JavaScript

MDN Web Docs (Mozilla) — Português do Brasil

Fonte oficial: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript
Gerado para o projeto doc-frog

JavaScript

JavaScript® (às vezes abreviado para **JS**) é uma linguagem leve, interpretada e baseada em objetos com *funções de primeira classe*, mais conhecida como a linguagem de script para páginas Web, mas usada também em vários outros ambientes sem browser, tais como node.js, Apache CouchDB e Adobe Acrobat. O JavaScript é uma linguagem baseada em protótipos, multi-paradigma e dinâmica, suportando estilos de orientação a objetos, imperativos e declarativos (como por exemplo a programação funcional). Saiba mais sobre o JavaScript.

Essa seção do site é dedicada à linguagem JavaScript e não às partes que são específicas para páginas Web e outros ambientes. Para mais informações sobre as APIs específicas para páginas Web, por favor consulte as seções Web APIs e DOM.

O padrão JavaScript é ECMAScript. Desde 2012, todos os navegadores modernos possuem suporte total ao ECMAScript 5.1. Navegadores mais antigos suportam pelo menos ECMAScript 3. Em 17 de Junho de 2015, a ECMA International publicou a sexta versão do ECMAScript, que é oficialmente chamado de ECMAScript 2015, e foi inicialmente conhecido como ECMAScript 6 ou ES6. Desde então, as especificações do ECMAScript são lançadas anualmente. Essa documentação faz referência à última versão de referência, que atualmente é a ECMAScript 2018.

Não se deve confundir o JavaScript com a linguagem de programação Java. Tanto *Java* quanto *JavaScript* são marcas registradas da Oracle nos Estados Unidos da América e em outros países. No entanto, as duas linguagens de programação possuem sintaxe, semânticas e usos muito diferentes.

Tutoriais

Aprenda a programar em JavaScript com guias e tutoriais.

Para iniciantes

Vá para a Área de aprendizado de JavaScript se você quer aprender JavaScript mas não tem experiência prévia com JavaScript ou programação. Os módulos completos que estão disponíveis lá são os seguintes:

Primeiros passos em JavaScript

> Respostas para algumas perguntas básicas como *O que é JavaScript?*, *Com o que se parece?* e *O que se pode fazer?*, bem como funcionalidades importantes do JavaScript, tais como variáveis, strings, números e vetores.

Elementos básicos do JavaScript

> Continuamos nossa cobertura das funcionalidades fundamentais do JavaScript, direcionando nossa atenção para tipos de blocos de código encontrados comumente, como expressões condicionais, laços, funções e eventos.

Introduzindo objetos em JavaScript

> O entendimento da natureza da orientação à objetos do JavaScript é importante se você quiser levar o seu conhecimento da linguagem para o próximo nível e escrever códigos mais eficientes, por isso oferecemos esse módulo para te ajudar.

Guia do JavaScript

Guia do JavaScript

> Um guia muito mais detalhado sobre a linguagem JavaScript, para pessoas que possuem experiência prévia com JavaScript ou outra linguagem de programação.

Intermediário

Uma re-introdução ao JavaScript

> Uma visão geral para aqueles que *pensam* que conhecem JavaScript.

Estruturas de dados do JavaScript

> Um resumo das estruturas de dados disponíveis em JavaScript.

Comparações de igualdade e uniformidade

> O JavaScript fornece três operações diferentes para comparar valores: igualdade estrita utilizando `===`, igualdade ampla usando `==` e o método `Object.is()`.

Avançado

Herança e a cadeia de protótipos

> Explicação da herança baseada em protótipos, que costuma ser amplamente mal entendida e subestimada.

Modo estrito

> O modo estrito define que você não pode usar nenhuma variável antes de inicializá-la. É uma variante restrita do ECMAScript 5, para um desempenho mais performático e uma depuração mais fácil.

Vetores JavaScript tipados

> Vetores JavaScript tipados oferecem um mecanismo para acesso a dados binários brutos.

Gerenciamento de memória

> Ciclo de vida da memória e coleta de lixo em JavaScript.

Modelo de concorrência e o loop de eventos

JavaScript tem um modelo de concorrência baseado em *loop de eventos*.

Referência

Navegue pela documentação completa da Referência de JavaScript.

Objetos globais

Conheça os objetos padrão nativos `Array`, `Boolean`, `Date`, `Error`, `Function`, `JSON`, `Math`, `Number`, `Object`, `RegExp`, `String`, `Map`, `Set`, `WeakMap` e `WeakSet`, entre outros.

Expressões e operadores

Saiba mais sobre o comportamento dos operadores de JavaScript `instanceof`, `typeof`, `new`, `this`, a precedência dos operadores e muito mais.

Instruções e declarações

Saiba como `do-while`, `for-in`, `for-of`, `try-catch`, `let`, `var`, `const`, `if-else`, `switch` e outras instruções e palavras-chave do JavaScript funcionam.

Funções

Aprenda como trabalhar com funções em JavaScript para desenvolver suas aplicações.

Ferramentas & recursos

Ferramentas úteis para escrever e depurar seu código **JavaScript**.

Ferramentas do Firefox para desenvolvedores

Scratchpad, Web Console, JavaScript Profiler, Debugger e muito mais.

JavaScript Shells

Um shell JavaScript permite que você teste rapidamente fragmentos de código JavaScript.

TogetherJS

Colaboração fácil. Ao adicionar o TogetherJS ao seu site, seus usuários podem ajudar uns aos outros em tempo real!

Stack Overflow

Perguntas do Stack Overflow com a tag *JavaScript*.

Versões do JavaScript e notas de lançamento

Navegue no histórico de funcionalidades do JavaScript e no status das implementações.

JSFiddle

Edite JavaScript, CSS e HTML e obtenha resultados em tempo real. Utilize recursos externos e colabore com sua equipe online.

Guia JavaScript

O Guia JavaScript mostra como usar JavaScript e dá uma visão geral da linguagem. Se você quer começar com JavaScript ou programação em geral, consulte os artigos na área de aprendizagem. Se você precisar de informações mais aprofundadas sobre algum recurso da linguagem, consulte a referência JavaScript.

Capítulos

Este guia está dividido em vários capítulos:

- Introdução

  Sobre este guia Sobre JavaScript JavaScript e Java ECMAScript Ferramentas Olá Mundo

- Sintaxe e tipos

  Sintaxe básica e comentários Declarações Escopo de variável Hoisting Estruturas de dados e Tipos Literais

- Controle de fluxo e manipulação de erro

  `if...else` `switch` `try/catch/throw` Objetos de erro Promises

- Laços e iteração

  `for while do...while break/continue for..in for..of`

- Funções

  Definição de funções Chamando funções Escopo da função Closures Argumentos & parâmetros Funções de seta

- Expressões e operadores

  Atribuição & comparações Operadores aritméticos Operadores lógicos & bit a bit Operador condicional (Ternário)

- Números e datas Números

Objeto de números Objeto Math Objeto Data

- Formatação de texto

  Strings Objetos de strings Modelos de strings Internacionalização

- Expressões Regular

  Criando uma expressão regular Padrão de escrita de uma expressão regular Trabalhando com expressões regular

- Coleções Indexadas

  Arrays Arrays digitados

- Coleções Chaveadas

  Maps WeakMaps Set WeakSet

- Trabalhando com objetos

  Objetos e propriedades Criando Objetos Definindo Métodos Getter e setter

- Detalhes do modelo de objetos

  POO baseada na prototipagem Criando hierarquias de objetos Herança

- Iteradores e geradores

  Iterators Iterables Generators

- Meta programação

  `Proxy` Handlers e traps Proxy revogável `Refletor`

- Próxima

Closures

Uma **closure** é a combinação de uma função com as referências ao estado que a circunda (o **ambiente léxico**). Em outras palavras, uma closure lhe dá acesso ao escopo de uma função externa a partir de uma função interna. Em JavaScript, as closures são criadas toda vez que uma função é criada, no momento da criação da função.

Escopo léxico

Considere a função abaixo:

```
function init() {
  var name = "Mozilla"; // name é uma variável local criada pelo init
  function displayName() {
    // displayName() é a função interna, uma closure
    console.log(name); // usa a variável declarada na função pai
  }
  displayName();
}
init();
```

A função `init()` cria uma variável local chamada `name`, e depois define uma função chamada `displayName()`. `displayName()` é uma função aninhada (uma *closure*) — ela é definida dentro da função `init()`, e está disponivel apenas dentro do corpo daquela função. Diferente de init(), `displayName()` não tem variáveis locais próprias, e ao invés disso reusa a variável `name` declarada na função pai.

Rode o código e veja que isso funciona. Este é um exemplo de *escopo léxico:* em JavaScript, o escopo de uma variável é definido por sua localização dentro do código fonte (isto é aparentemente *léxico*) e funções aninhadas têm acesso às variáveis declaradas em seu escopo externo.

Closure

Agora considere o seguinte exemplo:

```
function makeFunc() {
  var name = "Mozilla";
  function displayName() {
    alert(name);
  }
  return displayName;
}

var myFunc = makeFunc();
myFunc();
```

Se você rodar este código o mesmo terá exatamente o mesmo efeito que o `init()` do exemplo anterior: a palavra "Mozilla" será mostrada na caixa de alerta. O que é diferente - e interessante - é o fato de que a função interna do `displayName()` foi retornada da função externa antes de ser executada.

Pode parecer não muito intuitivo de que o código de fato funciona. Normalmente variáveis locais de uma função, apenas existem pela duração de sua execução. Uma vez que `makeFunc()` terminou de executar, é razoável esperar que a variável `name` não será mais necessária. Dado que o código ainda funciona como o esperado, este não é o caso.

A solução para tal problema é que a função `myFunc` tornou-se uma `closure`. Uma closure trata-se de um tipo especial de objeto que combina duas coisas: a função e o ambiente onde a função foi criada. Este ambiente consiste de quaisquer variáveis que estavam no escopo naquele momento em que a função foi criada. Neste caso, `myFunc` é a closure que incorpora tanto a função `displayName` quanto a palavra *Mozilla* que existia quando a closure foi criada.

Aqui temos um exemplo um pouco mais interessante, a função `makeAdder`:

```
function makeAdder(x) {
  return function (y) {
    return x + y;
  };
}

var add5 = makeAdder(5);
var add10 = makeAdder(10);

print(add5(2)); // 7
print(add10(2)); // 12
```

Neste exemplo definimos a função `makeAdder(x)` que toma um único argumento `x` e retorna uma nova função. A função retornada toma então um único argumento, `y`, e retorna então a soma de `x` e de `y`.

Na essência o `makeAdder` trata-se de uma *função fábrica* - irá construir outras funções que podem adicionar um determinado valor específico a seu argumento. No exemplo acima usamos a fábrica de funções para criar duas novas funções - uma que adiciona 5 ao argumento, e outra que adiciona 10.

Ambas as funções `add5` e `add10` são closures. Compartilham o mesmo corpo de definição de função mas armazenam diferentes ambientes. No ambiente da `add5`, por exemplo, `x` equivale a 5, enquanto na `add10` o valor de x é 10.

Closures na prática

Esta é a teoria — mas closures são realmente úteis? Vamos considerar suas aplicações práticas. Uma closure deixa você associar dados (do ambiente) com uma função que trabalha estes dados. Isto está diretamente ligado com programação orientada a objetos, onde objetos nos permitem associar dados (as propriedades do objeto) utilizando um ou mais métodos.

Consequentemente, você pode utilizar uma closure em qualquer lugar onde você normalmente utilizaria um objeto de único método.

Situações onde você poderia utilizar isto são comuns em ambientes web. Muitos códigos escritos em JavaScript para web são baseados em eventos - nós definimos algum comportamento e então, o atribuimos a um evento que será disparado pelo usuário (quando uma tecla for pressionada, por exemplo). Nosso código normalmente é utilizado como callback: uma função que será executada como resposta ao evento.

Aqui temos um exemplo prático: suponha que queremos adicionar alguns botões para ajustar o tamanho do texto de uma página. Um jeito de fazer seria especificar o tamanho da fonte no elemento body e então definir o tamanho dos outros elementos da página (os cabeçalhos, por exemplo) utilizando a unidade relativa em:

```
body {
  font-family: Helvetica, Arial, sans-serif;
  font-size: 12px;
}

h1 {
  font-size: 1.5em;
}
h2 {
  font-size: 1.2em;
}
```

Nossos botões interativos de tamanho de texto podem alterar a propriedade font-size do elemento body, e os ajustes serão refletidos em outros elementos graças à unidade relativa.

O código JavaScript:

```
function makeSizer(size) {
  return function () {
    document.body.style.fontSize = size + "px";
  };
}

var size12 = makeSizer(12);
var size14 = makeSizer(14);
var size16 = makeSizer(16);
```

`size12`, `size14` e `size16` agora são funções que devem redimensionar o texto do elemento body para 12, 14 e 16 pixels respectivamente. Nós podemos designá-las a botões (neste caso, links) como feito a seguir:

```
document.getElementById("size-12").onclick = size12;
document.getElementById("size-14").onclick = size14;
document.getElementById("size-16").onclick = size16;
```

```
<a href="#" id="size-12">12</a>
<a href="#" id="size-14">14</a>
<a href="#" id="size-16">16</a>
```

View on JSFiddle

Emulando métodos privados com closures

Linguagens como Java oferecem a habilidade de declarar métodos privados, o que significa que eles só poderão ser chamados por outros métodos na mesma classe.

O JavaScript não oferece uma maneira nativa de fazer isso, mas é possível emular métodos privados usando closures. Métodos privados não são somente úteis para restringir acesso ao código: eles também oferecem uma maneira eficaz de gerenciar seu namespace global, evitando que métodos não essenciais baguncem a interface pública do seu código.

Veja como definir algumas funções públicas que acessam funções e variáveis privadas, usando closures que também é conhecido como module pattern:

```
var Counter = (function () {
  var privateCounter = 0;
  function changeBy(val) {
    privateCounter += val;
  }
  return {
    increment: function () {
      changeBy(1);
    },
    decrement: function () {
      changeBy(-1);
    },
    value: function () {
      return privateCounter;
    },
  };
})();

alert(Counter.value()); /* Alerts 0 */
Counter.increment();
Counter.increment();
alert(Counter.value()); /* Alerts 2 */
Counter.decrement();
alert(Counter.value()); /* Alerts 1 */
```

Tem muita coisa acontecendo aqui. Nos exemplos anteriores cada closure teve o seu próprio ambiente; aqui nós criamos um ambiente único que é compartilhado por três funções: `Counter.increment`, `Counter.decrement` e `Counter.value`.

O ambiente compartilhado é criado no corpo de uma função anônima, da qual é executada assim que é definida. O ambiente contém dois itens privados: uma variável chamada `privateCounter` e uma função chamada `changeBy`. Nenhum desses itens privados podem ser acessados diretamente de fora da função anônima. Ao invés disso, eles devem ser acessados pelas três funções públicas que são retornadas.

Aquelas três funções públicas são closures que compartilham o mesmo ambiente. Graças ao escopo léxico do JavaScript, cada uma delas tem acesso a variável `privateCounter` e à função `changeBy`.

> **Nota:** Você perceberá que estamos definindo uma função anônima que cria um contador , e então o executamos imediatamente e atribuímos o resultado à variável `Counter`. Poderíamos armazenar essa função em uma variável separada e usá-la para criar diversos contadores.

```
var makeCounter = function () {
  var privateCounter = 0;
  function changeBy(val) {
    privateCounter += val;
  }
  return {
    increment: function () {
      changeBy(1);
    },
    decrement: function () {
      changeBy(-1);
    },
    value: function () {
      return privateCounter;
    },
  };
};

var Counter1 = makeCounter();
var Counter2 = makeCounter();
alert(Counter1.value()); /* Alerts 0 */
Counter1.increment();
Counter1.increment();
alert(Counter1.value()); /* Alerts 2 */
Counter1.decrement();
alert(Counter1.value()); /* Alerts 1 */
alert(Counter2.value()); /* Alerts 0 */
```

Observe como cada um dos contadores mantém a sua independência em relação ao outro. Seu ambiente durante a execução da função `makeCounter()` é diferente a cada vez que ocorre. A variável `privateCounter` contém uma instância diferente a cada vez.

> **Nota:** Usar closures desta maneira oferece uma série de benefícios que estão normalmente associados a programação orientada a objetos, em particular

> encapsulamento e ocultação de dados.

Criando closures dentro de loops: Um erro comum

Antes da introdução da palavra chave `let` no JavaScript 1.7, um problema comum ocorria com closures quando eram criadas dentro de um loop. Considere o exemplo:

```
<p id="help">Notas úteis aparecerão aqui</p>
<p>E-mail: <input type="text" id="email" name="email" /></p>
<p>Nome: <input type="text" id="name" name="name" /></p>
<p>Idade: <input type="text" id="age" name="age" /></p>
```

```
function showHelp(help) {
  document.getElementById("help").innerHTML = help;
}

function setupHelp() {
  var helpText = [
    { id: "email", help: "Seu e-mail" },
    { id: "name", help: "Seu nome completo" },
    { id: "age", help: "Sua idade (você deve ter mais de 16 anos)" },
  ];

  for (var i = 0; i < helpText.length; i++) {
    // O culpado é o uso do `var` nesta linha
    var item = helpText[i];
    document.getElementById(item.id).onfocus = function () {
      showHelp(item.help);
    };
  }
}

setupHelp();
```

View on JSFiddle

O array `helpText` define três dicas úteis, cada uma associada ao ID de um input no documento. O loop percorre essas definições, atrelando um evento `onfocus` para cada um que mostra o método de ajuda associado.

Se você tentar executar esse código, Você verá que não vai funcionar como esperado. Não importa em qual campo ocorre o focus, a mensagem sobre a sua idade será mostrada.

O motivo disto é que as funções atreladas ao `onfocus` são closures; elas consistem na definição da função e do ambiente capturado do escopo da função `setupHelp`. Três closures foram criados, mas todos eles compartilham o mesmo ambiente. No momento em que os callbacks do `onfocus` são executados, o loop segue seu curso e então a variável `item` (compartilhada por todos os três closures) fica apontando para a última entrada na lista `helpText`.

Uma solução seria neste caso usar mais closures: em particular, usar uma fábrica de funções como descrito anteriormente:

```
function showHelp(help) {
  document.getElementById("help").textContent = help;
}

function makeHelpCallback(help) {
  return function () {
    showHelp(help);
  };
}

function setupHelp() {
  var helpText = [
    { id: "email", help: "Seu e-mail" },
    { id: "name", help: "Seu nome completo" },
    { id: "age", help: "Sua idade (você deve ter mais de 16 anos)" },
  ];

  for (var i = 0; i < helpText.length; i++) {
    var item = helpText[i];
    document.getElementById(item.id).onfocus = makeHelpCallback(item.help);
  }
}

setupHelp();
```

View on JSFiddle

Isto funciona conforme o esperado. Ao invés dos callbacks compartilharem o mesmo ambiente, a função `makeHelpCallback` cria um novo ambiente para cada um no qual `help` se refere à string correspondente do array `helpText`.

Uma outra maneira de escrever o mesmo usando closures anônimas é:

```
function showHelp(help) {
  document.getElementById("help").textContent = help;
}
```

```
function setupHelp() {
  var helpText = [
    { id: "email", help: "Seu e-mail" },
    { id: "name", help: "Seu nome completo" },
    { id: "age", help: "Sua idade (você deve ter mais de 16 anos)" },
  ];

  for (var i = 0; i < helpText.length; i++) {
    (function () {
      var item = helpText[i];
      document.getElementById(item.id).onfocus = function () {
        showHelp(item.help);
      };
    })(); // Expressão de função invocada imediatamente com o valor atual do item (preservado até a iteração).
  }
}

setupHelp();
```

Se você não quiser usar mais closures, você pode usar a palavra-chave `let` ou `const`:

```
function showHelp(help) {
  document.getElementById("help").textContent = help;
}

function setupHelp() {
  var helpText = [
    { id: "email", help: "Seu e-mail" },
    { id: "name", help: "Seu nome completo" },
    { id: "age", help: "Sua idade (você deve ter mais de 16 anos)" },
  ];

  for (let i = 0; i < helpText.length; i++) {
    const item = helpText[i];
    document.getElementById(item.id).onfocus = () => {
      showHelp(item.help);
    };
  }
}

setupHelp();
```

Este exemplo usa `const` em vez de `var`, portanto cada closure vincula a variável com escopo de bloco, o que significa que nenhuma closure adicional é necessária.

Outra alternativa poderia ser usar `forEach()` para iterar sobre o array `helpText` e anexar um ouvinte a cada `<input>`, conforme mostrado:

```
function showHelp(help) {
  document.getElementById("help").textContent = help;
}

function setupHelp() {
  var helpText = [
    { id: "email", help: "Seu e-mail" },
    { id: "name", help: "Seu nome completo" },
    { id: "age", help: "Sua idade (você deve ter mais de 16 anos)" },
  ];

  helpText.forEach(function (text) {
    document.getElementById(text.id).onfocus = function () {
      showHelp(text.help);
    };
  });
}

setupHelp();
```

Considerações de performance

Não é sábio criar funções dentro de outras funções se a closure não for necessário para uma tarefa em particular, pois ele afetará a performance do script de forma bem negativa tanto em velocidade de processamento quanto em consumo de memória.

Por exemplo, ao criar uma nova classe/objeto, os métodos devem normalmente estar associados ao protótipo do objeto do que definido no construtor. O motivo disso é que sempre que o construtor for chamado os métodos serão reatribuídos (isto é, para cada criação de objeto).

Considere o seguinte exemplo pouco prático porém demonstrativo:

```
function MyObject(name, message) {
  this.name = name.toString();
  this.message = message.toString();
  this.getName = function () {
    return this.name;
  };
```

```
  this.getMessage = function () {
    return this.message;
  };
}
```

O código anterior não aproveita os benefícios dos closures e portanto poderia ser reformulado assim:

```
function MyObject(name, message) {
  this.name = name.toString();
  this.message = message.toString();
}
MyObject.prototype = {
  getName: function () {
    return this.name;
  },
  getMessage: function () {
    return this.message;
  },
};
```

Ou assim:

```
function MyObject(name, message) {
  this.name = name.toString();
  this.message = message.toString();
}
MyObject.prototype.getName = function () {
  return this.name;
};
MyObject.prototype.getMessage = function () {
  return this.message;
};
```

Nos dois exemplos anteriores, o protótipo herdado pode ser compartilhado por todos os objetos, e as definições de métodos não precisam ocorrer sempre que o objeto for criado. Veja Detalhes do modelo de objeto para mais detalhes.

Controle de Fluxo e Manipulação de Erro

- Anterior
- Próxima

O JavaScript suporta um conjunto compacto de declarações, especificamente de fluxo de controle, que você pode utilizar para atribuir uma grande interatividade a páginas web. Este capítulo fornece uma visão geral destas declarações.

Veja a Referência do JavaScript para detalhes sobre as declarações mostradas neste capítulo. No código em JavaScript, o caractere ponto e vírgula (`;`) é utilizado para separar declarações.

Toda expressão também é uma declaração. Veja Expressões e Operadores para informações completas sobre expressões.

Declaração em bloco

Uma declaração em bloco é utilizada para agrupar declarações. O bloco é delimitado por um par de chaves:

```
{
   declaracao_1;
   declaracao_2;
   .
   .
   .
   declaracao_n;
}
```

Exemplo

Declarações em bloco são utilizadas geralmente com declarações de fluxo de controle (ex. `if`, `for`, `while`).

```
while (x < 10) {
  x++;
}
```

Aqui, `{ x++; }` é a declaração de bloco.

Importante: Antes de ECMAScript 6 o JavaScript **não** possuía escopo de bloco. Variáveis introduzidas dentro de um bloco possuem como escopo a função ou o script em que o bloco está contido, e, definir tais variáveis tem efeito muito além do bloco em si. Em outras palavras, declarações de bloco não introduzem um escopo. Embora blocos "padrão" sejam uma sintaxe válida não utilize-os, em JavaScript, pensando que funcionam como em C ou Java porque eles não funcionam da maneira que você acredita. Por exemplo:

```
var x = 1;
{
  var x = 2;
}
console.log(x); // exibe 2
```

Este código exibe 2 porque a declaração `var x` dentro do bloco possui o mesmo escopo que a declaração `var x` antes do bloco. Em C ou Java, o código equivalente exibiria 1.

Declarações condicionais

Uma declaração condicional é um conjunto de comandos que são executados caso uma condição especificada seja verdadeira. O JavaScript suporta duas declarações condicionais: `if...else` e `switch`.

Declaração if...else

Use a declaração `if` para executar alguma declaração caso a condição lógica for verdadeira. Use a cláusula opcional `else` para executar alguma declaração caso a condição lógica for falsa. Uma declaração `if` é declarada da seguinte maneira:

```
if (condicao) {
  declaracao_1;
} else {
  declaracao_2;
}
```

onde `condicao` pode ser qualquer expressão que seja avaliada como verdadeira ou falsa. Veja Boolean para uma explicação sobre o que é avaliado como `true` e `false`. Se `condicao` for avaliada como verdadeira, declaracao_1 é executada; caso contrário, `declaracao_2` é executada. `declaracao_1` e `declaracao_2` podem ser qualquer declaração, incluindo declarações `if` aninhadas.

Você pode também combinar declarações utilizando `else if` para obter várias condições testadas em sequência, como o seguinte:

```
if (condicao) {
  declaracao_1;
} else if (condicao_2) {
  declaracao_2;
} else if (condicao_n) {
  declaracao_n;
} else {
  declaracao_final;
}
```

Para executar várias declarações, agrupe-as em uma declaração em bloco (`{ ... }`). Em geral, é uma boa prática sempre utilizar declarações em bloco, especialmente ao aninhar declarações `if`:

```
if (condicao) {
    declaracao_1_executada_se_condicao_for_verdadeira;
    declaracao_2_executada_se_condicao_for_verdadeira;
} else {
    declaracao_3_executada_se_condicao_for_falsa;
    declaracao_4_executada_se_condicao_for_falsa;
}
```

Recomenda-se não utilizar atribuições simples em uma expressão condicional porque o símbolo de atribuição poderia ser confundido com o de igualdade ao dar uma olhada no código. Por exemplo, não utilize o seguinte código:

```
if (x = y) {
  /* faça a coisa certa */
}
```

Caso tenha que utilizar uma atribuição em uma expressão condicional, uma prática comum é colocar parênteses adicionais em volta da atribuição. Por exemplo:

```
if ((x = y)) {
  /* faça a coisa certa */
}
```

Valores avaliados como falsos

Os seguintes valores são avaliados como falsos:

- `false`
- `undefined`
- `null`
- `0`
- `NaN`
- string vazia (`""`)

Todos os outros valores, incluindo todos os objetos, são avaliados como verdadeiros quando passados para uma declaração condicional.

Não confunda os valores booleanos primitivos `true` e `false` com os valores de `true` e `false` do objeto Boolean. Por exemplo:

```
var b = new Boolean(false);
if (b) // esta condição é avaliada como verdadeira
if (b == true) // esta condição é avaliada como falsa
```

Exemplo

No exemplo a seguir, a função `verifiqueDados` retorna verdadeiro se o número de caracteres em um objeto `Text` for três; caso contrário, exibe um alerta e retorna `falso`.

```
function verifiqueDados() {
  if (document.form1.tresCaracteres.value.length == 3) {
    return true;
  } else {
    alert(
      "Informe exatamente três caracteres. " +
        document.form1.tresCaracteres.value +
        " não é válido.",
    );
    return false;
  }
}
```

Declaração switch

Uma declaração `switch` permite que um programa avalie uma expressão e tente associar o valor da expressão ao rótulo de um `case`. Se uma correspondência é encontrada, o programa executa a declaração associada. Uma declaração `switch` se parece com o seguinte:

```
switch (expressao) {
   case rotulo_1:
      declaracoes_1
      [break;]
   case rotulo_2:
      declaracoes_2
      [break;]
   ...
   default:
      declaracoes_padrao
      [break;]
}
```

O programa primeiramente procura por uma cláusula `case` com um rótulo que corresponda ao valor da expressão e então transfere o controle para aquela cláusula, executando as declaracoes associadas. Se nenhum rótulo correspondente é encontrado, o programa procura pela cláusula opcional `default` e, se encontrada, transfere o controle àquela cláusula, executando as declarações associadas. Se nenhuma cláusula `default` é encontrada, o programa continua a execução a partir da declaracao seguinte ao `switch`. Por convenção, a cláusula `default` é a última, mas não é necessário que seja assim.

A instrução `break` associada a cada cláusula `case`, garante que o programa sairá do `switch` assim que a declaração correspondente for executada e que continuará a execução a partir da declaração seguinte ao `switch`. Se a declaração `break` for omitida, o programa continua a execução a partir da próxima declaração dentro do `switch`.

Exemplo

No exemplo a seguir, se `tipofruta` for avaliada como `"Banana"`, o programa faz a correspondência do valor com case `"Banana"` e executa a declaração associada. Quando o `break` é encontrado, o programa termina o `switch` e executa a declaração seguinte ao condicional. Se o `break` fosse omitido, a declaração de case `"Cereja"` também seria executada.

```
switch (tipofruta) {
  case "Laranja":
    console.log("O quilo da laranja está R$0,59.<br>");
    break;
  case "Maçã":
    console.log("O quilo da maçã está R$0,32.<br>");
    break;
  case "Banana":
    console.log("O quilo da banana está R$0,48.<br>");
    break;
  case "Cereja":
    console.log("O quilo da cereja está R$3,00.<br>");
    break;
  case "Manga":
    console.log("O quilo da manga está R$0,56.<br>");
    break;
  case "Mamão":
    console.log("O quilo do mamão está R$2,23.<br>");
    break;
  default:
    console.log("Desculpe, não temos " + tipofruta + ".<br>");
}
console.log("Gostaria de mais alguma coisa?<br>");
```

Declarações de Manipulação de Error

Você pode chamar uma exceção usando a declaração `throw` e manipulá-la usando a declaração `try...catch`.

- Declaração throw
- Declaração `try...catch`

Tipos de exceções

Praticamente pode-se utilizar `throw` em qualquer objeto de JavaScript. Todavia, nem todos os objetos ativados por `throw` são igualmente criados. Embora seja bastante comum tratar números ou strings como erros usando `throw`, é frequentemente mais eficiente usar alguns tipos de exceções especificamente criadas para esses propósitos:

- ECMAScript exceptions
- `DOMException` and `DOMError`

<code>Declaração throw</code>

Use a declaração `throw` para lançar uma exceção. Quando você lança uma exceção, você especifica a expressão contendo o valor a ser lançado:

```
throw expressão;
```

Você pode lançar qualquer expressão, não apenas expressões de um tipo específico. O código a seguir lança várias exceções de diferentes tipos:

```
throw "Error2"; // tipo string
throw 42; // tipo numérico
throw true; // tipo booleano
throw {
  toString: function () {
    return "Eu sou um objeto!";
  },
};
```

> **Nota:** Você pode especificar um objeto quando você lança uma exceção. Você pode então, referenciar as propriedades de um objeto no bloco catch. O exemplo a seguir cria um objeto myUserException do tipo userException e o usa em uma declaração throw.

```
// Cria um objeto do tipo UserException
function UserException(mensagem) {
  this.mensagem = mensagem;
  this.nome = "UserException";
}

// Realiza a conversão da exceção para uma string adequada quando usada como uma string.
// (ex. pelo console de erro)
UserException.prototype.toString = function () {
  return this.name + ': "' + this.message + '"';
};

// Cria uma instância de um tipo de objeto e lança ela
throw new UserException("Valor muito alto");
```

<code>Declaração try...catch</code>

A declaração `try...catch` coloca um bloco de declarações em try, e especifica uma ou mais respostas para uma exceção lançada. Se uma exceção é lançada, a declaração `try...catch` pegá-a.

A declaração `try...catch` é composta por um bloco `try`, que contém uma ou mais declarações, e zero ou mais blocos `catch`, contendo declarações que especificam o que fazer se uma exceção é lançada no bloco `try`. Ou seja, você deseja que o bloco `try` tenha sucesso, e se ele não tiver êxito, você quer o controle passado para o bloco `catch`. Se qualquer declaração do bloco `try` (ou em uma função chamada dentro do bloco `try`) lança uma exceção, o controle é imediatamente mudado para o bloco `catch`. Se nenhuma exceção é lançada no bloco `try`, o bloco `catch` é ignorado. O bloco `finally` executa após os blocos `try` e `catch` executarem, mas antes das declarações seguinte ao bloco `try...catch`.

O exemplo a seguir usa a declaração `try...catch`. O exemplo chama uma função que recupera o nome de um mês no array com base no valor passado para a função. Se o valor não corresponde ao número de um mês (1-12), uma exceção é lançada com o valor "`InvalidMonthNo`" e as declarações no bloco `catch` define a váriavel `monthName` para `unknown`.

```
function getMonthName(mo) {
  mo = mo - 1; // Ajusta o número do mês para o índice do array (1 = Jan, 12 = Dec)
  var months = [
    "Jan",
    "Feb",
    "Mar",
    "Apr",
    "May",
    "Jun",
    "Jul",
    "Aug",
    "Sep",
    "Oct",
    "Nov",
    "Dec",
  ];
  if (months[mo]) {
    return months[mo];
  } else {
    throw "InvalidMonthNo"; //lança uma palavra-chave aqui usada.
  }
}

try {
```

```
  // statements to try
  monthName = getMonthName(myMonth); // função poderia lançar uma exceção
} catch (e) {
  monthName = "unknown";
  logMyErrors(e); // passa a exceção para o manipulador de erro -> sua função local.
}
```

O bloco `catch`

Você pode usar um bloco `catch` para lidar com todas as exceções que podem ser geradas no bloco `try`.

```
catch (catchID) {
  declaracoes
}
```

O bloco `catch` específica um identificador (`catchID` na sintaxe anterior), que contém o valor especificado pela declaração `throw`; você pode usar esse identificador para obter informações sobre a exceção que foi lançada. JavaScript cria este identificador quando o bloco `catch` é inserido; o identificador dura enquanto o bloco `catch` está em execução, depois que termina a execução do bloco `catch`, o identificador não estará mais disponível.

Por exemplo, o seguinte código lança uma exceção. Quando a exceção ocorre, o controle é transferido para o bloco `catch`.

```
try {
  throw "myException"; // lança  uma exceção
} catch (e) {
  // declarações de lidar com as exceções
  logMyErrors(e); // passar a exceção para o manipulador de erro
}
```

O bloco `finally`

O bloco `finally` contém instruções para executar após os blocos `try` e `catch`, mas antes das declarações seguinte a declaração `try...catch`. O bloco `finally` é executado com ou sem o lançamento de uma exceção. Se uma exceção é lançada, a declaração no bloco `finally` executa, mesmo que nenhum bloco `catch` processe a exceção.

Você pode usar bloco `finally` para deixar a falha de seu script agradável quando uma exceção ocorre; por exemplo, você pode precisar liberar um recurso que seu script tem amarrado. O exemplo a seguir abre um arquivo e então executa instruções que usam o arquivo (JavaScript do lado do servidor permite que você acesse arquivos). Se um exceção é lançada enquanto o arquivo é aberto, o bloco `finally` fecha o arquivo antes do script falhar.

```
openMyFile();
try {
  writeMyFile(theData); //Isso pode lançar um erro
} catch (e) {
  handleError(e); // Se temos um erro temos que lidar com ele
} finally {
  closeMyFile(); // Sempre feche o recurso
}
```

Se o bloco `finally` retornar um valor, este valor se torna o valor de toda a entrada `try-catch-finally`, independente de quaisquer declarações de retorno nos blocos `try` e `catch`:

```
function f() {
  try {
    console.log(0);
    throw "bogus";
  } catch (e) {
    console.log(1);
    return true; // essa declaração de retorno é suspensa
    // até que o bloco finally seja concluído
    console.log(2); // não executa
  } finally {
    console.log(3);
    return false; // substitui o "return" anterior
    console.log(4); // não executa
  }
  // "return false" é executado agora
  console.log(5); // não executa
}
f(); // exibe 0, 1, 3; retorna false
```

Substituições de valores de retorno pelo bloco `finally` também se aplica a exceções lançadas ou re-lançadas dentro do bloco `catch`:

```
function f() {
  try {
    throw "bogus";
  } catch (e) {
    console.log('captura interior "falso"');
    throw e; // essa instrução throw é suspensa até
    // que o bloco finally seja concluído
  } finally {
    return false; // substitui "throw" anterior
  }
  // "return false" é executado agora
```

```
}

try {
  f();
} catch (e) {
  // isto nunca é executado porque o throw dentro
  // do catch é substituído
  // pelo return no finally
  console.log('captura exterior "falso"');
}

// SAIDA
// captura interior "falso"
```

Aninhando declarações try...catch

Você pode aninhar uma ou mais declarações `try...catch`. Se uma declaração `try...catch` interior não tem um bloco `catch`, o delimitador do bloco `try...catch` da declaração `catch` é verificado por uma correspondência.

Utilizando objetos de <code>erro</code>

Dependendo do tipo de erro, você pode ser capaz de usar as propriedade 'name' e 'message' para pegar uma mensagem mais refinada. A propriedade 'name' fornece a classe geral de erro (ex., 'DOMException' ou 'Error'), enquanto 'message' geralmente oferece uma mensagem mais sucinta do que se poderia obter através da conversão do objeto de erro para uma string.

Se você está lançando suas próprias exceções, a fim de tirar proveito dessas propriedades (como o seu bloco catch não discrimina entre suas próprias exceções e as exceções próprias da linguagem), você pode usar o construtor Error. Por exemplo:

```
function doSomethingErrorProne () {
  if (ourCodeMakesAMistake()) {
    throw (new Error('A mensagem'));
  } else {
    doSomethingToGetAJavascriptError();
  }
}
....
try {
  doSomethingErrorProne();
}
catch (e) {
  console.log(e.name); // exibe 'Error'
  console.log(e.message); // exibe 'A mensagem' ou uma mensagem de erro em JavaScript
}
```

Promises

Começando com ECMAScript 6, JavaScript ganha suporte para objetos `Promise` que lhe permite controlar o fluxo de operações diferídas e assíncronas.

Uma Promise assume um destes estados:

- *pending*: estado inicial, não `fulfilled`, ou `rejected`.
- *fulfilled*: operação bem sucedida.
- *rejected*: operação falha.
- *settled*: A Promise é fulfilled ou rejected, mas não pending.



Carregando uma imagem com XHR

Um exemplo simples usando Promise e `XMLHttpRequest` para carregar uma imagem disponível no repositório MDN GitHub promise-test. Você também pode vê-lo executando. Cada etapa está comentada o que lhe permite seguir de perto a arquitetura Promise e arquitetura XHR. Aqui está a versão não comentada, mostrando o fluxo `Promise` para que você possa ter uma ideia:

```
function imgLoad(url) {
  return new Promise(function (resolve, reject) {
    var request = new XMLHttpRequest();
    request.open("GET", url);
    request.responseType = "blob";
```

```
    request.onload = function () {
      if (request.status === 200) {
        resolve(request.response);
      } else {
        reject(
          Error(
            "Image didn't load successfully; error code:" + request.statusText,
          ),
        );
      }
    };
    request.onerror = function () {
      reject(Error("There was a network error."));
    };
    request.send();
  });
}
```

Para uma informação mais detalhada, consulte a página de referência `Promise`.

- Anterior
- Próxima

Estrutura de dados do Javascript

Todas as linguagens de programação têm estruturas de dados embutidas, mas geralmente diferem de uma linguagem para outra. Este artigo tenta listar as estruturas de dados internas disponíveis em JavaScript e quais propriedades elas possuem. Estes podem ser usados para construir outras estruturas de dados. Sempre que possível, são feitas comparações com outras línguas.

Tipagem dinâmica e fraca

JavaScript é uma linguagem dinâmica com tipos dinâmicos. As variáveis em JavaScript não estão diretamente associadas a nenhum tipo de valor específico, e qualquer variável pode receber (e reatribuir) valores de todos os tipos:

```
let foo = 42; // foo agora é um número
foo = "bar"; // foo agora é uma string
foo = true; // foo agora é um booleano
```

JavaScript também é uma linguagem de tipagem fraca, o que significa que permite a conversão implícita de tipo quando uma operação envolve tipos incompatíveis, em vez de gerar erros de tipo.

```
const foo = 42; //foo é um número
const result = foo + "1"; // JavaScript coage foo para uma string, então ela pode ser concatenada com o outro operando
console.log(resultado); // 421
```

Coerções implícitas são muito convenientes, mas podem ser uma arma em potencial se os desenvolvedores não pretendem fazer a conversão ou pretendem converter na outra direção (por exemplo, string para número em vez de número para string). Para symbols e BigInts, o JavaScript desativou intencionalmente certas conversões de tipo implícito.

Tipos em JavaScript

O conjunto de tipos na linguagem JavaScript consiste em *primitive values* e *objects*.

- Valores primitivos (dados imutáveis representados diretamente no nível mais baixo do idioma)
  - tipo Boolean
  - Tipo Null
  - Tipo Undefined
  - Tipo Number
  - tipo BigInt
  - tipo String
  - Tipo Symbol
- Objetos (coleções de propriedades)

Valores primitivos

Todos os tipos, exceto objetos, definem valores imutáveis (ou seja, valores que não podem ser alterados). Por exemplo, Strings são imutáveis. Referimo-nos a valores desses tipos como "*valores primitivos*".

Tipo Boolean

Boolean representa uma entidade lógica e pode ter dois valores: `true` e `false`. Consulte Boolean e `Boolean` para obter mais detalhes.

Tipo Null

O tipo Null tem exatamente um valor: `null`. Consulte `null` e Null para obter mais detalhes.

Tipo Undefined

Uma variável que não recebeu um valor tem o valor `undefined`. Consulte `undefined` e Undefined para mais detalhes.

Tipos numéricos

ECMAScript tem dois tipos numéricos integrados: Number e BigInt — junto com o valor relacionado NaN.

Tipo Number

O tipo Number é um valor IEEE 754 de formato binário de 64 bits de precisão dupla. É capaz de armazenar números de ponto flutuante positivos entre 2^-1074 (`Number.MIN_VALUE`) e 2^1024 (`Number.MAX_VALUE`), bem como números de ponto flutuante negativos entre -(2^-1074) e -(2^1024), mas só pode armazenar com segurança inteiros no intervalo -(2^53 − 1) (`Number.MIN_SAFE_INTEGER`) para 2^53 − 1 (`Number.MAX_SAFE_INTEGER`).

> **Nota:** Você pode verificar se um número está dentro do intervalo de inteiros seguros usando `Number.isSafeInteger()`. Fora do intervalo de `Number.MIN_SAFE_INTEGER` a `Number.MAX_SAFE_INTEGER`, o JavaScript não pode mais representar inteiros com segurança; eles serão representados por uma aproximação de ponto flutuante de precisão dupla.

Valores fora do intervalo ±(2^-1074 a 2^1024) são convertidos automaticamente:

- Valores positivos maiores que `Number.MAX_VALUE` são convertidos em `+Infinity`.
- Valores positivos menores que `Number.MIN_VALUE` são convertidos em `+0`.
- Valores negativos menores que `-Number.MAX_VALUE` são convertidos para `-Infinity`.
- Valores negativos maiores que `-Number.MIN_VALUE` são convertidos para `-0`.

`+Infinity` e `-Infinity` se comportam de forma semelhante ao infinito matemático, mas com algumas pequenas diferenças; veja `Number.POSITIVE_INFINITY` e `Number.NEGATIVE_INFINITY` para detalhes.

O tipo Number tem apenas um inteiro com múltiplas representações: `0` é representado como `-0` e `+0` (onde `0` é um alias para `+0`). Na prática, quase não há diferença entre as diferentes representações; por exemplo, `+0 === -0` é `true`. No entanto, você pode perceber isso quando divide por zero:

```
console.log(42 / +0); // Infinity
console.log(42 / -0); // -Infinity
```

Embora um número geralmente represente apenas seu valor, o JavaScript fornece operadores bitwise.

> **Nota:** Embora os operadores bit a bit *podem* ser usados para representar vários valores booleanos em um único número usando mascaramento de bits, isso geralmente é considerada uma má prática. JavaScript oferece outros meios para representar um conjunto de booleanos (como uma matriz de booleanos ou um objeto com valores booleanos atribuídos a propriedades nomeadas). O mascaramento de bits também tende a tornar o código mais difícil de ler, entender e manter.

Pode ser necessário usar essas técnicas em ambientes muito restritos, como ao tentar lidar com as limitações do armazenamento local ou em casos extremos (como quando cada bit na rede conta). Esta técnica só deve ser considerada quando for a última medida que pode ser tomada para otimizar o tamanho.

Tipo BigInt

O tipo BigInt é uma primitiva numérica em JavaScript que pode representar inteiros com precisão arbitrária. Com BigInts, você pode armazenar e operar com segurança em grandes números inteiros, mesmo além do limite de números inteiros seguros.

Um BigInt é criado anexando `n` ao final de um inteiro ou chamando o construtor.

Você pode obter o maior valor seguro que pode ser incrementado com Numbers usando a constante `Number.MAX_SAFE_INTEGER`. Com a introdução do BigInts, você pode operar com números além do `Number.MAX_SAFE_INTEGER`.

Este exemplo demonstra, onde incrementar o `Number.MAX_SAFE_INTEGER` retorna o resultado esperado:

```
// BigInt
const x = BigInt(Number.MAX_SAFE_INTEGER); // 9007199254740991n
x + 1n === x + 2n; // false porque 9007199254740992n e 9007199254740993n são desiguais

// Number
Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER + 2; // true porque ambos são 9007199254740992
```

Você pode usar os operadores `+`, `*`, `-`, `**` e `%` com BigInts — assim como com Numbers. Um BigInt não é estritamente igual a um Number, mas é vagamente assim.

Um BigInt se comporta como um Number nos casos em que é convertido em booleano: `if`, `||`, `&&`, `Boolean`, `!`.

`BigInt`s não podem ser operados de forma intercambiável com Numbers. Em vez disso, um `TypeError` será lançado.

NaN

`NaN` ("**N**ot a **N**umber") é normalmente encontrado quando o resultado de uma operação aritmética não pode ser expresso como um número. É também o único valor em JavaScript que não é igual a si mesmo.

Tipo String

O tipo String do JavaScript é usado para representar dados textuais. É um conjunto de "elementos" de valores inteiros sem sinal de 16 bits. Cada elemento na String ocupa uma posição na String. O primeiro elemento está no índice `0`, o próximo no índice `1` e assim por diante. O comprimento de uma String é o número de elementos nela.

As strings JavaScript são imutáveis. Isso significa que uma vez que uma string é criada, não é possível modificá-la.

No entanto, ainda é possível criar outra string com base em uma operação na string original. Por exemplo:

- Uma substring do original escolhendo letras individuais ou usando `substring()`.
- Uma concatenação de duas strings usando o operador de concatenação (+) ou `concat()`.

Cuidado com a tipagem do seu código!

Pode ser tentador usar strings para representar dados complexos. Fazer isso traz benefícios a curto prazo:

- É fácil construir strings complexas com concatenação.
- Strings são fáceis de depurar (o que você vê impresso é sempre o que está na string).
- Strings são o denominador comum de muitas APIs (campos de entrada, valores de armazenamento local , `XMLHttpRequest` respostas ao usar `responseText`, etc.) e pode ser tentador trabalhar apenas com strings.

Com convenções, é possível representar qualquer estrutura de dados em uma string. Isso não faz com que seja uma boa ideia. Por exemplo, com um separador, pode-se emular uma lista (enquanto um array JavaScript seria mais adequado). Infelizmente, quando o separador é usado em um dos elementos da "lista", a lista é quebrada. Um caractere de escape pode ser escolhido, etc. Tudo isso requer convenções e cria uma carga de manutenção desnecessária.

Use strings para dados textuais. Ao representar dados complexos, *parse* strings e use a abstração apropriada.

Tipo Symbol

Um Symbol é um valor primitivo **único** e **imutável** e pode ser usado como a chave de uma propriedade Object (veja abaixo). Em algumas linguagens de programação, os símbolos são chamados de "átomos".

Para mais detalhes veja a página de referência `Symbol`.

Objetos

Na ciência da computação, um objeto é um valor na memória que possivelmente é referenciado por um identificador.

Propriedades

Em JavaScript, os objetos podem ser vistos como uma coleção de propriedades. Com a sintaxe literal de objeto, um conjunto limitado de propriedades é inicializado; então as propriedades podem ser adicionadas e removidas. Os valores de propriedade podem ser valores de qualquer tipo, incluindo outros objetos, o que permite construir estruturas de dados complexas. As propriedades são identificadas usando valores *key*. Um valor *key* é um String value ou um Symbol value.

Existem dois tipos de propriedades de objeto: A *data* property e a *accessor* property. Cada propriedade tem *attributes* correspondentes. Cada atributo é acessado internamente pelo mecanismo JavaScript, mas você pode defini-los por meio de `Object.defineProperty()` ou lê-los por meio de `Object.getOwnPropertyDescriptor()`. Você pode ler mais sobre as várias nuances na página `Object.defineProperty()`.

Propriedade de dados

As propriedades de dados associam uma chave a um valor. Pode ser descrito pelos seguintes atributos:

`value`

O valor recuperado por um acesso get da propriedade. Pode ser qualquer valor JavaScript.

`gravável`

Um valor booleano que indica se a propriedade pode ser alterada com uma atribuição.

`enumerável`

Um valor booleano que indica se a propriedade pode ser enumerada por um loop `for...in`. Consulte também Enumerabilidade e propriedade de propriedades para saber como a enumerabilidade interage com outras funções e sintaxes.

`configurável`

Um valor booleano que indica se a propriedade pode ser excluída, pode ser alterada para uma propriedade acessora e pode ter seus atributos alterados.

Propriedade do acessor

Associa uma chave com uma das duas funções de acesso (`get` e `set`) para recuperar ou armazenar um valor.

> **Nota:** É importante reconhecer que é o acessor *property* — não o acessor *method*. Podemos fornecer acessores semelhantes a classes a um objeto JavaScript usando uma função como valor — mas isso não torna o objeto uma classe.

Uma propriedade do acessor tem os seguintes atributos:

`get`

Uma função chamada com uma lista de argumentos vazia para recuperar o valor da propriedade sempre que um acesso de obtenção ao valor for executado. Consulte também getters. Pode ser `indefinido`.

`definir`

Uma função chamada com um argumento que contém o valor atribuído. Executado sempre que se tenta alterar uma propriedade especificada. Consulte também setters. Pode ser `indefinido`.

`enumerável`

Um valor booleano que indica se a propriedade pode ser enumerada por um loop `for...in`. Consulte também Enumerabilidade e propriedade de propriedades para saber como a enumerabilidade interage com outras funções e sintaxes.

`configurável`

Um valor booleano que indica se a propriedade pode ser excluída, pode ser alterada para uma propriedade de dados e pode ter seus atributos alterados.

Objetos e funções "normais"

Um objeto JavaScript é um mapeamento entre *keys* e *values*. Chaves são strings (ou símbolos), e *values* pode ser qualquer coisa. Isso torna os objetos um ajuste natural para hashmaps.

Funções são objetos regulares com a capacidade adicional de serem *callable*.

Datas

Ao representar datas, a melhor opção é usar o utilitário `Date` integrado em JavaScript.

Coleções indexadas: Arrays e Arrays tipados

Arrays são objetos regulares para os quais existe uma relação específica entre propriedades de chave inteira e a propriedade `length`.

Além disso, arrays herdam de `Array.prototype`, que fornece a eles um punhado de métodos convenientes para manipular arrays. Por exemplo, `indexOf()` (pesquisando um valor no array) ou [`push()`](/en- US/docs/Web/JavaScript/Reference/Global_Objects/Array/push) (adicionando um elemento à matriz) e assim por diante. Isso torna os Arrays um candidato perfeito para representar listas ou conjuntos.

Typed Arrays apresentam uma visão semelhante a um array de um buffer de dados binários subjacente e oferecem muitos métodos que têm semântica semelhante aos equivalentes de array. "Typed array" é um termo abrangente para uma variedade de estruturas de dados, incluindo `Int8Array`, `Float32Array`, etc. Verifique a página typed array para obter mais informações .

Coleções chaveadas: Maps, Sets, WeakMaps, WeakSets

Essas estruturas de dados usam referências de objetos como chaves. `Set` e `WeakSet` representante reenviam um conjunto de objetos, enquanto `Map` e `WeakMap` associam um valor a um objeto.

A diferença entre `Maps` e `WeakMaps` é que no primeiro, as chaves de objeto podem ser enumeradas. Isso permite otimizações de coleta de lixo no último caso.

Você pode implementar `Maps` e `Sets` você mesmo. No entanto, como os objetos não podem ser comparados (no sentido de `<` "menor que", por exemplo), nem o mecanismo expõe sua função de hash para objetos, o desempenho da pesquisa seria necessariamente linear. Implementações nativas deles (incluindo `WeakMaps`) podem ter um desempenho de pesquisa que é aproximadamente logarítmico para tempo constante.

Normalmente, para vincular dados a um nó DOM, pode-se definir propriedades diretamente no objeto ou usar atributos `data-*`. Isso tem a desvantagem de que os dados estão disponíveis para qualquer script em execução no mesmo contexto. `Maps` e `WeakMaps` facilitam *privately* vincular dados a um objeto.

Dados estruturados: JSON

JSON (**J**ava**S**cript **O**bject **N**otation) é um formato leve de intercâmbio de dados, derivado do JavaScript, mas usado por muitas linguagens de programação. JSON cria estruturas de dados universais.

Consulte JSON e `JSON` para obter mais detalhes.

Mais objetos na biblioteca padrão

JavaScript tem uma biblioteca padrão de objetos internos.

Por favor, dê uma olhada na referência para saber mais sobre objetos.

Determinando tipos usando o operador <code>typeof</code>

O operador `typeof` pode ajudá-lo a encontrar o tipo de sua variável.

Leia a página de referência para obter mais detalhes e casos extremos.

Veja também

- Estruturas de dados e algoritmos JavaScript por Oleksii Trekhleb
- Coleção de Nicholas Zakas de estrutura de dados comum e algoritmos comuns em JavaScript.
- Search Tre(i)es implementados em JavaScript
- Tipos de dados e valores na especificação ECMAScript

Enumerabilidade e posse de propriedades

Propriedades enumeráveis são aquelas propriedades cuja flag interna [[Enumerable]] é verdadeira (true), que é o padrão para propriedades criadas via assinatura simples ou através de um inicializador (propriedades definidas através de Object.defineProperty e tipo padrão [[Enumerable]] falso (false)).Propriedades enumeráveis aparecem em for...in loops exceto se o nome da propriedade for um objeto Symbol. Posse de propriedades é determinada pelo fato da propriedade pertencer ao objeto diretamente e não a uma cadeira de protótipos. Propriedades de um objeto pode também ser recuperadas diretamente. Há um número de built-in de detecção, iteração/enumeração e recuperação de propriedades, com o gráfico mostrando que estão disponíveis.O código de exemplo a seguir demostra como obter as categorias que faltam.

Propriedade de enumerabilidade e posse - métodos internos de detecção, recuperação, e iteração

<table>
<tr><th>Funcionalidade</th><th colspan="3">Próprio objeto</th><th>Próprio objeto e sua cadeia de caracteres</th></tr>
<tr><td>Detecção</td><td colspan="3"><table><tr><th>Enumerável</th><th>Não enumerável</th><th>Enumerável e Não enumerável</th></tr><tr><td><code>propertyIsEnumerable</code></td><td><code>hasOwnProperty</code> e não <code>propertyIsEnumerable</code></td><td><code>hasOwnProperty</code></td></tr></table></td><td>Não disponível sem código extra</td></tr>
<tr><td>Recuperação</td><td colspan="3"><table><tr><th>Enumerável</th><th>Não enumerável</th><th>Enumerável e Não enumerável</th></tr><tr><td><code>Object.keys</code></td><td><code>getOwnPropertyNames</code> filtrou-se para incluir as propriedades quando não passado <code>propertyIsEnumerable</code></td><td><code>getOwnPropertyNames</code></td></tr></table></td><td>Não disponível sem código extra</td></tr>
<tr><td>Iteração</td><td colspan="3"><table><tr><th>Enumerável</th><th>Não enumerável</th><th>Enumerável e Não enumerável</th></tr><tr><td>Iterar <code>Object.keys</code></td><td>itera <code>getOwnPropertyNames</code> filtrou-se para incluir as propriedades quando não passado <code>propertyIsEnumerable</code></td><td><code>iterar</code><code>getOwnPropertyNames</code></td></tr></table></td><td><table><tr><th>Enumerável</th><th>Não enumerável</th><th>Enumerável e Não enumerável</th></tr><tr><td><code>for..in</code></td><td>Não disponível sem código extra</td><td>Não disponível sem código extra</td></tr></table></td></tr>
</table>

Obtendo propriedades por enumerabilidade/posse

Note que não é o algoritmo mais eficiente para todos os casos, mas útil para uma demonstração rápida.

- Detecção pode ocorrer por `SimplePropertyRetriever.theGetMethodYouWant(obj).indexOf(prop) > -1`
- Iteração pode ocorrer por `SimplePropertyRetriever.theGetMethodYouWant(obj).forEach(function (value, prop) {});` (ou use `filter()`, `map()`, etc.)

```
var SimplePropertyRetriever = {
  getOwnEnumerables: function (obj) {
    return this._getPropertyNames(obj, true, false, this._enumerable);
    // Ou poderia usar for..in filtrado com hasOwnProperty ou apenas isto: return Object.keys(obj);
  },
  getOwnNonenumerables: function (obj) {
    return this._getPropertyNames(obj, true, false, this._notEnumerable);
  },
  getOwnEnumerablesAndNonenumerables: function (obj) {
    return this._getPropertyNames(
      obj,
      true,
      false,
      this._enumerableAndNotEnumerable,
    );
    // Ou apenas use: return Object.getOwnPropertyNames(obj);
  },
  getPrototypeEnumerables: function (obj) {
    return this._getPropertyNames(obj, false, true, this._enumerable);
  },
  getPrototypeNonenumerables: function (obj) {
    return this._getPropertyNames(obj, false, true, this._notEnumerable);
  },
  getPrototypeEnumerablesAndNonenumerables: function (obj) {
    return this._getPropertyNames(
      obj,
      false,
      true,
      this._enumerableAndNotEnumerable,
    );
  },
  getOwnAndPrototypeEnumerables: function (obj) {
    return this._getPropertyNames(obj, true, true, this._enumerable);
    // Ou poderia usar filtrado for..in
  },
  getOwnAndPrototypeNonenumerables: function (obj) {
    return this._getPropertyNames(obj, true, true, this._notEnumerable);
  },
  getOwnAndPrototypeEnumerablesAndNonenumerables: function (obj) {
    return this._getPropertyNames(
      obj,
```

```
      true,
      true,
      this._enumerableAndNotEnumerable,
    );
  },
  // Private static property checker callbacks
  _enumerable: function (obj, prop) {
    return obj.propertyIsEnumerable(prop);
  },
  _notEnumerable: function (obj, prop) {
    return !obj.propertyIsEnumerable(prop);
  },
  _enumerableAndNotEnumerable: function (obj, prop) {
    return true;
  },
  // Inspirado por http://stackoverflow.com/a/8024294/271577
  _getPropertyNames: function getAllPropertyNames(
    obj,
    iterateSelfBool,
    iteratePrototypeBool,
    includePropCb,
  ) {
    var props = [];

    do {
      if (iterateSelfBool) {
        Object.getOwnPropertyNames(obj).forEach(function (prop) {
          if (props.indexOf(prop) === -1 && includePropCb(obj, prop)) {
            props.push(prop);
          }
        });
      }
      if (!iteratePrototypeBool) {
        break;
      }
      iterateSelfBool = true;
    } while ((obj = Object.getPrototypeOf(obj)));

    return props;
  },
};
```

Tabela de Detecção

	`in`	`for..in`	`hasOwnProperty`	`propertyIsEnumerable`	`in Object.keys`	`in Object.getOwnPropertyNames`	**in Object.getOwnPropertyDescriptors**
Enumerável	true	true	true	true	true	true	true
Não enumerável	true	false	true	false	false	true	true
Inherited Enumerable	true	true	false	false	false	false	false
Inherited Nonenumerable	true	false	false	false	false	false	false
Account for Symbols keys	true	false	true	true	false	false	true

Veja também

- `in`
- `for..in`
- `hasOwnProperty`
- `propertyIsEnumerable`
- `getOwnPropertyNames`
- `Object.keys`
- `Object.getOwnPropertyDescriptors`

Comparações de igualdade e uniformidade

JavaScript fornece três diferentes operações de comparação de valores:

- igualdade estrita (ou "três iguais" ou "identidade") usando `===`,
- igualdade ampla ("dois iguais") usando `==`,
- e `Object.is` (novo no ECMAScript 6).

A escolha de qual operação usar depende da ordem de comparação que você está procurando executar.

Em resumo, dois iguais irá realizar uma conversão de tipo ao comparar duas coisas; três iguais irá fazer a mesma comparação, sem conversão de tipo (simplesmente retornando false se os tipos forem diferentes); e `Object.is` irá se comportar da mesma forma que três iguais, mas com tratamento especial para `NaN` e `-0` e `+0` de modo que os dois últimos não são referidos como sendo o mesmo, enquanto `Object.is(NaN, NaN)` será `true`. (Comparando `NaN` com `NaN` ordinariamente - ou seja, usando tanto iguais duplo ou iguais triplo - avalia como `false`, porque IEEE 754 diz isso.) Note que a distinção entre todos eles têm a ver com seu manuseio dos primitivos;

nenhum deles compara se os parâmetros são conceitualmente semelhante em estrutura. Para qualquer objetos não-primitivo x e y que têm a mesma estrutura, mas são objetos distintos entre si, todas as formas acima será avaliada como falsa.

Igualdade estrita usando <code>===</code>

Igualdade estrita compara dois valores para a igualdade. Nenhum valor é convertido implicitamente para algum outro valor antes de serem comparados. Se os valores tem tipos diferentes, os valores são considerados não-iguais. Caso contrário, se os valores têm o mesmo tipo e não são números, eles são considerados iguais, se tiverem o mesmo valor. Finalmente, se ambos os valores são números, eles são considerados iguais se ambos não são `NaN` e são do mesmo valor, ou se um é `+0` e outro é `-0`.

```
var num = 0;
var obj = new String("0");
var str = "0";

console.log(num === num); // true
console.log(obj === obj); // true
console.log(str === str); // true

console.log(num === obj); // false
console.log(num === str); // false
console.log(obj === str); // false
console.log(null === undefined); // false
console.log(obj === null); // false
console.log(obj === undefined); // false
```

Igualdade estrita é quase sempre a operação de comparação correta a se usar. Para todos os valores, exceto os números, ele usa a semântica óbvia: um valor é apenas igual a ele mesmo. Para os números que ele usa uma semântica ligeiramente diferente para encobrir dois casos extremamente diferentes. A primeira é que o zero de ponto flutuante é positiva ou negativamente assinada. Isso é útil em representar certas soluções matemáticas, mas como a maioria das situações não se preocupam com a diferença entre `+0` e `-0`, igualdade estrita trata-os como o mesmo valor. A segunda é que ponto flutuante inclui o conceito de um valor não-numérico, `NaN`, para representar a solução para certos problemas matemáticos mal definidos: infinito negativo adicionado ao infinito positivo, por exemplo. Igualdade estrita trata `NaN` como desigual a qualquer outro valor - incluindo o próprio. (O único caso em que `(x! == X)` é `true` é quando `x` é `NaN`.)

Igualdade ampla usando ==

Igualdade ampla compara dois valores para a igualdade, *após* converter ambos os valores para um tipo comum. Após as conversões (um ou ambos os lados podem sofrer conversões), a comparação de igualdade final é realizada exatamente como `===` executa. Igualdade ampla é *simétrica*: `A == B` sempre tem semântica idêntica à `B == A` para quaisquer valores de `A` e `B`.

A comparação de igualdade é realizado da seguinte forma para os operandos dos vários tipos:

		Operand B					
		Undefined	Null	Number	String	Boolean	Object
Operand A	Undefined	`true`	`true`	`false`	`false`	`false`	`IsFalsy(B)`
	Null	`true`	`true`	`false`	`false`	`false`	`IsFalsy(B)`
	Number	`false`	`false`	`A === B`	`A === ToNumber(B)`	`ToNumber(B) === A`	`ToPrimitive(B) == A`
	String	`false`	`false`	`B === ToNumber(A)`	`A === B`	`ToNumber(A) === ToNumber(B)`	`ToPrimitive(B) == A`
	Boolean	`false`	`false`	`ToNumber(A) === B`	`ToNumber(A) === ToNumber(B)`	`A === B`	`false`
	Object	`IsFalsy(A)`	`IsFalsy(A)`	`ToPrimitive(A) == B`	`ToPrimitive(A) == B`	`false`	`A === B`

Na tabela acima, `ToNumber(A)` tenta converter seu argumento para um número antes de comparação. Seu comportamento é equivalente a `+A` (o operador + unário). `ToPrimitive(A)` tenta converter seu argumento de objeto para um valor primitivo, tentando invocar sequências diferentes de `A.toString` e `A.valueOf` métodos em `A`.

Tradicionalmente, e de acordo com ECMAScript, todos os objetos são amplamente desiguais a `undefined` e `null`. Mas a maioria dos navegadores permitem uma classe muito limitada de objetos (especificamente, o objeto `document.all` para qualquer página), em alguns contextos, para agir como se eles *emulassem* o valor `undefined`. Igualdade ampla é um desses contexto. Portanto, o método `IsFalsy(A)` é avaliada como verdadeira se, e somente se, A for um objeto que *emula* `undefined`. Em todos os outros casos, um objeto não é amplamente igual a `undefined` ou `null`.

```
var num = 0;
var obj = new String("0");
var str = "0";

console.log(num == num); // true
console.log(obj == obj); // true
console.log(str == str); // true

console.log(num == obj); // true
console.log(num == str); // true
console.log(obj == str); // true
console.log(null == undefined); // true

// both false, except in rare cases
console.log(obj == null);
console.log(obj == undefined);
```

Alguns desenvolvedores consideram que nunca é uma boa idéia, praticamente, usar a igualdade ampla. O resultado de uma comparação usando a igualdade estrita é mais fácil de prever, e como nenhum tipo de coerção ocorre, a avaliação pode ser mais rápida.

Igualdade de mesmo valor

Igualdade de mesmo valor aborda um caso de uso final: determinar se dois valores são *funcionalmente idêntico* em todos os contextos. (Este caso de uso demonstra uma instância de Liskove substitution principle. Princípio de substituição de Liskove.) Uma instância ocorre quando é feita uma tentativa de transformar uma propriedade imutável:

```
// Add an immutable NEGATIVE_ZERO property to the Number constructor.
Object.defineProperty(Number, "NEGATIVE_ZERO", {
  value: -0,
  writable: false,
  configurable: false,
  enumerable: false,
});

function attemptMutation(v) {
  Object.defineProperty(Number, "NEGATIVE_ZERO", { value: v });
}
```

`Object.defineProperty` irá disparar uma exceção ao tentar alterar uma propriedade imutável que realmente iria alterá-lo, mas ele não faz nada se nenhuma mudança real for solicitada. Se `v` for `-0`, nenhuma mudança foi solicitada, e nenhum erro será disparado. Mas se `v` for `+0`, `Number.NEGATIVE_ZERO` não teria mais o seu valor imutável. Internamente, quando uma propriedade imutável é redefinida, o valor recém-especificado é comparado com o valor atual usando a igualdade de mesmo valor.

Igualdade de mesmo valor é fornecida pelo método `Object.is`.

Igualdade abstrata, igualdade estrita, e de mesmo valor na especificação

No ES5, a comparação efetuada por `==` é descrita em Seção 11.9.3, O Algoritmo da Igualdade Abstrata. A comparação `===` está em 11.9.6, O Algoritmo de Igualdade Estrita. (Veja estes. Eles são breve e legível. Dica: leia o algoritmo de igualdade estrita primeiro.) ES5 também descreve, em Seção 9.12, o Algoritmo de Mesmo Valor para uso internamente pelo motor de JS. É em grande parte o mesmo que o Algoritmo de Igualdade Estrita, exceto que 11.9.6.4 e 9.12.4 diferem em manipulação do `Number`. ES6 simplesmente se propõe a expor este algoritmo através de `Object.is`.

Podemos ver que com iguais duplos e triplos, com exceção de fazer uma verificação de tipo inicial 11.9.6.1, o Algoritmo de Igualdade Estrita é um subconjunto do Algoritmo Igualdade Abstrata, porque 11.9.6.2-7 correspondem a 11.9.3.1.a -f.

Um modelo para a compreensão comparações de igualdade?

Antes do ES6, você poderia dizer, de iguais duplos e iguais triplos, que um é uma versão "melhorada" do outro. Por exemplo, alguém poderia dizer que dois iguais é uma versão estendida de três iguais, porque o anterior faz tudo o que este último faz, mas com conversão de tipo nos seus operandos. Por exemplo, 6 == "6". (Alternativamente, alguém poderia dizer que dois iguais é a linha de base, e três iguais são uma versão melhorada, porque requer dois operandos para ser do mesmo tipo, por isso adiciona uma restrição extra. Qual deles é o melhor modelo para compreensão depende como você optar por ver as coisas.)

No entanto, esta forma de pensar sobre os operadores de igualdade embutida não é um modelo que pode ser esticado para permitir um lugar para Object.is de ES6 neste "espectro". Object.is não é simplesmente "mais frouxo" que dois iguais ou "mais rigorosa" do que três iguais, nem se encaixa em algum lugar entre eles (ex: sendo tanto mais rigorosas que dois iguais, mas mais flexível do que três iguais). Podemos ver a partir da tabela de comparações de igualdade abaixo que isto é devido à forma que Object.is manipula NaN. Observe que, se Object.is(NaN, NaN) avalia como false, nós *poderiamos* dizer que ele se encaixa no espectro amplo/estrito como um forma ainda mais rigorosa das três iguais, um que distingue entre -0 e +0. A manipulação do NaN significa que este é não-verdadeiro, entretanto. Infelizmente, Object.is simplesmente tem que ser pensado em termos de suas características específicas, ao invés de sua amplitude ou rigorosidade no que diz respeito aos operadores de igualdade.

x	y	==	===	Object.is
undefined	undefined	true	true	true
null	null	true	true	true
true	true	true	true	true
false	false	true	true	true
"foo"	"foo"	true	true	true
{ foo: "bar" }	x	true	true	true
0	0	true	true	true
+0	-0	true	true	false
0	false	true	false	false
""	false	true	false	false
""	0	true	false	false
"0"	0	true	false	false
"17"	17	true	false	false
[1,2]	"1,2"	true	false	false
new String("foo")	"foo"	true	false	false
null	undefined	true	false	false
null	false	false	false	false
undefined	false	false	false	false

x	y	==	===	Object.is
`{ foo: "bar" }`	`{ foo: "bar" }`	`false`	`false`	`false`
`new String("foo")`	`new String("foo")`	`false`	`false`	`false`
`0`	`null`	`false`	`false`	`false`
`0`	`NaN`	`false`	`false`	`false`
`"foo"`	`NaN`	`false`	`false`	`false`
`NaN`	`NaN`	`false`	`false`	`true`

Quando usar <code>Object.is</code> ao invés de três iguais

Além da forma como trata `NaN`, geralmente, a única vez que o comportamento especial do Object.is em direção a zeros é provável que seja interessante é na busca de determinados regimes de meta-programação, especialmente em relação a descritores de propriedade quando é desejável para o seu trabalho para espelhar algumas das características de Object.defineProperty. Se o seu caso de uso não exige isso, sugere-se a evitar Object.is e usar `===` em vez disso. Mesmo se suas exigências envolvem comparações entre dois valores NaN avaliarem para true, geralmente é mais fácil para caso especial o NaN checar (usando o isNaN método disponível a partir de versões anteriores do ECMAScript) do que descobrir como cálculos circundantes podem afetar o sinal de todos os zeros que você encontra em sua comparação.

Aqui está uma lista exaustiva de métodos e operadores integrados que pode causar uma distinção entre -0 e +0 a manifestar-se em seu código:

`- (negação unário)`

> É óbvio que negando 0 produz -0. Mas a abstração de uma expressão pode causar -0 a fluência em quanto você não percebe isso. Por exemplo, considere:
>
> ```
> let stoppingForce = obj.mass * -obj.velocity;
> ```
>
> If obj.velocity is 0 (ou calcula para 0), um -0 é introduzido naquele lugar e propaga-se em stoppingForce.

`Math.atan2Math.ceilMath.powMath.round`

> É possível que um -0 para ser introduzido em uma expressão como um valor de retorno desses métodos, em alguns casos, mesmo quando nenhum -0 exista como um dos parâmetros. Por exemplo, usando Math.pow para levantar -Infinity a potência de qualquer número, expoente ímpar negativo avaliada como -0. Consulte a documentação para os métodos individuais.

`Math.floorMath.maxMath.minMath.sinMath.sqrtMath.tan`

> É possível obter um -0 valor de retorno para fora destes métodos em alguns casos em que um -0 existe como um dos parâmetros. Por exemplo, `Math.min(-0, +0)` resulte em -0. Consulte a documentação para os métodos individuais.

`~<<>>`

> Cada um destes operadores usa o algoritmo ToInt32 internamente. Uma vez que existe apenas uma representação para 0 no tipo integer de 32 bits interno, -0 não vai sobreviver a uma ida e volta após uma operação inversa. Por exemplo, tanto `Object.is(~~(-0), -0)` e `Object.is(-0 << 2 >> 2, -0)` avaliar como `false`.

Baseando-se em `Object.is` quando a sinalização de zeros não é levado em conta podem ser perigosos. Claro que, quando a intenção é fazer a distinção entre `-0` e `+0`, isso faz exatamente o que é desejado.

Veja também

- Tabela de Comparação JS

Expressões e operadores

- Anterior
- Próxima

Este artigo descreve expressões e operadores de JavaScript, incluindo operadores de atribuição, comparação, aritméticos, bit a bit, lógicos, de strings e especiais.

Operadores

O JavaScript possui os tipos de operadores a seguir. Esta seção descreve os operadores e contém informações sobre precedência de operadores.

- Operadores de atribuição
- Operadores de comparação
- Operadores aritméticos
- Operadores bit a bit
- Operadores lógicos
- Operadores de string
- Operador condicional (ternário)
- Operador vírgula
- Operadores unário
- Operadores relacionais

O JavaScript possui tanto operadores *binários* quanto *unários* e um operador ternário, o operador condicional. Um operador binário exige dois operandos, um antes do operador e outro depois:

```
operando1 operador operando2
```

Por exemplo, `3+4` ou `x*y`.

Um operador unário exige um único operando, seja antes ou depois do operador:

```
operador operando
```

ou

```
operando operador
```

Por exemplo, x++ ou ++x.

Operadores de atribuição

Um operador de atribuição atribui um valor ao operando à sua esquerda baseado no valor do operando à direita. O operador de atribuição básico é o igual (=), que atribui o valor do operando à direita ao operando à esquerda. Isto é, x = y atribui o valor de y a x.

Os outros operadores de atribuição são encurtamentos de operadores padrão, como mostrado na tabela a seguir.

Nome	Operador encurtado	Significado
Atribuição	x = y	x = y
Atribuição de adição	`x += y`	`x = x + y`
Atribuição de subtração	`x -= y`	`x = x - y`
Atribuição de multiplicação	`x *= y`	`x = x * y`
Atribuição de divisão	`x /= y`	`x = x / y`
Atribuição de resto	`x %= y`	`x = x % y`
Atribuição exponencial	x **= y	x = x ** y
Atribuição bit-a-bit por deslocamento á esquerda	`x <<= y`	`x = x << y`
Atribuição bit-a-bit por deslocamento á direita	`x >>= y`	`x = x >> y`
Atribuiçãode bit-a-bit deslocamento á direita não assinado	`x >>>= y`	`x = x >>> y`
Atribuição AND bit-a-bit	`x &= y`	`x = x & y`
Atribuição XOR bit-a-bit	`x ^= y`	`x = x ^ y`
Atribuição OR bit-a-bit	`x |= y`	`x = x | y`

Operadores de comparação

Um operador de comparação compara seus operandos e retorna um valor lógico baseado em se a comparação é verdadeira. Os operandos podem ser numéricos, strings, lógicos ou objetos. Strings são comparadas com base em ordenação lexográfica utilizando valores Unicode. Na maioria dos casos, se dois operandos não são do mesmo tipo, o JavaScript tenta convertê-los para um tipo apropriado. Isto geralmente resulta na realização de uma comparação numérica. As únicas exceções a esta regra são os operadores `===` e o `!==`, que realizam comparações de igualdade e desigualdade "estritas". Estes operadores não tentam converter os operandos em tipos compatíveis antes de verificar a igualdade. A tabela a seguir descreve os operadores de comparação levando em conta o seguinte código:

```
var var1 = 3;
var var2 = 4;
```

Operador	Descrição	Exemplos que retornam verdadeiro
Igual (==)	Retorna verdadeiro caso os operandos sejam iguais.	`3 == var1` `"3" == var1` `3 == '3'`
Não igual (!=)	Retorna verdadeiro caso os operandos não sejam iguais.	`var1 != 4` `var2 != "3"`
Estritamente igual (===)	Retorna verdadeiro caso os operandos sejam iguais e do mesmo tipo. Veja também `Object.is` e igualdade em JS.	`3 === var1`
Estritamente não igual (!==)	Retorna verdadeiro caso os operandos não sejam iguais e/ou não sejam do mesmo tipo.	`var1 !== "3"` `3 !== '3'`
Maior que (>)	Retorna verdadeiro caso o operando da esquerda seja maior que o da direita.	`var2 > var1` `"12" > 2`
Maior que ou igual (>=)	Retorna verdadeiro caso o operando da esquerda seja maior ou igual ao da direita.	`var2 >= var1` `var1 >= 3`
Menor que (<)	Retorna verdadeiro caso o operando da esquerda seja menor que o da direita.	`var1 < var2` `"12" < "2"`
Menor que ou igual (<=)	Retorna verdadeiro caso o operando da esquerda seja menor ou igual ao da direita.	`var1 <= var2` `var2 <= 5`

> **Nota:** (**=>**) não é um operador, mas a notação para função de seta

Operadores aritméticos

Operadores aritméticos tomam valores numéricos (sejam literais ou variáveis) como seus operandos e retornam um único valor numérico. Os operadores aritméticos padrão são os de soma (+), subtração (-), multiplicação (*) e divisão (/). Estes operadores trabalham da mesma forma como na maioria das linguagens de programação quando utilizados com números de ponto flutuante (em particular, repare que divisão por zero produz um `NaN`). Por exemplo:

```
console.log(1 / 2); /* imprime 0.5 */
console.log(1 / 2 == 1.0 / 2.0); /* isto também é verdadeiro */
```

Em complemento às operações aritméticas padrões (+, -, * /), o JavaScript disponibiliza os operadores aritméticos listados na tabela a seguir.

Operador	Descrição	Exemplo
Módulo (%)	Operador binário. Retorna o inteiro restante da divisão dos dois operandos.	12 % 5 retorna 2.
Incremento (++)	Operador unário. Adiciona um ao seu operando. Se usado como operador prefixado (++x), retorna o valor de seu operando após a adição. Se usado como operador pósfixado (x++), retorna o valor de seu operando antes da adição.	Se x é 3, então ++x define x como 4 e retorna 4, enquanto x++ retorna 3 e, somente então, define x como 4.
Decremento (--)	Operador unário. Subtrai um de seu operando. O valor de retorno é análogo àquele do operador de incremento.	Se x é 3, então --x define x como 2 e retorna 2, enquanto x-- retorna 3 e, somente então, define x como 2.
Negação (-)	Operador unário. Retorna a negação de seu operando.	Se x é 3, então -x retorna -3.
Adição (+)	Operador unário. Tenta converter o operando em um número, sempre que possível.	+"3" retorna 3.+true retorna 1.
Operador de exponenciação (**)	Calcula a base elevada á potência do expoente, que é, base`expoente`	2 ** 3 retorna 8.10 ** -1 retorna 0.1

Operadores bit a bit

Operadores bit a bit tratam seus operandos como um conjunto de 32 bits (zeros e uns), em vez de tratá-los como números decimais, hexadecimais ou octais. Por exemplo, o número decimal nove possui uma representação binária 1001. Operadores bit a bit realizam suas operações nestas representações, mas retornam valores numéricos padrões do JavaScript.

A tabela a seguir resume os operadores bit a bit do JavaScript.

Operador	Expressão	Descrição
AND	`a & b`	Retorna um 1 para cada posição em que os bits da posição correspondente de ambos operandos sejam uns.
OR	`a | b`	Retorna um 0 para cada posição em que os bits da posição correspondente de ambos os operandos sejam zeros.
XOR	`a ^ b`	Retorna um 0 para cada posição em que os bits da posição correspondente são os mesmos. [Retorna um 1 para cada posição em que os bits da posição correspondente sejam diferentes.]
NOT	`~ a`	Inverte os bits do operando.
Deslocamento à esquerda	`a << b`	Desloca `a` em representação binária `b` bits à esquerda, preenchendo com zeros à direita.
Deslocamento à direita com propagação de sinal	`a >> b`	Desloca `a` em representação binária `b` bits à direita, descartando bits excedentes.
Deslocamento à direita com preenchimento zero	`a >>> b`	Desloca `a` em representação binária `b` bits à direita, descartando bits excedentes e preenchendo com zeros à esquerda.

Operadores bit a bit lógicos

Conceitualmente, os operadores bit a bit lógicos funcionam da seguinte maneira:

- Os operandos são convertidos em inteiros de 32 bits e expressos como uma série de bits (zeros e uns). Números com representação maior que 32 bits terão seus bits truncados. Por exemplo, o seguinte inteiro tem representação binária maior que 32 bits será convertido em um inteiro de 32 bits.

```
Antes: 11100110111110100000000000000110000000000001
Depois:            10100000000000000110000000000001
```

- Cada bit do primeiro operando é pareado com o bit correspondente do segundo operando: primeiro bit com primeiro bit, segundo bit com segundo bit e assim por diante.
- O operador é aplicado a cada par de bits e o resultado é construído bit a bit.

Por exemplo, a representação binária de nove é 1001 e a representação binária de quinze é 1111. Desta forma, quando operadores bit a bit são aplicados a estes valores, os resultados são como se segue:

Expressão	Resultado	Descrição binária
`15 & 9`	`9`	`1111 & 1001 = 1001`
`15 | 9`	`15`	`1111 | 1001 = 1111`
`15 ^ 9`	`6`	`1111 ^ 1001 = 0110`
`~15`	`-16`	`~ 0000 0000 … 0000 1111 = 1111 1111 … 1111 0000`
`~9`	`-10`	`~ 0000 0000 … 0000 1001 = 1111 1111 … 1111 0110`

> **Nota:** No quadro acima perceba que todos os 32 bits são invertidos quando usa-se o operador bit a bit NOT, e que os bits mais significativos (extrema esquerda) são definidos com 1 que representam valores negativos (representação complemento de dois).

Operadores de deslocamento bit a bit

Os operadores de deslocamento bit a bit possui dois operandos: o primeiro é uma quantidade a ser deslocada e o segundo especifica o número de posições binárias as quais o primeiro operando deverá ser deslocado. A direção da operação de deslocamento é controlada pelo operador utilizado.

Operadores de deslocamento convertem seus operandos em inteiros de 32 bits e retornam um resultado do tipo do operando à esquerda.

Os operadores de deslocamento são listados na tabela a seguir.

Operador	Descrição	Exemplo
`Deslocamento à esquerda (<<)`	Este operador desloca o primeiro operando pelo número especificado de bits à esquerda. Bits excedentes deslocados para fora do limite à esquerda são descartados. Bits zero são inseridos à direita.	`9<<2` produz 36 porque 1001 deslocado 2 bits à esquerda se torna 100100, que é 36.
Deslocamento à direita com propagação de sinal (`>>`)	Este operador desloca o primeiro operando pelo número especificado de bits à direita. Bits excedentes deslocados para fora do limite à direita são descartados. Cópias dos bits mais à esquerda são deslocadas a partir da esquerda.	`9>>2` produz 2 porque 1001 deslocado 2 bits à direita se torna 10, que é 2. De forma similar, `-9>>2` produz -3 porque o sinal é preservado.
Deslocamento à direita com preenchimento zero (`>>>`)	Este operador desloca o primeiro operando pelo número especificado de bits à direita. Bits excedentes deslocados para fora do limite à direita são descartados. Bits zero são inseridos à esquerda.	`19>>>2` produz 4 porque 10011 deslocado 2 bits à direita se torna 100, que é 4. Para números não negativos o deslocamento à direita com propagação de sinal e o deslocamento à direita com preenchimento zero produzem o mesmo resultado.

Operadores lógicos

Operadores lógicos são utilizados tipicamente com valores booleanos (lógicos); neste caso, retornam um valor booleano. Entretanto, os operadores && e || na verdade retornam o valor de um dos operandos especificados, de forma que se esses operadores forem utilizados com valores não-booleanos, eles possam retornar um valor não-booleano. Os operadores lógicos são descritos na seguinte tabela.

Operador	Utilização	Descrição
`AND lógico (&&)`	`expr1 && expr2`	(E lógico) - Retorna `expr1` caso possa ser convertido para falso; senão, retorna `expr2`. Assim, quando utilizado com valores booleanos, `&&` retorna verdadeiro caso ambos operandos sejam verdadeiros; caso contrário, retorna falso.
`OU lógico (||)`	`expr1 || expr2`	(OU lógico) - Retorna `expr1` caso possa ser convertido para verdadeiro; senão, retorna `expr2`. Assim, quando utilizado com valores booleanos, `||` retorna verdadeiro caso ambos os operandos sejam verdadeiro; se ambos forem falsos, retorna falso.
`NOT lógico (!)`	`!expr`	(Negação lógica) Retorna falso caso o único operando possa ser convertido para verdadeiro; senão, retorna verdadeiro.

Exemplos de expressões que podem ser convertidas para falso são aquelas que são avaliados como nulo, 0, string vazia ("") ou `undefined`.

O código a seguir mostra exemplos do operador && (E lógico).

```
var a1 = true && true; // t && t retorna true
var a2 = true && false; // t && f retorna false
var a3 = false && true; // f && t retorna false
var a4 = false && 3 == 4; // f && f retorna false
var a5 = "Gato" && "Cão"; // t && t retorna Cão
var a6 = false && "Gato"; // f && t retorna false
var a7 = "Gato" && false; // t && f retorna false
```

O código a seguir mostra exemplos do operador || (OU lógico).

```
var o1 = true || true; // t || t retorna true
var o2 = false || true; // f || t retorna true
var o3 = true || false; // t || f retorna true
var o4 = false || 3 == 4; // f || f retorna false
var o5 = "Gato" || "Cão"; // t || t retorna Gato
var o6 = false || "Gato"; // f || t retorna Gato
var o7 = "Gato" || false; // t || f retorna Gato
```

O código a seguir mostra exemplos do operador ! (negação lógica).

```
var n1 = !true; // !t retorna false
var n2 = !false; // !f retorna true
var n3 = !"Gato"; // !t retorna false
```

Avaliação de curto-circuito

Como expressões lógicas são avaliadas da esquerda para a direita, elas são testadas como possíveis avaliações de "curto-circuito" utilizando as seguintes regras:

- `false` && *qualquercoisa* é avaliado em curto-circuito como falso.
- `true` || *qualquercoisa* é avaliado em curto-circuito como verdadeiro.

As regras de lógica garantem que estas avaliações estejam sempre corretas. Repare que a parte *qualquercoisa* das expressões acima não é avaliada, de forma que qualquer efeito colateral de fazê-lo não produz efeito algum.

Operadores de string

Além dos operadores de comparação, que podem ser utilizados em valores string, o operador de concatenação (+) concatena dois valores string, retornando outra string que é a união dos dois operandos.

Por exemplo,

```
 console.log("minha " + "string"); // exibe a string "minha string".
```

O operador de atribuição encurtado += também pode ser utilizado para concatenar strings.

Por exemplo,

```
var minhaString = "alfa";
minhaString += "beto"; // É avaliada como "alfabeto" e atribui este valor a minhastring.
```

Operador condicional (ternário)

O operador condicional é o único operador JavaScript que utiliza três operandos. O operador pode ter um de dois valores baseados em uma condição. A sintaxe é:

```
condicao ? valor1 : valor2
```

Se `condicao` for verdadeira, o operador terá o valor de `valor1`. Caso contrário, terá o valor de `valor2`. Você pode utilizar o operador condicional em qualquer lugar onde utilizaria um operador padrão.

Por exemplo,

```
var status = idade >= 18 ? "adulto" : "menor de idade";
```

Esta declaração atribui o valor "adulto" à variável `status` caso `idade` seja dezoito ou mais. Caso contrário, atribui o valor "menor de idade".

Operador vírgula

O operador vírgula (`,`) simplesmente avalia ambos de seus operandos e retorna o valor do segundo. Este operador é utilizado primariamente dentro de um laço `for` para permitir que multiplas variáveis sejam atualizadas cada vez através do laço.

Por exemplo, se `a` é uma matriz bidimensional com 10 elementos em um lado, o código a seguir utiliza o operador vírgula para incrementar duas variáveis de uma só vez. O código imprime os valores dos elementos diagonais da matriz:

```
for (var i = 0, j = 9; i <= 9; i++, j--)
  console.log("a[" + i + "][" + j + "]= " + a[i][j]);
```

Operadores unário

Um operador unário é uma operação com apenas um operando.

`delete`

O operador `delete` apaga um objeto, uma propriedade de um objeto ou um elemento no índice especificado de uma matriz. A sintaxe é:

```
delete nomeObjeto;
delete nomeObjeto.propriedade;
delete nomeObjeto[indice];
delete propriedade; // válido apenas dentro de uma declaração with
```

onde `nomeObjeto` é o nome de um objeto, `propriedade` é uma propriedade existente e `indice` é um inteiro que representa a localização de um elemento em uma matriz.

A quarta forma é permitida somente dentro de uma declaração `with` para apagar uma propriedade de um objeto.

Você pode utilizar o operador `delete` para apagar variáveis declaradas implicitamente mas não aquelas declaradas com `var`.

Se o operador `delete` for bem-sucedido, ele define a propriedade ou elemento para `undefined`. O operador `delete` retorna verdadeiro se a operação for possível; ele retorna falso se a operação não for possível.

```
x = 42;
var y = 43;
meuobj = new Number();
meuobj.h = 4; // cria a propriedade h
delete x; // retorna true (pode apagar se declarado implicitamente)
delete y; // retorna false (não pode apagar se declarado com var)
delete Math.PI; // retorna false (não pode apagar propriedades predefinidas)
delete meuobj.h; // retorna true (pode apagar propriedades definidas pelo usuário)
delete meuobj; // retorna true (pode apagar se declarado implicitamente)
```

Apagando elementos de array

Quando você apaga um elemento de um array, o tamanho do array não é afetado. Por exemplo, se você apaga `a[3]`, o valor de `a[4]` ainda estará em `a[4]` e `a[3]` passa a ser `undefined`.

Quando o operador `delete` remove um elemento do array, aquele elemento não pertence mais ao array. No exemplo a seguir, `arvores[3]` é removido com `delete`. Entretanto, `arvores[3]` ainda é endereçável e retorna `undefined`.

```
var arvores = new Array(
  "pau-brasil",
  "loureiro",
```

```
  "cedro",
  "carvalho",
  "sicômoro",
);
delete arvores[3];
if (3 in arvores) {
  // isto não é executado
}
```

Se você quer que um elemento do array exista, mas tenha um valor indefinido, utilize a palavra-chave `undefined` em vez do operador `delete`. No exemplo a seguir, o valor `undefined` é atribuído a `arvores[3]`, mas o elemento da matriz ainda existe:

```
var arvores = new Array(
  "pau-brasil",
  "loureiro",
  "cedro",
  "carvalho",
  "sicômoro",
);
arvores[3] = undefined;
if (3 in arvores) {
  // isto será executado
}
```

typeof

O operador `typeof` é utilizado em qualquer uma das seguintes formas:

```
typeof operando;
typeof (operando);
```

O operador `typeof` retorna uma string indicando o tipo do operando sem avaliação. `operando` é uma string, variável, palavra-chave ou objeto cujo tipo deve ser retornado. Os parênteses são opcionais.

Suponha que você defina as seguintes variáveis:

```
var meuLazer = new Function("5 + 2");
var forma = "redondo";
var tamanho = 1;
var hoje = new Date();
```

O operador `typeof` retornaria o seguinte resultado para aquelas variáveis:

```
typeof meuLazer; // retorna "function"
typeof forma; // retorna "string"
typeof tamanho; // retorna "number"
typeof hoje; // retorna "object"
typeof naoExiste; // retorna "undefined"
```

Para as palavras-chave `true` e `null`, o `typeof` retorna os seguintes resultados:

```
typeof true; // retorna "boolean"
typeof null; // retorna "object"
```

Para um número ou uma string, o `typeof` retorna os seguintes resultados:

```
typeof 62; // retorna "number"
typeof "Olá mundo"; // retorna "string"
```

Para valores de propriedades, o `typeof` retorna o tipo do valor que a propriedade possui:

```
typeof document.lastModified; // retorna "string"
typeof window.length; // retorna "number"
typeof Math.LN2; // retorna "number"
```

Para métodos e funções, o `typeof` retorna os seguintes resultados:

```
typeof blur; // retorna "function"
typeof eval; // retorna "function"
typeof parseInt; // retorna "function"
typeof forma.split; // retorna "function"
```

Para objetos predefinidos, o `typeof` retorna os seguintes resultados:

```
typeof Date; // retorna "function"
typeof Function; // retorna "function"
typeof Math; // retorna "object"
typeof Option; // retorna "function"
typeof String; // retorna "function"
```

`void`

O operador `void` é utilizado de qualquer uma das seguintes formas:

```
void (expressao);
void expressao;
```

O operador `void` especifica que uma expressão deve ser avaliada sem retorno de valor. `expressao` é uma expressão JavaScript que deve ser avaliada. Os parênteses em torno da expressão são opcionais, mas é uma boa prática utilizá-los.

Você pode utilizar o operador `void` para especificar uma expressão como um link de hipertexto. A expressão é avaliada mas não é carregada no lugar do documento atual.

O código a seguir cria um link de hipertexto que não faz coisa alguma quando clicado pelo usuário. Quando o usuário clica no link, `void(0)` é avaliado como indefinido, que não tem efeito em JavaScript.

```
<a href="javascript:void(0)">Clique aqui para fazer nada</a>
```

O código a seguir cria um link de hipertexto que submete um formulário quando clicado pelo usuário.

```
<a href="javascript:void(document.form.submit())">Clique aqui para enviar</a>
```

Operadores relacionais

Um operador relacional compara seus operando e retorna um valor booleano baseado em se a comparação é verdadeira.

`in`

O operador `in` retorna verdadeiro se a propriedade especificada estiver no objeto especificado. A sintaxe é:

```
nomePropriedadeOuNumero in nomeObjeto;
```

onde `nomePropriedadeOuNumero` é uma string ou uma expressão numérica que representa um nome de propriedade ou um índice de um array, e `nomeObjeto` é o nome de um objeto.

Os exemplos a seguir mostram alguns usos do operador `in`.

```
// Arrays
var arvores = new Array(
  "pau-brasil",
  "loureiro",
  "cedro",
  "carvalho",
  "sicômoro",
);
0 in arvores; // retorna verdadeiro
3 in arvores; // retorna verdadeiro
6 in arvores; // retorna falso
"cedro" in arvores; // retorna falso (você deve especificar o número do índice,
// não o valor naquele índice)
"length" in arvores; // retorna verdadeiro (length é uma propriedade de Array)

// Objetos predefinidos
"PI" in Math; // retorna verdadeiro
var minhaString = new String("coral");
"length" in minhaString; // retorna verdadeiro

// Objetos personalizados
var meucarro = { marca: "Honda", modelo: "Accord", ano: 1998 };
"marca" in meucarro; // retorna verdadeiro
"modelo" in meucarro; // retorna verdadeiro
```

`instanceof`

O operador `instanceof` retorna verdadeiro se o objeto especificado for do tipo de objeto especificado. A sintaxe é:

```
nomeObjeto instanceof tipoObjeto;
```

onde `nomeObjeto` é o nome do objeto a ser comparado com `tipoObjeto`, e `tipoObjeto` é um tipo de objeto como `Date` ou `Array`.

Utilize o `instanceof` quando você precisar confirmar o tipo de um objeto em tempo de execução. Por exemplo, ao capturar exceções você pode desviar para um código de manipulação de exceção diferente dependendo do tipo de exceção lançada.

Por exemplo, o código a seguir utiliza o `instanceof` para determinar se `dia` é um objeto `Date`. Como `dia` é um objeto `Date`, as declarações do `if` são executadas.

```
var dia = new Date(1995, 12, 17);
if (dia instanceof Date) {
  // declarações a serem executadas
}
```

Precedência de operadores

A *precedência* de operadores determina a ordem em que eles são aplicados quando uma expressão é avaliada. Você pode substituir a precedência dos operadores utilizando parênteses.

A tabela a seguir descreve a precedência de operadores, da mais alta para a mais baixa.

Tipo de operador	Operadores individuais
membro	`. []`
chamada / criação de instância	`() new`
negação / incremento	`! ~ - + ++ -- typeof void delete`
multiplicação / divisão / resto ou módulo	`* / %`
adição / subtração	`+ -`
deslocamento bit a bit	`<< >> >>>`
relacional	`< <= > >= in instanceof`
igualdade	`== != === !==`
E bit a bit	`&`
OU exclusivo bit a bit	`^`
OU bit a bit	`|`
E lógico	`&&`
OU lógico	`||`
condicional	`?:`
atribuição	`= += -= *= /= %= <<= >>= >>>= &= ^= |=`
vírgula	`,`

Uma versão mais detalhada desta tabela, com links adicionais para detalhes de cada operador, pode ser vista em Referência do JavaScript.

Expressões

Uma *expressão* consiste em qualquer unidade válida de código que é resolvida como um valor.

Conceitualmente, existem dois tipos de expressões: aquelas que atribuem um valor a uma variável e aquelas que simplesmente possuem um valor.

A expressão `x = 7` é um exemplo do primeiro tipo. Esta expressão utiliza o *operador* = para atribuir o valor sete à variável `x`. A expressão em si é avaliada como sete.

O código `3 + 4` é um exemplo do segundo tipo de expressão. Esta expressão utiliza o operador + para somar três e quatro sem atribuir o resultado, sete, a uma variável.

O JavaScript possui as seguintes categorias de expressão:

- Aritmética: é avaliada como um número, por exemplo 3.14159. (Geralmente utiliza operadores aritméticos).
- String: é avaliada como uma string de caracteres, por exemplo, "Fred" ou "234". (Geralmente utiliza operadores de string).
- Lógica: é avaliada como verdadeira ou falsa. (Costuma envolveroperadores lógicos).
- Expressões primárias: Palavras reservadas e expressões gerais do JavaScript.
- Expressão lado esquerdo: atribuição à esquerda de valores.

Expressões primárias

Palavras reservadas e expressões gerais do JavaScript.

`this`

Utilize a palavra reservada `this` para se referir ao objeto atual. Em geral, o `this` se refere ao objeto chamado em um método. Utilize o `this` das seguintes formas:

```
this["nomePropriedade"];
this.nomePropriedade;
```

Suponha uma função chamada `valide` que valida a propriedade `valor` de um objeto, dado o objeto e os valores máximo e mínimo:

```
function valide(obj, minimo, maximo) {
  if (obj.valor < minimo || obj.valor > maximo) alert("Valor inválido!");
}
```

Você poderia chamar `valide` em cada manipulador de evento `onChange` de um formulário utilizando `this` para passar o elemento do formulário, como no exemplo a seguir:

```
<b>Informe um número entre 18 e 99:</b>
<input type="text" name="idade" size="3" onChange="valide(this, 18, 99);" />
```

Operador de agrupamento

O operador de agrupamento `( )` controla a precedência de avaliação de expressões. Por exemplo, você pode substituir a precedência da divisão e multiplicação para que a adição e subtração sejam avaliadas primeiro.

```
var a = 1;
var b = 2;
var c = 3;

// Precedência padrão
a + b * c; // 7
// a avaliação padrão pode ser assim
a + (b * c); // 7

// Agora substitui a precedência
// soma antes de multiplicar
(a + b) * c; // 9

// o que é equivalente a
a * c + b * c; // 9
```

Comprehensions

Comprehensions são uma característica experimental de JavaScript, marcada para ser inclusa em uma versão futura do ECMAScript. Existem duas versões de Comprehensions:

`[for (x of y) x]` Comprehensions de array.

**`(for (x of y) y)` ** `gerador de` comprehensions

Comprehensions existem em muitas linguagens de programação e permitem que você rapidamente monte um novo array com base em um existente, por exemplo:

```
[for (i of [ 1, 2, 3 ]) i*i ];
// [ 1, 4, 9 ]

var abc = [ "A", "B", "C" ];
[for (letras of abc) letras.toLowerCase()];
// [ "a", "b", "c" ]
```

Expressão lado esquerdo

Atribuição à esquerda de valores.

`new`

Você pode utilizar o operador new para criar uma instância de um tipo de objeto definido pelo usuário ou de um dos tipos de objeto predefinidos: `Array`, `Boolean`, `Date`, `Function`, `Image`, `Number`, `Object`, `Option`, `RegExp` ou `String`. No servidor, você pode também utilizar `DbPool`, `Lock`, `File` ou `SendMail`. Utilize o operador `new` da seguinte forma:

```
var nomeObjeto = new tipoObjeto([parametro1, parametro2, ..., parametroN]);
```

`super`

A palavra reservada super é utilizada para chamar a função pai de um objeto. É útil para nas classes para a chamada do construtor pai, por exemplo:

```
super([argumentos]); //chama o construtor pai.
super.funcaoDoPai([argumentos]);
```

`Operador spread`

O operador `spread` permite que uma expressão seja expandido em locais onde são esperados vários argumentos (para chamadas de função) ou vários elementos (para arrays).

Exemplo: Se você tem um array e deseja criar um novo array com os elementos do array já existente sendo parte do novo array, a sintaxe do array não será suficiente e você terá de usar uma combinação de push, splice, concat, etc. Com a sintaxe `spread`, isto torna-se muito mais sucinto:

```
var partes = ["ombro", "joelhos"];
var musica = ["cabeca", ...partes, "e", "pés"];
```

Da mesma forma, o operador `spread` funciona com chamadas de função:

```
function f(x, y, z) {}
var args = [0, 1, 2];
f(...args);
```


- Anterior
- Próxima

Funções

- Anterior
- Próxima

Funções são blocos de construção fundamentais em JavaScript. Uma função é um procedimento de JavaScript - um conjunto de instruções que executa uma tarefa ou calcula um valor. Para usar uma função, você deve defini-la em algum lugar no escopo do qual você quiser chamá-la.

Veja também o capítulo de referência sobre funções JavaScript para conhecer os detalhes.

Definindo Funções

Declarando uma função

A **definição da função** (também chamada de **declaração de função**) consiste no uso da palavra chave `function`, seguida por:

- Nome da Função.
- Lista de argumentos para a função, entre parênteses e separados por vírgulas.
- Declarações JavaScript que definem a função, entre chaves `{ }`.

Por exemplo, o código a seguir define uma função simples chamada `square`:

```
function square(numero) {
  return numero * numero;
}
```

A função `square` recebe um argumento chamado `numero`. A função consiste em uma instrução que indica para retornar o argumento da função (isto é, `numero`) multiplicado por si mesmo. A declaração `return` especifica o valor retornado pela função.

```
return numero * numero;
```

Parâmetros primitivos (como um número) são passados para as funções por **valor**; o valor é passado para a função, mas se a função altera o valor do parâmetro, esta mudança não reflete globalmente ou na função chamada.

Se você passar um objeto (ou seja, um valor não primitivo, tal como `Array` ou um objeto definido por você) como um parâmetro e a função alterar as propriedades do objeto, essa mudança é visível fora da função, conforme mostrado no exemplo a seguir:

```
function minhaFuncao(objeto) {
  objeto.make = "Toyota";
}

var meucarro = { make: "Honda", model: "Accord", year: 1998 };
var x, y;

x = meucarro.make; // x recebe o valor "Honda"

minhaFuncao(meucarro);
y = meucarro.make; // y recebe o valor "Toyota"
// (a propriedade make foi alterada pela função)
```

Expressão de função

Embora a declaração de função acima seja sintaticamente uma declaração, funções também podem ser criadas por uma **expressão de função**. Tal função pode ser **anônima**; ele não tem que ter um nome. Por exemplo, a função `square` poderia ter sido definida como:

```
var square = function (numero) {
  return numero * numero;
};
var x = square(4); //x recebe o valor 16
```

No entanto, um nome pode ser fornecido com uma expressão de função e pode ser utilizado no interior da função para se referir a si mesma, ou em um debugger para identificar a função em stack traces:

```
var fatorial = function fac(n) {
  return n < 2 ? 1 : n * fac(n - 1);
};

console.log(fatorial(3));
```

As expressões de função são convenientes ao passar uma função como um argumento para outra função. O exemplo a seguir mostra uma função `map` sendo definida e, em seguida, chamada com uma função anônima como seu primeiro parâmetro:

```
function map(f, a) {
  var result = []; // Cria um novo Array
  var i;
  for (i = 0; i != a.length; i++) result[i] = f(a[i]);
  return result;
}
```

O código a seguir:

```
map(
  function (x) {
    return x * x * x;
  },
  [0, 1, 2, 5, 10],
);
```

retorna [0, 1, 8, 125, 1000].

Em JavaScript, uma função pode ser definida com base numa condição. Por exemplo, a seguinte definição de função define `minhaFuncao` somente se `num` é igual a 0:

```
var minhaFuncao;
if (num == 0) {
  minhaFuncao = function (objeto) {
    objeto.make = "Toyota";
  };
}
```

Além de definir funções, você também pode usar o construtor `Function` para criar funções a partir de uma string em tempo real, como no método `eval()`.

Um **método** é uma função invocada por um objeto. Leia mais sobre objetos e métodos em Trabalhando com Objetos.

Chamando funções

A definição de uma função não a executa. Definir a função é simplesmente nomear a função e especificar o que fazer quando a função é chamada. Chamar a função executa realmente as ações especificadas com os parâmetros indicados. Por exemplo, se você definir a função `square`, você pode chamá-la do seguinte modo:

```
square(5);
```

A declaração anterior chama a função com o argumento 5. A função executa as instruções e retorna o valor 25.

Funções devem estar no escopo quando são chamadas, mas a declaração de uma função pode ser puxada para o topo (aparecem abaixo da chamada no código), como neste exemplo:

```
console.log(square(5));
/* ... */
function square(n){return n*n}
```

O escopo de uma função é a função na qual ela é declarada, ou todo o programa se ela é declarada no nível superior.

> **Nota:** Isso funciona apenas quando a definição da função usa a sintaxe acima (ex., `function funcNome(){ }`. O código a seguir não vai funcionar.

```
console.log(square(5));
var square = function (n) {
  return n * n;
};
```

Os argumentos de uma função não estão limitados a strings e números. Você pode passar objetos para uma função. A função `show_props` (definido em Trabalhando com Objetos) é um exemplo de uma função que recebe um objeto como um argumento.

Um função pode chamar a si mesma. Por exemplo, a função que calcula os fatoriais recursivamente:

```
function fatorial(n) {
  if (n == 0 || n == 1) return 1;
  else return n * fatorial(n - 1);
}
```

Você poderia, então, calcular os fatoriais de um a cinco:

```
var a, b, c, d, e;
a = fatorial(1); // a recebe o valor 1
b = fatorial(2); // b recebe o valor 2
c = fatorial(3); // c recebe o valor 6
d = fatorial(4); // d recebe o valor 24
e = fatorial(5); // e recebe o valor 120
```

Há outras maneiras de chamar funções. Muitas vezes há casos em que uma função precisa ser chamada dinamicamente, ou o número de argumentos de uma função varia, ou em que o contexto da chamada de função precisa ser definido para um objeto específico determinado em tempo de execução. Acontece que as funções são, por si mesmas, objetos, e esses objetos por sua vez têm métodos (veja objeto `Function`). Um desses, o método `apply()`, pode ser usado para atingir esse objetivo.

Escopo da função

As variáveis definidas no interior de uma função não podem ser acessadas de nenhum lugar fora da função, porque a variável está definida apenas no escopo da função. No entanto, uma função pode acessar todas variáveis e funções definida fora do escopo onde ela está definida. Em outras palavras, a função definida no escopo global pode acessar todas as variáveis definidas no escopo global. A função definida dentro de outra função também pode acessar todas as variáveis definidas na função hospedeira e outras variáveis ao qual a função hospedeira tem acesso.

```
// As seguintes variáveis são definidas no escopo global
var num1 = 20,
  num2 = 3,
  nome = "Chamahk";

// Esta função é definida no escopo global
function multiplica() {
  return num1 * num2;
}

multiplica(); // Retorna 60
```

```
// Um exemplo de função aninhada
function getScore() {
  var num1 = 2,
    num2 = 3;

  function add() {
    return nome + " scored " + (num1 + num2);
  }

  return add();
}

getScore(); // Retorna "Chamahk scored 5"
```

Escopo e a pilha de função

Recursão

Uma função pode referir-se e chamar a si própria. Há três maneiras de uma função referir-se a si mesma:

1. o nome da função
2. `arguments.callee`
3. uma variável no escopo que se refere a função

Por exemplo, considere a seguinte definição de função:

```
var foo = function bar() {
  // declaracoes
};
```

Dentro do corpo da função, todos, a seguir, são equivalentes:

1. `bar()`
2. `arguments.callee()`
3. `foo()`

Uma função que chama a si mesma é chamada de função recursiva. Em alguns casos, a recursividade é análoga a um laço. Ambos executam o código várias vezes, e ambos necessitam de uma condição (para evitar um laço infinito, ou melhor, recursão infinita, neste caso). Por exemplo, o seguinte laço:

```
var x = 0;
while (x < 10) {
  // "x < 10" a condição do laço
  // faça coisas
  x++;
}
```

pode ser convertido em função recursiva e uma chamada para a função:

```
function loop(x) {
  if (x >= 10)
    // "x >= 10" a condição de parada (equivalente a "!(x < 10)")
    return;
  // faça coisas
  loop(x + 1); // chamada recursiva
}
loop(0);
```

No entanto, alguns algoritmos não podem ser simples laços iterativos. Por exemplo, conseguir todos os nós da estrutura de uma árvore (por exemplo, o DOM) é mais fácil se feito recursivamente:

```
function walkTree(node) {
  if (node == null)
    //
    return;
  // faça algo com o nó
  for (var i = 0; i < node.childNodes.length; i++) {
    walkTree(node.childNodes[i]);
  }
}
```

Em comparação ao laço da função, cada chamada recursiva realiza outras chamadas recursivas.

É possível converter qualquer algoritmo recursivo para um não recursivo, mas muitas vezes a lógica é muito mais complexa e exige o uso de pilhas. Na verdade a própria recursão usa pilha: a pilha de função.

O comportamento da pilha pode ser vista a seguir no exemplo:

```
function foo(i) {
  if (i < 0) return;
  document.writeln("begin:" + i);
  foo(i - 1);
```

```
  document.writeln("end:" + i);
}
foo(3);
```

que produz:

```
begin:3
begin:2
begin:1
begin:0
end:0
end:1
end:2
end:3
```

Funções aninhadas e closures

Você pode aninhar uma função dentro de outra. A função aninhada (interna) é acessível apenas para a função que a contém (exterior). Isso constitui também uma *`closure`*. Uma closure é uma expressão (tipicamente uma função) que pode ter variáveis livres em conjunto com um ambiente que conecta estas variáveis (que "fecha" a expressão).

Uma vez que uma função aninhada é uma closure, isto significa que uma função aninhada pode "herdar" os argumentos e variáveis de sua função de contenção. Em outras palavras, a função interior contém o escopo da função exterior.

Em resumo:

- A função interna só pode ser acessada a partir de declarações em função externa.
- A função interna forma uma closure: a função interna pode usar os argumentos e variáveis da função externa, enquanto a função externa não pode usar os argumentos e variáveis da função interna.

O exemplo a seguir mostra as funções aninhadas:

```
function addSquares(a, b) {
  function square(x) {
    return x * x;
  }
  return square(a) + square(b);
}
a = addSquares(2, 3); // retorna 13
b = addSquares(3, 4); // retorna 25
c = addSquares(4, 5); // retorna 41
```

Uma vez que a função interna forma uma *closure*, você pode chamar a função externa e especificar argumentos para a função externa e interna:

```
function fora(x) {
  function dentro(y) {
    return x + y;
  }
  return dentro;
}
fn_inside = fora(3); // Pense nisso como: Receba uma função que adicionará 3 ao que quer que você repasse para ela
result = fn_inside(5); // retorna 8

result1 = fora(3)(5); // retorna 8
```

Preservação de variáveis

Observe como `x` é preservado quando `dentro` é retornado. Uma *closure* deve preservar os argumentos e variáveis em todos os escopos que ela referencia. Uma vez que cada chamada fornece potencialmente argumentos diferentes, uma nova closure é criada para cada chamada de `fora`. A memória só poderá ser liberada quando o `dentro` retornado já não é mais acessível.

Isso não é diferente de armazenar referências em outros objetos, mas muitas vezes é menos óbvio, porque um não define diretamente as referências e não pode inspecioná-las.

Múltiplas funções aninhadas

Funções podem ter múltiplo aninhamento, por exemplo, a função (A) contém a função (B) que contém a função (C). Tanto as funções B e C formam uma *closure*, então B pode acessar A, e C pode acessar B. Além disso, uma vez que C pode acessar B que pode acessar A, C também pode acessar A. Assim, a *closure* pode conter vários escopos; eles recursivamente contém o escopo das funções que os contém. Isso é chamado *`encadeamento de escopo`*. (O motivo de ser chamado "encadeamento" será explicado mais tarde).

Considere o seguinte exemplo:

```
function A(x) {
  function B(y) {
    function C(z) {
      alert(x + y + z);
    }
    C(3);
  }
  B(2);
```

```
}
A(1); // Exibe um alerta com o valor 6 (1 + 2 + 3)
```

Neste exemplo, `C` acessa `y` do `B` e `x` do `A`. Isso pode ser feito porque:

1. B forma uma *closure* incluindo `A`, isto é, `B` pode acessar argumentos e variáveis de `A`.
2. C forma uma *closure* incluindo `B`.
3. Devido a closure `B` inclui `A`, a *closure* `C` inclui `A`, `C` pode acessar tanto argumentos e variáveis de `B` como de `A`. Em outras palavras, `C` *encadeia* o escopo de `B` e `A`, nesta ordem.

O inverso, no entanto, não é verdadeiro. `A` não pode acessar `C`, porque `A` não pode acessar qualquer argumento ou variável de `B`. Assim, `C` é privado somente a `B`.

Conflitos de nome

Quando dois argumentos ou variáveis nos escopos da *closure* tem o mesmo nome, há um *conflito de nome*. Mas escopos internos tem prioridade, por isso o escopo mais interno tem a maior prioridade, enquanto que o escopo mais externo tem a menor. Esta é a cadeia de escopo. O primeiro da cadeia é o escopo mais interno, e o último é o escopo mais externo. Considere o seguinte:

```
function fora() {
  var x = 10;
  function dentro(x) {
    return x;
  }
  return dentro;
}
result = fora()(20); // retorna 20 em vez de 10
```

O conflito de nome acontece na declaração `return x` e está entre o parâmetro `x` de `dentro` e a variável `x` de `fora`. A cadeia de escopo aqui é {`dentro`, `fora`, objeto global}. Por isso o `x` de `dentro` tem precedência sobre o `x` de `fora`, e 20 (`x` de `dentro`) é retornado em vez de 10 (`x` de fora).

Closures

Closures são um dos recursos mais poderosos de JavaScript. JavaScript permite o aninhamento de funções e garante acesso completo à função interna a todas as variáveis e funções definidas dentro da função externa (e todas as outras variáveis e funções que a função externa tem acesso). No entanto, a função externa não tem acesso às variáveis e funções definidas dentro da função interna. Isto proporciona uma espécie de segurança para as variáveis da função interna. Além disso, uma vez que a função interna tem acesso ao escopo da função externa, as variáveis e funções definidas na função externa vão durar na memória mais do que a própria função externa, isto se a função interna permanecer na memória mais tempo do que a função externa. Uma *closure* é criada quando a função interna é de alguma forma disponibilizada para qualquer escopo fora da função externa.

```
var pet = function (nome) {
    // A função externa define uma variável "nome"
    var getNome = function () {
      return nome; // A função interna tem acesso à variável "nome"  da função externa
    };

    return getNome; // Retorna a função interna, expondo-a assim para escopos externos
  },
  myPet = pet("Vivie");

myPet(); // Retorna "Vivie"
```

Ela pode ser mais complexa que o código acima. Um objeto contendo métodos para manipular as variáveis da função externa pode ser devolvida.

```
var criarPet = function (nome) {
  var sex;

  return {
    setNome: function (newNome) {
      nome = newNome;
    },

    getNome: function () {
      return nome;
    },

    getSex: function () {
      return sex;
    },

    setSex: function (newSex) {
      if (
        typeof newSex == "string" &&
        (newSex.toLowerCase() == "macho" || newSex.toLowerCase() == "fêmea")
      ) {
        sex = newSex;
      }
    },
  };
};

var pet = criarPet("Vivie");
pet.getNome(); // Vivie
```

```
pet.setNome("Oliver");
pet.setSex("macho");
pet.getSex(); // macho
pet.getNome(); // Oliver
```

Nos códigos acima, a variável `nome` da função externa é acessível para as funções internas, e não há nenhuma outra maneira para acessar as variáveis internas, exceto pelas funções internas. As variáveis internas da função interna atuam como armazenamento seguro para as funções internas. Elas armazenam "persistentes", mas seguros, os dados com os quais as funções internas irão trabalhar. As funções não tem que ser atribuídas a uma variável, ou ter um nome.

```
var getCode = (function () {
  var secureCode = "0]Eal(eh&2"; // Um código que não queremos que pessoas de fora sejam capazes de modificar

  return function () {
    return secureCode;
  };
})();

getCode(); // Retorna o secureCode
```

Há, no entanto, uma série de armadilhas que se deve ter cuidado ao usar closures. Se uma função fechada define uma variável com o mesmo nome de uma variável em um escopo externo, não há nenhuma maneira de se referir para a variável em um escopo externo novamente.

```
var createPet = function (nome) {
  // Função externa define uma variável chamada "nome"
  return {
    setNome: function (nome) {
      // Função fechada define uma variável chamada "nome"
      nome = nome; // ??? Como podemos acessar o "nome" definido pela função externa ???
    },
  };
};
```

A palavra reservada this é muito complicada em *closures*, elas têm de ser usadas com muito cuidado. O objeto ao que `this` se refere depende completamente de onde a função foi chamada, ao invés de onde ela foi definida.

Usando objeto de argumentos

Os argumentos de uma função são mantidos em um objeto do tipo array. Dentro de uma função, você pode endereçar os argumentos passados para ele conforme:

```
arguments[i];
```

onde `i` é um número ordinal do argumento, começando com zero. Então, o primeiro argumento passado para a função seria `arguments[0]`. O número total de argumentos é indicado por `arguments.length`.

Usando o objeto `arguments`, você pode chamar a função com mais argumentos do que o formalmente declarado. Isso muitas vezes é útil se você não sabe de antemão quantos argumentos serão passados para a função. Você pode usar `arguments.length` para determinar a quantidade de argumentos passados para a função, e então acessar cada argumento usando o objeto `arguments`.

Por exemplo, considere uma função que concatena várias *strings*. O argumento formal para a função é uma *string* que especifica os caracteres que separam os itens para concatenar. A função definida como segue:

```
function myConcat(separador) {
  var result = "", // inicializa a lista
    i;
  // itera por meio de argumentos
  for (i = 1; i < arguments.length; i++) {
    result += arguments[i] + separador;
  }
  return result;
}
```

Você pode passar qualquer quantidade de argumentos para esta função, e ela concatena cada argumento na *string* "list":

```
// retorna "red, orange, blue, "
myConcat(", ", "red", "orange", "blue");

// retorna "elephant; giraffe; lion; cheetah; "
myConcat("; ", "elephant", "giraffe", "lion", "cheetah");

// retorna "sage. basil. oregano. pepper. parsley. "
myConcat(". ", "sage", "basil", "oregano", "pepper", "parsley");
```

> **Nota:** A variável `arguments` é "como um array", mas não é um array. Ela é como um array pois possui um índice numerado e a propriedade `length`. No entanto, não possui todos os métodos de manipulação de array.

Veja objeto `Function` na referência do JavaScript para maiores informações.

Parâmetros de função

Começando com ECMAScript 6, há dois tipos novos de parâmetros: parâmetros padrão e parâmetros rest.

Parâmetros padrão

Em JavaScript, parâmetros padrões de funções são `undefined`. No entanto, em algumas situações pode ser útil definir um valor padrão diferente. Isto é onde os parâmetros padrão podem ajudar.

No passado, a estratégia geral para definir padrões era testar os valores de parâmetro no corpo da função e atribuir um valor se eles fossem `undefined`. Se, no exemplo a seguir, nenhum valor é fornecido para `b` na chamada, seu valor seria `undefined` ao avaliar `a*b` e a chamada para `multiplicar` retornaria `NaN`. No entanto, isso é pego com a segunda linha neste exemplo:

```
function multiplicar(a, b) {
  b = typeof b !== "undefined" ? b : 1;

  return a * b;
}

multiplicar(5); // 5
```

Com parâmetros padrão, a verificação no corpo da função não é mais necessária. Agora você pode simplesmente colocar `1` como valor padrão para `b` no campo de declaração de parâmetros:

```
function multiplicar(a, b = 1) {
  return a * b;
}

multiplicar(5); // 5
```

Mais detalhes, consulte parâmetros padrão na referência.

Parâmetros rest

A sintaxe de parâmetro rest permite representar um número indefinido de argumentos como um *array*. No exemplo, usamos parâmetros *rest* para coletar argumentos do segundo argumento ao último. Então os multiplicamos pelo primeiro argumento. Neste exemplo é usado uma *arrow function*, que será introduzida na próxima seção.

```
function multiplicar(multiplicador, ...args) {
  return args.map((x) => multiplicador * x);
}

var arr = multiplicar(2, 1, 2, 3);
console.log(arr); // [2, 4, 6]
```

Funções de seta

Uma expressão função de seta (anteriormente conhecida como **função de seta gorda**) tem uma sintaxe pequena em comparação com a expressão de função e lexicalmente vincula o valor `this`. Funções de seta são sempre anônimas. Consulte também no blog hacks.mozilla.org no post: "ES6 In Depth: Arrow functions".

Dois fatores influenciaram a introdução de funções de seta: funções mais curtas e o léxico `this`.

Funções curtas

Em alguns padrões funcionais, funções curtas são bem-vindas. Compare:

```
var a = ["Hydrogen", "Helium", "Lithium", "Beryllium"];

var a2 = a.map(function (s) {
  return s.length;
});

var a3 = a.map((s) => s.length);
```

Léxico this

Até as funções de seta, cada nova função definia seu próprio valor this (um novo objeto no caso de um construtor, indefinido em chamadas de função no modo estrito, o objeto de contexto se a função é chamada como um "método de objeto", etc.). Isso pode ser irritante com um estilo de programação orientada a objetos.

```
function Pessoa() {
  // O construtor Pessoa() define 'this' como sendo ele.
  this.idade = 0;
  setInterval(function crescer() {
    // No modo não estrito, a função crescer define 'this'
    // como o objeto global, o que é diferente do 'this'
    // definido pelo construtor Pessoa().
    this.idade++;
  }, 1000);
}
var p = new Pessoa();
```

No ECMAScript 3/5, este problema foi resolvido atribuindo o valor em `this` a uma variável que poderia ser fechada.

```
function Pessoa() {
  var self = this; // Alguns preferem 'that' em vez de 'self'.
  // Escolha um e seja consistente.
```

```
  self.idade = 0;

  setInterval(function crescer() {
    // A chamada de retorno refere-se à variável 'self' na qual
    // o valor é o objeto esperado.
    self.idade++;
  }, 1000);
}
```

Como alternativa, uma função vinculada poderia ser criada para que o valor da propriedade `this` seja passado para a função `crescer()`.

Funções de seta capturam o valor `this` do contexto delimitado, então o código a seguir funciona conforme o esperado.

```
function Pessoa(){
  this.idade = 0;

  setInterval(() => {
    this.idade++; // propriedade |this|refere ao objeto pessoa
  }, 1000);
}

var p = new Pessoa();
```

Funções pré-definidas

JavaScript tem várias funções pré-definidas:

`eval()`

O método **`eval()`** avalia código JavaScript representado como uma *string*.

`uneval()`

O método **`uneval()`** cria uma representação de *string* do código-fonte de um `Object`.

`isFinite()`

A função global **`isFinite()`** determina se o valor passado é um número finito. Se necessário, o parâmetro é primeiro convertido para um número.

`isNaN()`

A função **`isNaN()`** determina se um valor é `NaN` ou não. Nota: coerção dentro da função `isNaN` tem regras interessantes; você pode, alternativamente, querer usar `Number.isNaN()`, como definido no ECMAScript 6, ou você pode usar `typeof` para determinar se o valor não é um número.

`parseFloat()`

A função **`parseFloat()`** analisa um argumento do tipo *string* e retorna um número de ponto flutuante.

`parseInt()`

A função **`parseInt()`** analisa um argumento do tipo *string* e retorna um inteiro da base especificada (base do sistema numérico).

`decodeURI()`

A função **`decodeURI()`** decodifica uma *Uniform Resource Identifier* (URI) criada anteriormente por `encodeURI` ou por uma rotina similar.

`decodeURIComponent()`

O método **`decodeURIComponent()`** decodifica um componente *Uniform Resource Identifier* (URI) criado anteriormente por `encodeURIComponent` ou por uma rotina similar.

`encodeURI()`

O método **`encodeURI()`** codifica um Uniform Resource Identifier (URI), substituindo cada ocorrência de determinados caracteres por um, dois, três, ou quatro sequências de escape que representa a codificação UTF-8 do caractere (só serão quatro sequências de escape para caracteres compostos de dois caracteres "substitutos").

`encodeURIComponent()`

O método **`encodeURIComponent()`** codifica um componente *Uniform Resource Identifier* (URI), substituindo cada ocorrência de determinados caracteres por um, dois, três, ou quatro sequências de escape que representa a codificação UTF-8 do caractere (só serão quatro sequências de escape para caracteres compostos de dois caracteres "substitutos").

`escape()`

O método obsoleto **`escape()`** calcula uma nova *string* na qual certos caracteres foram substituídos por uma sequência de escape hexadecimal. Use `encodeURI` ou `encodeURIComponent` em vez disso.

`unescape()`

O método obsoleto **`unescape()`** calcula uma nova *string* na qual sequências de escape hexadecimais são substituídas pelo caractere que ela representa. As sequências de escape podem ser introduzidas por uma função como `escape`. Por `unescape()` estar obsoleto, use `decodeURI()` ou `decodeURIComponent` ao invés dele.

- Anterior

- Próxima

Sintaxe e tipos

- Anterior
- Próxima

Este capítulo trata sobre a sintaxe básica do JavaScript, declarações de variáveis, tipos de dados e literais.

Sintaxe básica

JavaScript pega emprestado a maior parte de sua sintaxe do Java, mas também é influenciado por Awk, Perl e Python.

JavaScript é **case-sensitive** e usa o conjunto de caracteres **Unicode.** Por exemplo, a palavra Früh (que significa "cedo" em Alemão) pode ser usada como nome de variável.

```
var Früh = "foobar";
```

Mas a variável `früh` não é a mesma que `Früh` porque JavaScript é case sensitive.

No JavaScript, instruções são chamadas de declaração e são separadas por um ponto e vírgula (;). Espaços, tabulação e uma nova linha são chamados de espaços em branco. O código fonte dos scripts em JavaScript são lidos da esquerda para a direita e são convertidos em uma sequência de elementos de entrada como simbolos, caracteres de controle, terminadores de linha, comentários ou espaço em branco. ECMAScript também define determinadas palavras-chave e literais, e tem regras para inserção automática de ponto e vírgula (ASI) para terminar as declarações. No entanto, recomenda-se sempre adicionar ponto e vírgula no final de suas declarações; isso evitará alguns imprevistos. Para obter mais informações, consulte a referência detalhada sobre a gramática léxica do JavaScript.

Comentários

A sintaxe dos comentários em JavaScript é semelhante como em C++ e em muitas outras linguagens:

```
// comentário de uma linha

/* isto é um comentário longo
   de múltiplas linhas.
 */

/* Você não pode, porém, /* aninhar comentários */ SyntaxError */
```

Declarações

Existem três tipos de declarações em JavaScript.

`var`

Declara uma variável, opcionalmente, inicializando-a com um valor.

`let`

Declara uma variável local de escopo do bloco, opcionalmente, inicializando-a com um valor.

`const`

Declara uma constante de escopo de bloco, apenas de leitura.

Variáveis

Você usa variáveis como nomes simbólicos para os valores em sua aplicação. O nome das variáveis, chamados de identificadores, obedecem determinadas regras.

Um identificador JavaScript deve começar com uma letra, underline (_), ou cifrão ($); os caracteres subsequentes podem também ser números (0-9). Devido JavaScript ser case-sensitive, letras incluem caracteres de "A" a "Z" (maiúsculos) e caracteres de "a" a "z" (minúsculos).

Você pode usar a ISO 8859-1 ou caracteres Unicode tal como os identificadores å e ü. Você pode também usar as sequências de escape Unicode como caracteres e identificadores.

Alguns exemplos de nomes legais são `Numeros_visitas`, `temp99`, e `_nome`.

Declarando variáveis

Você pode declarar uma variável de três formas:

- Com a palavra chave `var`. Por exemplo, `var x = 42`. Esta sintaxe pode ser usada para declarar tanto variáveis locais como variáveis globais.
- Por simples adição de valor. Por exemplo, `x = 42`. Isso declara uma variável global. Essa declaração gera um aviso de advertência no JavaScript. Você não deve usar essa variante.
- Com a palavra chave `let`. Por exemplo, `let y = 13`. Essa sintaxe pode ser usada para declarar uma variável local de escopo de bloco. Veja escopo de variável abaixo.

Classificando variáveis

Uma variável declarada usando a declaração `var` ou `let` sem especificar o valor inicial tem o valor `undefined`.

Uma tentativa de acessar uma variável não declarada resultará no lançamento de uma exceção `ReferenceError`:

```
var a;
console.log("O valor de a é " + a); // saída "O valor de a é undefined"
console.log("O valor de b é " + b); // executa uma exception de erro de referência (ReferenceError)
```

Você pode usar `undefined` para determinar se uma variável tem um valor. No código a seguir, não é atribuído um valor de entrada na variável e a declaração `if` será avaliada como verdadeira (`true`).

```
var input;
if (input === undefined) {
  facaIsto();
} else {
  facaAquilo();
}
```

O valor `undefined` se comporta como falso (`false`), quando usado em um contexto booleano. Por exemplo, o código a seguir executa a função `myFunction` devido o elemento `myArray` ser undefined:

```
var myArray = [];
if (!myArray[0]) myFunction();
```

O valor `undefined` converte-se para `NaN` quando usado no contexto numérico.

```
var a;
a + 2; // Avaliado como NaN
```

Quando você avalia uma variável nula, o valor nulo se comporta como 0 em contextos numéricos e como falso em contextos booleanos. Por exemplo:

```
var n = null;
console.log(n * 32); // a saída para o console será 0.
```

Escopo de variável

Quando você declara uma variável fora de qualquer função, ela é chamada de variável *global*, porque está disponível para qualquer outro código no documento atual. Quando você declara uma variável dentro de uma função, é chamada de variável *local*, pois ela está disponível somente dentro dessa função.

JavaScript antes do ECMAScript 6 não possuía escopo de declaração de bloco; pelo contrário, uma variável declarada dentro de um bloco de uma *função* é uma variável local (ou contexto *global*) do bloco que está inserido a função. Por exemplo o código a seguir exibirá 5, porque o escopo de `x` está na função (ou contexto global) no qual `x` é declarado, não o bloco, que neste caso é a declaração `if`.

```
if (true) {
  var x = 5;
}
console.log(x); // 5
```

Esse comportamento é alterado, quando usado a declaração `let` introduzida pelo ECMAScript 6.

```
if (true) {
  let y = 5;
}
console.log(y); // ReferenceError: y não está definido
```

Variável de elevação

Outra coisa incomum sobre variáveis em JavaScript é que você pode utilizar a variável e declará-la depois, sem obter uma exceção. Este conceito é conhecido como **hoisting**; variáveis em JavaScript são num sentido "hoisted" ou lançada para o topo da função ou declaração. No entanto, as variáveis que são "hoisted" retornarão um valor `undefined`. Então, mesmo se você usar ou referir a variável e depois declará-la e inicializá-la, ela ainda retornará undefined.

```
/**
 * Exemplo 1
 */
console.log(x === undefined); // exibe "true"
var x = 3;

/**
 * Exemplo 2
 */
// retornará um valor undefined
var myvar = "my value";

(function () {
  console.log(myvar); // undefined
  var myvar = "local value";
})();
```

Os exemplos acima serão interpretados como:

```
/**
 * Exemplo 1
 */
var x;
```

```
console.log(x === undefined); // exibe "true"
x = 3;

/**
 * Exemplo 2
 */
var myvar = "um valor";

(function () {
  var myvar;
  console.log(myvar); // undefined
  myvar = "valor local";
})();
```

Devido o hoisting, todas as declarações `var` em uma função devem ser colocadas no início da função. Essa recomendação de prática deixa o código mais legível.

Variáveis Globais

Variáveis globais são propriedades do *objeto global*. Em páginas web o objeto global é a `window`, assim você pode configurar e acessar variáveis globais utilizando a sintaxe `window.variavel`.

Consequentemente, você pode acessar variáveis globais declaradas em uma janela ou frame ou frame de outra janela. Por exemplo, se uma variável chamada phoneNumber é declarada em um documento, você pode consultar esta variável de um frame como `parent.phoneNumber`.

Constantes

Você pode criar uma constante apenas de leitura por meio da palavra-chave `const`. A sintaxe de um identificador de uma constante é semelhante ao identificador de uma variável: deve começar com uma letra, sublinhado ou cifrão e pode conter caractere alfabético, numérico ou sublinhado.

```
const PI = 3.14;
```

Uma constante não pode alterar seu valor por meio de uma atribuição ou ser declarada novamente enquanto o script está em execução. Deve ser inicializada com um valor.

As regras de escopo para as constantes são as mesmas para as váriaveis `let` de escopo de bloco. Se a palavra-chave `const` for omitida, presume-se que o identificador represente uma variável.

Você não pode declarar uma constante com o mesmo nome de uma função ou variável que estão no mesmo escopo. Por exemplo:

```
// Isto irá causar um  erro
function f() {}
const f = 5;

// Isto também irá causar um erro.
function f() {
  const g = 5;
  var g;

  //declarações
}
```

Estrutura de dados e tipos

Tipos de dados

O mais recente padrão ECMAScript define sete tipos de dados:

- Seis tipos de dados são os chamados primitivos:
  - Boolean. `true` e `false`.
  - null. Uma palavra-chave que indica valor nulo. Devido JavaScript ser case-sensitive, `null` não é o mesmo que `Null`, `NULL`, ou ainda outra variação.
  - undefined. Uma propriedade superior cujo valor é indefinido.
  - Number. `42` ou `3.14159`.
  - String. "Howdy"
  - Symbol (novo em ECMAScript 6). Um tipo de dado cuja as instâncias são únicas e imutáveis.
- e Object

Embora esses tipos de dados sejam uma quantidade relativamente pequena, eles permitem realizar funções úteis em suas aplicações. `Objetos` e `funções` são outros elementos fundamentais na linguagem. Você pode pensar em objetos como recipientes para os valores, e funções como métodos que suas aplicações podem executar.

Conversão de tipos de dados

JavaScript é uma linguagem dinamicamente tipada. Isso significa que você não precisa especificar o tipo de dado de uma variável quando declará-la, e tipos de dados são convertidos automaticamente conforme a necessidade durante a execução do script. Então, por exemplo, você pode definir uma variável da seguinte forma:

```
var answer = 42;
```

E depois, você pode atribuir uma string para a mesma variável, por exemplo:

```
answer = "Obrigado pelos peixes...";
```

Devido JavaScript ser dinamicamente tipado, essa declaração não gera uma mensagem de erro.

Em expressões envolvendo valores numérico e string com o operador +, JavaScript converte valores numérico para strings. Por exemplo, considere a seguinte declaração:

```
x = "A resposta é " + 42; // "A resposta é 42"
y = 42 + " é a resposta"; // "42 é a resposta"
```

Nas declarações envolvendo outros operadores, JavaScript não converte valores numérico para strings. Por exemplo:

```
"37" - 7; // 30
"37" + 7; // "377"
```

Convertendo strings para números

No caso de um valor que representa um número está armazenado na memória como uma string, existem métodos para a conversão.

- `parseInt()`
- `parseFloat()`

parseInt irá retornar apenas números inteiros, então seu uso é restrito para a casa dos decimais. Além disso, é uma boa prática ao usar parseInt incluir o parâmetro da base. O parâmetro da base é usado para especificar qual sistema númerico deve ser usado.

Uma método alternativo de conversão de um número em forma de string é com o operador + (operador soma):

```
"1.1" + "1.1" = "1.11.1"
(+"1.1") + (+"1.1") = 2.2
// Nota: Os parênteses foram usados para deixar mais legível o código, ele não é requirido.
```

Literais

Você usa literais para representar valores em JavaScript. Estes são valores fixados, não variáveis, que você `literalmente` insere em seu script. Esta seção descreve os seguintes tipos literais:

- Array literal
- Literais boolean
- Literais de ponto flutuante
- Inteiros
- Objeto literal
- String literal

Array literal

Um array literal é uma lista de zero ou mais expressões, onde cada uma delas representam um elemento do array, inseridas entre colchetes (`[]`). Quando você cria um array usando um array literal, ele é inicializado com os valores especificados como seus elementos, e seu comprimento é definido com o número de elementos especificados.

O exemplo a seguir cria um array `coffees` com três elementos e um comprimento de três:

```
var coffees = ["French Roast", "Colombian", "Kona"];
```

> **Nota:** Um array literal é um tipo de inicializador de objetos. Veja Usando inicializadores de Objetos.

Se um array é criado usando um literal no topo do script, JavaScript interpreta o array cada vez que avalia a expressão que contêm o array literal. Além disso, um literal usado em uma função é criado cada vez que a função é chamada.

Array literal são também um array de objetos. Veja `Array` e Coleções indexadas para detalhes sobre array de objetos.

Vírgulas extras em array literal

Você não precisa especificar todos os elementos em um array literal. Se você colocar duas vírgulas em uma linha, o array é criado com `undefined` para os elementos não especificados. O exemplo a seguir cria um array chamado `fish`:

```
var fish = ["Lion", , "Angel"];
```

Esse array tem dois elementos com valores e um elemento vazio (`fish[0]` é "Lion", `fish[1]` é `undefined`, e `fish[2]` é "Angel").

Se você incluir uma vírgula à direita no final da lista dos elementos, a vírgula é ignorada. No exemplo a seguir, o comprimento do array é três. Não há nenhum `myList[3]`. Todas as outras vírgulas na lista indicam um novo elemento.

> **Nota:** Vírgulas à direita podem criar erros em algumas versões de navegadores web antigos, é recomendável removê-las.

```
var myList = ["home", , "school", ];
```

No exemplo a seguir, o comprimento do array é quatro, e `myList[0]` e `myList[2]` são `undefined`.

```
var myList = [, "home", , "school"];
```

No exemplo a seguir, o comprimento do array é quatro, e `myList[1]` e `myList[3]` são `undefined`. Apenas a última vírgula é ignorada.

```
var myList = ["home", , "school", ,];
```

Entender o comportamento de vírgulas extras é importante para a compreensão da linguagem JavaScript, no entanto, quando você escrever seu próprio código: declarar explicitamente os elementos em falta como `undefined` vai aumentar a clareza do código, e consequentemente na sua manutenção.

Literais Boolean

O tipo Boolean tem dois valores literal: `true` e `false`.

Não confunda os valores primitivos Boolean `true` e `false` com os valores `true` e `false` do objeto Boolean. O objeto Boolean é um invólucro em torno do tipo de dado primitivo. Veja `Boolean` para mais informação.

Inteiros

Inteiros podem ser expressos em decimal (base 10), hexadecimal (base 16), octal (base 8) e binário (base 2).

- Decimal inteiro literal consiste em uma sequência de dígitos sem um 0 (zero).
- 0 (zero) em um inteiro literal indica que ele está em octal. Octal pode incluir somente os dígitos 0-7.
- 0x (ou 0X) indica um hexadecimal. Inteiros hexadecimais podem incluir dígitos (0-9) e as letras a-f e A-F.
- 0b (ou 0B) indica um binário. Inteiros binário podem incluir apenas os dígitos 0 e 1.

Alguns exemplos de inteiros literal são:

```
0, 117 and -345 (decimal, base 10)
015, 0001 and -077 (octal, base 8)
0x1123, 0x00111 and -0xF1A7 (hexadecimal, "hex" or base 16)
0b11, 0b0011 and -0b11 (binário, base 2)
```

Para maiores informações, veja Literais numérico na referência Léxica.

Literais de ponto flutuante

Um literal de ponto flutuante pode ter as seguintes partes:

- Um inteiro decimal que pode ter sinal (precedido por "+" ou "-"),
- Um ponto decimal ("."),
- Uma fração (outro número decimal),
- Um expoente.

O expoente é um "e" ou "E" seguido por um inteiro, que pode ter sinal (precedido por "+" ou "-"). Um literal de ponto flutuante deve ter no mínimo um dígito e um ponto decimal ou "e" (ou "E").

Mais sucintamente, a sintaxe é:

```
[(+|-)][digitos][.digitos][(E|e)[(+|-)]digitos]
```

Por exemplo:

```
3.1415926
-.123456789
-3.1E+12
.1e-23
```

Objeto literal

Um objeto literal é uma lista de zero ou mais pares de nomes de propriedades e valores associados de um objeto, colocado entre chaves (`{}`). Você não deve usar um objeto literal no início de uma declaração. Isso levará a um erro ou não se comportará conforme o esperado, porque o `{` será interpretado como início de um bloco.

Segue um exemplo de um objeto literal. O primeiro elemento do objeto `carro` define uma propriedade, `meuCarro`, e atribui para ele uma nova string, "Punto"; o segundo elemento, a propriedade `getCarro`, é imediatamente atribuído o resultado de chamar uma função (`tipoCarro("Fiat")`); o terceiro elemento, a propriedade `especial`, usa uma variável existente (`vendas`).

```
var vendas = "Toyota";

function tipoCarro(nome) {
  if (nome == "Fiat") {
    return nome;
  } else {
    return "Desculpa, não vendemos carros " + nome + ".";
  }
}

var carro = {
  meuCarro: "Punto",
  getCarro: tipoCarro("Fiat"),
```

```
  especial: vendas,
};

console.log(carro.meuCarro); // Punto
console.log(carro.getCarro); // Fiat
console.log(carro.especial); // Toyota
```

Além disso, você pode usar um literal numérico ou string para o nome de uma propriedade ou aninhar um objeto dentro do outro. O exemplo a seguir usar essas opções.

```
var carro = { carros: { a: "Saab", b: "Jeep" }, 7: "Mazda" };

console.log(carro.carros.b); // Jeep
console.log(carro[7]); // Mazda
```

Nomes de propriedades de objeto podem ser qualquer string, incluindo uma string vazia. Caso o nome da propriedade não seja um identificador JavaScript ou número, ele deve ser colocado entre aspas. Nomes de propriedades que não possuem identificadores válido, também não podem ser acessadas pela propriedade de ponto (`.`), mas podem ser acessadas e definidas com a notação do tipo array ("`[]`").

```
var unusualPropertyNames = {
  "": "Uma string vazia",
  "!": "Bang!"
}
console.log(unusualPropertyNames."");   // SyntaxError: string inesperada
console.log(unusualPropertyNames[""]);  // Um string vazia
console.log(unusualPropertyNames.!);    // SyntaxError: símbolo ! inesperado
console.log(unusualPropertyNames["!"]); // Bang!
```

Observe:

```
var foo = { a: "alpha", 2: "two" };
console.log(foo.a); // alpha
console.log(foo[2]); // two
//console.log(foo.2);  // Error: missing ) after argument list
//console.log(foo[a]); // Error: a não está definido
console.log(foo["a"]); // alpha
console.log(foo["2"]); // two
```

Expressão Regex Literal

Um regex literal é um padrão entre barras. A seguir um exemplo de regex literal.

```
var re = /ab+c/;
```

String Literal

Uma string literal são zero ou mais caracteres dispostos em aspas duplas (`"`) ou aspas simples (`'`). Uma sequência de caracteres deve ser delimitada por aspas do mesmo tipo; ou seja, as duas aspas simples ou ambas aspas duplas. A seguir um exemplo de strings literais.

```
"foo"
'bar'
"1234"
"um linha \n outra linha"
"John's cat"
```

Você pode chamar qualquer um dos métodos do objeto string em uma string literal - JavaScript automaticamente converte a string literal para um objeto string temporário, chama o método, em seguida, descarta o objeto string temporário. Você também pode usar a propriedade `String.length` com uma string literal:

```
console.log("John's cat".length);
// Irá exibir a quantidade de caracteres na string incluindo o espaço em branco.
// Nesse caso, 10 caracteres.
```

Você deve usar string literal, a não ser que você precise usar um objeto string. Veja `String` para detalhes sobre objetos de strings.

Uso de caracteres especiais em string

Além dos caracteres comuns, você também pode incluir caracteres especiais em strings, como mostrado no exemplo a seguir.

```
"uma linha \n outra linha";
```

A tabela a seguir lista os caracteres especiais que podem ser usados em strings no JavaScript.

Caracter	Descrição
`\0`	Byte nulo
`\b`	Backspace
`\f`	Alimentador de formulário
`\n`	Nova linha
`\r`	Retorno do carro

Caracter	Descrição
`\t`	Tabulação
`\v`	Tabulação vertical
`\'`	Apóstrofo ou aspas simples
`\"`	Aspas dupla
`\\`	Caractere de barra invertida
`\xxx`	Caractere com a codificação Latin-1 especificada por três dígitos octal *XXX* entre 0 e 377. Por exemplo, \251 é sequência octal para o símbolo de direitos autorais.
`\xXX`	Caractere com a codificação Latin-1 especificada por dois dígitos hexadecimal *XX* entre 00 e FF. Por exemplo, \xA9 é a sequência hexadecimal para o símbolo de direitos autorais.
`\uXXXX`	Caractere Unicode especificado por quatro dígitos hexadecimal *XXXX*. Por exemplo, \u00A9 é a sequência Unicode para o símbolo de direitos autorais. Veja sequências de escape Unicode.

Caracteres de escape

Para caracteres não listados na tabela, se precedidos de barra invertida ela é ignorada, seu uso está absoleto e deve ser ignorado.

Você pode inserir uma aspa dentro de uma string precendendo-a com uma barra invertida. Isso é conhecido como *escaping* das aspas. Por exemplo:

```
var quote = "Ele lê \"The Cremation of Sam McGee\" de R.W. Service.";
console.log(quote);
```

O resultado disso seria:

```
Ele lê "The Cremation of Sam McGee" de R.W. Service.
```

Para incluir uma barra invertida dentro de uma string, você deve escapar o caractere de barra invertida. Por exemplo, para atribuir o caminho do arquivo `c:\temp` para uma string, utilize o seguinte:

```
var home = "c:\\temp";
```

Você também pode escapar quebras de linhas, precedendo-as com barra invertida. A barra invertida e a quebra de linha são ambas removidas da string.

```
var str =
  "esta string \
está quebrada \
em várias\
linhas.";
console.log(str); // esta string está quebrada em várias linhas.
```

Embora JavaScript não tenha sintaxe "heredoc", você pode adicionar uma quebra de linha e um escape de quebra de linha no final de cada linha:

```
var poema =
  "Rosas são vermelhas\n\
Violetas são azuis,\n\
Esse seu sorriso\n\
é o que me seduz. (Lucas Pedrosa)";
```

Mais informação

Este capítulo focou na sintaxe básica das declarações e tipos. Para saber mais sobre a linguagem JavaScript, veja também os seguintes capítulos deste guia:

- Controle de fluxo e manipulação de erro
- Laços e iteração
- Funções
- Expressões e operadores

No próximo capítulo, veremos a construção de controle de fluxos e manipulação de erro.

- Anterior
- Próxima

Coleções Indexadas

- Anterior
- Próxima

Este capítulo introduz coleções de dados que são ordenados por um valor indexado. Isso inclui construtores de arrays e array-like como objetos `Array` e `TypedArray`.

Objeto <code>Array</code>

Um array é um conjunto de valores ordenados que você o referencia com um nome e um índice. Por exemplo, você pode ter um array chamado `emp` que contém nomes de funcionários indexados por seus números de funcionários. Então `emp[1]` poderia ser o funcionário número 1, `emp[2]` o funcionário número 2 e assim por diante.

JavaScript não possui um tipo de dados array específico. No entanto, você pode usar o objeto predefinido `Array` e seus métodos para trabalhar com arrays em suas aplicações. O objeto `Array` tem métodos para manipular arrays de várias maneiras como join, reverse e sort. Ele tem uma propriedade para determinar o tamanho do array e outras propriedades para usar com expressões regulares.

Criando um array

As declarações a seguir criam arrays equivalentes:

```
var arr = new Array(elemento0, elemento1, ..., elementoN);
var arr = Array(elemento0, elemento1, ..., elementoN);
var arr = [elemento0, elemento1, ..., elementoN];
```

`elemento0, elemento1, ..., elementoN` é uma lista de valores para os elementos do array. Quando esses valores são especificados, o array é inicializado com eles como elementos deste array. A propriedade do comprimento do array é definida pelo número de argumentos.

A sintaxe dos colchetes é chamada de "array literal" ou "inicializador de array". É uma abreviação de outras formas de criação de array e é a forma preferida de criação. Veja Array literal para detalhes.

Para criar um array com tamanho diferente de zero mas sem nenhum item, qualquer esquema abaixo pode ser utilizado:

```
var arr = new Array(comprimentoDoArray);
var arr = Array(comprimentoDoArray);

// Estes produzem exatamente o mesmo efeito
var arr = [];
arr.length = comprimentoDoArray;
```

> **Nota:** No código acima, `comprimentoDoArray` deve ser um `Número`. De outra maneira, um array com um único elemento (o valor passado) será criado. Chamar `arr.length` retornará `comprimentoDoArray`, mas o array na verdade, contem elementos vazios (undefined). Executar um loop `for...in` no array, não retornará nenhum dos elementos do array.

Além de poderem ser definidos como uma nova variável, como mostrado acima, arrays também podem ser atribuídos como uma propriedade de um novo objeto, ou de um objeto existente:

```
var obj = {};
// ...
obj.prop = [elemento0, elemento1, ..., elementoN];

// OU
var obj = {prop: [elemento0, elemento1, ...., elementoN]}
```

Se você deseja inicializar um array com um único elemento, e este elemento é um `Número`, você precisa usar a sintáxe dos colchetes. Quando um único valor de `Número` é passado para o construtor do Array(), ou para uma função, ele é interpretado como um `comprimentoDoArray`, e não como um elemento único.

```
var arr = [42]; // Cria um array com apenas um elemento:
// o número 42.

var arr = Array(42); // Cria um array sem elementos
// e arr.length é definido como 42; isso é
// equivalente a:
var arr = [];
arr.length = 42;
```

Chamar `Array(N)` resulta em um `RangeError`, se `N` é um número não inteiro cuja porção fracionária não é zero. O exemplo a seguir ilustra esse comportamento.

```
var arr = Array(9.3); // RangeError: Invalid array length
```

Se o seu código precisa criar arrays com elementos singulares de um tipo de dados arbitrário, é mais seguro usar arrays literais. Ou então, crie um array vazio antes de adicionar um elemento singular nele.

Povoando um array

Você pode povoar (inserir elementos) a um array atribuindo valores aos seus elementos. Por exemplo:

```
var emp = [];
emp[0] = "Casey Jones";
emp[1] = "Phil Lesh";
emp[2] = "August West";
```

> **Nota:** se você fornece um valor não inteiro ao operador do array, como no código acima, a propriedade será criada no objeto representando o array, ao invés do elemento do array.

```
var arr = [];
arr[3.4] = "Oranges";
console.log(arr.length); // 0
console.log(arr.hasOwnProperty(3.4)); // verdadeiro
```

Você também pode povoar o array quando o cria:

```
var myArray = new Array("Olá", myVar, 3.14159);
var myArray = ["Manga", "Maçã", "Laranja"];
```

Referenciando os elementos do array

Você referencia os elementos do array através do uso de elementos numéricos ordinais. Por exemplo, suponha que você definiu o seguinte array:

```
var myArray = ["Vento", "Chuva", "Fogo"];
```

Você então se refere ao primeiro elemento do array como em `myArray[0]` e ao segundo elemento do array como em `myArray[1]`. O índice do elemento começa com zero.

> **Nota:** o operador do array (colchetes) também é usado para acessar as propriedades do array (arrays também são objetos em JavaScript). Por exemplo,

```
var arr = ["um", "dois", "três"];
arr[2]; // três
arr["length"]; // 3
```

Compreendendo o comprimento

Sobe o ponto de vista da implementação, arrays JavaScript armazenam na realidade elementos como propriedades de objetos padrões, usando o índice do array como o nome da propriedade. O comprimento da propriedade é especial: ele sempre retorna o índice do último mais um (no exemplo seguinte Dusty é indexado no 30, então cats.length retorna 30 + 1). Lembre-se, índices de arrays JavaScript são baseados no zero: eles começam no 0, não no 1. Isso significa que o comprimento da propriedade será um a mais que o maior índice armazenado no array:

```
var gatos = [];
gatos[30] = ["Dusty"];
console.log(gatos.length); // 31
```

Você também pode atribuir um valor à propriedade `length`. Ao escrever um valor menor que o número de itens armazenados, trunca o array: escrevendo zero limpa-o completamente:

```
var gatos = ["Dusty", "Misty", "Twiggy"];
console.log(gatos.length); // 3

gatos.length = 2;
console.log(gatos); // mostra "Dusty, Misty" - Twiggy foi removido

gatos.length = 0;
console.log(gatos); // nada é apresentado; o array gatos está vazio

gatos.length = 3;
console.log(gatos); // [undefined, undefined, undefined]
```

Iteração em arrays

Uma operação comum é a de iterar sobre os valores de um array, processando cada elemento de alguma maneira. A maneira mais simples para fazer isso é como segue:

```
var cores = ["vermelho", "verde", "azul"];
for (var i = 0; i < cores.length; i++) {
  console.log(cores[i]);
}
```

Se você sabe que nenhum dos elemnetos no seu array é avaliado como `false` em um contexto booleano — se o seu array consiste apenas de nodos do DOM, como exemplo, você pode usar um idioma mais eficiente:

```
var divs = document.getElementsByTagName("div");
for (var i = 0, div; (div = divs[i]); i++) {
  /* Processa div de alguma forma */
}
```

Isso evita a sobrecarga da checagem do comprimento do array, e garante que a variável div seja reatribuida ao item atual cada vez que o loop for adicionado por conveniência.

O método `forEach()` disponibiliza um outro jeito de iterar sobre/em um array:

```
var cores = ["vermelho", "verde", "azul"];
cores.forEach(function (cor) {
  console.log(cor);
});
// vermelho
// verde
// azul
```

Alternativamente, você pode encurtar o código para o parâmetro do `forEach` com Arrow Functions ES6.

```
var cores = ["vermelho", "verde", "azul"];
cores.forEach((cor) => console.log(cor));
// vermelho
// verde
// azul
```

A função passada para o `forEach` é executada uma vez para cada item no array, com o item do array passado como o argumento para a função. Valores não atribuídos não são iterados no loop `forEach`.

Note que os elementos de um array que foram omitidos quando o array foi definido, não são listados quando iterados pelo `forEach`, mas são listados quando `undefined` foi manualmente atribuído ao elemento:

```
var array = ["primeiro", "segundo", , "quarto"];

array.forEach(function (elemento) {
  console.log(elemento);
});
// primeiro
// segundo
// quarto

if (array[2] === undefined) {
  console.log("array[2] is undefined"); // verdadeiro
}

array = ["primeiro", "segundo", undefined, "quarto"];

array.forEach(function (elemento) {
  console.log(elemento);
});
// primeiro
// segundo
// undefined
// quarto
```

Como elementos JavaScript são salvos como propriedades de objetos padronizados, não é aconselhável iterar sobre arrays JavaScript usando loops `for...in`, porque elementos normais e todas as propriedades numeráveis serão listadas.

Métodos dos arrays

O objeto `Array` possui os seguintes métodos:

`concat()` une dois arrays e retorna um novo array.

```
var myArray = new Array("1", "2", "3");
myArray = myArray.concat("a", "b", "c");
// myArray agora é ["1", "2", "3", "a", "b", "c"]
```

`join(deliminator = ',')` une todos os elementos de um array dentro de um string.

```
var myArray = new Array("Vento", "Chuva", "Fogo");
var lista = myArray.join(" - "); // lista é "Vento - Chuva - Fogo"
```

`push()` adiciona um ou mais elementos no fim de um array e retorna o comprimento resultante do array.

```
var myArray = new Array("1", "2");
myArray.push("3"); // myArray é agora ["1", "2", "3"]
```

`pop()` remove o último elemento de um array e retorna esse elemento.

```
var myArray = new Array("1", "2", "3");
var ultimo = myArray.pop();
// myArray é agora ["1", "2"], ultimo = "3"
```

`shift()` remove o primeiro elemento de um array e retorna esse elemento.

```
var myArray = new Array("1", "2", "3");
var primeiro = myArray.shift();
// myArray agora é ["2", "3"], primeiro é "1"
```

`unshift()` adiciona um ou mais elementos ao início do array e retorna o novo comprimento do array.

```
var myArray = new Array("1", "2", "3");
myArray.unshift("4", "5");
// myArray torna-se ["4", "5", "1", "2", "3"]
```

`slice(start_index, upto_index)` extrai uma seção de um array e retorna um novo array.

```
var myArray = new Array("a", "b", "c", "d", "e");
myArray = myArray.slice(1, 4); // inicia no índice 1 e extrai todos os elementos
// até o índice 3, retornado [ "b", "c", "d"]
```

`splice(index, count_to_remove, addElement1, addElement2, ...)` remove elementos de um array e (opcionalmente) o substitui, e retorna os itens que foram removidos do array.

```
var myArray = new Array("1", "2", "3", "4", "5");
myArray.splice(1, 3, "a", "b", "c", "d");
// myArray é agora ["1", "a", "b", "c", "d", "5"]
// Este código iniciou no índice um (ou onde o "2" estava),
// removeu 3 elementos a partir dali, e então inseriu todos os elementos
// consecutivos em seus lugares.
```

`reverse()` transpõe (inverte) os elementos de um array, *in situ*: o primeiro elemento do array se torna o último e o último torna-se o primeiro, e retorna uma referência para o array.

```
var myArray = new Array("1", "2", "3");
myArray.reverse();
// transpõe o array de modo que myArray = [ "3", "2", "1" ]
```

`sort()` ordena os elementos de um array *in situ*, e retorna uma referência para o array.

```
var myArray = new Array("Neve", "Chuva", "Fogo");
myArray.sort();
// ordena o array de modo que myArray = [ "Chuva", "Fogo", "Neve" ]
```

`sort()` também pode 'pegar' uma função callback para determinar como os elementos do array são comparados.

O método sort, assim como outros métodos abaixo que tomam um callback são conhecidos como *métodos iterativos*, porque eles iteram sobre o array de alguma forma. Cada um pega um segundo argumento opcional chamado `thisObject`. Se fornecido, `thisObject` se torna o valor da palavra chave `this` dentro do corpo da função callback. Se não fornecido, como em outros casos onde uma função é invocada fora do contexto explícito de um objeto, `this` fará referência ao objeto global (`window`).

A função callback é na verdade chamada com três argumentos. O primeiro é o valor do item corrente, o segundo é o índice do array e o terceiro é uma referência ao próprio array. Funções javaScript ignoram qualquer argumento que não são nomeados na lista de parâmetros, portanto é seguro prover uma função callback que toma somente um único argumento, como a função `alert`.

A função abaixo compara dois valores e retorna um dos tres valores: -1, 0 ou 1.

Por exemplo, o seguinte trecho de código vai ordenar pela última letra da string:

```
var sortFn = function (a, b) {
  if (a[a.length - 1] < b[b.length - 1]) return -1;
  if (a[a.length - 1] > b[b.length - 1]) return 1;
  if (a[a.length - 1] == b[b.length - 1]) return 0;
};
myArray.sort(sortFn);
// ordena o array de modo que myArray = ["Chuva","Neve","Fogo"]
```

- se `a` for menor que `b` pelo sistema de ordenação, retorna -1 (ou qualquer número negativo)
- se `a` for maior que `b` pelo sistema de ordenação, retorna 1 (ou qualquer número positivo)
- se `a` e `b` forem considerados equivalentes, retornará 0.

`indexOf(searchElement[, fromIndex])` busca `searchElement` no array e retorna o índice da primeira ocorrência.

```
var a = ["a", "b", "a", "b", "a"];
console.log(a.indexOf("b")); // mostra 1
// Agora tente novamente, iniciando após o último resultado de busca
console.log(a.indexOf("b", 2)); // mostra 3
console.log(a.indexOf("z")); // mostra -1, porque 'z' não foi encontrado
```

`lastIndexOf(searchElement[, fromIndex])` funciona como `indexOf`, mas começa no fim e busca de trás para a frente.

```
var a = ["a", "b", "c", "d", "a", "b"];
console.log(a.lastIndexOf("b")); // mostra 5
// Agora tente novamente, iniciando antes do último resultado de busca
console.log(a.lastIndexOf("b", 4)); // mostra 1
console.log(a.lastIndexOf("z")); // mostra -1
```

`forEach(callback[, thisObject])` executa um `callback` em cada item do array e retorna undefined.

```
var a = ["a", "b", "c"];
a.forEach(function (element) {
  console.log(elemento);
});
// mostra cada item por vez
```

`map(callback[, thisObject])` retorna um novo array do valor retornado da execução do callback em cada item do array.

```
var a1 = ["a", "b", "c"];
var a2 = a1.map(function (item) {
  return item.toUpperCase();
```

```
});
console.log(a2); // logs ['A', 'B', 'C']
```

`filter(callback[, thisObject])` retorna um novo array contendo os items verdadeiros ao executar o callback.

```
var a1 = ["a", 10, "b", 20, "c", 30];
var a2 = a1.filter(function (item) {
  return typeof item === "number";
});
console.log(a2); // mostra [10, 20, 30]
```

`every(callback[, thisObject])` retorna verdadeiro se o `callback` retornar verdadeiro para cada item no array.

```
function isNumber(valor) {
  return typeof valor === "number";
}
var a1 = [1, 2, 3];
console.log(a1.every(isNumber)); // mostra true
var a2 = [1, "2", 3];
console.log(a2.every(isNumber)); // mostra false
```

`some(callback[, thisObject])` retorna verdadeiro se o `callback` retornar verdadeiro para pelo menos um item no array.

```
function isNumber(valor) {
  return typeof valor === "number";
}
var a1 = [1, 2, 3];
console.log(a1.some(isNumber)); // mostra true
var a2 = [1, "2", 3];
console.log(a2.some(isNumber)); // mostra true
var a3 = ["1", "2", "3"];
console.log(a3.some(isNumber)); // mostra false
```

`reduce(callback[, initialValue])` aplica `callback(firstValue, secondValue)` para reduzir a lista de itens para um único valor e retorna este valor.

```
var a = [10, 20, 30];
var total = a.reduce(function (primeiro, segundo) {
  return primeiro + segundo;
}, 0);
console.log(total); // mostra 60
```

`reduceRight(callback[, initalvalue])` funciona como `reduce()`, mas inicia com o último elemento.

`reduce` e `reduceRight` são os métodos iterativos menos óbvios dos arrays. Eles devem ser usados para algorítmos que combinam dois valores de maneira recursiva com a finalidade de reduzir uma sequência para um único valor.

Arrays multidimensionais

Arrays podem ser aninhados, significando que um array pode conter outro array como seu elemento. Usando essa característica dos arrays JavaScript, arrays multidimensionais pode ser criados.

O código a seguir cria dois arrays multidimensionais:

```
var a = new Array(4);
for (i = 0; i < 4; i++) {
  a[i] = new Array(4);
  for (j = 0; j < 4; j++) {
    a[i][j] = "[" + i + "," + j + "]";
  }
}
```

Esse exemplo cria um array com as seguintes linhas:

```
Linha 0: [0,0] [0,1] [0,2] [0,3]
Linha 1: [1,0] [1,1] [1,2] [1,3]
Linha 2: [2,0] [2,1] [2,2] [2,3]
Linha 3: [3,0] [3,1] [3,2] [3,3]
```

Arrays e expressões regulares

Quando um array é o resultado de uma equivalência entre uma expressão regular e um string, o array retorna propriedades e elementos que disponibilizam a informação sobre a correspondência. Um array é o valor retornado de `RegExp.exec()`, `String.match()`, e `String.split()`. Para informações sobre o uso de arrays com expressões regulares, veja Expressões Regulares.

Trabalhando com objetos array-like

Alguns objetos JavaScript, como a `NodeList` retornada pelo `document.getElementsByTagName()` ou o objeto acessível dentro do `arguments` de uma função, se parecem e se comportam superficialmente como arrays, mas não compartilham de todos os seus métodos. O objeto `arguments` fornece um atributo `length` mas não implementa o método `forEach()`, por exemplo.

Métodos Array prototype podem ser chamados contra outros objetos array-like. Por exemplo:

```
function printArguments() {
  Array.prototype.forEach.call(arguments, function (item) {
    console.log(item);
  });
}
```

Métodos Array prototype também podem ser usados em strings, desde que eles forneçam acesso sequencial a seus caracteres de maneira similar às arrays:

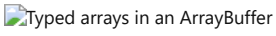
```
Array.prototype.forEach.call("uma string", function (chr) {
  console.log(chr);
});
```

Arrays Tipados

Arrays tipados no JavaScript são objetos array-like e provêm um mecanismo para acessar dados binários crus. Como você já sabe, objetos `Array` crescem e encolhem dinamicamente e podem ter um valor JavaScript. O motor do JavaScript executa otimizações para que os arrays sejam rápidos. Contudo, à medida que as aplicações web se tornam cada vez mais poderosas, com a adição de funcionalidades como manipulação de áudio e vídeo, acesso a dados crus usando WebSockets, etc., ficou claro que existem momentos em que seria útil para o código JavaScript ser capaz de rapida e facilmente manipular dados binários crus em arrays tipados.

Buffers e views: arquitetura do array tipado

Para alcançar máxima flexibilidade e eficiência, as views de array tipado do JavaScript dividem a implementação em **buffers** e **views.** Um buffer (implementado pelo objeto `ArrayBuffer`) é um objeto que representa um monte de dados; não possui nenhum formato específico e não oferece nenhum mecanismo para acessar seu conteúdo. Para acessar a memória contida em um buffer, você precisa usar uma view. Uma view provê um contexto — ou seja, um tipo de dado, um offset inicial e número de elementos — que transforma o dado em um array tipado real.



ArrayBuffer

O `ArrayBuffer` é um tipo de dado usado para representar um buffer de dados binários de tamanho fixo genérico. Você não pode manipular diretamente o conteúdo de um `ArrayBuffer`; ao invés disso, você deve criar uma view de array tipado ou uma `DataView` que represente o buffer em um formato específico, e use esta view para ler e modificar o conteúdo do buffer.

Views de arrays tipados

Views de arrays tipados possuem nomes autodescritivos e provêm views para todos os tipos numéricos usuais como `Int8`, `Uint32`, `Float64` e assim por diante. Existe uma view de array tipado especial, o `Uint8ClampedArray`. Ela fixa os valores entre 0 e 255. Isto é útil para Canvas data processing, por exemplo.

Para mais informações, veja Arrays tipados no JavaScript e documentos de referência para os diferentes objetos `TypedArray`.

- Anterior
- Próxima

Herança e cadeia de protótipos (prototype chain)

JavaScript é um pouco confuso para desenvolvedores com experiência em linguagens baseadas em classes (como Java ou C++), porque é dinâmico e não dispõe de uma implementação de uma `class` (a palavra-chave `class` foi introduzida no ES2015, mas é syntax sugar, o JavaScript permanece baseado em `prototype`).

Quando se trata de herança, o JavaScript tem somente um construtor: objetos. Cada objeto tem um link interno para um outro objeto chamado `prototype`. Esse objeto `prototype` também tem um atributo `prototype`, e assim por diante até o que o valor `null` seja encontrado como sendo o seu `prototype`. `null` que, por definição, não tem `prototype`, e age como um link final nesta **cadeia de protótipos** (prototype chain).

Isto muitas vezes é considerado como um dos pontos fracos do JavaScript, mas o modelo de herança prototipal é de fato mais potente do que o modelo clássico. É, por exemplo, relativamente trivial construir um "modelo clássico" (como na implementaçao de `class`), enquanto o contrário é uma tarefa muito mais difícil.

1 *N. da T: Como em uma implementação de fila em C, por exemplo.*

Herança com o encadeamento de protótipos

Propriedades de heranças

Objetos em JavaScript são "sacos" dinâmicos de propriedades (a que se refere as próprias propriedades) e cada um tem um link para um objeto `prototype`. Eis o que acontece quando se tenta acessar uma propriedade:

```
// Vamos criar um objeto o da função f com suas próprias propriedades a e b:
let f = function () {
   this.a = 1;
   this.b = 2;
}
let o = new f(); // {a: 1, b: 2}

// adicionar propriedades no protótipo da função f
f.prototype.b = 3;
f.prototype.c = 4;

// não defina o protótipo f.prototype = {b: 3, c: 4}; isso vai quebrar a cadeia de protótipos
// o. [[Prototype]] possui propriedades bec.
// o. [[Prototype]]. [[Prototype]] é Object.prototype.
// Finalmente, o. [[Prototype]]. [[Prototype]]. [[Prototype]] é nulo.
// Este é o fim da cadeia de protótipos, como nulo,
// por definição, não possui [[Prototype]].
```

```
// Assim, a cadeia completa de protótipos se parece com:
// {a: 1, b: 2} ---> {b: 3, c: 4} ---> Object.prototype ---> null dfdf

console.log(o.a); // 1
// Existe uma propriedade 'a' no objeto o? Sim, e seu valor é 1.

console.log(o.b); // 2
// Existe uma propriedade própria 'b' em o? Sim, e seu valor é 2.
// O protótipo também tem uma propriedade 'b', mas não é visitado.
// Isso é chamado de sombreamento de propriedade(Property Shadowing)

console.log(o.c); // 4
// Existe uma propriedade própria 'c' em o? Não, verifique seu protótipo.
// Existe uma propriedade 'c' própria em o. [[Prototype]]? Sim, seu valor é 4.

console.log(o.d); // undefined
// Existe uma propriedade 'd' própria em o? Não, verifique seu prototype.
// Existe uma propriedade 'd' em o. [[Prototype]]? Não, verifique seu prototype.
// o. [[Prototype]]. [[Prototype]] é Object.prototype e não há propriedade 'd' por padrão, verifique seu prototype.
// o. [[Prototype]]. [[Prototype]]. [[Prototype]] é nulo, pare de pesquisar,
// nenhuma propriedade encontrada, retorne indefinido.
```

Code Link

Atribuir uma propriedade a um objeto cria uma propriedade nele. A única exceção às regras de obtenção e definição de comportamento é quando há uma propriedade herdada com um getter or a setter.

Herança de "métodos"

JavaScript não tem "métodos" como os que conhecemos em linguagens baseadas em classes. Em JavaScript, qualquer função pode ser adicionada em um objeto em forma de propriedade. Uma herança de funções age como a herança de quaisquer outras propriedades que não sejam funções, e podemos inclusive realizar sobre-escrita de função (*method overriding*)!

Quando uma herança de função é executada, o valor de `this` aponta para o objeto que herdou as propriedades, não para o objeto prototype onde as propriedades foram escritas originalmente.

```
var o = {
  a: 2,
  m: function (b) {
    return this.a + 1;
  },
};

console.log(o.m()); // 3
// Ao chamar 'o.m' neste caso, "this" refere-se a 'o'

var p = Object.create(o);
// 'p' é um objeto que foi herdado de 'o'

p.a = 12; // cria uma propriedade 'a' no objeto 'p'
console.log(p.m()); // 13
// Ao chamar 'p.m', 'this' refere-se a 'p'
// Então, quando 'p' herda a função de 'm' de 'o', 'this.a' representa 'p.a' que é a própria propriedade 'a' de 'p'
```

Maneiras de criar objetos e resultados dos protótipos encadeados

Objetos criados com sintaxe de construtores

```
var o = { a: 1 };

// O recém-criado objeto 'o' tem Object.prototype como o seu [[Prototype]]
// 'o' não tem uma propriedade chamada 'hasOwnProperty'
// hasOwnProperty é uma propriedade própria de Object.prototype. Então 'o' herda hasOwnProperty de Object.prototype

// Object.prototype tem null como seu protótipo.
// o ---> Object.prototype ---> null

var a = ["yo", "whadup", "?"];

// Arrays herdam de Array.prototype (que tem métodos como indexOf, forEach, etc.)
// A cadeia de protótipos se parece com isso:
// a ---> Array.prototype ---> Object.prototype ---> null

function f() {
  return 2;
}

// Funções herdam de Function.prototype (que tem métodos como call, bind, etc.)
// f ---> Function.prototype ---> Object.prototype ---> null
```

Com um construtor

Um "construtor" em JavaScript é "somente" uma função que passa a ser chamada com o operador new.

```
function Graph() {
  this.vertexes = [];
  this.edges = [];
}

Graph.prototype = {
  addVertex: function (v) {
    this.vertexes.push(v);
  },
};

var g = new Graph();
// 'g' é um objeto com as propriedades 'vertexes' e 'edges'.
// g.[[Prototype]] é o valor de Graph.prototype quando new Graph() é executada.
```

Com Object.create

ECMAScript 5 introduziu o novo método: Object.create. Invocando este método podemos criar novos objetos. O prototype destes novos objetos é o primeiro argumento do método:

```
var a = { a: 1 };
// a ---> Object.prototype ---> null

var b = Object.create(a);
// b ---> a ---> Object.prototype ---> null
console.log(b.a); // 1 (inherited)

var c = Object.create(b);
// c ---> b ---> a ---> Object.prototype ---> null

var d = Object.create(null);
// d ---> null
console.log(d.hasOwnProperty); // undefined, porque não herda de Object.prototype
```

Performance

O tempo de pesquisa para as propriedades que estão no alto da cadeia de protótipos pode ter um impacto negativo no desempenho, e isso pode ser significativo no código em que o desempenho é crítico. Além disso, tentar acessar propriedades inexistentes irá sempre atravessar a cadeia cheia do protótipo (full prototype chain).

Porém, quando estamos interagindo com as propriedades de um objeto, **toda** propriedade que está na cadeia do prototype (prototype chain) vai ser enumerada.

Para verificar se um objeto tem uma propriedade definida em si mesmo e não em algum lugar na sua cadeia de protótipo, é necessário usar o método `hasOwnProperty` que todos os objetos herdam do Object.prototype.

`hasOwnProperty` é a única alternativa em JavaScript que lida com propriedades sem atravessar a cadeia de protótipos.

> **Nota: Não** é suficiente apenas verificar se o valor da propriedade é `undefined` para saber se ela existe. A propriedade pode muito bem existir e não ter sido inicializada, sendo assim o seu valor undefined.

Má Pratica: Estender protótipos nativos

Um erro frequentemente cometido por programadores é estender um Object.prototype.

Esta técnica é chamada de "monkey patching" e quebra o encapsulamento. Não existe uma boa razão para desorganizar tipos nativos do JavaScript para adicionar uma nova funcionalidade ao mesmo.

O único bom motivo para estender um protótipo nativo do JavaScript é para dar suporte a novas "features" do JavaScript; por exemplo: Array.forEach, etc.

Conclusão

É essencial entender bem "prototypal inheritance" antes de escrever códigos complexos. Tome cuidado com o tamanho da sua cadeia de protótipos, quebre a cadeia caso necessário para evitar problemas de performance. Nunca estenda protótipos nativos a menos que seja para conseguir compatibilidade com novas "features" do JavaScript.

Introdução

- Anterior
- Próxima

Este capítulo apresenta o JavaScript e discute alguns de seus conceitos fundamentais.

O que você realmente já deveria saber?

Este guia assume que você tem os seguintes conhecimentos básicos:

- Um entendimento geral da internet e da World Wide Web (WWW).
- Um bom conhecimento de HyperText Markup Language (HTML)
- Alguma experiência em programação. Se você é novo em programação, veja alguns tutorias na página inicial sobre JavaScript.

Onde encontrar informações sobre JavaScript

A documentação de JavaScript na MDN inclui o seguinte:

- Aprendendo sobre a internet fornece informações aos iniciantes e introduz os conceitos básicos de programação e da internet.
- Guia JavaScript (este guia) dá uma visão geral sobre a linguagem de programação JavaScript e seus objetos.
- Referência JavaScript provê um material de referência detalhado da linguagem JavaScript.

Se você é novo em JavaScript, comece com os artigos na área de aprendizado e com o Guia JavaScript. Quando você já possuir uma compreensão dos princípios básicos, você pode usar a Referência JavaScript para ter mais detalhes sobre objetos e declarações.

O que é JavaScript?

JavaScript é uma linguagem de script orientada a objetos e plataforma cruzada usada para tornar as páginas da Web interativas (por exemplo, com animações complexas, botões clicáveis, menus pop-up etc.). Há também versões mais avançadas do lado do servidor do JavaScript, como Node.js, que permitem adicionar mais funcionalidades a um site do que baixar arquivos (como colaboração em tempo real entre vários computadores). Dentro de um ambiente de host (por exemplo, um navegador da Web), o JavaScript pode ser conectado aos objetos de seu ambiente para fornecer controle programático sobre eles.

JavaScript tem uma biblioteca padrão de objetos, como: `Array`, `Date`, e `Math`, e um conjunto de elementos que formam o núcleo da linguagem, tais como: operadores, estruturas de controle e declarações. O núcleo do JavaScript pode ser estendido para uma variedade de propósitos, complementando assim a linguagem:

- *O lado cliente do JavaScript* estende-se do núcleo linguagem, fornecendo objetos para controlar um navegador web e seu *Document Object Model* (DOM). Por exemplo, as extensões do lado do cliente permitem que uma aplicação coloque elementos em um formulário HTML e responda a eventos do usuário, como cliques do mouse, entrada de formulário e de navegação da página.
- *O lado do servidor do JavaScript* estende-se do núcleo da linguagem, fornecendo objetos relevantes à execução do JavaScript em um servidor. Por exemplo, as extensões do lado do servidor permitem que uma aplicação comunique-se com um banco de dados, garantindo a continuidade de informações de uma chamada para a outra da aplicação, ou executar manipulações de arquivos em um servidor.

Isso significa que, no navegador, o JavaScript pode alterar a aparência da página da Web (DOM). E, da mesma forma, o JavaScript Node.js no servidor pode responder a solicitações personalizadas enviadas por código executado no navegador.

JavaScript e Java

JavaScript e Java são similares em algumas coisas, mas são diferentes em outras. O JavaScript assemelha-se ao Java, porém não possui tipagem estática e checagem rigída de tipos como o Java. JavaScript segue a sintaxe básica do Java, convenções de nomenclatura e construções de controle de fluxo, razões pelas quais esta linguagem foi renomeada de LiveScript para JavaScript.

Em contraste com o sistema em tempo de compilação das classes construídas por declarações no Java, JavaScript suporta um sistema em tempo de execução com base em um pequeno número de tipos de dados representando valores numéricos, booleanos, e strings. JavaScript tem um modelo de objeto baseado em protótipo em vez do modelo, mais comum, de objeto baseado em classes. O modelo baseado em protótipo fornece herança dinâmica; isto é, o que é herdado pode variar para objetos individuais. JavaScript também suporta funções sem quaisquer requisitos especiais declarativos. As funções podem ser propriedades de objetos, executando como métodos.

JavaScript é uma linguagem mais livre em comparação a Java. Você não tem de declarar todas as variáveis, classes e métodos. Você não tem que se preocupar com o fato dos métodos serem públicos, privados ou protegidos, e você não tem que implementar interfaces. Variáveis, parâmetros e tipo de retorno da função não são explicitamente tipados.

Java é uma linguagem de programação baseada em classes, projetada para execução rápida e segurança de tipos. Segurança de tipo significa que, por exemplo, você não pode converter um número inteiro em Java para uma referência de objeto ou acessar a memória privada corrompendo bytecodes Java. O modelo baseado em classes do Java significa que os programas são exclusivamente constituídos por classes e seus métodos. Herança de classe do Java e tipagem forte geralmente requerem hierarquias de objetos fortemente acoplados. Esses requisitos tornam a programação em Java mais complexa do que a programação em JavaScript.

Em contraste, JavaScript descende em espírito de uma linhagem de linguagens menores com tipagem dinâmica, como HyperTalk e dBASE. Essas linguagens de script oferecem ferramentas de programação para um público muito mais amplo por causa de sua sintaxe mais fácil, funções especializadas embutidas e requisitos mínimos para a criação de objetos.

JavaScript	Java
Orientada a objeto. Sem distinção entre tipos e objetos. A herança é feita através do protótipo e as propriedades e métodos podem ser adicionadas a qualquer objeto dinamicamente.	Baseada em classes. Objetos são divididos em classes e instâncias com toda a herança através da hierarquia da classe. Classes e instâncias não podem ter propriedades ou métodos adicionados dinamicamente.
Os tipos de dados das variáveis não precisam ser declarados (tipagem dinâmica, vagamente tipado)	Os tipos de dados das variáveis devem ser declarados (tipagem estática, fortemente tipado).
Não pode escrever automaticamente no disco rigído.	Pode escrever automaticamente no disco rigído.

JavaScript e a especificação ECMAScript

O JavaScript é padronizado pela Ecma International — a associação Europeia para a padronização de sistemas de comunicação e informação (antigamente ECMA era um acrônimo para European Computer Manufacturers Association) para entregar uma linguagem de programação padronizada, internacional baseada em JavaScript. Esta versão padronizada de JavaScript, chamada ECMAScript, comporta-se da mesma forma em todas as aplicações que suportam o padrão. As empresas podem usar a linguagem de padrão aberto para desenvolver a sua implementação de JavaScript. O padrão ECMAScript é documentado na especificação ECMA-262.

O padrão ECMA-262 também é aprovado pela ISO (International Organization for Standardization) como ISO-16262. Você também pode encontrar a especificação no site da Ecma International. A especificação ECMAScript não descreve o Document Object Model (DOM), que é padronizado pelo World Wide Web Consortium (W3C) e/ou WHATWG (Web Hypertext Application Technology Working Group). O DOM define a maneira na qual os objetos do documento HTML estão expostos no seu script. Para ter uma ideia melhor sobre as diferentes tecnologias que são usadas durante a programação com JavaScript, consultar o artigo Visão Geral das tecnologias JavaScript.

Documentação JavaScript versus especificação ECMAScript

A especificação ECMAScript é um conjunto de requisitos para a implementação ECMAScript. É útil se você desejar implementar recursos compatíveis com os padrões da linguagem em sua implementação ECMAScript ou *engine* (como SpiderMonkey no Firefox, ou v8 no Chrome).

O documento ECMAScript *não* se destina a ajudar programadores de script. Utilize a documentação JavaScript para obter informações ao escrever scripts.

A especificação ECMAScript utiliza terminologia e sintaxe que podem ser desconhecidos para um programador JavaScript. Embora a descrição da linguagem possa ser diferente no ECMAScript, a linguagem em si continua sendo a mesma. JavaScript suporta todas as funcionalidades descritas na especificação ECMAScript.

A documentação JavaScript descreve aspectos da linguagem que são apropriadas para um programador JavaScript.

Começando com JavaScript

Para começar a aprender JavaScript é fácil: Tudo o que você precisa é de um navegador web moderno. Versões recentes do Firefox, Chrome, Microsoft Edge, e Safari todas suportam os recursos discutidos neste guia.

Uma ferramenta muito útil para explorar o JavaScript é o JavaScript Console (às vezes chamado de Web Console, ou apenas o console): é uma ferramenta que permite inserir JavaScript e executá-lo na página atual.

A captura de tela aqui mostra o Firefox Web Console, mas todos os navegadores modernos vêm com um console que funciona de maneira semelhante.

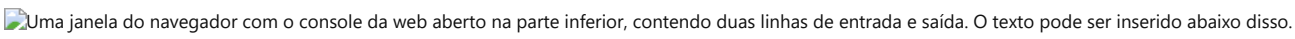
Abrindo o console

As instruções exatas para abrir o console variam de um navegador para outro:

- Abrindo o console no Firefox
- Abrindo o console no Chrome
- Abrindo o console no Microsoft Edge

Inserindo e executando JavaScript

O console aparece na parte inferior da janela do navegador. Na parte inferior do console há uma linha de entrada que você pode usar para inserir JavaScript, e a saída aparece no painel acima:


Uma janela do navegador com o console da web aberto na parte inferior, contendo duas linhas de entrada e saída. O texto pode ser inserido abaixo disso.

O console funciona exatamente da mesma forma que `eval`: a última expressão digitada é retornada. Por uma questão de simplicidade, pode-se imaginar que toda vez que algo é inserido no console, ele é realmente cercado por `console.log` em torno de `eval`, assim:

```
console.log(eval("3 + 5"));
```

Entrada de várias linhas no console

Por padrão, se você pressionar `Enter` após inserir uma linha de código, a string digitada será executada. Para inserir entrada multilinha:

- Se a string digitada estiver incompleta (por exemplo, você digitou `function foo() {`), o console tratará `Enter` como uma quebra de linha e permitirá que você digite outra linha.
- Se você segurar `Shift` enquanto pressiona `Enter`, o console tratará isso como uma quebra de linha e permitirá que você digite outra linha.
- Somente no Firefox, você pode ativar modo de entrada multi-linha, em onde você pode inserir várias linhas em um mini-editor e, em seguida, executar tudo quando estiver pronto.

Para começar a escrever JavaScript, abra o console, copie o seguinte código e cole na linha de comando:

```
(function () {
  "use strict";
  /* Ínicio do seu código */
  function greetMe(seuNome) {
    alert(`Olá ${seuNome}`);
  }

  greetMe("Mundo");
  /* Fim do seu código */
})();
```

Pressione `Enter` para vê-lo abrir em seu navegador!

O que vem em seguida?

Nas páginas seguintes, este guia apresentará a sintaxe e os recursos da linguagem JavaScript, para que você possa escrever aplicativos mais complexos.

Mas, por enquanto, lembre-se de sempre incluir `(function(){"use strict";` antes de seu código e adicionar `})();` ao final de seu código. Os artigos modo estrito e IIFE explicam o que eles fazem, mas por enquanto eles podem ser considerados em fazer o seguinte:

1. Evitam a semântica em JavaScript que engana os iniciantes.
2. Impedem que trechos de código executados no console interajam entre si (por exemplo, ter algo criado em uma execução de console sendo usado para uma execução de console diferente).

- Anterior
- Próxima

Iteratores e geradores

- Anterior

- Próxima

Processar cada item em uma coleção é uma operação muito comum. O JavaScript disponibiliza uma série de maneiras de iterar sobre uma coleção, desde um simples laço `for`, até um `map()` e também com o `filter()`. Iteradores e Geradores trazem o conceito da interação ocorrer diretamente no núcleo da linguagem e prover um mecanismo para a customização do comportamento dos laços `for...of`.

Para detalhes, também veja:

- Protocolos de iteração
- `for...of`
- `function*` e `Generator`
- `yield` e `yield*`

Iterators (Iteradores)

Um objeto é um **iterator (iterador)** quando sabe como acessar itens numa coleção, um por vez, enquanto mantém rastreada a posição atual em uma dada sequência. Em JavaScript um iterator é um objeto que oferece o método `next()`, o qual retorna o próximo item da sequência. Este método retorna um objeto com duas propriedades: `done` e `value`.

Uma vez criado, um objeto iterator pode ser usado explicitamente ao chamar repetidas vezes o método `next()`.

```
const makeIterator = (array) => {
  let nextIndex = 0;

  return {
    next: () => {
      return nextIndex < array.length
        ? { value: array[nextIndex++], done: false }
        : { done: true };
    },
  };
};
```

Uma vez inicializado, o método `next()` pode ser chamado para acessar os pares chave/valor do objeto da vez.

```
let it = makeIterator(["yo", "ya"]);
console.log(it.next().value); // 'yo'
console.log(it.next().value); // 'ya'
console.log(it.next().done); // true
```

Iterables (Iteráveis)

Um objeto é iterável **(iterable),** se ele define seu comportamento de iteração, como no caso de quais valores percorridos em um laço do tipo `for..of`. Alguns tipos embutidos, como o `Array`, ou o `Map`, têm um comportamento iterável padrão, enquanto outros tipos (como o`Object`) não possuem.

Para que um objeto seja **iterable**, o objeto precisa implementar o método **@@iterator**, significando que o objeto (ou um dos objetos na cadeia de prototipagem - prototype chain) precisa ter uma propriedade com uma chave `Symbol.iterator`:

Iterables definidos pelo usuário

Você pode fazer seus próprios iteráveis da seguinte maneira:

```
var myIterable = {};
myIterable[Symbol.iterator] = function* () {
  yield 1;
  yield 2;
  yield 3;
};
[...myIterable]; // [1, 2, 3]
```

Iterables Built-in (Iteráveis Embutidos)

`String`, `Array`, `TypedArray`, `Map` e `Set` são iteráveis embutidos, pois o protótipo dos objetos de todos eles têm o método `Symbol.iterator`.

Syntaxes expecting iterables

Algumas declarações e expressões esperam por iteradores, por exemplo o `for-of` loops, spread operator, `yield*`, e destructuring assignment.

```
for (let value of ["a", "b", "c"]) {
  console.log(value);
}
// "a"
// "b"
// "c"

[..."abc"]; // ["a", "b", "c"]

function* gen() {
  yield* ["a", "b", "c"];
}

gen().next(); // { value:"a", done:false }
```

```
[(a, b, c)] = new Set(["a", "b", "c"]);
a; // "a"
```

Generators

Enquanto os iteradores são ferramentas muito úteis, sua criação requer um cuidado devido à necessidade de manter explícito seu estado interno. **Generators** provêm uma alternativa poderosa: eles te permitem definir um algoritmo iterativo escrevendo uma função simples que pode manter seu estado próprio.

Generator é um tipo especial de função que trabalha como uma factory para iteradores. A função vira um generator se ela contém uma ou mais expressões `yield` e se ela usa a sintaxe `function*`.

```
function* idMaker() {
  var index = 0;
  while (true) yield index++;
}

var gen = idMaker();

console.log(gen.next().value); // 0
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
// ...
```

Generators avançados

Generators computam seus valores "yielded" por demanda, que os permitem representar sequências de forma eficiente que costumam ser trabalhosas ao serem computadas, ou até sequências infinitas como demonstradas acima.

O método `next()` também aceita um valor que pode ser usado para modificar o estado interno de um generator. O valor passado pro `next()` será tratado como o resultado da última expressão yield que pausou o generator.

Aqui um gerador de sequência Fibonacci usando `next(x)` pra restartar a sequência:

```
function* fibonacci() {
  var fn1 = 1;
  var fn2 = 1;
  while (true) {
    var current = fn2;
    fn2 = fn1;
    fn1 = fn1 + current;
    var reset = yield current;
    if (reset) {
      fn1 = 1;
      fn2 = 1;
    }
  }
}

var sequence = fibonacci();
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 2
console.log(sequence.next().value); // 3
console.log(sequence.next().value); // 5
console.log(sequence.next().value); // 8
console.log(sequence.next().value); // 13
console.log(sequence.next(true).value); // 1
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 2
console.log(sequence.next().value); // 3
```

> **Nota:** Como um ponto de interesse, chamando `next(undefined)` é o mesmo que chamar `next()`. Entretanto, estartar um novo generator com qualquer valor que não seja undefined na chamada next() terá `TypeError` exception.

Você pode forçar um generator a lançar uma exceção chamando o seu método `throw()` e passando o valor da exceção que ele deve lançar. Essa exceção será lançada do contexto suspenso atual do generator, como se o `yield` atualmente suspenso fosse um `throw`.

Se um yield não for encontrado durante o processo de lançamento de um thrown exception, então o exception será propagado através da chamada do `throw()`, e pra subsequente chamada do `next()` que terá a propriedade done resultando em `true`.

Generators têm o método `return(value)` que retorna o valor pego e finaliza o generator.

- Anterior
- Próxima

Coleções chaveadas

- Anterior
- Próxima

Este capítulo apresenta coleções de dados que são ordenados por uma chave; Objetos Map e Set contêm elementos que são iteráveis em ordem de inserção.

Mapas

<code>Objeto Map</code>

ECMAScript 6 apresenta uma nova estrutura de dados para mapear valores para valores. Um objeto `Map` é um simples mapa de chave/valor que pode ter seus elementos iterados por ordem de inserção.

O código a seguir mostra algumas operações básicas com um `Map`. Veja também a página de referência do `Map` para mais exemplos e a API completa. Você pode usar um laço `for...of` para retornar um array de `[chave, valor]` para cada iteração.

```
var sayings = new Map();
sayings.set("dog", "woof");
sayings.set("cat", "meow");
sayings.set("elephant", "toot");
sayings.size; // 3
sayings.get("fox"); // undefined
sayings.has("bird"); // false
sayings.delete("dog");
sayings.has("dog"); // false

for (var [key, value] of sayings) {
  console.log(key + " goes " + value);
}
// "cat goes meow"
// "elephant goes toot"

sayings.clear();
sayings.size; // 0
```

<code>Object</code> e <code>Map</code> comparados

Tradicionalmente, objetos tem sido usado para mapear strings para valores. Objetos permitem que você defina chaves para valores, recupere esses valores, exclua chaves e detecte se algo está armazenado em uma chave. Objetos `Map`, contudo, possuem algumas vantagens que os tornam mapas melhores.

- As chaves de um `Objeto` são `Strings`, onde elas podem ser de qualquer valor para um `Map`.
- Você pode obter o tamanho de um `Map` facilmente enquanto que para um `Object`, você tem que obter manualmente o seu tamanho.
- A iteração de mapas é por ordem de inserção dos elementos.
- Um `Object` tem um protótipo, então existem chaves padrão no mapa. (este pode ser ignorado usando `map = Object.create(null)`).

Estes dois tipos podem ajudar você a decidir se usa um `Map` ou um `Object`:

- Use mapas sobre objetos quando as chaves forem desconhecidas até a execução, e quando todas as chaves são do mesmo tipo e todos valores são do mesmo tipo.
- Use mapas caso haja a necessidade de armazenar valores primitivos como chaves, porque objetos tratam cada chave como uma string ou um valor numérico, valor booleano ou qualquer outro valor primitivo.
- Use objetos quando há uma lógica que opera em elementos individuais.

<code>Objeto WeakMap</code>

O objeto `WeakMap` é uma coleção de pares de chave/valor em que as **chaves são somente objetos** e seus valores podem ser valores arbitrários. As referências do objeto nas chaves são *fracamente* mantidas, isso significa que eles são alvo da garbage collection (coleta de lixo) se não houver nenhuma outra referência para o objeto. A API `WeakMap` é a mesma API do `Map`.

Uma diferença para objetos `Map` é que chaves `WeakMap` não são enumeráveis (isto é, não há um método que dê a você uma lista de chaves). Se eles fossem, a lista dependeria do estado da coleta de lixo, introduzindo um não-determinismo.

Para mais informações e código de exemplo, veja também "Por quê WeakMap?" na página de referência `WeakMap`.

Um caso de uso de objetos `WeakMap` é armazenar dados privados para um objeto ou ocultar detalhes de implementação. O exemplo a seguir é Nick Fitzgerald a partir de um post "Ocultando detalhes de implementação com WeakMaps ECMAScript 6" em seu blog. Os dados privados e métodos pertencem ao objeto e são armazenados nos objetos WeakMap `privados`. Tudo exposto na instância e o protótipo é público, todo o restante é inacessível a partir do mundo externo por que `privado` não é exportado pelo módulo.

```
const privates = new WeakMap();

function Public() {
  const me = {
    // Dados privados vem aqui
  };
  privates.set(this, me);
}

Public.prototype.method = function () {
  const me = privates.get(this);
  // Faça coisas com dado privado em `me`...
};

module.exports = Public;
```

Conjuntos

<code>Objeto Set</code>

Objetos `Set` são coleções de valores. Você pode iterar seus elementos em ordem de inserção. Um valor em um `Set` só pode ocorrer uma vez; ele é único em uma coleção `Set`.

O código a seguir mostra algumas operações básicas com um `Set`. Veja também a página de referência `Set` para mais exemplos e a API completa.

```
var mySet = new Set();
mySet.add(1);
mySet.add("some text");
mySet.add("foo");

mySet.has(1); // true
mySet.delete("foo");
mySet.size; // 2

for (let item of mySet) console.log(item);
// 1
// "some text"
```

Conversão entre Array e Set

Você pode criar um `Array` a partir de um Set usando `Array.from` ou o operador de propagação. Além disso, o construtor `Set` aceita um `Array` para converter em outra direção. Observe novamente que objetos `Set` armazenam valores únicos, então quaisquer elementos duplicados de um Array são excluídos quando convertidos.

```
Array.from(mySet);
[...mySet2];

mySet2 = new Set([1, 2, 3, 4]);
```

<code>Array</code> e <code>Set</code> comparados

Tradicionalmente, um conjunto de elementos tem sido armazenados em arrays JavaScript em muitas situações. O novo objeto `Set`, contudo, tem algumas vantagens:

- Checar se um elemento existe em uma coleção usando `indexOf` para arrays é lento.
- Objetos `Set` permitem que você exclua elementos por seu valor. Com um array você teria que unir baseado no índice do elemento.
- O valor `NaN` não pode ser encontrado com `indexOf` no array.
- Objetos `Set` armazenam valores únicos, você não tem que manter o controle de duplicidades manualmente por você mesmo.

<code>Objeto WeakSet</code>

Objetos `WeakSet` são coleções de objetos. Um objeto no `WeakSet` só pode ocorrer uma vez; Isto é único em coleções `WeakSet` e objetos não são enumeráveis.

As principais diferenças para objetos `Set` são:

- Em contraste com `Sets`, `WeakSets` são **apenas coleções de objetos** e não de valores arbitrários de qualquer tipo.
- O `WeakSet` é *fraco*: Referências para objetos são fracamente realizadas na coleção. Se não há outra referência para um objeto armazenado no `WeakSet`, eles podem ser coletados pelo coletor de lixo. Isso significa também que não há uma lista de objetos correntes armazenados na coleção. `WeakSets` não são enumeráveis.

Os casos de uso do objeto `WeakSet` são limitados. Eles não vão desperdiçar memória para ser seguro o uso de elementos DOM como chaves e marcá-los para propósitos de controle, por exemplo.

Igualdade de chave e valor do <code>Map</code> and <code>Set</code>

Ambos, a igualdade de chaves de objetos `Map` e a igualdade de valor de objetos `Set`, são baseados no "algoritmo mesmo valor de zero":

- Igualdade funciona como o operador de comparação de identidade `===`.
- `-0` e `+0` são considerados iguais.
- `NaN` é considerado igual a ela mesmo (contrário de `===`).

- Anterior
- Próxima

Uma reintrodução ao JavaScript (Tutorial de JS)

Introdução

Por que uma reintrodução? Porque JavaScript é conhecida como a mais incompreendida linguagem de programação do mundo. Embora muitas vezes ridicularizada como um brinquedo, por baixo de sua simplicidade enganosa estão alguns recursos poderosos da linguagem, que agora é usado por um número incrível de aplicações de alto nível, mostrando que o conhecimento mais profundo desta tecnologia é uma habilidade importante para qualquer desenvolvedor web, mobile ou desktop.

É sempre bom começar com a história da linguagem. A JavaScript foi criada em 1995 por Brendan Eich, um engenheiro da Netscape, e lançada pela primeira vez com o Netscape 2 no início de 1996. Foi inicialmente chamada de LiveScript, mas logo foi rebatizada, em uma decisão de marketing malfeita, para tentar crescer sobre a popularidade da linguagem Java da Sun Microsystem - apesar das duas terem muito pouco em comum. Esta tem sido uma fonte de confusão desde então.

A Microsoft lançou uma versão compatível com a maior parte da linguagem, chamada de JScript, junto com o IE, três meses mais tarde. A Netscape apresentou a linguagem a Ecma International, uma organização européia de normalização, o que resultou na primeira edição do padrão ECMAScript em 1997. O padrão recebeu uma atualização significativa com o ECMAScript Edição 3 em 1999, e manteve-se praticamente estável desde então. A quarta edição foi abandonada, devido a diferenças políticas relativas à complexidade da linguagem. Muitas partes da quarta edição formam a base da nova edição ECMAScript 5, publicado em dezembro de 2009.

Esta estabilidade foi uma grande notícia para os desenvolvedores, pois isto proporcionou que várias implementações em JavaScript tivessem muito tempo para se firmar. Eu vou focar quase exclusivamente no dialeto da edição 3. Para que seja facil se familiarizar, vou utilizar o termo JavaScript por todo o texto.

Diferentemente da maioria das linguagens de programação , a linguagem JavaScript não possui o conceito de entrada e saída. Ela é projetada para funcionar como uma linguagem de script em um ambiente de terceiros, e cabe ao ambiente fornecer mecanismos para a comunicação com o mundo exterior. O ambiente de terceiros (hospedeiro) mais comum é o navegador, mas interpretadores JavaScript também pode ser encontrados no Adobe Acrobat, Photoshop, imagens SVG, Widget engine do Yahoo! , bem como ambientes de servidor, como Node.js. No entanto, a lista aqui apresentada das áreas nas quais a JavaScript é utilizada é apenas o começo. Ela também inclui bancos de dados NoSQL, como o código-fonte aberto Apache CouchDB, computadores embarcados, ou ambientes de trabalho completos, como o GNOME (um dos GUIs mais populares para os sistemas operacionais GNU / Linux) .

Visão Geral

A JavaScript é uma linguagem dinâmica orientada a objetos; tem tipos e operadores, objetos e métodos. Sua sintaxe vem das linguagens Java e C, por isso tantas estruturas dessas linguagens se aplicam a JavaScript também. Uma das principais diferenças é que o JavaScript não tem classes; em vez disso, a funcionalidade de classe é realizada por protótipos de objetos. A outra diferença principal é que as funções são objetos, dando as funções a capacidade para armazenar código executável e serem passadas como parametro para qualquer outro objeto.

Vamos começar pelo bloco de construção de qualquer linguagem: os tipos. Programas JavaScript manipulam valores, e esses valores todos pertencem a um tipo. Tipos de JavaScript são :

- números
- strings
- booleanos
- funções
- objetos

... Ops, e o "indefinido" e o "nulo"- , que parecem um pouco estranhos. E arrays (vetores), que são um tipo especial de objeto. E as datas e expressões regulares, que são objetos que você ganha de graça. E para ser tecnicamente preciso, as funções são apenas um tipo especial de objeto. Assim, a lista de tipos se parece mais com isto:

- números (numbers)
- strings (strings)
- booleanos (booleans)
- objetos (objects)
    - funções (functions)
    - vetores (arrays)
    - datas (dates)
    - expressoes regulares (regexp)
- nulo (null)
- indefinido (undefined)

E existem também alguns tipos para erros. As coisas são muito mais fáceis se ficarmos com a primeira lista, no entanto.

Números

Números em JavaScript são "valores de precisão dupla no formato IEEE 754", de acordo com a especificação. Isto tem algumas consequências interessantes. Não existe essa coisa de inteiro em JavaScript, então você deve ser um pouco cuidadoso com seus cálculos se você está acostumado a matemática em C ou Java. Cuidado com coisas como:

```
0.1 + 0.2 == 0.30000000000000004
```

Na prática, valores inteiros são tratados como inteiros de 32 bits (e são armazenados dessa forma em algumas implementações do navegador), que podem ser importantes para as operações bit a bit. Para mais detalhes, consulte The Complete JavaScript Number Reference.

Os operadores numéricos padrões são suportados, incluindo adição, subtração, módulo (ou resto) aritmético e assim por diante. Há também um objeto embutido que eu esqueci de mencionar mais cedo chamado Math para manipular funções e constantes matemáticas mais avançadas:

```
Math.sin(3.5);
var d = Math.PI * r * r;
```

Você pode converter uma string em um inteiro usando a função embutida `parseInt()`. Ela tem um segundo parâmetro opcional para a base da conversão, parâmetro esse que você deveria sempre prover:

```
> parseInt("123", 10)
123
```

Se você quiser converter um número binário em um inteiro, basta mudar a base:

```
> parseInt("11", 2)
3
```

Similarmente, você pode fazer a conversão de números de ponto flutuante usando a função embutida `parseFloat()` que usa a base 10 sempre, ao contrário de seu primo `parseInt()`.

Você também pode usar o operador unário + para converter valores em números:

```
> + "42"
42
```

Um valor especial chamado `NaN` (sigla de "Not a Number ou Não é Número") é retornado se a string não é um valor numérico:

```
> parseInt("hello", 10)
NaN
```

`NaN` é tóxico: Se você provê-lo como uma entrada para qualquer operação matemática o resultado também será `NaN`:

```
> NaN + 5
NaN
```

Você pode testar se é `NaN` usando a função embutida `isNaN()`:

```
> isNaN(NaN)
true
```

JavaScript também tem os valores especiais `Infinity` e `-Infinity`:

```
> 1 / 0
Infinity
> -1 / 0
-Infinity
```

Você pode testar se o valor é `Infinity`, `-Infinity` e `NaN` usando a função embutida `isFinite()`:

```
> isFinite(1/0)
false
> isFinite(-Infinite)
false
> isFinite(NaN)
false
```

> **Nota:** As funções `parseInt()` e `parseFloat()` fazem a conversão da string até alcançarem um caracter que não é válido para o formato numérico especificado, então elas retornam o número convertido até aquele ponto. Contudo, o operador "+" simplesmente converte a string em `NaN` se tiver algum caracter inválido nela. Apenas tente por si mesmo converter a string "10.2abc" usando cada um desses métodos no console e entenderá melhor essas diferenças.

Strings

Strings em JavaScript são sequências de caracteres. Para ser mais exato, elas são sequências de Unicode characters, em que cada um deles é representado por um número de 16-bits. Isso deveria ser uma notícia bem-vinda para aqueles que tiveram que lidar com internacionalização.

Se você quiser representar um único caractere, você só tem que usar uma string de tamanho 1.

Para obter o tamanho de uma string, acesse sua propriedade `length`:

```
> "hello".length
5
```

Essa é nossa primeira pincelada com objetos JavaScript! Eu mencionei que strings também são objetos? De modo que elas têm métodos:

```
> "hello".charAt(0)
h
> "hello, world".replace("hello", "goodbye")
goodbye, world
> "hello".toUpperCase()
HELLO
```

Outros tipos

No JavaScript há distinção entre `null`, que é um objeto do tipo 'object' para indicar deliberadamente uma ausência de valor, do `undefined`, que é um objeto do tipo 'undefined' para indicar um valor não inicializado — isto é, que um valor não foi atribuído ainda. Vamos falar sobre variáveis depois, mas em JavaScript é possível declarar uma variável sem atribuir um valor para a mesma. Se você fizer isso, a variável será do tipo `undefined`.

JavaScript tem um tipo boolean, ao qual são possíveis os valores `true` e `false` (ambos são palavras-chave). Qualquer valor pode ser convertido em um boolean, de acordo com as seguintes regras:

1. `false`, `0`, a string vazia(`""`), `NaN`, `null`, e `undefined` todos tornam-se `false`
2. todos os outros valores tornam-se `true`

Você pode fazer essa conversão explicitamente usando a função `Boolean()`:

```
> Boolean("")
false
```

```
> Boolean(234)
true
```

Contudo, essa é uma necessidade rara, uma vez que JavaScript silenciosamente fará essa conversão quando for esperado um boolean, como em uma instrução `if`. Por isso, algumas vezes falamos simplesmente em "valores true" e "valores false" nos referindo a valores que se tornaram `true` e `false`, respectivamente, quando convertidos em boolean. Alternativamente, esses valores podem ser chamados de "truthy" (verdade/verdadeiro) e "falsy" (falso/falsidade), respectivamente.

Operações booleanas como `&&` (*and* lógico), `||` (*or* lógico), e `!` (*not* lógico) são suportadas.

Variáveis

Novas variáveis em JavaScript são declaradas usando a palavra-chave `var`:

```
var a;
var name = "simon";
```

Se você declarar uma variável sem atribuir qualquer valor a ela, seu tipo será `undefined`.

Uma diferença importante de outras linguagens como Java é que em JavaScript, blocos não tem escopo; somente funções tem escopo. De modo que se uma variável é definida usando `var` numa instrução composta (por exemplo dentro de uma estrutura de controle `if`), ela será visível por toda a função.

Obs: A definição de variáveis usando o let foi introduzida no ECMAScript 6. O let permite escopo de bloco, ou seja, é possível definir uma variável em um bloco `if`, e esta variável ter escopo limitado ao bloco `if`- por exemplo.

Operadores

Operadores numéricos de JavaScript são `+`, `-`, `*`, `/` e `%` - que é o operador resto. Valores são atribuídos usando `=`, e temos também as instruções de atribuição compostas, como `+=` e `-=`. Essas são o mesmo que `x = x operador y`.

```
x += 5;
x = x + 5;
```

Você pode usar `++` e `--` para incrementar ou decrementar respectivamente. Eles podem ser usados como operadores tanto antes como depois.

O operador `+` também faz concatenação de string:

```
> "hello" + " world"
hello world
```

Se você adicionar uma string a uma número (ou outro valor) tudo será convertido em uma string primeiro. Isso talvez seja uma pegadinha para você:

```
> "3" + 4 + 5
345
> 3 + 4 + "5"
75
```

Adicionar uma string em branco a algo é uma maneira melhor de fazer a conversão.

Comparações em JavaScript podem ser feitas usando `<`, `>`, `<=` e `>=`. Isso funciona tanto para strings como para números. A igualdade é um pouco menos simples. O operador igual-duplo faz a coersão de tipo se você colocar tipos diferentes, algumas vezes com resultados interessantes:

```
> "dog" == "dog"
true
> 1 == true
true
```

Para evitar a coersão de tipo, use o operador igual-triplo:

```
> 1 === true
false
> true === true
true
```

Temos também os operadores `!=` e `!==` .

JavaScript também tem operações de bit-a-bit. Se quiser usá-las, elas estarão lá.

Estruturas de Controle

JavaScript tem um conjunto de estruturas de controle similar à outras linguagens na família do C. Instruções condicionais são suportadas pelo `if` e `else`; você pode encadeá-los se quiser:

```
var name = "kittens";
if (name == "puppies") {
  name += "!";
} else if (name == "kittens") {
  name += "!!";
} else {
  name = "!" + name;
}
name == "kittens!!";
```

JavaScript tem as estruturas de repetição com os laços `while` e `do-while`. O primeiro é bom para repetições básicas; o segundo é para os casos em que você queira que o corpo da repetição seja executado pelo menos uma vez:

```
while (true) {
  // an infinite loop!
}

var input;
do {
  input = get_input();
} while (inputIsNotValid(input));
```

O laço `for` do JavaScript é o mesmo que no C e Java: ele lhe permite prover as informações para o seu laço em uma única linha.

```
for (var i = 0; i < 5; i++) {
  // Will execute 5 times
}
```

Os operadores `&&` e `||` usam a lógica de curto-circuito, o que quer dizer que, o segundo operando ser executado dependerá do primeiro operando. Isso é útil para checagem de objetos null antes de acessar seus atributos:

```
var name = o && o.getName();
```

Ou para configuração de valores-padrões:

```
var name = otherName || "default";
```

JavaScript tem um operador ternário para expressões condicionais:

```
var allowed = age > 18 ? "yes" : "no";
```

A instrução switch pode ser usada para múltiplas ramificações baseadas em um número ou uma string:

```
switch (action) {
  case "draw":
    drawit();
    break;
  case "eat":
    eatit();
    break;
  default:
    donothing();
}
```

Se você não adicionar a instrução `break`, a execução irá "cair" no próximo nível. Isso é algo que raramente vai querer fazer — de fato vale mais a pena colocar um comentário especificando essa "queda" para o próximo nível, pois isso o ajudará na hora de fazer a depuração de seu código:

```
switch (a) {
  case 1: // queda
  case 2:
    eatit();
    break;
  default:
    donothing();
}
```

A cláusula default é opcional. Se quiser, pode colocar expressões tanto no switch como nos cases; Comparações acontecem entre os dois usando o operador `===`:

```
switch (1 + 3) {
  case 2 + 2:
    yay();
    break;
  default:
    neverhappens();
}
```

Objetos

Objetos JavaScript são simplesmente coleções de pares nome-valor. Como tal, eles são similares à:

- Dicionários em Python
- Hashes em Perl e Ruby
- Hash tables em C e C++
- HashMaps em Java
- Vetores associativos em PHP

Esse tipo de estrutura de dados é largamente utilizada, o que atesta sua versatilidade. Uma vez que tudo em JavaScript é um objeto (tipos básicos principais), qualquer programa JavaScript naturalmente envolve uma grande quantidade de buscas em tabelas hash, o que é algo bom, pois elas são bem rápidas!

A parte "name" é uma string JavaScript, enquanto o valor pode ser qualquer valor JavaScript — incluindo mais objetos. Isso permite que você construa estruturas de dados de qualquer complexidade.

Há basicamente duas formas de criar um objeto vazio:

```
var obj = new Object();
```

e:

```
var obj = {};
```

Elas são semanticamente equivalentes; a segunda forma é chamada de sintaxe de objeto literal e é mais conveniente. Essa sintaxe é também o coração do formato JSON e deveria ser sempre preferida.

Uma vez criada, as propriedades de um objeto podem novamente ser acessadas de uma das seguintes formas:

```
obj.name = "Simon";
var name = obj.name;
```

E...

```
obj["name"] = "Simon";
var name = obj["name"];
```

Estas também são semânticamente equivalentes. A segunda forma tem a vantagem de que o valor da chave é passado através de uma string, que pode ser calculada em tempo de execução, muito embora esse método previna o uso de alguns mecanismos tais como a otimização e a minificação. Outra vantagem é a possibilidade de se atribuir palavras-reservadas aos nomes das propriedades:

```
obj.for = "Simon"; // Erro de sintaxe, pois 'for' é uma palavra reservada
obj["for"] = "Simon"; // Funciona bem
```

A sintaxe de objeto literal pode ser usada para inicializar completamente um objeto:

```
var obj = {
  name: "Carrot",
  for: "Max",
  details: {
    color: "orange",
    size: 12,
  },
};
```

O acesso aos atributos podem ser encadeados:

```
> obj.details.color
orange
> obj["details"]["size"]
12
```

Vetores

Vetores em JavaScript são, de fato, um tipo especial de objeto. Eles funcionam de forma muito similar à objetos regulares (propriedades numéricas podem naturalmente ser acessadas somente usando a sintaxe [], colchetes), porém eles tem uma propriedade mágica chamada '`length`'. Ela sempre é o maior índice de um vetor mais 1.

A forma tradicional de se criar um vetor em JavaScript é a seguinte:

```
> var a = new Array();
> a[0] = "dog";
> a[1] = "cat";
> a[2] = "hen";
> a.length
3
```

Existe uma notação mais conveniente usando um vetor literal:

```
> var a = ["dog", "cat", "hen"];
> a.length
3
```

Deixar uma vírgula à direita no final de um vetor literal gerará inconsistência entre os navegadores, portanto não faça isso.

Note que `array.length` não é necessariamente o número de itens em um vetor. Considere o seguinte:

```
> var a = ["dog", "cat", "hen"];
> a[100] = "fox";
> a.length
101
```

Lembre-se — o tamanho de um vetor é o maior índice mais 1.

Se você fizer referência a um índice de vetor inexistente, obterá um `undefined`:

```
> typeof a[90]
undefined
```

Você pode iterar sobre um vetor da seguinte forma:

```
for (var i = 0; i < a.length; i++) {
  // Faça algo com a[i]
}
```

Isso é um pouco ineficaz visto que você está procurando a propriedade length uma vez a cada iteração. Uma melhoria poderia ser:

```
for (var i = 0, len = a.length; i < len; i++) {
  // Faça algo com a[i]
}
```

Uma forma mais elegante ainda poderia ser:

```
for (var i = 0, item; (item = a[i++]);) {
  // Faça algo com item
}
```

Aqui nós estamos declarando duas variáveis. A atribuição na parte do meio do laço `for` é também testada — se for verdadeira, o laço continuará. Uma vez que o `i` é incrementado toda vez, os itens do array serão atribuídos a variável item sequencialmente. A iteração é finalizada quando item "falsy" é encontrado (tal como o `undefined`, false ou zero).

Note que esse truque só deveria ser usado em vetores que você sabe não conter valores "falsy" (vetores de objeto ou nós DOM por exemplo). Se você iterar sobre dados numéricos que possam ter o 0 ou sobre dados string que possam ter uma string vazia, você deveria usar a segunda forma como alternativa.

Uma outra forma de iterar é usar o laço `for...in`. Note que se alguém adicionou novas propriedades ao `Array.prototype`, elas também podem ser iteradas usando este laço:

```
for (var i in a) {
  // Do something with a[i]
}
```

Se quiser adicionar um item a um vetor, simplesmente faça desse jeito:

```
a[a.length] = item; // é o mesmo que a.push(item);
```

Vetores vem com vários métodos:

Nome do método	Descrição
`a.toString()`	Retorna uma string com o toString()de cada elemento separado por vírgulas.
`a.toLocaleString()`	Retorna uma string com o toLocaleString()de cada elemento separado por vírgulas.
`a.concat(item[, itemN])`	Retorna um novo vetor com os itens adicionados nele.
`a.join(sep)`	Converte um vetor em uma string com os valores do vetor separados pelo valor do parâmetro sep
`a.pop()`	Remove e retorna o último item.
`a.push(item[, itemN])`	`Push` adiciona um ou mais itens ao final.
`a.reverse()`	Reverte o vetor.
`a.shift()`	Remove e retorna o primeiro item.
`a.slice(start, end)`	Retorna um sub-vetor.
`a.sort([cmpfn])`	Prover uma função opcional para fazer a comparação.
`a.splice(start, delcount[, itemN])`	Permite que você modifique um vetor por apagar uma seção e substituí-lo com mais itens.
`a.unshift([item])`	Acrescenta itens ao começo do vetor.

Funções

Junto com objetos, funções são os componentes principais para o entendimento do JavaScript. A função mais básica não poderia ser mais simples:

```
function add(x, y) {
  var total = x + y;
  return total;
}
```

Isso demonstra tudo o que há para se saber sobre funções básicas. Uma função JavaScript pode ter 0 ou mais parâmetros declarados. O corpo da função pode conter tantas instruções quantas quiser e pode declarar suas próprias variáveis que são de escopo local àquela função. A instrução `return` pode ser usada para retornar um valor em qualquer parte da função, finalizando a função. Se nenhuma instrução de retorno for usada (ou um retorno vazio sem valor), o JavaScript retorna `undefined`.

Os parâmetros nomeados se parecem mais com orientações do que com outra coisa. Você pode chamar a função sem passar o parâmetro esperado, nesse caso eles receberão o valor `undefined`.

```
> add()
NaN // Você não pode executar adição em undefined
```

Você também pode passar mais argumentos do que a função está esperando:

```
> add(2, 3, 4)
5 // adicionado os dois primeiros; 4 foi ignorado
```

Pode parecer um pouco bobo, mas no corpo da função você tem acesso a uma variável adicional chamada `arguments`, que é um objeto parecido com um vetor que contém todos os valores passados para a função. Vamos rescrever a função add para tomarmos tantos valores quanto quisermos:

```
function add() {
    var sum = 0;
    for (var i = 0, j = arguments.length; i < j; i++) {
        sum += arguments[i];
    }
    return sum;
}

> add(2, 3, 4, 5)
14
```

Isso realmente não é muito mais útil do que escrever `2 + 3 + 4 + 5`. Vamos criar uma função para calcular média:

```
function avg() {
    var sum = 0;
    for (var i = 0, j = arguments.length; i < j; i++) {
        sum += arguments[i];
    }
    return sum / arguments.length;
}
> avg(2, 3, 4, 5)
3.5
```

Isso é muito útil, mas introduz um novo problema. A função `avg()` precisa de uma lista de argumentos separados por vírgula — mas e se o que quiser for procurar a média de um vetor? Você poderia apenas rescrever a função da seguinte forma:

```
function avgArray(arr) {
    var sum = 0;
    for (var i = 0, j = arr.length; i < j; i++) {
        sum += arr[i];
    }
    return sum / arr.length;
}
> avgArray([2, 3, 4, 5])
3.5
```

Porém, seria legal se pudéssemos reusar a função que já tínhamos criado. Felizmente, JavaScript lhe permite chamar a função, e chamá-la com um conjunto arbitrário de argumentos, usando o método `apply()` presente em qualquer objeto função.

```
> avg.apply(null, [2, 3, 4, 5])
3.5
```

O segundo argumento do `apply()` é o vetor para usar como argumento; o primeiro será discutido mais tarde. Isso enfatiza o fato que funções também são objetos.

JavaScript lhe permite criar funções anônimas.

```
var avg = function () {
  var sum = 0;
  for (var i = 0, j = arguments.length; i < j; i++) {
    sum += arguments[i];
  }
  return sum / arguments.length;
};
```

Isso é semanticamente equivalente a forma `function avg()`. É extremamente poderoso como ele lhe permite colocar a definição completa de uma função em qualquer lugar, que você normalmente poria uma expressão. Isso lhe permite todo tipo de truques engenhosos. Aqui está uma maneira de "esconder" algumas variáveis locais — como escopo de bloco em C:

```
> var a = 1;
> var b = 2;
> (function() {
    var b = 3;
    a += b;
})();
> a
4
> b
2
```

JavaScript lhe permite chamar funções recursivamente. Isso é particularmente útil quando estamos lidando com estruturas de árvore, como quando estavamos navegando no DOM.

```
function countChars(elm) {
  if (elm.nodeType == 3) {
    // TEXT_NODE
    return elm.nodeValue.length;
  }
  var count = 0;
  for (var i = 0, child; (child = elm.childNodes[i]); i++) {
    count += countChars(child);
  }
  return count;
}
```

Isso destaca um problema potencial com funções anônimas: Como chamá-las recursivamente se elas não tem um nome? JavaScript lhe permite nomear expressões de função para isso. Você pode usar EFIIs nomeadas (Expressões Funcionais Imediatamente Invocadas), conforme abaixo:

```
var charsInBody = (function counter(elm) {
  if (elm.nodeType == 3) {
    // TEXT_NODE
    return elm.nodeValue.length;
  }
  var count = 0;
  for (var i = 0, child; (child = elm.childNodes[i]); i++) {
    count += counter(child);
  }
  return count;
})(document.body);
```

O nome provido para a função anônima conforme acima só é (ou no mínimo só deveria ser) visível ao escopo da própria função. Isso tanto permite que mais otimizações sejam feitas pela engine como deixa o código mais legível.

Objetos Personalizados

> **Nota:** Para uma discursão mais detalhada de programação orientada a objetos em JavaScript, veja Introdução a JavaScript Orientado a Objeto.

Na clássica Programação Orientada a Objetos, objetos são coleções de dados e métodos que operam sobre esses dados. JavaScript é uma linguagem baseada em protótipos que não contém a estrutura de classe, como tem em C++ e Java. (Algumas vezes isso é algo confuso para o programador acostumado a linguagens com estrutura de classe). Em vez disso, JavaScript usa funções como classes. Vamos considerar um objeto pessoa com os campos primeiro e último nome. Há duas formas em que o nome talvez possa ser exibido: como "primeiro nome segundo nome" ou como "último nome, primeiro nome". Usando as funções e objetos que discutimos anteriormente, aqui está uma forma de fazer isso:

```
function makePerson(first, last) {
    return {
        first: first,
        last: last
    }
}
function personFullName(person) {
    return person.first + ' ' + person.last;
}
function personFullNameReversed(person) {
    return person.last + ', ' + person.first
}
> s = makePerson("Simon", "Willison");
> personFullName(s)
Simon Willison
> personFullNameReversed(s)
Willison, Simon
```

Isso funciona, mas é muito feio. Você termina com dúzias de funções em seu escopo global. O que nós realmente precisamos é uma forma de anexar uma função a um objeto. Visto que funções são objetos, isso é fácil:

```
function makePerson(first, last) {
    return {
        first: first,
        last: last,
        fullName: function() {
            return this.first + ' ' + this.last;
        },
        fullNameReversed: function() {
            return this.last + ', ' + this.first;
        }
    }
}
> s = makePerson("Simon", "Willison")
> s.fullName()
Simon Willison
> s.fullNameReversed()
Willison, Simon
```

Há algo aqui que não havíamos visto anteriormente: a palavra-chave '`this`'. Usada dentro de uma função, '`this`' refere-se ao objeto corrente. O que aquilo de fato significa é especificado pelo modo em que você chamou aquela função. Se você chamou-a usando notação ponto ou notação colchete em um objeto, aquele objeto torna-se '`this`'. Se a notação ponto não foi usada pela chamada, '`this`' refere-se ao objeto global. Isso é uma frequente causa de erros. Por exemplo:

```
> s = makePerson("Simon", "Willison")
> var fullName = s.fullName;
> fullName()
undefined undefined
```

Quando chamamos `fullName()`, '`this`' está ligado ao objeto global. Visto que não há variáveis globais chamadas `first` ou `last` obtemos `undefined` para cada um.

Podemos tirar vantagem da palavra chave '`this`' para melhorar nossa função `makePerson`:

```
function Person(first, last) {
  this.first = first;
  this.last = last;
  this.fullName = function () {
    return this.first + " " + this.last;
  };
  this.fullNameReversed = function () {
    return this.last + ", " + this.first;
  };
}
var s = new Person("Simon", "Willison");
```

Nós introduzimos uma outra palavra-chave: '`new`'. `new` é fortemente relacionada a '`this`'. O que ele faz é criar um novo objeto vazio, e então chamar a função especificada com '`this`' para atribuir aquele novo objeto. Funções que são desenhadas para ser chamadas pelo '`new`' são chamadas de funções construtoras. Uma prática comum é capitular essas funções como um lembrete de chamá-las com o `new`.

Nossos objetos pessoa estão ficando melhor mas ainda existem algumas arestas feias. Toda vez que criamos um objeto pessoa, criamos duas marcas de nova função dentro dele — não seria melhor se este código fosse compartilhado?

```
function personFullName() {
  return this.first + " " + this.last;
}
function personFullNameReversed() {
  return this.last + ", " + this.first;
}
function Person(first, last) {
  this.first = first;
  this.last = last;
  this.fullName = personFullName;
  this.fullNameReversed = personFullNameReversed;
}
```

Assim está melhor: estamos criando as funções de método apenas uma vez, e atribuimos referências para elas dentro do construtor. Podemos fazer algo melhor do que isso? A resposta é sim:

```
function Person(first, last) {
  this.first = first;
  this.last = last;
}
Person.prototype.fullName = function () {
  return this.first + " " + this.last;
};
Person.prototype.fullNameReversed = function () {
  return this.last + ", " + this.first;
};
```

`Person.prototype` é um objeto compartilhado por todas as instâncias de `Person`. Este forma parte da cadeia de buscas (que tem um nome especial, cadeia de protótipos ou "prototype chain"): toda a vez que você tentar acessar uma propriedade de `Person` que não está configurada, Javascript irá verificar em `Person.prototype` para ver se esta propriedade existe por lá. Como resultado, qualquer coisa atribuída à `Person.prototype` torna-se disponível para todas as instâncias deste construtor, através do objeto `this`.

Esta é uma ferramenta incrivelmente poderosa. JavaScript permite a você modificar algo prototipado em qualquer momento no seu programa. Isto significa que você pode adicionar métodos extras para objetos pré-existentes, em tempo de execução:

```
> s = new Person("Simon", "Willison");
> s.firstNameCaps();
TypeError on line 1: s.firstNameCaps is not a function
> Person.prototype.firstNameCaps = function() {
    return this.first.toUpperCase()
}
> s.firstNameCaps()
SIMON
```

Curiosamente, você pode também adicionar coisas para o protótipo de objetos built-in de Javascript. Vamos adicionar um método para `String` que retorna a string invertida:

```
> var s = "Simon";
> s.reversed()
```

```
TypeError on line 1: s.reversed is not a function
> String.prototype.reversed = function() {
    let start = 0, end = this.length - 1
    result = new Array(this.length)
    while(start <= end) {
      result[start] = this[end]
      result[end] = this[start]
      start++, end--
    }
    return result.join('')
}
> s.reversed()
nomiS
```

Nosso novo método funciona inclusive em string literais!

```
> "This can now be reversed".reversed()
desrever eb won nac sihT
```

Como antes mencionado, o protótipo forma parte de uma cadeia. A raiz dessa cadeia é `Object.prototype`, dos quais inclui o método `toString()` — este é o método que é chamado quando você tenta representar um objeto como uma string. Isto é útil para depurar os nossos objetos `Person`:

```
> var s = new Person("Simon", "Willison");
> s
[object Object]
> Person.prototype.toString = function() {
    return '<Person: ' + this.fullName() + '>';
}
> s
<Person: Simon Willison>
```

Lembra como `avg.apply()` tinha um primeiro argumento null? Nós podemos revisitar isto, agora. O primeiro argumento para `apply()` é o objeto que deve ser tratado como '`this`'. Por exemplo, aqui está uma implementação trivial de '`new`':

```
function trivialNew(constructor, ...args) {
  var o = {}; // Create an object
  constructor.apply(o, args);
  return o;
}
```

Isto não é exatamente uma réplica de `new` porque não configura a cadeia de protótipos. `apply()` é difícil de ilustrar — não é algo que você usa com frequência, mas é útil conhecer a respeito. No código acima, `...args` é chamado de "argumentos rest" — como o nome indica, isto contém o restante dos parâmetros.

Ao chamar

```
var bill = trivialNew(Person, "Willian", "Orange");
```

é equivalente a

```
var bill = new Person("Willian", "Orange");
```

`apply()` tem uma função irmã de nome `call`, que novamente permite você configurar o '`this`' mas toma uma lista expandida de argumentos, ao invés de um array.

```
function lastNameCaps() {
  return this.last.toUpperCase();
}
var s = new Person("Simon", "Willison");
lastNameCaps.call(s);
// Is the same as:
s.lastNameCaps = lastNameCaps;
s.lastNameCaps();
```

Funções Internas

Em JavaScript é permitido declarar uma função dentro de outras funções. Nós já vimos isso antes, com uma versão preliminar da função `makePerson()`. Um detalhe importante sobre funções aninhadas em JavaScript é que elas podem acessar as variáveis do escopo das funções parente:

```
function parentFunc() {
  var a = 1;

  function nestedFunc() {
    var b = 4; // parentFunc can't use this
    return a + b;
  }
  return nestedFunc(); // 5
}
```

Isto permite um compromisso maior de utilidade ao escrever um código de melhor mantenibilidade. Se uma função depende de uma ou mais funções que não são úteis para outras parte do seu código, você pode aninhar estas funções utilitárias dentro da função que será chamada. Isto mantém o número de funções que estão no escopo global baixo, o que é sempre uma boa coisa.

Isto é também um ótimo contador de atração de variáveis globais. Quando se escreve um código complexo, é sempre tentador usar as variáveis globais para compartilhar valores entre múltiplas funções — do qual leva a um código que é difícil de manter. Funções aninhadas podem compartilhar variáveis em seus parentes, então você pode usar este mecanismo para acoplar e juntar funções, quando isto fizer sentido, sem poluir o seu "namespace" global — 'globais locais' se preferir. Esta técnica deve ser usada com cautela, mas é uma habilidade a se ter.

Clausuras (Closures)

Isto nos leva a uma das abstrações mais poderosas que JavaScript tem a oferecer — mas também a mais potencionalmente confusa. O que isto faz?

```
function makeAdder(a) {
  return function (b) {
    return a + b;
  };
}
var x = makeAdder(5);
var y = makeAdder(20);
x(6); // ?
y(7); // ?
```

O nome da função `makeAdder` já diz tudo: ela cria novas funções 'adder', na qual, quando chamada com um argumento, adiciona o argumento com a que foi criada.

O que está acontecendo aqui é muito parecido com o que estava acontencedo com as funções internas vistas anterioremente: uma função definida dentro de uma outra função tem acessso às variáveis da função de fora. A única diferença aqui é que a função de fora retornou e, como consequência do senso comum, deve dizer que todas as variáveis locais não existem mais. Mas elas *ainda* existem — caso contrário a função adicionadora não seria capaz de funcionar. Mais ainda, há duas "cópias" diferentes de variáveis locais para `makeAdder` — uma na qual o `a` é 5 e a outra na qual `a` é 20. Então, o resultado dessas chamadas de funções é o seguinte:

```
x(6); // returns 11
y(7); // returns 27
```

Eis o que acontece na verdade: sempre que o JavaScript executa uma função, um objeto de 'escopo' é criado para guardar as variáveis locais criadas dentro desta função. Ela é inicializada com quaisquer variáveis passadas como parâmetros da função. Isto é similar ao objeto global, em que todas as variáveis globais e funções vivem, mas com algumas diferenças importantes: primeiro, um novo objeto de escopo é criado toda a vez que uma função começa a executar, e segundo, diferente do objeto global (que nos navegadores é acessado com `window`) estes objetos não podem ser diretamente acessados através do seu código JavaScript. Não há nenhum mecanismo para iterar sobre as propriedades do escopo corrente do objeto, por exemplo.

Então, quando `makeAdder` é chamado, um objeto de escopo é criado com uma única propriedade: `a`, a qual é o argumento passado para a função `makeAdder`. `makeAdder` então retorna uma nova função criada. Normalmente o coletor de lixo de JavaScript poderia limpar o objeto de escopo criado para `makeAdder` neste ponto, mas a função de retorno mantém uma referência ao objeto de escopo. Como resultado, o objeto de escopo não será coletado como lixo até que não haja mais referências para função objeto que `makeAdder` retornou.

Objetos de escopo formam uma cadeia chamada de cadeia de escopos, similar a cadeia de protótipos usadas no sistema de objetos de JavaScript.

Uma clausura é a combinação de uma função e o objeto de escopo na qual é criado. Clausuras permitem você guardar estado — de tal forma, elas podem ser frequentemente utilizadas no lugar de objetos.

Laços e iterações

- Anterior
- Próxima

Laços oferecem um jeito fácil e rápido de executar uma ação repetidas vezes. Este capítulo do guia do JavaScript abordará diferentes formas de iterações existentes no JavaScript.

Você pode pensar em um laço de repetição como um jogo onde você manda o seu personagem andar X passos em uma direção e Y passos em outra; por exemplo, a ideia "vá 5 passos para leste" pode ser expressa em um laço desta forma:

```
var passo;
for (passo = 0; passo < 5; passo++) {
  // Executa 5 vezes, com os valores de passos de 0 a 4.
  console.log("Ande um passo para o leste");
}
```

Existem várias formas diferentes de laços, mas eles essencialmente fazem a mesma coisa: repetir uma ação múltiplas vezes (inclusive você poderá repetir 0 vezes). Os vários mecanismos diferentes de laços oferecem diferentes formas de determinar quando este irá começar ou terminar. Há várias situações em que é mais fácil resolver um problema utilizando um determinado tipo de laço do que outros.

Os possíveis laços de repetição em JavaScript:

- for_statement
- do...while_statement
- while_statement
- label_statement
- break_statement
- continue_statement
- for...in_statement
- for...of_statement

<code>Declaração for</code>

Um laço for é repetido até que a condição especificada seja falsa. O laço for no JavaScript é similar ao Java e C. Uma declaração for é feita da seguinte maneira:

```
for ([expressaoInicial]; [condicao]; [incremento])
  declaracao
```

Quando um for é executado, ocorre o seguinte:

1. A expressão `expressao Inicial` é inicializada e, caso possível, é executada. Normalmente essa expressão inicializa um ou mais contadores, mas a sintaxe permite expressões de qualquer grau de complexidade. Podendo conter também declaração de variáveis.
2. A expressão `condicao` é avaliada. caso o resultado de `condicao` seja verdadeiro, o laço é executado. Se o valor de `condicao` é falso, então o laço terminará. Se a expressão `condicao` é omitida, a `condicao` é assumida como verdadeira.
3. A instrução é executada. Para executar múltiplas declarações, use uma declaração em bloco ({ ... }) para agrupá-las.
4. A atualização da expressão `incremento`, se houver, executa, e retorna o controle para o passo 2.

Exemplo

A função a seguir contém uma declaração `for` que contará o número de opções selecionadas em uma lista (um elemento `<select>` permite várias seleções). Dentro do `for` é declarado uma váriavel `i` inicializada com zero. A declaração `for` verifica se `i` é menor que o número de opções no elemento `<select>`, executa sucessivas declaração `if`, e incrementa `i` de um em um a cada passagem pelo laço.

```
<form name="selectForm">
  <p>
    <label for="tipoMusica"
      >Escolha alguns tipos de música, em seguida, clique no botão
      abaixo:</label
    >
    <select id="tipoMusica" name="tipoMusica" multiple="multiple">
      <option selected="selected">R&B</option>
      <option>Jazz</option>
      <option>Blues</option>
      <option>New Age</option>
      <option>Classico</option>
      <option>Ópera</option>
    </select>
  </p>
  <p><input id="btn" type="button" value="Quantos foram selecionados?" /></p>
</form>

<script>
  function howMany(selectObject) {
    var numeroSelecionadas = 0;
    for (var i = 0; i < selectObject.options.length; i++) {
      if (selectObject.options[i].selected) {
        numeroSelecionadas++;
      }
    }
    return numeroSelecionadas;
  }

  var btn = document.getElementById("btn");
  btn.addEventListener("click", function () {
    alert(
      "Total de opções selecionadas: " +
        howMany(document.selectForm.tipoMusica),
    );
  });
</script>
```

<code>Declaração do...while</code>

A instrução do...while repetirá até que a condição especificada seja falsa.

```
do
  declaracao
while (condicao);
```

A instrução será executada uma vez antes da condição ser verificada. Para executar multiplas instruções utilize uma declaração de bloco ({ ... }) para agrupá-las. Caso a `condicao` seja verdadeira, então o laço será executado novamente. Ao final de cada execução, a `condicao` é verificada. Quando a condição contida no while for falsa a execução do laço é terminada e o controle é passado para a instrução seguinte a `do...while`.

Exemplo

No exemplo a seguir, o laço é executado pelo menos uma vez e irá executar até que `i` seja menor que 5.

```
do {
  i += 1;
  console.log(i);
} while (i < 5);
```

<code>Declaração while</code>

Uma declaração while executa suas instruções, desde que uma condição especificada seja avaliada como verdadeira. Segue uma declaração `while`:

```
while (condicao)
  declaracao
```

Se a condição se tornar falsa, a declaração dentro do laço para a execução e o controle é passado para a instrução após o laço.

O teste da condição ocorre antes que o laço seja executado. Desta forma se a condição for verdadeira o laço executará e testará a condição novamente. Se a condição for falsa o laço termina e passa o controle para as instruções após o laço.

Para executar múltiplas declarações, use uma declaração em bloco ({ ... }) para agrupar essas declarações.

Exemplo 1

O `while` a seguir executará enquanto `n` for menor que três:

```
n = 0;
x = 0;
while (n < 3) {
  n++;
  x += n;
}
```

A cada iteração, o laço incrementa `n` e adiciona este valor para `x`. Portanto, `x` e `n` recebem os seguintes valores:

- Depois de executar pela primeira vez: `n` = 1 e `x` = 1
- Depois da segunda vez: `n` = 2 e `x` = 3
- Depois da terceira vez: `n` = 3 e `x` = 6

Depois de executar pela terceira vez, a condição `n < 3` não será mais verdadeira, então o laço encerrará.

Exemplo 2

Evite laços infinitos. Tenha certeza que a condição do laço eventualmente será falsa; caso contrário, o laço nunca terminará. O while a seguir executará para sempre pois sua condição nunca será falsa:

```
while (true) {
  console.log("Olá, mundo");
}
```

<code>Declaração label</code>

Um label provê um identificador que permite que este seja referenciado em outro lugar no seu programa. Por exemplo, você pode usar uma label para identificar um laço, e então usar `break` ou `continue` para indicar quando o programa deverá interromper o laço ou continuar sua execução.

Segue a sintaxe da instrução label:

```
label : declaracao
```

Um label pode usar qualquer idenficador que não seja uma palavra reservada do JavaScript. Você pode identificar qualquer instrução com um label.

Exemplo

Neste exemplo, o label `markLoop` idenfica um laço `while`.

```
markLoop: while (theMark == true) {
  facaAlgo();
}
```

<code>Declaração break</code>

Use break para terminar laços, `switch`, ou um conjunto que utiliza label.

- Quando você utiliza `break` sem um label, ele encerrará imediatamente o laço mais interno `while`, `do-while`, `for`, ou `switch` e transferirá o controle para a próxima instrução.
- Quando você utiliza `break` com um label, ele encerrará o label específico.

Segue a sintaxe do break:

1. `break;`
2. `break label;`

Na primeira opção será encerrado o laço de repetição mais interno ou `switch`. Já na segunda opção será encerrada o bloco de código referente a label.

Exemplo 1

O exemplo a seguir percorre os elementos de um array até que ele encontre o índice do elemento que possui o valor contido em `theValue`:

```
for (i = 0; i < a.length; i++) {
  if (a[i] == theValue) {
    break;
  }
}
```

Exemplo 2: Utilizando break em label

```
var x = 0;
var z = 0;
labelCancelaLaco: while (true) {
  console.log("Laço exterior: " + x);
  x += 1;
  z = 1;
  while (true) {
    console.log("Laço interior: " + z);
    z += 1;
    if (z === 10 && x === 10) {
      break labelCancelaLaco;
    } else if (z === 10) {
      break;
    }
  }
}
```

<code>Declaração continue</code>

A declaração continue pode ser usada para reiniciar uma instrução `while`, `do-while`, `for`, ou `label`.

- Quando você utiliza `continue` sem uma label, ele encerrará a iteração atual mais interna de uma instrução `while`, `do-while`, ou `for` e continuará a execução do laço a partir da próxima iteração. Ao contrário da instrução `break`, `continue` não encerra a execução completa do laço. Em um laço `while`, ele voltará para a condição. Em um laço `for`, ele pulará para a expressão de incrementação.
- Quando você utiliza `continue` com uma label, o `continue` será aplicado ao laço identificado por esta label.

Segue a sintaxe do `continue`:

1. `continue;`
2. `continue`*`label;`*

Exemplo 1

O exemplo a seguir mostra um laço `while` utilizando `continue` que executará quando o valor de `i` for igual a 3. Desta forma, `n` recebe os valores um, três, sete, e doze.

```
i = 0;
n = 0;
while (i < 5) {
  i++;
  if (i == 3) {
    continue;
  }
  n += i;
}
```

Exemplo 2

Uma instrução label `checkiandj` contém uma instrução label `checkj`. Se o `continue` for executado, o programa terminará a iteração atual de `checkj` e começará a próxima iteração. Toda vez que o `continue` for executado, `checkj` recomeçará até que a condição do `while` for falsa. Quando isto ocorrer `checkiandj` executará até que sua condição seja falsa.

Se o `continue` estivesse referenciando `checkiandj`, o programa deveria continuar do topo de `checkiandj`.

```
checkiandj: while (i < 4) {
  console.log(i);
  i += 1;
  checkj: while (j > 4) {
    console.log(j);
    j -= 1;
    if (j % 2 == 0) {
      continue checkj;
    }
    console.log(j + " é estranho.");
  }
  console.log("i = " + i);
  console.log("j = " + j);
}
```

<code>Declaração for...in</code>

A declaração for...in executa iterações a partir de uma variável específica, percorrendo todas as propriedades de um objeto. Para cada propriedade distinta, o JavaScript executará uma iteração. Segue a sintaxe:

```
for (variavel in objeto) {
  declaracoes
}
```

Exemplo

A função a seguir recebe em seu argumento um objeto e o nome deste objeto. Então executará uma iteração para cada elemento e retornará uma lista de string, que irá conter o nome da propriedade e seu valor.

```
function dump_props(obj, obj_name) {
  var result = "";
  for (var i in obj) {
    result += obj_name + "." + i + " = " + obj[i] + "<br>";
  }
  result += "<hr>";
  return result;
}
```

Para um objeto chamado `car` com propriedades `make` e `model`, o resultado será:

```
car.make = Ford;
car.model = Mustang;
```

Arrays

Embora seja tentador usar esta forma para interagir com os elementos de um Array, a declaração **for...in** irá retornar o nome pré-definido da propriedade ao invés do seu index numérico. Assim é melhor usar o tradicional for com index numérico quando interagir com arrays, pois o **for...in** interage com as propriedades definidas pelo programador ao invés dos elementos do array.

Declaração <code>for...of</code>

A declaração for...of cria uma laço com objetos interativos ((incluindo, `Array`, `Map`, `Set`, assim por conseguinte), executando uma iteração para o valor de cada propriedade distinta.

```
for (variavel of objeto) {
  declaracoes
}
```

O exemplo a seguir mostra a diferença entre o `for...of` e o `for...in`. Enquanto o `for...in` interage com o nome das propriedades, o `for...of` interage com o valor das propriedades.

```
let arr = [3, 5, 7];
arr.foo = "hello";

for (let i in arr) {
  console.log(i); // logs "0", "1", "2", "foo"
}

for (let i of arr) {
  console.log(i); // logs "3", "5", "7"
}
```


- Anterior
- Próxima

Gerenciamento de Memória

Introdução

Linguagens de baixo nível, como C, tem primitivas de gerenciamento de memória de baixo nível como `malloc()` e `free()`. Em contrapartida, os valores do JavaScript são alocados quando coisas (objetos, strings, etc.) são criadas e "automaticamente" liberadas quando não são mais usadas. Este último processo se chama *garbage collection*. Facilmente se torna uma fonte de confusão e dá a impressão aos desenvolvedores JavaScript (e outras linguagens de alto nível) que eles não precisam se preocupar com o consumo de memória. Isto é um erro.

Ciclo de vida da memória

Independentemente da linguagem de programação, o ciclo de vida da memória é praticamente sempre o mesmo:

1. Alocar a memória que você precisa
2. Utilizar a memória alocada (ler, escrever)
3. Liberar a memória alocada quando não é mais necessária

A primeira e a segunda parte são explícitas em todas as linguagens. A última parte é explicita em linguagens de baixo nível, porém implícito em linguagens de alto nível como JavaScript.

Alocação no JavaScript

Inicialização de valor

A fim de não incomodar o programador com alocações, o JavaScript faz isso com os valores conforme são declarados.

```
var n = 123; // aloca memória para um número
var s = "azerty"; // aloca memória para uma string

var o = {
  a: 1,
  b: null,
```

```
}; // aloca memória para um objeto e seus valores contidos

// (assim como o objeto) aloca memória para o vetor e
// seus valores contidos
var a = [1, null, "abra"];

function f(a) {
  return a + 2;
} // aloca uma função (que é um objeto que pode ser chamado)

// expressões de funções também alocam um objeto
someElement.addEventListener(
  "click",
  function () {
    someElement.style.backgroundColor = "blue";
  },
  false,
);
```

Alocação via chamada de uma função

Algumas funções quando chamadas resultam na alocação de um objeto.

```
var d = new Date();
// aloca um elemento do DOM
var e = document.createElement("div");
```

Alguns métodos alocam novos valores ou objetos:

```
var s = "azerty";
var s2 = s.substr(0, 3); // s2 é uma nova string
// Como as strings são valores imutáveis,
// o JavaScript pode decidir não alocar memória,
// mas apenas armazenar o intervalo [0, 3].

var a = ["ouais ouais", "nan nan"];
var a2 = ["generation", "nan nan"];
var a3 = a.concat(a2);
// novo vetor com 4 elementos sendo
// a concatenação dos elementos a e a2
```

Utilização de valores

A utilização de valores basicamente significa leitura e escrita em memória alocada. Isto pode ser feito ao ler ou escrever o valor de uma variável ou a propriedade de um objeto ou até mesmo ao passar um argumento para uma função.

Libere quando a memória não for mais necessária

A maioria dos problemas relacionados ao gerenciamento de memória aparecem nesta fase. A tarefa mais complicada aqui é descobrir quando "a memória alocada não é mais necessária". Geralmente exige que o desenvolvedor determine a onde no programa tal pedaço da memória não é mais necessária e liberá-la.

Linguagens de alto nível vêm com um pedaço de software chamado "garbage collector" (coletor de sujeira), cujo trabalho é monitorar a alocação de memória a fim de descobrir quando um pedaço de código não é mais necessário e neste caso, automaticamente liberá-lo. Este processo é algo aproximado já que, em geral, saber se um pedaço de memória é necessário é algo indecidível (que não pode ser resolvido através de um algoritmo).

Garbage collection

Como foi mencionado acima, em geral o problema de automaticamente descobrir se a memória "não é mais necessária" é indecidível. Como consequência, os *garbage collections* implementam uma limitação de uma solução ao problema em geral. Esta seção irá explicar os conceitos necessários para entender os principais algoritmos de garbage collection e suas limitações.

Referências

O principal conceito de algoritmos do garbage collection depende do conceito de *referência*. Dentro do contexto de gerenciamento de memória, é dito que um objeto faz referência a outro, caso o primeiro tenha acesso a este último (de maneira implícita ou explícita). Por exemplo, um objeto JavaScript tem uma referência ao seu prototype (referência implícita) e para os valores de suas propriedades (referência explícita).

Neste contexto, o conceito de "objeto" se extende para algo mais abrangente do que os objetos comuns do JavaScript, e também contém escopos de função (ou o escopo lexical global).

Referência de contagem do garbage collection

Este é o algoritmo mais ingênuo de garbage collection. este algoritmo reduz a definição de "um objeto não mais necessário" para "um objeto não tem outro objeto referenciando ele". Um objeto pode ser coletado pelo garbage collector se não existir referência apontando para este objeto.

Exemplo

```
var o = {
  a: {
    b: 2,
  },
};
// 2 objetos são criados. Um é referenciado pelo outro como uma de suas propriedades.
```

```
// O outro é referenciado pelo fato de ser atribuído à variável 'o'.
// Obviamente, nenhum pode ser coletado pelo garbage collector

var o2 = o; // a varável 'o2' é a segunda coisa que tem
// uma referência ao objeto
o = 1; // agora, o objeto que estava originalmente no 'o'
// tem uma referência única
// encorporada pela variável 'o2'

var oa = o2.a; // Referência para a propriedade 'a' do objeto.
// Este objeto agora tem 2 referências: uma como uma propriedade,
// a outra como a variável 'oa'

o2 = "yo"; // O objeto que estava originalmente em 'o' agora não tem
// nenhuma referência para ele.
// Ele poderia ser coletado pelo garbage collector.
// Entretanto o que era sua propriedade 'a' continua sendo referenciada
// pela variável 'oa', então ele não pode ser coletado.

oa = null; // O que era a propriedade 'a' do objeto original em 'o'
// não tem mais refêrencia para ele. Então pode ser coletado.
```

Limitação : ciclos

Esse algoritmo ingênuo tem a limitação de que objetos que referenciam um ao outro (e formam um ciclo), podem ser "não mais necessários" e ainda assim não serem coletados.

```
function f() {
  var o = {};
  var o2 = {};
  o.a = o2; // o referencia o2
  o2.a = o; // o2 referencia o

  return "azerty";
}

f();
// Dois objetos são criados e referenciam um ao outro criando assim um ciclo.
// Eles não vão sair do escopo da função depois dela ser chamada, então eles
// são efetivamente inúteis e podem ser liberados.
// Entretanto, o algoritmo contador de referências considera que desde que ambos
// os objetos sejam referenciados pelo menos uma vez,
// nenhum deles podem ser coletados.
```

Exemplo da vida real

Internet Explorer 6 e 7 são conhecidos por terem um coletor com contador de referências para os objetos do DOM. Ciclos são um erro comum que podem gerar erros na memória:

```
var div;
window.onload = function () {
  div = document.getElementById("minhaDiv");
  div.referenciaCircular = div;
  div.muitosDados = new Array(10000).join("*");
};
```

No exemplo acima, o elemento do DOM "minhaDiv" tem uma referência circular para ela mesma na propriedade "referenciaCircular". Se a propriedade não for removida ou anulada explicitamente, o contador de referências do coletor sempre terá pelo menos uma referência intacta e irá manter o elemento do DOM na memória mesmo se ele for removido da árvore do DOM. Se o elemento do DOM retém muitos dados (ilustrado no exemplo acima com a propriedade "muitosDados"), a memória consumida por esses dados não será liberada.

Algoritmo de varredura e rotulação

Esse algoritmo reduz a definição de "um objeto não é mais necessário" para "um objeto é inacessível".

Esse algoritmo assume o conhecimento de uma lista de objetos chamada *roots* (raízes) (no JavaScript, o root é o objeto global). Periodicamente, o coletor iniciará por esses roots, encontrando todos os objetos que são referenciados por esses roots, então todos os objetos referenciados por eles, etc. Começando pelos roots, o coletor encontrará todos os objetos acessíveis e coletará todos os objetos inacessíveis.

Esse algoritmo é melhor que o anterior pois "um objeto que não tem referência" leva a esse objeto inacessível. O oposto não é verdadeiro como nós vimos com os ciclos.

A partir de 2012, todos os navegadores modernos vem com um coletor com varredura e rotulação. Todas as melhorias feitas nessa área do coletor do JavaScript (geracional/incremental/concorrência/coletor paralelo) nos últimos anos são implementações melhoradas deste algoritmo, mas não melhorias para o coletor propriamente nem a redução da definição de quando "um objeto não é mais necessário".

Ciclos não são mais um problema

No primeiro exemplo, depois do retorno da chamada da função, os 2 objetos não são mais referenciados por algo acessível pelo objeto global. Consequentemente, eles não serão acessíveis pelo coletor.

A mesma coisa acontece com o segundo exemplo. Uma vez que o div e o seu gerenciador de eventos se tornem inacessíveis pelos roots, ambos podem ser coletados, apesar de referenciar um ao outro.

Limitação: objetos devem ser feitos explicitamente inacessíveis

Apesar disso ser marcado como uma limitação, isso é algo raro na prática e é por isso que ninguém se importa muito sobre o coletor.

Veja também

- IBM article on "Memory leak patterns in JavaScript" (2007)
- Kangax article on how to register event handler and avoid memory leaks (2010)
- Performance

Meta programação

- Anterior

Começando com ECMAScript 6, o JavaScript ganha suporte para os objetos `Proxy` e `Reflect`, permitindo você interceptar e definir o comportamento personalizado para operações fundamentais da linguagem (por exemplo, pesquisa de propriedade, atribuição, enumeração, invocação de função, etc). Com a ajuda destes dois objetos você será capaz de programar a nível meta em JavaScript.

Proxies

Introduzido em ECMAScript 6, objetos `Proxy` permitem que você intercepte determinadas operações e implementar comportamentos personalizados. Por exemplo, receber uma propriedade em um objeto:

```
var handler = {
  get: function (target, name) {
    return name in target ? target[name] : 42;
  },
};
var p = new Proxy({}, handler);
p.a = 1;
console.log(p.a, p.b); // 1, 42
```

O objeto Proxy define um `target` (um objeto vazio aqui) e um objeto `handler` em que um `get` *trap* é implementado. Aqui, um objeto que está em proxy não retornará indefinido quando receber propriedades indefinidas, mas, ao contrário, retornar o número 42.

Exemplos adicionais estão disponíveis na página de referência de `Proxy` .

Terminologia

Os seguintes termos são usados quando se fala sobre a funcionalidade de proxies.

handler

Espaço reservado de objeto que contenha traps.

traps

Os métodos que fornecem acesso de propriedade. Isto é análogo ao conceito de traps em sistemas operacionais.

target

Objeto que o proxy está virtualizando. Ele é frequentemente usado como backend de armazenamento para o proxy. Invariantes (semânticas que permanecem inalteradas) relativas a objetos que não podem ser extendidos ou propriedades que não podem ser configuradas são comparadas com o target.

invariantes

Semânticas que permanecem inalteradas na execução de operações personalizadas são chamados de *invariantes*. Se você violar as invariantes de um manipulador, um `TypeError` será lançado.

Handlers e traps

A tabela a seguir resume as traps disponíveis aos objetos do tipo Proxy. Veja as páginas de referência para explicações detalhadas e exemplos.

Handler / trap	Interceptions	Invariants

`handler.getPrototypeOf()`	`Object.getPrototypeOf()`
	`Reflect.getPrototypeOf()`
	`__proto__`
	`Object.prototype.isPrototypeOf()`
	`instanceof`
		• O método `getPrototypeOf` deve retornar um object ou `null`.
		• Se `target` não puder ser extendido, o método `Object.getPrototypeOf(proxy)` deve retornar o mesmo valor que `Object.getPrototypeOf(target)`.

`handler.setPrototypeOf()`	`Object.setPrototypeOf()` `Reflect.setPrototypeOf()`	Se `target` não puder ser extendido, o parâmetro `prototype` dever ter o mesmo valor que `Object.getPrototypeOf(target)`.
`handler.isExtensible()`	`Object.isExtensible()` `Reflect.isExtensible()`	`Object.isExtensible(proxy)` deve retornar o mesmo valor que `Object.isExtensible(target)`.
`handler.preventExtensions()`	`Object.preventExtensions()` `Reflect.preventExtensions()`	`Object.preventExtensions(proxy)` retorna `true` somente se `Object.isExtensible(proxy)` retornar `false`.
`handler.getOwnPropertyDescriptor()`	`Object.getOwnPropertyDescriptor()` `Reflect.getOwnPropertyDescriptor()`	• `getOwnPropertyDescriptor` deve retornar um object ou `undefined`. • Uma propriedade não pode ser descrita como não existente se ela existir como uma propriedade própria e não configurável do

Handler / trap	Interceptions	Invariants
		objeto alvo. • Uma propriedade não pode ser relatada como inexistente, se existir como uma propriedade própria do objeto de destino e o objeto de destino não for extensível. • Uma propriedade não pode ser relatada como existente, se não existir como uma propriedade própria do objeto de destino e o objeto de destino não for extensível. • Uma propriedade não pode ser relatada como não configurável, se não existir como uma propriedade própria do objeto de destino ou se existir como uma propriedade própria configurável do objeto de destino. • O resultado de `Object.getOwnPropertyDescriptor(target)` pode ser aplicado ao objeto de destino usando `Object.defineProperty` e não emitirá uma exceção.
`handler.defineProperty()`	`Object.defineProperty()` `Reflect.defineProperty()`	• Uma propriedade não pode ser adicionada se o objeto de destino não for extensível. • Uma propriedade não pode ser adicionada como ou modificada para não ser configurável, se não existir como uma propriedade própria não configurável do objeto de destino. • Uma propriedade pode não ser não configurável, se existir uma propriedade configurável correspondente do objeto de destino. • Se uma propriedade tiver uma propriedade de objeto de destino correspondente `Object.defineProperty(target, prop, descriptor)` não lançará uma exceção. • No modo estrito, um valor de retorno `false` do manipulador `defineProperty` manipulador lançará um `TypeError` exceção.
`handler.has()`	Property query: `foo in proxy` Inherited property query: `foo in Object.create(proxy)` `Reflect.has()`	• Uma propriedade não pode ser relatada como inexistente, se existir como uma propriedade própria não configurável do objeto de destino. • Uma propriedade não pode ser relatada como inexistente, se existir como uma propriedade própria do objeto de destino e o objeto de destino não for extensível.
`handler.get()`	1 Property access: `proxy[foo]`and `proxy.bar` Inherited property access: `Object.create(proxy)[foo]` `Reflect.get()`	• O valor relatado para uma propriedade deve ser igual ao valor da propriedade do objeto de destino correspondente se a propriedade do objeto de destino for uma propriedade de dados não gravável e não configurável. • O valor relatado para uma propriedade deve ser indefinido se a propriedade do objeto de destino correspondente for uma propriedade acessora não configurável que tenha sido indefinida como seu atributo [[Get]].
`handler.set()`	Property assignment: `proxy[foo] = bar` and `proxy.foo = bar` Inherited property assignment: `Object.create(proxy)[foo] = bar` `Reflect.set()`	• Não é possível alterar o valor de uma propriedade para ser diferente do valor da propriedade do objeto de destino correspondente se a propriedade do objeto de destino correspondente for uma propriedade de dados não gravável e não configurável. • Não é possível definir o valor de uma propriedade se a propriedade do objeto de destino correspondente for uma propriedade acessadora não configurável que tenha `undefined` como seu atributo [[Set]]. • No modo estrito, um valor de retorno `false` do manipulador `set` lançará uma exceção `TypeError`.
`handler.deleteProperty()`	Property deletion: `delete proxy[foo]` and `delete proxy.foo` `Reflect.deleteProperty()`	Uma propriedade não pode ser excluída, se existir como uma propriedade própria não configurável do objeto de destino.
`handler.enumerate()`	Property enumeration / for...in: `for (var name in proxy) {...}` `Reflect.enumerate()`	O método `enumerate` deve retornar um objeto.

`handler.ownKeys()`	`Object.getOwnPropertyNames()`
	`Object.getOwnPropertySymbols()`
	`Object.keys()`
	`Reflect.ownKeys()`
		• O resultado de `ownKeys` é uma lista.
		• O tipo de cada elemento da lista de resultados é `String` ou `Symbol`.
		• A lista de resultados deve conter as chaves de todas as propriedades próprias não configuráveis do objeto de destino.
		• Se o objeto de destino não for extensível, a Lista de resultados deverá conter todas as chaves das próprias propriedades do objeto de destino e nenhum outro valor.

`handler.apply()`	`proxy(..args)` `Function.prototype.apply()` and	Não há invariantes para o método `handler.apply`

Handler / trap	Interceptions	Invariants
	`Function.prototype.call()` `Reflect.apply()`	
`handler.construct()`	`new proxy(...args)` `Reflect.construct()`	O resultado deve ser um `Object`.

Proxy Revogável

O método `Proxy.revocable()` é utilizado para criar um objeto Proxy revogável. Isso significa que o proxy pode ser revogado através da função `revoke`, desligando-o. Depois disso, qualquer operação com o proxy lançará um `TypeError`.

```
var revocable = Proxy.revocable(
  {},
  {
    get: function (target, name) {
      return "[[" + name + "]]";
    },
  },
);
var proxy = revocable.proxy;
console.log(proxy.foo); // "[[foo]]"

revocable.revoke();

console.log(proxy.foo); // TypeError é lançado
proxy.foo = 1; // TypeError novamente
delete proxy.foo; // ainda um TypeError
typeof proxy; // "object", typeof não desencadeia nenhuma trap
```

Reflexão

`Reflect` é um objeto embutido que contém métodos que permitem a criação de operações interceptáveis em JavaScript. Os métodos são iguais àqueles de proxy handlers. `Reflect` não é um objeto do tipo function.

`Reflect` auxilia no encaminhamento de operações padrão do handler para o target.

`Reflect.has()`, por exemplo, tem o mesmo efeito prático que o operador in, com a facilidade de ser utilizado como uma função:

```
Reflect.has(Object, "assign"); // true
```

Uma função <code>apply</code> melhorada

Em ES5, você normalmente utiliza o método `Function.prototype.apply()` para invocar uma função com um dado valor para `this` e `arguments` fornecido como um array (ou um objeto parecido com um array).

```
Function.prototype.apply.call(Math.floor, undefined, [1.75]);
```

Com `Reflect.apply` essa operação se torna menos verbosa e mais fácil de compreender:

```
Reflect.apply(Math.floor, undefined, [1.75]);
// 1;

Reflect.apply(String.fromCharCode, undefined, [104, 101, 108, 108, 111]);
// "hello"

Reflect.apply(RegExp.prototype.exec, /ab/, ["confabulation"]).index;
// 4

Reflect.apply("".charAt, "ponies", [3]);
// "i"
```

Verificando se a definição da propriedade obteve sucesso

Com `Object.defineProperty`, a qual retorna um `object` em caso de sucesso ou lança um `TypeError` em caso contrário, você utilizaria um bloco `try...catch` para capturar qualquer erro que tenha ocorrido ao definir uma propriedade. Devido ao fato de `Reflect.defineProperty` retornar um status do tipo `Boolean`, você pode simplesmente utilizar aqui um bloco `if...else`:

```
if (Reflect.defineProperty(target, property, attributes)) {
  // success
} else {
  // failure
}
```


- Anterior

Módulos JavaScript

- Anterior

Este guia fornece tudo o que você precisa para começar com a sintaxe de módulo do JavaScript.

Um background em módulos

Os programas JavaScript começaram muito pequenos - a maior parte do seu uso nos primeiros dias era para executar tarefas isoladas de script, fornecendo um pouco de interatividade às suas páginas da Web sempre que necessário, de modo que scripts grandes geralmente não eram necessários. Com o avanço rápido da tecnologia agora temos aplicativos completos sendo executados em navegadores com muito JavaScript, além de JavaScript ser usado em outros contextos (Node.js, por exemplo).

Portanto, fez sentido nos últimos anos começar a pensar em fornecer mecanismos para dividir programas JavaScript em módulos separados que podem ser importados quando necessário. O Node.js possui essa capacidade há muito tempo e existem várias bibliotecas e estruturas JavaScript que permitem o uso do módulo (por exemplo, outros CommonJS e AMD-sistemas de módulos baseados em RequireJS, e mais recentemente Webpack e Babel).

A boa notícia é que os navegadores modernos começaram a dar suporte nativamente à funcionalidade do módulo, e é sobre isso que este artigo trata. Isso só pode ser uma coisa boa - os navegadores podem otimizar o carregamento de módulos, tornando-o mais eficiente do que ter que usar uma biblioteca e fazer todo esse processamento extra no lado do cliente e viagens de ida e volta extras.

Suporte do navegador

O uso de módulos JavaScript nativos depende do`import` e `export` afirmações; estes são suportados nos navegadores da seguinte maneira:

importa

exporta

Apresentando um exemplo

Para demonstrar o uso dos módulos, criamos um conjunto simples de exemplos que você pode encontrar no GitHub. Estes exemplos demonstram um conjunto simples de módulos que criam um`<canvas>` em uma página da Web e desenhe (e relate informações sobre) formas diferentes na tela.

Estes são bastante triviais, mas foram mantidos deliberadamente simples para demonstrar claramente os módulos.

> **Nota: Nota: Se você deseja fazer o download dos exemplos e executá-los localmente, precisará executá-los por meio de um servidor da web local.**

Exemplo de uma estrutura básica

No nosso primeiro exemplo (consulte basic-modules) nós temos uma estrutura de arquivos da seguinte maneira:

```
index.html
main.js
modules/
    canvas.js
    square.js
```

> **Nota: Nota: Todos os exemplos neste guia têm basicamente a mesma estrutura; o exposto acima deve começar a ficar bem familiar.**

Os dois módulos do diretório modules são descritos abaixo:

- `canvas.js` — contém funções relacionadas à configuração da tela:
    - `create()` — cria uma tela com uma largura e altura especificadas dentro de um invólucro `<div>` com um ID especificado, que é anexado dentro de um elemento pai especificado. Retorna um objeto que contém o contexto 2D da tela e o ID do wrapper.
    - `createReportList()` — cria uma lista não ordenada anexada dentro de um elemento de wrapper especificado, que pode ser usado para gerar dados de relatório. Retorna o ID da lista.
- `square.js` — contém:
    - `name` — uma constante contendo a string 'square'.
    - `draw()` — desenha um quadrado em uma tela especificada, com um tamanho, posição e cor especificados. Retorna um objeto que contém o tamanho, a posição e a cor do quadrado.
    - `reportArea()` — grava a área de um quadrado em uma lista de relatórios específica, considerando seu tamanho.
    - `reportPerimeter()` — grava o perímetro de um quadrado em uma lista de relatórios específica, considerando seu comprimento.

Aside — <code>.mjs</code> versus <code>.js</code>

Neste artigo, usamos extensões .js para nossos arquivos de módulo, mas em outros recursos você pode ver a extensão .mjs usada. A documentação da V8 recomenda isso, por exemplo. Os motivos apresentados são:

- É bom para maior clareza, ou seja, deixa claro quais arquivos são módulos e quais são JavaScript regulares.
- Ele garante que seus arquivos de módulo sejam analisados como um módulo por tempos de execução, como Node.js, e construir ferramentas como Babel.

No entanto, decidimos continuar usando .js, pelo menos por enquanto. Para que os módulos funcionem corretamente em um navegador, você precisa garantir que seu servidor os esteja servindo com um cabeçalho Content-Type que contenha um tipo MIME JavaScript, como text / javascript. Caso contrário, você receberá um erro estrito de verificação do tipo MIME, de acordo com as linhas "O servidor respondeu com um tipo MIME não JavaScript" e o navegador não executará seu JavaScript. A maioria dos servidores já define o tipo correto para arquivos .js, mas ainda não para arquivos .mjs. Servidores que já veiculam arquivos .mjs incluem corretamente GitHub Pages e `http-server` para Node.js.

Tudo bem se você já estiver usando esse ambiente ou se não estiver, mas souber o que está fazendo e tiver acesso (ou seja, você pode configurar o servidor para definir a configuração correta `Content-Type` para arquivos `.mjs`). No entanto, isso pode causar confusão se você não controlar o servidor do qual está servindo arquivos ou publicar arquivos para uso público, como estamos aqui.

Para fins de aprendizado e portabilidade, decidimos manter o `.js`.

Se você realmente valoriza a clareza de usar .mjs para módulos versus usar .js para arquivos JavaScript "normais", mas não deseja se deparar com o problema descrito acima, sempre poderá usar .mjs durante o desenvolvimento e convertê-los em .js durante sua etapa de construção.

Também é importante notar que:

- Algumas ferramentas podem nunca suportar .mjs, comoTypeScript.
- O atributo `<script type="module">`é usado para indicar quando um módulo está sendo apontado, como você verá abaixo.

Exportando recursos do módulo

A primeira coisa que você faz para obter acesso aos recursos do módulo é exportá-los. Isso é feito usando o `export` declaração.

A maneira mais fácil de usá-lo é colocá-lo na frente de qualquer item que você queira exportar para fora do módulo, por exemplo:

```
export const name = "square";

export function draw(ctx, length, x, y, color) {
  ctx.fillStyle = color;
  ctx.fillRect(x, y, length, length);

  return {
    length: length,
    x: x,
    y: y,
    color: color,
  };
}
```

Você pode exportar funções, `var`, `let`, `const`, e — como veremos mais tarde - classes. Eles precisam ser itens de nível superior; você não pode usar a exportação dentro de uma função, por exemplo.

Uma maneira mais conveniente de exportar todos os itens que você deseja exportar é usar uma única instrução de exportação no final do arquivo do módulo, seguida por uma lista separada por vírgula dos recursos que você deseja exportar envoltos em chaves. Por exemplo:

```
export { name, draw, reportArea, reportPerimeter };
```

Importando recursos para o seu script

Depois de exportar alguns recursos do seu módulo, é necessário importá-los para o script para poder usá-los. A maneira mais simples de fazer isso é a seguinte:

```
import { name, draw, reportArea, reportPerimeter } from "./modules/square.js";
```

Você usa o `import` , seguida por uma lista separada por vírgula dos recursos que você deseja importar agrupados em chaves, seguidos pela palavra-chave de, seguida pelo caminho para o arquivo do módulo - um caminho relativo à raiz do site, que para nossa `basic-modules` exemplo seria`/js-examples/modules/basic-modules`.

No entanto, escrevemos o caminho de maneira um pouco diferente - estamos usando a sintaxe de ponto (.) Para significar "o local atual", seguido pelo caminho além do arquivo que estamos tentando encontrar. Isso é muito melhor do que escrever todo o caminho relativo a cada vez, pois é mais curto e torna o URL portátil - o exemplo ainda funcionará se você o mover para um local diferente na hierarquia do site.

Então, por exemplo:

```
/js-examples/modules/basic-modules/modules/square.js
```

torna-se

```
./modules/square.js
```

Você pode ver essas linhas em ação em `main.js`.

> **Nota: Nota: Em alguns sistemas de módulos, você pode omitir a extensão do arquivo e o ponto**(e.g. `'/modules/square'`). Isso não funciona nos módulos JavaScript nativos.

Depois de importar os recursos para o seu script, você pode usá-los exatamente como eles foram definidos no mesmo arquivo. O seguinte é encontrado em `main.js`, abaixo das linhas de importação:

```
let myCanvas = create("myCanvas", document.body, 480, 320);
let reportList = createReportList(myCanvas.id);

let square1 = draw(myCanvas.ctx, 50, 50, 100, "blue");
reportArea(square1.length, reportList);
reportPerimeter(square1.length, reportList);
```

> **Nota: Nota: Embora os recursos importados estejam disponíveis no arquivo, eles são visualizações somente leitura do recurso que foi exportado. Você não pode alterar a variável importada, mas ainda pode modificar propriedades semelhantes à const. Além disso, esses recursos são importados como ligações ativas, o que significa que eles podem mudar de valor mesmo que você não possa modificar a ligação ao contrário de const.**

Aplicando o módulo ao seu HTML

Agora, apenas precisamos aplicar o módulo main.js. à nossa página HTML. Isso é muito semelhante ao modo como aplicamos um script regular a uma página, com algumas diferenças notáveis.

Primeiro de tudo, você precisa incluir `type="module"` no `<script>` elemento, para declarar esse script como um módulo. Para importar o `main.js` script, usamos este:

```
<script type="module" src="https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Modules/main.js"></script>
```

Você também pode incorporar o script do módulo diretamente no arquivo HTML, colocando o código JavaScript no corpo do elemento <script>:

```
<script type="module">/* JavaScript module code here */</script>
```

O script para o qual você importa os recursos do módulo atua basicamente como o módulo de nível superior. Se você o omitir, o Firefox, por exemplo, exibirá um erro "SyntaxError: as declarações de importação podem aparecer apenas no nível superior de um módulo".

Você só pode usar `import` e `export` instruções dentro de módulos, não scripts regulares.

Outras diferenças entre módulos e scripts padrão

- Você precisa prestar atenção nos testes locais - se você tentar carregar o arquivo HTML localmente (i.e. com um arquivo `://` URL), você encontrará erros do CORS devido a requisitos de segurança do módulo JavaScript. Você precisa fazer seus testes através de um servidor.
- Além disso, observe que você pode obter um comportamento diferente das seções de script definidas dentro dos módulos e não nos scripts padrão. Isso ocorre porque os módulos usam strict mode automaticamente.
- Não há necessidade de usar o atributo deferir (consulte `<script>` attributes) ao carregar um script de módulo; módulos são adiados automaticamente.
- Os módulos são executados apenas uma vez, mesmo que tenham sido referenciados em várias tags <script>.
- Por último, mas não menos importante, vamos esclarecer: os recursos do módulo são importados para o escopo de um único script - eles não estão disponíveis no escopo global. Portanto, você poderá acessar apenas os recursos importados no script para o qual eles foram importados e não poderá acessá-los no console JavaScript, por exemplo. Você ainda receberá erros de sintaxe mostrados no DevTools, mas não poderá usar algumas das técnicas de depuração que você esperava usar.

Exportações padrão versus exportações nomeadas

A funcionalidade que exportamos até agora foi composta por **named exports** — cada item (seja uma função, const, etc.) foi referido por seu nome na exportação e esse nome também foi usado para se referir a ele na importação.

Há também um tipo de exportação chamado **default export** — isso foi projetado para facilitar a função padrão fornecida por um módulo e também ajuda os módulos JavaScript a interoperar com os sistemas de módulos CommonJS e AMD existentes (conforme explicado em ES6 In Depth: Modules por Jason Orendorff; procure por "Exportações padrão").

Vejamos um exemplo ao explicar como ele funciona. Nos nossos módulos básicos `square.js` você pode encontrar uma função chamada `randomSquare()` que cria um quadrado com cor, tamanho e posição aleatórios. Queremos exportar isso como padrão, portanto, na parte inferior do arquivo, escrevemos isso:

```
export default randomSquare;
```

Note a falta dos colchetes.

Em vez disso, poderíamos acrescentar `export default` na função e defina-a como uma função anônima, assim:

```
export default function(ctx) {
  ...
}
```

No nosso arquivo main.js., importamos a função padrão usando esta linha:

```
import randomSquare from "./modules/square.js";
```

Isso ocorre porque há apenas uma exportação padrão permitida por módulo e sabemos que *randomSquare* é isso.

```
import { default as randomSquare } from "./modules/square.js";
```

> **Nota: Nota: A sintaxe as para renomear itens exportados é explicada abaixo no** Renaming imports and exports seção.

Evitando conflitos de nomenclatura

Até agora, nossos módulos de desenho de forma de tela parecem estar funcionando bem. Mas o que acontece se tentarmos adicionar um módulo que lide com o desenho de outra forma, como um círculo ou triângulo? Essas formas provavelmente teriam funções associadas, como `draw()`, `reportArea()`, etc. também; se tentássemos importar diferentes funções com o mesmo nome para o mesmo arquivo de módulo de nível superior, acabaríamos com conflitos e erros.

Felizmente, existem várias maneiras de contornar isso. Veremos isso nas próximas seções.

Renomeando importações e exportações

Dentro dos colchetes da instrução de importação e exportação, você pode usar a palavra-chave junto com um novo nome de recurso, para alterar o nome de identificação que será usado para um recurso dentro do módulo de nível superior.

Por exemplo, os dois itens a seguir executariam o mesmo trabalho, embora de uma maneira ligeiramente diferente:

```
// inside module.js
export { function1 as newFunctionName, function2 as anotherNewFunctionName };

// inside main.js
import { newFunctionName, anotherNewFunctionName } from "./modules/module.js";
```

```
// inside module.js
export { function1, function2 };

// inside main.js
import {
  function1 as newFunctionName,
  function2 as anotherNewFunctionName,
} from "./modules/module.js";
```

Vejamos um exemplo real. Na nossa renaming diretório, você verá o mesmo sistema de módulos do exemplo anterior, exceto que adicionamos `circle.js` e `triangle.js` módulos para desenhar e relatar círculos e triângulos.

Dentro de cada um desses módulos, temos recursos com os mesmos nomes sendo exportados e, portanto, cada um tem o mesmo `export` declaração na parte inferior:

```
export { name, draw, reportArea, reportPerimeter };
```

Ao importá-los para o main.js, se tentarmos usar

```
import { name, draw, reportArea, reportPerimeter } from "./modules/square.js";
import { name, draw, reportArea, reportPerimeter } from "./modules/circle.js";
import { name, draw, reportArea, reportPerimeter } from "./modules/triangle.js";
```

O navegador geraria um erro como "SyntaxError: redeclaration of import name" (Firefox).

Em vez disso, precisamos renomear as importações para que sejam únicas:

```
import {
  name as squareName,
  draw as drawSquare,
  reportArea as reportSquareArea,
  reportPerimeter as reportSquarePerimeter,
} from "./modules/square.js";

import {
  name as circleName,
  draw as drawCircle,
  reportArea as reportCircleArea,
  reportPerimeter as reportCirclePerimeter,
} from "./modules/circle.js";

import {
  name as triangleName,
  draw as drawTriangle,
  reportArea as reportTriangleArea,
  reportPerimeter as reportTrianglePerimeter,
} from "./modules/triangle.js";
```

Observe que você pode resolver o problema nos arquivos do módulo, e.g.

```
// in square.js
export {
  name as squareName,
  draw as drawSquare,
  reportArea as reportSquareArea,
  reportPerimeter as reportSquarePerimeter,
};
```

```
// in main.js
import {
  squareName,
  drawSquare,
  reportSquareArea,
  reportSquarePerimeter,
} from "./modules/square.js";
```

E funcionaria da mesma forma. Qual o estilo que você usa depende de você, no entanto, sem dúvida faz mais sentido deixar o código do módulo em paz e fazer as alterações nas importações. Isso faz especialmente sentido quando você está importando de módulos de terceiros sobre os quais você não tem controle.

24/5000

Criando um objeto de módulo

O método acima funciona bem, mas é um pouco confuso e longo. Uma solução ainda melhor é importar os recursos de cada módulo dentro de um objeto de módulo. O seguinte formulário de sintaxe faz isso:

```
import * as Module from "./modules/module.js";
```

Isso captura todas as exportações disponíveis no module.js e as torna disponíveis como membros de um objeto `Module`, efetivamente dando o seu próprio namespace. Então, por exemplo:

```
Module.function1()
Module.function2()
etc.
```

Novamente, vejamos um exemplo real. Se você for ao nosso module-objects diretório, você verá o mesmo exemplo novamente, mas reescrito para aproveitar essa nova sintaxe. Nos módulos, as exportações são todas da seguinte forma simples:

```
export { name, draw, reportArea, reportPerimeter };
```

As importações, por outro lado, são assim:

```
import * as Canvas from "./modules/canvas.js";

import * as Square from "./modules/square.js";
import * as Circle from "./modules/circle.js";
import * as Triangle from "./modules/triangle.js";
```

Em cada caso, agora você pode acessar as importações do módulo abaixo do nome do objeto especificado, por exemplo:

```
let square1 = Square.draw(myCanvas.ctx, 50, 50, 100, "blue");
Square.reportArea(square1.length, reportList);
Square.reportPerimeter(square1.length, reportList);
```

Agora você pode escrever o código da mesma forma que antes (contanto que inclua os nomes dos objetos quando necessário) e as importações sejam muito mais limpas.

Módulos e classes

Como sugerimos anteriormente, você também pode exportar e importar classes; essa é outra opção para evitar conflitos no seu código e é especialmente útil se você já tiver o código do módulo gravado em um estilo orientado a objetos.

Você pode ver um exemplo do nosso módulo de desenho de forma reescrito com as classes ES em nosso classes diretório. Como exemplo, o `square.js` O arquivo agora contém todas as suas funcionalidades em uma única classe:

```
class Square {
  constructor(ctx, listId, length, x, y, color) {
    ...
  }

  draw() {
    ...
  }

  ...
}
```

que exportamos então:

```
export { Square };
```

No main.js, nós o importamos assim:

```
import { Square } from "./modules/square.js";
```

E então use a classe para desenhar nosso quadrado:

```
let square1 = new Square(myCanvas.ctx, myCanvas.listId, 50, 50, 100, "blue");
square1.draw();
square1.reportArea();
square1.reportPerimeter();
```

Módulos de agregação

Haverá momentos em que você desejará agregar módulos. Você pode ter vários níveis de dependências, nos quais deseja simplificar as coisas, combinando vários submódulos em um módulo pai. Isso é possível usando a sintaxe de exportação dos seguintes formulários no módulo pai:

```
export * from "x.js";
export { name } from "x.js";
```

Por exemplo, veja nosso module-aggregation diretório. Neste exemplo (com base no exemplo de classes anteriores), temos um módulo extra chamado shapes.js, que agrega toda a funcionalidade de circle.js, square.js e triangle.js juntos. Também movemos nossos submódulos para dentro de um subdiretório dentro do diretório modules chamado shapes. Portanto, a estrutura do módulo neste exemplo é:

```
modules/
  canvas.js
  shapes.js
  shapes/
    circle.js
    square.js
    triangle.js
```

Em cada um dos submódulos, a exportação é da mesma forma, e.g.

```
export { Square };
```

Em seguida, vem a parte de agregação. Dentro de shapes.js, incluímos as seguintes linhas:

```
export { Square } from "./shapes/square.js";
export { Triangle } from "./shapes/triangle.js";
export { Circle } from "./shapes/circle.js";
```

Eles capturam as exportações dos submódulos individuais e os disponibilizam efetivamente no módulo shapes.js.

> **Nota: Nota: As exportações mencionadas no shapes.js são basicamente redirecionadas pelo arquivo e realmente não existem nele, portanto, você não poderá escrever nenhum código relacionado útil dentro do mesmo arquivo.**

Portanto, agora no arquivo main.js., podemos obter acesso às três classes de módulos substituindo

```
import { Square } from "./modules/square.js";
import { Circle } from "./modules/circle.js";
import { Triangle } from "./modules/triangle.js";
```

com a seguinte linha única:

```
import { Square, Circle, Triangle } from "./modules/shapes.js";
```

Carregamento dinâmico do módulo

A parte mais recente da funcionalidade dos módulos JavaScript a estar disponível nos navegadores é o carregamento dinâmico de módulos. Isso permite que você carregue módulos dinamicamente somente quando eles forem necessários, em vez de precisar carregar tudo antecipadamente. Isso tem algumas vantagens óbvias de desempenho; vamos ler e ver como isso funciona.

Essa nova funcionalidade permite que você ligue `import()` como uma função, passando o caminho para o módulo como um parâmetro. Retorna um`Promise`, que cumpre com um objeto de módulo (consulte Creating a module object) dando acesso às exportações desse objeto, e.g.

```
import("./modules/myModule.js").then((module) => {
  // Do something with the module.
});
```

Vejamos um exemplo. No dynamic-module-imports diretório, temos outro exemplo baseado em nosso exemplo de classes. Desta vez, no entanto, não estamos desenhando nada na tela quando o exemplo é carregado. Em vez disso, incluímos trêsbuttons — "Circle", "Square", e "Triangle" — que, quando pressionado, carrega dinamicamente o módulo necessário e, em seguida, usa-o para desenhar os shape.

Neste exemplo, fizemos apenas alterações em nossa `index.html` e `main.js` arquivos - as exportações do módulo permanecem as mesmas de antes.

No main.js, pegamos uma referência a cada botão usando um`Document.querySelector()` chamada, por exemplo:

```
let squareBtn = document.querySelector(".square");
```

Em seguida, anexamos um ouvinte de evento a cada botão para que, quando pressionado, o módulo relevante seja carregado dinamicamente e usado para desenhar a forma(shape):

```
squareBtn.addEventListener("click", () => {
  import("./modules/square.js").then((Module) => {
    let square1 = new Module.Square(
      myCanvas.ctx,
      myCanvas.listId,
      50,
      50,
      100,
      "blue",
    );
    square1.draw();
    square1.reportArea();
    square1.reportPerimeter();
  });
});
```

Observe que, como o cumprimento da promessa retorna um objeto de módulo, a classe é então transformada em uma sub-característica do objeto, portanto, agora precisamos acessar o construtor com `Module.` anexado a ele, e.g. `Module.Square( ... )`.

Solução de problemas

Aqui estão algumas dicas que podem ajudá-lo se você estiver com problemas para fazer seus módulos funcionarem. Sinta-se livre para adicionar à lista se descobrir mais!

- Mencionamos isso antes, mas para reiterar: arquivos `.js` precisa ser carregado com um tipo MIME de `text/javascript` (ou outro tipo MIME compatível com JavaScript, mas `text/javascript` é recomendável), caso contrário, você receberá um erro estrito de verificação do tipo MIME como "O servidor respondeu com um tipo MIME não JavaScript".
- Se você tentar carregar o arquivo HTML localmente (i.e. com um arquivo `://` URL), você encontrará erros do CORS devido a requisitos de segurança do módulo JavaScript. Você precisa fazer seus testes através de um servidor. As páginas do GitHub são ideais, pois também servem arquivos .js com o tipo MIME correto.
- Como .mjs é uma extensão de arquivo não padrão, alguns sistemas operacionais podem não reconhecê-lo ou tentar substituí-lo por outra. Por exemplo, descobrimos que o macOS estava adicionando silenciosamente .js ao final dos arquivos .mjs e ocultando automaticamente a extensão do arquivo. Então, todos os nossos arquivos foram lançados como x.mjs.js. Depois de desativarmos ocultar automaticamente as extensões de arquivo e treiná-lo para aceitar .mjs, tudo bem.

Veja também

- Using JavaScript modules on the web, por Addy Osmani e Mathias Bynens
- ES modules: A cartoon deep-dive, Postagem no blog Hacks por Lin Clark
- ES6 in Depth: Modules, Publicação de blog de Hacks por Jason Orendorff
- Livro de Axel Rauschmayer Exploring JS: Modules

- Anterior

Números e datas

- Anterior
- Próxima

Este capítulo apresenta como utilizar números e datas em JavaScript.

Números

Em Javascript, todos os números são implementados em double-precision 64-bit binary format IEEE 754 (Por exemplo, um número entre -(253 -1) e 253 -1). **Não havendo especificação de tipo Integer**. Além de ser capaz de representar números de ponto flutuante, o tipo de número tem três valores simbólicos: `+Infinity`, `-Infinity`, and `NaN` (not-a-number). Veja também Estruturas e Tipos de Dados em Javascript em contexto com outros tipos primitivos em JavaScript.

Você pode usar quatro tipos de números literais: decimal, binário, octal, e hexadecimal.

Números Decimais

```
1234567890;
42;

// Cuidado quando usar zeros à esquerda:

0888; // 888 interpretado como decimal
0777; // interpretado como octal  em modo no-strict (511 em decimal)
```

Note que literais decimais podem começar com zero (`0`) seguido por outro digito decimal, porém se o próximo dígito depois do primeiro zero for menor do que 8, o número será analisado como um número octal.

Números Binários

A sintaxe para números Binários, usa um zero à esquerda seguido de uma letra minúscula ou maiúscula "B" (`0b` or `0B`). Se os dígitos depois de 0b não forem 0 ou 1, a seguinte exceção `SyntaxError` é lançada: "Missing binary digits after 0b".

```
var FLT_SIGNBIT = 0b10000000000000000000000000000000; // 2147483648
var FLT_EXPONENT = 0b01111111100000000000000000000000; // 2139095040
var FLT_MANTISSA = 0b00000000011111111111111111111111; // 8388607
```

Números octais

A sintaxe dos números octais usa um zero na frente. Se os dígitos depois do 0 estiverem fora do alcance 0 a 7, o número será interpretado como um número decimal.

```
var n = 0755; // 493
var m = 0644; // 420
```

Modo estrito no ECMAScript 5 proíbe a sintaxe octal. A sintaxe Octal não é parte do ECMAScript 5, mas é suportada por todos os navegadores prefixando o número octal com zero: `0644 === 420` e `"\045" === "%"`. Em ECMAScript 6 números Octais são suportados prefixando o número com "`0o`" isto é.

```
var a = 0o10; // ES6: Octal
```

Numeros hexadecimais

A sintaxe numérica Hexadecimal usa um 0 na frente seguido por uma letra "X" maiúscula ou minúscula (`0x` ou `0X`). Se os dígidos depois do 0x estiverem fora do alcance (0123456789ABCDF), o seguinte erro de sintaxe (`SyntaxError`) ocorrerá: "Identifier starts immediately after numeric literal" (O identificador começa imediatamente depois do literal numérico).

```
0xffffffffffffffff; // 295147905179352830000
0x123456789abcdef; // 81985529216486900
0xa; // 10
```

Exponenciação

```
1e3; // 1000
2e6; // 2000000
0.1e2; // 10
```

<code>Objeto Number</code>

Um objeto built-in `Number` tem propriedades para constantes numéricas, tais como valor máximo, não número e infinito. Você não pode alterar os valores dessas propriedades e elas são usadas assim:

```
var maiorNum = Number.MAX_VALUE; //Valor máximo
var menorNum = Number.MIN_VALUE; //Valor mínimo
var infiniteNum = Number.POSITIVE_INFINITY; //Infinito positivo
var negInfiniteNum = Number.NEGATIVE_INFINITY; //Infinito negativo
var notANum = Number.NaN; //Não é numeral
```

Você sempre se refere a uma propriedade do objeto predefinido `Number` como mostrado acima, e não como uma propriedade de um objeto `Number que você mesmo criou.`

A tabela à seguir sumariza as propriedades do objeto `Number.`

Propriedade	Descrição
`Number.MAX_VALUE`	O maior número representável.
`Number.MIN_VALUE`	O menor número representável.
`Number.NaN`	Valor "not a number" especial
`Number.NEGATIVE_INFINITY`	Valor especial infinito negativo; retornado em overflow
`Number.POSITIVE_INFINITY`	Valor especial infinito positivo; retornado em overflow
`Number.EPSILON`	Diferença entre um e o menor valor maior do que um que pode ser representado como um `Number`.
`Number.MIN_SAFE_INTEGER`	Mínimo safe integer em JavaScript.
`Number.MAX_SAFE_INTEGER`	Máximo safe integer em JavaScript.

Método	Descrição
`Number.parseFloat()`	Analisa um argumento string e retorna um número float. O mesmo que a função global `parseFloat()`.
`Number.parseInt()`	Analisa um argumento string e retorna um inteiro da raiz ou base especificada. O mesmo que a função global`parseInt()`.
`Number.isFinite()`	Determina se o valor passado é um número finito.
`Number.isInteger()`	Determina se o valor passado é um inteiro.
`Number.isNaN()`	Determina se o valor passado é `NaN`. A versão mais robusta da original `isNaN()`.
`Number.isSafeInteger()`	Determina se o valor passado é um safe integer.

O protótipo `Number` provê métodos para resgatar informações de objetos `Number` em vários formatos. A tabela a seguir sumariza os métodos de `Number.prototype`.

Método	Descrição
`toExponential()`	Retorna uma string representando o número em uma notação exponencial.
`toFixed()`	Retorna uma string representando o número em notação com ponto-fíxo.
`toPrecision()`	Retorna uma string representando o número em uma precisão especificada na notação de ponto-fíxo.

<code>Objeto Math</code>

O objeto `Math` tem propriedades e métodos para constantes matemáticas e funções. Por exemplo, o `PI do objeto` Math tem o valor de pi (3,141...), que você usaria em uma aplicação como

```
Math.PI;
```

Similarmente, funções matemáticas padrão são métodos do Math. Isto inclui funções trigonométricas, logarítmicas, exponenciais, e outras funções. Por exemplo, se você quiser usar a função trigonométrica seno, basta escrever

```
Math.sin(1.56);
```

Note que todos os métodos trigonométricos pegam argumentos em radianos.

A tabela a seguir sumariza os métodos do objeto Math.

Método	Descrição
`abs()`	Valor absoluto
`sin()`, `cos()`, `tan()`	Funções trigonométricas padrão; Argumentos em radianos

Método	Descrição
`asin()`, `acos()`, `atan()`, `atan2()`	Funções trigonométricas inversas; retorna valores em radianos
`sinh()`, `cosh()`, `tanh()`	Funções trigonométricas hiperbólicas; retorna valores em radianos.
`asinh()`, `acosh()`, `atanh()`	Funções trigonométricas hiperbólicas inversas; retorna valores em radianos.
`pow()`, `exp()`, `expm1()`, `log10()`, `log1p()`, `log2()`	Funções exponenciais e logarítmicas.
`floor()`, `ceil()`	Retorna o maior/menor inteiro que é menor/maior inteiro que ou igual ao argumento.
`min()`, `max()`	Retorna menor ou maior (respectivamente) de uma lista separada por vírgula de argumentos numéricos
`random()`	Retorna um número aleatório entre 0 e 1.
`round()`, `fround()`, `trunc()`,	Funções de arredondamento e truncamento.
`sqrt()`, `cbrt()`, `hypot()`	Raiz quadrada, raiz cúbica, raiz quadrada da soma de argumentos ao quadrado.
`sign()`	O sinal de um número, indicando se o número é positivo, negativo ou zero.
`clz32()`, `imul()`	Número de zeros à esquerda na representação binária de 32-bits. The result of the C-like 32-bit multiplication of the two arguments.

Diferentemente de muitos outros objetos, você nunca cria um objeto Math por conta própria. Você sempre deve utilizar o objeto Math nativo.

<code>Objeto Date</code>

JavaScript não possui dados do tipo data. No entanto, você pode usar o objeto `Date` e seus métodos para trabalhar com datas e horas nas suas aplicações. O objeto Date tem um grande número de métodos para setar, recuperar e manipular datas. Ele não tem nenhuma propriedade.

JavaScript manipula datas de maneira semelhante ao Java. As duas linguagens tem muitos dos mesmos métodos para lidar com datas e ambas armazenam datas como números em milisegundos, desde 1 de janeiro de 1970, às 00:00:00 (January 1, 1970, 00:00:00).

A abrangência do objeto Date é de -100,000,000 dias até 100,000,000 dias relativos a 01 de janeiro de 1970 UTC.

Para criar um objeto Date:

```
var dateObjectName = new Date([parameters]);
```

onde `dateObjectName` é o nome do objeto Date que está sendo criado; ele pode ser um novo objeto ou uma propriedade de um objeto existente.

A chamada de Date sem a palavra reservada `new`, simplesmente converte a data para uma representação dela como string.

Os `parâmetros` do código acima podem ser qualquer um a seguir:

- Nada: cria a data e hora de hoje. Por exemplo, `today = new Date();`.
- Uma string representando uma data da seguinte forma: "Mês dia, ano, horas:minutos:segundos". Por exemplo, `Xmas95 = new Date("25 de dezembro de 1995, 13:30:00")`. Se você omitir as horas, minutos ou segundos, o valor será setado para zero.
- Um conjunto de valores inteiros para ano, mês e dia. Por exemplo, `var Xmas95 = new Date(1995, 11, 25)`.
- Um conjunto de valores inteiros par ano, mês, dia, hora, minuto e segundos. Por exemplo, `var Xmas95 = new Date(1995, 11, 25, 9, 30, 0);`.

Métodos do objeto Date

Os métodos do objeto Date para manipular data e hora pertencem às seguintes categorias:

- Métodos "set", para setar valores de data e hora em objetos Date.
- Métodos "get", para recuperar valores de data e hora de objetos Date.
- Métodos "to", para retornar valores de string de objetos Date.
- Métodos parse e UTC, para parsear string de Data.

Com os métods "get" e "set", você pode recuperar e setar segundos, minutos, horas, dia e mês, dia da semana, meses e anos, separadamente. Existe um método `getDay` que retorna o dia da semana, mas não existe um método `setDay` correspondente, porque o dia da semana é setado automaticamente. Estes métodos utilizam números inteiros para representar estes valores da seguinte maneira:

- Segundos e minutos: de 0 a 59
- Horas: de 0 a 23
- Dia: 0 (Domingo) a 6 (Sábado)
- Data: 1 a 31 (dia do mês)
- Meses: 0 (Janeiro) a 11 (Dezembro)
- Ano: anos desde 1900

Por exemplo, suponha que você queira definir a seguinite data:

```
var Xmas95 = new Date("December 25, 1995");
```

Então `Xmas95.getMonth()` retorna 11 e `Xmas95.getFullYear()` retorna 1995.

Os métodos `getTime` e `setTime` são úteis para comparar datas. O método `getTime` retorna o número dos milisegundos desde 1 de janeiro de 1970, às 00:00:00 para um objeto Date.

Por exemplo, o código a seguir mostra os números dos dias que ainda faltam do ano vigente:

```
var hoje = new Date();
var fimAno = new Date(1995, 11, 31, 23, 59, 59, 999); // Seta dia e mês
fimAno.setFullYear(hoje.getFullYear()); // Seta o ano para esse ano
var msPorDia = 24 * 60 * 60 * 1000; // Quantidade de milisegundos por dia
var diasRestantes = (fimAno.getTime() - hoje.getTime()) / msPorDia;
var diasRestantes = Math.round(diasRestantes); //retorna os dias restantes no ano
```

Este exemplo cria um objeto Date chamado `hoje` que contém a data de hoje. Ele, então, cria o objeto Date chamado `fimAno`e seta o ano para o ano vigente. Então, usando o número de milisegundos por dia, ele computa o número de dias entre hoje e `fimAno`, usando `getTime` e arredondando os números de dias.

O método `parse` é útil para associar valores de strings de data para objetos Date existentes. Por exemplo, o código a seguir usa `parse` e `setTime` para associar um valor de data ao objeto `IPOdate`:

```
var IPOdate = new Date();
IPOdate.setTime(Date.parse("Aug 9, 1995"));
```

No exemplo a seguir, a função `JSClock()` retorna o tempo no formato de um relógio digital.

```
function JSClock() {
  var tempo = new Date();
  var hora = tempo.getHours();
  var minuto = tempo.getMinutes();
  var segundo = tempo.getSeconds();
  var temp = "" + (hora > 12 ? hora - 12 : hora);
  if (hora == 0) temp = "12";
  temp += (minuto < 10 ? ":0" : ":") + minuto;
  temp += (segundo < 10 ? ":0" : ":") + segundo;
  temp += hora >= 12 ? " P.M." : " A.M.";
  return temp;
}
```

A função `JSClock` primeiro cria um objeto new `Date` chamado tempo; já que nenhum argumento é passado, `tempo` é criado com data e hora atuais. Ela então chama os métodos `getHours`, `getMinutes` e `getSeconds` e associa o valor à hora, minuto e segundo atuais à hora, minuto e segundo.

As próximas quatro declarações constroem uma string baseada em time. A primeira declaração cria uma variável `temp`, associando um valor utilizando uma expressão condicional; se hora é maior que 12, (hora - 12), senão simplesmente hora, a não ser que hora seja 0 que, nesse caso, será 12.

A próxima declaração anexa um valor `minuto` a `temp`. Se o valor de minuto for menos que 10, a expressão condicional acrescenta uma string com um 0 na frente; senão ela acrescenta uma string com dois pontos. Então a declaração anexa um valor `segundo` a `temp` do mesmo jeito.

Finalmente, a expressão condicional anexa "P.M." a `temp` se `hora` for 12 ou maior; senão ela anexa "A.M." a `temp`.

- Anterior
- Próxima

Expressões Regulares

- Anterior
- Próxima

Expressões regulares são padrões utilizados para selecionar combinações de caracteres em uma string. Em JavaScript, expressões regulares também são objetos. Elas podem ser utilizadas com os métodos `exec` e `test` do objeto `RegExp`, e com os métodos `match`, `replace`,`search`, e `split` do objeto `String`. Este capítulo descreve o uso de expressões regulares em JavaScript.

Criando uma Expressão Regular

Há duas maneiras de construir uma expressão regular:

Usando uma expressão literal, que consiste em um padrão fechado entre barras, como o exemplo a seguir:

```
const re = /ab+c/;
```

As expressões regulares na forma literal são compiladas quando o script é carregado. Esta forma de construção possui melhor performace quando a expressão regular utilizada é uma constante, ou seja, não muda durante a execução.

Ou chamando o construtor do objeto `RegExp`:

```
let re = new RegExp("ab+c");
```

Usando o construtor, a compilação da expressão regular é realizada em tempo de execução. Use o construtor quando souber que o padrão da expressão regular irá mudar ou quando o padrão for desconhecido, oriundo de outra fonte, uma entrada de usuário por exemplo.

> **Nota:** Se você já está familiarizado com as formas de uma expressão regular, também pode ler o resumo para uma rápida pesquisa de um padrão específico.

Escrevendo um padrão de expressão regular

Um padrão de expressão é composto por um conjunto de caracteres simples, como `/abc/`, ou uma combinação de caracteres simples e especiais, como `/ab*c/` ou `/Capitulo (\d+)\.\d*/`. O último exemplo contém parênteses, que são usados como um mecanismo de armazenamento. A correspondência feita por essa parte da expressão é armazenada para uso posterior, como descrito em: Using Parenthesized Substring Matches.

Uso de Padrões Simples

Padrões simples são construídos utilizando os caracteres que você deseja encontrar correspondências diretas. Por exemplo, o padrão `/abc/` encontra combinações de caracteres em strings apenas quando os caracteres 'abc' forem encontrados juntos e na ordem especificada. Esse padrão será encontrado com sucesso nas strings "Olá, você conhece o **abc**?" e "Os mais recentes aviões evoluíram do sl**abc**raft.". Em ambos os casos, a correspondência estará no subconjunto 'abc'. Porém, o padrão não será encontrado no texto "Grab crab" pois apesar de conter os mesmos caractes do padrão 'ab c', estes não aparecem na ordem especificada.

Uso dos Caracteres Especiais

Quando for necessário buscar algo além de uma correspondência direta, como encontrar uma ou mais ocorrências da letra 'b', ou encontrar espaços em branco, será necessário adicionar caracteres especiais ao padrão. Por exemplo, para encontrar uma única correspondência de 'a' seguido de nenhum ou mais 'b's seguido de 'c', o padrão a utilizar seria `/ab*c/`. O caractere `*` seleciona zero ou mais ocorrências do item que o precede. Se aplicada ao texto 'cbb**abbbbc**debc', essa expressão regular encontraria o subconjunto grifado no texto.

A tabela abaixo fornece uma lista completa dos caracteres especiais que podem ser utilizados nas expressões regulares, com sua respectiva descrição. Para testar os exemplos de expressão regular você pode usar o regExr.

Caracteres especiais utilizados em expressões regulares.

Caractere	Descrição
\	Aplicado conforme as seguintes regras: Uma barra invertida que preceda um caractere não especial significa que o caractere seguinte é especial e não deve ser interpretado de forma literal. Por exemplo, o caractere 'b' quando não precedido de uma barra invertida significará uma ocorrência do próprio caractere 'b' minúsculo, porém se precedido da barra invertida `'\b'` ele passará a significar a ocorrência do caractere especial [fronteira do caractere](#special-word-boundary). Quando a barra invertida preceder um caractere especial isso significará que o próximo caractere deve ser interpretado de forma literal. Por exemplo o padrão `/a*/`, que selecionará a ocorrência de zero ou mais caracteres 'a' quando utilizado sem a \ para escape. Por outro lado no padrão `/a\*/` o asterisco deixa de ter seu significado especial, pois a '\' de escape fará com que o '*' seja interpretado de forma literal, passando o padrão a selecionar o caractere 'a' seguido do caractere '*'. Quando utilizar o construtor RegExp("padrao"), não se esqueça de fazer o escape do caractere \, já que esse caractere é também utilizado como caractere de escape em strings.
^	Corresponde ao início do texto. Se a flag multilinhas é setada para true, também se aplica imediatamente após um caractere de quebra de linha. Por exemplo, `/^A/` não corresponde ao 'A' em "Um Alvo", mas corresponde ao 'A' em "Alvo Encontrado". Este caractere tem um significado diferente quando aparece como o primeiro caractere em um conjunto padrão de caracteres. Veja conjunto de caracteres negados ou complementados para detalhes e mais exemplos.
$	Corresponde ao final do texto. Se a flag multilinhas é setada para true, também se aplica imediatamente antes de um caractere de quebra de linha. Por exemplo, `/r$/` não corresponde ao 'r' em "corre", mas acha correspondência em "correr".
*	Corresponde a expressão que o precede repetida 0 ou mais vezes. Equivalente a {0,} Por exemplo, `/bo*/` acha uma correspondência para 'boo' em "boolean" e 'b' em "A bird warbled", mas nenhuma em "A goat grunted".
+	Corresponde a expressão que o precede repetido 1 ou mais vezes. Equivalente a {1,}. Por exemplo, `/a+/` acha correspondência para o 'a' em "candy" e todos os "as" em "caaaaaaandy", mas nâo encontra em "cndy".
?	Corresponde a expressão que o precede repetido 0 ou 1 vez. Equivalente à {0,1}. Por exemplo, `/e?le?/` encontra o 'el' em "angel" e o 'le' em "angle" e também o 'l' em "oslo". Se usado imediatamente após qualquer um dos quantificadores `*`, `+`, `?` ou `{}`, faz o quantificador não guloso (combinando o número mínimo de vezes), como um oposto para o padrão que é guloso (combinar o número máximo possível). Por exemplo, aplicando `/\d+/` em "123abc" encontra "123". Mas aplicando `/\d+?/`, apenas "1" será encontrado. Também usado em declarações lookahead, descritas sob `x(?=y)` e `x(?!y)`logo abaixo na tabela.
.	(O ponto decimal) corresponde com qualquer caracter, exceto o caracter de nova linha. Por exemplo, `/.n/` acha correspondência para o 'an' e 'on' em "nove dias restantes para onze de agosto.", mas não encontra 'no' em 'nove'.

Caractere	Descrição
`(x)`	Pesquisa correspondência com o caractere 'x' e memoriza-o, como a o exemplo a seguir mostra. Os parênteses são chamados *parênteses de captura*. Por exemplo, o '(foo)' e '(bar)' no padrão `/(foo) (bar) \1 \2/` encontra e memoriza a primeira das duas palavras na string "foo bar foo bar". O \1 e \2 no padrão combina as duas últimas palavras da string. Note que \1, \2, \n são utilizados na parte correspondente do regex.
`(?:x)`	Pesquisa correspondência com o caractere 'x' porém não o memoriza. Os parênteses são chamados de *parênteses de não-captura* e permitem que você defina uma subexpressão para operadores de expressão regular trabalhar com eles. Considere essa expressão de exemplo `/(?:foo){1,2}/`. Se a expressão era `/foo{1,2}/`, o {1,2} poderia ser aplicado apenas para o último 'o' em 'foo'. Com os parênteses de não-captura, o {1,2} é aplicado para toda a palavra 'foo'.
`x(?=y)`	Pesquisa correspondência em 'x' apenas se 'x' é seguido por 'y'. Isso é chamado de *lookahead*. Por exemplo, `/Jack(?=Sprat)/` busca 'Jack' apenas se é seguido por 'Sprat'. `/Jack(?=Sprat|Frost)/` busca 'Jack' apenas se ele é seguido por 'Sprat' ou 'Frost'. No entanto, 'Sprat' nem 'Frost' faz parte do resultado retornado.
`x(?!y)`	Pesquisa correspondência em 'x' apenas se 'x' não é seguido por 'y'. Isso é chamado *negação lookahead*. Por exemplo, `/\d+(?!\.)/` encontra um número apenas se ele não for seguido por um ponto decimal. A expressão regular `/\d+(?!\.)/.exec("3.141")` encontra '141' mas não '3.141'.
`x|y`	Pesquisa correspondência em 'x' ou 'y'. Por exemplo, `/verde|vermelha/` encontra 'verde' em "maçã verde" e 'vermelha' em "maçã vermelha."
`{n}`	Pesquisa n ocorrências correspondentes ao caracter precedido. Onde, `n` deve ser um inteiro positivo. Por exemplo, `/a{2}/` não encontra o 'a' em "candy," mas encontra-o se houver a quantidade de a's informarda em "caandy," e os dois primeiros a's em "caaandy."
`{n,m}`	Pesquisa a n menor correspondência e a m maior correspondência do caractere precedido. Quando n ou m é zero, ele poderá ser omitido. Onde, `n` e `m` devem ser inteiros positivo. Por exemplo, `/a{1,3}/` não encontra nada em "cndy", mas encontra o 'a' em "candy", encontra os dois primeiros a's em "caandy," e encontra os três primeiros a's em "caaaaaaandy". Observe que, ao fazer a correspondência de "caaaaaaandy", serão encontrados apenas os "aaa", mesmo que a string tenha mais a's.
`[xyz]`	Um conjunto de caracteres. Pesquisa correspondência para qualquer um dos caracteres entre colchetes. Você pode especificar um intervalo de caracteres usando hífen. Caracteres especiais (como o ponto (.) e o asterisco(*)) não tem significado algum quando está dentro de um conjunto de caracteres. Não necessita utilizar escape neles. Mas, se utilizar escape também irá funcionar. Por exemplo, `[abcd]` é o mesmo que `[a-d]`. Com a expressão será encontrado o 'b' em "beijo" e o 'c' em "chop". A expressão `/[a-z.]+/` e `/[\w.]+/` ambos encontraram as letras que formam "test.i.ng".
`[^xyz]`	Um conjunto de caracteres negados ou complementados. Isto é, combina com qualquer coisa que não esteja listado entre os colchetes. Você pode especificar um intervalo de caracteres usando hífen. Tudo que funciona no conjunto de caracteres (apresentado acima) também funciona aqui. Por exemplo, `[^abc]` é o mesmo que `[^a-c]`. Com a expressão será encontrado inicialmente 'e' em "beijo" e 'h' em "chop."
`[\b]`	Pesquisa correspondência com espaço em branco (U+0008). É preciso utilizar os colchetes se você quer encontrar um espaço em branco. (Não confunda-o com `\b`.)
`\b`	Pesquisa correspondência em uma fronteira de caractere. Uma fronteira de caractere corresponde a posição onde um caractere/palavra não é seguido ou antecedido por outro caractere/palavra. Isto é, em fronteira de caractere não pode haver nenhum caractere ou espaço, seu tamanho deve ser vazio. (não confunda-o com [\b].) Exemplos: `/\bmoo/` encontra a substring 'moo' em "moon" ; `/oo\b/` não encontra o 'oo' em "moon", devido o 'oo' ser seguido por 'n' que é um caractere; `/oon\b/` encontra a substring 'oon' em "moon", devido 'oon' ser o fim da string, ou seja, não é seguido por nenhum caractere; `/\w\b\w/` não encontrará nada, pois o caractere nunca será seguido por um não caractere e um caractere. **Nota:** O mecanismo de expressão regular no JavaScript define um conjunto específico de caracteres para serem caracteres "palavras". Qualquer caractere que não esteja neste conjunto é considerado uma quebra de palavra. Este conjunto de caractere é bastante limitado: consiste apenas no alfabeto romano tanto maiúsculo como minúsculo, digítos decimais, e o caractere sublinhado. Caracteres acentuados, tal como "é" ou "ã" são, infelizmente, tratados como palavras quebradas.

Caractere	Descrição
`\B`	Pesquisa correspondência que não seja em uma fronteira de caractere. Para a correspondência é associada uma posição onde o caractere anterior e o próximo tem as mesmas características: ambos são caractere/palavra, ou ambos não sejam caractere/palavra. O início e o fim de uma string não considerados como não caractere/palavra. Por exemplo, /\B../ encontra correspondente 'oo' em "boolean", e /y\B./ encontra correspondente 'ye' em "possibly yesterday."
`\c`*X*	Onde X é um caractere pertencente ao conjunto A-Z. Encontra correspondência de um caractere de controle em uma string. Por exemplo, `/\cM/` encontra correspondente control-M (U+000D) em uma string.
`\d`	Encontra correspondência com um número. Equivalente a [0-9]. Por exemplo, `/\d/` ou `/[0-9]/` encontra correspondente '8' em "Dróide BB8".
`\D`	Encontra correspondência com um caractere que não seja número. Equivalente a [^0-9]. Por exemplo, `/\D/` ou `/[^0-9]/` ecntra correspondente 'C' em "C3 está ativada."
`\f`	Encontra correspondência com um caractere de escape *avanço de página* (U+000C).
`\n`	Encontra correspondência com um caractere de escape *quebra de linha* (U+000A).
`\r`	Encontra correspondência com um caractere de escape *retorno de carro* (U+000D).
`\s`	Encontra correspondência com um único caractere de espaço em branco, espaço, tabulação, avanço de página, quebra de linha. Equivalente a `[\f\n\r\t\v\u00A0\u1680\u180e\u2000\u2001\u2002\u2003\u2004\u2005\u2006\u2007\u2008\u2009\u200a\u2028\u2029\u2028\u2029\u202f\u205f\u3000]`. Por exemplo, `/\s\w*/` encontra correspondente ' bar' em "foo bar."
`\S`	Encontra correspondência em um único caractere que não seja espaço em branco. Equivalente a `[^\f\n\r\t\v\u00A0\u1680\u180e\u2000\u2001\u2002\u2003\u2004\u2005\u2006\u2007\u2008\u2009\u200a\u2028\u2029\u2028\u2029\u202f\u205f\u3000]`. Por exemplo, `/\S\w*/` encontra correspondente 'foo' em "foo bar."
`\t`	Encontra correspondência em uma tabulação (U+0009).
`\v`	Encontra correspondência em uma tabulação vertical (U+000B).
`\w`	Encontra correspondência de qualquer caractere alfanumérico incluindo underline. Equivalente a `[A-Za-z0-9_]`. Por exemplo, `/\w/` encontra correspondente 'a' em "apple," '5' em "$5.28," e '3' em "3D."
`\W`	Encontra correspondência em um não caractere. Equivalente a `[^A-Za-z0-9_]`. Por exemplo, `/\W/` ou `/[^A-Za-z0-9_]/` encontra correspondente '%' em "50%."
`\`*num*	Onde num é um inteiro positivo. Faz referência a substring pertencente à um grupo, um grupo é definido entre parênteses. Grupos são numerados de 1 até 9. Por exemplo, /(muito) (cacique) pra \2 \1/ encontra 'muito cacique pra cacique muito' em 'Na aldeia tem muito cacique pra cacique muito.'
`\0`	Encontra correspondência em um caractere NULL (U+0000). Não adicione outro número após o zero, pois `\0<digitos> é um escape para número octal`.
`\xhh`	Encontra correspondência com o código hh (dois valores hexadecimal).
`\uhhhh`	Encontra correspondência com o código hhh (três valores hexadecimal).
\u{hhhh}	(funciona apenas com a flag u) Encontra correspondência com o valor Unicode hhhh (dígitos hexadecimais).

Usando Parênteses

Usar parênteses em volta de qualquer parte de uma expressão regular faz com que essa parte seja lembrada para ser usada depois, como descrito em Usando as Substrings entre Parênteses na Expressão Regular.

Por Exemplo, a expressão `/Capitulo (\d+)\.\d*/` ilustra caracteres adicionais escapados e especiais e indica que parte do padrão deve ser lembrado. Corresponde precisamente aos caracteres 'Capitulo ' seguidos por um ou mais caracteres numéricos (`\d` significa qualquer caracter numérico e `+` significa 1 ou mais vezes), seguidos por um ponto decimal (que é um caracter especial; preceder com um \ significa que a expressão regular deve buscar pelo caracter literal '.'), seguido por qualquer caracter numérico 0 ou mais vezes (`\d` significa caracter numérico, `*` significa 0 ou mais vezes). Além disso, os parenteses são usados para relembrar os primeiros caracteres numéricos correspondentes.

Esse padrão é encontrado em "Abra o capitulo 4.3, parágrafo 6" o '4' é relembrado. O padrão não é encontrado em "Capitulo 3 e 4", porque essa string não tem um período após o '3'.

Para encontrar uma substring sem que a correspondência seja relembrada, dentro dos parênteses inicie o padrão com `?:`. Por exemplo, `(?:\d+)` corresponde a um ou mais caracteres numéricos mas não relembra os caracteres correspondentes.

Trabalhando com expressões regulares

Expressões Regulares são usadas com os metodos `test` e `exec` do objeto `RegExp`e com os metodos `match`, `replace`, `search`, e `split` do objeto `String`. Estes metodos são explicados em detalhe em JavaScript Reference.

Metodo	Descrição
`exec`	Um método `RegExp` que execute uma pesquisa por uma correspondência em uma string. Retorna um array de informações.
`test`	Um método `RegExp` que testa uma correspondência em uma string. Retorna true ou false.
`match`	Um método `String` que executa uma pesquisa por uma correspondência em uma string. Retorna uma array de informações ou null caso não haja uma correspondência.
`search`	Um método `String` que testa uma correspondência em uma string. Retorna o indice da correspondência ou -1 se o teste falhar.
`replace`	Um método `String` que executa uma pesquisa por uma correspondência em uma string, e substitui a substring correspondênte por uma substring de substituição.
`split`	Um método `String` que usa uma expressão regular ou uma string fixa para quebrar uma string dentro de um array de substrings.

Quando você quer saber se um padrão é encontrado em uma string, use o método `test` ou `search`; para mais informações (mas execução mais lenta) use o método `exec` ou `match`. Se você usar `exec` ou `match` e se houver correspondência, estes métodos retornam um array e atualizam as propriedades do objeto da expressão regular associada e também do objeto da expressão regular predfinada `RegExp`. Se não houver corespondência, o método `exec` retorna `null` (convertido para `false`).

No seguinte exemplo, o script usa o método `exec` para encontrar uma correspondência em uma string.

```
var myRe = /d(b+)d/g;
var myArray = myRe.exec("cdbbdbsbz");
```

Se você não precisa acessar as propriedades da expressão regular, uma alternativa de criar `myArray` é com esse script:

```
var myArray = /d(b+)d/g.exec("cdbbdbsbz");
```

Se você quiser construir a expressão regular a partir de uma string, outra alternativa é esse script:

```
var myRe = new RegExp("d(b+)d", "g");
var myArray = myRe.exec("cdbbdbsbz");
```

Com esses scripts, a correspondência é encontrada o array é retornado e são atualizadas as propriedades mostradas na tabela a seguir.

Table 4.3 Resultados da execução de expressões regulares.

Objeto	Propriedade or indice	Descrição	Nesse exemplo
`myArray`		A string correspondente e todas as substrings relembradas.	`["dbbd", "bb"]`
	`index`	O índice do encontro em relação à string original.	`1`
	`input`	A string original.	`"cdbbdbsbz"`
	`[0]`	Os últimos caracteres encontrados.	`"dbbd"`
`myRe`	`lastIndex`	O índice no qual iniciar a próxima partida. (Esta propriedade é configurada apenas se a expressão regular usar a opção g, descrita em [Advanced Searching With Flags](#Advanced_Searching_With_Flags).)	`5`
	`source`	O texto da expressão regular. Atualizado quando a expressão é criada, não executada.	`"d(b+)d"`

Como mostrado na segunda forma deste exemplo, você pode usar uma expressão regular criada com um inicializador de objeto sem atribuí-la à uma variável. Contudo, se você o fizer, toda ocorrência é uma nova expressão regular. Assim sendo, se você usar esta forma sem atribuí-la à uma variável, você não pode subsequentemente acessar as propriedades da expressão regular. Assumamos que tenha este script, por exemplo:

```
var myRe = /d(b+)d/g;
var myArray = myRe.exec("cdbbdbsbz");
console.log("O útltimo índice é " + myRe.lastIndex);
```

Este script mostra:

```
O último índice é 5
```

Contudo, se tem este script:

```
var myArray = /d(b+)d/g.exec("cdbbdbsbz");
console.log("O último índice é " + /d(b+)d/g.lastIndex);
```

Ele mostra:

```
O último índice é 0
```

As ocorrências de `/d(b+)d/g` nas duas declarações são objetos expressões regulares diferentes e consequentemente têm diferentes valores para suas propriedades `lastIndex`. Se precisa acessar as propriedades de uma expressão regular criada com um inicializaor de objeto, você deve primeiro atribuí-la à uma variável.

Usando as Substrings entre Parênteses na Expressão Regular

A inclusão de parênteses em um padrão de expressão regular faz com que a sub-correspondência correspondente seja lembrada. Por exemplo, `/a(b)c/` corresponde aos caracteres 'abc' e lembra 'b'. Para recuperar essas correspondências de substring entre parênteses, use o `Arra[1]`, ..., `[n]`.

O número de substring entre parênteses possíveis é ilimitado. A matriz retornada contém tudo o que foi encontrado. Os exemplos a seguir ilustram como usar parênteses entre parênteses.

Exemplo 1

O script a seguir usa o método `replace()` para alternar as palavras na string. Para o texto de substituição, o script usa o `$1` e `$2` na substituição para indicar a primeira e a segunda correspondências de substring entre parênteses.

```
var re = /(\w+)\s(\w+)/;
var str = "John Smith";
var newstr = str.replace(re, "$2, $1");
console.log(newstr);
```

Isto imprime "Smith, John".

Pesquisa avançada com Flags

As expressões regulares possuem quatro flags opcionais as quais se incluem a pesquisa global e case insensitive. Para realizar uma pesquisa global, utilize a flag g. Para realizar uma pesquisa sem diferenciar letras maiúsculas de minúsculas, utilize a flag i. Para realizar uma pesquisa multi-linhas, utilize a flag m. Ao realizar uma pesquisa "sticky", o ponto de partida será a posição corrente da string alvo, use com a flag y.Estas flags podem ser usadas separadamente ou juntas, em qualquer ordem, e serão inclusas como parte da expressão regular.

Para incluir um sinalizador com a expressão regular, use esta sintaxe:

```
var re = /pattern/flags;
```

ou

```
var re = new RegExp("pattern", "flags");
```

Observe que os sinalizadores são parte integrante de uma expressão regular. Eles não podem ser adicionados ou removidos posteriormente.

Por exemplo, `re = /\w+\s/g` cria uma expressão regular que procura um ou mais caracteres seguidos por um espaço e procura essa combinação em toda a cadeia.

```
var re = /\w+\s/g;
var str = "fee fi fo fum";
var myArray = str.match(re);
console.log(myArray);
```

Isso exibe ["fee ", "fi ", "fo "]. Neste exemplo, você pode substituir a linha

```
var re = /\w+\s/g;
```

por:

```
var re = new RegExp("\\w+\\s", "g");
```

e obtenha o mesmo resultado.

O sinalizador `m` é usado para especificar que uma sequência de entrada de múltiplas linhas deve ser tratada como várias linhas. Se o sinalizador `m` for usado, `^` e `$` combinam no início ou no final de qualquer linha na sequência de entrada, em vez do início ou no final de toda a sequência.

Exemplos

Os exemplos que se seguem mostram mais usos expressões regulares.

Alterando o pedido em uma string de entrada

O exemplo a seguir ilustra a formação de expressões regulares e o uso de `string.split()` e `string.replace()`. Ela limpa uma string de entrada formatada com nomes (primeiro nome sobrenome) separados por espaço em branco, tabulações e exatamente um ponto e virgula. Por fim, inverte a ordem do nome (sobrenome e primeiro nome) e ordena a lista

```
// A cadeia de nomes contém vários espaços e guias,
// e pode ter vários espaços entre o nome e o sobrenome.
var names = "Harry Trump ;Fred Barney; Helen Rigby ; Bill Abel ; Chris Hand ";

var output = ["---------- String original\n", names + "\n"];

// Prepare dois padrões de expressão regular e armazenamento em array.
// Divide a string em elementos da matriz.

// pattern: possível espaço em branco, em seguida, ponto e vírgula, em seguida, possível espaço em branco
var pattern = /\s*;\s*/;
```

```
// Quebra string em pedaços separados pelo padrão acima e
// armazene as partes em uma matriz chamada nameList
var nameList = names.split(pattern);

// new pattern: um ou mais caracteres, espaços e caracteres.
// Use parênteses para "memorizar" partes do padrão.
// As partes memorizadas são referenciadas mais tarde.
pattern = /(\w+)\s+(\w+)/;

// Nova matriz para armazenar nomes sendo processados.
var bySurnameList = [];

// Exibe a matriz de nomes e preenche a nova matriz
// com nomes separados por vírgula, último primeiro.
//
// O método replace remove qualquer coisa que corresponda ao padrão
// e substitui-o pela sequência memorizada - segunda parte memorizada
// seguido por espaço de vírgula seguido pela primeira parte memorizada.
//
// As variáveis $ 1 e $ 2 se referem às partes
// memorizado enquanto corresponde ao padrão.

output.push("---------- Após Separar pela Expressão Regular");

var i, len;
for (i = 0, len = nameList.length; i < len; i++) {
  output.push(nameList[i]);
  bySurnameList[i] = nameList[i].replace(pattern, "$2, $1");
}

// Exibe a nova matriz.
output.push("---------- Nomes Invertidos");
for (i = 0, len = bySurnameList.length; i < len; i++) {
  output.push(bySurnameList[i]);
}

// Classifica pelo sobrenome e exibe a matriz classificada.
bySurnameList.sort();
output.push("---------- Ordenado");
for (i = 0, len = bySurnameList.length; i < len; i++) {
  output.push(bySurnameList[i]);
}

output.push("---------- Fim");

console.log(output.join("\n"));
```

Usando caracteres especiais para verificar entradas

No exemplo a seguir, é esperado que o usuário informe um número de telefone. Quando o usuário pressionar o botão "Check", o script verificará a validade do número. Se o número for válido (a sequência de caracteres especificada corresponderá a expressão regular), então, o script exibe uma mensagem agradecendo o usuário e confirmando o seu número. Se o número for inválido, o script informa ao usuário que o número de telefone não é válido.

A expressão regular procura por zero ou uma ocorrência de parênteses de abertura \(?, seguido de três dígitos \d{3}, seguido de zero ou uma ocorrência de parênteses de fechamento \)?, seguido de um hífen, barra ou ponto decimal e, quando encontrado, guarda o caractere ([-\/\.]), seguido de três dígitos \d{3}, seguido por um caractere de hífen, barra ou ponto decimal que fora guardado \1, seguido por quatro dígitos \d{4}.

Com o evento Change ativo, quando o usuário pressionar Enter, o valor será capturado por RegExp.input.

```
<!doctype html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1" />
    <meta http-equiv="Content-Script-Type" content="text/javascript" />
    <script type="text/javascript">
      var re = /\(?\d{3}\)?([-\/\.])\d{3}\1\d{4}/;
      function testInfo(phoneInput) {
        var OK = re.exec(phoneInput.value);
        if (!OK)
          window.alert(
            RegExp.input + " Não é um número de telefone com código de área!",
          );
        else window.alert("Obrigado, o seu número de telefone é " + OK[0]);
      }
    </script>
  </head>
  <body>
    <p>
      Informe o seu número de telefone (com código de área) e então clique em
      "Check". <br />O formato esperado é ###-###-####.
    </p>
    <form action="#">
      <input id="phone" /><button
        onclick="testInfo(document.getElementById('phone'));">
```

```
        Check
      </button>
    </form>
  </body>
</html>
```

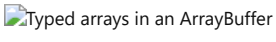
- Anterior
- Próxima

Arrays tipados no JavaScript

Os arrays tipados do Javascript são objetos parecidos com arrays convencionais, e fornecem um mecanismo para lidar com dados binários em sua forma "bruta". Como você já deve saber, objetos do tipo `Array` aumentam e encolhem dinamicamente, e podem conter qualquer tipo de valor possível no Javascript. Os interpretadores Javascript realizam otimizações para que estes arrays sejam rápidos de se manipular. Todavia, à medida em que as aplicações web tornam-se mais e mais poderosas, adicionando recursos como manipulação de áudio e vídeo, acesso a dados brutos utilizando WebSockets, e assim por diante, tornou-se claro que há momentos em que seria útil ao código Javascript ser capaz de manipular dados binários "brutos" em arrays tipados, de forma rápida e fácil.Entretanto, os arrays tipados não devem ser confundidos com arrays convencionais, já que invocar o método `Array.isArray()` em um array tipado retornará `false`. Além disso, nem todos os métodos disponíveis para arrays normais são suportados pelos arrays tipados (ex. push e pop).

Buffers e views: a arquitetura dos arrays tipados

Para alcançar a máxima flexibilidade e eficiência, os arrays tipados do Javascript dividem sua implementação entre **buffers** e **views**. Um buffer (implementado pelo objeto `ArrayBuffer`) é um objeto que representa um fragmento (chunk) de dados; Não possui formato para ser discutido, e não oferece mecanismos para acessar seu conteúdo. Para acessar os dados contidos em um buffer, você precisa utilizar uma view. Uma view fornece um contexto — isto é, um tipo de dado, o deslocamento inicial, e o número de elementos — que transforma estes dados em um array tipado.


Typed arrays in an ArrayBuffer

ArrayBuffer

O `ArrayBuffer` é um tipo de dado que é utilizado para representar um buffer genérico de dados binários, com comprimento fixo. Você não pode manipular diretamente o conteúdo de um `ArrayBuffer`; ao invés, você cria uma view de array tipado ou um objeto do tipo `DataView`, que representa o buffer em um formato específico, e os utiliza para ler e escrever no conteúdo do buffer.

Views de arrays tipados

As views de arrays tipados possuem nomes auto descritivos e fornecem exibições para todos os tipos numéricos habituais, como `Int8`, `Uint32`, `Float64` e assim por diante. Há uma view de array tipado especial, a view `Uint8ClampedArray`. Ela comporta os valores entre `0` e `255`. Isto é útil no processamento de dados do Canvas, por exemplo.

DataView

O objeto do tipo `DataView` é uma interface de baixo nível que fornece uma API no estilo *getter/setter* para ler e escrever dados arbitrários ao buffer. Isto é útil quando se está lidando com diferentes tipos de dados, por exemplo. As views de arrays tipados estão em ordem de bytes nativa (veja Endianness) da sua plataforma. Com uma `DataView` você é capaz de controlar a ordem dos bytes. Esta ordem é *big-endian* por padrão e pode ser alterada para *little-endian* através dos métodos *getters* e *setters* citados anteriormente.

Web APIs que utilizam arrays tipados

`FileReader.prototype.readAsArrayBuffer()`

> O método `FileReader.prototype.readAsArrayBuffer()` inicia a leitura do conteúdo de um objeto do tipo Blob ou FIle especificado.

`XMLHttpRequest.prototype.send()`

> O método `send()` das instâncias de `XMLHttpRequest` agora suportam arrays tipados e objetos do tipo `ArrayBuffer` como argumento.

`ImageData.data`

> É objeto do tipo `Uint8ClampedArray` que representa uma matriz unidimensional que contém dados na ordem RGBA, com valores numéricos entre `0` e `255` inclusive.

Exemplos

Utilizando views com buffers

Primeiramente, nós precisamos criar um buffer, com um tamanho fixo de 16 bytes neste caso:

```
var buffer = new ArrayBuffer(16);
```

Neste ponto, nós temos um fragmento de memória cujos bytes estão todos pré-inicializados em `0`. Não há muito que possamos fazer com isto, no entanto. Nós podemos confirmar que possuem de fato 16 bytes de comprimento, e isso é tudo sobre ele:

```
if (buffer.byteLength === 16) {
  console.log("Sim, são 16 bytes.");
} else {
  console.log("Ah não, o tamanho não está certo!");
}
```

Antes de realmente começarmos a trabalhar com este buffer, precisamos criar uma view. Vamos criar uma view que trata os dados neste buffer como um array de inteiros assinados de 32 bits:

```
var int32View = new Int32Array(buffer);
```

Agora nós podemos acessar os campos neste array assim como em um array convencional:

```
for (var i = 0; i < int32View.length; i++) {
  int32View[i] = i * 2;
}
```

Isto preenche as 4 posições do array (4 posições com 4 bytes cada uma, produzindo um total de 16 bytes) com os valores `0`, `2`, `4` e `6`.

Multiplas views dos mesmos dados

As coisas começar a ficar realmente interessantes quando você considera que é possível criar múltiplas views para os mesmos dados. Por exemplo, dado o código acima, nós podemos continuar desta forma:

```
var int16View = new Int16Array(buffer);

for (var i = 0; i < int16View.length; i++) {
  console.log("Posição " + i + ": " + int16View[i]);
}
```

Aqui nós criamos uma view de inteiros de 16 bits que compartilha o mesmo buffer que a view de 32 bits existente, e emitimos todos os valores no buffer como inteiros de 16 bits. Agora nós obtemos a saída `0`, `0`, `2`, `0`, `4`, `0`, `6`, `0`.

Você pode ir um pouco mais longe, no entanto. Considere isto:

```
int16View[0] = 32;
console.log("Entra 0 na matriz de 32 bits agora é " + int32View[0]);
```

A saída disso é "Entrada 0 na matriz de 32 bits agora é 32". Em outras palavras, as duas matrizes são, na verdade, apenas vistas no mesmo buffer de dados, tratando-as como formatos diferentes. Você pode fazer isso com qualquer tipo de visualização.

Trabalhando com Estruturas de Dados Complexas

Ao combinar um único buffer com várias exibições de diferentes tipos, iniciando em diferentes deslocamentos no buffer, você pode interagir com objetos de dados contendo vários tipos de dados. Isso permite, por exemplo, interagir com estruturas de dados complexas de WebGL, arquivos de dados ou estruturas C que você precisa usar ao usar js-ctypes.

Considere esta estrutura em C:

```
struct someStruct {
  unsigned long id;
  char username[16];
  float amountDue;
};
```

Você pode acessar um buffer contendo dados neste formato assim:

```
var buffer = new ArrayBuffer(24);

// ... leia os dados no buffer ...

var idView = new Uint32Array(buffer, 0, 1);
var usernameView = new Uint8Array(buffer, 4, 16);
var amountDueView = new Float32Array(buffer, 20, 1);
```

Então você pode acessar, por exemplo, o devido valor com `amountDueView[0]`.

> **Nota:** O alinhamento da estrutura de dados em uma estrutura em C depende da plataforma. Tome precauções e considerações para essas diferenças de preenchumento.

Conversão para arrays normais

Depois de processar uma array tipada, às vezes é útil convertê-la de volta em uma array normal para se beneficiar do `Array` prototype. Isso pode ser feito usando `Array.from`, ou usando o seguinte código onde `Array.from` não é suportado.

```
var typedArray = new Uint8Array([1, 2, 3, 4]),
  normalArray = Array.prototype.slice.call(typedArray);
normalArray.length === 4;
normalArray.constructor === Array;
```

Veja também

- Getting `ArrayBuffer`s or typed arrays from *Base64*-encoded strings
- `StringView` – a C-like representation of strings based on typed arrays
- Faster Canvas Pixel Manipulation with Typed Arrays
- Typed Arrays: Binary Data in the Browser
- Endianness

Usando promises

- Anterior
- Próxima

Uma `Promise` é um objeto que representa a eventual conclusão ou falha de uma operação assíncrona. Como a maioria das pessoas consomem promises já criadas, este guia explicará o consumo de promises devolvidas antes de explicar como criá-las.

Essencialmente, uma promise é um objeto retornado para o qual você adiciona callbacks, em vez de passar callbacks para uma função.

Por exemplo, em vez de uma função old-style que espera dois callbacks, e chama um deles em uma eventual conclusão ou falha:

```
function successCallback(result) {
  console.log("It succeeded with " + result);
}

function failureCallback(error) {
  console.log("It failed with " + error);
}

doSomething(successCallback, failureCallback);
```

...funções modernas retornam uma promise e então você pode adicionar seus callbacks:

```
const promise = doSomething();
promise.then(successCallback, failureCallback);
```

...ou simplesmente:

```
doSomething().then(successCallback, failureCallback);
```

Nós chamamos isso de *chamada de função assíncrona*. Essa convenção tem várias vantagens. Vamos explorar cada uma delas.

Garantias

Ao contrário dos callbacks com retornos de funções old-style, uma promise vem com algumas garantias:

- Callbacks nunca serão chamados antes da conclusão da execução atual do loop de eventos do JavaScript.
- Callbacks adicionadas com .then mesmo *depois* do sucesso ou falha da operação assíncrona, serão chamadas, como acima.
- Multiplos callbacks podem ser adicionados chamando-se .then várias vezes, para serem executados independentemente da ordem de inserção.

Mas o benefício mais imediato das promises é o encadeamento.

Encadeamento

Uma necessidade comum é executar duas ou mais operações assíncronas consecutivas, onde cada operação subsequente começa quando a operação anterior é bem sucedida, com o resultado do passo anterior. Nós conseguimos isso criando uma *cadeia de promises*.

Aqui está a mágica: a função `then` retorna uma nova promise, diferente da original:

```
const promise = doSomething();
const promise2 = promise.then(successCallback, failureCallback);
```

ou

```
const promise2 = doSomething().then(successCallback, failureCallback);
```

Essa segunda promise representa a conclusão não apenas de `doSomething()`, mas também do `successCallback` ou `failureCallback` que você passou, que podem ser outras funções assíncronas que retornam uma promise. Quando esse for o caso, quaisquer callbacks adicionados a `promise2` serão enfileiradas atrás da promise retornada por `successCallback` ou `failureCallback`.

Basicamente, cada promise representa a completude de outro passo assíncrono na cadeia.

Antigamente, realizar operações assíncronas comuns em uma linha levaria à clássica pirâmide da desgraça:

```
doSomething(function (result) {
  doSomethingElse(
    result,
    function (newResult) {
      doThirdThing(
        newResult,
        function (finalResult) {
          console.log("Got the final result: " + finalResult);
        },
        failureCallback,
      );
    },
    failureCallback,
  );
}, failureCallback);
```

Ao invés disso, com funções modernas, nós atribuímos nossas callbacks às promises retornadas, formando uma *cadeia de promise*:

```
doSomething()
  .then(function (result) {
    return doSomethingElse(result);
  })
  .then(function (newResult) {
    return doThirdThing(newResult);
  })
  .then(function (finalResult) {
    console.log("Got the final result: " + finalResult);
  })
  .catch(failureCallback);
```

Os argumentos para `then` são opcionais, e `catch(failureCallback)` é uma abreviação para `then(null, failureCallback)`. Você pode também pode ver isso escrito com arrow functions:

```
doSomething()
  .then((result) => doSomethingElse(result))
  .then((newResult) => doThirdThing(newResult))
  .then((finalResult) => {
    console.log(`Got the final result: ${finalResult}`);
  })
  .catch(failureCallback);
```

Importante: Sempre retorne um resultado, de outra forma as callbacks não vão capturar o resultado da promise anterior.

Encadeando depois de um catch

É possivel encadear *depois* de uma falha, i.e um `catch`. Isso é muito útil para realizar novas ações mesmo depois de uma falha no encadeamento. Leia o seguinte exemplo:

```
new Promise((resolve, reject) => {
  console.log("Initial");

  resolve();
})
  .then(() => {
    throw new Error("Something failed");

    console.log("Do this");
  })
  .catch(() => {
    console.log("Do that");
  })
  .then(() => {
    console.log("Do this whatever happened before");
  });
```

Isso vai produzir o seguinte texto:

```
Initial
Do that
Do this whatever happened before
```

Observe que o texto "Do this" não foi impresso por conta que o erro "Something failed" causou uma rejeição.

Propagação de erros

Na pirâmide da desgraça vista anteriormente, você pode se lembrar de ter visto `failureCallback` três vezes, em comparação a uma única vez no fim da corrente de promises:

```
doSomething()
  .then((result) => doSomethingElse(result))
  .then((newResult) => doThirdThing(newResult))
  .then((finalResult) => console.log(`Got the final result: ${finalResult}`))
  .catch(failureCallback);
```

Basicamente, uma corrente de promises para se houver uma exceção, procurando por catch handlers no lugar. Essa modelagem de código segue bastante a maneira de como o código síncrono funciona:

```
try {
  const result = syncDoSomething();
  const newResult = syncDoSomethingElse(result);
  const finalResult = syncDoThirdThing(newResult);
  console.log(`Got the final result: ${finalResult}`);
} catch (error) {
  failureCallback(error);
}
```

Essa simetria com código assíncrono resulta no *syntactic sugar* `async/await` presente no ECMAScript 2017:

```
async function foo() {
  try {
```

```
    const result = await doSomething();
    const newResult = await doSomethingElse(result);
    const finalResult = await doThirdThing(newResult);
    console.log(`Got the final result: ${finalResult}`);
  } catch (error) {
    failureCallback(error);
  }
}
```

É construído sobre promises, por exemplo, `doSomething()` é a mesma função que antes. Leia mais sobre a sintaxe aqui.

Por pegar todos os erros, até mesmo exceções jogadas(*thrown exceptions*) e erros de programação, as promises acabam por solucionar uma falha fundamental presente na pirâmide da desgraça dos callbacks. Essa característica é essencial para a composição funcional das operações assíncronas.

Criando uma Promise em torno de uma callback API antiga

Uma `Promise` pode ser criada do zero utilizando o seu construtor. Isto deve ser necessário apenas para o envolvimento de APIs antigas.

Em um mundo ideal, todas as funções assíncronas já retornariam promises. Infelizmente, algumas APIs ainda esperam que os retornos de sucesso e/ou falha sejam passados da maneira antiga. O exemplo por excelência é o `setTimeout()` function:

```
setTimeout(() => saySomething("10 seconds passed"), 10000);
```

Misturar chamadas de retorno e promises de *old-style* é problemático. Se `saySomething` falhar ou contiver um erro de programação, nada o captura.

Por sorte nós podemos envolvê-la em uma promise. É uma boa prática envolver funções problemáticas no menor nivel possível, e nunca chamá-las diretamente de novo:

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

wait(10000)
  .then(() => saySomething("10 seconds"))
  .catch(failureCallback);
```

Basicamente, um construtor de promises pega uma função executora que nos deixa resolver ou rejeitar uma promise manualmente. Desde que `setTimeout` não falhe, nós deixamos a rejeição de fora neste caso.

Composição

`Promise.resolve()` e `Promise.reject()` são atalhos para se criar manualmente uma promise que já foi resolvida ou rejeitada, respectivamente. Isso pode ser útil em algumas situações.

`Promise.all()` e `Promise.race()` são duas ferramentas de composição para se executar operações assíncronas em paralelo.

Uma composição sequencial é possível usando JavaScript de uma forma esperta:

```
[func1, func2].reduce((p, f) => p.then(f), Promise.resolve());
```

Basicamente reduzimos um vetor de funções assíncronas a uma cadeia de promises equivalentes a: `Promise.resolve().then(func1).then(func2);`

Isso também pode ser feito com uma função de composição reutilizável, que é comum em programação funcional:

```
const applyAsync = (acc, val) => acc.then(val);
const composeAsync =
  (...funcs) =>
  (x) =>
    funcs.reduce(applyAsync, Promise.resolve(x));
```

A função composeAsync aceitará qualquer número de funções como argumentos e retornará uma nova função que aceita um valor inicial a ser passado pelo pipeline de composição. Isso é benéfico porque alguma, ou todas as funções, podem ser assíncronas ou síncronas, e é garantido de que serão executadas na ordem correta.

```
const transformData = composeAsync(func1, asyncFunc1, asyncFunc2, func2);
transformData(data);
```

No ECMAScript 2017, uma composição sequencial pode ser feita de forma mais simples com async/await:

```
for (const f of [func1, func2]) {
  await f();
}
```

Cronometragem

Para evitar surpresas, funções passadas para `then` nunca serão chamadas sincronamente, mesmo com uma função já resolvida:

```
Promise.resolve().then(() => console.log(2));
console.log(1); // 1, 2
```

Ao invés de rodar imediatamente, a função passada é colocada em uma micro tarefa, o que significa que ela roda depois que a fila estiver vazia no final do atual processo de evento de loop do Javascript, ou seja: muito em breve:

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

wait().then(() => console.log(4));
Promise.resolve()
  .then(() => console.log(2))
  .then(() => console.log(3));
console.log(1); // 1, 2, 3, 4
```

Ver também

- `Promise.then()`
- Promises/A+ specification
- Venkatraman.R - JS Promise (Part 1, Basics)
- Venkatraman.R - JS Promise (Part 2 - Using Q.js, When.js and RSVP.js)
- Venkatraman.R - Tools for Promises Unit Testing
- Nolan Lawson: We have a problem with promises — Common mistakes with promises

- Anterior
- Próxima

Trabalhando com objetos

A linguagem JavaScript é projetada com base em um simples paradigma orientado a objeto. Um objeto é uma coleção de propriedades, e uma propriedade é uma associação entre um nome (ou *chave*) e um valor. Um valor de propriedade pode ser uma função, que é então considerada um *método* do objeto. Além dos objetos que são pré-definidos no browser, você pode definir seus próprios objetos.

Este capítulo descreve como usar objetos, propriedades, funções, e métodos, e como criar seus próprios objetos.

Visão geral de objetos

Objetos em JavaScript, assim como em muitas outras linguagens de programação, podem ser comparados com objetos na vida real. O conceito de objetos em JavaScript pode ser entendido com objetos tangíveis da vida real.

Em JavaScript, um objeto é uma entidade independente, com propriedades e tipos. Compare-o com uma xícara, por exemplo. Uma xícara é um objeto, com propriedades. Uma xícara tem uma cor, uma forma, peso, um material de composição, etc. Da mesma forma, objetos em JavaScript podem ter propriedades, que definem suas características.

Objetos e propriedades

Um objeto em JavaScript tem propriedades associadas a ele. Uma propriedade de um objeto pode ser explicada como uma variável que é ligada ao objeto. Propriedades de objetos são basicamente as mesmas que variáveis normais em JavaScript, exceto pelo fato de estarem ligadas a objetos. As propriedades de um objeto definem as características do objeto. Você acessa as propriedades de um objeto com uma simples notação de ponto:

```
nomeDoObjeto.nomeDaPropriedade;
```

Como as variáveis em JavaScript, o nome do objeto (que poderia ser uma variável normal) e um nome de propriedade diferem em maiúsculas/minúsculas (por exemplo, cor e Cor são propriedades diferentes). Você pode definir uma propriedade atribuindo um valor a ela. Por exemplo, vamos criar um objeto chamado `meuCarro` e dar a ele propriedades chamadas `fabricacao`, `modelo`, e `ano`, conforme mostrado a seguir:

```
var meuCarro = new Object();
meuCarro.fabricacao = "Ford";
meuCarro.modelo = "Mustang";
meuCarro.ano = 1969;
```

Propriedades não definidas de um objeto são `undefined` (e não `null`).

```
meuCarro.semPropriedade; //undefined
```

Propriedades de objetos em JavaScript podem também ser acessadas ou alteradas usando-se notação de colchetes. Objetos são às vezes chamados de *arrays associativos*, uma vez que cada propriedade é associada com um valor de string que pode ser usado para acessá-la. Então, por exemplo, você poderia acessar as propriedades do objeto `meuCarro` como se segue:

```
meuCarro["fabricacao"] = "Ford";
meuCarro["modelo"] = "Mustang";
meuCarro["ano"] = 1969;
```

Um nome de propriedade de um objeto pode ser qualquer string JavaScript válida, ou qualquer coisa que possa ser convertida em uma string, incluindo uma string vazia. No entanto, qualquer nome e propriedade que não é um identificador JavaScript válido (por exemplo, um nome de propriedade que tem um espaço ou um hífen, ou que começa com um número) só pode ser acessado(a) usando-se a notação de colchetes. Essa notação é também muito útil quando nomes de propriedades devem ser determinados dinamicamente (quando o nome da propriedade não é determinado até o momento de execução). Exemplos são mostrados a seguir:

```
var meuObj = new Object(),
  str = "minhaString",
  aleat = Math.random(),
  obj = new Object();

meuObj.tipo = "Sintaxe de ponto";
meuObj["data de criacao"] = "String com espaco";
meuObj[str] = "valor de String";
```

```
meuObj[aleat] = "Numero Aleatorio";
meuObj[obj] = "Objeto";
meuObj[""] = "Mesmo uma string vazia";

console.log(meuObj);
```

Você pode também acessar propriedades usando um valor de string que está armazenado em uma variável:

```
var nomeDaPropriedade = "fabricacao";
meuCarro[nomeDaPropriedade] = "Ford";

nomeDaPropriedade = "modelo";
meuCarro[nomeDaPropriedade] = "Mustang";
```

Você pode usar a notação de colchetes com o comando for...in para iterar por todas as propriedades enumeráveis de um objeto. Para ilustrar como isso funciona, a seguinte função mostra as propriedades de um objeto quando você passa o objeto e o nome do objeto como argumentos para a função:

```
function mostrarProps(obj, nomeDoObj) {
  var resultado = "";
  for (var i in obj) {
    if (obj.hasOwnProperty(i)) {
      resultado += nomeDoObj + "." + i + " = " + obj[i] + "\n";
    }
  }
  return resultado;
}
```

Então, a chamada de função `mostrarProps(meuCarro, "meuCarro")` retornaria o seguinte:

```
meuCarro.fabricacao = Ford
meuCarro.modelo = Mustang
meuCarro.ano = 1969
```

Objetos: tudo

Em JavaScript, quase tudo é um objeto. Todos os tipos primitivos - com exceção de `null` e `undefined` - são tratados como objetos. Eles podem receber propriedades (propriedades atribuídas de alguns tipos não são persistentes), e possuem todas as características de objetos.

Enumerando todas as propriedades de um objeto

Começando com a ECMAScript 5, há três formas nativas de se listar (ou "caminhar por") as propriedades de um objeto:

- for...in loops Esse método caminha por todas as propriedades enumeráveis de um objeto e sua cadeia de protótipos
- Object.keys(o) Esse método retorna um array com todos os nomes ("chaves") de propriedades próprios de um objeto `o` (mas não na cadeia de protótipos).
- Object.getOwnPropertyNames(o) Esse método retorna um array contendo todos os nomes de propriedades próprios (enumeráveis ou não) de um objeto `o`.

Antes, na ECMAScript 5, não existia uma forma nativa de se listar todas as propriedades de um objeto. No entanto, isso pode ser feito com a seguinte função:

```
function listarTodasAsPropriedades(o) {
  var objectoASerInspecionado;
  var resultado = [];

  for (
    objectoASerInspecionado = o;
    objectoASerInspecionado !== null;
    objectoASerInspecionado = Object.getPrototypeOf(objectoASerInspecionado)
  ) {
    resultado = resultado.concat(
      Object.getOwnPropertyNames(objectoASerInspecionado),
    );
  }

  return resultado;
}
```

Isso pode ser útil para revelar propriedades "escondidadas" (propriedades na cadeia de protótipos que não são acessíveis através do objeto, porque outra propriedade possui o mesmo nome anteriormente na cadeia de protótipos). A listagem de propriedades acessíveis só pode ser facilmente feita através da remoção de valores duplicados no array.

Criando novos objetos

JavaScript possui um número de objetos pré-definidos. Além disso, você pode criar seus próprios objetos. Você pode criar um objeto usando um objeto inicializador. Alternativamente, você pode primeiro criar uma função construtora e depois instanciar um objeto usando aquela função e o operador `new`.

Usando inicializadores de objeto

Além de criar objetos usando uma função construtora, você pode criar objetos usando um inicializador de objeto. O uso de inicializadores de objeto é às vezes conhecido como criar objetos com notação literal. O termo "inicializador de objeto" é consistente com a terminologia usada por C++.

A sintaxe para um objeto usando-se um inicializador de objeto é:

```
var obj = {
  propriedade_1: valor_1, // propriedade_# pode ser um identificador...
  2: valor_2, // ou um numero...
  // ...,
  "propriedade n": valor_n,
}; // ou uma string
```

onde `obj` é o nome do novo objeto, cada `propriedade_i` é um identificador (um nome, um número, ou uma string literal), e cada `valor_i` é uma expressão cujo valor é atribuído à `propriedade_i`. O `obj` e a atribuição são opcionais; se você não precisa fazer referência a esse objeto em nenhum outro local, você não precisa atribuí-lo a uma variável. (Note que você pode precisar colocar o objeto literal entre parentêses se o objeto aparece onde um comando é esperado, de modo a não confundir o literal com uma declaração de bloco.)

Se um objeto é criado com um inicializador de objeto em um script de alto nível, JavaScript interpreta o objeto a cada vez que avalia uma expressão contendo o objeto literal. Além disso, um inicializador usado em uma função é criado toda vez que a função é chamada.

O seguinte comando cria um objeto e o atribui à variável `x` somente se a expressão `cond` é verdadeira.

```
if (cond) var x = { hi: "there" };
```

O seguinte exemplo cria `minhaHonda` com três propriedades. Note que a propriedade `motor` é também um objeto com suas próprias propriedades.

```
var minhaHonda = {
  cor: "vermelho",
  rodas: 4,
  motor: { cilindros: 4, tamanho: 2.2 },
};
```

Você pode também usar inicializadores de objeto para criar arrays. Veja arrays literais.

Usando uma função construtora

Alternativamente, você pode criar um objeto com estes dois passos:

1. Defina o tipo de objeto escrevendo uma função construtora. Há uma forte convenção, e com boa razão, de se usar uma letra inicial maiúscula.
2. Crie uma instância do objeto com `new`.

Para definir um tipo de objeto, crie uma função para o tipo de objeto que especifique seu nome, suas propriedades e seus métodos. Por exemplo, suponha que você queira criar um tipo objeto para carros. Você quer que esse tipo de objeto seja chamado `carro`, e você quer ele tenha propriedades de marca, modelo, e ano. Para fazer isto, você escreveria a seguinte função:

```
function Carro(marca, modelo, ano) {
  this.marca = marca;
  this.modelo = modelo;
  this.ano = ano;
}
```

Note o uso de `this` para atribuir valores às propriedades do objeto com base nos valores passados para a função.

Agora você pode criar um objeto chamado `meucarro` como se segue:

```
var meucarro = new Carro("Eagle", "Talon TSi", 1993);
```

Esse comando cria `meucarro` e atribui a ele valores especificados para suas propriedade. Então o valor de `meucarro.marca` é a string "Eagle", `meucarro.ano` é o inteiro 1993, e assim por diante.

Você pode criar qualquer número de objetos `carro` com o uso de `new`. Exemplo,

```
var carroDeKen = new Carro("Nissan", "300ZX", 1992);
var carroDeVPG = new Carro("Mazda", "Miata", 1990);
```

Um objeto pode ter uma propriedade que por si só também é um objeto. Por exemplo, suponha que você define um objeto chamado `pessoa` como se segue:

```
function Pessoa(nome, idade, sexo) {
  this.nome = nome;
  this.idade = idade;
  this.sexo = sexo;
}
```

e então você instancia dois novos objetos `pessoa` da seguinte forma:

```
var jose = new Pessoa("Jose Silva", 33, "M");
var paulo = new Pessoa("Paulo Santos", 39, "M");
```

Então, você pode reescrever a definição de `carro` de modo a incluir uma propriedade `dono` que recebe um objeto `pessoa`, como se segue:

```
function Carro(marca, modelo, ano, dono) {
  this.marca = marca;
  this.modelo = modelo;
  this.ano = ano;
```

```
  this.dono = dono;
}
```

Para instanciar os novos objetos, você então usa o seguinte:

```
var carro1 = new Carro("Eagle", "Talon TSi", 1993, jose);
var carro2 = new Carro("Nissan", "300ZX", 1992, paulo);
```

Perceba que ao invés de passar uma string literal ou um valor inteiro na hora de criar os novos objetos, os comandos acima passam os objetos `jose` e `paulo` como os argumentos para os donos. Então se você quiser descobrir o nome do dono de `carro2`, você pode acessar a seguinte propriedade:

```
carro2.dono;
```

Note que você pode sempre adicionar uma propriedade a um objeto definido anteriormente. Por exemplo, o comando

```
carro1.cor = "preto";
```

adiciona uma propriedade `cor` ao `carro1`, e dá a ele o valor `"preto."` No entanto, isso não afeta nenhum outro objeto. Para adicionar a nova propriedade a todos os objetos do mesmo tipo, você deve adicionar a propriedade na definição do tipo de objeto `carro`.

Usando o método Object.create

Objetos podem também ser criados usando-se o método `Object.create()`. Esse método pode ser muito útil, pois permite que você escolha o objeto protótipo para o objeto que você quer criar, sem a necessidade de se definir uma função construtora.

```
// Encapsulamento das propriedades e métodos de Animal
var Animal = {
  tipo: "Invertebrados", // Propriedades de valores padrão
  qualTipo: function () {
    // Método que ira mostrar o tipo de Animal
    console.log(this.tipo);
  },
};

// Cria um novo tipo de animal chamado animal1
var animal1 = Object.create(Animal);
animal1.qualTipo(); // Saída:Invertebrados

// Cria um novo tipo de animal chamado Peixes
var peixe = Object.create(Animal);
peixe.tipo = "Peixes";
peixe.qualTipo(); // Saída: Peixes
```

Herança

Todos os objetos em JavaScript herdam de pelo menos um outro objeto. O objeto "pai" é conhecido como o protótipo, e as propriedades herdadas podem ser encontradas no objeto `prototype` do construtor.

Indexando Propriedades de Objetos

Você pode se referir a uma propriedade de um objeto pelo seu nome de propriedade ou pelo seu índice ordinal. Se você inicialmente definiu uma propriedade pelo nome, você deve sempre se referir a ela pelo nome, e se você inicialmente definir uma propriedade por um índice, você deve sempre se referir a ela pelo índice.

Esta restrição se aplica quando você cria um objeto e suas propriedades com uma função construtora (como fizemos anteriormente com o objeto do tipo carro) e quando você define propriedades individuais explicitamente (por exemplo, `meuCarro.cor = "vermelho"`). Se você inicialmente definir uma propriedade do objeto com um índice, tal como `meuCarro[5] = "25 mpg"`, você pode subsequentemente referir-se á propriedade somente como `meuCarro[5]`.

A exceção a esta regra é a objetos refletidos a partir do HTML, como o conjunto de formulários. Você pode sempre se referir a objetos nessas matrizes por seu número de ordem (com base em onde eles aparecem no documento) ou seu nome (se definido). Por exemplo, se a segunda tag `<FORM>` em um documento tem um atributo `NAME` de "meuFormulario", você pode se referir ao formulário como `document.forms[1]` ou `document.forms["meuFormulario"]` ou `document.meuFormulario`.

Definindo propriedades para um tipo de objeto

Você pode adicionar uma propriedade a um tipo de objeto definido anteriormente, utilizando a propriedade prototype. Esta define uma propriedade que é partilhada por todos os objetos do tipo especificado, em vez de apenas uma instância do objeto. O código a seguir adiciona uma propriedade `cor` para todos os objetos do tipo `Carro`, em seguida adiciona um valor a propriedade `cor` do objeto `carro1`.

```
Carro.prototype.cor = null;
carro1.cor = "preto";
```

Consulte a propriedade `prototype` do objeto `Function` na Referência JavaScript para mais informações.

Definindo métodos

Um *método* é uma função associada a um objeto, ou, simplesmente, um método é uma propriedade de um objeto que é uma função. Métodos são definidos da forma que as funções normais são definidas, exceto que eles tenham que ser atribuídos como propriedade de um objeto. São exemplos:

```
nomeDoObjeto.nomedometodo = nome_da_funcao;

var meuObjeto = {
  meuMetodo: function (parametros) {
```

```
    // ...faça algo
  },
};
```

Onde `nomeDoObjeto` é um objeto existente, `nomedometodo` é o nome que você atribuiu ao método, e `nome_da_funcao` é o nome da função.

Em seguida, você pode chamar o método no contexto do objeto da seguinte forma:

```
objeto.nomedometodo(parametros);
```

Você pode definir métodos para um tipo de objeto incluindo uma definição de metodo na função construtora do objeto. Por exemplo, você poderia definir uma função que iria formatar e mostrar as propriedades do objeto `carro` previamente definido; por exemplo,

```
function mostreCarro() {
  var resultado =
    "Um belo " + this.ano + " " + this.fabricacao + " " + this.modelo;
  pretty_print(resultado);
}
```

onde `pretty_print` é uma função que mostra uma linha horizontal e uma string. Observe o uso de `this` para referenciar o objeto ao qual o método pertence.

Você pode fazer desta função um método de `carro`, adicionando seu estado à definição do objeto.

```
this.mostreCarro = mostreCarro;
```

Assim, a definição completa de `carro` seria agora, parecida com essa:

```
function Carro(fabricacao, modelo, ano, proprietario) {
  this.fabricacao = fabricacao;
  this.modelo = modelo;
  this.ano = ano;
  this.proprietario = proprietario;
  this.mostreCarro = mostreCarro;
}
```

Então você pode chamar o método `mostreCarro` para cada objeto seguinte:

```
carro1.mostreCarro();
carro2.mostreCarro();
```

Usando <code>this</code> para referências de objetos

JavaScript tem uma palavra-chave especial, this, que você pode usar dentro de um método para referenciar o objeto corrente. Por exemplo, suponha que você tenha uma função chamada `validate` que valida o `valor` da propriedade de um objeto, dado o objeto e os valores altos e baixos:

```
function validate(obj, lowval, hival) {
  if (obj.value < lowval || obj.value > hival) alert("Valor inválido!");
}
```

Então, você poderia chamar `validate` no manipulador de evento `onchange` em cada elemento do formulário, usando `this` para passar o elemento, como no exemplo a seguir:

```
<input type="text" name="age" size="3" onChange="validate(this, 18, 99)" />
```

No geral, `this` referencia o objeto chamando um método.

Quando combinado com a propriedade `form` , `this` pode referenciar a forma original do objeto atual. No exemplo seguinte, o formulário `myForm` contém um objeto `Text` e um botão. Quando o usuário clica no botão, o valor do objeto `Text` é definido como nome do formulário. O manipulador de eventos `onclick` do botão usa `this.form` para referenciar a forma original, `myForm`.

```
<form name="myForm">
  <p>
    <label
      >Nome do form:<input type="text" name="text1" value="Beluga"
    /></label>
  </p>
  <p>
    <input
      name="button1"
      type="button"
      value="Mostre o Nome do Form"
      onclick="this.form.text1.value = this.form.name" />
  </p>
</form>
```

Definindo getters e setters

Um getter é um método que obtém o valor de uma propriedade específica. Um setter é um método que define o valor de uma propriedade específica. Você pode definir getters e setters em qualquer objeto de núcleo pré-definido ou objeto definido pelo usuário que suporta a adição de novas propriedades. A sintaxe para definir getters e setters usa a sintaxe literal do objeto.

O código a seguir ilustra como getters e setters podem funcionar para um objetoodefinido pelo usuário.

```
var o = {
  a: 7,
  get b() {
    return this.a + 1;
  },
  set c(x) {
    this.a = x / 2;
  },
};

console.log(o.a); // 7
console.log(o.b); // 8
o.c = 50;
console.log(o.a); // 25
```

As propriedades do objeto `o` são:

- `o.a` — um número
- `o.b` — um getter que retorna `o.a` + 1
- `o.c` — um setter que define o valor de `o.a` pela metade do valor definindo para `o.c`

Observe que nomes de função de getters e setters definidos em um objeto literal usando "[gs]et *property*()" (ao contrário de `__define[GS]etter__`) não são os próprios nomes dos getters, embora a sintaxe `[gs]et propertyName(){ }` possa induzir ao erro e você pensar de outra forma. Para nomear uma função getter ou setter usando a sintaxe "[gs]et *property*()", define explicitamente um função nomeada programaticamente usando `Object.defineProperty` (ou o legado fallback `Object.prototype.__defineGetter__`).

O código a seguir ilustra como getters e setters podem extender o protótipo `Date` para adicionar a propriedade `ano` para todas as instâncias de classes `Date` pré-definidas. Ele usa os métodos `getFullYear` e `setFullYear` existentes da classe `Date` para suportar o getter e setter da propriedade `ano`.

Estes estados definem um getter e setter para a propriedade `ano`:

```
var d = Date.prototype;
Object.defineProperty(d, "year", {
  get: function () {
    return this.getFullYear();
  },
  set: function (y) {
    this.setFullYear(y);
  },
});
```

Estes estados usam o getter e setter em um objeto `Date`:

```
var now = new Date();
console.log(now.year); // 2000
now.year = 2001; // 987617605170
console.log(now);
// Wed Apr 18 11:13:25 GMT-0700 (Pacific Daylight Time) 2001
```

A principio, getters e setters podem ser ou

- definidos usando objetos inicializadores, ou
- adicionar posteriormente para qualquer objeto a qualquer tempo usando um método getter ou setter adicionado

Ao definir getters e setters usando objetos inicializadores tudo o que você precisa fazer é prefixar um método getter com `get` e um método setter com `set`. Claro, o método getter não deve esperar um parâmetro, enquanto o método setter espera exatamente um parâmetro (novo valor para definir). Por exemplo:

```
var o = {
  a: 7,
  get b() {
    return this.a + 1;
  },
  set c(x) {
    this.a = x / 2;
  },
};
```

Getters e setters podem também ser adicionado em um objeto a qualquer hora depois da criação usando o método `Object.defineProperties`. O primeiro parâmetro deste método é o objeto no qual você quer definir o getter ou setter. O segundo parâmetro é um objeto cujos nomes das propriedades são os nomes getter ou setter, e cujo valores das propriedades são objetos para definição de funções getter ou setter. Aqui está um exemplo que define o mesmo getter e setter usado no exemplo anterior:

```
var o = { a: 0 };

Object.defineProperties(o, {
  b: {
    get: function () {
      return this.a + 1;
    },
  },
```

```
  c: {
    set: function (x) {
      this.a = x / 2;
    },
  },
});

o.c = 10; // Roda o setter, que associa 10 / 2 (5) para a propriedade 'a'
console.log(o.b); // Roda o getter, que yields a + 1 ou 6
```

Escolher qual das duas formas depende do seu estilo de programação e tarefa na mão. Se você já vai para o inicializador de objeto ao definir um protótipo, provavelmente a maior parte do tempo escolherá a primeira forma. Esta forma é mais compacta e natural. No entanto, se você precisar adicionar getters e setters mais tarde - porque você não escreveu o protótipo ou objeto particular - então a segunda forma é a única possível. A segunda forma provavelmente melhor representa a natureza dinâmica do JavaScript - mas pode tornar o código difícil de ler e entender.

Removendo propriedades

Você pode remover uma propriedade não herdada usando o operador `delete`. O código a seguir mostra como remover uma propriedade.

```
//Criando um novo objeto, myobj, com duas propriedades, a e b.
var myobj = new Object();
myobj.a = 5;
myobj.b = 12;

//Removendo a propriedade a, deixando myobj com apenas a propriedade b.
delete myobj.a;
console.log("a" in myobj); // yields "false"
```

Você também pode usar `delete` para remover uma variável global se a `var` keyword não estiver sendo usada para declarar a variável:

```
g = 17;
delete g;
```

Comparando Objetos

Em JavaScript, objetos são um tipo de referência. Dois objetos distintos nunca são iguais, mesmo que tenham as mesmas propriedades. Apenas comparando o mesmo objeto de referência com ele mesmo produz verdadeiro.

```
// Duas variáveis, dois objetos distintos com as mesmas propriedades
var fruit = { name: "apple" };
var fruitbear = { name: "apple" };

fruit == fruitbear; // return false
fruit === fruitbear; // return false
```

```
// Duas variáveis, um único objeto
var fruit = { name: "apple" };
var fruitbear = fruit; // assign fruit object reference to fruitbear

// Here fruit and fruitbear are pointing to same object
fruit == fruitbear; // return true
fruit === fruitbear; // return true
```

Para mais informações sobre comparação de operadores, veja Operadores de comparação.

Veja também

- Para se aprofundar, leia sobre os detalhes do modelo de objetos javaScript.
- Para saber mais sobre classes em ECMAScript6 (uma nova forma de criar objetos), veja o capítulo JavaScript classes.

- Anterior
- Próxima

Referência JavaScript

Sobre essa Referência

Objetos Globais

Funções e escopo de função

- `argumentos`
- Modo Strict

Declarações

Operadores e outras palavras-chave

Comentários

- Comentários de código (`//` and `/* */`)

E4X (extensão)

Declarações Globais:

- default XML namespace

Funções Globais:

- isXMLName()

Construtores Globais:

- Namespace()
- QName()
- XML()
- XMLList()

Apêndice A - Palavras Reservadas

Apêndice B - Funcionalidades Obsoletas

Classes

Classes em JavaScript são introduzidas no ECMAScript 2015 e são simplificações da linguagem para as heranças baseadas nos protótipos. A sintaxe para classes **não** introduz um novo modelo de herança de orientação a objetos em JavaScript. Classes em JavaScript provêm uma maneira mais simples e clara de criar objetos e lidar com herança.

Definindo classes

As Classes são, de fato, "funções especiais", e, assim como você pode definir "function expressions" e "function declarations", a sintaxe de uma classe possui dois componentes: "class expressions" e "class declarations".

Declarando classes

Uma maneira de definir uma classe é usando uma declaração de classe. Para declarar uma classe, você deve usar a palavra-chave `class` seguida pelo nome da classe (aqui "Retangulo").

```
class Retangulo {
  constructor(altura, largura) {
    this.altura = altura;
    this.largura = largura;
  }
}
```

Uso antes da declaração (Hoisting - Tradução Literal: Lançamento)

Uma diferença importante entre **declarações de funções** das **declarações de classes**, é que declararações de funções são hoisted e declarações de classes não são. Primeiramente deve declarar sua classe para só então acessá-la, pois do contrário o código a seguir irá lançar uma exceção: `ReferenceError`:

```
const p = new Retangulo(); // Erro de referência (ReferenceError)

class Retangulo {}
```

Expressões de Classes

Uma **Expressão de Classe** (class expression) é outra forma para definir classes. Expressões de Classes podem possuir nomes ou não (anônimas). O nome dado para uma expressão de classe é local ao corpo da classe.

```
// sem nome
let Retangulo = class {
  constructor(altura, largura) {
    this.altura = altura;
    this.largura = largura;
  }
};

// nomeada
let Retangulo = class Retangulo {
  constructor(altura, largura) {
    this.altura = altura;
    this.largura = largura;
  }
};
```

Nota: As **expressões de classe** também sofrem com o mesmo problema de hoisted mencionados em **declarações** de classe.

Corpo de uma classe e definições de métodos

O corpo de uma classe é a parte que está entre chaves `{}`. É aí onde você define os membros da classe, como os métodos, ou os construtores.

Modo Estrito (strict mode)

Os corpos das Declarações de Classes e das Expressões de Classes são executados em modo estrito.

Construtor

O método `constructor` é um tipo especial de método para criar e iniciar um objeto criado pela classe. Só pode existir um método especial com o nome "constructor" dentro da classe. Um erro de sintaxe `SyntaxError` será lançado se a classe possui mais do que uma ocorrência do método `constructor`.

Um construtor pode usar a palavra-chave `super` para chamar o construtor de uma classe pai.

Métodos Protótipos

Veja também definições de métodos (method definitions).

```
class Retangulo {
  constructor(altura, largura) {
    this.altura = altura;
    this.largura = largura;
  }
  //Getter
  get area() {
    return this.calculaArea();
  }

  calculaArea() {
    return this.altura * this.largura;
  }
}

const quadrado = new Retangulo(10, 10);

console.log(quadrado.area);
```

Métodos estáticos

A palavra-chave `static` define um método estático de uma classe. Métodos estáticos são chamados sem a instanciação da sua classe e não podem ser chamados quando a classe é instanciada. Métodos estáticos são geralmente usados para criar funções de utilidades por uma aplicação.

```
class Ponto {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }

  static distancia(a, b) {
    const dx = a.x - b.x;
    const dy = a.y - b.y;

    return Math.hypot(dx, dy);
  }
}

const p1 = new Ponto(5, 5);
const p2 = new Ponto(10, 10);

p1.distancia; //undefined
p2.distancia; //undefined

console.log(Ponto.distancia(p1, p2));
```

Empacotando com protótipos e métodos estáticos

Quando um método estático ou protótipo é chamado sem um objeto "this" configurado (ou com "this" como boolean, string, number, undefined ou null), então o valor "this" será **`undefined`** dentro da função chamada. Autoboxing não vai acontecer. O comportamento será o mesmo se escrevemos o código no modo não-estrito.

```
class Animal {
  falar() {
    return this;
  }
  static comer() {
    return this;
  }
}

let obj = new Animal();
obj.falar(); // Animal {}
let falar = obj.falar;
falar(); // undefined

Animal.comer(); // class Animal
let comer = Animal.comer;
comer(); // undefined
```

Se escrevemos o código acima usando classes baseadas em função tradicional, então o autoboxing acontecerá com base no valor de "this" para o qual a função foi chamada.

```
function Animal() {}

Animal.prototype.falar = function () {
  return this;
};

Animal.comer = function () {
  return this;
};

let obj = new Animal();
let falar = obj.falar;
falar(); // objeto global

let comer = Animal.comer;
comer(); // objeto global
```

Propriedades de instância

Propriedades de instâncias devem ser definidas dentro dos métodos da classe:

```
class Retangulo {
  constructor(altura, largura) {
    this.altura = altura;
    this.largura = largura;
  }
}
```

Propriedades de dados estáticos e propriedades de dados prototipados (prototype) devem ser definidos fora da declaração do corpo da classe.

```
Retangulo.larguraEstatico = 20;
Retangulo.prototype.larguraPrototipagem = 25;
```

Sub classes com o <code>extends</code>

A palavra-chave `extends` é usada em uma *declaração de classe*, ou em uma *expressão de classe* para criar uma classe como filha de uma outra classe.

```
class Animal {
  constructor(nome) {
    this.nome = nome;
  }

  falar() {
    console.log(this.nome + " emite um barulho.");
  }
}

class Cachorro extends Animal {
  falar() {
    console.log(this.nome + " latidos.");
  }
}

let cachorro = new Cachorro("Mat");
cachorro.falar();
```

Se existir um contrutor nas subclasses, é necessário primeiro chamar super() antes de usar a keyword "this".

Também é possivel ampliar (extends) "classes" baseadas em funções tradicionais.

```
function Animal(nome) {
  this.nome = nome;
}

Animal.prototype.falar = function () {
  console.log(this.nome + " faça barulho.");
};

class Cachorro extends Animal {
  falar() {
    console.log(this.nome + " lati.");
  }
}

let cachorro = new Cachorro("Mitzie");
cachorro.falar(); // Mitzie lati.
```

Note que classes não extendem objetos normais (não construíveis). Se você quer herdar de um objeto, é necessário utilizar `Object.setPrototypeOf()`:

```
let Animal = {
  falar() {
    console.log(this.nome + " faça barulho.");
  },
};

class Cachorro {
  constructor(nome) {
    this.nome = nome;
  }
}

Object.setPrototypeOf(Cachorro.prototype, Animal);

let cachorro = new Cachorro("Mitzie");
cachorro.falar(); //Mitzie faça barulho.
```

Species

Você pode querer retornar um objeto `Array` na sua classe `MinhaArray` derivada de array. O padrão Species permite a sobrescrita do construtor padrão.

Por exemplo, quando utilizando um método como `map()` que retorna o construtor padrão, você pode querer que esse método retorne um objeto `Array` ao invés do objeto `MinhaArray`. O `Symbol.species` te permite fazer isso:

```
class MinhaArray extends Array {
  // Sobrescreve species para o construtor da classe pai Array
  static get [Symbol.species]() {
    return Array;
  }
}

let a = new MinhaArray(1, 2, 3);
let mapped = a.map((x) => x * x);

console.log(mapped instanceof MyArray); // false
console.log(mapped instanceof Array); // true
```

Chamada da classe pai com <code>super</code>

A palavra-chave (keyword) `super` é utilizada para chamar funções que pertencem ao pai do objeto.

```
class Gato {
  constructor(nome) {
    this.nome = nome;
  }

  falar() {
    console.log(this.nome + " faça barulho.");
  }
}

class Leao extends Gato {
  falar() {
    super.falar();
    console.log(this.nome + " roars.");
  }
}

let leao = new Leao("Fuzzy");
leao.falar();

// Fuzzy faça barulho.
// Fuzzy roars.
```

Mix-ins

Subclasses abstratas ou *mix-ins* são templates para classes. Uma classe do ECMAScript pode apenas ter uma classe pai, assim sendo, não é possível a classe ter herança múltipla.

Para se ter um comportamento similar ao de herança múltipla no ECMAScript usa-se mix-ins, uma forma de implementar mix-ins é usar um template de subclasse que é uma função que instancia uma classe base e retorna uma subclasse extendida desta classe base:

```
class Humano {
  constructor(nome) {
    this.nome = nome;
  }
  andar() {
    return this.nome + " andou um passo";
  }
}

const HumanoFalante = (Base) =>
  class extends Base {
    falar() {
```

```
      return this.nome + " diz: olá mundo!";
    }
  };

const HumanoFalanteMixado = (Base) => class extends Base {};

const HumanoFinal = HumanoFalanteMixado(HumanoFalante(Humano));

const humano = new HumanoFinal("Bill Gates");

console.log(humano.andar());
console.log(humano.falar());
```

Rodando com Scratchpad

Uma classe não pode ser redefinida. Se você estiver rodando código com Scratchpad (Menu do Firefox Ferramentas > Web Developer > Scratchpad) e você acionar 'Run' a uma definição de uma classe com o mesmo nome duas vezes, você verá um confuso SyntaxError: redeclaration of let <class-name> (Erro de Sintaxe: redeclaração de let <nome-da-classe>).

Para reacionar (re-run) uma definição, use o menu do Scratchpad em Execute > Reload and Run (Executar > Atualizar e Rodar). Por favor, vote no bug #1428672.

Veja também

- Funções
- Declaração de `classes` (class declaration)
- Expressão de `classes` (class expression)
- `super`
- Blog post: "ES6 In Depth: Classes"

constructor

O construtor é um método especial para criar e inicializar um objeto criado a partir de uma classe.

Sintaxe

```
constructor([argumentos]) { ... }
```

Descrição

Apenas um método especial com o nome *constructor* pode existir em uma classe. O erro `SyntaxError` será mostrado se a classe contiver mais de um método *constructor.*

Um construtor pode usar a palavra reservada *super* para se referir ao construtor da classe pai (**superior**).

Um construtor padrão será usado se você não especificá-lo*.*

Exemplos

Usando o método constructor

O trecho de código foi fornecido por exemplo de classes (live demo).

```
class Quadrado extends Poligono {
  constructor(comprimento) {
    // super chama o construtor da classe pai que vai atribuir comprimento para
    // os atributos comprimento e altura herdados pela nossa classe filha Quadrado
    super(comprimento, comprimento);
    // Nas classes filhas, super() deve ser chamado antes de usar o this. Sem ele
    // vai ocorrer um erro de referência. O this agora se refere a classe filha Quadrado
    this.nome = "Quadrado";
  }

  // os atributos a seguir são herdados da classe pai Poligono: altura, comprimento e area.

  get area() {
    return this.altura * this.comprimento;
  }

  set area(valor) {
    this.area = valor;
  }
}
```

Construtores padrão (constructors)

Um construtor padrão será usado pela classe caso nenhum seja especificado. Para as classes de base o construtor padrão é:

```
constructor() {} //construtor criado por padrão
```

Para as classes filhas o construtor padrão é:

```
constructor(...args) {
  super(...args);
```

```
}
```

Veja também

- super()
- `class` expressão
- `class` declaração
- Classes

extends

A palavra chave **`extends`** é usada em uma class declarations ou class expressions para criar uma classe filha de outra classe.

Sintaxe

```
class ChildClass extends ParentClass { ... }
```

Descrição

A palavra chave extends pode ser usada para tanto classes filhas quanto objetos filhos pré-construidos.

O `.prototype` da extensão deve ser um `Object` ou `null`.

Exemplos

Usando <code>extends</code>

O primeiro exemplo cria uma classe chamada `Square` a partir de uma classe chamada `Polygon`. Este exemplo foi extraido deste live demo (source).

```
class Square extends Polygon {
  constructor(length) {
    // Here, it calls the parent class' constructor with lengths
    // provided for the Polygon's width and height
    super(length, length);
    // Nota: In derived classes, super() must be called before you
    // can use 'this'. Leaving this out will cause a reference error.
    this.name = "Square";
  }

  get area() {
    return this.height * this.width;
  }

  set area(value) {
    this.area = value;
  }
}
```

Usando <code>extends</code> com objetos pré-construidos

Este exemplo extende o objeto pré-construido `Date`. Este exemplo foi extraido deste live demo (source).

```
class myDate extends Date {
  constructor() {
    super();
  }

  getFormattedDate() {
    var months = [
      "Jan",
      "Feb",
      "Mar",
      "Apr",
      "May",
      "Jun",
      "Jul",
      "Aug",
      "Sep",
      "Oct",
      "Nov",
      "Dec",
    ];
    return (
      this.getDate() + "-" + months[this.getMonth()] + "-" + this.getFullYear()
    );
  }
}
```

Estendendo <code>null</code>

Estender de `null` funciona como em uma classe normal, exceto que o objeto prototype não herda de `Object`.

```
class nullExtends extends null {
  constructor() {}
}

Object.getPrototypeOf(nullExtends); // Function.prototype
Object.getPrototypeOf(nullExtends.prototype); // null
```

Ver também

- Classes
- super

static

A palavra chave **static** define um método estático para a classe. Métodos estáticos não são chamados na instâncias da classe. Em vez disso, eles são chamados na própria classe. Geralmente, são funções utilitárias, como funções para criar ou clonar objetos.

Sintaxe

```
static nomeDoMetodo() { ... }
```

Descrição

Chamadas a métodos estáticos são feitas diretamente na classe e não podem ser feitas em uma instância da classe. Métodos estáticos são comumente utilizados como funções utilitárias.

Chamada de métodos estáticos

De outro método estático

Para chamar um método estático dentro de outro método estático da mesma classe, podemos utilizar a palavra reservada `this`.

```
class ChamadaDoMetodoEstatico {
  static metodoEstatico() {
    return "O método estático foi chamado";
  }
  static outroMetodoEstatico() {
    return this.metodoEstatico() + " de outro método estático";
  }
}
ChamadaDoMetodoEstatico.metodoEstatico();
// 'O método estático foi chamado'

ChamadaDoMetodoEstatico.outroMetodoEstatico();
// 'O método estático foi chamado de outro método estático'
```

De outro construtor e outros métodos

Métodos estáticos não são diretamente acessíveis utilizando-se `this` a partir de métodos não estáticos. É necessário chamá-los usando o nome da classe:

```
nomeDaClasse.nomeDoMetodoEstatico();
```

ou chamando o método como uma propriedade do construtor:

```
this.constructor.nomeDoMetodoEstatico().
```

```
class ChamadaDoMetodoEstatico {
  constructor() {
    console.log(ChamadaDoMetodoEstatico.MetodoEstatico());
    // 'O método estático foi chamado.'

    console.log(this.constructor.MetodoEstatico());
    // 'O método estático foi chamado.'
  }

  static MetodoEstatico() {
    return "O método estático foi chamado.";
  }
}
```

Exemplos

O exemplo a seguir demonstra várias coisas. Ele mostra como um método estático é implementado em uma classe e como uma classe com um membro estático pode virar uma subclasse. Por fim, ele mostra como um método estático pode e não pode ser chamado.

```
class Tripple {
  static tripple(n) {
    n = n | 1;
    return n * 3;
  }
}
```

```
class BiggerTripple extends Tripple {
  static tripple(n) {
    return super.tripple(n) * super.tripple(n);
  }
}

console.log(Tripple.tripple());
console.log(Tripple.tripple(6));
console.log(BiggerTripple.tripple(3));
var tp = new Tripple();
console.log(tp.tripple()); //Logs 'tp.tripple is not a function'.
```

Veja também

- `class` expression
- `class` declaration
- Classes

Deprecated and obsolete features

Esta página lista funcionalidades do JavaScript que foram descontinuadas (ou seja, continuam disponíveis mas têm remoção planejada) e obsoletas (ou seja, não estão mais disponíveis).

Funcionalidades descontinuadas

Estas *features* descontinuadas ainda podem ser usadas, mas com cautela, porque espera-se que sejam removidas por completo no futuro. Aconselha-se que remova-as do seu código.

Propriedades RegExp

As propriedades a seguir foram descontinuadas. Isto não afeta o uso delas em replacement strings:

Propriedade	Descrição
`$1-$9`	Encontra *substrings* dentro de parênteses. **Atenção:** Usar estas propriedades pode resultar em problemas, porque extensões do navegador podem modifica-las. Evite-as!
`$_`	Ver `input`.
`$*`	Ver `multiline`.
`$&`	Ver `lastMatch`.
`$+`	Ver `lastParen`.
`$``	Ver `leftContext`.
`$'`	Ver `rightContext`.
`input`	A string encontrada por uma expressão regular.
`lastMatch`	Os últimos caracteres encontrados.
`lastParen`	A última *substring* entre parênteses que foi encontrada.
`leftContext`	A penúltima *substring* encontrada.
`rightContext`	A *substring* que segue a que foi encontrada por último.

As propriedades a seguir pertencem a instâncias de um `RegExp`, não mais ao objeto `RegExp`:

Propriedades	Descrição
`global`	Determina se deve ou não testar a expressão regular com todas as combinações possíveis em uma *string* ou só com a primeira.
`ignoreCase`	Determina se deve ou não ignorar a capitalização ao tentar encontrar uma combinação em uma *string*.
`lastIndex`	O índince que aponta o começo da próxima combinação.
`multiline`	Determina se deve ou não procurar por *strings* em várias linhas diferentes.
`source`	O texto do padrão.

Métodos RegExp

- O método `compile()` foi descontinuado.
- O método `valueOf` não é mais especializado para `RegExp`. Use `Object.valueOf()`.

Propriedades de Função

- As propriedades `caller` e `arguments` foram descontinuadas, porque elas estouram o chamador de funções. Ao invés da propriedade *arguments*, deve-se usar o objeto `arguments` dentro dos fechamentos da função.

Legacy generator

- `Legacy generator function statement` e `Legacy generator function expression` foram descontinuados. No lugar deles, use `function* statement` e `function* expression`.
- O `JS1.7/JS1.8 Array comprehension` e o `JS1.7/JS1.8 Generator comprehension` foram descontinuados.

Iterador

- `StopIteration` foi descontinuado.
- `Iterator` foi descontinuado.

Métodos de objeto

- `watch` e `unwatch` foram descontinuados. No lugar deles, use `Proxy` .
- `__iterator__` foi descontinuado.
- `Object.prototype.__noSuchMethod__` foi descontinuado. Use `Proxy` em seu lugar.

Métodos de data

- `getYear` e `setYear` foram afetados pelo Bug do Milênio e foram reagrupados em `getFullYear` e `setFullYear`.
- Deve-se usar `toISOString` ao invés do método descontinuado `toGMTString` em códigos novos.
- `toLocaleFormat` foi descontinuado.

Funções

- Expression closures foram descontinuadas. Use `functions` normais ou arrow functions no lugar delas.

Proxy

- Proxy.create e Proxy.createFunction foram descontinuadas. Use `Proxy` no lugar delas.
- As armadilhas a seguir ficaram obsoletas:
  - `hasOwn` (Erro do Firefox 980565, Firefox 33).
  - `getEnumerablePropertyKeys` (Erro do Firefox 783829, Firefox 37)
  - `getOwnPropertyNames` (Erro do Firefox 1007334, Firefox 33)
  - `keys` (Erro do Firefox 1007334, Firefox 33)

let

- `let blocks` e `let expressions` foram descontinuados.

Sequências de escape

- Sequências de escape octais (\ seguido por um, dois ou três dígitos octais) foram descontinuadas em literais *string* e expressões regulares.
- As funções `escape` e `unescape` foram descontinuadas. Use `encodeURI`, `encodeURIComponent`, `decodeURI` ou `decodeURIComponent` para codificar e decodificar sequências de escape para caracteres especiais.

Métodos de string

- HTML wrapper methods como `String.prototype.fontsize` e `String.prototype.big`.
- `String.prototype.quote` foi removido do Firefox 37.
- parâmetros flag não padrões em `String.prototype.search`, `String.prototype.match`, e `String.prototype.replace` foram depreciados.

Features obsoletas

Estas *features* obsoletas foram totamente removidas do JavaScript e não podem ser usadas a partir da versão indicada do JavaScript.

Object

- The `Object.prototype.__proto__` accessors are deprecated. Use `Object.getPrototypeOf` and `Object.setPrototypeOf` instead. This does not apply to the `__proto__` literal key in object literals.
- The `Object.prototype.__defineGetter__`, `Object.prototype.__defineSetter__`, `Object.prototype.__lookupGetter__`, and `Object.prototype.__lookupSetter__` methods are deprecated. Use `Object.getOwnPropertyDescriptor` and `Object.defineProperty` instead.

Function

Property	Description
`arity`	Number of formal arguments.

Array

Property	Description
`Array.observe()`	Asynchronously observing changes to Arrays.
`Array.unobserve()`	Remove observers.

Number

- `Number.toInteger()`

ParallelArray

- `ParallelArray`

Statements

- `for each...in` is deprecated. Use `for...of` instead.
- Destructuring `for...in` is deprecated. Use `for...of` instead.

E4X

See E4X for more information.

Sharp variables

See Sharp variables in JavaScript for more information.

JavaScript error reference

Lista de erros "globais".

- Erro de sintaxe falta) depois da lista de argumentos
- Erro de sintaxe: declaração de função requer um nome
- Erro de Sintaxe: faltando : depois da propriedade id
- Error: Permission denied to access property "x"
- InternalError: too much recursion
- RangeError: argument is not a valid code point
- RangeError: invalid array length
- RangeError: invalid date
- RangeError: precision is out of range
- RangeError: repeat count must be non-negative
- ReferenceError: "x" não está definido
- ReferenceError: Atribuição à variável não declarada "x"
- ReferenceError: can't access lexical declaration`X' before initialization
- ReferenceError: invalid assignment left-hand side
- SyntaxError: as declarações da cabeça do laço for-in podem não ter inicializadores
- SyntaxError: flag de expressão regular inválida "x"
- SyntaxError: illegal character
- SyntaxError: JSON.parse: bad parsing
- SyntaxError: missing } after property list
- SyntaxError: missing formal parameter
- SyntaxError: missing variable name
- SyntaxError: redeclaration of formal parameter "x"
- SyntaxError: uma declaração na cabeça de um laço for-of não pode ter um inicializador
- SyntaxError: Unexpected token
- SyntaxError: unterminated string literal
- TypeError: "x" is (not) "y"
- TypeError: "x" is not a function
- TypeError: "x" is read-only
- TypeError: atribuição inválida para const "x"
- TypeError: can't assign to property "x" on "y": not an object
- TypeError: can't define property "x": "obj" is not extensible
- TypeError: can't redefine non-configurable property "x"
- TypeError: invalid Array.prototype.sort argument
- TypeError: property "x" is non-configurable and can't be deleted
- TypeError: setting getter-only property "x"
- Warning: -file- is being assigned a //# sourceMappingURL, but already has one
- Warning: unreachable code after return statement

Warning: -file- is being assigned a //# sourceMappingURL, but already has one

O aviso "-file- is being assigned a //# sourceMappingURL, ocorre quando um source map foi especificado mais de uma vez para uma determinada origem JavaScript.

Mensagem

```
Warning: -file- is being assigned a //# sourceMappingURL, but already has one.
```

Tipo de erro

Um aviso. A execução do JavaScript não será interrompida.

O que deu errado?

Um source map foi especificado mais de uma vez para uma determinada origem JavaScript.

As fontes JavaScript são frequentemente combinadas e minificadas para tornar a entrega mais eficiente do servidor. Com source maps, o depurador pode mapear o código que está sendo executado para os arquivos de origem originais. Há duas maneiras de atribuir um source map, usando um comentário ou definindo um cabeçalho para o arquivo JavaScript.

Exemplos

Configurando source maps

Configurando um source map usando um comentário no arquivo:

```
//# sourceMappingURL=http://example.com/path/to/your/sourcemap.map
```

Ou, alternativamente, você pode definir um cabeçalho para seu arquivo JavaScript:

```
X-SourceMap: /path/to/file.js.map
```

Veja também

- Como usar um source map– Documentação do Firefox Tools
- Introdução aos source maps (2012)

TypeError: invalid Array.prototype.sort argument

A exceção JavaScript "invalid Array.prototype.sort argument" ocorre quando o argumento de `Array.prototype.sort()` não é `undefined` ou uma função que compara os seus operandos.

Mensagem

```
TypeError: The comparison function must be either a function or undefined (V8-based)
TypeError: invalid Array.prototype.sort argument (Firefox)
TypeError: Array.prototype.sort requires the comparator argument to be a function or undefined (Safari)
```

Tipo de erro

`TypeError`

O que ocorreu de errado?

Espera-se que o argumento de `Array.prototype.sort()` seja `undefined` ou uma função que compare os seus operandos.

Exemplos

Casos inválidos

```
[1, 3, 2].sort(5); // TypeError

const cmp = { asc: (x, y) => x >= y, dsc: (x, y) => x <= y };
[1, 3, 2].sort(cmp[this.key] || "asc"); // TypeError
```

Casos válidos

```
[1, 3, 2].sort(); // [1, 2, 3]

const cmp = { asc: (x, y) => x >= y, dsc: (x, y) => x <= y };
[1, 3, 2].sort(cmp[this.key || "asc"]); // [1, 2, 3]
```

Veja também

- `Array.prototype.sort()`

SyntaxError: flag de expressão regular inválida "x"

A exceção JavaScript "flag de expressão regular inválida" ocorre quando as flags de uma expressão regular contêm qualquer flag que não seja uma das flag: `g`, `i`, `m`, `s`, `u`, `y` ou `d`.

Também pode ser lançada se a expressão contiver mais do que uma instância de uma flag válida.

Mensagem

```
SyntaxError: Invalid regular expression flags (V8-based)
SyntaxError: invalid regular expression flag x (Firefox)
SyntaxError: Invalid regular expression: invalid flags (Safari)
```

Tipo de erro

`SyntaxError`

O que ocorreu de errado?

A expressão regular contém flags inválidas, ou foram utilizadas flags válidas mais de uma vez na expressão.

As flags válidas (permitidas) são listadas em Expressões regulares > Pesquisa avançada com flags, e reproduzidas abaixo:

Flag	Descrição
`g`	Pesquisa Global. Veja `global`
`i`	Pesquisa case-sensitive. Veja `ignoreCase`.
`m`	Pesquisa em múltiplas linhas. Veja `multiline`.
`s`	Permitir que `.` se igualem a novas linhas. Veja `dotAll`.
`u`	Unicode; trata um padrão como uma sequência de pontos de código Unicode. Veja `unicode`.
`y`	Realiza uma pesquisa "sticky" que coincida a partir da posição atual na cadeia de destino. Veja `sticky`
`d`	Índices. Geração de índices para a correspondência de substrato. Veja `hasIndices`

Exemplos

Em uma expressão regular literal, que consiste em um padrão fechado entre barras, as flags são definidas após a segunda barra. As flags de expressão regular podem ser utilizadas separadamente ou em conjunto em qualquer ordem. Esta sintaxe mostra como declarar as flags usando a expressão regular literal:

```
const re = /pattern/flags;
```

Podem também ser definidos na função construtora do objecto `RegExp` (segundo parâmetro):

```
const re = new RegExp("pattern", "flags");
```

Aqui está um exemplo que mostra a utilização apenas de flags corretas.

```
/foo/g;
/foo/gims;
/foo/uy;
```

Abaixo está um exemplo que mostra a utilização de algumas flags inválidas `b`, `a` e `r`:

```
/foo/bar;

// SyntaxError: invalid regular expression flag "b"
```

O código abaixo está incorrecto, porque `W`, `e` e `b` não são flags válidas.

```
const obj = {
  url: /docs/Web,
};

// SyntaxError: invalid regular expression flag "W"
```

Uma expressão contendo duas barras é interpretada como uma expressão regular literal. Muito provavelmente a intenção era criar uma cadeia literal, usando citações simples ou duplas, como se mostra abaixo:

```
const obj = {
  url: "/docs/Web",
};
```

Veja também

- Expressões Regulares
- XRegEx flags – biblioteca de expressões regulares que fornece quatro novas bandeiras (`n`, `s`, `x`, `A`)

ReferenceError: can't access lexical declaration`X' before initialization

Mensagem

```
ReferenceError: Use before delaration (Edge)
ReferenceError: can't access lexical declaration `X' before initialization (Firefox)
ReferenceError: 'x' is not defined (Chrome)
```

Tipo de Erro

`ReferenceError`

O que está errado

Uma variável léxica foi acessada antes de ser inicializada. Isso acontece dentro de qualquer declaração de bloco, quando as declarações `let` ou `const` são acessadas antes de serem definidas.

Exemplos

Errado

Neste caso, a variável "foo" é redeclarada usando `let`.

```
function test() {
  let foo = 33;
  if (true) {
    let foo = foo + 55;
    // ReferenceError: can't access lexical
    // declaration `foo' before initialization
  }
}
test();
```

Certo

Para mudar "foo" dentro do bloco if você precisa remover o `let` que causa a redeclaração.

```
function test() {
  let foo = 33;
```

```
  if (true) {
    foo = foo + 55;
  }
}
test();
```

Veja também

- Temporal Dead Zone e erros com let.

TypeError: can't assign to property "x" on "y": not an object

A exceção do modo strict do Javascript "can't assign to property" ocorre quando é feita a tentativa de criar uma propriedade em um valor primitivo como um symbol, uma string, um number ou um boolean. Valores primitivos não podem ter nenhuma propriedade.

Mensagem

```
TypeError: can't assign to property "x" on {y}: not an object (Firefox)
TypeError: Cannot create property 'x' on {y} (Chrome)
```

Tipo de Erro

`TypeError`.

O que deu errado?

No `Strict_mode`, um `TypeError` é gerado quando é feita a tentativa de criar uma propriedade em um valor primitivo como um symbol, uma string, um number ou um boolean. Valores primitivos não podem ter nenhuma propriedade.

O problema pode ser que um valor inesperado está em um lugar inesperado, ou que uma variante de objeto de uma `String` ou um `Number` é esperado.

Exemplos

Casos inválidos

```
"use strict";

var foo = "minha string";
// A linha a seguir não faz nada se não estiver em modo strict.
foo.bar = {}; // TypeError: can't assign to property "bar" on "my string": not an object
```

Corrigindo o problema

Corrija o código para evitar que um primitivo seja usado em tais lugares, ou corrija o problema criando o objeto homólogo `Object`.

```
"use strict";

var foo = new String("minha string");
foo.bar = {};
```

Veja também

- `Strict_mode`
- Primitivo

TypeError: can't define property "x": "obj" is not extensible

A exceção de modo strict do Javascript "can't define property "x": "obj" is not extensible" ocorre quando `Object.preventExtensions()` marcou um objeto como não extensível, de modo que ele não terá propriedades além das que ele tinha no momento em que foi marcado como não extensível.

Mensagem

```
TypeError: Cannot create property for a non-extensible object (Edge)
TypeError: can't define property "x": "obj" is not extensible (Firefox)
TypeError: Cannot define property: "x", object is not extensible. (Chrome)
```

Tipo de Erro

`TypeError`

O que deu errado?

Geralmente, um objeto é extensível e novas propriedades podem ser adicionadas a ele. Contudo, neste caso `Object.preventExtensions()` marcou o objeto como não extensível, de modo que ele não terá propriedades além das que ele tinha no momento em que foi marcado como não extensível.

Exemplos

Adicionando uma nova propriedade para um objeto não extensível

No modo strict, a tentativa de adicionar uma nova propriedade em um objeto não extensível lança um `TypeError`. No modo sloppy, a adição da propriedade "x" é silenciosamente ignorada.

```
"use strict";

var obj = {};
Object.preventExtensions(obj);

obj.x = "foo";
// TypeError: can't define property "x": "obj" is not extensible
```

Em ambos os casos, modo strict e modo sloppy, chamar `Object.defineProperty()` lança uma exceção quando é adicionada uma nova propriedade em um objeto não extensível.

```
var obj = {};
Object.preventExtensions(obj);

Object.defineProperty(obj, "x", { value: "foo" });
// TypeError: can't define property "x": "obj" is not extensible
```

Para corrigir este erro, você precisa remover todas as chamadas para `Object.preventExtensions()`, ou movê-las para uma posição para que a propriedade seja adicionada antes e apenas depois o objeto seja marcado como não extensível. Naturalmente, você pode remover a tentativa de adicionar a propriedade, se você não precisar dela.

```
"use strict";

var obj = {};
obj.x = "foo"; // adiciona a propriedade antes e só então previne extensões

Object.preventExtensions(obj);
```

Veja também

- `Object.preventExtensions()`

TypeError: property "x" is non-configurable and can't be deleted

A exceção do Javascript "property is non-configurable and can't be deleted" ocorre quando é feita a tentativa de deletar uma propriedade, mas a propriedade não é configurável.

Mensagem

```
TypeError: Calling delete on 'x' is not allowed in strict mode (Edge)
TypeError: property "x" is non-configurable and can't be deleted. (Firefox)
TypeError: Cannot delete property 'x' of #<Object> (Chrome)
```

Tipo de Erro

`TypeError` apenas no modo strict.

O que deu errado?

Foi feita a tentativa de deletar uma propriedade, mas esta propriedade não é configurável. O atributo `configurable` controla se a propriedade pode ser deletada do objeto e se seus atributos (Exceto `writable`) podem ser alterados.

Este erro acontece apenas no modo strict. No modo de código não estrito, a operação retorna `false`.

Exemplos

Tentando excluir uma propriedade não configurável

Propriedades não configuráveis não são comuns, mas podem ser criadas usando `Object.defineProperty()` ou `Object.freeze()`.

```
"use strict";
var obj = Object.freeze({ name: "Elsa", score: 157 });
delete obj.score; // TypeError

("use strict");
var obj = {};
Object.defineProperty(obj, "foo", { value: 2, configurable: false });
delete obj.foo; // TypeError

("use strict");
var frozenArray = Object.freeze([0, 1, 2]);
frozenArray.pop(); // TypeError
```

Também existem algumas propriedades não configuráveis embutidas no Javascript. Talvez você tenha tentado deletar uma constante matemática.

```
"use strict";
delete Math.PI; // TypeError
```

Veja também

- Operador delete

- `Object.defineProperty()`
- `Object.freeze()`

TypeError: can't redefine non-configurable property "x"

A exceção do Javascript "can't redefine non-configurable property" ocorre quando é feita a tentativa de redefinir uma propriedade, mas essa propriedade não é configurável.

Mensagem

```
TypeError: Cannot modify non-writable property {x} (Edge)
TypeError: can't redefine non-configurable property "x" (Firefox)
TypeError: Cannot redefine property: "x" (Chrome)
```

Tipo de Erro

`TypeError`

O que deu errado?

Foi feita a tentativa de redefinir uma propriedade, mas essa propriedade não é configurável. O atributo `configurable` controla se a propriedade pode ser deletada do objeto e se seus atributos (Exceto `writable`) podem ser alterados. Geralmente, propriedades de um objeto criado por um inicializador de objeto são configuráveis. Contudo, por exemplo, ao usar `Object.defineProperty()`, a propriedade não é configurável por padrão.

Exemplos

Propriedades não configuráveis criadas por <code>Object.defineProperty</code>

`Object.defineProperty()` cria uma propriedade não configurável se você não especificá-las como configurável.

```
var obj = Object.create({});
Object.defineProperty(obj, "foo", { value: "bar" });

Object.defineProperty(obj, "foo", { value: "baz" });
// TypeError: can't redefine non-configurable property "foo"
```

Você vai precisar alterar a propriedade "foo" para configurável, se você pretende redefini-la posteriormente no código.

```
var obj = Object.create({});
Object.defineProperty(obj, "foo", { value: "bar", configurable: true });
Object.defineProperty(obj, "foo", { value: "baz", configurable: true });
```

Veja também

- [[Configurable]]
- `Object.defineProperty()`

TypeError: setting getter-only property "x"

A exceção apenas do modo strict do Javascript "setting getter-only property" ocorre quando é feita a tentativa de definir um novo valor para uma propriedade para qual apenas um getter está especificado.

Mensagem

```
TypeError: Assignment to read-only properties is not allowed in strict mode (Edge)
TypeError: setting getter-only property "x" (Firefox)
TypeError: Cannot set property "prop" of #<Object> which has only a getter (Chrome)
```

Tipo de Erro

`TypeError` no modo strict apenas.

O que deu errado?

Tem uma tentativa de definir um novo valor para uma propriedade para qual apenas um getter está especificado. Isto será silenciosamente ignorado no modo non-strict, enquanto vai lançar um `TypeError` no modo strict.

Exemplos

Propriedades sem setter

O exemplo abaixo mostra como definir um getter para uma propriedade. Isso não especifica um setter, então um `TypeError` vai ser lançado quando for feita a tentativa de definir a propriedade `temperatura` para `30`. Para mais detalhes veja também a página `Object.defineProperty()`.

```
"use strict";

function Arquivo() {
  var temperatura = null;
  Object.defineProperty(this, "temperatura", {
    get: function () {
      console.log("get!");
      return temperatura;
```

```
    },
  });
}

var arq = new Arquivo();
arq.temperatura; // 'get!'

arq.temperatura = 30;
// TypeError: setting getter-only property "temperatura"
```

Para corrigir este erro, você vai ter que remover a linha 16, onde é feita a tentativa de definir a propriedade temperatura, ou você vai precisar implementar um setter para a propriedade, desta forma, por exemplo:

```
"use strict";

function Arquivo() {
  var temperatura = null;
  var arquivo = [];

  Object.defineProperty(this, "temperatura", {
    get: function () {
      console.log("get!");
      return temperatura;
    },
    set: function (value) {
      temperatura = value;
      arquivo.push({ val: temperatura });
    },
  });

  this.getArquivo = function () {
    return arquivo;
  };
}

var arq = new Arquivo();
arq.temperatura; // 'get!'
arq.temperatura = 11;
arq.temperatura = 13;
arq.getArquivo(); // [{ val: 11 }, { val: 13 }]
```

Veja também

- `Object.defineProperty()`
- `Object.defineProperties()`

SyntaxError: illegal character

Mensagem

```
SyntaxError: Invalid character (Edge)
SyntaxError: illegal character (Firefox)
SyntaxError: Invalid or unexpected token (Chrome)
```

Tipo do erro

`SyntaxError`

O que deu errado?

Há um token inválido ou inesperado que não pertence a esta posição no código. Utilize um editor que realce a sintaxe e cuidadosamente verifique seu código em relação a desajustes como um sinal de menos (-) no lugar de um travessão (–) ou aspas simples (') no lugar de aspas não padronizadas(“).

Exemplos

Caracteres desajustados

Alguns carateres parecem similares, mas irão impedir que o analisador interprete seu código. Exemplos famosos disso são as aspas, o sinal de menos ou o ponto e vírgula (o ponto de interrogação grego (U+37e) é quase igual)

```
“Isso parece uma string”;
// SyntaxError: illegal character

42 – 13;
// SyntaxError: illegal character
```

Isto deve funcionar:

```
"Isso é realmente uma string";

42 - 13;
```

Caracteres esquecidos

É fácil esquecer um caractere aqui ou ali.

```
var colors = ['#000', #333', '#666'];
// SyntaxError: illegal character
```

Adicione as aspas que faltam no `'#333'`.

```
var colors = ["#000", "#333", "#666"];
```

Caracteres escondidos

Quando copiamos e colamos código de fontes externas, pode haver caracteres inválidos. Cuidado!

```
var foo = 'bar';
// SyntaxError: illegal character
```

Quando inspeciona-se este código em um editor como o Vim, você pode ver que, na verdade, há um caractere zero-width space (ZWSP) (U+200B).

```
var foo = 'bar';<200b>
```

Veja também

- Lexical grammar

RangeError: invalid array length

Mensagem

```
RangeError: invalid array length (Firefox)
RangeError: Invalid array length (Chrome)
RangeError: Invalid array buffer length (Chrome)
```

Tipo de erro

`RangeError`

O que houve de errado?

Um comprimento inválido de array pode aparecer nas seguintes situações:

- Quando criamos um `Array` ou um `ArrayBuffer` que tem um comprimento que ou é negativo ou é maior ou igual a 232, ou
- quando configuramos a propriedade `Array.length` a um valor que é negativo, ou maior ou igual a 232.

Porque `Array` e `ArrayBuffer` tem um comprimento limitado? A propriedade `length` de um `Array` ou um `ArrayBuffer` é representado por um inteiro 32-bit unsigned, que pode apenas armazenar valores que estão no intervalo de 0 a 232-1.

Se você está criando um `Array`, utilizando o construtor, você provavelmente quer usar a notação literal, onde o primeiro argumento é interpretado como o comprimento do `Array`.

Ao contrário, você poderia querer travar o comprimento antes de ajustar a propriedade do comprimento, ou utilizá-lo com um argumento do construtor.

Exemplos

Casos inválidos

```
new Array(Math.pow(2, 40));
new Array(-1);
new ArrayBuffer(Math.pow(2, 32));
new ArrayBuffer(-1);

let a = [];
a.length = a.length - 1; // define -1 à propriedade length

let b = new Array(Math.pow(2, 32) - 1);
b.length = b.length + 1; // define 2^32 à propriedade length
```

Casos válidos

```
[Math.pow(2, 40)]; // [ 1099511627776 ]
[-1]; // [ -1 ]
new ArrayBuffer(Math.pow(2, 32) - 1);
new ArrayBuffer(0);

let a = [];
a.length = Math.max(0, a.length - 1);

let b = new Array(Math.pow(2, 32) - 1);
b.length = Math.min(0xffffffff, b.length + 1);

// 0xffffffff é a notação hexadecimal de 2^32 - 1
// que também pode ser escrito como (-1 >>> 0)
```

Veja também

- `Array`
- `Array.length`
- `ArrayBuffer`

ReferenceError: invalid assignment left-hand side

Mensagem

```
ReferenceError: invalid assignment left-hand side
```

Tipo do erro

`ReferenceError`.

O que deu errado?

Ouve uma declaração inesperada em algum lugar. Isso pode ocorrer devido a uma confusão entre um Operador de atribuição e um Operador de comparação, por exemplo. Enquanto um `"="` define uma variavel, `"=="` ou `"==="` são usados para fazer uma comparação entre valores.

Exemplos

```
if (Math.PI = 3 || Math.PI = 4) {
  console.log('de modo algum!');
}
// ReferenceError: invalid assignment left-hand side

var str = 'Olá, '
+= 'sou eu '
+= 'que você está procurando?';
// ReferenceError: invalid assignment left-hand side
```

Na declaração `if` você deve usar um operador de comparação ("=="), e para a atribuição da string, apenas o operador mais ("+") é necessario

```
if (Math.PI == 3 || Math.PI == 4) {
  console.log("de modo algum!");
}

var str = "Olá, " + "do " + "outro lado!";
```

Veja também

- Operadores de atribuição
- Operadores de comparação

TypeError: atribuição inválida para const "x"

Mensagem

```
TypeError: invalid assignment to const "x" (Firefox)
TypeError: Assignment to constant variable. (Chrome)
TypeError: Assignment to const (Edge)
TypeError: Redeclaration of const 'x' (IE)
```

Tipo de erro

`TypeError`

O que deu errado?

Uma constante é um valor que não pode ser alterado pelo programa durante a execução normal. Ele não pode mudar através de reatribuição e não pode ser redeclarado. Em JavaScript, as constantes são declaradas usando a palavra-chave `const`.

Exemplos

Redeclaração inválida

Atribuir um valor ao mesmo nome de constante no mesmo escopo de bloco lançará o erro.

```
const COLUNAS = 80;

// ...

COLUNAS = 120; // TypeError: invalid assignment to const `COLUNAS'
```

Corrigindo o erro

Existem várias opções para corrigir esse erro. Verifique o que se pretendia alcançar com a constante em questão.

Renomear

Se você quis declarar outra constante, escolha outro nome e renomeie. Esse nome de constante já está sendo usado nesse escopo.

```
const COLUNAS = 80;
const COLUNAS_LARGAS = 120;
```

`const`, `let` ou `var`?

Não use const se você não quis declarar uma constante. Talvez você quisesse declarar uma variável com escopo de bloco com `let` ou uma variável global com `var`.

```
let colunas = 80;

// ...

let colunas = 120;
```

Escopo

Verifique se você está no escopo correto. Essa constante deve aparecer nesse escopo ou deveria aparecer em uma função, por exemplo?

```
const COLUNAS = 80;

function configurarAmbienteTelaGrande() {
  const COLUNAS = 120;
}
```

<code>const</code> e imutabilidade

A declaração `const` cria uma referência somente leitura para um valor. Isso **não significa** que o valor que ela contém é imutável, apenas que o identificador da variável não pode ser reatribuído. Por exemplo, caso o conteúdo seja um objeto, isso significa que o objeto em si ainda pode ser alterado. Isso significa que você não pode alterar o valor armazenado em uma variável:

```
const obj = { foo: "bar" };
obj = { foo: "baz" }; // TypeError: invalid assignment to const `obj'
```

Mas você pode alterar as propriedades em uma variável:

```
obj.foo = "baz";
obj; // Object { foo: "baz" }
```

Veja também

- `const`
- `let`
- `var`

RangeError: invalid date

Mensagem

```
RangeError: invalid date (Firefox)
RangeError: invalid time value (Chrome)
RangeError: Provided date is not in valid range (Chrome)
```

Tipo de erro

`RangeError`

O que houve de errado?

Uma string apontando para uma data inválida foi disponibilizada para `Date` ou `Date.parse()`.

Exemplos

Casos inválidos

Strings irreconhecíveis ou data contendo elementos de strings formatadas com valores ISO ilegais, geralmente retornam `NaN`. No entanto, dependendo da implementação, strings com formatos em não conformidade ISO, também podem disparar `RangeError: invalid date`, como os seguintes casos no Firefox:

```
new Date("foo-bar 2014");
new Date("2014-25-23").toISOString();
new Date("foo-bar 2014").toString();
```

Isto, no entanto, retorna `NaN` no Firefox:

```
Date.parse("foo-bar 2014"); // NaN
```

Para mais detalhes, veja a documentação `Date.parse()`.

Casos válidos

```
new Date("05 October 2011 14:48 UTC");
```

Veja também

- `Date`
- `Date.prototype.parse()`
- `Date.prototype.toISOString()`

SyntaxError: as declarações da cabeça do laço for-in podem não ter inicializadores

O JavaScript em modo estrito-somente exceção "as declarações for-in loop head podem não ter inicializadores" isso ocorre quando a cabeça de um for...in contém uma expressão inicializadora, tal como `for (var i = 0 in obj)`. Isto não é permitido em laços for-in em modo estrito. Além disso, declarações léxicas com inicializadores como `for (const i = 0 in obj)` também não são permitidas fora do modo estrito.

Mensagem

```
SyntaxError: for-in loop variable declaration may not have an initializer. (V8-based)
SyntaxError: for-in loop head declarations may not have initializers (Firefox)
SyntaxError: a lexical declaration in the head of a for-in loop can't have an initializer (Firefox)
SyntaxError: Cannot assign to the loop variable inside a for-in loop header. (Safari)
```

Tipo de erro

`SyntaxError`.

O que ocorreu errado?

A cabeça de um laço for...in contém uma expressão inicializadora. Ou seja, uma variável é declarada e atribuída um valor `for (var i = 0 in obj)`. No modo não estrito, esta declaração de cabeça é silenciosamente ignorada e comporta-se como `for (var i in obj)`. No modo estrito, contudo, é lançado um `SyntaxError`. Além disso, declarações léxicas com inicializadores como `for (const i = 0 in obj)` também não são permitidas fora do modo estrito, e produzirão sempre um `SyntaxError`.

Exemplos

Esse exemplo lança um `SyntaxError`:

```
const obj = { a: 1, b: 2, c: 3 };

for (const i = 0 in obj) {
  console.log(obj[i]);
}

// SyntaxError: for-in loop head declarations may not have initializers
```

Laço for-in válido

Pode remover o inicializador (`i = 0`) na cabeça do laço de entrada.

```
const obj = { a: 1, b: 2, c: 3 };

for (const i in obj) {
  console.log(obj[i]);
}
```

Iteração de array

O laço for...in não deve ser usado para iteração de Array. Pretendia utilizar um laço `for` em vez de um laço `for-in` para iterar um `Array`? O laço `for` permite também definir um inicializador então:

```
const arr = ["a", "b", "c"];

for (let i = 2; i < arr.length; i++) {
  console.log(arr[i]);
}

// "c"
```

Veja também

- `for...in`
- `for...of` – also disallows an initializer in both strict and non-strict mode.
- `for` – preferred for array iteration, allows to define an initializer.

SyntaxError: uma declaração na cabeça de um laço for-of não pode ter um inicializador

A exceção JavaScript "a declaration in the head of a for-of loop can't have an initializer" ocorre quando a cabeça de um for...of loop contém uma expressão inicializadora tal como `for (const i = 0 of iteravel)`. Isto não é permitido no laço `for-of`.

Mensagem

```
SyntaxError: for-of loop variable declaration may not have an initializer. (V8-based)
SyntaxError: a declaration in the head of a for-of loop can't have an initializer (Firefox)
```

```
SyntaxError: Cannot assign to the loop variable inside a for-of loop header. (Safari)
```

Tipo de erro

`SyntaxError`

O que ocorreu errado?

The head of a for...of loop contains an initializer expression. That is, a variable is declared and assigned a value `for (const i = 0 of iterable)`. This is not allowed in for-of loops. You might want a `for` loop that does allow an initializer.

A cabeça de um laço for...of contém uma expressão inicializadora. Ou seja, uma variável é declarada e atribuída um valor `for (const i = 0 in iteravel)`. Isto não é permitido no laço for-of. Você pode querer utilizar um laço `for` que permita um inicializador.

Exemplos

Laços for-of inválidos

```
const iteravel = [10, 20, 30];

for (const valor = 50 of iteravel) {
  console.log(valor);
}

// SyntaxError: a declaration in the head of a for-of loop can't
// have an initializer
```

Laço for-of válido

É necessário remover o inicializador (`valor = 50`) na cabeça do laço `for-of`. Talvez pretendesse fazer de 50 um valor de offset, nesse caso poderia adicioná-lo ao corpo do laço, por exemplo.

```
const iteravel = [10, 20, 30];

for (let valor of iteravel) {
  valor += 50;
  console.log(valor);
}
// 60
// 70
// 80
```

Veja também

- `for...of`
- `for...in` – disallows an initializer in strict mode as well (SyntaxError: for-in loop head declarations may not have initializers)
- `for` – allows to define an initializer when iterating.

SyntaxError: JSON.parse: bad parsing

Mensagem

```
SyntaxError: JSON.parse: unterminated string literal
SyntaxError: JSON.parse: bad control character in string literal
SyntaxError: JSON.parse: bad character in string literal
SyntaxError: JSON.parse: bad Unicode escape
SyntaxError: JSON.parse: bad escape character
SyntaxError: JSON.parse: unterminated string
SyntaxError: JSON.parse: no number after minus sign
SyntaxError: JSON.parse: unexpected non-digit
SyntaxError: JSON.parse: missing digits after decimal point
SyntaxError: JSON.parse: unterminated fractional number
SyntaxError: JSON.parse: missing digits after exponent indicator
SyntaxError: JSON.parse: missing digits after exponent sign
SyntaxError: JSON.parse: exponent part is missing a number
SyntaxError: JSON.parse: unexpected end of data
SyntaxError: JSON.parse: unexpected keyword
SyntaxError: JSON.parse: unexpected character
SyntaxError: JSON.parse: end of data while reading object contents
SyntaxError: JSON.parse: expected property name or '}'
SyntaxError: JSON.parse: end of data when ',' or ']' was expected
SyntaxError: JSON.parse: expected ',' or ']' after array element
SyntaxError: JSON.parse: end of data when property name was expected
SyntaxError: JSON.parse: expected double-quoted property name
SyntaxError: JSON.parse: end of data after property name when ':' was expected
SyntaxError: JSON.parse: expected ':' after property name in object
SyntaxError: JSON.parse: end of data after property value in object
SyntaxError: JSON.parse: expected ',' or '}' after property value in object
SyntaxError: JSON.parse: expected ',' or '}' after property-value pair in object literal
SyntaxError: JSON.parse: property names must be double-quoted strings
SyntaxError: JSON.parse: expected property name or '}'
SyntaxError: JSON.parse: unexpected character
SyntaxError: JSON.parse: unexpected non-whitespace character after JSON data
```

Tipo de erro

`SyntaxError`

O que ocorreu de errado?

`JSON.parse()` converte uma string para JSON. A string precisa ser um JSON válido e retornará esse erro caso a sintaxe esteja errada.

Exemplos

<code>JSON.parse()</code> não permite vírgulas à direita

As duas linhas retornarão um SyntaxError:

```
JSON.parse("[1, 2, 3, 4,]");
JSON.parse('{"foo": 1,}');
// SyntaxError JSON.parse: unexpected character
// at line 1 column 14 of the JSON data
```

Retire as virgulas sobrando para converter o JSON corretamente:

```
JSON.parse("[1, 2, 3, 4]");
JSON.parse('{"foo": 1}');
```

Propriedades precisão ser escritas com aspas duplas

Você não pode usar aspas simples em propriedades, exemplo: 'foo'.

```
JSON.parse("{'foo': 1}");
// SyntaxError: JSON.parse: expected property name or '}'
// at line 1 column 2 of the JSON data
```

Em vez disso use "foo":

```
JSON.parse('{"foo": 1}');
```

Zeros à esquerda e pontos decimais

Você não pode usar zeros à esquerda, como 01, e pontos decimais precisam ser seguidos de pelo menos um digito.

```
JSON.parse('{"foo": 01}');
// SyntaxError: JSON.parse: expected ',' or '}' after property value
// in object at line 1 column 2 of the JSON data

JSON.parse('{"foo": 1.}');
// SyntaxError: JSON.parse: unterminated fractional number
// at line 1 column 2 of the JSON data
```

Em vez disso, use o 1 sem o zero, e pelo menos um digito após um ponto decimal:

```
JSON.parse('{"foo": 1}');
JSON.parse('{"foo": 1.0}');
```

Veja também

- `JSON`
- `JSON.parse()`
- `JSON.stringify()`

Erro de Sintaxe: faltando : depois da propriedade id

Mensagem

```
SyntaxError: missing : after property id
```

Tipo de erro

`SyntaxError`

O que deu errado?

Ao criar objetos com a sintaxe do iniciador do objeto, dois pontos (`:`) separam chaves e valores para as propriedades do objeto.

```
var obj = { propertyKey: "value" };
```

Exemplos

Dois-pontos contra sinal de igualdade

Este código falha, pois o sinal de igualdade não pode ser usado dessa maneira nesta sintaxe do iniciador do objeto.

```
var obj = { propertyKey = 'value' };
// SyntaxError: missing : after property id
```

Correto seria usar um dois-pontos, ou usar colchetes para atribuir uma nova propriedade depois que o objeto já foi criado.

```
var obj = { propertyKey: "value" };

// or alternatively

var obj = {};
obj["propertyKey"] = "value";
```

Propriedades vazia

Você não pode criar propriedades vazias como esta:

```
var obj = { propertyKey; };
// SyntaxError: missing : after property id
```

Se você precisa definir uma propriedade sem um valor, você pode usar `null` como um valor.

```
var obj = { propertyKey: null };
```

Propriedades computadas

Se você criar uma chave de propriedade de uma expressão, você precisa usar colchetes. Caso contrário, o nome da propriedade não pode ser computado:

```
var obj = { 'b'+'ar': 'foo' };
// SyntaxError: missing : after property id
```

Coloque a expressão entre parênteses `[]`:

```
var obj = { ["b" + "ar"]: "foo" };
```

Veja também

- Inicializador de objeto

SyntaxError: missing } after property list

Mensagem

```
SyntaxError: missing } after property list
```

Tipo de erro

`SyntaxError`

O que deu errado?

Aconteceu um engano na sintaxe do inicializador do objeto em algum lugar. Pode ser que você esqueceu de colocar uma chave, mas também pode ser uma vírgula que foi esquecida, por exemplo. Verifique também se alguma chave de finalização ou parêntesis estão em algum lugar que não deveriam estar. Indente ou formate o código de uma maneira legível pode te ajudar a enxergar no meio dessa selva.

Exemplos

Vírgula esquecida

Muitas vezes esquecemos uma vígula no inicializador de objeto:

```
var obj = {
  a: 1,
  b: { minhaProp: 2 }
  c: 3
};
```

O código correto deve ser:

```
var obj = {
  a: 1,
  b: { minhaProp: 2 },
  c: 3,
};
```

Veja também

- Inicializador de objeto

SyntaxError: missing formal parameter

Mensagem

```
SyntaxError: missing formal parameter (Firefox)
```

Tipo de error

`SyntaxError`

O que deu errado?

"Parâmetro Formal" é uma maneira elegante de dizer "parâmetro de função". Faltam parâmetros válidos em sua função. Na declaração de uma função os parâmetros devem ser identifiers, em vez de um valor como números, strings ou objetos. Declarar funções e chamar funções são dois passos diferentes. Declarações exigem identificadores como parâmetros e é apenas ao chamar (invocar) a função que você fornece os valores que a função deve usar.

Em JavaScript, identificadores podem obter apenas caracteres alfanuméricos (ou "$" e "_"), e não podem começar com um número. Um identificador difere de uma **string** porque uma string é um dado, enquanto que um identificador é parte do código.

Exemplos

Parâmetros de função devem ser identificadores ao se configurar uma função. Todas essas declarações de função abaixo falham, pois estão fornecendo valores para seus parâmetros:

```
function quadrado(3) {
  return numero * numero;
};
// SyntaxError: missing formal parameter

function saudacao("Olá") {
  return saudacao;
};
// SyntaxError: missing formal parameter

function log({ obj: "valor"}) {
  console.log(arg)
};
// SyntaxError: missing formal parameter
```

Você terá de usar identificadores nas declarações de função:

```
function quadrado(numero) {
  return numero * numero;
}

function saudar(saudacao) {
  return saudacao;
}

function log(arg) {
  console.log(arg);
}
```

You can then call these functions with the arguments you like:

```
quadrado(2); // 4

saudar("Olá"); // "Olá"

log({ obj: "valor" }); // Object { obj: "valor" }
```

Ver também

- Outros erros referentes a parâmetros formais:
  - SyntaxError: Parâmetro formal mal formado
  - SyntaxError: redeclaração de parâmetro formal "x"

Erro de sintaxe falta) depois da lista de argumentos

Mensagem

```
Erro de sintaxe: Falta ) depois da lista de argumento
```

Tipo de Erro

`Erro de sintaxe`.

O que houve de errado?

Ocorreu um erro quando a função foi chamada. Pode ter sido um erro de escrita, falta de operador, ou uma string fora das aspas, por exemplo.

Exemplos

Pela falta do operador "+" para fazer a concatenação da string, o JavaScript esperou um argumento para a função log ser "PI: ". Nesse caso, deveria ser finalizado com parênteses de fechamento ')'.

```
console.log("PI: " Math.PI);
// SyntaxError: missing ) after argument list
```

Você pode corrigir a chamada do log adicionand o operador "+":

```
console.log("PI: " + Math.PI);
// "PI: 3.141592653589793"
```

Veja também

- Funções

RangeError: repeat count must be non-negative

Mensagem

```
RangeError: argument out of range
RangeError: repeat count must be non-negative (Firefox)
RangeError: Invalid count value (Chrome)
```

Tipo de Erro

`RangeError`

O que está errado?

O método `String.prototype.repeat()` foi usado. Ele tem um parâmetro `count` indicando o número de vezes para repetir a string. Esse contador deve ser maior ou igual à 0 menor que `Infinity` (infinito positivo). O intervalo de vaores permitidos pode ser descrito por: [0, +∞).

Exemplos

Errado

```
"abc".repeat(-1); // RangeError
```

Certo

```
"abc".repeat(0); // ''
"abc".repeat(1); // 'abc'
"abc".repeat(2); // 'abcabc'
"abc".repeat(3.5); // 'abcabcabc' (count será convertido para inteiro)
```

Veja também

- `String.prototype.repeat()`

SyntaxError: missing variable name

Mensagem

```
SyntaxError: missing variable name (Firefox)
SyntaxError: Unexpected token = (Chrome)
```

Tipo de erro

`SyntaxError`

O que deu errado?

O nome de uma variável está faltando. Isto é provavelmente devido a um erro de sintaxe no seu código. Provavelmente uma vírgula está errada em algum lugar. Totalmente compreensível! Nomear as coisas é tão difícil.

Exemplos

Falta um nome de variável

```
var = "foo";
```

É difícil chegar com bons nomes de variáveis. Nós todos estivemos lá.

```
var ohGodWhy = "foo";
```

Palavras-chave reservadas não podem ser nomes de variáveis

Existem alguns nomes de variáveis que são palavras-chave resevadas. Você não pode usar isso. Desculpa :(

```
var debugger = "whoop";
// SyntaxError: missing variable name
```

Declarando múltiplas variáveis

Preste especial atenção às vírgulas ao declarar múltiplas variáveis. Existe um excesso de vírgula? Você acidentalmente adicionou vírgulas em vez de ponto e vírgula?

```
var x, y = "foo",
var x, = "foo"

var first = document.getElementById('one'),
var second = document.getElementById('two'),

// SyntaxError: missing variable name
```

A versão corrigida:

```
var x,
  y = "foo";
var x = "foo";

var first = document.getElementById("one");
var second = document.getElementById("two");
```

Arrays

`Array` literais em JavaScript precisam de colchetes ao redor dos valores. Isso não funcionará:

```
var arr = 1,2,3,4,5;
// SyntaxError: missing variable name
```

Isso seria correto:

```
var arr = [1, 2, 3, 4, 5];
```

Veja também

- Bons nomes de variáveis
- `var`
- Declarações de variáveis no Guia JavaScript

TypeError: "x" is not a function

Mensagem

```
TypeError: Object doesn't support property or method {x} (Edge)
TypeError: "x" is not a function
```

Tipo do erro

`TypeError`.

O que deu errado?

Houve a tentativa de obter um valor de uma função, mas o valor não é exatamente uma função. Algum código espera que você forneça uma função, mas não foi o que aconteceu.

Talvez exista um erro de digitação no nome da função? Talvez o objeto no qual você está chamando o método não tenha essa função? Por exemplo, objetos no JavaScript não tem a função `map`, mas o objeto Array tem.

Existem várias funções internas que precisam de uma função (callback). Você terá que prover uma função para obter esses métodos funcionando corretamente:

- Quando estiver trabalhando com os objetos `Array` ou `TypedArray`:
  - `Array.prototype.every()`, `Array.prototype.some()`, `Array.prototype.forEach()`, `Array.prototype.map()`, `Array.prototype.filter()`, `Array.prototype.reduce()`, `Array.prototype.reduceRight()`, `Array.prototype.find()`
- Quando estiver trabalhando com os objetos `Map` ou `Set`:
  - `Map.prototype.forEach()` e `Set.prototype.forEach()`

Exemplos

Um erro de digitação no nome da função

Nesse caso, que ocorre com bastante frequência, existe um erro de digitação no nome do método:

```
var x = document.getElementByID("foo");
// TypeError: document.getElementByID is not a function
```

O nome correto da função é `getElementById`:

```
var x = document.getElementById("foo");
```

Função chamada no objeto errado

Para alguns métodos, é necessário fornecer a função (callback) que irá funcionar apenas em objetos específicos. Nesse exemplo, é utilizado `Array.prototype.map()` que irá funcionar apenas em objetos `Array`.

```
var obj = { a: 13, b: 37, c: 42 };

obj.map(function (num) {
  return num * 2;
});

// TypeError: obj.map is not a function
```

Ao invés disso, use uma array:

```
var numbers = [1, 4, 9];

numbers.map(function (num) {
  return num * 2;
});

// Array [2, 8, 18]
```

Função compartilha o nome com uma propriedade pré-existente

Algumas vezes, ao criar uma classe, pode haver uma propriedade e função com os mesmos nomes. Ao chamar a função, o compilador não consegue identificar a função.

```
var Dog = function () {
  this.age = 11;
  this.color = "black";
  this.name = "Ralph";
  return this;
};

Dog.prototype.name = function (name) {
  this.name = name;
  return this;
};

var myNewDog = new Dog();
myNewDog.name("Cassidy"); //Uncaught TypeError: myNewDog.name is not a function
```

Então, use um nome diferente para a propriedade

```
var Dog = function () {
  this.age = 11;
  this.color = "black";
  this.dogName = "Ralph"; //Using this.dogName instead of .name
  return this;
};

Dog.prototype.name = function (name) {
  this.dogName = name;
  return this;
};

var myNewDog = new Dog();
myNewDog.name("Cassidy"); //Dog { age: 11, color: 'black', dogName: 'Cassidy' }
```

Veja também

- Funções

RangeError: argument is not a valid code point

Mensagem

```
RangeError: Invalid code point {0} (Edge)
RangeError: {0} is not a valid code point (Firefox)
RangeError: Invalid code point {0} (Chrome)
```

Tipo de Erro

`RangeError`

O que está errado?

`String.fromCodePoint()` dispara esse erro quando são passados valores `NaN`, inteiros negativos (-1), não inteiros (5.4), ou valores maiores que 0x10FFFF (1114111).

Um code point é um valor Unicode; isto é,é um valor inteiro entre `0` e `0x10FFFF`.

Exemplos

Errado

```
String.fromCodePoint("_"); // RangeError
String.fromCodePoint(Infinity); // RangeError
String.fromCodePoint(-1); // RangeError
String.fromCodePoint(3.14); // RangeError
String.fromCodePoint(3e-2); // RangeError
String.fromCodePoint(NaN); // RangeError
```

Certo

```
String.fromCodePoint(42); // "*"
String.fromCodePoint(65, 90); // "AZ"
String.fromCodePoint(0x404); // "\u0404"
String.fromCodePoint(0x2f804); // "\uD87E\uDC04"
String.fromCodePoint(194564); // "\uD87E\uDC04"
String.fromCodePoint(0x1d306, 0x61, 0x1d307); // "\uD834\uDF06a\uD834\uDF07"
```

Veja também

- `String.fromCodePoint()`

ReferenceError: "x" não está definido

Mensagem

```
ReferenceError: "x" is not defined
```

Tipo de erro

`ReferenceError`

O que deu errado?

Há uma variavel inexistente referenciada em algum lugar. Essa variável precisa ser declarada ou você precisa ter certeza que ela está disponível no seu atual script ou escopo.

> **Nota:** Quando carregar uma biblioteca (como o JQuery) tenha certeza que ela está carregada antes que você acesse as variáveis dela, como "$". Coloque na tag `<script>` para carregar a biblioteca antes do seu código usá-lo.

Exemplos

Variável não declarada

```
foo.substring(1); // ReferenceError: foo is not defined
```

A variável "foo" não está definida em lugar nenhum. Ela precisa ser uma string e assim o método `String.prototype.substring()` irá funcionar.

```
var foo = "bar";
foo.substring(1); // "ar"
```

Escopo Errado

Uma variável precisa estar disponível no atual contexto de execução. Variáveis definidas dentro de uma function não podem ser acessadas de outros lugares fora da função, porque a variável é definida apenas no escopo da função

```
function numbers() {
  var num1 = 2,
    num2 = 3;
  return num1 + num2;
}

console.log(num1); // ReferenceError num1 is not defined.
```

Entretanto, uma função pode acessar todas as variáveis e funções definidas dentro do escopo no qual elas estão definidas. Em outras palavras, uma função definida no escopo global pode acessar todas as variáveis no escopo global.

```
var num1 = 2,
  num2 = 3;

function numbers() {
  return num1 + num2;
}

console.log(num1); // 2
```

Veja também

- Scope

- Declarando variáveis no JavaScript
- Escopo de função no JavaScript

RangeError: precision is out of range

A exceção JavaScript "precision is out of range" (precisão fora de alcance) ocorre quando um número que está fora do intervalo que vai de 0 à 20 (ou 21) foi passado para `toFixed` ou `toPrecision`.

Mensagem

```
RangeError: The number of fractional digits is out of range (Edge)
RangeError: The precision is out of range (Edge)
RangeError: precision {0} out of range (Firefox)
RangeError: toExponential() argument must be between 0 and 20 (Chrome)
RangeError: toFixed() digits argument must be between 0 and 20 (Chrome)
RangeError: toPrecision() argument must be between 1 and 21 (Chrome)
```

Tipo de erro

`RangeError`

O que houve de errado?

Existe um argumento de precisão fora do intervalo em um desses métodos:

- `Number.prototype.toExponential()`
- `Number.prototype.toFixed()`
- `Number.prototype.toPrecision()`

Po padrão, o intervalo permitido para os métodos acima é de 0 à 20 (ou 21). Todavia, a especificação ECMAScript nos permite extender o intervalo.

Método	Firefox (SpiderMonkey)	Chrome, Opera (V8)
`Number.prototype.toExponential()`	0 to 100	0 to 20
`Number.prototype.toFixed()`	-20 to 100	0 to 20
`Number.prototype.toPrecision()`	1 to 100	1 to 21

Exemplos

Casos inválidos

```
(77.1234).toExponential(-1); // RangeError
(77.1234).toExponential(101); // RangeError

(2.34).toFixed(-100); // RangeError
(2.34).toFixed(1001); // RangeError

(1234.5).toPrecision(-1); // RangeError
(1234.5).toPrecision(101); // RangeError
```

Casos válidos

```
(77.1234).toExponential(4); // 7.7123e+1
(77.1234).toExponential(2); // 7.71e+1

(2.34).toFixed(1); // 2.3
(2.35).toFixed(1); // 2.4 (perceba que ele foi arredondado nesse caso)

(5.123456).toPrecision(5); // 5.1235
(5.123456).toPrecision(2); // 5.1
(5.123456).toPrecision(1); // 5
```

Veja também

- `Number.prototype.toExponential()`
- `Number.prototype.toFixed()`
- `Number.prototype.toPrecision()`

Error: Permission denied to access property "x"

Mensagem

```
Error: Permission denied to access property "x"
```

Tipo de erro

`Error`.

O que houve de errado?

Tentativa de acessar um objeto no qual você não possui permissões. Provavelmente um `<iframe>` carregado de um dominio diferente no qual foi violado o same-origin policy.

Exemplos

```
<!doctype html>
<html>
  <head>
    <iframe
      id="myframe"
      src="http://www1.w3c-test.org/common/blank.html"></iframe>
    <script>
      onload = function () {
        console.log(frames[0].document);
        // Error: Permission denied to access property "document"
      };
    </script>
  </head>
  <body></body>
</html>
```

Veja também

- `<iframe>`
- Same-origin policy

TypeError: "x" is read-only

Mensagem

```
TypeError: "x" is read-only (Firefox)
TypeError: 0 is read-only (Firefox)
TypeError: Cannot assign to read only property 'x' of #<Object> (Chrome)
TypeError: Cannot assign to read only property '0' of [object Array] (Chrome)
```

Tipo de Erro

`TypeError`

O que deu errado?

A variável global ou propriedade do objeto foi definida como propriedade somente-leitura. (Tecnicamente, esse é um dado de não-escrita.)

Esse erro ocorre apenas em código no strict mode. No modo não strict mode, essa atribuição é ignorada silenciosamente.

Exemplos

Casos inválidos

Propriedades somente-leitura não são super comuns, mas elas podem ser criadas utilizando `Object.defineProperty()` ou `Object.freeze()`.

```
"use strict";
var obj = Object.freeze({ nome: "Elsa", pontuacao: 157 });
obj.pontuacao = 0; // TypeError

("use strict");
Object.defineProperty(this, "CONTADOR_PULMAO", { value: 2, writable: false });
CONTADOR_PULMAO = 3; // TypeError

("use strict");
var frozenArray = Object.freeze([0, 1, 2]);
frozenArray[0]++; // TypeError
```

Existem também algumas propriedades somente-leitura nativas do Javascript. Talvez você já tentou redefinir um constante matemática.

```
"use strict";
Math.PI = 4; // TypeError
```

Desculpe, você não pode fazer isso.

A variável global `undefined` também é somente-leitura, então você não pode silenciar o infame erro "undefined is not a function" fazendo isso:

```
"use strict";
undefined = function () {}; // TypeError: "undefined" is read-only
```

Casos válidos

```
"use strict";
var obj = Object.freeze({ nome: "Score", pontos: 157 });
obj = { nome: obj.nome, pontos: 0 }; // substituindo com o novo objeto funciona

("use strict");
var CONTADOR_PULMAO = 2; // uma `var` funciona, porque ela não é somente-leitura
CONTADOR_PULMAO = 3; // ok (anatomicamente improvável, porém...)
```

Veja também

- `Object.defineProperty()`
- `Object.freeze()`
- "Quais animais tem três pulmões?" no answers.com
- Klingons (uma outra resposta para essa pergunta)

SyntaxError: redeclaration of formal parameter "x"

Mensagem

```
SyntaxError: Let/Const redeclaration (Edge)
SyntaxError: redeclaration of formal parameter "x" (Firefox)
SyntaxError: Identifier "x" has already been declared (Chrome)
```

Tipo de Erro

`SyntaxError`

O que deu errado?

O mesmo nome de variável ocorre como parâmetro da função e é então redeclarada atribuindo `let` no corpo da função. Redeclarando a mesma variável dentro da mesma função ou no escopo do bloco usando `let` não é permitido em JavaScript.

Exemplos

Neste caso, a variável "arg" redeclara o argumento.

```
function f(arg) {
  let arg = "foo";
}

// SyntaxError: redeclaration of formal parameter "arg"
```

Se você quiser mudar o valor de "arg" no corpo da função, você pode fazer, mas você não precisa declarar novamente a mesma variável. Em outras palavras: você pode omitir a palavra chave `let`. Se você quiser criar uma nova variável, você precisa renomea-la, pois isso conflita com o parâmetro da função.

```
function f(arg) {
  arg = "foo";
}

function f(arg) {
  let bar = "foo";
}
```

Notas de compatibilidade

- Antes do Firefox 49, isto foi lançado como um `TypeError` (Erro do Firefox 1275240).

Veja também

- `let`
- `const`
- `var`
- Declaring variables in the JavaScript Guide

Warning: unreachable code after return statement

Mensagem

```
Warning: unreachable code after return statement (Firefox)
```

Tipo de Erro

Warning

O que deu errado?

Codigo inacessível depois da declaração de retorno pode ocorrer nas seguintes situações:

- Quando usado em uma expressão após a declaração de um `return` , ou
- quando usado em uma expressão **return** sem ponto e vírgula porém incluindo uma expressão diretamente depois.

Quando existe uma expressão após uma declaração válida de `return`, um warning é dado para indicar que o código depois da declaração `return` é inacessível, significando que ele pode nunca ser executado.

Porque eu devo usar ponto-e-virgula após declarações `return`? No caso de declarações `return` sem ponto-e-vírgula, ele pode deixar obscuro se o desenvolvedor quis usar o return na proxima linha, ou parar a execução e retornar. O warning indica que há uma ambiguidade no modo que a declaração `return` foi escrita.

Warnings não serão mostrado para declarações return sem ponto-e-vírgula nas seguintes situações:

- `throw`
- `break`
- `var`

- `function`

Exemplos

Casos Inválidos

```
function f() {
  var x = 3;
  x += 4;
  return x; // return exits the function immediately
  x -= 3; // so this line will never run; it is unreachable
}

function f() {
  return; // this is treated like `return;`
  3 + 4; // so the function returns, and this line is never reached
}
```

Casos Válidos

```
function f() {
  var x = 3;
  x += 4;
  x -= 3;
  return x; // OK: return after all other statements
}

function f() {
  return 3 + 4; // OK: semicolon-less return with expression on the same line
}
```

Ver também

- Automatic Semicolon Insertion

SyntaxError: unterminated string literal

Mensagem

```
SyntaxError: unterminated string literal
```

Tipo de Erro

`SyntaxError`

O que deu errado?

Há uma `String` não finalizada em algum lugar do código. String literais devem se fechadas com aspas simples (`'`) ou duplas (`"`) . O JavaScript não faz distinção entre strings com aspas simples ou aspas duplas. As sequências de escape trabalham com strings criadas tanto com aspas simples, quanto com aspas duplas. Para consertar este erro, verifique se:

- você abriu e fechou sua string literal com aspas (simples ou duplas),
- você escapou sua string literal corretamente,
- sua string funciona corretamente em várias linhas, se houver.

Exemplos

Múltiplas linhas

Você não pode dividir sua string em múltiplas linhas em Javascript:

```
var longString = "This is a very long string which needs
                 to wrap across multiple lines because
                 otherwise my code is unreadable.";
// SyntaxError: unterminated string literal
```

Ao invés disso, você deve utilizar o operador + operator, uma barra invertida(\), ou modelos literais. Uma forma de uso do operador +, seria assim:

```
var longString =
  "This is a very long string which needs " +
  "to wrap across multiple lines because " +
  "otherwise my code is unreadable.";
```

Ou você pode usar uma barra invertida ("\") ao final de cada linha para indicar que a string continuará na próxima linha. Certifique-se de que não há espaço ou qualquer outro caractere após a barra invertida (excento quando for uma quebra de linha), ou como indentação; caso contrário, não funcionará. Esta forma ficaria assim:

```
var longString =
  "This is a very long string which needs \
to wrap across multiple lines because \
otherwise my code is unreadable.";
```

Uma outra possibilidade é usar um modelo literal, que é suportado pelos ambientes ECMAScript 2015:

```
var longString = `This is a very long string which needs
                  to wrap across multiple lines because
                  otherwise my code is unreadable.`;
```

Veja também

- `String`
- Modelos literais

InternalError: too much recursion

Mensagem

```
Error: Out of stack space (Edge)
InternalError: too much recursion (Firefox)
RangeError: Maximum call stack size exceeded (Chrome)
```

Tipo de erro

`InternalError`.

O que deu errado?

Uma função que invoca a si mesma é chamada *função recursiva*. Assim que uma condição é atendida, a função para de se invocar.

De certa forma, recursão é análoga a um loop. Ambos executam o mesmo código múltiplas vezes, e ambos exigem uma condição (para evitar um loop infinito ou, nesse caso, uma recursão infinita). Quando são feitas excessivas invocações, ou a função não possui uma condição para interromper a recursividade, este erro é lançado.

Exemplos

Esta função recursiva roda 10 vezes, como definido pela condição de saída.

```
function loop(x) {
  if (x >= 10) {
    // "x >= 10" é a condição de saída
    return;
  }
  loop(x + 1); // o código recursivo em si
}
loop(0);
```

Mudar esta condição para um valor extremamente alto não vai funcionar:

```
function loop(x) {
  if (x >= 1000000000000) return;
  loop(x + 1);
}
loop(0);

// InternalError: too much recursion
```

A seguinte função recursiva não possui condição de saída. Assim, ela vai continuar se invocando indefinidamente (até que o erro seja lançado e a execução interrom).

```
function loop(x) {
  // Não há condição de saída

  loop(x + 1); // Código recursivo
}

loop(0);

// InternalError: too much recursion
```

Veja também

- Recursion
- Funções recursivas

ReferenceError: Atribuição à variável não declarada "x"

Message

```
ReferenceError: Atribuição à variável não declarada "x"(Firefox)
ReferenceError: "x" não está definido (Chrome)
ReferenceError: Variável indefinida em strict mode (Edge)
```

Tipo de erro

`ReferenceError` aviso em *strict mode* somente.

O que deu errado?

Um valor foi atribuído a uma variável não declarada. Em outras palavras, houve uma atribuição sem a palavra-chave var. Existem algumas diferenças entre as variáveis declaradas e não declaradas, o que pode levar a resultados inesperados e é por isso que o JavaScript apresenta um erro no modo estrito.

Três coisas a observar sobre as variáveis declaradas e não declaradas:

- Variáveis declaradas são limitadas no contexto de execução em que são declarados. Variáveis não declaradas são sempre globais.
- Variáveis declaradas são criados antes de qualquer código ser executado. Variáveis não declaradas não existem até que o código atribuido for executado.
- Variáveis declaradas são uma propriedade não-configurável de seu contexto de execução (função ou global). Variáveis não declaradas são configuráveis (por exemplo, pode ser excluído).

Para mais exemplos, veja a página de referência `var`.

Erros sobre atribuições de variáveis não declaradas ocorrem apenas em código de modo estrito. Em código não-estrito, eles são silenciosamente ignorados.

Exemplos

Casos inválidos

neste caso, a variavel "bar" é uma variavel não declarada.

```
function foo() {
  "use strict";
  bar = true;
}
foo(); // ReferenceError: assignment to undeclared variable bar
```

Casos válidos

Para fazer uma variável declarada "bar", você pode adicionar a palavra-chave var na frente dele.

```
function foo() {
  "use strict";
  var bar = true;
}
foo();
```

Veja também

- *Strict mode*

SyntaxError: Unexpected token

Mensagem

```
SyntaxError: expected expression, got "x"
SyntaxError: expected property name, got "x"
SyntaxError: expected target, got "x"
SyntaxError: expected rest argument name, got "x"
SyntaxError: expected closing parenthesis, got "x"
SyntaxError: expected '=>' after argument list, got "x"
```

Tipo de erro

`SyntaxError`

O que deu errado?

Uma construção específica da linguagem era esperada, mas algo não foi fornecido. Isto deve ser um simples erro de digitação.

Exemplos

Expressão esperada

Por exemplo, quando funções são chamadas, vírgulas sem um valor na sequência não são permitidas. O JavaScript esperarará outro argumento, como pode ser qualquer expressão.

```
Math.max(2, 42,);
// SyntaxError: expected expression, got ')'
```

O correto é omitir a vírgula ou adicionar outro argumento:

```
Math.max(2, 42);
Math.max(2, 42, 13 + 37);
```

Veja também

- `Math.max()`

TypeError: "x" is (not) "y"

Mensagem

```
Tipo do erro: "x" é (não é) "y"

Exemplos:
Tipo do erro: "x" é indefinido
Tipo do erro: "x" é nulo
Tipo do erro: "x" indefinido" não é um objeto
Tipo do erro: "x" não é um objeto ou é nulo
Tipo do erro: "x" não é um símbolo
```

Tipo de erro

`TypeError`.

O que deu errado?

Houve um tipo inesperado. Isso ocorre muitas vezes com valores `undefined` ou `null`.

Além disso, certos métodos, tais como `Object.create()` ou `Symbol.keyFor()`, requerem um tipo específico, que deve ser fornecido.

Exemplos

Casos inválidos

```
// casos undefined e null nos quais o método substring não vai funcionar
var foo = undefined;
foo.substring(1); // TypeError: foo is undefined

var foo = null;
foo.substring(1); // TypeError: foo is null

// Certos métodos podem precisar de um tipo específico
var foo = {};
Symbol.keyFor(foo); // TypeError: foo is not a symbol

var foo = "bar";
Object.create(foo); // TypeError: "foo" is not an object or null
```

Consertando o problema

Para consertar o pointeiro nulo para valores `undefined` ou `null`, você pode utilizar o operador typeof , por exemplo.

```
if (typeof foo !== "undefined") {
  // Agora nós sabemos que foo está definido, então podemos prosseguir.
}
```

Veja também

- `undefined`
- `null`

Erro de sintaxe: declaração de função requer um nome

Mensagem

```
Errodesintaxe: Identificador Esperado(Edge)
Errodesintaxe: declaração de função requer um nome [Firefox]
Errodesintaxe: Token inesperado ( [Chrome]
```

Tipo de erro

`Errodesintaxe`

O que estava errado?

Existe uma declaração no código que requer um nome. Você precisa checar como as funções são definidas e se você precisa providenciar um nome, se a função em questão precisa ser uma expressão de função, um IIFE ou se o código da função está colocado corretamente neste contexto.

Exemplos

Statements vs expressions

Uma *declaração de função* (ou *declaração de função*) requer um nome, isso não vai funcionar:

```
function () {
  return 'Olha mundo';
}
// SyntaxError: function statement requires a name
```

Você pode usar uma expressão de função ao invés de uma atribuição.

```
var saudar = function () {
  return "Ola mundo";
};
```

Ou, sua função pode ser pretendida a ser uma IIFE (Immediately Invoked Function Expression), qual é uma função que será em breve definida. Você vai precisar de um pouco mais de colchetes neste caso:

```
(function () {})();
```

Funçoes etiquetadas

Se usar labels, precisará providenciar um nome de função depois da palavra `function` . Isto não funciona:

```
function Saudacao() {
  alemao: function () {
    return "Moin";
  }
}
// SyntaxError: a função declaração requer um nome
```

Isso funciona ,veja o exemplo:

```
function Saudacao() {
  alemao: function g() {
    return "Moin";
  }
}
```

Métodos de Objetos

Se pretende criar um metodo para um objeto, voce precisa-rá criar um objeto (hehehe). A seguir uma sintaxe sem nome depois de `function` é válida.

```
var saudacao = {
  alemao: function () {
    return "Moin";
  },
};
```

Callback Sintaxe

Alem disso,cheque sua sintaxe usando callbacks. Colchetes e virgulas ficam facilmente atrapalhar e dificultar.

```
promessa.then(
  function() {
    console.log("sucesso");
  });
  function() {
    console.log("erro");
}
// SyntaxError: function statement requires a name
```

O correto seria:

```
promise.then(
  function() {
    console.log("success");
  },
  function() {
    console.log("error");
  }
);//sempre que abrir feche();
```

Veja também

- Guia de funções em JavaScript
- Funções de Declaração
- Funções de Expressão
- Funções imediatas (IIFE)
- Etiqueta

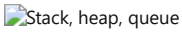
Modelo de Concorrência e Event Loop

O JavaScript possui um modelo de concorrência baseado em um **event loop** (laço de eventos), em português), responsável pela execução do código, coleta e processamento de eventos e execução de subtarefas enfileiradas. Este modelo é bem diferente de outras linguagens, como C ou Java, por exemplo.

Conceitos de runtime (tempo de execução)

Os próximos tópicos irão explicar teoricamente o modelo. Interpretadores modernos de JavaScript implementam e otimizam fortemente as semânticas descritas.

Representação visual



Pilha (Stack)

As chamadas de funções criam uma pilha de *frames* (quadros).

```
function foo(b) {
  let a = 10;
  return a + b + 11;
}

function bar(x) {
  let y = 3;
  return foo(x * y);
}

const baz = bar(7); // atribui 42 a baz
```

Ordem das operações:

1. Quando chamamos a função `bar`, o primeiro *frame* é criado contendo argumentos e variáveis locais de `bar`.
2. Quando a função `bar` chama `foo`, o segundo *frame* é criado e é colocado no topo da pilha, contendo os argumentos e as variáveis locais de `foo`.
3. Quando `foo` retorna, o *frame* do topo é removido da pilha (deixando apenas o *frame* da chamada de `bar`).
4. Quando `bar` retorna, a pilha fica vazia.

Note que os argumentos e variáveis locais podem continuar existindo, pois são armazenados fora da pilha — de forma que podem ser acessados por quaisquer funções aninhadas muito depois de sua função externa ter retornado.

Acervo (Heap)

Os objetos são alocados em um *heap* (acervo), que é apenas um nome para denotar uma grande região não estruturada da memória.

Fila (Queue)

O *runtime* do JavaScript contém uma fila de mensagens, que é uma lista de mensagens a serem processadas. Cada mensagem é associada a uma função, que é chamada para lidar com a mensagem.

Em algum ponto durante o *event loop* (laço de eventos), o *runtime* começa a manipular as mensagens na fila, começando pela mais antiga. Para fazer isso, a mensagem é removida da fila e sua função correspondente é chamada com a mensagem como um parâmetro de *input* (entrada). Como de costume, chamar uma função cria um novo *frame* (quadro) na pilha para uso dessa função.

O processamento de funções continua até que a pilha fique novamente vazia, então o *event loop* processará a próxima mensagem na fila (se houver uma).

Laço de Eventos (Event loop)

O ***Event loop*** tem esse nome por causa da forma que normalmente é implementado, que se assemelha a:

```
while (queue.waitForMessage()) {
  queue.processNextMessage();
}
```

`queue.waitForMessage` aguarda, de maneira síncrona, receber uma mensagem (se não houver nenhuma já disponível esperando para ser tratada).

Processamento Completo ("Run-to-completion")

Cada mensagem é processada completamente antes de outra mensagem ser processada. Isso oferece um bom fundamento ao pensar sobre o seu *software*, incluindo o fato de que, independente de quando uma função é executada, ela não pode ser interrompida e, portanto, será executada por completo antes que outro código rode (e modifique dados manipuláveis pela função). Isso é diferente do C, por exemplo, no qual uma função que está sendo executada em uma *thread* (um fluxo de execução), pode ser interrompida a qualquer momento para executar um outro código em outra *thread*.

Um aspecto negativo deste modelo é que se uma mensagem levar muito tempo para ser finalizada, a aplicação web ficará indisponível para processar as interações do usuário, como cliques ou rolagens. O navegador mitiga este problema através do aviso: "*Um script desta página pode estar ocupado, ou parou de responder*". Uma boa prática a seguir é fazer o processamento de mensagens curtas, e, se possível, dividir uma longa mensagem em múltiplas mensagens menores.

Adicionando mensagens

Nos navegadores, as mensagens são adicionadas a qualquer momento que um evento é acionado se este possuir um *listener* (ouvinte). Caso não haja, o evento será ignorado. Assim, um clique em um elemento com um manipulador de eventos de clique adicionará uma mensagem, como qualquer outro evento.

A função `setTimeout` é chamada com 2 argumentos: uma mensagem para adicionar à fila (queue) e um valor em tempo (opcional, o padrão é `0`). O valor de tempo (*time value*) representa o intervalo (mínimo) com que a mensagem será realmente enviada à fila. Se não houver outra mensagem na fila, a mensagem será processada logo após o intervalo. No entanto, caso haja mensagens, a mensagem `setTimeout` terá que esperar até que outras mensagens sejam processadas. Por esse motivo, o segundo argumento indica um tempo *mínimo* e não um tempo *garantido*.

Aqui está um exemplo que demonstra esse conceito (`setTimeout` não é executado imediatamente após o temporizador expirar):

```
const s = new Date().getSeconds();

setTimeout(function () {
  // imprime "2", o que significa que o callback não é chamado imediatamente após 500 milissegundos.
  console.log("Ran after " + (new Date().getSeconds() - s) + " seconds");
}, 500);
```

```
while (true) {
  if (new Date().getSeconds() - s >= 2) {
    console.log("Good, looped for 2 seconds");
    break;
  }
}
```

Intervalos de zero segundos

O intervalo zero não significa, necessariamente, que o *callback* será disparado após zero milissegundos. Chamar `setTimeout` com um intervalo de `0` (zero) milissegundos não executa a função do *callback* após intervalo dado.

A execução depende do número de mensagens em espera na fila. No exemplo abaixo, a mensagem `''this is just a message''` será escrita no console antes que a mensagem do *callback* seja processada, porque, como informado previamente, o intervalo definido na função indica o tempo *mínimo* necessário para que a aplicação processe a requisição e não o tempo *exato* de processamento.

Basicamente, `setTimeout` precisa esperar que todo o código das mensagens enfileiradas seja concluído, mesmo que você tenha especificado um tempo limite específico para o seu `setTimeout`.

```
(function () {
  console.log("this is the start");

  setTimeout(function cb() {
    console.log("Callback 1: this is a msg from call back");
  }); // tem um valor de tempo padrão de 0

  console.log("this is just a message");

  setTimeout(function cb1() {
    console.log("Callback 2: this is a msg from call back");
  }, 0);

  console.log("this is the end");
})();

// "this is the start"
// "this is just a message"
// "this is the end"
// "Callback 1: this is a msg from call back"
// "Callback 2: this is a msg from call back"
```

Múltiplos runtimes comunicando-se em conjunto

Um *web worker* ou um `iframe` com uma diferente origem (cross-origin) tem seu próprio *stack*, *heap* e *queue* de messagens. Dois *runtimes* distintos só podem se comunicar por meio do envio de mensagens via método `postMessage`. Este método adiciona uma mensagem ao outro *runtime*, se este escutar os eventos de `message`.

Sem bloqueio

Uma propriedade muito interessante do modelo *event loop* é que o JavaScript, ao contrário de muitas outras linguagens, nunca bloqueia. A manipulação de E/S (*I/O*) é tipicamente realizada através de eventos e *callbacks*. Portanto, quando uma aplicação está esperando pelo retorno de uma consulta do IndexedDB ou de uma requisição XHR, ela ainda pode processar outras coisas, como as ações do usuário.

Exceções de legado existem, como por exemplo, `alert` ou XHR síncrono, mas é considerado uma boa prática evitá-los. Tome cuidado, exceções a exceção existem (mas geralmente são, mais do que qualquer coisa, *bugs* de implementação).

Veja também

- Event loops no HTML standard
- Node.js Event Loop

Funções

De modo geral, função é um "subprograma" que pode ser *chamado* por código externo (ou interno no caso de recursão) à função. Assim como o programa em si, uma função é composta por uma sequência de instruções chamada *corpo da função*. Valores podem ser *passados* para uma função e ela vai *retornar* um valor.

Em JavaScript, funções são objetos de primeira classe, pois elas podem ter propriedades e métodos como qualquer outro objeto. O que as difere de outros objetos é que as funções podem ser chamados. Em resumo, elas são objetos `Function`.

Para mais exemplos e explicações, veja também o Guia de JavaScript sobre funções.

Descrição

Toda função em JavaScript é um objeto `Function`. Veja `Function` para informação das propriedades e métodos dos objetos `Function`.

Funções não são como procedimentos (*procedure*). Uma função sempre retorna um valor, mas um procedimento pode ou não retornar um valor.

Para retornar um valor diferente do padrão, uma função deve ter uma instrução return que especifica o valor a ser retornado. Uma função sem um `return` retornará um valor padrão. No caso de um método construtor chamado com a palavra reservada `new`, o valor padrão é o valor do parâmetro `this`. Para todas as outras funções, o valor padrão de retorno é `undefined`.

Os parâmetros de uma função são chamados de argumentos da função. Argumentos são passados para a função *por valor*. Se uma função muda o valor de um argumento, esta mudança não é refletida globalmente ou na chamada da função. Contudo, referência de objetos são valores também, e eles são especiais: se a

função muda as propriedades do objeto referenciado, estas mudanças são visíveis fora da função, como é mostrado no exemplo a seguir:

```
/* Declare a função 'minhaFunção' */
function minhaFuncao(objeto) {
  objeto.marca = "Toyota";
}

/*
 * Declare a variável 'meucarro';
 * crie e inicialize um novo Objeto;
 * atribua referência para 'meucarro'
 */
var meucarro = {
  marca: "Honda",
  modelo: "Accord",
  ano: 1998,
};

/* Exibe 'Honda' */
console.log(meucarro.marca);

/* Passe a referência do objeto para a função */
minhaFuncao(meucarro);

/*
 * Exibe 'Toyota' como valor para a propriedade 'marca'
 * do objeto, mudado pela função.
 */
console.log(meucarro.marca);
```

A palavra reservada `this` não se refere a função sendo executada no momento, então você deve referenciar um objeto `Function` pelo nome, mesmo dentro do corpo da função.

Definindo funções

Há várias maneiras de definir funções:

Declaração de função (Instrução <code>function</code>)

Há um sintaxe especial para declarar funções (veja Instrução function para detalhes):

```
function nome([param[, param[, ... param]]]) {
   instruções
}
```

`nome`

O nome da função.

`param`

O nome de um argumento a ser passado para a função.

`instruções`

As instruções que formam o corpo da função.

A expressão function (Operador <code>function</code>)

Uma expressão function é parecida com uma declaração function e tem até a mesma sintaxe (veja Operador function para detalhes):

```
function [nome]([param] [, param] [..., param]) {
   instruções
}
```

`nome`

O nome da função. Pode ser omitido, e neste caso a função é conhecida como função anônima.

`param`

O nome de um argumento a ser passado para a função.

`instruções`

As instruções que formam o corpo da função.

O gerador de declaração de função (instrução <code>function*</code>)

> **Nota:** Expressão Arrow function é uma tecnologia experimental, parte da proposta do ECMAScript 6, e ainda não é amplamente suportada pelos navegadores.

Uma expressão arrow function tem uma sintaxe curta e conecta seu valor lexicamente (veja arrow functions para mais detalhes):

```
([param] [, param]) => {
   instruções
}

param => expressão
```

`param`

O nome de um argumento. Quando não há argumentos deve ser indicado com o uso dos parênteses ( ). Para apenas um argumento os parênteses não são obrigatórios. (por exemplo `foo => 1`)

`instruções ou expressão`

Múltiplas instruções precisam ser envolvidas por chaves. Uma única expressão não requer chaves. A expressão também é implicitamente o valor de retorno dessa função.

O construtor <code>Function</code>

> **Nota:** O uso do construtor Function para criar funções não é recomendado uma vez que é requerido o corpo da função como string, o que pode impedir algumas otimizações por parte do motor JS e pode também causar outros problemas.

Como todos os outros objetos, objetos `Function` podem ser criados usando o operador new:

```
new Function (arg1, arg2, ... argN, corpoDaFuncao)
```

`arg1, arg2, ... argN`

Nenhum ou mais nomes para serem usados pela função como nomes formais de argumentos. Cada um deve ser uma string em conformidade com as regras para um identificador JavaScript válido ou uma lista com tais strings separadas por vírgula; por exemplo "x", "oValor", ou "a, b".

corpoDaFuncao

Uma string contento as instruções JavaScript correspondendo a definição da função.

Invocar o construtor Function como uma função (sem usar o operador new) the o mesmo efeito de invocá-lo como um construtor comum.

Parâmetros de Função

> **Nota:** Parâmetros Default e Rest são tecnologias experimentais, parte da proposta do ECMAScript 6, e ainda não são amplamente suportados pelos navegadores.

Parâmetros Default

Parâmetro Default de função permite que parâmetros formais sejam inicializados com valores padrão se nenhum valor ou valor undefined é passado. Para mais detalhes, veja parâmetros default.

Parâmetros Rest

A sintaxe do parâmetro rest permite representar um número indefinido de argumentos em forma de array. Para mais detalhes, veja parâmetros rest.

O objeto <code>arguments</code>

Você pode referenciar aos argumentos da função dentro da função utilizando o objeto arguments. Veja arguments.

- `arguments`: Um objeto array-like contendo os argumentos passados para a função atualmente em execução.
- `arguments.callee` : A função atualmente em execução.
- `arguments.caller`: A função que invocou a função atualmente em execução.
- `arguments.length`: O número de argumentos passados para a função.

Definindo Métodos de Funções

Funções Getter e setter

Você pode definir getters (métodos de acesso) and setters (métodos de alteração) em qualquer objeto interno padrão ou objeto definido pelo usuário que ofereça suporte à adição de novas propriedades. A sintaxe para definir getters e setters usa a sintaxe literal do objeto.

get

Vincula uma propriedade de objeto a uma função que será chamada quando essa propriedade é requisitada.

set

Vincula uma propriedade de objeto a uma função a ser chamada quando há uma tentativa de definir essa propriedade.

Sintaxe dos métodos de funções

> **Nota:** Definições de métodos são tecnologias experimentais, parte da proposta do ECMAScript 6, e ainda não são amplamente suportados pelos navegadores.

Começando com ECMAScript 6, você é capaz de definir métodos próprios em uma sintaxe mais curta, semelhante aos getters e setters. Veja method definitions para mais informações.

```
var obj = {
  foo() {},
  bar() {},
};
```

<code>Function</code> constructor vs. function declaration vs. function expression

Compare o seguinte:

Uma função definida com `Function` *constructor* atribuída à variável `multiply`

```
var multiply = new Function("x", "y", "return x * y");
```

Uma *function expression* de uma função anônima atribuída à variável `multiply`

```
var multiply = function (x, y) {
  return x * y;
};
```

Uma *function expression* de uma função chamada `func_name` atribuída à variável `multiply`

```
var multiply = function func_name(x, y) {
  return x * y;
};
```

Diferenças

Todas fazem aproximadamente a mesma coisa, com algumas diferenças sutis:

Há uma distinção entre o nome da função e a variável à qual a função é atribuída. O nome da função não pode ser alterado, enquanto a variável à qual a função está atribuída pode ser reatribuída. O nome da função pode ser usado somente dentro do corpo da função. A tentativa de usá-lo fora do corpo da função resulta em um *error* (ou *undefined* se o nome da função foi declarado anteriormente por meio de uma declaração de variável). Por exemplo:

```
var y = function x() {};
alert(x); // throws an error
```

O nome da função também aparece quando a função é serializada através do `método toString de Function()`.

Por outro lado, a variável à qual a função é atribuída é limitada somente pelo seu escopo, garantindo-se que ele inclui o escopo onde a função é declarada.

Como o quarto exemplo mostra, o nome da função pode ser diferente da variável à qual a função é atribuída. Eles não têm relação um com o outro. Uma declaração de função também cria uma variável com o mesmo nome da função. Assim, ao contrário daquelas definidas por expressões de função, funções definidas por declarações de função podem ser acessadas pelo seu nome no escopo onde elas foram definidas:

Uma função definida por '`new Function`' não possui um nome. Entretanto, na máquina de JavaScript SpiderMonkey, a forma serializada da função aparece como se ela tivesse o nome "anonymous". Por exemplo, `alert(new Function())` gera como saída:

```
function anonymous() {}
```

Já que a função na realidade não possui um nome, `anonymous` não é uma variável que possa ser acessada dentro da função. Por exemplo, o código seguinte iria resultar em um erro:

```
var foo = new Function("alert(anonymous);");
foo();
```

Ao contrário das funções definidas por expressões de função ou pelo construtor `Function`, uma função definida por uma declaração de função pode ser usada antes da declaração da função em si. Por exemplo:

```
foo(); // alerts FOO!
function foo() {
  alert("FOO!");
}
```

Uma função definida por uma expressão de função herda o escopo atual. Isto é, a função forma um closure. Por outro lado, uma função definida por um construtor `Function` não herda qualquer escopo a não ser o escopo global (que todas as funções herdam).

Funções definidas por expressões de função e declarações de função são analisadas somente uma vez, enquanto aquelas definidas pelo construtor `Function` não são. Isto é, a string que forma o corpo da função, passada para o construtor `Function` precisa ser analisada toda vez que o construtor é chamado. Embora uma expressão de função crie um closure a cada vez, o corpo da função não é reanalisado, assim expressões de função ainda são mais rápidas do que "`new Function(...)`". Assim, o construtor `Function` deve geralmente ser evitado sempre que possível.

Deve ser notado, entretanto, que expressões de função e declarações de função aninhadas dentro de uma função gerada pela análise da string de um construtor `Function` não são analisadas repetidamente. Por exemplo:

```
var foo = new Function(
  "var bar = 'FOO!';\nreturn(function() {\n\talert(bar);\n});",
```

```
)();
foo(); // O segmento "function() {\n\talert(bar);\n}" do corpo da função não é reanalisado.
```

Uma declaração de função é muito facilmente (e às vezes de forma não intencional) transformada em uma expressão de função. Uma declaração de função deixa de ser o que ela é nos casos em que:

- se torna parte de uma expressão; ou
- não é mais um "elemento fonte" de uma função ou do script em si. Um "elemento fonte" é uma instrução não aninhada no script ou no corpo de uma função:

```
var x = 0; // elemento fonte
if (x == 0) {
  // elemento fonte
  x = 10; // não é um elemento fonte
  function boo() {} // não é um elemento fonte
}
function foo() {
  // elemento fonte
  var y = 20; // elemento fonte
  function bar() {} // elemento fonte
  while (y == 10) {
    // elemento fonte
    function blah() {} // não é um elemento fonte
    y++; // não é um elemento fonte
  }
}
```

Examples

```
// declaração de função
function foo() {}

// expressão de função
(function bar() {});

// expressão de função
x = function hello() {};

if (x) {
  // expressão de função
  function world() {}
}

// declaração de função
function a() {
  // declaração de função
  function b() {}
  if (0) {
    // expressão de função
    function c() {}
  }
}
```

Definindo uma função condicionalmente

As funções podem ser definidas condicionalmente usando instruções de função (uma extensão permitida para o padrão ECMA-262 Edição 3) ou o construtor 'Function'. Observe que essas instruções de funções não são mais permitidas no ES5 strinct. Além disso, esse recurso não funciona de forma consistente em navegadores, portanto, você não deve confiar nele.

No script a seguir, a função 'zero' nunca é definida e não pode ser chamada, porque 'if (0)' avalia sua condição como falso:

```
if (0) {
  function zero() {
    document.writeln("Isso é zero.");
  }
}
```

If the script is changed so that the condition becomes '`if (1)`', function `zero` is defined.

Nota: Although this kind of function looks like a function declaration, it is actually an expression (or statement), since it is nested within another statement. See differences between function declarations and function expressions.

Nota: Some JavaScript engines, not including SpiderMonkey, incorrectly treat any function expression with a name as a function definition. This would lead to `zero` being defined, even with the always-false `if` condition. A safer way to define functions conditionally is to define the function anonymously and assign it to a variable:

```
if (0) {
  var zero = function () {
    document.writeln("Aqui tem zero.");
  };
}
```

Exemplo

Exemplo: Retornando um número formatado

A função a seguir retorna uma sequência contendo a representação formatada de um número preenchido com zeros à esquerda.

```
// Esta função retorna uma string preenchida com zeros à esquerda
function padZeros(num, totalLen) {
  var numStr = num.toString(); // Inicializar valor de retorno como string
  var numZeros = totalLen - numStr.length; // Calcular números de zeros
  for (var i = 1; i <= numZeros; i++) {
    // Preencher a quantidade de zeros
    numStr = "0" + numStr;
  }
  return numStr; // Retorna a string já preenchida
}
```

The following statements call the padZeros function.

```
var resultado;
resultado = padZeros(42, 4); // retorno "0042"
resultado = padZeros(42, 2); // retorno "42"
resultado = padZeros(5, 4); // retorno "0005"
```

Example: Determining whether a function exists

You can determine whether a function exists by using the `typeof` operator. In the following example, a test is peformed to determine if the `window` object has a property called `noFunc` that is a function. If so, it is used; otherwise some other action is taken.

```
if ("function" == typeof window.noFunc) {
  // use noFunc()
} else {
  // do something else
}
```

Note that in the `if` test, a reference to `noFunc` is used—there are no brackets "()" after the function name so the actual function is not called.

Veja também

- `Function`
- `function` statement
- `function` operator

Arguments object

O objeto **`arguments`** é como um objeto Array correspondendo aos argumentos passados para uma função.

Sintaxe

```
arguments
```

Descrição

O objeto `arguments` é uma variável local disponível dentro de todas as funções. Você pode referenciar os argumentos de uma função dentro da função usando o objeto `arguments`. Esse objeto contém um registro para cada argumento fornecido para a função, com o índice do primeiro registro começando em 0. Por exemplo, se são passados três argumentos para uma função, você pode referenciá-los como a seguir:

```
arguments[0];
arguments[1];
arguments[2];
```

Os argumentos também podem ser definidos:

```
arguments[1] = "novo valor";
```

O objeto `argumentos` não é um `Array`. É similar a um Array, mas não possui as propriedades de `Array`, exceto `length`. Por exemplo, ele não possui o método `pop`. Entretanto, ele pode ser convertido em um `Array` real:

```
var args = Array.prototype.slice.call(arguments);
var args = [].slice.call(arguments);

// ES2015
var args = Array.from(arguments);
```

> **Aviso:** Usar slice nos argumentos impedem otimizações em alguns motores JavaScript (V8 por exemplo - mais informações). Se você precisar, tente construir um novo array iterando através dos argumentos do objeto. Uma alternativa seria usar o construtor do `Array` como uma função:
>
> ```
> var args =
>   arguments.length === 1 ? [arguments[0]] : Array.apply(null, arguments);
> ```

Você pode usar o objeto `arguments` se você chamar uma função com mais argumentos do que ele é formalmente declarado para aceitar. Esta técnica é útil para funções que podem ser passada em um número de variáveis de argumentos. Você pode usar `arguments.length` para determinar o número de argumentos passado

para a função, e então processar cada argumento usando o objeto `arguments`. Para determinar o número de parâmetros declarados na assinatura da função, use a propriedade `Function.length`.

Usando typeof com Arguments

O comando `typeof arguments` retorna 'object'.

```
console.log(typeof arguments); // 'object'
```

O `typeof` de argumentos individuais pode ser determinado com o uso do índice.

```
console.log(typeof arguments[0]); //this will return the typeof individual arguments.
```

Usando a Sintaxe Spread com Arguments

Você também pode usar o método `Array.from()` ou o operador spread para converter argumentos em um Array real:

```
var args = Array.from(arguments);
var args = [...arguments];
```

Propriedades

`arguments.callee`

> Referência para a função atualmente em execução.

`arguments.caller`

> Referência para a função que invocou a função atualmente em execução.

`arguments.length`

> Referência para o número de argumentos passados para a função.

`arguments[@@iterator]`

> Retorna um novo objeto Array Iterator que contém os valores para cada índice dos argumentos.

Exemplos

Definindo uma função que concatena várias strings

Este exemplo define uma função que concatena várias strings. O único argumento formal para a função é uma string que especifica os caracteres que separam os itens a concatenar. A função é definida como se segue:

```
function myConcat(separator) {
  var args = Array.prototype.slice.call(arguments, 1);
  return args.join(separator);
}
```

Você pode passar qualquer número de argumentos para essa função e ela cria uma lista usando cada argumento como um item na lista.

```
// returns "red, orange, blue"
myConcat(", ", "red", "orange", "blue");

// returns "elephant; giraffe; lion; cheetah"
myConcat("; ", "elephant", "giraffe", "lion", "cheetah");

// returns "sage. basil. oregano. pepper. parsley"
myConcat(". ", "sage", "basil", "oregano", "pepper", "parsley");
```

Definindo uma função que cria listas HTML

Este exemplo define uma função que cria uma string contatenado HTML para uma lista. O único argumento formal para a função é uma string que é "u" se a lista for desordenada (marcadores), ou "o" se a lista for ordenada (numerada). A função é definida como se segue:

```
function list(type) {
  var result = "<" + type + "l><li>";
  var args = Array.prototype.slice.call(arguments, 1);
  result += args.join("</li><li>");
  result += "</li></" + type + "l>"; // end list

  return result;
}
```

Você pode passar qualquer número de argumentos para essa função, e ela adiciona cada argumento como um item para uma lista do tipo indicado. Por exemplo:

```
var listHTML = list("u", "One", "Two", "Three");

/* listHTML is:

"<ul><li>One</li><li>Two</li><li>Three</li></ul>"
```

```
*/
```

Rest, default e parâmetros desestruturados

O objeto `arguments` pode ser usado em conjunto com os parâmetros rest, default e destruturados.

```
function foo(...args) {
  return args;
}
foo(1, 2, 3); // [1,2,3]
```

Enquanto a presença dos parâmetros rest, default, ou desestruturados não alteram o comportamento do objeto `arguments` no código em strict mode, existe uma sutil diferença para o código non-strict.

Quando uma função não-strict **não** contém parâmetros rest, default, ou desestruturados, os valores no objeto `arguments` **modificam** os valores dos argumentos (e vice-versa). Veja o código:

```
function func(a) {
  arguments[0] = 99; // updating arguments[0] also updates a
  console.log(a);
}
func(10); // 99
```

e

```
function func(a) {
  a = 99; // updating a also updates arguments[0]
  console.log(arguments[0]);
}
func(10); // 99
```

Quando uma função não-strict **contém** parâmetros rest, default, ou desestruturados, os valores no objeto `arguments` **não modificam** os valores dos argumentos (e vice-versa). Ao invés disso, eles refletem os argumentos fornecidos no momento da chamada:

```
function func(a = 55) {
  arguments[0] = 99; // updating arguments[0] does not also update a
  console.log(a);
}
func(10); // 10
```

e

```
function func(a = 55) {
  a = 99; // updating a does not also update arguments[0]
  console.log(arguments[0]);
}
func(10); // 10
```

e

```
function func(a = 55) {
  console.log(arguments[0]);
}
func(); // undefined
```

Veja também

- `Function`

Arrow functions

Uma **expressão *arrow function*** possui uma sintaxe mais curta quando comparada a uma expressão de função (*function expression*) e não tem seu próprio *this*, *arguments*, *super* ou *new.target*. Estas expressões de funções são melhor aplicadas para funções que não sejam métodos, e elas não podem ser usadas como construtores (*constructors*).

Experimente

```
const materials = ["Hydrogen", "Helium", "Lithium", "Beryllium"];

console.log(materials.map((material) => material.length));
// Expected output: Array [8, 6, 7, 9]
```

Sintaxe

Sintaxe básica

```
(param1, param2, …, paramN) => { statements }
(param1, param2, …, paramN) => expression
// equivalente a: => { return expression; }
```

```
// Parênteses são opcionais quando só há um nome de parâmetro:
(singleParam) => { statements }
singleParam => { statements }

// A lista de parâmetros para uma função sem parâmetros deve ser escrita com um par de parênteses.
() => { statements }
```

Sintaxe avançada

```
// Envolva o corpo da função em parênteses para retornar uma expressão literal de objeto:
params => ({foo: bar})

// Parâmetros rest (rest parameters) e parâmetros padrões (default parameters) são suportados
(param1, param2, ...rest) => { statements }
(param1 = defaultValue1, param2, …, paramN = defaultValueN) => { statements }

// Desestruturação (destructuring) dentro da lista de parâmetros também é suportado
var f = ([a, b] = [1, 2], {x: c} = {x: a + b}) => a + b + c;
f(); // 6
```

Descrição

Veja também "ES6 In Depth: Arrow functions" on hacks.mozilla.org.

Dois fatores influenciaram a introdução das *arrow functions*: funções mais curtas e a inexistência da palavra chave `this`.

Funções mais curtas

```
var elements = [
  'Hydrogen',
  'Helium',
  'Lithium',
  'Beryllium'
];

elements.map(function(element) {
  return element.length;
}); // esta sentença retorna o array: [8, 6, 7, 9]

// A função regular acima pode ser escrita como a arrow function abaixo
elements.map((element) => {
  return element.length;
}); // [8, 6, 7, 9]

// Quando só existe um parâmetro, podemos remover os parênteses envolvendo os parâmetros:
elements.map(element => {
  return element.length;
}); // [8, 6, 7, 9]

// Quando a única sentença em uma arrow function é `return`, podemos remover `return` e remover
// as chaves envolvendo a sentença
elements.map(element => element.length); // [8, 6, 7, 9]

// Neste caso, porque só precisamos da propriedade length, podemos usar o parâmetro de destruição (destructing parameter):
// Note que a string `"length"` corresponde a propriedade que queremos obter enquanto que a
// obviamente propriedade não especial `lengthFooBArX` é só o nome de uma variável que pode ser mudado
// para qualquer nome válido de variável que você quiser
elements.map(({ "length": lengthFooBArX }) => lengthFooBArX); // [8, 6, 7, 9]

// Esta atribuição de parâmetro de destruição (destructing parameter) pode ser escrita como visto abaixo. Entretanto, note que
// não há um específico `"length"` para selecionar qual propriedade nós queremos obter. Ao invés disso, o nome literal
// da própria variável `length` é usado como a propriedade que queremos recuperar do objeto.
elements.map(({ length }) => length); // [8, 6, 7, 9]
```

Sem <code>this</code> separado

Antes das *arrow functions*, toda nova função definia seu próprio valor de this (baseado em como a função era chamada, um novo objeto no caso de um construtor, *undefined* em chamadas de funções com modo estrito (*strict mode*), o objeto base se a função é chamada como um "método de objeto", etc.). Este comportamento é importuno com um estilo de programação orientado a objeto.

```
function Person() {
  // O contrutor Person() define `this` como uma instância dele mesmo.
  this.age = 0;

  setInterval(function growUp() {
    // Em modo não estrito, a função growUp() define `this`
    // como o objeto global (porque é onde growUp() é executado.),
    // que é diferente ao `this`
    // definido pelo construtor Person().
    this.age++;
  }, 1000);
}

var p = new Person();
```

No ECMAScript 3/5, este comportamento era corrigido definindo o valor em `this` à uma variável que pudesse ser encapsulada.

```
function Person() {
  var that = this;
  that.age = 0;

  setInterval(function growUp() {
    // A chamada a função refere à variáevel `that` da qual
    // o valor é o objeto esperado.
    that.age++;
  }, 1000);
}
```

Alternativamente, uma função de ligação (bound function) pode ser criada para que o valor pré-atribuido à `this` seja passado para a função alvo de ligação (a função `growUp()` no exemplo acima.

Uma *arrow function* não tem seu próprio `this;` o valor `this` do contexto léxico encapsulado é usado. Ex: *Arrow functions* seguem as regras normais de pesquisa de variáveis. Então, ao procurar por `this`, que não está no escopo atual elas acabam encontrando `this` no escopo encapsulado. Assim, no código a seguir, o `this` dentro da função que é passado para `setInterval` tem o mesmo valor do `this` na função lexicamente encapsulada:

```
function Person() {
  this.age = 0;

  setInterval(() => {
    this.age++; // |this| corretamente se refere ao objeto Person
  }, 1000);
}

var p = new Person();
```

Relação com *strict mode*

Considerando que `this` vem do contexto léxico envolvente, as regras do modo estrito (*strict mode*) em relação ao `this` são ignoradas.

```
var f = () => {
  "use strict";
  return this;
};
f() === window; // ou o objeto global
```

O restante das regras do modo estrito (*strict mode)* são aplicadas normalmente.

Invocação por call ou apply

Já que as *arrow functions* não têm o próprio `this`, os métodos `call()` ou `apply()` só podem passar parâmetros. `thisArg` é ignorado.

```
var adder = {
  base: 1,

  add: function(a) {
    var f = v => v + this.base;
    return f(a);
  },

  addThruCall: function(a) {
    var f = v => v + this.base;
    var b = {
      base: 2
    };

    return f.call(b, a);
  }
};

console.log(adder.add(1));         // Deve logar 2
console.log(adder.addThruCall(1)); // Deve logar 2 ainda
```

Sem ligação (binding) de argumentos (<code>arguments</code>)

Arrow functions não tem o próprio objeto argumentos (arguments object). Assim, neste exemplo, `arguments` é simplesmente uma referência aos argumentos do escopo encapsulado:

```
var arguments = [1, 2, 3];
var arr = () => arguments[0];

arr(); // 1

function foo(n) {
  var f = () => arguments[0] + n; // ligação implícita dos argumentos de foo. arguments[0] é n
  return f();
}

foo(3); // 6
```

Na maioria dos casos, usar parâmetros rest (*rest parameters*) é uma boa alternativa a usar um objeto `arguments`.

```
function foo(n) {
  var f = (...args) => args[0] + n;
  return f(10);
}

foo(1); // 11
```

Arrow functions usadas como métodos

Como afirmado anteriormente, expressões *arrow function* são melhores para funções que não sejam métods. Vamos ver o que acontece quando tentamos usá-las como métodos.

```
'use strict';

var obj = {
  i: 10,
  b: () => console.log(this.i, this),
  c: function() {
    console.log(this.i, this);
  }
}

obj.b(); // imprime undefined, Window {...} (ou o objeto global)
obj.c(); // imprime 10, Object {...}
```

Arrow functions não tem o próprio `this`. Outro exemplo envolvendo `Object.defineProperty()`:

```
'use strict';

var obj = {
  a: 10
};

Object.defineProperty(obj, 'b', {
  get: () => {
    console.log(this.a, typeof this.a, this); // undefined 'undefined' Window {...} (ou o objeto global)
    return this.a + 10; // representa o objeto global 'Window', portanto 'this.a' retorna 'undefined'
  }
});
```

Uso do operador <code>new</code>

Arrow functions não podem ser usadas como construtores e lançarão um erro quando usadas com o `new`.

Uso da propriedade <code>prototype</code>

Arrow functions não têm a propriedade `prototype`.

```
var Foo = () => {};
console.log(Foo.prototype); // undefined
```

Uso da palavra chave <code>yield</code>

A palavra chave `yield` não pode ser usada no corpo de uma *arrow function* (exceto quando permitido dentro de funções aninhadas dentro delas). como consequência, *arrow functions* não podem ser usadas como geradoras (*generators*).

Corpo de função

Arrow functions podem ter um corpo conciso ("*concise body*") ou o usual corpo em bloco ("*block body*").

Em um *concise body*, apenas uma expressão é especificada, a qual se torna o valor de retorno implícito. Em um *block body*, você precisa explicitamente usar a declaração de retorno, ou seja, o `return`.

```
var func = x => x * x;
// sintaxe de concise body. O "return" é implícito

var func = (x, y) => { return x + y; };
// Em um função com block body, é necessário um "return" explícito
```

Retornando objetos literais

Tenha em mente que retornar objetos literais usando a sintaxe de corpo conciso (*concise body*) `params => {object:literal}` não funcionará como esperado.

```
var func = () => { foo: 1 };
// Chamar func() retornará undefined!

var func = () => { foo: function() {} };
// SyntaxError (Erro de sintaxe): a declaração da função requer um nome
```

Isto acontece porque o código dentro das chaves ({}) é convertido como uma sequência de sentenças (ex: `foo` é tratado como um título, não como uma chave num objeto literal).

Se lembre de envolver o objeto literal em parênteses.

```
var func = () => ({foo: 1});
```

Quebras de linha

Uma *arrow function* não pode conter uma quebra de linha entre seus parâmetros e sua flecha.

```
var func = (a, b, c)
           => 1;
// SyntaxError (Erro de sintaxe): esperada expressão, mas obteve '=>'
```

Entretanto, isto pode ser corrigido ao usar parênteses ou colocar a quebra de linha dentro dos argumentos como visto abaixo para garantir que o código permaneça bonito e leve.

```
var func = (a, b, c) => 1;
// SyntaxError (erro de sintaxe) não é lançado
```

Ordem de análise

Apesar de a flecha numa *arrow function* não ser um operador, *arrow functions* possuem regras especiais de análise que interagem diferentemente com precedência de operador (operator precedence) comparadas à funções comuns.

```
let callback;

callback = callback || function() {}; // ok

callback = callback || () => {};
// SyntaxError (Erro de sintaxe): argumentos inválidos de arrow-function

callback = callback || (() => {});    // ok
```

Mais exemplos

```
// Uma arrow function vazia retorna undefined
let empty = () => {};

(() => 'foobar')();
// Retorna "foobar"
// (esta é uma Expressão de Função Invocada Imediatamente (Immediately Invoked Function Expression)
// veja 'IIFE' no glossário)

var simple = a => a > 15 ? 15 : a;
simple(16); // 15
simple(10); // 10

let max = (a, b) => a > b ? a : b;

// Mapeamento, filtragem, ... simples de array

var arr = [5, 6, 13, 0, 1, 18, 23];

var sum = arr.reduce((a, b) => a + b);
// 66

var even = arr.filter(v => v % 2 == 0);
// [6, 0, 18]

var double = arr.map(v => v * 2);
// [10, 12, 26, 0, 2, 36, 46]

// Cadeias de promessa (promisse chains) mais concisas
promise.then(a => {
  // ...
}).then(b => {
  // ...
});

// Arrow functions sem parâmetros que são visualmente mais fáceis de analisar
setTimeout( () => {
  console.log('E aconteço antes');
  setTimeout( () => {
    // deeper code
    console.log('Eu aconteço depois');
  }, 1);
}, 1);
```

Parâmetros Predefinidos

Os parâmetros predefinidos de uma função permitem que parâmetros regulares sejam inicializados com com valores iniciais caso `undefined` ou nenhum valor seja passado.

Sintaxe

```
function [nome]([param1[ = valorPredefinido1 ][, ..., paramN[ = valorPredefinidoN ]]]) {
   instruções
}
```

Descrição

Em JavaScript, os parâmetros de funções tem `undefined` como valor predefinido. Contudo, em alguns casos pode ser útil utilizar algum outro valor. É nesta situação em que os parâmetros predefinidos podem ser úteis.

No passado, a estratégia de definir valores padrão para parâmetros era testar os valores do parâmetros no corpo da função e atribuir um valor se este for **`undefined`.** No exemplo a seguir, se nenhum valor for fornecido para **b** na chamada, este valor será **`undefined`**, quando `a*b` for calculado resultaria em `NaN`. No entanto, isto é capturado na segunda linha definindo um valor padrão para b:

```
function multiply(a, b) {
  b = typeof b !== "undefined" ? b : 1;

  return a * b;
}

multiply(5, 2); // 10
multiply(5, 1); // 5
multiply(5); // 5
```

Com o parâmetros predefinidos a checagem no corpo da função nao é mais necessária. Agora você pode simplesmente colocar 1 como valor padrão para `b` na declaração da função:

```
function multiply(a, b = 1) {
  return a * b;
}

multiply(5, 2); // 10
multiply(5, 1); // 5
multiply(5); // 5
```

Exemplos

Passando <code>undefined</code> vs. outros valores "falsy"

Na segunda chamada aqui, mesmo se o segundo argumento é definido explicitamente como `undefined` (com exceção de `null`) quando chamado, o valor para o argumento `num` será o padrão.

```
function test(num = 1) {
  console.log(typeof num);
}

test();          // 'number' (num é definido para 1)
test(undefined); // 'number' (num é definido para 1 também)

// teste com outros values "falsy":
test('');        // 'string' (num é definido para '')
test(null);      // 'object' (num é definido para null)
```

Avaliado em tempo de chamada

Os parâmetros predefinidos são avaliados no momento da chamada da função, então diferente de ex.: Python, um novo objeto é criado cada vez que a funçao é chamada.

```
function append(value, array = []) {
  array.push(value);
  return array;
}

append(1); //[1]
append(2); //[2], not [1, 2]
```

Este mesmo comportamento é aplicado para funções e variáveis:

```
function callSomething(thing = something()) {
  return thing;
}

function something() {
  return "sth";
}

callSomething(); //sth
```

Parâmetros predefinidos estão disponíveis para os parâmetros seguintes à sua definição

Parâmetros que já foram avaliados ficam disponíveis para uso para os parâmetros seguintes:

```
function singularAutoPlural(singular, plural = singular+"s",
                            rallyingCry = plural + " ATTACK!!!") {
  return [singular, plural, rallyingCry ];
}

//["Gecko","Geckos", "Geckos ATTACK!!!"]
singularAutoPlural("Gecko");

//["Fox","Foxes", "Foxes ATTACK!!!"]
singularAutoPlural("Fox","Foxes");

//["Deer", "Deer", "Deer ... change."]
singularAutoPlural("Deer", "Deer", "Deer peaceably and respectfully
   petition the government for positive change.")
```

Esta funcionalidade torna-se uma maneira direta e demonstra quantos casos extremos são manipulados.

```
function go() {
  return ":P";
}

function withDefaults(
  a,
  b = 5,
  c = b,
  d = go(),
  e = this,
  f = arguments,
  g = this.value,
) {
  return [a, b, c, d, e, f, g];
}
function withoutDefaults(a, b, c, d, e, f, g) {
  switch (arguments.length) {
    case 0:
      a;
    case 1:
      b = 5;
    case 2:
      c = b;
    case 3:
      d = go();
    case 4:
      e = this;
    case 5:
      f = arguments;
    case 6:
      g = this.value;
    default:
  }
  return [a, b, c, d, e, f, g];
}

withDefaults.call({ value: "=^_^=" });
// [undefined, 5, 5, ":P", window, arguments, "=^_^="]

withoutDefaults.call({ value: "=^_^=" });
// [undefined, 5, 5, ":P", window, arguments, "=^_^="]
```

Funções definidadas dentro do corpo da função

Introduzido no Gecko 33. Funções declaradas no corpo da função não podem ser referenciada dentro de parâmetos padrão e lançará um `ReferenceError` (atualmente um `TypeError` no SpiderMonkey, veja Erro do Firefox 1022967). Parâmetros padrão são sempre executados primeiro, declarações de funções dentro do corpo de outra função são avaliadas depois.

```
// Não funciona! Throws ReferenceError.
function f(a = go()) {
  function go() {
    return ":P";
  }
}
```

Parâmetros sem valor padrão depois de parâmetros com valores padrão

Antes do Gecko 26, o seguinte código resultaria em um `SyntaxError`. Isto foi corrigido no Erro do Firefox 777060 e funciona como esperado em versões posteriores:

```
function f(x = 1, y) {
  return [x, y];
}
```

```
f(); // [1, undefined]
```

Parâmetro desestruturado com valores padrões

É possível definir valores padrões com a notação destructuring assignment:

```
function f([x, y] = [1, 2], { z: z } = { z: 3 }) {
  return x + y + z;
}

f(); // 6
```

Veja também

- Original proposal at ecmascript.org

getter

A sintaxe de **get** associa uma propriedade de um objeto a uma função que será chamada quando tal propriedade é acessada.

Sintaxe

```
{get prop() { ... } }
{get [expression]() { ... } }
```

Parâmetros

`prop`

O nome da propriedade à qual a função dada será associada.

expression

A partir do ECMAScript 6, você pode também utilizar expressões para um nome computado de uma propriedade para associar à função dada.

Descrição

Às vezes é desejável que se permita acesso a uma propriedade que retorna um valor computado dinamicamente, ou você pode querer refletir o status de uma variável interna sem requerer o uso de chamadas de método explícitas. Em Javascript, isso pode ser feito com o uso de um *getter*. Não é possível simultaneamente ter um getter associado a uma propriedade e a mesma possuir um valor, embora seja possível usar um getter e um setter em conjunto para criar algo como uma pseudo-propriedade.

Note o seguinte quando for trabalhar com a sintaxe *get*:

- Pode haver um identificador que seja um número ou uma string;
- Deve haver exatamente zero parâmetros (veja Incompatible ES5 change: literal getter and setter functions must now have exactly zero or one arguments para mais informações)
- Não deve haver mais de um getter para a mesma propriedade nem uma propriedade comum como o mesmo nome do getter (`{ get x() { }, get x() { } }` and `{ x: ..., get x() { } }` são proibidos).

Um getter pode ser removido usando o operador `delete`.

Exemplos

Definindo um getter em novos objetos em inicializadores de objetos

O exemplo abaixo irá criar a pseudo-propriedade `latest` para o objeto `obj`, que irá retornar o último item do array em `log`.

```
var log = ["test"];
var obj = {
  get latest() {
    if (log.length == 0) return undefined;
    return log[log.length - 1];
  },
};
console.log(obj.latest); // Retornará "test".
```

Note que a tentativa de atribuir um valor a `latest não irá alterá-la`.

Removendo um getter usando o operador <code>delete</code>

Se você quer remover um getter, você pode apenas utilizar `delete`.

```
delete obj.latest;
```

Definindo um getter em objetos existentes usando <code>defineProperty</code>

Para adicionar um getter a um objeto existente a qualquer momento, use `Object.defineProperty()`.

```
var o = { a: 0 };
```

```
Object.defineProperty(o, "b", {
  get: function () {
    return this.a + 1;
  },
});

console.log(o.b); // Executa o getter, que retornará a + 1 (que é 1)
```

Usando uma propriedade com nome computado

> **Nota:** Propriedades com nome computado são uma tecnologia experimenta, parte da proposta do ECMAScript 6 e não é amplamente suportada pelos navegadores ainda. O exemplo abaixo irá disparar um `SyntaxError em ambientes sem suporte.`

```
var expr = "foo";

var obj = {
  get [expr]() {
    return "bar";
  },
};

console.log(obj.foo); // "bar"
```

Smart / self-overwriting / lazy getters

Getters lhe dão uma maneira de definir uma propriedade de um objeto, mas eles não calculam o valor da propriedade até que sejam acessados. Um getter adia o custo de cálculo do valor até que o valor seja necessário e, se nunca o for, você não precisa pagar esse custo.

Uma técnica de otimização adicional para atrasar o cálculo do valor de uma propriedade e cacheá-lo para acesso futuro são os **smart ou memoized getters**. O valor é calculado na primeira vez que o getter é invocado, sendo então cacheado para que acessos subsequentes retornem o valor em cache sem recalculá-lo. Isso é util nas seguintes situações:

- Se o cálculo da propriedade é caro (usa muita RAM ou CPU, cria um worker, retorna um arquivo remoto, etc).
- Se o valor não é necessário agora. Ele será utilizado depois ou, em alguns casos, nem mesmo o será.
- Se for utilizado, ele será acessado diversas vezes e não há necessidade de recalcular um valor que nunca será alterado, ou não deveria ser recalculado.

Isso significa que você não deveria utilizar um lazy getter para uma propriedade cujo valor você espera mudar, porque o getter nunca irá recalcular o valor.

No exemplo a seguir, o objeto tem um getter como uma propriedade própria (own property). Ao tentar obter essa propriedade que foi removida, ela será readicionada, mas implicitamente como uma propriedade comum desta vez. Finalmente, o valor será retornado.

```
get notifier() {
  delete this.notifier;
  return this.notifier = document.getElementById("bookmarked-notification-anchor");
},
```

Para o Firefox, veja também o código do módulo XPCOMUtils.jsm, que define a função defineLazyGetter().

Veja também

- setter
- `delete`
- `Object.defineProperty()`
- `Object.prototype.__defineGetter__()`
- `Object.prototype.__defineSetter__()`
- Defining Getters and Setters in JavaScript Guide

Definições de Método

No ECMAScript 2015 foi introduzida uma sintaxe reduzida para definição de métodos em inicializadores de objetos. É uma abreviação para uma função atribuída ao nome do método.

Experimente

```
const obj = {
  foo() {
    return "bar";
  },
};

console.log(obj.foo());
// Expected output: "bar"
```

Sintaxe

```
var obj = {
  propriedade( parametros… ) {},
  *generator( parametros… ) {},
// também com chaves computadas:
  [propriedade]( parameters… ) {},
```

```
  *[generator]( parametros… ) {},
// compare ES5 sintaxe para getter/setter:
  get propriedade() {},
  set propriedade(valor) {}
};
```

Descrição

A sintaxe reduzida é similar à da getter e setter introduzida no ECMAScript 5.

Dado o seguinte código:

```
var obj = {
  foo: function () {},
  bar: function () {},
};
```

Agora você pode reduzi-lo para isto:

```
var obj = {
  foo() {},
  bar() {},
};
```

Generator methods

Os generator methods também podem ser definidos utilizando a sintaxe reduzida.

- Observe que o asterisco (*) na sintaxe reduzida deve estar antes do nome da propriedade generator. Assim, `* g(){}` funcionará, porém `g *(){}` não.
- Se o método não for generator, sua definição não pode conter a palavra-chave `yield`. Dessa forma, generator functions legadas também não funcionarão, lançando um `SyntaxError`. Sempre utilize `yield` em conjunto com o asterisco (*)

```
// Utilizando a propriedade com nome (pre-ES6)
var obj2 = {
  g: function* () {
    var indice = 0;
    while (true) yield indice++;
  },
};

// O mesmo objeto utilizando a sintaxe reduzida
var obj2 = {
  *g() {
    var indice = 0;
    while (true) yield indice++;
  },
};

var coisa = obj2.g();
console.log(coisa.next().value); // 0
console.log(coisa.next().value); // 1
```

Métodos assíncronos

Funções assíncronas também podem ser definidas usando a sintaxe reduzida.

```
// Utilizando a propriedade com nome (pre-ES6)
var obj3 = {
  f: async function () {
    await alguma_promise;
  },
};

// O mesmo objeto com a sintaxe reduzida
var obj3 = {
  async f() {
    await alguma_promise;
  },
};
```

Generator methods assíncronos

Os generator methods também podem ser assíncronos

```
var obj4 = {
  f: async function* () {
    yield 1;
    yield 2;
    yield 3;
  },
};

// O mesmo objeto com a sintaxe reduzida
```

```
var obj4 = {
  async *f() {
    yield 1;
    yield 2;
    yield 3;
  },
};
```

Métodos reduzidos não são construíveis

Métodos assim definidos não são construtores e lançarão um `TypeError` se você tentar instanciá-los.

```
var obj = {
  metodo() {},
};
new obj.metodo(); // TypeError: obj.method is not a constructor

var obj = {
  *g() {},
};
new obj.g(); // TypeError: obj.g is not a constructor (modificado no ES2016)
```

Exemplos

Caso de teste simples

```
var obj = {
  a: "foo",
  b() {
    return this.a;
  },
};
console.log(obj.b()); // "foo"
```

Nome de propriedades computados

A sintaxe reduzida também suporta nome de propriedades computados.

```
var bar = {
  foo0: function () {
    return 0;
  },
  foo1() {
    return 1;
  },
  ["foo" + 2]() {
    return 2;
  },
};

console.log(bar.foo0()); // 0
console.log(bar.foo1()); // 1
console.log(bar.foo2()); // 2
```

Veja também

- `get`
- `set`
- Gramática léxica

Parâmetros Rest

A sintaxe de **rest parameter (parâmetros rest)** nos permite representar um número indefinido de argumentos como um array.

Sintaxe

```
function(a, b, ...theArgs) {
  // ...
}
```

Descrição

Se o último argumento nomeado de uma função tiver prefixo com `...`, ele irá se tornar um array em que os elemento de 0 (inclusive) até theArgs.length (exclusivo) são disponibilizados pelos argumentos atuais passados à função.

No exemplo acima, `theArgs` irá coletar o terceiro argumento da função (porquê o primeiro é mapeado para `a`, e o segundo para `b`) e assim por diante em todos os argumentos consecutivos.

Diferença entre rest parameters e <code>arguments</code> object

Há três diferenças principais entre *rest parameters* e os `arguments` objects:

- *rest parameters* são os únicos que não foram atribuidos a um nome separado, enquanto os `arguments` object contêm todos os argumentos passados para a função;
- o objeto `arguments` não é um array, enquanto rest parameters são instâncias `Array`, isso significa que métodos como `sort`, `map`, `forEach` ou `pop` podem ser aplicados diretamente;
- o objeto `arguments` possui a funcionalidade adicional de especificar ele mesmo (como a propriedade `callee`).

De arguments para array

Rest parameters foram criados para reduzir o código padrão que foi induzida pelos argumentos

```
// Antes rest parameters, o seguinte codigo pode ser encontrado
function f(a, b){
  var args = Array.prototype.slice.call(arguments, f.length);

  // ...
}

// esse é o equivalente

function(a, b, ...args) {

}
```

Exemplos

Como `theArgs` é um array, você pode pegar número de elementos usando a propriedade `length`:

```
function fun1(...theArgs) {
  console.log(theArgs.length);
}

fun1(); // 0
fun1(5); // 1
fun1(5, 6, 7); // 3
```

No próximo exemplo, nós usamos o rest parâmetro para buscar argumentos do segundo parâmetro para o fim. Nós multiplicamos eles pelo primeiro parâmetro:

```
function multiply(multiplier, ...theArgs) {
  return theArgs.map(function (element) {
    return multiplier * element;
  });
}

var arr = multiply(2, 1, 2, 3);
console.log(arr); // [2, 4, 6]
```

O próximo exemplo mostra como você pode usar metodos do Array em rest params, mas não no objeto `arguments`:

```
function sortRestArgs(...theArgs) {
  var sortedArgs = theArgs.sort();
  return sortedArgs;
}

console.log(sortRestArgs(5, 3, 7, 1)); // Exibe 1,3,5,7

function sortArguments() {
  var sortedArgs = arguments.sort();
  return sortedArgs; // isso nunca irá ocorrer
}

// throws a TypeError: arguments.sort is not a function
console.log(sortArguments(5, 3, 7, 1));
```

a fim de usar o objeto `arguments`, você precisará converte-lo para um array antes.

Veja também

- Arguments object
- Array
- Functions
- Spread Operator
- Original proposal at ecmascript.org
- JavaScript arguments object and beyond

setter

A sintaxe **`set`** liga a propriedade da função para ser chamada quando existe uma tentativa de definir valor para essa propriedade.

Sintaxe

```
{set prop(val) { . . . }}
{set [expression](val) { . . . }}
```

Parâmetros

`prop`

Nome da propriedade ligada a função.

`val`

`Um apelido para variável que será atribuidada a prop.`

expression

Iniciando com ECMAScript 6, você pode usar também expressões para computar o nome da propriedade a ser definida na função.

Descrição

Em JavaScript, um *setter* pode ser usado para executar uma função sempre que se tenta mudar uma propriedade específica. *Setters* são geralmente usados em conjunto com *getters*, para criar um tipo de pseudo-propriedade. No entanto é impossível ter-se um setter para uma propriedade que contenha um valor real.

Observe os seguintes itens ao trabalhar com a sintaxe *set*:

- Ele pode ter um identificador que é um número ou uma string;
- Deve ter exatamente um parâmetro (para mais informações, acesse: Mudança do ES5 incompatível: funções getter e setter literais tem exatamemente zero ou um parâmetros);
- Não deve ter a mesma nomenclatura seja ela para propriedades ou função. (`{ set x(v) { }, set x(v) { } }` e `{ x: ..., set x(v) { } }` é proibido.)

Um *setter* pode ser deletado usando o operator `delete`.

Exemplos

Definindo um setter em new Objects no inicializador

Vai definir um pseudo-property *current* para este objeto, quando atribuido um valor, vai mudar `log` com o valor passado:

```
var o = {
  set current(str) {
    this.log[this.log.length] = str;
  },
  log: [],
};
```

Observe que *current* não está definido e qualquer tentativa de acesso irá resultar em *undefined*.

Removendo um setter com o operador <code>delete</code>

Se você deseja remover o *setter*, poderá usar somente o `delete`:

```
delete o.current;
```

Definindo um setter para um Object existente usando o <code>defineProperty</code>

Para adicionar um *setter* para um Object existentem mais tarde, use `Object.defineProperty()`.

```
var o = { a: 0 };

Object.defineProperty(o, "b", {
  set: function (x) {
    this.a = x / 2;
  },
});

o.b = 10; // Runs the setter, which assigns 10 / 2 (5) to the 'a' property
console.log(o.a); // 5
```

Usando um nome de propriedade computada

> **Nota: Atenção:** Propriedades computadas são uma tecnologia experimental, parte da proposta ECMAScript 6, e não é amplamente suportada por todos os browsers ainda. Ele vai retornar um syntax error en ambientes que não tem suporte.

```
var expr = "foo";

var obj = {
  baz: "bar",
  set [expr](v) {
    this.baz = v;
  },
};

console.log(obj.baz); // "bar"
obj.foo = "baz"; // run the setter
console.log(obj.baz); // "baz"
```

SpiderMonkey-specific notes

- Starting with JavaScript 1.8.1, setters are no longer called when setting properties in object and array initializers.
- From SpiderMonkey 38 on, a setter with a rest parameter is a `SyntaxError` as per the ES6 specification.

See also

- getter
- `delete`
- `Object.defineProperty()`
- `Object.prototype.__defineGetter__()`
- `Object.prototype.__defineSetter__()`
- Defining Getters and Setters in JavaScript Guide

Objetos Globais

Resumo

Este capítulo documenta todos os objetos nativos do JavaScript padrão, assim como seus métodos e propriedades.

O termo "objetos globais" (ou objetos nativos por padrão) aqui não deve ser confundido com o de **objeto global**. Aqui, objetos globais se referem aos **objetos no escopo global** (somente se o modo estrito/*strict mode* do ECMAScript 5 não for usado; Nesse caso retorna `undefined`). O **objeto global** pode ser acessado usando o operador `this` no escopo global. De fato, o escopo global **consiste em** propriedades do objeto global, incluindo propriedades herdadas, se houver.

Outros objetos no escopo global também são criados pelo desenvolvedor ou fornecido pela aplicação *host*. Os objetos disponíveis no *host* no contexto do browser são documentados na API reference. Para maiores informações sobre as distinções entre DOM e core JavaScript, veja visão geral das tecnologias JavaScript.

Objetos padrão (por categoria)

Propriedades de valor

Propriedades globais retornam um valor simples; eles não tem propriedades ou métodos.

- `Infinity`
- `NaN`
- `undefined`
- `null` literal

Propriedades de função

Estas funções globais —funções que são chamadas globalmente ao invés de em um objeto—retornam diretamente seus resultados a quem chama.

- `eval()`
- `uneval()`
- `isFinite()`
- `isNaN()`
- `parseFloat()`
- `parseInt()`
- `decodeURI()`
- `decodeURIComponent()`
- `encodeURI()`
- `encodeURIComponent()`
- `escape()`
- `unescape()`

Objetos fundamentais

Estes são objetos básicos e fundamentais nos quais todos os outros objetos são baseados. Isso inclui objetos que representam objetos genéricos, funções e erros.

- `Object`
- `Function`
- `Boolean`
- `Symbol`
- `Error`
- `EvalError`
- `InternalError`
- `RangeError`
- `ReferenceError`
- `StopIteration`
- `SyntaxError`
- `TypeError`
- `URIError`

Números e datas

Estes são objetos base para a representação de números, datas e cálculos matemáticos.

- `Number`
- `Math`

- `Date`

Processamento de texto

Estes objetos representam strings e manipulam as mesmas.

- `String`
- `RegExp`

Coleções indexadas

Estes objetos representam coleções de dados que são ordenados pelo valor de um índice. Isso inclui arrays (tipados) e arrays baseados em outros construtores, como `[]`.

- `Array`
- `Float32Array`
- `Float64Array`
- `Int16Array`
- `Int32Array`
- `Int8Array`
- `Uint16Array`
- `Uint32Array`
- `Uint8Array`
- `Uint8ClampedArray`
- `ParallelArray`

Coleções chaveadas

Estes objetos representam coleções que usam chaves; estas contém elementos que são iteráveis na ordem de inserção.

- `Map`
- `Set`
- `WeakMap`
- `WeakSet`

Dados estruturados

Estes objetos representam e interagem com buffers de dados estruturados e dados codificados usando JavaScript Object Notation (JSON).

- `ArrayBuffer`
- `DataView`
- `JSON`

Controle de abstrações de objetos

- `Promise`
- `Generator`
- `GeneratorFunction`

Reflexão (reflection)

- `Reflect`
- `Proxy`

Internacionalização

Adições ao core do ECMAScript para funcionalidades sensíveis à linguagem.

- `Intl`
- `Intl.Collator`
- `Intl.DateTimeFormat`
- `Intl.NumberFormat`

Objetos não-padrão

- `Iterator`
- `ParallelArray`
- `StopIteration`

Outros

- `arguments`

Array

Sumário

O objeto `Array` do JavaScript é um objeto global usado na construção de 'arrays': objetos de alto nível semelhantes a listas.

Sintaxe

```
[element0, element1, ..., elementN]
new Array(element0, element1, ..., elementN)
new Array(arrayLength)
```

`element0, element1, ..., elementN`

> Um array JavaScript é inicializado com os elementos contém, exceto no caso onde um único argumento é passado para o construtor do `Array` e esse argumento é um número (veja o parâmetro arrayLength abaixo). Esse caso especial só se aplica para os arrays JavaScript criados com o construtor `Array` , e não para literais de array criados com a sintaxe de colchetes [].

`arrayLength`

> Se o único argumento passado para o construtor do `Array` for um número inteiro entre 0 e 232-1 (inclusive), um novo array com o tamanho desse número é retornado. Se o argumento for qualquer outro número, uma exceção `RangeError` é lançada.

Descrição

Arrays são objetos semelhantes a listas que vêm com uma série de métodos embutidos para realizar operações de travessia e mutação. Nem o tamanho de um array JavaScript nem os tipos de elementos são fixos. Já que o tamanho de um array pode ser alterado a qualquer momento e os dados podem ser armazenados em posições não contíguas, arrays JavaScript não tem a garantia de serem densos; isso depende de como o programador escolhe usá-los. De uma maneira geral, essas são características convenientes, mas, se esses recursos não são desejáveis para o seu caso em particular, você pode considerar usar arrays tipados.

Arrays não podem usar strings como índices (como em um array associativo), devem ser usados números inteiros. Definir ou acessar não-inteiros usando notação de colchetes (ou notação de ponto) não vai definir ou recuperar um elemento do array em si, mas sim definir ou acessar uma variável associada com a coleção de propriedades de objeto daquele array. As propriedades de objeto do array e a lista de elementos do array são separados, e as operações de travessia e mutação não podem ser aplicadas a essas propriedades nomeadas.

Accessando elementos de um array

Arrays JavaScript começam com índice zero: o primeiro elemento de um array está na posição `0` e o último elemento está na posição equivalente ao valor da propriedade `length` (tamanho) menos 1.

```
var arr = ["este é o primeiro elemento", "este é o segundo elemento"];
console.log(arr[0]); // exibe 'este é o primeiro elemento'
console.log(arr[1]); // exibe 'este é o segundo elemento'
console.log(arr[arr.length - 1]); // exibe 'este é o segundo elemento'
```

Elementos de um array são somente propriedades de objetos, da forma que `toString` é uma propriedade. Contudo, note que tentando acessar o primeiro elemento de um array da seguinte forma causará um erro de sintaxe, pois o nome da propriedade é inválido:

```
console.log(arr.0); // um erro de sintaxe
```

Não há nada de especial a respeito de arrays JavaScript e suas propriedades que causam isso. As propriedades JavaScript que começam com um dígito não podem ser referenciadas com notação de ponto. Elas necesitam usar notação de colchetes para poderem ser acessadas. Por exemplo, se você tivesse um objeto com a propriedade "3d", também teria que ser referenciá-la usando notação de colchetes. Por exemplo:

```
var anos = [1950, 1960, 1970, 1980, 1990, 2000, 2010];
console.log(anos.0); // um erro de sintaxe
console.log(anos[0]); // funciona corretamente
```

```
renderer.3d.setTexture(model, 'personagem.png'); // um erro de sintaxe
renderer['3d'].setTexture(model, 'personagem.png'); //funciona corretamente
```

Note que no exemplo `3d`, `'3d'` teve de ser colocado entre aspas. É possivel também colocar entre aspas os índices de arrays JavaScript (ou seja, `years['2']` ao invés de `years[2]`), contudo isto não é necessário. O valor 2 em `years[2]` eventualmente será convertido a uma string pela engine do JavaScript através de uma conversão explicita com o método `toString`. E é por esta razão que `'2'` e `'02'` irão referenciar dois slots diferentes no objeto `anos` e o seguinte exemplo pode ser *`true`*:

```
console.log(anos["2"] != anos["02"]);
```

De forma similar, propriedades de objeto que sejam palavras reservadas(!) só podem ser acessadas como strings em notação de colchetes:

```
var promessa = {
  var: "texto",
  array: [1, 2, 3, 4],
};

console.log(promessa["var"]);
```

Relação entre <code>length</code> e propriedades numéricas

As propriedades `length` e numéricas de um array Javascript são conectadas. Varios dos métodos javascript pré-definidos (por exemplo, `join`, `slice`, `indexOf` etc.) levam em conta o valor da propriedade `length` de um array quando eles são chamados. Outros métodos (por exemplo, `push`, `splice` etc.) também resultam em uma atualização na propriedade `length` do array.

```
var frutas = [];
frutas.push("banana", "maça", "pêssego");

console.log(frutas.length); // 3
```

Quando configurar uma propriedade num array Javascript em que a propriedade é um índice valido do array e este índice está fora do atual limite do array, o array irá crescer para um tamanho grande o suficiente para acomodar o elemento neste índice, e a engine irá atualizar a propriedade *length* do array de acordo com isto:

```
frutas[5] = "manga";
console.log(frutas[5]); // 'manga'
console.log(Object.keys(frutas)); // ['0', '1', '2', '5']
console.log(frutas.length); // 6
```

Configurar a propriedade `length` diretamente, também resulta em um comportamento especial:

```
frutas.length = 10;
console.log(Object.keys(frutas)); // ['0', '1', '2', '5']
console.log(frutas.length); // 10
```

Diminuir o valor de `length`, entretanto, apaga elementos:

```
frutas.length = 2;
console.log(Object.keys(frutas)); // ['0', '1']
console.log(frutas.length); // 2
```

Criando um array usando o resultado de uma comparação

O resultado de uma comparação entre uma *expressão regular* e uma string pode criar um array Javascript. Este array tem propriedades e elementos que disponibilizam informações sobre a comparação. Esse array é o valor de retorno dos métodos `RegExp.exec`, `String.match`, e `String.replace`. Para explicar melhor sobre estas propriedades e elementos, veja o seguinte exemplo e então consulte a tabela abaixo:

```
// Encontra um d seguido por um ou mais b's seguido por um d
// Salva os b's encontrados e o d seguinte
// Ignora caixa (maiúscula/minúscula)

var minhaRegex = /d(b+)(d)/i;
var meuArray = minhaRegex.exec("cdbBdbsbz");
```

As propriedades e elementos retornados desta comparação são os seguintes:

Propriedade/Elemento	Descrição	Exemplo
`input`	Uma propriedade somente-leitura que reflete a string original a qual a expressão regular foi comparada.	cdbBdbsbz
`index`	Uma propriedade somente-leitura que é o índice baseado em zero da comparação na string.	1
`[0]`	Um elemento somente-leitura que especifica os ultimos caracteres que foram encontrados.	dbBd
`[1], ...[n]`	Elementos somente-leitura que especificam as *substrings* de comparações entre parênteses encontradas, se incluidas na expressão regular. O número de possíveis *substrings* entre parenteses é ilimitado.	[1]: bB [2]: d

Propriedades

`Array.length`

Propriedade comprimento do construtor `Array`, cujo valor é 1.

`get Array[@@species]`

A função de construtor que é utilizada para criar objetos derivados.

`Array`

Permite a adição de propriedades para todos os objetos array.

Métodos

`Array.from()`

Cria uma nova instância de `Array` a partir de um objeto semelhante ou iterável.

`Array.isArray()`

Retorna `true` se a variável é um array e `false` caso contrário.

`Array.of()`

Cria uma nova instância de `Array` com um número variável de argumentos, independentemente do número ou tipo dos argumentos.

Instâncias de <code>Array</code>

Todas as instâncias de `Array` herdam de `Array.prototype`. O protótipo do construtor `Array` pode ser modificado de forma a afetar todas as instâncias de `Array`.

Propriedades

Métodos

Métodos modificadores

Métodos de acesso

Métodos de iteração

Métodos genéricos de <code>Array</code>

> **Aviso: Métodos genéricos de arrays não seguem o padrão, são obsoletos e serão removidos em breve.**

Algumas vezes você poderá querer aplicar métodos de arrays para strings ou outros objetos parecidos com arrays (como em `argumentos` de funções). Ao fazer isto, você trata uma string como um array de caracteres (ou em outros casos onde trata-se não-arrays como um array). Por exemplo, para checar se cada caractere em uma varivável *str* é uma letra, você poderia escrever:

```
function isLetter(character) {
  return character >= "a" && character <= "z";
}

if (Array.prototype.every.call(str, isLetter))
  alert("A string '" + str + "' contém somente letras!");
```

Esta notação é um pouco despendiosa e o JavaScript 1.6 introduziu a seguinte abreviação genérica:

```
if (Array.every(isLetter, str))
  alert("A string '" + str + "' contém somente letras!");
```

Generics também estão disponíveis em `String`.

Estes não são atualmente parte dos padrões ECMAScript (através do ES2015 `Array.from()` pode se conseguir isto). A seguir segue uma adaptação para permitir o uso em todos os navegadores:

```
/*globals define*/
// Assumes Array extras already present (one may use shims for these as well)
(function () {
  "use strict";

  var i,
    // We could also build the array of methods with the following, but the
    //   getOwnPropertyNames() method is non-shimable:
    // Object.getOwnPropertyNames(Array).filter(function (methodName) {return typeof Array[methodName] === 'function'});
    methods = [
      "join",
      "reverse",
      "sort",
      "push",
      "pop",
      "shift",
      "unshift",
      "splice",
      "concat",
      "slice",
      "indexOf",
      "lastIndexOf",
      "forEach",
      "map",
      "reduce",
      "reduceRight",
      "filter",
      "some",
      "every",
      "isArray",
    ],
    methodCount = methods.length,
    assignArrayGeneric = function (methodName) {
      var method = Array.prototype[methodName];
      Array[methodName] = function (arg1) {
        return method.apply(arg1, Array.prototype.slice.call(arguments, 1));
      };
    };

  for (i = 0; i < methodCount; i++) {
    assignArrayGeneric(methods[i]);
  }
})();
```

Exemplos

Exemplo: Criando um array

O exemplo a seguir cria um array, `msgArray`, com *length* 0, então atribui valores para `msgArray[0]` e `msgArray[99]`, trocando o *length* do array para 100.

```
var msgArray = new Array();
msgArray[0] = "Hello";
msgArray[99] = "world";
```

```
if (msgArray.length == 100) console.log("O length é 100.");
```

Criando um Array

```
var frutas = ["Maçã", "Banana"];

console.log(frutas.length);
// 2
```

Acessar um item (index) do Array

```
var primeiro = frutas[0];
// Maçã

var ultimo = frutas[frutas.length - 1];
// Banana
```

Iterar um Array

```
frutas.forEach(function (item, indice, array) {
  console.log(item, indice);
});
// Maçã 0
// Banana 1
```

Adicionar um item ao final do Array

```
var adicionar = frutas.push("Laranja");
// ['Maçã', 'Banana', 'Laranja']
```

Remover um item do final do Array

```
var ultimo = frutas.pop(); // remove Laranja (do final)
// ['Maçã', 'Banana'];
```

Remover do início do Array

```
var primeiro = frutas.shift(); // remove Maçã do início
// ['Banana'];
```

Adicionar ao início do Array

```
var adicionar = frutas.unshift("Morango"); // adiciona ao início
// ['Morango', 'Banana'];
```

Procurar o índice de um item na Array

```
frutas.push("Manga");
// ['Morango', 'Banana', 'Manga']

var pos = frutas.indexOf("Banana");
// 1
```

Remover um item pela posição do índice

```
var removedItem = frutas.splice(pos, 1); // é assim que se remove um item
// ['Morango', 'Manga']
```

Remover itens de uma posição de índice

```
var vegetais = ["Repolho", "Nabo", "Rabanete", "Cenoura"];
console.log(vegetais);
// ['Repolho', 'Nabo', 'Rabanete', 'Cenoura']

var pos = 1,
  n = 2;

var itensRemovidos = vegetais.splice(pos, n);
// Isso é como se faz para remover itens, n define o número de itens a se remover,
// a partir da posição (pos) em direção ao fim da array.

console.log(vegetais);
// ['Repolho', 'Cenoura'] (o array original é alterado)

console.log(itensRemovidos);
// ['Nabo', 'Rabanete']
```

Copiar um Array

```
var copiar = frutas.slice(); // é assim que se copia
// ['Morango', 'Manga']
```

Exemplo: Criando um array bi-dimensional

O exemplo a seguir cria um tabuleiro de xadrez usando dois arrays bi-dimensionais de string. A primeira jogada é feita copiando o 'p' em 6,4 para 4,4. A posição antiga de 6,4 é colocada em branco.

```
var board = [
  ["R", "N", "B", "Q", "K", "B", "N", "R"],
  ["P", "P", "P", "P", "P", "P", "P", "P"],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  ["p", "p", "p", "p", "p", "p", "p", "p"],
  ["r", "n", "b", "q", "k", "b", "n", "r"],
];
console.log(board.join("\n") + "\n\n");

// Fazendo o King's Pawn avançar 2
board[4][4] = board[6][4];
board[6][4] = " ";
console.log(board.join("\n"));
```

Saída:

```
R,N,B,Q,K,B,N,R
P,P,P,P,P,P,P,P
 , , , , , , ,
 , , , , , , ,
 , , , , , , ,
 , , , , , , ,
p,p,p,p,p,p,p,p
r,n,b,q,k,b,n,r

R,N,B,Q,K,B,N,R
P,P,P,P,P,P,P,P
 , , , , , , ,
 , , , , , , ,
 , , , ,p, , ,
 , , , , , , ,
p,p,p,p, ,p,p,p
r,n,b,q,k,b,n,r
```

Utilizando um array para tabular um conjunto de valores

```
values = [];
for (var x = 0; x < 10; x++) {
  values.push([2 ** x, 2 * x ** 2]);
}
console.table(values);
```

Saída:

```
0  1  0
1  2  2
2  4  8
3  8  18
4  16  32
5  32  50
6  64  72
7  128  98
8  256  128
9  512  162
```

Ver também

- "Indexing object properties" in JavaScript Guide: "Working with objects"
- New in JavaScript 1.7: Array comprehensions
- New in JavaScript 1.6: Array extras
- Draft: Typed Arrays

Array.prototype.at()

O método **`at()`** recebe um valor inteiro e retorna o item referente ao index dele, permitindo valores positivos ou negativos. Valores negativos contam apartir do último item do array.

Isto não quer dizer que exista alguma coisa errada em utilizar a notação com colchetes. Por exemplo `array[0]` retornaria o primeiro item. Porém ao invés de utilizar o `array.length` para os últimos items; e.g `array[array.length-1]` para o último item, você pode utilizar o `array.at(-1)`. (Veja os exemplos abaixo)

Experimente

```
const array1 = [5, 12, 8, 130, 44];

let index = 2;

console.log(`An index of ${index} returns ${array1.at(index)}`);
// Expected output: "An index of 2 returns 8"

index = -2;

console.log(`An index of ${index} returns ${array1.at(index)}`);
// Expected output: "An index of -2 returns 130"
```

Sintaxe

```
at(index);
```

Parâmetros

`index`

> O index (posição) do elemento do array que será retornado. Quando um valor negativo é passado como argumento a contagem dos elementos é feita de trás para frente.

Valor retornado

O elemento correspondente ao index passado como argumento. Retorna `undefined` se o index passado não for encontrado.

Exemplos

Retornando o último item de um array

O exemplo a seguir exibe uma função que retorna o último elemento encontrado de um array específico.

```
// O array com items
const cart = ["apple", "banana", "pear"];

// A função que retorna o último item do array passado como argumento
function returnLast(arr) {
  return arr.at(-1);
}

// Pega o último item do array 'cart'
const item1 = returnLast(cart);
console.log(item1); // 'pear'

// Adiciona um item ao array 'cart'
cart.push("orange");
const item2 = returnLast(cart);
console.log(item2); // 'orange'
```

Comparando métodos

Este exemplo compara as diferentes maneiras de selecionar o penúltimo item de um `Array`. Todos os métodos abaixos são válidos, mas este exemplo destaca a sucintez e legibilidade do método `at()`.

```
// O array com items
const colors = ["red", "green", "blue"];

// Usando a propriedade length
const lengthWay = colors[colors.length - 2];
console.log(lengthWay); // 'green'

// Usando o método slice(). Perceba que um array é retornado
const sliceWay = colors.slice(-2, -1);
console.log(sliceWay[0]); // 'green'

// Usando o método at()
const atWay = colors.at(-2);
console.log(atWay); // 'green'
```

Veja também

- Um polyfill do método `Array.prototype.at` está disponível em `core-js`
- Um polyfill para o método at().
- `Array.prototype.find()` – retorna um valor baseado no teste fornecido.
- `Array.prototype.includes()` – verifica se um valor existe em um array.
- `Array.prototype.indexOf()` – retorna o index de um determinado elemento.

Array.prototype.concat()

O método **`concat()`** é utilizado para mesclar dois ou mais arrays. Esse método não altera os arrays existentes, mas, em vez disso, retorna um novo array.

Experimente

```
const array1 = ["a", "b", "c"];
const array2 = ["d", "e", "f"];
const array3 = array1.concat(array2);

console.log(array3);
// Expected output: Array ["a", "b", "c", "d", "e", "f"]
```

Sintaxe

```
concat()
concat(valor0)
concat(valor0, valor1)
concat(valor1, valor2, /*...,*/ valorN)
```

Parâmetros

`valorN` Optional

> Arrays e/ou valores para concatenar em um novo array. Se todos os parâmetros `valorN` forem omitidos, `concat` retorna uma shallow copy de todos os arrays existentes do qual é chamado. Veja a descrição abaixo para mais detalhes.

Valor de retorno

Uma nova instância `Array`.

Descrição

O método `concat` cria um novo array. O array irá primeiro ser preenchido pelos elementos no objeto no qual é chamado. Então, para cada argumento, seu valor será concatenado no array — para objetos normais ou primitivos, o próprio argumento se tornará um elemento do array final; para arrays ou objetos tipo array com a propriedade `Symbol.isConcatSpreadable` definida como um valor verdadeiro, cada elemento do argumento será adicionado independentemente ao array final. O método `concat` não recursa em argumentos de array aninhados.

O método `concat()` é um copying method. Ele não altera `this` ou quaquer um dos arrays fornecidos como argumentos, mas, em vez disso retorna uma shallow copy que contém os mesmos elementos dos arrays originais.

O método `concat()` preserva slots vazios se qualquer um dos arrays de origem for sparse.

O método `concat()` é genérico. O valor `this` é tratado da mesma forma como os outros argumentos (exceto que será convertido em um objeto primeiro), o que significa que objetos simples serão anexados diretamente ao array resultante, enquanto objetos do tipo array com verdadeiro `@@isConcatSpreadable` serão propagados no array resultante.

Exemplos

Concatenando dois arrays

O código a seguir concatena dois arrays:

```
const letras = ["a", "b", "c"];
const numeros = [1, 2, 3];

const alfaNumerico = letras.concat(numeros);
console.log(alfaNumerico);
// resulta em ['a', 'b', 'c', 1, 2, 3]
```

Concatenando três arrays

O código a seguir concatena três arrays:

```
const num1 = [1, 2, 3];
const num2 = [4, 5, 6];
const num3 = [7, 8, 9];

const numeros = num1.concat(num2, num3);

console.log(numeros);
// resulta em [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Concatenando valores em um array

O código a seguir concatena três valores em um array:

```
const letras = ["a", "b", "c"];

const alfaNumerico = letras.concat(1, [2, 3]);

console.log(alfaNumerico);
// resulta em ['a', 'b', 'c' , 1, 2, 3]
```

Concatenando arrays aninhados

O código a seguir concatena arrays aninhados e demonstra a retenção de referência:

```
const num1 = [[1]];
const num2 = [2, [3]];

const numeros = num1.concat(num2);

console.log(numeros);
// resulta em [[1], 2, [3]]

//modifica o primeiro elemento do num1
num1[0].push(4);

console.log(numeros);
// resulta em [[1, 4], 2, [3]]
```

Concatenanto objetos tipo array com Symbol.isConcatSpreadable

`concat` não trata todos os objetos tipo array como arrays por padrão — somente se `Symbol.isConcatSpreadable` estiver definido como um valor verdadeiro (P.ex. `true`)

```
const obj1 = { 0: 1, 1: 2, 2: 3, length: 3 };
const obj2 = { 0: 1, 1: 2, 2: 3, length: 3, [Symbol.isConcatSpreadable]: true };
console.log([0].concat(obj1, obj2));
// resulta em [ 0, { '0': 1, '1': 2, '2': 3, length: 3 }, 1, 2, 3 ]
```

Usando concat() em arrays sparse

Se qualquer uma das arrays é sparse, a array resultante será também sparse:

```
console.log([1, , 3].concat([4, 5])); // [1, empty, 3, 4, 5]
console.log([1, 2].concat([3, , 5])); // [1, 2, 3, empty, 5]
```

Chamando concat() em objetos não array

Se o valor `this` não for um array, ele será convertido em um objeto e então tratado da mesma forma que os argumentos para `concat()`. Nesse caso o valor de retorno é sempre um novo array simples.

```
console.log(Array.prototype.concat.call({}, 1, 2, 3)); // [{}, 1, 2, 3]
console.log(Array.prototype.concat.call(1, 2, 3)); // [ [Number: 1], 2, 3 ]
const arrayLike = { [Symbol.isConcatSpreadable]: true, length: 2, 0: 1, 1: 2 };
console.log(Array.prototype.concat.call(arrayLike, 3, 4)); // [1, 2, 3, 4]
```

Veja também

- Polyfill of `Array.prototype.concat` in `core-js` with fixes and implementation of modern behavior like `Symbol.isConcatSpreadable` support
- Indexed collections
- `Array`
- `Array.prototype.push()`
- `Array.prototype.unshift()`
- `Array.prototype.splice()`
- `String.prototype.concat()`
- `Symbol.isConcatSpreadable`

Array.prototype.copyWithin()

O método **`copyWithin()`** copia parte de um array para outra localização no mesmo array e o retorna sem alterar seu tamanho.

Experimente

```
const array1 = ["a", "b", "c", "d", "e"];

// Copy to index 0 the element at index 3
console.log(array1.copyWithin(0, 3, 4));
// Expected output: Array ["d", "b", "c", "d", "e"]

// Copy to index 1 all elements from index 3 to the end
console.log(array1.copyWithin(1, 3));
// Expected output: Array ["d", "d", "e", "d", "e"]
```

Sintaxe

```
copyWithin(target)
copyWithin(target, start)
copyWithin(target, start, end)
```

Parâmetros

`target`

Índice de base zero à qual copiar a sequência para, convertido para inteiro.

- Índice negativo será contado a partir do final do array — se `target < 0`, `target + array.length` é utilizado.
- Se `target < -array.length`, `0` é utilizado.

- Se `target >= array.length`, nada é copiado.
- Se `target` é posicionado após `start` depois da normalização, a cópia só acontece até o final do `array.length` (em outras palavras, `copyWithin()` nunca estende o array).

`start` Optional

Índice de base zero à qual inicia a cópia dos elementos a partir de, convertido para inteiro.

- Índice negativo será contado a partir do final do array — se `start < 0`, `start + array.length` é utilizado.
- Se `start < -array.length` ou `start` é omitido, `0` é utilizado.
- Se `start >= array.length`, nada é copiado.

`end` Optional

Índice de base zero à qual termina a cópia dos elementos a partir de, convertido para inteiro. `copyWithin()` copia até, mas não inclui o `end`.

- Índice negativo será contado a partir do final do array — se `end < 0`, `end + array.length` é utilizado.
- Se `end < -array.length`, `0` é utilizado.
- Se `end >= array.length` ou `end` é omitido, `array.length` é utilizado, fazendo com que todos os elementos até o final sejam copiados.
- Se `end` é posicionado antes ou em `start` após a normalização, nada será copiado.

Valor de retorno

O array modificado.

Descrição

O método `copyWithin()` funciona como o `memmove` do C e C++, e é um método de alta performance para troca de dados de um `Array`. Isso se aplica especialmente ao método `TypedArray` de mesmo nome. A sequência é copiada e colada como uma operação; a sequência colada terá os valores copiados mesmo quando a região de copiar e colar se sobrepuserem.

O método `copyWithin()` é um método mutável. Ele não altera o comprimento de `this`, mas mudará o conteúdo de `this` e criará novas propriedades ou excluirá propriedades existentes, se necessário.

O método `copyWithin()` preserva slots vazios. Se a região a ser copiada for sparse, os novos índices correspondentes dos slots vazios são excluídos e também se tornam slots vazios.

O método `copyWithin()` é genérico. Ele apenas espera que o valor de `this` tenha uma propriedade `length` e propriedades integer-keyed. Embora as strings também sejam semelhantes a arrays, esse método não é adequado para ser aplicado nelas, pois as strings são imutáveis.

Exemplos

Usando copyWithin()

```
console.log([1, 2, 3, 4, 5].copyWithin(-2));
// [1, 2, 3, 1, 2]

console.log([1, 2, 3, 4, 5].copyWithin(0, 3));
// [4, 5, 3, 4, 5]

console.log([1, 2, 3, 4, 5].copyWithin(0, 3, 4));
// [4, 2, 3, 4, 5]

console.log([1, 2, 3, 4, 5].copyWithin(-2, -3, -1));
// [1, 2, 3, 3, 4]
```

Usando copyWithin() em arrays sparse

`copyWithin()` propagará slots vazios(empty).

```
console.log([1, , 3].copyWithin(2, 1, 2)); // [1, empty, empty]
```

Chamando copyWithin() em objetos não array

O método `copyWithin()` lê a propriedade `length` do `this` e então manipula os índices inteiros envolvidos.

```
const arrayLike = {
  length: 5,
  3: 1,
};
console.log(Array.prototype.copyWithin.call(arrayLike, 0, 3));
// { '0': 1, '3': 1, length: 5 }
console.log(Array.prototype.copyWithin.call(arrayLike, 3, 1));
// { '0': 1, length: 5 }
// A propriedade '3' é excluída porque a fonte copiada é um slot vazio.
```

Veja também

- Polyfill de `Array.prototype.copyWithin` no `core-js`
- Coleções indexadas
- `Array`
- `TypedArray.prototype.copyWithin()`

Array.prototype.entries()

O método **`entries()`** retorna um novo objeto **`Array Iterator`** que contém os pares chave/valor para cada índice no array.

Sintaxe

```
arr.entries()
```

Exemplos

```
var arr = ["a", "b", "c"];
var eArr = arr.entries();

console.log(eArr.next().value); // [0, 'a']
console.log(eArr.next().value); // [1, 'b']
console.log(eArr.next().value); // [2, 'c']
```

O mesmo que acima, utilizando um loop for...of

```
var arr = ["a", "b", "c"];
var eArr = arr.entries();

for (let e of eArr) {
  console.log(e);
}
// [0, 'a']
// [1, 'b']
// [2, 'c']
```

Veja também

- `Array.prototype.keys()`
- `Array.prototype.forEach()`
- `Array.prototype.every()`
- `Array.prototype.some()`

Array.prototype.every()

O método `every()` testa se todos os elementos do array passam pelo teste implementado pela função fornecida. Este método retorna um valor booleano.

Experimente

```
const isBelowThreshold = (currentValue) => currentValue < 40;

const array1 = [1, 30, 39, 29, 10, 13];

console.log(array1.every(isBelowThreshold));
// Expected output: true
```

Sintaxe

```
arr.every(callback[, thisArg])
```

Parâmetros

`callback`

Função que testa cada elemento, recebe três parametros:

`currentValue` (obrigatório)

O elemento atual sendo processado na array.

`index` (opcional)

O índice do elemento atual sendo processado na array.

`array` (opcional)

O array de origem.

`thisArg`

Opcional. Valor a ser usado como `this` quando o `callback` é executado.

Valor de retorno

true se a função de callback retorna um valor truthy para cada um dos elementos do array; caso contrário, **false**.

Descrição

O método `every` executa a função `callback` fornecida uma vez para cada elemento presente no array, até encontrar algum elemento em que a função retorne um valor false (valor que se torna false quando convertido para boolean). Se esse elemento é encontrado, o método `every` imediatamente retorna false. Caso contrário, se a função `callback` retornar true para todos elementos, o método retorna true. A função `callback` é chamada apenas para os elementos do array original que tiverem valores atribuídos; os elementos que tiverem sido removidos ou os que nunca tiveram valores atribuídos não serão considerados.

A função `callback` é chamada com três argumentos: o valor do elemento corrente, o índice do elemento corrente e o array original que está sendo percorrido.

Se o parâmetro `thisArg` foi passado para o método `every`, ele será repassado para a função `callback` no momento da chamada para ser utilizado como o `this`. Caso contrário, o valor `undefined` será repassado para uso como o *`this`*. O valor do `this` a ser repassado para o `callback` é determinado de acordo com as regras usuais para determinar o this visto por uma função.

O método `every`não modifica o array original.

A lista de elementos que serão processados pelo `every` é montada antes da primeira chamada da função `callback`. Se um elemento for acrescentado ao array original após a chamada ao `every` , ele não será visível para o callback. Se os elementos existentes forem modificados, os valores que serão repassados serão os do momento em que o método `every` chamar o `callback`. Elementos removidos não serão considerados.

`every` funciona como o qualificador "for all" em matemática. Particularmente, para um vetor vazio, é retornado true. (É verdade por vacuidade que todos os elementos do conjunto vazio satisfazem qualquer condição.)

Exemplos

Testando tamanho de todos os elementos do vetor

O exemplo a seguir testa se todos elementos no array são maiores que 10.

```
function isBigEnough(element, index, array) {
  return element >= 10;
}
[12, 5, 8, 130, 44].every(isBigEnough); // false
[12, 54, 18, 130, 44].every(isBigEnough); // true
```

Usando arrow functions

Arrow functions fornecem sintaxe mais curta para o mesmo teste.

```
[12, 5, 8, 130, 44].every((elem) => elem >= 10); // false
[12, 54, 18, 130, 44].every((elem) => elem >= 10); // true
```

Polyfill

`every` foi adicionado ao padrão ECMA-262 na 5ª edição; como tal, pode não estar presente em outras implementações do padrão. Você pode contornar isso adicionando o seguinte código no começo dos seus scripts, permitindo o uso de every em implementações que não o suportam nativamente. Esse algoritimo é exatamente o mesmo especificado no ECMA-262, 5ª edição, assumindo que `Object` e `TypeError` tem os seus valores originais e que `callbackfn.call` retorna o valor original de `Function.prototype.call`

```
if (!Array.prototype.every) {
  Array.prototype.every = function (callbackfn, thisArg) {
    "use strict";
    var T, k;

    if (this == null) {
      throw new TypeError("this is null or not defined");
    }

    // 1. Let O be the result of calling ToObject passing the this
    //    value as the argument.
    var O = Object(this);

    // 2. Let lenValue be the result of calling the Get internal method
    //    of O with the argument "length".
    // 3. Let len be ToUint32(lenValue).
    var len = O.length >>> 0;

    // 4. If IsCallable(callbackfn) is false, throw a TypeError exception.
    if (typeof callbackfn !== "function") {
      throw new TypeError();
    }

    // 5. If thisArg was supplied, let T be thisArg; else let T be undefined.
    if (arguments.length > 1) {
      T = thisArg;
    }

    // 6. Let k be 0.
    k = 0;

    // 7. Repeat, while k < len
    while (k < len) {
      var kValue;

      // a. Let Pk be ToString(k).
      //   This is implicit for LHS operands of the in operator
      // b. Let kPresent be the result of calling the HasProperty internal
      //    method of O with argument Pk.
      //   This step can be combined with c
      // c. If kPresent is true, then
      if (k in O) {
```

```
        // i. Let kValue be the result of calling the Get internal method
        //    of O with argument Pk.
        kValue = O[k];

        // ii. Let testResult be the result of calling the Call internal method
        //     of callbackfn with T as the this value and argument list
        //     containing kValue, k, and O.
        var testResult = callbackfn.call(T, kValue, k, O);

        // iii. If ToBoolean(testResult) is false, return false.
        if (!testResult) {
          return false;
        }
      }
      k++;
    }
    return true;
  };
}
```

Veja também

- `Array.prototype.forEach()`
- `Array.prototype.some()`
- `TypedArray.prototype.every()`

Array.prototype.fill()

O método **`fill()`** preenche todos os valores do array a partir do índice inicial a um índice final com um valor estático.

Sintaxe

```
arr.fill(valor[, início = 0[, fim = this.length]])
```

Parâmetros

`valor`

Valor para preencher o array.

`início`

Opcional. Índice inicial.

`fim`

Opcional. Índice final.

Descrição

O intervalo de preenchimento dos elementos é [`início`, `fim`).

O método **`fill`** pode receber até três argumentos `valor`, `início` e `fim`. Os argumentos `início` e `fim` são opcionais com valor padrão `0 (valor)` e o tamanho do objeto (`fim`).

Se o `início` for negativo, ele será tratado como `tamanho + início` onde `tamanho é o tamanho total do array`. Se o `fim` for negativo, ele será tratado como `tamanho + fim`.

A função **fill** é intencionalmente genérica, ele não precisa que o valor do this seja um objeto Array.

O método **fill** é um método mutável, ele irá mudar o objeto em si, e retorná-lo, não somente uma cópia do objeto.

Exemplos

```
[1, 2, 3].fill(4); // [4, 4, 4]
[1, 2, 3].fill(4, 1); // [1, 4, 4]
[1, 2, 3].fill(4, 1, 2); // [1, 4, 3]
[1, 2, 3].fill(4, 1, 1); // [1, 2, 3]
[1, 2, 3].fill(4, -3, -2); // [4, 2, 3]
[1, 2, 3].fill(4, NaN, NaN); // [1, 2, 3]
Array(3).fill(4); // [4, 4, 4]
[].fill.call({ length: 3 }, 4); // {0: 4, 1: 4, 2: 4, length: 3}
```

Polyfill

```
if (!Array.prototype.fill) {
  Array.prototype.fill = function (value) {
    // Passo 1-2.
    if (this == null) {
      throw new TypeError("this is null or not defined");
    }

    var O = Object(this);
```

```
    // Passo 3-5.
    var len = O.length >>> 0;

    // Passo 6-7.
    var start = arguments[1];
    var relativeStart = start >> 0;

    // Passo 8.
    var k =
      relativeStart < 0
        ? Math.max(len + relativeStart, 0)
        : Math.min(relativeStart, len);

    // Passo 9-10.
    var end = arguments[2];
    var relativeEnd = end === undefined ? len : end >> 0;

    // Passo 11.
    var final =
      relativeEnd < 0
        ? Math.max(len + relativeEnd, 0)
        : Math.min(relativeEnd, len);

    // Passo 12.
    while (k < final) {
      O[k] = value;
      k++;
    }

    // Passo 13.
    return O;
  };
}
```

[1] Começando com Chrome 36, isto era disponível com uma mudança nas preferencias. Em chrome://flags, ativar a entrada "Enable Experimental JavaScript".

Ver também

- `Array`
- `TypedArray.prototype.fill()`

Array.prototype.filter()

O método **`filter()`** cria um novo array com todos os elementos que passaram no teste implementado pela função fornecida.

Experimente

```
const words = ["spray", "elite", "exuberant", "destruction", "present"];

const result = words.filter((word) => word.length > 6);

console.log(result);
// Expected output: Array ["exuberant", "destruction", "present"]
```

Sintaxe

```
var newArray = arr.filter(callback[, thisArg])
```

Parâmetros

`callback`

Função é um predicado, para testar cada elemento do array. Retorna `true` para manter o elemento, `false` caso contrário, recebendo três argumentos:

`element`

O elemento que está sendo processado no array.

`index`

O índice do elemento atual que está sendo processado no array.

`array`

O array para qual `filter` foi chamada.

`thisArg Optional`

Opcional. Valor a ser usado como `this` durante a execução do `callback`.

Valor de retorno

Um novo array com os elementos que passaram no teste.

Descrição

`filter()` chama a função callback fornecida, uma vez para cada elemento do array, e constrói um novo array com todos os valores para os quais o `callback` retornou o valor `true` ou um valor que seja convertido para true. O `callback` é chamado apenas para índices do array que possuem valores atribuídos; Ele não é invocado para índices que foram excluídos ou para aqueles que não tiveram valor atribuído. Elementos do array que não passaram no teste do `callback` são simplesmente ignorados, e não são incluídos no novo array.

`callback` é invocado com estes três argumentos:

1. o valor do elemento
2. o índice do elemento
3. o objeto do array a ser preenchido

Se o parâmetro `thisArg` for provido para o filter, ele será passado para o `callback` quando invocado, para ser usado como o valor do this. Caso contrário, será passado `undefined` como o valor de `this`. O valor do `this` finalmente observado pela função de `callback` é determinado de acordo com a regra que define o valor do `this` geralmente visto por uma função.

`filter()` não altera o `array` a partir da qual foi invocado.

O intervalo de elementos processados pela função `filter()` é definido antes da invocação do primeiro callback. Elementos que forem adicionados ao array depois da invocação do `filter()` não serão visitados pelo `callback`. Se elementos existentes no array forem alterados ou deletados, os valores deles que serão passados para o callback são os que eles tiverem quando o `filter()` visitá-los; Elementos que forem deletados não são visitados.

Exemplos

Exemplo: Filtrando todos os valores pequenos

Os exemplos a seguir usam `filter()` para criar um `array` filtrado em que todos os elementos com valores menores que 10 são removidos.

```
function isBigEnough(value) {
  return value >= 10;
}
var filtered = [12, 5, 8, 130, 44].filter(isBigEnough);
// filtered is [12, 130, 44]
```

Exemplo: Filtrando entradas inválidas para JSON

O exemplo a seguir usa `filter()` para criar um `JSON` filtrado com todos seus elementos diferentes de zero, e `id` numérico.

```
var arr = [
  { id: 15 },
  { id: -1 },
  { id: 0 },
  { id: 3 },
  { id: 12.2 },
  {},
  { id: null },
  { id: NaN },
  { id: "undefined" },
];

var invalidEntries = 0;

function filterByID(obj) {
  if ("id" in obj && typeof obj.id === "number" && !isNaN(obj.id)) {
    return true;
  } else {
    invalidEntries++;
    return false;
  }
}

var arrByID = arr.filter(filterByID);

console.log("Filtered Array\n", arrByID);
// [{ id: 15 }, { id: -1 }, { id: 0 }, { id: 3 }, { id: 12.2 }]

console.log("Number of Invalid Entries = ", invalidEntries);
// Number of Invalid Entries = 4
```

Procurando em um array

O exemplo a seguir usa `filter()` para filtrar o conteúdo de um array baseado em um critério de busca

```
var fruits = ["apple", "banana", "grapes", "mango", "orange"];

/**
 * Array filters items based on search criteria (query)
 */
function filterItems(query) {
  return fruits.filter(function (el) {
    return el.toLowerCase().indexOf(query.toLowerCase()) > -1;
  });
}
```

```
console.log(filterItems("ap")); // ['apple', 'grapes']
console.log(filterItems("an")); // ['banana', 'mango', 'orange']
```

Implementação ES2015

```
const fruits = ["apple", "banana", "grapes", "mango", "orange"];

/**
 * Array filters items based on search criteria (query)
 */
const filterItems = (query) => {
  return fruits.filter(
    (el) => el.toLowerCase().indexOf(query.toLowerCase()) > -1,
  );
};

console.log(filterItems("ap")); // ['apple', 'grapes']
console.log(filterItems("an")); // ['banana', 'mango', 'orange']
```

Polyfill

`filter()` foi adicionado ao padrão ECMA-262 na 5ª edição; assim como pode não estar presente em todas as implementações do padrão. Você pode trabalhar com isso adicionando o seguinte código no início de seus scripts, permitindo o uso do `filter()` na implementação ECMA-262 que não tem suporte nativo. Esse algoritmo é exatamente aquele especificado na 5ª edição do ECMA-262, assumindo que `fn.call` veja o valor original de `Function.prototype.call()`, e que `Array.prototype.push()` tenha seu valor original.

```
if (!Array.prototype.filter) {
  Array.prototype.filter = function (fun /*, thisArg*/) {
    "use strict";

    if (this === void 0 || this === null) {
      throw new TypeError();
    }

    var t = Object(this);
    var len = t.length >>> 0;
    if (typeof fun !== "function") {
      throw new TypeError();
    }

    var res = [];
    var thisArg = arguments.length >= 2 ? arguments[1] : void 0;
    for (var i = 0; i < len; i++) {
      if (i in t) {
        var val = t[i];

        // NOTE: Technically this should Object.defineProperty at
        //       the next index, as push can be affected by
        //       properties on Object.prototype and Array.prototype.
        //       But that method's new, and collisions should be
        //       rare, so use the more-compatible alternative.
        if (fun.call(thisArg, val, i, t)) {
          res.push(val);
        }
      }
    }

    return res;
  };
}
```

Veja também

- `Array.prototype.forEach()`
- `Array.prototype.every()`
- `Array.prototype.some()`
- `Array.prototype.reduce()`

Array.prototype.find()

O método **`find()`** retorna o **valor** do **primeiro elemento** do array que satisfizer a função de teste provida. Caso contrario, `undefined` é retornado.

Experimente

```
const array1 = [5, 12, 8, 130, 44];

const found = array1.find((element) => element > 10);

console.log(found);
// Expected output: 12
```

Veja também o método `findIndex()`, que retorna o **índice** do elemento encontrado no array ao invés do seu valor.

Se você precisa encontrar a posição de um elemento ou se um elemento existe em um array, use `Array.prototype.indexOf()` ou `Array.prototype.includes()`.

Sintaxe

```
arr.find(callback(element[, index[, array]])[, thisArg])
```

Parâmetros

`callback`

Função executada a cada iteração do array, recebendo três argumentos:

`element`

O elemento atual que está sendo processado no array.

`index`Optional

O índice do elemento atualmente sendo processado no array.

`array`Optional

O array sobre o qual `find` foi chamado.

`thisArg`Optional

Opcional. Objeto usado como `this` quando executando o `callback`.

Valor retornado

O **valor** do **primeiro elemento** do array que satisfaz a função de teste fornecida; caso contrário, `undefined`.

Descrição

O método `find` executa a função `callback` uma vez para cada elemento presente no array até que encontre um onde `callback` retorne o valor true. Se o elemento é encontrado, `find` retorna imediatamente o valor deste elemento. Caso contrário, `find` retorna `undefined`. O `callback` é acionado para todos os índices do array de `0` a `tamanho-1`, não apenas para aqueles que possuem valores atribuídos. Sendo assim, ele pode ser menos eficiente para arrays muito grandes em que existem outros métodos que só visitam os índices que tenham valor atribuído.

O `callback` é acionado com três argumentos: o valor do elemento, o índice do elemento e o objeto do Array que está sendo executado.

Se um parâmetro `thisArg` é provido ao `find`, ele será usado como o `this` para cada acionamento do `callback`. Se não for provido, então `undefined` é usado.

O `find` não altera a array à qual foi acionado.

O conjunto dos elementos processados por `find` é definido antes do primeiro acionamento do `callback`. Elementos que são anexados à array após o início da chamada ao `find` não serão visitados pelo `callback`. Se um elemento existente ainda não visitado da array for alterado pelo `callback`, o valor passado ao `callback` quando o visitar será o valor no momento que `find` visita o índice daquele elemento; elementos excluídos ainda são visitados.

Exemplos

Encontrar um objeto em um array por uma de suas propriedades

```
const inventory = [
  { name: "apples", quantity: 2 },
  { name: "bananas", quantity: 0 },
  { name: "cerejas", quantity: 5 },
];

function isCherries(fruit) {
  return fruit.name === "cerejas";
}

console.log(inventory.find(isCherries));
// { name: 'cerejas', quantity: 5 }
```

Utilizando arrow function

```
const inventory = [
  { name: "maças", quantity: 2 },
  { name: "bananas", quantity: 0 },
  { name: "cherries", quantity: 5 },
];

const result = inventory.find((fruit) => fruit.name === "cherries");

console.log(result); // { name: 'cherries', quantity: 5 }
```

Encontrar um número primo em um array

O exemplo a seguir encontra um elemento dentro da array que é número primo (ou retorna `undefined` se não houverem números primos).

```
function isPrime(element, index, array) {
  var start = 2;
```

```
  while (start <= Math.sqrt(element)) {
    if (element % start++ < 1) {
      return false;
    }
  }
  return element > 1;
}

console.log([4, 6, 8, 12].find(isPrime)); // undefined, not found
console.log([4, 5, 8, 12].find(isPrime)); // 5
```

Polyfill

Este método foi adicionado à especificação do ECMAScript 2015 e pode não estar disponível em todas as implementações do JavaScript. Entretanto, você pode fazer um polyfill para o `Array.prototype.find` com o trecho de código abaixo:

```
if (!Array.prototype.find) {
  Array.prototype.find = function (predicate) {
    if (this === null) {
      throw new TypeError("Array.prototype.find called on null or undefined");
    }
    if (typeof predicate !== "function") {
      throw new TypeError("predicate must be a function");
    }
    var list = Object(this);
    var length = list.length >>> 0;
    var thisArg = arguments[1];
    var value;

    for (var i = 0; i < length; i++) {
      value = list[i];
      if (predicate.call(thisArg, value, i, list)) {
        return value;
      }
    }
    return undefined;
  };
}
```

Veja também

- `Array.prototype.findIndex()`
- `Array.prototype.every()`

Array.prototype.findIndex()

O método **`findIndex()`** retorna o **índice** no array do primeiro elemento que satisfizer a função de teste provida. Caso contrário, retorna -1, indicando que nenhum elemento passou no teste.

Veja também o método `find()`, que retorna o **valor** de um elemento encontrado no array em vez de seu índice.

Sintaxe

```
arr.findIndex(callback[, thisArg])
```

Parâmetros

`callback`

Função para executar em cada valor no array, tomando três argumentos:

`element`

O elemento atual sendo processado no array.

`index`

O índice do elemento atual sendo processado no array.

`array`

O array sobre o qual `findIndex` foi chamado.

`thisArg`

Opcional. Objeto para usar como `this` na execução do `callback`.

Descrição

O método `findIndex` executa a função `callback` uma vez para cada elemento presente no array até encontrar um onde o `callback` retorna um valor verdadeiro. Se tal elemento for encontrado, `findIndex` imediatamente retorna o índice deste elemento. Caso contrário, `findIndex` retorna -1. `callback` é invocado apenas para índices no array que têm valores atribuídos; nunca é invocado para índices que foram deletados ou que nunca tiveram valores atribuídos.

`callback` é invocado com três argumentos: o valor do elemento, o índice do elemento e o objeto Array sendo percorrido.

Se um parâmetro `thisArg` for fornecido para `findIndex`, ele será usado como o `this` para cada invocação do `callback`. Se não for fornecido, então `undefined` é usado.

`findIndex` não modifica o array sobre o qual é chamado.

A série de elementos processados por `findIndex` é definida antes da primeira invocação do `callback`. Elementos que são adicionados ao array depois que a chamada a `findIndex` começa não serão visitados pelo `callback`. Se um elemento existente não visitado do array for modificado pelo `callback`, seu valor passado ao `callback` será o valor no momento em que `findIndex` visitar o índice deste elemento; elementos que forem deletados não são visitados.

Exemplos

Encontrar o índice de um número primo em um array

O seguinte exemplo encontra o índice de um elemento no array que é um número primo (ou retorna -1 se não houver número primo).

```
function isPrime(element, index, array) {
  var start = 2;
  while (start <= Math.sqrt(element)) {
    if (element % start++ < 1) {
      return false;
    }
  }
  return element > 1;
}

console.log([4, 6, 8, 12].findIndex(isPrime)); // -1, não encontrado
console.log([4, 6, 7, 12].findIndex(isPrime)); // 2
```

Polyfill

Esse método foi adicionado à especificação do ECMAScript 6 e pode não estar disponível em todas as implementações de JavaScript ainda. Contudo, você pode fazer o polyfill de `Array.prototype.findIndex` com o seguinte trecho de código:

```
if (!Array.prototype.findIndex) {
  Array.prototype.findIndex = function (predicate) {
    if (this === null) {
      throw new TypeError(
        "Array.prototype.findIndex called on null or undefined",
      );
    }
    if (typeof predicate !== "function") {
      throw new TypeError("predicate must be a function");
    }
    var list = Object(this);
    var length = list.length >>> 0;
    var thisArg = arguments[1];
    var value;

    for (var i = 0; i < length; i++) {
      value = list[i];
      if (predicate.call(thisArg, value, i, list)) {
        return i;
      }
    }
    return -1;
  };
}
```

- `Array.prototype.find()`
- `Array.prototype.indexOf()`

Array.prototype.flat()

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

O método **`flat()`** cria um novo array com todos elementos sub-arrays concatenados nele de forma recursiva até a profundidade especificada.

Experimente

```
const arr1 = [0, 1, 2, [3, 4]];

console.log(arr1.flat());
// expected output: Array [0, 1, 2, 3, 4]

const arr2 = [0, 1, [2, [3, [4, 5]]]];

console.log(arr2.flat());
// expected output: Array [0, 1, 2, Array [3, Array [4, 5]]]

console.log(arr2.flat(2));
// expected output: Array [0, 1, 2, 3, Array [4, 5]]
```

```
console.log(arr2.flat(Infinity));
// expected output: Array [0, 1, 2, 3, 4, 5]
```

Sintaxe

```
flat();
flat(depth);
```

Parâmetros

`depth` Optional

O nível de profundidade especifíca o quão profundo um array aninhando deve ser achatado. O padrão é 1.

Retorno

Um novo array com os elementos sub-array concatenados nele.

Exemplos

Achatando arrays aninhados

```
var arr1 = [1, 2, [3, 4]];
arr1.flat();
// [1, 2, 3, 4]

var arr2 = [1, 2, [3, 4, [5, 6]]];
arr2.flat();
// [1, 2, 3, 4, [5, 6]]

var arr3 = [1, 2, [3, 4, [5, 6]]];
arr3.flat(2);
// [1, 2, 3, 4, 5, 6]

const arr4 = [1, 2, [3, 4, [5, 6, [7, 8]]]];
arr4.flat(Infinity);
// [1, 2, 3, 4, 5, 6, 7, 8]
```

Achatando e buracos em array

o método flat remove espaços vazios do array:

```
var arr5 = [1, 2, , 4, 5];
arr5.flat();
// [1, 2, 4, 5]
```

Alternativa

<code>reduce</code> e <code>concat</code>

```
var arr = [1, 2, [3, 4]];

// Achatar array de nível único
arr.flat();
// É equivalente à
arr.reduce((acc, val) => acc.concat(val), []);
// [1, 2, 3, 4]

// Ou com sintaxe de decomposição
const flattened = (arr) => [].concat(...arr);
```

<code>reduce</code> + <code>concat</code> + <code>isArray</code> + recursividade

```
var arr = [1, 2, [3, 4, [5, 6]]];

// Para achatamentos mais profundos, use recursividade com reduce e concat
function flatDeep(arr, d = 1) {
  return d > 0
    ? arr.reduce(
        (acc, val) =>
          acc.concat(Array.isArray(val) ? flatDeep(val, d - 1) : val),
        [],
      )
    : arr.slice();
}

flatDeep(arr, Infinity);
// [1, 2, 3, 4, 5, 6]
```

Especificações

Especificação	Status	Comentários
`Array.prototype.flat` proposal	Draft	

Veja Também

- `Array.prototype.flatMap()`
- `Array.prototype.map()`
- `Array.prototype.reduce()`
- `Array.prototype.concat()`

Array.prototype.flatMap()

O método **`flatMap()`** primeiro mapeia cada elemento usando uma função de mapeamento e, em seguida, nivela o resultado em um novo array. É idêntico a um `map` seguido por um `flat` de profundidade 1, mas **`flatMap`** é bastante útil e mescla ambos em um método um pouco mais eficiente.

Experimente

```
const arr1 = [1, 2, 1];

const result = arr1.flatMap((num) => (num === 2 ? [2, 2] : 1));

console.log(result);
// Expected output: Array [1, 2, 2, 1]
```

Sintaxe

```
var new_array = arr.flatMap(function callback(currentValue[, index[, array]]) {
    // retorna o elemento para new_array
}[, thisArg])
```

Parâmetros

`callback`

Função que produz um elemento de uma nova Array, pegando três argumentos:

`currentValue`

O valor atual sendo processo na array.

`index`Optional

O index do valor atual sendo processo na array.

`array`Optional

O `map` da array que foi chamado.

`thisArg`Optional

Valor para ser usado como `this` quando `callback` estiver sendo executado.

Valor de retorno

Uma nova array com cada elemento sendo o resultado da função callback e achatado ao valor de 1.

Descrição

Veja `Array.prototype.map()` para uma detalhada descrição da função callback. O método `flatMap` é idêntico ao `map` seguido por um chamado a `flatten` de profundidade 1.

Exemplos

<code>map</code> e <code>flatMap</code>

```
var arr1 = [1, 2, 3, 4];

arr1.map((x) => [x * 2]);
// [[2], [4], [6], [8]]

arr1.flatMap((x) => [x * 2]);
// [2, 4, 6, 8]

//  Só o primeiro nível
arr1.flatMap((x) => [[x * 2]]);
// [[2], [4], [6], [8]]
```

Enquanto que acima poderíamos alcançar apenas com a utilização de map, já aqui temos um exemplo onde `flatMap` é mais apropriado.

Vamos gerar uma lista de palavras a partir de uma lista de sentenças.

```
let arr1 = ["it's Sunny in", "", "California"];
```

```
arr1.map((x) => x.split(" "));
// [["it's","Sunny","in"],[""],["California"]]

arr1.flatMap((x) => x.split(" "));
// ["it's","Sunny","in", "", "California"]
```

Perceba, o comprimento da lista de saída pode ser diferente do comprimento da lista de entrada.

Alternativa

<code>reduce</code> e <code>concat</code>

```
var arr1 = [1, 2, 3, 4];
arr1.flatMap((x) => [x * 2]);
// é equivalente a
arr1.reduce((acc, x) => acc.concat([x * 2]), []);
// [ 2, 4, 6, 8 ]
```

Especificações

Specification	Status	Comment
`Array.prototype.flatMap` proposal	Rascunho	

Veja também

- `Array.prototype.flatten()`
- `Array.prototype.map()`
- `Array.prototype.reduce()`
- `Array.prototype.concat()`

Array.prototype.forEach()

O método `forEach()` executa uma dada função em cada elemento de um array.

Experimente

```
const array1 = ["a", "b", "c"];

array1.forEach((element) => console.log(element));

// Expected output: "a"
// Expected output: "b"
// Expected output: "c"
```

Sintaxe

```
arr.forEach(callback(currentValue [, index [, array]])[, thisArg]);
```

Parâmetros

`callback`

Função para executar em cada elemento, recebendo três argumentos:

`currentValue`

O valor atual do elemento sendo processado no array.

`index` Optional

O índice do elemento atual sendo processado no array.

`array` Optional

O array que `forEach()` está sendo aplicado.

`thisArg` Optional

Opcional. Valor a ser usado como `this` quando executar `callback`.

Valor retornado

`undefined`.

Descrição

O `forEach` executa o `callback` fornecido uma vez para cada elemento da ordem com um valor atribuido. Ele não é invocado para propriedades de índices que foram deletados ou que não foram inicializados (por ex. em arrays esparsos).

`callback` é invocado com **três argumentos**:

- o **valor do elemento**
- o **índice do elemento**

- o **array que está sendo percorrido**

Se um parâmetro `thisArg` for passado para `forEach()`, ele será passado para o `callback` quando invocado como valor para `this`. Caso contrário, o valor `undefined` será passado como valor para `this`. O valor de `this` assumido no `callback` é determinado de acordo com as regras usuais para determinação do `this` visto por uma função.

O intervalo dos elementos processados por `forEach()` é determinado antes da primeira invocação do `callback`. Elementos que forem adicionados ao array depois da chamada ao `forEach()` começar não serão visitados pelo `callback`. Se os valores dos elementos existentes do array forem alterados, o valor passado para o `callback` será o valor no momento em que o `forEach()` visitá-los; elementos que forem deletados antes de serem visitados não serão visitados.

`forEach()` executa a a função `callback` uma vez para cada elemento do array – diferentemente de `map()` ou `reduce()`, ele sempre retorna o valor `undefined` e não é encadeável. O caso de uso típico é alterar o array no final do loop.

> **Nota:** A única maneira de parar ou interromper um loop `forEach()` é disparando uma exceção. Se você precisa desse recurso, o método `forEach()` é a ferramenta errada. Você estará mais bem servido com um loop simples nesse caso. Se estiver testando o array de elementos para um predicado e precisar de um valor de retorno Boleano, você pode usar `every()` ou `some()`. Se estiverem disponíveis, os novos métodos `find()` e `findIndex()` também podem ser usados para terminação antecipada em predicados verdadeiros.

Exemplos

Imprimindo os conteúdos de uma ordem

Os códigos a seguir logam uma linha para cada elemento na ordem:

```
function logArrayElements(element, index, array) {
  console.log("a[" + index + "] = " + element);
}
[2, 5, 9].forEach(logArrayElements);
// logs:
// a[0] = 2
// a[1] = 5
// a[2] = 9
```

Função para cópia de um objeto

O código a seguir cria uma cópia para cada objeto dado. Há diferentes formas de criar uma cópia para um objeto. Esta é somente uma forma de explicar como `Array.prototype.forEach` funciona. Ela usa um grupo de novas funções ECMAScript 5 Object.*

```
function copy(o) {
  var copy = Object.create(Object.getPrototypeOf(o));
  var propNames = Object.getOwnPropertyNames(o);

  propNames.forEach(function (name) {
    var desc = Object.getOwnPropertyDescriptor(o, name);
    Object.defineProperty(copy, name, desc);
  });

  return copy;
}

var o1 = { a: 1, b: 2 };
var o2 = copy(o1); // o2 looks like o1 now
```

Compatibilidade

`forEach` é uma adição recente para o ECMA-262 standard; assim sendo, pode não estar presente em outras implementações do standard. Você pode contornar isto pela inserção do código a seguir no começo de seus scripts, permitindo o uso de `forEach` em implementações que normalmente não possuem este suporte.

```
if (!Array.prototype.forEach) {
  Array.prototype.forEach = function (fn, scope) {
    for (var i = 0, len = this.length; i < len; ++i) {
      fn.call(scope, this[i], i, this);
    }
  };
}
```

Um algorítimo 100% verdadeiro para a 5ª Edição do ECMA-262, pode ser visto abaixo:

O algoritmo é exatamente o especificado na 5ª Edição da ECMA-262, assumindo `Object` e `TypeError` possuem seus valores originais e avalia `callback.call` para o valor original de `Function.prototype.call`.

```
// Production steps of ECMA-262, Edition 5, 15.4.4.18
// Reference: http://es5.github.com/#x15.4.4.18
if (!Array.prototype.forEach) {
  Array.prototype.forEach = function forEach(callback, thisArg) {
    var T, k;

    if (this == null) {
      throw new TypeError("this is null or not defined");
    }

    // 1. Let O be the result of calling ToObject passing the |this| value as the argument.
```

```
    var O = Object(this);

    // 2. Let lenValue be the result of calling the Get internal method of O with the argument "length".
    // 3. Let len be ToUint32(lenValue).
    var len = O.length >>> 0; // Hack to convert O.length to a UInt32

    // 4. If IsCallable(callback) is false, throw a TypeError exception.
    // See: http://es5.github.com/#x9.11
    if ({}.toString.call(callback) !== "[object Function]") {
      throw new TypeError(callback + " is not a function");
    }

    // 5. If thisArg was supplied, let T be thisArg; else let T be undefined.
    if (thisArg) {
      T = thisArg;
    }

    // 6. Let k be 0
    k = 0;

    // 7. Repeat, while k < len
    while (k < len) {
      var kValue;

      // a. Let Pk be ToString(k).
      //   This is implicit for LHS operands of the in operator
      // b. Let kPresent be the result of calling the HasProperty internal method of O with argument Pk.
      //   This step can be combined with c
      // c. If kPresent is true, then
      if (Object.prototype.hasOwnProperty.call(O, k)) {
        // i. Let kValue be the result of calling the Get internal method of O with argument Pk.
        kValue = O[k];

        // ii. Call the Call internal method of callback with T as the this value and
        // argument list containing kValue, k, and O.
        callback.call(T, kValue, k, O);
      }
      // d. Increase k by 1.
      k++;
    }
    // 8. return undefined
  };
}
```

Veja também

- `Array.prototype.find()`
- `Array.prototype.findIndex()`
- `Array.prototype.map()`
- `Array.prototype.every()`
- `Array.prototype.some()`
- `Map.prototype.forEach()`
- `Set.prototype.forEach()`

Array.from()

Resumo

O método **`Array.from()`** cria uma nova instância `de um Array` quando for passado um array-like ou um iterable object como argumento.

No ES6, a sintaxe de classe permite a subclassificação de classes nativas e classes definidas pelo usuário; como resultado, os métodos estáticos pertencentes a classe, como **`Array.from`**, são "herdadas" por subclasses do **`Array`** e criam novas instâncias da subclasse, não do **`Array`**.

Sintaxe

```
Array.from(arrayLike[, mapFn[, thisArg]])
```

Parâmetros

`arrayLike`

Um array-like ou um objeto iterável para converter em array.

`mapFn`

Opcional. Função Map que será chamada para cada elemento do array.

`thisArg`

Opcional. Valor a ser utilizado como this quando a mapFn for chamada.

Descrição

`Array.from()` deixa você criar um Array de:

- objetos array-like (objetos com a propriedade `length` e elementos indexados); ou
- Objetos iteráveis (objetos onde você pode coletar seus elementos, assim como `Map` e `Set`).

`Array.from()` tem um parametro opcional `mapFn`, que permite executar a função `map` para cada elemento do array (ou subclasse de objeto) que está sendo criado. Simplificando, `Array.from(obj, mapFn, thisArg)` é o mesmo que `Array.from(obj).map(mapFn, thisArg)`, com a excessão de não cria um array intermediário . Isso é importante, principalmente para certas subclasses de array, como typed array, no qual o array intermediário iria necessáriamente ter o valor truncado para encaixar-se no tipo apropriado.

A propriedade `length` do método `from()` é `1`.

No ES2015, a sintaxe de class permite a definição de subclasses tanto internas quando definidas pelo usuário. Como resultado, métodos estáticos como `Array.from()` são "herdados" pelas subclasses de `Array`, e cria novas instâncias da subclasse, não de `Array`.

Polyfill

`Array.from` foi adicionado ao padrão ECMA-262 em sua 6ª edição; desta forma, não está presente na aplicações anteriores da especificação. Na ausência do código nativo, você pode inserir o código seguinte no início do script, permitindo `o uso parcial da funcionalidade do Array.from`. Esse algorítmo é equivalente ao especificado no ECMA-262, 6ª edição, exceto que `Object` e `TypeError` tem seus valores originais e que `callback.call` retorna o valor original de `Function.prototype.call`. Além disso, os verdadeiros iterables não podem ser representados genéricamente no polyfill, criando a principal distinção do que foi definido na especificação.

```
// Production steps of ECMA-262, Edition 6, 22.1.2.1
// Reference: https://people.mozilla.org/~jorendorff/es6-draft.html#sec-array.from
if (!Array.from) {
  Array.from = (function () {
    var toStr = Object.prototype.toString;
    var isCallable = function (fn) {
      return typeof fn === "function" || toStr.call(fn) === "[object Function]";
    };
    var toInteger = function (value) {
      var number = Number(value);
      if (isNaN(number)) {
        return 0;
      }
      if (number === 0 || !isFinite(number)) {
        return number;
      }
      return (number > 0 ? 1 : -1) * Math.floor(Math.abs(number));
    };
    var maxSafeInteger = Math.pow(2, 53) - 1;
    var toLength = function (value) {
      var len = toInteger(value);
      return Math.min(Math.max(len, 0), maxSafeInteger);
    };

    // The length property of the from method is 1.
    return function from(arrayLike /*, mapFn, thisArg */) {
      // 1. Let C be the this value.
      var C = this;

      // 2. Let items be ToObject(arrayLike).
      var items = Object(arrayLike);

      // 3. ReturnIfAbrupt(items).
      if (arrayLike == null) {
        throw new TypeError(
          "Array.from requires an array-like object - not null or undefined",
        );
      }

      // 4. If mapfn is undefined, then let mapping be false.
      var mapFn = arguments.length > 1 ? arguments[1] : void undefined;
      var T;
      if (typeof mapFn !== "undefined") {
        // 5. else
        // 5. a If IsCallable(mapfn) is false, throw a TypeError exception.
        if (!isCallable(mapFn)) {
          throw new TypeError(
            "Array.from: when provided, the second argument must be a function",
          );
        }

        // 5. b. If thisArg was supplied, let T be thisArg; else let T be undefined.
        if (arguments.length > 2) {
          T = arguments[2];
        }
      }

      // 10. Let lenValue be Get(items, "length").
      // 11. Let len be ToLength(lenValue).
      var len = toLength(items.length);

      // 13. If IsConstructor(C) is true, then
      // 13. a. Let A be the result of calling the [[Construct]] internal method of C with an argument list containing the single item len.
      // 14. a. Else, Let A be ArrayCreate(len).
```

```
      var A = isCallable(C) ? Object(new C(len)) : new Array(len);

      // 16. Let k be 0.
      var k = 0;
      // 17. Repeat, while k < len… (also steps a - h)
      var kValue;
      while (k < len) {
        kValue = items[k];
        if (mapFn) {
          A[k] =
            typeof T === "undefined"
              ? mapFn(kValue, k)
              : mapFn.call(T, kValue, k);
        } else {
          A[k] = kValue;
        }
        k += 1;
      }
      // 18. Let putStatus be Put(A, "length", len, true).
      A.length = len;
      // 20. Return A.
      return A;
    };
  })();
}
```

Exemplos

```
// Array-like object (arguments) para um Array
function f() {
  return Array.from(arguments);
}

f(1, 2, 3);
// [1, 2, 3]

// Qualquer iterable object ...
// com Set
var s = new Set(["foo", window]);
Array.from(s);
// ["foo", window]

// Map
var m = new Map([
  [1, 2],
  [2, 4],
  [4, 8],
]);
Array.from(m);
// [[1, 2], [2, 4], [4, 8]]

// String
Array.from("foo");
// ["f", "o", "o"]

// Usando um arrow function como função map para
// manipular os elementos
Array.from([1, 2, 3], (x) => x + x);
// [2, 4, 6]

// Gerando uma sequência de números
Array.from({ length: 5 }, (v, k) => k);
// [0, 1, 2, 3, 4]
```

Veja também

- `Array`
- `Array.prototype.map()`
- `TypedArray.from()`

Array.prototype.includes()

Sumário

O método `includes()` determina se um array contém um determinado elemento, retornando `true` ou `false` apropriadamente.

Experimente

```
const array1 = [1, 2, 3];

console.log(array1.includes(2));
// Expected output: true

const pets = ["cat", "dog", "bat"];
```

```
console.log(pets.includes("cat"));
// Expected output: true

console.log(pets.includes("at"));
// Expected output: false
```

Sintaxe

```
array.includes(searchElement[, fromIndex])
```

Parâmetros

`searchElement`

O elemento a buscar

`fromIndex`

Opcional. A posição no array de onde a busca pelo `searchElement` se iniciará. Por padrão, 0.

Exemplos

```
[1, 2, 3].includes(2); // true
[1, 2, 3].includes(4); // false
[1, 2, 3].includes(3, 3); // false
[1, 2, 3].includes(3, -1); // true
[1, 2, NaN].includes(NaN); // true
```

Polyfill

```
// https://tc39.github.io/ecma262/#sec-array.prototype.includes
if (!Array.prototype.includes) {
  Object.defineProperty(Array.prototype, "includes", {
    value: function (searchElement, fromIndex) {
      // 1. Let O be ? ToObject(this value).
      if (this == null) {
        throw new TypeError('"this" is null or not defined');
      }

      var o = Object(this);

      // 2. Let len be ? ToLength(? Get(O, "length")).
      var len = o.length >>> 0;

      // 3. If len is 0, return false.
      if (len === 0) {
        return false;
      }

      // 4. Let n be ? ToInteger(fromIndex).
      //    (If fromIndex is undefined, this step produces the value 0.)
      var n = fromIndex | 0;

      // 5. If n ≥ 0, then
      //  a. Let k be n.
      // 6. Else n < 0,
      //  a. Let k be len + n.
      //  b. If k < 0, let k be 0.
      var k = Math.max(n >= 0 ? n : len - Math.abs(n), 0);

      // 7. Repeat, while k < len
      while (k < len) {
        // a. Let elementK be the result of ? Get(O, ! ToString(k)).
        // b. If SameValueZero(searchElement, elementK) is true, return true.
        // c. Increase k by 1.
        // NOTE: === provides the correct "SameValueZero" comparison needed here.
        if (o[k] === searchElement) {
          return true;
        }
        k++;
      }

      // 8. Return false
      return false;
    },
  });
}
```

Especificações

Proposta ES7: https://github.com/domenic/Array.prototype.contains/blob/master/spec.md

Veja Também

- `TypedArray.prototype.includes()`
- `String.prototype.includes()`
- `Array.prototype.indexOf()`

Array.prototype.indexOf()

O método **`indexOf()`** retorna o primeiro índice em que o elemento pode ser encontrado no array, retorna -1 caso o mesmo não esteja presente.

Experimente

```
const beasts = ["ant", "bison", "camel", "duck", "bison"];

console.log(beasts.indexOf("bison"));
// Expected output: 1

// Start from index 2
console.log(beasts.indexOf("bison", 2));
// Expected output: 4

console.log(beasts.indexOf("giraffe"));
// Expected output: -1
```

Sintaxe

```
array.indexOf(elementoDePesquisa, [pontoInicial = 0])
```

Parâmetros

`elementoDePesquisa`

Elemento a ser pesquisado no array.

`pontoInicial`

O índice para iniciar a procura. Se o índice for maior ou igual ao tamanho do array, é retornado -1 e signfica que o item não será procurado. Se o `pontoInicial` é fornecido com um número negativo, é tomado como deslocamento da extremidade do array. Nota: se o `pontoInicial` fornecido é negativo, a procura no array acontece de frente para trás. Se o `pontoInicial` fornecido é 0, então o array inteiro será pesquisado. Padrão: 0 (pesquisa em todo array).

Descrição

`indexOf()` compara o `elementoDePesquisa` com os elementos do Array usando igualdade estrita (o mesmo método usado pelo ===, ou triple-equals operator).

Exemplos

Usando <code>indexOf()</code>

O exemplo seguinte usa indexOf() para localizar valores em um array

```
var array = [2, 5, 9];
array.indexOf(2); // 0
array.indexOf(7); // -1
array.indexOf(9, 2); // 2
array.indexOf(2, -1); // -1
array.indexOf(2, -3); // 0
```

Encontrar todas as ocorrências de um elemento

```
var indices = [];
var array = ["a", "b", "a", "c", "a", "d"];
var elemento = "a";
var idx = array.indexOf(elemento);
while (idx != -1) {
  indices.push(idx);
  idx = array.indexOf(elemento, idx + 1);
}
console.log(indices);
// [0, 2, 4]
```

Encontrar se um elemento existe ou não e atualizar o array

```
function atualizarColecaoVegetais(vegetais, vegetal) {
  if (vegetais.indexOf(vegetal) === -1) {
    vegetais.push(vegetal);
    console.log("Nova coleção de vegetais é : " + vegetais);
  } else if (vegetais.indexOf(vegetal) > -1) {
    console.log(vegetal + " já existe na coleção de vegetais.");
  }
}

var vegetais = ["batata", "tomate", "pimenta", "pimentao"];

atualizarColecaoVegetais(vegetais, "espinafre");
// Nova coleção de vegatais é : batata,tomate,pimenta,pimentao,espinafre
```

```
atualizarColecaoVegetais(vegetais, "espinafre");
// espinafre já existe na coleção de vegetais.
```

Polyfill

`indexOf()` foi adicionado ao ECMA-262 standard em sua 5 edição; como tal, não pode estar presente em todos navegadores.Você pode contornar isso utilizando o seguinte codigo no inicio de seus scripts. Isto permitirá que voce use o `indexOf()` quando não possuir suporte nativo. Esse algoritmo corresponde ao especificado no ECMA-262, edição 5, assumindo `TypeError` e `Math.abs()` tem seus valores originais.

```
// Passos para a produção do ECMA-262, Edition 5, 15.4.4.14
// Referência: http://es5.github.io/#x15.4.4.14
if (!Array.prototype.indexOf) {
  Array.prototype.indexOf = function (elementoDePesquisa, pontoInicial) {
    var k;

    //1. Deixe-o ser o resultado da chamada de toObject
    // passando o valor de this como argumento.
    if (this == null) {
      throw new TypeError(
        '"this" é nulo (null) ou não foi definido (undefined)',
      );
    }

    var O = Object(this);

    // 2. Deixar o tamanhoValor ser o resultado da
    // chamada do método interno Get de 0 com o
    // argumento "length"
    // 3. Deixar o  tamanhoValor ser um ToUint32(tamanhoValor).
    var tamanho = O.length >>> 0;

    // 4. se o tamanho é 0, retorna -1.
    if (tamanho === 0) {
      return -1;
    }

    // 5. Se o argumento pontoInicial for passado,
    // use o ToInteger(pontoInicial); senao use 0.
    var n = +pontoInicial || 0;

    if (Math.abs(n) === Infinity) {
      n = 0;
    }

    //6. Se n >= tamanho, retorna -1.
    if (n >= tamanho) {
      return -1;
    }

    // 7. Se n>= 0, entao k seja n.
    // 8. Senao, n<0, k seja tamanho - abs(n).
    // Se k é menor que 0, entao k seja 0.
    k = Math.max(n >= 0 ? n : tamanho - Math.abs(n), 0);

    // 9. Repita, enquanto k < tamanho
    while (k < tamanho) {
      // a. Deixe Pk ser ToString(k).
      //    isto é implicito para operandos LHS de um operador

      // b. Deixe o kPresent  ser o resultado da chamada do método
      //    interno de 0 com argumento Pk
      //    Este passo pode ser combinado com c.
      // c. Se kPresent é true, entao
      //    i.  Deixe o  elementK ser o resultado da chamada do metodo
      //        interno Get de 0 com argumento ToString(k)
      //   ii.  Deixe o resultado ser aplicado pelo Algoritmo de
      //        Comparação de Igualdade Estrita (Strict Equality Comparison)
      //        para o elementoDePesquisa e elementK
      //  iii.  caso verdadeiro, retorne k.
      if (k in O && O[k] === elementoDePesquisa) {
        return k;
      }
      k++;
    }
    return -1;
  };
}
```

Veja também

- `Array.prototype.lastIndexOf()`
- `TypedArray.prototype.indexOf()`

Array.isArray()

Sumário

O método **`Array.isArray()`** retorna `true` se um objeto é uma array, e `false` se não é.

Sintaxe

```
Array.isArray(obj)
```

Parâmetros

`obj`

O objeto a ser verificado.

Descrição

Se o objeto é um `Array`, retorna `true`(verdadeiro), caso contrário é retornado `false`(falso).

Veja o artigo "Determinando com absoluta precisão se um objeto em Javascript é ou não uma array" para mais detalhes.

Exemplos

```
// todas as chamadas conseguintes retornam true
Array.isArray([]);
Array.isArray([1]);
Array.isArray(new Array());
// Pequeno detalhe: Array.prototype por si só é uma array:
Array.isArray(Array.prototype);

// todas as conseguintes retornam false
Array.isArray();
Array.isArray({});
Array.isArray(null);
Array.isArray(undefined);
Array.isArray(17);
Array.isArray("Array");
Array.isArray(true);
Array.isArray(false);
Array.isArray({ __proto__: Array.prototype });
```

Polyfill

Executando o seguinte código antes de qualquer outro, o método `Array.isArray()` será criado, caso o browser não o disponibilize nativamente.

```
if (!Array.isArray) {
  Array.isArray = function (arg) {
    return Object.prototype.toString.call(arg) === "[object Array]";
  };
}
```

Veja Também

- `Array`

Array.prototype.join()

O método **`join()`** junta todos os elementos de um array (ou um array-like object) em uma string e retorna esta string.

Experimente

```
const elements = ["Fire", "Air", "Water"];

console.log(elements.join());
// Expected output: "Fire,Air,Water"

console.log(elements.join(""));
// Expected output: "FireAirWater"

console.log(elements.join("-"));
// Expected output: "Fire-Air-Water"
```

Sintaxe

```
arr.join([separador = ','])
```

Parâmetros

`separador` Optional

Especifica uma string para separar cada elemento adjacente do array. O separador é convertido em uma string se necessário. Se omitido, os elementos do array são separados com uma vírgula (","). Se o `separador` for uma string vazia, todos os elementos são juntados sem nenhum caracter entre eles.

Valor de retorno

Uma string com todos os elementos do array juntos. Se `arr.length` é `0`, uma string vazia é retornada.

Descrição

As conversões em string de todos os elementos de um array são juntados em apenas uma string.

> **Aviso:** Se um elemento é `undefined` ou `null`, ele é convertido em uma string vazia.

Exemplos

Juntando um array de quatro formas diferentes

O exemplo interativo a seguir cria um array, `a`, com três elementos, e o junta três vezes: a primeira com virgulas, a segunda so junta os elementos e a terceira com um sinal de menos.

Experimente

```
const elements = ["Fire", "Air", "Water"];

console.log(elements.join());
// Expected output: "Fire,Air,Water"

console.log(elements.join(""));
// Expected output: "FireAirWater"

console.log(elements.join("-"));
// Expected output: "Fire-Air-Water"
```

Juntando um array-like object (objeto estilo-array)

O exemplo abaixo junta um array-like object (ex: o objeto `arguments`), chamando `Function.prototype.call` no `Array.prototype.join`.

```
function f(a, b, c) {
  var s = Array.prototype.join.call(arguments);
  console.log(s); // '1,a,true'
}
f(1, "a", true);
//saida esperada: "1,a,true"
```

Veja Também

- `String.prototype.split()`
- `Array.prototype.toString()`
- `TypedArray.prototype.join()`

Array.prototype.keys()

Sumário

O método **`keys()`** retorna um novo **`Array Iterator`** que contém as chaves para cada *index* do array.

Sintaxe

```
arr.keys()
```

Exemplos

Exemplo: uso básico

```
var arr = ["a", "b", "c"];
var iterator = arr.keys();

console.log(iterator.next()); // { value: 0, done: false }
console.log(iterator.next()); // { value: 1, done: false }
console.log(iterator.next()); // { value: 2, done: false }
console.log(iterator.next()); // { value: undefined, done: true }
```

Exemplo: keys iterator não ignora lacunas

```
var arr = ["a", , "c"];
var sparseKeys = Object.keys(arr);
var denseKeys = [...arr.keys()];
console.log(sparseKeys); // [0, 2]
console.log(denseKeys); // [0, 1, 2]
```

Veja também

- `Array.prototype.values()`
- `Array.prototype.entries()`

Array.prototype.lastIndexOf()

O método **`lastIndexOf()`** retorna o ultimo índice que um certo elemento pode ser encontrado no array, ou -1 se o elemento não estiver presente. O array é pesquisado de trás para frente, começando pelo `fromIndex`.

Sintaxe

```
arr.lastIndexOf(searchElement[, fromIndex = arr.length - 1])
```

Parâmetros

`searchElement`

> Elemento para ser localizado no array.

`fromIndex`

> Opcional. O índice ao qual a busca será iniciada de traz para frente. O valor padrão é o tamanho total do array menos um (array.length -1), ou seja, todo o array será pesquisado. Se o índice for maior ou igual ao tamanho do array, o array inteiro será pesquisado. Se for negativo, ele é tomado como deslocamento no final do array. Note que mesmo se o índice for negativo, o array ainda será pesquisado de traz para frente. Se o índice calculado for menor que 0, -1 será retornado, ou seja, o array não será pesquisado.

Descrição

`lastIndexOf` compara `searchElement` a elementos do Array usando igualdade rigorosa (o mesmo método usado pelo operador ===, ou "igual triplo").

Exemplos

Exemplo: Usando <code>lastIndexOf</code>

O seguinte exemplo utiliza `lastIndexOf` para localizar elementos em um array.

```
var array = [2, 5, 9, 2];
array.lastIndexOf(2); // 3
array.lastIndexOf(7); // -1
array.lastIndexOf(2, 3); // 3
array.lastIndexOf(2, 2); // 0
array.lastIndexOf(2, -2); // 0
array.lastIndexOf(2, -1); // 3
```

Exemplo: Encontrando todas as ocorrências de um elemento

O seguinte exemplo utiliza `lastIndexOf` para encontrar todos os índices de um elemento em um dado array, utilizando `push` para adicioná-los em outro array quando são encontrados.

```
var indices = [];
var array = ["a", "b", "a", "c", "a", "d"];
var element = "a";
var idx = array.lastIndexOf(element);
while (idx != -1) {
  indices.push(idx);
  idx = idx > 0 ? array.lastIndexOf(element, idx - 1) : -1;
}

console.log(indices);
// [4, 2, 0]
```

Note que devemos tratar o caso `idx == 0` separadamente aqui pois o elemento será sempre encontrado independente do parâmetro `fromIndex` se ele for o primeiro elemento do array. Isso é diferente do método `indexOf`.

Polyfill

`lastIndexOf` foi adicionado ao padrão ECMA-262 na 5ª edição; devido a isso, não deve estar presente em outras implementações do padrão. Você pode contornar isso inserindo o seguinte código no início dos seus scripts, permitindo o uso de `lastIndexOf` em implementações que não o suportam nativamente. Esse algorítimo é exatamente o mesmo especificado no padrão ECMA-262, 5ª edição, assumindo que `Object`, `TypeError`, `Number`, `Math.floor`, `Math.abs`, e `Math.min` possuem seus valores originais.

```
// Production steps of ECMA-262, Edition 5, 15.4.4.15
// Reference: http://es5.github.io/#x15.4.4.15
if (!Array.prototype.lastIndexOf) {
  Array.prototype.lastIndexOf = function (searchElement /*, fromIndex*/) {
    "use strict";

    if (this === void 0 || this === null) {
      throw new TypeError();
    }

    var n,
      k,
      t = Object(this),
      len = t.length >>> 0;
    if (len === 0) {
      return -1;
```

```
    }

    n = len - 1;
    if (arguments.length > 1) {
      n = Number(arguments[1]);
      if (n != n) {
        n = 0;
      } else if (n != 0 && n != 1 / 0 && n != -(1 / 0)) {
        n = (n > 0 || -1) * Math.floor(Math.abs(n));
      }
    }

    for (k = n >= 0 ? Math.min(n, len - 1) : len - Math.abs(n); k >= 0; k--) {
      if (k in t && t[k] === searchElement) {
        return k;
      }
    }
    return -1;
  };
}
```

Novamente, perceba que essa implementação foca na absoluta compatibilidade com `lastIndexOf` no Firefox e no motor JavaScript SpiderMonkey, incluíndo vários casos que são, indiscutivelmente, extremos. Se você pretende usar isso em aplicações reais, é possível calcular `from` com um código menos complicado se você ignorar esses casos.

Veja também

- `Array.prototype.indexOf()`
- `TypedArray.prototype.lastIndexOf()`

Array.prototype.length

A propriedade **`length`** representa um inteiro de 32-bit sem sinal, que especifica o número de elementos em um array.

Property attributes of `Array.prototype.length`	
Writable	yes
Enumerable	no
Configurable	no

Sintaxe

```
arr.length
```

Descrição

O valor da propriedade length é um inteiro com um sinal positivo e valor menor que 2 elevado na 32ª potência(232).

Você pode setar o a propriedade `length` para truncar um array a qualquer momento. Quando você extende um array mudando sua propriedade `length`, o numero de elementos atuais não é incrementado; por exemplo, se você setar o tamanho para 3 quando ele é atualmente 2, o array continua somente com 2 elementos. Assim, a propriedade `length` não diz nada sobre o tamanho de valores definidos no array. Veja também Relacionamento entre `length` e propriedades numéricas.

Exemplos

Iterando sobre um array

No exemplo a seguir `numbers` é iterado considerando a propriedade `length` para ver quantos elementos ele tem. O valor de cada elemento recebe então o dobro.

```
var numbers = [1, 2, 3, 4, 5];

for (var i = 0; i < numbers.length; i++) {
  numbers[i] *= 2;
}
// numbers is now [2, 4, 6, 8, 10]
```

Encurtando um array

O exemplo a seguir encurta o array `statesUS` para um tamanho de 50 se o tamanho corrente for maior do que 50.

```
if (statesUS.length > 50) {
  statesUS.length = 50;
}
```

Veja também

- `Array`

Array.prototype.map()

Experimente

```
const array1 = [1, 4, 9, 16];

// Pass a function to map
const map1 = array1.map((x) => x * 2);

console.log(map1);
// Expected output: Array [2, 8, 18, 32]
```

Resumo

O método **map()** invoca a função `callback` passada por argumento para cada elemento do Array e devolve um novo Array como resultado.

Sintaxe

```
arr.map(callback[, thisArg])
```

Parâmetros

`callback`

Função cujo retorno produz o elemento do novo Array. Recebe três argumentos:

`valorAtual`

O valor do elemento original do Array de origem.

`indice`

O índice do elemento atual que está sendo processado no array.

`array`

O Array de origem.

`thisArg`

Opcional. Valor a ser utilizado como o *`this`* no momento da execução da função `callback`.

Descrição

O método **`map`** chama a função `callback` recebida por parâmetro para cada elemento do Array original, em ordem, e constrói um novo array com base nos retornos de cada chamada. A função `callback` é chamada apenas para os elementos do array original que tiverem valores atribuídos; os elementos que estiverem como `undefined`, que tiverem sido removidos ou os que nunca tiveram valores atribuídos não serão considerados.

A função `callback` é chamada com três argumentos: o valor do elemento corrente, o índice do elemento corrente e o array original que está sendo percorrido.

Se o parametro `thisArg` foi passado para o método `map`, ele será repassado para a função `callback` no momento da invocação para ser utilizado como o `this`. Caso contrário, o valor `undefined` será repassado para uso como o *`this`*. O valor do `this` a ser repassado para o `callback` deve respeitar as regras para determinar como o this é acessado por uma função (em inglês).

O método `map` não modifica o array original. No entanto, a função `callback` invocada por ele pode fazê-lo.

A lista de elementos que serão processados pelo `map` é montada antes da primeira invocação à função `callback`. Se um elemento for acrescentado ao array original após a chamada ao `map`, ele não será visível para o `callback`. Se os elementos existentes forem modificados, os valores que serão repassados serão os do momento em que o método `map` invocar o `callback`. Elementos removidos não serão visitados.

Exemplos

Exemplo: Mapeando um array de números para um array de raízes quadradas

O código a seguir mapeia um array de números e cria um novo array contendo o valor da raiz quadrada de cada número do primeiro array.

```
var numbers = [1, 4, 9];
var roots = numbers.map(Math.sqrt);
// roots é [1, 2, 3], numbers ainda é [1, 4, 9]
```

Exemplo: Mapeando um array de números usando uma função <code>callback</code> que contém um argumento

O código a seguir mostrar como o método `map` funciona quando a função `callback` possui apenas um argumento. Esse argumento será automaticamente atribuído para cada elemento do array conforme o `map` itera sobre o array original.

```
var numbers = [1, 4, 9];
var doubles = numbers.map(function (num) {
  return num * 2;
});
// doubles é agora [2, 8, 18]. numbers ainda é [1, 4, 9]
```

Exemplo: usando <code>map</code> genericamente

Esse exemplo demonstra como usar o map em um `String` para recuperar a representação em ASCII de cada caracter em um array de bytes:

```
var map = Array.prototype.map;
var a = map.call("Hello World", function (x) {
  return x.charCodeAt(0);
});
// a agora vale [72, 101, 108, 108, 111, 32, 87, 111, 114, 108, 100]
```

Exemplo: usando <code>map</code> genericamente com <code>querySelectorAll</code>

Esse exemplo demonstra como iterar sobre uma coleção de objetos recuperada através de querySelectorAll. Nesse caso, vamos pegar todos os options selecionados na tela e imprimir no console:

```
var elems = document.querySelectorAll("select option:checked");
var values = [].map.call(elems, function (obj) {
  return obj.value;
});
```

Exemplo: Usando <code>map</code> para inverter uma string

```
var str = "12345";
[].map
  .call(str, function (x) {
    return x;
  })
  .reverse()
  .join("");

// Output: '54321'
// Bonus: utilize '===' para verificar se a string original e a nova string são palíndromos
```

Exemplo: Caso de uso mais complexo

(inspirado nesse post) (em inglês)

É uma prática comum utilizar o callback com apenas um argumento (o elemento atual do array original). Algumas funções também são comumente utilizadas com um argumento, mesmo tendo argumentos adicionais opcionais. Esses hábitos podem resultar em comportamentos indesejado:

```
// Considere:
["1", "2", "3"].map(parseInt);
// Enquanto era de se esperar [1, 2, 3]
// O resultado atual é [1, NaN, NaN]

// parseInt é normalmente usado com apenas um argumento, mas ele possui dois.
// O primeiro é uma expressão, e o segundo o radical.
// Para a função callback, o Array.prototype.map repassa 3 argumentos:
// o elemento corrente, o indice e o array original
// O terceiro argumento é ignorado pelo parseInt, mas o segundo não, o que certamente gerou o comportamento inesperado. Veja o post para maiores detalhes

function returnInt(element) {
  return parseInt(element, 10);
}

["1", "2", "3"].map(returnInt); // [1, 2, 3]
// O resultado atual é um array de números (como esperado)

// Bonus: Um jeito mais simples de conseguir o mesmo resultado sem nenhuma "pegadinha do Malandro":
["1", "2", "3"].map(Number); // [1, 2, 3]
```

Polyfill

(veja definição em Polyfill)

O método `map` foi introduzido ao padrão ECMA-262 na sua 5ª edição, o que significa que ele pode não estar presente em todas as implementações desse padrão. Você pode contornar esse problema inserindo o código a seguir no início dos seus scripts, permitindo o uso do `map` mesmo que ele não esteja sendo suportado nativamente. Esse algoritmo é exatamente o especificado no ECMA-262 5ª edição, assumindo que `Object`, `TypeError`, e `Array` tenham seus valores originais, e que `callback.call` avalie para o valor original de `Function.prototype.call`.

```
// Production steps of ECMA-262, Edition 5, 15.4.4.19
// Reference: http://es5.github.io/#x15.4.4.19
if (!Array.prototype.map) {
  Array.prototype.map = function (callback, thisArg) {
    var T, A, k;

    if (this == null) {
      throw new TypeError(" this is null or not defined");
    }

    //  1. Let O be the result of calling ToObject passing the |this|
    //    value as the argument.
    var O = Object(this);

    // 2. Let lenValue be the result of calling the Get internal
    //    method of O with the argument "length".
    // 3. Let len be ToUint32(lenValue).
```

```
    var len = O.length >>> 0;

    // 4. If IsCallable(callback) is false, throw a TypeError exception.
    // See: http://es5.github.com/#x9.11
    if (typeof callback !== "function") {
      throw new TypeError(callback + " is not a function");
    }

    // 5. If thisArg was supplied, let T be thisArg; else let T be undefined.
    if (arguments.length > 1) {
      T = thisArg;
    }

    // 6. Let A be a new array created as if by the expression new Array(len)
    //    where Array is the standard built-in constructor with that name and
    //    len is the value of len.
    A = new Array(len);

    // 7. Let k be 0
    k = 0;

    // 8. Repeat, while k < len
    while (k < len) {
      var kValue, mappedValue;

      // a. Let Pk be ToString(k).
      //   This is implicit for LHS operands of the in operator
      // b. Let kPresent be the result of calling the HasProperty internal
      //    method of O with argument Pk.
      //   This step can be combined with c
      // c. If kPresent is true, then
      if (k in O) {
        // i. Let kValue be the result of calling the Get internal
        //    method of O with argument Pk.
        kValue = O[k];

        // ii. Let mappedValue be the result of calling the Call internal
        //     method of callback with T as the this value and argument
        //     list containing kValue, k, and O.
        mappedValue = callback.call(T, kValue, k, O);

        // iii. Call the DefineOwnProperty internal method of A with arguments
        // Pk, Property Descriptor
        // { Value: mappedValue,
        //   Writable: true,
        //   Enumerable: true,
        //   Configurable: true },
        // and false.

        // In browsers that support Object.defineProperty, use the following:
        // Object.defineProperty(A, k, {
        //   value: mappedValue,
        //   writable: true,
        //   enumerable: true,
        //   configurable: true
        // });

        // For best browser support, use the following:
        A[k] = mappedValue;
      }
      // d. Increase k by 1.
      k++;
    }

    // 9. return A
    return A;
  };
}
```

Veja também

- `Array.prototype.forEach()`
- `Map` object
- `Array.from()`

Array.of()

O método **`Array.of()`** cria um nova instância de `Array` com um número variável de argumentos, independentemente do número ou do tipo dos argumentos.

A diferença entre o **`Array.of()`** e o construtor de **`Array`** é no tratamento dos argumentos inteiros: **`Array.of(7)`** cria um array com um único elemento, `7`, enquanto **`Array(7)`** cria um array vazio de propriedade `length` igual a `7` (**Nota**: isso quer dizer um array com `7` espaços vazios, e não com valores do tipo `undefined`).

```
Array.of(7);       // [7]
Array.of(1, 2, 3); // [1, 2, 3]
```

```
Array(7);          // array com 7 espaços vazios
Array(1, 2, 3);    // [1, 2, 3]
```

Syntaxe

```
Array.of(element0[, element1[, ...[, elementN]]])
```

Parâmetros

`elementN`

Elementos usados para criar o `array`.

Valor de retorno

Uma nova instância de `Array`.

Descrição

Esta função é parte do padrão ECMAScript 6 (ou ECMAScript 2015).

Para maiores informações veja:

- `Array.of`
- `Array.from` proposal
- `Array.of` polyfill.

Exemplos

```
Array.of(1); // [1]
Array.of(1, 2, 3); // [1, 2, 3]
Array.of(undefined); // [undefined]
```

Polyfill

Executando o seguinte código antes de qualquer outro c[odigo criará o `Array.of()` se ele não for disponível nativamente.

```
if (!Array.of) {
  Array.of = function () {
    return Array.prototype.slice.call(arguments);
    // Or
    let vals = [];
    for (let prop in arguments) {
      vals.push(arguments[prop]);
    }
    return vals;
  };
}
```

Ver também

- `Array`
- `Array.from()`
- `TypedArray.of()`

Array.prototype.pop()

Resumo

O método **`pop()`** remove o **último** elemento de um array e retorna aquele elemento.

Sintaxe

```
arr.pop()
```

Descrição

O método `pop` remove o último elemento de um array e retorna aquele valor.

Esse método é intencionalmente genérico. Podendo ser utilizado por call ou apply em objetos que implementam arrays.

Se você chamar `pop()` em um array vazio, ele retorna o valor `undefined`.

Exemplos

Exemplo: Removendo o último elemento de um array

O código seguinte cria o array `meuPeixe` contendo quatro elementos e então remove seu último elemento.

```
var meuPeixe = ["acara-bandeira", "palhaco", "mandarim", "esturjao"];

console.log(meuPeixe); // ['acara-bandeira', 'palhaco', 'mandarim', 'esturjao']
```

```
var meuPeixePop = meuPeixe.pop();

console.log(meuPeixe); // ['acara-bandeira', 'palhaco', 'mandarim' ]

console.log(meuPeixePop); // 'esturjao'
```

Veja também

- `Array.prototype.push()`
- `Array.prototype.shift()`
- `Array.prototype.unshift()`

Array.prototype.push()

Resumo

O método **push()** adiciona um ou mais elementos ao final de um array e retorna o novo comprimento desse array.

```
var numeros = [1, 2, 3];
numeros.push(4);

console.log(numeros); // [1, 2, 3, 4]

numeros.push(5, 6, 7);

console.log(numeros); // [1, 2, 3, 4, 5, 6, 7]
```

Sintaxe

```
arr.push(elemento1, ..., elementoN)
```

Parâmetros

`elementoN`

Os elementos a serem incluídos ao final do array.

Retorno

O novo valor da propriedade `length` do objeto no qual o método foi chamado.

Descrição

O método `push` adiciona valores a um array.

Esse método é intencionalmente genérico. Podendo ser utilizado por `call()` ou `apply()` em objetos que implementam arrays. O método push depende da propriedade length para determinar onde começar a inserir os valores. Caso a propriedade `length` não possa ser convertida em número, é utilizado 0 como índice. Isto inclui a possibilidade de `length` não existir, nesse caso, essa propriedade será criada.

Os únicos objetos que implementam nativamente array são as strings, porém elas não são adequadas para a aplicação desse método, pois são imutáveis.

Exemplos

Exemplo: Adicionando elementos a um array

O seguinte código cria um array `esportes` que contém dois elementos. Então adiciona dois elementos a ele. A variável `total` contém o novo comprimento do array.

```
var esportes = ["futebol", "beisebol"];
var total = esportes.push("handebol", "natacao");

console.log(esportes); // ['futebol, 'beisebol', 'handebol', 'natacao']
console.log(total); // 4
```

Exemplo: Fusão de dois arrays

Este exemplo utiliza `apply()` para adicionar todos os elementos de um segundo array.

```
var vegetais = ["cenoura", "batata"];
var maisVegetais = ["aipo", "beterraba"];

// Adiciona o segundo array no primeiro
// Equivalente a vegetais.push('aipo', 'beterraba');
Array.prototype.push.apply(vegetais, maisVegetais);

console.log(vegetais); // ['cenoura', 'batata', 'aipo', 'beterraba']
```

Exemplo: Utilizando um object como um array-like

Como mencionado acima, `push` é intencionalmente genérico, e podemos usar isso para nossa vantagem. `Array.prototype.push` pode trabalhar em um objeto muito bem, como mostra este exemplo. Observe que não criamos um array para armazenar uma coleção de objetos. Em vez disso, armazenamos a coleção no objeto em si e usamos a chamada em `Array.prototype.push` para enganar o método e pensar que estamos lidando com um array, e ele simplesmente funciona, graças à forma como o JavaScript nos permite estabelecer o contexto de execução quando queremos.

```
var obj = {
  length: 0,

  addElem: function addElem(elem) {
    // obj.length é automaticamente incrementado
    // toda vez que um elemento for adicionado.
    [].push.call(this, elem);
  },
};

// Vamos adicionar alguns objetos vazios apenas para ilustrar.
obj.addElem({});
obj.addElem({});
console.log(obj.length);
// → 2
```

Observe que, embora `obj` não seja um array, o método `push` aumentou com sucesso a propriedade de comprimento (`length`) do `obj` como se estivéssemos lidando com um array.

Veja também

- `Array.prototype.pop()`
- `Array.prototype.shift()`
- `Array.prototype.unshift()`
- `Array.prototype.concat()`

Array.prototype.reduce()

O método **`reduce()`** executa uma função **reducer** (fornecida por você) para cada elemento do array, resultando num único valor de retorno.

Experimente

```
const array1 = [1, 2, 3, 4];

// 0 + 1 + 2 + 3 + 4
const initialValue = 0;
const sumWithInitial = array1.reduce(
  (accumulator, currentValue) => accumulator + currentValue,
  initialValue,
);

console.log(sumWithInitial);
// Expected output: 10
```

A função **reducer** recebe quatro parâmetros:

1. Acumulador (`acc`)
2. Valor Atual (`cur`)
3. Index Atual (`idx`)
4. Array original (`src`)

O valor de retorno da sua função **reducer** é atribuída ao acumulador. O acumulador, com seu valor atualizado, é repassado para cada iteração subsequente pelo array, que por fim, se tornará o valor resultante, único, final.

Sintaxe

```
array.reduce(callback( acumulador, valorAtual[, index[, array]] )[, valorInicial])
```

Parâmetros

`callback`

Função que é executada em cada valor no array (exceto no primeiro, se nenhum `valorInicial` for passado); recebe quatro argumentos:

`acumulador`

É o valor inicial (ou o valor do callback anterior). Este valor inicia com o `valorInicial` e será retornado na última iteração.

`valorAtual`

Opcional. O índice do elemento atual que está sendo processado no array. Começa a partir do index `0` se um `valorInicial` for fornecido. Do contrário, começa do index `1`.

`valorInicial`

Opcional. Valor a ser usado como o primeiro argumento da primeira chamada da função `callback`. Se nenhum `valorInicial` é fornecido, o primeiro elemento do array será usado como o valor inicial do `acumulador` e o `valorAtual` não será lido. Chamar `reduce()` em uma array vazia sem valor inicial retornará um erro.

Valor retornado

O valor que resulta da execução da função de callback do "reducer" até a conclusão de todo o array.

Descrição

O método `reduce()` executa a função de `callback` uma vez para cada elemento presente no array, excluindo furos (valores indefinidos), recebendo quatro argumentos:

1. *acumulador* - valor inicial (ou o valor do callback anterior),
2. *valorAtual* - o valor do elemento atual
3. *index* - o índice atual e
4. `array` - o array onde a iteração está ocorrendo.

A primeira vez que o callback é chamado, o `acumulador` e o `valorAtual` podem ter um de dois valores possíveis. Se o `valorInicial` tiver sido fornecido na chamada à função `reduce()`, então o `acumulador` será igual ao `valorInicial` e o `valorAtual` será igual ao primeiro valor no array. Caso nenhum `valorInicial` seja fornecido, `acumulador` será igual ao primeiro valor no array, e `valorAtual` será igual ao segundo.

> **Nota:** Se o `valorInicial` não tiver sido passado como argumento, então reduce() executará o callback da função começando a partir do índice 1 (index 1), pulando o primeiro índice (`index 0`). Se o `valorInicial` for passado como argumento, a função irá começar no index 0.

Se a array estiver vazia e o `valorInicial` não tiver sido informado, uma exceção do tipo `TypeError` será lançada.

Se a array possuir somente um elemento (independente da posição) e o `valorInicial` não tiver sido fornecido, ou se `valorInicial` for fornecido, mas a array estiver vazia, o valor será retornado sem que a função de `callback` seja chamada.

É mais seguro prover um `valorInicial`, porque existem até *quatro* possíveis saídas sem o `valorInicial`, como mostrado no exemplo:

```
var maxCallback = (acc, cur) => Math.max(acc.x, cur.x);
var maxCallback2 = (max, cur) => Math.max(max, cur);

// reduce() sem valores iniciais
[{ x: 22 }, { x: 42 }].reduce(maxCallback); // 42
[{ x: 22 }].reduce(maxCallback); // { x: 22 }
[].reduce(maxCallback); // TypeError

// map/reduce; melhor solução, funciona para vetores vazios e tambem para vetores grandes
[{ x: 22 }, { x: 42 }].map((el) => el.x).reduce(maxCallback2, -Infinity);
```

Como funciona o reduce()

Suponha que o seguinte uso de reduce() tenha ocorrido:

```
[0, 1, 2, 3, 4].reduce(function (acumulador, valorAtual, index, array) {
  return acumulador + valorAtual;
});
// 10
```

O callback será invocado quatro vezes, com os argumentos e valores em cada chamada sendo:

	`acumulador`	valorAtual	index	`array`	valor de retorno
primeira chamada	`0`	`1`	`1`	`[0, 1, 2, 3, 4]`	`1`
segunda chamada	`1`	`2`	`2`	`[0, 1, 2, 3, 4]`	`3`
terceira chamada	`3`	`3`	`3`	`[0, 1, 2, 3, 4]`	`6`
quarta chamada	`6`	`4`	`4`	`[0, 1, 2, 3, 4]`	`10`

O valor retornado pelo `reduce` será o da última chamada à callback (`10`).

Você também pode usar uma Arrow Function em vez de uma função completa. O código abaixo produz a mesma saída que o código do bloco acima:

```
[0, 1, 2, 3, 4].reduce((accum, curr) => accum + curr);
```

Se você informar um `valorInicial` como o segundo argumento de reduce, o resultado será:

```
[0, 1, 2, 3, 4].reduce(function (acumulador, valorAtual, indice, array) {
  return acumulador + valorAtual;
}, 10);

// 20
```

callback	`acumulador`	`valorAtual`	index	`array`	`valor de retorno`
primeira chamada	`10`	`0`	`0`	`[0, 1, 2, 3, 4]`	`10`
segunda chamada	`10`	`1`	`1`	`[0, 1, 2, 3, 4]`	`11`
terceira chamada	`11`	`2`	`2`	`[0, 1, 2, 3, 4]`	`13`
quarta chamada	`13`	`3`	`3`	`[0, 1, 2, 3, 4]`	`16`
quinta chamada	`16`	`4`	`4`	`[0, 1, 2, 3, 4]`	`20`

O retorno da última chamada `20`,é retornado como resultado da função `reduce()`.

Exemplos

Soma todos os valores de uma array

```
let total = [0, 1, 2, 3].reduce(function (acumulador, valorAtual) {
  return acumulador + valorAtual;
}, 0);
// retorna 6
```

outra alternativa é usar uma arrow function:

```
var total = [0, 1, 2, 3].reduce(
  (acumulador, valorAtual) => acumulador + valorAtual,
  0,
);
```

Soma de valores de um objeto de um array

Para resumir os valores contidos em um array, você **deve** fornecer um `valorInicial`, para que cada item passe por sua função.

```
var valorInicial = 0;
var soma = [{ x: 1 }, { x: 2 }, { x: 3 }].reduce(function (
  acumulador,
  valorAtual,
) {
  return acumulador + valorAtual.x;
}, valorInicial);

console.log(soma); // retorna 6
```

Utilizando uma arrow function:

```
var valorInicial = 0;
var soma = [{ x: 1 }, { x: 2 }, { x: 3 }].reduce(
  (acumulador, valorAtual) => acumulador + valorAtual.x,
  valorInicial,
);

console.log(soma); // retorna 6
```

Redução de um array de arrays

```
let reduzido = [
  [0, 1],
  [2, 3],
  [4, 5],
].reduce(function (acumulador, valorAtual) {
  return acumulador.concat(valorAtual);
}, []);
// reduzido é [0, 1, 2, 3, 4, 5]
```

Utilizando uma arrow function:

```
let reduzido = [
  [0, 1],
  [2, 3],
  [4, 5],
].reduce((acumulador, valorAtual) => acumulador.concat(valorAtual), []);
```

Contando valores iguais em um objeto

```
let nomes = ["Alice", "Bob", "Tiff", "Bruce", "Alice"];

let quantidadeNomes = nomes.reduce(function (todosNomes, nome) {
  if (nome in todosNomes) {
    todosNomes[nome]++;
  } else {
    todosNomes[nome] = 1;
  }
  return todosNomes;
}, {});
// quantidade de nomes:
// { 'Alice': 2, 'Bob': 1, 'Tiff': 1, 'Bruce': 1 }
```

Agrupando objetos por uma propriedade

```
let pessoas = [
  { nome: "Alice", idade: 21 },
  { nome: "Max", idade: 20 },
  { nome: "Jane", idade: 20 },
];

function agruparPor(objetoArray, propriedade) {
  return objetoArray.reduce(function (acc, obj) {
    let key = obj[propriedade];
    if (!acc[key]) {
```

```
      acc[key] = [];
    }
    acc[key].push(obj);
    return acc;
  }, {});
}

let grupodePessoas = agruparPor(pessoas, "idade");
// grupodePessoas é:
// {
//   20: [
//     { nome: 'Max', idade: 20 },
//     { nome: 'Jane', idade: 20 }
//   ],
//   21: [{ nome: 'Alice', idade: 21 }]
// }
```

Juntando arrays contidos num array de objetos usando o operador spread e o <code>valorInicial</code>

```
// friends - um array de objetos
// onde o campo "books" é a lista de livros favoritos
var friends = [
  {
    name: "Anna",
    books: ["Bible", "Harry Potter"],
    age: 21,
  },
  {
    name: "Bob",
    books: ["War and peace", "Romeo and Juliet"],
    age: 26,
  },
  {
    name: "Alice",
    books: ["The Lord of the Rings", "The Shining"],
    age: 18,
  },
];

// allbooks - lista que contém todos os livros de friends +
// lista adicional contida em valorInicial
var allbooks = friends.reduce(
  function (prev, curr) {
    return [...prev, ...curr.books];
  },
  ["Alphabet"],
);

// allbooks = [
//   'Alphabet', 'Bible', 'Harry Potter', 'War and peace',
//   'Romeo and Juliet', 'The Lord of the Rings',
//   'The Shining'
// ]
```

Removendo itens duplicados num array

> **Nota:** Se você estiver usando um ambiente compatível com `Set` and `Array.from()`, você pode usar `let orderedArray = Array.from(new Set(myArray))` para obter um array em que os itens duplicados tenham sido removidos.

```
let arr = [1, 2, 1, 2, 3, 5, 4, 5, 3, 4, 4, 4, 4];
let result = arr.sort().reduce((init, current) => {
  if (init.length === 0 || init[init.length - 1] !== current) {
    init.push(current);
  }
  return init;
}, []);
console.log(result); //[1,2,3,4,5]
```

Substituindo .filter().map() por .reduce()

Usar `Array.filter()` seguido por `Array.map()` faz com que o array seja percorrido duas vezes. Você pode obter o mesmo resultado percorrendo o array apenas uma vez com `Array.reduce()`, o que é, portanto, mais eficiente. (Se você gosta de for loops, você pode usar filter e map percorrendo o array apenas uma vez com `Array.forEach()`).

```
const numbers = [-5, 6, 2, 0];

const doubledPositiveNumbers = numbers.reduce((accumulator, currentValue) => {
  if (currentValue > 0) {
    const doubled = currentValue * 2;
    accumulator.push(doubled);
  }
  return accumulator;
}, []);
```

```
console.log(doubledPositiveNumbers); // [12, 4]
```

Rodando promises em sequência

```
/**
 * Roda promises de um promise array de uma maneira encadeada
 *
 * @param {array} arr - promise arr
 * @return {Object} promise object
 */
function runPromiseInSequense(arr) {
  return arr.reduce((promiseChain, currentPromise) => {
    return promiseChain.then((chainedResult) => {
      return currentPromise(chainedResult).then((res) => res);
    });
  }, Promise.resolve());
}

// promise function 1
function p1() {
  return new Promise((resolve, reject) => {
    resolve(5);
  });
}

// promise function 2
function p2(a) {
  return new Promise((resolve, reject) => {
    resolve(a * 2);
  });
}

// promise function 3
function p3(a) {
  return new Promise((resolve, reject) => {
    resolve(a * 3);
  });
}

const promiseArr = [p1, p2, p3];
runPromiseInSequense(promiseArr).then((res) => {
  console.log(res); // 30
});
```

Escrever map usando reduce

```
if (!Array.prototype.mapUsingReduce) {
  Array.prototype.mapUsingReduce = function (callback, thisArg) {
    return this.reduce(function (mappedArray, currentValue, index, array) {
      mappedArray[index] = callback.call(thisArg, currentValue, index, array);
      return mappedArray;
    }, []);
  };
}

[1, 2, , 3].mapUsingReduce(
  (currentValue, index, array) => currentValue + index + array.length,
); // [5, 7, , 10]
```

Polyfill

`Array.prototype.reduce` foi adicionado ao padrão ECMA-262 na quinta edição; e portanto, pode não estar presente em todas as implementações do padrão. Você pode contornar isso inserindo o código a seguir no início de seus scripts, permitindo o uso do `reduce()` em implementações que não possuem suporte nativo a ele.

```
// Etapas de produção para o ECMA-262, Edition 5, 15.4.4.21
// Referencia: http://es5.github.io/#x15.4.4.21
if (!Array.prototype.reduce) {
  Array.prototype.reduce = function (callback /*, valorInicial*/) {
    "use strict";
    if (this == null) {
      throw new TypeError(
        "Array.prototype.reduce chamado é nulo (null) ou indefinido (undefined)",
      );
    }
    if (typeof callback !== "function") {
      throw new TypeError(callback + " não é uma função");
    }
    var t = Object(this),
      len = t.length >>> 0,
      k = 0,
      value;
    if (arguments.length == 2) {
      value = arguments[1];
    } else {
```

```
      while (k < len && !(k in t)) {
        k++;
      }
      if (k >= len) {
        throw new TypeError(
          "Reduce possui um array vazio sem um valor inicial",
        );
      }
      value = t[k++];
    }
    for (; k < len; k++) {
      if (k in t) {
        value = callback(value, t[k], k, t);
      }
    }
    return value;
  };
}
```

Leia também

- `Array.prototype.reduceRight()`

Array.prototype.reduceRight()

Sumário

O método **`reduceRight()`** aplica à uma função um acumulador e cada valor do array (da direita para esquerda) é reduzido para um valor único.

Sintaxe

```
arr.reduceRight(callback[, initialValue])
```

Parâmetros

`callback`

Função para executar em cada valor do array, recebendo quatro argumentos:

`previousValue`

O valor anteriormente retornado na ultima invocação do callback, ou o `initialValue`, se este for o recebido. (Ver abaixo.)

`currentValue`

O valor atualmente sendo processado no array.

`index`

O índice do valor atualmente sendo processado no array.

`array`

O array que foi chamado para ser reduzido.

`initialValue`

Opcional. Objeto para ser usado como argumento inicial da primeria chamada do callback.

Descrição

`reduceRight` executa a função callback uma vez para cada elemento presente no array, excluindo buracos no array, recebendo quatro argumentos: o valor inicial (ou o valor da chamada anterior do callback), o valor do elemento atual, o índice do elemento atual, e o array onde a operação está acontecendo.

A chamada ao callback reduceRight irá parecer com uma chamada assim:

```
array.reduceRight(function (previousValue, currentValue, index, array) {
  // ...
});
```

A primeira vez que a função é chamada, o `previousValue` e o `currentValue` podem ser um de dois valores. Se um `initialValue` foi recebido na chamada do `reduceRight`, então o `previousValue` sera iqual ao `initialValue` e o `currentValue` será igual ao ultimo valor no array. Se o `initialValue` não foi recebido, então o `previousValue` será igual ao ultimo valor no array e o `currentValue` será igual ao penultimo valor no array.

Se o array é vazio e nenhum `initialValue` foi recebido, `TypeError` será lançado. Se o array somente tem um elemento (independentemente da posição dele) e o `initialValue` não foi recebido, ou se o `initialValue` foi recebido mas o array é vazio, o valor em si será retornado sem chamar o `callback`.

Alguns exemplos de execuções da função e como será parecida a chamada:

```
[0, 1, 2, 3, 4].reduceRight(
  function (previousValue, currentValue, index, array) {
    return previousValue + currentValue;
  },
);
```

O callback será invocado quatro vezes, com os argumentos e valores de retornos em cada chamada será como o seguinte:

	previousValue	currentValue	index	array	return value
Primeira chamada	4	3	3	[0, 1, 2, 3, 4]	7
Segunda chamada	7	2	2	[0, 1, 2, 3, 4]	9
Terceira chamada	9	1	1	[0, 1, 2, 3, 4]	10
Quarta chamada	10	0	0	[0, 1, 2, 3, 4]	10

O valor retornado pelo `reduceRight` será o valor retornado pela ultima chamada ao callback(`10`).

E se você também passou um `initialValue`, o resultado irá ser como a seguir:

```
[0, 1, 2, 3, 4].reduceRight(function (
  previousValue,
  currentValue,
  index,
  array,
) {
  return previousValue + currentValue;
}, 10);
```

	previousValue	currentValue	index	array	return value
Primeira chamada	10	4	4	[0, 1, 2, 3, 4]	14
Segunda chamada	14	3	3	[0, 1, 2, 3, 4]	17
Terceira chamada	17	2	2	[0, 1, 2, 3, 4]	19
Quarta chamada	19	1	1	[0, 1, 2, 3, 4]	20
Quinta chamada	20	0	0	[0, 1, 2, 3, 4]	20

O valor retornado pelo `reduceRight` desta vez será, obviamente, `20`.

Exemplos

Exemplo: Somando todos os valores presente em um array

```
var total = [0, 1, 2, 3].reduceRight(function (a, b) {
  return a + b;
});
// total == 6
```

Exemplo: Juntando um array de arrays

```
var flattened = [
  [0, 1],
  [2, 3],
  [4, 5],
].reduceRight(function (a, b) {
  return a.concat(b);
}, []);
// flattened is [4, 5, 2, 3, 0, 1]
```

Polyfill

`reduceRight` foi adicionado no padrão ECMA-262 em sua Quinta edição; sendo assim pode não estar presente em todas as implementações deste padrão. Você pode contornar isso adicionando o seguinte codigo ao inicio do seu script, adicionando a possibilidade de uso do `reduceRight` em implementações que não o suportam nativamente.

```
// Production steps of ECMA-262, Edition 5, 15.4.4.22
// Reference: http://es5.github.io/#x15.4.4.22
if ("function" !== typeof Array.prototype.reduceRight) {
  Array.prototype.reduceRight = function (callback /*, initialValue*/) {
    "use strict";
    if (null === this || "undefined" === typeof this) {
      throw new TypeError("Array.prototype.reduce called on null or undefined");
    }
    if ("function" !== typeof callback) {
      throw new TypeError(callback + " is not a function");
    }
    var t = Object(this),
      len = t.length >>> 0,
      k = len - 1,
      value;
    if (arguments.length >= 2) {
      value = arguments[1];
    } else {
      while (k >= 0 && !(k in t)) {
        k--;
      }
      if (k < 0) {
```

```
        throw new TypeError("Reduce of empty array with no initial value");
      }
      value = t[k--];
    }
    for (; k >= 0; k--) {
      if (k in t) {
        value = callback(value, t[k], k, t);
      }
    }
    return value;
  };
}
```

Ver também

- `Array.prototype.reduce()`

Array.prototype.reverse()

Resumo

O método **`reverse()`** inverte os itens de um array. O primeiro elemento do array se torna o último e o último torna-se o primeiro.

Sintaxe

```
arr.reverse()
```

Parâmetros

Nenhum.

Descrição

O método `reverse` transpõe os elementos do objeto array no mesmo lugar, mutando o array, e retornando uma referência para o mesmo.

Exemplos

Exemplo: Invertendo os elementos em um array

O seguinte exemplo cria um array `myArray`, contendo três elementos, em seguida inverte-o.

```
var myArray = ["one", "two", "three"];
myArray.reverse();

console.log(myArray); // ['three', 'two', 'one']
```

Veja também

- `Array.prototype.join()`
- `Array.prototype.sort()`
- `TypedArray.prototype.reverse()`

Array.prototype.shift()

Sumário

O método **`shift()`** remove o **primeiro** elemento de um array e retorna esse elemento. Este método muda o tamanho do array.

Experimente

```
const array1 = [1, 2, 3];

const firstElement = array1.shift();

console.log(array1);
// Expected output: Array [2, 3]

console.log(firstElement);
// Expected output: 1
```

Sintaxe

```
arr.shift()
```

Valor de retorno

O elemento removido do array; `undefined` se o array estiver vazio.

Descrição

O método shift remove o elemento de índice zero, diminui em 1 os indices dos demais valores e retorna o valor removido. Se a propriedade `length` for 0, então `undefined` é retornado.

`shift` é intencionalmente genérico; esse método pode ser chamado ou aplicado para objetos parecidos com arrays. Objetos que não contém a propriedade `length` representando o tamanho de uma série consecutiva, começando em zero, podem não se comportar de maneira correta.

Exemplos

Removendo um elemento de array

O código a seguir mostra o array `minhaLista` antes e depois de remover seu primeiro elemento. Ele também mostra o elemento removido.

```
var minhaLista = ["anjo", "casa", "mandarim", "medico"];

console.log("minhaLista antes: " + minhaLista);
// minhaList antes: ['anjo', 'casa', 'mandarim', 'medico']
var shifted = minhaLista.shift();

console.log("minhaLista depois: " + minhaLista);
// minhaList depois: ['casa', 'mandarim', 'medico']
console.log("Elemento removido: " + shifted);
// Elemento removido: anjo
```

Usando o método shift() dentro de um loop de while

O médodo `shift()` é frequentemente usado como condição dentro de um loop de while. No exemplo a seguir, cada iteração removerá o elemento seguinte do array, até que ele esteja vazio:

```
var nomes = ["André", "Eduardo", "Paulo", "Cris", "João"];
while ((i = nomes.shift()) !== undefined) {
  console.log(i);
}
// André Eduardo Paulo Cris João
```

Veja também

- `Array.prototype.push()`
- `Array.prototype.pop()`
- `Array.prototype.unshift()`
- `Array.prototype.concat()`

Array.prototype.slice()

O método **`slice()`** retorna uma cópia de parte de um array a partir de um subarray criado entre as posições `início` e `fim` (fim não é incluído) de um array original. O Array original não é modificado.

Experimente

```
const animals = ["ant", "bison", "camel", "duck", "elephant"];

console.log(animals.slice(2));
// Expected output: Array ["camel", "duck", "elephant"]

console.log(animals.slice(2, 4));
// Expected output: Array ["camel", "duck"]

console.log(animals.slice(1, 5));
// Expected output: Array ["bison", "camel", "duck", "elephant"]

console.log(animals.slice(-2));
// Expected output: Array ["duck", "elephant"]

console.log(animals.slice(2, -1));
// Expected output: Array ["camel", "duck"]

console.log(animals.slice());
// Expected output: Array ["ant", "bison", "camel", "duck", "elephant"]
```

Syntaxe

```
arr.slice([início[,fim]])
```

Parâmetros

`início` Optional

Índice baseado em zero no qual se inicia a extração.

Como um índice negativo, `início` indica um deslocamento em relação ao fim da sequência. `slice(-2)` extrai os dois últimos elementos do array.

Se `início` for omitido, `slice` inicia a partir do índice `0`.

Se `início` for maior que o comprimento do array, é retornado um array vazio.

`fim` Optional

Índice baseado em zero o qual é o final da extração. `slice` extrai até, não incluindo, `fim`.

`slice(1,4)` extrai do segundo até o quarto elemento (elementos de índice 1, 2 e 3).

Como índice negativo, `fim` indica um deslocamento em relação ao fim do array. `slice(2,-1)` extrai o terceiro elemento através do segundo-para-o-último elemento no array.

Se `fim` for omitido ou for maior que o tamanho do array, `slice` considerará o último elemento do array como sendo o `fim` (`arr.length`).

Valor de retorno

Um novo array contendo os elementos extraídos.

Descrição

`slice` não altera o array original. Retorna uma cópia de elementos do array original. Elementos do array original são copiados para o array retornado da seguinte maneira:

- Para referências de objeto (e não o objeto real), `slice` copia referencias de objeto em um novo array. Ambos, o original e o novo array referem-se ao mesmo objeto. Se um objeto referenciado é alterado, as alterações são visiveis em ambos, no novo array e no array original.
- Para strings e números (não objetos `String` e `Number`), `slice` copia strings e números em um novo array. Alterações na string ou número em um array não afetam o outro array.

Se um novo elemento é adicionado a qualquer array, o outro não é afetado.

Exemplos

Retorna uma parte de um array existente

```
// Exemplo para extrair 'Laranja' e 'Limao' do array frutas
var frutas = ["Banana", "Laranja", "Limao", "Maçã", "Manga"];
var citricos = frutas.slice(1, 3);

// citricos contem ['Laranja','Limao']
```

Usando <code>slice</code>

No exemplo seguinte, `slice` cria um novo array, `novoCarro`, do original `meuCarro`. Ambos incluem uma referência ao objeto, `meuHonda`. Quando a cor de `meuHonda` é alterada para roxo, ambos os arrays sofrem alteração.

```
// Usando slice para criar novoCarro a partir de meuCarro.
var meuHonda = {
  cor: "vermelho",
  rodas: 4,
  motor: { cilindros: 4, tamanho: 2.2 },
};
var meuCarro = [meuHonda, 2, "perfeitas condições", "comprado em 1997"];
var novoCarro = meuCarro.slice(0, 2);

// Exibe os valores de meuCarro, novoCarro, e a cor de meuHonda
//  referenciado de ambos arrays.
console.log("meuCarro = " + meuCarro.toSource());
console.log("novoCarro = " + novoCarro.toSource());
console.log("meuCarro[0].cor = " + meuCarro[0].cor);
console.log("novoCarro[0].cor = " + novoCarro[0].cor);

// Altera a cor de meuHonda.
meuHonda.cor = "roxo";
console.log("A nova cor do meu Honda é " + meuHonda.cor);

// Exibe a cor de meuHonda referenciado de ambos arrays.
console.log("meuCarro[0].cor = " + meuCarro[0].cor);
console.log("novoCarro[0].cor = " + novoCarro[0].cor);
```

Esse script escreve:

```
meuCarro = [{cor:'vermelho', rodas:4, motor:{cilindros:4, tamanho:2.2}}, 2,'perfeitas condições', 'comprado em 1997']
novoCarro = [{cor:'vermelho', rodas:4, motor:{cilindros:4, tamanho:2.2}},2]
meuCarro[0].cor = vermelho
novoCarro[0].cor = vermelho
A nova cor do meu Honda é roxo
meuCarro[0].cor = roxo
novoCarro[0].cor = roxo
```

Objetos Array-like

O método `slice` pode também ser chamado para converter objetos ou coleções Array-like em um novo Array. Você só precisa encadear o método no Array. Os `arguments` dentro de uma função são um exemplo de 'objeto array-like'.

```
function list() {
  return Array.prototype.slice.call(arguments);
}
```

```
var list1 = list(1, 2, 3); // [1, 2, 3]
```

Ligações podem ser feitas com a função `.call` de `Function` and it can also be reduced using `[].slice.call(arguments)` ao invés de `Array.prototype.slice.call`. De qualquer forma, ela pode ser simplificada com `bind`.

```
var unboundSlice = Array.prototype.slice;
var slice = Function.prototype.call.bind(unboundSlice);

function list() {
  return slice(arguments);
}

var list1 = list(1, 2, 3); // [1, 2, 3]
```

Simplificando o comportamento entre navegadores

Embora os objetos de host (como objetos DOM) não sejam obrigados pela especificação a seguir o comportamento do Mozilla quando convertidos por Array.prototype.slice e IE <9 não o fazem, versões do IE começando pela versão 9 permitem isso. "Shimming" pode permitir um comportamento confiável entre navegadores. Enquanto outros navegadores modernos continuem suportando essa habilidade, como atualmente IE, Mozilla, Chrome, Safari, e Opera fazem, desenvolvedores lendo (DOM-supporting) o código slice confiando neste shim não serão desencaminhados pela semântica; eles podem confiar seguramente na semântica para fornecer o agora aparentemente comportamento padrão de fato. (Isso também corrige o problema com IE < 9 onde o segundo argumento do `slice` era explicitamente `null`/`undefined`)

```
(function () {
  "use strict";
  var _slice = Array.prototype.slice;

  try {
    // Produzirá erro no IE < 9
    _slice.call(document.documentElement);
  } catch (e) {
    // Funciona para arrays, objetos array-like,
    // NamedNodeMap (atributos, entidades, notações),
    // NodeList (por exemplo, getElementsByTagName), HTMLCollection (por exemplo, childNodes),
    // e não vai falhar em outros objetos do DOM (como falham no IE < 9)
    Array.prototype.slice = function (begin, end) {
      end = typeof end !== "undefined" ? end : this.length;

      // Para arrays, chamamos o método nativo
      if (Object.prototype.toString.call(this) === "[object Array]") {
        return _slice.call(this, begin, end);
      }

      // Para array-like, o processo é manual.
      var i,
        cloned = [],
        size,
        len = this.length;

      // Lidando com valor negativo para "begin"
      var start = begin || 0;
      start = start >= 0 ? start : Math.max(0, len + start);

      // Lidando com valor negativo para "end"
      var upTo = typeof end == "number" ? Math.min(end, len) : len;
      if (end < 0) {
        upTo = len + end;
      }

      // Tamanho real do corte feito pelo slice
      size = upTo - start;

      if (size > 0) {
        cloned = new Array(size);
        if (this.charAt) {
          for (i = 0; i < size; i++) {
            cloned[i] = this.charAt(start + i);
          }
        } else {
          for (i = 0; i < size; i++) {
            cloned[i] = this[start + i];
          }
        }
      }

      return cloned;
    };
  }
})();
```

Veja também

- `Function.prototype.call()`

- `Function.prototype.bind()`
- `Array.prototype.splice()`

Array.prototype.some()

O método **`some()`** testa se ao menos um dos elementos no array passa no teste implementado pela função atribuída e retorna um valor **`true`** ou **`false`**.

Experimente

```
const array = [1, 2, 3, 4, 5];

// Checks whether an element is even
const even = (element) => element % 2 === 0;

console.log(array.some(even));
// Expected output: true
```

Sintaxe

```
arr.some(callback[, thisArg])
```

Parâmetros

`callback`

Função para testar cada elemento, recebendo três argumentos:

`currentValue`

O valor atual do elemento sendo processado no array.

`index`

O índice do elemento atual sendo processado no array.

`array`

O array onde o método `some()` foi chamado.

`thisArg`

Opcional. Valor para usar como *`this`* durante a execução do `callback`.

Valor de retorno

Esta função retorna **`true`** se a função callback retornar **`true`** para qualquer elemento do array; caso contrário, **`false`**.

Descrição

`some()` executa a função callback uma vez para cada elemento presente no array até achar um onde o `callback` retorne um valor *true*. Se em qualquer dos elementos o valor for encontrado, `some()` imediatamente retorna `true`. Caso contrario, `some()` retorna `false`. `callback` é invocado somente para índices do array que contenham valor definido; não é invocado para índices que foram deletados ou os quais nunca tiveram valor definido.

`callback` é invocado com três argumentos: o valor do elemento, o índice do elemento, e o array onde a função foi chamada.

Se o parâmetro `thisArg` foi passado ao `some()`, ele sera passado ao `callback` quando o mesmo for invocado, para ser usado como o valor de `this` internamente na função callback. Caso contrario, o valor `undefined` será passado para uso como `this`. O valor `this` observado pela `callback` é determinado de acordo com as regras usuais para determinar o que é visto por uma função.

`some()` não altera o array dentro do qual ele é chamado.

O intervalo de elementos processado por `some()` é definido antes da primeira invocação da `callback`. Elementos contidos no array antes da chamada `some()` ser iniciada não serão testados pela *callback*. Se algum elemento pertencente ao array for alterado pela `callback`, o valor passado para a `callback` será o valor do momento em que a função `some()` encontra o índice daquele elemento. Elementos deletados não são testados.

Exemplos

Testando valores de elementos de um array

O exemplo a seguir testa se algum elemento de um array é maior que 10.

```
function isBiggerThan10(element, index, array) {
  return element > 10;
}
[2, 5, 8, 1, 4].some(isBiggerThan10); // false
[12, 5, 8, 1, 4].some(isBiggerThan10); // true
```

Testando valores de elementos de um array usando arrow functions

Arrow functions fornece uma sintaxe mais curta para o mesmo teste.

```
[2, 5, 8, 1, 4].some((elem) => elem > 10); // false
[12, 5, 8, 1, 4].some((elem) => elem > 10); // true
```

Polyfill

`some()` was added to the ECMA-262 standard in the 5th edition; as such it may not be present in all implementations of the standard. You can work around this by inserting the following code at the beginning of your scripts, allowing use of `some()` in implementations which do not natively support it. This algorithm is exactly the one specified in ECMA-262, 5th edition, assuming `Object` and `TypeError` have their original values and that `fun.call` evaluates to the original value of `Function.prototype.call()`.

```
// Production steps of ECMA-262, Edition 5, 15.4.4.17
// Reference: http://es5.github.io/#x15.4.4.17
if (!Array.prototype.some) {
  Array.prototype.some = function (fun /*, thisArg*/) {
    "use strict";

    if (this == null) {
      throw new TypeError("Array.prototype.some called on null or undefined");
    }

    if (typeof fun !== "function") {
      throw new TypeError();
    }

    var t = Object(this);
    var len = t.length >>> 0;

    var thisArg = arguments.length >= 2 ? arguments[1] : void 0;
    for (var i = 0; i < len; i++) {
      if (i in t && fun.call(thisArg, t[i], i, t)) {
        return true;
      }
    }

    return false;
  };
}
```

Veja também

- `Array.prototype.forEach()`
- `Array.prototype.every()`
- `TypedArray.prototype.some()`

Array.prototype.sort()

O método **`sort()`** ordena os elementos do próprio array e retorna o array. A ordenação não é necessariamente estável. A ordenação padrão é de acordo com a pontuação de código unicode.

A complexidade do tempo de execução ou a quantidade de memória utilizada pela ordenação não pode ser garantido e depende da implementação realizada.

Sintaxe

```
arr.sort(funcaoDeComparacao)
```

Parâmetros

`funcaoDeComparacao` Optional

> Especifica uma função que define a ordenação. Se omitido, o array é ordenado de acordo com a pontuação de código Unicode de cada um dos caracteres, de acordo com a conversão de cada elemento para string.

`primeiroElemento`

> O primeiro elemento para a comparação.

`segundoElemento`

> O segundo elemento para comparação.

Valor de Retorno

O array ordenado. Note que o array é ordenado de acordo com a pontuação de código Unicode de cada um dos caracteres, e nenhuma cópia é feita.

Descrição

Se **`funcaoDeComparacao`** não for informado, os elementos serão ordenados de acordo com a sua conversão para texto e o texto comparado na pontuação unicode do texto convertido. Por exemplo, "banana" vem antes de "cherry". Em uma ordenação numérica, 9 vem antes de 80, mas porque os números são convertidos para texto e, "80" vem antes de "9" na ordenação Unicode.

```
var fruit = ["cherries", "apples", "bananas"];
fruit.sort(); // ['apples', 'bananas', 'cherries']

var scores = [1, 10, 2, 21];
scores.sort(); // [1, 10, 2, 21]
// Observe que 10 vem antes do 2,
```

```
// porque '10' vem antes de '2' em ponto de código Unicode.

var things = ["word", "Word", "1 Word", "2 Words"];
things.sort(); // ['1 Word', '2 Words', 'Word', 'word']
// Em Unicode, números vêem antes de letras maiúsculas,
// as quais vêem antes das minúsculas.
```

Se o parâmetro `funcaoDeComparacao` for fornecido, o array será ordenado de acordo com o valor de retorno da `funcaoDeComparacao`. Considerando que `a` e `b` são dois elementos sendo comparados, então:

- Se `funcaoDeComparacao(a, b)` for menor que 0, ordena `a` para um índice anterior a `b`, i.e. a vem primeiro.
- Se `funcaoDeComparacao(a, b)` retornar 0, deixa `a` e `b` inalterados em relação um ao outro, mas ordenado em relação a todos os outros elementos. Nota: O padrão ECMAscript não garante este comportamento, e, portanto, nem todos os navegadores (e.g. Versões do Mozilla anteriores a 2003) respeitarão isto.
- Se `funcaoDeComparacao(a, b)` é maior que 0, ordena b para um índice anterior que a.
- `funcaoDeComparacao(a, b)` sempre deve retornar o mesmo valor dado um par específico de elementos a e b como seus dois parametros. Se resultados inconsistentes são retornados, então a ordenação é indefinida.

Então, a função de comparação tem a seguinte forma:

```
function comparar(a, b) {
  if (a é menor que b em algum critério de ordenação) {
    return -1;
  }
  if (a é maior que b em algum critério de ordenação) {
    return 1;
  }
  // a deve ser igual a b
  return 0;
}
```

Para comparar números ao invés de texto, a função de comparação pode simplesmente subtrair b de a. A função abaixo irá ordenar o array em ordem crescente:

```
function compararNumeros(a, b) {
  return a - b;
}
```

O método de ordenação pode convenientemente ser usada com funções anônimas (e closures):

```
var numbers = [4, 2, 5, 1, 3];
numbers.sort(function (a, b) {
  return a - b;
});
console.log(numbers);
```

Objetos podem ser ordenados de acordo com o valor de uma de suas propriedades.

```
var items = [
  { name: "Edward", value: 21 },
  { name: "Sharpe", value: 37 },
  { name: "And", value: 45 },
  { name: "The", value: -12 },
  { name: "Magnetic" },
  { name: "Zeros", value: 37 },
];
items.sort(function (a, b) {
  if (a.name > b.name) {
    return 1;
  }
  if (a.name < b.name) {
    return -1;
  }
  // a must be equal to b
  return 0;
});
```

Exemplos

Criando, exibindo, e ordenando um array

O exemplo abaixo cria quatro arrays e mostra seu conteúdo original, então o conteúdo dos arrays ordenado. Os arrays numéricos são ordenados sem a função de comparação, e então, com a função.

```
var stringArray = ["Blue", "Humpback", "Beluga"];
var numericStringArray = ["80", "9", "700"];
var numberArray = [40, 1, 5, 200];
var mixedNumericArray = ["80", "9", "700", 40, 1, 5, 200];

function compararNumeros(a, b) {
  return a - b;
}

console.log("stringArray:", stringArray.join());
```

```
console.log("Ordenada:", stringArray.sort());

console.log("numberArray:", numberArray.join());
console.log("Ordenada sem função de comparação:", numberArray.sort());
console.log("Ordenada com compararNumeros:", numberArray.sort(compararNumeros));

console.log("numericStringArray:", numericStringArray.join());
console.log("Ordenada sem função de comparação:", numericStringArray.sort());
console.log(
  "Ordenada com compararNumeros:",
  numericStringArray.sort(compararNumeros),
);

console.log("mixedNumericArray:", mixedNumericArray.join());
console.log("Ordenada sem função de comparação:", mixedNumericArray.sort());
console.log(
  "Ordenada com compararNumeros:",
  mixedNumericArray.sort(compararNumeros),
);
```

Este exemplo gera a saída abaixo. Como as saídas mostram, quando a função de comparação é usada, os números são ordenados corretamente, sejam eles números ou strings numéricas.

```
stringArray: Blue,Humpback,Beluga
Ordenada: Beluga,Blue,Humpback

numberArray: 40,1,5,200
Ordenada sem função de comparação: 1,200,40,5
Ordenada com compararNumeros: 1,5,40,200

numericStringArray: 80,9,700
Ordenada sem função de comparação: 700,80,9
Ordenada com compararNumeros: 9,80,700

mixedNumericArray: 80,9,700,40,1,5,200
Ordenada sem função de comparação: 1,200,40,5,700,80,9
Ordenada com compararNumeros: 1,5,9,40,80,200,700
```

Ordenando caracteres não-ASCII

Para ordenar strings com caracteres não-ASCII, i.e. strings com caracteres acentuados (e, é, è, a, ä, etc.), strings de línguas diferentes do Inglês: use `String.localeCompare`. Esta função pode comparar estes caracteres, então eles aparecerão na ordem correta.

```
var items = ["réservé", "premier", "cliché", "communiqué", "café", "adieu"];
items.sort(function (a, b) {
  return a.localeCompare(b);
});

// items é ['adieu', 'café', 'cliché', 'communiqué', 'premier', 'réservé']
```

Ordenando com mapa

A `funcaoDeComparacao` pode ser invocada múltiplas vezes por elemento do array. Dependendo da natureza da `funcaoDeComparacao`, isto pode causar um excesso processamento. Quanto mais trabalho a `funcaoDeComparacao` fizer, e quanto mais elementos houverem para ordenar, seria mais inteligente considerar o uso de um mapa para a ordenação. A idéia é percorrer o array uma vez para extrair os valores já processados para a ordenação e armazenar em um array temporário, ordenar o array temporário e então percorrer o array temporário para conseguir a ordenação correta.

```
// o array a ser ordenado
var list = ["Delta", "alpha", "CHARLIE", "bravo"];

// array temporário que armazena os objetos com o índice e o valor para ordenação
var mapped = list.map(function (el, i) {
  return { index: i, value: el.toLowerCase() };
});

// ordenando o array mapeado contendo os dados resumidos
mapped.sort(function (a, b) {
  return +(a.value > b.value) || +(a.value === b.value) - 1;
});

// container para o resultado ordenado
var result = mapped.map(function (el) {
  return list[el.index];
});
```

Veja também

- `Array.prototype.reverse()`
- `String.prototype.localeCompare()`

Array.prototype.splice()

Resumo

O método **splice()** altera o conteúdo de uma lista, adicionando novos elementos enquanto remove elementos antigos.

Experimente

```
const months = ["Jan", "March", "April", "June"];
months.splice(1, 0, "Feb");
// Inserts at index 1
console.log(months);
// Expected output: Array ["Jan", "Feb", "March", "April", "June"]

months.splice(4, 1, "May");
// Replaces 1 element at index 4
console.log(months);
// Expected output: Array ["Jan", "Feb", "March", "April", "May"]
```

Sintaxe

```
array.splice(indice[, deleteCount[, elemento1[, ...[, elementoN]]]])

array.splice(indice) // SpiderMonkey/Firefox extension
```

Parâmetros

`indice`

Índice o qual deve iniciar a alterar a lista. Se maior que o tamanho total da mesma, nenhum elemento será alterado. Se negativo, irá iniciar a partir daquele número de elementos a partir do fim.

`deleteCount`

Um inteiro indicando o número de antigos elementos que devem ser removidos.

Se o parâmetro `deleteCount` não é especificado, ou se é maior que o número de elementos restantes na lista iniciando pelo índice, então todos os elementos até o fim da lista serão deletados.

Se `deleteCount` é 0, nenhum elemento é removido. Neste caso você deve especificar pelo menos um novo elemento.

`elemento1, ..., elementoN`

Os elementos a adicionar na lista. Se você não especificar nenhum elemento, splice simplesmente removerá elementos da mesma.

Retorno

Uma lista contendo os elementos removidos. Se apenas um elemento é removido, por exemplo, uma lista contendo apenas um elemento é retornada. Se nenhum elemento é removido, uma lista vazia é retornada.

Descrição

Se você especificar um número diferente de elementos a inserir comparado ao número de elementos que você está removendo, a lista terá um tamanho diferente no final da execução.

Exemplo

Exemplo: Removendo 0 elementos do indice 2, e inserindo "drum"

O script a seguir ilustra o uso do *splice*:

```
var myFish = ["angel", "clown", "mandarin", "surgeon"];

//remove 0 elementos a partir do índice 2, e insere "drum"
var removed = myFish.splice(2, 0, "drum");
//myFish é ["angel", "clown", "drum", "mandarin", "surgeon"]
//removed é [], nenhum elemento removido

//remove 1 elemento do índice 3
removed = myFish.splice(3, 1);
//myFish é ["angel", "clown", "drum", "surgeon"]
//removed é ["mandarim"]

//remove 1 elemento a partir do índice 2, e insere "trumpet"
removed = myFish.splice(2, 1, "trumpet");
//myFish é ["angel", "clown", "trumpet", "surgeon"]
//removed é ["drum"]

//remove 2 elementos a partir do índice 0, e insere "parrot", "anemone" e "blue"
removed = myFish.splice(0, 2, "parrot", "anemone", "blue");
//myFish é ["parrot", "anemone", "blue", "trumpet", "surgeon"]
//removed é ["angel", "clown"]

//remove 2 elementos a partir do indice 3
removed = myFish.splice(3, Number.MAX_VALUE);
//myFish é ["parrot", "anemone", "blue"]
//removed é ["trumpet", "surgeon"]
```

Veja Também

- `push` / `pop` - adiciona/remove elementos do final do array;
- `unshift` / `shift` - adiciona/remove elementos do início do array;
- `concat` - returns a new array comprised of this array joined with other array(s) and/or value(s).

Array.prototype[@@iterator]()

O valor inicial da propriedade **`@@iterator`** é o mesmo objeto função que o valor inicial da propriedade `values()`.

Sintaxe

```
arr[Symbol.iterator]()
```

Exemplos

Iteração usando laço <code>for...of</code>

```
var arr = ["w", "y", "k", "o", "p"];
// seu navegador deve suportar laço for..of
// e variáveis de escopo let em laços for
for (let letter of arr) {
  console.log(letter);
}
```

Iteração alternativa

```
var arr = ["w", "y", "k", "o", "p"];
var eArr = arr[Symbol.iterator]();
console.log(eArr.next().value); // w
console.log(eArr.next().value); // y
console.log(eArr.next().value); // k
console.log(eArr.next().value); // o
console.log(eArr.next().value); // p
```

Veja também

- `Array.prototype.keys()`
- `Array.prototype.entries()`
- `Array.prototype.forEach()`
- `Array.prototype.every()`
- `Array.prototype.some()`
- `Array.prototype.values()`

Array.prototype[@@unscopables]

A propriedade de símbolo **`@@unscopable`** contém nomes de propriedades que não foram incluídos no padrão ECMAScript antes da versão ES2015. Essas propriedades são excluídas do statement bindings `with`.

Sintaxe

```
arr[Symbol.unscopables]
```

Descrição

As propriedades padrão de array que são excluídas dos bindings `with` são: copyWithin, entries, fill, find, findIndex, includes, keys, e values.

Veja `Symbol.unscopables` para saber como definir `unscopables` para seus objetos.

Property attributes of `Array.prototype[@@unscopables]`	
Writable	no
Enumerable	no
Configurable	yes

Exemplos

O código a seguir funciona bem no ES5 e abaixo. No entanto, no ECMAScript 2015 e posterior, o método `Array.prototype.keys()` foi introduzido. Isso significa que dentro de ambientes `with`, as "keys" seriam agora o método e não a variável. É aqui que a propriedade de símbolo `@@unscopables` `Array.prototype[@@unscopables]` entra em ação e impede que alguns métodos do Array estejam sendo definidos na instrução `with`.

```
var keys = [];

with (Array.prototype) {
  keys.push("alguma coisa");
}

Object.keys(Array.prototype[Symbol.unscopables]);
// ["copyWithin", "entries", "fill", "find", "findIndex",
//  "includes", "keys", "values"]
```

Veja também

- `Symbol.unscopables`

Array.prototype.toLocaleString()

O método **`toLocaleString()`** retorna uma representaçao de elementos de um array. Os elementos são convertidos para Strings utilizando seus respectivos métodos `toLocaleString` e essas cadeias são separadas por uma sequência específica de localidade (separados por virgula ",").

Sintaxe

```
array.toLocaleString();
```

Parâmetros

`locales` Optional

Uma string com uma tag de linguagem BCP 47, ou um array como strings. Para uma forma geral e uma interpretação do arquivo `locales`, veja a página `Intl`.

`options` Optional

Um objeto com as propriedades de configurações, para números veja `Number.prototype.toLocaleString()`, e para datas veja `Date.prototype.toLocaleString()`.

Retorno

Uma string que representa os elementos de um array.

Exemplos

Usando <code>toLocaleString</code>

Os elementos de um array são convertidos para strings usando seus respectivos métodos `toLocaleString`:

- `Object`: `Object.prototype.toLocaleString()`
- `Number`: `Number.prototype.toLocaleString()`
- `Date`: `Date.prototype.toLocaleString()`

```
var numero = 1337;
var data = new Date();
var meuArray = [numero, data, "foo"];

var resultado = meuArray.toLocaleString();

console.log(resultado);
// saída '1337,July 26, 2015 at 20:02:23 GMT-3,foo'
// se estiver no Brasil com o fuso horario GMT-0300 (BRT) de Brasília.
```

Para mais exemplos, veja as páginas `Intl`, `NumberFormat`, e `DateTimeFormat`.

Veja também

- `Array.prototype.toString()`

Array.prototype.toString()

O método **`toString()`** retorna uma string representando um array específico e seus elementos.

Sintaxe

```
arr.toString()
```

Parâmetros

Nenhum.

Descrição

O objeto `Array` substitui o método toString() de `Object`. Para objetos do tipo Array, o método toString() concatena todos os valores em apenas uma string. Segue exemplo abaixo, de como ele se comporta.

```
var monthNames = ["Jan", "Feb", "Mar", "Apr"];
var myVar = monthNames.toString(); // atribui 'Jan,Feb,Mar,Apr' para myVar.
```

JavaScript chama o método toString automaticamente quando um Array é para ser representado como um valor de texto ou quando um Array é designado em uma concatenação.

Semântica ECMAScript 5

Implementado no JavaScript 1.8.5 (Firefox 4), e compatível com a 5ª versão do ECMAScript, a função toString() é genérica e pode ser usada em qualquer Objeto. Se o objeto tem um método join(), ele será chamado e esse valor será devolvido. Do contrário, `Object.prototype.toString()` será chamado, e o valor será retornado.

Veja também

- `Object.prototype.toSource()`

Array.prototype.unshift()

Introdução

O método **`unshift()`** adiciona um ou mais elementos no início de um array e retorna o número de elementos (propriedade `length`) atualizado.

Sintaxe

```
arr.unshift([element1[, ...[, elementN]]])
```

Parâmetros

`elementN`

Os elementos a serem adicionados no começo do array.

Retorna

A nova propriedade `length` do objeto acima onde o método foi chamado.

Descrição

O método `unshift` insere os valores fornecidos no início de um objeto do tipo array.

`unshift` é intencionalmente genérico; este método pode ser chamado via call ou apply em objetos que se assemelham aos arrays. Objetos que não contêm uma propriedade `length` que reflete a última de uma série consecutiva de propriedades numéricas, iniciada por 0, podem não comportar-se de maneira significativa.

Exemplos

```
var arr = [1, 2];

arr.unshift(0); // result of call is 3, the new array length
// arr is [0, 1, 2]

arr.unshift(-2, -1); // = 5
// arr is [-2, -1, 0, 1, 2]

arr.unshift([-3]);
// arr is [[-3], -2, -1, 0, 1, 2]
```

Veja também

- `Array.prototype.push()`
- `Array.prototype.pop()`
- `Array.prototype.shift()`

Array.prototype.values()

O método **`values()`** retorna um novo objeto de **`Array Iterator`** que contém os valores para cada índice no array.

Sintaxe

```
arr.values()
```

Exemplos

Iteração usando <code>for...of</code> loop

```
var arr = ["w", "y", "k", "o", "p"];
var eArr = arr.values();
// seu navegador deve suportar for..of loop
// e deixar variáveis let-scoped no for loops
for (let letter of eArr) {
  console.log(letter);
}
```

Iteração alternativa

```
var arr = ["w", "y", "k", "o", "p"];
var eArr = arr.values();
console.log(eArr.next().value); // w
console.log(eArr.next().value); // y
console.log(eArr.next().value); // k
console.log(eArr.next().value); // o
console.log(eArr.next().value); // p
```

Notas específicas do Firefox

- Devido a um problema de compatibilidade, `Array.prototype.values()` foi removido do SpiderMonkey por enquanto.

Ver também

- `Array.prototype.keys()`
- `Array.prototype.entries()`
- `Array.prototype.forEach()`
- `Array.prototype.every()`
- `Array.prototype.some()`

ArrayBuffer

O objeto **`ArrayBuffer`** é um tipo de dado usado para representar um genérico, buffer de dados binários de tamanho fixo. Você não pode manipular diretamente os conteúdos de um `ArrayBuffer`; em vez disso, você cria um objeto `ArrayBufferView` que representa o buffer em um formato específico, e usa para ler e escrever os conteúdos do buffer.

Experimente

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);

console.log(buffer.byteLength);
// Expected output: 8
```

Syntax

```
new ArrayBuffer(length)
```

Parameters

`length`

The size, in bytes, of the array buffer to create.

Return value

A new `ArrayBuffer` object of the specified size. Its contents are initialized to 0.

Exceptions

A `RangeError` is thrown if the `length` is larger than `Number.MAX_SAFE_INTEGER` (>= 2 ** 53) or negative.

Description

The `ArrayBuffer` constructor creates a new `ArrayBuffer` of the given length in bytes.

Getting an array buffer from existing data

- From a Base64 string
- From a local file

Properties

`ArrayBuffer.length`

The `ArrayBuffer` constructor's length property whose value is 1.

`get ArrayBuffer[@@species]`

The constructor function that is used to create derived objects.

`ArrayBuffer`

Allows the addition of properties to all `ArrayBuffer` objects.

Methods

`ArrayBuffer.isView(arg)`

Returns `true` if `arg` is one of the ArrayBuffer views, such as typed array objects or a `DataView`. Returns `false` otherwise.

`ArrayBuffer.transfer(oldBuffer [, newByteLength])`

Returns a new `ArrayBuffer` whose contents are taken from the `oldBuffer`'s data and then is either truncated or zero-extended by `newByteLength`.

Instances

All `ArrayBuffer` instances inherit from `ArrayBuffer`.

Properties

Methods

`ArrayBuffer.slice()`

Has the same functionality as `ArrayBuffer.prototype.slice()`.

Exemplo

In this example, we create a 8-byte buffer with a `Int32Array` view referring to the buffer:

```
var buffer = new ArrayBuffer(8);
var view   = new Int32Array(buffer);
```

Compatibility notes

Starting with ECMAScript 2015, `ArrayBuffer` constructors require to be constructed with a `new` operator. Calling an `ArrayBuffer` constructor as a function without `new`, will throw a `TypeError` from now on.

```
var dv = ArrayBuffer(10);
// TypedError: calling a builtin ArrayBuffer constructor
// without new is forbidden
```

```
var dv = new ArrayBuffer(10);
```

Veja também

- JavaScript typed arrays
- `SharedArrayBuffer`

AsyncFunction

O **construtor** **`AsyncFunction`** cria um novo objeto `async function`. Em JavaScript cada função assíncrona é atualmente um objeto do tipo `AsyncFunction`.

Note que`AsyncFunction` não é um objeto global. Ele poderia ser obtido analisando o seguinte código:

```
Object.getPrototypeOf(async function () {}).constructor;
```

Sintaxe

```
new AsyncFunction([arg1[, arg2[, ...argN]],] functionBody)
```

Parameters

`arg1, arg2, ... argN`

> Nomes que vão ser usados pela função como argumentos formais. Cada um deve ser uma string que corresponde a um indentificador JavaScript válido ou uma lista das strings separadas com uma vírgula; por exemplo "`x`", "`oValor`", or "`a,b`".

`functionBody`

> Uma string contendo as declarações JavaScript que compõem a definição da função.

Descrição

Objetos `async function` criados com o construtor `AsyncFunction` são tratados quando a função é criada. Isso é menos eficiente que declarar uma função async com a `expression async function` e chama-la com seu código, porque essas funções são tratadas com o resto do código.

Todos os argumentos passado para a função são tratados como nomes dos identificadores dos parâmetros na função que vai ser criada, na ordem que eles são passados.

> **Nota:** `async functions` criadas com o construtor `AsyncFunction` não cria closures para seus contextos de criação; elas sempre criadas no escopo global. Quando rodar eles, eles só poderão acessar suas variáveis local e as globais, mas não as que estão no escopo que o construtor foi `AsyncFunction` chamado. Isso é diferente de usar `eval` com código para uma expressão async function.

Invocar o construtor `AsyncFunction` como uma função (sem usar o operador `new`) tem o mesmo efeito de invocá-lo como um construtor.

Propriedades

`AsyncFunction.length`

> A propriedade tamanho do construtor da `AsyncFunction` cujo valor é 1.

`AsyncFunction`

> Permite a adição de propriedades para todos os objetos async function.

<code>AsyncFunction</code> prototype object

Propriedades

<code>AsyncFunction</code> instances

Instância `AsyncFunction` herdam métodos e propriedades do `AsyncFunction`. Com todos os contrutores, que podem mudar o prototype do objeto construtor para fazer mudanças em todas as instâncias do `AsyncFunction`.

Exemplos

Criando uma async function a partir do construtor de uma <code>AsyncFunction</code>

```
function resolveAfter2Seconds(x) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

var AsyncFunction = Object.getPrototypeOf(async function () {}).constructor;

var a = new AsyncFunction(
  "a",
  "b",
  "return await resolveAfter2Seconds(a) + await resolveAfter2Seconds(b);",
);

a(10, 20).then((v) => {
  console.log(v); // imprime 30 após 4 seconds
});
```

Veja também

- `async function function`
- `async function expression`
- `Function`
- `function statement`
- `function expression`
- Funções e escopo de função

Atomics

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

O objeto **`Atomics`** fornece operações atômicas como metodos estáticos. Eles são usados com objetos `SharedArrayBuffer`.

As operações atômicas estão localizadas no modulo `Atomics`. Diferente de outros *global objects*, `Atomics` não é um construtor. Você não deve usa-lo com o `new` operator ou invocar objetos `Atomics` como funções. Todas as propriedades e método do `Atomics` são estáticos (como é o caso com o objeto `Math`, por exemplo).

Métodos

Operações Atômicas

Quando a memória é compartilhada, multiplas threads podem ser lidas e escritas no mesmo dado da memória. Operações atômicas garantem que os valores previstos sejam lidos e escritos, estas operações são finalizadas antes da próxima operação iniciar e que as mesmas não sejam interrompidas.

`Atomics.add()`

: Adiciona o valor recebido na posiçao recebida no array. Retorna o valor anterior nesta posição.

`Atomics.and()`

: Calcula um bit a bit AND na posição recebida no array. Retorna o valor anterior nesta posição.

`Atomics.compareExchange()`

: Armazena o valor recebido na posição recebida no array, se este foi igual ao valor recebido. Retorna o valor anterior nesta posição.

`Atomics.exchange()`

: Armazena o valor recebido na posição recebida no array. Retorna o valor anterior.

`Atomics.load()`

: Retorna o valor na posição recebida no array.

`Atomics.or()`

: Calcula um bit a bit OR na posição recebida no array. Retorna o valor anterior nesta posição.

`Atomics.store()`

: Armazena o valor recebido na posição recebida no array. Retorna o valor.

`Atomics.sub()`

: Subtrai o valor recebido na posição recebida no array. Retorna o valor anterior nesta posição.

`Atomics.xor()`

: Calcula um bit a bit XOR na posição recebida no array. Retorna o valor anterior nesta posição.

Wait e wake

Os métodos `wait()` e `wake()` são modelados no Linux futexes ("fast user-space mutex") e fornece formas de aguardar até que certas condições se tornem `true` e são tipicamente usadas como construtores de bloco.

`Atomics.wait()`

> Verifica se a posição informada no array ainda contém a valor recebido e dorme à espera ou até o tempo limite. Retorna `"ok"`, `"not-equal"`, ou `"timed-out"`. Se a espera não for permitida no agente de chamada ele irá lançar uma exceção de erro (muitos navegadores não permitem `wait()` na thread main do navegador).

`Atomics.wake()`

> "Acorda" alguns agentes que estavam "dormindo" na lista de espera na posição recebida do array. Retorna o número de agentes que estão sendo "acordados".

`Atomics.isLockFree(size)`

> Uma otimização primitiva que pode ser usada para determinar se deve ser usado lock ou operações atômicas. Retorna `true`, se uma operação atômica em matrizes de um dado tamanho do elemento vai ser implementado utilizando uma operação atômica de hardware (como oposição a lock). Só para experientes.

Observações de compatibilidade

[3] A especificação de compartilhamento de memória está sendo estabilizada. Anterior ao SpiderMonkey 48, a última API e semântica não foram implementadas ainda. As alterações entre as versões 46 e 48 do Firefox são:

- Os métodos `Atomics.futexWakeOrRequeue()` e `Atomics.fence()` foram totalmente removidos (Erro do Firefox 1259544 e Erro do Firefox 1225028).
- O método `Atomics.wait()` e `Atomics.wake()` foram nomeados como `Atomics.futexWait()` e `Atomics.futexWake()` (Erro do Firefox 1260910). Os nomes antigos foram mantidos como alias, e serão removidos na versão 49 (Erro do Firefox 1262062).
- As propriedades `Atomics.OK`, `Atomics.TIMEDOUT`, `Atomics.NOTEQUAL` foram removidas. O método `Atomics.wait()` agora retorna "ok", "timed-out" e "not-equal" (Erro do Firefox 1260835).
- O parâmetro `count` do método `Atomics.wake()` foi alterado: este agora possui o valor padrão `+Infinity`, e não `0` (Erro do Firefox 1253350).

Veja Também

- `ArrayBuffer`
- JavaScript typed arrays
- Web Workers
- parlib-simple – a simple library providing synchronization and work distribution abstractions.
- Shared Memory – a brief tutorial

Atomics.add()

O método estátitico **`Atomics.add()`** adiciona um dado valor em uma determinada posição no array e retorna o valor antigo daquela posição. Esta operação atômica garante que nenhuma outra escrita ocorra até que o valor modificado seja escrito de volta.

Sintaxe

```
Atomics.add(typedArray, index, value)
```

Parâmetros

`typedArray`

> Um array tipado de inteiros compartilhado. Pode ser `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, ou `Uint32Array`.

`index`

> A posição no `typedArray` onde o `value` será adicionado.

`value`

> Número que será adicionado.

Valor de retorno

O valor antigo na determinada posição (`typedArray[index]`).

Exceções

- Lança uma exceção `TypeError`, se o `typedArray` não for um dos tipos de inteiro permitidos.
- Lança uma exceção `TypeError`, se o `typedArray` não for tipo array tipado compartilhado.
- Lança uma exceção `RangeError`, se o `index` estiver fora dos limites no `typedArray`.

Exemplos

```
var sab = new SharedArrayBuffer(1024);
var ta = new Uint8Array(sab);

Atomics.add(ta, 0, 12); // retorna 0, o valor antigo
Atomics.load(ta, 0); // 12
```

Veja também

- `Atomics`
- `Atomics.sub()`

BigInt

`BigInt` é um objeto nativo que fornece um modo de representar números inteiros maiores que 2^53, que é o maior número que o JavaScript consegue, com exatidão, representar com o tipo primitivo `Number`.

Sintaxe

```
BigInt(value);
```

Parâmetros

`value`

O valor numérico do objeto que está sendo criado. Pode ser uma *string* ou um número inteiro.

> **Nota: Observação**: `BigInt()` não é usado com o operador `new`.

Descrição

Um `BigInt` é criado com a acrescentação de `n` ao final de um inteiro literal — `10n` — ou chamando a função `BigInt()`.

```
const theBiggestInt = 9007199254740991n;

const alsoHuge = BigInt(9007199254740991);
// ↪ 9007199254740991n

const hugeString = BigInt("9007199254740991");
// ↪ 9007199254740991n

const hugeHex = BigInt("0x1fffffffffffff");
// ↪ 9007199254740991n

const hugeBin = BigInt(
  "0b11111111111111111111111111111111111111111111111111111",
);
// ↪ 9007199254740991n
```

Isso é parecido com um `Number` em algumas partes, mas difere-se em alguns assuntos importantes — ele não pode ser usado com métodos no objeto `Math` e não pode ser misturado em operações ou qualquer instância de `Number`.

> **Aviso:** `Number` e `BigInt` não podem ser misturados em operações — eles devem ser manipulados com o mesmo tipo.
>
> Tenha cuidado com a conversão e desconversão de valores, visto que a precisão de `BigInt` pode ser perdida com a conversão para `Number`.

Informações do tipo

Quando testado com `typeof` , um `BigInt` vai devolver "bigint":

```
typeof 1n === "bigint"; // true
typeof BigInt("1") === "bigint"; // true
```

Quando envolvido em um `Object`, um `BigInt` vai ser considerado como um tipo normal de "object".

```
typeof Object(1n) === "object"; // true
```

Operadores

Os seguintes operadores podem ser usados com `BigInt`s (ou com `BigInt`s envolvidos em objetos): `+`, `*`, `-`, `**`, `%`.

```
const previousMaxSafe = BigInt(Number.MAX_SAFE_INTEGER);
// ↪ 9007199254740991

const maxPlusOne = previousMaxSafe + 1n;
// ↪ 9007199254740992n

const theFuture = previousMaxSafe + 2n;
// ↪ 9007199254740993n, isso funciona agora!

const multi = previousMaxSafe * 2n;
// ↪ 18014398509481982n

const subtr = multi - 10n;
// ↪ 18014398509481972n

const mod = multi % 10n;
// ↪ 2n
```

```
const bigN = 2n ** 54n;
// ↪ 18014398509481984n

bigN * -1n;
// ↪ -18014398509481984n
```

O operador `/` também funciona com o esperado com números inteiros. No entanto, desde que esses sejam `BigInt`s e não `BigDecimal`s, essa operação vai arredondar para 0, o que significa que não vai retornar qualquer valor fracional.

> **Aviso:** Uma operação com um resultado fracional será arredondado com `BigInt`.

```
const expected = 4n / 2n;
// ↪ 2n

const rounded = 5n / 2n;
// ↪ 2n, e não 2.5n
```

Comparações

Um `BigInt` não é estritamente igual a um `Number`, mas é mais ou menos assim.

```
0n === 0;
// ↪ false

0n == 0;
// ↪ true
```

Um `Number` e um `BigInt` podem ser comparado normalmente.

```
1n < 2;
// ↪ true

2n > 1;
// ↪ true

2 > 2;
// ↪ false

2n > 2;
// ↪ false

2n >= 2;
// ↪ true
```

Eles podem ser misturados em *arrays* e ordenados.

```
const mixed = [4n, 6, -12n, 10, 4, 0, 0n];
// ↪  [4n, 6, -12n, 10, 4, 0, 0n]

mixed.sort();
// ↪ [-12n, 0, 0n, 10, 4n, 4, 6]
```

Observe que comparações com `BigInt`s envolvidos em `Object` atuam com outros objetos, indicando somente a igualdade onde a mesma instância do objeto é comparada.

```
0n === Object(0n); // false
Object(0n) === Object(0n); // false

const o = Object(0n);
o === o; // true
```

Condicionais

A `BigInt` comporta-se como `Number` nos casos onde ele é convertido para um `Boolean`: através da função `Boolean`; quando usado com operadores lógicos `Logical Operators` `||`, `&&`, e `!`; ou dentro de um teste condicional como um `if statement`.

```
if (0n) {
  console.log("Olá de um if!");
} else {
  console.log("Olá de um else!");
}

// ↪ "Olá de um else!"

0n || 12n;
// ↪ 12n

0n && 12n;
// ↪ 0n
```

```
Boolean(0n);
// ↪ false

Boolean(12n);
// ↪ true

!12n;
// ↪ false

!0n;
// ↪ true
```

Métodos

`BigInt.asIntN()`

Limita um BigInt entre -2expoente-1 e 2expoente-1-1

`BigInt.asUintN()`

Limita um BigInt entre 0 e 2expoente-1

Propriedades

`BigInt`

Permite a adição de propriedades a um objeto `BigInt`.

Instâncias <code>BigInt</code>

Todas as instâncias `BigInt` são herdades de BigInt.prototype. O protótipo do objeto do construtor do BigInt pode ser modificado para afetar todas as instâncias de `BigInt`.

Métodos

Exemplos

Calculando números primos

```
function isPrime(p) {
  for (let i = 2n; i * i <= p; i++) {
    if (p % i === 0n) return false;
  }
  return true;
}

// Toma um BigInt como argumento e retorna um BigInt
function nthPrime(nth) {
  let maybePrime = 2n;
  let prime = 0n;

  while (nth >= 0n) {
    if (isPrime(maybePrime)) {
      nth -= 1n;
      prime = maybePrime;
    }
    maybePrime += 1n;
  }

  return prime;
}

nthPrime(20n);
// ↪ 73n
```

Boolean

O objeto **`Boolean`** é um objeto wrapper para um valor booleano.

Sintaxe

```
new Boolean([value])
```

Parâmetros

`value`

Opcional. O valor inicial do objeto `Boolean`.

Descrição

O valor passado como primeiro parâmetro é convertido para um valor boleano, se necessário. Se o valor é omitido ou é `0`, `-0`, `null`, `false`, `NaN`, `undefined` ou é uma string vazia(`""`), o objeto terá um valor inicial de `false`. Todos outros valores, incluindo qualquer objeto ou string `"false"`, criam um objeto com valor inicial `true`.

Não confunda os valores primitivos Boolean `true` e `false` com os valores `true` and `false` do objeto `Boolean`.

Qualquer objeto cujo o valor não é `undefined` ou `null`, incluindo um objeto `Boolean` que o valor seja `false`, é avaliado para `true` quando passa por uma declaração condicional. Por exemplo, a condição a seguir `if` a declaração é avaliada como `true`:

```
var x = new Boolean(false);
if (x) {
  // esse código é executado
}
```

Esse comportamento não se aplica aos primitivos `Boolean`. Por exemplo, a condição a seguir `if` a declaração é avaliada como `false`:

```
var x = false;
if (x) {
  // esse código não é executado
}
```

Não use um objeto `Boolean` para converter um valor não-boleano para um valor boleano. Ao invés disso use `Boolean` como uma função para executar essa tarefa:

```
var x = Boolean(expression); // preferido
var x = new Boolean(expression); // não use
```

Se você especificar qualquer objeto, incluindo um objeto `Boolean` cujo valor é `false`, como valor inicial de um objeto `Boolean`, o novo objeto `Boolean` terá o valor de `true`.

```
var myFalse = new Boolean(false); // valor inicial false
var g = new Boolean(myFalse); // valor inicial true
var myString = new String("Hello"); // objeto String
var s = new Boolean(myString); // valor inicial true
```

Não use um um objeto `Boolean` no lugar de um primitivo `Boolean`.

Propriedades

`Boolean.length`

> Propriedade Length cujo valor é 1.

`Boolean`

> Representa o protótipo para o construtor `Boolean`.

Métodos

O objeto global `Boolean` contém métodos próprios, entretanto, ele herda alguns métodos através da cadeia de protótipos:

Instâncias <code>Boolean</code>

Todas instâncias `Boolean` herdam de `Boolean`. Assim como todos os construtores, o protótipo do objeto dita as propriedades e métodos herdados.

Propriedades

Métodos

Exemplos

Criando objetos <code>Boolean</code> com um valor inicial <code>false</code>

```
var bNoParam = new Boolean();
var bZero = new Boolean(0);
var bNull = new Boolean(null);
var bEmptyString = new Boolean("");
var bfalse = new Boolean(false);
```

Criando objetos <code>Boolean</code> com um valor inicial <code>true</code>

```
var btrue = new Boolean(true);
var btrueString = new Boolean("true");
var bfalseString = new Boolean("false");
var bSuLin = new Boolean("Su Lin");
var bArrayProto = new Boolean([]);
var bObjProto = new Boolean({});
```

Veja também

- `Boolean`
- Boolean
- Boolean data type (Wikipedia)

Boolean.prototype.toString()

O método **`toString()`** retorna uma string representando o objeto Boolean específico.

Sintaxe

```
bool.toString()
```

Valor retornado

Uma string representando o objeto `Boolean` específico.

Descrição

O objeto `Boolean` sobrepõe o método `toString` do objeto `Object`; ele não herda `Object.prototype.toString()`. Para objetos Boolean, o método `toString` retorna uma representação do objeto como string.

O JavaScript chama o método `toString` automaticamente quando uma `Boolean` deve ser representado como texto ou quando uma `Boolean` é referenciada como uma concatenação de string.

Para objetos e valores `Boolean`, o método nativo `toString` retorna a string "`true`" ou "`false`", dependendo do valor do objeto boleano.

Exemplos

Usando <code>toString</code>

No código a seguir, `flag.toString()` retorna "`true`":

```
var flag = new Boolean(true);
var myVar = flag.toString();
```

Veja também

- `Object.prototype.toString()`

Boolean.prototype.valueOf()

O método **`valueOf()`** retorna o valor primitivo de um objeto `Boolean`.

Sintaxe

```
bool.valueOf()
```

Parâmetros

Nenhum.

Descrição

O método `valueOf` do `Boolean` retorna o valor primitivo de um objeto `Boolean` ou um literal `Boolean` como tipo de dado Boolean.

Esse método é geralmente chamado internamente pelo JavaScript e não explicitamente no código.

Exemplos

Usando <code>valueOf</code>

```
x = new Boolean();
myVar = x.valueOf(); // atribui o valor false à variável myVar
```

Veja também

- `Object.prototype.valueOf()`

DataView

O **`DataView`** provê uma interface de baixo nível para leitura e escrita de múltiplos tipos de número em um `ArrayBuffer`, independentemente da extremidade (*endianness*) da plataforma.

Experimente

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

// Create a couple of views
const view1 = new DataView(buffer);
const view2 = new DataView(buffer, 12, 4); // From byte 12 for the next 4 bytes
view1.setInt8(12, 42); // Put 42 in slot 12

console.log(view2.getInt8(0));
// Expected output: 42
```

Sintaxe

```
new DataView(buffer [, byteOffset [, byteLength]])
```

Parâmetros

`buffer`

> `ArrayBuffer` ou `SharedArrayBuffer` existente para usar como armazenamento de um novo objeto `DataView`.

`byteOffset` Optional

> A mudança, em bytes, do primeiro byte determinado em um buffer, que será referenciado pela nova view. Se não for especificado, a view do buffer começará no primeiro byte.

`byteLength` Optional

> O número de elementos no array de bytes. Se não especificado, o tamanho da view será do mesmo tamanho do buffer.

Retorno

Um novo objeto `DataView` que representa o buffer de dados especificado. (Provavelmente não foi uma descrição muito útil.)

Você pode pensar nesse objeto retornado como um "intérprete" de um array buffer de bytes - ele sabe como converter números para inserir em um buffer corretamente, tanto ao ler quanto ao gravar. Isso significa lidar com conversões *integer, float, endianness* e outros detalhes da representação de números em formato binário.

Exceções

`RangeError`

> Lançado se o `byteOffset` ou `byteLength` especificados ultrapassarem o final do buffer.
>
> Por exemplo, se o buffer tem 16 bytes de comprimento, o `byteOffset` é 8 e o `byteLength` é 10, esse erro será lançado porque a view resultante tenta estender 2 bytes acima do comprimento total do buffer.

Descrição

Endianness

Formatos de números *Multi-byte* são representados de maneira diferente na memória, dependendo da arquitetura da máquina, veja Endianness para mais informações. Assessores de `DataView` fornecem controle explícito de como o dado será acessado, independente do *endianness* da arquitetura em execução.

```
var littleEndian = (function () {
  var buffer = new ArrayBuffer(2);
  new DataView(buffer).setInt16(0, 256, true /* littleEndian */);
  // Int16Array uses the platform's endianness.
  return new Int16Array(buffer)[0] === 256;
})();
console.log(littleEndian); // true or false
```

Valores inteiros de 64 bits

Como JavaScript atualmente não inclui suporte padrão para valores inteiros de 64 bits, `DataView` não oferece operações nativas de 64 bits. Como solução alternativa, você poderia implementar sua própria função getUint64() para obter um valor com a precisão de `Number.MAX_SAFE_INTEGER`, o que pode ser bom para determinados casos.

```
function getUint64(dataview, byteOffset, littleEndian) {
  // split 64-bit number into two 32-bit (4-byte) parts
  const left = dataview.getUint32(byteOffset, littleEndian);
  const right = dataview.getUint32(byteOffset + 4, littleEndian);

  // combine the two 32-bit values
  const combined = littleEndian
    ? left + 2 ** 32 * right
    : 2 ** 32 * left + right;

  if (!Number.isSafeInteger(combined))
    console.warn(combined, "exceeds MAX_SAFE_INTEGER. Precision may be lost");

  return combined;
}
```

Como alternativa, se você precisar de um intervalo completo de 64 bits, poderá criar um `BigInt`.

```
function getUint64BigInt(dataview, byteOffset, littleEndian) {
  // split 64-bit number into two 32-bit (4-byte) parts
  const left = dataview.getUint32(byteOffset, littleEndian);
  const right = dataview.getUint32(byteOffset + 4, littleEndian);

  // combine the two 32-bit values as their hex string representations
  const combined = littleEndian
    ? right.toString(16) + left.toString(16).padStart(8, "0")
    : left.toString(16) + right.toString(16).padStart(8, "0");

  return BigInt(`0x${combined}`);
}
```

Propriedades

Todas as instâncias de `DataView` herdam `DataView` e permitem a adição de propriedades a todos os objetos DataView.

Métodos

Exemplo

```
var buffer = new ArrayBuffer(16);
var dv = new DataView(buffer, 0);

dv.setInt16(1, 42);
dv.getInt16(1); //42
```

Notas de compatibilidade

Começando com o Firefox 40, `DataView` deve ser construído com o operador `new` . Chamando `DataView()` como uma função sem o `new`, irá lançar um `TypeError` de agora em diante.

```
var dv = DataView(buffer, 0);
// TypeError: calling a builtin DataView constructor without new is forbidden
```

```
var dv = new DataView(buffer, 0);
```

Veja também

- jDataView: Biblioteca javascript que faz o polyfill e extende a API do `DataView` para todos os browsers e para o Node.js.
- `ArrayBuffer`
- `SharedArrayBuffer`

Date

Resumo

Cria uma instância JavaScript de **`Date`** que representa um único momento no tempo. Objetos Date são baseados no valor de tempo que é o número de milisegundos desde 1º de Janeiro de 1970 (UTC).

Construtor

```
new Date();
new Date(valor);
new Date(dataString);
new Date(ano, mês, dia, hora, minuto, segundo, milissegundo);
```

> **Nota:** Note que objetos JavaScript `Date` só podem ser instanciados chamando JavaScript `Date` como um construtor: chamá-lo como uma função regular (ou seja, sem o operador new) irá retornar uma string ao invés de um objeto `Date`; ao contrário de outros tipos de objetos JavaScript, objetos JavaScript `Date` não têm sintaxe literal.

Parâmetros para o constructor Date

Nota: Quando Date for chamado como um construtor com mais de um argumento, se os valores forem maiores do que seu limite lógico (e.g. se 13 for fornecido como um valor para mês ou 70 for o valor para minuto), o valor adjacente será ajustado. E.g. new Date(2013, 13, 1) é equivalente a new Date(2014, 1, 1), ambos criam uma data para 2014-02-01 (note que o mês começa em 0). Similarmente para outros valores: new Date(2013, 2, 1, 0, 70) é equivalente a new Date(2013, 2, 1, 1, 10), pois ambos criam uma data para 2013-03-01T01:10:00.

`value`

Um valor inteiro representando o número de milisegundos desde 1 de Janeiro de 1970 00:00:00 UTC (Era Unix ou Marco Zero).

`dataString`

Um valor do tipo String que representa uma data. A string deverá estar uma formato reconhecido pelo método `Date.parse()` (IETF-compliant RFC 2822 timestamps e também uma versão da ISO8601).

`year`

Um valor inteiro que representa o ano. Valores de 0 a 99 correspondem aos anos de 1900 a 1999. Veja o exemplo abaixo.

`month`

Um valor inteiro que representa o mês, começando com 0 para Janeiro até 11 para Dezembro.

`day`

Um valor inteiro que representa o dia do mês.

`hour`

Um valor inteiro que representa a hora do dia.

minute

Um valor inteiro que representa o segmento de um minuto de tempo.

second

Um valor inteiro que representa o segmento de segundo do tempo.

millisecond

Um valor inteiro que representa o segmento de milisegundo do tempo.

Descrição

- Se nenhum argumento for fornecido, o construtor criará um objeto JavaScript Date com a data e hora corrente de acordo com as configurações do sistema.
- Se ao menos 2 argumentos forem fornecidos, os argumentos ausentes serão configurados como 1 (se o dia estiver ausente) ou 0 para todos os outros.
- A data do JavaScript é baseada no valor de tempo em milisegundos desde a meia noite de 01 de Janeiro de 1970, UTC. Um dia corresponde a 86.400,000 milisegundos. O intervalo do objeto Date no JavaScript é de -100.000,000 dias a 100.000,000 dias relativo a 01 de Janeiro de 1970, UTC.
- O objeto Date no JavaScript tem um comportamento uniforme nas plataformas. O valor do tempo pode ser transmitido entre sistemas para representar o mesmo instante no tempo e se for usado para criar um objeto de data local, ele refletirá o tempo local equivalente.
- O objeto Date JavaScript suporta vários métodos UTC (universal), assim como métodos de tempo locais. UTC, também conhecido como Tempo Médio de Greenwich (Greenwich Mean Time, GMT), refere-se ao tempo como definido pelo Padrão de Tempo Mundial (World Time Standard). O tempo local é o tempo conhecido pelo computador onde o JavaScript é executado.
- Invocar o objeto Date no JavaScript como uma função (i.e., sem o operador new) retornatá uma string representando a data e hora corrente.

Propriedades

`Date`

Permite adicionar propriedades a um objeto javaScript Date.

Date.length

O valor de `Date.length` é 7. Esse é o número de argumentos manipulados pelo construtor.

Métodos

`Date.now()`

Retorna o valor numérico correspondente ao tempo corrente - o número de milisegundos passados desde 1 de Janeiro de 1970 00:00:00 UTC.

`Date.parse()`

Analisa uma string que representa uma data e retorna o número de milisegundos desde 1 de Janeiro, 1970, 00:00:00, hora local.

`Date.UTC()`

Aceita os mesmos parâmetros como a forma mais longa do construtor (i.e. 2 até 7) e retorna o número de milisegundos desde 1 de Janeiro, 1970, 00:00:00 UTC.

Instâncias JavaScript de <code>Date</code>

Todas as instâncias `Date` são herdadas de `Date`. O objeto protótipo do construtor `Date` pode ser modificado para afetar todas as instâncias de `Date`.

Métodos

Exemplos

Várias formas de se criar um objeto Date

Os seguintes exemplos mostram várias formas de se criar datas em JavaScript:

> **Nota:** A conversão de *strings* com o construtor de `Date` (`Date.parse` é equivalente ao contrutor) é fortemente desencorajada devido às inconsistências e diferenças dos navegadores.

```
var today = new Date();
var birthday = new Date("December 17, 1995 03:24:00");
var birthday = new Date("1995-12-17T03:24:00");
var birthday = new Date(1995, 11, 17);
var birthday = new Date(1995, 11, 17, 3, 24, 0);
```

Anos com dois dígitos mapeados para 1900 - 1999

Para criar e obter datas entre os anos 0 e 99 os métodos `Date.prototype.setFullYear()` e `Date.prototype.getFullYear()` devem ser usados.

```
var data = new Date(98, 1); // Dom Fev 01 1998 00:00:00 GMT+0000 (GMT)

// Métodos em desuso, 98 mapeia para 1998 aqui também
data.setYear(98); // Dom Fev 01 1998 00:00:00 GMT+0000 (GMT)

data.setFullYear(98); // Sab Fev 01 0098 00:00:00 GMT+0000 (BST)
```

Calculando o tempo decorrido

Os seguintes exemplos mostram como determinar o tempo decorrido entre duas datas no JavaScript em milissegundos.

Devido aos tamanhos diferentes dos dias (devido à mudança do horário de verão), meses e dias, expressar o tempo decorrido em unidades maiores que horas, minutos e segundos requer analisar os problemas e deve ser cuidadosamente investigado antes de se tentar utilizar.

```
// usando objetos Date
var inicio = Date.now();

// o evento para o tempo vai aqui:
facaAlgoPorUmLongoTempo();
var fim = Date.now();
var decorrido = fim - inicio; // tempo decorrido em milisegundos
```

```
// utilizando métodos embutidos
var inicio = new Date();

// o evento para o tempo vai aqui:
facaAlgoPorUmLongoTempo();
var fim = new Date();
var decorrido = fim.getTime() - inicio.getTime(); // tempo decorrido em milisegundos
```

```
// para testar uma função e obter o seu retorno
function imprimirTempoDecorrido(fTeste) {
  var nHoraInicial = Date.now(),
    vRetorno = fTeste(),
    nHoraFinal = Date.now();

  alert(
    "Tempo decorrido: " + String(nHoraFinal - nHoraInicial) + " milisegundos",
  );
  return vRetorno;
}

retornoDaSuaFuncao = imprimirTempoDecorrido(suaFuncao);
```

> **Nota:** Em navegadores que suportam a API de Desempenho Web (Web Performance API) com o recurso de tempo de alta resolução, `Performance.now()` pode fornecer medidas de tempo decorrido mais confiáveis e precisas do que `Date.now()`.

Date.prototype.getDate()

Resumo

O método **`getDate()`** retorna o dia do mês da data especificada de acordo com a hora local.

Sintaxe

```
dateObj.getDate()
```

Parâmetros

Nenhum.

Retorna

O valor retornado por `getDate()` é um inteiro entre 1 e 31.

Exemplos

Exemplo: Usando <code>getDate()</code>

O segundo statement abaixo atribui o valor 25 à variável day, baseado no valor do objeto `Date` `Xmas95`.

```
var Xmas95 = new Date("December 25, 1995 23:15:30");
var day = Xmas95.getDate();

console.log(day); // 25
```

Veja também

- `Date.prototype.getUTCDate()`
- `Date.prototype.getUTCDay()`
- `Date.prototype.setDate()`

Date.prototype.getDay()

Resumo

O método **`getDay()`** retorna o dia da semana para a data especificada de acordo com a hora local, onde 0 representa o Domingo.

Sintaxe

```
dateObj.getDay()
```

Parâmetros

Nenhum.

Retorna

O valor retornado por `getDay()` é um inteiro que corresponde com o dia da semana: 0 para Domingo, 1 para Segunda-Feira, 2 para Terça-Feira, e assim por diante.

Exemplos

Exemplo: Usando <code>getDay()</code>

O segundo statement abaixo atribui o valor 1 à variável weekday (dia da semana), baseado no valor do objeto `Date` `Xmas95`. A data 25 de Dezembro de 1995 é uma Segunda-Feira.

```
var Xmas95 = new Date("December 25, 1995 23:15:30");
var weekday = Xmas95.getDay();

console.log(weekday); // 1
```

Veja também

- `Date.prototype.getUTCDate()`
- `Date.prototype.getUTCDay()`
- `Date.prototype.setDate()`

Date.prototype.getFullYear()

Resumo

O método **`getFullYear()`** retorna o ano da data especificada de acordo com a hora local.

Use este método ao invés do `getYear()`.

Sintaxe

```
dateObj.getFullYear()
```

Parâmetros

Nenhum.

Retorna

O valor retornado por `getFullYear()` é um número absoluto. Para datas entre os anos 1000 e 9999, `getFullYear()` retorna um número de quatro dígitos, por exemplo, 1995. Use esta função para ter certeza de que o ano é compatível com os anos depois de 2000.

Exemplos

Exemplo: Usando <code>getFullYear()</code>

O exemplo seguinte atribui o valor de quatro dígitos do ano corrente à variável year.

```
var today = new Date();
var year = today.getFullYear();
```

Veja também

- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setFullYear()`
- `Date.prototype.getYear()`

Date.prototype.getHours()

Resumo

O método **`getHours()`** retorna a hora para a data especificada, de acordo com a hora local.

Sintaxe

```
dateObj.getHours()
```

Parâmetros

Nenhum.

Retorna

O valor retornado por `getHours()` é um inteiro entre 0 e 23.

Exemplos

Exemplo: Usando <code>getHours()</code>

O segundo statement abaixo atribui o valor 23 à variável `hours`, baseado no valor do objeto `Date` `Xmas95`.

```
var Xmas95 = new Date("December 25, 1995 23:15:30");
var hours = Xmas95.getHours();

console.log(hours); // 23
```

Veja também

- `Date.prototype.getUTCHours()`
- `Date.prototype.setHours()`

Date.prototype.getMilliseconds()

O método **`getMilliseconds()`** retorna os milissegundos em uma data específica de acordo com o horário local.

Sintaxe

```
dateObj.getMilliseconds()
```

Retorna

Um número, entre 0 e 999, representando os milissegundos de uma data de acordo com o horário local.

Exemplos

Usando o <code>getMilliseconds()</code>

O exemplo a seguir atribui a porção de milissegundos do horário atual à variável `milliseconds`:

```
var today = new Date();
var milliseconds = today.getMilliseconds();
```

Veja também

- `Date.prototype.getUTCMilliseconds()`
- `Date.prototype.setMilliseconds()`

Date.prototype.getMinutes()

O método **`getMinutes()`** retorna os minutos em uma data específica de acordo com o horário local.

Sintaxe

```
dateObj.getMinutes()
```

Retorna

Um número inteiro, entre 0 e 59, representando os minutos em uma data de acordo com o horário local.

Exemplos

Usando o <code>getMinutes()</code>

The second statement below assigns the value 15 to the variable `minutes`, based on the value of the `Date` object `Xmas95`.

No exemplo abaixo, a segunda linha atribui o valor 15 à variável `minutes`, baseado no valor de objeto `Date` `Xmas95`.

```
var Xmas95 = new Date("December 25, 1995 23:15:30");
var minutes = Xmas95.getMinutes();

console.log(minutes); // 15
```

Veja também

- `Date.prototype.getUTCMinutes()`
- `Date.prototype.setMinutes()`

Date.prototype.getMonth()

O método **`getMonth()`** retorna o mês na data especificada de acordo com o horário local, como um valor zero-based (onde o zero indica o primeiro mês do ano).

Sintaxe

```
dateObj.getMonth()
```

Parâmetros

Nenhum.

Retorno

O valor retornado pelo método `getMonth()` é um inteiro entre 0 e 11. 0 corresponde a Janeiro, 1 a Fevereiro, e assim sucessivamente.

Exemplos

Usando o <code>getMonth()</code>

A segunda declaração abaixo atribui o valor 11 à variavel `month`, baseado no valor do objeto `Date` `Xmas95`.

```
var Xmas95 = new Date("December 25, 1995 23:15:30");
var month = Xmas95.getMonth();

console.log(month); // 11
```


Veja também

- `Date.prototype.getUTCMonth()`
- `Date.prototype.setMonth()`

Date.prototype.getSeconds()

O método **`getSeconds()`** retorna os segundos de uma data específica de acordo com o horário local.

Sintaxe

```
dateObj.getSeconds()
```

Retorna

Um número inteiro, entre 0 e 59, representando os segundos de uma data específica de acordo com o horário local.

Exemples

Usando o <code>getSeconds()</code>

The second statement below assigns the value 30 to the variable `seconds`, based on the value of the `Date` object `Xmas95`.

No exemplo a seguir, a segunda linha atribui o valor 30 à variável `seconds`, baseado no valor do objeto `Date` `Xmas95`.

```
var Xmas95 = new Date("December 25, 1995 23:15:30");
var seconds = Xmas95.getSeconds();

console.log(seconds); // 30
```

Veja também

- `Date.prototype.getUTCSeconds()`
- `Date.prototype.setSeconds()`

Date.prototype.getTime()

O método **`getTime()`** retorna o valor numérico correspondente ao horário da data especificada de acordo com o horário universal.

Você pode usar este método para atribuir uma data e horário a outro objeto `Date`. Este método é funcionalmente equivalente ao método `valueOf()`.

Sintaxe

```
dateObj.getTime()
```

Retorna

Um número representando os milissegundos passados entre 1 de Janeiro de 1970 00:00:00 UTC e a data atual.

Exemplos

Usando <code>getTime()</code> para duplicar datas

Construindo um objeto de data com um horário idêntico.

```
// Sendo o mês iniciado em zero, birthday será 10 de Janeiro de 1995
var birthday = new Date(1994, 12, 10);
var copy = new Date();
copy.setTime(birthday.getTime());
```

Medindo tempo de execução

Subtrair duas chamadas subsequentes a `getTime()` em objetos `Date` recém criados resultará no intervalo de tempo entre essas duas chamadas. Isso pode ser usado para calcular o tempo de execução de algumas operações. Veja também `Date.now()` para evitar instanciar objetos `Date` desnecessariamente.

```
var end, start;

start = new Date();
for (var i = 0; i < 1000; i++) {
  Math.sqrt(i);
}
end = new Date();

console.log("Operation took " + (end.getTime() - start.getTime()) + " msec");
```

See also

- `Date.prototype.setTime()`
- `Date.prototype.valueOf()`
- `Date.now()`

Date.prototype.getTimezoneOffset()

O método **`getTimezoneOffset()`** retorna a diferença, em minutos, do deslocamento de fuso horário entre o UTC (Tempo Universal Coordenado) e a localidade atual.

Sintaxe

```
dateObj.getTimezoneOffset()
```

Parâmetros

Nenhum

Valor de retorno

O deslocamento de fuso horário é a diferença, em minutos, entre o UTC (Tempo Universal Coordenado) e o horário local. Perceba que isto significa que o deslocamento será negativo se o fuso horário local está à direita do UTC e positivo se está a esquerda. Por exemplo, se seu fuso horário é UTC+10(Padrão da Austrália Oriental), -600 será retornado. O Horário de verão evita que este valor seja sempre o mesmo para uma determinada localidade.

Exemplo

Utilizando <code>getTimezoneOffset()</code>

```
var x = new Date();
var currentTimeZoneOffsetInHours = x.getTimezoneOffset() / 60;
```

Date.prototype.getUTCDate()

O método **`getUTCDate()`** retorna o dia (data) do mês na data especificada de acordo com o horário universal.

Sintaxe

```
dateObj.getUTCDate()
```

Retorna

Um número inteiro, entre 1 e 31, representando o dia do mês na data especificada de acordo com o horário universal.

Exemplos

Usando <code>getUTCDate()</code>

O exemplo a seguir atribui o dia da data atual à variável `day`:

```
var today = new Date();
var day = today.getUTCDate();
```

Veja também

- `Date.prototype.getDate()`
- `Date.prototype.getUTCDay()`
- `Date.prototype.setUTCDate()`

Date.prototype.getUTCDay()

O método **`getUTCDay()`** retorna o dia da semana na data especificada de acordo com o horário universal, onde 0 representa domingo.

Sintaxe

```
dateObj.getUTCDay()
```

Parâmetros

Nenhum.

Valor retornado

O valor retornado por `getUTCDay()` é um inteiro correspondente ao dia da semana: 0 para domingo, 1 para segunda-feira, 2 para terça-feira e assim por diante.

Exemplos

Usando <code>getUTCDay()</code>

O exemplo a seguir atribui a parte do dia da semana da data atual à variável `diaDaSemana`.

```
var hoje = new Date();
var diaDaSemana = hoje.getUTCDay();
```

Veja também

- `Date.prototype.getUTCDate()`
- `Date.prototype.getDay()`
- `Date.prototype.setUTCDate()`

Date.prototype.getUTCFullYear()

O método **`getUTCFullYear()`** retorna o ano na data indicada de acordo com o horário universal.

Experimente

```
const date1 = new Date("December 31, 1975, 23:15:30 GMT+11:00");
const date2 = new Date("December 31, 1975, 23:15:30 GMT-11:00");

console.log(date1.getUTCFullYear());
// Expected output: 1975

console.log(date2.getUTCFullYear());
// Expected output: 1976
```

Sintaxe

```
dateObj.getUTCFullYear()
```

Retorna

Um número que representa o ano na data indicada de acordo com o horário universal.

Descrição

O valor retornado por `getUTCFullYear()` é um número absoluto compatível com ano 2000, por exemplo, 1995.

Exemplos

Usando <code>getUTCFullYear()</code>

O exemplo a seguir atribui o valor de 4 dígitos do ano atual à variável `year`.

```
var today = new Date();
var year = today.getUTCFullYear();
```

Veja também

- `Date.prototype.getFullYear()`
- `Date.prototype.setFullYear()`

Date.prototype.getUTCHours()

O método **`getUTCHours()`** retorna o número de horas na data especificada de acordo com o horário universal (UTC).

Experimente

```
const date1 = new Date("December 31, 1975, 23:15:30 GMT+11:00");
const date2 = new Date("December 31, 1975, 23:15:30 GMT-11:00");

console.log(date1.getUTCHours());
// Expected output: 12

console.log(date2.getUTCHours());
// Expected output: 10
```

Sintaxe

```
dateObj.getUTCHours()
```

Valor retornado

Um número inteiro, entre 0 e 23, representando as horas em uma data de acordo com o horário universal (UTC).

Exemplos

Usando <code>getUTCHours()</code>

O exemplo a seguir atribui a porção de horas do horário corrente à variável `hours`.

```
var today = new Date();
var hours = today.getUTCHours();
```

Veja também

- `Date.prototype.getHours()`
- `Date.prototype.setUTCHours()`

Date.prototype.getUTCMilliseconds()

o método **`getUTCMilliseconds()`** retorna os milisegundos na data especificada, de acordo com o horário universal (UTC).

Sintaxe

```
dateObj.getUTCMilliseconds()
```

Valor de retorno

Um número inteiro, entre 0 e 999, representando os milisegundos na data especificada de acordo com o horário universal (UTC).

Exemplos

Usando <code>getUTCMilliseconds()</code>

O exemplo a seguir associa a parte dos milissegundos do tempo atual à variável `milissegundos`.

```
var hoje = new Date();
var milissegundos = today.getUTCMilliseconds();
```

Veja também

- `Date.prototype.getMilliseconds()`
- `Date.prototype.setUTCMilliseconds()`

Date.prototype.getUTCMinutes()

O método **`getUTCMinutes()`** retorna os minutos na data especificada de acordo com o tempo universal.

Experimente

```
const date1 = new Date("1 January 2000 03:15:30 GMT+07:00");
const date2 = new Date("1 January 2000 03:15:30 GMT+03:30");

console.log(date1.getUTCMinutes()); // 31 Dec 1999 20:15:30 GMT
// Expected output: 15

console.log(date2.getUTCMinutes()); // 31 Dec 1999 23:45:30 GMT
// Expected output: 45
```

Sintaxe

```
dateObj.getUTCMinutes()
```

Valor retornado

Um número inteiro, entre 0 e 59, representando os minutos na data especificada de acordo com o horário universal (UTC).

Exemplos

Usando <code>getUTCMinutes()</code>

O seguinte exemplo atribui a parte dos minutos do tempo atual para a variável `minutes`.

```
var today = new Date();
var minutes = today.getUTCMinutes();
```

Veja também

- `Date.prototype.getMinutes()`
- `Date.prototype.setUTCMinutes()`

Date.prototype.getUTCMonth()

A função **`getUTCMonth()`** retorna o mês da data especificada de acordo com o horário universal, como um valor iniciado em zero (aonde zero indica o primeiro mês do ano).

Sintaxe

```
dateObj.getUTCMonth()
```

Valor retornado

Um número inteiro, entre 0 e 11, correspondente ao mês da data informada de acordo com o horário universal. 0 para Janeiro, 1 para Fevereiro, 2 para Março, e assim em diante.

Exemplos

Usando <code>getUTCMonth()</code>

O exemplo a seguir atribui a parte do mês da data atual a variável `month`.

```
var today = new Date();
var month = today.getUTCMonth();
```

Veja também

- `Date.prototype.getMonth()`
- `Date.prototype.setUTCMonth()`

Date.prototype.getUTCSeconds()

O método **`getUTCSeconds()`** retorna os segundos na data informada de acordo com a hora universal.

Experimente

```
const moonLanding = new Date("July 20, 1969, 20:18:04 UTC");

console.log(moonLanding.getUTCSeconds());
// Expected output: 4
```

Sintaxe

```
dateObj.getUTCSeconds()
```

Valor de retorno

Um número inteiro, entre 0 e 59, representando os segundos da data informada de acordo com a hora universal.

Exemplos

Usando <code>getUTCSeconds()</code>

The following example assigns the seconds portion of the current time to the variabl

O exemplo a seguir atribui a parte dos segundos da hora atual à variável `seconds`.

```
var today = new Date();
var seconds = today.getUTCSeconds();
```

Veja também

- `Date.prototype.getSeconds()`
- `Date.prototype.setUTCSeconds()`

Date.prototype.getYear()

Resumo

O método **`getYear()`** retorna o ano especificado pela data de acordo com a hora local. Por conta do fato de que `getYear()` não retorna anos completos ("problema do ano 2000"), este método não é mais utilizado e foi substituido pelo método `getFullYear()`.

Sintaxe

```
dateObj.getYear()
```

Parâmetros

Nenhum.

Retorna

O método `getYear()` retorna o ano menos 1900; então:

- Para ano maior ou igual a 2000, o valor retornado por `getYear()` é 100 ou mais. Por exemplo, se o ano for 2026, `getYear()` retorna 126.

- Para ano entre e incluindo 1900 e 1999, o valor retornado por `getYear()` é entre 0 e 99. Por exemplo, se o anor for 1976, `getYear()` retorna 76.
- Para ano menor que 1900, o valor retornado por `getYear()` é menor que 0. Por exemplo, se o ano for 1800, `getYear()` retorna -100.

Para levar em consideração anos antes e depois de 2000, você deve utilizar `getFullYear()` ao invés de `getYear()` para que o ano seja especificado por completo.

Retrocombatibilidade

Comportamento em JavaScript 1.2 e anteriores

O método `getYear()` retorna um ano de 2 ou 4 dígitos:

- Para anos entre e incluindo 1900 e 1999, o valor retornado por `getYear()` é o ano menos 1900. Por exemplo, se o ano é 1976, o valor retornado é 76.
- Para anos menores que 1900 ou maiores que 1999, o valor retornado por `getYear()` é um ano de 4 dígitos. Por exemplo, se o ano é 1856, o valor retornado é 1856. Se o ano é 2026, o valor retornado é 2026.

Exemplos

Exemplo: Anos entre 1900 e 1999

O segundo statement atribui o valor 95 à variável `year`.

```
var Xmas = new Date("December 25, 1995 23:15:00");
var year = Xmas.getYear(); // returns 95
```

Exemplo: Anos depois de 1999

O segundo statement atribui o valor 100 à variável `year`.

```
var Xmas = new Date("December 25, 2000 23:15:00");
var year = Xmas.getYear(); // returns 100
```

Exemplo: Anos antes de 1900

O segundo statement atribui o valor -100 à variável `year`.

```
var Xmas = new Date("December 25, 1800 23:15:00");
var year = Xmas.getYear(); // returns -100
```

Exemplo: Setando e recuperando um ano entre 1900 e 1999

O segundo statement atribui o valor 95 à variável `year`, representanto o ano 1995.

```
var Xmas.setYear(95);
var year = Xmas.getYear(); // returns 95
```

Veja também

- `Date.prototype.getFullYear()`
- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setYear()`

Date.now()

Resumo

O método **`Date.now()`** retorna o número de milisegundos decorridos desde 1 de janeiro de 1970 00:00:00 UTC.

Sintaxe

```
var timeInMs = Date.now();
```

Parâmetros

Nenhum.

Descrição

O método **`now()`** retorna os milisegundos decorridos desde 1 de janeiro de 1970 00:00:00 UTC até agora como um `Number`.

Por que **`now`** é um método estático de **`Date`**, você sempre o usará como **`Date.now()`**.

Polyfill

Este método foi padronizado no ECMA-262 5th edition. Em ambientes que não estão atualizados para suportar este método você pode suprir esta carência utilizando o seguinte:

```
if (!Date.now) {
  Date.now = function now() {
    return new Date().getTime();
  };
}
```

Veja também

- `window.performance.now` - fornece timestamps com sub-millisegundo precisão para uso em medir o desempenho da página
- `console.time` / `console.timeEnd`

Date.parse()

Resumo

O método **`Date.parse()`** analisa uma representação de data em *string*, e retorna o número de milisegundos desde 01 de Janeiro de 1970, 00:00:00 UTC ou NaN se a string não for reconhecida ou, em alguns casos, contiver valores inválidos de data (ex. 2015-02-31).

O uso de `Date.parse` não é recomendado, uma vez que até ES5 a análise de strings era inteiramente dependente de implementação. Ainda existem muitas diferenças de como diferentes plataformas analisam strings de data, então strings de data devem ser manualmente analisadas (uma biblioteca pode ajudar caso seja necessário atender a vários formatos)

Sintaxe

Chamada direta:

```
Date.parse(dateString)
```

Chamada implícita:

```
new Date(dateString)
```

Parâmetros

`dateString`

Uma *string* de representação de datas no format RFC2822 ou ISO 8601 (outros formatos podem ser utilizados, mas os resultados podem não ser os esperados).

Descrição

O método `parse()` analisa uma *string* de data (como `"Dec 25, 1995"`) e retorna o número de milisegundos desde 01 de Janeiro de 1970, 00:00:00 UTC. Esta função é útil para definir valores de data baseados em valores de *string*, por exemplo em conjunto com o método `setTime()` e o objeto `Date`.

Dada uma string representando um tempo, `parse()` retorna o valor temporal. É aceito o RFC2822 / IETF sintaxe de data (RFC2822 Section 3.3), ex.: `"Mon, 25 Dec 1995 13:30:00 GMT"`. A função entende as abreviações dos fusos horários continentais dos EUA, mas para uso geral, use um deslocamento de fuso horário, por exemplo, `"Seg, 25 Dez 1995 13:30:00 +0430"` (4 horas, 30 minutos à leste do Meridiano de Greenwich). Se o fuso horário não é especificado e a string está em um formato ISO reconhecido pela ES5, então é adotado o formato UTC (Tempo Universal Coordenado). GMT e UTC são considerados equivalentes. O fuso horário do seu horário local é usado para interpreatar argumentos na RFC2822 Formato 3.3 (ou qualquer formato não reconhecido como ISO 8601 na ES5) que não contenha informação de fuso horário.

Suporte ao formato ECMAScript 5 ISO-8601

Astring Data e hora pode ser no formato ISO 8601. Por exemplo, `"2011-10-10"` (apenas data) ou `"2011-10-10T14:48:00"` (data e hora) podem ser passados ou convertidos. O fuso horario UTC é usado para interpretar argumentos no formato ISO 8601 que não contenham informação de fuso horário (note que ECMAScript ed 6 draft especifica que a *string* do tipo data e hora sem um fuso horário são tratados como data local, não UTC).

While time zone specifiers are used during date string parsing to interpret the argument, the value returned is always the number of milliseconds between January 1, 1970 00:00:00 UTC and the point in time represented by the argument.

Because `parse()` is a static method of `Date`, it is called as `Date.parse()` rather than as a method of a `Date` instance.

Differences in assumed time zone

Given a date string of `"March 7, 2014"`, `parse()` assumes a local time zone, but given an ISO format such as `"2014-03-07"` it will assume a time zone of UTC. Therefore `Date` objects produced using those strings will represent different moments in time unless the system is set with a local time zone of UTC. This means that two date strings that appear equivalent may result in two different values depending on the format of the string that is being converted (this behavior is changed in ECMAScript ed 6 so that both will be treated as local).

Fall-back to implementation-specific date formats

The ECMAScript specification states: If the String does not conform to the standard format the function may fall back to any implementation–specific heuristics or implementation–specific parsing algorithm. Unrecognizable strings or dates containing illegal element values in ISO formatted strings shall cause `Date.parse()` to return `NaN`.

However, invalid values in date strings not recognized as ISO format as defined by ES5 may or may not result in `NaN`, depending on the browser and values provided, e.g.:

```
// Non-ISO string with invalid date values
new Date("23/25/2014");
```

will be treated as a local date of 25 November, 2015 in Firefox 30 and an invalid date in Safari 7. However, if the string is recognized as an ISO format string and it contains invalid values, it will return `NaN` in all browsers compliant with ES5:

```
// ISO string with invalid values
new Date("2014-25-23").toISOString();
// returns "RangeError: invalid date" in all es5 compliant browsers
```

SpiderMonkey's implementation-specific heuristic can be found in `jsdate.cpp`. The string `"10 06 2014"` is an example of a non–conforming ISO format and thus falls back to a custom routine. See also this rough outline on how the parsing works.

```
new Date("10 06 2014");
```

will be treated as a local date of 6 October, 2014 and not 10 June, 2014. Other examples:

```
new Date("foo-bar 2014").toString();
// returns: "Invalid Date"

Date.parse("foo-bar 2014");
// returns: NaN
```

Examples

Example: Using <code>Date.parse()</code>

If `IPOdate` is an existing `Date` object, it can be set to August 9, 1995 (local time) as follows:

```
IPOdate.setTime(Date.parse("Aug 9, 1995"));
```

Some other examples of parsing non–standard date strings:

```
Date.parse("Aug 9, 1995");
```

Returns `807937200000` in time zone GMT-0300, and other values in other time zones, since the string does not specify a time zone and is not ISO format, therefore the time zone defaults to local.

```
Date.parse("Wed, 09 Aug 1995 00:00:00 GMT");
```

Returns `807926400000` no matter the local time zone as GMT (UTC) is provided.

```
Date.parse("Wed, 09 Aug 1995 00:00:00");
```

Returns `807937200000` in time zone GMT-0300, and other values in other time zones, since there is no time zone specifier in the argument and it is not ISO format, so is treated as local.

```
Date.parse("Thu, 01 Jan 1970 00:00:00 GMT");
```

Returns `0` no matter the local time zone as a time zone GMT (UTC) is provided.

```
Date.parse("Thu, 01 Jan 1970 00:00:00");
```

Returns `14400000` in time zone GMT-0400, and other values in other time zones, since no time zone is provided and the string is not in ISO format, therfore the local time zone is used.

```
Date.parse("Thu, 01 Jan 1970 00:00:00 GMT-0400");
```

Returns `14400000` no matter the local time zone as a time zone GMT (UTC) is provided.

See also

- `Date.UTC()`

Date.prototype.setDate()

O método **`setDate()`** configura o dia do objeto `Date` relativamente ao início do mês configurado previamente.

Experimente

```
const event = new Date("August 19, 1975 23:15:30");

event.setDate(24);

console.log(event.getDate());
// Expected output: 24

event.setDate(32);
// Only 31 days in August!

console.log(event.getDate());
// Expected output: 1
```

Sintaxe

```
dateObj.setDate(dayValue)
```

Parâmetros

`dayValue`

Um número inteiro que representa o dia do mês.

Valor de retorno

O número de milisegundos entre 1o de janeiro de 1970 00:00:00 UTC e a data indicada (o objeto `Date` é mudado no lugar).

Descrição

Se o `dayValue` estiver fora da extensão de valores de data para o mês, `setDate()` vai atualizar o objeto `Date` consequentemente.

Por exemplo, se 0 for fornecido para `dayValue`, a data será configurada para o último dia do mês anterior.

Se um número negativo for fornecido para `dayValue`, a data será configurada contando-se regressivamente desde o último dia do mês anterior. -1 resultará na data sendo configurada para um dia antes do último dia o mês anterior.

Exemplos

Usando <code>setDate()</code>

```
var theBigDay = new Date(1962, 6, 7); // 1962-07-07 (7th of July 1962)
theBigDay.setDate(24); // 1962-07-24 (24th of July 1962)
theBigDay.setDate(32); // 1962-08-01 (1st of August 1962)
theBigDay.setDate(22); // 1962-08-22 (22th of August 1962)
theBigDay.setDate(0); // 1962-07-31 (31th of July 1962)
theBigDay.setDate(98); // 1962-10-06 (6th of October 1962)
theBigDay.setDate(-50); // 1962-08-11 (11th of August 1962)
```

Veja também

- `Date.prototype.getDate()`
- `Date.prototype.setUTCDate()`

Date.prototype.setFullYear()

O método **`setFullYear()`** atribui o ano completo para a data especificada de acordo com o horário universal (UTC). Retorna uma nova data e hora.

Experimente

```
const event = new Date("August 19, 1975 23:15:30");

event.setFullYear(1969);

console.log(event.getFullYear());
// Expected output: 1969

event.setFullYear(0);

console.log(event.getFullYear());
// Expected output: 0
```

Sintaxe

```
dateObj.setFullYear(yearValue[, monthValue[, dateValue]])
```

Parâmetros

`yearValue`

Um inteiro especificando o valor número de um ano, por exemplo, 1995.

`monthValue`

Opcional. Um inteiro entre 0 e 11 representando os meses Janeiro até Dezembro.

`dateValue`

Opcional. Um inteiro entre 1 e 31 representando o dia do mês. Se você especificar o parâmetro `dateValue`, você deve também especificar `monthValue`.

Valor retornado

O número de milisegundos entre 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se você não especificar os parâmetros `monthValue` e `dateValue`, os valores retornados dos métodos `getMonth()` e `getDate()` são usados.

Se um parâmetro que você especificou está fora do alcance esperado, `setFullYear()` tenta atualizar os outros parâmetros e a informação de data no objeto `Date` adequadamente. Por exemplo, se você especificar 15 para o `monthValue`, o ano será incrementado em 1 (`yearValue + 1`), e 3 é usado para o mês.

Exemplos

Usando <code>setFullYear()</code>

```
var theBigDay = new Date();
theBigDay.setFullYear(1997);
```

Veja também

- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setUTCFullYear()`
- `Date.prototype.setYear()`

Date.prototype.setHours()

O método **`setHours()`** atribui as horas para uma data especificada de acordo com a hora local e retorna o número de milissegundos desde do dia 1 de Janeiro de 1970 00:00:00 UTC até o tempo representado pela instância `Date` atualizada.

Experimente

```
const event = new Date("August 19, 1975 23:15:30");
event.setHours(20);

console.log(event);
// Expected output: "Tue Aug 19 1975 20:15:30 GMT+0200 (CEST)"
// Note: your timezone may vary

event.setHours(20, 21, 22);

console.log(event);
// Expected output: "Tue Aug 19 1975 20:21:22 GMT+0200 (CEST)"
```

Sintaxe

```
dateObj.setHours(hoursValue[, minutesValue[, secondsValue[, msValue]]])
```

Versões anteriores ao JavaScript 1.3

```
dateObj.setHours(hoursValue)
```

Parâmetros

`hoursValue`

Idealmente, um número inteiro entre 0 e 23, representando a hora. Se um valor maior que 23 for fornecido, a data e hora será incrementada pelas horas extras.

`minutesValue`

Opcional. Idealmente, um número inteiro entre 0 e 59, representando os minutos. Se um valor maior que 59 for fornecido, a data e hora será incrementada em minutos extras.

`secondsValue`

Opcional. Idealmente, um número inteiro entre 0 e 59, representando os segundos. Se um valor maior que 59 for fornecido, a data e hora será incrementada em segundos extras. Se você especificar o parâmetro `secondsValue`, também deverá especificar parâmetro `minutesValue`.

`msValue`

Opcional. Idealmente, um número entre 0 e 999, representando os milissegundos. Se um valor maior que 999 for fornecido, a data e hora será incrementada em milissegundos extras. Se você especificar o parâmetro `msValue`, você também deve especificar `minutesValue` e `secondsValue`.

Valor de retorno

O número de milissegundos entre dia 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se você não especifica os parâmetros `minutesValue`, `secondsValue`, e `msValue`, os valores retornados dos métodos `getMinutes()`, `getSeconds()`, e `getMilliseconds()` são usados.

Se um parâmetro que você especificar estiver fora do intervalo esperado, `setHours()` tenta atualizar as informações de data no objeto `Date`. Por exemplo, se você usa 100 para `secondsValue`, os minutos serão incrementados em 1 (`minutesValue + 1`), e 40 serão usados para os segundos.

Exemplos

Usando <code>setHours()</code>

```
var theBigDay = new Date();
theBigDay.setHours(7);
```

Veja também

- `Date.prototype.getHours()`
- `Date.prototype.setUTCHours()`

Date.prototype.setMilliseconds()

O método **`setMilliseconds()`** atribui os milissegundos para a data específica de acordo com a hora local.

Experimente

```
const event = new Date("August 19, 1975 23:15:30");

console.log(event.getMilliseconds());
// Expected output: 0

event.setMilliseconds(456);

console.log(event.getMilliseconds());
// Expected output: 456
```

Sintaxe

```
dateObj.setMilliseconds(millisecondsValue)
```

Parâmetros

`millisecondsValue`

Um número entre 0 e 999, representando os milissegundos.

Valor de retorno

O número de milissegundos entre dia 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se você especifica um número fora do alcance esperado, a informação de data no objeto `Date` é atualizado de acordo. Por exemplo, se você especifica 1005, o número de segundos é incrementado em 1, e 5 é utilizado para os milissegundos.

Exemplos

Usando <code>setMilliseconds()</code>

```
var theBigDay = new Date();
theBigDay.setMilliseconds(100);
```

Veja também

- `Date.prototype.getMilliseconds()`
- `Date.prototype.setUTCMilliseconds()`

Date.prototype.setMinutes()

O método **`setMinutes()`** atribui os minutos para uma data específica de acordo com o horário local.

Experimente

```
const event = new Date("August 19, 1975 23:15:30");

event.setMinutes(45);

console.log(event.getMinutes());
// Expected output: 45

console.log(event);
// Expected output: "Tue Aug 19 1975 23:45:30 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Sintaxe

```
dateObj.setMinutes(minutesValue[, secondsValue[, msValue]])
```

Versões anteriores ao JavaScript 1.3

```
dateObj.setMinutes(minutesValue)
```

Parâmetros

`minutesValue`

Um inteiro entre 0 e 59, representando os mintuos.

`secondsValue`

Opcional. Um inteiro entre 0 e 59, representando os segundos. Se você especificar o parâmetro `secondsValue`, você também deve especificar `minutesValue`.

`msValue`

Opcional. Um número entre 0 e 999, representando os milissegundos. Se você especificar o parâmetro `msValue`, você deve também especificar `minutesValue` e `secondsValue`.

Valor de retorno

O número de milissegundos entre dia 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se você não especifica os parâmetros `secondsValue` e `msValue`, os valores retornados dos métodos `getSeconds()` e `getMilliseconds()` são usados.

Se um parâmetro que você especificou estiver fora do alcance esperado, `setMinutes()` tentará atualizar a informação da data no objeto `Date`. Por exemplo, se você usa 100 para `secondsValue`, os minutos serão incrementados em 1 (`minutesValue + 1`), e 40 serão usados para segundos.

Exemplos

Usando <code>setMinutes()</code>

```
var theBigDay = new Date();
theBigDay.setMinutes(45);
```

Veja também

- `Date.prototype.getMinutes()`
- `Date.prototype.setUTCMinutes()`

Date.prototype.setMonth()

O método **`setMonth()`** atribui o mês para uma data específica de acordo com o ano corrente.

Experimente

```
const event = new Date("August 19, 1975 23:15:30");

event.setMonth(3);

console.log(event.getMonth());
// Expected output: 3

console.log(event);
// Expected output: "Sat Apr 19 1975 23:15:30 GMT+0100 (CET)"
// Note: your timezone may vary
```

Sintaxe

```
dateObj.setMonth(monthValue[, dayValue])
```

Versões anteriores ao JavaScript 1.3

```
dateObj.setMonth(monthValue)
```

Parâmetros

`monthValue`

Um inteiro baseado em zero representando o mês do ano a partir do começo do ano. Então, 0 representa Janeiro, 11 representa Dezembro, -1 representa Dezembro do ano passado, e 12 representa Janeiro do ano seguinte.

`dayValue`

Opcional. Um inteiro de 1 a 31, representando o dia do mês.

Valor de retorno

O número de milissegundos entre dia 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se você não especifica o parâmetro `dayValue`, o valor retornado do método `getDate()` é utilizado.

Se um parâmetro que você especificou estiver fora do alcance esperado, `setMonth()` tenta atualizar a informação da data no objeto `Date`. Por exemplo, se você usa 15 para `monthValue`, o ano será incrementado em 1, e 3 será usado para o mês.

O dia corrente do mês terá um impacto no comportamento deste método. Conceitualmente ele irá adicionar o número de dias dado pelo dia corrente do mês ao primeiro dia do novo mês especificado como parâmetro, retornando uma nova data. Por exemplo, se o valor corrente é 31 de agosto de 2016, chamando `setMonth()` com um valor de 1 irá retornar 2 de Março de 2016. Isso acontece porque Fevereiro de 2016 tem 29 dias.

Exemplos

Usando <code>setMonth()</code>

```
var theBigDay = new Date();
theBigDay.setMonth(6);

//Cuidado com transições de final de mês
var endOfMonth = new Date(2016, 7, 31);
endOfMonth.setMonth(1);
console.log(endOfMonth); //Wed Mar 02 2016 00:00:00
```

Veja também

- `Date.prototype.getMonth()`
- `Date.prototype.setUTCMonth()`

Date.prototype.setSeconds()

O método **`setSeconds()`** atribui os segundos na data especificada de acordo com o tempo local.

Experimente

```
const event = new Date("August 19, 1975 23:15:30");

event.setSeconds(42);

console.log(event.getSeconds());
// Expected output: 42

console.log(event);
// Expected output: "Sat Apr 19 1975 23:15:42 GMT+0100 (CET)"
// Note: your timezone may vary
```

Sintaxe

```
dateObj.setSeconds(secondsValue[, msValue])
```

Versões anteriores ao JavaScript 1.3

```
dateObj.setSeconds(secondsValue)
```

Parâmetros

`secondsValue`

Um inteiro entre 0 e 59, representando os segundos.

`msValue`

Opcional. Um número entre 0 e 999, representando os milissegundos.

Valor de retorno

O número de milissegundos entre 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se você não especifica o parâmetro `msValue`, o valor retornado do método `getMilliseconds()` é utilizado.

Se um parâmetro que você especificou está fora do alcance esperado, `setSeconds()` tentará atualizar a informação da data no objeto `Date`. Por exemplo, se você usar 100 para `secondsValue`, os minutos guardados no objeto `Date` serão incrementados em 1, e 40 serão usados para os segundos.

Exemplos

Usando <code>setSeconds()</code>

```
var theBigDay = new Date();
theBigDay.setSeconds(30);
```

Veja também

- `Date.prototype.getSeconds()`
- `Date.prototype.setUTCSeconds()`

Date.prototype.setTime()

O método **`setTime()`** atribui ao objecto `Date` a hora representada pelo número de milisegundos desde 1 de janeiro de 1970 as 00:00:00 UTC.

Experimente

```
const launchDate = new Date("July 1, 1999, 12:00:00");
const futureDate = new Date();
futureDate.setTime(launchDate.getTime());

console.log(futureDate);
// Expected output: "Thu Jul 01 1999 12:00:00 GMT+0200 (CEST)"
```

```
const fiveMinutesInMillis = 5 * 60 * 1000;
futureDate.setTime(futureDate.getTime() + fiveMinutesInMillis);

console.log(futureDate);
// Expected output: "Thu Jul 01 1999 12:05:00 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Sintáxe

```
dateObj.setTime(timeValue)
```

Parâmetros

`timeValue`

Um inteiro representando o número de milisegundos desde 1 de janeiro 1970, 00:00:00 UTC.

Valor retornado

O número de milisegundos entre 1 de janeiro de 1970 00:00:00 UTC e a data atualizada (efetivamente, o valor do argumento).

Descrição

Use o método `setTime()` para ajudar a atribuir data e hora para outro objeto `Date`.

Exemplos

Usando <code>setTime()</code>

```
var theBigDay = new Date("July 1, 1999");
var sameAsBigDay = new Date();
sameAsBigDay.setTime(theBigDay.getTime());
```

Veja também

- `Date.prototype.getTime()`
- `Date.prototype.setUTCHours()`

Date.prototype.setUTCDate()

O método **`setUTCDate()`** atribui o dia do mês para uma data especifica de acordo com o horário universal.

Experimente

```
const event = new Date("August 19, 1975 23:15:30 GMT-3:00");

console.log(event.getUTCDate());
// Expected output: 20

event.setUTCDate(19);

console.log(event.getUTCDate());
// Expected output: 19
```

Sintaxe

```
dateObj.setUTCDate(dayValue)
```

Parâmetros

`dayValue`

Um inteiro entre 1 e 31, representando o dia do mês.

Valor de retorno

O número de milissegundos entre 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se um parâmetro que você especificou está fora do alcance esperado, `setUTCDate()` tentará atualizar a informação da data no objeto `Date`. Por exemplo, se você usar 40 para `dayValue`, e o mês guardado no objeto `Date` é Junho, o dia será trocado para 10 e o mês será incrementado para Julho.

Exemplos

Usando <code>setUTCDate()</code>

```
var theBigDay = new Date();
theBigDay.setUTCDate(20);
```

Veja também

- `Date.prototype.getUTCDate()`
- `Date.prototype.setDate()`

Date.prototype.setUTCFullYear()

O método **`setUTCFullYear()`** atribui o ano completo na data especificada de acordo com o horário universal.

Experimente

```
const event = new Date("December 31, 1975 23:15:30 GMT-3:00");

console.log(event.getUTCFullYear());
// Expected output: 1976

console.log(event.toUTCString());
// Expected output: "Thu, 01 Jan 1976 02:15:30 GMT"

event.setUTCFullYear(1975);

console.log(event.toUTCString());
// Expected output: "Wed, 01 Jan 1975 02:15:30 GMT"
```

Sintaxe

```
dateObj.setUTCFullYear(yearValue[, monthValue[, dayValue]])
```

Parâmetros

`yearValue`

Um inteiro especificando o valor numérico do ano, por exemplo, 1995.

`monthValue`

Opcional. Um inteiro entre 0 e 11 representando os meses Janeiro até Dezembro.

`dayValue`

Opcional. Um inteiro entre 1 e 31 representando o dia do mês. Se você especificar o parâmetro `dayValue`, você deve especificar `monthValue` também.

Valor de retorno

O número de milissegundos entre 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se você não especificar os parâmetros `monthValue` e `dayValue`, os valores retornados dos métodos `getUTCMonth()` e `getUTCDate()` serão utilizados.

Se um parâmetro que você especificou está fora do alcance esperado, `setUTCFullYear()` tentará atualizar os outros parâmetros e a informação da data no objeto `Date`. Por exemplo, se você especificar 15 para o `monthValue`, o ano será incrementado em 1 (`yearValue + 1`), e 3 será usado para o mês.

Exemplos

Usando <code>setUTCFullYear()</code>

```
var theBigDay = new Date();
theBigDay.setUTCFullYear(1997);
```

Veja também

- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setFullYear()`

Date.prototype.setUTCHours()

O método **`setUTCHours()`** atribui a hora para a data especificada de acordo com o horário universal, e retorna o número de milissegundos desde 1 de Janeiro de 1970 00:00:00 UTC até o horário representado pela instância `Date` atualizada.

Experimente

```
const event = new Date("August 19, 1975 23:15:30 GMT-3:00");

console.log(event.toUTCString());
// Expected output: "Wed, 20 Aug 1975 02:15:30 GMT"

console.log(event.getUTCHours());
// Expected output: 2

event.setUTCHours(23);

console.log(event.toUTCString());
// Expected output: "Wed, 20 Aug 1975 23:15:30 GMT"
```

Sintaxe

```
dateObj.setUTCHours(hoursValue[, minutesValue[, secondsValue[, msValue]]])
```

Parâmetros

`hoursValue`

Um inteiro entre 0 e 23, representando a hora.

`minutesValue`

Opcional. Um inteiro entre 0 e 59, representando os minutos.

`secondsValue`

Opcional. Um inteiro entre 0 e 59, representando os segundos. Se você especificar o parâmetro `secondsValue`, você também deve especificar `minutesValue`.

`msValue`

Opcional. Um número entre 0 e 999, representando os milissegundos. Se você especificar o parâmetro `msValue`, você também deve especificar `minutesValue` e `secondsValue`.

Valor de retorno

O número de milissegundos entre 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se você não especificar os parâmetros `minutesValue`, `secondsValue`, e `msValue`, os valores retornados dos métodos `getUTCMinutes()`, `getUTCSeconds()`, e `getUTCMilliseconds()` serão utilizados.

Se o parâmetro que você especificou estiver fora do alcance esperado, `setUTCHours()` tentará atualizar a informação da data no objeto `Date`. Por exemplo, se você usa 100 para `secondsValue`, os minutos serão incrementados em 1 (`minutesValue + 1`), e 40 será usado para os segundos.

Exemplos

Usando <code>setUTCHours()</code>

```
var theBigDay = new Date();
theBigDay.setUTCHours(8);
```

Veja também

- `Date.prototype.getUTCHours()`
- `Date.prototype.setHours()`

Date.prototype.setUTCMilliseconds()

O método **`setUTCMilliseconds()`** atribui os milissegundos para a data especificada de acordo com o horário universal.

Experimente

```
const date1 = new Date("2018-01-24T12:38:29.069Z");

console.log(date1.getUTCMilliseconds());
// Expected output: 69

date1.setUTCMilliseconds(420);

console.log(date1.getUTCMilliseconds());
// Expected output: 420
```

Sintaxe

```
dateObj.setUTCMilliseconds(millisecondsValue)
```

Parâmetros

`millisecondsValue`

Um número entre 0 e 999, representando os milissegundos.

Valor de retorno

O número de milissegundos entre 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se o parâmetro que você especificou estiver fora do alcance esperado, `setUTCMilliseconds()` tentará atualizar a informação da data no objeto `Date`. Por exemplo, se você usar 1100 para `millisecondsValue`, os segundos guardados no objeto `Date` serão incrementados em 1, e 100 serão usados para os milissegundos.

Exemplos

Usando <code>setUTCMilliseconds()</code>

```
var theBigDay = new Date();
theBigDay.setUTCMilliseconds(500);
```

Veja também

- `Date.prototype.getUTCMilliseconds()`
- `Date.prototype.setMilliseconds()`

Date.prototype.setUTCMinutes()

O método **`setUTCMinutes()`** atribui os minutos para uma data específica de acordo com o horário universal.

Experimente

```
const date1 = new Date("December 31, 1975, 23:15:30 GMT+11:00");

console.log(date1.getUTCMinutes());
// Expected output: 15

date1.setUTCMinutes(25);

console.log(date1.getUTCMinutes());
// Expected output: 25
```

Sintaxe

```
dateObj.setUTCMinutes(minutesValue[, secondsValue[, msValue]])
```

Parâmetros

`minutesValue`

Um inteiro entre 0 e 59, representando os minutos.

`secondsValue`

Opcional. Um inteiro entre 0 e 59, reprensentando os segundos. Se você especificar o parâmetro `secondsValue`, você também deve especificar `minutesValue`.

`msValue`

Opcional. Um número entre 0 e 999, representando os milissegundos. Se você especificar o parâmetro `msValue`, você também deve especificar `minutesValue` e `secondsValue`.

Valor de retorno

O número de milissegundos entre 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se você não especificar os parâmetros `secondsValue` e `msValue`, os valores retornados dos métodos `getUTCSeconds()` e `getUTCMilliseconds()` são utilizados.

Se o parâmetro que você especificou estiver fora do alcance esperado, `setUTCMinutes()` tentará atualizar a informação da data no objeto `Date`. Por exemplo, se você usar 100 para `secondsValue`, os minutos serão incrementados em 1 (`minutesValue + 1`), e 40 será usado para os segundos.

Exemplos

Usando <code>setUTCMinutes()</code>

```
var theBigDay = new Date();
theBigDay.setUTCMinutes(43);
```

Veja também

- `Date.prototype.getUTCMinutes()`
- `Date.prototype.setMinutes()`

Date.prototype.setUTCMonth()

O método **`setUTCMonth()`** atribui o mês para a data especificada de acordo com o horário universal.

Experimente

```
const event = new Date("December 31, 1975 23:15:30 GMT-3:00");

console.log(event.toUTCString());
// Expected output: "Thu, 01 Jan 1976 02:15:30 GMT"

console.log(event.getUTCMonth());
// Expected output: 0

event.setUTCMonth(11);
```

```
console.log(event.toUTCString());
// Expected output: "Wed, 01 Dec 1976 02:15:30 GMT"
```

Sintaxe

```
dateObj.setUTCMonth(monthValue[, dayValue])
```

Parâmetros

`monthValue`

Um inteiro entre 0 e 11, representando os meses de Janeiro até Dezembro.

`dayValue`

Opcional. Um inteiro de 1 a 31, representando o dia do mês.

Valor de retorno

O número de milissegundos entre 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se você não especificar o parâmetro `dayValue`, o valor retornado do método `getUTCDate()` é utilizado.

Se um parâmetro que você especificou está fora do alcance especificado, `setUTCMonth()` tentará atualizar a informação da data no objeto `Date`. Por exemplo, se você usar 15 para `monthValue`, o ano irá incrementar em 1, e 3 será usado para o mês.

Exemplos

Usando <code>setUTCMonth()</code>

```
var theBigDay = new Date();
theBigDay.setUTCMonth(11);
```

Veja também

- `Date.prototype.getUTCMonth()`
- `Date.prototype.setMonth()`

Date.prototype.setUTCSeconds()

O método **`setUTCSeconds()`** atribui os segundos para a data especificada de acordo com o horário universal.

Experimente

```
const date1 = new Date("December 31, 1975, 23:15:30 GMT+11:00");

console.log(date1.getUTCSeconds());
// Expected output: 30

date1.setUTCSeconds(39);

console.log(date1.getUTCSeconds());
// Expected output: 39
```

Sintaxe

```
dateObj.setUTCSeconds(secondsValue[, msValue])
```

Parâmetros

`secondsValue`

Um inteiro entre 0 e 59, representando os segundos.

`msValue`

Opcional. Um número entre 0 e 999, representando os milissegundos.

Valor de retorno

O número de milissegundos entre 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se você não especifica o parâmetro `msValue`, o valor retornado pelo método `getUTCMilliseconds()` é utilizado.

Se o parâmetro que você especificou está fora do alcance esperado, `setUTCSeconds()` tentará atualizar a informação da data no objeto `Date`. Por exemplo, se você usar 100 para `secondsValue`, os minutos guardados no objeto `Date` irá ser incrementado em 1, e 40 será usado para os segundos.

Exemplos

Usando <code>setUTCSeconds()</code>

```
var theBigDay = new Date();
theBigDay.setUTCSeconds(20);
```

Veja também

- `Date.prototype.getUTCSeconds()`
- `Date.prototype.setSeconds()`

Date.prototype.setYear()

O método **`setYear()`** atribui o ano para a data especificada de acordo com o horário local. Devido `setYear()` não colocar o anos cheios ("problema do ano 2000"), ele não é mais utilizado e foi substituído pelo método `setFullYear()`.

Sintaxe

```
dateObj.setYear(yearValue)
```

Parâmetros

`yearValue`

Um inteiro.

Valor de retorno

O número de milissegundos entre 1 de Janeiro de 1970 00:00:00 UTC e a data atualizada.

Descrição

Se `yearValue` é um número entre 0 e 99 (inclusivo), então o ano para `dateObj` é atribuido para `1900 + yearValue`. Caso contrário, o ano para `dateObj` é atribuido para `yearValue`.

Exemplos

Usando <code>setYear()</code>

As duas primeiras linhas atribuem o ano para 1996. O terceiro atribui o ano para 2000.

```
var theBigDay = new Date();

theBigDay.setYear(96);
theBigDay.setYear(1996);
theBigDay.setYear(2000);
```

Veja também

- `Date.prototype.getFullYear()`
- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setFullYear()`
- `Date.prototype.setUTCFullYear()`

Date.prototype[@@toPrimitive]

O método **`[@@toPrimitive]()`** converte o objeto `Date` para um valor primitivo.

Sintaxe

```
Date()[Symbol.toPrimitive](hint);
```

Valor de retorno

O valor primitivo do objeto `Date`. Dependendo do argumento, o método pode retornar uma cadeia de caracteres (*string*) ou um número.

Descrição

O método `[@@toPrimitive]()` do objeto `Date` retorna um valor primitivo, que pode ser tanto um tipo numérico quanto uma string.

Se `hint` é `"string"` ou `"default"`, `[@@toPrimitive]()` tenta chamar o método `toString`. Se a propriedade `toString` não existe, ele tenta chamar o método `valueOf` e se o `valueOf` não existir, `[@@toPrimitive]()` joga um `TypeError`.

Se `hint` é `"number"`, `[@@toPrimitive]()` tenta primeiro chamar `valueOf`, e se ele falha, chama `toString`.

O JavaScript chama o método `[@@toPrimitive]()` para converter um objeto para um valor primitivo. Você raramente precisa invocar o método `[@@toPrimitive]()` em si; JavaScript automaticamente o invoca quando encontra um objeto onde um valor primitivo é esperado.

Exemplos

Retornando primitivos de data

```
const testDate = new Date(1590757517834);
// "Date Fri May 29 2020 14:05:17 GMT+0100 (British Summer Time)"

testDate[Symbol.toPrimitive]("string");
// Returns "Date Fri May 29 2020 14:05:17 GMT+0100 (British Summer Time)"

testDate[Symbol.toPrimitive]("number");
// Returns "1590757517834"

testDate[Symbol.toPrimitive]("default");
// Returns "Date Fri May 29 2020 14:05:17 GMT+0100 (British Summer Time)"
```

Veja também

- `Symbol.toPrimitive`

Date.prototype.toDateString()

O método **`toDateString()`** retorna a porção da data de um objeto `Date` em um formato legível para humanos em inglês dos Estados Unidos.

Sintaxe

```
objetoDate.toDateString()
```

Descrição

Exemplares de `Date` referem-se a um ponto específico no tempo. Chamar `toString()` retorna a data num formato legível para humanos em inglês dos Estados Unidos. No SpiderMonkey, isso consiste na porção da data (dia, mês e ano) seguida pela porção da hora (horas, minutos, segundos e fuso horário). Ás vezes, deseja-se obter uma string apenas da porção da data; Isso pode ser conseguido com o método `toDateString()`.

`O método toDateString()` é especialmente útil porque motores que implementam o padrão ECMA-262 podem divergir na string retornada pelo `toString()` de objetos `Date` , pois o formato é dependente da implentação e abordagens simples que repartem a string podem não produzir resultados consistentes nos vários motores.

Exemplos

Um uso básico de <code>toDateString()</code>

```
var d = new Date(1993, 6, 28, 14, 39, 7);

console.log(d.toString()); // exibe Wed Jul 28 1993 14:39:07 GMT-0600 (PDT) no log
console.log(d.toDateString()); // exibe Wed Jul 28 1993 no log
```

Ver também

- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toTimeString()`
- `Date.prototype.toString()`

Date.prototype.toISOString()

O método **`toISOString()`** retorna uma cadeia de caracteres (*string*) simplificada no formato ISO extendido (ISO 8601), que é sempre 24 ou 27 caracteres de tamanho (`YYYY-MM-DDTHH:mm:ss.sssZ` ou `±YYYYYY-MM-DDTHH:mm:ss.sssZ`, respectivamente). O fuso horário é sempre o deslocamento zero UTC, como denotado pelo sufixo "`Z`".

Experimente

```
const event = new Date("05 October 2011 14:48 UTC");
console.log(event.toString());
// Expected output: "Wed Oct 05 2011 16:48:00 GMT+0200 (CEST)"
// Note: your timezone may vary

console.log(event.toISOString());
// Expected output: "2011-10-05T14:48:00.000Z"
```

Sintaxe

```
dateObj.toISOString()
```

Valor de retorno

Uma *string* representando a data no formato ISO 8601 de acordo com o horário universal.

Polyfill

Este método foi padronizado na quinta edição da ECMA-262. Motores que não foram atualizados para suportar este método podem funcionar com uma gambiarra na ausência deste método da seguinte forma:

```
if (!Date.prototype.toISOString) {
  (function () {
    function pad(number) {
      if (number < 10) {
        return "0" + number;
      }
```

```
      return number;
    }

    Date.prototype.toISOString = function () {
      return (
        this.getUTCFullYear() +
        "-" +
        pad(this.getUTCMonth() + 1) +
        "-" +
        pad(this.getUTCDate()) +
        "T" +
        pad(this.getUTCHours()) +
        ":" +
        pad(this.getUTCMinutes()) +
        ":" +
        pad(this.getUTCSeconds()) +
        "." +
        (this.getUTCMilliseconds() / 1000).toFixed(3).slice(2, 5) +
        "Z"
      );
    };
  })();
}
```

Exemplos

Usando <code>toISOString()</code>

```
let today = new Date("05 October 2011 14:48 UTC");

console.log(today.toISOString()); // Retorna 2011-10-05T14:48:00.000Z
```

O exemplo acima usa uma conversão de uma *string* não-padrão que pode não ser convertida corretamente em navegadores que não sejam da Mozilla..

Veja também

- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toTimeString()`
- `Date.prototype.toUTCString()`

Date.prototype.toJSON()

O método **`toJSON()`** retorna uma representação representation do objeto `Date` como string.

Experimente

```
const event = new Date("August 19, 1975 23:15:30 UTC");

const jsonDate = event.toJSON();

console.log(jsonDate);
// Expected output: "1975-08-19T23:15:30.000Z"

console.log(new Date(jsonDate).toUTCString());
// Expected output: "Tue, 19 Aug 1975 23:15:30 GMT"
```

Sintaxe

```
dateObj.toJSON()
```

Retorno

Uma representação da data como string.

Descrição

Instâncias de `Date` referem-se a um específico ponto no tempo. Invocar `toJSON()` retorna uma string (usando `toISOString()`) representando o valor do objeto `Date`. Por padrão este método é destinado a serializar objetos `Date` em serializações JSON.

Exemplos

Usando <code>toJSON()</code>

```
var jsonDate = new Date().toJSON();
var backToDate = new Date(jsonDate);

console.log(jsonDate); //2015-10-26T07:46:36.611Z
```

Veja também

- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toTimeString()`
- `Date.prototype.toUTCString()`

Date.prototype.toLocaleDateString()

O método **`toLocaleDateString()`** retorna uma string com a representação de parte da data baseando-se no idioma. Os novos argumentos `locales` e `options` deixam as aplicações especificarem o idioma cujas convenções de formatação devem ser usadas e permitem customizar o comportamento da função. Em implementações antigas, nas quais ignoram os argumentos `locales` e `options`, o locale usado e a forma da string retornada são inteiramente dependente da implementação nativa.

Sintáxe

```
dateObj.toLocaleDateString([locales [, options]])
```

Parametros

Verifique a seção Compatibilidade entre navegadores para ver quais navegadores dão suporte aos argumentos `locales` e `options`, e o Example: Verificação para suporte dos argumentos `locales` e `options` para detectar a funcionalidade.

O valor padrão para cada propriedade do componente date-time é `undefined` , mas se as propriedades `weekday`, `year`, `month`, `day` são todas `undefined` , então `year`, `month` and `day` são assumidos como "`numeric`".

Return value

Uma representação em string de parte da data dada a instância `Date` de acordo com as convenções específicas do idioma.

Exemplos

Usando <code>toLocaleDateString()</code>

Em uso básico sem especificação de locale, uma string formatada no padrão do locale e com as opções padrões é retornada.

```
var date = new Date(Date.UTC(2012, 11, 12, 3, 0, 0));

// toLocaleDateString() sem argumentos depende da implementação,
// o locale padrão, e o time zone padrão
console.log(date.toLocaleDateString());
// → "12/11/2012" se rondando em locale en-US com time zone de America/Los_Angeles
```

Checagem para o suporte dos argumentos <code>locales</code> e <code>options</code>

Os argumentos `locales` e `options` não são suportados em todos os browser ainda. Para verificar se uma uma implementação já suporta eles, você pode usar o requisito de que tags de idioma ilegal são rejeitadas com uma exceção `RangeError`:

```
function toLocaleDateStringSupportsLocales() {
  try {
    new Date().toLocaleDateString("i");
  } catch (e) {
    return e.name === "RangeError";
  }
  return false;
}
```

Usando <code>locales</code>

Esse exemplo mostra algumas das variações em formatos de data localizados. A fim de obter oformato do idioma usado na interface de usuário da usa aplicação, certifique-se de especificar esse idioma (e possivelmente algumas outros idiomas de reserva) usando o argumento `locales`:

```
var date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// os formatos abaixo assumem o time zone local do locale;
// America/Los_Angeles for the US

// Inglês americano usa a ordem mês-dia-ano
console.log(date.toLocaleDateString("en-US"));
// → "12/19/2012"

// Inglês Britânico usa a ordem dia-mês-ano
console.log(date.toLocaleDateString("en-GB"));
// → "20/12/2012"

// Coreano usa a ordem ano-mês-dia
console.log(date.toLocaleDateString("ko-KR"));
// → "2012. 12. 20."

// Árabe na maioria dos países de língua árabe usa dígitos árabes reais
console.log(date.toLocaleDateString("ar-EG"));
// → "٢٠/١٢/٢٠١٢"

// para Japonês, aplicações podem querer usar o calendário japonês,
// onde 2012 foi o ano 24 da era Heisei
console.log(date.toLocaleDateString("ja-JP-u-ca-japanese"));
// → "24/12/20"

// quando um idioma que pode não ser suportado é requerido, tal como
// Balinês, inclua um idioma de reserva, nesse caso Indonésio
```

```
console.log(date.toLocaleDateString(["ban", "id"]));
// → "20/12/2012"
```

Usando <code>options</code>

O resultados gerados por `toLocaleDateString()` podem ser customizado usando o argumento `options`:

```
var date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// requer um dia da semana jutamente com uma data longa
var options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
console.log(date.toLocaleDateString("de-DE", options));
// → "Donnerstag, 20. Dezember 2012"

// uma aplicação pode querer usar UTC e fazê-lo visível
options.timeZone = "UTC";
options.timeZoneName = "short";
console.log(date.toLocaleDateString("en-US", options));
// → "Thursday, December 20, 2012, GMT"
```

Performance

Ao formatar um grande número de datas, é melhor criar um objeto `Intl.DateTimeFormat` e usar a função fornecido porsua propriedade `format`.

Veja também

- `Intl.DateTimeFormat`
- `Date.prototype.toLocaleString()`
- `Date.prototype.toLocaleTimeString()`
- `Date.prototype.toString()`

Date.prototype.toLocaleTimeString()

O método **`toLocaleTimeString()`** retorna uma string com uma representação sensível ao idioma de uma porção de tempo desta data. Os novos argumentos `locales` e `options` possibilitam aplicações especificarem que formato de linguagem deve ser usado, podendo customizar o comportamento da função. Em implementações antigas, que ignoram os argumentos `locales` e `options`, o local utilizado e o formato retornado da string são implementações completamente dependentes.

Experimente

```
// Depending on timezone, your results will vary
const event = new Date("August 19, 1975 23:15:30 GMT+00:00");

console.log(event.toLocaleTimeString("en-US"));
// Expected output: "1:15:30 AM"

console.log(event.toLocaleTimeString("it-IT"));
// Expected output: "01:15:30"

console.log(event.toLocaleTimeString("ar-EG"));
// Expected output: "١:١٥:٣٠ ص"
```

Sintaxe

```
dateObj.toLocaleTimeString([locales[, options]])
```

Parâmetros

Confira a seção Browser compatibility para ver o suporte em browsers dos argumentos `locales` e `options`, e Checking for support for `locales` and `options` arguments para ver suas features.

O valor default para cada propriedade do componente date-time é `undefined`, mas se as propriedades `hour`, `minute`, `second` são todas `undefined`, então `hour`, `minute`, e `second` espera-se que seja `"numeric"`.

Valor retornado

Uma string representando uma porção de tempo de uma instância `Date`, fornecida de acordo com as convenções específicas do idioma.

Exemplos

Usando <code>toLocaleTimeString()</code>

Em um uso simples, sem especificar uma localidade, é retornado uma string formatada de uma localidade default e com opções default.

```
var date = new Date(Date.UTC(2012, 11, 12, 3, 0, 0));

// toLocaleTimeString() sem argumentos, depende da implementação,
// da localidade padrão, e do fuso horário padrão
```

```
console.log(date.toLocaleTimeString());
// → "7:00:00 PM" se rodar em en-US com o fuso horário de America/Los_Angeles
```

Verificando o suporte para os argumentos <code>locales</code> e <code>options</code>

Os argumentos `locales` e `options` não são suportados em todos os browsers ainda. Para conferir se uma implementação já os suporta, você pode usar a exigência que tags ilegais de liguagem são rejeitadas com uma exceção `RangeError`:

```
function toLocaleTimeStringSupportsLocales() {
  try {
    new Date().toLocaleTimeString("i");
  } catch (e) {
    return e.name === "RangeError";
  }
  return false;
}
```

Usando <code>locales</code>

Este exemplo mostra algumas das variações em um formato de tempo localizado. Para obter o formato do idioma usado na interface do usuário da sua aplicação, tenha certeza de especificar esse idioma (e possivelmente alguns idiomas de retorno) usando o argumento `locales`:

```
var date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// Os formatos abaixo assumem o fuso horário local da região;
// America/Los_Angeles for the US

// US English usa o formato 12 horas com AM/PM
console.log(date.toLocaleTimeString("en-US"));
// → "7:00:00 PM"

// British English usa o formato 24 horas sem AM/PM
console.log(date.toLocaleTimeString("en-GB"));
// → "03:00:00"

// Korean usa o formato 12 horas com AM/PM
console.log(date.toLocaleTimeString("ko-KR"));
// → "오후 12:00:00"

// Arabic na maiorias dos países que falam árabe usa-se os dígitos arábicos reais
console.log(date.toLocaleTimeString("ar-EG"));
// → "م ٧:٠٠:٠٠"

// quando solicitar um idioma que talvez não seja suportado, como o
// Balinese, inclua um idioma de fallback, nesse caso Indonesian
console.log(date.toLocaleTimeString(["ban", "id"]));
// → "11.00.00"
```

Usando <code>options</code>

Os resultados fornecidos por `toLocaleTimeString()` podem ser customizados usando o argumento `options`:

```
var date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// uma aplicação pode querer usar UTC e tornar isso visível
var options = { timeZone: "UTC", timeZoneName: "short" };
console.log(date.toLocaleTimeString("en-US", options));
// → "3:00:00 AM GMT"

// ás vezes, até em US precise usar o formato 24 horas
console.log(date.toLocaleTimeString("en-US", { hour12: false }));
// → "19:00:00"
```

Performance

Quando se formata um grande número de datas, é aconselhável criar um objeto `Intl.DateTimeFormat` e usar a função fornecida pela propriedade dele: `format`.

Veja também

- `Intl.DateTimeFormat`
- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toLocaleString()`
- `Date.prototype.toTimeString()`
- `Date.prototype.toString()`

Date.prototype.toString()

O método **`toString()`** retorna uma cadeia de caracteres (*string*) representando o objeto `Date` especificado.

Experimente

```
const event = new Date("August 19, 1975 23:15:30");
```

```
console.log(event.toString());
// Expected output: "Tue Aug 19 1975 23:15:30 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Sintaxe

```
dateObj.toString()
```

Valor de retorno

Uma *string* representando a data.

Descrição

Instâncias de `Date` herdam o método `toString()` de `Date`, não `Object`. `Date.prototype.toString()` retorna uma *string* com a representação de Date no formato especificado na ECMA-262 que pode ser resumida como:

- Dia da semana: 3 letras do nome do dia da semana em Inglês, e.g. "Sat"
- espaço
- Nome do mês: 3 letras do nome do mês em Inglês, e.g. "Sep"
- espaço
- Dia: Dia do mês em 2 dígitos, e.g. "01"
- espaço
- Ano: Ano no formato de 4 dígitos, e.g. "2018"
- espaço
- Hora: Hora do dia em 2 dígitos, e.g. "14"
- dois-pontos
- Minuto: Minutos da hora em 2 dígitos, e.g. "53"
- dois-pontos
- Segundos: Segundos do minuto em 2 dígitos, e.g. "26"
- espaço
- A *string* "GMT"
- Sinal do deslocamento do fuso horário, pode ser:
  - "+" para deslocamentos positivos (0 ou maior)
  - "-" para deslocamentos negativos (menores que 0)
- Dois dígitos da hora do deslocamento, e.g. "14"
- Dois dígitos dos minutos do deslocamento, e.g. "00"
- Opcionalmente, um nome de fuso horário consistindo de:
  - espaço
  - parênteses esquerdo, i.e. "("
  - Uma representação de *string* dependente da implementação do fuso horário, que pode ser uma abreviação ou o nome completo (não há um padrão para nomes ou abreviação de fusos horários), e.g. "Line Islands Time" ou "LINT"
  - parênteses direito, i.e. ")"

E.g. "Sat Sep 01 2018 14:53:26 GMT+1400 (LINT)"

Até o ECMAScript 2018 (edição 9), o formato da *string* retornada pelo `Date.prototype.toString` era dependente da implementação. Portanto, não se deve confiar que ela está no formato especificado.

O método `toString()` é automaticamente chamado quando a data deve ser representada como texto, e.g. `console.log(new Date())`, ou quando a data é utilizada em uma concatenação de *string*, como `var today = 'Today is ' + new Date()`.

`toString()` é um método genérico, ele não requer que seu `this` seja uma instância de `Date`. Entretanto, ele deve ter uma propriedade interna `[[TimeValue]]` que não pode ser construída usando JavaScript nativo, então ela é efetivamente limitada a ser usada com instâncias `Date`. Se chamado em uma instância que não seja Date, um `TypeError` é jogado.

Exemplos

Usando <code>toString()</code>

O exemplo asseguir atribui o valor de `toString()` de um objeto `Date` para `myVar`:

```
var x = new Date();
var myVar = x.toString(); // atribui uma string em myVar no mesmo formato que este:
```

```
// Mon Sep 08 1998 14:36:22 GMT-0700 (PDT)
```

Veja também

- `Object.prototype.toString()`
- `Date.prototype.toDateString()`
- `Date.prototype.toLocaleString()`
- `Date.prototype.toTimeString()`

Date.prototype.toTimeString()

O método **`toTimeString()`** retorna uma porção de tempo de um objeto `Date` de forma legível para um ser humano em Inglês Americano.

Experimente

```
const event = new Date("August 19, 1975 23:15:30");

console.log(event.toTimeString());
// Expected output: "23:15:30 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Sintaxe

```
dateObj.toTimeString()
```

Valor de retorno

Uma *string* representando a porção de tempo da data de forma legível para um ser humano em Inglês Americano.

Descrição

Instâncias de `Date` referem-se a um ponto específico no tempo. Chamando `toString()` irá retornar uma data formatada de forma legível para um ser humano em Inglês Americano. No SpiderMonkey, isso consiste na porção da data (dia, mês e ano) seguido pela porção de tempo (horas, minutos, segundos e fuso horário). Algumas vezes é desejável obter a *string* da porção do tempo; tal coisa pode ser atingida utilizando o método `toTimeString()`.

O `toTimeString()` método é especialmente útil por que motores parecidos que implementam a ECMA-262 podem ter *strings* diferentes obtidas de `toString()` para objetos `Date`, como o formato é dependente de implementação; abordagens de corte simples de *strings* pode não produzir resultados consistentes entre múltiplos motores.

Exemplos

Uso básico de <code>toTimeString()</code>

```
var d = new Date(1993, 6, 28, 14, 39, 7);

console.log(d.toString()); // Wed Jul 28 1993 14:39:07 GMT-0600 (PDT)
console.log(d.toTimeString()); // 14:39:07 GMT-0600 (PDT)
```

Veja também

- `Date.prototype.toLocaleTimeString()`
- `Date.prototype.toDateString()`
- `Date.prototype.toString()`

Date.prototype.toUTCString()

O método **`toUTCString()`** converte uma data para uma cadeia de caracteres (*string*), usando o fuso horário UTC.

Baseado na RFC7231 e modificado de acordo com a ECMA-262 toUTCString, ele pode ter valores negativos na versão de 2021.

Experimente

```
const event = new Date("14 Jun 2017 00:00:00 PDT");

console.log(event.toUTCString());
// Expected output: "Wed, 14 Jun 2017 07:00:00 GMT"
```

Sintaxe

```
dateObj.toUTCString()
```

Valor de retorno

Uma *string* representando a data usando o fuso horário UTC.

Descrição

O valor retornado por `toUTCString()` é uma *string* no formato `Www, dd Mmm yyyy hh:mm:ss GMT`, onde:

Formato da String	Descrição
`Www`	Dia da semana, em três letras (e.g. Sun, Mon, ...)

Formato da String	Descrição
`dd`	Dia do mês, como dois dígitos com zero à esquerda se requisitado
`Mmm`	Mês, em três letras (e.g. Jan, Feb, ...)
`yyyy`	Ano, como 4 ou mais dígitos com zeros à esquerda se requisitado
`hh`	Hora, como dois dígitos com zero à esquerda se requisitado
`mm`	Minutos, como dois dígitos com zero à esquerda se requisitado
`ss`	Segundos, como dois dígitos com zero à esquerda se requisitado

Antes do ECMAScript 2018, o formato do valor de retorno variava de acordo com a plataforma. O valor de retorno mais comum era um carimbo de data formatada na RFC-1123, que é uma versão relativamente atualizada dos carimbos de data da RFC-822.

Exemplos

Usando <code>toUTCString()</code>

```
let today = new Date("Wed, 14 Jun 2017 00:00:00 PDT");
let UTCstring = today.toUTCString(); // Wed, 14 Jun 2017 07:00:00 GMT
```

Veja também

- `Date.prototype.toLocaleString()`
- `Date.prototype.toDateString()`
- `Date.prototype.toISOString()`

Date.UTC()

O método **`Date.UTC()`** aceita os mesmos parâmetros que o construtor mais longo de **`Date`** e retorna o total de milisegundos desde 1º de Janeiro de 1970 às 00:00:00 (horário universal). O valor retornado por **`Date.UTC()`** pode ser usado como parâmetro para criar uma instância de `Date`.

Sintaxe

```
Date.UTC(ano, mês[, dia[, hora[, minuto[, segundo[, milisegundo]]]]])
```

Parâmetros

`ano`

Um valor inteiro representando o ano, após 1970.

`mês`

Um valor inteiro que representa o mês, começando com 0 para Janeiro até 11 para Dezembro.

`dia`

Opcional. Um valor inteiro entre 1 e 31 representando o dia do mês.

`hora`

Opcional. Um valor inteiro entre 0 e 23 representando a hora do dia.

`minuto`

Opcional. Um valor entre 0 e 59 representando os minutos.

`segundo`

Opcional. Um valor entre 0 e 59 representando os segundos.

`milisegundo`

Opcional. Um valor entre 0 e 999 representando os milisegundos.

Descrição

`UTC()` recebe argumentos de uma data separados por vírgula e retorna um número representando o total de milisegundos entre o dia 1º de Janeiro de 1970 às 00:00:00 (horário universal) e a data e hora que você especificou.

Você deve especificar o ano completo para o argumento ano. Por exemplo, 1998. Se o ano fornecido for um valor entre 0 e 99 o método irá converter este valor para o século 20 (1900 + ano); Por exemplo, se você utilizar 95, então o ano 1995 será utilizado.

O método `UTC()` se diferencia do construtor de `Date` de duas maneiras:

- `Date.UTC()` utiliza o horário universal em vez do horário local.
- `Date.UTC()` retorna um valor inteiro em vez de uma instância de `Date`.

Se você fornecer um argumento fora do intervalo esperado, o método UTC() atualiza os demais argumentos para permitir este valor. Por exemplo, se você utilizar 15 para mês, será adicionado 1 ao argumento ano (ano + 1) e será utilizado 3 para o argumento mês.

`UTC()` é um método estático, por conta disto você sempre irá chamar `Date.UTC()` em vez de chamar um método de um objeto `Date` que você tenha criado.

Exemplos

Utilizando <code>Date.UTC()</code>

A seguinte expressão cria uma instância de `Date` utilizando UTC em vez do horário local:

```
var dataUniversal = new Date(Date.UTC(96, 11, 1, 0, 0, 0));
```

Veja Também

- `Date.parse()`

Date.prototype.valueOf()

O método **`valueOf()`** retorna o valor primitivo do objeto `Date`.

Experimente

```
const date1 = new Date(Date.UTC(96, 1, 2, 3, 4, 5));

console.log(date1.valueOf());
// Expected output: 823230245000

const date2 = new Date("02 Feb 1996 03:04:05 GMT");

console.log(date2.valueOf());
// Expected output: 823230245000
```

Sintaxe

```
dateObj.valueOf()
```

Valor de retorno

O número de milissegundos entre 1 de Janeiro de 1970 00:00:00 UTC e a data.

Descrição

O método `valueOf()` retorna o valor primitivo do objeto `Date` como um tipo de dado numérico, o número de milissegundos desde a meia noite do dia 1 de Janeiro de 1970 UTC.

Este método é funcionalmente equivalente ao método `Date.prototype.getTime()`.

Este método é geralmente chamado internamente pelo JavaScript e não explícito em código.

Exemplos

Usando <code>valueOf()</code>

```
var x = new Date(56, 6, 17);
var myVar = x.valueOf(); // atribui -424713600000 to myVar
```

Veja também

- `Object.prototype.valueOf()`
- `Date.prototype.getTime()`

decodeURI()

A função **`decodeURI()`** decodifica um Uniform Resource Identifier (URI) criado anteriormente por `encodeURI()` ou por uma rotina semelhante.

Experimente

```
const uri = "https://mozilla.org/?x=шеллы";
const encoded = encodeURI(uri);
console.log(encoded);
// Expected output: "https://mozilla.org/?x=%D1%88%D0%B5%D0%BB%D0%BB%D1%8B"

try {
  console.log(decodeURI(encoded));
  // Expected output: "https://mozilla.org/?x=шеллы"
} catch (e) {
  // Catches a malformed URI
  console.error(e);
}
```

Sintaxe

```
decodeURI(encodedURI)
```

Parâmetros

`encodedURI`

Um identificador de recurso uniforme codificado completo.

Valor retornado

Uma nova string representando a versão não codificada do URI (Uniform Resource Identifier) codificado fornecido.

Exceções

Lança uma exceção `URIError` ("sequência de URI malformada") quando o `encodedURI` contém sequências de caracteres inválidos.

Descrição

Substitui cada sequência de escape no URI codificado pelo caractere que ele representa, mas não decodifica sequências de escape que não poderiam ter sido introduzidas por `encodeURI`. O caractere "#" não é decodificado a partir de sequência de escape.

Exemplos

Decodificando uma URL Cyrillic

```
decodeURI(
  "https://developer.mozilla.org/ru/docs/JavaScript_%D1%88%D0%B5%D0%BB%D0%BB%D1%8B",
);
// "https://developer.mozilla.org/ru/docs/JavaScript_шеллы"
```

Captura de erros

```
try {
  var a = decodeURI('%E0%A4%A');
} catch(e) {
  console.error(e);
}

// URIError: sequência de URI malformada
```

Ver também

- `decodeURIComponent()`
- `encodeURI()`
- `encodeURIComponent()`

decodeURIComponent()

O método **`decodeURIComponent()`** decodifica um componente Identificador Uniforme de Recursos (URI) criado anteriormente por um `encodeURIComponent` ou por uma rotina semelhante.

Sintaxe

```
decodeURIComponent(encodedURI)
```

Parâmetros

`encodedURI`

Um componente codificado de um Identificador Uniforme de Recursos.

Descrição

Substitui cada sequência escapada no componente URI codificado com o caractere que a representa.

Exemplos

Decodificando um componente URL Cirílico

```
decodeURIComponent("JavaScript_%D1%88%D0%B5%D0%BB%D0%BB%D1%8B");
// "JavaScript_шеллы"
```

Veja também

- `decodeURI`
- `encodeURI`
- `encodeURIComponent`

encodeURI()

A função **`encodeURI()`** codifica a URI substituindo cada instância de certos caracteres por um, dois, três ou quatro sequências de escape representando a codificação UTF-8 do caracter (será somente quatro sequências de escape para caracteres compostos de dois caracteres substitutos).

Experimente

```
const uri = "https://mozilla.org/?x=шеллы";
const encoded = encodeURI(uri);
console.log(encoded);
// Expected output: "https://mozilla.org/?x=%D1%88%D0%B5%D0%BB%D0%BB%D1%8B"

try {
  console.log(decodeURI(encoded));
  // Expected output: "https://mozilla.org/?x=шеллы"
} catch (e) {
  // Catches a malformed URI
  console.error(e);
}
```

Sintaxe

```
encodeURI(URI)
```

Parâmetros

`URI`

Uma URI completa.

Valor de retorno

Uma nova cadeia de caracteres representando a cadeia de caracteres provida, codificada como uma URI.

Descrição

A função `encodeURI()` não codifica caracteres que possuem significado especial (caracteres reservados) para a URI. O exemplo seguinte mostra todas as partes que o "esquema" da URI pode possivelmente conter. Note como certos caracteres são usados para dar significado especial:

```
http://username:password@www.example.com:80/path/to/file.php?foo=316&bar=this+has+spaces#anchor
```

Consequentemente, `encodeURI()` **não** codifica caracteres que são necessários para formular uma URI completa. Também, `encodeURI()` **não** codifica alguns caracteres adicionais, conhecidos como "marcas não reservadas (unreserved marks)", que não tem propósito reservado mas são permitidos na URI "como são". (Veja RFC2396)

`encodeURI()` escapa todos os caracteres **exceto**:

```
Não escapado:

    A-Z a-z 0-9 ; , / ? : @ & = + $ - _ . ! ~ * ' ( ) #
```

Exemplos

encodeURI vs encodeURIComponent

`encodeURI()` se difere de `encodeURIComponent()` como a seguir:

```
var set1 = ";,/?:@&=+$#"; // Caracteres reservados
var set2 = "-_.!~*'()"; // Marcas não reservadas
var set3 = "ABC abc 123"; // Caracteres alfanuméricos + Espaço

console.log(encodeURI(set1)); // ;,/?:@&=+$#
console.log(encodeURI(set2)); // -_.!~*'()
console.log(encodeURI(set3)); // ABC%20abc%20123 (o espaço é codificado como %20)

console.log(encodeURIComponent(set1)); // %3B%2C%2F%3F%3A%40%26%3D%2B%24%23
console.log(encodeURIComponent(set2)); // -_.!~*'()
console.log(encodeURIComponent(set3)); // ABC%20abc%20123 (o espaço é codificado como %20)
```

Note que `encodeURI()` por si só não pode formar requisições `GET` e `POST`, como para `XMLHttpRequest`, porque "&", "+", e "=" não são codificados, que são tratados como caracteres especiais em requisições `GET` e `POST`. `encodeURIComponent()`, entretanto, codifica esses caracteres.

Codificando um substituto solitário alto joga

Um `URIError` será jogado se uma tentativa de codificar um substituto que não é parte de um par alto-baixo, e.g.,

```
// par alto-baixo ok
console.log(encodeURI("\uD800\uDFFF"));

// substituto alto solitário joga "URIError: malformed URI sequence"
console.log(encodeURI("\uD800"));

// substituto baixo solitário joga "URIError: malformed URI sequence"
console.log(encodeURI("\uDFFF"));
```

Codificando para IPv6

Se você deseja seguir a RFC3986 mais recente para URLs, que faz colchetes ser reservado (para IPv6) e então não será codificado quando formando algo que possa ser parte da URL (como o host), o seguinte código pode ajudar:

```
function fixedEncodeURI(str) {
  return encodeURI(str).replace(/%5B/g, "[").replace(/%5D/g, "]");
}
```

Veja também

- `decodeURI()`
- `encodeURIComponent()`
- `decodeURIComponent()`

encodeURIComponent()

O método **`encodeURIComponent()`** codifica uma URI (Identificador recurso uniforme) substituindo cada ocorrência de determinados caracteres por um, dois, três, ou quatro seqüências de escape que representam a codificação UTF-8 do caractere (só será quatro seqüências de escape para caracteres compostos por dois caracteres "substituto").

Syntaxe

```
encodeURIComponent(str);
```

Parâmetros

`str`

String. Uma sequência URI.

Descrição

`encodeURIComponent` escapa todos os caracteres, exceto: alfabeto, dígitos decimais, `- _ . ! ~ * ' ( )`

Note-se que um`URIError` será gerada se uma tentativas para codificar um substituto que não faz parte de um par de alta-baixa, por exemplo,

```
// high-low par ok
console.log(encodeURIComponent("\uD800\uDFFF"));

// lone high surrogate throws "URIError: malformed URI sequence"
console.log(encodeURIComponent("\uD800"));

// lone low surrogate throws "URIError: malformed URI sequence"
console.log(encodeURIComponent("\uDFFF"));
```

Para previnir requisões inesperadas ao servidor, deve-se chamar `encodeURIComponent` ou qualquer parâmetro fornecido pelo usuário que será passado como parte da URI. Por exemplo, um usuário poderia digitar "`Thyme &time=again`" para uma variável `commentario`. Ao não usar `encodeURIComponent` nessa variável irá ser obetido `commentario=Thyme%20&time=again`. Note que o ampersa e o sinal de igual marcam um novo par de chave e valor. Então ao invés de ter um POST com a chave `commentario` igual a "`Thyme &time=again`", tem-se chaves em POST, uma igual a "`Thyme`" e outra (`time`) igual a `again`.

Para `application/x-www-form-urlencoded`, espaços são substituídos por '+', então pode-se querer seguir um `encodeURIComponent` substituição com uma substituição adicional de "%20" com "+".

Para ser mais rigoroso à aderência da RFC 3986 (qual reserva !, ', (,), e *), mesmo que esses caracteres não tenham usos formalizados de delimitação de URI, o seguinte pode ser usado com segurança:

```
function ajustadoEncodeURIComponent(str) {
  return encodeURIComponent(str).replace(/[!'()*]/g, function (c) {
    return "%" + c.charCodeAt(0).toString(16);
  });
}
```

Exemplos

O exemplo seguinte provê o encoding especial requerido pelo UTF-8 nos parâmetros `Content-Disposition` e `Link` no cabeçalho de uma Response (e.g., UTF-8 filenames):

```
var fileName = "my file(2).txt";
var header =
  "Content-Disposition: attachment; filename*=UTF-8''" +
  encodeRFC5987ValueChars(fileName);

console.log(header);
// logs "Content-Disposition: attachment; filename*=UTF-8''my%20file%282%29.txt"

function encodeRFC5987ValueChars(str) {
  return (
    encodeURIComponent(str)
      // Note that although RFC3986 reserves "!", RFC5987 does not,
      // so we do not need to escape it
      .replace(/['()]/g, escape) // i.e., %27 %28 %29
      .replace(/\*/g, "%2A")
      // The following are not required for percent-encoding per RFC5987,
      // so we can allow for a little better readability over the wire: |`^
      .replace(/%(?:7C|60|5E)/g, unescape)
```

```
  );
}
```

Veja também

- `decodeURI`
- `encodeURI`
- `decodeURIComponent`

Error

O construtor de **`Error`** cria um objeto de erro. Instâncias de objetos `Error` são lançadas quando erros de tempo de execução ocorrem. O objeto `Error` também pode ser usado como objeto base para exceções definidas pelo usuário. Veja abaixo tipos de erro padrões embutidos.

Sintaxe

```
new Error([message[, fileName[, lineNumber]]])
```

Parâmetros

`message`

Opcional. Descrição do erro legível para humanos.

`fileName`

Opcional. O valor da propriedade `fileName` no objeto de erro criado. O padrão é o nome do arquivo contendo o código que chamou o construtor de `Error()`.

`lineNumber`

Opcional. O valor da propriedade `lineNumber` no objeto de `Error` criado. O padrão é o número da linha contendo a invocação do construtor `Error()`.

Descrição

Erros em tempo de execução resultam em novos objetos `Error` sendo criados e lançados.

Esta página documenta o uso do objeto `Error` em si e seu uso como uma função construtora. Para uma lista de propriedades e métodos herdados por instâncias de `Error`, veja `Error`.

Tipos de erro

Além do construtor genérico de `Error`, existem outros seis construtores principais de erro no JavaScript. Para exceções em *client-side*, veja Exceções na captura de Instruções.

`EvalError`

Cria uma instância representando um erro que ocorre na função global. `eval()`.

`InternalError`

Cria uma instância representando um erro que ocorre quando um erro interno na *engine* do JavaScript é lançado. Ex: "too much recursion".

`RangeError`

Cria uma instância representando um erro que ocorre quando um valor ou parâmetro numérico está fora de seus limites válidos.

`ReferenceError`

Cria uma instância representando um erro que ocorre ao de-referenciar uma referência inválida.

`SyntaxError`

Cria uma instância representando um erro que ocorre ao fazer o *parse* do código em `eval()`.

`TypeError`

Cria uma instância representando um erro que ocorre quando uma variável ou parâmetro não é de um tipo válido.

`URIError`

Cria uma instância representando um erro que ocorre quando são passados parâmetros inválidos para `encodeURI()` ou `decodeURI()`.

Propriedades

`Error`

Permite a criação de propriedades para instâncias de `Error`.

Métodos

O objeto `Error` global não contém métodos próprios, entretanto, ele herda alguns métodos através da cadeia de *prototypes*.

<code>Instâncias de Error</code>

Propriedades

Métodos

Exemplos

Lançando um erro genérico

Geralmente você cria um objeto `Error` com a intenção de lançá-lo usando a palavra-chave `throw`. Você pode capturar o erro usando uma construção de `try...catch`:

```
try {
  throw new Error("Oooops!");
} catch (e) {
  alert(e.name + ": " + e.message);
}
```

Capturando um erro específico

Você pode escolher por capturar apenas tipos de erro específicos testando o tipo do erro com a propriedade `constructor` de erro ou, se você está escrevendo para *engines* de JavaScript modernas, a palavra-chave `instanceof`:

```
try {
  Objeto.Metodo();
} catch (e) {
  if (e instanceof EvalError) {
    alert(e.name + ": " + e.message);
  } else if (e instanceof RangeError) {
    alert(e.name + ": " + e.message);
  }
  // ... etc
}
```

Tipos de erro customizados

Você pode escolher definir seus próprios tipos de erro derivando de `Error` para conseguir usar `throw new MeuErro()` e usar `instanceof MeuErro` para checar o tipo de erro na captura da exceção. A forma comum para isso está demonstrada abaixo

> **Aviso:** Note que as instâncias `MeuErro` lançadas vão reportar valores de `lineNumber` e `fileName` incorretos, ao menos no Firefox.

Veja também "esta discussão no Stackoverflow (em inglês): What's a good way to extend Error in JavaScript?".

```
// Cria um novo objeto que herda o construtor de Error através do prototype.
function MeuErro(message) {
  this.name = "MeuErro";
  this.message = message || "Mensagem de erro padrão";
  this.stack = new Error().stack;
}
MeuErro.prototype = Object.create(MeuErro.prototype);
MeuErro.prototype.constructor = MeuErro;

try {
  throw new MeuErro();
} catch (e) {
  console.log(e.name); // 'MeuErro'
  console.log(e.message); // 'Mensagem de erro padrão'
}

try {
  throw new MeuErro("Mensagem customizada");
} catch (e) {
  console.log(e.name); // 'MeuErro'
  console.log(e.message); // 'Mensagem customizada'
}
```

Veja também

- `Error`
- `throw`
- `try...catch`

Error.prototype.cause

A propriedade **`cause`** indica a causa original específica de um erro.

É usado quando captura e relança um erro com uma mensagem mais específica ou útil para ter acesso ao erro original.

Valor

Este é o valor que foi passado para o construtor `Error()` no argumento `options.cause`.

O valor pode ser de qualquer tipo. Você não deve criar suposições que o erro que você apanhou tem um `Error` como sua `cause`, e da mesma forma você não pode ter certeza que a variável vinculada na declaração `catch` e um `Error` qualquer. O exemplo abaixo "Forncenendo dados estruturados é a causa do erro" mostra uma casa onde um não erro é deliberadamente fornecido como causa

Exemplos

Relançando um erro com a causa

Isso é útil algumas vezes para capturar um erro e relançá-lo com uma nova mensagem. E nesse caso você deve passar o erro original no construtor para o novo `Error` aparecer.

```
try {
  connectToDatabase();
} catch (err) {
  throw new Error("Falha na comunicação com o banco de dados.", { cause: err });
}
```

Para exemplos mais detalhados veja Erro > Diferenciar entre erros semelhantes.

Fornecendo dados estruturados como a causa de erro

Mensagens de erro escritas para o humano consumir pode ser inapropriado para a análise de máquina, já que estão sujeitos a mudanças de reformulação ou pontuação ele pode parar qualquer análise de escrita existente para consumi-los. Então quando um erro é disparado de uma função, é uma alternativa para a leitura humana da mensagem de erro, você pode em vez disso fornecer a causa com dados estruturados, para análise de máquina.

```
function makeRSA(p, q) {
  if (!Number.isInteger(p) || !Number.isInteger(q)) {
    throw new Error("RSA key generation requires integer inputs.", {
      cause: { code: "NonInteger", value: [p, q] },
    });
  }
  if (!areCoprime(p, q)) {
    throw new Error("RSA key generation requires two co-prime integers.", {
      cause: { code: "NonCoprime", values: [p, q] },
    });
  }
  // rsa algorithm…
}
```

Veja também

- `Error.prototype.message`
- `Error.prototype.toString()`

Error.prototype.message

A propriedade **`message`** é a descrição legível para humanos de um erro.

Descrição

Esta propriedade contém uma breve descrição do erro se ele estiver disponível ou foi definido. A propriedade `message` combinada com a propriedade `name` é usada pelo metodo `Error.prototype.toString()` para criar a string de representação do erro.

Por padrão, a propriedade `message` é uma string vazia, mas esse comportamento pode ser substituído por uma instância especificando uma mensagem como o primeiro argumento para o construtor `Error`.

Exemplos

Disparando um erro personalizado

```
const e = new Error("Não foi possível analisar a entrada");
// e.message é 'Não foi possível analisar a entrada'
throw e;
```

Veja também

- `Error.prototype.name`
- `Error.prototype.toString()`

Error.prototype.name

A propriedade **`name`** representa o nome do tipo do erro. O valor inicial é "Error".

Descrição

Por padrão, instâncias `Error` recebem o nome "Error". A propriedade `name`, em adição na propriedade `message`, é usado pelo método `Error.prototype.toString()` para criar a string de representação do erro.

Exemplos

Disparando um erro personalizado

```
const e = new Error("Malformed input"); // e.name é 'Error'

e.name = "ParseError";
```

```
throw e;
// e.toString() vai retornar 'ParseError: Malformed input'
```

Veja também

- `Error.prototype.message`
- `Error.prototype.toString()`

Error.prototype.toString()

O método **`toString()`** retorna uma string representando o objeto `Error`.

Sintaxe

```
e.toString()
```

Valor de retorno

A string representando o objeto `Error` especificado.

Descrição

O objeto `Error` sobrescreve o método `Object.prototype.toString()` herdado por todos os objetos. Sua semântica é a seguinte (assumindo que `Object` e `String` tem seus valores originais):

```
Error.prototype.toString = function () {
  "use strict";

  var obj = Object(this);
  if (obj !== this) {
    throw new TypeError();
  }

  var name = this.name;
  name = name === undefined ? "Error" : String(name);

  var msg = this.message;
  msg = msg === undefined ? "" : String(msg);

  if (name === "") {
    return msg;
  }
  if (msg === "") {
    return name;
  }

  return name + ": " + msg;
};
```

Exemplos

Usando toString()

```
var e = new Error("fatal error");
console.log(e.toString()); // 'Error: fatal error'

e.name = undefined;
console.log(e.toString()); // 'Error: fatal error'

e.name = "";
console.log(e.toString()); // 'fatal error'

e.message = undefined;
console.log(e.toString()); // ''

e.name = "hello";
console.log(e.toString()); // 'hello'
```

Veja também

- `Error.prototype.toSource()`

escape()

A função obsoleta `escape() retorna uma nova string com` certos caracteres substituídos por sua sequência hexadecial. Use `encodeURI` ou `encodeURIComponent` em seu lugar.

Sintaxe

```
escape(str)
```

Parâmetros

`str`

Uma string para ser codificada.

Descrição

A função **`escape`** é uma propriedade do *global object*. Caracteres especiais são codificados, com a exceção de: `@*_+-./`

O formato hexadecimal de caracteres, que o valor unitário do código é **0xFF or less,** é uma sequência de escape de 2 digitos: %xx. Para caracteres com um código unitário maior, uma sequência de 4 digitos %**u**xxxx é usada.

Exemplos

```
escape("abc123"); // "abc123"
escape("äöü"); // "%E4%F6%FC"
escape("ć"); // "%u0107"

// Caracteres Especiais
escape("@*_+-./"); // "@*_+-./"
```

Veja Também

- `encodeURI`
- `encodeURIComponent`

eval()

Resumo

A função **`eval()`** computa um código JavaScript representado como uma string.

Sintaxe

```
eval(string);
```

Parâmetros

`string`

Uma sequência de caracteres que representa uma expressão JavaScript, declaração, ou sequência de declarações. A expressão pode incluir variáveis e propriedades de objetos existentes.

Retorno

É o valor computado do código passado para a função. Se o valor estiver vazio, retornará `undefined`.

Descrição

`eval() é uma função de propriedade do objeto global (window).`

O argumento da função `eval()` é uma string. Se a string representa uma expressão, `eval()` avalia a expressão. Se o argumento representa uma ou mais declarações em JavaScript, `eval()` avalia as declarações. Não chame o `eval()` para avaliar uma expressão aritmética; JavaScript avalia expressões aritméticas automaticamente.

Se você construir uma expressão aritmética como uma string, você pode usar `eval()` para calcular o resultado depois. Por exemplo, suponha que você tenha uma variável x. Você pode adiar a avaliação de uma expressão envolvendo x atribuindo o valor de string da expressão, dizer `"3 * x + 2"`, a uma variável, e, em seguida, chamando `eval()` em um ponto posterior no seu script.

Se o argumento de `eval()` não é uma string, `eval()` retorna o argumento inalterado. No exemplo a seguir, o construtor `String` é especificado, e `eval()` retorna um objeto `String` em vez de avaliar a string.

```
eval(new String("2 + 2")); // retorna um objeto String contendo "2 + 2"
eval("2 + 2"); // retorna 4
```

Você pode contornar esta limitação de forma genérica usando `toString()`.

```
var expression = new String("2 + 2");
eval(expression.toString()); // retorna 4
```

Se você usar a função `eval` *indiretamente*, invocando-a por outra referência além de `eval`, a partir do ECMAScript 5 funciona no escopo global ao invés do escopo local. Significa que, por exemplo, aquelas declarações de funções criam funções globais e que o código que está sendo avaliado não tem acesso às variáveis locais dentro do escopo onde está sendo chamada.

```
function test() {
  var x = 2,
    y = 4;
  console.log(eval("x + y")); // Chamada direta, usa o escopo local, resulta em 6
  var geval = eval; // equivalente a chamar eval no escopo global
  console.log(geval("x + y")); // Chamada indireta, usa o escopo global, lança uma exceção ReferenceError porque `x` não foi declarado
  (0, eval)("x + y"); // outro exemplo de chamada indireta
}
```

Nunca use <code>eval</code>!

`eval()` é uma função perigosa, que executa o código passado com os privilégios do caller. Se você executar o `eval()` com uma sequência de caracteres que podem ser afetados por uma parte maliciosa, você pode acabar executando código malicioso na máquina do usuário com as permissões da sua página/extensão. Mais importante ainda, o código de terceiros pode ver o escopo em que `eval()` foi chamado, o que pode levar a possíveis ataques como `Function` não é suscetível.

`eval()` é geralmente mais lento do que as alternativas, uma vez que tem de invocar o interpretador JS, enquanto muitos outros construtores são otimizados por *engines* de JS modernos.

Os interpretadores modernos convertem javascript em código de máquina. Resumindo, o nome das variáveis será desmanchado. Portanto, o uso de `eval` forçará o navegador a fazer buscas caras para descobrir o endereço e seu valor no código de máquina. Além disso, novos valores podem ser introduzidos em `eval` como mudanças no tipo da variável, fazendo o navegador recalcular todo o código de máquina gerado anteriormente. Felizmente, existem alternativas mais seguras (e rápidas) ao `eval()` para usos comuns.

Acessando propriedades dos membros

Você não deve utilizar `eval()` para converter nomes de propriedades em propriedades. Considere o seguinte exemplo onde as propriedades do objeto a ser acessado não são conhecidas até o código ser executado. Isso pode ser feito com eval:

```
var obj = { a: 20, b: 30 };
var propname = getPropName(); //retorna "a" ou "b"

eval("var result = obj." + propname);
```

No entanto, `eval()` não é necessário aqui. De fato, sua utilização não é recomendada. Ao invés disso, utilize os operadores de acesso, que são mais rápidos e seguros:

```
var obj = { a: 20, b: 30 };
var propname = getPropName(); //retorna "a" ou "b"
var result = obj[propname]; //  obj[ "a" ] é o mesmo como obj.a
```

Utilize funções ao invés de avaliar snippets de código

JavaScript possui first-class functions, o que significa que você pode passar os argumentos para outras APIs, armazená-los em variáveis e propriedades de objeto, e assim por diante. Muitas APIs de DOM foram desenvolvidas com isso em mente, então você pode (e deve) escrever:

```
// ao invés de setTimeout(" ... ", 1000) use:
setTimeout(function() { ... }, 1000);

// ao invés de elt.setAttribute("onclick", "...") use:
elt.addEventListener("click", function() { ... } , false);
```

Closures são úteis como forma de criar funcções parametrizáveis sem concatenar strings.

Analisando JSON (convertendo string para objetos JavaScript)

Se a string para a qual você está chamando o `eval()` contém dados (por exemplo, um array: `"[1, 2, 3]"`), ao invés de código, você deve considerar mudar para JSON, que permite a string usar um subconjunto de sintaxe Javascript para representar dados. Veja também Downloading JSON and JavaScript in extensions.

Perceba que como a sintaxe JSON é limitada comparada com a sintaxe JavaScript, muitos literais JavaScript válidos não serão analisados como JSON. Por exemplo, trailing commas não são permitidas em JSON, e nomes de propriedades (keys) em literais de objetos devem ser colocados entre aspas. Certifique-se de usar um serializados JSON para gerar strings que serão analisadas como JSON mais tarde.

Passar dados em vez de códigos

Por exemplo, uma extensão concebida para raspar conteúdos de páginas web pode ter as regras de raspagem definidas no XPath em vez de código JavaScript.

Rodando o código com privilégios limitados

Se você precisa executar o código, considere executá-lo com privilégios limitados. Esse conselho se aplica principalmente para extensões e aplicações XUL, que podem usar Components.utils.evalInSandbox para obter o resultado.

Exemplos

Os exemplos a seguir mostram o retorno do `document.write`. No JavaScript rodando no server-side, você pode obter o mesmo resultado chamando o método `write()ao invés` de usar o `document.write()`.

Exemplo: Usando <code>eval</code>

No código a seguir, ambas as declarações contendo `eval()` retornam 42. A primeira avalia a string `"x + y + 1"`; a segunda avalia a string `"42"`.

```
var x = 2;
var y = 39;
var z = "42";
eval("x + y + 1"); // returns 42
eval(z); // returns 42
```

Exemplo: Using <code>eval</code> to evaluate a string of JavaScript statements

O exemplo a seguir usa `eval()` para avaliar a string `str`. Essa string consiste de instruções JavaScript que abrem uma caixa de diálogo de alerta e atribuem ao `z` o valor de 42 se `x` for cinco, e do contrário, atribui 0 a `z`. Quando a segunda instrução é executada, `eval()` fará com que essas instruções sejam executadas e também irá avaliar o conjunto de instruções e retornará o valor atribuído a z.

```
var x = 5;
var str = "if (x == 5) {alert('z is 42'); z = 42;} else z = 0; ";

document.write("<P>z is ", eval(str));
```

Exemplo: A última expressão é avaliada

`eval()` retorna o valor da última expressão avaliada.

```
var str = "if ( a ) { 1+1; } else { 1+2; }";
var a = true;
var b = eval(str); // returns 2

alert("b is : " + b);

a = false;
b = eval(str); // returns 3

alert("b is : " + b);
```

Exemplo: avaliar uma string definindo a função necessária "(" and ")" como prefixo e sufixo

```
var fctStr1 = "function a() {}";
var fctStr2 = "(function a() {})";
var fct1 = eval(fctStr1); // return undefined
var fct2 = eval(fctStr2); // return a function
```

Veja também

- `uneval()`
- Member operators

EvalError

O Objeto **`EvalError`** indica um erro em relação a função global `eval()`. Esta exceção não é mais lançada pelo JavaScript, no entanto, o objeto `EvalError` ainda permanece compatível.

Sintaxe

```
new EvalError([message[, fileName[, lineNumber]]])
```

Parâmetros

`message`

Opcional. Descrição do erro legível para humanos

`fileName`

Opcional. O nome do arquivo que contém o código que causa a exceção

`lineNumber`

Opcional. O número da linha do código que causa a exceção

Propriedades

`EvalError`

Permite a adição de propriedades para um objeto `EvalError`.

Métodos

O global `EvalError` não contém métodos próprios, no entando, ele irá herdar alguns métodos através da cadeia de protótipos.

<code>Instâncias do EvalError</code>

Propriedades

Métodos

Exemplos

`EvalError` não é usado na especificação ECMAScript e deste modo não será lançado pelo tempo de execução. No entando, o objeto em si permanece para compatibilidade com versões anteriores da especificação.

Criando um <code>EvalError</code>

```
try {
  throw new EvalError("Hello", "someFile.js", 10);
} catch (e) {
  console.log(e instanceof EvalError); // true
  console.log(e.message); // "Hello"
```

```
  console.log(e.name); // "EvalError"
  console.log(e.fileName); // "someFile.js"
  console.log(e.lineNumber); // 10
  console.log(e.columnNumber); // 0
  console.log(e.stack); // "@Scratchpad/2:2:9\n"
}
```

Veja também

- `Error`
- `EvalError`
- `eval()`

Float32Array

A array tipada **`Float32Array`** representa uma array de 32-bits contendo floats (correspondendo ao tipo de dados `float` em C) na ordem dos bytes da plataforma. Se o controle sobre a ordem dos bytes for precisa, use `DataView`. Os conteúdos iniciam em `0`. Uma vez estabelecido, você pode referenciar os elementos na array usando os métodos do objeto ou usando a síntaxe padrão das arrays (Isto é, usando colchetes)

Síntaxe

```
new Float32Array(); // novo no ES2017
new Float32Array(length);
new Float32Array(typedArray);
new Float32Array(object);
new Float32Array(buffer [, byteOffset [, length]]);
```

Para mais informações sobre a síntaxe de construção e os parâmetros, veja *TypedArray*.

Propriedades

`Float32Array.BYTES_PER_ELEMENT`

Retorna um número valor do tamanho do elemento. `4` no caso de uma `Float32Array`.

Float32Array.length

Método estático cujo o valor é `0`. Para o tamanho atual (número de elementos), veja `Float32Array.prototype.length`.

`Float32Array.name`

Retorna o valor em string do nome do constructor. No caso de `Float32Array` é "Float32Array".

`Float32Array.prototype`

Prototype para os objetos *TypedArray*.

Métodos

`Float32Array.from()`

Cria uma nova `Float32Array` de um array-like ou um objeto iterável. Veja também `Array.from()`.

`Float32Array.of()`

Cria uma nova `Float32Array` with um número variável de argumentos. Veja também `Array.of()`.

<code>Float32Array</code> prototype

Todos os objetos `Float32Array` herdam de `%TypedArray%.prototype`.

Propriedades

`Float32Array.prototype.constructor`

Retorna a função cujo criou o protótipo da instância. Este é o construtor padrão da Array `Float32Array`.

`Float32Array.prototype.buffer` Somente leitura

Retorna os `ArrayBuffer` referenciado pela Array `Float32Array` fixada na hora de sua construção e é **somente leitura**.

`Float32Array.prototype.byteLength` Somente leitura

Retorna o tamanho (em bytes) da Array `Float32Array` do começo de seu `ArrayBuffer`. Fixado na hora de sua construção e é **somente leitura**.

`Float32Array.prototype.byteOffset` Somente leitura

Retorna o deslocamento (em bytes) da Array `Float32Array` do começo de seu `ArrayBuffer`. Fixado na hora de sua construção e é **somente leitura**.

`Float32Array.prototype.length` Somente leitura

Retorna o número de elementos na Array `Float32Array` do começo de seu `ArrayBuffer`. Fixado na hora de sua construção e é **somente leitura**.

Métodos

`Float32Array.prototype.copyWithin()`

Copia a sequência dos elementos dentro da Array. Veja também `Array.prototype.copyWithin()`.

`Float32Array.prototype.entries()`

Retorna um novo objeto `Array Iterator` que contém os pares chave/valor para cada índex na array. Veja também `Array.prototype.entries()`.

`Float32Array.prototype.every()`

Testa quando todos os elementos na array passam no teste proveniente de uma função. Veja também `Array.prototype.every()`.

`Float32Array.prototype.fill()`

Preenche todos os elementos de uma array do índex inicial ao índex final com um valor estático. Veja também `Array.prototype.fill()`.

`Float32Array.prototype.filter()`

Cria uma nova array com todos os elementos dessa array para a função de filtragem que retornar `true`. Veja também `Array.prototype.filter()`.

`Float32Array.prototype.find()`

Retorna o valor na array encontrado, se um elemento na array satizfaz o teste proveniente de uma função de teste ou `undefined` se não for encontrado. Veja também `Array.prototype.find()`.

`Float32Array.prototype.findIndex()`

Retorna o index encontrado na array, se um elemento na array satizfaz o teste proveniente de uma função de teste ou -1 se não for encontrado. Veja também `Array.prototype.findIndex()`.

`Float32Array.prototype.forEach()`

Chama uma função para cada elemento na array. Veja também `Array.prototype.forEach()`.

`Float32Array.prototype.includes()`

Determina quando a array tipada inclui um certo elemento, retornando `true` ou `false` como apropriado. Veja também `Array.prototype.includes()`.

`Float32Array.prototype.indexOf()`

Retorna o primeiro (menor) index de um elemento dentro de uma array igual ao valor expecificado, ou -1 se nenhum for encontrado. Veja também `Array.prototype.indexOf()`.

`Float32Array.prototype.join()`

Junta todos os elementos de uma array em uma string. Veja também `Array.prototype.join()`.

`Float32Array.prototype.keys()`

Retorna um novo `Array Iterator` cujo contem as chaves para cada index na array. Veja também `Array.prototype.keys()`.

`Float32Array.prototype.lastIndexOf()`

Retorna o último (maior) índex de um elemento dentro da array igual ao valor especificado, ou -1 se nenhum for encontrado. Veja também `Array.prototype.lastIndexOf()`.

`Float32Array.prototype.map()`

Cria uma nova array com os resultados da função chamada em cada elemento nesta array. Veja também `Array.prototype.map()`.

`Float32Array.prototype.move()`

Versão antiga não padrão de `Float32Array.prototype.copyWithin()`.

`Float32Array.prototype.reduce()`

Aplica uma função contra um acumulador e cada valor na array (da esquerda para a direita) com o intuito de reduzí-la a um único valor. Veja também `Array.prototype.reduce()`.

`Float32Array.prototype.reduceRight()`

Aplica uma função contra um acumulador e cada valor na array (da direita para a esquerda) como o intuito de reduzí-la a um único valor. See also `Array.prototype.reduceRight()`.

`Float32Array.prototype.reverse()`

Reverte a ordem se todos os elementos em uma array - o primeiro se torna o último, e o último de torna o primeiro. Veja também `Array.prototype.reverse()`.

`Float32Array.prototype.set()`

Armazena múltiplos valores na array tipada, lendo os valores de entrada de uma array especificada.

`Float32Array.prototype.slice()`

Extrai uma seção de uma array e retorna uma nova array. Veja também `Array.prototype.slice()`.

`Float32Array.prototype.some()`

Retorna true se pelo menos um elemento nesta array satisfazer a função de teste proveniente de uma função de teste. Veja também `Array.prototype.some()`.

`Float32Array.prototype.sort()`

Classifica os elementos de uma array no lugar e retorna a array. Veja também `Array.prototype.sort()`.

`Float32Array.prototype.subarray()`

Retorna uma nova Array `Float32Array` a partir de um ponto inicial e um fim pelo índex do elemento.

`Float32Array.prototype.values()`

Retorna um novo objeto `Array Iterator` que contém os valores para cada índex na Array. Veja também `Array.prototype.values()`.

`Float32Array.prototype.toLocaleString()`

Retorna uma string localizada representando a array e seus elementos. Veja também `Array.prototype.toLocaleString()`.

`Float32Array.prototype.toString()`

Retorna uma string representando a array e seus elementos. Veja também `Array.prototype.toString()`.

`Float32Array.prototype[@@iterator]()`

Retorna um novo objeto `Array Iterator` que contém os valores para cada índex na Array.

Exemplos

Diferentes modos de criar uma Array `Float32Array`:

```
// Pelo seu tamanho
var float32 = new Float32Array(2);
float32[0] = 42;
console.log(float32[0]); // 42
console.log(float32.length); // 2
console.log(float32.BYTES_PER_ELEMENT); // 4

// Por uma array
var arr = new Float32Array([21, 31]);
console.log(arr[1]); // 31

// Por uma outra TypedArray
var x = new Float32Array([21, 31]);
var y = new Float32Array(x);
console.log(y[0]); // 21

// Por um ArrayBuffer
var buffer = new ArrayBuffer(16);
var z = new Float32Array(buffer, 0, 4);

// Por um iterável
var iterable = (function* () {
  yield* [1, 2, 3];
})();
var float32 = new Float32Array(iterable);
// Float32Array[1, 2, 3]
```

Notas de compatibilidade

Começando com o ECMAScript 2015, construtores da Array `Float32Array` requerem o uso do operador `new`. Chamando o construtor da Array `Float32Array` como uma função, sem o `new`, irá gerar um erro `TypeError` de agora em diante.

```
var dv = Float32Array([1, 2, 3]);

// TypeError: chamando o construtor da função interna Float32Array
// sem o new é proibido
```

```
var dv = new Float32Array([1, 2, 3]);
```

Veja também

- JavaScript arrays tipadas
- `ArrayBuffer`
- `DataView`

Function

Resumo

O **`construtor Function`** cria um novo objeto `Function`. Chamar o construtor diretamente pode criar funções dinamicamente, mas sofre com problemas de segurança e desempenho semelhante (mas muito menos significativo) a `eval`. No entanto, diferentemente de eval, a Função construtora cria funções que executam somente no escopo global.

Sintaxe

```
new Function ([arg1[, arg2[, ...argN]],] functionBody)
```

Parâmetros

`arg1, arg2, ... argN`

> Nomes para serem usados pela função como nomes formais de argumentos. Cada um deve ser uma string que corresponde para uma válida identidade JavaScript ou uma lista de certas strings separadas com uma vírgula; por exemplo "x", "theValue". our "a,b".

`functionBody`

> Uma string que contém as instruções JavaScript que compõem a definição da função.

Descrição

Objetos `Function` criados com o construtor `Function` são *parseados* quando a função é criada. Isto é menos eficiente que criar com uma expressão de função ou um declaração de função e chamando-a dentro do seu código, porque tais funções são *parseadas* com o resto do código.

Todos os argumentos passados para a função são tratados como os nomes dos indetificadores dos parâmetros na função a ser criada, na mesma ordem na qual eles foram passados.

> **Nota:** Funções criadas com o construtor `Function não criam closures` para o seu contexto de criação; elas sempre são criadas no escopo global. Quando executadas, elas terão acesso apenas às suas variáveis locais ou globais, não terão acesso às variáveis do escopo na qual o construtor `Function` foi chamado. Isto é diferente de usar `eval` com o código de uma expressão de função.

Invocar o construtor `Function` como uma função (sem usar o operador `new`) tem o mesmo efeito de chamá-la como um construtor.

Propriedades e Métodos da <code>Function</code>

O objeto global `Function` não tem métodos ou propriedades próprias, no entanto, como ela é uma função, ela herda alguns métodos e propriedades através do prototype chain do `Function`.

<code>Function</code> prototype object

Propriedades

Métodos

<code>Function</code> instances

`Function` instances inherit methods and properties from `Function`. As with all constructors, you can change the constructor's prototype object to make changes to all `Function` instances.

Exemplos

Exemplos: Especificando argumentos com o construtor <code>Function</code>

O código a seguir cria um objeto `Function` que recebe dois argumentos.

```
// O exemplo pode ser executado direto no seu console JavaScript

// Cria uma função que recebe 2 argumentos e retorna a soma entre os dois:
var adder = new Function("a", "b", "return a + b");

// Chamada da função
adder(2, 6);
// > 8
```

Os argumentos "a" e "b" são os argumentos que serão usados no corpo da função, "`return a + b`".

Exemplo: Um atalho recursivo para modificar o DOM em massa

Creating functions with the `Function` constructor is one of the ways to dynamically create an indeterminate number of new objects with some executable code into the global scope from a function. The following example (a recursive shortcut to massively modify the DOM) is impossible without the invocation of the `Function` constructor for each new query if you want to avoid closures.

```
<!doctype html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>
      MDN Example - a recursive shortcut to massively modify the DOM
    </title>
    <script type="text/javascript">
      var domQuery = (function () {
        var aDOMFunc = [
          Element.prototype.removeAttribute,
          Element.prototype.setAttribute,
          CSSStyleDeclaration.prototype.removeProperty,
```

```
          CSSStyleDeclaration.prototype.setProperty,
        ];

        function setSomething(bStyle, sProp, sVal) {
          var bSet = Boolean(sVal),
            fAction = aDOMFunc[bSet | (bStyle << 1)],
            aArgs = Array.prototype.slice.call(arguments, 1, bSet ? 3 : 2),
            aNodeList = bStyle ? this.cssNodes : this.nodes;

          if (bSet && bStyle) {
            aArgs.push("");
          }
          for (
            var nItem = 0, nLen = this.nodes.length;
            nItem < nLen;
            fAction.apply(aNodeList[nItem++], aArgs)
          );
          this.follow = setSomething.caller;
          return this;
        }

        function setStyles(sProp, sVal) {
          return setSomething.call(this, true, sProp, sVal);
        }
        function setAttribs(sProp, sVal) {
          return setSomething.call(this, false, sProp, sVal);
        }
        function getSelectors() {
          return this.selectors;
        }
        function getNodes() {
          return this.nodes;
        }

        return function (sSelectors) {
          var oQuery = new Function(
            "return arguments.callee.follow.apply(arguments.callee, arguments);",
          );
          oQuery.selectors = sSelectors;
          oQuery.nodes = document.querySelectorAll(sSelectors);
          oQuery.cssNodes = Array.prototype.map.call(
            oQuery.nodes,
            function (oInlineCSS) {
              return oInlineCSS.style;
            },
          );
          oQuery.attributes = setAttribs;
          oQuery.inlineStyle = setStyles;
          oQuery.follow = getNodes;
          oQuery.toString = getSelectors;
          oQuery.valueOf = getNodes;
          return oQuery;
        };
      })();
    </script>
  </head>

  <body>
    <div class="testClass">Lorem ipsum</div>
    <p>Some text</p>
    <div class="testClass">dolor sit amet</div>

    <script type="text/javascript">
      domQuery(".testClass")
        .attributes("lang", "en")("title", "Risus abundat in ore stultorum")
        .inlineStyle("background-color", "black")("color", "white")(
        "width",
        "100px",
      )("height", "50px");
    </script>
  </body>
</html>
```

See also

- function Expression
- function Statement
- Functions and function scope

Function.prototype.apply()

O método **`apply()`** chama uma função com um dado valor `this` e `arguments` providos como uma array (ou um objeto parecido com um array).

> **Nota:** A sintaxe desta função é quase idêntica a essa da `call()`, a diferença é que `call()` aceita uma **lista de argumentos**, enquanto `apply()` aceita um **array de**

> **argumentos**.

Sintaxe

```
fun.apply(thisArg, [argsArray])
```

Parâmetros

`thisArg`

> O valor de `this` é fornecido para a chamada de *fun*. Note que isso talvez não seja o valor real visto pelo método: se um método é uma função em código non-strict mode , `null` e `undefined` serão substituidos com o objeto global, e os valores primitivos serão embalados.

`argsArray`

> Um objeto parecido com um array (array-like), especificando os argumentos com os quais *fun* deve ser chamado, ou `null` ou `undefined` se não houverem argumentos que possam ser passados para a função. Começando com ECMAScript5 esses argumentos podem ser um objeto genérico array-like ao invés de um array. Veja abaixo a informação de compatibilidade de browsers.

Descrição

Você pode atribuir um objeto *`this`* diferente quando chamar uma função existente. *`this`* refere-se ao objeto atual, o objeto da chamada. Com *`apply`*, você pode escrever um método apenas uma vez e então herdá-lo em outro objeto, sem ter que reescrever o método para o novo objeto.

`apply` é muito parecido com `call()`, exceto pelo tipo de argumentos que ele suporta. Você pode usar um array de argumentos em vez de conjunto de parâmetros nomeados. Com `apply`, você pode usar um array literal, por exemplo, `fun.apply(this, ['comer', 'bananas'])`, ou um objeto `Array`, por exemplo `fun.apply(this, new Array('comer', 'bananas'))`.

Você pode também usar `arguments` para o parâmetro `argsArray`. `arguments`é uma variável local de uma função. Ele pode ser utilizado para todos os argumentos não especificados do objeto chamado. Assim, você não tem que saber os argumentos do objeto chamado quando você usa o método `apply`. Você pode usar `arguments` para passar todos os argumentos para o objeto da chamada. O objeto chamado fica então responsável por manipular os argumentos.

Desde a 5a versão do ECMAScript você pode utilizar qualquer tipo de objeto que é parecido com um array (array-like), então na prática isso significa que ele vai ter uma propriedade `length` e propriedades inteiras no intervalor (`0... length`). Como um exemplo, você pode agora usar um `NodeList` ou um objeto personalizado como `{ 'length': 2, '0': 'comer', '1': 'bananas' }`.

> **Nota:** Muitos navegadores, incluindo o Chrome 14 e o Internet Explorer 9, ainda não aceitam objetos parecidos com array e irão lançar uma exceção.

Exemplos

Usando <code>apply</code> para cadeia de construtores

Você pode usar `apply` para encadear construtores em um objeto, similar ao Java. No exemplo seguinte nós iremos criar um método de `Função` global chamado `construct`, que fará você capaz de usar um objeto parecido com um array com um construtor ao invés de uma lista de argumentos

```
Function.prototype.construct = function (aArgs) {
  var oNew = Object.create(this.prototype);
  this.apply(oNew, aArgs);
  return oNew;
};
```

> **Nota:** O método `Object.create()` usado acima é relativamente novo. Para um método alternativo utilizando closures, por favor considere a seguinte alternativa.
>
> ```
> Function.prototype.construct = function (aArgs) {
>   var fConstructor = this,
>     fNewConstr = function () {
>       fConstructor.apply(this, aArgs);
>     };
>   fNewConstr.prototype = fConstructor.prototype;
>   return new fNewConstr();
> };
> ```

Exemplo de uso:

```
function MyConstructor() {
  for (var nProp = 0; nProp < arguments.length; nProp++) {
    this["property" + nProp] = arguments[nProp];
  }
}

var myArray = [4, "Hello world!", false];
var myInstance = MyConstructor.construct(myArray);

console.log(myInstance.property1); // logs 'Hello world!'
console.log(myInstance instanceof MyConstructor); // logs 'true'
console.log(myInstance.constructor); // logs 'MyConstructor'
```

> **Nota:** Este método não nativo `Function.construct` não irá funcionar com alguns construtores nativos (como `Date`, por exemplo). Nestes casos você tem que usar o método `Function.prototype.bind` (por exemplo, imagine ter um array como o seguinte, para ser usado com o construtor `Date`: `[2012, 11, 4]`; Neste caso você tem que escrever algom como: `new (Function.prototype.bind.apply(Date, [null].concat([2012, 11, 4])))()` - de qualquer maneira essa não é a melhor forma de fazer as coisas e provavelmente não deve ser utilizado em qualquer ambiente de produção.

Usando <code>apply</code> e funções embutidas

A forma inteligente com que `apply` é utilizado permite à você usar funções nativas que de outra forma provavelmente teriam que ser escritas iterando sobre um array de valores. Aqui, como exemplo, iremos utilizar `Math.max/Math.min` para achar o valor máximo/mínimo value em um array.

```
/* número min/max em um array */
var numbers = [5, 6, 2, 3, 7];

/* utilizando Math.min/Math.max apply */
var max = Math.max.apply(
  null,
  numbers,
); /* Isso está prestes a ser igual a Math.max(numbers[0], ...)
                                      ou Math.max(5, 6, ...) */
var min = Math.min.apply(null, numbers);

/* vs. algoritmo simples baseado em loop */
((max = -Infinity), (min = +Infinity));

for (var i = 0; i < numbers.length; i++) {
  if (numbers[i] > max) {
    max = numbers[i];
  }
  if (numbers[i] < min) {
    min = numbers[i];
  }
}
```

Mas tome cuidado: ao utilizar o `apply` desta forma, você corre o risco de exceder o limite de argumentos do JavaScript. As consequências de fazer applying em uma função com muitos argumentos (pense em algo como dezenas de centenas de argumentos) varia de acordo com os engines (JavaScriptCore tem um limite de argumentos de 65536 hard-coded), visto que o limite (na verdade, até mesmo a natureza de qualquer comportamento de um stack excessivamente grande) não é especificado. Algumas engines irão jogar uma excessão. De uma forma mais incisiva, outras engines irão limitar de forma arbitrária o número de argumentos que poderção ser aplicados à função. (Para ilustrar esse último caso: se uma engine dessas tem um limite de quatro argumentos [obviamente, os limites atuais são significativamente maiores], isso seria como se os argumentos `5, 6, 2, 3` do exemplo anterior fossem passados ao `apply`, ao invés do array completo.) Se o valor do seu array puder crescer à casa das dezenas de centenas, use uma estratégia híbrida: aplique suas funções em cada bloco de array por vez:

```
function minOfArray(arr) {
  var min = Infinity;
  var QUANTUM = 32768;

  for (var i = 0, len = arr.length; i < len; i += QUANTUM) {
    var submin = Math.min.apply(null, arr.slice(i, Math.min(i + QUANTUM, len)));
    min = Math.min(submin, min);
  }

  return min;
}

var min = minOfArray([5, 6, 2, 3, 7]);
```

Usando apply em "monkey-patching"

Apply pode ser a melhor forma de monkey-patch uma função nativa do Firefox, ou de bibliotecas em JS. Dada uma função `someobject.foo`, você poderá modificar a função de uma maneira hackeresca, como por exemplo:

```
var originalfoo = someobject.foo;
someobject.foo = function () {
  // Faça coisas antes de chamar a função
  console.log(arguments);
  // Chama a função como se ela estivesse sido chamada normalmente:
  originalfoo.apply(this, arguments);
  // Rode as coisas que vem depois, aqui.
};
```

Esse método é especialmente útil quando você quer fazer debug de eventos, ou interagir com algo que não tem nenhuma API como os diversos eventos `.on([event]...` events, por exemplo aqueles utilizáveis no Devtools Inspector).

Veja também

- Objeto `arguments`
- `Function.prototype.bind()`
- `Function.prototype.call()`
- Funções e escopo de funções

Function.arguments

A propriedade **`function.arguments`** diz respeito a um objeto tipo array (array-like object) correspondente aos argumentos passados para uma função. Use somente a variável `arguments` em vez disso.

Descrição

A sintaxe `function.arguments` está obsoleta. A forma recomendada de acessar o objeto `arguments` disponível dentro das funções, é simplesmente referenciar a variável `arguments`.

No caso de recursão, ou seja, uma função f aparecer várias vezes na pilha de chamadas, o valor de `f.arguments` representa os argumentos correspondentes a invocação mais recente da função.

O valor da propriedade arguments é normalmente nulo (`null`) se não houver nenhuma invocação pendente da função em andamento (ou seja, a função foi chamada mas ainda não retornou).

Exemplos

```
function f(n) {
  g(n - 1);
}

function g(n) {
  console.log("before: " + g.arguments[0]);
  if (n > 0) {
    f(n);
  }
  console.log("after: " + g.arguments[0]);
}

f(2);

console.log("returned: " + g.arguments);

// Output

// before: 1
// before: 0
// after: 0
// after: 1
// returned: null
```

Especificações

Não faz parte de nenhuma especificação.

Veja também

- `arguments` object
- Functions and function scope

Function.prototype.bind()

O método **`bind()`** cria uma nova função que, quando chamada, tem sua palavra-chave `this` definida com o valor fornecido, com uma sequência determinada de argumentos precedendo quaisquer outros que sejam fornecidos quando a nova função é chamada.

Experimente

```
const module = {
  x: 42,
  getX: function () {
    return this.x;
  },
};

const unboundGetX = module.getX;
console.log(unboundGetX()); // The function gets invoked at the global scope
// Expected output: undefined

const boundGetX = unboundGetX.bind(module);
console.log(boundGetX());
// Expected output: 42
```

Sintaxe

```
function.bind(thisArg[, arg1[, arg2[, ...]]])
```

Parâmetros

`thisArg`

> O valor a ser passado como parâmetro `this` para a função de destino quando a função vinculada é chamada. O valor é ignorado se a função ligada é construída usando o operador `new`.

`arg1, arg2, ...`

Argumentos que precedem outros argumentos fornecidos para a função vinculada ao invocar a função de destino.

Valor de retorno

Uma cópia da função fornecida com o valor `this` especificado e argumentos iniciais.

Descrição

A função `bind()` cria uma nova **função vinculada** (*bound function*). Uma função vinculada é um **objeto de função exótico** (termo da **ECMAScript 2015**) que encapsula o objeto de função original. Chamar uma função vinculada geralmente resulta na execução de sua **função encapsulada**.

Uma função vinculada tem as seguintes propriedades internas:

- **[[BoundTargetFunction]]** - o objeto de função encapsulado;
- **[[BoundThis]]** - o valor que sempre é passado como `this` quando se chama a função encapsulada;
- **[[BoundArguments]]** - uma lista de valores cujos elementos são usados como os primeiros argumentos para qualquer chamada da função encapsulada;
- **[[Call]]** - executa código associado com este objeto. Invocado através de uma expressão de chamada de função. Os argumentos para o método interno são um valor `this` e uma lista contendo os argumentos passados para a função por uma expressão de chamada.

Quando a função vinculada é chamada, ela chama seu método interno **[[Call]]** na **[[BoundTargetFunction]],** na forma `Call(boundThis, args)`, onde `boundThis` é **[[BoundThis]]** e `args` é **[[BoundArguments]]** seguido pelos argumentos passados pela chamada de função.

Uma função vinculada também pode ser construída usando-se o operador `new`; ao fazê-lo, o resultado é o mesmo que seria se a função alvo tivesse sido construída. O valor de `this` fornecido é ignorado, porém os argumentos precedentes são fornecidos à função emulada.

Exemplos

Criando uma função vinculada

O uso mais simples de `bind()` é fazer com que uma função que, independentemente da chamada, é chamada com um determinado valor `this`. Um erro comum para programadores JavaScript novatos é extrair um método de um objeto e, em seguida, chamar essa função e esperar que ele use o objeto original como o seu `this` (por exemplo, usando esse método num código baseado em *callback*). Sem a devida atenção, no entanto, o objeto original é normalmente perdido. Criar uma função vinculada a partir da função, usando o objeto original, resolve perfeitamente esse problema:

```
this.x = 9; //this aqui se refere ao objeto global "window" do navegador
var module = {
  x: 81,
  getX: function () {
    return this.x;
  },
};

module.getX(); // 81

var retrieveX = module.getX;
retrieveX();
// retorna 9 - a função foi invocada no escopo global

// Criando uma nova função com 'this' vinculada ao módulo
// Programadores novatos podem confundir a variável x
// global com a propriedade x do módulo
var boundGetX = retrieveX.bind(module);
boundGetX(); // 81
```

Funções parcialmente aplicadas

O próximo uso mais simples de `bind()` é criar uma função com argumentos iniciais pré-especificados. Esses argumentos (caso existam) acompanham o valor `this` fornecido e então são inseridos no início dos argumentos passados para a função alvo, seguidos pelos argumentos passados para a função vinculada, sempre que a função vinculada é chamada.

```
function list() {
  return Array.prototype.slice.call(arguments);
}

var list1 = list(1, 2, 3); // [1, 2, 3]

// Cria uma função com um argumento principal predefinido
var leadingThirtysevenList = list.bind(null, 37);

var list2 = leadingThirtysevenList();
// [37]

var list3 = leadingThirtysevenList(1, 2, 3);
// [37, 1, 2, 3]
```

Com <code>setTimeout</code>

Por padrão, dentro de `window.setTimeout()` a palavra-chave `this` vai ser definida com o objeto `window` (ou com o objeto `global`). Ao trabalhar com métodos de classes que requerem que `this` se refira à instâncias de classes, você pode vincular `this` explicitamente à função de *callback*, de modo a manter a instância.

```
function LateBloomer() {
  this.petalCount = Math.ceil(Math.random() * 12) + 1;
}
```

```
// Declarar bloom depois de um intervalo de 1 segundo
LateBloomer.prototype.bloom = function () {
  window.setTimeout(this.declare.bind(this), 1000);
};

LateBloomer.prototype.declare = function () {
  console.log("I am a beautiful flower with " + this.petalCount + " petals!");
};

var flower = new LateBloomer();
flower.bloom();
// depois de 1 segundo, ativa o método 'declare'
```

Funções vinculadas usadas como construtores

> **Aviso:** Esta seção demonstra capacidades do JavaScript e documenta alguns casos de borda do método `bind()`. Os métodos mostrados abaixo não são os melhores jeitos de se fazer as coisas e provavelmente não deveriam ser usados em nenhum ambiente produtivo.

Funções vinculadas são automaticamente adequadas para uso com o operador `new` para construir novas instâncias criadas pela função alvo. Quando uma função vinculada é usada para construir um valor, o `this` fornecido é ignorado. Porém, argumentos fornecidos ainda são prefixados à chamada do construtor:

```
function Point(x, y) {
  this.x = x;
  this.y = y;
}

Point.prototype.toString = function () {
  return this.x + "," + this.y;
};

var p = new Point(1, 2);
p.toString(); // '1,2'

// não suportado no polyfill abaixo,
// funciona bem com o bind nativo:

var YAxisPoint = Point.bind(null, 0 /*x*/);

var emptyObj = {};
var YAxisPoint = Point.bind(emptyObj, 0 /*x*/);

var axisPoint = new YAxisPoint(5);
axisPoint.toString(); // '0,5'

axisPoint instanceof Point; // true
axisPoint instanceof YAxisPoint; // true
new Point(17, 42) instanceof YAxisPoint; // true
```

Note que você não precisa fazer nada de especial para criar uma função vinculada para usar com `new`. O corolário é que você não precisa fazer nada de especial para criar uma função vinculada que será chamada de forma clara, mesmo que você preferisse que a função vinculada fosse somente chamada usando-se `new`.

```
// Exemplo pode ser executado diretamente no seu console JavaScript
// ...continuando o exemplo acima

// Ainda pode ser chamada como uma função normal
// (apesar de que isso geralmente não é desejado)
YAxisPoint(13);

emptyObj.x + "," + emptyObj.y;
// >  '0,13'
```

Se você quer suportar o uso de uma função vinculada somente através de `new`, ou somente a chamando, a função alvo deve impor essa restrição.

Criando atalhos

`bind()` itambém é útil em casos onde você quer criar um atalho para uma função que requer um valor específico de `this`.

Tome por exemplo `Array.prototype.slice`, que você quer usar para converter um objeto *array-like* em um vetor verdadeiro. Você poderia criar um atalho assim:

```
var slice = Array.prototype.slice;

// ...

slice.apply(arguments);
```

Com `bind()`, isso pode ser simplificado. No seguinte trecho de código, `slice` é uma função vinculada à função `apply()` de `Function`, com o valor `this` definido com a função `slice()` de `Array`. Isso significa que chamadas adicionais de `apply()` podem ser eliminadas:

```
// mesmo que "slice" no exemplo anterior
var unboundSlice = Array.prototype.slice;
var slice = Function.prototype.apply.bind(unboundSlice);
```

```
// ...

slice(arguments);
```

Polyfill

A função `bind` é uma adição à ECMA-262, 5ª. edição; como tal, pode não estar presente em todos os navegadores. Você pode contornar isso parcialmente inserindo o seguinte código no começo de seus *scripts*, permitindo o uso de muita parte da funcionalidade de `bind()` em implementações que não a suportam nativamente.

```
if (!Function.prototype.bind) {
  Function.prototype.bind = function (oThis) {
    if (typeof this !== "function") {
      // mais próximo possível da função interna
      // IsCallable da ECMAScript 5
      throw new TypeError(
        "Function.prototype.bind - what is trying to be bound is not callable",
      );
    }

    var aArgs = Array.prototype.slice.call(arguments, 1),
      fToBind = this,
      fNOP = function () {},
      fBound = function () {
        return fToBind.apply(
          this instanceof fNOP ? this : oThis,
          aArgs.concat(Array.prototype.slice.call(arguments)),
        );
      };

    fNOP.prototype = this.prototype;
    fBound.prototype = new fNOP();

    return fBound;
  };
}
```

Algumas das muitas diferenças (é bem possível que haja outras, já que esta lista não pretende seriamente ser completa) entre este algoritmo e o algoritmo especificado são:

- Esta implementação parcial depende dos métodos internos `Array.prototype.slice()`, `Array.prototype.concat()`, `Function.prototype.call()` e `Function.prototype.apply()` possuírem seus valores originais.
- Esta implementação parcial cria funções que não tem um `caller` imutável como "mecanismo de defesa" e propriedades `arguments` que lançam um `TypeError` ao usar *get*, *set*, ou ao deletar. (Isto pode ser adicionado se a implementação suporta `Object.defineProperty`, ou parcialmente implementado sem um comportamento *throw-on-delete* se a implementação suporta as extensões `Object.prototype.__defineGetter__()` e `Object.prototype.__defineSetter__()`)
- Esta implementação parcial cria funções que tem uma propriedade `prototype`. (Funções vinculadas apropriadas não a tem.)
- Esta implementação parcial cria funções vinculadas cuja propriedade `length` não cumpre com a regra da ECMA-262: cria funções com comprimento zero, quando uma implementação completa, dependendo do comprimento da função alvo e do número de argumentos pre-especificados, pode retornar um comprimento não-nulo.

Se você escolher utilizar esta implementação parcial, **você não deve confiar em casos onde o comportamento é diferente da ECMA-262, 5ª. edição!** Porém, com algum cuidado (e talvez com modificação adicional para atender necessidades específicas), esta implementação parcial pode ser uma ponte razoável para quando `bind()` for amplamente implementada de acordo com a especificação.

Veja também

- `Function.prototype.apply()`
- `Function.prototype.call()`
- Functions

Function.prototype.call()

Introdução

O método **`call()`** invoca uma função com um dado valor `this` e argumentos passados individualmente.

> **Nota:** Apesar de a sintaxe desta função ser quase idêntica à de `apply()`, a principal diferença é que `call()` aceita uma **lista de argumentos**, enquanto `apply()` aceita **um único array de argumentos.**

Sintaxe

```
fun.call(thisArg[, arg1[, arg2[, ...]]])
```

Parâmetros

`thisArg`

O valor de `this` proveu a chamada para *fun*. Note que `this` pode não ser o valor atual visto pelo método: se esse método é uma função em non-strict mode code, `null` e `undefined` serão reescritos com o objeto global, e valores primitivos serão encaixados.

`arg1, arg2, ...`

Argumentos para o objeto.

Descrição

Você pode atribuir um objeto `this` diferente quando executar uma função existente. `this` refere-se ao objeto atual, o objeto em execução. Com `call`, você pode escrever um método uma vez e então herdá-lo em outro objeto, sem ter que reescrever o método para o novo objeto.

Exemplos

Exemplo: Usando <code>call</code> para encadear construtores para um objeto

Você pode usar `call` para encadear construtores para um objeto, similar ao Java. No seguinte exemplo, o construtor do objeto `Product` é definido com dois parâmetros, `name` e `price`. Outras duas funções `Food` e `Toy` executam `Product` passando `this`, `name` e `price`. O Produto inicializa as propriedades `name` e `price`, ambos definem o `category`.

```
function Product(name, price) {
  this.name = name;
  this.price = price;

  if (price < 0) {
    throw RangeError(
      "Cannot create product " + this.name + " with a negative price",
    );
  }

  return this;
}

function Food(name, price) {
  Product.call(this, name, price);
  this.category = "food";
}

Food.prototype = Object.create(Product.prototype);

function Toy(name, price) {
  Product.call(this, name, price);
  this.category = "toy";
}

Toy.prototype = Object.create(Product.prototype);

var cheese = new Food("feta", 5);
var fun = new Toy("robot", 40);
```

Exemplo: Usando o <code>call</code> para chamar funções anônimas

Neste exemplo, criamos uma função anônima que usa o `call` para executá-lo em todos os objetos em um array(vetor). O principal propósito da função anônima aqui é adicionar uma função print para todo o objeto, que está disponível para imprimir o índice correto do objeto no array. Não foi necessário passar o valor do objeto como `this` , mas isso foi feito apenas para explicação.

```
var animais = [
  { especie: "Lion", nome: "King" },
  { especie: "Whale", nome: "Fail" },
];

for (var i = 0; i < animais.length; i++) {
  (function (i) {
    this.print = function () {
      console.log("#" + i + " " + this.especie + ": " + this.nome);
    };
    this.print();
  }).call(animais[i], i);
}
```

Usando <code>call</code> para chamar a função e especificar o contexto de 'this'

No exemplo abaixo, quando vamos chamar a apresentação, o valor de this será associado ao objeto i.

```
function apresentacao() {
  var resposta = [this.pessoa, 'é um excelente', this.funcao].join(' ');
  console.log(resposta);
}

var i = {
  pessoa: 'Douglas Crockford', funcao: 'Desenvolvedor Javascript'
};

apresentacao.call(i); // Douglas Crockford é um excelente Desenvolvedor Javascript
```

Veja também

- `Function.prototype.bind()`
- `Function.prototype.apply()`
- Introdução à JavaScript Orientado à Objetos

Function.caller

> **Não padrão:** Este recurso não é padronizado. Não recomendamos o uso de recursos não padronizados em produção, pois eles têm suporte limitado nos navegadores e podem mudar ou ser removidos. No entanto, podem ser uma alternativa viável em casos específicos onde não exista uma opção padrão.

A propriedade **`function.caller`** retorna a função que invocou a função especificada.

Descrição

Se a função `f` foi invocada pelo codigo mais alto nível, o valor de `f.caller` é `null`, caso contrario, o valor será a função a qual invocou `f`.

Esta propriedade substitui a propriedade obsoleta `arguments.caller` do objeto `arguments`.

A propriedade especial `__caller__`, a qual retornou o objeto de ativação do chamador, permitindo assin reconstruir o stack, foi removido por motivo de segurança.

Notas

Note que no caso de recurção, você não pode reconstruir o stack de chamadas usando esta propriedade. Considere:

```
function f(n) {
  g(n - 1);
}
function g(n) {
  if (n > 0) {
    f(n);
  } else {
    stop();
  }
}
f(2);
```

No momento em que `stop()` é chamado o stack será:

```
f(2) -> g(1) -> f(1) -> g(0) -> stop()
```

O seguinte é true:

```
stop.caller === g && f.caller === g && g.caller === f;
```

então se você tentou recuperar o stack trace na função `stop()` assim:

```
var f = stop;
var stack = "Stack trace:";
while (f) {
  stack += "\n" + f.name;
  f = f.caller;
}
```

o loop nunca irá parar.

Exemplos

Verificando o valor da propriedade <code>caller</code> de uma função

O código a seguir verifica o valor da propriedade `caller` de uma função.

```
function myFunc() {
  if (myFunc.caller == null) {
    return "The function was called from the top!";
  } else {
    return "This function's caller was " + myFunc.caller;
  }
}
```

Especificações

Não faz parte de nenhuma especificação. Implementado no JavaScript 1.5.

Ver também

- Implementation bug for SpiderMonkey Erro do Firefox 65683

Function.displayName

> **Não padrão:** Este recurso não é padronizado. Não recomendamos o uso de recursos não padronizados em produção, pois eles têm suporte limitado nos

> navegadores e podem mudar ou ser removidos. No entanto, podem ser uma alternativa viável em casos específicos onde não exista uma opção padrão.

A propriedade **`function.displayName`** retorna o nome de exibição da função.

Descrição

Quando definida, a propriedade `displayName` retorna o nome de exibição da função.

```
function doSomething() {}

console.log(doSomething.displayName); // "undefined"

var popup = function (content) {
  console.log(content);
};

popup.displayName = "Show Popup";

console.log(popup.displayName); // "Show Popup"
```

Você pode definir uma função com uma nome de exibição em um function expression:

```
var object = {
  someMethod: function () {},
};

object.someMethod.displayName = "someMethod";

console.log(object.someMethod.displayName); // logs "someMethod"

try {
  someMethod;
} catch (e) {
  console.log(e);
}
// ReferenceError: someMethod is not defined
```

Você pode mudar dinamicamente o`displayName` de uma função:

```
var object = {
  // anonymous
  someMethod: function (value) {
    arguments.callee.displayName = "someMethod (" + value + ")";
  },
};

console.log(object.someMethod.displayName); // "undefined"

object.someMethod("123");
console.log(object.someMethod.displayName); // "someMethod (123)"
```

Exemplos

Geralmente, é preferida por consoles e perfis em vez de `func.name` mostrar o nome de uma função.

Entrando com o seguinte em um console, isso deverá mostrar algo como "`function My Function()`":

```
var a = function () {};
a.displayName = "My Function";

a; // "function My Function()"
```

Especificações

Não faz parte de denhuma especificação.

Function.length

A propriedade **`length`** especifica o número de argumentos esperados pela função.

Property attributes of `Function.length`	
Writable	no
Enumerable	no
Configurable	yes

Descrição

`length é uma propriedade de um objeto função`, e indica quantos argumentos a função espera, i.e. o número de parametros formais. Este número não incluí o rest parameter. Por contraste, `arguments.length` é local para a função e fornece o número de argumentos que foram realmente passados.

Propriedade de Dados do construtor <code>Function</code>

O construtor `Function` é propriamente um objeto `Function`. A proproedade de dados do seu `length` tem o valor de 1. Os atributos da propriedade são: Escrita: `false`, Enumerável: `false`, Configurável: `true`.

Propriedades do objeto prototype de <code>Function</code>

A propriedade length do objeto prototype `Function` tem o valor de 0.

Exemplos

```
console.log(Function.length); /* 1 */

console.log(function () {}.length); /* 0 */
console.log(function (a) {}.length); /* 1 */
console.log(function (a, b) {}.length); /* 2 etc. */
console.log(
  function (...args) {}.length,
); /* 0, rest parameter is not counted */
```

Veja tambem

- `Function`

Function.name

A propriedade somente-leitura **`name`** de um objeto `Function` indica o nome da função como especificado quando esta foi criada, ou `"anonymous"` para funções criadas anonimamente.

Experimente

```
const func1 = function () {};

const object = {
  func2: function () {},
};

console.log(func1.name);
// Expected output: "func1"

console.log(object.func2.name);
// Expected output: "func2"
```

Property attributes of `Function.name`	
Writable	no
Enumerable	no
Configurable	yes

> **Nota:** Note que em implementações não-standard anteriores à ES2015 o atributo `configurable` tinha também o valor `false`.

Exemplos

Nome de declaração da função

A propriedade `name` retorna o nome de uma declaração de função.

```
function doSomething() {}
doSomething.name; // "doSomething"
```

Nome do construtor da função

Funções criadas com a sintaxe `new Function(...)` ou somente `Function(...)` criam objetos `Function` com o nome "anonymous".

```
(new Function).name; // "anonymous"
```

Nomes de função inferidos

Variáveis e métodos podem inferir o nome de uma função anônima a partir de sua posição sintática (novo na ECMAScript 2015).

```
var f = function () {};
var object = {
  someMethod: function () {},
};

console.log(f.name); // "f"
console.log(object.someMethod.name); // "someMethod"
```

Você pode definir uma função com um nome numa expressão de função:

```
var object = {
  someMethod: function object_someMethod() {},
};
console.log(object.someMethod.name); // grava o log "object_someMethod"

try {
  object_someMethod;
} catch (e) {
  console.log(e);
}
// ReferenceError: object_someMethod is not defined
```

Você não pode mudar o nome de uma função, pois a propriedade é somente-leitura:

```
var object = {
  // anonymous
  someMethod: function () {},
};

object.someMethod.name = "otherMethod";
console.log(object.someMethod.name); // someMethod
```

Para mudá-lo, você poderia no entanto usar `Object.defineProperty()`.

Nomes curtos de métodos

```
var o = {
  foo() {},
};
o.foo.name; // "foo";
```

Nomes de funções vinculadas

`Function.bind()` produz uma função cujo nome é "bound " seguido do nome da função.

```
function foo() {}
foo.bind({}).name; // "bound foo"
```

Nomes de função para getters e setters

Ao usar propriedades acessórias `get` e `set`, "get" ou "set" aparecerão no nome da função.

```
var o = {
  get foo() {},
  set foo(x) {},
};

var descriptor = Object.getOwnPropertyDescriptor(o, "foo");
descriptor.get.name; // "get foo"
descriptor.set.name; // "set foo";
```

Nomes de funções em classes

Você pode usar `obj.constructor.name` para checar a "classe" de um objeto (porém leia com atenção os avisos abaixo):

```
function Foo() {} // Sintaxe ES2015: class Foo {}

var fooInstance = new Foo();
console.log(fooInstance.constructor.name); // grava o log "Foo"
```

> **Aviso:** O interpretador vai definir a propriedade interna `Function.name` somente se uma função não tiver uma propriedade já com o nome *name* (veja a seção 9.2.11 da ECMAScript2015 Language Specification). Porém, a ES2015 especifica que a palavra-chave *static* de maneira que métodos estáticos serão definidos como OwnProperty da função construtora de classe (ECMAScript2015, 14.5.14.21.b + 12.2.6.9).

Portanto não podemos obter o nome de virtualmente qualquer classe com um método estático `name()`:

```
class Foo {
  constructor() {}
  static name() {}
}
```

Com um método `static name()`, `Foo.name` não guarda mais o nome verdadeiro da classe mas uma referência ao objeto de função `name()`. A definição de classe acima, escrita em sintaxe ES2015, se comportará de maneira similar ao seguinte trecho de código em sintaxe ES5 no Chrome ou no Firefox:

```
function Foo() {}
Object.defineProperty(Foo, "name", { writable: true });
Foo.name = function () {};
```

Tentar obter a classe de `fooInstance` via `fooInstance.constructor.name` não nos dará de maneira alguma o nome da classe, mas sim uma referência ao método estático da classe. Exemplo:

```
var fooInstance = new Foo();
console.log(fooInstance.constructor.name); // grava o name() da função no log
```

Você pode ver também, a partir do exemplo de sintaxe ES5, que, no Chrome ou no Firefox, a nossa definição estática de `Foo.name` se torna *writable*. A predefinição interna na ausência de uma definição estática customizada é somente-leitura:

```
Foo.name = "Hello";
console.log(Foo.name); // logs "Hello" if class Foo has a static name() property but "Foo" if not.
```

Portanto, você não pode assumir que a propriedade interna `Function.name` sempre guardará um nome de classe..

Símbolos como nome de função

Se um `Symbol` é usado como nome de função e o símbolo tem uma descrição, o nome do método será a descrição entre colchetes.

```
var sym1 = Symbol("foo");
var sym2 = Symbol();
var o = {
  [sym1]: function(){},
  [sym2]: function(){}
};

o[sym1].name; // "[foo]"
o[sym2].name; // ""
```

Compressores e minificadores JavaScript

> **Aviso:** Tenha cuidado ao usar `Function.name` e transformações de código-fonte, como aquelas executadas por compressores (minificadores) ou obfuscadores de JavaScript. Estas ferramentas são comumente usadas como parte de processos de *build* de JavaScript para reduzir os tamanhos de programas antes da implementação em produção. Tais transformações frequentemente mudam nomes de função durante o *build*.

Código fonte do tipo:

```
function Foo() {}
var foo = new Foo();

if (foo.constructor.name === "Foo") {
  console.log("'foo' is an instance of 'Foo'");
} else {
  console.log("Oops!");
}
```

pode ser comprimido e se tornar:

```
function a() {}
var b = new a();
if (b.constructor.name === "Foo") {
  console.log("'foo' is an instance of 'Foo'");
} else {
  console.log("Oops!");
}
```

Na versão descomprimida, o programa cai no bloco-verdade e grava o log *'foo' is an instance of 'Foo'*. Todavia, na versão comprimida ele se comporta diferentemente, e cai no bloco `else`. Se você depende de `Function.name`, como no exemplo acima, tenha certeza que seu processo de *build* não mude nomes de função, ou então não assuma que uma função terá um nome determinado.

Function.prototype.toString()

O método **`toString()`** retorna uma string representando o código fonte da função.

Experimente

```
function sum(a, b) {
  return a + b;
}

console.log(sum.toString());
// Expected output: "function sum(a, b) {
//                     return a + b;
//                   }"

console.log(Math.abs.toString());
// Expected output: "function abs() { [native code] }"
```

Sintaxe

```
function.toString()
```

Valor de retorno

Uma string representando o código fonte da função.

Descrição

O objeto da `Function` substitui o método `toString` herdado de `Object`; ele não herda `Object.prototype.toString`. Para objetos `Function` definidos pelo usuário, o método `toString` retorna uma string contendo o seguimento de texto de origem que foi usado para definir a função

O JavaScript chama o método `toString` automaticamente quando uma `Function` pode ser representada como um valor de texto. e.x. quando uma função é concatenada com uma string.

O método `toString()` lançará uma exceção do tipo `TypeError` ("Function.prototype.toString called on incompatible object") se o valor `this` do objeto não é um objeto do tipo `Function`.

```
Function.prototype.toString.call("foo"); // TypeError
```

Se o método `toString()` é chamado por objetos de funções embutidas ou por uma função criada por `Function.prototype.bind`, `toString()` retorna uma string de uma função nativa que parece

```
"function () {\n    [native code]\n}";
```

Se o método `toString()` é chamado por uma função criada pelo contrutor de `Function`, `toString()` retorna o código fonte de uma declaração de função sintetizada chamada "anonymous" usando os parâmetros passados e o corpo da função.

Exemplos

Função	Function.prototype.toString resultado
`function f(){}`	`"function f(){}"`
`class A { a(){} }`	`"class A { a(){} }"`
`function* g(){}`	`"function* g(){}"`
`a => a`	`"a => a"`
`({ a(){} }.a)`	`"a(){}"`
`({ *a(){} }.a)`	`"*a(){}"`
`({ [0](){} }[0])`	`"[0](){}"`
`Object.getOwnPropertyDescriptor({ get a(){} }, "a").get`	`"get a(){}"`
`Object.getOwnPropertyDescriptor({ set a(x){} }, "a").set`	`"set a(x){}"`
`Function.prototype.toString`	`"function toString() { [native code] }"`
`(function f(){}.bind(0))`	`"function () { [native code] }"`
`Function("a", "b")`	`"function anonymous(a\n) {\nb\n}"`

Notas específicas do Firefox

- Desde o Firefox 17 a função `Function.prototype.toString()` tem sido implementada salvando o fonte da função. O descompilador foi removido o que torna o parâmetro `indentation` desnecessário. Veja Erro do Firefox 761723 para mais detalhes.
- Do Firefox 38 para o 63 a função `Function.prototype.toString()` lançava exceções para objetos `Proxy` (Erro do Firefox 1100936 e Erro do Firefox 1440468).

Veja também

- `Object.prototype.toString()`

Generator

O objeto **`Generator`** é retornado por generator function e conforme iterable protocol e o iterator protocol.

Síntaxe

```
function* gen() {
  yield 1;
  yield 2;
  yield 3;
}

var g = gen(); // "Generator { }"
```

Métodos

`Generator.prototype.next()`

Retorna o valor fornecido pela expressão `yield`.

`Generator.prototype.return()`

Retorna o valor fornecido a finaliza o generator.

`Generator.prototype.throw()`

Lança um erro no generator.

Example

An infinite iterator

```
function* idMaker() {
  var index = 0;
  while (true) yield index++;
}

var gen = idMaker(); // "Generator { }"

console.log(gen.next().value); // 0
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
// ...
```

Objeto legacy generator

Firefox (SpiderMonkey) também implementa a versão anterior do generator em JavaScript 1.7, onde o asterisco (*) na declaração da função não era necessário (somente era necessário usar a palavra reservada `yield` no corpo da função). Contudo, legacy generators estão obsoletos. Não os use, eles serão removidos (Erro do Firefox 1083482).

Métodos legacy generator

`Generator.prototype.next()`

Retorna o valor fornecido pela expressão `yield`. Isto corresponde ao `next()` do ES6.

`Generator.prototype.close()`

Fecha o generator, então quando chamar `next()` um erro `StopIteration` será lançado. Isto corresponde ao método `return()` do ES6.

`Generator.prototype.send()`

Usado para enviar um valor para o generator. Este valor é retordo pela expressão `yield`, e retorna o valor fornecido pelo pelo next `yield`. `send(x)` corresponde ao `next(x)` do ES6.

`Generator.prototype.throw()`

Lança um erro no generator. Isto corresponde ao método `throw() do ES6.`

Exemplo do Legacy generator

```
function* fibonacci() {
  var a = yield 1;
  yield a * 2;
}

var it = fibonacci();
console.log(it); // "Generator {  }"
console.log(it.next()); // 1
console.log(it.send(10)); // 20
```

```
console.log(it.close()); // undefined
console.log(it.next()); // throws StopIteration (Como o generator está fechado)
```

Veja Também

Legacy generators

- The legacy generator function
- The legacy generator function expression
- `StopIteration`
- The legacy Iterator protocol

ES6 generators

- Functions
- `function`
- `function expression`
- `Function`
- `function*`
- `function* expression`
- `GeneratorFunction`
- The Iterator protocol

globalThis

A propriedade global **`globalThis`** retorna um objeto global de nível superior.

Experimente

```
function canMakeHTTPRequest() {
  return typeof globalThis.XMLHttpRequest === "function";
}

console.log(canMakeHTTPRequest());
// Expected output (in a browser): true
```

Sintaxe

```
globalThis
```

Descrição

Historicamente, o acesso ao escopo global exigiu uma sintaxe diferente em diferentes ambientes JavaScript. Na web você pode usar `window`, `self` ou `frames` - porém em Web Workers somente `self` funcionará. Em Node.js nada disso funciona e você deve usar `global`. A palavra-chave `this` poderia ser usada dentro de funções em execução no modo *sloppy*, mas `this` será `undefined` em módulos, e dentro de funções em execução no *strict mode*.

A propriedade `globalThis` fornece uma maneira padrão de acessar o objeto global entre ambientes. Ao contrário de propriedades semelhantes, como `window` e `self`, é garantido que funcione em contextos *window* e *non-window*. Dessa forma, você pode acessar o objeto global de maneira consistente sem precisar saber em qual ambiente o código está sendo executado.

Para ajudá-lo a lembrar o nome, lembre-se que no escopo global, o valor de `this` é `globalThis`.

Nome

Várias outras opções de nomes populares, como `self` e `global` foram removidas da discussão devido ao seu potencial em quebrar a compatibilidade com o código existente.

Exemplos

Antes de `globalThis`, a única maneira confiável de obter o objeto global para um ambiente era `Function('return this')()`. No entanto, isso causa violações CSP em algumas configurações, então es6-shim usa uma verificação como essa, por exemplo:

```
var getGlobal = function () {
  if (typeof self !== "undefined") {
    return self;
  }
  if (typeof window !== "undefined") {
    return window;
  }
  if (typeof global !== "undefined") {
    return global;
  }
  throw new Error("unable to locate global object");
};

var globals = getGlobal();

if (typeof globals.setTimeout !== "function") {
  // sem setTimeout neste ambiente!
}
```

Com `globalThis` disponível, a busca global adicional entre ambientes não é mais necessária:

```
if (typeof globalThis.setTimeout !== "function") {
  // sem setTimeout neste ambiente!
}
```

Especificações

Especificação	Status	Comentário
globalThis proposal	Stage 3	

Infinity

Sumário

A propriedade global **`Infinity`** é um valor numérico que representa infinito.

Property attributes of `Infinity`	
Writable	no
Enumerable	no
Configurable	no

Sintaxe

```
Infinity
```

Descrição

`Infinity` é uma propriedade do *objeto global*, ou seja, é uma varíavel no escopo global.

O valor inicial de `Infinity` é `Number.POSITIVE_INFINITY`. O valor `Infinity` (positivo) é maior do que qualquer outro número. Este valor se comporta matematicamente como infinito; por exemplo, qualquer número positivo multiplicado por `Infinity` é `Infinity`, e qualquer coisa dividida por `Infinity` é 0.

Pela especificação ECMAScript 5, `Infinity` é somente leitura (implementado no JavaScript 1.8.5 / Firefox 4).

Veja também

- `Number.NEGATIVE_INFINITY`
- `Number.POSITIVE_INFINITY`

Int16Array

A matriz do tipo **`Int16Array`** representa uma matriz de inteiros assinados de 16 bits com dois complementos na ordem de bytes da plataforma. Se o controle sobre a ordem de bytes for necessário, use `DataView`. O conteúdo é inicializado em 0. Uma vez estabelecido, você pode fazer referência a elementos na matriz usando os métodos do objeto ou usando a sintaxe de índice da matriz padrão (ou seja, usando a notação de colchetes).

Sintaxe

```
new Int16Array(); // novo no ES2017
new Int16Array(length);
new Int16Array(typedArray);
new Int16Array(object);
new Int16Array(buffer [, byteOffset [, length]]);
```

Para obter mais informações sobre a sintaxe do construtor e os parâmetros, consulte *TypedArray*.

Propriedades

`Int16Array.BYTES_PER_ELEMENT`

Retorna um valor numérico do tamanho do elemento. `2` no caso de um `Int16Array`.

Int16Array.length

Propriedade de comprimento estático cujo valor é 0. Para o comprimento real (número de elementos), consulte `Int16Array.prototype.length`.

`Int16Array.name`

Retorna o valor da string do nome do construtor. No caso do tipo `Int16Array`: "Int16Array".

`Int16Array.prototype`

Protótipo para os objetos TypedArray.

Métodos

`Int16Array.from()`

Cria um novo `Int16Array` de um objeto semelhante a uma matriz ou iterável. Veja também `Array.from()`.

`Int16Array.of()`

Cria um novo `Int16Array` com um número variável de argumentos. Veja também `Array.of()`.

<code>Int16Array</code> prototype

Todos `Int16Array` objetos herdam de `%TypedArray%.prototype`.

Propriedades

`Int16Array.prototype.constructor`

Retorna a função que criou o protótipo de uma instância. Este é o construtor `Int16Array` por padrão.

`Int16Array.prototype.buffer` Somente leitura

Retorna o `ArrayBuffer` referenciado pelo `Int16Array` Fixado em tempo de construção e **apenas leitura**.

`Int16Array.prototype.byteLength` Somente leitura

Retorna o tamanho (em bytes) de um `Int16Array` desde o ínicio `ArrayBuffer`. Fixado em tempo de construção e **apenas leitura**.

`Int16Array.prototype.byteOffset` Somente leitura

Retorna o deslocamento (em bytes) de um `Int16Array` desde o ínicio `ArrayBuffer`. Fixado em tempo de construção e **apenas leitura**.

`Int16Array.prototype.length` Somente leitura

Retorna o número de elementos em um `Int16Array`. Fixado em tempo de construção e **apenas leitura**.

Métodos

`Int16Array.prototype.copyWithin()`

Copia uma sequência de elementos da matriz dentro da matriz. Veja também `Array.prototype.copyWithin()`.

`Int16Array.prototype.entries()`

Retorna um novo `Array Iterator` objeto que contém os pares de chave / valor para cada índice na matriz. Veja também `Array.prototype.entries()`.

`Int16Array.prototype.every()`

Testa se todos os elementos na matriz passam no teste fornecido por uma função. Veja também `Array.prototype.every()`.

`Int16Array.prototype.fill()`

Preenche todos os elementos de uma matriz de um índice inicial para um índice final com um valor estático. Veja também `Array.prototype.fill()`.

`Int16Array.prototype.filter()`

Cria uma nova matriz com todos os elementos dessa matriz para os quais a função de filtragem fornecida retorna true. Veja também `Array.prototype.filter()`.

`Int16Array.prototype.find()`

Retorna o valor encontrado na matriz, se um elemento na matriz satisfizer a função de teste fornecida ou `undefined` se não encontrado. Veja também `Array.prototype.find()`.

`Int16Array.prototype.findIndex()`

Retorna o índice encontrado na matriz, se um elemento na matriz satisfizer a função de teste fornecida ou -1, se não for encontrado. Veja também `Array.prototype.findIndex()`.

`Int16Array.prototype.forEach()`

Chama uma função para cada elemento na matriz. Veja também `Array.prototype.forEach()`.

`Int16Array.prototype.includes()`

Determina se uma matriz tipificada inclui um determinado elemento, retornando `true` ou `false`. Veja também `Array.prototype.includes()`.

`Int16Array.prototype.indexOf()`

Retorna o primeiro (menos) índice de um elemento dentro da matriz igual ao valor especificado ou -1 se nenhum for encontrado. Veja também `Array.prototype.indexOf()`.

`Int16Array.prototype.join()`

Junta todos os elementos de um array em uma string. Veja também `Array.prototype.join()`.

`Int16Array.prototype.keys()`

Retorna um novo `Array Iterator` que contém uma chave para cada índice no array. Veja também `Array.prototype.keys()`.

`Int16Array.prototype.lastIndexOf()`

Retorna o último (maior) índice de um elemento dentro da matriz igual ao valor especificado ou -1 se nenhum for encontrado. Veja também `Array.prototype.lastIndexOf()`.

`Int16Array.prototype.map()`

Cria uma nova matriz com os resultados da chamada de uma função fornecida em todos os elementos dessa matriz. Veja também `Array.prototype.map()`.

`Int16Array.prototype.move()`

Versão não-padrão anterior de `Int16Array.prototype.copyWithin()`.

`Int16Array.prototype.reduce()`

Aplique uma função contra um acumulador e cada valor da matriz (da esquerda para a direita) para reduzi-lo a um único valor. Veja também `Array.prototype.reduce()`.

`Int16Array.prototype.reduceRight()`

Aplique uma função contra um acumulador e cada valor da matriz (da direita para a esquerda) para reduzi-lo a um único valor. Veja também `Array.prototype.reduceRight()`.

`Int16Array.prototype.reverse()`

Inverte a ordem dos elementos de um array - o primeiro torna-se o último e o último torna-se o primeiro. Veja também `Array.prototype.reverse()`.

`Int16Array.prototype.set()`

Armazena vários valores na matriz tipada, lendo valores de entrada de uma matriz especificada.

`Int16Array.prototype.slice()`

Extrai uma seção de uma matriz e retorna uma nova matriz. Veja também `Array.prototype.slice()`.

`Int16Array.prototype.some()`

Retorna true se pelo menos um elemento nessa matriz satisfizer a função de teste fornecida. Veja também `Array.prototype.some()`.

`Int16Array.prototype.sort()`

Classifica os elementos de uma matriz e retorna a matriz. Veja também `Array.prototype.sort()`.

`Int16Array.prototype.subarray()`

Retorna um novo `Int16Array` a partir do índice de elemento inicial e final fornecido.

`Int16Array.prototype.values()`

Retorna um novo objeto `Array Iterator` que contém os valores para cada índice na matriz. Veja também `Array.prototype.values()`.

`Int16Array.prototype.toLocaleString()`

Retorna uma string localizada representando a matriz e seus elementos. Veja também `Array.prototype.toLocaleString()`.

`Int16Array.prototype.toString()`

Retorna uma string representando a matriz e seus elementos. Veja também `Array.prototype.toString()`.

`Int16Array.prototype[@@iterator]()`

Retorna um novo objeto `Array Iterator` que contém os valores para cada índice na lista.

Exemplos

Diferentes opções de criar um `Int16Array`:

```
// De um tamanho
var int16 = new Int16Array(2);
int16[0] = 42;
console.log(int16[0]); // 42
console.log(int16.length); // 2
console.log(int16.BYTES_PER_ELEMENT); // 2

// De uma array
var arr = new Int16Array([21, 31]);
console.log(arr[1]); // 31

// De um outro TypedArray
var x = new Int16Array([21, 31]);
var y = new Int16Array(x);
console.log(y[0]); // 21

// De um ArrayBuffer
var buffer = new ArrayBuffer(8);
var z = new Int16Array(buffer, 0, 4);

// De um interável
```

```
var iterable = (function* () {
  yield* [1, 2, 3];
})();
var int16 = new Int16Array(iterable);
// Int16Array[1, 2, 3]
```

Notas de compatibilidade

A partir do ECMAScript 2015, os construtores `Int16Array` precisam ser construídos com um operador `new`. Chamar um construtor `Int16Array` como uma função sem o `new`, irá resultar em `TypeError`.

```
var dv = Int16Array([1, 2, 3]);
// TypeError: calling a builtin Int16Array constructor
// without new is forbidden
```

```
var dv = new Int16Array([1, 2, 3]);
```

Ver também

- JavaScript typed arrays
- `ArrayBuffer`
- `DataView`

InternalError

> **Não padrão:** Este recurso não é padronizado. Não recomendamos o uso de recursos não padronizados em produção, pois eles têm suporte limitado nos navegadores e podem mudar ou ser removidos. No entanto, podem ser uma alternativa viável em casos específicos onde não exista uma opção padrão.

O **objeto** `InternalError` indica que um erro ocorreu internamente na engine do JavaScript.

Isso ocorre quando algo é muito grande, por exemplo:

- "too many switch cases",
- "too many parentheses in regular expression",
- "array initializer too large",
- "too much recursion".

Construtor

`InternalError()`

Cria um um novo objeto `InternalError`.

Instance properties

`InternalError.prototype.message`

Error message. Inherited from `Error`.

`InternalError.prototype.name`

Error name. Inherited from `Error`.

`InternalError.prototype.fileName`

Path to file that raised this error. Inherited from `Error`.

`InternalError.prototype.lineNumber`

Line number in file that raised this error. Inherited from `Error`.

`InternalError.prototype.columnNumber`

Column number in line that raised this error. Inherited from `Error`.

`InternalError.prototype.stack`

Stack trace. Inherited from `Error`.

Examples

Too much recursion

This recursive function runs 10 times, as per the exit condition.

```
function loop(x) {
  if (x >= 10) {
    // "x >= 10" is the exit condition
    return;
  }
  // do stuff
  loop(x + 1); // the recursive call
```

```
}
loop(0);
```

Setting this condition to an extremely high value, won't work:

```
function loop(x) {
  if (x >= 1000000000000) return;
  // do stuff
  loop(x + 1);
}
loop(0);

// InternalError: too much recursion
```

For more information, see InternalError: too much recursion.

Specifications

Not part of any standard.

See also

- `Error`
- InternalError: too much recursion

Intl

O objeto **`Intl`** é o namespace para a API de Internacionalização do ECMAScript , que fornece comparação de string sensível à línguagem, formatação de números, e formatação de data e hora. Os construtores para os objetos `Collator`, `NumberFormat`, e `DateTimeFormat` são propriedades do objecto `Intl`. Esta página documenta essas propriedades, bem como funcionalidades comuns aos construtores de internacionalização e outras funções sensíveis de linguagem.

Propriedades

`Intl.Collator`

> Construtor para *collators*, objetos que permitem comparação de string sensível a linguagem.

`Intl.DateTimeFormat`

> Construtor para objetos que permitem formatação de data e hora sensível a linguagem.

`Intl.NumberFormat`

> Construtor para objetos que permitem formatação de número sensível a linguagem.

Métodos

`Intl.getCanonicalLocales()`

> Retorna os nomes canônicos de local (ex.: en-US, pt-BR).

Identificação e negociação de local

Os construtores de internacionalização, assim como diversos métodos de outros construtores que são sensíveis a idioma (listados em Veja também) usam um padrão comum para identificar locais e determinar qual será utilizado: todos aceitam argumentos `locales` e `options` e negociam o(s) local(is) requisitado entre os locais suportados usando um algoritmo especificado na propriedade `options.localeMatcher`.

Argumento <code>locales</code>

O argumento `locales` deve ser uma string contendo uma tag de linguagem BCP 47 ou um array dessas tags. Se o argumento `locales` não for passado ou estiver indefinido, será utilizado o local padrão do runtime.

Uma tag de linguagem BCP 47 identifica um idioma ou local (a diferença entre ambos neste caso é difusa). Em sua forma mais comum, ela pode conter, nesta ordem: um código de idioma, um código de escrita e um código de país, todos eparados por hífen. Exemplos:

- `"hi"`: Hindi.
- `"de-AT"`: Alemão como usado na Áustria.
- `"zh-Hans-CN"`: Chinês com escrita simplificada como usado na China.

As subtags identificando idiomas, escritas, países (regiões) e (raramente utilizadas) variantes nas tags de linguagem BCP 47 podem ser consultadas no Registro de Subtags de Linguagem da IANA.

BCP 47 também permite extensões, e uma delas é relevante para as funções JavaScript de internacionalização: a extensão `"u"` (Unicode). Ela pode ser utilizada para requisitar uma customização do comportamento específico local de um objeto `Collator`, `NumberFormat`, ou `DateTimeFormat`. Exemplos:

- `"de-DE-u-co-phonebk"`: Use the phonebook variant of the German sort order, which expands umlauted vowels to character pairs: ä → ae, ö → oe, ü → ue.
- `"th-TH-u-nu-thai"`: Use Thai digits (๐, ๑, ๒, ๓, ๔, ๕, ๖, ๗, ๘, ๙) in number formatting.
- `"ja-JP-u-ca-japanese"`: Use the Japanese calendar in date and time formatting, so that 2013 is expressed as the year 25 of the Heisei period, or 平成 25.

Locale negotiation

The `locales` argument, after stripping off all Unicode extensions, is interpreted as a prioritized request from the application. The runtime compares it against the locales it has available and picks the best one available. Two matching algorithms exist: the `"lookup"` matcher follows the Lookup algorithm specified in BCP 47; the

`"best fit"` matcher lets the runtime provide a locale that's at least, but possibly more, suited for the request than the result of the Lookup algorithm. If the application doesn't provide a `locales` argument, or the runtime doesn't have a locale that matches the request, then the runtime's default locale is used. The matcher can be selected using a property of the `options` argument (see below).

If the selected language tag had a Unicode extension substring, that extension is now used to customize the constructed object or the behavior of the function. Each constructor or function supports only a subset of the keys defined for the Unicode extension, and the supported values often depend on the language tag. For example, the `"co"` key (collation) is only supported by `Collator`, and its `"phonebk"` value is only supported for German.

<code>options</code> argument

The `options` argument must be an object with properties that vary between constructors and functions. If the `options` argument is not provided or is undefined, default values are used for all properties.

One property is supported by all language sensitive constructors and functions: The `localeMatcher` property, whose value must be a string `"lookup"` or `"best fit"` and which selects one of the locale matching algorithms described above.

Veja também

- Introduction: The ECMAScript Internationalization API
- Constructors
  - `Intl.Collator`
  - `Intl.DateTimeFormat`
  - `Intl.NumberFormat`
- Methods
  - `String.prototype.localeCompare()`
  - `Number.prototype.toLocaleString()`
  - `Date.prototype.toLocaleString()`
  - `Date.prototype.toLocaleDateString()`
  - `Date.prototype.toLocaleTimeString()`

Intl.Collator

O objeto **`Intl.Collator`** permite uma comparação de strings sensível à linguagem.

Experimente

```
console.log(["Z", "a", "z", "ä"].sort(new Intl.Collator("de").compare));
// Expected output: Array ["a", "ä", "z", "Z"]

console.log(["Z", "a", "z", "ä"].sort(new Intl.Collator("sv").compare));
// Expected output: Array ["a", "z", "Z", "ä"]

console.log(
  ["Z", "a", "z", "ä"].sort(
    new Intl.Collator("de", { caseFirst: "upper" }).compare,
  ),
);
// Expected output: Array ["a", "ä", "Z", "z"]
```

Construtor

`Intl.Collator()`

Cria um novo objeto `Collator`.

Métodos estáticos

`Intl.Collator.supportedLocalesOf()`

Retorna um array contendo as localizações (`locales`) providas que são suportados sem a necessidade de depender da localização padrão do runtime.

Métodos de instância

`Intl.Collator.prototype.compare`

Função Getter que compara duas strings conforme a ordenação deste objeto `Intl.Collator`.

`Intl.Collator.prototype.resolvedOptions()`

Retorna um novo objeto com as propriedades que refletem as opções de localização (`locale`) e agrupamento (`collation`) computadas durante a inicialização do objeto.

Exemplos

Usando Collator

O exemplo a seguir demonstra os diferentes potenciais resultados para uma string que aparece antes, depois, ou no mesmo nível que outra:

```
console.log(new Intl.Collator().compare("a", "c")); // -1, ou algum outro valor negativo
console.log(new Intl.Collator().compare("c", "a")); // 1, ou algum outro valor positivo
console.log(new Intl.Collator().compare("a", "a")); // 0
```

É importante notar que os resultados mostrados no código acima podem variar entre navegadores e versões de navegadores. Isso por os valores serem específicos a implementação. Isso ocorre, pois a implementação define somente que os valores para antes e depois sejam negativo e positivo.

Usando localizações

Os resultados retornados por `Intl.Collator.prototype.compare()` variam entre linguagens. Para obter a sequência de ordenação da linguagem usada na interface da sua aplicação, garanta a especificação desta linguagem (e possivelmente algumas linguagens reserva) usando o argumento `locales`:

```
// Em alemão, ä está no mesmo nível de a
console.log(new Intl.Collator("de").compare("ä", "z"));
// -1, ou algum outro valor negativo

// Em Sueco, ä vem depois de z
console.log(new Intl.Collator("sv").compare("ä", "z"));
// 1, ou algum outro valor positivo
```

Usando opções

Os resultados retornados por `Intl.Collator.prototype.compare()` podem ser customizados usando o argumento `options`:

```
// Em Alemão, ä tem a como sua letra base
console.log(new Intl.Collator("de", { sensitivity: "base" }).compare("ä", "a"));
// 0

// Em Sueco, ä e a são letras base separadas
console.log(new Intl.Collator("sv", { sensitivity: "base" }).compare("ä", "a"));
// 1, ou algum outro valor positivo
```

Veja também

- `Intl`

Intl.ListFormat

O objeto **`Intl.ListFormat`** habilita a formatação de lista de acordo com o idioma.

Experimente

```
const vehicles = ["Motorcycle", "Bus", "Car"];

const formatter = new Intl.ListFormat("en", {
  style: "long",
  type: "conjunction",
});
console.log(formatter.format(vehicles));
// Expected output: "Motorcycle, Bus, and Car"

const formatter2 = new Intl.ListFormat("de", {
  style: "short",
  type: "disjunction",
});
console.log(formatter2.format(vehicles));
// Expected output: "Motorcycle, Bus oder Car"

const formatter3 = new Intl.ListFormat("en", { style: "narrow", type: "unit" });
console.log(formatter3.format(vehicles));
// Expected output: "Motorcycle Bus Car"
```

Construtor

`Intl.ListFormat()`

Criar um novo objeto `Intl.ListFormat`.

Métodos estáticos

`Intl.ListFormat.supportedLocalesOf()`

Retorna um array com as localidades fornecidas com suporte sem precisar depender da localidade padrão do tempo de execução.

Métodos de instância

`Intl.ListFormat.prototype.format()`

Retorna uma string formatada especifica do idioma representando os elementos da lista.

`Intl.ListFormat.prototype.formatToParts()`

Retorna um array de objetos representando os diferentes componentes que podem ser usados para formatar uma lista de valores com reconhecimento de localidade.

Exemplos

Utilizando format

O exemplo a seguir mostra como criar o List formatter utilizando o idioma Português.

```
const list = ["Moto", "Ônibus", "Carro"];

console.log(
  new Intl.ListFormat("pt-BR", { style: "long", type: "conjunction" }).format(
    list,
  ),
);
// > Moto, Ônibus e Carro

console.log(
  new Intl.ListFormat("pt-BR", { style: "short", type: "disjunction" }).format(
    list,
  ),
);
// > Moto, Ônibus ou Carro

console.log(
  new Intl.ListFormat("pt-BR", { style: "narrow", type: "unit" }).format(list),
);
// > Moto Ônibus Carro
```

Utilizando formatToParts

O exemplo a seguir mostra como criar o List formatter retornando as partes formatadas.

```
const list = ["Moto", "Ônibus", "Carro"];
console.log(
  new Intl.ListFormat("pt-BR", {
    style: "long",
    type: "conjunction",
  }).formatToParts(list),
);

// [ { "type": "element", "value": "Moto" },
//   { "type": "literal", "value": ", " },
//   { "type": "element", "value": "Ônibus" },
//   { "type": "literal", "value": ", e " },
//   { "type": "element", "value": "Carro" } ];
```

Veja também

- `Intl`
- Um polyfill para `Intl.ListFormat` no FormatJS

Construtor Intl.ListFormat()

O construtor **`Intl.ListFormat()`** cria objetos `Intl.ListFormat` que habilitam a formatação de lista de acordo com o idioma.

Experimente

```
const vehicles = ["Motorcycle", "Bus", "Car"];

const formatter = new Intl.ListFormat("en", {
  style: "long",
  type: "conjunction",
});
console.log(formatter.format(vehicles));
// Expected output: "Motorcycle, Bus, and Car"

const formatter2 = new Intl.ListFormat("de", {
  style: "short",
  type: "disjunction",
});
console.log(formatter2.format(vehicles));
// Expected output: "Motorcycle, Bus oder Car"

const formatter3 = new Intl.ListFormat("en", { style: "narrow", type: "unit" });
console.log(formatter3.format(vehicles));
// Expected output: "Motorcycle Bus Car"
```

Sintaxe

```
new Intl.ListFormat();
new Intl.ListFormat(locales);
new Intl.ListFormat(locales, options);
```

Parâmetros

`locales` Optional

Uma string com a tag de idioma BCP 47, ou um array de tais string. Veja de forma geral a interpretação do argumento `locales`, na página Intl.

`options` Optional

Um objeto com algumas ou todas as seguintes propriedades:

`localeMatcher`

O algoritmo de correspondência de localidade para ser utilizado. Os possíveis valores são "`lookup`" e "`best fit`"; o valor padrão é "`best fit`". Para mais informações sobre esta opção, veja a página Intl.

`type`

O formato de saída da mensagem. Os possíveis valores são "`conjunction`" que representa listas com "e" (padrão, e.g., "`A, B, e C`"), ou "`disjunction`" que representa listas com "ou" (e.g., "`A, B, ou C`"). "`unit`" que representa listas com valores de unidade (e.g., "`5 libras, 12 onças`").

`style`

O tamanho da lista formatada. Os possíveis valores são: "`long`" (padrão, e.g., "`A, B, e C`"); "`short`" (e.g., "`A, B, C`"), ou "`narrow`" (e.g., "`A B C`"). quando `style` é "`short`" ou "`narrow`", "`unit`" é o único valor de tipo permitido.

Exemplos

Utilizando format

O exemplo a seguir mostra como criar um List formatter utilizando o idioma Português.

```
const list = ["Moto", "Ônibus", "Carro"];

console.log(
  new Intl.ListFormat("pt-BR", { style: "long", type: "conjunction" }).format(
    list,
  ),
);
// > Moto, Ônibus e Carro

console.log(
  new Intl.ListFormat("pt-BR", { style: "short", type: "disjunction" }).format(
    list,
  ),
);
// > Moto, Ônibus ou Carro

console.log(
  new Intl.ListFormat("pt-BR", { style: "narrow", type: "unit" }).format(list),
);
// > Moto Ônibus Carro
```

Veja também

- `Intl.ListFormat`
- `Intl`

Intl.NumberFormat

O objeto **`Intl.NumberFormat`** é um construtor para objetos que habilita formatação de número sensível a linguagem.

Experimente

```
const number = 123456.789;

console.log(
  new Intl.NumberFormat("de-DE", { style: "currency", currency: "EUR" }).format(
    number,
  ),
);
// Expected output: "123.456,79 €"

// The Japanese yen doesn't use a minor unit
console.log(
  new Intl.NumberFormat("ja-JP", { style: "currency", currency: "JPY" }).format(
    number,
  ),
);
// Expected output: "￥123,457"

// Limit to three significant digits
console.log(
  new Intl.NumberFormat("en-IN", { maximumSignificantDigits: 3 }).format(
    number,
  ),
```

```
);
// Expected output: "1,23,000"
```

Sintaxe

```
new Intl.NumberFormat([locales[, options]])
Intl.NumberFormat.call(this[, locales[, options]])
```

Parâmetros

`locales`

Opcional. Uma string com uma tag de linguagem BCP 47 ou uma matriz delas. Para a forma geral e interpretação do argumento `locales`, veja Intl page. A seguinte chave extendida Unicode é permitida:

`nu`

O sistema de numeração que será usado. Os valores permitidos são: `"arab"`, `"arabext"`, `"bali"`, `"beng"`, `"deva"`, `"fullwide"`, `"gujr"`, `"guru"`, `"hanidec"`, `"khmr"`, `"knda"`, `"laoo"`, `"latn"`, `"limb"`, `"mlym"`, `"mong"`, `"mymr"`, `"orya"`, `"tamldec"`, `"telu"`, `"thai"`, `"tibt"`.

`options`

Opcional. Um objeto com alguns ou todas as seguintes propriedades:

`localeMatcher`

O algoritmo de comparação de localização para utilizar. Os valores permitidos são `"lookup"` e `"best fit"`; o padrão é `"best fit"`. Para mais informações sobre esta opção, veja Intl page.

`style`

O estilo do formato a ser utilizado. Os valores permitidos são `"decimal"` para formato de número simples, `"currency"` para formato monetário e `"percent"` para formato percentual; o padrão é `"decimal"`.

`currency`

A moeda para usar na formatação monetária. Os valores permitidos são os códigos de moedas da ISO 4217, como `"USD"` para dólar estadunidense, `"EUR"` para euro, ou `"CNY"` para RMB chinês — veja a Lista de códigos de moedas e fundos atuais. Não há valor padrão; se o `style` for `"currency"`, a propriedade `currency` deverá ser informada.

`currencyDisplay`

Como será mostrada a moeda na formatação monetária. Os valores permitidos são `"symbol"` para usar um símbolo de moeda localizado como €, `"code"` para usar o código de moeda ISO, `"name"` para usar o nome da moeda localizado como `"dollar"`; o padrão é `"symbol"`.

`useGrouping`

Se usar separadores de agrupamento, como separadores de milhares ou milhares/cem mil/dez milhões. Os valores permitidos são `true` e `false`; o padrão é `true`.

As próximas propriedades se dividem em dois grupos: `minimumIntegerDigits`, `minimumFractionDigits`, e `maximumFractionDigits` no primeiro grupo, `minimumSignificantDigits` and `maximumSignificantDigits` em outro. Se pelo menos uma propriedade do segundo grupo for informado, então o primeiro grupo é ignorado.

`minimumIntegerDigits`

A quantidade mínima de dígitos inteiros para utilizar. É possível usar valores de 1 a 21; o padrão é 1.

`minimumFractionDigits`

A quantidade mínima de dígitos fracionados para utilizar. É possível usar valores de 0 a 20; o padrão para formatos de números simples e percentuais é 0; o padrão para formatos monetários é a menor unidade de dígitos fornecidos pela lista de códigos de moedas ISO 4217 (2 se a lista não fornecer a informação).

`maximumFractionDigits`

O número máximo de dígitos fracionados para utilizar. É possível usar valores de 0 a 20; o padrão para a formatação de número simples é o maior entre `minimumFractionDigits` e 3; o padrão para formatos monetários é o maior número de dígitos entre `minimumFractionDigits` e o fornecido pela lista de códigos de moedas ISO 4217 (2 se a lista não fornecer a informação); o padrão para a formatação percentual é o maior número entre `minimumFractionDigits` e 0.

`minimumSignificantDigits`

A quantidade mínima de dígitos significantes para usar. Os valores permitidos são de 1 a 21; o padrão é 1.

`maximumSignificantDigits`

A quantidade máxima de dígitos significantes para usar. Os valores permitidos são de 1 a 21; o padrão é `minimumSignificantDigits`.

Descrição

Propriedades

`Intl.NumberFormat.prototype`

Pertime a inclusão de propriedades a todos os objetos.

Métodos

`Intl.NumberFormat.supportedLocalesOf()`

Retorna uma matriz contendo as localizações fornecidas que são suportadas sem retornar a localização padrão em tempo de execução.

Instâncias <code>NumberFormat</code>

Propriedades

As instâncias de `NumberFormat` herdam as seguntes propriedades de seu protótipo:

Methods

As instâncias de `NumberFormat` herdam os seguintes métodos de seu protótipo:

Exemplos

Uso básico

No uso básico sem a especificação de uma localização, o método retornará uma string formatada com a localização e as opções padrão.

```
var numero = 3500;

console.log(new Intl.NumberFormat().format(numero));
// → '3,500' se a localização for U.S. English
```

Usando <code>locales</code>

Este exemplo mostra algumas variações de formatos de números localizados. A fim de obter o formato da linguagem utilizada na interface do usuário da sua aplicação, tenha certeza de especificar a língua (e possivelmente algumas línguas reservas) usando o argumento `locales`:

```
var numero = 123456.789;

// O alemão usa vírgula como separador de decimal e ponto para milhares
console.log(new Intl.NumberFormat("de-DE").format(numero));
// → 123.456,789

// O árabe usa dígitos reais árabes em muitos países que falam árabe
console.log(new Intl.NumberFormat("ar-EG").format(numero));
// → ١٢٣٤٥٦٫٧٨٩

// A Índia usa separadores de milhares/cem mil/dez milhões
console.log(new Intl.NumberFormat("en-IN").format(numero));
// → 1,23,456.789

// A chave de extensão nu requer um sistema de numeração, ex. decimal chinês
console.log(new Intl.NumberFormat("zh-Hans-CN-u-nu-hanidec").format(numero));
// → 一二三,四五六.七八九

// Quando informada uma língua sem suporte, como balinês,
// inclua uma língua reseva, neste caso indonésio
console.log(new Intl.NumberFormat(["ban", "id"]).format(numero));
// → 123.456,789
```

Usando <code>options</code>

Os resultados podem ser personalizados usando o argumento `options`:

```
var numero = 123456.789;

// informando um formato de moeda
console.log(
  new Intl.NumberFormat("de-DE", { style: "currency", currency: "EUR" }).format(
    numero,
  ),
);
// → 123.456,79 €

// o yen japonês não tem uma unidade menor
console.log(
  new Intl.NumberFormat("ja-JP", { style: "currency", currency: "JPY" }).format(
    numero,
  ),
);
// → ￥123,457

// limitando a três dígitos significativos
console.log(
  new Intl.NumberFormat("en-IN", { maximumSignificantDigits: 3 }).format(
```

```
    numero,
  ),
);
// → 1,23,000
```

Veja também

Intl.RelativeTimeFormat

O objeto **`Intl.RelativeTimeFormat`** é um construtor de objetos que permitem uma formatação de tempo relativa sensível ao idioma.

Experimente

```
const rtf1 = new Intl.RelativeTimeFormat("en", { style: "short" });

console.log(rtf1.format(3, "quarter"));
// Expected output: "in 3 qtrs."

console.log(rtf1.format(-1, "day"));
// Expected output: "1 day ago"

const rtf2 = new Intl.RelativeTimeFormat("es", { numeric: "auto" });

console.log(rtf2.format(2, "day"));
// Expected output: "pasado mañana"
```

Sintaxe

```
new Intl.RelativeTimeFormat([locales[, options]])
```

Parâmetros

`locales`

Opcional. Uma string com uma tag da linguagem BCP 47, ou um array dessas strings. Para a forma geral e interpretação do argumento `locales` , acesse: Página Intl.

`options`

Opcional. Um objeto com algumas ou todas as seguintes propriedades:

- `localeMatcher` O algoritmo para comparação de local a ser usado. Os valores possíveis são `"lookup"` e `"best fit"`; o padrão é `"best fit"`. Para informações sobre esta opção, veja `Intl`.
- `numeric` O formato de saída da mensagem. Valores possíveis são:
  - `"always"` (padrão, e.g., `há 1 dia`),
  - ou `"auto"` (e.g., `ontem`). O valor`"auto"` permite que não seja sempre necessário o uso de valores númericos na saída.
- `style` O comprimento da mensagem internacionalizada. Valores possíveis são:
  - `"long"` (padrão, e.g., `in 1 month`)
  - `"short"` (e.g., `in 1 mo.`),
  - ou `"narrow"` (e.g., `in 1 mo.`). O estilo narrow pode ser similar ao short em alguns locais.

Descrição

Propriedades

`Intl.RelativeTimeFormat.prototype`

Permite a adição de propriedades para todos os objetos.

Métodos

`Intl.RelativeTimeFormat.supportedLocalesOf()`

Retorna um array contendo os valores disponíveis dentre os que foram passados como parâmetro sem ter de recorrer ao local padrão do ambiente.

Instâncias <code>RelativeTimeFormat</code>

Propriedades

Instâncias `RelativeTimeFormat` herdam as seguintes propriedades do seu protótipo:

Métodos

Instâncias `RelativeTimeFormat` herdam as seguintes propriedades do seu protótipo:

Exemplos

Uso básico do <code>format</code>

O exemplo a seguir mostra como criar um formatador de tempo relativo usando a língua portuguesa.

```
// Crie um formatador de tempo relativo no seu local
// com os valores padrão sendo passados explicitamente.
const rtf = new Intl.RelativeTimeFormat("pt", {
  localeMatcher: "best fit", // outros valores: "lookup"
  numeric: "always", // outros valores: "auto"
  style: "long", // outros valores: "short" ou "narrow"
});

// Formatação de tempo relativa usando valor negativo (-1).
rtf.format(-1, "day");
// > "há 1 dia"

// Formatação de tempo relativa usando valor positivo (1).
rtf.format(1, "day");
// > "em 1 dia"
```

Usando a opção <code>auto</code>

Se a opção `numeric:auto` é passada, serão produzidas as strings `ontem` ou `amanhã` ao invés de `há 1 dia` ou `em 1 dia`. Isso permite que não seja necessário sempre utilizar valores numéricos na saída.

```
// Crie um formatador de tempo relativo no seu local
// com o valor "auto" passado para a propriedade numeric.
const rtf = new Intl.RelativeTimeFormat("pt", { numeric: "auto" });

// Formatação de tempo relativa usando valor negativo (-1).
rtf.format(-1, "day");
// > "ontem"

// Formatação de tempo relativa usando valor positivo (1).
rtf.format(1, "day");
// > "amanhã"
```

Usando <code>formatToParts</code>

O exemplo a seguir mostra como criar um formatador de tempo relativo que retorna partes formatadas

```
const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });

// Formatação de tempo relativa usando a unidade day.
rtf.formatToParts(-1, "day");
// > [{type: "literal", value: "ontem"}]

rtf.formatToParts(100, "day");
// > [{type: "literal", value: "em "},
// >  { type: "integer", value: "100", unit: "day" },
// >  {type: "literal", value: " dias"]
```

Especificações

Especificação	Estágio	Comentário
Intl.RelativeTime proposal	Stage 3	

Veja também

- The Intl.RelativeTimeFormat API

isFinite()

`A função global isFinite()` determina se o valor transmitido é um número finito. Se necessário, o parâmetro é primeiro convertido a um número.

Sintaxe

```
isFinite(testValue)
```

Parametros

`testValue`

O valor a ser testado para a finitude

Descrição

`isFinite` é uma função de nível superior é não é associada com qualquer objeto.

Você pode usar esta função para determinar se um número é um número finito. `A função isFinite` examina o número em seu argumento. Se o argumento é `NaN`, positivo infinito, ou negativo infinito, este método retorna `false`; de outra forma , ele retorna `true`.

Exemplos

```
isFinite(Infinity); // false
isFinite(NaN); // false
```

```
isFinite(-Infinity); // false

isFinite(0); // true
isFinite(2e64); // true
isFinite(null); // true

isFinite("0"); // true, teria sido false com o
// mais robusto Number.isFinite("0")
```

Veja também

- `Number.isFinite()`
- `Number.NaN()`
- `Number.POSITIVE_INFINITY`
- `Number.NEGATIVE_INFINITY`

isNaN()

A função **`isNaN()`** determina se o valor é `NaN` ou não. Tenha cuidado, o `cast` em `isNaN` tem regras para serem observadas. Você pode ficar interessado no `Number.isNaN()` que foi definido no ECMAScript 6 ou você pode usar `typeof` para determinar se o valor é Not-A-Number, `NaN`.

Experimente

```
function milliseconds(x) {
  if (isNaN(x)) {
    return "Not a Number!";
  }
  return x * 1000;
}

console.log(milliseconds("100F"));
// Expected output: "Not a Number!"

console.log(milliseconds("0.0314E+2"));
// Expected output: 3140
```

Syntaxe

```
isNaN(value)
```

Parâmetros

`value`

O valor a ser testado.

Valor retornado

`true` se o valor for `NaN`; otherwise, **`false`**.

Descrição

A necessidade da função <code>isNaN</code>

Ao contrário de todas as outras possibilidades de valores no JavaScript, não é possivel confiar nos operadores de igualdade `(== e ===)` para determina se o valor é `NaN` ou não, porque ambos, `NaN == NaN` e `NaN === NaN`, terá como valor de retorno: `false`. Daí a necessidade da funçao `isNAN`.

Origem dos valores <code>NaN</code>

Os valores `NaN` são gerados quando operações aritiméticas tem como valores: *undefined* ou *unrepresentable*. Como valores, não fazem necessariamente condições de estouro. `NaN` também resulta da tentativa de coerção para valores numéricos, de valores não numéricos, para que o valor numérico primitivo seja disponível.

Por exemplo, divida zero por zero e o resultado será `NaN` , mas se dividir outros números por zero ele não será.

Comportamento confuso do caso especial

Desde as primeiras versões da especificação da função isNaN , o seu comportamento para argumentos não numéricos tem sido confuso. Quando o argumento para a função `isNaN` não é do tipo* **Number***, o valor é primeiro convertido para um número. O valor resultante é em seguida testado para determinar se se trata de `Number.isNaN()`. Assim, para os não-números que quando forçados ao tipo numérico resultado em um valor numérico não-`NaN` válido (particularmente strings vazias e primitivas booleanas, que quando submetidas ao `cast` dão valores numéricos zero ou um), o "falso" valor retornado pode ser inesperado; a cadeia vazia , por exemplo, é certamente "not-a-number" A confusão decorre do fato de que o termo, " não é um número " , tem um significado específico para os números representados como valores de ponto flutuante IEEE- 794 . A função deve ser interpretada como respondendo à pergunta: "este valor, quando convertido para um valor numérico, um valor IEEE -794 ' not-a-number"?

A ultima versão do ECMAScript (ES6) contém A função `Number.isNaN()`. `Number.isNaN(x)` será a real forma para testar se x é NaN ou não. Mesmo com `Number.isNaN`, porém, o significado de NaN continua ser a precisão numérica, e não simplesmente, "não é um número = `NaN`, *not a number*". Paralelamente , na expressão `Number.isNaN`, a expressão `(x != x)` é a forma mais confiável para testar se a variável `x` é `NaN` ou não, assim o resultado não é sujeito ao falso positivo que faz `isNaN` não ser confiável.

A polyfill for `isNaN` would be (the polyfill leverages the unique never-equal-to-itself characteristic of `NaN`):

```
var isNaN = function (value) {
  var n = Number(value);
  return n !== n;
};
```

Exemplos

```
isNaN(NaN); // true
isNaN(undefined); // true
isNaN({}); // true

isNaN(true); // false
isNaN(null); // false
isNaN(37); // false

// strings
isNaN("37"); // false: "37" is converted to the number 37 which is not NaN
isNaN("37.37"); // false: "37.37" is converted to the number 37.37 which is not NaN
isNaN(""); // false: the empty string is converted to 0 which is not NaN
isNaN(" "); // false: a string with spaces is converted to 0 which is not NaN

// dates
isNaN(new Date()); // false
isNaN(new Date().toString()); // true

// Esse é um falso positivo e é a razão para isNaN não seja totalmente confiável.
isNaN("blabla"); // true: "blabla" é convertido para número.
// A análise desse número falha e retorna NaN como resultado.
```

Useful special-case behavior

There is a more usage oriented way to think of `isNaN()`: If `isNaN(x)` returns `false`, you can use `x` in an arithmetic expression not making the expression return `NaN`. If it returns `true`, `x` will make every arithmetic expression return `NaN`. This means that in JavaScript, `isNaN(x) == true` is equivalent to `x - 0` returning `NaN` (though in JavaScript `x - 0 == NaN` always returns false, so you can't test for it). Actually, `isNaN(x)`, `isNaN(x - 0)`, `isNaN(Number(x))`, `Number.isNaN(x - 0)`, and `Number.isNaN(Number(x))` always return the same and in JavaScript `isNaN(x)` is just the shortest possible form to express each of these terms.

You can use this, for example, to test whether an argument to a function is arithmetically processable (usable "like" a number), or if it's not and you have to provide a default value or something else. This way you can have a function that makes use of the full versatility JavaScript provides by implicitly converting values depending on context.

Veja também

- `NaN`
- `Number.isNaN()`

JSON

Resumo

O Objeto **`JSON`** contém métodos para parsing JavaScript Object Notation (JSON) e conversão de valores para JSON. Ele não pode ser chamado ou construído e, além de suas propriedades de dois métodos, ele não possui uma funcionalidade interessante.

Descrição

JavaScript Object Notation

JSON é uma sintaxe para serialização de objetos, matrizes, números, strings, booleanos, e `null`. Baseia-se em sintaxe Javascript, mas é distinta desta: alguns Javascript não são JSON, e alguns JSON não são Javascript.

JavaScript tipo	JSON diferenças
Objetos e Arrays	Os nomes das propriedades devem ser strings com aspas duplas; as vírgulas à direita são proibidas.
Números	Zeros à esquerda são proibidos; um ponto decimal deve ser seguido por pelo menos um dígito.
Strings	Apenas um conjunto limitado de caracteres pode ser escapado; certos caracteres de controle são proibidos; o separador de linha Unicode (U+2028) e o separador de parágrafo (U+2029) caracteres são permitidos; strings devem ter aspas duplas.Veja o exemplo a seguir, onde `JSON.parse()` funciona bem e um `SyntaxError` é lançado ao avaliar o código como JavaScript: var code = '"\u2028\u2029"'; JSON.parse(code); // works fine eval(code); // fails

A sintaxe completa do JSON é a seguinte:

```
JSON = null
    or true or false
    or JSONNumber
    or JSONString
    or JSONObject
    or JSONArray

JSONNumber = - PositiveNumber
          or PositiveNumber
```

```
PositiveNumber = DecimalNumber
               or DecimalNumber . Digits
               or DecimalNumber . Digits ExponentPart
               or DecimalNumber ExponentPart
DecimalNumber = 0
              or OneToNine Digits
ExponentPart = e Exponent
             or E Exponent
Exponent = Digits
         or + Digits
         or - Digits
Digits = Digit
       or Digits Digit
Digit = 0 through 9
OneToNine = 1 through 9

JSONString = ""
           or " StringCharacters "
StringCharacters = StringCharacter
                 or StringCharacters StringCharacter
StringCharacter = any character
                  except " or \ or U+0000 through U+001F
                or EscapeSequence
EscapeSequence = \" or \/ or \\ or \b or \f or \n or \r or \t
               or \u HexDigit HexDigit HexDigit HexDigit
HexDigit = 0 through 9
         or A through F
         or a through f

JSONObject = { }
           or { Members }
Members = JSONString : JSON
        or Members , JSONString : JSON

JSONArray = [ ]
          or [ ArrayElements ]
ArrayElements = JSON
              or ArrayElements , JSON
```

Espaços em branco podem estar presentes em qualquer lugar, exceto dentro de um JSONNumber (números não devem conter espaço em branco) ou JSONString (onde ele é interpretado como o caractere correspondente na string, ou causaria um erro). O caractere de tabulação (U+0009), retorno de carro (U+000D), retorno de linha (U+000A), e espaço (U+0020) são os únicos caracteres em branco válidos.

Métodos

`JSON.parse()`

> Analisar uma seqüência como JSON, opcionalmente transformar o valor produzido e suas propriedades, e retornar o valor.

`JSON.stringify()`

> Retorna uma string JSON correspondente ao valor especificado, opcionalmente, pode incluir apenas determinados propriedades ou substituir valores de propriedade de acordo com a definição feita pelo usuário.

Polyfill

O objeto JSON não é suportado em navegadores mais antigos. Você pode contornar este problema inserindo o seguinte código no início de seus scripts, permitindo o uso de JSON e navegadores sem suporte (como Internet Explorer 6).

O algoritmo a seguir é uma imitação do objeto nativo JSON:

```
if (!window.JSON) {
  window.JSON = {
    parse: function (sJSON) {
      return eval("(" + sJSON + ")");
    },
    stringify: (function () {
      var toString = Object.prototype.toString;
      var isArray =
        Array.isArray ||
        function (a) {
          return toString.call(a) === "[object Array]";
        };
      var escMap = {
        '"': '\\"',
        "\\": "\\\\",
        "\b": "\\b",
        "\f": "\\f",
        "\n": "\\n",
        "\r": "\\r",
        "\t": "\\t",
      };
      var escFunc = function (m) {
        return (
          escMap[m] ||
```

```
          "\\u" + (m.charCodeAt(0) + 0x10000).toString(16).substr(1)
        );
      };
      var escRE = /[\\"\u0000-\u001F\u2028\u2029]/g;
      return function stringify(value) {
        if (value == null) {
          return "null";
        } else if (typeof value === "number") {
          return isFinite(value) ? value.toString() : "null";
        } else if (typeof value === "boolean") {
          return value.toString();
        } else if (typeof value === "object") {
          if (typeof value.toJSON === "function") {
            return stringify(value.toJSON());
          } else if (isArray(value)) {
            var res = "[";
            for (var i = 0; i < value.length; i++)
              res += (i ? ", " : "") + stringify(value[i]);
            return res + "]";
          } else if (toString.call(value) === "[object Object]") {
            var tmp = [];
            for (var k in value) {
              if (value.hasOwnProperty(k))
                tmp.push(stringify(k) + ": " + stringify(value[k]));
            }
            return "{" + tmp.join(", ") + "}";
          }
        }
        return '"' + value.toString().replace(escRE, escFunc) + '"';
      };
    })(),
  };
}
```

Veja também

- `Date.prototype.toJSON()`

JSON.parse()

O método **`JSON.parse()`** analisa uma string JSON, construindo o valor ou um objeto JavaScript descrito pela string. Uma função **reviver** opcional pode ser fornecida para executar uma transformação no objeto que será retornado.

Experimente

```
const json = '{"result":true, "count":42}';
const obj = JSON.parse(json);

console.log(obj.count);
// Expected output: 42

console.log(obj.result);
// Expected output: true
```

Sintaxe

```
JSON.parse(text[, reviver])
```

Parâmetros

`text`

A string para analisar como JSON. Veja o objeto `JSON` para uma descrição da sintaxe JSON.

`reviver` Optional

Se for uma função, prescreve como o valor originalmente produzido pela análise será transformado antes de ser retornado.

Valor retornado

O `Object` correspondente ao `text` JSON fornecido.

Exceções

Lança uma exceção `SyntaxError` se a string a ser analisada não for um JSON válido.

Exemplos

Usando <code>JSON.parse()</code>

```
JSON.parse("{}"); // {}
JSON.parse("true"); // true
JSON.parse('"foo"'); // "foo"
```

```
JSON.parse('[1, 5, "false"]'); // [1, 5, "false"]
JSON.parse("null"); // null
```

Usando o parâmetro <code>reviver</code>

Se um `reviver` for especificado, o valor calculado pela análise será *transformado* antes de ser retornado. Especificamente, o valor computado e todas as suas propriedades (começando com as propriedades mais aninhadas e prosseguindo para o próprio valor original) são executadas individualmente através do `reviver`. Em seguida, ele é chamado, com o objeto contendo a propriedade sendo processada como `this`, e com o nome da propriedade como uma string, e o valor da propriedade como argumentos. Se a função `reviver` retornar `undefined` (ou não retornar nenhum valor, por exemplo, se a execução cair no final da função), a propriedade será excluída do objeto. Caso contrário, a propriedade é redefinida para ser o valor de retorno.

Se o `reviver` apenas transformar alguns valores e não outros, certifique-se de retornar todos os valores não transformados como estão, caso contrário, eles serão excluídos do objeto resultante.

```
JSON.parse(
  '{"p": 5}',
  (key, value) =>
    typeof value === "number"
      ? value * 2 // retorna o valor * 2 para números
      : value, // retorna tudo sem alteração
);

// { p: 10 }

JSON.parse('{"1": 1, "2": 2, "3": {"4": 4, "5": {"6": 6}}}', (key, value) => {
  console.log(key); // mostra o nome da propriedade atual, o último é "".
  return value; // retorna o valor da propriedade inalterada.
});

// 1
// 2
// 4
// 6
// 5
// 3
// ""
```

<code>JSON.parse()</code> não permite vírgulas à direta

```
// ambos retornarão um SyntaxError
JSON.parse("[1, 2, 3, 4, ]");
JSON.parse('{"foo" : 1, }');
```

Veja Também

- `JSON.stringify()`

JSON.stringify()

Resumo

O método **`JSON.stringify()`** converte valores em javascript para uma String JSON. Esses valores podem ser substituidos especificando a função *replacer*, ou incluindo somente as propriedades específicas, quando o array do replacer for especificado.

Sintaxe

```
JSON.stringify(valor[, replacer[, espaço]])
```

Parâmetros

`valor`

O valor a ser convertido para uma string JSON.

`replacer` Optional

Uma função que altera o comportamento do processo de transformação em string, ou um array de objetos `String` e `Number` que servem como uma lista branca para selecionar as propriedades do objeto valor a ser incluído na string JSON. Se este valor for nulo ou não fornecido, todas as propriedades do objeto são incluídas na string JSON resultante.

`espaço` Optional

Um objeto `String` ou `Number` que é usado para inserir espaço em branco na saída da string JSON para propósito de legibilidade. Se isto for um `Number`, indica o número de caracteres espaço para usar como espaço em branco; este número é limitado em 10 se for maior que isso. Valores menores que 1 indicam que nenhum espaço deve ser usado. Se isto for uma `String`, a string (ou os primeiros 10 caracteres da string, se for maior que isso) é usado como espaço em branco. Se esse parâmetro não for fornecido (ou for null), nenhum espaço em branco é usado.

Descrição

`JSON.stringify()` converte um valor para uma notação JSON que o representa:

- Se o valor tiver um método `toJSON()`, é responsável por definir quais dados serão serializados.

- `Boolean`, `Number`, and `String` os objetos são convertidos para os valores primitivos correspondentes durante a stringificação, de acordo com a semântica de conversão.
- Se `undefined`, uma função, ou um symbol é encontrado durante a conversão é omitido (quando é encontrado em um objeto) ou censurado para `null` (quando é encontrado em um Array). `JSON.stringify` pode também somente retornar `undefined` ao passar valores puros como `JSON.stringify(function(){})` ou `JSON.stringify(undefined)`.
- Todas as propriedades com chave de símbolo serão completamente ignoradas, mesmo quando usar a função `replacer`.
- Propriedades não enumeráveis serão ignoradas.

```
JSON.stringify({}); // '{}'
JSON.stringify(true); // 'true'
JSON.stringify("foo"); // '"foo"'
JSON.stringify([1, "false", false]); // '[1,"false",false]'
JSON.stringify({ x: 5 }); // '{"x":5}'

JSON.stringify({ x: 5, y: 6 });
// '{"x":5,"y":6}' or '{"y":6,"x":5}'
JSON.stringify([new Number(1), new String("false"), new Boolean(false)]);
// '[1,"false",false]'

// Symbols:
JSON.stringify({ x: undefined, y: Object, z: Symbol("") });
// '{}'
JSON.stringify({ [Symbol("foo")]: "foo" });
// '{}'
JSON.stringify({ [Symbol.for("foo")]: "foo" }, [Symbol.for("foo")]);
// '{}'
JSON.stringify({ [Symbol.for("foo")]: "foo" }, function (k, v) {
  if (typeof k === "symbol") {
    return "a symbol";
  }
});
// '{}'
```

O parâmetro <code>replacer</code> parameter

O parâmetro `replacer` pode ser uma função ou uma matriz. Como função, são necessários dois parâmetros, a `chave` e os valores que estão sendo especificados. O objeto no qual a `chave` foi encontrada é fornecido como substituto desse parâmetro. Inicialmente, ele é chamado com uma chave vazia que representa o objeto que está sendo codificado e, em seguida, é chamado para cada propriedade no `objeto` ou matriz que está sendo codificada. Ele deve retornar o valor que deve ser adicionado à cadeia JSON, da seguinte maneira:

- Se você retornar um `Number`, a string correspondente a esse número é usada como o valor da propriedade quando adicionada à sequência JSON.
- If you return a `String`, that string is used as the property's value when adding it to the JSON string.
- If you return a `Boolean`, "true" or "false" is used as the property's value, as appropriate, when adding it to the JSON string.
- If you return any other object, the object is recursively stringified into the JSON string, calling the `replacer` function on each property, unless the object is a function, in which case nothing is added to the JSON string.
- If you return `undefined`, the property is not included in the output JSON string.

> **Nota:** Você não pode usar a função `replacer` para remover valoeres em uma `array`. Se você retornar `undefined`, `null` será usado no lugar.

Exemplo de uma função

```
function replacer(key, value) {
  if (typeof value === "string") {
    return undefined;
  }
  return value;
}

var foo = {
  fundação: "Mozilla",
  modelo: "caixa",
  semana: 45,
  transporte: "carro",
  mês: 7,
};
var jsonString = JSON.stringify(foo, replacer);
```

O resultado do JSON é: `{"semana":45,"mês":7}`.

Exemplo em uma array

Se `replacer` é usado em uma array, os valores da array indicam os nomes das propriedades no objeto, que devem ser incluídas na sequência JSON resultante.

```
JSON.stringify(foo, ["semana", "mês"]);
// '{"semana":45,"mês":7}', apenas mantém as propriedades "semana" e "mês"
```

O argumento <code>space</code>

O argumento `space` O argumento pode ser usado para controlar o espaçamento na sequência final. Se for um número, os níveis sucessivos na stringficação serão recuados por esse número de caracteres de espaço (até 10). Se for uma sequência, os níveis sucessivos serão recuados por essa sequência (ou pelos dez primeiros

caracteres).

```
JSON.stringify({ a: 2 }, null, " ");
// '{
//  "a": 2
// }'
```

O uso de um caractere de tabulação imita a aparência padrão de impressão pretty-print.

```
JSON.stringify({ um: 1, dois: 2 }, null, "\t");
// r:
// '{
//     "um": 1,
//     "dois": 2
// }'
```

Comportamento de <code>toJSON()</code>

Se um objeto sendo stringificado tiver uma propriedade denominada toJSON() cujo valor é uma função, o método `toJSON()` personaliza o comportamento da stringificação JSON, em vez de o objeto ser serializado, o valor retornado pelo método `toJSON()` quando chamado será serializado. Por exemplo:

```
var obj = {
  foo: "foo",
  toJSON: function () {
    return "bar";
  },
};
JSON.stringify(obj); // '"bar"'
JSON.stringify({ x: obj }); // '{"x":"bar"}'
```

Exemplo de uso de <code>JSON.stringify()</code> com <code>localStorage</code>

No caso em que você deseja armazenar um objeto criado por seu usuário e permitir que ele seja restaurado mesmo após o fechamento do navegador, o exemplo a seguir é um modelo para a aplicabilidade de `JSON.stringify()`:

> **Aviso:** As funções não são um tipo de dados JSON válido, portanto, elas não funcionarão. Também alguns objetos como `Date` será uma string depois `JSON.parse()`.

```
// Criando um exemplo em JSON
var seção = {
  telas: [],
  estado: true,
};
session.screens.push({ nome: "telaA", largura: 450, altura: 250 });
session.screens.push({ nome: "telaB", largura: 650, altura: 350 });
session.screens.push({ nome: "telaC", largura: 750, altura: 120 });
session.screens.push({ nome: "telaD", largura: 250, altura: 60 });
session.screens.push({ nome: "telaE", largura: 390, altura: 120 });
session.screens.push({ nome: "telaF", largura: 1240, altura: 650 });

// Convertendo a string JSON em JSON.stringify()
// Salvando com localStorage no nome da sessão
localStorage.setItem("seção", JSON.stringify(seção));

// Exemplo de como transformar a String gerada por meio de:
// JSON.stringify() e salva em localStorage no objeto JSON novamente
var seçãoRestaurada = JSON.parse(localStorage.getItem("seção"));

// Agora, a variável seçãoRestaurada contém o objeto que foi salvo
// no localStorage
console.log(seçãoRestaurada);
```

Veja também

- `JSON.parse()`

Map

O objeto **`Map`** contém pares de chave-valor e lembra a ordem original da inserção das chaves. Qualquer valor (objetos e valores primitivos) podem ser usados como chave ou valor.

Experimente

```
const map1 = new Map();

map1.set("a", 1);
map1.set("b", 2);
map1.set("c", 3);

console.log(map1.get("a"));
// Expected output: 1
```

```
map1.set("a", 97);

console.log(map1.get("a"));
// Expected output: 97

console.log(map1.size);
// Expected output: 3

map1.delete("b");

console.log(map1.size);
// Expected output: 2
```

Descrição

Um objeto `Map` itera seus elementos na order da inserção - um loop `for...of` retorna um array de `[key, value]` para cada iteração

Igualdade de chaves

- A igualdade de chaves é baseada no algoritimo `sameValueZero`.
- O `NaN` é considerado o mesmo que o `NaN` (apesar de `NaN !== NaN`) e todos os outros valores são considerados de acordo com a semântica do operador `===`.
- Na especificação atual do ECMAScript, `-0` e `+0` são considerados iguais, embora não tenha sido nos esboços anteriores. Veja "*Equivalência de valor entre -0 e 0*" em Compatibilidade com navegadores para mais detalhes;

Objetos vs. Maps

Um `Object` é similar o `Map` - ambos permitem que valores sejam definidos a chaves, retornar esses valores, remover as chaves, e detectar se algo está armazenado na chave. Por esta razão (e porque não existem outras alternativas construídas), o `Objeto` tem sido usado como `Map` historicamente.

Porém, existem diferenças importantes que fazem o `Map` ser preferido em alguns casos:

	Map	Objeto
Chaves acidentais	O `Map` não contém nenhuma chave por padrão. Ele só contém o que é definido explicitamente nele.	Um `Objeto` tem um prototype, então ele contém por padrão algumas chaves que podem conflitar com suas próprias chaves se você não for cuidadoso. **Nota:** Apartir do ES5, isso pode ser ignorado com o `Object.create(null)`, mas isso raramente é feito.
Tipos das chaves	As chaves do `Map` podem ser qualquer valor (incluindo funções, objetos, ou qualquer outro tipo primitivo).	A chave de um `Objeto` deve ser uma `String` ou um `Symbol`.
Ordem das chaves	As chaves dentro do `Map` são ordenadas de forma simples, de maneira direta: O objeto `Map` itera suas entradas, chaves, e valor na ordem em que foram inseridas.	Embora as chaves de um `Objeto` comum hoje sejam ordenadas, nem sempre esse foi o caso, e a ordem é complexa. Como resultado é melhor não confiar na ordem das propriedades. A ordem foi definida primeiramente para suas próprias propriedades apenas no ECMAScript 2015; no ECMAScript 2020 a ordem definida é por propriedades herdadas também. Veja o OrdinaryOwnPropertyKeys e numerateObjectProperties operações de especificações abstraídas. Mas note que nenhum mecanismo itera **todas** as propriedades do objeto; cada um dos vários mecanismos incluem diferentes subconjuntos de propriedades. (`for-in` incluí apenas propriedades enumeráveis com chaves do tipo string `Object.keys` inclui apenas chaves próprias e enumeráveis do tipo string; `Object.getOwnPropertyNames` incluí o próprio, e propriedades com chaves do tipo string, mesmo que não enumeráveis `Object.getOwnPropertySymbols` faz o mesmo só que para propriedades `Símbolos` como chave, etc.)
Tamanho	O número de items dentro de um `Map` é facilmente retornado pela propriedade `size`	O número de items dentro de um `Objeto` deve ser determinado manualmente
Iteração	Um `Map` é `iterável`, então ele pode ser diretamente iterável	O `Objeto` não implementa o protocolo de iteração, e os objetos não podem ser iterados diretamente usando o for...of (por padrão). **Nota:** • Um objeto pode implementar o protocolo de iteração, ou pode ser iterado usando o `Object.keys`ou`Object.entries`. • A declaração for...in permite que você itere pela propriedades enumeráveis de um objeto.
Performance	Perfoma melhor em cenários envolvendo adições e remoções	Não é otimizado para adições e remoções frequentes de pares chave-valor.

	Map	Objeto
	frequentes em pares chave-valor.	
Serialização e análise sintática	Não há suporte nativo para a serialização ou análise sintática. (Mas você pode construir sua própria serialização e conversão para o `Map` usando o `JSON.stringify()` como o argumento *replacer*, e usando o `JSON.parse()` com o argumento *reviver*. Veja a questão no Stack Overflow How do you JSON.stringify an ES6 Map?).	Suporte nativo para serialização de `Objeto` para JSON, usando `JSON.stringify()`. Suporte nativo para conversão de JSON para `Objeto` usando `JSON.parse()`.

Definindo propriedades no objeto

Definir propriedades no objeto também funciona em objetos `Map`, e pode causar um confusão considerável.

Portanto, isso aparenta funcionar de certa forma:

```
const wrongMap = new Map();
wrongMap["bla"] = "blaa";
wrongMap["bla2"] = "blaaa2";

console.log(wrongMap); // Map { bla: 'blaa', bla2: 'blaaa2' }
```

Mas esse jeito de definir propriedades não interage com a estrura de dados do `Map`. Dessa forma é usada a implementação genérica do objeto. O valor 'bla' não é armazenado no `Map` para queries. Outras operaçãoes nos dados irão falhar.

```
wrongMap.has("bla"); // false
wrongMap.delete("bla"); // false
console.log(wrongMap); // Map { bla: 'blaa', bla2: 'blaaa2' }
```

A maneira correta para armazenar dados dentro do `Map` é através do `set(key,value)`

```
const contacts = new Map();
contacts.set("Jessie", { phone: "213-555-1234", address: "123 N 1st Ave" });
contacts.has("Jessie"); // true
contacts.get("Hilary"); // undefined
contacts.set("Hilary", { phone: "617-555-4321", address: "321 S 2nd St" });
contacts.get("Jessie"); // {phone: "213-555-1234", address: "123 N 1st Ave"}
contacts.delete("Raymond"); // false
contacts.delete("Jessie"); // true
console.log(contacts.size); // 1
```

Construtor

`Map()`

Cria um novo objeto `Map`.

propriedades estáticas

`get Map[@@species]`

A função do construtor que é usada para criar apartir de objetos.

propriedades da instância

`Map.prototype.size`

Retorna o número de pares chave/valor no objeto `Map`.

Métodos da instância

`Map.prototype.clear()`

Remove todos os pares chave/valor do objeto `Map`.

`Map.prototype.delete(key)`

Retorna `true` se o elemento no objeto `Map` existia e tenha sido removido, ou `false` se o elemento não existia. `Map.prototype.has(key)` irá retornar `false` após isso.

`Map.prototype.get(key)`

Retorna o valor associado à chave, ou `undefined` se não há nada.

`Map.prototype.has(key)`

Retorna uma asserção booleana se o valor tenha sido associado à chave no objeto `Map` ou não.

`Map.prototype.set(key, value)`

Define o `valor` para a `chave` no objeto `Map`. Retorna o objeto `Map`

Métodos iterativos

`Map.prototype[@@iterator]()`

Retorna um novo objeto iterador que contèm **um array de `[chave, valor]`** para cada elemento do objeto `Map` na ordem em que foram inseridos.

`Map.prototype.keys()`

Retorna um novo objeto iterador que contèm as **chaves** de cada elemento do objeto `Map` na ordem em que foram inseridos.

`Map.prototype.values()`

Retorna um novo objeto iterador que contém os **valores** para cada elemento do objeto `Map` na ordem em que foram inseridos.

`Map.prototype.entries()`

Retorna um novo objeto iterador que contèm **um array de `[chave, valor]`** para cada elemento no objeto `Map` na ordem em que foram inseridos.

`Map.prototype.forEach(callbackFn[, thisArg])`

Invoca o `callbackFn` uma vez para cada par chave-valor presente no objeto `Map`, na ordem em que foram inseridos. Se um parâmetro `thisArg` é provido para o `forEach`, será usado o valor de `this` para cada callback.

Exemplos

Usando o objeto Map

```
const myMap = new Map();

const keyString = "a string";
const keyObj = {};
const keyFunc = function () {};

// setting the values
myMap.set(keyString, "value associated with 'a string'");
myMap.set(keyObj, "value associated with keyObj");
myMap.set(keyFunc, "value associated with keyFunc");

myMap.size; // 3

// getting the values
myMap.get(keyString); // "valor associado a 'a string'"
myMap.get(keyObj); // "valor associado a keyObj"
myMap.get(keyFunc); // "valor associado a keyFunc"

myMap.get("a string"); // "valor associado a 'a string'"
// porque keyString === 'a string'
myMap.get({}); // undefined, porque keyObj !== {}
myMap.get(function () {}); // undefined, porque keyFunc !== function () {}
```

Usando NaN como chaves de Map

`NaN` também pode ser usado como chave. Apesar de todo o `NaN` não ser igual a ele mesmo (`NaN !== NaN`), o exemplo a seguir funciona porque não é possível distinguir um `NaN` de outros.

```
const myMap = new Map();
myMap.set(NaN, "not a number");

myMap.get(NaN);
// "not a number"

const otherNaN = Number("foo");
myMap.get(otherNaN);
// "not a number"
```

Iterando o Map com for..of

`Maps` podem ser iterados usando um loop `for..of`:

```
const myMap = new Map();
myMap.set(0, "zero");
myMap.set(1, "one");

for (const [key, value] of myMap) {
  console.log(key + " = " + value);
}
// 0 = zero
// 1 = um
```

```
for (const key of myMap.keys()) {
  console.log(key);
}
// 0
// 1

for (const value of myMap.values()) {
  console.log(value);
}
// zero
// one

for (const [key, value] of myMap.entries()) {
  console.log(key + " = " + value);
}
// 0 = zero
// 1 = one
```

Iterando o Map com forEach()

`Maps` podem ser iterados usando o método `forEach()`:

```
myMap.forEach(function (value, key) {
  console.log(key + " = " + value);
});
// 0 = zero
// 1 = um
```

Relação com Arrays

```
const kvArray = [
  ["key1", "value1"],
  ["key2", "value2"],
];

// Use o construtor padrão do Map para transformar um array 2D chave-valor em um map
const myMap = new Map(kvArray);

myMap.get("key1"); // retorna "value1"

// Use o Array.from() para transformar um map em um Array bidimensional de chaves e valores
console.log(Array.from(myMap)); // Irá exibir para você o mesmo Array como um Array chave-valor

// Uma forma sucinta de fazer o mesmo, utilizando a sintaxe spread
console.log([...myMap]);

// Ou use os iteradores keys() ou values(), e os converta para um array
console.log(Array.from(myMap.keys())); // ["key1", "key2"]
```

Clonando e mesclando Maps

Assim como `Arrays`, o map também pode ser clonado:

```
const original = new Map([[1, "um"]]);

const clone = new Map(original);

console.log(clone.get(1)); // um
console.log(original === clone); // false (Útil para comparações superficiais)
```

> **Nota:** Tenha em mente que *o dado em si* não é clonado.

Maps podem ser mesclados, mantendo as chaves únicas:

```
const first = new Map([
  [1, "one"],
  [2, "two"],
  [3, "three"],
]);

const second = new Map([
  [1, "uno"],
  [2, "dos"],
]);

// Mescla dois maps. A última chave a se repetir vence.
// O operador spread essenciamente converte um Map para um Array
const merged = new Map([...first, ...second]);

console.log(merged.get(1)); // uno
console.log(merged.get(2)); // dos
console.log(merged.get(3)); // three
```

Maps podem ser mesclados com array também:

```
const first = new Map([
  [1, "one"],
  [2, "two"],
  [3, "three"],
]);

const second = new Map([
  [1, "uno"],
  [2, "dos"],
]);

// Mescla maps com um array. A última chave a se repetir vence.
const merged = new Map([...first, ...second, [1, "eins"]]);

console.log(merged.get(1)); // eins
console.log(merged.get(2)); // dos
console.log(merged.get(3)); // three
```

Veja também

- Um polyfill do `Map` está disponível em `core-js`
- `Set`
- `WeakMap`
- `WeakSet`

Map.prototype.clear()

O método **`clear()`** remove todos os elementos de um objeto `Map`.

Experimente

```
const map1 = new Map();

map1.set("bar", "baz");
map1.set(1, "foo");

console.log(map1.size);
// Expected output: 2

map1.clear();

console.log(map1.size);
// Expected output: 0
```

Sintaxe

```
clear();
```

Valor retornado

`undefined`.

Exemplos

Usando o clear()

```
var myMap = new Map();
myMap.set("bar", "baz");
myMap.set(1, "foo");

myMap.size; // 2
myMap.has("bar"); // true

myMap.clear();

myMap.size; // 0
myMap.has("bar"); // false
```

Veja também

- `Map`

Map.prototype.delete()

O método **`delete()`** remove o elemento específico do objeto `Map` através de sua chave.

Experimente

```
const map1 = new Map();
map1.set("bar", "foo");
```

```
console.log(map1.delete("bar"));
// Expected result: true
// True indicates successful removal

console.log(map1.has("bar"));
// Expected result: false
```

Sintaxe

```
delete key;
```

Parâmetros

`key`

A chave do elemento que será removido do objeto `Map`.

Valor retornado

`true` se o elemento do `Map` existia e foi removido, ou `false` se o elemento não existe.

Exemplos

Usando o delete()

```
var myMap = new Map();
myMap.set("bar", "foo");

myMap.delete("bar"); // Retorna true. Removido com sucesso.
myMap.has("bar"); // Retorna false. O elemento "bar" não está mais presente
```

Veja também

- `Map`

Map.prototype.entries()

O método **`entries()`** retorna um novo objeto **iterador** que contém os pares `[key, value]` de cada elemento no objeto `Map` na ordem em que foram inseridos. Nesse caso em particular, esse objeto iterador também é iterativo. então o loop for-of pode ser usado. Quando o protocolo `[Symbol.iterator]` é usado, ele retorna uma função que, quando invocada, retorna o própio iterador.

Experimente

```
const map1 = new Map();

map1.set("0", "foo");
map1.set(1, "bar");

const iterator1 = map1.entries();

console.log(iterator1.next().value);
// Expected output: Array ["0", "foo"]

console.log(iterator1.next().value);
// Expected output: Array [1, "bar"]
```

Sintaxe

```
entries();
```

Valor retornado

Um novo objeto iterador `Map`.

Exemplos

Usando o entries()]

```
let myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

let mapIter = myMap.entries();

console.log(mapIter.next().value); // ["0", "foo"]
console.log(mapIter.next().value); // [1, "bar"]
console.log(mapIter.next().value); // [Object, "baz"]
```

Veja também

- `Map.prototype.keys()`

- `Map.prototype.values()`

Map.prototype.forEach()

O método **`forEach()`** executa a função que foi provida uma vez para cada par de chave-valor no objeto `Map`, na order em que foram inseridos.

Experimente

```
function logMapElements(value, key, map) {
  console.log(`m[${key}] = ${value}`);
}

new Map([
  ["foo", 3],
  ["bar", {}],
  ["baz", undefined],
]).forEach(logMapElements);

// Expected output: "m[foo] = 3"
// Expected output: "m[bar] = [object Object]"
// Expected output: "m[baz] = undefined"
```

Sintaxe

```
// Arrow function
forEach(() => { ... } )
forEach((value) => { ... } )
forEach((value, key) => { ... } )
forEach((value, key, map) => { ... } )

// função de Callback
forEach(callbackFn)
forEach(callbackFn, thisArg)

// função de callback inline
forEach(function callbackFn() { ... })
forEach(function callbackFn(value) { ... })
forEach(function callbackFn(value, key) { ... })
forEach(function callbackFn(value, key, map) { ... })
forEach(function callbackFn(value, key, map) { ... }, thisArg)
```

Parâmetros

`callbackFn`

A função que será executada em cada entrada do map. Ela receberá os seguintes argumentos:

`value` Optional

O valor de cada iteração.

`key` Optional

A chave de cada iteração.

`map` Optional

O map que está sendo iterado.

`thisArg` Optional

O valor para usar como `this` quando o callback for executado.

Valor retornado

`undefined`.

Descrição

O método `forEach` executa o `callback` fornecido uma vez para cada chave do map que realmente existe. Ele não é invocado para chaves que foram deletadas. Porém, é executado para valores que estão presentes mesmo tendo o valor `undefined`.

O `callback` é invocado com **três argumentos**:

- O `value` de entrada
- A `key` de entrada
- o **objeto** `Map` que está sendo percorrido

Se o parâmetro `thisArg` for fornecido para o `forEach`, ele será passado para o `callback` quando for invocado, para ser usado como o valor de `this`. De outra forma, o valor `undefined` será usado como valor de `this`. O valor de `this` será observado pelo `callback` em última análise e será determinado de acordo com as regras usuais para a determinação do `this` visto por uma função.

Cada valor será visitado uma vez, exceto no caso em que foram deletados e adicionados novamente antes do `forEach` ser finalizado. O `callback` não é invocado para valores que foram removidos antes de terem sidos visitados. Novos valores adicionados antes do `forEach` terminar serão visitados.

Exemplos

Exibindo o conteúdo de um objeto Map

O código a seguir imprime uma linha para cada elemento do objeto `Map`:

```
function logMapElements(value, key, map) {
  console.log(`map.get('${key}') = ${value}`);
}
new Map([
  ["foo", 3],
  ["bar", {}],
  ["baz", undefined],
]).forEach(logMapElements);
// logs:
// "map.get('foo') = 3"
// "map.get('bar') = [object Object]"
// "map.get('baz') = undefined"
```

Veja também

- `Array.prototype.forEach()`
- `Set.prototype.forEach()`

Map.prototype.get()

O método **`get()`** retorna um elemento específico do objeto `Map`. Se o valor associado à chave for um objeto, será retornado a referência do objeto e qualquer mudança no mesmo irá afetar o valor que está dentro do objeto `Map`.

Experimente

```
const map1 = new Map();
map1.set("bar", "foo");

console.log(map1.get("bar"));
// Expected output: "foo"

console.log(map1.get("baz"));
// Expected output: undefined
```

Sintaxe

```
get(key);
```

Parâmetros

`key`

A chave do elemento que será retornado do objeto `Map`.

Valor retornado

O elemento associado à chave especificada, ou `undefined` se a chave não puder ser encontrada no objeto `Map`.

Exemplos

Usando o get()

```
let myMap = new Map();
myMap.set("bar", "foo");

myMap.get("bar"); // Returns "foo"
myMap.get("baz"); // Returns undefined
```

Veja também

- `Map`
- `Map.prototype.set()`
- `Map.prototype.has()`

Map.prototype.has()

O método **`has()`** retorna um booleano indicando se o elemento com a chave especificada existe ou não.

Experimente

```
const map1 = new Map();
map1.set("bar", "foo");

console.log(map1.has("bar"));
// Expected output: true
```

```
console.log(map1.has("baz"));
// Expected output: false
```

Sintaxe

```
has(key);
```

Parâmetros

`key`

A chave do elemento que será testado no objeto `Map`.

Valor retornado

`true` se o elemento com a chave especificada existe no objeto `Map`, de outra forma será retornado `false`.

Exemplos

Usando has()

```
let myMap = new Map();
myMap.set("bar", "foo");

myMap.has("bar"); // retorna true
myMap.has("baz"); // retorna false
```

Veja também

- `Map`
- `Map.prototype.set()`
- `Map.prototype.get()`

Map.prototype.keys()

O método **`keys()`** retorna um novo objeto **iterador** que contém as chaves de cada elemento dentro do objeto `Map` na ordem em que foram inseridos.

Experimente

```
const map1 = new Map();

map1.set("0", "foo");
map1.set(1, "bar");

const iterator1 = map1.keys();

console.log(iterator1.next().value);
// Expected output: "0"

console.log(iterator1.next().value);
// Expected output: 1
```

Sintaxe

```
keys();
```

Valor retornado

Um novo objeto `Map` iterador.

Exemplos

Usando keys()

```
var myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

var mapIter = myMap.keys();

console.log(mapIter.next().value); // "0"
console.log(mapIter.next().value); // 1
console.log(mapIter.next().value); // Object
```

Veja também

- `Map.prototype.entries()`
- `Map.prototype.values()`

Map() constructor

O **construtor** `Map()` cria objetos `Map`.

Sintaxe

```
new Map();
new Map(iterable);
```

Parâmetros

`iterable` Optional

Um `Array` ou outro objeto iterável dos quais os elementos são pares chave-valor. (Por exemplo, arrays com dois elementos como `[[ 1, 'one' ],[ 2, 'two' ]]`.). Cada par chave-valor é adicionado ao novo `Map`.

Exemplos

Criando um novo Map

```
let myMap = new Map([
  [1, "one"],
  [2, "two"],
  [3, "three"],
]);
```

Veja também

- Um polyfill de `Map` esta disponível em `core-js`
- `Set`
- `WeakMap`
- `WeakSet`

Map.prototype.set()

O método **`set()`** adiciona ou atualiza um elemento com a chave especificada e um valor do objeto `Map`.

Experimente

```
const map1 = new Map();
map1.set("bar", "foo");

console.log(map1.get("bar"));
// Expected output: "foo"

console.log(map1.get("baz"));
// Expected output: undefined
```

Sintaxe

```
set(key, value);
```

Parâmetros

`key`

A chave do elemento que será adicionado ao objeto `Map`.

`value`

O valor do elemento que será adicionado ao objeto `Map`.

Valor retornado

O objeto `Map`.

Exemplos

Usando o set()

```
let myMap = new Map();

// Adiciona um elemento ao map
myMap.set("bar", "foo");
myMap.set(1, "foobar");

// Atualiza um elemento no map
myMap.set("bar", "baz");
```

Usando o <code>set</code> de forma encadeada

Já que o método `set()` retorna o mesmo objeto `Map`, você pode encadear a chamada do método como abaixo:

```
// Adiciona novos elementos ao map de forma encadeada.
myMap.set("bar", "foo").set(1, "foobar").set(2, "baz");
```

Veja também

- `Map`
- `Map.prototype.get()`
- `Map.prototype.has()`

Map.prototype.size

A propriedade acessora **`size`** retorna o número de elementos que estão dentro do objeto `Map`

Experimente

```
const map1 = new Map();

map1.set("a", "alpha");
map1.set("b", "beta");
map1.set("g", "gamma");

console.log(map1.size);
// Expected output: 3
```

Descrição

O valor de `size` é um inteiro que representa a quantidade de entradas que o objeto `Map` possui. A função acessora set para `size` é `undefined`; você não poderá alterar essa propriedade.

Exemplos

Usando o size

```
var myMap = new Map();
myMap.set("a", "alpha");
myMap.set("b", "beta");
myMap.set("g", "gamma");

myMap.size; // 3
```

Veja também

- `Map`

Map.prototype[@@iterator]()

O valor inicial da propriedade **`@@iterator`** é o mesmo objeto função que o valor inicial do método `entries`

Experimente

```
const map1 = new Map();

map1.set("0", "foo");
map1.set(1, "bar");

const iterator1 = map1[Symbol.iterator]();

for (const item of iterator1) {
  console.log(item);
}
// Expected output: Array ["0", "foo"]
// Expected output: Array [1, "bar"]
```

Sintaxe

```
myMap[Symbol.iterator];
```

Valor retornado

A função iterativa do map, que é a `entries()` por padrão

Exemplos

Usando o [@@iterator]()

```
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

const mapIter = myMap[Symbol.iterator]();

console.log(mapIter.next().value); // ["0", "foo"]
```

```
console.log(mapIter.next().value); // [1, "bar"]
console.log(mapIter.next().value); // [Object, "baz"]
```

Usando o [@@iterator]() com for..of

```
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

for (const entry of myMap) {
  console.log(entry);
}
// ["0", "foo"]
// [1, "bar"]
// [{}, "baz"]

for (const [key, value] of myMap) {
  console.log(`${key}: ${value}`);
}
// 0: foo
// 1: bar
// [Object]: baz
```

Veja também

- `Map.prototype.entries()`
- `Map.prototype.keys()`
- `Map.prototype.values()`

get Map[@@species]

O Acessor de propriedade **`Map[@@species]`** retorna o construtor do `Map`.

Descrição

O acessor `species` retorna o valor padrão do construtor dos objetos `Map` Subclasses de construtores podem sobrescrever isso para mudar o construtor atribuído.

Exemplos

Species em objetos ordinários

A propriedade *species* retorna a função padrão do construtor, que é o construtor `Map` para objetos `Map`.

```
Map[Symbol.species]; // function Map()
```

Species em objetos derivados

Em uma objeto derivado (e.g. seu custom map `MyMap`), a `specie` do `MyMap` é o construtor do `MyMap`. Porém, você deve querer sobrescrever isso, para retornar o objeto `Map` pai nos métodos derivados de sua classe.

```
class MyMap extends Map {
  // Overwrite MyMap species to the parent Map constructor
  static get [Symbol.species]() {
    return Map;
  }
}
```

Veja também

- `Map`
- `Symbol.species`

Map.prototype.values()

O método **`values()`** retorna um novo objeto **iterador** que contém o valor de cada elemento dentro do objeto `Map` na ordem em que foram inseridos.

Experimente

```
const map1 = new Map();

map1.set("0", "foo");
map1.set(1, "bar");

const iterator1 = map1.values();

console.log(iterator1.next().value);
// Expected output: "foo"

console.log(iterator1.next().value);
// Expected output: "bar"
```

Sintaxe

```
values();
```

Valor retornado

Um novo objeto `Map` iterador.

Exemplos

Usando values()

```
var myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

var mapIter = myMap.values();

console.log(mapIter.next().value); // "foo"
console.log(mapIter.next().value); // "bar"
console.log(mapIter.next().value); // "baz"
```

Veja também

- `Map.prototype.entries()`
- `Map.prototype.keys()`

Math

Math é um objeto embutido que tem propriedades e métodos para constantes e funções matemáticas. Não é um objeto de função.

Descrição

Ao contrário de outros objetos globais, `Math` não é um construtor. Todas as propriedades e métodos de `Math` são estáticos. Você pode referenciar a constante PI como `Math.PI` e você pode chamar a função de seno como `Math.sin(x)`, onde `x` é o argumento do método. Constantes são definidas com a precisão total de números reais em JavaScript.

Propriedades

`Math.E`

Constante de Euler e base dos logaritmos naturais, aproximadamente 2.718.

`Math.LN2`

Logaritmo natural de 2, aproximadamente 0.693.

`Math.LN10`

Logaritmo natural de 10, aproximadamente 2.303.

`Math.LOG2E`

Logaritmo de E na base 2, aproximadamente 1.443.

`Math.LOG10E`

Logaritmo de E na base 10, aproximadamente 0.434.

`Math.PI`

Relação entre a circunferência de um círculo e o seu diâmetro, aproximadamente 3.14159.

`Math.SQRT1_2`

Raiz quadrada de 1/2; Equivale a 1 dividido pela raiz quadrada de 2, aproximadamente 0.707.

`Math.SQRT2`

Raiz quadrada de 2, aproximadamente 1.414.

Métodos

> **Nota:** Note que as funções trigonométricas (sin(), cos(), tan(), asin(), acos(), atan(), atan2()) recebem ou retornam ângulos em radianos. Divida por (`Math.PI/180`) para converter radianos em graus, ou multiplique por esse valor para fazer a conversão inversa.

`Math.abs(x)`

Retorna o módulo, ou valor absoluto, de um número (

$|x|$

).

`Math.acos(x)`

Retorna o arco-coseno de um número ($\arccos x$).

`Math.acosh(x)`

Retorna o arco-coseno hiperbólico de um número.

`Math.asin(x)`

Retorna o arco-seno de um número ($\arcsin x$).

`Math.asinh(x)`

Retorna o arco-seno hiperbólico de um número.

`Math.atan(x)`

Retorna o arco-tangente de um número ($\arctan x$).

`Math.atanh(x)`

Retorna o arco-tangente hiperbólico de um número ($\arctan x$).

`Math.atan2(x, y)`

Retorna o arco-tangente do quociente de seus argumentos.

`Math.cbrt(x)`

Retorna a raiz cúbica de um número ($\sqrt[3]{x}$).

`Math.ceil(x)`

Retorna o menor inteiro que é maior ou igual a um número.

`Math.cos(x)`

Retorna o coseno de um número ($\cos x$).

`Math.cosh(x)`

Retorna o coseno hiperbólico de um número .

`Math.exp(x)`

Retorna e^x, onde *x* é o argumento, e e é a constante de Euler (2.718...), a base do logaritmo natural.

`Math.expm1(x)`

Retorna e^x - 1.

`Math.floor(x)`

Retorna o maior inteiro que é menor ou igual a um número.

`Math.fround(x)`

Retorna a mais próxima representação de ponto flutuante de precisão-única de um número.

`Math.hypot([x[,y[,…]]])`

Retorna a raiz quadrada da soma dos quadrados dos argumentos (

$\sqrt{x^2 + y^2 + \ ...}$

).

`Math.imul(x)`

Retorna o resultado de uma multiplicação de inteiro de 32-bit.

`Math.log(x)`

Retorna o logaritmo natural (

$\log_e x$

ou

$\ln x$

) de um número.

`Math.log1p(x)`

Retorna o logaritmo natural de 1 + x (

$\log_e(1 + x)$

ou

$\ln(1 + x)$

) de um número.

`Math.log10(x)`

Retorna o logaritmo de x na base 10 (

$\log_{10} x$

).

`Math.log2(x)`

Retorna o logaritmo de x na base 2 (

$\log_2 x$

).

`Math.max([x[,y[,…]]])`

Retorna o maior dentre os parâmetros recebidos.

`Math.min([x[,y[,…]]])`

Retorna o menor dentre os parâmetros recebidos.

`Math.pow(x,y)`

Retorna a base x elevada à potência y do expoente, ou seja,

x^y

.

`Math.random()`

Retorna um número pseudo-aleatório entre 0 e 1.

`Math.round(x)`

Retorna o valor arrendodado de x, para o valor inteiro mais próximo.

`Math.sign(x)`

Retorna o sinal de x, indicando se é positivo, negativo ou zero.

`Math.sin(x)`

Retorna o seno de um número ($\sin x$).

`Math.sinh(x)`

Retorna o seno hiperbólico de um número ($\sinh x$).

`Math.sqrt(x)`

Retorna a raiz quadrada positiva de um número ($\sqrt{x}$).

`Math.tan(x)`

Retorna a tangente de um número ($\tan x$).

`Math.tanh(x)`

Retorna a tangente hiperbólica de um número ($\tanh x$).

`Math.toSource()`

Retorna a string `"Math"`.

`Math.trunc(x)`

Retorna a parte inteira de `x`, removendo quaisquer dígitos fracionários.

Veja também

- `Number`

Math.abs()

Sumário

A função **`Math.abs(x)`** retorna o valor absoluto de um número "x", tal qual:

$$\text{Math.abs}\ (x) = |x| = \begin{cases} x & \text{if} \quad x > 0 \\ 0 & \text{if} \quad x = 0 \\ -x & \text{if} \quad x < 0 \end{cases}$$

Sintaxe

```
var abs = Math.abs(x);
```

Parâmetros

`x`

Um número.

Valor de Retorno

O valor absoluto do número passado

Descrição

Por `abs` ser um método estático de `Math`, você sempre o usará como `Math.abs()` ao ínves de usar como método de um objeto `Math` criado por você. (`Math` não é um construtor);

Exemplos

Comportamento de <code>Math.abs()</code>

Passando um string não-numérica ou variável indefinida/vazia retorna `NaN`. Passando null retorna 0.

```
Math.abs("-1"); // 1
Math.abs(-2); // 2
Math.abs(null); // 0
Math.abs(""); // 0
Math.abs([]); // 0
Math.abs([2]); // 2
Math.abs([1, 2]); // NaN
Math.abs({}); // NaN
Math.abs("string"); // NaN
Math.abs(); // NaN
```

Veja Também

- `Math.ceil()`
- `Math.floor()`
- `Math.round()`
- `Math.sign()`
- `Math.trunc()`

Math.acos()

A função **`Math.acos()`** retorna o arco cosseno (em radianos de um numero, sendo esse

$$\forall x \in [-1; 1], \text{Math.acos } (x) = \arccos(x) = \text{the unique} y \in [0; \pi] \text{such that} \cos(y) = x$$

Syntax

```
Math.acos(x)
```

Parâmetros

Um numero.

Retorno

O arco cosseno(em radianos) se o valor passado como parâmetro for entre -1 e 1; caso contrario retornará `NaN`.

Description

O `Math.acos()` metodo retorna um numero entre 0 e π radians para valores passado como parâmetros entre -1 e 1. Se o valor estiver fora dessa variação será retornado `NaN`.

Porque `acos() é um metodo estático`, você sempre usará `Math.acos() ao invés de criar um Objecto Math(Math não é um construtor).`

Exemplos

Usando <code>Math.acos()</code>

```
Math.acos(-2); // NaN
Math.acos(-1); // 3.141592653589793
Math.acos(0); // 1.5707963267948966
Math.acos(0.5); // 1.0471975511965979
Math.acos(1); // 0
Math.acos(2); // NaN
```

Para valores menores que -1 ou maiores que than 1, `Math.acos()` o método retornrá `NaN`.

Veja também

- `Math.asin()`
- `Math.atan()`
- `Math.atan2()`
- `Math.cos()`
- `Math.sin()`
- `Math.tan()`

Math.acosh()

A função Math.acosh() retorna o arco cosseno hiperbólico de um número, onde

$$\forall x \geq 1, \text{Math.acosh } (x) = \text{arcosh } (x) = \text{the unique} y \geq 0 \text{such that} \cosh(y) = x$$

Sintaxe

```
Math.acosh(x)
```

Parâmetros

`x`

Um número.

Valor retornado

The hyperbolic arc-cosine of the given number. If the number is less than **1**, `NaN`.

O arco cosseno hiperbólico do número recebido. Se o número for menor que 1, `NaN` é retornado.

Descrição

Por `acosh()` ser um método estático de `Math`, deve-se sempre usá-lo como `Math.acosh()`, e não como um método de um objeto `Math` que você criou.

Exemplos

Usando <code>Math.acosh()</code>

```
Math.acosh(-1); // NaN
Math.acosh(0); // NaN
Math.acosh(0.5); // NaN
Math.acosh(1); // 0
Math.acosh(2); // 1.3169578969248166
```

For values less than 1 `Math.acosh()` returns `NaN`.

Para valores menores que 1, `Math.acosh()` retornará `NaN`.

Polyfill

Para todo $x \geq 1$, temos $\operatorname{arcosh}(x) = \ln\left(x + \sqrt{x^2 - 1}\right)$. Dessa maneira, este comportamento pode ser emulado da seguinte maneira:

```
Math.acosh =
  Math.acosh ||
  function (x) {
    return Math.log(x + Math.sqrt(x * x - 1));
  };
```

See also

- `Math.asinh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.sinh()`
- `Math.tanh()`

Math.asin()

A função Math.asin() retorna o arco seno (em radianos) de um número, onde

$$\forall x \in [-1; 1], \text{Math.asin}(x) = \arcsin(x) = \text{o único valor } y \in \left[-\frac{\pi}{2}; \frac{\pi}{2}\right] \text{tal que } \sin(y) = x$$

Sintaxe

```
Math.asin(x)
```

Parâmetros

`x`

Um número.

Valor retornado

O arco seno (em radianos) do parâmetro recebido se o parâmetro estiver entre **-1** e **1**; senão, `NaN`.

Descrição

O método `Math.asin()` retorna um valor numérico entre $-\frac{\pi}{2}$ e $\frac{\pi}{2}$ radianos para todo `x` entre -1 e 1. Se o valor de `x` estiver fora deste intervalo `NaN` é retornado.

Por `asin()` ser um método estático de `Math`, deve-se sempre usá-lo como `Math.asin()`, e não como um método de um objeto `Math` que você criou.

Exemplos

Usando <code>Math.asin()</code>

```
Math.asin(-2); // NaN
Math.asin(-1); // -1.5707963267948966 (-pi/2)
Math.asin(0); // 0
Math.asin(0.5); // 0.5235987755982989
Math.asin(1); // 1.5707963267948966 (pi/2)
Math.asin(2); // NaN
```

Para valores menores que -1 ou maiores que 1, `Math.asin()` retorna `NaN`.

Veja também

- `Math.acos()`

- `Math.atan()`
- `Math.atan2()`
- `Math.cos()`
- `Math.sin()`
- `Math.tan()`

Math.asinh()

A função **`Math.asinh()`** retorna o arco-seno hiperbólico de um número, isto é:

$$\text{Math.asinh}\ (x) = \text{arsinh}\ (x) = \text{the unique} y \text{such that} \sinh(y) = x$$

Experimente

```
console.log(Math.asinh(1));
// Expected output: 0.881373587019543

console.log(Math.asinh(0));
// Expected output: 0

console.log(Math.asinh(-1));
// Expected output: -0.881373587019543

console.log(Math.asinh(2));
// Expected output: 1.4436354751788103
```

Sintaxe

```
Math.asinh(x)
```

Parâmetros

`x`

Um número.

Valor de retorno

O arco-seno hiperbólico de um dado número.

Descrição

Como `asinh()` é um métodos estático de `Math`, você deve sempre chamá-lo como `Math.asinh()`, ao invés de um método de um objeto `Math` que você criou (`Math` não é um construtor).

Exemplos

Usando <code>Math.asinh()</code>

```
Math.asinh(1); // 0.881373587019543
Math.asinh(0); // 0
```

Polyfill

Como uma solução simples a expressçao pode ser usada diretamente para uma emulação grosseira pela seguinte função:

```
Math.asinh =
  Math.asinh ||
  function (x) {
    if (x === -Infinity) {
      return x;
    } else {
      return Math.log(x + Math.sqrt(x * x + 1));
    }
  };
```

Apesar de formalmente correta, ela sofre de algumas problemas relacionadas à computação de ponto flutuante. Resultados precisos precisam de tratamento especial de positivos/negativos e argumentos pequenos/grandes como feitos por exemplo em em glibc ouGNU Scientific Library.

Veja também

- `Math.acosh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.sinh()`
- `Math.tanh()`

Math.atan()

A função **`Math.atan()`** retorna a arco tangente (em radianos) de um número, onde

$$\text{Math.atan}\ (x) = \arctan(x) = \text{the unique} y \in \left[-\frac{\pi}{2}; \frac{\pi}{2}\right] \text{such that} \tan(y) = x$$

Sintaxe

```
Math.atan(x)
```

Parâmetros

`x`

Um número.

Valor retornado

O arco tangente (em radianos) do parâmetro recebido.

Descrição

O método `Math.atan()` retorna um valor numérico entre $-\frac{\pi}{2}$ e $\frac{\pi}{2}$ radianos.

Por `atan()` ser um método estático de `Math`, deve-se sempre usá-lo como `Math.atan()`, e não como um método de um objeto `Math` que você criou.

Exemplos

Usando <code>Math.atan()</code>

```
Math.atan(1); // 0.7853981633974483
Math.atan(0); // 0
Math.atan(-0); // -0

Math.atan(Infinity); //  1.5707963267948966
Math.atan(-Infinity); // -1.5707963267948966

// The angle that the line [(0,0);(x,y)] forms with the x-axis in a Cartesian coordinate system
Math.atan(y / x);
```

Talvez você queira evitar usar `±Infinity` por motivos estéticos. Nesse caso, `Math.atan2()` com `0` como segundo parâmentro pode ser uma solução melhor.

Veja também

- `Math.acos()`
- `Math.asin()`
- `Math.atan2()`
- `Math.cos()`
- `Math.sin()`
- `Math.tan()`

Math.atan2()

A função **`Math.atan2()`** retorna o arco tangente do coeficiente dos argumentos passado.

Sintaxe

```
Math.atan2(y, x)
```

Parâmentros

`y`

Primeiro numero.

`x`

Segundo numero.

Retorno

O arco tagente do coegiente dos parâmetros

Descrição

O `Math.atan2()` método retorna um valor numérico entre -π e π representando o ângulo teta entre (x, y). Assim indo no sentido anti-horario ao ângulo, medido em radianos, entre o eixo X positivo, e o ponto`(x, y)`.Nota os argumentos para essa função: primeiro o eixo Y(ordenadas) e o eixo X(absissas) como segundo parâmetro.

 A simple diagram showing the angle returned by atan2(y, x)

`Math.atan2()os argumentos são passados separados x e y enquanto no Math.atan() é passado a razão entre esses argumentos.`

Porque `atan2()` é um método estático de `Math`, você sempre usará `Math.atan2()`, ao inves de um objeto Math criado (`Math` não é um construtor).

Examples

Using <code>Math.atan2()</code>

```
Math.atan2(90, 15); // 1.4056476493802699
Math.atan2(15, 90); // 0.16514867741462683

Math.atan2(±0, -0);               // ±PI.
Math.atan2(±0, +0);               // ±0.
Math.atan2(±0, -x);               // ±PI for x > 0.
Math.atan2(±0, x);                // ±0 for x > 0.
Math.atan2(-y, ±0);               // -PI/2 for y > 0.
Math.atan2(y, ±0);                // PI/2 for y > 0.
Math.atan2(±y, -Infinity);        // ±PI for finite y > 0.
Math.atan2(±y, +Infinity);        // ±0 for finite y > 0.
Math.atan2(±Infinity, x);         // ±PI/2 for finite x.
Math.atan2(±Infinity, -Infinity); // ±3*PI/4.
Math.atan2(±Infinity, +Infinity); // ±PI/4.
```

Veja também

- `Math.acos()`
- `Math.asin()`
- `Math.atan()`
- `Math.cos()`
- `Math.sin()`
- `Math.tan()`

Math.atanh()

A função **`Math.atanh()`** retorna o arco tangente hiperbólico de um número, onde

$$\forall x \in (-1, 1), \text{Math.atanh}(x) = \text{arctanh}(x) = \text{the unique} y \text{such that} \tanh(y) = x$$

Sintaxe

```
Math.atanh(x)
```

Parâmetros

`x`

O número.

Valor retornado

O arco tangente hiperbólico do parâmetro recebido.

Descrição

Por `atanh()` ser um método estático de `Math`, deve-se sempre usá-lo como `Math.atanh()`, e não como um método de um objeto `Math` que você criou.

Exemplos

Usando <code>Math.atanh()</code>

```
Math.atanh(-2); // NaN
Math.atanh(-1); // -Infinity
Math.atanh(0); // 0
Math.atanh(0.5); // 0.5493061443340548
Math.atanh(1); // Infinity
Math.atanh(2); // NaN
```

For values greater than 1 or less than -1, `NaN` is returned.

Para valores menores que -1 ou maiores que 1, `NaN` é retornado.

Polyfill

Para $|x| < 1$, temos $\text{artanh}(x) = \frac{1}{2}\ln\left(\frac{1+x}{1-x}\right)$, esse comportamento pode ser emulado com a seguinte função:

```
Math.atanh =
  Math.atanh ||
  function (x) {
    return Math.log((1 + x) / (1 - x)) / 2;
  };
```

Veja também

- `Math.acosh()`
- `Math.asinh()`
- `Math.cosh()`
- `Math.sinh()`
- `Math.tanh()`

Math.cbrt()

A função **`Math.cbrt()`** retorna a raiz cúbica de um número, isto é

$$Math.cbrt(x) = \sqrt[3]{x} = \text{y, tal que} y^3 = x$$

Sintaxe

```
Math.cbrt(x)
```

Parâmetros

`x`

Um número.

Valor de retorno

A raiz cúbica do número fornecido.

Descrição

Porque `cbrt()` é um método estático de `Math`, você sempre irá utilizar como `Math.cbrt()`, ao invés de um método de um objeto `Math` que você tenha criado (`Math` não é um construtor).

Exemplos

Utilizando <code>Math.cbrt()</code>

```
Math.cbrt(NaN); // NaN
Math.cbrt(-1); // -1
Math.cbrt(-0); // -0
Math.cbrt(-Infinity); // -Infinity
Math.cbrt(0); // 0
Math.cbrt(1); // 1
Math.cbrt(Infinity); // Infinity
Math.cbrt(null); // 0
Math.cbrt(2); // 1.2599210498948734
```

Polyfill

Para todo $x \geq 0$, temos $\sqrt[3]{x} = x^{1/3}$, então isto pode ser simulado pela seguinte função:

```
if (!Math.cbrt) {
  Math.cbrt = function (x) {
    var y = Math.pow(Math.abs(x), 1 / 3);
    return x < 0 ? -y : y;
  };
}
```

Veja também

- `Math.pow()`
- `Math.sqrt()`

Math.ceil()

Resumo

`A função Math.ceil(x)` retorna o menor número inteiro maior ou igual a "x".

Sintaxe

```
Math.ceil(x)
```

Parâmetros

`x`

Um número.

Valor de retorno

O menor inteiro maior ou igual ao número fornecido.

Descrição

Por `ceil` ser um método estático de `Math`, você sempre usará como `Math.ceil()`, e não como um método do objeto `Math` que você criou.

Exemplos

Usando <code>Math.ceil()</code>

O exemplo a seguir mostra um exemplo de uso de `Math.ceil()`.

```
Math.ceil(0.95); // 1
Math.ceil(4); // 4
Math.ceil(7.004); // 8
Math.ceil(-0.95); // -0
Math.ceil(-4); // -4
Math.ceil(-7.004); // -7
```

Ajuste decimal

```
// Closure
(function () {
  /**
   * Decimal adjustment of a number.
   *
   * @param  {String}  type  The type of adjustment.
   * @param  {Number}  value  The number.
   * @param  {Integer}  exp    The exponent (the 10 logarithm of the adjustment base).
   * @returns  {Number}      The adjusted value.
   */
  function decimalAdjust(type, value, exp) {
    // If the exp is undefined or zero...
    if (typeof exp === "undefined" || +exp === 0) {
      return Math[type](value);
    }
    value = +value;
    exp = +exp;
    // If the value is not a number or the exp is not an integer...
    if (isNaN(value) || !(typeof exp === "number" && exp % 1 === 0)) {
      return NaN;
    }
    // Shift
    value = value.toString().split("e");
    value = Math[type](+(value[0] + "e" + (value[1] ? +value[1] - exp : -exp)));
    // Shift back
    value = value.toString().split("e");
    return +(value[0] + "e" + (value[1] ? +value[1] + exp : exp));
  }

  // Decimal round
  if (!Math.round10) {
    Math.round10 = function (value, exp) {
      return decimalAdjust("round", value, exp);
    };
  }
  // Decimal floor
  if (!Math.floor10) {
    Math.floor10 = function (value, exp) {
      return decimalAdjust("floor", value, exp);
    };
  }
  // Decimal ceil
  if (!Math.ceil10) {
    Math.ceil10 = function (value, exp) {
      return decimalAdjust("ceil", value, exp);
    };
  }
})();

// Round
Math.round10(55.55, -1); // 55.6
Math.round10(55.549, -1); // 55.5
Math.round10(55, 1); // 60
Math.round10(54.9, 1); // 50
Math.round10(-55.55, -1); // -55.5
Math.round10(-55.551, -1); // -55.6
Math.round10(-55, 1); // -50
Math.round10(-55.1, 1); // -60
// Floor
Math.floor10(55.59, -1); // 55.5
Math.floor10(59, 1); // 50
Math.floor10(-55.51, -1); // -55.6
Math.floor10(-51, 1); // -60
// Ceil
Math.ceil10(55.51, -1); // 55.6
Math.ceil10(51, 1); // 60
Math.ceil10(-55.59, -1); // -55.5
Math.ceil10(-59, 1); // -50
```

Veja também

- O objeto `Math` pertence a.
- `Math.abs()`
- `Math.floor()`
- `Math.round()`

- `Math.sign()`
- `Math.trunc()`

Math.clz32()

A função Math.clz32 () retorna o número de zero bit inicial na representação binária de 32 bits de um número.

Experimente

```
// 00000000000000000000000000000001
console.log(Math.clz32(1));
// Expected output: 31

// 00000000000000000000000000000100
console.log(Math.clz32(4));
// Expected output: 29

// 00000000000000000000001111101000
console.log(Math.clz32(1000));
// Expected output: 22
```

Sintaxe

```
Math.clz32(x)
```

Parâmetros

x

Um número.

Valor de retorno

O número de zero bits à esquerda na representação binária de 32 bits do número fornecido.

Descrição

"`clz32`" é short para`CountLeadingZeroes32`.

Se x não for um número, ele será convertido em um número primeiro e depois convertido em um número inteiro não assinado de 32 bits.

Se o número inteiro não assinado de 32 bits convertido for 0, retorne 32, porque todos os bits são 0.

Essa função é particularmente útil para sistemas que são compilados para JS, como o Emscripten.

Exemplos

Usando <code>Math.clz32()</code>

```
Math.clz32(1); // 31
Math.clz32(1000); // 22
Math.clz32(); // 32

var stuff = [NaN, Infinity, -Infinity, 0, -0, null, undefined, "foo", {}, []];
stuff.every((n) => Math.clz32(n) == 32); // true

Math.clz32(true); // 31
Math.clz32(3.5); // 30
```

Contagem dos principais e mais além

No momento, não há Math.clon para "Count Leading Ones" (chamado "clon", não "clo", porque "clo" e "clz" são muito semelhantes, especialmente para pessoas que não falam inglês). No entanto, uma função clon pode ser criada facilmente, invertendo os bits de um número e passando o resultado para Math.clz32. Fazer isso funcionará porque o inverso de 1 é 0 e vice-versa. Assim, a inversão dos bits inverterá a quantidade medida de 0s (de Math.clz32), fazendo com que Math.clz32 conte o número de unidades em vez de contar o número de zeros.

Considere a seguinte palavra de 32 bits:

```
var a = 32776; // 00000000000000001000000000001000 (16 zeros à esquerda)
Math.clz32(a); // 16

var b = ~32776; // 11111111111111110111111111110111 (32776 inverso, 0 zeros à esquerda)
Math.clz32(b); // 0 (isso é igual a quantos líderes existem em um)
```

Usando essa lógica, uma função clon pode ser criada da seguinte maneira:

```
var clz = Math.clz32;
function clon(integer) {
  return clz(~integer);
}
```

Além disso, essa técnica pode ser estendida para criar funções inumeráveis "Contagem de zeros à direita" e funções de contagem de zeros, como mostrado abaixo. A função ctrz abaixo preenche todos os bits altos com o bit mais baixo preenchido e depois os anula para apagar todos os bits mais altos definidos, de modo que o clz

possa ser usado.

```
var clz = Math.clz32;
function ctrz(integer){ // contar zeros à direita
   // 1. preencha todos os bits mais altos após o primeiro
    integer |= integer << 16;
    integer |= integer << 8;
    integer |= integer << 4;
    integer |= integer << 2;
    integer |= integer << 1;
// 2. Agora, a inversão dos bits revela os bits mais baixos
    return 32 - clz(~integer) |0; // `|0`garante coerção inteira
}
function ctron(integer){ // conta os que estão à direita
     // Nenhum operador shift-fill-in-with-ones está disponível em
     // JavaScript, portanto, o código abaixo é o mais rápido
    return ctrz(~integer);
/ * Implementação alternativa para fins demonstrativos:
        // 1. apaga todos os bits mais altos após o primeiro zero
       integer &= (integer << 16) | 0xffff;
       integer &= (integer << 8 ) | 0x00ff;
       integer &= (integer << 4 ) | 0x000f;
       integer &= (integer << 2 ) | 0x0003;
       integer &= (integer << 1 ) | 0x0001;
      // 2. Agora, reverter os bits revela os zeros mais baixos
       return 32 - clon(~integer) |0;
    */
}
```

Transforme essas funções auxiliares no módulo ASM.JS; então, você tem uma verdadeira obra-prima de desempenho. Situações como essas são exatamente para o que o ASM.JS foi projetado.

```
var countTrailsMethods = (function (stdlib, foreign, heap) {
  "use asm";
  var clz = stdlib.Math.clz32;
  function ctrz(integer) {
    // count trailing zeros
    integer = integer | 0; // coerce to an integer
    // 1. preencha todos os bits mais altos após o primeiro
    // ASM js, por algum motivo, não permite ^ =, & = ou | =
    integer = integer | (integer << 16);
    integer = integer | (integer << 8);
    integer = integer | (integer << 4);
    integer = integer | (integer << 2);
    integer = integer | (integer << 1);
    // 2. Agora, a inversão dos bits revela os bits mais baixos
    return (32 - clz(~integer)) | 0;
  }
  function ctron(integer) {
    //contar os últimos
    integer = integer | 0; // coagir a um número inteiro
    return ctrz(~integer) | 0;
  }
  // infelizmente, o ASM.JS exige objetos compactos lentos:
  return { a: ctrz, b: ctron };
})(window, null, null);
var ctrz = countTrailsMethods.a;
var ctron = countTrailsMethods.b;
```

Polyfill-"Trecho de código"

O seguinte polyfill é o mais eficiente.

```
if (!Math.clz32)
  Math.clz32 = (function (log, LN2) {
    return function (x) {
      // Seja n ToUint32 (x).
      // Seja p o número de zero bits iniciais em
      // a representação binária de 32 bits de n.
      // Retornar p.
      var asUint = x >>> 0;
      if (asUint === 0) {
        return 32;
      }
      return (31 - ((log(asUint) / LN2) | 0)) | 0; // the "| 0" acts like math.floor
    };
  })(Math.log, Math.LN2);
```

Veja também

- `Math`
- `Math.imul`

Math.cos()

A função **`Math.cos()`** ,que é estatica, retorna o cosseno de um ângulo, que deve estar em radianos. A expressão é $\frac{\text{length}_{\text{adjacent}}}{\text{length}_{\text{hypotenuse}}}$.

Experimente

```
function getCircleX(radians, radius) {
  return Math.cos(radians) * radius;
}

console.log(getCircleX(1, 10));
// Expected output: 5.403023058681398

console.log(getCircleX(2, 10));
// Expected output: -4.161468365471424

console.log(getCircleX(Math.PI, 10));
// Expected output: -10
```

Sintaxe

```
Math.cos(x)
```

Parâmetros

x

Ângulo em radianos

Valor de retorno

O cosseno do número: x

Descrição

`Math.cos()` é um método que retorna um número entre -1 e 1, que representa o cosseno de um ângulo.

Já que `cos()` é um método estático de `Math`, você sempre deve usar `Math.cos()`, ao invez de criar um objeto de `Math` (`Math` não tem construtor).

Exemplos

Usando <code>Math.cos()</code>

```
Math.cos(0); // 1
Math.cos(1); // 0.5403023058681398

Math.cos(Math.PI); // -1
Math.cos(2 * Math.PI); // 1
```

Veja também

- `Math.acos()`
- `Math.asin()`
- `Math.atan()`
- `Math.atan2()`
- `Math.sin()`
- `Math.tan()`

Math.cosh()

A função **`Math.cosh()`** retorna o cosseno hiperbólico de um número, que pode ser expressada usando constante e:

$$\text{Math.cosh(x)} = \frac{e^{x} + e^{-x}}{2}$$

Experimente

```
console.log(Math.cosh(0));
// Expected output: 1

console.log(Math.cosh(1));
// Expected output: 1.543080634815244 (approximately)

console.log(Math.cosh(-1));
// Expected output: 1.543080634815244 (approximately)

console.log(Math.cosh(2));
// Expected output: 3.7621956910836314
```

Sintaxe

```
Math.cosh(x)
```

Parâmetros

`x`

Um número.

Valor de retorno

O cosseno hiperbólico do número dado.

Descrição

Por `cosh()` ser um método estático de `Math`, sempre utilize como `Math.cosh()`, ao invés de como um método de um objeto `Math` que você criou (`Math` não é um construtor).

Exemplos

Usando <code>Math.cosh()</code>

```
Math.cosh(0); // 1
Math.cosh(1); // 1.5430806348152437
Math.cosh(-1); // 1.5430806348152437
```

Polyfill

Isto pode ser emulado com a ajuda da função `Math.exp()`:

```
Math.cosh =
  Math.cosh ||
  function (x) {
    return (Math.exp(x) + Math.exp(-x)) / 2;
  };
```

ou usando apenas uma chamada da função `Math.exp()`:

```
Math.cosh =
  Math.cosh ||
  function (x) {
    var y = Math.exp(x);
    return (y + 1 / y) / 2;
  };
```

See also

- `Math.acosh()`
- `Math.asinh()`
- `Math.atanh()`
- `Math.sinh()`
- `Math.tanh()`

Math.E

A propriedade **`Math.E`** representa a base dos logarítmos naturais, aproximadamente 2.718.

$$\text{Math.E} = e \approx 2.718$$

Experimente

```
function compoundOneYear(interestRate, currentVal) {
  return currentVal * Math.E ** interestRate;
}

console.log(Math.E);
// Expected output: 2.718281828459045

console.log((1 + 1 / 1000000) ** 1000000);
// Expected output: 2.718280469 (approximately)

console.log(compoundOneYear(0.05, 100));
// Expected output: 105.12710963760242
```

Property attributes of `Math.E`	
Writable	no
Enumerable	no
Configurable	no

Descrição

Como `E` é uma propriedade estática de `Math`, sempre use-a como `Math.E`, ao invés de uma propriedade de um objeto `Math` que você criou (`Math` não é um construtor).

Exemplos

Usando <code>Math.E</code>

A seguinte função retorna o valor de *e*:

```
function getNapier() {
  return Math.E;
}

getNapier(); // 2.718281828459045
```

Veja também

- `Math.exp()`
- `Math.log()`
- `Math.log1p()`

Math.exp()

Sumário

A função **`Math.exp()`** retorna `ex`, onde `x` é o argumento, e `e` é a Constante de Euler, a base dos logaritmos naturais

Síntaxe

```
Math.exp(x);
```

Parâmetros

`x`

Um número.

Descrição

Pelo fato de `exp` ser um método estático de `Math`, você sempre utiliza-o como `Math.exp()`, não como um método do objeto `Math` que você criou.

Exemplos

Exemplo: Usando <code>Math.exp</code>

```
Math.exp(-1); // 0.36787944117144233
Math.exp(0); // 1
Math.exp(1); // 2.718281828459045
```

Veja também

- O objeto `Math` ao qual ele pertence.
- `Math.E`
- `Math.log()`
- `Math.pow()`

Math.expm1()

The **`Math.expm1()`** function returns `ex - 1`, where `x` is the argument, and e the base of the natural logarithms.

Experimente

```
console.log(Math.expm1(0));
// Expected output: 0

console.log(Math.expm1(1));
// Expected output: 1.718281828459045

console.log(Math.expm1(-1));
// Expected output: -0.6321205588285577

console.log(Math.expm1(2));
// Expected output: 6.38905609893065
```

Syntax

```
Math.expm1(x)
```

Parameters

`x`

Um número.

Return value

Um número representando `ex - 1`, onde `e` é Euler's number e `x` ié o argumento.

Description

Porque `expm1()` é um método estático de is `Math`, você sempre o usurá como `Math.expm1()`, do que como um método de um objeto `Math` que você criou (`Math` não é um contrutor).

Polyfill

This can be emulated with the help of the `Math.exp()` function:

```
Math.expm1 =
  Math.expm1 ||
  function (x) {
    return Math.exp(x) - 1;
  };
```

Examples

Using <code>Math.expm1()</code>

```
Math.expm1(-1); // -0.6321205588285577
Math.expm1(0); // 0
Math.expm1(1); // 1.718281828459045
```

See also

- `Math.E`
- `Math.exp()`
- `Math.log()`
- `Math.log10()`
- `Math.log1p()`
- `Math.log2()`
- `Math.pow()`

Math.floor()

Resumo

A função **`Math.floor(x)`** retorna o menor número inteiro dentre o número "x".

Sintaxe

```
Math.floor(x)
```

Parâmetros

`x`

Um número.

Descrição

Por `floor` ser um método estático de `Math`, você sempre irá usar como `Math.floor()`,e não como método do objeto `Math que você criou`.

Exemplos

Exemplo: Usando <code>Math.floor</code>

```
Math.floor(45.95); //  45
Math.floor(-45.95); // -46
```

Exemplo: Ajuste Decimal

```
// Closure
(function () {
  /**
   * Decimal adjustment of a number.
   *
   * @param  {String}  type  The type of adjustment.
   * @param  {Number}  value  The number.
   * @param  {Integer}  exp    The exponent (the 10 logarithm of the adjustment base).
   * @returns  {Number}      The adjusted value.
   */
  function decimalAdjust(type, value, exp) {
    // If the exp is undefined or zero...
    if (typeof exp === "undefined" || +exp === 0) {
      return Math[type](value);
    }
    value = +value;
    exp = +exp;
    // If the value is not a number or the exp is not an integer...
```

```
    if (isNaN(value) || !(typeof exp === "number" && exp % 1 === 0)) {
      return NaN;
    }
    // Shift
    value = value.toString().split("e");
    value = Math[type](+(value[0] + "e" + (value[1] ? +value[1] - exp : -exp)));
    // Shift back
    value = value.toString().split("e");
    return +(value[0] + "e" + (value[1] ? +value[1] + exp : exp));
  }

  // Decimal round
  if (!Math.round10) {
    Math.round10 = function (value, exp) {
      return decimalAdjust("round", value, exp);
    };
  }
  // Decimal floor
  if (!Math.floor10) {
    Math.floor10 = function (value, exp) {
      return decimalAdjust("floor", value, exp);
    };
  }
  // Decimal ceil
  if (!Math.ceil10) {
    Math.ceil10 = function (value, exp) {
      return decimalAdjust("ceil", value, exp);
    };
  }
})();

// Round
Math.round10(55.55, -1); // 55.6
Math.round10(55.549, -1); // 55.5
Math.round10(55, 1); // 60
Math.round10(54.9, 1); // 50
Math.round10(-55.55, -1); // -55.5
Math.round10(-55.551, -1); // -55.6
Math.round10(-55, 1); // -50
Math.round10(-55.1, 1); // -60
// Floor
Math.floor10(55.59, -1); // 55.5
Math.floor10(59, 1); // 50
Math.floor10(-55.51, -1); // -55.6
Math.floor10(-51, 1); // -60
// Ceil
Math.ceil10(55.51, -1); // 55.6
Math.ceil10(51, 1); // 60
Math.ceil10(-55.59, -1); // -55.5
Math.ceil10(-59, 1); // -50
```

Veja também

- The `Math` object it belongs to.
- `Math.abs`
- `Math.ceil()`
- `Math.round()`

Math.hypot()

A função **`Math.hypot()`** retorna a raiz quadrada do somatório do quadrado de seus parâmetros, ou seja

$$\text{Math.hypot}\ (v_1, v_2, \ \dots, v_n) = \sqrt{\sum_{i=1}^{n} v_i^2} = \sqrt{v_1^2 + v_2^2 + \ \dots \ + v_n^2}$$

1, v_2, \dots, v_n)} = \sqrt{\sum{i=1}^n v_i^2} = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}

Sintaxe

```
Math.hypot([value1[, value2[, ...]]])
```

Parâmetros

`value1, value2, ...`

Números.

Valor retornado

A raiz quadrada do somatório do quadrado dos parâmetros recebidos. Se um ou mais argumentos não puderem ser convertidos para um número, `NaN` é retornado.

Descrição

Para calcular a hipotenusa de um triângulo retângulo, ou o módulo de um número complexo, é usada a fórmula `Math.sqrt(v1*v1 + v2*v2)` ($\sqrt{v1^2 + v2^2}$) onde v1 e v2 são, ou os lados de um triângulo, ou a parte real e a imaginário de um número complexo. Para calcular a distância entre duas ou mais dimensões, basta adicionar mais

exponenciações dentro da raiz quadrada, por exemplo `Math.sqrt(v1*v1 + v2*v2 + v3*v3 + v4*v4)` ($\sqrt{v1^2 + v2^2 + v3^2 + v4^2}$).

A função Math.hypot() torna esta tarefa mais rápida e mais fácil, basta executar `Math.hypot(v1, v2)` , or `Math.hypot(v1, v2, v3, v4, ...)` .

Dessa maneira também se evita problemas se a magnitude dos seus número for muito grande. O maio número que se pode representar em um *double float* em JavasScript é `Number.MAX_VALUE` = 1.797...e+308. Se os seu números são maior que 1e154, calcular o quadrado deles resultará em `Infinity`, estragando os seus resultados. Por exemplo, `Math.sqrt(1e200*1e200 + 1e200*1e200) = Infinity`. Se você usar a função `Math.hypot()`, você receberá uma resposta aceitável: `Math.hypot(1e200, 1e200) = 1.4142...e+200`. Isto também é verdade para número muito pequenos. `Math.sqrt(1e-200*1e-200 + 1e-200*1e-200) = 0`, mas `Math.hypot(1e-200, 1e-200) = 1.4142...e-200` é uma boa resposta.

> **Nota:** Por `hypot()` ser um método estático de `Math`, deve-se sempre usá-lo como `Math.hypot()`, e não como um método de um objeto `Math` que você criou.

Se nenhum parâmetro for passado, o resultado é +0.

Se um ou mais parâmetros não puderem ser convertidos para um número, o resultado será `NaN`.

Com apenas um parâmetro, `Math.hypot()` se comporta como `Math.abs()`.

Examples

Usando <code>Math.hypot()</code>

```
Math.hypot(3, 4); // 5
Math.hypot(3, 4, 5); // 7.0710678118654755
Math.hypot(); // 0
Math.hypot(NaN); // NaN
Math.hypot(3, 4, "foo"); // NaN, +'foo' => NaN
Math.hypot(3, 4, "5"); // 7.0710678118654755, +'5' => 5
Math.hypot(-3); // 3, the same as Math.abs(-3)
```

Polyfill

O comportamento de `Math.hypot()` pode ser emulado com a seguinte função:

```
Math.hypot =
  Math.hypot ||
  function () {
    var y = 0;
    var length = arguments.length;

    for (var i = 0; i < length; i++) {
      if (arguments[i] === Infinity || arguments[i] === -Infinity) {
        return Infinity;
      }
      y += arguments[i] * arguments[i];
    }
    return Math.sqrt(y);
  };
```

Ver também

- `Math.abs()`
- `Math.pow()`
- `Math.sqrt()`

Math.LN10

A propriedade **`Math.LN10`** representa o logaritmo natural de 10, aproximadamente 2.302:

$$\text{Math.LN10} = \ln(10) \approx 2.302$$

Property attributes of `Math.LN10`	
Writable	no
Enumerable	no
Configurable	no

Descrição

Como `LN10` é uma propriedade estática de `Math`, você sempre deve usa-lo como `Math.LN10`, ao invés de uma propriedade do objeto `Math` que você tenha criado (`Math` não é um construtor).

Exemplos

Usando <code>Math.LN10</code>

A seguinte função retorna o log natural de 10:

```
function getNatLog10() {
  return Math.LN10;
}
```

```
getNatLog10(); // 2.302585092994046
```

Veja também

- `Math.exp()`
- `Math.log()`
- `Math.log10()`

Math.LN2

A propriedade **`Math.LN2`** representa o logaritmo natural (também conhecido como logaritmo neperiano) de 2, que é aproximadamente 0.693:

$$\text{Math.LN2} = \ln(2) \approx 0.693$$

Property attributes of `Math.LN2`	
Writable	no
Enumerable	no
Configurable	no

Descrição

Por `LN2` ser uma propriedade estática de `Math`, deve-se sempre usá-la como `Math.LN2`, e não como uma propriedade de um objeto `Math` que você criou.

Exemplos

Usando <code>Math.LN2</code>

A função a seguir retorna o logaritmo natural de 2:

```
function getNatLog2() {
  return Math.LN2;
}

getNatLog2(); // 0.6931471805599453
```

Veja também

- `Math.exp()`
- `Math.log()`
- `Math.log2()`

Math.log()

A função **`Math.log()`** retorna o logaritmo natural(base `e`) de um número, que é:

$$\forall x > 0, \text{Math.log}\ (x) = \ln(x) = \text{the unique} y \text{such that} e^{y} = x$$

Sintaxe

```
Math.log(x)
```

Parâmetros

`x`

Um número.

Retorno

O logaritmo natural (base `e`) de um número dado. Se o número é negativo, `NaN` é retornado.

Descrição

Se o valor de `x` é negativo, o retorno será sempre `NaN`.

Por `log()` ser um método estático de `Math`, você sempre o usará como `Math.log()`, ao invés de um método de um objeto de `Math` que você criou(`Math` não é um construtor).

Caso você precise do logaritmo natural de 2 ou 10, use as constantes `Math.LN2` ou `Math.LN10` . Caso você precise de um logaritmo de base 2 ou 10, use `Math.log2()` ou `Math.log10()` . Caso você precise utilizar logaritmo de outras bases, use Math.log(x) / Math.log(outraBase) como no exemplo abaixo; talvez você queira pré-calcular 1 / Math.log(outraBase) .

Exemplos

Usando <code>Math.log()</code>

```
Math.log(-1); // NaN, out of range
Math.log(0); // -Infinity
Math.log(1); // 0
Math.log(10); // 2.302585092994046
```

Usando <code>Math.log()</code> como uma base diferente

As funções a seguir retornam o logaritmo de `y` na base `x` (ie. $\log_x y$):

```
function getBaseLog(x, y) {
  return Math.log(y) / Math.log(x);
}
```

Caso você execute `getBaseLog(10, 1000)` será retornado `2.9999999999999996` devido ao arredondamento de ponto-flutuante, o qual é bem próximo do retorno exato de 3.

Veja também

- `Math.exp()`
- `Math.log1p()`
- `Math.log10()`
- `Math.log2()`
- `Math.pow()`

Math.log10()

A função **`Math.log10()`** retorna o logaritmo de base 10 de um número, que é

$$\forall x > 0, \text{Math.log10}\ (x) = \log_{10}(x) = \text{the unique} y \text{such that} 10^y = x$$

Sintaxe

```
Math.log10(x)
```

Parâmetros

`x`

Um número.

Retorno

O logaritmo de base 10 de um número. Caso o número seja negativo, `NaN` é retornado.

Descrição

Caso o valor de `x` seja menor que 0, então o retorno será sempre `NaN`.

Por `log10()` ser um método estático de `Math`, você sempre o usará como `Math.log10()`, ao invés de usá-lo como método de um objeto `Math` criado (`Math` não é um construtor).

Esta função é equivalente a Math.log(x) / Math.log(10). Para log10(e) use a constante `Math.LOG10E` que é 1 / `Math.LN10`.

Exemplos

Usando <code>Math.log10()</code>

```
Math.log10(2); // 0.3010299956639812
Math.log10(1); // 0
Math.log10(0); // -Infinity
Math.log10(-2); // NaN
Math.log10(100000); // 5
```

Polyfill

Isso pode ser simulado a partir da seguinte função:

```
Math.log10 =
  Math.log10 ||
  function (x) {
    return Math.log(x) * Math.LOG10E;
  };
```

Veja também

- `Math.exp()`
- `Math.log()`
- `Math.log1p()`
- `Math.log2()`
- `Math.pow()`

Math.LOG10E

A propriedade **`Math.LOG10E`** representa o logaritmo com base 10 de `e`, aproximadamente 0.434:

$$\text{Math.LOG10E} = \log_{10}(e) \approx 0.434$$

Property attributes of `Math.LOG10E`	
Writable	no
Enumerable	no
Configurable	no

Descrição

Por `LOG10E` ser uma propriedade estática de `Math`, deve-se sempre usá-la como `Math.LOG10E`, e não como propriedade de um objeto `Math` criado por você (`Math` não é um construtor).

Exemplos

Usando <code>Math.LOG10E</code>

A função a seguir retorna o logaritmo com base 10 de `e`:

```
function getLog10e() {
  return Math.LOG10E;
}

getLog10e(); // 0.4342944819032518
```

Veja também

- `Math.exp()`
- `Math.log()`
- `Math.log10()`

Math.log1p()

A função **`Math.log1p()`** retorna o logaritmo natural (base `e`) de 1 + um número, isto é

$$\forall x > -1, \text{Math.log1p}(x) = \ln(1 + x)$$

Experimente

```
console.log(Math.log1p(1));
// Expected output: 0.6931471805599453

console.log(Math.log1p(0));
// Expected output: 0

console.log(Math.log1p(-1));
// Expected output: -Infinity

console.log(Math.log1p(-2));
// Expected output: NaN
```

Sintaxe

```
Math.log1p(x)
```

Parâmetros

`x`

Um número.

Valor de retorno

O logaritmo natural (base `e`) de **1** mais o número fornecido. Se o número for menor que **-1**, `NaN` será retornado.

Descrição

Para valores muito pequenos de *x*, adicionando 1 pode reduzir ou eliminar precisão. Valores double floats costuman te dar em torno de 15 digitos de precisão no JavaScript. 1 + 1e-15 = 1.000000000000001, porém, 1 + 1e-16 = 1.000000000000000 e portanto, exatamente 1.0 naquele resultado, porque os números que passam de 15 digitos são arredondados.

Quando você calcula log(1 + x), você obterá um resultado muito perto de x, se x for um valor pequeno (isto é, porque eles são chamados logaritmos 'naturais'). Se você calcular Math.log(1 + 1.1111111111e-15) você obterá uma resposta perto de1.1111111111e-15. Ao invés, você vai acabar obtendo o logaritmo de 1.0000000000000111022 (o arrendondamento é feito em binário, portanto, as vezes isso pode parecer estranho), então você obterá o resultado 1.11022...e-15, com somente 3 digitos corretos. Se, ao invés, você calcular Math.log1p(1.1111111111e-15) você terá um retorno mais preciso de 1.1111111110999995e-15 com 15 digitos corretos de precisão (na verdade 16 nesse caso).

Se o valor de `x` for menor que -1, o valor retornado será sempre `NaN`.

Por conta do `log1p()` ser um metódo estático de `Math`, você sempre chamará como `Math.log1p()`, ao invés de chamar como um método de um objeto `Math` que você tenha criado (`Math` não é um construtor).

Exemplos

Usando <code>Math.log1p()</code>

```
Math.log1p(1); // 0.6931471805599453
Math.log1p(0); // 0
Math.log1p(-1); // -Infinity
Math.log1p(-2); // NaN
```

Polyfill

Isto pode ser implementado com a seguinte função:

```
Math.log1p =
  Math.log1p ||
  function (x) {
    return Math.log(1 + x);
  };
```

Veja também

- `Math.exp()`
- `Math.log()`
- `Math.expm1()`
- `Math.log10()`
- `Math.log2()`
- `Math.pow()`

Math.log2()

A função **`Math.log2()`** retorna o logaritmo de base 2 de um número, que é

$$\forall x > 0, \text{Math.log2}\ (x) = \log_2(x) = \text{the unique} y \text{such that} 2^y = x$$

Sintaxe

```
Math.log2(x)
```

Parâmetros

x Um número.

Retorno

O logaritmo de base 2 de um número. Caso o número seja negativo, `NaN` é retornado.

Descrição

Caso o valor de x seja menor que 0, então o retorno será sempre `NaN`.

Por `log2()` ser um método estático de `Math`, você sempre o usará como `Math.log2()`, ao invés de usá-lo como método de um objeto `Math` criado (`Math` não é um construtor).

Esta função é equivalente a `Math.log(x)/Math.log(2)`. Para log2(e) use a constante `Math.LOG2E` que é 1 / `Math.LN2`.

Exemplos

Usando <code>Math.log2()</code>

```
Math.log2(3); // 1.584962500721156
Math.log2(2); // 1
Math.log2(1); // 0
Math.log2(0); // -Infinity
Math.log2(-2); // NaN
Math.log2(1024); // 10
```

Polyfill

Este Polyfill simula a função `Math.log2`. Note que é retornado um valor não prcesiso Observe que é retornado valores imprecisos em algumas entradas (like 1 << 29), envolva em `Math.round()` se estiver trabalhando com máscaras de bits.

```
Math.log2 =
  Math.log2 ||
  function (x) {
    return Math.log(x) * Math.LOG2E;
  };
```

Veja também

- `Math.exp()`
- `Math.log()`
- `Math.log10()`
- `Math.log1p()`

- `Math.pow()`

Math.LOG2E

A propriedade **`Math.LOG2E`** representa o logaritmo com base 2 de e, aproximadamente 1.442:

$$\text{Math.LOG2E} = \log_2(e) \approx 1.442$$

Property attributes of `Math.LOG2E`	
Writable	no
Enumerable	no
Configurable	no

Descrição

Por `LOG2E` ser uma propriedade estática de `Math`, deve-se sempre usá-la como `Math.LOG2E`, e não como propriedade de um objeto `Math` criado por você (`Math` não é um construtor).

Exemplos

Usando <code>Math.LOG2E</code>

A função a seguir retorna o logaritmo com base 2 de e:

```
function getLog2e() {
  return Math.LOG2E;
}

getLog2e(); // 1.4426950408889634
```

Veja também

- `Math.exp()`
- `Math.log()`
- `Math.log2()`

Math.max()

Sumário

A função **`Math.max()`** retorna o maior de um ou mais números.

Sintaxe

```
Math.max([valor1[,valor2, ...]])
```

Parâmetros

`valor1, valor2, ...`

Números.

Valor de retorno

O maior dos números passados como argumentos. Se pelo menos um dos argumentos não puder ser convertido para um número `NaN` é retornado.

Descrição

Por `max` ser um método estático em `Math`, você sempre irá usá-lo da seguinte maneira `Math.max()`, e não como um método da classe `Math` que você tenha instanciado.

Se nenhum argumento for passado o resultado sempre será - `Infinity`.

Se um dos argumentos não puder ser convertido em um número, o resultado será `NaN`.

Exemplos

Usando <code>Math.max</code>

```
Math.max(10, 20); //  20
Math.max(-10, -20); // -10
Math.max(-10, 20); //  20
```

Retornando o maior elemento de um array

`Array.reduce()` pode ser usada para encontrar o maior elemento em um vetor numérico, comparando cada valor:

```
var arr = [1, 2, 3];
var max = arr.reduce(function (a, b) {
  return Math.max(a, b);
}, -Infinity);
```

A função a seguir utiliza `Function.prototype.apply()` para encontrar o elemento de maior valor dentro do array. `getMaxOfArray([1,2,3])` é equivalente a `Math.max(1, 2, 3)`, mas você pode usar `getMaxOfArray` em arrays construídos programaticamente e o ideal é utilizá-la somente em arrays com relativamente poucos elementos.

```
function getMaxOfArray(numArray) {
  return Math.max.apply(null, numArray);
}
```

O novo operador spread é um modo curto de se escrever a solução com `apply` para retornar o maior valor de um array.

```
var arr = [1, 2, 3];
var max = Math.max(...arr);
// max: 3
```

Entretanto, tanto spread(`...`) quanto `apply` irão ou falhar ou retornar o resultado errado caso o array tenha muitos elementos, porque eles tentam passar o array de elementos como parâmetros de funções. Veja usando apply e funções embutidas para mais detalhes. A solução com `reduce` não apresenta esse problema.

Veja também

- O objeto `Math` pertence a.
- `Math.min()`

Math.min()

A função **`Math.min()`** retorna o menor valor passado como parâmetro, ou `NaN` se o parâmetro não é ou não pode ser convertido para um número.

Syntax

```
Math.min([valor1[, valor2[, ...]]])
```

Parâmetros

`valor1, valor2, ...`

Números.

Descrição

Por que `min()` é um método estático de `Math`, você sempre usa como `Math.min()`, e não como um método de um objeto `Math` que você criou (`Math` não é um construtor).

Se nenhum argumento for dado, o resultado é `Infinity`.

Se pelo menos um dos argumentos não pode ser convertido para um número, o resultado é `NaN`.

Exemplos

Usando <code>Math.min()</code>

Este encontra o min de `x` e `y` e atribui a `z` :

```
var x = 10, y = -20;
var z = Math.min(x, y);
```

Cortando um valor com <code>Math.min()</code>

`Math.min()` é muitas vezes usado para cortar um valor sempre menor do que ou igual a um limite. Por exemplo, este.

```
var x = f(foo);

if (x > boundary) {
  x = boundary;
}
```

pode ser escrita como este

```
var x = Math.min(f(foo), boundary);
```

`Math.max()` pode ser usado de uma maneira semelhante ao corte de um valor na outra extremidade.

Veja Também

- `Math.max()`

Math.PI

A propriedade **`Math.PI`** representa a proporção entre circunferência de um círculo com o seu diâmetro, aproximadamente 3.14159:

$$\text{Math.PI} = \pi \approx 3.14159$$

Experimente

```
function calculateCircumference(radius) {
  return 2 * Math.PI * radius;
}

console.log(Math.PI);
// Expected output: 3.141592653589793

console.log(calculateCircumference(10));
// Expected output: 62.83185307179586
```

Property attributes of `Math.PI`	
Writable	no
Enumerable	no
Configurable	no

Descrição

Como `PI` é uma propriedade estática de `Math`, sempre use-a como `Math.PI`, ao invés de uma propriedade de um objeto `Math` que você criou (`Math` não é um construtor).

Exemplos

Usando <code>Math.PI</code>

A seguinte função usa `Math.PI` para calcular a circunferência de um círculo com um dado raio.

```
function calcularCircunferencia(raio) {
  return 2 * Math.PI * raio;
}

calcularCircunferencia(1); // 6.283185307179586
```

Veja também

- `Math`

Math.pow()

A função **`Math.pow()`** retorna a *base* elevada ao expoente *power*, ou seja, `baseexpoente`.

Sintaxe

```
Math.pow(base, expoente)
```

Parâmetros

`base`

O número da base.

`expoente`

O expoente usado para elevar a `base`.

Descrição

Como `pow()` é um método estático de `Math`, você sempre irá usá-lo como `Math.pow()`, ao invés de usá-lo como um método de um objeto do tipo `Math` que você tenha criado (`Math` não é um construtor).

Exemplos

Usando <code>Math.pow()</code>

```
// simples
Math.pow(7, 2); // 49
Math.pow(7, 3); // 343
Math.pow(2, 10); // 1024
// expoentes fracionários
Math.pow(4, 0.5); // 2 (raiz quadrada de 4)
Math.pow(8, 1 / 3); // 2 (raiz cúbica de 8)
Math.pow(2, 0.5); // 1.4142135623730951 (raiz quadrada de 2)
Math.pow(2, 1 / 3); // 1.2599210498948732 (raiz cúbica de 2)
// expoentes com sinais
Math.pow(7, -2); // 0.02040816326530612 == (1/7)2 == (1/49)
Math.pow(8, -1 / 3); // 0.5 == (1/8)1/3 == 1/2
// bases com sinal
Math.pow(-7, 2); // 49 (quadrados sempre são positivos)
Math.pow(-7, 3); // -343 (cubos podem ser negativos conforme a base)
Math.pow(-7, 0.5); // NaN (números negativos não tem uma raiz quadrada real)
// devido ao fato que raízes "par" e "ímpar" são próximas,
// e limitam a precisão de ponto flutuante,
```

```
// bases negativas com expoentes fracionários sempre retornam NaN
Math.pow(-7, 1 / 3); // NaN
```

Veja também

- `Math.cbrt()`
- `Math.exp()`
- `Math.log()`
- `Math.sqrt()`
- Operador Exponencial

Math.random()

Sumário

A função **`Math.random()`** retorna um número pseudo-aleatório no intervalo `[0, 1[`, ou seja, de 0 (inclusivo) até, mas não incluindo, 1 (exclusivo), que depois você pode dimensionar para um intervalo desejado. A implementação seleciona uma semente para o algoritmo de geração de números aleatórios; esta semente não pode ser escolhida ou reatribuída.

> **Nota:** **`Math.random()`** não gera números criptograficamente seguros. Não a use para nada relacionado a segurança. Use a API Web Crypto, mais precisamente o método `window.crypto.getRandomValues()`.

Sintaxe

```
Math.random()
```

Valor retornado

Um número pseudo-aleatório entre 0 (inclusivo) e 1 (exclusivo).

Exemplos

Note que os números em JavaScript são pontos flutuantes que seguem o padrão IEEE 754 com comportamento *arredondar-para-o-par-mais-próximo*, os intervalos que serão citados nos exemplos a seguir (exceto o exemplo do `Math.random()`), não são exatas. Se limites extremamente grandes forem escolhidos (253 ou maior), em raros casos é possível que o limite superior (que seria exclusivo) seja retornado.

Gerando um número aleatório entre 0 (inclusivo) e 1 (exclusivo)

```
function getRandom() {
  return Math.random();
}
```

Gerando um número aleatório entre dois valores

Este exemplo retorna um número entre dois valores definidos. O valor retornado será maior ou igual a `min`, e menor que `max`.

```
function getRandomArbitrary(min, max) {
  return Math.random() * (max - min) + min;
}
```

Gerando um número inteiro aleatório entre dois valores

Este exemplo retorna um número *inteiro* entre dois valores definidos. O valor não poderá ser menor que `min` (ou do próximo inteiro maior que `min`, caso `min` não seja inteiro), e será menor (mas não igual) a `max`.

```
function getRandomInt(min, max) {
  min = Math.ceil(min);
  max = Math.floor(max);
  return Math.floor(Math.random() * (max - min) + min);
}
```

> **Nota:** Pode ser tentandor usar `Math.round()` para arredondar `min` e `max`, mas dessa maneira a aleatoriedade dos números seguiria uma distribuição não-uniforme, que talvez não seja o que você precisa.

Gerando um número inteiro aleatório entre dois valores, inclusive

A função `getRandomInt()` acima tem intervalo com o valor mínimo incluído e o máximo excluído. Mas se você precisar que a função inclua, tanto o mínimo quanto o máximo, em seus resultados? A função `getRandomIntInclusive()` abaixo faz isso.

```
function getRandomIntInclusive(min, max) {
  min = Math.ceil(min);
  max = Math.floor(max);
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

Math.round()

Resumo

A função **`Math.round()`** retorna o valor de um número arredondado para o inteiro mais proximo.

Sintaxe

```
Math.round(x)
```

Parâmetros

x

Um número.

Retorno

O valor de um número dado aproximado para o inteiro mais próximo

Descrição

Se a parte fracionária do número for maior ou igual a 0.5, o argumento x é arredondado para o próximo número inteiro acima, entretanto se a parte fracionária do `número` for menor que 0.5, então o valor de x é arredondado para o próximo número inteiro abaixo. Se a parte fracionária for exatamente igual a 0.5, o número é arredondado para o próximo inteiro na direção de +∞.

Por `round` ser um método estático de `Math`, você sempre irá usá-lo como `Math.round()`, ao invés de usá-lo como um método da instância do objeto `Math` que você criou.

Exemplos

Exemplo: Uso de <code>Math.round</code>

```
// Retorna o valor 20
x = Math.round(20.49);

// Retorna o valor 21
x = Math.round(20.5);

// Retorna o valor -20
x = Math.round(-20.5);

// Retorna o valor -21
x = Math.round(-20.51);

// Retorna 1 (!)
// Note o erro de arredondamento por causa da inacurácia de aritmética de ponto flutuante
// Compare o exemplo abaixo com Math.round(1.005, -2)
x = Math.round(1.005 * 100) / 100;
```

Exemplo: Arredondamento decimal

```
// Closure
(function () {
  /**
   * Ajuste decimal de um número.
   *
   * @param  {String}  type  O tipo de arredondamento.
   * @param  {Number}  value  O número a arredondar.
   * @param  {Integer}  exp    O expoente (o logaritmo decimal da base pretendida).
   * @returns  {Number}      O valor depois de ajustado.
   */
  function decimalAdjust(type, value, exp) {
    // Se exp é indefinido ou zero...
    if (typeof exp === "undefined" || +exp === 0) {
      return Math[type](value);
    }
    value = +value;
    exp = +exp;
    // Se o valor não é um número ou o exp não é inteiro...
    if (isNaN(value) || !(typeof exp === "number" && exp % 1 === 0)) {
      return NaN;
    }
    // Transformando para string
    value = value.toString().split("e");
    value = Math[type](+(value[0] + "e" + (value[1] ? +value[1] - exp : -exp)));
    // Transformando de volta
    value = value.toString().split("e");
    return +(value[0] + "e" + (value[1] ? +value[1] + exp : exp));
  }

  // Arredondamento decimal
  if (!Math.round) {
    Math.round = function (value, exp) {
      return decimalAdjust("round", value, exp);
    };
```

```
  }
  // Decimal arredondado para baixo
  if (!Math.floor) {
    Math.floor = function (value, exp) {
      return decimalAdjust("floor", value, exp);
    };
  }
  // Decimal arredondado para cima
  if (!Math.ceil) {
    Math.ceil = function (value, exp) {
      return decimalAdjust("ceil", value, exp);
    };
  }
})();

// Round (arredondamento)
Math.round(55.55, -1); // 55.6
Math.round(55.549, -1); // 55.5
Math.round(55, 1); // 60
Math.round(54.9, 1); // 50
Math.round(-55.55, -1); // -55.5
Math.round(-55.551, -1); // -55.6
Math.round(-55, 1); // -50
Math.round(-55.1, 1); // -60
Math.round(1.005, -2); // 1.01 -- compare este resultado com Math.round(1.005*100)/100 no exemplo acima
// Floor (para baixo)
Math.floor(55.59, -1); // 55.5
Math.floor(59, 1); // 50
Math.floor(-55.51, -1); // -55.6
Math.floor(-51, 1); // -60
// Ceil (para cima)
Math.ceil(55.51, -1); // 55.6
Math.ceil(51, 1); // 60
Math.ceil(-55.59, -1); // -55.5
Math.ceil(-59, 1); // -50
```

Método de arredondamento PHP

O código abaixo pode ser utilizado para adicionar sua própria versão do Math.round ao seu namespace no qual tenha um parâmetro de precisão. Diferentemente do arredondamento decimal do exemplo acima, esse método não realiza conversão de e para strings, e o parâmetro de precisão funciona da mesma maneira que PHP e Excel onde um positivo 1 seria arredondado para 1 casa decimal e -1 seria arredondado para os decimais.

```
var myNamespace = {};

myNamespace.round = function(number, precision) {
    var factor = Math.pow(10, precision);
    var tempNumber = number * factor;
    var roundedTempNumber = Math.round(tempNumber);
    return roundedTempNumber / factor;
};

myNamespace.round(1234.5678, 1); // 1234.6
myNamespace.round(1234.5678, -1); // 1230
```

Veja também

- `Math.abs()`
- `Math.ceil()`
- `Math.floor()`

Math.sign()

A função **`Math.sign()`** retorna o sinal de um número, indicando se o número é positivo, negativo ou zero.

Sintaxe

```
Math.sign(x)
```

Parâmetros

`x`

Se o parametro passado ao for numerico, a função irá converter o parametro em Um número.

Valor retornado

Um número representando o sinal do argumento fornecido.

Se o argumento é um número positivo, o retorno será 1;

Se o numero passado for negativo, o retorno será -1

Se o argumento for um zero positivo , o retorno será +0

Se o argumento for 0 negativo , o retorno será -0

. Qualquer outro valor que não seja numérico (string transformada em numero por ex.), o retorno será `NaN`

Descrição

Pela razão de `sign()` ser um método estático de `Math`, você sempre deve usá-lo como `Math.sign()`, e não como um método do objeto `Math` que você criou (`Math` não é um construtor).

Esta função possui 5 tipos de valores retornados, `1`, `-1`, `0`, `-0`, `NaN`, que representam "número positivo", "número negativo", "zero positivo", "zero negativo" e `NaN`, respectivamente.

O argumento passado para esta função será implicitamente convertido para o tipo `numérico`.

Exemplos

Usando <code>Math.sign()</code>

```
Math.sign(3); //  1
Math.sign(-3); // -1
Math.sign("-3"); // -1
Math.sign(0); //  0
Math.sign(-0); // -0
Math.sign(NaN); // NaN
Math.sign("foo"); // NaN
Math.sign(); // NaN
```

Polyfill

```
if (!Math.sign) {
  Math.sign = function (x) {
    // Se x é NaN, o resultado é NaN.
    // Se x é -0, o resultado é -0.
    // Se x é +0, o resultado é +0.
    // Se x é negativo e não -0, o resultado é -1.
    // Se x é positivo e não +0, o resultado é +1.
    return (x > 0) - (x < 0) || +x;
    // Uma representação mais estética é mostrada abaixo
    //
    // ( (x > 0) ? 1 : 0 )  // se x é positivo então mais um
    //          +           // senão (porque não poder ser - e +)
    // ( (x < 0) ? -1 : 0 ) // se x é negativo então menos um
    //         ||           // se x é 0, -0, NaN, ou não é um número,
    //         +x           // Então o resultado será x, (ou) se x não é
    //                      // um número, então converte x para número
  };
}
```

No polyfill acima, nenhuma coerção de tipo extra é necessária para tornar numéricos as expressões `(x > 0) ou (x < 0)` , porque subtraindo-as um do outro força uma conversão de tipo de booleano para numérico.

Veja também

- `Math.abs()`
- `Math.ceil()`
- `Math.floor()`
- `Math.round()`
- `Math.trunc()`

Math.sin()

A função **`Math.sin()`** retorna o seno de um número.

Experimente

```
function getCircleY(radians, radius) {
  return Math.sin(radians) * radius;
}

console.log(getCircleY(1, 10));
// Expected output: 8.414709848078965

console.log(getCircleY(2, 10));
// Expected output: 9.092974268256818

console.log(getCircleY(Math.PI, 10));
// Expected output: 1.2246467991473533e-15
```

Sintaxe

```
Math.sin(x)
```

Paramêtros

`x`

Um número (dado em radianos)

Valor retornado

O seno de um número dado.

Descrição

O método `Math.sin()` retorna um valor numérico entre -1 e 1, que representa o seno de um angulo dado em radianos.

Como `sin()` é um método estático de `Math`, você sempre o usa como `Math.sin()`, ao invés de um método de um objeto Math que você criou (`Math` não é um construtor).

Exemplos

Usando <code>Math.sin()</code>

```
Math.sin(0); // 0
Math.sin(1); // 0.8414709848078965

Math.sin(Math.PI / 2); // 1
```

Veja também

- `Math.acos()`
- `Math.asin()`
- `Math.atan()`
- `Math.atan2()`
- `Math.cos()`
- `Math.tan()`

Math.sinh()

A função **`Math.sinh()`** retorna o seno hiperbólico de um número, que pode ser expresso usando a constante e:

$$\text{Math.sinh(x)} = \frac{e^x - e^{-x}}{2}$$

Experimente

```
console.log(Math.sinh(0));
// Expected output: 0

console.log(Math.sinh(1));
// Expected output: 1.1752011936438014

console.log(Math.sinh(-1));
// Expected output: -1.1752011936438014

console.log(Math.sinh(2));
// Expected output: 3.626860407847019
```

Sintáxe

```
Math.sinh(x)
```

Parâmetros

`x`

Um número.

Valor retornado

O seno hiperbólico do número dado.

Descrição

Como `sinh()` é um método estático de `Math`, você sempre deve usar como `Math.sinh()`, ao invés de um novo objeto instanciado `Math` (`Math` não é um construtor).

Exemplos

Usando <code>Math.sinh()</code>

```
Math.sinh(0); // 0
Math.sinh(1); // 1.1752011936438014
```

Polyfill

Isso pode ser emulado com a ajuda da função `Math.exp()`:

```
Math.sinh =
  Math.sinh ||
  function (x) {
    return (Math.exp(x) - Math.exp(-x)) / 2;
  };
```

ou usando apenas uma chamada para a função `Math.exp()`:

```
Math.sinh =
  Math.sinh ||
  function (x) {
    var y = Math.exp(x);
    return (y - 1 / y) / 2;
  };
```

Veja também

- `Math.acosh()`
- `Math.asinh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.tanh()`

Math.sqrt()

Resumo

A função **Math.sqrt()** retorna a raiz quadrada de um número ($\sqrt{x}$) .

Sintaxe

```
Math.sqrt(x)
```

Parâmetros

`x`

Um número.

Valor retornado

A raiz quadrada do número recebido. Se o número for negativo, a função retornará `NaN`.

Descrição

Se o valor de `x` for negativo, `Math.sqrt()` retorna `NaN`.

Por `sqrt` ser um método estático de `Math`, deve-se sempre usá-lo como `Math.sqrt()`, e não como um método de um objeto `Math` que você criou.

Exemplos

Exemplo: Usando <code>Math.sqrt</code>

```
Math.sqrt(9); // 3
Math.sqrt(2); // 1.414213562373095

Math.sqrt(1); // 1
Math.sqrt(0); // 0
Math.sqrt(-1); // NaN
```

Ver também

- `Math.cbrt()`
- `Math.exp()`
- `Math.log()`
- `Math.pow()`

Math.SQRT1_2

A propriedade `Math.SQRT1_2` representa a raiz quadrada de $\frac{1}{2}$, que é aproximadamente 0.707:

$$\text{Math.SQRT1_2} = \sqrt{\frac{1}{2}} = \frac{1}{\sqrt{2}} \approx 0.707$$

Property attributes of `Math.SQRT1_2`	
Writable	no
Enumerable	no
Configurable	no

Descrição

Por `SQRT1_2` ser um método estático de `Math`, deve-se sempre usá-lo como `Math.SQRT1_2()`, e não como um método de um objeto `Math` que você criou.

Exemplos

Usando <code>Math.SQRT1_2</code>

A função a seguir retorna 1 sobre a raiz quadrada de 2:

```
function getRoot1_2() {
  return Math.SQRT1_2;
}

getRoot1_2(); // 0.7071067811865476
```

Veja também

- `Math.pow()`
- `Math.sqrt()`

Math.SQRT2

A propriedade **`Math.SQRT2`** representa a raiz quadrada de 2, que é aproximadamente 1.414:

$$\text{Math.SQRT2} = \sqrt{2} \approx 1.414$$

Property attributes of `Math.SQRT2`	
Writable	no
Enumerable	no
Configurable	no

Descrição

Por `SQRT2` ser uma propriedade estática de `Math`, deve-se sempre usá-la como `Math.SQRT2`, e não como propriedade de um objeto `Math` criado por você (`Math` não é um construtor).

Examples

Usando <code>Math.SQRT2</code>

A função a seguir retorna a raiz quadrada de 2:

```
function getRoot2() {
  return Math.SQRT2;
}

getRoot2(); // 1.4142135623730951
```

Veja também

- `Math.pow()`
- `Math.sqrt()`

Math.tan()

Resumo

A função **`Math.tan()`** retorna a tangente de um número.

Sintaxe

```
Math.tan(x)
```

Parâmetros

`x`

Um número representando um ângulo em radianos.

Descrição

O método `tan` retorna um valor numérico que representa a tangente do ângulo.

Como `tan` é um método estático de `Math`, use sempre `Math.tan()`, ao invés de um método de um objeto `Math` que você tenha criado.

Exemplos

Exemplo: Usando <code>Math.tan</code>

A seguinte função retorna a tangente da variável `x`:

```
function getTan(x) {
  return Math.tan(x);
}
```

Como a função `Math.tan()` trabalha com radianos, mas normalment epe mais fácil trabalhar em graus, a seguinte função aceita um valor em graus, converte-o para radianos e retorna a tangente.

```
function getTanDeg(deg) {
  var rad = (deg * Math.PI) / 180;
  return Math.tan(rad);
}
```

Math.tanh()

A função **`Math.tanh()`** retorna a tangente hiperbólica de um número, que é:

$$\tanh x = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{e^{2x} - 1}{e^{2x} + 1}$$

Experimente

```
console.log(Math.tanh(-1));
// Expected output: -0.7615941559557649

console.log(Math.tanh(0));
// Expected output: 0

console.log(Math.tanh(Infinity));
// Expected output: 1

console.log(Math.tanh(1));
// Expected output: 0.7615941559557649
```

Syntax

```
Math.tanh(x)
```

Parameters

`x`

Um número.

Return value

Uma tangente hiperbólica de um número dado.

Description

Because `tanh()` is a static method of `Math`, you always use it as `Math.tanh()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Examples

Using <code>Math.tanh()</code>

```
Math.tanh(0); // 0
Math.tanh(Infinity); // 1
Math.tanh(1); // 0.7615941559557649
```

Polyfill

This can be emulated with the help of the `Math.exp()` function:

```
Math.tanh =
  Math.tanh ||
  function (x) {
    var a = Math.exp(+x),
      b = Math.exp(-x);
    return a == Infinity ? 1 : b == Infinity ? -1 : (a - b) / (a + b);
  };
```

See also

- `Math.acosh()`
- `Math.asinh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.sinh()`

Math.trunc()

O método **`Math.trunc()`** retorna a parte inteira de um número, descartando suas casas decimais.

Experimente

```
console.log(Math.trunc(13.37));
// Expected output: 13

console.log(Math.trunc(42.84));
// Expected output: 42

console.log(Math.trunc(0.123));
// Expected output: 0

console.log(Math.trunc(-0.123));
// Expected output: -0
```

Sintaxe

```
Math.trunc(x)
```

Parâmetros

`x`

Um número.

Valor de retorno

A parte inteira de um dado número.

Descrição

Diferente dos demais métodos em Math: `Math.floor()`, `Math.ceil()` e `Math.round()`, o retorno esperado da função `Math.trunc()` é simples e direto, ela apenas trunca o número passado a ela como parâmetro, removendo todas as casas decimais dele, não importando se o número é positivo ou negativo.

Portanto, se o argumento passado for um número positivo, `Math.trunc()` será equivalente a `Math.floor()`, caso contrário `Math.trunc()` será equivalente a `Math.ceil()`.

O argumento passado a esse método será convertido a um tipo numérico implicitamente.

Já que `trunc()` é um método estático em `Math`, sempre utilize `Math.trunc()`, ao invés de um método existente no objeto que você criou (`Math` não é um construtor).

Exemplos

Usando <code>Math.trunc()</code>

```
Math.trunc(13.37); // 13
Math.trunc(42.84); // 42
Math.trunc(0.123); //  0
Math.trunc(-0.123); // -0
Math.trunc("-1.123"); // -1
Math.trunc(NaN); // NaN
Math.trunc("foo"); // NaN
Math.trunc(); // NaN
```

Polyfill

(um*Polyfill* é um pedaço de código que o desenvolvedor pode colocar na sua página para garantir compatibilidade do método. No exemplo abaixo, caso o navegador não tiver o método `trunc` na classe `Math`, ele será criado);

```
Math.trunc =
  Math.trunc ||
  function (x) {
    return x < 0 ? Math.ceil(x) : Math.floor(x);
  };
```

Veja também

- `Math.abs()`
- `Math.ceil()`
- `Math.floor()`
- `Math.round()`
- `Math.sign()`

NaN

Resumo

A propriedade global **`NaN`** é um valor especial que significa *Not-A-Number* (não é um número).

Property attributes of `NaN`	
Writable	no
Enumerable	no

Property attributes of `NaN`	
Configurable	no

Sintaxe

```
NaN
```

Descrição

`NaN` é uma propriedade do *objeto global*.

O valor inicial de `NaN` é Not-A-Number - o mesmo valor de `Number.NaN`. Nos navegadores modernos, o `NaN` é uma propriedade somente leitura e não configurável. Mesmo quando não for este o caso, evite sobrescrevê-lo.

Não é usual a utilização do `NaN`. Ele é retornado quando uma operação matemática falha (Math.sqrt(-1)) ou quando uma função tenta transformar uma string em inteiro (parseInt("blabla")).

Testando um valor <code>NaN</code>

Os operadores de igualdade (`==` e `===`) não podem ser usados para testar um valor NaN. Ao invés disso, utilize `Number.isNaN()` ou `isNaN()`.

```
NaN === NaN; // falso
Number.NaN === NaN; // falso
isNaN(NaN); // verdadeiro
isNaN(Number.NaN); // verdadeiro
```

Veja também

- `Number.NaN`
- `Number.isNaN()`
- `isNaN()`

Número

Sumário

O objeto JavaScript **`Number`** é um objeto encapsulado que permite você trabalhar com valores numéricos. Um objeto `Number` é criado utilizando o construtor `Number()`.

Construtor

```
new Number(value);
```

Parâmetros

`value`

O valor numérico do objeto sendo criado.

Descrição

Os principais usos para o objeto `Number` são:

- Se o objeto não pode ser convertido para um número, é retornado `NaN`.
- Fora do contexo de um construtor (Ex., Sem o operador `new`, `Number` pode ser utilizador para realizar uma conversão de tipo.

Propriedades

`Number.EPSILON`

O menor intervalo entre dois números representáveis.

`Number.MAX_SAFE_INTEGER`

O inteiro máximo seguro em JavaScript (253 -1).

`Number.MAX_VALUE`

O maior número representável positivo.

`Number.MIN_SAFE_INTEGER`

O inteiro mínimo seguro em JavaScript (-(253 -1)).

`Number.MIN_VALUE`

O número mínimo representável positivo - isto é, o número positivo mais próximo de zero (sem ser zero na verdade).

`Number.NaN`

Valor especial que não é número.

`Number.NEGATIVE_INFINITY`

Valor especial representando infinito negativo; retornado no "overflow".

`Number.POSITIVE_INFINITY`

Valor especial representando infinito; retornado no "overflow".

`Number`

Permite a adição de propriedades a um objeto Number.

Methods

`Number.isNaN()`

Determina se o valor passado é NaN.

`Number.isFinite()`

Determina se o tipo e o valor passado é um número finito.

`Number.isInteger()`

Determina se o tipo do valor passado é inteiro.

`Number.isSafeInteger()`

Determina se o tipo do valor passado é um inteiro seguro (número entre -(253 -1) e 253 -1).

`~~Number.toInteger()~~`

~~Usado para avaliar o valor passado e convertê-lo a um inteiro (ou infinito), mas foi removido.~~

`Number.parseFloat()`

O valor é o mesmo que `parseFloat` do objeto global.

`Number.parseInt()`

O valor é o mesmo que `parseInt` do objeto global.

Instâncias <code>Number</code>

Toda instância `Number` herdam de `Number`. O objeto 'prototype' do construtor `Number` pode ser modificado para afetar todas as instâncias `Number`.

Métodos

Exemplos

Exemplo: Usando o objeto <code>Number</code> para atribuir valores a variáveis numéricas

O seguinte exemplo usa as propriedades do objeto `Number` para atribuir valores a várias variáveis numéricas:

```
var biggestNum = Number.MAX_VALUE;
var smallestNum = Number.MIN_VALUE;
var infiniteNum = Number.POSITIVE_INFINITY;
var negInfiniteNum = Number.NEGATIVE_INFINITY;
var notANum = Number.NaN;
```

Exemplo: Intervalo inteiro para <code>Number</code>

O seguinte exemplo mostra os valores inteiros mínimo e máximo que podem ser representados como objeto `Number` (para mais detalhes, referir-se ao padrão EcmaScript standard (EcmaScript standard), capítulo *8.5 O tipo de número (The Number Type*):

```
var maxInt = 9007199254740992;
var minInt = -9007199254740992;
```

Ao analisar dados que foram serializados para JSON, valores inteiros que caem fora desse intervalo podem ser corrompidos quando o analisador JSON os converte ao tipo `Number`. Usando `String` em vez disso é uma possível alternativa para se evitar um resultado indesejado.

Exemplo: Usando <code>Number para converter um objeto Date</code>

O exemplo a seguir converte o objeto `Date` para um valor numérico usando `Number como uma função`:

```
var d = new Date("December 17, 1995 03:24:00");
print(Number(d));
```

Isto resulta em "819199440000".

Converte 'string' numérica em números

```
Number("123"); // 123
Number("12.3"); // 12.3
Number(""); // 0
Number("0x11"); // 17
```

```
Number("0b11"); // 3
Number("0o11"); // 9
Number("foo"); // NaN
Number("100a"); // NaN
```

Veja também

- `NaN`
- O objeto global `Math`

Number.EPSILON

A propriedade **`Number.EPSILON`** representa a diferença entre 1 e o menor ponto flutuante maior que 1.

Você não tem que criar um objeto `Number` para acessar esta propriedade estática (use `Number.EPSILON`).

Experimente

```
const result = Math.abs(0.2 - 0.3 + 0.1);

console.log(result);
// Expected output: 2.7755575615628914e-17

console.log(result < Number.EPSILON);
// Expected output: true
```

Property attributes of `Number.EPSILON`	
Writable	no
Enumerable	no
Configurable	no

Descrição

A propriedade `EPSILON` tem o valor de aproximadamente `2.220446049250313080847263336181 6E-16`, ou `2-52`.

Polyfill

```
if (Number.EPSILON === undefined) {
  Number.EPSILON = Math.pow(2, -52);
}
```

Exemplos

Testando igualdade

```
x = 0.2;
y = 0.3;
z = 0.1;
equal = Math.abs(x - y + z) < Number.EPSILON;
```

Veja também

- O objeto `Number` que esta propriedade pertence.

Number.isFinite()

O método **`Number.isFinite()`** determina se o valor passado é um número finito.

Sintaxe

```
Number.isFinite(valor)
```

Parâmetros

`valor`

O valor a ser testado.

Retorno

Um `Boolean` indicando se o valor passado é ou não um número finito.

Descrição

Em comparação com a função global `isFinite()`, esse método não força a conversão do parâmetro para número. Isso significa que só valores do tipo número, que são também finitos, retornam `true`.

Exemplos

```
Number.isFinite(Infinity); // false
Number.isFinite(NaN); // false
```

```
Number.isFinite(-Infinity); // false

Number.isFinite(0); // true
Number.isFinite(2e64); // true

Number.isFinite("0"); // false, teria sido true com a função
// global isFinite('0')
Number.isFinite(null); // false, teria sido true com a função
// global isFinite(null)
```

Polyfill

```
Number.isFinite =
  Number.isFinite ||
  function (value) {
    return typeof value === "number" && isFinite(value);
  };
```

Veja também

- O objeto `Number` ao qual pertence.

Number.isInteger()

O método **`Number.isInteger()`** determina se o valor passado é um inteiro.

Experimente

```
function fits(x, y) {
  if (Number.isInteger(y / x)) {
    return "Fits!";
  }
  return "Does NOT fit!";
}

console.log(fits(5, 10));
// Expected output: "Fits!"

console.log(fits(5, 11));
// Expected output: "Does NOT fit!"
```

Sintaxe

```
Number.isInteger(value)
```

Parâmetros

`value`

O valor a testar se é um inteiro.

Valor retornado

Um `Boolean` indicando se o valor é inteiro ou não.

Descrição

Se o alvo for um inteiro, retorna `true`, senão retorna `false`. Se o valor é `NaN` ou infinito, retorna `false`.

Exemplos

```
Number.isInteger(0); // true
Number.isInteger(1); // true
Number.isInteger(-100000); // true

Number.isInteger(0.1); // false
Number.isInteger(Math.PI); // false

Number.isInteger(Infinity); // false
Number.isInteger(-Infinity); // false
Number.isInteger("10"); // false
Number.isInteger(true); // false
Number.isInteger(false); // false
Number.isInteger([1]); // false
```

Polyfill

```
Number.isInteger =
  Number.isInteger ||
  function (value) {
    return (
      typeof value === "number" &&
      isFinite(value) &&
      Math.floor(value) === value
```

```
    );
  };
```

Ver também

- O objeto `Number` a qual este método pertence.

Number.isNaN()

Resumo

O método **`Number.isNaN()`** determina se o valor passado é `NaN`. Versão mais robusta do original global `isNaN`.

Sintaxe

```
Number.isNaN(testValue)
```

Parâmetros

`testValue`

O valor a ser testado por `NaN`.

Descrição

Devido a ambos os operadores de igualdade, == and ===, avaliar a `false` quando está verificando se `NaN` *é* NaN, a função `Number.isNaN` se torna necessária. Esta situação é diferente de todas as outras comparações de valor possível em JavaScript.

Em comparação a função global `isNaN`, Number.isNaN não sofre do problema de forçar a conversão do parâmetro para um número. Isso significa que ele é seguro para passar valores que, normalmente, se convertem em NaN, mas na verdade não são o mesmo valor que NaN. Isto também significa que apenas os valores do número do tipo, que são também NaN, retorna `true`.

Exemplos

```
Number.isNaN(NaN); // true
Number.isNaN(Number.NaN); // true
Number.isNaN(0 / 0); // true

// everything else: false
Number.isNaN(undefined);
Number.isNaN({});

Number.isNaN(true);
Number.isNaN(null);
Number.isNaN(37);

Number.isNaN("37");
Number.isNaN("37.37");
Number.isNaN("");
Number.isNaN(" ");
Number.isNaN("NaN");
Number.isNaN("blabla"); // e.g. este teria sido true com isNaN
```

Veja também

- O `Number` objeto pertence a.

Number.isSafeInteger()

O método **`Number.isSafeInteger()`** determina se o valor fornecido é seja um número seguro.

Experimente

```
function warn(x) {
  if (Number.isSafeInteger(x)) {
    return "Precision safe.";
  }
  return "Precision may be lost!";
}

console.log(warn(Math.pow(2, 53)));
// Expected output: "Precision may be lost!"

console.log(warn(Math.pow(2, 53) - 1));
// Expected output: "Precision safe."
```

Um inteiro seguro é um inteiro que:

- pode ser representando exatamente como um número IEEE-754 de dupla precisão e
- pode ser representando exatamente como um número IEEE-754 de precisão dupla e
- cuja representação IEEE-754 não pode ser o resultado do arrendodamento de qualquer outro número inteiro para se ajustar ao IEEE-754.

Exemplo, `253 - 1` é um inteiro seguro: pode ser exatamente representado, e nenhum outro numero arredondado existe para ele na represetanção IEEE-754. Em contexto, `253` *não* é um inteiro seguro: pode ser representado em IEEE-754, mas um inteiro `253 + 1` não pode ser diretamente representado em IEEE-754 mas instanciado do arrendamento de `253` sob arrendamento para o mais próximo e do arrendamento de zero a zero. Os inteiros seguros consistem em todos os inteiros de `-(253 - 1)` inclusive para `253 - 1` (sendo ± `9007199254740991` ou ± 9,007,199,254,740,991).

A manipulação de valores entre ~9 quadrilhões com precisão total requer o uso de arbitrary precision arithmetic library (biblioteca aritmética de precisão arbitrária). Veja What Every Programmer Needs to Know about Floating Point Arithmetic (o que todo programador precisa saber sobre aritmética de ponto flutuante) para mais informações sobre represetanções de número de ponto flutuante.

Para números inteiros maiores, considere o uso do tipo `BigInt`.

Sintaxe

```
Number.isSafeInteger(valorTest)
```

Parâmetros

`valorTest`

O valor a ser testado pode ser um número inteiro seguro.

Retorno

Um `Boolean` indica se o valor fornecido é um número seguro ou não.

Exemplos

```
Number.isSafeInteger(3); // true
Number.isSafeInteger(Math.pow(2, 53)); // false
Number.isSafeInteger(Math.pow(2, 53) - 1); // true
Number.isSafeInteger(NaN); // false
Number.isSafeInteger(Infinity); // false
Number.isSafeInteger("3"); // false
Number.isSafeInteger(3.1); // false
Number.isSafeInteger(3.0); // true
```

Polyfill (caso não exista suporte)

```
Number.isSafeInteger =
  Number.isSafeInteger ||
  function (value) {
    return (
      Number.isInteger(value) && Math.abs(value) <= Number.MAX_SAFE_INTEGER
    );
  };
```

Veja também

- O objeto pertence a `Number`
- `Number.MIN_SAFE_INTEGER`
- `Number.MAX_SAFE_INTEGER`
- `BigInt`

Number.MAX_SAFE_INTEGER

A constante **`Number.MAX_SAFE_INTEGER`** representa o maior inteiro seguro no JavaScript (`253 - 1`).

Para inteiros maiores, considere usar `BigInt`.

Experimente

```
const x = Number.MAX_SAFE_INTEGER + 1;
const y = Number.MAX_SAFE_INTEGER + 2;

console.log(Number.MAX_SAFE_INTEGER);
// Expected output: 9007199254740991

console.log(x);
// Expected output: 9007199254740992

console.log(x === y);
// Expected output: true
```

Property attributes of `Number.MAX_SAFE_INTEGER`	
Writable	no
Enumerable	no
Configurable	no

Descrição

A constante `MAX_SAFE_INTEGER` tem o valor de `9007199254740991` (9,007,199,254,740,991 ou ~9 quadrilhões). A razão por trás deste número é que o JavaScript usa o formato de número de ponto-flutuante de precisão-dupla como especificado na IEEE 754 e pode seguramente representar número entre `-(253 - 1)` e `253 - 1`.

Seguro neste contexto se refere a habilidade de representar inteiros exatamente e corretamente compará-los. Por exemplo, `Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER + 2` será avaliado para verdadeiro, que é matematicamente incorreto. Veja `Number.isSafeInteger()` para mais informação.

Este campo não existe em navegadores antigos. Usando ele sem checar sua existência, como `Math.max(Number.MAX_SAFE_INTEGER, 2)`, irá gerar resultados indesejados como NaN.

Por `MAX_SAFE_INTEGER` ser uma propriedade estática de `Number`, você sempre deve usar como `Number.MAX_SAFE_INTEGER`, ao invés de uma propriedade do objeto `Number` que você criou.

Polyfill

```
if (!Number.MAX_SAFE_INTEGER) {
  Number.MAX_SAFE_INTEGER = 9007199254740991; // Math.pow(2, 53) - 1;
}
```

Exemplos

Retorno do valor de MAX_SAFE_INTEGER

```
Number.MAX_SAFE_INTEGER; // 9007199254740991
```

Números maiores que o inteiro seguro

Isso retorna 2 por quê em pontos flutuantes, o valor é na verdade o final decimal "1" exceto em casos subnormais de precisão como zero.

```
Number.MAX_SAFE_INTEGER * Number.EPSILON; // 2
```

Veja também

- `Number.MIN_SAFE_INTEGER`
- `Number.isSafeInteger()`
- `BigInt`

Number.MAX_VALUE

A propriedade **`Number.MAX_VALUE`** representa o maior valor numérico representável em JavaScript.

Experimente

```
function multiply(x, y) {
  if (x * y > Number.MAX_VALUE) {
    return "Process as Infinity";
  }
  return x * y;
}

console.log(multiply(1.7976931348623157e308, 1));
// Expected output: 1.7976931348623157e+308

console.log(multiply(1.7976931348623157e308, 2));
// Expected output: "Process as Infinity"
```

Property attributes of `Number.MAX_VALUE`	
Writable	no
Enumerable	no
Configurable	no

Descrição

A propriedade `MAX_VALUE` tem o valor de aproximadamente `1.79E+308`, ou 21024. Valores maiores que `MAX_VALUE` são representados como `Infinity`.

Por `MAX_VALUE` ser uma propriedade estática de `Number`, você sempre deve usar como `Number.MAX_VALUE`, ao invés de uma propriedade do objeto `Number` que você criou.

Exemplos

Usando <code>MAX_VALUE</code>

O código a seguir multiplica dois valores numéricos. Se o resultado é menor ou igual a `MAX_VALUE`, a função `func1` é chamada; caso contrário, a função `func2` é chamada.

```
if (num1 * num2 <= Number.MAX_VALUE) {
  func1();
} else {
  func2();
}
```

Veja também

- `Number.MIN_VALUE`
- O objeto `Number` a qual ela pertence

Number.MIN_SAFE_INTEGER

A constante **`Number.MIN_SAFE_INTEGER`** representa o menor inteiro seguro no JavaScript (`-(253 - 1)`).

Para representar inteiros menores do que isso, considere usar `BigInt`.

Experimente

```
const x = Number.MIN_SAFE_INTEGER - 1;
const y = Number.MIN_SAFE_INTEGER - 2;

console.log(Number.MIN_SAFE_INTEGER);
// Expected output: -9007199254740991

console.log(x);
// Expected output: -9007199254740992

console.log(x === y);
// Expected output: true
```

Property attributes of `Number.MIN_SAFE_INTEGER`	
Writable	no
Enumerable	no
Configurable	no

Descrição

A constante `MIN_SAFE_INTEGER` tem o valor de `-9007199254740991` (-9,007,199,254,740,991 ou -9 quadrilhões). A razão por trás deste número é que o JavaScript usa o formato de número de ponto-flutuante de precisão-dupla como especificado na IEEE 754 e pode seguramente representar número entre `-(253 - 1)` e `253 - 1`. Veja `Number.isSafeInteger()` para mais informações.

Por `MIN_SAFE_INTEGER` ser uma propriedade estática de `Number`, você sempre deve usar como `Number.MIN_SAFE_INTEGER`, ao invés de uma propriedade do objeto `Number` que você criou.

Exemplos

Usando MIN_SAFE_INTEGER

```
Number.MIN_SAFE_INTEGER; // -9007199254740991
-(Math.pow(2, 53) - 1); // -9007199254740991
```

Veja também

- `Number.MAX_SAFE_INTEGER`
- `Number.isSafeInteger()`
- `BigInt`

Number.MIN_VALUE

A propriedade **`Number.MIN_VALUE`** representa o menor valor positivo numérico representável em JavaScript.

Experimente

```
function divide(x, y) {
  if (x / y < Number.MIN_VALUE) {
    return "Process as 0";
  }
  return x / y;
}

console.log(divide(5e-324, 1));
// Expected output: 5e-324

console.log(divide(5e-324, 2));
// Expected output: "Process as 0"
```

Property attributes of `Number.MIN_VALUE`	
Writable	no
Enumerable	no
Configurable	no

Descrição

A propriedade `MIN_VALUE` é o número mais próximo de 0, não o número mais negativo, que o JavaScript pode representar.

`MIN_VALUE` tem o valor de aproximadamente `5e-324`. Valores menores que `MIN_VALUE` ("valores de underflow") são convertidos para 0.

Por `MIN_VALUE` ser uma propriedade estática de `Number`, você sempre deve usar como `Number.MIN_VALUE`, ao invés de uma propriedade do objeto `Number` que você criou.

Exemplos

Usando <code>MIN_VALUE</code>

O seguinte código divide dois valores numéricos. Se o resultado é maior ou igual a `MIN_VALUE`, a função `func1` é chamada; caso contrário, a função `func2` é chamada.

```
if (num1 / num2 >= Number.MIN_VALUE) {
  func1();
} else {
  func2();
}
```

Veja também

- `Number.MAX_VALUE`

Number.NaN

A propriedade **`Number.NaN`** representa Not-A-Number (Não-Número). Equivalente a `NaN`.

Você não precisa criar um objeto `Number` para acessar esta propriedade estática (use `Number.NaN`).

Property attributes of `Number.NaN`	
Writable	no
Enumerable	no
Configurable	no

Veja também

- O objeto global `NaN`.
- O objeto `Number` ao qual ele pertence.

Number.NEGATIVE_INFINITY

A propriedade **`Number.NEGATIVE_INFINITY`** representa o valor Infinito negativo.

Experimente

```
function checkNumber(smallNumber) {
  if (smallNumber === Number.NEGATIVE_INFINITY) {
    return "Process number as -Infinity";
  }
  return smallNumber;
}

console.log(checkNumber(-Number.MAX_VALUE));
// Expected output: -1.7976931348623157e+308

console.log(checkNumber(-Number.MAX_VALUE * 2));
// Expected output: "Process number as -Infinity"
```

Property attributes of `Number.NEGATIVE_INFINITY`	
Writable	no
Enumerable	no
Configurable	no

Descrição

O valor de `Number.NEGATIVE_INFINITY` é o mesmo que o valor negativo da propriedade do objeto global `Infinity`.

O valor se comporta um pouco diferente do que o infinito matemático:

- Qualquer valor positivo, incluindo `POSITIVE_INFINITY`, mutiplicado por `NEGATIVE_INFINITY` é `NEGATIVE_INFINITY`.
- Qualquer valor negativo, incluindo `NEGATIVE_INFINITY`, multiplicado por `NEGATIVE_INFINITY` é `POSITIVE_INFINITY`.
- Qualquer valor positivo dividido por `NEGATIVE_INFINITY` é zero negativo.
- Qualquer valor negativo dividido por `NEGATIVE_INFINITY` é zero positivo.
- Zero multiplicado por `NEGATIVE_INFINITY` é `NaN`.
- `NaN` multiplicado por `NEGATIVE_INFINITY` é `NaN`.
- `NEGATIVE_INFINITY`, dividido por qualquer valor negativo exceto `NEGATIVE_INFINITY`, é `POSITIVE_INFINITY`.
- `NEGATIVE_INFINITY`, divido por qualquer valor positivo exceto `POSITIVE_INFINITY`, é `NEGATIVE_INFINITY`.
- `NEGATIVE_INFINITY`, divido por ou `NEGATIVE_INFINITY` ou `POSITIVE_INFINITY`, é `NaN`.

Você pode usar a propriedade `Number.NEGATIVE_INFINITY` para indicar uma condição de erro que retorna um número finito em caso de sucesso. Nota que, usar `isFinite` seria mais apropriado neste caso.

Por `NEGATIVE_INFINITY` ser uma propriedade estática de `Number`, você sempre a usa como `Number.NEGATIVE_INFINITY`, ao invés de ser uma propriedade do objeto `Number` que você criou.

Exemplos

Usando <code>NEGATIVE_INFINITY</code>

No seguinte exemplo, a variável `smallNumber` é atribuída um valor que é menor que o valor mínimo. Quando o `if` executa, `smallNumber` tem o valor `-Infinity`, então é colocado em `smallNumber` um valor mais manejável antes de continuar.

```
var smallNumber = -Number.MAX_VALUE * 2;

if (smallNumber === Number.NEGATIVE_INFINITY) {
  smallNumber = returnFinite();
}
```

Veja também

- `Number.POSITIVE_INFINITY`
- `Number.isFinite()`
- `Infinity`
- `isFinite()`

Number.parseFloat()

Resumo

O método **Number.parseFloat()** converte a string recebida como argumento e a retorna como um número de ponto flutuante. Este método se comporta de maneira idêntica a da função global `parseFloat` e é parte da especificação ECMAScript 6 (seu propósito é a modularização dos objetos globais).

Sintaxe

```
Number.parseFloat(string)
```

Parâmetros

`string`

Uma string que represente o valor que se deseja converter.

Retorno

`string`

Um número de ponto flutuante a partir da string dada. Se a string não puder ser convertida em para um número, `NaN` é retornado.

Descrição

Este método tem a mesma funcionalidade do método global: `parseFloat()`

```
Number.parseFloat === parseFloat; // true
```

Por favor veja `parseFloat()` para mais detalhes e exemplos.

Veja também

- O objeto `Number` ao qual o método parseFloat pertence.
- O método global `parseFloat()`.

Number.parseInt()

O método **`Number.parseInt()`** converte um argumento de *string* e retorna um inteiro da raiz ou base específica.

Experimente

```
function roughScale(x, base) {
  const parsed = Number.parseInt(x, base);
  if (Number.isNaN(parsed)) {
    return 0;
  }
  return parsed * 100;
}

console.log(roughScale(" 0xF", 16));
// Expected output: 1500

console.log(roughScale("321", 2));
// Expected output: 0
```

Sintaxe

```
Number.parseInt(string,[ radix])
```

Parâmetros

`string`

O valor a ser convertido. Se este argumento não for uma *string*, então ele é convertido a um usando a operação abstrata `ToString`. O espaço em branco inicial neste argumento é ignorado.

`radix_` Optional_

Um inteiro entre `2` e `36` que representa a *raiz* (a base no sistema numérico matemático) de uma `string`. Tome cuidado—o padrão ***não*** é `10`!

Valor de retorno

Um inteiro convertido de uma dada `string`.

Se a `radix` é menor que `2` ou maior que `36`, e o primeiro caracter que não é um espaço em branco não puder ser convertido para um número, `NaN` é retornado.

Polyfill

```
if (Number.parseInt === undefined) {
  Number.parseInt = window.parseInt;
}
```

Exemplos

Number.parseInt vs parseInt

Este método tem a mesma funcionalidade que o método global `parseInt()`:

```
Number.parseInt === parseInt; // true
```

e é parte do ECMAScript 2015 (sua proposta é a modularização dos globais). Por favor veja `parseInt()` para mais detalhes e exemplos.

Veja também

- O objeto `Number` a qual ela pertence.
- O método global `parseInt()`.

Number.POSITIVE_INFINITY

A propriedade **`Number.POSITIVE_INFINITY`** representa o valor positivo infinito.

Você não precisa criar um objeto `Number` para utilizar a propriedade estática (use `Number.POSITIVE_INFINITY`).

Property attributes of `Number.POSITIVE_INFINITY`	
Writable	no
Enumerable	no
Configurable	no

Descrição

O valor de `Number.POSITIVE_INFINITY` é o mesmo valor da propriedade `Infinity` do objeto global.

Esse valor se comporta ligeiramente diferente do infinito matemático:

- Qualquer valor positivo, incluindo `POSITIVE_INFINITY`, multiplicado por `POSITIVE_INFINITY` é `POSITIVE_INFINITY`.
- Qualquer valor negativo, incluindo, `NEGATIVE_INFINITY`, multiplicado por `POSITIVE_INFINITY` é `NEGATIVE_INFINITY`.
- Qualquer número positivo dividido por `POSITIVE_INFINITY` é Zero positivo.
- Qualquer número negativo dividido por `POSITIVE_INFINITY` é Zero negativo.
- Zero multiplicado por `POSITIVE_INFINITY` é `NaN`.
- `NaN` multiplicado por `POSITIVE_INFINITY` é `NaN`.
- `POSITIVE_INFINITY`, dividido por qualquer valor negativo exceto `NEGATIVE_INFINITY`, é `NEGATIVE_INFINITY`.
- `POSITIVE_INFINITY`, dividido por qualquer valor positivo exceto `POSITIVE_INFINITY`, é `POSITIVE_INFINITY`.
- `POSITIVE_INFINITY`, dividido por `NEGATIVE_INFINITY` ou `POSITIVE_INFINITY`, é `NaN`.

Você pode usar a propriedade `Number.POSITIVE_INFINITY` para indicar uma condição de erro que retorna um número finito no caso de sucesso. Sobretudo, `isFinite` seria mais apropriado nesse caso.

Exemplos

Usando <code>POSITIVE_INFINITY</code>

No exemplo a seguir, a variável `bigNumber` recebe um valor maior que o valor máximo. Quando as declarações `if` executam, `bigNumber` tem o valor `Infinity`, então `bigNumber` recebe um valor mais gerenciável antes de continuar.

```
var bigNumber = Number.MAX_VALUE * 2;

if (bigNumber == Number.POSITIVE_INFINITY) {
  bigNumber = returnFinite();
}
```

Ver também

- `Number.NEGATIVE_INFINITY`
- `Number.isFinite()`
- `Infinity`
- `isFinite()`

Number.prototype.toFixed()

O método **`toFixed()`** formata um número utilizando notação de ponto fixo.

Sintaxe

```
numObj.toFixed([dígitos])
```

Parâmetros

`dígitos`

> Opcional. O número de dígitos que aparecem depois do ponto decimal; este pode ser um valor entre 0 e 20, inclusive, e algumas implementacões podem suportar uma variação de números maiores. Se este argumento for omitido, será tratado como 0.

Retorno

Uma string representando o número usando notação em ponto fixo.

Throws

`RangeError`

> Se `dígitos` for muito pequeno ou muito grande. Valores entre 0 e 20, inclusive, não irão causar o `RangeError`. É permitido às implementações suportar valores maiores e menores.

`TypeError`

> Se este método for chamado em um objeto que não é `Number`.

Descrição

Uma string representando `numObj` que não usa notação exponencial e tem exatamente `dígitos` dígitos depois da casa decimal. O número será arredondado se necessário, e será adicionado zeros a parte após a virgula para que este tenha o tamanho que foi especificado. Se o `numObj` for maior que `1e+21`, entao será invocado o método `Number.prototype.toString()` e será retornado uma string em notação exponencial.

Exemplos

Utilizando <code>toFixed</code>

```
var numObj = 12345.6789;

numObj.toFixed(); // Retorna '12346': note o arredondamento, não possui nenhuma parte fracionária
numObj.toFixed(1); // Retorna '12345.7': note o arredondamento
numObj.toFixed(6); // Retorna '12345.678900': note que adicionou zeros
(1.23e20).toFixed(2); // Retorna '123000000000000000000.00'
(1.23e-10).toFixed(2); // Retorna '0.00'
(2.34).toFixed(1); // Retorna '2.3'
(2.35).toFixed(1); // Retorna '2.4'. Note que arredonda para cima neste caso.
-(2.34).toFixed(1); // Retorna -2.3 (devido à precedência do operador, literais de números negativos não retornam uma string...)
(-2.34).toFixed(1); // Retorna '-2.3' (...a menos que se utilize parênteses)
```

Veja também

- `Number.prototype.toExponential()`
- `Number.prototype.toPrecision()`
- `Number.prototype.toString()`

Number.prototype.toLocaleString()

O método **`toLocaleString()`** retorna uma string com uma representação sensível a linguagem deste número.

Os novos argumentos `locales` e `options` permitem às aplicações especificar a linguagem cujas convenções de formatações serão utilizadas e personalizar o comportamento da função. Nas implementações anteriores, que ignorava os argumentos `locales` e `options` arguments, a localização utilizada e a forma de retornar a string erão totalmente dependente da implementação.

Sintaxe

```
numObj.toLocaleString([locales [, options]])
```

Parâmetros

Dê uma olhada na seção Compatibilidade do Navegador para verificar quais navegadores suportam os argumentos `locales` e `options`, e o Exemplo: Verificando o suporte dos argumentos `locales` e `options` para detecção desta característica.

> **Nota:** ECMAScript Internationalization API, implementada com o Firefox 29, incluiu o argumento `locales` ao método `Number.toLocaleString()`. Se o argumento for `undefined`, este método retorna os dígitos de localização especificados pelo SO, enquanto que as versões anteriores doFirefox retornavam os dígitos Árabe Ocidental. Esta mudança foi relatada como uma regressão que afeta a retrocompatibilidade que será corrigida em breve. (Erro do Firefox 999003)

Exemplos

Usando <code>toLocaleString</code>

No uso básico sem a especificação de uma localização, o método retornará uma string formatada com a localização e as opções padrão.

```
var numero = 3500;

console.log(numero.toLocaleString()); // Mostra "3,500" se a localização for U.S. English
```

Verificando o suporte dos argumentos <code>locales</code> e <code>options</code>

Os argumentos `locales` e `options` não são suportados por todos os navegadores ainda. Para verificar pelo suporte das implementações do ES5.1 e posteriores, a requisição de tags de linguagem ilegais são rejeitadas com uma exceção `RangeError` pode ser usada da seguinte forma:

```
function toLocaleStringSupportsLocales() {
  var numero = 0;
  try {
    numero.toLocaleString("i");
  } catch (e) {
    return e.name === "RangeError";
  }
  return false;
}
```

Antes da ES5.1, implementações que não exigiam um tratamento de erro se `toLocaleString` fosse chamada com argumentos.

Uma verificação que funciona em todos os casos, incluindo aqueles que suportam ECMA-262 antes da edição 5.1, é testar pelas especificações de característicadas da ECMA-402 que exigem suporte de opções regionais para `Number.prototype.toLocaleString` diretamente:

```
function toLocaleStringSupportsOptions() {
  return !!(
    typeof Intl == "object" &&
    Intl &&
    typeof Intl.NumberFormat == "function"
  );
}
```

Estes testes para um objeto `Intl` global, verifica se ele não é `null` e se uma propriedade `NumberFormat` é uma função.

Usando <code>locales</code>

Este exemplo mostra algumas variações de formatos de números localizados. A fim de obter o formato da linguagem utilizada na interface do usuário da sua aplicação, tenha certeza de especificar a língua (e possivelmente algumas línguas reservas) usando o argumento `locales`:

```
var numero = 123456.789;

// O alemão usa vírgula como separador de decimal e ponto para milhares
console.log(numero.toLocaleString("de-DE"));
// → 123.456,789

// O árabe usa dígitos Árabes Orientais em muitos países que falam árabe
console.log(numero.toLocaleString("ar-EG"));
// → ١٢٣٤٥٦,٧٨٩

// A Índia usa separadores de milhares/cem mil/dez milhões
console.log(numero.toLocaleString("en-IN"));
// → 1,23,456.789

// A chave de extensão nu requer um sistema de numeração, ex. decimal chinês
console.log(numero.toLocaleString("zh-Hans-CN-u-nu-hanidec"));
// → 一二三,四五六.七八九

// Quando informada uma língua sem suporte, como balinês,
// inclua uma língua reseva, neste caso indonésio
console.log(numero.toLocaleString(["ban", "id"]));
// → 123.456,789
```

Usando <code>options</code>

Os resultados obtidos por `toLocaleString` pode ser personalizado usando o argumento `options`:

```
var numero = 123456.789;

// informando um formato de moeda
console.log(
  numero.toLocaleString("de-DE", { style: "currency", currency: "EUR" }),
```

```
);
// → 123.456,79 €

// o yen japonês não tem uma unidade menor
console.log(
  numero.toLocaleString("ja-JP", { style: "currency", currency: "JPY" }),
);
// → ￥123,457

// limitando a três dígitos significativos
console.log(numero.toLocaleString("en-IN", { maximumSignificantDigits: 3 }));
// → 1,23,000
```

Desempenho

Quando formatar uma grande quantidade de números, é melhor criar um objeto `NumberFormat` e usar a função fornecida pela propriedade `NumberFormat.format`.

Veja também

- `Number.prototype.toString()`

Number.prototype.toPrecision()

O método **`toPrecision()`** retorna uma string que representa o valor do objeto `Number` com uma precisão específica.

Experimente

```
function precise(x) {
  return x.toPrecision(4);
}

console.log(precise(123.456));
// Expected output: "123.5"

console.log(precise(0.004));
// Expected output: "0.004000"

console.log(precise(1.23e5));
// Expected output: "1.230e+5"
```

Sintaxe

```
numObj.toPrecision([precisão])
```

Parâmetros

`precisão`

> Opcional. Um inteiro especificando o número de algarismos significativos.

Retorno

Uma string representando um objeto `Number` em notação de ponto fixo ou exponencial arredondada segundo o parâmetro `precisão`. Veja a discussão sobre arredondamento feita na documentação do método `Number.prototype.toFixed()`, que também se aplica ao método `toPrecision()`.

Se o parâmetro `precisão` for omitido, este método terá o mesmo comportamento de `Number.prototype.toString()`. Se o parâmetro `precisão` for um valor não inteiro, ele será arredondado para a sua representação mais próxima em inteiro.

Exceções

`RangeError`

> Se o valor de `precisão` não estiver compreendido entre 1 e 100 (inclusive), um `RangeError` será lançado. É permitido às implementações suportar valores menores e maiores que esses, sendo um requisito do ECMA-262 que seja dado suporte a uma precisão de até 21 algarismos significativos.

Exemplos

Utilizando <code>toPrecision</code>

```
var numObj = 5.123456;

console.log(numObj.toPrecision()); // logs '5.123456'
console.log(numObj.toPrecision(5)); // logs '5.1235'
console.log(numObj.toPrecision(2)); // logs '5.1'
console.log(numObj.toPrecision(1)); // logs '5'

numObj = 0.000123;

console.log(numObj.toPrecision()); // logs '0.000123'
console.log(numObj.toPrecision(5)); // logs '0.00012300'
console.log(numObj.toPrecision(2)); // logs '0.00012'
console.log(numObj.toPrecision(1)); // logs '0.0001'
```

```
// observe que a notação exponencial pode ser retornado em alguns casos
console.log((1234.5).toPrecision(2)); // logs '1.2e+3'
```

Veja também

- `Number.prototype.toFixed()`
- `Number.prototype.toExponential()`
- `Number.prototype.toString()`

Number.prototype.toString()

Resumo

O método **`toString()`** retorna uma string representando o objeto `Number` especificado.

Sintaxe

```
numObj.toString([radix])
```

Parâmetros

`radix`

Opcional. Um inteiro entre 2 e 36 especificando a base utilizada para representar os valores numéricos.

Exceções

`RangeError`

se `toString()` receber um valor de radix fora do intervalo entre 2 e 36, uma exceção `RangeError` é lançada.

Descrição

O objeto `Number` sobrescreve o método `toString()` do objeto `Object`; ele não herda de `Object.prototype.toString()`. Para objetos `Number`, o método `toString()` retorna uma representação string do objeto na base especificada.

O método `toString()` analisa seu primeiro argumento e tenta retornar uma representação string na raiz (base) especificada. Para raizes maiores que 10, as letras do alfabeto indicam valores maiores que 9. Por exemplo, para números hexadecimais (base 16), letras entre `a` e `f` são utilizadas.

Se o `radix` não for especificado, a raiz assumida como preferencial é a 10.

Se o `numObj` for negativo, o sinal é preservado. Isto acontece mesmo se a raiz for 2; a string retornada é a representação binária positiva de `numObj` precedida por um sinal `-` e **não** o complemento de dois do `numObj`.

Exemplos

Exemplo: Usando <code>toString</code>

```
var count = 10;

console.log(count.toString()); // displays '10'
console.log((17).toString()); // displays '17'

var x = 6;

console.log(x.toString(2)); // displays '110'
console.log((254).toString(16)); // displays 'fe'

console.log((-10).toString(2)); // displays '-1010'
console.log((-0xff).toString(2)); // displays '-11111111'
```

Veja também

- `Number.prototype.toFixed()`
- `Number.prototype.toExponential()`
- `Number.prototype.toPrecision()`

Number.prototype.valueOf()

O método **`valueOf()`** retorna o valor primitivo contido no objeto `Number`.

Experimente

```
const numObj = new Number(42);
console.log(typeof numObj);
// Expected output: "object"

const num = numObj.valueOf();
console.log(num);
// Expected output: 42

console.log(typeof num);
// Expected output: "number"
```

Sintaxe

```
numObj.valueOf()
```

Valor retornado

Um número representando o valor primitivo do objeto `Number`.

Descrição

Este método normalmente é invocado internamente pelo JavaScript e não explicitamente em um código web.

Exemplos

Utilizando <code>valueOf</code>

```
var numObj = new Number(10);
console.log(typeof numObj); // object

var num = numObj.valueOf();
console.log(num); // 10
console.log(typeof num); // number
```

See also

- `Number.prototype.toSource()`

Object

O construtor `Object` cria um wrapper de objeto.

Sintaxe

```
// Object initialiser or literal
{ [ nameValuePair1[, nameValuePair2[, ...nameValuePairN] ] ] }

// Called as a constructor
new Object([value])
```

Parametros

`nameValuePair1, nameValuePair2, ... nameValuePairN`

Pares de nomes (strings) e valores (qualquer valor) em que o nome é separado do valor por dois pontos.

`value`

Qualquer valor.

Descrição

O construtor `Object` cria um wrapper de objeto para o valor fornecido. Se o valor for `null` ou `undefined`, ele criará e retornará um objeto vazio, caso contrário, retornará um objeto de um Type que corresponde ao dado valor. Se o valor já for um objeto, ele retornará o valor.

Quando chamado em um contexto não-construtor, o `object` se comporta de forma idêntica ao `new Object ()`.

Veja também object initializer / literal syntax.

Propriedades do construtor <code>Object</code>

`Object.length`

Tem um valor de 1.

`Object`

Permite a adição de propriedades a todos os objetos do tipo Object.

Métodos do construtor <code>Object</code>

`Object.assign()`

Copia os valores de todas as propriedades próprias enumeráveis de um ou mais objetos de origem para um objeto de destino.

`Object.create()`

Cria um novo objeto com o objeto e as propriedades do protótipo especificado.

`Object.defineProperty()`

Adiciona a propriedade nomeada descrita por um determinado descritor a um objeto.

`Object.defineProperties()`

Adiciona as propriedades nomeadas descritas pelos descritores fornecidos a um objeto.

`Object.entries()`

Retorna uma matriz contendo todos os pares `[key, value]` das próprias propriedades de string enumeráveis de um determinado objeto.

`Object.freeze()`

Congela um objeto: outro código não pode excluir ou alterar nenhuma propriedade.

`Object.fromEntries()`

Retorna um novo objeto de um iterável de pares de valor-chave (reverso à `Object.entries`).

`Object.getOwnPropertyDescriptor()`

Retorna um descritor de propriedade para uma propriedade nomeada em um objeto.

`Object.getOwnPropertyDescriptors()`

Retorna um objeto contendo todos os descritores de propriedade para um objeto.

`Object.getOwnPropertyNames()`

Retorna uma matriz contendo os nomes de todas as propriedades enumeráveis e não enumeráveis do próprio objeto fornecido.

`Object.getOwnPropertySymbols()`

Retorna uma matriz de todas as propriedades de símbolo encontradas diretamente sobre um determinado objeto.

`Object.getPrototypeOf()`

Retorna o protótipo do objeto especificado.

`Object.is()`

Compara se dois valores são o mesmo valor. Equivale a todos os valores de `NaN` (que diferem da Comparação de Igualdade Abstrata e da Comparação de Igualdade Estrita).

`Object.isExtensible()`

Determina se a extensão de um objeto é permitida.

`Object.isFrozen()`

Determina se um objeto foi congelado.

`Object.isSealed()`

Determina se um objeto está selado.

`Object.keys()`

Retorna uma matriz contendo os nomes de todas as propriedades de string enumeráveis do objeto fornecido.

`Object.preventExtensions()`

Impede qualquer extensão de um objeto.

`Object.seal()`

Impede que outro código exclua propriedades de um objeto.

`Object.setPrototypeOf()`

Define o protótipo (isto é, a propriedade interna `[[Prototype]]`).

`Object.values()`

Retorna uma matriz contendo os valores que correspondem a todas as propriedades de string enumeráveis do próprio objeto.

Instâncias de <code>Object</code> e <code>Object</code> de protótipo de objeto

Todos os objetos em JavaScript são descendentes do `Object`; todos os objetos herdam métodos e propriedades de `Object`, embora eles possam ser substituídos. Por exemplo, protótipos de outros construtores substituem a propriedade `constructor` e fornecer seus próprios métodos `toString()`. As alterações no objeto de protótipo `Object` são propagadas para todos os objetos, a menos que as propriedades e os métodos sujeitos a essas alterações sejam substituídos na cadeia de protótipos.

Propriedades (enUS)

Métodos (enUS)

Excluindo uma propriedade de um objeto

Não há nenhum método em um Objeto para excluir suas próprias propriedades (por exemplo, como `Map.prototype.delete()`). Para isso, é necessário usar o operador delete.

Exemplos

Usando <code>Object</code> com tipos <code>undefined</code> e <code>null</code>

Os exemplos a seguir armazenam um `Object` vazio na variável "o":

```
var o = new Object();
```

```
var o = new Object(undefined);
```

```
var o = new Object(null);
```

Using <code>Object</code> to create <code>Boolean</code> objects

Os exemplos a seguir armazenam objetos `Boolean` na variável "o":

```
// equivalent to o = new Boolean(true);
var o = new Object(true);
```

```
// equivalent to o = new Boolean(false);
var o = new Object(Boolean());
```

Veja também

- Object initializer

Object.prototype.__defineGetter__()

> **Aviso:** Esta característica está descontinuada em favor de definindo getters usando a sintaxe de inicialização de objetoou a API `Object.defineProperty()`. Enquanto esta característica é amplamente implementada, ela está somente descrita na especificação ECMAScript por causa do uso legado. Este método não deveria ser usado desde que exista existam melhores alternativas.

O método **`__defineGetter__`** conecta uma propriedade do objeto à uma função para ser chamada quando isso é consultado.

Sintaxe

```
obj.__defineGetter__(prop, func)
```

Parâmetros

`prop`

Uma string contendo o nome da propriedade para conectar a função dada.

`func`

Uma função para ser ligada a uma consulta da propriedade especificada.

Valor de Retorno

`undefined`.

Descrição

O `__defineGetter__` permite um getter ser definido sobre um objeto preexistente.

Exemplos

```
// Forma não-padrão e depreciada

var o = {};
o.__defineGetter__("gimmeFive", function () {
  return 5;
});
console.log(o.gimmeFive); // 5

// Formas padrão-compatível

// Usando o operador get
var o = {
  get gimmeFive() {
    return 5;
  },
};
console.log(o.gimmeFive); // 5

// Usando Object.defineProperty
var o = {};
Object.defineProperty(o, "gimmeFive", {
  get: function () {
    return 5;
```

```
  },
});
console.log(o.gimmeFive); // 5
```

Veja também

- `Object.prototype.__defineSetter__()`
- `get` operator
- `Object.defineProperty()`
- `Object.prototype.__lookupGetter__()`
- `Object.prototype.__lookupSetter__()`
- Guia JS: Definindo Getters e Setters
- [Blog Post] Depreciação de __defineGetter__ e __defineSetter__
- Erro do Firefox 647423

Object.prototype.__defineSetter__()

> **Aviso:** Esta funcionalidade está depreciada em favor da definição de setters usando a sintaxe de inicialização de objeto ou a API `Object.defineProperty()`.
>
> Entretando, como ele é largamente implementado e usado na Web, é bem improvável que os navegadores vão parar de implementá-lo.

O método **`__defineSetter__`** vincula uma propriedade de um objeto a uma função a ser chamada quando é feita uma tentativa de atribuir algo a aquela propriedade.

Sintaxe

```
obj.__defineSetter__(prop, fun)
```

Parâmetros

`prop`

Uma cadeia de caracteres (*string*) contendo o nome da propriedade que vai ser vinculada a função dada.

`fun`

A função a ser chamada quando houver uma tentativa de atribuir na propriedade especificada. Esta função toma a forma

```
function(val) { . . . }
```

`val`

Um apelido para a variável que contém o valor que se tentou atribuir a `prop`.

Valor de retorno

`undefined`.

Descrição

O método `__defineSetter__` permite um setter ser definido a um objeto pré-existente.

Exemplos

Não padronizados e forma depreciada

```
var o = {};
o.__defineSetter__("value", function (val) {
  this.anotherValue = val;
});
o.value = 5;
console.log(o.value); // undefined
console.log(o.anotherValue); // 5
```

Formas compatíveis padronizadas

```
// Usando o operador set
var o = {
  set value(val) {
    this.anotherValue = val;
  },
};
o.value = 5;
console.log(o.value); // undefined
console.log(o.anotherValue); // 5

// Usando Object.defineProperty
var o = {};
Object.defineProperty(o, "value", {
  set: function (val) {
    this.anotherValue = val;
  },
```

```
});
o.value = 5;
console.log(o.value); // undefined
console.log(o.anotherValue); // 5
```

Veja também

- `Object.prototype.__defineGetter__()`
- Operador `set`
- `Object.defineProperty()`
- `Object.prototype.__lookupGetter__()`
- `Object.prototype.__lookupSetter__()`
- JS Guide: Defining Getters and Setters
- [Blog Post] Deprecation of __defineGetter__ and __defineSetter__
- bug 647423

Object.prototype.__lookupGetter__()

O método **`__lookupGetter__`** retorna a função limite como uma getter para a específica propriedade.

Sintaxe

```
obj.__lookupGetter__(sprop)
```

Parâmetros

`sprop`

Uma sequência de caracteres contendo o nome da propriedade cuja getter deve retornar.

Valores de retorno

A função limite como uma getter para a específica propriedade.

Descrição

Se uma getter foi definida por uma propriedade de um objeto, não é possível referir-se a função getter através da propriedade, porque essa propriedade se refere ao retorno do valor daquela função. `__lookupGetter__` pode ser usado para obter referência à função getter.

Agora é possível fazer isso de um jeito uniforme usando:. `Object.getOwnPropertyDescriptor()` e `Object.getPrototypeOf()`.

Exemplos

```
var obj = {
  get foo() {
    return Math.random() > 0.5 ? "foo" : "bar";
  },
};

// Non-standard and deprecated way
obj.__lookupGetter__("foo");
// (function() { return Math.random() > 0.5 ? 'foo' : 'bar'; })

// Standard-compliant way
Object.getOwnPropertyDescriptor(obj, "foo").get;
// (function() { return Math.random() > 0.5 ? 'foo' : 'bar'; })
```

Ver também

- `Object.prototype.__lookupSetter__()`
- `get` operador
- `Object.getOwnPropertyDescriptor()` e `Object.getPrototypeOf()`
- `Object.prototype.__defineGetter__()`
- `Object.prototype.__defineSetter__()`
- JS Guide: Defining Getters and Setters

Object.prototype.__lookupSetter__()

O método **`__lookupSetter__`** retorna a função vinculada como *setter* para a propriedade especificada.

Sintaxe

```
obj.__lookupSetter__(sprop)
```

Parâmetros

`sprop`

Uma cadeia de caracteres (*string*) contendo o nome da propriedade a qual o *setter* deve ser retornado.

Valor de retorno

A função vinculada como *setter* para a propriedade especificada.

Descrição

Se o *setter* foi definido para uma propriedade do objeto, não era possível referenciar a função através da propriedade, porque aquela propriedade se refere ao valor de retorno da função. `__lookupSetter__` pode ser usado para obter a referência para a função *setter*.

É possível agora fazer isso de forma padronizada usando `Object.getOwnPropertyDescriptor()`.

Exemplos

Formas compatíveis e não padronizadas de obter um definidor de propriedades

```
var obj = {
  set foo(value) {
    this.bar = value;
  },
};

// Forma não padronizada e depreciada
obj.__lookupSetter__("foo");
// (function(value) { this.bar = value; })

// Forma compatível padronizada
Object.getOwnPropertyDescriptor(obj, "foo").set;
// (function(value) { this.bar = value; })
```

Veja também

- `Object.prototype.__lookupGetter__()`
- Operador `set`
- `Object.getOwnPropertyDescriptor()` e `Object.getPrototypeOf()`
- `Object.prototype.__defineGetter__()`
- `Object.prototype.__defineSetter__()`
- JS Guide: Defining Getters and Setters

Object.assign()

O método **`Object.assign()`** é usado para copiar os valores de todas as propriedades próprias enumeráveis de um ou mais objetos *de origem* para um objeto *destino*. Este método irá retornar o objeto *destino*.

Experimente

```
const target = { a: 1, b: 2 };
const source = { b: 4, c: 5 };

const returnedTarget = Object.assign(target, source);

console.log(target);
// Expected output: Object { a: 1, b: 4, c: 5 }

console.log(returnedTarget === target);
// Expected output: true
```

Sintaxe

```
Object.assign(destino, ...origens)
```

Parâmetros

`destino`

O objeto *destino*.

`origens`

Um ou mais objetos de *origem*.

Valor retornado

O objeto *destino* será retornado.

Descrição

O método `Object.assign()` copia apenas propriedades *enumeráveis* e *próprias* de um objeto *de origem* para um objeto destino. Ele usa `[[Get]]` na origem e `[[Put]]` no *destino*, então isto irá invocar *getters* e *setters*.

Portanto, ele *atribui* propriedades, em vez de simplesmente copiar ou definir novas propriedades. Isso pode fazê-lo impróprio para combinar novas propriedades com um *prototype* se os objetos *de origem* contiverem getters. Para copiar definições de propriedades, incluindo sua enumerabilidade, para *prototypes* `Object.getOwnPropertyDescriptor()` e `Object.defineProperty()` devem ser utilizadas no lugar.

Ambas as propriedades `String` e `Symbol` são copiadas.

No caso de erro, por exemplo, se uma propriedade não é *writable*, um `TypeError` será lançado e o objeto *destino* permanecerá inalterado. Note que `Object.assign()` não lança erros caso algum argumento *source* seja `null` ou `undefined`.

Exemplos

Clonando um objeto

```
var obj = { a: 1 };
var copy = Object.assign({}, obj);
console.log(copy); // { a: 1 }
```

Mesclando objetos

```
var o1 = { a: 1 };
var o2 = { b: 2 };
var o3 = { c: 3 };

var obj = Object.assign(o1, o2, o3);
console.log(obj); // { a: 1, b: 2, c: 3 }
console.log(o1); // { a: 1, b: 2, c: 3 }, target object itself is changed.
```

Copiando propriedades Symbol

```
var o1 = { a: 1 };
var o2 = { [Symbol("foo")]: 2 };

var obj = Object.assign({}, o1, o2);
console.log(obj); // { a: 1, [Symbol("foo")]: 2 }
```

Propriedades herdadas e não enumeráveis não podem ser copiadas

```
var obj = Object.create(
  { foo: 1 },
  {
    // foo is an inherit property.
    bar: {
      value: 2, // bar is a non-enumerable property.
    },
    baz: {
      value: 3,
      enumerable: true, // baz is an own enumerable property.
    },
  },
);

var copy = Object.assign({}, obj);
console.log(copy); // { baz: 3 }
```

Primitivas serão encapsuladas em objetos

```
var v1 = "123";
var v2 = true;
var v3 = 10;
var v4 = Symbol("foo");

var obj = Object.assign({}, v1, null, v2, undefined, v3, v4);
// Primitives will be wrapped, null and undefined will be ignored.
// Note, only string wrappers can have own enumerable properties.
console.log(obj); // { "0": "1", "1": "2", "2": "3" }
```

Exceções irão interromper a tarefa de cópia em execução

```
var target = Object.defineProperty({}, "foo", {
  value: 1,
  writeable: false,
}); // target.foo is a read-only property

Object.assign(target, { bar: 2 }, { foo2: 3, foo: 3, foo3: 3 }, { baz: 4 });
// TypeError: "foo" is read-only
// The Exception is thrown when assigning target.foo

console.log(target.bar); // 2, the first source was copied successfully.
console.log(target.foo2); // 3, the first property of the second source was copied successfully.
console.log(target.foo); // 1, exception is thrown here.
console.log(target.foo3); // undefined, assign method has finished, foo3 will not be copied.
console.log(target.baz); // undefined, the third source will not be copied either.
```

Copiando acessores

```
var obj = {
  foo: 1,
  get bar() {
    return 2;
```

```
  },
};

var copy = Object.assign({}, obj);
console.log(copy);
// { foo: 1, bar: 2 }, the value of copy.bar is obj.bar's getter's return value.

// This is an assign function which can copy accessors.
function myAssign(target, ...sources) {
  sources.forEach((source) => {
    Object.defineProperties(
      target,
      Object.keys(source).reduce((descriptors, key) => {
        descriptors[key] = Object.getOwnPropertyDescriptor(source, key);
        return descriptors;
      }, {}),
    );
  });
  return target;
}

var copy = myAssign({}, obj);
console.log(copy);
// { foo:1, get bar() { return 2 } }
```

Polyfill

Este polyfill não suporta propriedades `Symbol`, visto que ES5 não possui símbolos:

```
if (!Object.assign) {
  Object.defineProperty(Object, "assign", {
    enumerable: false,
    configurable: true,
    writable: true,
    value: function (target) {
      "use strict";
      if (target === undefined || target === null) {
        throw new TypeError("Cannot convert first argument to object");
      }

      var to = Object(target);
      for (var i = 1; i < arguments.length; i++) {
        var nextSource = arguments[i];
        if (nextSource === undefined || nextSource === null) {
          continue;
        }
        nextSource = Object(nextSource);

        var keysArray = Object.keys(Object(nextSource));
        for (
          var nextIndex = 0, len = keysArray.length;
          nextIndex < len;
          nextIndex++
        ) {
          var nextKey = keysArray[nextIndex];
          var desc = Object.getOwnPropertyDescriptor(nextSource, nextKey);
          if (desc !== undefined && desc.enumerable) {
            to[nextKey] = nextSource[nextKey];
          }
        }
      }
      return to;
    },
  });
}
```

Veja também

- `Object.defineProperties()`
- Enumerabilidade e posse de propriedades

Object.prototype.constructor

Sumário

Retorna uma referência para a função `Object` que cria a instância do protótipo. Note que o valor desse protótipo é uma referência para a própria função, não uma string contendo o nome da função. O valor é apenas *read-only* para valores primitivos como `1`, `true` e `"test"`.

Descrição

Todos os objetos herdam a propriedade *construtor* de seu protótipo:

```
var o = {};
o.constructor === Object; // true
```

```
var a = [];
a.constructor === Array; // true

var n = new Number(3);
n.constructor === Number; // true
```

Exemplos

Exemplo: Apresentando o construtor de um objeto

O exemplo a seguir cria um protótipo, `Tree`, e um objeto desse tipo, `theTree`. O exemplo, então, apresenta a propriedade *constructor* do objeto `theTree`.

```
function Tree(name) {
  this.name = name;
}

var theTree = new Tree("Redwood");
console.log("theTree.constructor is " + theTree.constructor);
```

Esse exemplo apresenta a seguinte saída:

```
theTree.constructor is function Tree(name) {
  this.name = name;
}
```

Exemplo: Mudando o construtor de um objeto

O exemplo a seguir apresenta como modificar o valor do construtor de um objeto genérico. Apenas `true`, `1` e `"test"` não serão afetados sendo que eles tem `construtores` *read-only* nativos. Esse exemplo apresenta que nem sempre é seguro depender da propriedade `constructor` de um objeto.

```
function Type() {}

var types = [
  new Array(),
  [],
  new Boolean(),
  true, // remains unchanged
  new Date(),
  new Error(),
  new Function(),
  function () {},
  Math,
  new Number(),
  1, // remains unchanged
  new Object(),
  {},
  new RegExp(),
  /(?:)/,
  new String(),
  "test", // remains unchanged
];

for (var i = 0; i < types.length; i++) {
  types[i].constructor = Type;
  types[i] = [
    types[i].constructor,
    types[i] instanceof Type,
    types[i].toString(),
  ];
}

console.log(types.join("\n"));
```

Esse exemplo apresenta a seguinte saída:

```
function Type() {},false,
function Type() {},false,
function Type() {},false,false
function Boolean() {
    [native code]
},false,true
function Type() {},false,Mon Sep 01 2014 16:03:49 GMT+0600
function Type() {},false,Error
function Type() {},false,function anonymous() {

}
function Type() {},false,function () {}
function Type() {},false,[object Math]
function Type() {},false,0
function Number() {
    [native code]
},false,1
function Type() {},false,[object Object]
function Type() {},false,[object Object]
```

```
function Type() {},false,/(?:)/
function Type() {},false,/(?:)/
function Type() {},false,
function String() {
    [native code]
},false,test
```

Object.create()

O método **`Object.create()`** cria um novo objeto, utilizando um outro objeto existente como protótipo para o novo objeto a ser criado.

Experimente

```
const person = {
  isHuman: false,
  printIntroduction: function () {
    console.log(`My name is ${this.name}. Am I human? ${this.isHuman}`);
  },
};

const me = Object.create(person);

me.name = "Matthew"; // "name" is a property set on "me", but not on "person"
me.isHuman = true; // Inherited properties can be overwritten

me.printIntroduction();
// Expected output: "My name is Matthew. Am I human? true"
```

Sintaxe

```
Object.create(proto[, propertiesObject])
```

Parâmetros

`proto`

O objeto que deve ser o protótipo do objeto recém-criado.

`propertiesObject`

Opcional. Se especificado e não `undefined`, um objeto cuja as propriedades próprias enumeráveis (isto é, aquelas propriedades definidas sobre si mesmo, e *não* propriedades enumeráveis ao longo da sua cadeia protótipa) especificam os nomes das propriedades a serem adicionadas ao objeto recém-criado, com os nomes das propriedades correspondentes. Essas propriedades correspondem ao segundo argumento de `Object.defineProperties()`.

Retorno

Um novo objeto com o protótipo de objeto e propriedades especificadas.

Exceções

Uma exceção `TypeError` se o parâmetro `proto` não for `null` ou um objeto.

Exemplos

Herança tradicional com <code>Object.create()</code>

A seguir, um exemplo de como usar `Object.create()` para realizar uma herança tradicional. Isto é para herança simples, que é a única herança suportada pelo JavaScript.

```
// Shape - superclasse
function Shape() {
  this.x = 0;
  this.y = 0;
}

// método da superclasse
Shape.prototype.move = function (x, y) {
  this.x += x;
  this.y += y;
  console.info("Shape moved.");
};

// Rectangle - subclasse
function Rectangle() {
  Shape.call(this); // chama construtor-pai.
}

// subclasse extende superclasse
Rectangle.prototype = Object.create(Shape.prototype);
Rectangle.prototype.constructor = Rectangle;

var rect = new Rectangle();

console.log("Rect é uma instância de Rectangle?", rect instanceof Rectangle); // true
```

```
console.log("Rect é uma instância de Shape?", rect instanceof Shape); // true
rect.move(1, 1); // Saída: 'Shape moved.'
```

Caso queira realizar herança de múltiplos objetos, então mixins ("mistura") são uma possibilidade.

```
function MyClass() {
  SuperClass.call(this);
  OtherSuperClass.call(this);
}

MyClass.prototype = Object.create(SuperClass.prototype); // herança
mixin(MyClass.prototype, OtherSuperClass.prototype); // mixin

MyClass.prototype.myMethod = function () {
  // faz algo
};
```

A função `mixin` copia as funções do protótipo da superclasse para o protótipo da subclasse, a função mixin precisa ser fornecida pelo usuário. Um exemplo de uma função do tipo mixin seria jQuery.extend().

Usando argumento <code>propertiesObject</code> com <code>Object.create()</code>

```
var o;

// cria um objeto com protótipo null
o = Object.create(null);

o = {};
// equivalente a:
o = Object.create(Object.prototype);

// Exemplo em que criamos um objeto com algumas propriedades
// (Note que o segundo parâmetro mapeia as chaves para *descritores de propriedade*.)
o = Object.create(Object.prototype, {
  // foo é uma 'propriedade de valor' ('value property') normal
  foo: { writable: true, configurable: true, value: "hello" },
  // bar é uma propriedade getter-setter (accessor)
  bar: {
    configurable: false,
    get: function () {
      return 10;
    },
    set: function (value) {
      console.log("Setting `o.bar` to", value);
    },
    /* com os ES5 Accessors nosso código pode ser escrito como:
    get() { return 10; },
    set(value) { console.log('setting `o.bar` to', value); } */
  },
});

function Constructor() {}
o = new Constructor();
// equivalente a:
o = Object.create(Constructor.prototype);
// Claro, se há de fato um código de inicialização na função
// Constructor, o Object.create() não pode refleti-la

// Cria um novo objeto cujo protóptipo é um objeto novo, vazio
// e adiciona a propriedade 'p' com o valor 42.
o = Object.create({}, { p: { value: 42 } });

// por padrão, propriedades NÃO SÃO escritas, enumeradas ou configuráveis:
o.p = 24;
o.p;
// 42

o.q = 12;
for (var prop in o) {
  console.log(prop);
}
// 'q'

delete o.p;
// false

// especificar uma propriedade ES3
o2 = Object.create(
  {},
  {
    p: {
      value: 42,
      writable: true,
      enumerable: true,
      configurable: true,
```

```
    },
  },
);
```

Polyfill

Este polyfill cobre o caso de uso principal que é a crição de um novo objeto em que o protótipo foi escolhido mas não leva em consideração o segundo argumento.

Note que, enquanto a configuração `null` as `[[Prototype]]` é suportada no ES5 `Object.create`, este polyfill não suporta devido à limitação inerente em versões do ECMAScript inferiores a 5.

```
if (typeof Object.create != "function") {
  Object.create = (function () {
    var Temp = function () {};
    return function (prototype) {
      if (arguments.length > 1) {
        throw Error("Second argument not supported");
      }
      if (typeof prototype != "object") {
        throw TypeError("Argument must be an object");
      }
      Temp.prototype = prototype;
      var result = new Temp();
      Temp.prototype = null;
      return result;
    };
  })();
}
```

Veja também

- `Object.defineProperty()`
- `Object.defineProperties()`
- `Object.prototype.isPrototypeOf()`
- Post de John Resig sobre getPrototypeOf()

Object.defineProperties()

O método **`Object.defineProperties()`** define uma nova propriedade ou modifica uma existente no objeto, retornando o objeto.

Sintaxe

```
Object.defineProperties(obj, props)
```

Parâmetros

`obj`

O objeto no qual se cria ou modifica suas propriedades.

`props`

Um objeto do qual propriedades enumeráveis constitui descritores para as propriedades serem definidas ou modificadas. Descritores de propriedade presente nos objetos provém em dois principais tipos: descritores de dados e de acesso (veja `Object.defineProperty()` para mais detalhes). Descritores têm as seguintes chaves:

`configurable`

`true` se e somente se o tipo deste descritor de propriedades pode ser modificada e se a propriedade pode ser apagada do objeto correspondente. **Valor padrão é `false`.**

`enumerable`

`true` se e somente se este propriedade aparece durante enumeração das propriedade sobre o objeto correspondente. **Valor padrão é `false`.**

`value`

O valor associado com a propriedade. Pode ser qualquer valor válido em JavaScript value (número, objeto, função, etc). **Valor padrão é `undefined`.**

`writable`

`true` se e somente se o valor associado com a propriedade pode ser modificada com um assignment operator. **Valor padrão é `false`.**

`get`

Uma função a qual serve com um getter para a propriedade, ou `undefined` se não existe getter. A retorno da função será usado como o valor da propriedade. **Valor padrão é `undefined`.**

`set`

Uma função a qual server com um setter para a propriedade, ou `undefined` se não existe setter. A função receberá como argumento somente o novo valor sendo atribuído à propriedade. **Valor padrão é `undefined`.**

Valor de retorno

O objeto que foi passado para a função.

Descrição

`Object.defineProperties`, em essência, define todas as propriedades correspondentes para as propriedades próprias enumeráveis de `props` sobre o objeto `obj`.

Exemplo

```
var obj = {};
Object.defineProperties(obj, {
  property1: {
    value: true,
    writable: true,
  },
  property2: {
    value: "Hello",
    writable: false,
  },
  // etc. etc.
});
```

Polyfill

Assumindo uma execução intocada com todos os nomes e propriedades referindo para seus valores iniciais, `Object.defineProperties` é quase completamente equivalente (note o comentário em `isCallable`) para a seguinte reimplementação em JavaScript:

```
function defineProperties(obj, properties) {
  function convertToDescriptor(desc) {
    function hasProperty(obj, prop) {
      return Object.prototype.hasOwnProperty.call(obj, prop);
    }

    function isCallable(v) {
      // NB: modify as necessary if other values than functions are callable.
      return typeof v === "function";
    }

    if (typeof desc !== "object" || desc === null)
      throw new TypeError("bad desc");

    var d = {};

    if (hasProperty(desc, "enumerable")) d.enumerable = !!desc.enumerable;
    if (hasProperty(desc, "configurable")) d.configurable = !!desc.configurable;
    if (hasProperty(desc, "value")) d.value = desc.value;
    if (hasProperty(desc, "writable")) d.writable = !!desc.writable;
    if (hasProperty(desc, "get")) {
      var g = desc.get;

      if (!isCallable(g) && typeof g !== "undefined")
        throw new TypeError("bad get");
      d.get = g;
    }
    if (hasProperty(desc, "set")) {
      var s = desc.set;
      if (!isCallable(s) && typeof s !== "undefined")
        throw new TypeError("bad set");
      d.set = s;
    }

    if (("get" in d || "set" in d) && ("value" in d || "writable" in d))
      throw new TypeError("identity-confused descriptor");

    return d;
  }

  if (typeof obj !== "object" || obj === null) throw new TypeError("bad obj");

  properties = Object(properties);

  var keys = Object.keys(properties);
  var descs = [];

  for (var i = 0; i < keys.length; i++)
    descs.push([keys[i], convertToDescriptor(properties[keys[i]])]);

  for (var i = 0; i < descs.length; i++)
    Object.defineProperty(obj, descs[i][0], descs[i][1]);

  return obj;
}
```

Veja também

- `Object.defineProperty()`

- `Object.keys()`
- Enumerabilidade e direito de propriedades

Object.defineProperty()

O método **`Object.defineProperty()`** define uma nova propriedade diretamente em um objeto, ou modifica uma propriedade já existente em um objeto, e retorna o objeto.

> **Nota:** Você invoca este método diretamente no construtor do `Object` ao invés de invocar em uma instância do tipo `Object`.

Sintaxe

```
Object.defineProperty(obj, prop, descriptor)
```

Parâmetros

`obj`

O objeto no qual será definida a propriedade.

`prop`

O nome da propriedade que será definida ou modificada.

`descriptor`

O descritor para a propriedade que será definida ou modificada.

Valor de retorno

O objeto que foi passado à função.

Descrição

Esse método permite uma precisa inclusão ou modificação de uma propriedade em um objeto. Enquanto a inclusão de propriedades através de atribuição cria propriedades que são visíveis durante a enumeração (por repetições `for...in` ou pelo método `Object.keys`), e cujos valores podem ser alterados e deletados, esse método permite a modificação deste comportamento padrão. Por padrão, valores incluídos utilizando `Object.defineProperty()` são imutáveis.

Os descritores de propriedades presentes nos objetos se apresentam em duas variedades: descritores de dados e descritores de assessores. Um *descritor de dado* é uma propriedade que contém um valor, podendo este ser gravável ou não. Um *descritor de assessor* é uma propriedade definida como um par de funções getter-setter. Um descritor deve ser de uma destas variedades; não pode ser de ambas.

Ambos os descritores de dados e de assessor são objetos. Eles compartilham as seguintes chaves obrigatórias:

`configurable`

`true` se e somente se o tipo deste descritor de propriedade pode ser alterado e se a propriedade pode ser deletada do objeto correspondente. **Valor padrão é `false`.**

`enumerable`

`true` se e somente se esta propriedade aparece durante enumeração das propriedades no objeto correspondente. **Valor padrão é `false`.**

Um descritor de dados também possui as seguintes chaves opcionais:

`value`

O valor associado com a propriedade. Pode ser qualquer valor válido em Javascript (número, objeto, função, etc). **Valor padrão é `undefined`.**

`writable`

`true` se e somente se o valor associado com a propriedade pode ser modificada com um operador de atribuição. **Valor padrão é `false`.**

Um descritor de assessor também possui as seguintes chaves opcionais:

`get`

Uma função que servirá como um getter da propriedade, ou `undefined` se não houver getter. Quando a propriedade é acessada, esta função é chamada sem argumentos e com `this` define para o objeto no qual a propriedade é acessada (este pode não ser o objeto sobre o qual a propriedade está definida devido a herança). O valor retornado será usado como valor da propriedade. **Valor padrão é `undefined`.**

`set`

A função que servirá como um setter para a propriedade, ou `undefined` se não houver setter. Quando a propriedade é atribuída, esta função é chamada com um argumento (o valor sendo atribuído para a propriedade) e com `this` configura o objeto através do qual a propriedade é atribuída. **Valor padrão é `undefined`.**

Se um descritor tem nenhum das chaves `value`, `writable`, `get` e `set`, ele é tratado como um descritor de dados. Se um descritor tem ambas chaves `value` ou `writable` e `get` ou `set` keys, uma exceção é lançada.

Tenha em mente que estes atributos não são necessariamente as propriedades do próprio descritor. Propriedades herdadas serão consideradas também. Para garantir que estes padrões sejam preservados, você pode congelar o `Object` previamente, declarar todas as opções explicitamente, ou apontar para `null` com `Object.create(null)`.

```
// usando __proto__
var obj = {};
var descriptor = Object.create(null); // não herdar propriedades
// não enumerável, não configurável, não gravável por padrão
descriptor.value = "static";
Object.defineProperty(obj, "key", descriptor);

// declarando explicitamente
Object.defineProperty(obj, "key", {
  enumerable: false, // não enumerável
  configurable: false, // não configurável
  writable: false, // não gravável
  value: "static",
});

// reciclando um mesmo objeto
function withValue(value) {
  var d =
    withValue.d ||
    (withValue.d = {
      enumerable: false,
      writable: false,
      configurable: false,
      value: null,
    });
  d.value = value;
  return d;
}

// ... e ...
Object.defineProperty(obj, "key", withValue("static"));

// se o método freeze estiver disponível, prevenir as propriedades
// (value, get, set, enumerable, writable, configurable) de serem
// incluídas ou removidas do protótipo do objeto
(Object.freeze || Object)(Object.prototype);
```

Exemplos

Se você deseja utilizar o método `Object.defineProperty` com uma sintaxe estilo *flags-binárias*, veja os exemplos adicionais.

Criando uma propriedade

Quando a propriedade especificada não existe no objeto, `Object.defineProperty()` cria uma nova propriedade conforme descrito anteriormente. Campos podem ser omitidos no descritor, e os valores padrão para esses campos serão introduzidos.

Todos os campos do tipo Boolean possuem como valor padrão `false`. Os campos `value`, `get`, e `set` possuem como padrão `undefined`. Uma propriedade que é definida sem os valores para `get`/`set`/`value`/`writable` é dita "genérica" e classificada como um descritor de dados.

```
var o = {}; // Criar um novo objeto

// Exemplo de propriedade de objeto inserida através
// de defineProperty com descritor do tipo dado
Object.defineProperty(o, "a", {
  value: 37,
  writable: true,
  enumerable: true,
  configurable: true,
});
// A propriedade 'a' existe no objeto com valor 37

// Exemplo de propriedade de objeto inserida através
// de defineProperty com descritor do tipo assessor
var bValue = 38;

Object.defineProperty(o, "b", {
  get: function () {
    return bValue;
  },
  set: function (newValue) {
    bValue = newValue;
  },
  enumerable: true,
  configurable: true,
});

o.b; // 38
// A propriedade 'b' existe no objeto com valor 38
// O valor de o.b será sempre idêntico a bValue, a
// menos que o.b seja redefinido

// Você não pode combinar ambos os tipos:
Object.defineProperty(o, "conflict", {
  value: 0x9f91102,
  get: function () {
```

```
    return 0xdeadbeef;
  },
});
// lança um TypeError: value existe apenas em descritores
// de dado, get existe apenas em descritores de assessor
```

Modificando uma propriedade

Quando uma propriedade já existe, `Object.defineProperty()` tenta modificá-la de acordo com os valores do descritor e a configuração atual do objeto. Se o descritor antigo possuía seu atributo `configurable` como `false` a propriedade é chamada "não configurável" e nenhum atributo pode ser alterado (exceto a alteração irreversível de *writable* para *false*). Não é possível alternar o tipo de uma propriedade entre dados e assessor quando esta for não-configurável.

Um `TypeError` é lançado quando são realizadas tentativas de se alterar propriedades não-configuráveis (exceto o atributo `writable`) a menos que o valor atual e o novo sejam os mesmos.

O atributo writable

Quando o atributo `writable` de uma propriedade é definido como `false`, a propriedade é dita "não-gravável". Seu valor não poderá ser alterado.

```
var o = {}; // Cria um novo objeto

Object.defineProperty(o, "a", {
  value: 37,
  writable: false,
});

console.log(o.a); // escreve 37

o.a = 25; // Nenhum erro é lançado (no modo strict seria
// lançado mesmo que o valor fosse o mesmo)

console.log(o.a); // escreve 37. A atribuição não teve efeito.
```

Como visto no exemplo, tentativas de escrita em uma propriedade não-gravável não alteram seu valor, mas também não lançam erros.

O atributo enumerable

O atributo `enumerable` de uma propriedade define se ela deve ser exibida em uma repetição `for...in` e por `Object.keys()` ou não.

```
var o = {};

Object.defineProperty(o, "a", {
  value: 1,
  enumerable: true,
});

Object.defineProperty(o, "b", {
  value: 2,
  enumerable: false,
});

Object.defineProperty(o, "c", {
  value: 3,
}); // o valor padrão para enumerable é false

o.d = 4; // o valor padrão para enumerable é true quando
// a propriedade é criada em uma atribuição

for (var i in o) {
  console.log(i);
}
// escreve 'a' e 'd' (em ordem indefinida)

Object.keys(o); // ['a', 'd']

o.propertyIsEnumerable("a"); // true
o.propertyIsEnumerable("b"); // false
o.propertyIsEnumerable("c"); // false
```

O atributo configurable

O atributo `configurable` controla ao mesmo se uma propriedade pode ser deletada do objeto, e se seus atributos (exceto a mudança de `writable` para `false`) podem ser alterados.

```
var o = {};

Object.defineProperty(o, "a", {
  get: function () {
    return 1;
  },
  configurable: false,
});

Object.defineProperty(o, "a", {
```

```
  configurable: true,
}); // lança um TypeError

Object.defineProperty(o, "a", {
  enumerable: true,
}); // lança um TypeError

Object.defineProperty(o, "a", {
  set: function () {},
}); // lança um TypeError (o atributo set já estava definido)

Object.defineProperty(o, "a", {
  get: function () {
    return 1;
  },
}); // lança um TypeError
// (mesmo o novo get fazendo exatamente a mesma coisa)

Object.defineProperty(o, "a", {
  value: 12,
}); // lança um TypeError

console.log(o.a); // escreve 1
delete o.a; // Nada acontece
console.log(o.a); // escreve 1
```

Se o atributo `configurable` de `o.a` fosse `true`, nenhum dos erros seria lançado e a propriedade estaria deletada ao final.

Incluindo propriedades e valores padrão

É importante considerar a forma como os valores padrão para atributos são aplicados. Normalmente existe diferença entre usar a notação por ponto para atribuir um valor e usar `Object.defineProperty()`, como pode ser visto no exemplo abaixo:

```
var o = {};

o.a = 1;

// é equivalente a:
Object.defineProperty(o, "a", {
  value: 1,
  writable: true,
  configurable: true,
  enumerable: true,
});

// Por outro lado,
Object.defineProperty(o, "a", { value: 1 });

// é equivalente a:
Object.defineProperty(o, "a", {
  value: 1,
  writable: false,
  configurable: false,
  enumerable: false,
});
```

Setters e getters customizados

O exemplo abaixo mostra como implementar um objeto auto-arquivável. Quando a propriedade `temperature` é atribuída, o array `archive` recebe uma nova entrada de log.

```
function Archiver() {
  var temperature = null;
  var archive = [];

  Object.defineProperty(this, "temperature", {
    get: function () {
      console.log("get!");
      return temperature;
    },
    set: function (value) {
      temperature = value;
      archive.push({ val: temperature });
    },
  });

  this.getArchive = function () {
    return archive;
  };
}

var arc = new Archiver();
arc.temperature; // 'get!'
arc.temperature = 11;
```

```
arc.temperature = 13;
arc.getArchive(); // [{ val: 11 }, { val: 13 }]
```

Neste exemplo, um getter sempre retorna o mesmo valor.

```
var pattern = {
  get: function () {
    return (
      "Eu sempre retorno esta string, " + "não importa o que você atribuiu"
    );
  },
  set: function () {
    this.myname = "esta string é meu nome";
  },
};

function TestDefineSetAndGet() {
  Object.defineProperty(this, "myproperty", pattern);
}

var instance = new TestDefineSetAndGet();
instance.myproperty = "test";

console.log(instance.myproperty);
// Eu sempre retorno esta string, não importa o que você atribuiu

console.log(instance.myname); // esta string é meu nome
```

Propriedades de Herança

Se uma propriedade de acessor é herdada, métodos `get` e `set` serão chamados quando a propriedade é acessada e modificada sobre os objetos descendentes. Se estes métodos usam uma variável para armazenar o valor, este valor será compartilhada por todos os objetos.

```
function myclass() {
}

var value;
Object.defineProperty(myclass.prototype, "x", {
  get() {
    return value;
  },
  set(x) {
    value = x;
  }
});

var a = new myclass();
var b = new myclass();
a.x = 1;
console.log(b.x); // 1
```

Isto pode ser corrigido armazenando o valor em outra propriedade. Em métodos `get` e `set`, `this` aponta para o objeto no qual é usado para acessar ou modificar a propriedade.

```
function myclass() {
}

Object.defineProperty(myclass.prototype, "x", {
  get() {
    return this.stored_x;
  },
  set(x) {
    this.stored_x = x;
  }
});

var a = new myclass();
var b = new myclass();
a.x = 1;
console.log(b.x); // undefined
```

Ao contrário das propriedades do acessor, propriedades do valor serão sempre configuradas sobre o próprio objeto, não sobre um protótipo. Entretanto, se uma propriedade de valor não-gravável é herdada, ele ainda previne de modicação a propriedade do objeto.

```
function myclass() {
}

myclass.prototype.x = 1;
Object.defineProperty(myclass.prototype, "y", {
  writable: false,
  value: 1
});

var a = new myclass();
```

```
a.x = 2;
console.log(a.x); // 2
console.log(myclass.prototype.x); // 1
a.y = 2; // Ignorado, lança no modo strict
console.log(a.y); // 1
console.log(myclass.prototype.y); // 1
```

Notas de compatibilidade

Redefinindo a propriedade <code>length</code> de um objeto <code>Array</code>

É possível redefinir a propriedade `length` de arrays, sujeita às restrições de redefinição usuais. (A propriedade `length` é inicialmente não configurável, não enumerável, mas gravável. Assim, em um array que não foi modificado, é possível alterar o valor da propriedade `length` ou torná-la não-gravável. Não é permitido alterar sua enumerabilidade ou configurabilidade, ou quando se encontrar não-gravável, alterar seu valor ou torná-la gravável novamente.) Entretanto, nem todos os browsers permitem esta redefinição.

Das versões 4 até 22 do Firefox, um `TypeError` é lançado em qualquer tentativa (seja ela permitida ou não) de redefinir a propriedade `length` de um array.

Versões do Chrome que implementam `Object.defineProperty()` em algumas circunstâncias ignoram um valor para *length* diferente do valor atual da propriedade `length` do array. Em algumas circustâncias, alterar o atributo `writable` falha de forma silenciosa (sem lançar uma exceção). Além disso, alguns métodos que modificam o array como {jsxref("Array.prototype.push")}} não respeitam uma propriedade *length* não-gravável.

Versões do Safari que implementam `Object.defineProperty()` ignoram um valor para `length` diferente do valor atual da propriedade `length`, e tentantivas de alterar o atributo `writable` executam sem erros embora não modifiquem seu comportamento.

Apenas o Internet Explorer 9 a posteriores, e o Firefox 23 e posteriores, parecem implementar total e corretamente a redefinição da propriedade `length` de arrays. Por enquanto, não confie que a redefinição da propriedade `length` vá funcionar, mesmo que de uma forma particular. E mesmo quando você *puder* confiar, existem boas razões para não fazer isso.

Notas específicas para o Internet Explorer 8

O Internet Explorer 8 implementa o método `Object.defineProperty()` para uso apenas em objetos DOM. Algumas observações:

- Tentativas de usar `Object.defineProperty()` em objetos nativos lançam um erro.
- Todos os atributos da propriedade devem ter seu valor definido. Os atributos `configurable`, `enumerable` e `writable` devem ser `true` para um descritor do tipo dado, e `true` para `configurable` e `false` para `enumerable` em descritores do tipo assessor. (?) Qualquer tentativa de usar outros valores (?) resultará no lançamento de um erro.
- Reconfigurar uma propriedade exige que ela seja deletada anteriormente. Se a propriedade não for deletada, ela permanecerá inalterada após a tentativa de reconfiguração.

Veja também

- Enumerabilidade and posse de propriedades
- `Object.defineProperties()`
- `Object.propertyIsEnumerable()`
- `Object.getOwnPropertyDescriptor()`
- `Object.prototype.watch()`
- `Object.prototype.unwatch()`
- `get`
- `set`
- `Object.create()`
- Exemplos adicionais de `Object.defineProperty`
- `Reflect.defineProperty()`

Object.entries()

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

O método **`Object.entries()`** retorna uma array dos próprios pares `[key, value]` enumeráveis de um dado objeto, na mesma ordem dos objetos providos através do loop `for...in` (sendo a diferença que o for-in loop enumera também propriedades dispostas na cadeia de prototipagem - *prototype chain*).

Sintaxe

```
Object.entries(obj)
```

Parâmetros

`obj`

O objeto cujos próprios pares `[key, value]` de propriedades enumeráveis deverão ser retornados.

Valor de retorno

Uma array de pares `[key, value]` enumeráveis de propriedades de um dado objeto.

Descrição

`Object.entries()` retorna um array cujos elementos são também arrays correspondentes aos pares de propriedades `[key, value]` enumeráveis encontrados diretamente sobre o objeto. A ordem das propriedades é a mesma que seria se fossem iteradas as propriedades do objeto manualmente.

Exemplos

```
var obj = { foo: "bar", baz: 42 };
console.log(Object.entries(obj)); // [ ['foo', 'bar'], ['baz', 42] ]

// objeto array-like
var obj = { 0: "a", 1: "b", 2: "c" };
console.log(Object.entries(obj)); // [ ['0', 'a'], ['1', 'b'], ['2', 'c'] ]

// objeto array-like com ordenação aleatória de chave (key)
var an_obj = { 100: "a", 2: "b", 7: "c" };
console.log(Object.entries(an_obj)); // [ ['2', 'b'], ['7', 'c'], ['100', 'a'] ]

// getFoo é uma propriedade que não é enumerável
var my_obj = Object.create(
  {},
  {
    getFoo: {
      value: function () {
        return this.foo;
      },
    },
  },
);
my_obj.foo = "bar";
console.log(Object.entries(my_obj)); // [ ['foo', 'bar'] ]

// argumento não-objeto será convertido (conversão implícita) para um objeto
console.log(Object.entries("foo")); // [ ['0', 'f'], ['1', 'o'], ['2', 'o'] ]

// itera graciosamente através de chave-valor (key-value)
var obj = { a: 5, b: 7, c: 9 };
for (var [key, value] of Object.entries(obj)) {
  console.log(key + " " + value); // "a 5", "b 7", "c 9"
}

// Ou, usando array extras
Object.entries(obj).forEach(([key, value]) => {
  console.log(key + " " + value); // "a 5", "b 7", "c 9"
});
```

Convertendo um <code>Object</code> em um <code>Map</code>

O construtor `new Map()` aceita entradas iteráveis. Com o `Object.entries`, você pode facilmente converter de `Object` para `Map`:

```
var obj = { foo: "bar", baz: 42 };
var map = new Map(Object.entries(obj));
console.log(map); // Map { foo: "bar", baz: 42 }
```

Polyfill

Para incluir suporte ao `Object.entries` em ambientes mais antigos, você pode achar um Polyfill nos repositórios: tc39/proposal-object-values-entries e es-shims/Object.entries.

Veja também

- Enumerabilidade e posse de propriedades
- `Object.keys()`
- `Object.values()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Object.getOwnPropertyNames()`

Object.freeze()

O método **`Object.freeze()`** congela um objeto: isto é, impede que novas propriedades sejam adicionadas a ele; impede que as propriedades existentes sejam removidas; e impede que propriedades existentes, ou sua inumerabilidade, configurabilidade, ou capacidade de escrita sejam alteradas. Em essência o objeto é efetivamente imutável. O método retorna o objeto congelado.

Sintaxe

```
Object.freeze(obj)
```

Parâmetros

`obj`

O objeto a ser congelado.

Valor de retorno

O objeto congelado.

Descrição

Nada pode ser adicionado ou removido do conjunto de propriedades de um objeto congelado. Qualquer tentativa de fazê-lo irá falhar, ou silenciosamente ou através de um `TypeError` exception (mais frequentemente, mas não exclusivamente, quando em strict mode).

Valores não podem ser alterados para propriedades de dados. As propriedades do Accessor (getters e setters) funcionam da mesma forma (e ainda dão a ilusão de que você está alterando o valor). Observe que valores que são objetos ainda podem ser modificados, a menos que eles também sejam congelados.

Exemplos

```
var obj = {
  prop: function () {},
  foo: "bar",
};

// Novas propriedades podem ser adicionadas, propriedades existentes podem ser alteradas ou removidas
obj.foo = "baz";
obj.lumpy = "woof";
delete obj.prop;

// Tanto o objeto que está sendo passado, bem como o objeto retornado será congelado.
// É desnecessário salvar o objeto retornado para congelar o objeto original.
var o = Object.freeze(obj);

o === obj; // true
Object.isFrozen(obj); // === true

// De agora em diante qualquer alteração irá falhar
obj.foo = "quux"; // silenciosamente não faz nada.
obj.quaxxor = "the friendly duck"; // silenciosamente não adiciona a propriedade

// ...e em modo strict tais tentativas irão lançar TypeErrors
function fail() {
  "use strict";
  obj.foo = "sparky"; // throws um TypeError
  delete obj.quaxxor; // throws um TypeError
  obj.sparky = "arf"; // throws um TypeError
}

fail();

// As tentativas de alteração através de Object.defineProperty também irão lançar
Object.defineProperty(obj, "ohai", { value: 17 }); // throws um TypeError
Object.defineProperty(obj, "foo", { value: "eit" }); // throws um TypeError
```

O exemplo a seguir mostra que valores do tipo objeto, em um objeto congelado, podem ser alterados (freeze é raso).

```
obj1 = {
  internal: {},
};

Object.freeze(obj1);
obj1.internal.a = "aValue";

obj1.internal.a; // 'aValue'

// Para fazer um obj completamente imutável, congele cada objeto em obj.
// Para fazer isso, nós usamos essa função.
function deepFreeze(obj) {
  // Recuperar os nomes de propriedade definidos em obj
  var propNames = Object.getOwnPropertyNames(obj);

  // Congelar as propriedades antes de congelar-se
  propNames.forEach(function (name) {
    var prop = obj[name];

    // Congele prop se for um objeto
    if (typeof prop == "object" && prop !== null) deepFreeze(prop);
  });

  // Congele-se (não faz nada se já estiver congelado)
  return Object.freeze(obj);
}

obj2 = {
  internal: {},
};

deepFreeze(obj2);
obj2.internal.a = "anotherValue";
obj2.internal.a; // undefined
```

Notas

Em ES5, se um argumento para este método não for um objeto (um primitivo), então isso irá causar um `TypeError`. Em ES6, um argumento não-objeto vai ser tratado como se fosse um objeto comum congelado e simplesmente retornado.

```
> Object.freeze(1)
TypeError: 1 is not an object // ES5 code

> Object.freeze(1)
1                             // ES6 code
```

Ver também

- `Object.isFrozen()`
- `Object.preventExtensions()`
- `Object.isExtensible()`
- `Object.seal()`
- `Object.isSealed()`

Object.fromEntries()

O método **`Object.fromEntries()`** transforma uma lista de pares chave-valor em um objeto.

Experimente

```
const entries = new Map([
  ["foo", "bar"],
  ["baz", 42],
]);

const obj = Object.fromEntries(entries);

console.log(obj);
// Expected output: Object { foo: "bar", baz: 42 }
```

Sintaxe

```
Object.fromEntries(iterable);
```

Parâmetros

`iterable`

Um iterável como `Array` ou `Map` ou qualquer outro objeto que implemente o protocolo iterável.

Valor de retorno

Um novo objeto com suas propriedades definidas pelas entradas fornecidadas pelo iterável.

Descrição

O método `Object.fromEntries()` recebe uma lista de pares chave-valor e retorna um novo objeto cujas propriedades foram definidas pelas entradas da lista. O parâmetro *iterable* deve ser um objeto que implemente o método `@@iterator`, que retorne um objeto iterador que produza um objeto *array-like* de dois elementos, onde o primeiro será a chave da propriedade e o segundo será o valor associado à chave.

`Object.fromEntries()` faz o inverso de `Object.entries()`.

Exemplos

Convertendo um <code>Map</code> em um <code>Object</code>

Com o método `Object.fromEntries`, é possível fazer a conveeção de um `Map` em um `Object`:

```
const map = new Map([
  ["foo", "bar"],
  ["baz", 42],
]);
const obj = Object.fromEntries(map);
console.log(obj); // { foo: "bar", baz: 42 }
```

Convertendo um <code>Array</code> em um <code>Object</code>

Com o método `Object.fromEntries`, é possível converter um `Array` em um `Object`:

```
const arr = [
  ["0", "a"],
  ["1", "b"],
  ["2", "c"],
];
const obj = Object.fromEntries(arr);
console.log(obj); // { 0: "a", 1: "b", 2: "c" }
```

Transformações de objetos

Com o método `Object.fromEntries`, seu inverso `Object.entries()`, e os métodos para manipulação de arrays, é possível fazer transformações em objetos como por exemplo:

```
const object1 = { a: 1, b: 2, c: 3 };

const object2 = Object.fromEntries(
  Object.entries(object1).map(([key, val]) => [key, val * 2]),
);

console.log(object2);
// { a: 2, b: 4, c: 6 }
```

Veja Também

- `Object.entries()`
- `Object.keys()`
- `Object.values()`
- `Map.prototype.entries()`
- `Map.prototype.keys()`
- `Map.prototype.values()`

Object.getOwnPropertyDescriptor()

O método **`Object.getOwnPropertyDescriptor()`** retorna um descritor de propriedades para uma propriedade (isto é, uma diretamente presente, e não pertencente ao objeto por força da cadeia de protótipo do objeto) de um dado objeto.

Sintaxe

```
Object.getOwnPropertyDescriptor(obj, prop)
```

Parâmetros

`obj`

O objeto no qual deve-se procurar pela propriedade.

`prop`

O nome da propriedade cuja descrição é para ser retornada.

Valor de Retorno

Um descritor de propriedades da propriedade especificada, se esta existir no objeto, ou `undefined`, caso contrário.

Descrição

Este método permite uma análise da descrição precisa de uma propriedade. Uma propriedade em Javascript consiste de um nome com formato *texto-valor* e um descritor de propriedades. Mais informações sobre os tipos de descritores de propriedades e seus atributos podem ser encontrados em `Object.defineProperty()`.

Um descritor de propriedade é um registro com alguns dos seguintes atributos:

`value`

o valor associado com a propriedade (somente para descritores de dados).

`writable`

`true` se, e somente se, o valor associado com a propriedade pode ser alterado (somente para descritores de dados).

`get`

Uma função que serve como um *getter*, para obter o valor da propriedade, ou `undefined` se não houver (somente para descritores de acesso).

`set`

Uma função que serve como um *setter*, para atribuir um valor à propriedade, ou `undefined` se não houver (somente para descritores de acesso).

`configurable`

`true` se, e somente se, o tipo deste descritor de propriedade pode ser alterado e se a propriedade pode ser excluída do objeto correspondente.

`enumerable`

`true` se, e somente se, esta propriedade aparece durante a enumeração das propriedades do objeto correspondente.

Exemplos

```
var o, d;

o = {
  get foo() {
    return 17;
  },
};
```

```
d = Object.getOwnPropertyDescriptor(o, "foo");
// d é { configurable: true, enumerable: true, get: /*A função getter*/, set: undefined }

o = { bar: 42 };
d = Object.getOwnPropertyDescriptor(o, "bar");
// d é { configurable: true, enumerable: true, value: 42, writable: true }

o = {};
Object.defineProperty(o, "baz", {
  value: 8675309,
  writable: false,
  enumerable: false,
});
d = Object.getOwnPropertyDescriptor(o, "baz");
// d é { value: 8675309, writable: false, enumerable: false, configurable: false }
```

Notas

No ES5, o primeiro parâmetro deste método não é um objeto (tipo primitivo), então ele irá gerar um `TypeError`. No ES6, um primeiro argumento não-objeto será primeiramente convertido para objeto.

```
Object.getOwnPropertyDescriptor("foo", 0);
// TypeError: "foo" is not an object  // Código ES5

Object.getOwnPropertyDescriptor("foo", 0);
// {configurable:false, enumerable:true, value:"f", writable:false}  // Código ES6
```

Veja Também

- `Object.defineProperty()`
- `Reflect.getOwnPropertyDescriptor()`

Object.getOwnPropertyDescriptors()

O **`Object.getOwnPropertyDescriptors()`** método retorna todas as descrições próprias da propriedade de um dado objeto.

Experimente

```
const object1 = {
  property1: 42,
};

const descriptors1 = Object.getOwnPropertyDescriptors(object1);

console.log(descriptors1.property1.writable);
// Expected output: true

console.log(descriptors1.property1.value);
// Expected output: 42
```

Sintaxe

```
Object.getOwnPropertyDescriptors(obj)
```

Parâmetro

`obj`

O objeto pelo o qual deseja obter todos os descritores de propriedade próprios.

Valor retornado

Um objeto contendo todas as propriedades descritivas de um objeto. Pode ser um objeto vazio, se não existir propriedade.

Descrição

Esse método permite examinar a descrição precisa de todas as propriedades de um objeto. Uma propriedade em JavaScript consiste de um nome com valor string ou um `Symbol` e uma propriedade descritora. Outras informações sobre propriedade de tipo descritoras e seus atributos podem ser encontradas em `Object.defineProperty()`.

Uma propriedade descritora é um registro com alguns dos seguintes atributos:

`value`

O valor associado com à propriedade (somente descritores de dados).

`writable`

`true` se somente o valor associado com a propriedade pode ser alterada (somente descritores de dados).

`get`

Uma função que serve como um capturador para a propriedade ou `undefined` se não existir um capturador (somente descritores de acesso).

`set`

Uma função que serve como um configurador para a propriedade, ou `undefined` se não existir um configurador (somente descritores de acesso).

`configurable`

`true` se e somente se o tipo da propriedade descritora pode ser alterada e se a propriedade pode ser deletada do objeto correspondente.

`enumerable`

`true` se e somente se essa propriedade se mostrar durante a enumeração da propriedade no objeto correspondente.

Exemplos

Criando um clone superficial

Enquanto o `Object.assign()` método vai copiar somente o enumerável e as próprias propriedades da origem de um objeto para o objeto alvo, você é capaz de usar esse método e `Object.create()` para uma cópia superficial entre dois objetos desconhecidos:

```
Object.create(
  Object.getPrototypeOf(obj),
  Object.getOwnPropertyDescriptors(obj),
);
```

Criando uma subclasse

Um modo típico de criar uma subclasse é definir a subclasse, configurar um protótipo para uma instância da superclasse e depois difinir as propriedades daquela instância. Isso pode ficar estranho especialmente para os capturadores e configuradores. Ao invés disso, você pode usar esse código para configurar o protótipo :

```
function superclass() {}
superclass.prototype = {
  // Define seu método e propriedades aqui
};
function subclass() {}
subclass.prototype = Object.create(superclass.prototype, {
  // Define seu método e propriedades aqui
});
```

Veja também

- `Object.getOwnPropertyDescriptor()`
- `Object.defineProperty()`
- Polyfill

Object.getOwnPropertyNames()

O método **`Object.getOwnPropertyNames()`** retorna um vetor com todas as propriedades (enumeráveis ou não) encontradas diretamente em um dado objeto.

Sintaxe

```
Object.getOwnPropertyNames(obj)
```

Parâmetros

`obj`

O objeto, cujas suas próprias propriedades, enumeráveis ou não, serão retornadas.

Descrição

`Object.getOwnPropertyNames()` retorna um vetor cujos elementos são strings correspondentes as propriedades enumeráveis ou não, encontradas em `obj`. A ordem das propriedades enumeráveis no vetor é consistente com a ordenação exposta por um laço `for...in` (ou por `Object.keys()`) nas propriedades do objeto. A ordenação das propriedades não-enumeráveis no vetor, e entre as propriedades enumeráveis, não está definida.

Exemplos

Usando <code>Object.getOwnPropertyNames()</code>

```
var arr = ["a", "b", "c"];
console.log(Object.getOwnPropertyNames(arr).sort());
// logs ["0", "1", "2", "length"]

// Array-like object
var obj = { 0: "a", 1: "b", 2: "c" };
console.log(Object.getOwnPropertyNames(obj).sort());
// logs ["0", "1", "2"]

// Logging property names and values using Array.forEach
Object.getOwnPropertyNames(obj).forEach(function (val, idx, array) {
  console.log(val + " -> " + obj[val]);
});
// logs
// 0 -> a
```

```
// 1 -> b
// 2 -> c

// non-enumerable property
var my_obj = Object.create(
  {},
  {
    getFoo: {
      value: function () {
        return this.foo;
      },
      enumerable: false,
    },
  },
);
my_obj.foo = 1;

console.log(Object.getOwnPropertyNames(my_obj).sort());
// logs ["foo", "getFoo"]
```

Se voce quer somente as propriedades enumeráveis, veja `Object.keys()` ou use um laço `for...in` (contudo, note que isto irá retornar propriedades enumeráveis não encontradas diretamente naquele objeto, mas também junto com a cadeia prototype do objeto a menos que o último seja filtrado com `hasOwnProperty()`).

Ítens na cadeia prototype não são listados:

```
function ParentClass() {}
ParentClass.prototype.inheritedMethod = function () {};

function ChildClass() {
  this.prop = 5;
  this.method = function () {};
}
ChildClass.prototype = new ParentClass();
ChildClass.prototype.prototypeMethod = function () {};

console.log(
  Object.getOwnPropertyNames(
    new ChildClass(), // ["prop", "method"]
  ),
);
```

Obtenha somente não-enumeráveis

Isto usa a função `Array.prototype.filter()` para remover as chaves enumeráveis (obtidas com `Object.keys()`) de uma lista com todas as chaves (obtidas com `Object.getOwnPropertyNames()`) deixando somente as chaves não-enumeráveis.

```
var target = myObject;
var enum_and_nonenum = Object.getOwnPropertyNames(target);
var enum_only = Object.keys(target);
var nonenum_only = enum_and_nonenum.filter(function (key) {
  var indexInEnum = enum_only.indexOf(key);
  if (indexInEnum == -1) {
    // not found in enum_only keys mean the key is non-enumerable,
    // so return true so we keep this in the filter
    return true;
  } else {
    return false;
  }
});

console.log(nonenum_only);
```

Notas

No ES5, se o argumento desse método não é um objeto (um tipo primitivo), então isso causará um `TypeError`. No ES6, um argumento diferente de objeto será transformado em um objeto.

```
Object.getOwnPropertyNames("foo");
// TypeError: "foo" is not an object (ES5 code)

Object.getOwnPropertyNames("foo");
// ["0", "1", "2", "length"]  (ES6 code)
```

Notas específicas para SpiderMonkey

Antes do SpiderMonkey 28, `Object.getOwnPropertyNames` não via propriedades não resolvidas de objetos `Error`. Isto foi resolvido em versões posteriores (Erro do Firefox 724768).

Veja também

- Enumerabilidade e posse de propriedades
- `Object.prototype.hasOwnProperty()`
- `Object.prototype.propertyIsEnumerable()`

- `Object.create()`
- `Object.keys()`
- `Array.forEach()`

Object.getOwnPropertySymbols()

O **`Object.getOwnPropertySymbols()`** método retorna uma array com todas propriedades de símbolo encontradas diretamente em um determinado objeto dado.

Experimente

```
const object1 = {};
const a = Symbol("a");
const b = Symbol.for("b");

object1[a] = "localSymbol";
object1[b] = "globalSymbol";

const objectSymbols = Object.getOwnPropertySymbols(object1);

console.log(objectSymbols.length);
// Expected output: 2
```

Sintaxe

```
Object.getOwnPropertySymbols(obj);
```

Parâmetro

`obj`

O objeto pelo qual as propriedades de símbolos são retornas.

Valor retornado

Uma array com todas as propriedades de símbolos encontradas diretamente pelo o objeto dado.

Descrição

Similar do `Object.getOwnPropertyNames()`, você pode obter todas as propriedades de símbolo de um objeto dado como um array de símbolos. Lembre que o próprio `Object.getOwnPropertyNames()` não contém as propriedades de símbolo de um objeto e somente as propriedades de string.

Como todos os objetos não possuem símbolos próprios inicialmente, o `Object.getOwnPropertySymbols()` retorna uma array vazia a menos que você tenha definido as propriedades de símbolo do seu objeto .

Exemplos

Usando getOwnPropertySymbols

```
var obj = {};
var a = Symbol("a");
var b = Symbol.for("b");

obj[a] = "localSymbol";
obj[b] = "globalSymbol";

var objectSymbols = Object.getOwnPropertySymbols(obj);

console.log(objectSymbols.length); // retorno esperado 2
console.log(objectSymbols); // retorno esperado (2) [Symbol(a), Symbol(b)]
console.log(objectSymbols[0]); // retorno esperado Symbol(a)
```

Veja também

- `Object.getOwnPropertyNames()`
- `Symbol`

Object.getPrototypeOf()

O método **`Object.getPrototypeOf()`** retorna o prototype (isto é, o valor da propriedade interna `[[Prototype]]`) do objeto especificado.

Sintaxe

```
Object.getPrototypeOf(obj)
```

Parâmetros

`obj`

O objeto cujo prototype será retornado.

Exemplos

```
var proto = {};
var obj = Object.create(proto);
Object.getPrototypeOf(obj) === proto; // true
```

Notas

No ES5 será lançada uma exceção `TypeError` se o parâmetro obj não for um objeto. No ES6, no entanto, esse valor será submetido a um `Object` equivalente ao seu tipo e retornado.

```
Object.getPrototypeOf("foo");
// TypeError: "foo" não é um objeto (código ES5)
Object.getPrototypeOf("foo");
// String.prototype                 (código ES6)
```

Notas específicas para o Opera

Embora as versões mais antigas do Opera não suportem `Object.getPrototypeOf()`, ele suporta a propriedade não oficial `Object.prototype.__proto__` desde de a sua versão 10.50.

Veja também

- `Object.prototype.isPrototypeOf()`
- `Object.setPrototypeOf()`
- `Object.prototype.__proto__`
- John Resig's post on getPrototypeOf
- `Reflect.getPrototypeOf()`

Object.hasOwn()

O método estático **`Object.hasOwn()`** retorna `true` se o objeto especificado tem a propriedade indicada como sua propriedade *own*. Se a propriedade é herdada, ou não existe, o método retorna `false`.

> **Nota:** `Object.hasOwn()` Tem a intenção de substituir `Object.hasOwnProperty()`.

Experimente

```
const object1 = {
  prop: "exists",
};

console.log(Object.hasOwn(object1, "prop"));
// Expected output: true

console.log(Object.hasOwn(object1, "toString"));
// Expected output: false

console.log(Object.hasOwn(object1, "undeclaredPropertyValue"));
// Expected output: false
```

Sintaxe

```
hasOwn(instance, prop);
```

Parâmetros

`instance`

A instância do objeto javascript para o teste.

`prop`

O nome `String` ou Symbol da propriedade para teste.

Retorna valor

`true` se o objeto especificado definiu diretamente a propriedade especificada. Por outro lado `false`.

Descrição

O método **`Object.hasOwn()`** retorna `true` se a propriedade especificada é uma propriedade direta do objeto - mesmo se o valor da propriedade é `null` ou `undefined`.

O método retorna `false` se a propriedade é herdada, ou não foi declarada. Diferente do operador `in`, este método não verifica pela propriedade específica na cadeia do objeto.

Ele é mais recomendado do que `Object.hasOwnProperty()` porque ele funciona para objetos criados utilizando `Object.create(null)` e com objetos que sobescreveram o método herdado `hasOwnProperty()`. É possível contornar esse problema chamando o `Object.prototype.hasOwnProperty()` em um objeto externo, `Object.hasOwn()` é mais intuítivo.

Exemplos

Utilizando hasOwn para o teste de propriedades existentes

O seguinte código mostra como determinar se o seguinte objeto `example` contém a propriedade `prop`.

```
const example = {};
Object.hasOwn(example, "prop"); // false = 'prop' ela não foi definida

example.prop = "exists";
Object.hasOwn(example, "prop"); // true - 'prop' ela foi definida

example.prop = null;
Object.hasOwn(example, "prop"); // true - a propriedade existe com o valor null

example.prop = undefined;
Object.hasOwn(example, "prop"); // true - a propriedade existe com o valor undefined
```

Propriedaedes direta vs. herdada

O seguinte exemplo diferencia entre propriedade direta e proprietade errada através do prototype chain:

```
const example = {};
example.prop = "exists";

// `hasOwn` Só retornará true para propriedades diretas will:
Object.hasOwn(example, "prop"); // returns true
Object.hasOwn(example, "toString"); // returns false
Object.hasOwn(example, "hasOwnProperty"); // returns false

// O operador `in` só retornara tru para propriedades diretas ou herdadas:
"prop" in example; // returns true
"toString" in example; // returns true
"hasOwnProperty" in example; // returns true
```

Iterando sobre as propriedades de um objeto

Para itererar sobre inúmeras propriedades de um objeto, você deve usar:

```
const example = { foo: true, bar: true };
for (const name of Object.keys(example)) {
  // …
}
```

Mas se você precisa utilizar `for...in`, você pode utilziar `Object.hasOwn()` para pular as propriedades herdadas.

```
const example = { foo: true, bar: true };
for (const name in example) {
  if (Object.hasOwn(example, name)) {
    // …
  }
}
```

Verificando se um index existe em um array

Os elementos de um `Array` são definidos como propriedades diretas, você pode utilizar o método `hasOwn()` para verificar se existe um index específico:

```
const fruits = ["Apple", "Banana", "Watermelon", "Orange"];
Object.hasOwn(fruits, 3); // true ('Orange')
Object.hasOwn(fruits, 4); // false - not defined
```

Casos problematicos para hasOwnProperty

Essa seção demonstra `hasOwn()` é imune aos problemas que afetam `hasOwnProperty`. Primeiramente, ele pode ser uasdos em objetos que foram reimplementados `hasOwnProperty()`:

```
const foo = {
  hasOwnProperty() {
    return false;
  },
  bar: "The dragons be out of office",
};

if (Object.hasOwn(foo, "bar")) {
  console.log(foo.bar); //true - reimplementação do hasOwnProperty() não afetara o objeto
}
```

Ele pode ser usado para testar objetos criados utilizando `Object.create(null)`. Esses não são herdados de `Object.prototype`, e assim o `hasOwnProperty()` é inacessível

```
const foo = Object.create(null);
foo.prop = "exists";
if (Object.hasOwn(foo, "prop")) {
  console.log(foo.prop); //true - Funciona independente de como o objeto é criado.
}
```

Veja também

- Polyfill of `Object.hasOwn` in `core-js`
- `Object.hasOwnProperty()`
- Enumerability and ownership of properties
- `Object.getOwnPropertyNames()`
- `for...in`
- `in`
- JavaScript Guide: Inheritance revisited

Object.prototype.hasOwnProperty()

Resumo

O método **`hasOwnProperty()`** retorna um booleano indicando se o objeto possui a propriedade especificada como uma propriedade definida no próprio objeto em questão (ao contrário de uma propriedade herdada).

Sintaxe

```
obj.hasOwnProperty(prop)
```

Parâmetros

`prop`

Uma `String` ou symbol indicando o nome da propriedade a ser verificada.

Valor de Retorno

Um `booleano` indicando se o objeto possui ou não a propriedade especificada como uma propriedade do próprio objeto e que a propriedade não é uma propriedade herdada.

Descrição

Todo objeto descendente de `Object` herda o método `hasOwnProperty`. Este método pode ser usado para determinar se um objeto possui a propriedade especificada como propriedade direta do objeto.

Diferentemente do operador `in`, este método não checa a cadeia prototípica do objeto.

Nota

o método `hasOwnProperty` retorna `true` mesmo se o valor da propridade em questão é `null` ou `undefined`

```
o = new Object();
o.propUm = null;
o.hasOwnProperty('propUm'); // retorna true
o.propDois = undefined;
o.hasOwnProperty('propDois'); // retorna true
```

Exemplos

Usando <code>hasOwnProperty</code> para testar a existência de uma propriedade

O seguinte exemplo determina se o objeto `o` possui uma propriedade com o nome `prop`:

```
o = new Object();
o.hasOwnProperty('prop'); // retorna false
o.prop = 'existe';
o.hasOwnProperty('prop'); // retorna true
```

Propriedades Diretas vs Propriedades Herdadas

O seguinte exemplo diferencia entre propriedade diretas e propriedade herdadas da cadeia prototípica:

```
o = new Object();
o.prop = "existe";
o.hasOwnProperty("prop"); // Retorna true
o.hasOwnProperty("toString"); // Retorna false
o.hasOwnProperty("hasOwnProperty"); // Retorna false
```

Percorrer através das propriedades de um objeto

O seguinte exemplo mostra como percorrer as propriedade de um objeto sem executar as propriedade herdadas.

Vale observar que o loop `for...in` percorre somente itens enumeráveis. Entretanto, o método hasOwnProperty também funciona com propriedades não enumeráveis.

```
var buz = {
  fog: "stack",
};

for (var nome in buz) {
  if (buz.hasOwnProperty(nome)) {
```

```
    alert("this is fog (" + nome + ") for sure. Value: " + buz[nome]);
  } else {
    alert(nome); // toString ou qualquer outra coisa
  }
}
```

Exemplo: <code>hasOwnProperty</code> como propriedade

JavaScript não protege o nome `hasOwnProperty`, assim, se existir a possibilidade do objeto possuir uma propriedade com esse nome, é necessário usar externamente `hasOwnProperty` para se ter o resultado correto:

```
var foo = {
  hasOwnProperty: function () {
    return false;
  },
  bar: "Here be dragons",
};

foo.hasOwnProperty("bar"); // Sempre retorna false

// Usando a propriedade hasOwnProperty de outro objeto e definindo 'this' como foo
({}).hasOwnProperty.call(foo, "bar"); // true

// Também é possível usar hasOwnProperty do objeto
Object.prototype.hasOwnProperty.call(foo, "bar"); // true
```

Observe que neste ultimo caso nenhum novo objeto é criado.

Veja Também

- Enumerability and ownership of properties
- `Object.getOwnPropertyNames()`
- for...in
- `in`
- JavaScript Guide: Inheritance revisted

Object.is()

O método **`Object.is()`** determina se dois valores correspondem ao mesmo valor.

Sintaxe

```
Object.is(value1, value2);
```

Parâmetros

`value1`

O primeiro valor a ser comparado.

`value2`

O segundo valor a ser comparado.

Return value

Um `Boolean` indicando se os dois argumentos possuem o mesmo valor ou não.

Descrição

`Object.is()` determina se dois valores correspondem ao mesmo valor. Dois valores correspondem ao mesmo valor se uma das seguintes condições for verdadeira:

- ambos são `undefined`
- ambos são `null`
- ambos são `true` ou ambos são `false`
- ambos são strings do mesmo comprimento, com os mesmos caracteres
- ambos são o mesmo objeto
- ambos são numéricos e
  - ambos são `+0`
  - ambos são `-0`
  - ambos são `NaN`
  - ou ambos são não-zero e ambos são não `NaN` e ambos correspondem ao mesmo valor

Isso *não é* o mesmo que ser igual de acordo com o operador `==`. O operador `==` aplica diversas coerções para ambos os lados (se eles não correspondem ao mesmo Tipo) antes de testar a igualdade (resultando em comportamentos como a comparação `"" == false` retornar `true`), enquanto `Object.is` não realiza a coerção de nenhum dos valores.

Isso também *não é* o mesmo que ser igual de acordo com o operador `===`. O operador `===` (assim como o operador `==`) trata os valores numéricos `-0` e `+0` como iguais e trata `Number.NaN` como não igual a `NaN`.

Exemplos

```
Object.is("foo", "foo"); // true
Object.is(window, window); // true

Object.is("foo", "bar"); // false
Object.is([], []); // false

var test = { a: 1 };
Object.is(test, test); // true

Object.is(null, null); // true

// Casos especiais
Object.is(0, -0); // false
Object.is(-0, -0); // true
Object.is(NaN, 0 / 0); // true
```

Polyfill para navegadores que não suportam ES6

`Object.is()` é uma adição proposta ao padrão ECMA-262; e como tal, pode não estar presente em todos os navegadores. Você pode contornar essa situação por meio da adição do seguinte código no começo de seus scripts. Isso permitirá a você utilizar `Object.is()`, mesmo quando não houver suporte por parte do navegador.

```
if (!Object.is) {
  Object.is = function (x, y) {
    // Algoritmo para verificar se os valores sao iguais
    if (x === y) {
      // Passos 1-5, 7-10
      // Passos 6.b-6.e: +0 != -0
      return x !== 0 || 1 / x === 1 / y;
    } else {
      // Passo 6.a: NaN == NaN
      return x !== x && y !== y;
    }
  };
}
```

Veja também

- Comparações de igualdade — uma comparação dos três recursos implementados para verificação de igualdade.

Object.isExtensible()

O método **`Object.isExtensible()`** verifica se um objeto pode ser extendido (se é ou não possível adicinar novas propriedades).

Experimente

```
const object1 = {};

console.log(Object.isExtensible(object1));
// Expected output: true

Object.preventExtensions(object1);

console.log(Object.isExtensible(object1));
// Expected output: false
```

Sintaxe

```
Object.isExtensible(obj)
```

Parâmetros

`obj`

O objeto a ser verificado.

Valor de retorno

Um valor booleano (`Boolean`) que indica se o objeto pode ser extendido.

Descrição

Objetos são extensíveis por padrão: novas propriedades podem ser adicionadas, e (em ambientes que suportam `Object.prototype.__proto__`) a propriedade __proto__ pode ser modificada. Um objeto pode ser marcado como não extensível usando `Object.preventExtensions()`, `Object.seal()`, ou `Object.freeze()`.

Exemplos

```
// Novos objetos podem ser extendidos.
var empty = {};
Object.isExtensible(empty); // === true

// ...mas isso pode mudar.
Object.preventExtensions(empty);
Object.isExtensible(empty); // === false
```

```
// Objetos selados, não podem ser extendidos.
var sealed = Object.seal({});
Object.isExtensible(sealed); // === false

// Objetos congelados também não podem ser extendidos.
var frozen = Object.freeze({});
Object.isExtensible(frozen); // === false
```

Notas

No ES5, se o argumento fornecido não for um objeto (um tipo primitivo), isso vai causar um erro do tipo `TypeError`. No ES2015, um argumento que não é um objeto será tratado como um objeto não extensível, simplesmente retornando `false`.

```
Object.isExtensible(1);
// TypeError: 1 is not an object (ES5 code)

Object.isExtensible(1);
// false                        (ES2015 code)
```

See also

- `Object.preventExtensions()`
- `Object.seal()`
- `Object.isSealed()`
- `Object.freeze()`
- `Object.isFrozen()`
- `Reflect.isExtensible()`

Object.isFrozen()

O método **`Object.isFrozen()`** determina se um objeto está frozen.

Experimente

```
const object1 = {
  property1: 42,
};

console.log(Object.isFrozen(object1));
// Expected output: false

Object.freeze(object1);

console.log(Object.isFrozen(object1));
// Expected output: true
```

Sintaxe

```
Object.isFrozen(obj)
```

Parameters

`obj`

O objeto que será verificado.

Return value

Um valor `Boolean` indicando se o objeto está frozen.

Descrição

Um objeto estará frozen se, e apenas se, ele não for extensible, todas as suas propriedades não são configuráveis e todas suas propriedades de dados (propriedades que não são asessores de propriedades com getters ou setters) não podem ser modificadas.

Examples

```
// Um novo objeto é extensível, então ele não está frozen.
Object.isFrozen({}); // === false

// Um objeto vazio que não é extensível
// é vagamente frozen.
var vacuouslyFrozen = Object.preventExtensions({});
Object.isFrozen(vacuouslyFrozen); // === true

// Um novo objeto com uma propriedade também é extensível,
// portanto não está frozen.
var oneProp = { p: 42 };
Object.isFrozen(oneProp); // === false

// Impedir extensões do objeto ainda
// não o torna frozen, pois a propriedade ainda será
```

```
// configurável (e modificável).
Object.preventExtensions(oneProp);
Object.isFrozen(oneProp); // === false

// ...Porém, deletando a propriedade o objeto se tornará
// vagamente frozen.
delete oneProp.p;
Object.isFrozen(oneProp); // === true

// Um objeto não extensível e não modificável,
// mas com uma propriedade configurável não será frozen.
var nonWritable = { e: "plep" };
Object.preventExtensions(nonWritable);
Object.defineProperty(nonWritable, "e", {
  writable: false,
}); // tornar não modificável
Object.isFrozen(nonWritable); // === false

// Alterando a propriedade para não configurável
// tornará o objeto frozen.
Object.defineProperty(nonWritable, "e", {
  configurable: false,
}); // make non-configurable
Object.isFrozen(nonWritable); // === true

// Um objeto não extensível com uma propriedade não configurável
// mas modificável não será frozen.
var nonConfigurable = { release: "the kraken!" };
Object.preventExtensions(nonConfigurable);
Object.defineProperty(nonConfigurable, "release", {
  configurable: false,
});
Object.isFrozen(nonConfigurable); // === false

// Alterando a propriedade para não modificável
// tornará o objeto frozen.
Object.defineProperty(nonConfigurable, "release", {
  writable: false,
});
Object.isFrozen(nonConfigurable); // === true

// Um objeto não extensível com um assessor de propriedade
// configurável não será frozen.
var accessor = {
  get food() {
    return "yum";
  },
};
Object.preventExtensions(accessor);
Object.isFrozen(accessor); // === false

// ...Mas alterando essa propriedade para não configurável
// o objeto se tornará frozen.
Object.defineProperty(accessor, "food", {
  configurable: false,
});
Object.isFrozen(accessor); // === true

// A forma mais fácil para um objeto ser frozen
// é se o método Object.freeze foi usado nele.
var frozen = { 1: 81 };
Object.isFrozen(frozen); // === false
Object.freeze(frozen);
Object.isFrozen(frozen); // === true

// Por definição, um objeto frozen não é extensível.
Object.isExtensible(frozen); // === false

// Também por definição, um objeto frozen estará sealed.
Object.isSealed(frozen); // === true
```

Notas

No ES5, se o argumento deste método não for um objeot (uma primitiva), ele irá lançar um `TypeError`. No ES2015, um argumento que não é objeto será sempre tratado como se fosse um objeto frozen, simplesmente retornando `true`.

```
Object.isFrozen(1);
// TypeError: 1 is not an object (ES5 code)

Object.isFrozen(1);
// true                          (ES2015 code)
```

Veja também

- `Object.freeze()`

- `Object.preventExtensions()`
- `Object.isExtensible()`
- `Object.seal()`
- `Object.isSealed()`

Object.prototype.isPrototypeOf()

O método **`isPrototypeOf()`** checa se um objeto existe em na cadeia de protótipos de um outro objeto.

> **Nota:** `isPrototypeOf()` difere do operador `instanceof`. Na expressão "`objeto instanceof UmaFuncaoQualquer`", a cadeia de protótipos do `objeto` é comparada com `UmaFuncaoQualquer.prototype`, e não com a própria função `UmaFuncaoQualquer`.

Experimente

```
function Foo() {}
function Bar() {}

Bar.prototype = Object.create(Foo.prototype);

const bar = new Bar();

console.log(Foo.prototype.isPrototypeOf(bar));
// Expected output: true
console.log(Bar.prototype.isPrototypeOf(bar));
// Expected output: true
```

Sintaxe

```
prototypeObj.isPrototypeOf(objeto)
```

Parâmetros

`objeto`

Objeto no qual será feito uma busca na cadeia de protótipos.

Retorno

Um `Boolean` indicando se prototypeObj está na cadeia de protótipos do objeto.

Erros possíveis

`TypeError`

Um `TypeError` é mostrado se `prototypeObj` é *undefined* ou *null*.

Descrição

O método `isPrototypeOf()` lhe permite checar se um objeto está ou não na cadeia de protótipos (cadeia hieráquica) de um outro objeto.

Em outras palavras, você pode descobrir se um objeto x (já instanciado) é herdeiro de um objeto y.

Exemplos

Este exemplo demonstra que `Baz.prototype`, `Bar.prototype`, `Foo.prototype` e `Object.prototype` estão na cadeia de protótipos de `baz` , ou seja, baz herda atributos de Baz, Bar e Foo:

```
function Foo() {}
function Bar() {}
function Baz() {}

Bar.prototype = Object.create(Foo.prototype);
Baz.prototype = Object.create(Bar.prototype);

const foo = new Foo();
const bar = new Bar();
const baz = new Baz();

// cadeia de protótipos:
// foo: Foo <- Object
// bar: Bar <- Foo <- Object
// baz: Baz <- Bar <- Foo <- Object
console.log(Baz.prototype.isPrototypeOf(baz)); // true
console.log(Baz.prototype.isPrototypeOf(bar)); // false
console.log(Baz.prototype.isPrototypeOf(foo)); // false
console.log(Bar.prototype.isPrototypeOf(baz)); // true
console.log(Bar.prototype.isPrototypeOf(foo)); // false
console.log(Foo.prototype.isPrototypeOf(baz)); // true
console.log(Foo.prototype.isPrototypeOf(bar)); // true
console.log(Object.prototype.isPrototypeOf(baz)); // true
```

O método`isPrototypeOf()`, junto com o operador `instanceof` vêm a ser útil se você tem um código que só pode funcionar quando estiver lidando com objetos que descendem de uma cadeia de protótipos específicos, por exemplo, para garantir que certos métodos ou propriedades estarão presentes naquele objeto que você precisa.

Por exemplo, checar se o objeto `baz` descende de `Foo.prototype`:

```
if (Foo.prototype.isPrototypeOf(baz)) {
  // fazer algo seguramente
}
```

Veja também

- `instanceof`
- `Object.getPrototypeOf()`
- `Object.setPrototypeOf()`
- `Object.prototype.__proto__`

Object.isSealed()

O método **`Object.isSealed()`** determina se um objeto está selado.

Experimente

```
const object1 = {
  property1: 42,
};

console.log(Object.isSealed(object1));
// Expected output: false

Object.seal(object1);

console.log(Object.isSealed(object1));
// Expected output: true
```

Sintaxe

```
Object.isSealed(obj)
```

Parâmetros

`obj`

O objeto que deverá ser verificado.

Valor retornado

Um `Boolean` indicando se o objeto fornecido está ou não selado.

Descrição

Retorna `true` se o objeto está selado, senão `false`. Um objeto está selado se ele for "não extensible" e se todas as suas propriedades estão como "não configuráveis" e assim sendo "não removíveis" (mas não necessariamente "não escrevíveis").

Exemplos

```
// Objetos não são selados por padrão.
var empty = {};
Object.isSealed(empty); // === false

// Se você fizer um objeto não extensível vazio,
// ele estará vagamente selado.
Object.preventExtensions(empty);
Object.isSealed(empty); // === true

// O mesmo não é verdadeiro em um objeto "não vazio",
// a não ser que todas as suas propriedades sejam "não configuráveis".
var hasProp = { fee: "fie foe fum" };
Object.preventExtensions(hasProp);
Object.isSealed(hasProp); // === false

// Mas torne-os todos "não configuráveis"
// e o objeto se tornará selado.
Object.defineProperty(hasProp, "fee", {
  configurable: false,
});
Object.isSealed(hasProp); // === true

// O jeito mais fácil de selar um objeto, com certeza,
// é Object.seal.
var sealed = {};
Object.seal(sealed);
Object.isSealed(sealed); // === true
```

```
// Um objeto selado é, por definição, não extensível.
Object.isExtensible(sealed); // === false

// Um objeto selado pode estar congelado,
// mas não precisa estar.
Object.isFrozen(sealed); // === true
// (todas as propriedades também não escrevíveis)

var s2 = Object.seal({ p: 3 });
Object.isFrozen(s2); // === false
// ('p' continua "escrevível")

var s3 = Object.seal({
  get p() {
    return 0;
  },
});
Object.isFrozen(s3); // === true
// (somente a configurabilidade importa nos assessores de propriedade)
```

Notas

No ES5, se o argumento para esse método não é um objeto (um primitivo), então ele irá causar um `TypeError`. No ES2015, um argumento que não seja objeto irá ser tratado como se fosse um objeto selado qualquer, simplesmente retornando `true`.

```
Object.isSealed(1);
// TypeError: 1 is not an object (ES5 code)

Object.isSealed(1);
// true                         (ES2015 code)
```

Veja também

- `Object.seal()`
- `Object.preventExtensions()`
- `Object.isExtensible()`
- `Object.freeze()`
- `Object.isFrozen()`

Object.keys()

O método **`Object.keys()`** retorna um array de propriedades enumeraveis de um determinado objeto, na mesma ordem em que é fornecida por um laço `for...in` (a diferença é que um laço for-in enumera propriedades que estejam na cadeia de protótipos).

Sintaxe

```
Object.keys(obj)
```

Parametros

`obj`

O objeto cujas propriedades são enumeráveis.

Descrição

`Object.keys()` retorna um array cujo os elementos são strings correspondentes para a propriedade enumerável encontrada diretamento sobre o objeto. A ordenação das propriedades é a mesma que a dada pelo loop sobre as propriedades do objeto manualmente.

Exemplos

```
var arr = ["a", "b", "c"];
console.log(Object.keys(arr)); // console: ['0', '1', '2']

// array com objeto
var obj = { 0: "a", 1: "b", 2: "c" };
console.log(Object.keys(obj)); // console: ['0', '1', '2']

// array como objeto com ordenação aleatória por chave
var an_obj = { 100: "a", 2: "b", 7: "c" };
console.log(Object.keys(an_obj)); // console: ['2', '7', '100']

// getFoo é uma propriedade que não é enumerável
var my_obj = Object.create(
  {},
  {
    getFoo: {
      value: function () {
        return this.foo;
      },
    },
  },
```

```
);
my_obj.foo = 1;

console.log(Object.keys(my_obj)); // console: ['foo']
```

Se você quiser todas as propriedades, mesmo que não enumeráveis, consulte:`Object.getOwnPropertyNames()`.

Notas

Em ES5, Se o argumento para o método this não é um objeto(um primitivo), em seguida ele irá causar um `TypeError`. Em ES2015, um argumento não-objeto será forçado a um objeto.

```
Object.keys("foo");
// TypeError: "foo" is not an object (ES5 code)

Object.keys("foo");
// ["0", "1", "2"]                  (ES2015 code)
```

Polyfill

Para adicionar suporte Object.keys compatíveis em ambientes mais antigos que não têm suporte nativo para isso, copie o seguinte trecho:

```
// De https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Object/keys
if (!Object.keys) {
  Object.keys = (function () {
    "use strict";
    var hasOwnProperty = Object.prototype.hasOwnProperty,
      hasDontEnumBug = !{ toString: null }.propertyIsEnumerable("toString"),
      dontEnums = [
        "toString",
        "toLocaleString",
        "valueOf",
        "hasOwnProperty",
        "isPrototypeOf",
        "propertyIsEnumerable",
        "constructor",
      ],
      dontEnumsLength = dontEnums.length;

    return function (obj) {
      if (
        typeof obj !== "object" &&
        (typeof obj !== "function" || obj === null)
      ) {
        throw new TypeError("Object.keys chamado de non-object");
      }

      var result = [],
        prop,
        i;

      for (prop in obj) {
        if (hasOwnProperty.call(obj, prop)) {
          result.push(prop);
        }
      }

      if (hasDontEnumBug) {
        for (i = 0; i < dontEnumsLength; i++) {
          if (hasOwnProperty.call(obj, dontEnums[i])) {
            result.push(dontEnums[i]);
          }
        }
      }
      return result;
    };
  })();
}
```

Por favor, note que o código acima inclui chaves não-enumeráveis no IE7 (e talvez IE8), ao passar em um objeto a partir de uma janela diferente.

Para um simples Browser Polyfill, veja Javascript - Object.keys Browser Compatibility.

Veja também

- Enumerability and ownership of properties
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Object.getOwnPropertyNames()`

Object.preventExtensions()

O método **`Object.preventExtensions()`** impede que novas propriedades sejam adicionadas a um objeto (isto é, impede futuras extensões ao objeto).

Syntax

```
Object.preventExtensions(obj)
```

Parâmetros

`obj`

O objeto a tornar-se não-extensível.

Valor de retorno

Um objeto tornado não-extensível.

Descrição

Um objeto é extensível se novas propriedades puderem der adicionadas ao mesmo. `Object.preventExtensions()` marca um objeto como não mais extensível, de forma que este nunca terá novas propriedades além daquelas que o objeto tinha quando foi marcado como não-extensível. Note que as propriedades de um objeto não-extensível, em geral, ainda poderão ser *apagadas*. Tentativas de adicionar novas propriedades a um objeto não-extensível falharão, tanto silenciosamente ou lançando uma exceção `TypeError` (mais comumente, mas não exclusivamente, quando em strict mode).

`Object.preventExtensions()` evita apenas a adição de novas propriedades diretas. Proprieades ainda poderão ser adicionadas ao protótipo do objeto.

É impossível tornar um objeto extensível novamente uma vez que o mesmo tenha se tornado não-extensível.

Exemplos

```
// Object.preventExtensions retorna o objeto
// tornado não-extensível.
var obj = {};
var obj2 = Object.preventExtensions(obj);
obj === obj2; // verdadeiro

// Objetos são extensíveis por padrão...
var empty = {};
Object.isExtensible(empty); // === verdadeiro

// ...mas isso pode ser mudado.
Object.preventExtensions(empty);
Object.isExtensible(empty); // === falso

// Object.defineProperty lança erro quando adiciona-se
// uma nova propriedade a um objeto não-extensível.
var nonExtensible = { removable: true };
Object.preventExtensions(nonExtensible);
Object.defineProperty(nonExtensible, "new", {
  value: 8675309,
}); // lança um TypeError

// No modo restrito, tentar adicionar novas propriedades a
// um objeto não-extensível lança um TypeError.
function fail() {
  "use strict";
  // lança um TypeError
  nonExtensible.newProperty = "FAIL";
}
fail();
```

O protótipo não-extensível de um objeto é imutável:

```
var fixed = Object.preventExtensions({});
// lança um 'TypeError'.
fixed.__proto__ = { oh: "hai" };
```

Notas

No ES5, se o argumento atribuído a este método não for um objeto (for um primitivo), isso causará um erro de tipo `TypeError`. No ES2015, um argumento não-objeto será tratado como se o mesmo fosse um objeto não-extensível comum, simplesmente retornando-o.

```
Object.preventExtensions(1);
// TypeError: 1 não é um objeto (código ES5)

Object.preventExtensions(1);
// 1                           (código ES2015)
```

Veja também

- `Object.isExtensible()`
- `Object.seal()`
- `Object.isSealed()`
- `Object.freeze()`
- `Object.isFrozen()`

- `Reflect.preventExtensions()`

Object.prototype.propertyIsEnumerable()

O método **`propertyIsEnumerable()`** retorna um booleano indicando quando a propriedade especificada é enumerável e é a propriedade do próprio objeto

Experimente

```
const object1 = {};
const array1 = [];
object1.property1 = 42;
array1[0] = 42;

console.log(object1.propertyIsEnumerable("property1"));
// Expected output: true

console.log(array1.propertyIsEnumerable(0));
// Expected output: true

console.log(array1.propertyIsEnumerable("length"));
// Expected output: false
```

Sintaxe

```
obj.propertyIsEnumerable(prop)
```

Parâmetros

`prop`

O nome da propriedade para teste

Valor de Retorno

O `Boolean` indicando se a propriedade especificada é enumeravel e é a propriedade do objeto

Descrição

Every object has a `propertyIsEnumerable` method. This method can determine whether the specified property in an object can be enumerated by a `for...in` loop, with the exception of properties inherited through the prototype chain. If the object does not have the specified property, this method returns `false`.

Exemplos

O uso basico de <code>propertyIsEnumerable</code>

O exemplos a seguir mostram o uso de `propertyIsEnumerable` em um objeto e um array:

```
var o = {};
var a = [];
o.prop = "is enumerable";
a[0] = "is enumerable";

o.propertyIsEnumerable("prop"); // returns true
a.propertyIsEnumerable(0); // returns true
```

Objetos User-defined vs. built-in

Os exemplos a seguir demostram a enumerabilidade da propriedade user-defined vs. built-in :

```
var a = ["is enumerable"];

a.propertyIsEnumerable(0); // returns true
a.propertyIsEnumerable("length"); // returns false

Math.propertyIsEnumerable("random"); // returns false
this.propertyIsEnumerable("Math"); // returns false
```

Propriedade Direct vs. inherited

```
var a = [];
a.propertyIsEnumerable("constructor"); // returns false

function firstConstructor() {
  this.property = "is not enumerable";
}

firstConstructor.prototype.firstMethod = function () {};

function secondConstructor() {
  this.method = function () {
    return "is enumerable";
  };
}
```

```
secondConstructor.prototype = new firstConstructor();
secondConstructor.prototype.constructor = secondConstructor;

var o = new secondConstructor();
o.arbitraryProperty = "is enumerable";

o.propertyIsEnumerable("arbitraryProperty"); // returns true
o.propertyIsEnumerable("method"); // returns true
o.propertyIsEnumerable("property"); // returns false

o.property = "is enumerable";

o.propertyIsEnumerable("property"); // returns true

// These return false as they are on the prototype which
// propertyIsEnumerable does not consider (even though the last two
// are iteratable with for-in)
o.propertyIsEnumerable("prototype"); // returns false (as of JS 1.8.1/FF3.6)
o.propertyIsEnumerable("constructor"); // returns false
o.propertyIsEnumerable("firstMethod"); // returns false
```

Veja também

- Enumerability and ownership of properties
- `for...in`
- `Object.keys()`
- `Object.defineProperty()`

Object.prototype.__proto__

> **Aviso: Cuidado:** Alterando o `[[Prototype]]` de um objeto é, pela natureza que as engines do Javascript modernos otimizam os acessos à propriedades, uma operação muito lenta, em ***TODOS*** os mecanismos browsers e JavaScript. Os efeitos no desempenho de alteração na herança são sutis e distantes, e não se limitam simplesmente ao tempo gasto em na declaração `obj.__proto__ = ...`, mas podem se estender para ***qualquer*** código que tenha acesso a ***qualquer*** objeto cujo `[[Prototype]]` foi alterado. Se você se preocupa com desempenho, evite configurar o `[[Prototype]]` de um objeto. Ao invés disso, crie um novo objeto com o `[[Prototype]]` desejado usando `Object.create()`.

> **Aviso: Cuidado:** Enquanto `Object.prototype.__proto__` é suportado hoje em dia em quase todos os navegadores, a existência e o comportamento exato foram padronizados na especificação ECMAScript 2015 como um recurso legado para assegurar compatibilidade com os navegadores. Para melhor suporte, recomenda-se que apenas `Object.getPrototypeOf()` seja usado em vez disso.

A propriedade `__proto__` de `Object` é uma propriedade de acesso (uma função getter e uma setter) que expõe o interno `[[Prototype]]` (ou um objeto ou `null`) de um objeto o qual é acessado.

O uso de `__proto__` é controverso, e foi desencorajado. Nunca foi incluído originalmente na especificação do idioma EcmaScript, mas os navegadores modernos decidiram implementá-lo de qualquer maneira. Somente recentemente, a propriedade `__proto__` foi padronizada na especificação de linguagem ECMAScript 2015 para navegadores para garantir compatibilidade, e então ser suportada no futuro. É obsoleta a favor de `Object.getPrototypeOf`/`Reflect.getPrototypeOf` e `Object.setPrototypeOf`/`Reflect.setPrototypeOf` (embora ainda, definir `[[Prototype]]` é uma operação lenta que deve ser evitada se o desempenho for uma preocupação).

A propriedade `__proto__` também pode ser usada em uma definição literal de objeto para definir o objeto `[[Prototype]]` na criação, como uma alternativa para `Object.create()`. Veja: object initializer / literal syntax.

Sintaxe

```
var shape = {};
var circle = new Circle();

// Define o objeto prototype.
// OBSOLETO. Isto é somente exemplo. NÃO FAÇA ISSO em código real.
shape.__proto__ = circle;

// Retorna o objeto prototype
console.log(shape.__proto__ === circle); // true
```

```
var shape = function () {};
var p = {
  a: function () {
    console.log("aaa");
  },
};
shape.prototype.__proto__ = p;

var circle = new shape();

circle.a(); //aaa

console.log(shape.prototype === circle.__proto__); //true

//ou
```

```
var shape = function () {};
var p = {
  a: function () {
    console.log("a");
  },
};

var circle = new shape();
circle.__proto__ = p;

circle.a(); //  a

console.log(shape.prototype === circle.__proto__); //false

//ou

function test() {}
test.prototype.myname = function () {
  console.log("myname");
};
var a = new test();

console.log(a.__proto__ === test.prototype); //true

a.myname(); //myname

//ou

var fn = function () {};
fn.prototype.myname = function () {
  console.log("myname");
};

var obj = {
  __proto__: fn.prototype,
};

obj.myname(); //myname
```

Nota: são dois underscores(underlines), seguidos de cinco caracteres "proto", seguidos por mais dois underscores(underlines).

Descrição

A função getter de `__proto__` expõe o valor interno de `[[Prototype]]` de um objeto. Para objetos criado usando um objeto literal, este valor é `Object`. Para os objetos criados usando literais de matrizes, esse valor é `Array`. Para funções, esse valor é `Function`. Para objeto criados usando `new fun`, onde `fun` é uma função construtora built-in fornecida pelo JavaScript (`Array`, `Boolean`, `Date`, `Number`, `Object`, `String`, e assim por diante — incluindo novos construtores adicionados como evolução do JavaScript), este valor é sempre `fun.prototype`. Para objetos criados usando `new fun`, onde `fun` é uma função definida em um script, esse valor é o valor de `fun.prototype`. (Ou seja, se o construtor não retornou um outro objeto explicitamente, ou o `fun.prototype` foi reatribuído desde que a instância foi criada).

O setter `__proto__` permite ao `[[Prototype]]` de um objeto sejá mutável. O objeto deve ser extensível de acordo com `Object.isExtensible()`: se não for, um erro `TypeError` é emitido. O valor fornecido deve ser um objeto ou `null`. Fornecer qualquer outro valor não fará nada.

Para entender como os prototypes são usados para herança, veja o artigo:Inheritance and the prototype chain.

A propriedade `__proto__` é simplesmente uma propriedade acessora `Object` consistindo de uma função getter e setter. Um acesso de propriedade para `__proto__` que eventualmente consulte `Object` irá encontrar esta propriedade, mas um acesso que não consulta `Object` não a encontrará. Se alguma outra propriedade `__proto__` for encontrada antes de consultar `Object`, essa propriedade irá ocultar a que encontrou `Object`.

Notas de compatibilidade

Enquanto a especificação ECMAScript 2015 dita que o suporte para `__proto__` é requerido *somente* para navegadores (apesar de ser normativo), outros ambientes podem suportar também para uso legado.

Veja também

- `Object.prototype.isPrototypeOf()`
- `Object.getPrototypeOf()`
- `Object.setPrototypeOf()`

Object.seal()

O método **`Object.seal()`** sela um Objeto, evitando que novas propriedades sejam adicionadas à ele e marcando todas as propriedades existentes como não configuráveis. Valores das propriedades atuais ainda podem ser alterados desde que essas propriedades sejam graváveis (writable).

Sintaxe

```
Object.seal(obj)
```

Parâmetros

`obj`

O Objeto que deve ser selado.

Valor de retorno

O Objeto sendo selado.

Descrição

Por padrão, objetos são extensible (novas propriedades podem ser adicionadas à eles). Selar um objeto evita que novas propriedades sejam adicionadas e marca todas as propriedades existentes como não configuráveis. Isto tem o efeito de tornar as propriedades no objeto fixas e imutáveis. Tornando todas as propriedades não configuráveis também evita que as mesmas sejam convertidas de propriedades de dados para propriedades de acesso e vice-versa, mas não evita que os valores das propriedades de dados sejam alterados. A tentativa de deletar ou adicionar propriedades à um objeto selado, ou converter uma propriedade de dado para uma propriedade de acesso ou vice-versa, irá falhar, seja silenciosamente como jogando o erro `TypeError` (mais comumente, mas não exclusivamente, quando em modo rigoroso strict mode de código).

A cadeia de prototipação permanece intocada. Entretanto, a propriedade `Object.prototype.__proto__` é selada também.

Retorna a referência ao Objeto passado.

Exemplos

```
var obj = {
  prop: function () {},
  foo: "bar",
};

// Novas propriedades podem ser adicionadas, propriedades existentes podem ser alteradas ou removidas.
obj.foo = "baz";
obj.lumpy = "woof";
delete obj.prop;

var o = Object.seal(obj);

o === obj; // true
Object.isSealed(obj); // === true

// Alterar o valor das propriedades em um objeto selado ainda funciona.
obj.foo = "quux";

// Mas você não pode converter propriedades de dados em propriedades de acesso, e vice-versa.
Object.defineProperty(obj, "foo", {
  get: function () {
    return "g";
  },
}); // throws a TypeError

// Agora quaisquer mudanças, que não sejam aos valores da das propriedades, irão falhar.
obj.quaxxor = "the friendly duck"; // silently doesn't add the property
delete obj.foo; // silently doesn't delete the property

// e em modo rigoroso (strict mode) tais tentativas irão jogar erros do tipo TypeErrors.
function fail() {
  "use strict";
  delete obj.foo; // throws a TypeError
  obj.sparky = "arf"; // throws a TypeError
}
fail();

// Tentativas através do Object.defineProperty também irão falhar.
Object.defineProperty(obj, "ohai", { value: 17 }); // lança um erro do tipo TypeError
Object.defineProperty(obj, "foo", { value: "eit" }); // altera o valor da propriedade existente
```

Notas

No ES5, se o argumento passado à este método não é um objeto (primitivo) , irá causar um erro `TypeError`. No ES6, qualquer argumento que não seja um objeto será tratado como se fosse um objeto ordinário selado e simplesmente irá retorná-lo

```
Object.seal(1);
// TypeError: 1 não é um Objeto (código ES5)

Object.seal(1);
// 1                            (código ES6)
```

Veja também

- `Object.isSealed()`
- `Object.preventExtensions()`
- `Object.isExtensible()`
- `Object.freeze()`
- `Object.isFrozen()`

Object.setPrototypeOf()

O método **`Object.setPrototypeOf()`** configura o 'prototype' (i.e., a propriedade interna `[[Prototype]]`) de um objeto específico para outro objeto ou `null`.

> **Aviso: Atenção:** Mudar o `[[Prototype]]` de um objeto é, pela natureza de como os modernos mecanismos JavaScript otimizam os acessos de propriedade, uma operação muito lenta, em ***todos*** navegadores e mecanismos JavaScript. O efeito sobre o desempenho de alterar a herança são sutis e vastas e não se limitam simplesmente ao tempo gasto em `obj.__proto__ = ...` statement, mas pode estender a ***qualquer*** código que tem acesso a ***qualquer*** objeto cujo `[[Prototype]]` foi alterado. Se você se preocupa com desempenho, você deveria evitar configurar o `[[Prototype]]` de um objeto. Em vez disso, crie um novo objeto com o `[[Prototype]] desejado usando` `Object.create()`.

Sintaxe

```
Object.setPrototypeOf(obj, prototype);
```

Parâmetros

`obj`

O objeto que deve ter seu 'prototype' definido.

`prototype`

O novo 'prototype' do objeto (um objeto ou `null`).

Valor de retorno

O objeto especificado.

Descrição

Gera uma exceção `TypeError` se o objeto cujo `[[Prototype]]` é para ser modificado não for extensível de acordo com `Object.isExtensible()`. Não faz nada se o parâmetro `'prototype'` não for um objeto ou `null` (ex., número, texto, boleano, ou `undefined`). Caso contrário, este método muda o `[[Prototype]]` do `obj` para um novo valor.

`Object.setPrototypeOf() é uma especificação` ECMAScript 2015. É geralmente considerada a maneira correta de definir o 'prototype' de um objeto, em relação à propriedade mais controversa `Object.prototype.__proto__`.

Exemplos

```
var dict = Object.setPrototypeOf({}, null);
```

Polyfill

Usando a propriedade mais antiga `Object.prototype.__proto__`, nós podemos facilmente definir `Object.setPrototypeOf` se já não estiver disponível:

```
// Funciona somente no Chrome e FireFox, não funciona no IE:
Object.setPrototypeOf =
  Object.setPrototypeOf ||
  function (obj, proto) {
    obj.__proto__ = proto;
    return obj;
  };
```

Adicionando 'Prototype' em cadeia

Uma combinação de `Object.getPrototypeOf()` e `Object.prototype.__proto__` permite anexar toda uma cadeia de 'prototype' a um novo objeto 'prototype':

```
/**
 *** Object.appendChain(@object, @prototype)
 *
 * Acrescente o primeiro 'prototype' não nativo de uma cadeia a um novo 'prototype'.
 * Retorna @object (se for um valor primitivo, será transformado em um objeto).
 *
 *** Object.appendChain(@object [, "@arg_name_1", "@arg_name_2", "@arg_name_3", "..."], "@function_body")
 *** Object.appendChain(@object [, "@arg_name_1, @arg_name_2, @arg_name_3, ..."], "@function_body")
 *
 * Adicione o primeiro 'prototype' não nativo de uma cadeia ao objeto nativo Function.prototype, então anexar a nova função
 * Function(["@arg"(s)], "@function_body") àquela cadeia.
 * Retorna a função.
 *
 **/

Object.appendChain = function (oChain, oProto) {
  if (arguments.length < 2) {
    throw new TypeError("Object.appendChain - Argumentos insuficientes");
  }
  if (typeof oProto !== "object" && typeof oProto !== "string") {
    throw new TypeError(
      "segundo argumento de Object.appendChain deve ser um objeto ou uma string",
    );
  }

  var oNewProto = oProto,
    oReturn =
      (o2nd =
```

```
      oLast =
        oChain instanceof this ? oChain : new oChain.constructor(oChain));

  for (
    var o1st = this.getPrototypeOf(o2nd);
    o1st !== Object.prototype && o1st !== Function.prototype;
    o1st = this.getPrototypeOf(o2nd)
  ) {
    o2nd = o1st;
  }

  if (oProto.constructor === String) {
    oNewProto = Function.prototype;
    oReturn = Function.apply(null, Array.prototype.slice.call(arguments, 1));
    this.setPrototypeOf(oReturn, oLast);
  }

  this.setPrototypeOf(o2nd, oNewProto);
  return oReturn;
};
```

Exemplos

Primeiro exemplo: Adicionar uma cadeia a um 'prototype'

```
function Mammal() {
  this.isMammal = "yes";
}

function MammalSpecies(sMammalSpecies) {
  this.species = sMammalSpecies;
}

MammalSpecies.prototype = new Mammal();
MammalSpecies.prototype.constructor = MammalSpecies;

var oCat = new MammalSpecies("Felis");

console.log(oCat.isMammal); // 'yes'

function Animal() {
  this.breathing = "yes";
}

Object.appendChain(oCat, new Animal());

console.log(oCat.breathing); // 'yes'
```

Segundo exemplo: Transformar um valor primitivo em uma instância de seu construtor e anexar sua cadeia a um 'prototype'

```
function MySymbol() {
  this.isSymbol = "yes";
}

var nPrime = 17;

console.log(typeof nPrime); // 'number'

var oPrime = Object.appendChain(nPrime, new MySymbol());

console.log(oPrime); // '17'
console.log(oPrime.isSymbol); // 'yes'
console.log(typeof oPrime); // 'object'
```

Terceiro exemplo: Anexar uma cadeia ao objeto Function.prototype e anexar uma nova função a essa cadeia

```
function Person(sName) {
  this.identity = sName;
}

var george = Object.appendChain(
  new Person("George"),
  'console.log("Hello guys!!");',
);

console.log(george.identity); // 'George'
george(); // 'Hello guys!!'
```

Veja também

- `Reflect.setPrototypeOf()`
- `Object.prototype.isPrototypeOf()`
- `Object.getPrototypeOf()`
- `Object.prototype.__proto__`

Object.prototype.toLocaleString()

O método **`toLocaleString()`** retorna uma cadeia de caracteres (*string*) representando o objeto. Este método é feito para ser sobrescrito por objetos derivados para propósitos de localização específica.

Experimente

```
const date1 = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

console.log(date1.toLocaleString("ar-EG"));
// Expected output: "٢٠‏/١٢‏/٢٠١٢ ٣:٠٠:٠٠ ص"

const number1 = 123456.789;

console.log(number1.toLocaleString("de-DE"));
// Expected output: "123.456,789"
```

Sintaxe

```
obj.toLocaleString()
```

Valor de retorno

Uma *string* representando o objeto.

Descrição

`toLocaleString` do `Object` retorna o resultado da chamada `toString()`.

Esta função é provida para dar aos objetos um método `toLocaleString` genérico, mesmo que nem todos os usem. Veja a lista abaixo.

Objetos sobrescrevendo toLocaleString

- `Array`: `Array.prototype.toLocaleString()`
- `Number`: `Number.prototype.toLocaleString()`
- `Date`: `Date.prototype.toLocaleString()`
- `TypedArray`: `TypedArray.prototype.toLocaleString()`
- `BigInt`: `BigInt.prototype.toLocaleString()`

Exemplos

Sobrescrita do Array toLocaleString()

Em objetos `Array`, `toLocaleString()` pode ser usado para imprimir valores do arranjo como uma *string*, opcionalmente com identificadores de localização específicos (como símbolos monetários) atrelados a eles:

Por exemplo:

```
const testArray = [4, 7, 10];

let euroPrices = testArray.toLocaleString("fr", {
  style: "currency",
  currency: "EUR",
});
// "4,00 €,7,00 €,10,00 €"
```

Sobrescrita do Date toLocaleString()

Em objetos `Date`, `toLocaleString()` é usado para imprimir datas mais apropriadas para localizações específicas:

Por exemplo:

```
const testDate = new Date(Date.now());
// "Date Fri May 29 2020 18:04:24 GMT+0100 (British Summer Time)"

let deDate = testDate.toLocaleString("de");
// "29.5.2020, 18:04:24"

var frDate = testDate.toLocaleString("fr");
//"29/05/2020 à 18:04:24"
```

Sobrescrita de Number toLocaleString()

Em objetos `Number`, `toLocaleString()` é usado para imprimir números para localizações específicas, e.g. com os corretos separadores:

Por exemplo:

```
const testNumber = 2901234564;
// "2901234564"

let deNumber = testNumber.toLocaleString("de");
// "2.901.234.564"
```

```
let frNumber = testNumber.toLocaleString("fr");
// "2 901 234 564"
```

Veja também

- `Object.prototype.toString()`

Object.prototype.toString()

Sumário

O método `toString()` retorna uma string representando o objeto.

Experimente

```
function Dog(name) {
  this.name = name;
}

const dog1 = new Dog("Gabby");

Dog.prototype.toString = function dogToString() {
  return `${this.name}`;
};

console.log(dog1.toString());
// Expected output: "Gabby"
```

Sintaxe

```
obj.toString();
```

Descrição

Todo objeto possui um método `toString()` que é chamado automaticamente quando o objeto precisa ser representado como um valor em texto ou quando o objeto é referenciado de uma maneira que requeira uma string. Por padrão, o método `toString()` é herdado de todo objeto descendente de `Object`. Se e o método não é sobrescrito em um objeto personalizado, `toString()` retorna "[object *type*]", onde `type` é o tipo do objeto. O código a seguir ilustra isso:

```
var o = new Object();
o.toString(); // retorna [object Object]
```

> **Nota:** Starting in JavaScript 1.8.5 `toString()` called on `null` returns `[object Null]`, and `undefined` returns `[object Undefined]`, as defined in the 5th Edition of ECMAScript and a subsequent Errata. See Using toString to detect object type.

Examples

Exemplo: Sobrepondo o método inicial <code>toString</code>

Você pode criar uma função para ser chamada no lugar do método `toString()`. O método `toString()` não requer parâmetros e deve retornar uma string. O método `toString()` criado por você pode ter o valor que quiser, mas será mais útil se usar informações do objeto.

O código abaixo define o objeto `Dog` e cria `theDog`, um objeto do tipo `Dog`:

```
function Dog(name, breed, color, sex) {
  this.name = name;
  this.breed = breed;
  this.color = color;
  this.sex = sex;
}

theDog = new Dog("Gabby", "Lab", "chocolate", "female");
```

Se você chamar o método `toString()` neste objeto, ele retornará o valor original herdado de `Object`:

```
theDog.toString(); // returns [object Object]
```

O código abaixo cria e faz com que `dogToString()` sobrescreva o `toString()` original. Esta função gera uma string contendo **name, breed, color, and sex** do objeto, na forma de "`propriedade = valor;`".

```
Dog.prototype.toString = function dogToString() {
  var ret =
    "Dog " +
    this.name +
    " is a " +
    this.sex +
    " " +
    this.color +
    " " +
    this.breed;
```

```
  return ret;
};
```

Usando este código, toda vez que `theDog` for usado em um texto (string), JavaScript automaticamente chamará a função `dogToString()`, a qual retornará:

```
Dog Gabby is a female chocolate Lab
```

Exemplo: Usando <code>toString()</code> para detectar a classe do objeto

`toString()` pode ser usado com qualquer objeto e permite que você determine sua classe. Para usar `Object.prototype.toString()` com qualquer objeto, deverá chamar `Function.prototype.call()` ou `Function.prototype.apply()` nele, passando o objeto que quer inspecionar como o primeiro parâmetro, chamado `thisArg`.

```
var toString = Object.prototype.toString;

toString.call(new Date()); // [object Date]
toString.call(new String()); // [object String]
toString.call(Math); // [object Math]

// Since JavaScript 1.8.5
toString.call(undefined); // [object Undefined]
toString.call(null); // [object Null]
```

Veja também

- `Object.prototype.toSource()`
- `Object.prototype.valueOf()`

Object.prototype.valueOf()

O método **`valueOf()`** retorna o valor primitivo do objeto especificado.

Experimente

```
function MyNumberType(n) {
  this.number = n;
}

MyNumberType.prototype.valueOf = function () {
  return this.number;
};

const object1 = new MyNumberType(4);

console.log(object1 + 3);
// Expected output: 7
```

Sintaxe

```
object.valueOf()
```

Valor retornado

O valor primitivo do objeto especificado.

Descrição

JavaScript chama o método `valueOf` para converter um objeto em um valor primitivo. Você raramente precisará chamar o método `valueOf` por ele mesmo; O JavaScript chamará ele automaticamente quando encontrar um objeto onde um valor primitivo for esperado.

Por padrão, o método `valueOf` é herdado por cada objeto descendente de `Object`. Todo núcleo embutido do objeto sobrescreve esse método para retornar um valor apropriado. Se um objeto não tem um valor primitivo, `valueOf` retorna o próprio objeto.

Você pode usar `valueOf` dentro do seu próprio código para converter um objeto embutido, em um valor primitivo. Quando você criar um objeto customizado, você pode sobrescrever `Object.prototype.valueOf()` para chamar um método customizado ao invés do método padrão `Object`.

Sobrescrevendo <code>valueOf</code> para objetos customizados

Você pode criar uma função para ser chamada no lougar do método padrão `valueOf`. Sua função não pode ter nenhum argumento.

Suponha que você tem um tipo de objeto `MyNumberType` e você quer criar um método `valueOf` para ele. O código a seguir atribui uma função definida por usuário para o método `valueOf` desse objeto:

```
MyNumberType.prototype.valueOf = function () {
  return customPrimitiveValue;
};
```

Com o código anterior no lugar, a qualquer hora um objeto do tipo `MyNumberType` é usado em um contexto onde deve ser representado como um valor primitivo, o JavaScript chama automaticamente a função definida no código anterior.

Um método `valueOf` de um objeto é geralmente chamado pelo JavaScript, mas você pode chamá-lo se quiser da seguinte maneira:

```
myNumberType.valueOf();
```

> **Nota:** Objetos em contexto de string convertidos através do método `toString()`, o que é diferente de objetos `String` convertendo para string primiriva utlizando `valueOf`. Todos os objetos têm uma conversão string, somente se "`[object type]`". Mas muitos objetos não convertem para number, boolean, or function.

Exemplos

Usando <code>valueOf</code>

```
function MyNumberType(n) {
  this.number = n;
}

MyNumberType.prototype.valueOf = function () {
  return this.number;
};

var myObj = new MyNumberType(4);
myObj + 3; // 7
```

Veja também

- `Object.prototype.toString()`
- `parseInt()`
- `Symbol.toPrimitive`

Object.values()

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

O método **`Object.values()`** retorna um array com os valores das propriedades de um dado objeto, na mesma ordem provida pelo `for...in` laço (sendo a diferença que o laço for-in também enumera as propriedades na cadeia prototype).

Sintaxe

```
Object.values(obj)
```

Parâmetros

`obj`

O objeto cujos valores das propriedades enumeradas serão retornados.

Valor de retorno

Um array contendo os valores das propriedades enumeradas do dado objeto.

Descrição

`Object.values()` retorna um array cujos elementos são os valores das propriedades enumeradas encontradas no objeto. A ordem das propriedades é a mesma que a dada pelo laço sobre os valores da propriedade do objeto manualmente.

Exemplos

```
var obj = { foo: "bar", baz: 42 };
console.log(Object.values(obj)); // ['bar', 42]

// array como objeto
var obj = { 0: "a", 1: "b", 2: "c" };
console.log(Object.values(obj)); // ['a', 'b', 'c']

// array como objeto com ordenação de chave aleatória
var an_obj = { 100: "a", 2: "b", 7: "c" };
console.log(Object.values(an_obj)); // ['b', 'c', 'a']

// getFoo é a propriedade a qual não é enumerável
var my_obj = Object.create(
  {},
  {
    getFoo: {
      value: function () {
        return this.foo;
      },
    },
  },
);
my_obj.foo = "bar";
console.log(Object.values(my_obj)); // ['bar']
```

```
// argumento não-objeto será coagido num objeto
console.log(Object.values("foo")); // ['f', 'o', 'o']
```

Biblioteca de Suporte

`Para adicionar compatibilidade ao suporte de Object.values` em ambientes antigos que nativamente não o suportam, você pode encontrar uma biblioteca de suporte nos repositórios tc39/proposal-object-values-entries ou no es-shims/Object.values.

Ver também

- Enumerability and ownership of properties
- `Object.keys()`
- `Object.entries()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Object.getOwnPropertyNames()`

parseFloat()

A função `parseFloat` analisa um argumento (convertendo-o para uma string primeiro caso necessário) e retorna um número de ponto flutuante (número decimal).

Experimente

```
function circumference(r) {
  return parseFloat(r) * 2.0 * Math.PI;
}

console.log(circumference(4.567));
// Expected output: 28.695307297889173

console.log(circumference("4.567abcdefgh"));
// Expected output: 28.695307297889173

console.log(circumference("abcdefgh"));
// Expected output: NaN
```

Sintaxe

```
parseFloat(string);
```

Parâmetros

`string`

> O valor para analisar. Se este argumento não é uma string, então ele é convertido para uma, usando a operação abstrata `ToString`. Espaços em branco são ignorados.

Retorna

Um número de ponto flutuante da `string` dada. Ou `NaN` quando o primeiro carácter, diferente de um espaço em branco, não pôde ser convertido para um número

Descrição

`parseFloat` é uma função top-level e não está associada a nenhum objeto.

`parseFloat` analisa um argumento string, e retorna um numero de ponto flutuante. Se ele encontrar um carácter diferente de um sinal (+ ou -), numeral (0-9), um ponto decimal, ou um expoente, ele retorna o valor até esse ponto e ignora esse caractere e todos os caracteres seguintes. Espaços a direita e a esquerda são permitidos.

Se o primeiro carácter não puder ser convertido para um número, `parseFloat` retorna `NaN`.

Para propósitos aritméticos, o valor `NaN` não é um número de qualquer raiz. Você pode chamar a função `isNaN` para determinar se o resultado do `parseFloat` é `NaN`. Se `NaN` for passado em operações aritméticas, a operação também retornará `NaN`.

`parseFloat` também pode analisar e retornar o valor `Infinity`. Você pode usar a função `isFinite` para determinar se a função é um número finito (not `Infinity`, `-Infinity`, ou `NaN`).

Exemplos

<code>parseFloat</code> retornando um número

O exemplo a seguir sempre retorna `3.14`:

```
parseFloat("3.14");
parseFloat("314e-2");
parseFloat("0.0314E+2");
parseFloat("3.14more non-digit characters");
```

<code>parseFloat</code> retornando <code>NaN</code>

O exemplo a seguir retorna `NaN`

```
parseFloat("FF2");
```

Uma função de análise estrita

As vezes é útil ter uma maneira mais rigorosa para analisar valores float, expressões regulares podem ajudar:

```
var filterFloat = function (value) {
  if (/^(\-|\+)?([0-9]+(\.[0-9]+)?|Infinity)$/.test(value))
    return Number(value);
  return NaN;
};

console.log(filterFloat("421")); // 421
console.log(filterFloat("-421")); // -421
console.log(filterFloat("+421")); // 421
console.log(filterFloat("Infinity")); // Infinity
console.log(filterFloat("1.61803398875")); // 1.61803398875
console.log(filterFloat("421e+0")); // NaN
console.log(filterFloat("421hop")); // NaN
console.log(filterFloat("hop1.61803398875")); // NaN
```

Observe que este código é somente um exemplo; ele não aceita números válidos, tais como 1. ou 0,5.

Veja também

- `parseInt()`
- `Number.parseFloat()`
- `Number.parseInt()`
- `isNaN()`

parseInt()

Sumário

A função **`parseInt()`** analisa um argumento `string` e retorna um inteiro na `base` especificada.

Sintaxe

```
parseInt(string, base);
```

Parâmetros

`string`

> O valor a analisar. Se o argumento `string` não for uma string, então o valor é convertido para uma string (utilizando a operação abstrata `ToString`). Os espaços em branco na `string` são ignorados.

`base`

> Um inteiro entre `2` e `36` que representa a *base* da `string` (sistemas numéricos matemáticos) mencionada no parâmetro anterior. Especifique `10` para o sistema numérico decimal comumente usado por humanos. **Sempre especifique este parâmetro** para eliminar confusão do leitor e para garantir o comportamento esperado. Implementações diferentes produzem resultados diferentes quando `base` não é especificado, normalmente assumindo o valor como `10`.

Valor de retorno

Um número inteiro analisado a partir da string fornecida. Se o primeiro caracter não puder ser convertido para um número, `NaN` é retornado.

Descrição

A função `parseInt` converte seu primeiro argumento para uma string, analisa, e retorna um inteiro ou `NaN`. Se não `NaN`, o valor retornado será a representação decimal inteira do primeiro argumento obtido como um número na base especificada. Por exemplo, uma*base* 10 indica para converter de um número decimal, 8 octal, 16 hexadecimal, e assim por diante. Para bases acima de `10`, as letras do alfabeto indicam numerais maiores que `9`. Por exemplo, para números hexadecimais (base 16), `A` até `F` são usados.

Se `parseInt` encontrar um caracter que não seja um numeral na base especificada, ele o ignora e a todos os caracteres subsequentes e retorna o valor inteiro analisado até aquele ponto. `parseInt` trunca números para valores inteiros. Espaços no início e fim são permitidos.

Se *base* é `undefined` ou 0 (ou ausente), JavaScript assume o seguinte:

- Se a `string` de entrada começa com "0x" ou "0X", a *base* é 16 (hexadecimal) e o restante da string é analisado.
- Se a `string` de entrada começa com "0", a *base* é oito (octal) ou 10 (decimal). Exatamente qual base é escolhida é dependente da implementação. O ECMAScript 5 especifica que 10 (decimal) seja utilizado, mas nem todos os browsers suportam isso ainda. Por essa razão **sempre especifique uma base quando estiver usando `parseInt`**.
- Se a `string` de entrada começa com qualquer outro valor, a base é 10 (decimal).

Se o primeiro caracter não puder ser convertido para um número, `parseInt` retorna `NaN`.

Para propósitos aritméticos, o valor `NaN` não é um número em qualquer base. Você pode chamar a função `isNaN` para determinar se o resultado de `parseInt` é `NaN`. Se `NaN` for passado em operações aritméticas, o resultado da operação também será `NaN`.

Para converter um número para seu literal string em uma base específica use `intValue.toString(base)`.

Exemplos

Exemplo: Usando <code>parseInt</code>

Os seguintes exemplos sempre retornam **15**:

```
parseInt(" 0xF", 16);
parseInt(" F", 16);
parseInt("17", 8);
parseInt(021, 8);
parseInt("015", 10);
parseInt(15.99, 10);
parseInt("FXX123", 16);
parseInt("1111", 2);
parseInt("15*3", 10);
parseInt("15e2", 10);
parseInt("15px", 10);
parseInt("12", 13);
```

Os seguintes exemplos sempre retornam **NaN**:

```
parseInt("Hello", 8); // Não é realmente um número
parseInt("546", 2);   // Dígitos não são válidos para representações binárias
```

Os seguintes exemplos sempre retornam **-15**:

```
parseInt("-F", 16);
parseInt("-0F", 16);
parseInt("-0XF", 16);
parseInt(-15.1, 10)
parseInt(" -17", 8);
parseInt(" -15", 10);
parseInt("-1111", 2);
parseInt("-15e1", 10);
parseInt("-12", 13);
```

O seguinte exemplo retorna **224**:

```
parseInt("0e0", 16);
```

Interpretação octal sem informar a base

Embora desencorajado pelo ECMAScript 3 e proibido pelo ECMAScript 5, muitas implementações interpretam uma string numérica começando com um `0` como octal. O exemplo abaixo pode ter um resultado octal, ou ele pode ter um resultado decimal. **Sempre especifique uma base para evitar este comportamento não confiável.**

```
parseInt("0e0"); // 0
parseInt("08"); // 0, '8' não é um dígito octal.
```

O ECMAScript 5 remove a interpretação octal

A especificação ECMAScript 5 da funcão `parseInt` não permite mais que as implementações tratem Strings começando com o caracter 0 como um valor octal. O ECMAScript 5 declara:

A função `parseInt` produz um valor inteiro ditado pela interpretação do conteúdo de uma argumento string de acordo com uma base específicada. Espaços em branco no início da string são ignorados. Se a base for undefined ou `0`, ela é assumida como `10` exceto quando o número começa com os pares de caracter `0x` or `0X`, e neste caso a base 16 é assumida. Se a base é `16`, o número pode também opcionalmente começar com os pares de caracter `0x` or `0X`.

Isto difere do ECMAScript 3, que desencoraja mas permite a interpretação octal.

Muitas implementações não adotaram este comportamento a partir de 2013, e porque browser antigos devem ser suportados, **sempre especifique uma base**.

Uma função de análise mais rigorosa

É útil em algum momento ter uma maneira mais rigorosa para analisar valores inteiros. Expressões regulares podem ajudar:

```
filterInt = function (value) {
  if (/^(\-|\+)?([0-9]+|Infinity)$/.test(value)) return Number(value);
  return NaN;
};

console.log(filterInt("421")); // 421
console.log(filterInt("-421")); // -421
console.log(filterInt("+421")); // 421
console.log(filterInt("Infinity")); // Infinity
console.log(filterInt("421e+0")); // NaN
console.log(filterInt("421hop")); // NaN
console.log(filterInt("hop1.61803398875")); // NaN
console.log(filterInt("1.61803398875")); // NaN
```

Veja também

- `parseFloat()`
- `Number.parseFloat()`
- `Number.parseInt()`
- `isNaN()`
- `Number.toString()`
- `Object.valueOf`

Promise

O objeto **`Promise`** representa a eventual conclusão (ou falha) de uma operação assíncrona e seu valor resultante.

Para saber como as promises funcionam e como você pode usá-las, recomendamos que você leia Usando promises primeiro.

Descrição

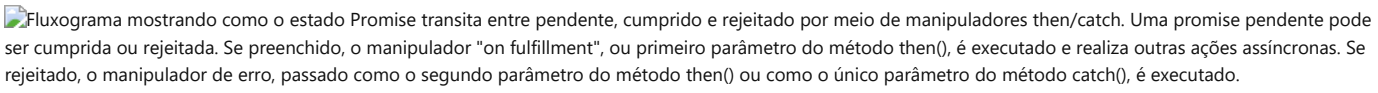
Uma **`Promise`** é um proxy para um valor não necessariamente conhecido quando a promise é criada. Ele permite que você associe manipuladores ao valor de sucesso ou motivo de falha de uma ação assíncrona. Isso permite que métodos assíncronos retornem valores como métodos síncronos: em vez de retornar imediatamente o valor final, o método assíncrono retorna uma *promise* para fornecer o valor em algum momento no futuro.

Uma `Promise` está em um destes estados:

- *pending*: estado inicial, nem cumprido nem rejeitado.
- *fulfilled*: significa que a operação foi concluída com sucesso.
- *rejected*: significa que a operação falhou.

O *estado eventual* de uma promise pendente pode ser *fulfilled* com um valor ou *rejected* com um motivo (erro). Quando uma dessas opções ocorre, os manipuladores associados enfileirados pelo método `then` de uma promise são chamados. Se a promise já tiver sido cumprida ou rejeitada quando um manipulador correspondente for anexado, o manipulador será chamado, portanto, não há condição de corrida entre a conclusão de uma operação assíncrona e a anexação de seus manipuladores.

Uma promise é considerada *resolvida* se for cumprida ou rejeitada, mas não pendente.

 Fluxograma mostrando como o estado Promise transita entre pendente, cumprido e rejeitado por meio de manipuladores then/catch. Uma promise pendente pode ser cumprida ou rejeitada. Se preenchido, o manipulador "on fulfillment", ou primeiro parâmetro do método then(), é executado e realiza outras ações assíncronas. Se rejeitado, o manipulador de erro, passado como o segundo parâmetro do método then() ou como o único parâmetro do método catch(), é executado.

Você também ouvirá o termo *resolved* usado com promises — isso significa que a promise é liquidada ou "bloqueada" para corresponder ao estado eventual de outra promise e resolvê-la ou rejeitá-la não tem efeito. O documento Estados e destinos da proposta original da Promise contém mais detalhes sobre a terminologia da promise. Coloquialmente, promises "resolvidas" geralmente são equivalentes a promises "cumpridas", mas, conforme ilustrado em "Estados e destinos", as promises resolvidas também podem estar pendentes ou rejeitadas. Por exemplo:

```
new Promise((resolveOuter) => {
  resolveOuter(
    new Promise((resolveInner) => {
      setTimeout(resolveInner, 1000);
    }),
  );
});
```

Essa promise já está *resolvida* no momento em que é criada (porque o `resolveOuter` é chamado de forma síncrona), mas é resolvido com outra promise e, portanto, não será *cumprida* até 1 segundo depois, quando a promise interna for cumprida. Na prática, a "resolução" muitas vezes é feita nos bastidores e não observável, e apenas o seu cumprimento ou rejeição o são.

> **Nota:** Várias outras linguagens têm mecanismos para avaliação preguiçosa e adiamento de uma computação, que eles também chamam de "promises", por exemplo, Esquema. As promises em JavaScript representam processos que já estão acontecendo, que podem ser encadeados com funções de retorno de chamada. Se você deseja avaliar lentamente uma expressão, considere usar uma função sem argumentos, por exemplo. `f = () => expressão` para criar a expressão avaliada lentamente e `f()` para avaliar a expressão imediatamente.

Promises em cadeia

Os métodos `Promise.prototype.then()`, `Promise.prototype.catch()` e `Promise.prototype.finally()` são usados para associar uma ação adicional com uma promise que se torna liquidada. Como `Promise.prototype.then()` e `Promise.prototype.catch()` métodos retornam promises, eles podem ser encadeados.

O método `.then()` aceita até dois argumentos; o primeiro argumento é uma função de retorno de chamada para o caso cumprido da promise e o segundo argumento é uma função de retorno de chamada para o caso rejeitado. Cada `.then()` retorna um objeto de promise recém-gerado, que pode ser usado opcionalmente para encadeamento; por exemplo:

```
const minhaPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("foo");
  }, 300);
});

minhaPromise
  .then(handleFulfilledA, handleRejectedA)
  .then(handleFulfilledB, handleRejectedB)
  .then(handleFulfilledC, handleRejectedC);
```

O processamento continua para o próximo elo da cadeia mesmo quando um `.then()` não possui uma função de retorno de chamada que retorna um objeto Promise. Portanto, uma cadeia pode omitir com segurança cada *rejection* função de retorno de chamada até o `.catch()` final.

Manipular uma promise rejeitada em cada `.then()` tem consequências mais adiante na cadeia de promises. Às vezes não há escolha, porque um erro deve ser tratado imediatamente. Nesses casos, devemos lançar um erro de algum tipo para manter o estado de erro na cadeia. Por outro lado, na ausência de uma necessidade imediata, é mais simples deixar de fora o tratamento de erros até uma instrução `.catch()` final. Um `.catch()` é realmente apenas um `.then()` sem um slot para uma função de retorno de chamada para o caso em que a promise é cumprida.

```
minhaPromise
  .then(handleFulfilledA)
  .then(handleFulfilledB)
  .then(handleFulfilledC)
  .catch(handleRejectedAny);
```

Usando Arrow Function Expressions para as funções de retorno de chamada, a implementação da cadeia de promises pode ser algo assim:

```
minhaPromise
  .then((valor) => `${valor} e barra`)
  .then((valor) => `${valor} e barra novamente`)
  .then((valor) => `${valor} e novamente`)
  .then((valor) => `${valor} e novamente`)
  .then((valor) => {
    console.log(valor);
  })
  .catch((erro) => {
    console.error(err);
  });
```

> **Nota:** Para uma execução mais rápida, todas as ações síncronas devem ser feitas preferencialmente em um manipulador, caso contrário, seriam necessários vários tiques para executar todos os manipuladores em sequência.

A condição de término de uma promise determina o estado "estabelecido" da próxima promise na cadeia. Um estado "cumprido" indica uma conclusão bem-sucedida da promise, enquanto um estado "rejeitado" indica falta de sucesso. O valor de retorno de cada promise cumprida na cadeia é passado para o próximo `.then()`, enquanto o motivo da rejeição é passado para a próxima função de tratamento de rejeição na cadeia.

As promises de uma corrente são aninhadas como bonecas russas, mas são estouradas como o topo de uma pilha. A primeira promise na cadeia está mais profundamente aninhada e é a primeira a aparecer.

```
(promise D, (promise C, (promise B, (promise A))))
```

Quando um `nextValue` é uma promise, o efeito é uma substituição dinâmica. O `return` faz com que uma promise seja exibida, mas a promise `nextValue` é colocada em seu lugar. Para o aninhamento mostrado acima, suponha que `.then()` associado a "promise B" retorne um `nextValue` de "promise X". O aninhamento resultante ficaria assim:

```
(promise D, (promise C, (promise X)))
```

Uma promise pode participar de mais de um aninhamento. Para o código a seguir, a transição de `promiseA` para um estado "settled" fará com que ambas as instâncias de `.then()` sejam invocadas.

```
const promiseA = new Promise(myExecutorFunc);
const promiseB = promiseA.then(handleFulfilled1, handleRejected1);
const promiseC = promiseA.then(handleFulfilled2, handleRejected2);
```

Uma ação pode ser atribuída a uma promise já "estabelecida". Nesse caso, a ação (se apropriado) será executada na primeira oportunidade assíncrona. Observe que as promises são garantidas como assíncronas. Portanto, uma ação para uma promise já "estabelecida" ocorrerá somente depois que a pilha for limpa e um tique-taque do relógio passar. O efeito é muito parecido com `setTimeout(action,10)`.

```
const promiseA = new Promise((resoluçãoFunc, rejeiçãoFunc) => {
  resoluçãoFunc(777);
});
// Neste ponto, "promiseA" já está liquidado.
promiseA.then((val) => console.log("log assíncrono tem val:", val));
console.log("log imediato");

// produz saída nesta ordem:
// registro imediato
// registro assíncrono tem valor: 777
```

Thenables

O ecossistema JavaScript fez várias implementações do Promise muito antes de se tornar parte da linguagem. Apesar de serem representados de forma diferente internamente, no mínimo, todos os objetos do tipo Promise implementam a interface *Thenable*. Um thenable implementa o método `.then()`, que é chamado com dois retornos de chamada: um para quando a promise é cumprida, um para quando for rejeitado. Promises também são possíveis.

Para interoperar com as implementações de Promise existentes, a linguagem permite o uso de thenables no lugar de promises. Por exemplo, `Promise.resolve` não apenas resolverá as promises, mas também rastreará os entãoables.

```
const aThenable = {
  then(onFulfilled, onRejected) {
    onCumprido({
      // O thenable é preenchido com outro thenable
```

```
      then(onFulfilled, onRejected) {
        onCumprido(42);
      },
    });
  },
};

Promise.resolve(aThenable); // Uma promise cumprida com 42
```

Rastreamento de objetos de configurações incumbentes

Um objeto de configurações é um ambiente que fornece informações adicionais quando o código JavaScript está em execução. Isso inclui o mapa do domínio e do módulo, bem como informações específicas do HTML, como a origem. O objeto de configurações incumbente é rastreado para garantir que o navegador saibas qual usar para um determinado pedaço de código de usuário.

Para melhor visualizar isso, podemos dar uma olhada mais de perto em como o reino pode ser um problema. Um **reino** pode ser pensado aproximadamente como o objeto global. O que é único sobre os realms é que eles contêm todas as informações necessárias para executar o código JavaScript. Isso inclui objetos como `Array` e `Error`. Cada objeto de configurações tem sua própria "cópia" e não são compartilhados. Isso pode causar algum comportamento inesperado em relação às promises. Para contornar isso, rastreamos algo chamado **objeto de configurações incumbentes**. Isso representa informações específicas do contexto do código do usuário responsável por uma determinada chamada de função.

Para ilustrar isso um pouco mais, podemos dar uma olhada em como um `<iframe>` embutido em um documento se comunica com seu host. Como todas as APIs da Web estão cientes do objeto de configurações incumbente, o seguinte funcionará em todos os navegadores:

```
<!doctype html> <iframe></iframe>
<!-- temos um reino aqui -->
<script>
  // temos um reino aqui também
  const bound = frames[0].postMessage.bind(frames[0], "alguns dados", "*");
  // bound é uma função embutida — não há usuário
  // código na pilha, então qual reino usamos?
  setTimeout(limite);
  // isso ainda funciona, porque usamos o mais novo
  // reino (o titular) na pilha
</script>
```

O mesmo conceito se aplica às promises. Se modificarmos um pouco o exemplo acima, obtemos isso:

```
<!doctype html> <iframe></iframe>
<!-- temos um reino aqui -->
<script>
  // temos um reino aqui também
  const bound = frames[0].postMessage.bind(frames[0], "alguns dados", "*");
  // bound é uma função incorporada — não há usuário
  // código na pilha — qual reino usamos?
  Promise.resolve(undefined).then(bound);
  // isso ainda funciona, porque usamos o mais novo
  // reino (o titular) na pilha
</script>
```

Se alterarmos isso para que o `<iframe>` no documento esteja ouvindo as mensagens de postagem, podemos observar o efeito do objeto de configurações incumbente:

```
<!-- y.html -->
<!doctype html>
<iframe src="https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Promise/x.html"></iframe>
<script>
  const bound = frames[0].postMessage.bind(frames[0], "alguns dados", "*");
  Promise.resolve(undefined).then(bound);
</script>
```

```
<!-- x.html -->
<!doctype html>
<script>
  window.addEventListener(
    "mensagem",
    (evento) => {
      document.querySelector("#texto").textContent = "olá";
      // este código será executado apenas em navegadores que rastreiam o objeto de configurações incumbente
      console.log(evento);
    },
    falso,
  );
</script>
```

No exemplo acima, o texto interno do `<iframe>` será atualizado somente se o objeto de configurações incumbente for rastreado. Isso porque, sem rastrear o titular, podemos acabar usando o ambiente errado para enviar a mensagem.

> **Nota:** Atualmente, o rastreamento de realm incumbente é totalmente implementado no Firefox e tem implementações parciais no Chrome e no Safari.

Construtor

`Promise()`

Cria um novo objeto `Promise`. O construtor é usado principalmente para encapsular funções que ainda não suportam promises.

Métodos estáticos

`Promise.all(iterable)`

Aguarde que todas as promises sejam cumpridas ou que alguma seja rejeitada.

Se a promise retornada for cumprida, ela será cumprida com uma matriz de agregação dos valores das promises cumpridas, na mesma ordem definida no iterável de várias promises.

Se for rejeitado, será rejeitado com o motivo da primeira promise no iterável que foi rejeitada.

`Promise.allSettled(iterable)`

Aguarde até que todas as promises sejam resolvidas (cada uma pode cumprir ou rejeitar).

Retorna uma Promise que cumpre depois que todas as promises fornecidas são cumpridas ou rejeitadas, com uma matriz de objetos que descrevem o resultado de cada promise.

`Promise.any(iterable)`

Pega um iterável de objetos Promise e, assim que uma das promises do iterável é cumprida, retorna uma única promise que cumpre com o valor dessa promise.

`Promise.race(iterable)`

Aguarde até que alguma das promises seja cumprida ou rejeitada.

Se a promise retornada for cumprida, ela será cumprida com o valor da primeira promise no iterável que foi cumprida.

Se for rejeitado, será rejeitado com o motivo da primeira promise que foi rejeitada.

`Promise.reject(reason)`

Retorna um novo objeto `Promise` que é rejeitado com o motivo fornecido.

`Promise.resolve(value)`

Retorna um novo objeto `Promise` que é resolvido com o valor fornecido. Se o valor for um thenable (ou seja, tem um método `then`), a promise retornada "seguirá" esse thenable, adotando seu estado eventual; caso contrário, a promise devolvida será cumprida com o valor.

Geralmente, se você não sabe se um valor é uma promise ou não, `Promise.resolve(value)` e trabalhe com o valor de retorno como uma promise.

Métodos de instância

Consulte o Guia do Microtask para saber mais sobre como esses métodos usam a fila e os serviços do Microtask.

`Promise.prototype.catch()`

Anexa um retorno de chamada do manipulador de rejeição à promise e retorna uma nova promise resolvendo o valor de retorno do retorno de chamada se for chamado ou seu valor de cumprimento original se a promise for cumprida.

`Promise.prototype.then()`

Anexa manipuladores de cumprimento e rejeição à promise e retorna uma nova promise resolvendo o valor de retorno do manipulador chamado ou seu valor estabelecido original se a promise não foi tratada (ou seja, se o manipulador relevante `onFulfilled` ou `onRejected` não é uma função).

`Promise.prototype.finally()`

Anexa um manipulador à promise e retorna uma nova promise que é resolvida quando a promise original é resolvida. O manipulador é chamado quando a promise é liquidada, seja cumprida ou rejeitada.

Exemplos

Exemplo básico

```
const minhaPrimeiraPromise = new Promise((resolve, reject) => {
  // Chamamos resolve(...) quando o que estávamos fazendo de forma assíncrona foi bem-sucedido e reject(...) quando falhou.
  // Neste exemplo, usamos setTimeout(...) para simular código assíncrono.
  // Na realidade, você provavelmente estará usando algo como XHR ou uma API HTML.
  setTimeout(() => {
    resolve("Sucesso!"); // Yay! Tudo ocorreu bem!
  }, 250);
});

minhaPrimeiraPromise.then((mensagem de sucesso) => {
  // successMessage é o que passamos na função resolve(...) acima.
  // Não precisa ser uma string, mas se for apenas uma mensagem de sucesso, provavelmente será.
  console.log(`Yay! ${successMessage}`);
});
```

Exemplo com diversas situações

Este exemplo mostra diversas técnicas para usar os recursos do Promise e diversas situações que podem ocorrer. Para entender isso, comece rolando até a parte inferior do bloco de código e examine a cadeia de promises. Após o fornecimento de uma promise inicial, uma cadeia de promises pode seguir. A cadeia é composta de chamadas `.then()`, e normalmente (mas não necessariamente) tem um único `.catch()` no final, opcionalmente seguido por `.finally()`. Neste exemplo, a cadeia de promises é iniciada por uma construção `new Promise()` escrita sob medida; mas, na prática, as cadeias de promises geralmente começam com uma função de API (escrita por outra pessoa) que retorna uma promise.

A função de exemplo `tetheredGetNumber()` mostra que um gerador de promise utilizará `reject()` ao configurar uma chamada assíncrona, ou dentro do call-back, ou ambos. A função `promiseGetWord()` ilustra como uma função de API pode gerar e retornar uma promise de maneira independente.

Observe que a função `troubleWithGetNumber()` termina com um `throw`. Isso é forçado porque uma cadeia de promises passa por todas as promises `.then()`, mesmo após um erro, e sem o `throw`, o erro pareceria "corrigido". Isso é um aborrecimento, e por esta razão, é comum omitir `rejectionFunc` em toda a cadeia de promises `.then()`, e ter apenas um único `rejectionFunc` no `catch()` final.

Este código pode ser executado em NodeJS. A compreensão é aprimorada ao ver os erros realmente ocorrerem. Para forçar mais erros, altere os valores de `threshold`.

```
// Para experimentar o tratamento de erros, os valores "threshold" causam erros aleatoriamente
const THRESHOLD_A = 8; // pode usar zero 0 para garantir o erro

function tetheredGetNumber(resolve, reject) {
  setTimeout(() => {
    const randomInt = Date.now();
    const value = randomInt % 10;
    if (value < THRESHOLD_A) {
      resolve(value);
    } else {
      reject(`Muito grande: ${value}`);
    }
  }, 500);
}

function determinaParidade(valor) {
  const isOdd = valor % 2 === 1;
  return { valor, isOdd };
}

function troubleWithGetNumber(reason) {
  const err = new Error("Problema para obter o número", { cause: reason });
  console.error(err);
  throw err;
}

function promiseGetWord(parityInfo) {
  return new Promise((resolve, reject) => {
    const { value, isOdd } = parityInfo;
    if (value >= THRESHOLD_A - 1) {
      reject(`Ainda muito grande: ${valor}`);
    } else {
      parityInfo.wordEvenOdd = isOdd ? "ímpar" : "par";
      resolve(parityInfo);
    }
  });
}

new Promise(tetheredGetNumber)
  .then(determineParity, troubleWithGetNumber)
  .then(promiseGetWord)
  .then((info) => {
    console.log(`Recebido: ${info.value}, ${info.wordEvenOdd}`);
    return info;
  })
  .catch((reason) => {
    if (reason.cause) {
      console.error("Já havia tratado o erro anteriormente");
    } else {
      console.error(`Problema com promiseGetWord(): ${reason}`);
    }
  })
  .finally((info) => console.log("Tudo pronto"));
```

Exemplo Avançado

Este pequeno exemplo mostra o mecanismo of a `Promise`. O método `testPromise()` é chamado cada vez que o `<button>` é clicado. Ele cria uma promise que será cumprida, usando `setTimeout()`, para a contagem de promises (número começando em 1) a cada 1-3 segundos, aleatoriamente. O construtor `Promise()` é usado para criar a promise.

O cumprimento da promise é registrado, por meio de um callback de cumprimento definido usando `p1.then()`. Alguns logs mostram como a parte síncrona do método é dissociada da conclusão assíncrona da promise.

Ao clicar no botão várias vezes em um curto espaço de tempo, você verá até as diferentes promises sendo cumpridas uma após a outra.

HTML

```
<button id="make-promise">Faça uma promise!</button>
<div id="log"></div>
```

JavaScript

```
"use strict";

let promiseCount = 0;

function testPromise() {
  const thisPromiseCount = ++promiseCount;
  const log = document.getElementById("log");
  // começar
  log.insertAdjacentHTML("beforeend", `${thisPromiseCount}) Iniciado<br>`);
  // Fazemos uma nova promise: prometemos uma contagem numérica dessa promise,
  // a partir de 1 (após esperar 3s)
  const p1 = new Promise((resolve, reject) => {
    // A função executora é chamada com a habilidade
    // para resolver ou rejeitar a promise
    log.insertAdjacentHTML(
      "beforeend",
      `${thisPromiseCount}) Construtor de promise<br>`,
    );
    // Este é apenas um exemplo para criar assincronismo
    setTimeout(
      () => {
        // Nós cumprimos a promise
        resolve(thisPromiseCount);
      },
      Math.random() * 2000 + 1000,
    );
  });

  // Definimos o que fazer quando a promise é resolvida com a chamada then(),
  // e o que fazer quando a promise é rejeitada com a chamada catch()
  p1.then((val) => {
    // Registra o valor de atendimento
    log.insertAdjacentHTML("beforeend", `${val}) Promise cumprida<br>`);
  }).catch((reason) => {
    // Registra o motivo da rejeição
    console.log(`Manuseie a promise rejeitada (${reason}) aqui.`);
  });
  // fim
  log.insertAdjacentHTML("beforeend", `${thisPromiseCount}) Promise feita<br>`);
}

const btn = document.getElementById("make-promise");
btn.addEventListener("click", testPromise);
```

Resultado

Carregando uma imagem com XHR

Outro exemplo simples usando `Promise` e `XMLHttpRequest` para carregar uma imagem está disponível no MDN GitHub js-examples. Você também pode ver em ação. Cada etapa é comentada e permite acompanhar de perto a arquitetura Promise e XHR.

Veja também

- Polyfill de `Promise` em `core-js`
- Usando promises
- Promises/A+ especificação
- Promises JavaScript: uma introdução
- Domenic Denicola: Callbacks, Promises, and Coroutines – Asynchronous Programming Patterns in JavaScript

Promise.all()

O método **`Promise.all(iterable)`** retorna uma única `Promise` que resolve quando todas as promises no argumento iterável forem resolvidas ou quando o iterável passado como argumento não contém promises. É rejeitado com o motivo da primeira promise que foi rejeitada.

Experimente

```
const promise1 = Promise.resolve(3);
const promise2 = 42;
const promise3 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, "foo");
});

Promise.all([promise1, promise2, promise3]).then((values) => {
  console.log(values);
});
// Expected output: Array [3, 42, "foo"]
```

Sintaxe

```
Promise.all(iterable);
```

Parâmetros

iterable

> Um objeto iterável, como um `Array` ou `String`.

Retorno

- Uma `Promise` **já resolvida** se o iterável passado for vazio.
- Uma `Promise` **resolvida assíncronamente** se o iterável passado não conter promises. Nota, Google Chrome 58 retorna uma promise **já resolvida** nesse caso.
- Uma `Promise` **pendente** em todos os outros casos. Essa promise retornada é então resolvida/rejeitada **assíncronamente** (tão logo a pilha esteja vazia) quando todas as promises no dado iterável forem resolvidas, ou se alguma das promises for rejeitada. Veja o exemplo sobre "Assincronia ou sincronia da Promise.all" abaixo. Os valores retornados estarão na ordem em que as promises foram passadas. independentemente da ordem em que forem concluídas.

Descrição

Esse método pode ser usado para agregar resultados de várias promises.

Resolução

A promise retornada é resolvida com um array contendo **todos** os valores dos iteráveis passados como argumento (como valores que não são promises).

- Se um iterável vazio é passado, então esse método retorna (sincronamente) uma promise já resolvida.
- Se todas as promises passadas forem resolvidas, ou não forem promises, a promise retornada por `Promise.all` é resolvida assincronamente.

Rejeição

Se qualquer uma das promises passadas for rejeitada, `Promise.all` assíncronamente é rejeitada com o valor da promise rejeitada, independentemente se outras promises foram resolvidas.

Exemplos

Utilizando Promise.all

`Promise.all` espera que todas as promises sejam resolvidas (ou a primeira rejeição).

```
var p1 = Promise.resolve(3);
var p2 = 1337;
var p3 = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("foo");
  }, 100);
});

Promise.all([p1, p2, p3]).then((valores) => {
  console.log(valores); // [3, 1337, "foo"]
});
```

Se o iterável conter valores que não são promises, eles serão ignorados, mas ainda serão contados no array da promise retornada (se a promise for resolvida):

```
// Essa será considerada como se o iterável passado fosse vazio, logo ela será resolvido
var p = Promise.all([1, 2, 3]);
// Essa será considerada como se o iterável passado contém apenas a promise resolvida com o valor "444", logo ela é resolvida
var p2 = Promise.all([1, 2, 3, Promise.resolve(444)]);
// Esse será considerada como se o iterável passado contém apenas o valor de rejeição "555" da promise, logo ela é rejeitada
var p3 = Promise.all([1, 2, 3, Promise.reject(555)]);

// Utilizando setTimeout para executar código depois que a pilha estiver vazia
setTimeout(function () {
  console.log(p);
  console.log(p2);
  console.log(p3);
});

// logs
// Promise { <estado>: "resolvida", <valor>: Array[3] }
// Promise { <estado>: "resolvida", <valor>: Array[4] }
// Promise { <estado>: "rejeitada", <razão>: 555 }
```

Assincronia ou sincronia da Promise.all

O exemplo a seguir demonstra a assincronia (ou sincronia, se o iterável passado for vazio) de `Promise.all`:

```
// Passamos o argumento como um array de promises que já estão resolvidas para disparar Promise.all a mais rápido possível
var arrayPromisesResolvidas = [Promise.resolve(33), Promise.resolve(44)];

var p = Promise.all(arrayPromisesResolvidas);
// Logando imediatamente o valor de p
console.log(p);

// Utilizando setTimeout para executar código depois que a pilha estiver vazia
```

```
setTimeout(function () {
  console.log("a pilha está vazia agora");
  console.log(p);
});

// logs, em ordem:
// Promise { <estado>: "pendente" }
// a pilha está vazia agora
// Promise { <estado>: "resolvida", <valor>: Array[2] }
```

A mesma coisa acontece se `Promise.all` for rejeitada:

```
var arrayPromisesMisturadas = [Promise.resolve(33), Promise.reject(44)];
var p = Promise.all(arrayPromisesMisturadas);
console.log(p);
setTimeout(function () {
  console.log("a pilha está vazia agora");
  console.log(p);
});

// logs
// Promise { <estado>: "pendente" }
// a pilha está vazia agora
// Promise { <estado>: "rejeitada", <razão>: 44 }
```

Mas, `Promise.all` resolve sincromamente **se e somente se** o iterável passado for vazio:

```
var p = Promise.all([]); // será resolvida imediatamente
var p2 = Promise.all([1337, "oi"]); // um valor que não é uma promise será ignorado, mas a avaliação será feita assíncronamente
console.log(p);
console.log(p2);
setTimeout(function () {
  console.log("a pilha está vazia agora");
  console.log(p2);
});

// logs
// Promise { <estado>: "resolvida", <valor>: Array[0] }
// Promise { <estado>: "pendente" }
// a pilha está vazia agora
// Promise { <estado>: "resolvida", <valor>: Array[2] }
```

Comportamente de falhar rapidamente de Promise.all

`Promise.all` é rejeitada se qualquer um dos elementos for rejeitado. Por exemplo, se você passar quartro promises que resolvem após um intervalo de tempo e uma promise que rejeita imediatamente, então `Promise.all` será rejeitada imediatamente.

```
var p1 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("um"), 1000);
});
var p2 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("dois"), 2000);
});
var p3 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("três"), 3000);
});
var p4 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("quatro"), 4000);
});
var p5 = new Promise((resolve, reject) => {
  reject(new Error("rejeitada"));
});

// Usando .catch:
Promise.all([p1, p2, p3, p4, p5])
  .then((valores) => {
    console.log(valores);
  })
  .catch((erro) => {
    console.log(erro.message);
  });

// No console:
// "rejeitada"
```

É possível mudar esse comportamente lidando com possíveis rejeições:

```
var p1 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("p1_resolução_atrasada"), 1000);
});

var p2 = new Promise((resolve, reject) => {
  reject(new Error("p2_rejeição_imediata"));
});
```

```
Promise.all([
  p1.catch((erro) => {
    return erro;
  }),
  p2.catch((erro) => {
    return erro;
  }),
]).then((valores) => {
  console.log(valores[0]); // "p1_resolução_atrasada"
  console.log(valores[1]); // "Erro: p2_rejeição_imediata"
});
```

Veja também

- `Promise`
- `Promise.race()`

Promise.allSettled()

O método **`Promise.allSettled()`** retorna uma promessa que é resolvida após todas as promessas dadas serem resolvidas ou rejeitadas, com um array de objetos que descrevem o resultado de cada promessa.

É tipicamente usado quando você tem múltiplas tarefas assíncronas que não são dependentes das conclusões umas das outras, ou quando você sempre quer ter o resultado de cada promessa.

Para comparação, a promessa retornada por `Promise.all()` pode ser mais apropriada para tarefas que dependem umas das outras, ou se você precisa que todas as tarefas sejam rejeitadas quando apenas uma é.

Experimente

```
const promise1 = Promise.resolve(3);
const promise2 = new Promise((resolve, reject) =>
  setTimeout(reject, 100, "foo"),
);
const promises = [promise1, promise2];

Promise.allSettled(promises).then((results) =>
  results.forEach((result) => console.log(result.status)),
);

// Expected output:
// "fulfilled"
// "rejected"
```

Sintaxe

```
promise.allSettled(iterable);
```

Parâmetros

`iterable`

Um objeto iterável, como um `Array`, onde cada membro é uma `Promise`.

Valor retornado

Uma `Promise` **pendente** que vai ser **preenchida assíncronamente** quando todas as promessas na coleção fornecida forem completas, sendo aceitas ou rejeitadas. Quando isso acontecer, é retornado um array contendo um resultado para cada promessa passada como entrada.

Para cada objeto no array retornado, existe uma string `status`. Se o status for `fulfilled`, então o campo `value` estará presente. Se o status for `rejected`, então o campo `reason` estará presente. O valor (value) ou o motivo da falha (reason) refletem o valor com que cada promessa foi completada (ou rejeitada).

Veja também

- Promessas
- Usando promessas
- Programando assincronamente de maneira elegante usando promessas
- `Promise`
- `Promise.all()`

Promise.prototype.catch()

O método **catch()** retorna uma Promise e lida apenas com casos rejeitados. Ele possui o mesmo comportamento de quando chamamos `Promise.prototype.then(undefined, onRejected)` (de fato, chamando `obj.catch(onRejected)` internamente é chamado `obj.then(undefined, onRejected)`).

Sintaxe

```
p.catch(onRejected);

p.catch(function(motivo) {
   // rejeição
});
```

Parâmetros

onRejected

Uma `Function` chamada quando a `Promise` é rejeitada. Esta função possui um argumento: **reason** da rejeição. O motivo da rejeição.

A Promise retornada pelo `catch()` é rejeitada apenas se `onRejected` cospe um erro ou se o o retorno da Promise foi rejeitada por si mesmo, ou seja, foi resolvida.

Valor de retorno

Internamente chamamos `Promise.prototype.then` sobre o objeto que é chamando passando parâmetros como `undefined` e `onRejected` no manipulador de eventos. Então retornamos o valor da chamada que é `Promise`.

> **Aviso:** O exemplo abaixo está cuspindo uma string. Isso é considerado uma má prática. Sempre cuspir uma instance de erro (Error). Em todo caso, a parte que faz a captura deve fazer verificaçoes sobre os argumentos para saber se é uma string ou um erro e você poderá perder informações valiosas como stack traces.

Demonstração de uma camada interna:

```
// Sobrescrevendo o techo original de  Promise.prototype.then/catch adicionando alguns logs
(function (Promise) {
  var originalThen = Promise.prototype.then;
  var originalCatch = Promise.prototype.catch;

  Promise.prototype.then = function () {
    console.log(
      "> > > > > > chamando .then em %o com argumentos: %o",
      this,
      arguments,
    );
    return originalThen.apply(this, arguments);
  };
  Promise.prototype.catch = function () {
    console.log(
      "> > > > > > chamando .catch em %o com argumentos: %o",
      this,
      arguments,
    );
    return originalCatch.apply(this, arguments);
  };
})(this.Promise);

// chamando um catch em uma Promise já resolvida.
Promise.resolve().catch(function XXX() {});

// logs:
// > > > > > > chamando .catch na Promise{} com os argumentos: Arguments{1} [0: function XXX()]
// > > > > > > chamando .then na Promise{} com os argumentos: Arguments{2} [0: undefined, 1: function XXX()]
```

Description

O método `catch` pode ser útil para manipulação de erros na composição da sua promise.

Exemplos

Usando o método <code>catch</code>

```
var p1 = new Promise(function (resolve, reject) {
  resolve("Sucesso");
});

p1.then(function (value) {
  console.log(value); // "Sucesso!"
  throw "Ah, não!";
})
  .catch(function (e) {
    console.log(e); // "Ah, não!"
  })
  .then(
    function () {
      console.log("Após um catch, a sequencia é restaurada");
    },
    function () {
      console.log("Não engatilhado devido ao catch");
    },
  );

// O seguinte se comporta da mesma maneira que o anterior
p1.then(function (value) {
  console.log(value); // "Sucesso!"
  return Promise.reject("Ah, não!");
})
  .catch(function (e) {
    console.log(e); // "Ah, não!"
```

```
  })
  .then(
    function () {
      console.log("Após um catch, a sequencia é restaurada");
    },
    function () {
      console.log("Não engatilhado devido ao catch");
    },
  );
```

Veja também

- `Promise`
- `Promise.prototype.then()`

Promise.prototype.finally()

O método **`finally()`** retorna uma `Promise`. Quando a promise for estabelecida, tenha ela sido realizada ou rejeitada, executa-se a função callback especificada. Isso permite a execução de um código que acontecerá independentemente da `Promise` ter sido realizada (com sucesso) ou rejeitada (com falha).

Assim, você pode evitar a duplicação do código em ambos os handlers `then()` e `catch()` da `Promise`.

Sintaxe

```
p.finally(quandoEstabelecida);

p.finally(function() {
   // concluída (realizada ou rejeitada)
});
```

Parâmetros

`quandoEstabelecida`

Uma `Function` chamada quando a `Promise` é concluída.

Valor de retorno

Retorna uma `Promise` onde o manipulador `finally` é definido como a função especificada, `quandoEstabelecida`.

Descrição

O método `finally()` pode ser útil quando você deseja realizar algum tipo de processamento ou limpeza quando a promise for estabelecida, independentemente de seu resultado (sucesso ou falha).

O método `finally()` é bastante similar a chamar `.then(quandoEstabelecida, quandoEstabelecida)`. Porém, existem algumas diferenças:

- Ao passar a função na linha (entre os parênteses do método), você precisa passá-la apenas uma vez, ao invés de ser forçado a declará-la duas vezes ou a definir em uma variável.
- O callback de `finally` não poderá receber nenhum argumento, já que não existem meios confiáveis de determinar se a promise foi realizada ou rejeitada. Seu uso é para quando você realmente *não se importa* com os possíveis motivos de uma falha ou possíveis valores retornados de um sucesso, não tendo, portanto, razão para fornecê-los.
- Diferentemente de `Promise.resolve(2).then(() => {}, () => {})` (será resolvido como `undefined`), `Promise.resolve(2).finally(() => {})` será resolvido como `2`.
- De maneira semelhante, diferentemente de `Promise.reject(3).then(() => {}, () => {})` (que será resolvido como `undefined`), `Promise.reject(3).finally(() => {})` será rejeitado como `3`.

> **Nota:** Um `throw` (ou retorno de uma promise rejeitada) no callback de `finally` rejeitará a nova promise com a razão de rejeição especificada na chamada de `throw()`.

Exemplos

```
let carregando = true;

fetch(minhaRequisicao)
  .then(function (resposta) {
    var tipoConteudo = response.headers.get("content-type");
    if (tipoConteudo && tipoConteudo.includes("application/json")) {
      return resposta.json();
    }
    throw new TypeError("Opa, isso não é JSON!");
  })
  .then(function (json) {
    /* processamento do seu JSON */
  })
  .catch(function (erro) {
    console.log(erro);
  })
  .finally(function () {
    carregando = false;
  });
```

Especificações

Specification	Status	Comment
TC39 proposal	Stage 4	

Veja também

- `Promise`
- `Promise.prototype.then()`
- `Promise.prototype.catch()`

Promise.race()

O método **`Promise.race(iterable)`** retorna uma promise que resolve ou rejeita assim que uma das promises no iterável resolver ou rejeitar, com o valor ou razão daquela promise.

Sintaxe

```
Promise.race(iterable);
```

Parâmetros

iterable

Um objeto iterável, como um `Array`. Veja iterável.

Descrição

A função `race` retorna uma `Promise` que é estabelecida da mesma forma que a primeira promise passada estabelecer. Ela resolve ou rejeita, o que acontecer primeiro.

Exemplos

Usando <code>Promise.race</code> – exemplos com <code>setTimeout</code>

```
var p1 = new Promise(function (resolve, reject) {
  setTimeout(resolve, 500, "one");
});
var p2 = new Promise(function (resolve, reject) {
  setTimeout(resolve, 100, "two");
});

Promise.race([p1, p2]).then(function (value) {
  console.log(value); // "two"
  // Ambos resolvem, mas p2 é mais rápido
});

var p3 = new Promise(function (resolve, reject) {
  setTimeout(resolve, 100, "three");
});
var p4 = new Promise(function (resolve, reject) {
  setTimeout(reject, 500, "four");
});

Promise.race([p3, p4]).then(
  function (value) {
    console.log(value); // "three"
    // p3 é mais rápido, então ela resolve
  },
  function (reason) {
    // Não é chamado
  },
);

var p5 = new Promise(function (resolve, reject) {
  setTimeout(resolve, 500, "five");
});
var p6 = new Promise(function (resolve, reject) {
  setTimeout(reject, 100, "six");
});

Promise.race([p5, p6]).then(
  function (value) {
    // Não é chamado
  },
  function (reason) {
    console.log(reason); // "six"
    // p6 é mais rápido, então ela rejeita
  },
);
```

Ver também

- `Promise`

Promise.reject()

O método **`Promise.reject(motivo)`** retorna um objeto Promise que é rejeitada com um dado motivo.

Sintaxe

```
Promise.reject(motivo);
```

Parâmetros

motivo

Motivo pelo qual este Promise foi rejeitado.

Descrição

O método estático `Promise.reject` retorna uma `Promise` que é rejeitada. Para fins de debug e captura de erros seletiva, é útil que o motivo seja uma `instanceof` `Error`.

Exemplos

Usando o método estático Promise.reject()

```
Promise.reject("Testando reject estático").then(
  function (motivo) {
    // não executado
  },
  function (motivo) {
    console.log(motivo); // "Testando reject estático"
  },
);

Promise.reject(new Error("falha")).then(
  function (erro) {
    // não executado
  },
  function (erro) {
    console.log(erro); // Stacktrace
  },
);
```

Ver também

- `Promise`
- Captura de erros seletiva com a biblioteca de Promise BlueBird

Promise.resolve()

O método **`Promise.resolve(value)`** retorna um objeto `Promise` que é resolvido com o valor passado. Se o valor for *thenable* (ex: tiver um método `then`), a promise retornada irá "seguir" esse *thenable*, adotando seu estado final; se o valor for uma promise, o objeto será o resultado da chamada Promise.resolve; do contrário a promise será realizada com o valor.

Sintaxe

```
Promise.resolve(value);
Promise.resolve(promise);
Promise.resolve(thenable);
```

Parametros

value

Argumento a ser resolvido pela `Promise`. Pode também ser uma `Promise` ou um thenable a resolver.

Valor retornado

A `Promise` que será resolvida com o valor passado ou com a `Promise` passada como valor, caso o valor seja um objeto `Promise`

Descrição

A função estática `Promise.resolve` retorna uma `Promise` de que será resolvida.

Examples

Usando o método estático <code>Promise.resolve</code>

```
Promise.resolve("Success").then(
  function (value) {
    console.log(value); // "Success"
  },
  function (value) {
    // not called
  },
);
```

Resolvendo um array

```
var p = Promise.resolve([1, 2, 3]);
p.then(function (v) {
  console.log(v[0]); // 1
});
```

Resolvendo outra <code>Promise</code>

```
var original = Promise.resolve(true);
var cast = Promise.resolve(original);
cast.then(function (v) {
  console.log(v); // true
});
```

A ordem invertida dos logs acontece devido ao fato de que os handlers são chamados assincronamente. Veja como o `then` funciona aqui.

Resolvendo thenables e disparando Errors

```
// Resolving a thenable object
var p1 = Promise.resolve({
  then: function (onFulfill, onReject) {
    onFulfill("fulfilled!");
  },
});
console.log(p1 instanceof Promise); // true, object casted to a Promise

p1.then(
  function (v) {
    console.log(v); // "fulfilled!"
  },
  function (e) {
    // not called
  },
);

// Thenable throws before callback
// Promise rejects
var thenable = {
  then: function (resolve) {
    throw new TypeError("Throwing");
    resolve("Resolving");
  },
};

var p2 = Promise.resolve(thenable);
p2.then(
  function (v) {
    // not called
  },
  function (e) {
    console.log(e); // TypeError: Throwing
  },
);

// Thenable throws after callback
// Promise resolves
var thenable = {
  then: function (resolve) {
    resolve("Resolving");
    throw new TypeError("Throwing");
  },
};

var p3 = Promise.resolve(thenable);
p3.then(
  function (v) {
    console.log(v); // "Resolving"
  },
  function (e) {
    // not called
  },
);
```

Veja também

- `Promise`

Promise.prototype.then()

Resumo

O método **then()** retorna uma **Promise**. Possui dois argumentos, ambos são "**call back functions**", sendo uma para o sucesso e outra para o fracasso da promessa.

Sintaxe

```
p.then(quandoRealizada, quandoRejeitada);

p.then(function(valor) {
   // sucesso
  }, function(motivo) {
  // rejeitada
});
```

Parametros

quandoRealizada

Uma `Function` chamada quando a **Promise** é cumprida (Sucesso). Essa função tem um argumento, o valor do cumprimento.

quandoRejeitada

Uma `Function` chamada quando a Promise é rejeitada. Essa função tem um argumento, o motivo da recusa.

Descrição

Assim como o método .then() e `Promise.prototype.catch()` retornam uma **Promise**, eles podem ser encadeados - uma operação chamada ***composition***.

Exemplos

Usando o método then

```
var p1 = new Promise(function (resolve, reject) {
  resolve("Success!");
  // or
  // reject ("Error!");
});

p1.then(
  function (value) {
    console.log(value); // Success!
  },
  function (reason) {
    console.log(reason); // Error!
  },
);
```

Encadeando

Já que o método then() devolve uma **Promise**, você pode facilmente encadeá-los.

```
var p2 = new Promise(function (resolve, reject) {
  resolve(1);
});

p2.then(function (value) {
  console.log(value); // 1
  return value + 1;
}).then(function (value) {
  console.log(value); // 2
});
```

No exemplo acima, o último **.then()** recebeu a soma value + 1, que resultou em 2, porém se o retorno de value + 1 fosse uma **Promise** que também retornasse value + 1, o resultado seria o mesmo. Note, no exemplo abaixo, que leva 1000ms para a impressão de 2 ocorrer.

```
var p2 = new Promise(function (resolve, reject) {
  resolve(1);
});

p2.then(function (value) {
  console.log(value); // 1
  return new Promise(function (resolve, reject) {
    setTimeout(function () {
      resolve(value + 1);
    }, 1000);
  });
}).then(function (value) {
  console.log(value); // 2
});
```

Veja também

- `Promise`
- `Promise.prototype.catch()`

Proxy

O objeto **Proxy** é usado para definir comportamentos customizados para operações fundamentais (por exemplo, pesquisa de propriedade, atribuição, enumeração, invocação de função, etc.).

Terminologia

handler

Objeto marcador que contém *traps*.

traps

Métodos que fornecem acesso à propriedade. Isto é análogo ao conceito de *traps* (armadilhas) em sistemas operacionais

target

Objeto que o proxy virtualiza. É frequentemente utilizado como back-end de armazenamento para o proxy. Os invariantes (semânticas que permanecem inalterados) em relação à não-extensibilidade do objeto ou propriedades não-configuráveis são verificados em relação ao *target*.

Sintaxe

```
var p = new Proxy(target, handler);
```

Parâmetros

`target`

Um objeto target (pode ser qualquer tipo de objeto, incluindo um array, uma função ou até mesmo outro *Proxy*) a ser envolvido com o `Proxy`.

`handler`

Um objeto cujas propriedades são funções que definem o comportamento do proxy quando uma operação é realizada sobre ele.

Métodos

`Proxy.revocable()`

Cria um objeto `Proxy` revogável.

Métodos para manipular objetos

O objeto manipulado é um objeto reservado que contém traps para `Proxy`.

Exemplos

Exemplo básico

Neste exemplo simples, o número `37` é retornado como o valor padrão quando o nome da propriedade não está no objeto. Usa-se o manipulador `get`.

```
var handler = {
  get: function (target, name) {
    return name in target ? target[name] : 37;
  },
};

var p = new Proxy({}, handler);
p.a = 1;
p.b = undefined;

console.log(p.a, p.b); // 1, undefined
console.log("c" in p, p.c); // false, 37
```

Encaminhamento de Proxy

Neste exemplo, estamos usando um objeto JavaScript nativo ao qual nosso proxy irá encaminhar todas as operações que são aplicadas para ele.

```
var target = {};
var p = new Proxy(target, {});

p.a = 37; // Operação encaminhada para o alvo

console.log(target.a); // 37. A operação foi devidamente encaminhada
```

Validação

Com um `Proxy`, você pode validar facilmente o valor passado para um objeto. Este exemplo usa o manipulador `set`.

```
let validator = {
  set: function (obj, prop, value) {
    if (prop === "age") {
      if (!Number.isInteger(value)) {
        throw new TypeError("The age is not an integer");
      }
      if (value > 200) {
        throw new RangeError("The age seems invalid");
      }
    }
```

```
    //O comportamento padrão para armazenar o valor
    obj[prop] = value;

    // Indique o sucesso
    return true;
  },
};

let person = new Proxy({}, validator);

person.age = 100;
console.log(person.age); // 100
person.age = "young"; // Lança uma exceção
person.age = 300; // Lança uma exceção
```

Construtor de extensão

Um proxy de função poderia facilmente estender um construtor com um novo construtor. Este exemplo usa os manipuladores `construct` e `apply`.

```
function extend(sup, base) {
  var descriptor = Object.getOwnPropertyDescriptor(
    base.prototype,
    "constructor",
  );
  base.prototype = Object.create(sup.prototype);
  var handler = {
    construct: function (target, args) {
      var obj = Object.create(base.prototype);
      this.apply(target, obj, args);
      return obj;
    },
    apply: function (target, that, args) {
      sup.apply(that, args);
      base.apply(that, args);
    },
  };
  var proxy = new Proxy(base, handler);
  descriptor.value = proxy;
  Object.defineProperty(base.prototype, "constructor", descriptor);
  return proxy;
}

var Person = function (name) {
  this.name = name;
};

var Boy = extend(Person, function (name, age) {
  this.age = age;
});

Boy.prototype.sex = "M";

var Peter = new Boy("Peter", 13);
console.log(Peter.sex); // "M"
console.log(Peter.name); // "Peter"
console.log(Peter.age); // 13
```

DOM manipulação de nós

Às vezes, você deseja alternar o atributo ou o nome da classe de dois elementos diferentes. Veja como usar o manipulador `set`.

```
let view = new Proxy(
  {
    selected: null,
  },
  {
    set: function (obj, prop, newval) {
      let oldval = obj[prop];

      if (prop === "selected") {
        if (oldval) {
          oldval.setAttribute("aria-selected", "false");
        }
        if (newval) {
          newval.setAttribute("aria-selected", "true");
        }
      }

      // O comportamento para armazenar o valor padrão
      obj[prop] = newval;

      // Indica o sucesso
      return true;
    },
  },
```

```
);

let i1 = (view.selected = document.getElementById("item-1"));
console.log(i1.getAttribute("aria-selected")); // 'true'

let i2 = (view.selected = document.getElementById("item-2"));
console.log(i1.getAttribute("aria-selected")); // 'false'
console.log(i2.getAttribute("aria-selected")); // 'true'
```

Correção de valor e uma propriedade extra

O objeto de proxy `produtos` avalia o valor passado e converte-o em uma matriz, se necessário. O objeto também suporta uma propriedade adicional chamada `latestBrowser` tanto em getters como em setters.

```
let products = new Proxy(
  {
    browsers: ["Internet Explorer", "Netscape"],
  },
  {
    get: function (obj, prop) {
      // An extra property
      if (prop === "latestBrowser") {
        return obj.browsers[obj.browsers.length - 1];
      }

      //  O comportamento para armazenar o valor padrão
      return obj[prop];
    },
    set: function (obj, prop, value) {
      // An extra property
      if (prop === "latestBrowser") {
        obj.browsers.push(value);
        return true;
      }

      // Converta o valor se não for uma matriz
      if (typeof value === "string") {
        value = [value];
      }

      //  O comportamento para armazenar o valor padrão
      obj[prop] = value;

      // Indicate success
      return true;
    },
  },
);

console.log(products.browsers); // ['Internet Explorer', 'Netscape']
products.browsers = "Firefox"; // pass a string (by mistake)
console.log(products.browsers); // ['Firefox'] <- no problem, the value is an array

products.latestBrowser = "Chrome";
console.log(products.browsers); // ['Firefox', 'Chrome']
console.log(products.latestBrowser); // 'Chrome'
```

Encontrando um item de objeto em uma matriz por propriedade

Esta proxy estende uma matriz com alguns recursos de utilidade. Como você vê, você pode "definir" propriedades flexíveis sem usar `Object.defineProperties`. Este exemplo pode ser adaptado para encontrar uma linha de tabela por sua célula. Nesse caso, o alvo será `table.rows`

```
let products = new Proxy(
  [
    { name: "Firefox", type: "browser" },
    { name: "SeaMonkey", type: "browser" },
    { name: "Thunderbird", type: "mailer" },
  ],
  {
    get: function (obj, prop) {
      // O comportamento para retornar o valor; Prop geralmente é um inteiro
      if (prop in obj) {
        return obj[prop];
      }

      // Obter o número de produtos; Com products.length
      if (prop === "number") {
        return obj.length;
      }

      let result,
        types = {};

      for (let product of obj) {
        if (product.name === prop) {
```

```
          result = product;
        }
        if (types[product.type]) {
          types[product.type].push(product);
        } else {
          types[product.type] = [product];
        }
      }

      // Obtém um produto por nome
      if (result) {
        return result;
      }

      // Obtém produtos por tipo
      if (prop in types) {
        return types[prop];
      }

      // Obtém tipos de produto
      if (prop === "types") {
        return Object.keys(types);
      }

      return undefined;
    },
  },
);

console.log(products[0]); // { name: 'Firefox', type: 'browser' }
console.log(products["Firefox"]); // { name: 'Firefox', type: 'browser' }
console.log(products["Chrome"]); // undefined
console.log(products.browser); // [{ name: 'Firefox', type: 'browser' }, { name: 'SeaMonkey', type: 'browser' }]
console.log(products.types); // ['browser', 'mailer']
console.log(products.number); // 3
```

Um exemplo completo de lista de <code>traps</code>

Agora, para criar uma lista completa de amostra de `traps`, para fins didáticos, tentaremos propor um objeto *não nativo* que seja particularmente adequado para este tipo de operação: o objeto global `docCookies` criado por a "little framework" publicada na página`document.cookie`.

```
/*
  var docCookies = ... get the "docCookies" object here:
  https://developer.mozilla.org/pt-BR/docs/DOM/document.cookie#A_little_framework.3A_a_complete_cookies_reader.2Fwriter_with_full_unicode_support
*/

var docCookies = new Proxy(docCookies, {
  get: function (oTarget, sKey) {
    return oTarget[sKey] || oTarget.getItem(sKey) || undefined;
  },
  set: function (oTarget, sKey, vValue) {
    if (sKey in oTarget) {
      return false;
    }
    return oTarget.setItem(sKey, vValue);
  },
  deleteProperty: function (oTarget, sKey) {
    if (sKey in oTarget) {
      return false;
    }
    return oTarget.removeItem(sKey);
  },
  enumerate: function (oTarget, sKey) {
    return oTarget.keys();
  },
  ownKeys: function (oTarget, sKey) {
    return oTarget.keys();
  },
  has: function (oTarget, sKey) {
    return sKey in oTarget || oTarget.hasItem(sKey);
  },
  defineProperty: function (oTarget, sKey, oDesc) {
    if (oDesc && "value" in oDesc) {
      oTarget.setItem(sKey, oDesc.value);
    }
    return oTarget;
  },
  getOwnPropertyDescriptor: function (oTarget, sKey) {
    var vValue = oTarget.getItem(sKey);
    return vValue
      ? {
          value: vValue,
          writable: true,
          enumerable: true,
          configurable: false,
        }
```

```
      : undefined;
  },
});

/* Teste Cookies */

console.log((docCookies.my_cookie1 = "First value"));
console.log(docCookies.getItem("my_cookie1"));

docCookies.setItem("my_cookie1", "Changed value");
console.log(docCookies.my_cookie1);
```

Notas Especificas Gecko

- No momento, `Object.getPrototypeOf(proxy)` retorna incondicionalmente um `Object.getPrototypeOf(target)`, porque a trap ES2015 getPrototypeOf não foi implementada (Erro do Firefox 795904, Erro do Firefox 888969).
- `Array.isArray(proxy)` retorna incondicionalmente um `Array.isArray(target)` (Erro do Firefox 1096753, Erro do Firefox 1111785).
- `Object.prototype.toString.call(proxy)` retorna incondicionalmente `Object.prototype.toString.call(target)`, porque ES2015 Symbol.toStringTag não foi implementado (Erro do Firefox 1114580).

Veja também

- "Proxies are awesome" Brendan Eich presentation at JSConf (slides)
- ECMAScript Harmony Proxy proposal page e ECMAScript página de semântica de proxy Harmony
- Tutorial em proxies
- SpiderMonkey specific Old Proxy API
- `Object.watch()` É um recurso não padrão, mas foi suportado no Gecko há muito tempo.

<code>Nota de licença</code>

Alguns conteúdos (texto, exemplos) nesta página foram copiados ou adaptados do ECMAScript wiki que contém à licença de conteúdo CC 2.0 BY-NC-SA.

ReferenceError

O objeto **`ReferenceError`** representa um erro quando uma variável não existente é referenciada.

Sintaxe

```
new ReferenceError([message[, fileName[, lineNumber]]])
```

Parâmetros

`message`

Opcional. Descrição legível do erro.

`fileName`

Opcional. O nome do arquivo contendo o código que causou a excessão.

`lineNumber`

Opcional. O número da linha do código que causou a excessão.

Descrição

O `ReferenceError` acontece quando é feita uma tentativa de referenciar uma variável que não foi declarada.

Propriedades

`ReferenceError`

Permite a inclusão de propriedades em um objeto `ReferenceError`.

Métodos

O `ReferenceError` não contém métodos próprios, porém, ele herda alguns métodos através da cadeia de protótipos.

Instâncias do <code>ReferenceError</code>

Propriedades

Métodos

Exemplos

Capturando um <code>ReferenceError</code>

```
try {
  var a = variavelNaoDefinida;
} catch (e) {
  console.log(e instanceof ReferenceError); // true
  console.log(e.message); // "variavelNaoDefinida não está definida"
```

```
  console.log(e.name); // "ReferenceError"
  console.log(e.fileName); // "Scratchpad/1"
  console.log(e.lineNumber); // 2
  console.log(e.columnNumber); // 6
  console.log(e.stack); // "@Scratchpad/2:2:7\n"
}
```

Criando um <code>ReferenceError</code>

```
try {
  throw new ReferenceError("Olá", "arquivoQualquer.js", 10);
} catch (e) {
  console.log(e instanceof ReferenceError); // true
  console.log(e.message); // "Olá"
  console.log(e.name); // "ReferenceError"
  console.log(e.fileName); // "arquivoQualquer.js"
  console.log(e.lineNumber); // 10
  console.log(e.columnNumber); // 0
  console.log(e.stack); // "@Scratchpad/2:2:9\n"
}
```

Veja também

- `Error`
- `ReferenceError`

Reflect

Reflect é um objeto nativo que provê métodos para operações JavaScript interceptáveis. Os métodos são os mesmos que o dos manipuladores de Proxy. `Reflect` não é um objeto de função, então não é construtível.

Descrição

Ao contrário da maioria dos objetos globais, `Reflect` não é um construtor. Você não pode usá-lo com o operador new ou invocar o objeto `Reflect` como uma função. Todas as propriedades e métodos do `Reflect` são estáticos (igual o objeto `Math`).

O objeto `Reflect` provê as seguintes funções estáticas as quais tem os mesmos nomes usados pelos métodos manipuladores de Proxy.

Alguns deste métodos são também os mesmos correspondentes aos métodos em `Object`, embora eles tenham diferenças sutis entre eles.

Métodos

`Reflect.apply(target, thisArgument, argumentsList)`

Chama uma função de destino com os argumentos, conforme especificado pelo parâmetro *`argumentsList`*. Veja também `Function.prototype.apply()`.

`Reflect.construct(target, argumentsList[, newTarget])`

O operador new como uma função. Equivalente a chamada `new target(...args)`. Também possui a opção de especificar um `prototype` diferente

`Reflect.defineProperty(target, propertyKey, attributes)`

Similar ao `Object.defineProperty()`. Retorna um `Boolean` com o valor `true` *se a propriedade foi definida com sucesso.*

`Reflect.deleteProperty(target, propertyKey)`

O operador delete como uma função. Equivalente a chamada `delete target[name]`.

`Reflect.get()`, "Reflect.get(*target*, *propertyKey*[, *receiver*])"}}

Uma função que retorna o valor da propriedade. Funciona como obter uma propriedade de um objeto (`target[propertyKey]`) como uma função.

`Reflect.getOwnPropertyDescriptor(target, propertyKey)`

Similar ao `Object.getOwnPropertyDescriptor()`. Retorna um descritor de propriedade da propriedade dada se existir no objeto, `undefined` caso contrário.

`Reflect.getPrototypeOf(target)`

Igual ao `Object.getPrototypeOf()`.

`Reflect.has(target, propertyKey)`

O operador in como função. Retorna um `Boolean` indicando se existe uma propriedade própria ou herdada.

`Reflect.isExtensible(target)`

Igual ao `Object.isExtensible()`. Retorna um `Boolean` com o valor `true` se o destino (parâmetro target) for extensível.

`Reflect.ownKeys(target)`

Retorna uma matriz das chaves de propriedade do próprio objeto de destino (não herdadas).

`Reflect.preventExtensions(target)`

Similar ao `Object.preventExtensions()`. Retorna um `Boolean` com o valor `true` se a atualização foi bem sucedida.

`Reflect.set(target, propertyKey, value[, receiver])`

Uma função que atribui valores a propriedades. Retorna um `Boolean` com o valor `true` se a atualização foi bem sucedida.

`Reflect.setPrototypeOf(target, prototype)`

Uma função que define o protótipo de um objeto. Retorna um `Boolean` com o valor `true` se a atualização foi bem sucedida.

Exemplos

Verificando se um objeto contém determinadas propriedades

```
const duck = {
  name: "Maurice",
  color: "white",
  greeting: function () {
    console.log(`Quaaaack! My name is ${this.name}`);
  },
};

Reflect.has(duck, "color");
// true
Reflect.has(duck, "haircut");
// false
```

Retornando as próprias chaves do objeto

```
Reflect.ownKeys(duck);
// [ "name", "color", "greeting" ]
```

Adicionando uma nova propriedade ao objeto

```
Reflect.set(duck, "eyes", "black");
// returns "true" if successful
// "duck" now contains the property "eyes: 'black'"
```

Veja também

- The `Proxy` global object.
- The `handler` object.

Reflect.apply()

O método estático **`Reflect.apply()`** chama uma função alvo com os argumentos especificados.

Sintaxe

```
Reflect.apply(target, thisArgument, argumentsList)
```

Parâmetros

target

Função que será chamada.

thisArgument

O valor de "`this`" que será usado pela function do target.

argumentsList

Um objeto do tipo array que especifica os argumentos com que o *`target`* deve ser chamado.

Valor de retorno

O resultado da função alvo chamada com o **`this`** e argumentos especificados.

Exceções

Um `TypeError`, se a função especificada no *target* não for invocável.

Descrição

No ES5, tipicamente é usado o método `Function.prototype.apply()` para chamar uma função com o valor de `this e argumentos` fornecidos como um array (ou um array-like object).

```
Function.prototype.apply.call(Math.floor, undefined, [1.75]);
```

Com o `Reflect.apply` isso se torna menos verboso e mais fácil de entender.

Exemplos

Usando <code>Reflect.apply()</code>

```
Reflect.apply(Math.floor, undefined, [1.75]);
// 1;

Reflect.apply(String.fromCharCode, undefined, [104, 101, 108, 108, 111]);
// "hello"

Reflect.apply(RegExp.prototype.exec, /ab/, ["confabulation"]).index;
// 4

Reflect.apply("".charAt, "ponies", [3]);
// "i"
```

Veja também

- `Reflect`
- `Function.prototype.apply()`

Reflect.construct()

The static **`Reflect.construct()`** method acts like the `new` operator, but as a function. It is equivalent to calling `new target(...args)`. It gives also the added option to specify a different prototype.

Experimente

```
function func1(a, b, c) {
  this.sum = a + b + c;
}

const args = [1, 2, 3];
const object1 = new func1(...args);
const object2 = Reflect.construct(func1, args);

console.log(object2.sum);
// Expected output: 6

console.log(object1.sum);
// Expected output: 6
```

Sintaxe

```
Reflect.construct(target, argumentsList[, newTarget])
```

Parametros

`target`

A função alvo à ser chamada.

`argumentsList`

Um objeto tipo array que especifica com quais argumentos `target` deveria ser chamada.

`newTarget` Optional

O construtor de quem o protótipo deveria ser usado. Veja também o `new.target` operador. Se `newTarget` não estiver presente, será `target`.

Return value

A new instance of `target` (or `newTarget`, if present), initialized by `target` as a constructor with the given arguments.

Exceptions

A `TypeError`, if `target` or `newTarget` are not constructors.

Description

`Reflect.construct` allows you to invoke a constructor with a variable number of arguments (which would also be possible by using the spread operator combined with the new operator).

```
var obj = new Foo(...args);
var obj = Reflect.construct(Foo, args);
```

<code>Reflect.construct()</code> vs <code>Object.create()</code>

Prior to the introduction of `Reflect`, objects could be constructed using an arbitrary combination of constructor and prototype by using `Object.create()`.

```
function OneClass() {
  this.name = "one";
}

function OtherClass() {
  this.name = "other";
}
```

```
// Calling this:
var obj1 = Reflect.construct(OneClass, args, OtherClass);

// ...has the same result as this:
var obj2 = Object.create(OtherClass.prototype);
OneClass.apply(obj2, args);

console.log(obj1.name); // 'one'
console.log(obj2.name); // 'one'

console.log(obj1 instanceof OneClass); // false
console.log(obj2 instanceof OneClass); // false

console.log(obj1 instanceof OtherClass); // true
console.log(obj2 instanceof OtherClass); // true
```

However, while the end result is the same, there is one important difference in the process. When using `Object.create()` and `Function.prototype.apply()`, the `new.target` operator will point to `undefined` within the function used as the constructor, since the `new` keyword is not being used to create the object.

When invoking `Reflect.construct()`, on the other hand, the `new.target` operator will point to the `newTarget` parameter if supplied, or `target` if not.

```
function OneClass() {
  console.log("OneClass");
  console.log(new.target);
}
function OtherClass() {
  console.log("OtherClass");
  console.log(new.target);
}

var obj1 = Reflect.construct(OneClass, args);
// Output:
//     OneClass
//     function OneClass { ... }

var obj2 = Reflect.construct(OneClass, args, OtherClass);
// Output:
//     OneClass
//     function OtherClass { ... }

var obj3 = Object.create(OtherClass.prototype);
OneClass.apply(obj3, args);
// Output:
//     OneClass
//     undefined
```

Examples

Using <code>Reflect.construct()</code>

```
var d = Reflect.construct(Date, [1776, 6, 4]);
d instanceof Date; // true
d.getFullYear(); // 1776
```

See also

- `Reflect`
- `new`
- `new.target`

Reflect.defineProperty()

O método estático **`Reflect.defineProperty()`** é como o `Object.defineProperty()`, mas retorna um `Boolean`.

Experimente

```
const object1 = {};

if (Reflect.defineProperty(object1, "property1", { value: 42 })) {
  console.log("property1 created!");
  // Expected output: "property1 created!"
} else {
  console.log("problem creating property1");
}

console.log(object1.property1);
// Expected output: 42
```

Sintaxe

```
Reflect.defineProperty(target, propertyKey, attributes)
```

Parâmetros

`target`

O objeto de destino onde será definida a propriedade.

`propertyKey`

O nome da propriedade a ser definida ou modificada.

`attributes`

Os atributos para a propriedade que está sendo definida ou modificada.

Valor de retorno

Um `Boolean` indicando se a propriedade foi ou não definida com êxito.

Erros

Um `TypeError`, se `target` não for um `Object`.

Descrição

O método `Reflect.defineProperty` permite a adição precisa ou a modificação de uma propriedade em um objeto. Para mais detalhes veja o `Object.defineProperty`, que é semelhante.

> **Nota: Uma diferença fundamental:** `Object.defineProperty` retorna o objeto ou lança um `TypeError` se a propriedade não tiver sido definida com êxito. `Reflect.defineProperty`, no entanto, simplesmente retorna um `Boolean` indicando se a propriedade foi ou não definida com êxito.

Exemplos

Usando <code>Reflect.defineProperty()</code>

```
let obj = {};
Reflect.defineProperty(obj, "x", { value: 7 }); // true
obj.x; // 7
```

Verificando se a definição da propriedade foi bem-sucedida

Com o `Object.defineProperty`, que retorna um objeto se for bem-sucedido ou lança um `TypeError`, você usaria um bloco try...catch para detectar qualquer erro que ocorreu ao definir uma propriedade.

Como `Reflect.defineProperty` retorna um status de sucesso booleano, você pode usar apenas um bloco `if...else` aqui:

```
if (Reflect.defineProperty(target, property, attributes)) {
  // success
} else {
  // failure
}
```

Veja também

- `Reflect`
- `Object.defineProperty()`

Reflect.set()

O método estático **`Reflect.set()`** define uma propriedade em um objeto.

Sintaxe

```
Reflect.set(alvo, propriedade, valor[, receptor])
```

Parâmetros

`alvo`

O objeto alvo onde a propriedade será definida.

`propriedade`

O nome da propriedade a ser definida.

valor

o valor a ser definido para a propriedade.

receptor

O valor do `this` fornecido para a chamada do alvo se um setter é encontrado.

Retorno

Um `Boolean` indicando se a definicão da propriedade ocorreu com sucesso ou não.

Exceções

Um `TypeError`, se o `alvo` não for um `Object`.

Descrição

O método `Reflect.set` permite que você defina uma propriedade em um objeto. Ele define a propriedade e is like the property accessor syntax as a function.

Exemplos

Usando <code>Reflect.set()</code>

```
// Object
var obj = {};
Reflect.set(obj, "prop", "value"); // true
obj.prop; // "value"

// Array
var arr = ["duck", "duck", "duck"];
Reflect.set(arr, 2, "goose"); // true
arr[2]; // "goose"

// É possível truncar o array
Reflect.set(arr, "length", 1); // true
arr; // ["duck"];

// Com apenas um argumento, propertKey e valor são undefined
var obj = {};
Reflect.set(obj); // true
Reflect.getOwnPropertyDescriptor(obj, "undefined");
// { value: undefined, writable: true, enumerable: true, configurable: true }
```

Veja também

- `Reflect`
- Property accessors

RegExp

Sumário

O construtor **`RegExp`** cria um objeto de expressão regular para realizar uma correspondência de texto com um padrão.

Para uma introdução à expressões regulares, leia o capítulo de Expressões Regulares no Guia de JavaScript.

Construtor

Notações literais e de construtores são possíveis:

```
/padrão/flags
new RegExp(padrão[, flags])
RegExp(padrão[, flags])
```

Parâmetros

`padrão`

O texto da expressão regular, ou como em ES5, outro objeto RegExp. Os padrões podem incluir caracteres especiais para correspondencia em um intervalo maior de valores do que uma string literal.

`flags`

Se especificado, `flags`indica os marcadores que podem ser adicionados, ou se um objeto é suficiente para o padrão, os valores de flags serão substituidos por qualquer uma das flags de objetos. O valor de `flags`é uma string contendo qualquer combinação dos seguintes valores:

`g`

corresponder globalmente; acha **todas** as correspondências em vez de parar após achar a primeira

`i`

ignorar maiúsc./minúsc.; Se a *flag u* estiver ativa, deve ser utilizado o *Unicode case folding*

`m`

multilinha; trata caracteres de início e fim (^ e $) ao operar sobre múltiplas linhas (ou seja, corresponder o início ou fim de *cada* linha (delimitado por \n ou \r), e não apenas o começo ou fim de toda a string de entrada)

`u`

unicode; trata o padrão como uma sequência de código unicode

`y`

aderente; corresponde apenas pelo index indicado pela propriedade `lastIndex` dessa expressão regular na string alvo (e não tenta corresponder de qualquer indexes posteriores).

Descrição

Há dois modos de criar um objeto `RegExp`: uma notação literal e um construtor. Para indicar strings, os parâmetros para a notação literal não usam aspas, enquanto os parâmetros para a função do construtor usam. Então, as seguintes expressões criam a mesma expressão regular:

```
/ab+c/i;
new RegExp("ab+c", "i");
```

A notação literal compila a expressão regular em tempo de execução. Use a notação literal quando a expressão regular permanecerá constante. Por exemplo, se você usar a notação literal para construir a expressão regular usada em um *loop*, a expressão regular não será recompilada a cada iteração

O construtor do objeto da expressão regular, por exemplo, `new RegExp('ab+c')`, fornece uma compilação em tempo de execução da expressão regular. Use a função construtora quando você sabe que o padrão da expressão regular será mudado, ou você não sabe o padrão e o está recebendo de outra fonte, como uma entrada do usuário.

Começando com ECMAScript 6, `new RegExp(/ab+c/, 'i')`, deixou de lançar um `TypeError` ("can't supply flags quando constructing one RegExp from another") quando o primeiro argumento é um `RegExp` e o segundo argumento `flags` está presente. Uma nova `RegExp` dos argumentos é criada ao invés disso.

Quando se usa a função construtora, as regras de escapar em uma string (preceder caracteres especiais com \ quando incluídos na string) são necessárias. Por exemplo, as declarações a seguir são equivalentes:

```
var re = /\w+/;
var re = new RegExp("\\w+");
```

Significado dos caracteres especiais nas expressões regulares

- Classes de Caracteres
- Conjuntos de Caracteres
- Limites
- Agrupamentos e back references
- Quantificadores

Classes de Caracteres	
Caractere	**Significado**
`.`	(O ponto) corresponde um único caracter qualquer *exceto* os caracteres de nova linha: `\n`, `\r`, `\u2028` ou `\u2029`. Note que a flag multilinha `m` não muda o comportamento do ponto. Então para corresponder um padrão por múltiplas linhas, o conjunto de caracteres `[^]` pode ser usado, que corresponderá qualquer caractere, incluindo novas linhas. Por exemplo, `/.y/` corresponde "my" e "ay", mas não "yes", em "yes make my day".
`\d`	Corresponde um caractere de dígito no alfabeto basic Latin. Equivalente a `[0-9]`. Por exemplo, `/\d/` ou `/[0-9]/` corresponde "2" em "B2 é o número da suíte".
`\D`	Corresponde qualquer caractere que não é um dígito no alfabeto basic Latin. Equivalente a `[^0-9]`. Por exemplo, `/\D/` ou `/[^0-9]/` corresponde "B" em "B2 é o suite number".
`\w`	Corresponde qualquer caractere alfanumérico do alfabeto basic Latin, incluindo o underline. Equivalente a `[A-Za-z0-9_]`. Por exemplo, `/\w/` corresponde "a" em "apple", "5" em "$5.28", e "3" em "3D".
`\W`	Corresponde qualquer caractere que não é um alfanumérico do alfabeto basic Latin. Equivalente a `[^A-Za-z0-9_]`. Por exemplo, `/\W/` ou `/[^A-Za-z0-9_]/` corresponde "%" em "50%".
`\s`	Corresponde um único caractere de espaço em branco, incluindo espaço, tabulação (tab), quebra de página, nova linha (LF) e outros espaços Unicode. Equivalente a `[ \f\n\r\t\v\u00a0\u1680\u180e\u2000\u2001\u2002\u2003\u2004\u2005\u2006\u2007\u2008\u2009\u200a\u2028\u2029\u202f\u205f\u3000]`. Por exemplo, `/\s\w*/` corresponde " bar" em "foo bar".

`\S`	Corresponde um único caractere que não seja um espaço em branco. Equivalente a `[^\f\n\r\t\v\u00a0\u1680\u180e\u2000\u2001\u2002\u2003\u2004\u2005\u2006\u2007\u2008\u2009\u200a\u2028\u2029\u202f\u205f\u3000]`. Por exemplo, `/\S\w*/` corresponde "foo" em "foo bar".
`\t`	Corresponde uma tabulação.
`\r`	Corresponde uma quebra de linha.
`\n`	Corresponde uma nova linha.
`\v`	Corresponde uma tabulação vertical.
`\f`	Corresponde uma quebra de página.
`[\b]`	Corresponde um caracter backspace. (Não confundir com `\b`)
`\0`	Corresponde um caractere NUL. Não coloque outro dígito seguinte a esse.
`\c`*X*	Onde *X* é uma letra de A - Z. Corresponde um caractere de controle em uma string. Por exemplo, `/\cM/` corresponde control-M em uma string.
`\x`*hh*	Corresponde o caractere com o código *hh* (dois dígitos hexadecimais).
`\u`*hhhh*	Corresponde o caractere com o valor Unicode *hhhh* (quatro dígitos hexadecimais).
`\`	Para caracteres que são geralmente tratados literalmente, indica que o próximo caractere é especial e não deve ser interpretado literalmente. Por exemplo, `/b/` corresponde o caractere "b". Colocando uma barra invertida antes do "b", ou seja, usando `/\b/`, o caractere se torna especial, significando corresponder o limite de uma palavra. *ou* Para caracteres que são geralmente tratados especialmente, indica que o próximo caractere não é especial e deve ser interpretado literalmente. Por exemplo, "*" é um caractere especial que significa 0 ou mais ocorrências do caractere precedente devem ser correspondidas; por exemplo, `/a*/` significa corresponder 0 ou mais "a"s. Para corresponder `*` literalmente, preceda-o com uma barra invertida; por exemplo, `/a\*/` corresponde "a*".
Conjuntos de Caracteres	
Caractere	**Significado**
`[xyz]`	Um conjunto de caracteres. Corresponde qualquer um dos caracteres cercados. Você pode especificar uma extensão de caracteres usando um hífen. Por exemplo, `[abcd]` é o mesmo que `[a-d]`. Eles correspondem o "b" em "banco" e o "c" em "cortar".
`[^xyz]`	Um conjunto de caracteres negativo ou complementado. Isto é, corresponde qualquer coisa que não esteja cercada nos colchetes. Você pode especificar uma extensão de caracteres usando um hífen. Por exemplo, `[^abc]` é o mesmo que `[^a-c]`. Eles inicialmente correspondem "n" em "banco" e "o" em "cortar".
Limites	
Caractere	**Significado**
`^`	Corresponde o início de uma entrada. Se a flag multilinha é utilizada, também corresponde imediatamente após um caractere de quebra de linha. Por exemplo, `/^A/` não corresponde o "A" em "an A", mas corresponde o primeiro "A" em "An A".
`$`	Corresponde o fim de uma entrada. Se a flag multilinha é utilizada, também corresponde imediatamente antes de um caractere de quebra de linha. Por exemplo, `/o$/` não corresponde o "o" em "cantor", mas corresponde em "canto".
`\b`	Corresponde um limite de palavra de largura zero, como entre uma letra e um espaço. (Não confundir com `[\b]`) Por exemplo, `/\bno/` corresponde o "no" em "de noite"; `/ly\b/` corresponde o "ly" em "possibly yesterday".
`\B`	Corresponde um limite de uma não palavra de largura zero, como entre duas letras ou entre dois espaços. Por exemplo, `/\Bte/` corresponde "te" em "de noite", e `/on\B/` corresponde "on" em "possivelmente ontem".
Agrupamentos e back references	

Caractere	Significado
(*x*)	Corresponde *x* e memoriza a correspondência. Esses são chamados parênteses de captura. Por exemplo, `/(foo)/` corresponde e memoriza "foo" em "foo bar". A substring correspondida pode ser chamada novamente dos elementos do array resultante `[1], ..., [n]` ou das propriedades predefinidas do objeto `RegExp` `$1, ..., $9`. Grupos de captura têm uma falta na performance. Se você não necessita que a substring correspondida seja chamada novamente, prefira parênteses de não-captura (veja mais abaixo).
n	Onde *n* é um inteiro positivo. A back reference to o last substring matching o n parenthetical no expressão regular (counting left parentheses). Por exemplo, `/apple(,)\sorange\1/` corresponde "apple, orange," em "apple, orange, cherry, peach". Um exemplo mais completo está a seguir nesta tabela.
(?:*x*)	Corresponde *x* mas não memoriza a correspondência. Esses são chamados parênteses de não-captura. A substring correspondida não pode ser chamada novamente dos elementos do array resultante `[1], ..., [n]` ou das propriedades predefinidas do objeto `RegExp` `$1, ..., $9`.
Quantificadores	
Caractere	**Significado**
*x**	Corresponde o item precedente *x* zero ou mais vezes. Por exemplo, `/assusto*/` corresponde "assustoooo" em "Um fantasma assustoooou" e "assust" em "Não me assustei", mas nada em "Um bode grunhiu". Note que no caso acima, o único caractere a ser selecionado é "o", pois ele é o caractere imediatamente anterior ao asterisco e não há qualquer operador de agrupamento que indique que mais caracteres deveriam ser submetidos à repetição. Se quiséssemos selecionar a ocorrência sequencial da palavra completa "blah", por exemplo, poderíamos utilizar algum agrupamento como `/(blah)*/` o que geraria a seleção de "blahblahblah" na frase "blahblahblah ração para gatos" ou de "blah" na frase "algoritmo blahut-arimoto".
x+	Corresponde o item precedente *x* uma ou mais vezes. Equivalente a `{1,}`. Por exemplo, `/a+/` corresponde o "o" em "doce" e todos os "o"s em "dooooooooce".
x?	Corresponde o item precedente *x* nenhuma ou uma vez. Por exemplo, `/e?le?/` corresponde o "el" em "angel" e o "le" em "angle." If used imediatamente after qualquer dos quantifiers `*`, `+`, `?`, ou `{}`, makes o quantifier non-greedy (matching o minimum number of vezes), como opposed to o default, which é greedy (matching o maximum number of vezes). Also used em lookahead assertions, described under `(?=)`, `(?!)`, e `(?:)` em this table.
x(?=*y*)	Corresponde *x* apenas se *x* é seguido por *y*. Por exemplo, `/Jack(?=Sprat)/` corresponde "Jack" apenas se for seguido por "Sprat". `/Jack(?=Sprat|Frost)/` corresponde "Jack" apenas se for seguido por "Sprat" ou "Frost". Porém, nem "Sprat" nem "Frost" são partes do resultado da correspondência.
x(?!*y*)	Corresponde *x* apenas se *x* não é seguido por *y*. Por exemplo, `/\d+(?!\.)/` corresponde um número apenas se não for seguido por um ponto. `/\d+(?!\.)/.exec('3.141')` corresponde "141", mas não "3.141".
x|*y*	Corresponde *x* ou *y*. Por exemplo, `/verde|vermelha/` corresponde "verde" em "maçã verde" e "vermelha" em "maçã vermelha". Entretanto, a primeira expressão tem preferência. Se uma string fosse corresponder às duas expressões, ela vai corresponder à que aparece do lado esquerdo do operador |. Por exemplo, `/maçãs|maçã/` corresponde "maçãs" na frase "*maçãs* vermelhas" e "maçã" na frase "*maçã* verde".
x{*n*}	Onde *n* é um número inteiro positivo. Corresponde exatamente *n* ocorrências do item precedente *x*. Por exemplo, `/o{2}/` não corresponde o "o" em "brigadeiro", mas corresponde todos os dois "o"s em "brigadeir*oo*", e o dois primeiros "o"s em "brigadeir*oo*o".
x{*n*,}	Onde *n* é um número inteiro positivo. Corresponde *pelo menos n* ocorrências do item precedente *x* (sem que haja um limite superior). Por exemplo, `/o{2,}/` não corresponde o "o" em "brigadeiro", mas corresponde todos os "o"s em "brigadeir*oo*" e em "brigadeir*oooooooo*".

x{*n*,*m*}	Onde *n* e *m* são números inteiros positivos. Corresponde pelo menos *n* e no máximo *m* ocorrências do item precedente *x*. Por exemplo, `/o{2,4}/` corresponde nada em "brigadeiro", os dois "o"s em "brigadeir*oo*", os três "o"s em "brigadeir*ooo*", e os primeiros quatro "o"s em "brigadeir*oooo*o". É importante perceber que no último caso a correspondência não inclui o último "o" de "brigadeiroooo". Isso se deve ao fato do operador quantificador ter definido o número máximo de ocorrências como 4, ignorando a quinta repetição do caractere.
*x**? *x*+? *x*?? *x*{*n*,}? *x*{*n*,*m*}?	**Operadores *non-greedy* ou *lazy* (não-gulosos ou preguiçosos)** Esses operadores com a `?` no final, operam de maneira semelhante aos seus análogos sem `?` mostrados acima, correspondendo múltiplas ocorrências do item precedente *x*. Entretanto, desta vez a ocorrência selecionada será a mínima possível. No exemplo `/(blah)*?/`, diante da frase "blahblahblah ração para gatos", nada seria selecionado, pois a ocorrência mínima aceita pelo operador `*?` seria ocorrência de 0 (zero) vezes da *string* "blah", o que resulta no nada. Já a expressão regular `/(blah)+?/`, diante da mesma frase, corresponderia a "blah", que é a ocorrência mínima aceita pelo operador `+?`, ou seja, 1 (uma) repetição da sequência "blah". Em outro exemplo com mais aplicação prática, digamos que se quer corresponder todos os termos que ficam entre aspas em um texto. Se fizéssemos simplesmente a *regex* `/".*"/`, diante de um texto com múltiplas ocorrências de termos entre aspas, como: '*Eu* "*gosto*" *muito de* "*estudar*" *regex*', a nossa expressão regular seria gulosa e selecionaria o maior texto possível que correspondesse à definição, nesse caso, selecionando '"*gosto*" *muito de* "*estudar*"', pois todo esse texto está compreendido entre a primeira aspa (antes de '*gosto*') e a última aspa (após '*estudar*'), o que é um resultado talvez indesejado. Se usarmos a regex `/".*?"/`, com o operador não-guloso, as correspondências para a mesma frase seriam '"*gosto*"' e '"*estudar*"' separadamente, conforme era a intenção inicial.

Propriedades

`RegExp`

Permite a adição de propriedades a todos os objetos.

`RegExp.length`

O valor of `RegExp.length` é 2.

Métodos

O objeto global `RegExp não possui métodos próprios, no entanto, herda alguns métodos através da cadeia de prototype.`

Objetos e instancias

Propriedades

Métodos

Exemplos

Exemplo: Usando uma expressão regular para mudar o fomato dos dados

O seguinte script usa o método `replace()` da instância de `String` para casar o nome no format *nome sobrenome* e produzir no formato *sobrenome, nome*. No texto substituto, o script usa `$1` e `$2` para indicar os respectivos parênteses de casamento no padrão da expressão regular.

```
var re = /(\w+)\s(\w+)/;
var str = "John Smith";
var newstr = str.replace(re, "$2, $1");
console.log(newstr);
```

Isto retornará "Smith, John".

Exemplo: Usando expressões regulares para quebrar linhas com diferentes fins de linha/quebras de linha

O final de linha padrão depende da plataforma utilizada (Unix, Windows, etc.). A divisão(*split*) de linha fornecida neste exemplo funciona com todas as plataformas.

```
var text = "Um texto\nE mais um pouco\r\nE ainda mais\rEsse é o fim";
var lines = text.split(/\r\n|\r|\n/);
console.log(lines); // prints [ 'Um texto', 'E mais um pouco', 'E ainda mais', 'Esse é o fim' ]
```

Note que a ordem dos padrões na expressão regular importa.

Exemplo: Usando expressão regular sobre múltiplas linhas

```
var s = "Please yes\nmake my day!";
s.match(/yes.*day/);
// Retorna null
s.match(/yes[^]*day/);
// Retorna'yes\nmake my day'
```

Exemplo: Using a expressão regular com o "sticky" flag

Este exemplo mostra como utilizar a sticky flag em expressões regulares.

```
var text = "First line\nSecond line";
var regex = /(\S+) line\n?/y;

var match = regex.exec(text);
console.log(match[1]); // prints 'First'
console.log(regex.lastIndex); // prints '11'

var match2 = regex.exec(text);
console.log(match2[1]); // prints 'Second'
console.log(regex.lastIndex); // prints '22'

var match3 = regex.exec(text);
console.log(match3 === null); // prints 'true'
```

One can test at run-time whether o sticky flag é supported, using `try { … } catch { … }`. Para this, either an `eval(…)` expression ou o `RegExp(regex-string, flags-string)` syntax must be used (since o `/regex/flags` notation é processed at compile-time, so throws an exception before o `catch` block é encountered). Por exemplo:

```
var supports_sticky;
try {
  RegExp("", "y");
  supports_sticky = true;
} catch (e) {
  supports_sticky = false;
}
console.log(supports_sticky); // prints 'true'
```

Exemplo: Expressão regular e Unicode caracteres

As mentioned above, `\w` ou `\W` only corresponde ASCII based caracteres; por exemplo, "a" to "z", "A" to "Z", "0" to "9" e "_". To match caracteres from other languages such como Cyrillic ou Hebrew, use `\uhhhh`, onde "hhhh" é o caractere's Unicode valor em hexadecimal. This exemplo demonstrates how one can separate out Unicode caracteres from uma palavra.

```
var text = "Образец text на русском языке";
var regex = /[\u0400-\u04FF]+/g;

var match = regex.exec(text);
console.log(match[0]); // prints 'Образец'
console.log(regex.lastIndex); // prints '7'

var match2 = regex.exec(text);
console.log(match2[0]); // prints 'на' [não print 'text']
console.log(regex.lastIndex); // prints '15'

// e assim vai
```

Here's an external resource para getting o complete Unicode block range para different scripts: Regexp-unicode-block.

Exemplo: Extracting subdomain name from URL

```
var url = "http://xxx.domain.com";
console.log(/[^.]+/.exec(url)[0].substr(7)); // prints 'xxx'
```

Veja também

- Regular Expressions chapter no JavaScript Guide
- `String.prototype.match()`
- `String.prototype.replace()`

RegExp.prototype.compile()

O método **`compile()`** está depreciado, é usado para (re-)compilar uma expressão regular durante a execução de um script. É basicamente o mesmo que o construtor `RegExp`.

Sintaxe

```
regexObj.compile(pattern, flags)
```

Parâmetros

`pattern`

É o texto da expressão regular.

`flags`

Se especificado, as flags podem ter qualquer combinação dos seguintes valores:

`g`

global.

`i`

ignorar caso.

`m`

multilinha; trata os caracteres iniciais e finais como (^ e $), trabalhando sobre várias linhas (isto é, corresponde ao início ou ao final de cada linha (delimitado por \n ou \r), não apenas o começo, ou o final de toda a cadeia de entrada).

`y`

sticky; corresponde apenas ao índice indicado pela propriedade `lastIndex` dessa expressão regular na cadeia de destino (e não tenta corresponder a partir de índices posteriores).

Descrição

O método de `compilação` é obsoleto. Você pode simplesmente usar o construtor `RegExp` para obter o mesmo efeito.

Exemplos

Usando <code>compile()</code>

O exemplo a seguir mostra como recompilar uma expressão regular com um novo padrão e um nova flag.

```
var regexObj = new RegExp("foo", "gi");
regexObj.compile("new foo", "g");
```

Veja também

- `RegExp`

RegExp.prototype.exec()

O método **`exec()`** executa a busca por um padrão em uma determinada string. Retorna um array, ou `null`.

Se você está precisa somente de um retorno verdadeiro/falso, use o método `RegExp.prototype.test()` ou `String.prototype.search()`.

Sintaxe

```
regexObj.exec(string)
```

Parâmetros

`string`

A string para comparar com a expressão regular

Valor retornado

Se a combinação acontecer, o método `exec()` o método retorna um array e atualiza as propriedades do objeto da expressão regular. Esse array retornado possui o texto combinado como primeiro item e depois um item para cada captura contendo o respectivo texto.

Se falhar, o retorno do método `exec()` será `null`.

Descrição

Considere o exemplo abaixo:

```
// Encontra combinações "quick brown" seguido de "jumps", ignorando caracteres entre eles
// Relembra "brown" e "jumps"
// Ignora caixa (maiúsculo e minúsculo)
var re = /quick\s(brown).+?(jumps)/gi;
var result = re.exec("The Quick Brown Fox Jumps Over The Lazy Dog");
```

A tabela a seguir provê os resultados do script:

Objeto	Propriedade/ Índice	Descrição	Exemplo
`result`	`[0]`	A string completa dos caracteres encontrados	`Quick Brown Fox Jumps`
	`[1], ...[n ]`	As combinações de substrings parametrizadas encontradas, se existir. A quantidade de possíveis substrings parametrizadas é ilimitado.	`[1] = Brown` `[2] = Jumps`
	`index`	O índice base 0 do valor encontrado na string.	`4`
	`input`	String original	`The Quick Brown Fox Jumps Over The Lazy Dog`
`re`	`lastIndex`	O índice que começa a próxima combinação encontrada. Quando "`g`" não é definido, este valor será sempre 0.	`25`
	`ignoreCase`	Indica se a flag "`i`" foi usada para ignorar caixa alta/baixa.	`true`

	`global`	Indica se a flag "`g`" foi usada para encontrar combinações de forma global.	`true`
	`multiline`	Indica se a flag "`m`" foi usada para pesquisar em strings de diversas linhas.	`false`
	`source`	Texto do padrão.	`quick\s(brown).+?(jumps)`

Exemplos

Procurando combinações sucessivas

If your regular expression uses the "`g`" flag, you can use the `exec()` method multiple times to find successive matches in the same string. When you do so, the search starts at the substring of `str` specified by the regular expression's `lastIndex` property (`test()` will also advance the `lastIndex` property). For example, assume you have this script:

```
var myRe = /ab*/g;
var str = "abbcdefabh";
var myArray;
while ((myArray = myRe.exec(str)) !== null) {
  var msg = "Found " + myArray[0] + ". ";
  msg += "Next match starts at " + myRe.lastIndex;
  console.log(msg);
}
```

This script displays the following text:

```
Found abb. Next match starts at 3
Found ab. Next match starts at 9
```

Nota: Do not place the regular expression literal (or `RegExp` constructor) within the `while` condition or it will create an infinite loop if there is a match due to the `lastIndex` property being reset upon each iteration. Also be sure that the global flag is set or a loop will occur here also.

Usando <code>exec()</code> com <code>RegExp</code> literais

You can also use `exec()` without creating a `RegExp` object:

```
var matches = /(hello \S+)/.exec("This is a hello world!");
console.log(matches[1]);
```

This will log a message containing 'hello world!'.

Veja também

- O capítulo de Expressões Regulares no Guia de Javascript.
- `RegExp`

RegExp.prototype.ignoreCase

O atributo **`ignoreCase`** indica se a flag "i" foi ou não utilizada com a expressão regular.`ignoreCase` é um atributo somente leitura de uma instância individual de expressãoregular.

Experimente

```
const regex1 = new RegExp("foo");
const regex2 = new RegExp("foo", "i");

console.log(regex1.test("Football"));
// Expected output: false

console.log(regex2.ignoreCase);
// Expected output: true

console.log(regex2.test("Football"));
// Expected output: true
```

Property attributes of `RegExp.prototype.ignoreCase`	
Writable	no
Enumerable	no
Configurable	yes

Descrição

O valor de `ignoreCase` é um `Boolean` `true` se a flag "`i`" foi usada; `false` caso contrário. A flag "i" indica que maiúsculas e minúsculas são equivalentes ao se tentar casar uma string com a expressão regular.

Você não pode alterar essa propriedade diretamente.

Exemplos

Usando <code>ignoreCase</code>

```
var regex = new RegExp("foo", "i");

console.log(regex.ignoreCase); // true
```

See also

- `RegExp.lastIndex`
- `RegExp.prototype.global`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`
- `RegExp.prototype.sticky`

RegExp.prototype.sticky

A propriedade **`sticky`** indica se a busca é "pegajosa" (percorre a string somente a partir do índice indicado pela propriedade `lastIndex` desta expressão regular). A propriedade `sticky` em um objeto de expressão regular é somente para leitura.

Experimente

```
const str1 = "table football";
const regex1 = new RegExp("foo", "y");

regex1.lastIndex = 6;

console.log(regex1.sticky);
// Expected output: true

console.log(regex1.test(str1));
// Expected output: true

console.log(regex1.test(str1));
// Expected output: false
```

Property attributes of `RegExp.prototype.sticky`	
Writable	no
Enumerable	no
Configurable	yes

Descrição

O valor de `sticky` é do tipo `Boolean` e será *true* quando a flag "y" for utilizada; senão, será *false*. A flag "y" indica que as correspondências ocorrerão apenas a partir do indice indicado pela propriedade `lastIndex` desta expressão regular na string alvo (e não buscará correspondência em nenhum índice anterior). Uma expressão regular definida como `sticky` e `global` ignora a flag `global`.

Você não pode alterar essa propriedade diretamente. Ela é somente para leitura.

Exemplos

Uilizando uma expressão regular com a flag sticky

```
var str = "#foo#";
var regex = /foo/y;

regex.lastIndex = 1;
regex.test(str); // true
regex.lastIndex = 5;
regex.test(str); // false (lastIndex é levado em conta com a flag sticky)
regex.lastIndex; // 0 (reinicia quando não ocorre correspondência)
```

Flag sticky ancorada

Por diversas versões, a engine SpiderMonkey do Firefox apresentou um bug na asserção de ^ com a flag *sticky* que fazia expressões iniciando com ^ e usando a flag *sticky* encontrarem correspondências onde não deveriam. O bug foi introduzido algum tempo após o Firefox 3.6 (que possuía a flag sticky mas não apresentava o bug) e corrigido em 2015. Talvez por este motivo, a especificação ES2015 destaca especificamente que:

> Quando a flag `y` for usada em um padrão, ^ indica que a correspondência ocorrerá apenas no início da entrada, ou (se `multiline` for `true`) no início de uma linha.

Exemplos de comportamento esperado:

```
var regex = /^foo/y;
regex.lastIndex = 2;
regex.test("..foo"); // false - índice 2 não é o início da string

var regex2 = /^foo/my;
regex2.lastIndex = 2;
regex2.test("..foo"); // false - índice 2 não é o início da string nem da linha
regex2.lastIndex = 2;
regex2.test(".\nfoo"); // true - índice 2 é o início da linha
```

Veja também

- `RegExp.lastIndex`
- `RegExp.prototype.global`
- `RegExp.prototype.ignoreCase`
- `RegExp.prototype.multiline`
- `RegExp.prototype.source`

RegExp.prototype.test()

O método **test()** executa uma busca por uma correspondência entre uma expressão regular e uma string. Retorna `true` ou `false`.

Sintaxe

```
regexObj.test(str)
```

Parâmetros

`str`

> A string que confrontará a expressão regular. Todos os valores são convertidos em strings, portanto, omiti-lo ou passar undefined faz com que test() procure a string "undefined".

Retorno

Boolean.

`true` se a string `str` corresponde com o que está descrito na expressão regular.

Caso contrário, retorna `false`.

Descrição

Use test() sempre que você quiser saber se um padrão está dentro de uma string(similar ao método `String.prototype.search()`); Para mais informações (porém com execução mais lenta) use o método `exec()` (similar ao método `String.prototype.match()`). Assim como `exec()` (ou em combinação com ele), test() quando chamado várias vezes dentro da mesma instância de expressão regular irá avançar seu ponteiro para a correspondência anterior.

Exemplos

Usando <code>test()</code>

Exemplo simples que verifica se `"hello"` está presente no início de uma string, retornando um booleano como resultado.

```
const str = 'hello world!';
const result = /^hello/.test(str);

console.log(result); // true
```

O exemplo a seguir mostra uma mensagem dependendo do sucesso do teste.

```
function testinput(re, str) {
  var midstring;
  if (re.test(str)) {
    midstring = " Contém ";
  } else {
    midstring = " não contém ";
  }
  console.log(str + midstring + re.source);
}
```

Gecko-specific notes

A priori no Grecko 8.0, `test()` foi implementado incorretamente; quando chamado sem parâmetros, ele encontrária uma correspondência com o valor de entrada anterior (`RegExp.input` property) no lugar de uma correspondência com `"undefined"`. Isso está conrrigido; agora `/undefined/.test()` resultará em `true`, no lugar de um erro.

See also

- Regular Expressions chapter in the JavaScript Guide
- `RegExp`

RegExp.prototype.toString()

O método **`toString()`** retorna uma string que representa uma expressão regular.

Experimente

```
console.log(new RegExp("a+b+c"));
// Expected output: /a+b+c/

console.log(new RegExp("a+b+c").toString());
// Expected output: "/a+b+c/"
```

```
console.log(new RegExp("bar", "g").toString());
// Expected output: "/bar/g"

console.log(new RegExp("\n", "g").toString());
// Expected output (if your browser supports escaping): "/\n/g"

console.log(new RegExp("\\n", "g").toString());
// Expected output: "/\n/g"
```

Sintaxe

```
toString();
```

Valor retornado

Uma string representando o objeto dado.

Descrição

O objeto `RegExp` subscreve o método `toString()` do objeto `Object`; ele não herda `Object.prototype.toString()`. Para objetos `RegExp`, o método `toString()` retorna uma representação da expressão regular como string.

Exemplos

UsandotoString()

O exemplo a seguir exibe o valor em string de um objeto `RegExp`

```
const myExp = new RegExp("a+b+c");
console.log(myExp.toString()); // logs '/a+b+c/'
const foo = new RegExp("bar", "g");
console.log(foo.toString()); // logs '/bar/g'
```

Expressões regulares vazias e escape

Introduzindo o ECMAScript 5, uma expressão regular vazia retorna a string "/(?:)/" e terminadores de linha como "\n" são utilizados

```
new RegExp().toString(); // "/(?:)/"
new RegExp("\n").toString() === "/\n/"; // verdadeiro, antes do ES5
new RegExp("\n").toString() === "/\\n/"; // verdadeiro, introduzindo o ES5
```

Veja também

- `Object.prototype.toString()`

Set

Sumário

O objeto **`Set`** permite que você armazene valores *únicos* de qualquer tipo, desde valores primitivos a referências a objetos.

Sintaxe

```
new Set([iterable]);
```

Parâmetros

iterable

> Se um objeto iterável é passado, todos os seus elementos serão adicionados ao novo Set. Se tal parâmetro não for específicado, ou se seu valor for `null`, o novo `Set` estará vazio.

Descrição

Objetos `Set` são coleções de valores nas quais é possível iterar os elementos em ordem de inserção. Um valor no `Set` **pode ocorrer apenas uma vez**; ele é único na coleção do `Set`.

Igualdade de valores

Como cada valor no Set deve ser único, a igualdade será checada e não é baseada no mesmo algoritmo que aquele usado no operador `===`. Especificamente, para `Sets`, `+0` (que é exatamente igual a `-0`) e `-0` são valores diferentes. No entanto, isto foi modificado na última especificação ECMAScript 2015. Iniciado com o Gecko 29.0 (Erro do Firefox 952870) e pelo recent nightly Chrome, +0 e -0 são tratados com sendo o mesmo valor em objetos conjunto (Set). Também, `NaN` e `undefined` podem ser armazenados em um conjunto Set.

Propriedades

`Set.length`

> O valor da propriedade comprimento é 0.
>
> Para contar o número de elementos de um `Set`, use `Set.prototype.size`.

`get Set[@@species]`

A função construtora usada para criar objetos derivados.

`Set`

Representa o prototype do constructor do `Set`. Permite a adição de propriedades para todos os objetos do tipo `Set`.

Instâncias <code>Set</code>

Todas as instâncias de `Set` herdam de `Set`.

Propriedades

Métodos

Exemplos

Utilizando o objeto <code>Set</code>

```
var meuSet = new Set();

meuSet.add(1); // meuSet [1]
meuSet.add(5); // meuSet [1, 5]
meuSet.add(5); // 5 já foi adicionando, portanto, meuSet [1, 5]
meuSet.add("texto");
var o = { a: 1, b: 2 };
meuSet.add(o);

meuSet.add({ a: 1, b: 2 }); //  o está referenciando outro objeto

meuSet.has(1); // true
meuSet.has(3); // false, 3 não foi adicionado ao set (Conjunto)
meuSet.has(5); // true
meuSet.has(Math.sqrt(25)); // true
meuSet.has("Texto".toLowerCase()); // true
meuSet.has(o); // true

meuSet.size; // 5

meuSet.delete(5); // remove 5 do set
meuSet.has(5); // false, 5 já foi removido

meuSet.size; // 4, nós simplesmente removemos um valor

console.log(meuSet); // Set { 1, 'texto', { a: 1, b: 2 }, { a: 1, b: 2 } }
```

Iterando objetos <code>Set</code>

```
// iterar sobre os itens em set
// loga os itens na ordem: 1, "texto"
for (let item of meuSet) console.log(item);

// loga os itens na ordem: 1, "texto"
for (let item of meuSet.keys()) console.log(item);

// loga os itens na ordem: 1, "texto"
for (let item of meuSet.values()) console.log(item);

// loga os itens na ordem: 1, "texto"
//(key e value são os mesmos aqui)
for (let [key, value] of meuSet.entries()) console.log(key);

// converte Set para um Array
var meuArr = [v for (v of mySet)]; // [1, "some text"]

// o seguinte também funcionará se for executado em um documento HTML
mySet.add(document.body);
mySet.has(document.querySelector("body")); // true

// a conversão entre Set e Array
mySet2 = Set([1,2,3,4]);
mySet2.size; // 4
[...mySet2]; // [1,2,3,4]

// intersecção pode ser simulado via
var intersection = new Set([...set1].filter(x => set2.has(x)));

// Iterar entradas set com forEach
meuSet.forEach(function(value) {
  console.log(value);
});

// 1
// 2
```

```
// 3
// 4
```

Implementando operações básicas entre conjuntos

```
function isSuperset(set, subset) {
  for (var elem of subset) {
    if (!set.has(elem)) {
      return false;
    }
  }
  return true;
}

function uniao(setA, setB) {
  var _uniao = new Set(setA);
  for (var elem of setB) {
    _uniao.add(elem);
  }
  return _uniao;
}

function interseccao(setA, setB) {
  var _interseccao = new Set();
  for (var elem of setB) {
    if (setA.has(elem)) {
      _interseccao.add(elem);
    }
  }
  return _interseccao;
}

function diferencaSimetrica(setA, setB) {
  var _diferenca = new Set(setA);
  for (var elem of setB) {
    if (_diferenca.has(elem)) {
      _diferenca.delete(elem);
    } else {
      _diferenca.add(elem);
    }
  }
  return _diferenca;
}

function diferenca(setA, setB) {
  var _diferenca = new Set(setA);
  for (var elem of setB) {
    _diferenca.delete(elem);
  }
  return _diferenca;
}

//Exemplos
var setA = new Set([1, 2, 3, 4]),
  setB = new Set([2, 3]),
  setC = new Set([3, 4, 5, 6]);

isSuperset(setA, setB); // => true
uniao(setA, setC); // => Set [1, 2, 3, 4, 5, 6]
interseccao(setA, setC); // => Set [3, 4]
diferencaSimetrica(setA, setC); // => Set [1, 2, 5, 6]
diferenca(setA, setC); // => Set [1, 2]
```

Relação com objetos <code>Array</code>

```
var myArray = ["value1", "value2", "value3"];

// Use o construtor regular de Set para transformar um array dentro de um Set
var mySet = new Set(myArray);

mySet.has("value1"); // retorna true

// Use o operador de propagação para transformar um Set em um Array.
alert(uneval([...mySet])); // Irá mostrar-lhe exatamente o mesmo Array como myArray
```

Removendo elementos duplicados de um <code>Array</code>

```
// Use para remover elementos duplicados de um Array

const numeros = [2, 3, 4, 4, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 5, 32, 3, 4, 5];

console.log([...new Set(numeros)]);

// [2, 3, 4, 5, 6, 7, 32]
```

Relação com objetos <code>String</code>

```
var texto = "India";

var meuSet = new Set(texto); // Set ['I', 'n', 'd', 'i', 'a']
meuSet.size; // 5
```

Veja também

- `Map`
- `WeakMap`
- `WeakSet`

Set.prototype.add()

O método **`add()`** acrescenta um novo elemento com o valor especificado no final de um objeto `Set`.

Sintaxe

```
mySet.add(value);
```

Parâmetros

value

Requerido. O valor de um elemento a ser adicionado ao objeto Set.

Valor de retorno

O objeto `Set`.

Exemplos

Usando o método <code>add</code>

```
var mySet = new Set();

mySet.add(1);
mySet.add(5).add("some text"); // pode ser encadeado

console.log(mySet);
// Set [1, 5, "some text"]
```

Notas específicas para Firefox

- Antes do Firefox 33, `Set.prototype.add retornava undefined` e não era possível encadear chamadas. Isto foi resolvido (Erro do Firefox 1031632). O comportamento pode ser encontrado no Chrome/v8 (issue).

Veja também

- `Set`
- `Set.prototype.delete()`
- `Set.prototype.has()`

Set.prototype.clear()

O método **`clear()`** remove todos os elementos de um objeto `Set`.

Sintaxe

```
mySet.clear();
```

Exemplos

Usando o método <code>clear</code>

```
var mySet = new Set();
mySet.add(1);
mySet.add("foo");

mySet.size; // 2
mySet.has("foo"); // true

mySet.clear();

mySet.size; // 0
mySet.has("bar"); // false
```

Veja também

- `Set`
- `Set.prototype.delete()`

Set.prototype.delete()

O método **`delete()`** remove o elemento especificado de um objeto `Set`.

Sintaxe

```
mySet.delete(value);
```

Parâmetros

value

Requerido. O valor do elemento a ser removido do objeto Set.

Return value

Retorna `true` se um elemento contido no objeto Set foi removido com sucesso; caso contrário `false`.

Exemples

Uso do método <code>delete</code>

```
var mySet = new Set();
mySet.add("foo");

mySet.delete("bar"); // Retorna false. Nenhum elemento "bar" foi encontrado para deletar.
mySet.delete("foo"); // Retorna true. remoção bem sucedida.

mySet.has("foo"); // Retorna false. O elemento "foo" não está mais presente.
```

Veja também

- `Set`
- `Set.prototype.clear()`

Set.prototype.entries()

O método **`entries()`** retorna um novo objeto Iterador (`Iterator`) que contém **um array de [valor, valor]** para cada elemento de um objeto `Set`em ordem de inserção. Os objetos `Set` não possuem `chaves (key)` como objetos do tipo `Map`. Entretanto, para manter a API similar a objetos `Map`, cada entrada (*entry*) tem o mesmo valor para sua chave (`key`) e valor (`value`), então uma matriz `array [valor, valor]` é retornado.

Sintaxe

```
mySet.entries()
```

Exemplos

Usando <code>entries()</code>

```
var mySet = new Set();
mySet.add("foobar");
mySet.add(1);
mySet.add("baz");

var setIter = mySet.entries();

console.log(setIter.next().value); // ["foobar", "foobar"]
console.log(setIter.next().value); // [1, 1]
console.log(setIter.next().value); // ["baz", "baz"]
```

Veja também

- `Set.prototype.keys()`
- `Set.prototype.values()`

Set.prototype.has()

O método **`has()`** retorna um valor booleano indicando se um elemento com o valor especificado existe em um objecto `Set` ou não.

Experimente

```
const set1 = new Set([1, 2, 3, 4, 5]);

console.log(set1.has(1));
// Expected output: true

console.log(set1.has(5));
// Expected output: true

console.log(set1.has(6));
// Expected output: false
```

Sintaxe

```
mySet.has(valor);
```

Parâmetros

`valor`

O valor para testar a existência no objeto `Set`.

Valor de retorno

Retorna `true` se um elemento com o valor especificado existe no objeto `Set`; `false` caso contrário.

> **Nota:** Tecnicamente falando, `has()` usa o algorítimo `sameValueZero` para determinar quando o elemento dado é encontrado.

Exemplos

Usando o método <code>has</code>

```
var mySet = new Set();
mySet.add("foo");

mySet.has("foo"); // retorna true
mySet.has("bar"); // retorna false

var set1 = new Set();
var obj1 = { key1: 1 };
set1.add(obj1);

set1.has(obj1); // retorna true
set1.has({ key1: 1 }); // retorna false porque obj1 e {'key': 1} fazem referência a objetos diferentes.
set1.add({ key1: 1 }); // agora set1 contém 2 registros
```

Veja também

- `Set`
- `Set.prototype.add()`
- `Set.prototype.delete()`

Set.prototype.values()

O método **`values()`** retorna um novo objeto `Iterator` que contem os valores para cada elemento dentro do objeto `Set` por ordem de inserção.

O método **`keys()`** é um alias desse método (por similaridade com os objetos `Map`); ele se comporta exatamente da mesma forma e retorna os **valores** dos elementos do `Set`.

Experimente

```
const set1 = new Set();
set1.add(42);
set1.add("forty two");

const iterator1 = set1.values();

console.log(iterator1.next().value);
// Expected output: 42

console.log(iterator1.next().value);
// Expected output: "forty two"
```

Sintaxe

```
mySet.values();
```

Valor retornado

Um novo objeto **`Iterator`** condendo os valores de cada elemento contido no `Set`, por ordem de inserção.

Exemplos

Usando <code>values()</code>

```
var mySet = new Set();
mySet.add("foo");
mySet.add("bar");
mySet.add("baz");

var setIter = mySet.values();

console.log(setIter.next().value); // "foo"
console.log(setIter.next().value); // "bar"
console.log(setIter.next().value); // "baz"
```

Veja também

- `Set.prototype.entries()`

String

Sumário

O objeto global **`String`** é um construtor para **strings,** ou uma sequência de caracteres.

Sintaxe

As formas literais de declaração de *String* são:

```
'string text'
"string text"
"中文 español English हिन्दी العربية português বাংলা русский 日本語 ਪੰਜਾਬੀ 한국어"
```

Além da forma regular, de caracteres de impressão, caracteres especiais podem ser codificados usando a *escape notation* (notação com barra invertida):

Codigo	Saida
\0	o caractere NULL
\'	aspas simples
\"	aspas duplas
\\	barra invertida
\n	nova linha
\r	carriage return
\v	tab vertical
\t	tab
\b	backspace
\f	form feed
\uXXXX	unicode codepoint
\xXX	the Latin-1 character

Ou, usando o objeto global `String` diretamente:

```
String(thing)
new String(thing)
```

Parâmetros

`thing`

Qualquer coisa a ser convertida para uma string.

Descrição

Strings são úteis para guardar dados que podem ser representados em forma de texto. Uma das operações mais usadas nas **strings** é checar seu `tamanho`, para construir e concatená-las usando os operadores `+` e `+=`, checando pela existência ou posição de *substrings* com o método `indexOf` , ou extrair *substrings* com o método `substring`.

Acesso à caractere

Há duas maneiras de acessar um caractere individual em uma string. A primeira é o método `charAt`:

```
return "cat".charAt(1); // returns "a"
```

A outra maneira (introduzido no ECMAScript 5) consiste em tratar a string como um objeto Array-like, onde os caráteres individuais correspondem a um índice numérico:

```
return "cat"[1]; // returns "a"
```

Para acesso de caracteres usando uma notação de colchetes, tentando deletar ou designar um valor a estas propriedades não haverá sucesso. As propriedades envolvidas não são nem escritas ou configuráveis. (Veja `Object.defineProperty` para mais informações.)

Comparando strings

Desenvolvedores de C têm a função strcmp() para comparar strings. No JavaScript, basta usar o operador maior que e menor que:

```
var a = "a";
var b = "b";
if (a < b)
  // verdadeiro
  print(a + " é menor que " + b);
```

```
else if (a > b) print(a + " é maior que " + b);
else print(a + " e " + b + " são iguais.");
```

Um resultado similar pode ser alcançado usando o método `localeCompare` herdado pelas instâncias de `String`.

Distinção entre String primitiva e objetos String

Note que o JavaScript distingue entre objetos String e valores de string primitivas. (O mesmo é válido para `Boolean` e `Numbers`.)

Strings literais (definidas por aspas duplas ou aspas simples) e strings retornadas da chamada da função String fora do contexto de uma função construtora (sem o uso da palavra chave new) são strings primitivas. O JavaScript converte automaticamente strings primitivas para objetos do tipo String, por isso é possível utilizar os métodos do objeto String através de strings primitivas. Em contextos onde um método é invocado de uma string primitiva ou uma propriedade é procurada, o JavaScript irá criar um objeto com a string primitiva e executar o método ou acessar a propriedade procurada.

```
var s_prim = "foo";
var s_obj = new String(s_prim);

console.log(typeof s_prim); // Loga "string"
console.log(typeof s_obj); // Loga "object"
```

String primitivas e objetos `String` também dão resultados diferentes quando usado `eval`. Primitivas passadas para `eval` são tratadas como código fonte; Objetos `String` são tratados como todos os outros objetos são, retornando o objeto. Por exemplo:

```
s1 = "2 + 2"; // cria uma string primitiva
s2 = new String("2 + 2"); // cria um objeto de String
console.log(eval(s1)); // retorna o número 4
console.log(eval(s2)); // retorna a string "2 + 2"
```

Por estas razões, o código pode quebrar quando encontra objetos `String` quando espera na verdade uma string primitiva, apesar de que geralmente autores não precisam se preocupar com a distinção.

Um objeto `String` pode ser convertido sempre para sua contraparte primitiva com o método `valueOf`.

```
console.log(eval(s2.valueOf())); // retorna o número 4
```

> **Nota:** Para uma outra possível abordagem para strings em JavaScript, favor ler o artigo sobre `StringView` – a C-like representation of strings based on typed arrays.

Propriedades

`String`

Permite a adição de propriedades a um objeto String.

Métodos

`String.fromCharCode()`

Retorna uma string criada usando a sequência especificada de valores Unicode.

`String.fromCodePoint()`

Retorna uma string criada usando a sequência especificada de posições de código.

<code>Métodos genéricos de Strings</code>

Métodos de instância `String` também estão disponíveis no Firefox a partir de JavaScript 1.6 (embora não faça parte dos padrões ECMAScript) no objeto String para aplicar métodos String a qualquer objeto:

```
var num = 15;
alert(String.replace(num, /5/, "2"));
```

Genéricos também estão disponíveis em métodos `Array`.

O seguinte é uma implementação para fornecer suporte a navegadores sem suporte:

```
/*globals define*/
// Assume que todos os métodos de instância String fornecidos
// já presentes (podem ser usadas implementações para este se não disponível)
(function () {
  "use strict";

  var i,
    // Nós também poderíamos construir o array de métodos com os seguintes,
    // mas o método getOwnPropertyNames() não é implementável:
    // Object.getOwnPropertyNames(String).filter(function (methodName)
    //  {return typeof String[methodName] === 'function'});
    methods = [
      "quote",
      "substring",
```

```
      "toLowerCase",
      "toUpperCase",
      "charAt",
      "charCodeAt",
      "indexOf",
      "lastIndexOf",
      "startsWith",
      "endsWith",
      "trim",
      "trimLeft",
      "trimRight",
      "toLocaleLowerCase",
      "toLocaleUpperCase",
      "localeCompare",
      "match",
      "search",
      "replace",
      "split",
      "substr",
      "concat",
      "slice",
    ],
    methodCount = methods.length,
    assignStringGeneric = function (methodName) {
      var method = String.prototype[methodName];
      String[methodName] = function (arg1) {
        return method.apply(arg1, Array.prototype.slice.call(arguments, 1));
      };
    };

  for (i = 0; i < methodCount; i++) {
    assignStringGeneric(methods[i]);
  }
})();
```

Instâncias de <code>String</code>

Propriedades

Métodos

Métodos não relacionados ao HTML

Métodos de envoltório HTML

Exemplos

Conversão de String

É possível usar `String` como uma alternativa "mais segura" `toString`, como embora normalmente ainda chama o `toString` subjacente, também funciona para `null` e `undefined`. Por exemplo:

```
var outputStrings = [];
for (let i = 0, n = inputValues.length; i < n; ++i) {
  outputStrings.push(String(inputValues[i]));
}
```

Veja também

- DOMString
- `StringView` – a C-like representation of strings based on typed arrays
- Binary strings

String.prototype.anchor()

O método **`anchor()`** cria uma string começando com uma tag inicial `<a name="...">`, um texto e uma tag final `</a>`.

> **Aviso:** Não use este método. Ao invés, use DOM APIs. Além disso, a especificação HTML não permite mais que o elemento <a> tenha um atributo **"name"**, portanto, esse método nem mesmo cria uma tag válida.

Sintaxe

```
str.anchor(name)
```

Parâmetros

`name`

Uma string que deve representar o valor do atributo *`name`*.

Valor retornado

Uma string começando com uma tag de início **, depois o valor da string e, em seguida, uma tag de fim **.

Descrição

Não use este método. Ao invés, use DOM APIs. Além disso, a especificação HTML não permite mais que o elemento *<a>* tenha um atributo **"name"**, portanto, esse método nem mesmo cria uma tag válida.

Exemplos

Usando <code>anchor()</code>

```
const nome = 'Ricardo';
console.log(nome.anchor('https://developer.mozilla.org/pt-BR/)');
```

irá retornar o seguinte código HTML:

```
'<a name="https://developer.mozilla.org/pt-BR/">Ricardo</a>'
```

Veja também

- `String.prototype.link()`

String.prototype.big()

O método **big()** cria um elemento HTML <big> fazendo com que o texto dentro dele seja exibido uma uma fonte maior.

> **Nota:** O elemento <big> foi removido no HTML5 e não deve mais ser usado. Em vez disso, web developers devem usar a propriedade CSS correspondente.

Sintaxe

```
str.big()
```

Valor retornado

Uma string contendo um elemento HTML.

Descrição

O método `big()` cria uma string dentro de uma tag `<big>`: `"<big>str</big>"`.

Exemplos

Usando <code>big()</code>

Os exemplos abaixo usam métodos do objeto String para alterar o tamanho de uma string:

```
var worldString = "Olá, mundo";

console.log(worldString.small()); // <small>Olá, mundo</small>
console.log(worldString.big()); // <big>Olá, mundo</big>
console.log(worldString.fontsize(7)); // <fontsize=7>Olá, Mundo</fontsize>
```

Com o objeto `element.style` você pode selecionar o atributo `style` do elemento e manipulá-lo de forma mais genérica, por exemplo:

```
document.getElementById("#oIdDoElemento").style.fontSize = "2em";
```

Veja também

- `String.prototype.fontsize()`
- `String.prototype.small()`

String.prototype.blink()

O método **`blink()`** cria um elemento HTML <blink> que faz uma string piscar.

> **Aviso:** A criação de textos que piscam é desaprovada por vários padrões de acessibilidade. O próprio elemento `<blink>` não é padrão e está obsoleto!

Sintaxe

```
str.blink()
```

Valor retornado

Uma string contendo um elemento HTML <blink>.

Descrição

O método `blink()` cria uma string dentro de uma tag `<blink>`: `"<blink>str</blink>"`.

Exemplos

Usando <code>blink()</code>

Os exemplos abaixo usam métodos do objeto String para alterar a formatação de uma string:

```
var worldString = "Olá, mundo";

console.log(worldString.blink()); // <blink>Olá, mundo</blink>
console.log(worldString.bold()); // <b>Olá, mundo</b>
console.log(worldString.italics()); // <i>Olá, mundo</i>
console.log(worldString.strike()); // <strike>Olá, mundo</strike>
```

Veja também

- `String.prototype.bold()`
- `String.prototype.italics()`
- `String.prototype.strike()`

String.prototype.bold()

O método **`bold()`** cria um elemento HTML `<b>` que faz com que uma string seja exibida em negrito.

Sintaxe

```
str.bold()
```

Valor retornado

Uma string contendo um elemento HTML `<b>` .

Descrição

O método `bold()` cria uma string dentro de uma tag `<b>`: `"<b>str</b>"`.

Exemplos

Usando <code>bold()</code>

Os exemplos abaixo usam métodos do objeto String para alterar a formatação de uma string:

```
var worldString = "Olá, mundo";

console.log(worldString.blink()); // <blink>Olá, mundo</blink>
console.log(worldString.bold()); // <b>Olá, mundo</b>
console.log(worldString.italics()); // <i>Olá, mundo</i>
console.log(worldString.strike()); // <strike>Olá, mundo</strike>
```

Veja também

- `String.prototype.blink()`
- `String.prototype.italics()`
- `String.prototype.strike()`

String.prototype.charAt()

O método **`charAt()`** retorna o caractere especificado a partir de uma string.

Sintaxe

```
str.charAt(index)
```

Parâmetros

`index`

Um inteiro entre `0` e `str.length - 1`. Se nenhum índice for definido, `charAt()` usará `0` como índice.

Valor retornado

Uma string representando o caractere no índice especificado. Uma string vazia se o **`index`** fornecido estiver fora do intervalo de índices da string `str`.

Descrição

Caracteres em uma string são indexados da esquerda para a direita. O índice do primeiro caractere é `0` (zero), e o índice do último caractere em uma string declarada como `stringName` é `stringName.length - 1`. Se o índice que você fornecer estiver fora do intervalo de índices da string, JavaScript retornará uma string vazia.

Se nenhum índice for passado para `charAt()`, `0` será usado por padrão.

Exemplos

Mostrando caracteres em diferente localizações em uma string

O exemplo a seguir mostra caracteres em diferentes locais em uma string `"Brave new world"`:

```
var anyString = "Brave new world";

console.log("A letra no índice 0 é '" + anyString.charAt(0) + "'");
console.log("A letra no índice 1 é '" + anyString.charAt(1) + "'");
console.log("A letra no índice 2 é '" + anyString.charAt(2) + "'");
console.log("A letra no índice 3 é '" + anyString.charAt(3) + "'");
console.log("A letra no índice 4 é '" + anyString.charAt(4) + "'");
console.log("A letra no índice 99 é '" + anyString.charAt(999) + "'");
```

As linhas acima retornam o seguinte:

```
A letra no índice 0 é 'B'
A letra no índice 1 é 'r'
A letra no índice 2 é 'a'
A letra no índice 3 é 'v'
A letra no índice 4 é 'e'
A letra no índice 99 é ''
```

Obtendo caracteres inteiros

O seguinte código fornece um meio de garantir que passar por um loop de string sempre forneça um caractere inteiro, mesmo se a string contiver caracteres que não estão no Plano Multilíngue Básico.

```
var str = "A \uD87E\uDC04 Z"; // We could also use a non-BMP character directly
for (var i = 0, chr; i < str.length; i++) {
  if ((chr = getWholeChar(str, i)) === false) {
    continue;
  }
  // Adapt this line at the top of each loop, passing in the whole string and
  // the current iteration and returning a variable to represent the
  // individual character

  console.log(chr);
}

function getWholeChar(str, i) {
  var code = str.charCodeAt(i);

  if (isNaN(code)) {
    return ""; // Position not found
  }
  if (code < 0xd800 || code > 0xdfff) {
    return str.charAt(i);
  }

  // High surrogate (could change last hex to 0xDB7F to treat high private
  // surrogates as single characters)
  if (0xd800 <= code && code <= 0xdbff) {
    if (str.length <= i + 1) {
      throw "High surrogate without following low surrogate";
    }
    var next = str.charCodeAt(i + 1);
    if (0xdc00 > next || next > 0xdfff) {
      throw "High surrogate without following low surrogate";
    }
    return str.charAt(i) + str.charAt(i + 1);
  }
  // Low surrogate (0xDC00 <= code && code <= 0xDFFF)
  if (i === 0) {
    throw "Low surrogate without preceding high surrogate";
  }
  var prev = str.charCodeAt(i - 1);

  // (could change last hex to 0xDB7F to treat high private
  // surrogates as single characters)
  if (0xd800 > prev || prev > 0xdbff) {
    throw "Low surrogate without preceding high surrogate";
  }
  // We can pass over low surrogates now as the second component
  // in a pair which we have already processed
  return false;
}
```

Em um ambiente ECMAScript 2016 que permite atribuição desestruturada, o seguinte código é uma alternativa mais sucinta e um pouco mais flexível, pois faz incremento para uma variável de incremento automaticamente (se o caractere justificar que seja um par substituto).

```
var str = "A\uD87E\uDC04Z"; // We could also use a non-BMP character directly
for (var i = 0, chr; i < str.length; i++) {
  [chr, i] = getWholeCharAndI(str, i);
  // Adapt this line at the top of each loop, passing in the whole string and
  // the current iteration and returning an array with the individual character
  // and 'i' value (only changed if a surrogate pair)
```

```
  console.log(chr);
}

function getWholeCharAndI(str, i) {
  var code = str.charCodeAt(i);

  if (isNaN(code)) {
    return ""; // Position not found
  }
  if (code < 0xd800 || code > 0xdfff) {
    return [str.charAt(i), i]; // Normal character, keeping 'i' the same
  }

  // High surrogate (could change last hex to 0xDB7F to treat high private
  // surrogates as single characters)
  if (0xd800 <= code && code <= 0xdbff) {
    if (str.length <= i + 1) {
      throw "High surrogate without following low surrogate";
    }
    var next = str.charCodeAt(i + 1);
    if (0xdc00 > next || next > 0xdfff) {
      throw "High surrogate without following low surrogate";
    }
    return [str.charAt(i) + str.charAt(i + 1), i + 1];
  }
  // Low surrogate (0xDC00 <= code && code <= 0xDFFF)
  if (i === 0) {
    throw "Low surrogate without preceding high surrogate";
  }
  var prev = str.charCodeAt(i - 1);

  // (could change last hex to 0xDB7F to treat high private surrogates
  // as single characters)
  if (0xd800 > prev || prev > 0xdbff) {
    throw "Low surrogate without preceding high surrogate";
  }
  // Return the next character instead (and increment)
  return [str.charAt(i + 1), i + 1];
}
```

Corrigindo <code>charAt()</code> para suportar caracteres não-Plano-Multilíngüe-Básico (PMB)

Embora o exemplo anterior possa ser mais útil para programas que devem suportar caracteres não BMP (uma vez que não exige que o chamador saiba onde qualquer caractere não BMP pode aparecer), no caso de desejar, na escolha de um caractere por índice, para tratar os pares substitutos em uma string como os caracteres únicos que eles representam, pode-se usar o seguinte:

```
function fixedCharAt(str, idx) {
  var ret = "";
  str += "";
  var end = str.length;

  var surrogatePairs = /[\uD800-\uDBFF][\uDC00-\uDFFF]/g;
  while (surrogatePairs.exec(str) != null) {
    var li = surrogatePairs.lastIndex;
    if (li - 2 < idx) {
      idx++;
    } else {
      break;
    }
  }

  if (idx >= end || idx < 0) {
    return "";
  }

  ret += str.charAt(idx);

  if (
    /[\uD800-\uDBFF]/.test(ret) &&
    /[\uDC00-\uDFFF]/.test(str.charAt(idx + 1))
  ) {
    // Go one further, since one of the "characters" is part of a surrogate pair
    ret += str.charAt(idx + 1);
  }
  return ret;
}
```

Veja também

- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`
- `String.prototype.charCodeAt()`
- `String.prototype.codePointAt()`

- `String.prototype.split()`
- `String.fromCodePoint()`
- JavaScript has a Unicode problem – Mathias Bynens

String.prototype.charCodeAt()

O método **`charCodeAt()`** retorna um número inteiro entre `0` e `65535` que representa a unidade de código UTF-16 no índice fornecido. A unidade de código UTF-16 corresponde ao ponto de código Unicode para pontos de códigos representáveis em uma única unidade de código UTF-16, mas também pode ser a primeira unidade de código de um par substituto não representável em uma única unidade de código UTF-16. Po exemplo: pontos de código Unicode > (0x10000). Se você quer o valor do ponto de código inteiro, use **`codePointAt`**().

Sintaxe

```
str.charCodeAt(index)
```

Parâmetros

`index`

Um inteiro maior ou igual a `0` e menor que o comprimento da string. Se não for um número, o padrão será `0`.

Valor retornado

Um número representando o valor de unidade de código UTF-16 do caractere no índice fornecido. O valor `NaN` é retornado se o índice estiver fora do intervalo aceitável.

Descrição

Os pontos de código Unicode variam de `0` a `1114111` (`0x10FFFF`). Os primeiros 128 pontos de código Unicode são uma correspondência direta da codificação de caracteres ASCII. (Para informações sobre Unicode, veja o JavaScript Guide.)

> **Nota:** O `charCodeAt()` sempre retornará um valor menor do que `65536`. Isso ocorre pois os pontos de código mais altos são representados por um par de pseudo-caracteres "substitutos" (de menor valor) que são usados para compreender o caracter real.
>
> Por isso, para examinar (ou reproduzir) o caractere completo para valores de caracteres individuais de valor `65536` e acima, é necessário recuperar não apenas o `charCodeAt(i)`, mas também o `charCodeAt(i+1)` (como se examinando/reproduzindo a string com duas letras), ou usar o `codePointAt(i)`. Veja o exemplo 2 e 3 (abaixo).

Compatibilidade com versões anteriores: Em versões históricas (como JavaScript 1.2) o método `charCodeAt()` retorna um número indicando o valor de conjunto de códigos ISO-Latin-1 do caractere no dado índice. O conjunto de códigos ISO-Latin-1 varia de `0` a `255`. Os primeiros `128` (do `0` ao `127`) são uma correspondência direta ao conjunto de caracteres ASCII.

Exemplos

Usando <code>charCodeAt()</code>

O exemplo a seguir retorna `65`, o valor Unicode para A.

```
"ABC".charCodeAt(0); // retorna 65
```

Corrigindo o <code>charCodeAt()</code> para manipular caracteres de Plano Multilingual não Básico se sua presença na string é desconhecida

Essa versão pode ser usada em loops **for** e afins quando não sabemos se caracteres de Plano Multilingual não Básico existem antes da posição do índice especificado.

```
function fixedCharCodeAt(str, idx) {
  // ex. fixedCharCodeAt('\uD800\uDC00', 0); // 65536
  // ex. fixedCharCodeAt('\uD800\uDC00', 1); // false
  idx = idx || 0;
  var code = str.charCodeAt(idx);
  var hi, low;

  // Substituto elevado (poderia mudar o último hex para 0xDB7F para tratar
  // substitutos privados elevados como caracteres únicos)
  if (0xd800 <= code && code <= 0xdbff) {
    hi = code;
    low = str.charCodeAt(idx + 1);
    if (isNaN(low)) {
      throw "High surrogate not followed by low surrogate in fixedCharCodeAt()";
    }
    return (hi - 0xd800) * 0x400 + (low - 0xdc00) + 0x10000;
  }
  if (0xdc00 <= code && code <= 0xdfff) {
    // Low surrogate
    // Retornamos false para permitir os loops pularem essa iteração já que já deveria
    //ter tratado os substitutos elevados acima, na iteração anterior
    return false;
    /*hi = str.charCodeAt(idx - 1);
    low = code;
    return ((hi - 0xD800) * 0x400) + (low - 0xDC00) + 0x10000;*/
  }
```

```
  return code;
}
```

Veja também

- `String.fromCharCode()`
- `String.prototype.charAt()`
- `String.fromCodePoint()`
- `String.prototype.codePointAt()`

String.prototype.codePointAt()

O método **`codePointAt()`** retorna um número inteiro não negativo que é o valor do ponto de código Unicode.

Experimente

```
const icons = "☃★♲";

console.log(icons.codePointAt(1));
// Expected output: "9733"
```

Sintaxe

```
str.codePointAt(pos)
```

Parâmetros

`pos`

A posição de um elemento em uma string a partir do qual retorna o valor do ponto de código.

Valor retornado

Um número que representa o valor do ponto de código do caractere na `pos` fornecida. Se não houver nenhum elemento na `pos`, `undefined` é retornado.

Descrição

Se não houver nenhum elemento na posição especificada, é retornado o valor de `undefined`. Se nenhum par substituto UTF-16 começar na `pos`, a unidade de código na `pos` será retornada.

Polyfill

O seguinte código cria no objeto global String a função `codePointAt()` conforme especificado em ECMAScript 2015 para navegadores sem suporte nativo:

```
/*! https://mths.be/codepointat v0.2.0 by @mathias */
if (!String.prototype.codePointAt) {
  (function () {
    "use strict"; // needed to support `apply`/`call` with `undefined`/`null`
    var defineProperty = (function () {
      // IE 8 only supports `Object.defineProperty` on DOM elements
      try {
        var object = {};
        var $defineProperty = Object.defineProperty;
        var result = $defineProperty(object, object, object) && $defineProperty;
      } catch (error) {}
      return result;
    })();
    var codePointAt = function (position) {
      if (this == null) {
        throw TypeError();
      }
      var string = String(this);
      var size = string.length;
      // `ToInteger`
      var index = position ? Number(position) : 0;
      if (index != index) {
        // better `isNaN`
        index = 0;
      }
      // Account for out-of-bounds indices:
      if (index < 0 || index >= size) {
        return undefined;
      }
      // Get the first code unit
      var first = string.charCodeAt(index);
      var second;
      if (
        // check if it’s the start of a surrogate pair
        first >= 0xd800 &&
        first <= 0xdbff && // high surrogate
        size > index + 1 // there is a next code unit
      ) {
        second = string.charCodeAt(index + 1);
        if (second >= 0xdc00 && second <= 0xdfff) {
```

```
          // low surrogate
          // https://mathiasbynens.be/notes/javascript-encoding#surrogate-formulae
          return (first - 0xd800) * 0x400 + second - 0xdc00 + 0x10000;
        }
      }
      return first;
    };
    if (defineProperty) {
      defineProperty(String.prototype, "codePointAt", {
        value: codePointAt,
        configurable: true,
        writable: true,
      });
    } else {
      String.prototype.codePointAt = codePointAt;
    }
  })();
}
```

Exemplos

Usando <code>codePointAt()</code>

```
"ABC".codePointAt(1); // retorna 66
"\uD800\uDC00".codePointAt(0); // retorna 65536

"XYZ".codePointAt(42); // retorna undefined
```

Criando um loop com <code>codePointAt()</code>

```
for (let codePoint of "\ud83d\udc0e\ud83d\udc71\u2764") {
  console.log(codePoint.codePointAt(0).toString(16));
}
// retorna '1f40e', '1f471', '2764'
```

Veja também

- `String.fromCodePoint()`
- `String.fromCharCode()`
- `String.prototype.charCodeAt()`
- `String.prototype.charAt()`

String.prototype.concat()

O método **`concat()`** combina o texto de duas ou mais strings e retorna uma nova string.

Sintaxe

```
str.concat(string2 [, ...stringN])
```

Parâmetros

`string2...stringN`

Strings para concatenar à **`string2`**.

Valor retornado

Uma nova string contendo a string original concatenada à string passada como parâmetro.

Descrição

A função **`concat()`** combina o texto de duas ou mais strings e retorna uma nova string. As alterações de texto de uma string não afetam a outra string.

Se o argumento passado não for do tipo string, o mesmo será convertido em uma string antes de ser concatenado.

Performance

É extremamente recomendado o uso dos operadores de atribuição (+, +=) em detrimento do método `concat()`.

Exemplos

Usando <code>concat()</code>

O exemplo a seguir concatena uma string à outra string originando uma terceira string.

```
var hello = "Olá, ";
console.log(hello.concat("Kevin", " tenha um bom dia."));

// retorna 'Olá, Kevin tenha um bom dia.'
```

Veja também

- `Array.prototype.concat()`
- Assignment operators

String.prototype.endsWith()

O método **`endsWith()`** indica se uma string termina com determinados caracteres, retornando `true` ou `false`.

Sintaxe

```
str.endsWith(stringSendoBuscada[, tamanho])
```

Parâmetros

`stringSendoBuscada`

Os caracteres a serem pesquisados no final da string.

`tamanho`

Opcional. Se fornecido, substitui o tamanho da string passada. Se omitido, o valor padrão é o tamanho da string.

Valor retornado

`true` se os caracteres passados forem encontrados no final da string. Do contrário, retorna **`false`**.

Descrição

Este método permite que você verifique se uma string termina ou não com determinados caracteres. Este método é case-sensitive.

Exemplos

Usando <code>endsWith()</code>

```
var str = "Ser ou não ser, eis a questão";

console.log(str.endsWith("questão")); // retorna true
console.log(str.endsWith("ser")); // retorna false
console.log(str.endsWith("ser", 14)); // retorna true
```

Polyfill

Este método foi adicionada na especificação ECMAScript 6 e talvez não esteja disponível em todos as implementações JavaScript ainda. No entanto, você pode criá-lo adicionando o seguinte código:

```
if (!String.prototype.endsWith)
  String.prototype.endsWith = function (searchStr, Position) {
    // This works much better than >= because
    // it compensates for NaN:
    if (!(Position < this.length)) Position = this.length;
    else Position |= 0; // round position
    return (
      this.substr(Position - searchStr.length, searchStr.length) === searchStr
    );
  };
```

Veja também

- `String.prototype.startsWith()`
- `String.prototype.includes()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`

String.prototype.fixed()

O método **`fixed()`** cria um elemento HTML `<tt>` que faz com que uma string seja exibida em uma fonte de densidade fixa.

Sintaxe

```
str.fixed()
```

Valor retornado

Uma string que representa o elemento HTML `<tt>`.

Descrição

O método `fixed()` cria uma string dentro de uma tag `<tt>`: "`<tt>str</tt>`".

Exemplos

Usando <code>fixed()</code>

O exemplo a seguir usa o método `fixed()` para alterar a formatação de uma string:

```
var worldString = "Olá, mundo";
console.log(worldString.fixed()); // "<tt>Olá, mundo</tt>"
```

Veja também

- `String.prototype.bold()`
- `String.prototype.italics()`
- `String.prototype.strike()`

String.prototype.fontcolor()

O método `fontcolor()` cria um elemento HTML <font> que faz com que uma string seja exibida na cor especificada.

> **Nota:** O elemento `<font>` foi removido do HTML5 e não deve mais ser usado. Em vez disso, web developers devem usar a propriedade CSS correspondente.

Sintaxe

```
str.fontcolor(color)
```

Parâmetros

`color`

Deve ser um string que expresse uma cor em formato hexadecimal ou o nome, em Inglês, de uma cor. Os nomes das cores estão listados na referência de cores CSS.

Valor retornado

Uma string contendo um elemento HTML <font>.

Descrição

Se você expressar uma cor em formato hexadecimal, deve usar o formato `rrggbb`. Por exemplo, os valores RGB hexadecimais para salmão são vermelho = FA, verde = 80 e azul = 72, portanto, o tripleto RGB para salmão é "FA8072".

Exemplos

Usando <code>fontcolor()</code>

O exemplo a seguir usa o método `fontcolor()` para alterar a cor de uma string, produzindo uma string com a tag HTML `<font>`.

```
var worldString = "Olá, mundo";

console.log(worldString.fontcolor("red") + " está vermelho nesta linha");
// '<font color="red">Olá, mundo</font> está vermelho nesta linha'

console.log(
  worldString.fontcolor("FF00") + " está vermelho em hexadecimal nesta linha",
);
// '<font color="FF00">Olá, mundo</font> está vermelho em hexadecimal nesta linha'
```

Com o objeto `element.style` você pode obter o atributo `style` do elemento e manipulá-lo de forma mais genérica, por exemplo:

```
document.getElementById("#oIdDoElemento").style.color = "red";
```

Veja também

- `String.prototype.fontsize()`

String.prototype.fontsize()

O método `fontsize()` cria um elemento HTML <font> que faz com que uma string seja exibida no tamanho da fonte especificada.

> **Nota:** O elemento `<font>` foi removido do HTML5 e não deve mais ser usado. Em vez disso, web developers devem usar a propriedade CSS correspondente.

Sintaxe

```
str.fontsize(size)
```

Parâmetros

`size`

Um número inteiro entre 1 e 7.

Valor retornado

Uma string contendo um elemento HTML <font>.

Descrição

Ao especificar o tamanho como um inteiro, você define o tamanho da fonte do texto para um dos 7 tamanhos definidos. Ao especificar `size` como uma string como "-2", você ajusta o tamanho da fonte do texto em relação ao tamanho definido na tag `<basefont>`.

Exemplos

Usando <code>fontsize()</code>

O exemplo a seguir usa métodos do objeto global String para alterar o tamanho de uma string:

```
var worldString = "Olá, mundo";

console.log(worldString.small()); // <small>Olá, mundo</small>
console.log(worldString.big()); // <big>Olá, mundo</big>
console.log(worldString.fontsize(7)); // <font size="7">Olá, mundo</fontsize>
```

Com o objeto `element.style` você pode obter o atributo `style` do elemento e manipulá-lo de forma mais genérica, por exemplo:

```
document.getElementById("#oIdDoElemento").style.fontSize = "0.7em";
```

Veja também

- `String.prototype.big()`
- `String.prototype.small()`

String.fromCharCode()

O método **`String.fromCharCode()`** retorna uma string criada ao usar uma sequência específica de valores Unicode.

Sintaxe

```
String.fromCharCode(num1[, ...[, numN]])
```

Parâmetros

`num1, ..., numN`

> Uma sequência de números que devem ser valores UTF-16. O tamanho é entre `0` e `65535` (`0xFFFF`). Números maiores do que `0xFFFF` são desconsiderados. Nenhuma verificação de vadalida é realizada.

Valor retornado

Uma string contendo os caracteres correspondentes à sequência de valores Unicode.

Descrição

Esse método retorna uma string e não um objeto `String`.

Como `fromCharCode()` é um método estático de `String`, você sempre o usará como `String.fromCharCode()`, ao invés de um método de um objeto `String` que você tenha criado.

Exemplos

Usando <code>fromCharCode()</code>

O seguinte exemplo retorna a string "ABC".

```
String.fromCharCode(65, 66, 67); // retorna "ABC"
```

Fazendo-o funcionar com valores maiores

Embora os valores Unicode mais comuns possam ser representados com um número de 16 bits (como esperado durante a padronização do JavaScript) e o fromCharCode() possa ser usado para retornar um único caracter dos valores mais comuns (por exemplo: valores UCS-2 que são os melhores subconjuntos do UTF-16 com os caractres mais comuns), a fim de resolver TODOS os valores Unicode legais (até 21 bits) o método fromCharCode() sozinho é inadequado. Como os caracteres de ponto de código mais alto usam 2 (valor menor) numeros "substitutos" para formar um único caracter, `String.fromCodePoint()` (parte do padrão ES2015) pode ser usado para retornar tal par e ainda representar adequadamente esses caracteres de valores altos.

Veja também

- `String.fromCodePoint()`
- `String.prototype.charAt()`
- `String.prototype.charCodeAt()`
- `String.prototype.codePointAt()`

String.fromCodePoint()

O método estático **String.fromCodePoint()** retorna uma seqüência de caracteres criado usando a seqüência especificada de pontos de código.

Syntax

```
String.fromCodePoint(num1[, ...[, numN]])
```

Parâmetros

`num1, ..., numN`

Uma sequência de pontos de código.

Exceções

`RangeError`

O `RangeError` é lançado se um ponto de código Unicode inválido é dado (por exemplo, "RangeError: NaN não é um ponto de código válido").

Descrição

Como o fromCodePoint() é um método estático do `String`, você sempre vai chamar esse método como **String.fromCodePoint()✓** em vez de usá-lo como um método de uma string que você criar, como **"minha string".fromCodePoint()❌**.

Exemplos

Usando <code>fromCodePoint()</code>

```
String.fromCodePoint(42); // "*"
String.fromCodePoint(65, 90); // "AZ"
String.fromCodePoint(0x404); // "\u0404"
String.fromCodePoint(0x2f804); // "\uD87E\uDC04"
String.fromCodePoint(194564); // "\uD87E\uDC04"
String.fromCodePoint(0x1d306, 0x61, 0x1d307); // "\uD834\uDF06a\uD834\uDF07"

String.fromCodePoint("_"); // RangeError
String.fromCodePoint(Infinity); // RangeError
String.fromCodePoint(-1); // RangeError
String.fromCodePoint(3.14); // RangeError
String.fromCodePoint(3e-2); // RangeError
String.fromCodePoint(NaN); // RangeError
```

```
// String.fromCharCode() alone cannot get the character at such a high code point
// The following, on the other hand, can return a 4-byte character as well as the
// usual 2-byte ones (i.e., it can return a single character which actually has
// a string length of 2 instead of 1!)
console.log(String.fromCodePoint(0x2f804)); // or 194564 in decimal
```

Polyfill

O método **String.fromCodePoint** foi adicionado ao padrão ECMAScript na versão 6 e pode não ser suportado em todos os navegadores da Web ou em todos os ambientes ainda. Use o código abaixo para um polyfill:

```
/*! http://mths.be/fromcodepoint v0.1.0 by @mathias */
if (!String.fromCodePoint) {
  (function () {
    var defineProperty = (function () {
      // IE 8 only supports `Object.defineProperty` on DOM elements
      try {
        var object = {};
        var $defineProperty = Object.defineProperty;
        var result = $defineProperty(object, object, object) && $defineProperty;
      } catch (error) {}
      return result;
    })();
    var stringFromCharCode = String.fromCharCode;
    var floor = Math.floor;
    var fromCodePoint = function () {
      var MAX_SIZE = 0x4000;
      var codeUnits = [];
      var highSurrogate;
      var lowSurrogate;
      var index = -1;
      var length = arguments.length;
      if (!length) {
        return "";
      }
      var result = "";
      while (++index < length) {
        var codePoint = Number(arguments[index]);
        if (
          !isFinite(codePoint) || // `NaN`, `+Infinity`, or `-Infinity`
          codePoint < 0 || // not a valid Unicode code point
          codePoint > 0x10ffff || // not a valid Unicode code point
          floor(codePoint) != codePoint // not an integer
        ) {
          throw RangeError("Invalid code point: " + codePoint);
        }
        if (codePoint <= 0xffff) {
```

```
          // BMP code point
          codeUnits.push(codePoint);
        } else {
          // Astral code point; split in surrogate halves
          // http://mathiasbynens.be/notes/javascript-encoding#surrogate-formulae
          codePoint -= 0x10000;
          highSurrogate = (codePoint >> 10) + 0xd800;
          lowSurrogate = (codePoint % 0x400) + 0xdc00;
          codeUnits.push(highSurrogate, lowSurrogate);
        }
        if (index + 1 == length || codeUnits.length > MAX_SIZE) {
          result += stringFromCharCode.apply(null, codeUnits);
          codeUnits.length = 0;
        }
      }
      return result;
    };
    if (defineProperty) {
      defineProperty(String, "fromCodePoint", {
        value: fromCodePoint,
        configurable: true,
        writable: true,
      });
    } else {
      String.fromCodePoint = fromCodePoint;
    }
  })();
}
```

Veja também

- `String.fromCharCode()`
- `String.prototype.charAt()`
- `String.prototype.codePointAt()`
- `String.prototype.charCodeAt()`

String.prototype.includes()

O método **includes()** determina se um conjunto de caracteres pode ser encontrado dentro de outra string, retornando `true` ou `false`.

Sintaxe

```
str.includes(searchString[, position])
```

Parâmetros

`searchString`

É o conjunto de caracteres que será pesquisado dentro desta string.

`position`

Opcional. É um número inteiro que indica por onde a busca iniciará, referente ao índice da string a ser pesquisada. O valor padrão é 0.

Valor retornado

`true` se o conjunto de caracteres for encontrado em algum lugar dentro da string sendo pesquisada. Do contrário, retorna `false`.

Descrição

Este método permite conferir se uma string contém um determinado conjunto de caracteres.

Case-sensitivity

O método `includes()` é case sensitive. Por exemplo, a seguinte expressão retorna `false`:

```
"Bandeira do Brasil".includes("brasil"); // retorna false
```

Exemplos

Utilizando <code>includes()</code>

```
var str = "Ser, ou não ser, eis a questão.";

console.log(str.includes("Ser")); // true
console.log(str.includes("questão")); // true
console.log(str.includes("não existe")); // false
console.log(str.includes("ser", 1)); // true
console.log(str.includes("SER")); // false
```

Implementação

Este método foi adicionado à especificação ECMAScript 6 e pode não estar disponível em todas as implementações JavaScript. No entanto, você pode facilmente implementar este método:

```
if (!String.prototype.includes) {
  String.prototype.includes = function () {
    "use strict";
    return String.prototype.indexOf.apply(this, arguments) !== -1;
  };
}
```

String.prototype.contains

No Firefox 18 - 39, o nome deste método era `contains()`. Ele foi renomeado para `includes()` no Erro do Firefox 1102219 devido ao seguinte motivo:

Foi reportado que alguns websites que utilizam MooTools 1.2 não funcionavam no Firefox 17. Esta versão do MooTools verifica se `String.prototype.contains()` existe e, se não existir, MooTools adiciona sua própria função. Com a implementação desta função no Firefox 17, o comportamento desta validação mudou de uma forma que códigos baseados na implementação da função `String.prototype.contains()` do MooTools parassem de funcionar. Como resultado, esta mudança foi desabilitada no Firefox 17 e `String.prototype.contains()` foi disponibilizada na versão seguinte, no Firefox 18.

MooTools 1.3 força sua própria versão do `String.prototype.contains()`, portanto websites baseados nela não devem parar de funcionar. No entanto, você deve notar que a assinatura do MooTools 1.3 e a assinatura ECMAScript 6 diferem (no segundo argumento). Posteriormente, MooTools 1.5+ mudou sua assinatura para o padrão ES6.

Veja também

- `Array.prototype.includes()`
- `TypedArray.prototype.includes()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`
- `String.prototype.startsWith()`
- `String.prototype.endsWith()`

String.prototype.indexOf()

Sumário

O método `indexOf()` retorna o índice da primeira ocorrência do valor fornecido em searchValue, começando a busca a partir de `fromIndex`. Retorna `-1` se o valor não for encontrado.

> **Nota:** Para o método de Array, veja `Array.prototype.indexOf()`.

Sintaxe

```
str.indexOf(searchValue[, fromIndex])
```

Parâmetros

`searchValue`

> Uma string representando o valor a ser buscado. Se nenhuma string for fornecida explicitamente, `searchValue` terá o valor de `undefined`, e esse valor será buscado na string atual. Por exemplo, `'undefined'.indexOf()` retornará `0`, já que `undefined` é encontrado na posição `0`. Já `'undefine'.indexOf()` retornará `-1`, já que `undefined` não pôde ser encontrado.

`fromIndex`

> Um número inteiro representando um índice da string original a partir da qual a busca deve começar. Por padrão é `0`. Se `fromIndex < 0`, a string toda é percorrida (equivale a passar 0). Se `fromIndex >= str.length`, o método retornará `-1`, já que a busca será iniciada após o final da string.

Valor retornado

O índice da primeira ocorrência de *searchValue*, ou `-1` se não for encontrado.

Uma string vazia no *searchValue* produz resultados estranhos. Sem `fromIndex`, ou com qualquer `fromIndex` menor que o comprimento da string, o valor retornado é o próprio `fromIndex`:

```
"Olá, mundo".indexOf(""); // retorna 0
"Olá, mundo".indexOf("", 0); // retorna 0
"Olá, mundo".indexOf("", 3); // retorna 3
"Olá, mundo".indexOf("", 8); // retorna 8
```

Entretanto, com qualquer `fromIndex` igual ou maior que o comprimento da string, o valor retornado é o comprimento da string:

```
"Olá, mundo".indexOf("", 11); // retorna 10
"Olá, mundo".indexOf("", 13); // retorna 10
"Olá, mundo".indexOf("", 22); // retorna 10
```

Descrição

Caracteres em uma string são indexados da esquerda para a direita. O índice do primeiro caractere é `0`, e o índice do último caractere de uma string chamada `stringName` é `stringName.length - 1`.

```
"Blue Whale".indexOf("Blue"); // retorna  0
"Blue Whale".indexOf("Whale"); // retorna  5
"Blue Whale".indexOf("Blute"); // retorna -1
"Blue Whale".indexOf("Whale", 0); // retorna  5
"Blue Whale".indexOf("Whale", 5); // retorna  5
"Blue Whale".indexOf("Whale", 7); // retorna -1
"Blue Whale".indexOf(""); // retorna  0
"Blue Whale".indexOf("", 9); // retorna  9
"Blue Whale".indexOf("", 10); // retorna 10
"Blue Whale".indexOf("", 11); // retorna 10
```

Verificando ocorrências

Note que um retorno `0` não implica em `true`, e `-1` não implica em `false`. Portanto, a maneira correta de se verificar se uma string específica está contida em outra string seria:

```
"Blue Whale".indexOf("Blue") !== -1; // true
"Blue Whale".indexOf("Bloe") !== -1; // false
```

Exemplos

Usando <code>indexOf()</code>

O exemplo a seguir usa `indexOf()` para localizar valores dentro da string "`Brave new world`".

```
var anyString = "Brave new world";

console.log(
  "O índice do primeiro w partindo do começo é " + anyString.indexOf("w"),
);
// Exibe 8

console.log(
  "O índice de 'new' partindo do começo é " + anyString.indexOf("new"),
);
// Exibe 6
```

<code>indexOf()</code> e sensibilidade a maiúsculas e minúsculas

O exemplo a seguir define duas variáveis string. Ambas contém a mesma string, exceto que a segunda string tem letras maiúsculas. O primeiro método `console.log()` exibe `19`. Porém, como o método `indexOf` é sensível a letras maiúsculas e minúsculas, a string `"cheddar"` não é encontrada em `myCapString`, portanto, o segundo método `console.log()` exibe `-1`.

```
var myString = "brie, pepper jack, cheddar";
var myCapString = "Brie, Pepper Jack, Cheddar";

console.log('myString.indexOf("cheddar") é ' + myString.indexOf("cheddar"));
// Exibe 19
console.log(
  'myCapString.indexOf("cheddar") é ' + myCapString.indexOf("cheddar"),
);
// Exibe -1
```

Usando <code>indexOf()</code> para contar as ocorrências de uma letra numa string

O exemplo a seguir atribui à variável `count` o número de ocorrências da letra `x` na string `str`:

```
const str = "Serx ou não ser, eisx a questão";
count = 0;
pos = str.indexOf("x"); // retorna 3

while (pos != -1) {
  count++;
  pos = str.indexOf("x", pos + 1 /* o mesmo que 3 + 1 */);
}

console.log(count);
```

Veja também

- `String.prototype.charAt()`
- `String.prototype.lastIndexOf()`
- `String.prototype.includes()`
- `String.prototype.split()`
- `Array.prototype.indexOf()`

String.prototype.italics()

O método `italics()` cria um elemento HTML `<i>` que faz com que uma string fique em itálico.

Sintaxe

```
str.italics()
```

Valor retornado

Uma string contendo um elemento HTML `<i>`.

Descrição

O método `italics()` cria uma string dentro de uma tag `<i>`: `"<i>str</i>"`.

Exemplos

Usando <code>italics()</code>

O exemplo a seguir usa métodos do objeto global String para alterar a formatação de uma string:

```
var worldString = "Olá, mundo";
console.log(worldString.blink()); // <blink>Olá, mundo</blink>
console.log(worldString.bold()); // <b>Olá, mundo</b>
console.log(worldString.italics()); // <i>Olá, mundo</i>
console.log(worldString.strike()); // <strike>Olá, mundo</strike>
```

Veja também

- `String.prototype.blink()`
- `String.prototype.bold()`
- `String.prototype.strike()`

String.prototype.lastIndexOf()

O método `lastIndexOf()` retorna o índice da última ocorrência do valor especificado encontrado na `String`. Quando `fromIndex` é especificado, a pesquisa é realizada de trás para frente. Retorna `-1` se o valor não for encontrado.

Sintaxe

```
str.lastIndexOf(searchValue[, fromIndex])
```

Parâmetros

`searchValue`

Uma string representando o valor a ser procurado. Se `searchValue` for uma string vazia, `str.length` é retornado.

`fromIndex`

Opcional. O índice no qual a pesquisa será iniciada de trás para frente. O valor padrão é `+Infinity`. Se `fromIndex >= str.length`, toda string é pesquisada. Se `fromIndex < 0`, o comportamento será o mesmo que seria com o índice `0`.

Valor retornado

O índice da última ocorrência referente ao valor especificado em `searchValue`. É retornado `-1` se nada for encontrado.

Descrição

Os caracteres em uma string são indexados da esquerda para a direita. O índice do primeiro caractere é `0`, e o índice do último caractere é `str.length - 1`.

```
"ricardo".lastIndexOf("r"); // retorna 4
"ricardo".lastIndexOf("a", 3); // retorna 3
"ricardo".lastIndexOf("a", 0); // retorna -1
"ricardo".lastIndexOf("x"); // retorna -1
"ricardo".lastIndexOf("r", -5); // retorna 0
"ricardo".lastIndexOf("r", 0); // retorna 0
"ricardo".lastIndexOf(""); // retorna 7
"ricardo".lastIndexOf("", 2); // retorna 2
```

Sensível a maiúsculas e minúsculas

O método `lastIndexOf()` é sensível a letras maiúsculas e minúsculas. Por exemplo, a seguinte expressão retorna `-1`:

```
"Blue Whale, Killer Whale".lastIndexOf("blue"); // retorna -1
```

Exemplos

Usando <code>lastIndexOf()</code>

O seguinte exemplo usa `lastIndexOf()` para localizar valores nas string `"Brave new world"`.

```
var anyString = "Brave new world";

console.log(
```

```
  "O índice do primeiro w a partir do final é " + anyString.lastIndexOf("w"),
);
// retorna 10

console.log(
  'O índice de "new" a partir do final é ' + anyString.lastIndexOf("new"),
);
// retorna 6
```

Veja também

- `String.prototype.charAt()`
- `String.prototype.indexOf()`
- `String.prototype.split()`
- `Array.prototype.indexOf()`
- `Array.prototype.lastIndexOf()`

String.length

A propriedade `length` de um objeto `String` contém o comprimento da string. `length` é uma propriedade `read-only` (somente leitura) de instâncias de string.

Sintaxe

```
str.length
```

Descrição

Essa propriedade retorna o número de unidades de código presentes na string. UTF-16, a codificação utilizada pelo JavaScript, utiliza uma unidade de código de 16-bits para representar os caracteres mais comuns, mas precisa usar duas unidades para caracteres menos usados, então é possível que o valor retornado por `length` não seja exatamente o número de caracteres na string.

ECMAScript 2016 (ed. 7) estabeleceu um comprimento máximo de `2^53 - 1` elementos. Anteriormente, nenhum comprimento máximo havia sido especificado. No Firefox, as strings têm um comprimento (`length`) máximo de `2**30 - 2` (~ 1 GB). Em versões anteriores ao Firefox 65, o comprimento máximo era `2**28 - 1` (~ 256 MB).

Para uma string vazia, `length` é `0`.

A propriedade estática `String.length` retorna o valor `1`.

Exemplos

Uso básico

```
var x = "Mozilla";
var empty = "";

console.log(
  "Mozilla possui " + x.length + " unidades de código de comprimento",
);
/* "Mozilla possui 7 unidades de código de comprimento" */

console.log("A string vazia possui um comprimento de " + empty.length);
/* "A string vazia possui um comprimento de 0" */
```

Atribuindo valor ao comprimento

```
let myString = "campainhas";

// A tentativa de atribuir um valor à propriedade .length
// de uma string não tem efeito observável.

myString.length = 4;
console.log(myString);
// retorna  "campainhas"
console.log(myString.length);
// retorna 10
```

Veja também

- JavaScript `String.length` and Internationalizing Web Applications

String.prototype.link()

O método `link()` cria uma string que representa o código para um elemento HTML `<a>` a ser usado como um link de hipertexto para outro URL.

Sintaxe

```
str.link(url)
```

Parâmetros

`url`

Qualquer string que especifique o atributo `href` da tag `<a>`. Deve ser um URL válido (relativo ou absoluto), com qualquer caractere `&` escapado como `&amp` e qualquer `"` caractere escapado como `&quot`.

Valor retornado

Uma string contendo um elemento HTML `<a>`.

Descrição

Use o método `link()` para criar um elemento HTML `<a>`. A string retornada pode então ser adicionada ao documento por meio de `document.write()` ou `element.innerHTML`.

Os links criados com o método `link()` tornam-se elementos na array de links do objeto `document`. Veja `document.links`.

Exemplos

Usando <code>link()</code>

O exemplo a seguir exibe a palavra "MDN" como um link que retorna o usuário à Mozilla Developer Network.

```
var hotText = "MDN";
var URL = "https://developer.mozilla.org/";

console.log("Clique para retornar à" + hotText.link(URL));
// Clique para retornar à <a href="https://developer.mozilla.org/">MDN</a>
```

Veja também

- `String.prototype.anchor()`

String.prototype.localeCompare()

O método `localeCompare()` retorna um número que indica se uma string de referência vem antes ou depois, ou é a mesma que a string fornecida na ordem de classificação.

Experimente

```
const a = "réservé"; // With accents, lowercase
const b = "RESERVE"; // No accents, uppercase

console.log(a.localeCompare(b));
// Expected output: 1
console.log(a.localeCompare(b, "en", { sensitivity: "base" }));
// Expected output: 0
```

Os novos argumentos *`locales`* e *`options`* permitem que os aplicativos especifiquem o idioma cuja ordem da ordenação deve ser usada e personalizem o comportamento da função. Em implementações mais antigas, que ignoram os argumentos *`locales`* e *`options`*, a localidade e a ordem de classificação usadas são totalmente dependentes da implementação.

Sintaxe

```
referenceStr.localeCompare(compareString[, locales[, options]])
```

Parâmetros

`compareString`

A string com a qual a *`referenceStr`* é comparada.

`locales` _e _`options`

Esses argumentos personalizam o comportamento da função e permitem que os aplicativos especifiquem o idioma cujas convenções de formatação devem ser usadas. Em implementações que ignoram os argumentos *`locales`* e *`options`*, a localidade usada e a forma da string retornada são inteiramente dependentes da implementação.

Consulte o construtor `Intl.Collator()` para obter detalhes sobre esses parâmetros e como usá-los.

Valor retornado

Um número **negativo** se *`referenceStr`* ocorrer antes de *`compareString`*. Um número **positivo** se o *`referenceStr`* ocorrer após *`compareString`*. `0` se eles forem equivalentes.

Descrição

Retorna um inteiro indicando se *`referenceStr`* vem antes, depois ou é equivalente a *`compareString`*.

- Negativo quando o *`referenceStr`* ocorre antes de *`compareString`*
- Positivo quando o *`referenceStr`* ocorre após *`compareString`*
- Retorna `0` se eles forem equivalentes

> **Aviso: NÃO confie em valores de retorno exatos de -1 ou 1!**

> Os resultados de números inteiros negativos e positivos variam entre os navegadores (bem como entre as versões dos navegadores) porque a especificação W3C exige apenas valores negativos e positivos. Alguns navegadores podem retornar `-2` ou `2`, ou mesmo algum outro valor negativo ou positivo.

Performance

Ao comparar um grande número de strings, como na classificação de grandes arrays, é melhor criar um objeto `Intl.Collator` e usar a função fornecida por sua propriedade `compare`.

Exemplos

Usando <code>localeCompare()</code>

```
// A letra "a" está antes de "c" produzindo um valor negativo
"a".localeCompare("c"); // -2 ou -1 (ou algum outro valor negativo)

// Alfabeticamente, a palavra "verificar" vem depois de "contra", produzindo um valor positivo
"verificar".localeCompare("contra"); // 2 ou 1 (ou algum outro valor positivo)

// "a" e "a" são equivalentes, resultando em um valor neutro de zero
"a".localeCompare("a"); // 0
```

Ordenar um array

`localeCompare()` permite a ordenação sem distinção entre maiúsculas e minúsculas em um array.

```
let items = ["réservé", "Premier", "Cliché", "communiqué", "café", "Adieu"];
items.sort((a, b) => a.localeCompare(b, "fr", { ignorePunctuation: true }));
// ['Adieu', 'café', 'Cliché', 'communiqué', 'Premier', 'réservé']
```

Verifique o suporte do navegador para os novos argumentos

Os argumentos *`locales`* e *`options`* ainda não são suportados em todos os navegadores.

Para verificar se uma implementação os suporta, use o argumento `"i"` (um requisito de rejeição das tags de linguagem ilegal) e procure uma exceção `RangeError`:

```
function localeCompareSupportsLocales() {
  try {
    "foo".localeCompare("bar", "i");
  } catch (e) {
    return e.name === "RangeError";
  }
  return false;
}
```

Usando <code>locales</code>

Os resultados fornecidos por `localeCompare()` variam entre os idiomas. Para obter a ordem de classificação do idioma usado na interface do usuário de seu aplicativo, certifique-se de especificar esse idioma (e possivelmente alguns idiomas substitutos) usando o argumento *`locales`*:

```
console.log("ä".localeCompare("z", "de")); // um valor negativo: em alemão, ä é classificado antes de z
console.log("ä".localeCompare("z", "sv")); // um valor positivo: em sueco, ä é classificado após z
```

Usando <code>options</code>

Os resultados fornecidos por `localeCompare()` podem ser personalizados usando o argumento *`options`*:

```
// em alemão, ä tem a como letra base
console.log("ä".localeCompare("a", "de", { sensitivity: "base" })); // 0

// em sueco, ä e a são letras de base separadas
console.log("ä".localeCompare("a", "sv", { sensitivity: "base" })); // um valor positivo
```

Ordenação numérica

```
// por padrão, "2" > "10"
console.log("2".localeCompare("10")); // 1

// numeric using options:
console.log("2".localeCompare("10", undefined, { numeric: true })); // -1

// numeric using locales tag:
console.log("2".localeCompare("10", "en-u-kn-true")); // -1
```

Veja também

- `Intl.Collator`

String.prototype.match()

Resumo

O método `match()` retorna uma correspondência entre uma string com uma expressão regular.

Sintaxe

```
str.match(regexp);
```

Parâmetros

`regexp`

Um objeto de expressão regular. Se *regexp* não for uma **`RegExp`**, o mesmo será convertido para uma nova RegExp usando `new RegExp(regexp)`.

Se você não fornecer nenhum parâmetro ao usar o método `match()`, você obterá um `Array` com uma string vazia: `[""]`.

Valor retornado

- Se o sinalizador `g` for usado, todos os resultados correspondentes à expressão regular serão retornados, mas a captura de grupos não.
- se o sinalizador `g` não for usado, apenas a primeira correspondência completa e seus grupos de captura relacionados serão retornados. Nesse caso, o item devolvido terá propriedades adicionais conforme descrito a seguir.

`array`

Um `Array` cujo conteúdo depende da presença ou ausência do sinalizador global (g), ou `null` se nenhuma correspondência for encontrada.

Propriedades adicionais

Conforme explicado acima, alguns resultados contêm propriedades adicionais conforme descrito abaixo.

`groups`

Um objeto de grupos de captura nomeados cujas chaves são os nomes, e valores são os grupos de captura ou `undefined` se nenhum grupo de captura nomeado foi definido. Consulte Grupos e Intervalos para obter mais informações.

`index`

O índice da pesquisa em que o resultado foi encontrado.

`input`

Uma cópia da string pesquisada.

Descrição

Se a expressão regular não incluir o sinalizador `g`, `str.match()` retornará o mesmo resultado que `RegExp.exec()`.

Veja também: métodos RegExp

- Se você precisar saber se uma string corresponde a uma expressão regular, use `RegExp.test()`.
- Se você quiser encontrar apenas uma correspondência, você pode querer usar `RegExp.exec()`.
- Se você deseja obter grupos de captura e o sinalizador global (g) está definido, você precisa usar `RegExp.exec()` ou `String.prototype.matchAll()` em vez disso.

Exemplos

Usando <code>match()</code>

No exemplo a seguir, `match()` é usado para encontrar "Capítulo", seguido de um ou mais caracteres numéricos, seguido por um ponto decimal e caracteres numéricos 0 ou mais vezes. A expressão inclui a flag `i` para que diferenças de maiúscula/minúscula sejam ignoradas.

```
const str = "Para maiores informações, veja o Capítulo 3.4.5.1";
const re = /(capítulo \d+(\.\d)*)/i;
const found = str.match(re);

console.log(found);

// retorna ["Capítulo 3.4.5.1",
            "Capítulo 3.4.5.1",
            ".1",
            index: 33,
            input: "Para maiores informações, veja o Capítulo 3.4.5.1"]

// "Capítulo 3.4.5.1" é a primeira correspondência e o primeiro valor
//  capturado a partir de (capítulo \d+(\.\d)*).
// ".1" é o útlimo valor de (\.\d).
// A propriedade "index" (33) é o índice de base zero da correspôndencia inteira.
// A propriedade "input" é a string original que foi analisada.
```

Usando as bandeiras (flags) global e ignore com
<code>match()</code>

O exemplo a seguir demonstra o uso das bandeiras (flags) global e ignore com `match()`. Todas as letras de `A` a `E` e `a` a `e` são retornadas, com cada letra sendo um elemento no array.

```
const str = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
const regexp = /[A-E]/gi;
const matches_array = str.match(regexp);
```

```
console.log(matches_array);
// ['A', 'B', 'C', 'D', 'E', 'a', 'b', 'c', 'd', 'e']
```

> **Nota:** Veja também `String.prototype.matchAll()` e Pesquisa avançada com sinalizadores.

Usando grupos de captura nomeados

Em navegadores que suportam grupos de captura nomeados, o código a seguir captura "`fox`" ou "`cat`" em um grupo denominado "`animal`":

```
const paragraph = "The quick brown fox jumps over the lazy dog. It barked.";

const capturingRegex = /(?<animal>fox|cat) jumps over/;
const found = paragraph.match(capturingRegex);
console.log(found.groups); // {animal: "fox"}
```

Usando <code>match()</code> sem parâmetros

```
const str = "nada se perde, tudo se transforma";

str.match(); // retorna [""]
```

Um objeto não-RegExp como parâmetro

Quando o parâmetro *regexp* é uma string ou um número, ele é convertido implicitamente em um `RegExp` usando o `new RegExp(regexp)`.

Se for um número positivo com um sinal positivo, `RegExp()` irá ignorar o sinal positivo.

```
const str1 =
  "NaN significa 'não é um número'. Infinity contem -Infinity e +Infinity em JavaScript.";
const str2 = "Meu avô tem 65 anos e minha avô tem 63.";
const str3 = "O contrato foi declarado null (nulo) e void (sem efeito)";
str1.match("número"); // "número" é um string. retorna ["número"]
str1.match(NaN); // o tipo de NaN é um número. retorna ["NaN"]
str1.match(Infinity); // o tipo de Infinity é um número. retorna ["Infinity"]
str1.match(+Infinity); // retorna ["Infinity"]
str1.match(-Infinity); // retorna ["-Infinity"]
str2.match(65); // retorna ["65"]
str2.match(+65); // Um número com sinal positivo. retorna ["65"]
str3.match(null); // retorna ["null"]
```

Veja também

- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`

String.prototype.matchAll()

O método `matchAll()` retorna um iterador de todos os resultados correspondentes a uma string em relação a uma expressão regular, incluindo grupos de captura.

Experimente

```
const regexp = /t(e)(st(\d?))/g;
const str = "test1test2";

const array = [...str.matchAll(regexp)];

console.log(array[0]);
// Expected output: Array ["test1", "e", "st1", "1"]

console.log(array[1]);
// Expected output: Array ["test2", "e", "st2", "2"]
```

Sintaxe

```
str.matchAll(regexp)
```

Parâmetros

`regexp`

Um objeto de expressão regular.

Se um objeto *`obj`* não-RegExp for passado, ele será convertido implicitamente em um `RegExp` usando `new RegExp(obj)`.

O objeto `RegExp` deve ter o sinalizador (flag) `/g`, caso contrário, um `TypeError` será retornado.

Valor retornado

Um iterador (que não é um iterável reinicializável).

Exemplos

Regexp.exec() e matchAll()

Antes da adição do `matchAll()` ao JavaScript, era possível usar chamadas regexp.exec (e regexes com a sinalização (flag) `/g`) em um loop para obter todas as correspondências:

```
const regexp = RegExp("foo[a-z]*", "g");
const str = "table football, foosball";
let match;

while ((match = regexp.exec(str)) !== null) {
  console.log(
    `Encontrou ${match[0]} início=${match.index} fim=${regexp.lastIndex}.`,
  );
  // retorna "Encontrou football início=6 fim=14."
  // retorna "Encontou foosball início=16 fim=24."
}
```

Com o `matchAll()` disponível, você pode evitar o loop `while` e executar com `g`.

Em vez disso, usando o `matchAll()`, você obtém um iterador para usar com o mais conveniente `for...of`, `array spread` ou construções `Array.from()`:

```
const regexp = RegExp("foo[a-z]*", "g");
const str = "table football, foosball";
const matches = str.matchAll(regexp);

for (const match of matches) {
  console.log(
    `Encontrou ${match[0]} início=${match.index} fim=${
      match.index + match[0].length
    }.`,
  );
}
// retorna "Encontrou football início=6 fim=14."
// retorna "Encontrou foosball início=16 fim=24."

// O iterador de correspondências se esgota após a iterção for..of
// Chame matchAll novamente para criar um novo iterador
Array.from(str.matchAll(regexp), (m) => m[0]);
// Array [ "football", "foosball" ]
```

`matchAll()` retornará uma exceção se o sinalizador (flag) `g` estiver ausente.

```
const regexp = RegExp("[a-c]", "");
const str = "abc";
str.matchAll(regexp);
// retorna TypeError
```

`matchAll()` cria internamente um clone da `regexp` - portanto, ao contrário de `regexp.exec()`, o `lastIndex` não muda conforme a string é verificada.

```
const regexp = RegExp("[a-c]", "g");
regexp.lastIndex = 1;
const str = "abc";
Array.from(str.matchAll(regexp), (m) => `${regexp.lastIndex} ${m[0]}`);
// Array [ "1 b", "1 c" ]
```

Melhor acesso para capturar grupos (do que String.prototype.match())

Outra razão convincente para usar `matchAll()` é o acesso aprimorado para capturar grupos.

Os grupos de captura são ignorados ao usar `match()` com o sinalizador global `/g`:

```
let regexp = /t(e)(st(\d?))/g;
let str = "test1test2";

str.match(regexp);
// Array ['test1', 'test2']
```

Usando o `matchAll()`, você pode acessar os grupos de captura facilmente:

```
let array = [...str.matchAll(regexp)];

array[0];
// ['test1', 'e', 'st1', '1', index: 0, input: 'test1test2', length: 4]
array[1];
// ['test2', 'e', 'st2', '2', index: 5, input: 'test1test2', length: 4]
```

Veja também

- `String.prototype.match()`
- Using regular expressions in JavaScript

- Capturing groups
- `RegExp`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`

String.prototype.normalize()

O método **`normalize()`** retorna a Forma de Normalização Unicode (Unicode Normalization Form) de uma dada string (se o valor não é uma string, ele será convertido para uma primeiramente).

Sintaxe

```
str.normalize([form])
```

Parâmetros

`form`

Opcional. Um dentre os seguintes valores: `"NFC"`, `"NFD"`, `"NFKC"`, ou `"NFKD"`, especificando o formato de normalização. Se o valor for omitido ou for `undefined`, `"NFC"` será utilizado.

- `NFC` — Formato de Normalização Canônico de Composição.
- `NFD` — Formato de Normalização Canônico de Decomposição.
- `NFKC` — Formato de Normalização de Compatibilidade de Composição.
- `NFKD` — Formato de Normalização de Compatibilidade de Decomposição.

Valor retornado

Uma string contendo a Forma de Normalização Unicode da string dada.

Erros lançados

`RangeError`

Em erro `RangeError` é lançado se `form` não for um dos valores especificados acima.

Descrição

O Unicode atribui um valor numérico exclusivo, denominado *ponto de código*, a cada caractere. Por exemplo, o ponto de código para "*A*" é fornecido como U+0041. No entanto, às vezes mais de um ponto de código, ou sequência de pontos de código, podem representar o mesmo caractere abstrato - o caractere `"ñ"`, por exemplo, pode ser representado por:

- Apenas um ponto de código U+00F1.
- O ponto de código para `"n"` (U+006E) seguido pelo ponto de código para o til (U+0303).

```
let string1 = "\u00F1";
let string2 = "\u006E\u0303";

console.log(string1); //  retorna ñ
console.log(string2); //  retorna ñ
```

No entanto, como os pontos de código são diferentes, a comparação de strings não os tratará como iguais. E como o número de pontos de código em cada versão é diferente, eles até mesmo possuem comprimentos diferentes.

```
let string1 = "\u00F1"; // ñ
let string2 = "\u006E\u0303"; // ñ

console.log(string1 === string2); // retorna false
console.log(string1.length); // retorna 1
console.log(string2.length); // retorna 2
```

O método `normalize()` ajuda a resolver esse problema convertendo uma string em uma forma normalizada comum para todas as sequências de pontos de código que representam os mesmos caracteres. Existem duas principais formas de normalização, uma baseada na **equivalência canônica** e a outra baseada na **compatibilidade**.

Normalização de equivalência canônica

Em Unicode, duas sequências de pontos de código têm equivalência canônica se representarem os mesmos caracteres abstratos e tiverem sempre a mesma aparência visual e comportamento (por exemplo, eles devem sempre ser classificados da mesma maneira).

Você pode usar o `normalize()` usando os argumentos `"NFD"` ou `"NFC"` para produzir uma forma de string que será a mesma para todas as strings canonicamente equivalentes. No exemplo abaixo, normalizamos duas representações do caractere `"ñ"`:

```
let string1 = "\u00F1"; // ñ
let string2 = "\u006E\u0303"; // ñ

string1 = string1.normalize("NFD");
string2 = string2.normalize("NFD");

console.log(string1 === string2); // retorna true
```

```
console.log(string1.length); // retorna 2
console.log(string2.length); // retorna 2
```

Formas compostas e decompostas

Observe que o comprimento da forma normalizada em `"NFD"` é 2. Isso porque `"NFD"` fornece a versão **decomposta** da forma canônica, na qual pontos de código únicos são divididos em vários combinados. A forma canônica decomposta para `"ñ"` é `"\u006E\u0303"`.

Você pode especificar `"NFC"` para obter a forma canônica **composta**, na qual vários pontos de código são substituídos por pontos de código únicos sempre que possível. A forma canônica composta para `"ñ"` é `"\u00F1"`:

```
let string1 = "\u00F1"; // ñ
let string2 = "\u006E\u0303"; // ñ

string1 = string1.normalize("NFC");
string2 = string2.normalize("NFC");

console.log(string1 === string2); // true
console.log(string1.length); // 1
console.log(string2.length); // 1
console.log(string2.codePointAt(0).toString(16)); // f1
```

Normalização de compatibilidade

No Unicode, duas sequências de pontos de código são compatíveis se representarem os mesmos caracteres abstratos e devem ser tratadas da mesma forma em algumas - mas não necessariamente em todas - aplicações.

Todas as sequências canonicamente equivalentes também são compatíveis, mas não o contrário.

Por exemplo:

- o ponto de código U+FB00 representa a ligadura `"ff"`. É compatível com dois pontos de código U+0066 consecutivos (`"ff"`).
- o ponto de código U+24B9 representa o símbolo `"Ⓓ"`. É compatível com o ponto de código U+0044 (`"D"`).

Em alguns aspectos (como classificação), eles devem ser tratados como equivalentes - e em alguns (como a aparência visual) não devem, portanto, não são canonicamente equivalentes.

Você pode usar o `normalize()` usando os argumentos `"NFKD"` ou `"NFKC"` para produzir uma forma de string que será a mesma para todas as strings compatíveis:

```
let string1 = "\uFB00";
let string2 = "\u0066\u0066";

console.log(string1); // ff
console.log(string2); // ff
console.log(string1 === string2); // false
console.log(string1.length); // 1
console.log(string2.length); // 2

string1 = string1.normalize("NFKD");
string2 = string2.normalize("NFKD");

console.log(string1); // ff <- aparência visual modificada
console.log(string2); // ff
console.log(string1 === string2); // true
console.log(string1.length); // 2
console.log(string2.length); // 2
```

Ao aplicar a normalização de compatibilidade, é importante considerar o que você pretende fazer com as strings, uma vez que a forma normalizada pode não ser apropriada para as aplicações. No exemplo acima, a normalização é apropriada para pesquisa, porque permite que um usuário encontre a string pesquisando por `"f"`. Mas pode não ser apropriado para exibição, porque a representação visual é diferente.

Como na normalização canônica, você pode solicitar formulários compatíveis decompostos ou compostos passando `"NFKD"` ou `"NFKC"`, respectivamente.

Exemplos

Usando <code>normalize()</code>

```
// String Inicial

// U+1E9B: CARACTERE LATINO - LETRA S COMPRIDA COM PONTO ACIMA
// U+0323: COMBINANDO PONTO ABAIXO
var str = "\u1E9B\u0323";

// Formato de Normalização Canônico de Composição (NFC)

// U+1E9B: CARACTERE LATINO - LETRA S COMPRIDA COM PONTO ACIMA
// U+0323: COMBINANDO PONTO ABAIXO
str.normalize("NFC"); // '\u1E9B\u0323'
str.normalize(); // igual à linha de cima

// Formato de Normalização Canônico de Decomposição (NFD)

// U+017F: CARACTERE LATINO - LETRA S COMPRIDA
// U+0323: COMBINANDO PONTO ABAIXO
```

```
// U+0307: COMBINANDO PONTO ACIMA
str.normalize("NFD"); // '\u017F\u0323\u0307'

// Formato de Normalização de Compatibilidade de Composição. (NFKC)

// U+1E69: CARACTERE LATINO - LETRA S COMPRIDA COM PONTO ACIMA E ABAIXO
str.normalize("NFKC"); // '\u1E69'

// Formato de Normalização de Compatibilidade de Decomposição (NFKD)

// U+0073: CARACTERE LATINO - LETRA S COMPRIDA
// U+0323: COMBINANDO PONTO ABAIXO
// U+0307: COMBINANDO PONTO ACIMA
str.normalize("NFKD"); // '\u0073\u0323\u0307'
```

Veja também

- Unicode Standard Annex #15, Unicode Normalization Forms
- Unicode equivalence

String.prototype.padEnd()

O método `padEnd()` preenche a string original com um determinado caractere, ou conjunto de caraceres, (repetidamente, se necessário) para que a string resultante alcance um determinado comprimento. O preenchimento é aplicado a partir do final (direita) da string original. A string original não é modificada.

Experimente

```
const str1 = "Breaded Mushrooms";

console.log(str1.padEnd(25, "."));
// Expected output: "Breaded Mushrooms........"

const str2 = "200";

console.log(str2.padEnd(5));
// Expected output: "200  "
```

Sintaxe

```
str.padEnd(targetLength [, padString])
```

Parâmetros

`targetLength`

> O comprimento da string resultante após a string original ter sido preenchida. Se o valor for menor do que o próprio comprimento da string original, a string original é retornada sem modificações.

`padString`

> Opcional. O caractere (ou caracteres) que deve completar a string atual. Caso o comprimento desta string seja muito longo, estando acima do comprimento alvo, ela será truncada e sua parte esquerda restante é aplicada. O valor padrão para esse parâmetro é " " (`U+0020`).

Valor retornado

Uma `String` cuja composição vem da string original, completada por um ou mais caracteres de preenchimento, respeitando o comprimento alvo.

Exemplos

Usando padEnd

```
"abc".padEnd(10); // "abc       "
"abc".padEnd(10, "foo"); // "abcfoofoof"
"abc".padEnd(6, "123456"); // "abc123"
"abc".padEnd(1); // "abc"
```

Polyfill

Rodando o seguinte código antes de qualquer código irá criar o método `String.prototype.padEnd()` caso ele não esteja disponível nativamente:

```
// https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/String/padEnd
if (!String.prototype.padEnd) {
  String.prototype.padEnd = function padEnd(targetLength, padString) {
    targetLength = targetLength >> 0; //floor if number or convert non-number to 0;
    padString = String(typeof padString !== "undefined" ? padString : " ");
    if (this.length > targetLength) {
      return String(this);
    } else {
      targetLength = targetLength - this.length;
      if (targetLength > padString.length) {
        padString += padString.repeat(targetLength / padString.length); //append to original to ensure we are longer than needed
      }
      return String(this) + padString.slice(0, targetLength);
```

```
    }
  };
}
```

Veja também

- `String.prototype.padStart()`

String.prototype.padStart()

O método `padStart()` preenche a string original com um determinado caractere, ou conjunto de caracteres, (várias vezes, se necessário) até que a string resultante atinja o comprimento fornecido. O preenchimento é aplicado antes do primeiro caractere da string original. A string original não é modificada.

Experimente

```
const str1 = "5";

console.log(str1.padStart(2, "0"));
// Expected output: "05"

const fullNumber = "2034399002125581";
const last4Digits = fullNumber.slice(-4);
const maskedNumber = last4Digits.padStart(fullNumber.length, "*");

console.log(maskedNumber);
// Expected output: "************5581"
```

Sintaxe

```
str.padStart(targetLength [, padString])
```

Parâmetros

`targetLength`

> O comprimento da string resultante uma vez que a string alvo tenha sido preenchida. Caso seu valor seja menor do que o comprimento da string alvo, é retornado a própria string alvo, sem alterações.

`padString`

> Opcional. O caractere, ou caracteres, que deve preencher a string alvo. Caso o comprimento dessa essa string de preenchimento seja superior ao `targetLength`, ela será truncada a partir da direita. O valor padrão é `" "` (`U+0020 'SPACE'`).

Valor de retorno

Uma `String` de comprimento específico com uma string de preenchimento aplicada a partir do seu início.

Exemplos

Exemplos básicos

```
"abc".padStart(10); // "       abc"
"abc".padStart(10, "foo"); // "foofoofabc"
"abc".padStart(6, "123465"); // "123abc"
"abc".padStart(8, "0"); // "00000abc"
"abc".padStart(1); // "abc"
```

Polyfill

Ao executar o seguinte código antes de qualquer outro código é criado o método `String.prototype.padStart()`, em casos onde ele não está disponível nativamente:

```
// https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/String/padStart
if (!String.prototype.padStart) {
  String.prototype.padStart = function padStart(targetLength, padString) {
    targetLength = targetLength >> 0; //truncate if number, or convert non-number to 0;
    padString = String(typeof padString !== "undefined" ? padString : " ");
    if (this.length >= targetLength) {
      return String(this);
    } else {
      targetLength = targetLength - this.length;
      if (targetLength > padString.length) {
        padString += padString.repeat(targetLength / padString.length); //append to original to ensure we are longer than needed
      }
      return padString.slice(0, targetLength) + String(this);
    }
  };
}
```

Veja também

- `String.prototype.padEnd()`

String.raw()

O método estático `String.raw()` é uma função tag de modelos literais, similar ao prefixo `r` no Python ou o prefixo `@` no C# para string literais (Mas não é *idêntico*, existe uma diferença, veja explicações nessa discussão). Ele é usado para pegar as strings no formato "cru" de modelos literais, isto é, substituições (ex: `${foo}`) são processados, mas "escapes" (ex:. `\n`) não são.

Experimente

```
// Create a variable that uses a Windows
// path without escaping the backslashes:
const filePath = String.raw`C:\Development\profile\aboutme.html`;

console.log(`The file was uploaded from: ${filePath}`);
// Expected output: "The file was uploaded from: C:\Development\profile\aboutme.html"
```

Sintaxe

```
String.raw(callSite, ...sub)

String.raw`templateString`
```

Parâmetros

`callSite`

Modelo bem-formatado de objeto de local de chamada, como `{ raw: ['foo', 'bar', 'baz'] }`.

`...substitutions`

Contém os valores das substituições.

`templateString`

Um modelo string, opcionalmente com substituições (`${...}`).

Valor retornado

A forma crua de uma string de um modelo string dado.

Exceções

`TypeError`

Um `TypeError` é jogado se o primeiro argumento não é um Objeto bem formado.

Descrição

Na maioria dos casos, `String.raw()` é usado com modelos de strings. A primeira sintaxe mencionada acima raramente é usada, porque o mecanismo JavaScript a chamará com os argumentos apropriados para você, assim como com outras funções de tag.

`String.raw()` é a única função de tag embutida de strings de template; ele funciona exatamente como a função de modelo padrão e executa a concatenação. Você pode até reimplementá-lo com o código JavaScript normal.

Exemplos

Usando <code>String.raw()</code>

```
String.raw`Hi\n${2 + 3}!`;
// 'Hi\n5!', o caractere após 'Hi'
// não é um caractere de quebra de linha,
// '\' e 'n' são dois caracteres.

String.raw`Hi\u000A!`;
// 'Hi\u000A!', o mesmo aqui, agora nós teremos os caracteres
//  \, u, 0, 0, 0, A, 6.
// Todos as formas de quebra de linha serão ineficazes
// e as contra barras estarão inclusas no valor retornado.
// Você pode confirmar isso verificando a propriedade .length
// da string.

let name = "Bob";
String.raw`Hi\n${name}!`;
// 'Hi\nBob!', substituições são processadas.

// Normalmente você não chamaria String.raw() como uma função,
// mas para simular `t${0}e${1}s${2}t` você pode fazer:
String.raw({ raw: "test" }, 0, 1, 2); // 't0e1s2t'
// Note que 'test', uma string, é um objeto array-like
// O código abaixo é equivalente a:
// `foo${2 + 3}bar${'Java' + 'Script'}baz`
String.raw(
  {
    raw: ["foo", "bar", "baz"],
  },
  2 + 3,
```

```
  "Java" + "Script",
); // 'foo5barJavaScriptbaz'
```

Veja também

- Modelos de strings
- `String`
- Gramática lexical

String.prototype.repeat()

O método **`repeat()`** constrói e retorna uma nova string com um determinado número de cópias concatenadas da string original.

Sintaxe

```
str.repeat(count);
```

Parâmetros

`count`

Um número inteiro entre 0 e `+Infinity`, indicando o número de vezes que a string deve ser repetida.

Valor retornado

Uma nova string contendo o número especificado de cópias da string original.

Exceções

- `RangeError`: o número de repetições não pode ser negativo.
- `RangeError`: o número de repetições deve ser menor que infinito e não deve ultrapassar o tamanho máximo da string.

Exemplos

```
"abc".repeat(-1); // RangeError
"abc".repeat(0); // ''
"abc".repeat(1); // 'abc'
"abc".repeat(2); // 'abcabc'
"abc".repeat(3.5); // 'abcabcabc' (o número será convertido para inteiro)
"abc".repeat(1 / 0); // RangeError

({ toString: () => "abc", repeat: String.prototype.repeat }).repeat(2);
// 'abcabc' (repeat() é um método genérico)
```

Polyfill

O método `repeat()` foi adicionado à especificação ECMAScript 2015 e pode ainda não estar disponível em todas as implementações do JavaScript. No entanto, você pode usar o seguinte polyfill para implementar o `String.prototype.repeat()`:

```
if (!String.prototype.repeat) {
  String.prototype.repeat = function (count) {
    "use strict";
    if (this == null) {
      throw new TypeError(
        "não é possível converter " + this + " para um objeto",
      );
    }
    var str = "" + this;
    count = +count;
    if (count != count) {
      count = 0;
    }
    if (count < 0) {
      throw new RangeError("o núm. de repetições não pode ser negativo");
    }
    if (count == Infinity) {
      throw new RangeError("o núm. de repetições deve ser menor que infinito");
    }
    count = Math.floor(count);
    if (str.length == 0 || count == 0) {
      return "";
    }

    // Ao Garantir que count seja um inteiro de 31 bits nos dá uma grande otimização
    // na parte principal. Porém, navegadores atuais (de agosto de 2014 pra cá)
    // não conseguem mais manipular strings de 1 << 28 chars ou maiores, então:
    if (str.length * count >= 1 << 28) {
      throw new RangeError(
        "o núm. de repetições não deve estourar o tamanho máx. de uma string",
      );
    }
    var rpt = "";
    for (var i = 0; i < count; i++) {
      rpt += str;
```

```
    }
    return rpt;
  };
}
```

Polyfill ES5

```
//#es5
'use strict';
(function(win){
 var typeOf=(function(w){var f=function f(x){return typeof(x)},o=w.Symbol,p;if(o && typeof(o)==='function' && typeof(o.iterator)==='symbol')
{p=o.prototype;f=function(x){return x && x.constructor===o && x!==p?'symbol':typeof x}};return f})(win),
 exist=function(o,p,t){return p in o && typeOf(o[p])===t};
 (function(w){
    var o=w.String.prototype;
    if(!exist(o,'repeat','function')){o.repeat=(function(A,E){return function(n){var i=n>>0,s=this,l=s.length,j;if(i===0||l<1)
{s=''}else{j=268435456;if(i<0||i>=j||i*l>j){throw new RE('Invalidcountvalue')}else if(i>0){s=A(++i).join(s)}};return s}})(w.Array,w.RangeError)};
 })(win);
})(window);

// teste:
console.clear();
console.log(
'abc'.repeat(false),//''
'abc'.repeat({}),//''
'abc'.repeat([]),//''
'abc'.repeat(['']),//''
'abc'.repeat([0]),//''
'abc'.repeat([0,1]),//''
'abc'.repeat([1,1]),//''
'abc'.repeat(0),//''
'abc'.repeat(.6),//''
'abc'.repeat(true),//'abc'
'abc'.repeat(1),//'abc'
'abc'.repeat(2),//'abcabc'
'abc'.repeat([2]),//'abcabc'
'abc'.repeat(3.5),//'abcabcabc'
''.repeat(2)//''
);
console.log(
'abc'.repeat(-Infinity),//RangeError: Invalid count value
'abc'.repeat(Infinity),//RangeError: Invalid count value
'abc'.repeat(1/0),//RangeError: Invalid count value
'abc'.repeat(-1)//RangeError: Invalid count value
);

/*
es5 src:
'use strict';
(function(win){

 var typeOf=(function(w){var f=function f(x){return typeof(x)},o=w.Symbol,p;if(o && typeof(o)==='function' && typeof(o.iterator)==='symbol')
{p=o.prototype;f=function(x){return x && x.constructor===o && x!==p?'symbol':typeof x}};return f})(win),
 exist=function(o,p,t){return p in o && typeOf(o[p])===t};

 (function(w){
    var o=w.String.prototype;
    if(!exist(o,'repeat','function')){
        o.repeat=(function(A,E){
            return function(n){
                var i=n>>0,s=this,l=s.length,j;
                if(i===0||l<1){s=''}else{
                    j=268435456;
                    if(i<0||i>=j||i*l>j){throw new RE('Invalidcountvalue')}else if(i>0){s=A(++i).join(s)}
                };
                return s
            };
        })(w.Array,w.RangeError);
    };
    //..
 })(win);

})(window);
*/
```

Polyfill ES6

```
//#es6

(w=>{

    const typeOf=(o=>{let f=x=>typeof x;if(o && 'function'===typeof o){const s='symbol';if(s===typeof o.iterator){const p=o.prototype;f=x=>x &&
x.constructor===o && x!==p?s:typeof x}};return f})(w.Symbol),

    exist=(o,p,t)=>p in o && typeOf(o[p])===t;
```

```
    (o=>{

        if(!exist(o,'repeat','function')){const A=w.Array,E=w.RangeError;o.repeat=function(n){var i=n>>0,s='';if(i!==0){let t=this;const
l=t.length;if(l!==0){if(i<0||i>=(t=268435456)||i*l>t){throw new E('Invalid count value')}else if(i>0){s=A(++i).join(t)}}};return s}};

    })(w.String.prototype);

})(window);

/*

es6 src:

(w=>{

    const typeOf=(o=>{let f=x=>typeof x;if(o && 'function'===typeof o){const s='symbol';if(s===typeof o.iterator){const p=o.prototype;f=x=>x &&
x.constructor===o && x!==p?s:typeof x}};return f})(w.Symbol),

    exist=(o,p,t)=>p in o && typeOf(o[p])===t;


    (o=>{

        if(!exist(o,'repeat','function')){

            const A=w.Array;

            o.repeat=function(n){var i=n>>0,s='';if(i!==0){let t=this;const l=t.length;if(l!==0){if(i<0||i>=(t=268435456)||i*l>t){throw new
RangeError('Invalid count value')}else if(i>0){s=A(++i).join(t)}}};return s};

        };

        //..

    })(w.String.prototype);


})(window);

*/


//test:

console.clear();

console.log(

'abc'.repeat(false),//''

'abc'.repeat({}),//''

'abc'.repeat([]),//''

'abc'.repeat(['']),//''

'abc'.repeat([0]),//''

'abc'.repeat([0,1]),//''

'abc'.repeat([1,1]),//''

'abc'.repeat(0),//''

'abc'.repeat(.6),//''

'abc'.repeat(true),//'abc'

'abc'.repeat(1),//'abc'

'abc'.repeat(2),//'abcabc'

'abc'.repeat([2]),//'abcabc'

'abc'.repeat(3.5),//'abcabcabc'

''.repeat(2)//''

);

console.log(

'abc'.repeat(-Infinity),//RangeError: Invalid count value

'abc'.repeat(Infinity),//RangeError: Invalid count value
```

```
'abc'.repeat(1/0),//RangeError: Invalid count value

'abc'.repeat(-1)//RangeError: Invalid count value

);
```

Veja também

- `String.prototype.concat()`

String.prototype.replace()

Resumo

O método `replace()` retorna uma nova string com algumas ou todas as correspondências de um padrão substituídas por um determinado caractere (ou caracteres). O padrão pode ser uma string ou uma `RegExp`, e a substituição pode ser uma string ou uma função a ser chamada para cada correspondência. Se o padrão for uma string, apenas a primeira ocorrência será substituída.

A string original não é modificada.

Experimente

```
const paragraph = "I think Ruth's dog is cuter than your dog!";

console.log(paragraph.replace("Ruth's", "my"));
// Expected output: "I think my dog is cuter than your dog!"

const regex = /Dog/i;
console.log(paragraph.replace(regex, "ferret"));
// Expected output: "I think Ruth's ferret is cuter than your dog!"
```

Sintaxe

```
str.replace(regexp|substr, newSubStr|function)
```

Parâmetros

`regexp`

Um objeto `RegExp` ou literal. A correspondência ou correspondências são substituídas por *`newSubStr`* ou o valor retornado pela `function` especificada.

`substr`

Uma `String` que será substituída por `newSubStr`. Ele é tratado como uma string textual e não é interpretado como uma expressão regular. Apenas a primeira ocorrência será substituída.

`newSubStr`

A `String` que substitui a `substr` recebida do parâmetro #1. Uma série de padrões de substituições especiais são suportados. Veja a seção "

Especificando uma string como parâmetro

" abaixo.

`function`

A função (function) chamada cria uma nova substring (para ser colocada no lugar da substring recebida pelo parametro #1). Os argumentos fornececidos para essa função estão descritos na seção "

Especificando uma função como parâmetro

" mais abaixo.

`flags`

Uma string especificando uma combinação de flags de expressão regular. O uso do parâmetro `flags` no método `String.prototype.replace()` é não-padrão. Ao invés de usar este parâmetro, use um objeto `RegExp` com as flags correspondentes. O valor deste parâmetro deve ser uma string consistindo em um ou mais dos seguintes caracteres para afetar a operação, tais como descrito:

`g`

Combinação global.

`i`

Ignora diferenças entre maiúsculas e minúsculas.

`m`

Combinação em várias linhas.

`y`

Sticky

> **Nota:** O argumento `flags` não funciona no v8 Core (Chrome e NodeJs).

Valor retornado

Uma nova string com alguma ou todas as combinações do padrão substituído(s) pelo valor de substituição.

Descrição

Este método não muda o objeto `String`. Ele simplesmente retorna uma nova string.

Para realizar uma pesquisa global e substituir, inclua a flag `g` na expressão regular ou se o primeiro parâmetro for uma string, inclua `g` no parâmetro flags.

Especificando uma string como parâmetro

A string substituidora pode incluir o seguinte padrão de substituição especial:

Padrão	Insere
`$$`	Insere um "$".
`$&`	Insere a string casada.
`$``	Insere a porção da string que precede a substring combinada.
`$'`	Insere a porção da string que segue a substring combinada.
`$n` ou `$nn`	Onde `n` ou `nn` são dígitos decimais, insere a *n*-ésima substring entre parêntesis casada, dado o primeiro argumento foi um objeto `RegExp`.

Especificando uma função como parâmetro

Você pode especificar uma função no segundo parâmetro. Neste caso, a função será chamada depois que a correspôndecia for encontrada. O resultado da função (valor retornado) será usado como a string substituta. (Atenção: os padrões de substituição citados acima não se aplicam neste caso). Note que a função será chamada múltiplas vezes para combinação que deve ser substituída se a expressão regular no primeiro parâmetro tiver a regra global.

Os parâmetros da função são:

Possíveis nomes	Valor fornecido
`match`	A substring encontrada. Corresponde ao `$&` acima.
`p1, p2, ...`	O enésimo parâmetro entre parênteses da RegEx no primeiro parâmetro na função `replace()` `RegExp`. (Corresponde a `$1`, `$2`, etc. acima.) Por exemplo, se `/(\a+)(\b+)/`, for o primeiro parâmetro, `p1` é a combinação para `\a+`, e `p2` para `\b+`.
`offset`	O offset da string encontrada em relação ao resto da string. Por exemplo, se a string for 'abcd' e a string a ser encontrada for 'bc', então este parâmetro terá o valor 1.
`string`	A string completa que está sendo examinada.

(O número exato de argumentos dependerá se o primeiro parâmetro for uma `RegExp` e de quantas combinações entre parênteses houver).

O exemplo a seguir irá substituir o valor de `newString` para `'abc - 12345 - #$*%'`:

```
function replacer(match, p1, p2, p3, offset, string) {
  // p1 não possui digitos,
  // p2 possui dígitos, e p3 não possui alfanuméricos
  return [p1, p2, p3].join(" - ");
}
var newString = "abc12345#$*%".replace(/([^\d]*)(\d*)([^\w]*)/, replacer);

// retorna "abc - 12345 - #$*%"
```

Exemplos

Definindo uma expressão regular com <code>replace()</code>

No exemplo a seguir foi definida uma expressão regular com a flag "`i`" (que ignora diferenças entre maiúsculas e minúsculas) no método `replace()`.

```
var str = "Twas the night before Xmas...";
var newstr = str.replace(/xmas/i, "Christmas");

console.log(newstr);
// retorna "Twas the night before Christmas..."
```

> **Nota:** Veja este guia para maiores explicações as sobre expressões regulares.

Usando <code>global</code> e <code>ignore</code> com <code>replace()</code>

Substituir globalmente, "`g`", só pode ser feito com uma expressão regular. No exemplo a seguir, a expressão regular inclui as flags global e ignore que permitem a função `replace()` substituir cada "`maçãs`" por "`laranjas`" na string.

```
var re = /maçãs/gi;
var str = "As maçãs são redondas. As maçãs são suculentas.";
```

```
var newstr = str.replace(re, "laranjas");

console.log(newstr);
// retorna
// "As laranjas são redondas. As laranjas são suculentas."
```

Trocando palavras em uma string

O script a seguir troca as palavras na string. Para o texto que vai substituir, o script usa grupos de captura e os padrões de substituição `$1` e `$2`.

```
var re = /(\w+)\s(\w+)/;
var str = "John Smith";
var newstr = str.replace(re, "$2, $1");
console.log(newstr); // Smith, John
```

Usando uma função que modifica os caracteres coincidentes

Neste exemplo, todas as ocorrências de letras maiúsculas na string são convertidas em minúsculas e um hífen é inserido antes do local de correspondência. O importante aqui é que é necessário uma operação adicional no item antes dele ser retornado como substituído.

A função de substituição aceita a string coincidida como parâmetro e usa ela para transformar os caracteres e concatenar um hífen antes de retornar.

```
function styleHyphenFormat(propertyName) {
  function upperToHyphenLower(match, offset, string) {
    return (offset ? "-" : "") + match.toLowerCase();
  }
  return propertyName.replace(/[A-Z]/g, upperToHyphenLower);
}
```

Dado o seguinte parâmetro: `styleHyphenFormat('borderTop')`, o valor retornado é `'border-top'`.

Como queremos transformar o resultado da coincidencia antes da substituição final, nós devemos usar uma função. Isto força que a transformação seja feita antes da chamada do método `toLowerCase()`. Se tivéssemos tentado isto sem a função, o método `toLowerCase()` não teria efeito.

```
let newString = propertyName.replace(/[A-Z]/g, "-" + "$&".toLowerCase()); // não funciona
```

Isso acontece porque `'$&'.toLowerCase()` será executada antes (resultando no mesmo que `'$&'`) em vez de usar os caracteres da string a ser transformada.

Substituindo graus Fahrenheit em Celsius

O exemplo a seguir converte graus Fahrenheit em Celsius. O grau Fahrenheit deve ser um número terminado com "`F`". A função retorna o número em Celsius terminando em "`C`". Por exemplo, se o valor de entrada for "`212F`", a função deve retornar "`100C`". Se o número for "`0F`", a função deve retornar "`-17.77777777777778C`".

A expressão regular `test` verifica por números que terminem com "`F`". O número de graus Fahrenheit é acessível pela função pelo segundo parâmetro, `p1`. A função calcula o Celsius baseado no Fahrenheit passado via string para a função `f2c()`. A `f2c()` então retorna o número em Celsius.

```
function f2c(x) {
  function convert(str, p1, offset, s) {
    return ((p1 - 32) * 5) / 9 + "C";
  }
  var s = String(x);
  var test = /(-?\d+(?:\.\d*)?)F\b/g;
  return s.replace(test, convert);
}
```

Use uma função com expressão regular para evitar loops for

O exemplo a seguir pega um padrão de string e converte em um array de objetos.

Entrada:

Uma string com caracteres: `x`, `-` e `_`

```
x-x_
x---x---x---x---
x-xxx-xx-x-
x_x_x___x___x___
```

Saída:

Um array de objetos. Um `'x'` denota um estado `'on'`, um `'-'` (hífen) denota um estado `'off'` e um `'_'` (underline) denota o comprimento do estado `'on'`.

```
[
  { on: true, length: 1 },
  { on: false, length: 1 },
  { on: true, length: 2 }
  ...
]
```

Código:

```
var str = "x-x_";
var retArr = [];
str.replace(/(x_*)|(-)/g, function (match, p1, p2) {
  if (p1) {
    retArr.push({ on: true, length: p1.length });
  }
  if (p2) {
    retArr.push({ on: false, length: 1 });
  }
});

console.log(retArr);
```

O código gera um array de 3 objetos como desejado sem usar uma função de loop.

Veja também

- `String.prototype.match()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`

String.prototype.replaceAll()

O método `replaceAll()` retorna uma nova string com todas as ocorrências de um padrão substituídas por uma substituição. O padrão pode ser uma string ou uma `RegExp`, e a substituição pode ser uma string ou uma função a ser chamada para cada ocorrência.

A *string* original é mantida sem modificação.

Experimente

```
const paragraph = "I think Ruth's dog is cuter than your dog!";

console.log(paragraph.replaceAll("dog", "monkey"));
// Expected output: "I think Ruth's monkey is cuter than your monkey!"

// Global flag required when calling replaceAll with regex
const regex = /Dog/gi;
console.log(paragraph.replaceAll(regex, "ferret"));
// Expected output: "I think Ruth's ferret is cuter than your ferret!"
```

Sintaxe

```
const newStr = str.replaceAll(regexp|substr, newSubstr|function)
```

> **Nota:** Quando usar uma `regexp`, você deve colocar o sinalizador (flag) global "`g`". Do contrário, será retornado um `TypeError`: "*replaceAll must be called with a global RegExp*".

Parâmetros

`regexp` (padrão)

Uma `RegExp` ou literal com o sinalizador (flag) global. As ocorrências serão substituídas por *`newSubstr`* ou pelo valor retornado da *`function`* (função) especificada.

`substr` (padrão)

Uma `String` que deve ser substituída por `newSubstr`. É tratada como uma string literal e não é interpretada como uma expressão regular (*`RegExp`*).

`newSubstr` (substituição)

É a `String` que substitui a `substring` especificada pelo parâmetro `regexp` ou `substr`. Um número de padrões especiais para substituição são suportados; veja a seção "

Especificando uma string como parâmetro

" abaixo.

`function` (substituição)

Uma função a ser chamada retornando a nova `substring` a ser usada para substituir as correspondências para a dada `regexp` ou `substr`. Os argumentos passados para esta função são descritos na seção "Especificando uma função como parâmetro" abaixo.

Valor de retorno

Um nova *string*, com todas as ocorrências de um padrão substituído por uma substituição.

Descrição

Este método não muda o objeto `String` original. Ele simplesmente retorna uma nova *string*.

Especificando uma string como parâmetro

A *string* de substituição pode incluir os seguimentos padrões especiais de substituição:

Padrão	Insere
`$$`	Insere um "`$`".
`$&`	Insere a *substring* correspondente.
`$``	Insere a porção da *string* que precede a *substring* correspondente.
`$'`	Insere a porção da *string* que sucede a *substring* correspondente.
`$n`	Onde `n` é um inteiro positivo menor que 100, insere a *n*-ésima *string* submatch entre parênteses, desde que o primeiro argumento seja um objeto `RegExp`. Note que isso é indexado começando pelo `1`.

Especificando uma função como parâmetro

Você pode especificar uma função como segundo parâmetro. Neste caso, a função será chamada depois da ocorrência ter sido encontrada. O resultado da função (valor de retorno) será usado como uma *string* para substituição. (**Nota:** Os padrões especiais mencionados acima **não se aplicam** neste caso.)

Note que a função será chamada múltiplas vezes para cada ocorrência a ser substituída se a expressão regular no primeiro parâmetro for global "`g`".

Os argumentos para funções são os seguintes:

Nome possível	Valor fornecido
`match`	A *substring* correspondida. (Corresponde ao `$&` acima.)
`p1, p2, ...`	A *n*-ésima string encontrada por um grupo de captura entre parênteses, desde que o primeiro argumento para `replace()` seja uma `RegExp`. (Corresponde a `$1`, `$2`, etc. acima.) Por exemplo, se `/(\a+)(\b+)/`, foi dado, `p1` é a correspondência para `\a+`, e `p2` para `\b+`.
`offset`	O deslocamento da substring correspondente em toda a string sendo examinada. (Por exemplo, se toda a string for '`abcd`' e a substring correspondente for '`bc`', este argumento será `1`.)
`string`	A *string* inteira será examinada.

(O número exato de argumentos depende se o primeiro argumento é um objeto `RegExp` — e, se então, quantas *subcorrespondências* entre parênteses são especificadas.)

Exemplos

Usando <code>replaceAll()</code>

```
"aabbcc".replaceAll("b", ".");
// 'aa..cc'
```

RegExp sem flag "g" retorando erro

Ao usar uma expressão regular para realizar uma pesquisa, a mesma deve conter a flag global "`g`". O código abaixo não irá funcionar:

```
'aabbcc'.replaceAll(/b/, '.');
TypeError: replaceAll must be called with a global RegExp
```

Já o código abaixo vai funcionar:

```
"aabbcc".replaceAll(/b/g, ".");
("aa..cc");
```

Veja também

- `String.prototype.replace()`
- `String.prototype.match()`
- `RegExp.prototype.exec()`
- `RegExp.prototype.test()`

String.prototype.search()

O método `search()` realiza uma busca por uma ocorrência entre uma expressão regular e uma `String`.

Sintaxe

```
str.search(regexp)
```

Parâmetros

`regexp`

> Um objeto contendo uma expressão regular. Se um objeto `obj` for passado como parâmetro e não for do tipo `RegExp`, ele será implicitamente convertido para uma `RegExp` através da instrução `new RegExp(obj)`.

Valor retornado

O índice na string do primeiro trecho que satisfaz a expressão regular. Do contrário, o valor `-1` será retornado.

Descrição

Quando você quiser saber se um padrão foi encontrado, e também saber seu índice dentro de uma string, use `search()`. (Se você deseja apenas saber se ele existe, use o método semelhante `test()` do protótipo RegExp, que retorna um booleano.)

Para obter mais informações (mas em compensação a execução se torna mais lenta), use `match()` (semelhante ao método `exec()` da RegExp).

Exemplos

Usando <code>search()</code>

O exemplo a seguir pesquisa uma string com dois objetos `regexp` diferentes para mostrar uma pesquisa bem-sucedida (valor positivo) vs. uma pesquisa mal-sucedida (-1).

```
let str = "hey JudE";
let re = /[A-Z]/g;
let reDot = /[.]/g;
console.log(str.search(re)); // retorna 4, que é o índice da primeira letra maiúscula "J"
console.log(str.search(reDot)); // retorna -1 pois não conseguiu encontrar o ponto "."
```

Notas específicas para a engine Gecko

- Antes do Gecko 8.0, `search()` foi implementado incorretamente. Quando era chamadosem parâmetros ou com `undefined`, ele buscava pela string '`undefined`', ao invés de buscar pela string vazia. Isto foi corrigido. Agora `'a'.search()` e `'a'.search(undefined)` corretamente retornam 0.
- A partir do Gecko 39, o argumento não-padrão `flags` está defasado (deprecated) e dispara um aviso no console (Erro do Firefox 1142351).
- A partir do Gecko 47, o argumento não-padrão `flags` não é mais suportado em builds *non-release* e em breve será removido inteiramente (Erro do Firefox 1245801).
- A partir do Gecko 49, o argumento não-padrão `flags` não é mais suportado (Erro do Firefox 1108382).

Veja também

- `String.prototype.match()`
- `RegExp.prototype.exec()`

String.prototype.slice()

Sumário

O método `slice()` extrai uma parte de uma string e a retorna como uma nova string, sem modificar a string original.

Experimente

```
const str = "The quick brown fox jumps over the lazy dog.";

console.log(str.slice(31));
// Expected output: "the lazy dog."

console.log(str.slice(4, 19));
// Expected output: "quick brown fox"

console.log(str.slice(-4));
// Expected output: "dog."

console.log(str.slice(-9, -5));
// Expected output: "lazy"
```

Sintaxe

```
str.slice(startIndex[, endIndex])
```

Paramêtros

`startIndex`

> É o índice, de base zero, onde se inicia a extração. Se for um número negativo, será o mesmo que executar *`str.length + startIndex`* (por exemplo, se *`startIndex`* for *`-3`*, será o mesmo que executar *`str.length - 3`*).
>
> ```
> const nome = "Ricardo";
> console.log(nome.slice(-3)); // retorna 'rdo'
> ```
>
> Se *`startIndex`* for maior ou igual a `str.length`, uma string vazia será retornada.
>
> ```
> const nome = "Ricardo";
> console.log(nome.slice(7)); // retorna <empty string>'
> ```

`endIndex`

> Opcional. É o índice, de base zero, onde termina a extração. O caractere que possuir este índice não será incluso.
>
> Se omitido ou possuir valor de *`undefined`*, ou for maior que `str.length`, *`slice()`* extrairá até o fim da string.
>
> ```
> const nome = "Ricardo";
>
> // omitindo fimSlice
> ```

```
console.log(nome.slice(0)); // retorna 'Ricardo'

// undefined fimSlice
let i; // variável criada sem atribuir valor
console.log(nome.slice(0, i)); // retorna 'Ricardo'

// passando número maior que o tamanho da string
console.log(nome.slice(0, 7)); // retorna 'Ricardo'
```

Se negativo, é o mesmo que executar *`str.length + endIndex`* onde *`str.length`* é o tamanho da string (por exemplo, se *`endIndex`* é *`-3`*, é como executar `str.length - 3`).

```
const nome = "Ricardo";
console.log(nome.slice(0, -3)); // retorna 'Rica'
```

Se for um valor diferente de indefinido e, ao mesmo tempo não for um número, uma string vazia será retornada

```
const nome = "Ricardo";
let i; // variável criada sem valor atribuído

// passando algum valor ≠ de undefined e ≠ de número
console.log(nome.slice(0, NaN)); // retorna <empty string>
console.log(nome.slice(2, false)); // retorna <empty string>
console.log(nome.slice(5, i)); // retorna 'Ricardo'
```

Se `endIndex` for definido e `startIndex` for negativo, `endIndex` deve ser negativo também, do contrário uma string vazia é retornada.

```
const nome = "Ricardo";
console.log(nome.slice(-3, 0)); // retorna <empty string>
```

Caso `endIndex` seja definido e `startIndex` e `endIndex` sejam ambos positivos ou negativos, `endIndex` deve ser maior que `startIndex`, do contrário uma string vazia é retornada.

```
const nome = "Ricado";
console.log(nome.slice(-1, -3)); // retorna <empty string>
console.log(nome.slice(3, 1)); // retorna <empty string>
```

Retorno

Uma nova string contento a porção extraída da string.

Descrição

`slice()` extrai um texto de uma string e retorna uma nova string. Modificações realizadas no texto de uma string não afetam a outra string.

`slice()` extrai até, mas não inclue *`endIndex`*.

`str.slice(1, 4)` extrai a partir do segundo caractere até o quarto caractere (ou seja, os caracteres de índices *`1`*, *`2`*, e *`3`*).

Por exemplo, *`str.slice(2, -1)`* extrai a partir do terceiro carácter até o penúltimo caractere da string.

```
const nome = "Ricardo";
console.log(nome.slice(2, -1)); // retorna 'card'
```

Exemplos

Exemplo: Usando <code>slice()</code> para criar uma nova string

O exemplo a seguir usa *`slice()`* para criar uma nova string.

```
var str1 = "A manhã está sobre nós", // o tamanho de str1 é 22
  str2 = str1.slice(3, 10),
  str3 = str1.slice(2, -2),
  str4 = str1.slice(13),
  str5 = str1.slice(22);
console.log(str2); // retorna 'anhã es'
console.log(str3); // retorna 'manhã está sobre n'
console.log(str4); // retorna 'sobre nós'
console.log(str5); // retorna <empty string>
```

Exemplo: Usando <code>slice()</code> com índices negativos

O exemplo a seguir usa o *`slice()`* com índices negativos.

```
var str = "A manhã está sobre nós";
str.slice(-3); // retorna 'nós'
str.slice(-3, -1); // retorna 'nó'
str.slice(0, -1); // retorna 'A manhã está sobre nó'
```

O exemplo abaixo percorre o índice no sentido anti-horário (de trás para frente) até chegar ao índice 11 da string, que será o início. Logo após, percorre o índice da string no sentido horário até chegar ao índice 16 da string, que será o fim.

```
console.log(str.slice(-11, 16)); // retorna "á sob"
```

O exemplo abaixo percorre o índice no sentido horário até chegar ao índice 10 da string, que será o início. Logo após, percorre o índice da string no sentido anti-horário até chegar ao índice 7 da string, que será o fim.

```
console.log(str.slice(10, -7)); // retorna "tá so"
```

O exemplo abaixo percorre o índice no sentido anti-horário até chegar ao índice 5 da string, que será o início. Logo após, percorre o índice da string novamente no sentido anti-horário até chegar ao índice 1 da string, que será o fim.

```
console.log(str.slice(-5, -1)); // retorna "e nó"
```

Veja também

- `String.prototype.substr()`
- `String.prototype.substring()`
- `Array.prototype.slice()`

String.prototype.small()

O método `small()` cria um elemento HTML `<small>` que faz com que uma string seja exibida em uma fonte pequena.

Sintaxe

```
str.small()
```

Valor retornado

Uma string contendo um elemento HTML `<small>`.

Descrição

O método `small()` cria uma string dentro de uma tag `<small>`: `"<small>str</small>"`.

Exemplos

Usando <code>small()</code>

O exemplo a seguir usa métodos do objeto global String para alterar o tamanho da string:

```
var worldString = "Olá, mundo";

console.log(worldString.small()); // <small>Olá, mundo</small>
console.log(worldString.big()); // <big>Olá, mundo</big>
console.log(worldString.fontsize(7)); // <font size="7">Olá, mundo</fontsize>
```

Com o objeto element.style você pode pegar o atributo de estilo do elemento e manipulá-lo de forma mais genérica, por exemplo:

```
document.getElementById("#oIdDoElemento").style.fontSize = "0.7em";
```

Veja também

- `String.prototype.fontsize()`
- `String.prototype.big()`

String.prototype.split()

O método `split()` divide uma `String` em uma lista ordenada de substrings, coloca essas substrings em um array e retorna o array. A divisão é feita procurando um padrão, onde o padrão é fornecido como o primeiro parâmetro na chamada do método.

Experimente

```
const str = "The quick brown fox jumps over the lazy dog.";

const words = str.split(" ");
console.log(words[3]);
// Expected output: "fox"

const chars = str.split("");
console.log(chars[8]);
// Expected output: "k"

const strCopy = str.split();
console.log(strCopy);
// Expected output: Array ["The quick brown fox jumps over the lazy dog."]
```

Sintaxe

```
str.split([separator[, limit]])
```

Parâmetros

- O caso mais simples é quando o separador é apenas um único caractere que será usado para dividir a string. Por exemplo, uma string contendo valores separados por tabulação (TSV) poderia ser analisada passando um caractere de tabulação como separador, como este: myString.split ("\ t").
- Se o `separator` contiver vários caracteres, toda a sequência de caracteres deve ser encontrada para ser dividida.
- Se o `separator` for omitido ou não aparecer dentro da `str`, o array retornará um elemento consistindo em toda a string.
- Se o `separator` aparecer no início (ou no final) da string, ele ainda terá o efeito de divisão. O resultado é uma string vazia (ou seja, `str.length == 0`), que aparece na primeira (ou última) posição do array retornado.
- Se o `separator` for uma string vazia (""), `str` será convertido em um array contendo cada um de seus caracteres em UTF-16.

`separator`

Opcional. Especifica o caractere, ou conjunto de caracteres, a ser usado para separar a string. O separador pode ser uma string ou uma expressão regular.

> **Aviso:** Quando uma string vazia (`""`) é usada como separador, a string não é dividida por caracteres percebidos pelo usuário (grapheme clusters) ou caracteres Unicode (pontos de código), mas por unidades de código UTF-16. Isso destrói pares substitutos. Consulte "Como você transforma uma string em um array de caracteres em JavaScript?" no StackOverflow.

`limite`

Opcional. Um número inteiro não negativo especificando um limite no número de divisões a serem encontradas. O método `split()` ainda dividirá em cada ocorrência do separador, até que o número de itens divididos corresponda ao limite ou que a string fique aquém do separador.

- O array pode conter menos entradas do que o `limit` se o final da string for alcançado antes que o limite seja atingido.
- Se o `limit` for `0`, `[]` será retornado.

Valor retornado

Um array de strings divididos em cada ponto onde o separador ocorre na string informada.

Descrição

Quando encontrado, o caractere designado como o `separator` é removido da string e as substrings são retornadas em um array. Se o `separator` não for encontrado ou for omitido, o array irá conter um elemento consistindo da string inteira. Se o `separator` for uma string vazia, `str` será convertida em um array de caracteres.

Se o separador for uma expressão regular que contenha parênteses de captura, cada vez que o `separator` for encontrado, os resultados (incluindo qualquer resultado `undefined`) dos parênteses de captura serão emendados ao array de saída. Entretanto, nem todos os navegadores possuem suporte a isso.

Exemplos

Usando <code>split()</code>

Quando a string está vazia, o `split()` irá retornar um array contendo uma string vazia ao invés de um array vazio. Se a string e o separador forem ambos strings vazias, um array vazio será retornado.

```
const myString = "";
const splits = myString.split();

console.log(splits);

// retorna [""]
```

O exemplo a seguir define uma função que divide uma string em um array de strings usando o separador especificado. Depois de dividir a string, a função exibe mensagens indicando a string original (antes da divisão), o separador usado, o número de elementos no array e os elementos individuais do array.

```
function splitString(stringToSplit, separator) {
  var arrayOfStrings = stringToSplit.split(separator);

  console.log('A string original é: "' + stringToSplit + '"');
  console.log('O separador é: "' + separator + '"');
  console.log(
    "O array tem " +
      arrayOfStrings.length +
      " elementos: " +
      arrayOfStrings.join(" / "),
  );
}

var tempestString = "Oh brave new world that has such people in it.";
var monthString = "Jan,Fev,Mar,Abr,Mai,Jun,Jul,Ago,Set,Out,Nov,Dez";

var space = " ";
var comma = ",";

splitString(tempestString, space);
splitString(tempestString);
splitString(monthString, comma);
```

Esse exemplo produz a saída a seguir:

```
A string original é: "Oh brave new world that has such people in it."
O separador é: " "
O array possui 10 elementos: Oh / brave / new / world / that / has / such / people / in / it.

A string original é: "Oh brave new world that has such people in it."
O separador é: "undefined"
O array possui 1 elementos: Oh brave new world that has such people in it.

A string original é: "Jan,Feb,Mar,Apr,May,Jun,Jul,Aug,Sep,Oct,Nov,Dec"
O separador é: ","
O array possui 12 elementos: Jan / Feb / Mar / Apr / May / Jun / Jul / Aug / Sep / Oct / Nov / Dec
```

Removendo espaços de uma string

No exemplo a seguir, `split()` procura por 0 ou mais espaços seguidos por um ponto e vírgula seguido por 0 ou mais espaços e, quando encontrar, remove os espaços e os pontos e vírgulas da string. `nameList` é o array retornado como resultado do `split()`.

```
var names = "Harry Trump ;Fred Barney; Helen Rigby ; Bill Abel ;Chris Hand ";

console.log(names);

var re = /\s*;\s*/;
var nameList = names.split(re);

console.log(nameList);
```

O retorno do código acima são duas linhas. A primeira linha registra a string original e a segunda linha registra o array resultante.

```
Harry Trump ;Fred Barney; Helen Rigby ; Bill Abel ;Chris Hand
[ "Harry Trump", "Fred Barney", "Helen Rigby", "Bill Abel", "Chris Hand " ]
```

Retornando um número limitado de divisões

No exemplo a seguir, o `split()` procura por 0 ou mais espaços em uma string e retorna as 3 primeiras divisões que encontrar.

```
var myString = "Hello World. How are you doing?";
var splits = myString.split(" ", 3);

console.log(splits);
```

O script exibirá o texto a seguir:

```
["Hello", "World.", "How"]
```

Parênteses de Captura

Se o `separator` contém parênteses de captura, os resultados correspondentes são retornados no array.

```
var myString = "Hello 1 word. Sentence number 2.";
var splits = myString.split(/(\d)/);

console.log(splits);
```

O script exibirá o texto a seguir:

```
[ "Hello ", "1", " word. Sentence number ", "2", "." ]
```

> **Nota:** `\d` corresponde à classe de caracteres para dígitos entre 0 e 9.

Revertendo uma String usando <code>split()</code>

> **Aviso:** Esta não é a melhor maneira de reverter uma string:
>
> ```
> const str = "asdfghjkl";
> const strReverse = str.split("").reverse().join("");
> // 'lkjhgfdsa'
>
> // split() retorna um array onde os métodos
> // reverse() e join() podem ser aplicados
> ```
>
> Não funciona se a string contém `grapheme clusters`, mesmo ao usar uma divisão compatível com Unicode. (Use, por exemplo, esrever no lugar.)
>
> ```
> const str = "résumé";
> const strReverse = str.split(/(?:)/u).reverse().join("");
> // retorna "émuśer"
> ```
>
> **Bonus:** use o operador `===` para testar se a string original era um palíndromo.

Veja também

- `String.prototype.charAt()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`
- `Array.prototype.join()`

String.prototype.startsWith()

O método **`startsWith()`** determina se uma string começa com os caracteres especificados, retornando `true` ou `false`.

Experimente

```
const str1 = "Saturday night plans";

console.log(str1.startsWith("Sat"));
// Expected output: true

console.log(str1.startsWith("Sat", 3));
// Expected output: false
```

Sintaxe

```
str.startsWith(searchString[, position])
```

Parâmetros

`searchString`

Os caracteres a serem procurados a partir do início dessa string.

`position`

Opcional. A posição nessa string na qual se inicia a busca pela *`searchString`*. O valor padrão é `0`.

Valor retornado

`true` se os caracteres fornecidos forem encontrados no início da string. Se não, **`false`**.

Descrição

Esse método permite determinar se uma string começa ou não com outra string. Esse método é case-sensitive (difere maiúsculas de minúsculas, e vice-versa).

Exemplos

Usando <code>startsWith()</code>

```
//startswith
let str = "Ser ou não ser, eis a questão.";

console.log(str.startsWith("Ser")); // true
console.log(str.startsWith("não ser")); // false
console.log(str.startsWith("não ser", 7)); // true
```

Polyfill

Este método foi adicionaldo à especificação ECMAScript 2015 e pode ainda não estar disponível em todas as implementações do JavaScript. No entanto, você pode usar o polyfill `String.prototype.startsWith()` adicionando o seguinte código:

```
if (!String.prototype.startsWith) {
    Object.defineProperty(String.prototype, 'startsWith', {
        value: function(search, rawPos) {
            var pos = rawPos > 0 ? rawPos|0 : 0;
            return this.substring(pos, pos + search.length) === search;
        }
    });
}
```

Um polyfill mais robusto (totalmente conforme com a especificação ES2015), mas com menor desempenho e compacto está disponível no GitHub por Mathias Bynens.

Veja também

- `String.prototype.endsWith()`
- `String.prototype.includes()`
- `String.prototype.indexOf()`
- `String.prototype.lastIndexOf()`

String.prototype.strike()

O método `strike()` cria um elemento HTML `<strike>` que faz com que uma string seja exibida com o texto riscado.

Sintaxe

```
str.strike()
```

Valor retornado

Uma string contendo um elemento HTML `<strike>`.

Descrição

O método `strike()` cria uma string dentro uma tag `<strike>`: "`<strike>str</strike>`".

Exemplos

Usando <code>strike()</code>

O exemplo a seguir usa métodos do objeto global String para alterar a formatação da string:

```
var worldString = "Olá, mundo";

console.log(worldString.blink()); // <blink>Olá, mundo</blink>
console.log(worldString.bold()); // <b>Olá, mundo</b>
console.log(worldString.italics()); // <i>Olá, mundo</i>
console.log(worldString.strike()); // <strike>Olá, mundo</strike>
```

Veja também

- `String.prototype.blink()`
- `String.prototype.bold()`
- `String.prototype.italics()`

String.prototype.sub()

O método `sub()` cria um elemento HTML `<sub>` que faz com que uma string seja exibida como subscrito (texto pequeno).

Sintaxe

```
str.sub()
```

Valor retornado

Uma string contendo um elemento HTML `<sub>`.

Descrição

O método `sub()` cria uma string dentro de uma tag `<sub>`: "`<sub>str</sub>`".

Exemplos

Usando os métodos <code>sub()</code> e <code>sup()</code>

Os exemplos seguintes usam o métodos `sub()` e `sup()` para formatar uma string:

```
var superText = "superscript";
var subText = "subscript";

console.log("This is what a " + superText.sup() + " looks like.");
// This is what a <sup>superscript</sup> looks like

console.log("This is what a " + subText.sub() + " looks like.");
// This is what a <sub>subscript</sub> looks like.
```

Veja também

- `String.prototype.sup()`

String.prototype.substr()

O método `substr()` retorna uma parte da string, começando no índice especificado e estendendo-se por um determinado número de caracteres posteriormente.

Experimente

```
const str = "Mozilla";

console.log(str.substr(1, 2));
// Expected output: "oz"

console.log(str.substr(2));
// Expected output: "zilla"
```

Sintaxe

```
str.substr(start[, length])
```

Parâmetros

`start`

Local para começar a extrair os caracteres.

`length`

Opcional. O número de caracteres a serem extraídos.

Valor de retorno

Uma nova string contendo a seção extraída da string fornecida.

Descrição

O `substr()` extrai caracteres de comprimento de uma `str`, contando a partir do índice inicial.

- Se o `start` for um número positivo, o índice começa a contar no início da string. Seu valor é limitado ao tamanho da string (`str.length`).
- Se o `start` for um número negativo, o índice começa a contar a partir do final da string. Seu valor é limitado ao tamanho da string (`-str.length`).

> **Nota:** No Microsoft JScript, valores negativos no argumento `start` não são considerados como referência ao final da string.

- Se `length` for omitido, `substr()` extrairá caracteres até o final da string.
- Se `length` for `undefined`, `substr()` extrai os caracteres até o final da string.
- Se `length` for um número negativo, ele será tratado como `0`.
- Para `start` e `length`, `NaN` é tratado como 0.

Exemplos

Usando <code>substr()</code>

```
var aString = "Mozilla";

console.log(aString.substr(0, 1)); // 'M'
console.log(aString.substr(1, 0)); // ''
console.log(aString.substr(-1, 1)); // 'a'
console.log(aString.substr(1, -1)); // ''
console.log(aString.substr(-3)); // 'lla'
console.log(aString.substr(1)); // 'ozilla'
console.log(aString.substr(-20, 2)); // 'Mo'
console.log(aString.substr(20, 2)); // ''
```

Polyfill

JScript da Microsoft não suporta valores negativos para o índice de `start`. Se você deseja usar esse recurso, você pode usar o seguinte código de compatibilidade para evitar esse erro:

```
// only run when the substr() function is broken
if ("ab".substr(-1) != "b") {
  /**
   *  Get the substring of a string
   *  @param  {integer}  start   where to start the substring
   *  @param  {integer}  length  how many characters to return
   *  @return {string}
   */
  String.prototype.substr = (function (substr) {
    return function (start, length) {
      // call the original method
      return substr.call(
        this,
        // did we get a negative start, calculate how much it is from the beginning of the string
        // adjust the start parameter for negative value
        start < 0 ? this.length + start : start,
        length,
      );
    };
  })(String.prototype.substr);
}
```

Veja também

- `String.prototype.slice()`
- `String.prototype.substring()`

String.prototype.substring()

Resumo

O método `substring()` retorna a parte da string entre os índices inicial e final, ou até o final da string.

Sintaxe

```
str.substring(indexStart[, indexEnd])
```

Parâmetros

`indexStart`

Um inteiro entre `0` e o comprimento da string (`str.length`), especificando a posição na string do primeiro caractere a ser incluído na substring retornada.

`indexEnd`

Opcional. Um inteiro entre `0` e o comprimento da string (`str.length`), especificando a posição na string do primeiro caractere a **não** ser mais incluído na substring retornada.

Valor retornado

Uma nova string contendo a parte especificada da string fornecida.

Descrição

`substring()` extrai caracteres desde `indexStart` até, mas **não incluindo**, `indexEnd`. Em particular:

- Se `indexStart` é igual a `indexEnd`, `substring()` retorna uma string vazia.
- Se `indexEnd` for omitido, `substring()` extrai caracteres até o fim da string.
- Se qualquer argumento for menor que `0` ou `NaN`, ele será tratado como `0`.
- Se qualquer argumento for maior que `stringName.length`, ele será tratado como se fosse `stringName.length`

Se `indexStart` for maior que `indexEnd`, então o efeito do `substring()` é como se os dois argumentos estivessem trocados, por exemplo, `str.substring(1, 0) == str.substring(0, 1)`.

Exemplos

Usando <code>substring()</code>

O seguinte exemplo usa `substring()` para mostrar caracteres da palavra 'Mozilla':

```
var anyString = "Mozilla";

// Mostra "Moz"
console.log(anyString.substring(0, 3));
console.log(anyString.substring(3, 0));

// Mostra "lla"
console.log(anyString.substring(4, 7));
console.log(anyString.substring(7, 4));

// Mostra "Mozill"
console.log(anyString.substring(0, 6));

// Mostra "Mozilla"
console.log(anyString.substring(0, 7));
console.log(anyString.substring(0, 10));
```

Usando <code>substring()</code> com <code>length</code>

O exemplo a seguir usa o método `substring()` e a propriedade `length` para extrair os últimos caracteres de uma string específica. Este método pode ser mais fácil de lembrar, visto que você não precisa saber os índices inicial e final como faria nos exemplos acima.

```
// Mostra 'illa', os últimos 4 caracteres
let anyString = "Mozilla";
let anyString4 = anyString.substring(anyString.length - 4);
console.log(anyString4);

// Mostra 'zilla', os últimos 5 caracteres
let anyString = "Mozilla";
let anyString5 = anyString.substring(anyString.length - 5);
console.log(anyString5);
```

A diferença entre <code>substring()</code> e <code>substr()</code>

Há uma diferença sutil entre os métodos `substring()` e `substr()`, então você deve ter cuidado para não confundi-los.

Os argumentos de `substring()` representam os índices inicial e final, enquanto os argumentos de `substr()` representam o índice inicial e o número de caracteres a serem incluídos na string retornada.

Além disso, `substr()` é considerado um **recurso legacy no ECMAScript** e pode ser removido em versões futuras, portanto, é melhor evitar usá-lo, se possível.

```
let text = "Mozilla";
console.log(text.substring(2, 5)); // retorna "zil"
console.log(text.substr(2, 3)); // retorna "zil"
```

Diferenças entre <code>substring()</code> e <code>slice()</code>

Os métodos `substring()` e `slice()` são quase idênticos, mas existem algumas diferenças sutis entre os dois, especialmente na forma como os argumentos negativos são tratados.

O método `substring()` troca seus dois argumentos se `indexStart` for maior que `indexEnd`, o que significa que uma string ainda será retornada. O método `slice()` retorna uma string vazia caso o mesmo ocorra.

```
let text = "Mozilla";
console.log(text.substring(5, 2)); // retorna "zil"
console.log(text.slice(5, 2)); // retorna ""
```

Se um ou ambos os argumentos forem negativos ou `NaN`, o método `substring()` os tratará como se fossem `0`.

```
console.log(text.substring(-5, 2)); // retorna "Mo"
console.log(text.substring(-5, -2)); // retorna ""
```

`slice()` também trata os argumentos `NaN` como `0`, mas quando recebe valores negativos, ele conta regressivamente a partir do final da string para encontrar os índices.

```
console.log(text.slice(-5, 2)); // retorna ""
console.log(text.slice(-5, -2)); // retorna "zil"
```

Veja a página `slice()` para mais exemplos com números negativos.

Substituindo uma <code>substring()</code> com uma string

O seguinte exemplo substitui uma substring dentro de uma string. Ela irá substituir ambos caracteres e substrings individualmente. A função invocada na linha final do exemplo altera a string `"Brave New World"` para `"Brave New Web"`.

```
function replaceString(oldS, newS, fullS) {
  // Substitui oldS por newS na string fullS
  for (var i = 0; i < fullS.length; i++) {
    if (fullS.substring(i, i + oldS.length) == oldS) {
      fullS =
        fullS.substring(0, i) +
        newS +
        fullS.substring(i + oldS.length, fullS.length);
    }
  }
  return fullS;
}

replaceString("World", "Web", "Brave New World");
```

Note que isto pode resultar em um loop infinito se `oldS` for um substring de `newS` — por exemplo, se você tentou substituir `"World"` com `"OtherWorld"`. O melhor método para substituir strings é o seguinte:

```
function replaceString(oldS, newS, fullS) {
  return fullS.split(oldS).join(newS);
}
```

O código acima serve como um exemplo para operações com substring. Se você precisa substituir substrings, na maioria das vezes você vai querer usar `String.prototype.replace()`.

Veja também

- `String.prototype.substr()`
- `String.prototype.slice()`

String.prototype.sup()

O método `sup()` cria um elemento HTML `<sup>` que faz com que uma string seja exibida como sobrescrito.

Sintaxe

```
str.sup()
```

Valor retornado

Uma string contendo um elemento HTML `<sup>`.

Descrição

O método `sup()` cria uma string dentro de uma tag `<sup>`: `"<sup>str</sup>"`

Exemplos

Usando os métodos <code>sub()</code> e <code>sup()</code>

O exemplo a seguir usa os métodos `sub()` e `sup()` para formatar uma string:

```
var superText = "superscript";
var supText = "supscript";

console.log("This is what a " + superText.sup() + " looks like.");
// "This is what a <sup>superscript</sup> looks like."

console.log("This is what a " + supText.sup() + " looks like.");
// "This is what a <sup>supscript</sup> looks like."
```

Veja também

- `String.prototype.sup()`

String.prototype[@@iterator]()

O método **`[@@iterator]()`** retorna um novo objeto `Iterator` que itera sobre os pontos de código do valor da String, retornando cada ponto de código como um valor String.

Sintaxe

```
str[Symbol.iterator]
```

Valor de Retorno

Novo objeto `Iterator`.

Exemplos

Usando <code>[@@iterator]()</code>

```
var str = "A\uD835\uDC68";

var strIter = str[Symbol.iterator]();

console.log(strIter.next().value); // "A"
console.log(strIter.next().value); // "\uD835\uDC68"
```

Usando <code>[@@iterator]()</code> com <code>for..of</code>

```
var str = "A\uD835\uDC68B\uD835\uDC69C\uD835\uDC6A";

for (var v of str) {
  console.log(v);
}
// "A"
// "\uD835\uDC68"
// "B"
// "\uD835\uDC69"
// "C"
// "\uD835\uDC6A"
```

Veja também

- Iteration protocols

String.prototype.toLocaleLowerCase()

O método `toLocaleLowerCase()` retorna o valor da string em minúsculas, de acordo com qualquer mapeamento de caixa de texto específico da localidade.

Experimente

```
const dotted = "İstanbul";

console.log(`EN-US: ${dotted.toLocaleLowerCase("en-US")}`);
// Expected output: "istanbul"

console.log(`TR: ${dotted.toLocaleLowerCase("tr")}`);
// Expected output: "istanbul"
```

Sintaxe

```
str.toLocaleLowerCase()
str.toLocaleLowerCase(locale)
str.toLocaleLowerCase([locale, locale, ...])
```

Parâmetros

`locale`

> Opcional. O parâmetro `locale` indica o local a ser usado para converter para minúsculas de acordo com qualquer mapeamento de caixa de texto específico da localidade. Se várias localidades forem fornecidas em um `Array`, a melhor localidade disponível é usada. A localidade padrão é a localidade atual do ambiente do host.

Valor retornado

Uma nova string que representa a string original convertida em minúsculas, de acordo com qualquer mapeamento da caixa de texto específico da localidade.

Exceções

- Um `RangeError` (" invalid language tag: xx_yy ") é lançado se um argumento `locale` não for uma tag de idioma válido.
- Um `TypeError` (" invalid element in locales argument ") é lançado se um elemento do array não for do tipo string.

Descrição

O método `toLocaleLowerCase()` retorna o valor da string convertida em minúsculas de acordo com qualquer mapeamento da caixa de texto específico da localidade. `toLocaleLowerCase()` não afeta o valor da string original. Na maioria dos casos, ele produzirá o mesmo resultado que `toLowerCase()`, mas para alguns locais, como turco, cujos mapeamentos da caixa de texto não seguem o mapeamento padrão em Unicode, pode haver um resultado diferente.

Exemplos

Usando <code>toLocaleLowerCase()</code>

```
"ALFABETO".toLocaleLowerCase(); // 'alfabeto'

"\u0130".toLocaleLowerCase("tr") === "i"; // true
"\u0130".toLocaleLowerCase("en-US") === "i"; // false

let locales = ["tr", "TR", "tr-TR", "tr-u-co-search", "tr-x-turkish"];
"\u0130".toLocaleLowerCase(locales) === "i"; // true
```

Veja também

- `String.prototype.toLocaleUpperCase()`
- `String.prototype.toLowerCase()`
- `String.prototype.toUpperCase()`

String.prototype.toLocaleUpperCase()

O método `toLocaleUpperCase()` retorna o valor da string em maiúsculas, de acordo com qualquer mapeamento de caixa de texto específico da localidade.

Experimente

```
const city = "istanbul";

console.log(city.toLocaleUpperCase("en-US"));
// Expected output: "ISTANBUL"

console.log(city.toLocaleUpperCase("TR"));
// Expected output: "İSTANBUL"
```

Sintaxe

```
str.toLocaleUpperCase()
str.toLocaleUpperCase(locale)
str.toLocaleUpperCase([locale, locale, ...])
```

Parâmetros

`locale`

> Opcional. O parâmetro `locale` indica a localização a ser usada para converter para maiúsculas de acordo com qualquer mapeamento de caixa de texto específico da localidade. Se vários locais forem fornecidos em um `Array`, a melhor localidade disponível é usada. A localidade padrão é a localidade atual do ambiente do host.

Valor retornado

Uma nova string que representa a string original convertida em maiúsculas, de acordo com qualquer mapeamento de caixa de texto específico da localidade.

Exceções

- Um `RangeError` ("invalid language tag: xx_yy") é lançado se um argumento `locale` não for uma tag de idioma válido.
- Um `TypeError` ("invalid element in locales argument") é lançado se um elemento do array não for do tipo string.

Descrição

O método `toLocaleUpperCase()` retorna o valor da string convertida em maiúsculas de acordo com qualquer mapeamento de caixa de texto específico da localidade. `toLocaleUpperCase()` não afeta o valor da string em si. Na maioria dos casos, ele produzirá o mesmo resultado que `toUpperCase()`, mas para alguns locais, como turco, cujos mapeamentos de caixa de texto não seguem o mapeamento de caixa de texto padrão em Unicode, pode haver um resultado diferente.

Observe também que a conversão não é necessariamente um mapeamento de caracteres 1:1, pois alguns caracteres podem resultar em dois (ou até mais) caracteres quando transformados em maiúsculas. Portanto, o comprimento da string resultante pode ser diferente do comprimento da string de entrada. Isso também implica que a conversão não é estável, então, por exemplo, o seguinte pode retornar `false`: `x.toLocaleLowerCase() === x.toLocaleUpperCase(). toLocaleLowerCase()`

Exemplos

Usando <code>toLocaleUpperCase()</code>

```
"alfabeto".toLocaleUpperCase(); // 'ALFABETO'

"Gesäß".toLocaleUpperCase(); // 'GESÄSS'

"i\u0307".toLocaleUpperCase("lt-LT"); // 'I'

let locales = ["lt", "LT", "lt-LT", "lt-u-co-phonebk", "lt-x-lietuva"];
"i\u0307".toLocaleUpperCase(locales); // 'I'
```

Veja também

- `String.prototype.toLocaleLowerCase()`
- `String.prototype.toLowerCase()`
- `String.prototype.toUpperCase()`

String.prototype.toLowerCase()

O método **`toLowerCase()`** retorna o valor da string que foi chamada convertido para minúsculo.

Sintaxe

```
str.toLowerCase()
```

Valor retornado

Uma nova string contendo o valor da string original convertido para minúsculo.

Descrição

O método `toLowerCase()` retorna o valor da string original convertido para minúsculo. `toLowerCase()` não altera o valor da string original.

Exemplos

Usando <code>toLowerCase()</code>

```
console.log("ALFABETO".toLowerCase()); // 'alfabeto'
```

Veja também

- `String.prototype.toLocaleLowerCase()`
- `String.prototype.toLocaleUpperCase()`
- `String.prototype.toUpperCase()`

String.prototype.toString()

O método **`toString()`** retorna uma string representando o objeto especificado.

Sintaxe

```
str.toString()
```

Descrição

O objeto `String` substitui o método `toString()` do objeto `Object`. Ele não herda `Object.prototype.toString()`. Para objetos `String`, o método `toString()` retorna uma representação de string do objeto e é o mesmo que o método `String.prototype.valueOf()`.

Exemplos

Usando <code>toString()</code>

O exemplo a seguir exibe o valor string de um objeto `String`:

```
var x = new String("Hello world");

console.log(x.toString()); // retorna 'Hello world'
```

Veja também

- `Object.prototype.toSource()`
- `String.prototype.valueOf()`

String.prototype.toUpperCase()

O método `toUpperCase()` retorna o valor da string original convertido em letras maiúsculas.

Sintaxe

```
str.toUpperCase()
```

Valor retornado

Uma nova string representando a string original convertida em maiúsculas.

Exceções

`TypeError`

Quando chamado em uma string contendo valor `null` ou `undefined`, por exemplo, `String.prototype.toUpperCase.call(undefined)`.

Descrição

O método `toUpperCase()` retorna o valor da string convertido para letras maiúsculas. `toUpperCase()` não altera o valor da string original.

Exemplos

Uso básico

```
console.log("alfabeto".toUpperCase()); // 'ALFABETO'
```

Veja também

- `String.prototype.toLocaleLowerCase()`
- `String.prototype.toLocaleUpperCase()`
- `String.prototype.toLowerCase()`

String.prototype.trim()

O método **`trim()`** remove os espaços em branco (whitespaces) do início e/ou fim de um texto. É considerado espaço em branco (espaço, tabulação, espaço fixo/rígido, etc.) e todo sinal de fim de linha de texto (LF, CR, etc.).

Sintaxe

```
str.trim()
```

Descrição

O método `trim()` retorna o texto sem espaços em branco no início e/ou fim da string. O `trim()` não altera o valor da string original.

Experimente

```
const greeting = "   Hello world!   ";

console.log(greeting);
// Expected output: "   Hello world!   ";

console.log(greeting.trim());
// Expected output: "Hello world!";
```

Exemplos

Usando <code>trim()</code>

Os exemplos a seguir retornam o valor de `'foo'` sem espaços em branco:

```
//.trim() removendo whitespace de ambos os lados

var str = "   foo  ";
console.log(str.trim()); // retorna 'foo'

// Outro exemplo de .trim() removendo whitespace de
// apenas um lado.

var str = "foo    ";
console.log(str.trim()); // retorna 'foo'
```

Polyfill

Executar o seguinte código antes antes de qualquer código irá criar o método `trim()` se o mesmo não estiver disponível nativamente.

```
if (!String.prototype.trim) {
  String.prototype.trim = function () {
    return this.replace(/^[\s\uFEFF\xA0]+|[\s\uFEFF\xA0]+$/g, "");
  };
}
```

Veja também

- `String.prototype.trimLeft()`
- `String.prototype.trimRight()`

String.prototype.trimEnd()

O método `trimEnd()` remove espaços do final de uma *string*. `trimRight()` é um apelido para este método.

Experimente

```
const greeting = "   Hello world!   ";

console.log(greeting);
// Expected output: "   Hello world!   ";

console.log(greeting.trimEnd());
// Expected output: "   Hello world!";
```

Sintaxe

```
str.trimEnd();
str.trimRight();
```

Valor retornado

Uma nova *string* representando a *string* original sem espaços ao seu final (direita).

Descrição

Os métodos `trimEnd()` / `trimRight()` retornam a *string* sem os espaços à direita dela. `trimEnd()` ou `trimRight()` não altera o valor da *string* original.

Aliasing

Para consistência com funções como `String.prototype.padEnd` o nome padrão do método é `trimEnd`. Entretanto, por razões de compatibilidade na *web*, `trimRight` permanece como um apelido para `trimEnd`. Em alguns motores isso significa:

```
String.prototype.trimRight.name === "trimEnd";
```

Exemplos

Usando <code>trimEnd()</code>

O exemplo a seguir mostra a *string* em caixa baixa `'  foo'`:

```
var str = "   foo  ";

console.log(str.length); // retorna 8

str = str.trimEnd();
console.log(str.length); // retorna 6
console.log(str); // retorna '   foo'
```

Veja também

- `String.prototype.trim()`
- `String.prototype.trimStart()`

String.prototype.trimStart()

O método `trimStart()` remove espaços do começo de uma *string*. `trimLeft()` é um apelido para este método.

Experimente

```
const greeting = "   Hello world!   ";

console.log(greeting);
// Expected output: "   Hello world!   ";

console.log(greeting.trimStart());
// Expected output: "Hello world!   ";
```

Sintaxe

```
str.trimStart();
str.trimLeft();
```

Valor retornado

Uma nova *string* representando a *string* original sem os espaços no começo (fim à esquerda).

Descrição

Os métodos `trimStart()` / `trimLeft()` retornam a *string* sem os espaços no fim à esquerda. `trimLeft()` ou `trimStart()` não altera o valor da *string* original.

Aliasing

Para consistência com funções como `String.prototype.padStart` o nome padrão do método é `trimStart`. Entretanto, por razões de compatibilidade na *web*, `trimLeft` permanece como um apelido para `trimStart`. Em alguns motores isso significa:

```
String.prototype.trimLeft.name === "trimStart";
```

Polyfill

```
//https://github.com/FabioVergani/js-Polyfill_String-trimStart

(function (w) {
  var String = w.String,
    Proto = String.prototype;

  (function (o, p) {
    if (p in o ? (o[p] ? false : true) : true) {
      var r = /^\s+/;
      o[p] =
        o.trimLeft ||
        function () {
          return this.replace(r, "");
        };
    }
  })(Proto, "trimStart");
})(window);

/*
ES6:
(w=>{
    const String=w.String, Proto=String.prototype;

    ((o,p)=>{
        if(p in o?o[p]?false:true:true){
            const r=/^\s+/;
            o[p]=o.trimLeft||function(){
                return this.replace(r,'')
            }
        }
    })(Proto,'trimStart');

})(window);
*/
```

Exemplos

Usando <code>trimStart()</code>

O seguinte exemplo mostra uma *string* em caixa baixa `'foo '`:

```
var str = "   foo  ";

console.log(str.length); // retorna 8

str = str.trimStart();
console.log(str.length); // retorna 5
console.log(str); // retorna 'foo  '
```

Veja também

- `String.prototype.trim()`
- `String.prototype.trimEnd()`

String.prototype.valueOf()

O método `valueOf()` retorna o valor primitivo de um objeto `String`.

Experimente

```
const stringObj = new String("foo");

console.log(stringObj);
// Expected output: String { "foo" }

console.log(stringObj.valueOf());
// Expected output: "foo"
```

Sintaxe

```
str.valueOf()
```

Valor retornado

Uma string representando o valor primitivo de um objeto `String`.

Descrição

O método `valueOf()` do objeto `String` retorna o valor primitivo de um objeto `String` como um dado do tipo string. Esse valor é equivalente a `String.prototype.toString()`.

Esse método é normalmente chamado internamente pelo JavaScript e não fica explícito no código.

Exemplos

Usando <code>valueOf()</code>

```
var x = new String("Olá, mundo");
console.log(x.valueOf()); // retorna 'Olá, mundo'
```

Veja também

- `String.prototype.toString()`
- `Object.prototype.valueOf()`

Símbolo

Sumário

A função `Symbol()` retorna um valor do tipo **símbolo (symbol)**, tem propriedades estáticas que expõem vários membros dos objetos nativos, possuem métodos estáticos que expõem o registro de símbolos globais e se parecem com uma classe de objeto nativo, mas estão incompletos como construtor porque não suportam a sintaxe "`new Symbol()`".

Cada valor símbolo retornado de `Symbol()` é único. Um símbolo pode ser usado como o identificador para propriedades de objetos; esse é o único propósito do tipo de dado. Algumas explicações sobre propósito e uso podem ser encontradas no verbete do glossário para Symbol.

O tipo de dado **símbolo** é um tipo de dado primitivo.

Sintaxe

```
Symbol([descrição])
```

Parâmetros

`descrição` Optional

String opcional. Uma descrição de símbolo no qual pode ser usado para debugar, mas não para acessar o próprio símbolo.

Descrição

Para criar um novo símbolo primitivo, simplesmente escreva `Symbol()` com uma string opcional para sua descrição:

```
var sym1 = Symbol();
var sym2 = Symbol("foo");
var sym3 = Symbol("foo");
```

O código acima cria três símbolos novos. Note que a função ***Symbol("foo")*** não faz a string ***"foo"*** ser um símbolo. Ela cria um novo símbolo a cada vez que é chamada:

```
Symbol("foo") === Symbol("foo"); // false
```

A sintaxe a seguir com o operador `new` vai resultar em um `TypeError`:

```
var sym = new Symbol(); // TypeError
```

Isso evita que os autores criem um objeto empacotador explícito de `Symbol` em vez de um novo valor de símbolo. O que pode surpreender, pois, geralmente é possível criar objetos empacotadores explícitos em torno de tipos de dados primitivos (por exemplo, `new Boolean`, `new String` e `new Number`).

Se você realmente quiser criar um objeto empacotador de `Symbol`, você pode usar a função `Object()`:

```
var sym = Symbol("foo");
typeof sym; // "symbol"
var symObj = Object(sym);
typeof symObj; // "object"
```

Símbolos compartilhados no registro global de símbolo

A sintaxe acima usando a função `Symbol()` não criará um símbolo global que estará disponível em todo o seu código. Para criar símbolos disponíveis em vários arquivos em um escopo como se fosse global, use os métodos `Symbol.for()` e `Symbol.keyFor()` para definir e configurar símbolos no registro global de símbolo.

Encontrando propriedades de símbolos em objetos

O método `Object.getOwnPropertySymbols()` retorna um array de símbolos e permite que você encontre propriedades de símbolos em um determinado objeto. Observe que cada objeto é inicializado sem suas próprias propriedades de símbolo, de modo que este array estará vazio, a menos que você estabeleça propriedades de símbolo no objeto.

Propriedades

`Symbol.length`

Propriedade de tamanho cujo valor é 1.

`Symbol`

Representa o protótipo do `Symbol` construtor.

Símbolos conhecidos

Em adição para seus próprios símbolos, JavaScript possui alguns símbolos built-in que representa os comportamentos internos da linguagem que não foram revelados para os desenvolvedores no ECMAScript 5 e anterior. Esses símbolos podem ser acessados usando as seguintes propriedades:

Symbol.hasInstance

Especificado como @@hasInstance. Um método que determina se um construtor de um objeto é reconhecido como a instância de um objeto. Usado por `instanceof`.

Symbol.isConcatSpreadable

Especificado como @@isConcatSpreadable. Um valor Booleano indicando se um objeto deve ser adicionado como elemento de uma array. Usado por `Array.prototype.concat()`.

Symbol.isRegExp

Especificado como @@isRegExp. Um valor Booleano indicando se um objeto pode ser usado como uma expressão regular.

Symbol.iterator

Especificado como @@iterator. Um método retornando o iterador padrão para um objeto. Usado por `for...of`.

Symbol.toPrimitive

Especificado como @@toPrimitive. Um método convertendo um objeto para um valor primitivo.

Symbol.toStringTag

Especificado como @@toStringTag. Um valor string usado para descrição padrão de um objeto. Usado por `Object.prototype.toString()`

Symbol.unscopables

Especificado como @@unscopables. Uma Array com valores string que são valores propriedade. Esses são excluídos das ligações com o objeto associado.

Métodos

`Symbol.for(key)`

Procura por símbolos existentes com as chaves dada e retorna as chaves se forem encontradas. Caso contrário um novo símbolo será criado no registro de símbolo global com essa chave.

`Symbol.keyFor(sym)`

Retorna um símbolo compartilhado do registro global de símbolo para o símbolo dado.

<code>Symbol</code> protótipo

Todos os símbolos herdados de `Symbol`.

Propriedades

Métodos

Exemplos

Usando o operador <code>typeof</code> com símbolos

O operador `typeof` pode ajudar a identificar os símbolos.

```
typeof Symbol() === "symbol";
typeof Symbol("foo") === "symbol";
typeof Symbol.iterator === "symbol";
```

Conversões de tipos de símbolos

Algumas anotações quando trabalhando com conversão de tipo de símbolos.

- Quando estiver tentando converter um símbolo para um número, um `TypeError` será retornado. (e.g. `+sym` or `sym | 0`).
- Quando usando operador de igualdade, `Object(sym) == sym` retorna `true`.
- `Symbol("foo") + "bar"` lança um `TypeError` (não pode converter um símbolo para string). Isso previne você de silenciosamente criar um novo nome de propriedade string a partir de um símbolo, por exemplo.
- A "safer" `String(sym)` conversion funciona como uma chamada para `Symbol.prototype.toString()` com símbolos, mas note que uma `new String(sym)` será lançada.

Símbolos e <code>for...in</code> iteração

Símbolos não são visíveis em `for...in` iterações. Além de, `Object.getOwnPropertyNames()` não retornará propriedades do objeto símbolo, entretanto, você pode fazer uso do `Object.getOwnPropertySymbols()` para conseguir esses resultados.

```
var obj = {};

obj[Symbol("a")] = "a";
obj[Symbol.for("b")] = "b";
obj["c"] = "c";
obj.d = "d";

for (var i in obj) {
  console.log(i); // logs "c" and "d"
}
```

Símbolos e <code>JSON.stringify()</code>

Propriedade com chave de símbolo vão ser completamente ignoradas quando usando `JSON.stringify()`:

```
JSON.stringify({ [Symbol("foo")]: "foo" });
// '{}'
```

Para mais detalhes, veja `JSON.stringify()`.

Objeto wrapper de símbolo como chave de propriedade

Quando um objeto wrapper de um símbolo é usado como uma chave de propriedade, esse objeto será coerced para o seu símbolo wrapper:

```
var sym = Symbol("foo");
var obj = { [sym]: 1 };
obj[sym]; // 1
obj[Object(sym)]; // still 1
```

Veja também

- Glossary: Symbol data type
- `typeof`
- Data types and data structures

Symbol.asyncIterator

O **`Symbol.asyncIterator`** é um símbolo conhecido que especifica o padrão AsyncIterator para um objeto. Se essa propriedade é configurada em um objeto, é um iterável assíncrono e pode ser usado in um `for await...of` loop.

Descrição

O `Symbol.asyncIterator` é um símbolo built-in que é usado para um acessar o método `@@asyncIterator` de um objeto. Para que um objeto seja iterável assíncrono, ele deve ter uma chave `Symbol.asyncIterator`.

Property attributes of `Symbol.asyncIterator`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Iteráveis assíncronos definidos pelo usuário

Você consegue definir seu próprio iterável assíncrono configurando a propriedade `[Symbol.asyncIterator]` em um objeto.

```
const myAsyncIterable = {
  async *[Symbol.asyncIterator]() {
    yield "hello";
    yield "async";
    yield "iteration!";
  },
};

(async () => {
  for await (const x of myAsyncIterable) {
    console.log(x);
    // resultado esperado:
    //    "hello"
    //    "async"
    //    "iteration!"
  }
})();
```

Quando criando uma API, lembre que iteráveis assíncronos são programados para representar algum iterável — como um fluxo de dados ou uma lista —, não para substituir completamente chamadas e eventos na maioria das situações.

Iteráveis assíncronos

Não há atualmente objetos Javascript built-in que tenha a chave `[Symbol.asyncIterator]` configurada por padrão. Entretanto, a WHATWG Streams estão configurando para que o primeiro objeto built-in seja um iterável assíncrono, com a recente chegada do `[Symbol.asyncIterator]` nas especificações.

Vejá também

- Iteration protocols
- for await... of

Symbol.prototype.description

A propriedade leitura somente **`description`** é uma string que retorna a descrição opcional de objetos `Symbol`.

Experimente

```
console.log(Symbol("desc").description);
// Expected output: "desc"

console.log(Symbol.iterator.description);
// Expected output: "Symbol.iterator"

console.log(Symbol.for("foo").description);
// Expected output: "foo"

console.log(`${Symbol("foo").description}bar`);
// Expected output: "foobar"
```

Descrição

`Symbol` objetos podem ser criados com uma uma descrição opcional na qual podem ser usados para debbuging mas não para acessar o próprio símbolo. A `Symbol.prototype.description` propriedade pode ser usada para ler essa descrição. É diferente do `Symbol.prototype.toString()` pois não contém a string incluida "`Symbol()`". Veja os exemplos.

Exemplos

Usando descrição

```
Symbol("desc").toString(); // "Symbol(desc)"
Symbol("desc").description; // "desc"
Symbol("").description; // ""
Symbol().description; // undefined

// símbolos conhecidos
Symbol.iterator.toString(); // "Symbol(Symbol.iterator)"
Symbol.iterator.description; // "Symbol.iterator"

// símbolos globais
Symbol.for("foo").toString(); // "Symbol(foo)"
Symbol.for("foo").description; // "foo"
```

Veja também

- `Symbol.prototype.toString()`
- Polyfill: https://npmjs.com/symbol.prototype.description

Symbol.hasInstance

O symbol bem-conhecido **`Symbol.hasInstance`** é usado para determinar se um objeto construtor reconhece um objeto como de sua instância. O comportamento do operador `instanceof` pode ser customizado por este symbol.

Property attributes of `Symbol.hasInstance`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Você pode implementar o comportamento customizado do seu `instanceof` deste jeito; por exemplo:

```
class MyArray {
  static [Symbol.hasInstance](instance) {
    return this.prototype.isPrototypeOf(instance) || Array.isArray(instance);
  }
}

console.log([] instanceof MyArray); // true
console.log(new MyArray() instanceof MyArray); // true
console.log(new Image() instanceof MyArray); // false

class MySubArray extends MyArray {}
console.log(new MySubArray() instanceof MySubArray); // true
```

```
console.log(new MySubArray() instanceof MyArray); // true
console.log(new MyArray() instanceof MySubArray); // false
```

Veja também

- `instanceof`
- `isPrototypeOf()`

Symbol.isConcatSpreadable

O **`Symbol.isConcatSpreadable`** é um símbolo conhecido que é usado para configurar se um objeto deve ser achatado para um elemento da array quando usado o método `Array.prototype.concat()`.

Experimente

```
const alpha = ["a", "b", "c"];
const numeric = [1, 2, 3];
let alphaNumeric = alpha.concat(numeric);

console.log(alphaNumeric);
// Expected output: Array ["a", "b", "c", 1, 2, 3]

numeric[Symbol.isConcatSpreadable] = false;
alphaNumeric = alpha.concat(numeric);

console.log(alphaNumeric);
// Expected output: Array ["a", "b", "c", Array [1, 2, 3]]
```

Descrição

O `@@isConcatSpreadable` símbolo (`Symbol.isConcatSpreadable`) pode também ser definido como uma propriedade própria ou herdada e seu valor é um booleano. Ele consegue controlar o comportamento das arrays e objetos semelhantes a array:

- For array objects, the default behavior is to spread (flatten) elements. `Symbol.isConcatSpreadable` can avoid flattening in these cases.
- For array-like objects, the default behavior is no spreading or flattening. `Symbol.isConcatSpreadable` can force flattening in these cases.

Property attributes of `Symbol.isConcatSpreadable`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Arrays

Por padrão, o `Array.prototype.concat()` espalha (alinha) arrays no seus resultados:

```
let alpha = ['a', 'b', 'c'],
let numeric = [1, 2, 3]

let alphaNumeric = alpha.concat(numeric)

console.log(alphaNumeric)  // Resultado: ['a', 'b', 'c', 1, 2, 3]
```

Quando configurando o `Symbol.isConcatSpreadable` para `false`, o comportamento padrão dele:

```
let alpha = ['a', 'b', 'c'],
let numeric = [1, 2, 3]

numeric[Symbol.isConcatSpreadable] = false
let alphaNumeric = alpha.concat(numeric)

console.log(alphaNumeric)  // Resultado: ['a', 'b', 'c', [1, 2, 3] ]
```

Objetos de array semelhantes

Para objetos de array semelhantes, o padrão não é espalhado. `Symbol.isConcatSpreadable`precisa ser configurado para `true` para poder conseguir um a array alinhada:

```
let x = [1, 2, 3];

let fakeArray = {
  [Symbol.isConcatSpreadable]: true,
  length: 2,
  0: "hello",
  1: "world",
};

x.concat(fakeArray); // [1, 2, 3, "hello", "world"]
```

> **Nota:** A propriedade `length` é usada para controlar o número de propriedade dos objetos para ser adicionado. No exemplo acima, `length:2` indica que duas propriedades tem de ser adicionado.

Veja também

- `Array.prototype.concat()`

Symbol.iterator

O símbolo conhecido **`Symbol.iterator`** especifica o iterador padrão para um objeto. Usado por `for...of`.

Experimente

```
const iterable1 = {};

iterable1[Symbol.iterator] = function* () {
  yield 1;
  yield 2;
  yield 3;
};

console.log([...iterable1]);
// Expected output: Array [1, 2, 3]
```

Descrição

Sempre que um objeto precisa ser iterado (como um começo de um `for..of` loop), o método `@@iterator` é chamado sem argumentos, e o **iterador** retornado é usado para obter os valores para serem iterados.

Alguns tipos built-in tem um comportamento padrão de iteração, enquanto outros tipos (assim como `Object`) não tem. O tipo built-in com um método `@@iterator` são:

- `Array.prototype[@@iterator]()`
- `TypedArray.prototype[@@iterator]()`
- `String.prototype[@@iterator]()`
- `Map.prototype[@@iterator]()`
- `Set.prototype[@@iterator]()`

Veja também Iteration protocols para mais informação.

Property attributes of `Symbol.iterator`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Iteráveis definidos pelo usuário

Podemos fazer nosso próprio iterável dessa forma:

```
var myIterable = {};
myIterable[Symbol.iterator] = function* () {
  yield 1;
  yield 2;
  yield 3;
};
[...myIterable]; // [1, 2, 3]
```

Ou iteráveis podem ser definidos diretamente dentro de uma classe ou um objeto usando computed property:

```
class Foo {
  *[Symbol.iterator] () {
    yield 1;
    yield 2;
    yield 3;
  }
}

const someObj = {
  *[Symbol.iterator] () {
    yield 'a';
    yield 'b';
  }
}

[...new Foo] // [ 1, 2, 3 ]
[...someObj] // [ 'a', 'b' ]
```

Iteráveis não bem formados

Se um método `@@iterator` não retornar um objeto iterador, então é um iterável não bem formado. Usando dessa forma é resultará em uma excessão de tempo ou comportamentos com erros:

```
var nonWellFormedIterable = {}
nonWellFormedIterable[Symbol.iterator] = () => 1
[...nonWellFormedIterable] // TypeError: [] is not a function
```

Veja também

- Iteration protocols
- `Array.prototype[@@iterator]()`
- `TypedArray.prototype[@@iterator]()`
- `String.prototype[@@iterator]()`
- `Map.prototype[@@iterator]()`
- `Set.prototype[@@iterator]()`

Symbol.match

O símbolo **`Symbol.match`** é conhecido por especificar a compatibilidade de uam expressão regular contra uma string. Essa função é chamada pelo método `String.prototype.match()`.

Experimente

```
const regexp1 = /foo/;
// console.log('/foo/'.startsWith(regexp1));
// Expected output (Chrome): Error: First argument to String.prototype.startsWith must not be a regular expression
// Expected output (Firefox): Error: Invalid type: first can't be a Regular Expression
// Expected output (Safari): Error: Argument to String.prototype.startsWith cannot be a RegExp

regexp1[Symbol.match] = false;

console.log("/foo/".startsWith(regexp1));
// Expected output: true

console.log("/baz/".endsWith(regexp1));
// Expected output: false
```

Descrição

Essa função também é usada para identificar se um objeto tem o comportamento de uma expressão regular. Por exemplo, os métodos `String.prototype.startsWith()`, `String.prototype.endsWith()` e `String.prototype.includes()`, verificar se o primeiro agumento é uma expressão regular e irá lançar um `TypeError` se eles forém. Agora, se o símbolo `match` é configurado para `false` (ou um valor Falsy), ele indica que o objeto não tem intensão de ser usado como um ojbeto de expressão regular

Property attributes of `Symbol.match`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Desativando a verificação <code>isRegExp</code>

O seguinte código vai lançar um `TypeError`:

```
"/bar/".startsWith(/bar/);

// Lança um TypeError, como /bar/ é uma expressão regular
// não Symbol.match não é modificado.
```

Entretanto, se você configurar `Symbol.match` para `false`, a verificação `isRegExp` (que usa a propriedade `match`) que o objeto não é uma expressão regular. O método `startsWith` e `endsWith` não vão lançar um `TypeError` como consequência.

```
var re = /foo/;
re[Symbol.match] = false;
"/foo/".startsWith(re); // true
"/baz/".endsWith(re); // false
```

Veja também

- `Symbol.replace`
- `Symbol.search`
- `Symbol.split`
- `RegExp.prototype[@@match]()`

Symbol.matchAll

O símbolo **`Symbol.matchAll`** é conhecido por retornar um iterador, que produz conrrespondências de uma expressão regular com uma string. Essa função é usada pelo método `String.prototype.matchAll()`.

Experimente

```
const re = /[0-9]+/g;
const str = "2016-01-02|2019-03-07";
const result = re[Symbol.matchAll](str);

console.log(Array.from(result, (x) => x[0]));
// Expected output: Array ["2016", "01", "02", "2019", "03", "07"]
```

Descrição

Esse símbolo é usado pelo `String.prototype.matchAll()` e especificado no `RegExp.prototype[@@matchAll]()`. Os próximos dois exemplos retornam o mesmo resultado:

```
"abc".matchAll(/a/);

/a/[Symbol.matchAll]("abc");
```

Esse método existe para costumizar o comportamento conrrespondente com as subclasses `RegExp`.

Property attributes of `Symbol.matchAll`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Usuando Symbol.matchAll

```
let re = /[0-9]+/g;
let str = "2016-01-02|2019-03-07";

const numbers = {
  *[Symbol.matchAll](str) {
    for (const n of str.matchAll(/[0-9]+/g)) yield n[0];
  },
};

console.log(Array.from(str.matchAll(numbers)));
//  Array ["2016", "01", "02", "2019", "03", "07"]
```

Veja `String.prototype.matchAll()` e `RegExp.prototype[@@matchAll]()` para mais exemplos.

Veja também

- `String.prototype.matchAll()`
- `RegExp.prototype[@@matchAll]()`

Symbol.replace

O símbolo **`Symbol.replace`** é conhecido por especificar o método que substitui as substrings conrrespondentes de uma string. Essa função é chamada pelo método `String.prototype.replace()`.

Para mais informações, veja `RegExp.prototype[@@replace]()` e `String.prototype.replace()`.

Experimente

```
class Replace1 {
  constructor(value) {
    this.value = value;
  }
  [Symbol.replace](string) {
    return `s/${string}/${this.value}/g`;
  }
}

console.log("foo".replace(new Replace1("bar")));
// Expected output: "s/foo/bar/g"
```

Property attributes of `Symbol.replace`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Usando Symbol.replace

```
class CustomReplacer {
  constructor(value) {
    this.value = value;
  }
  [Symbol.replace](string) {
    return string.replace(this.value, "#!@?");
  }
}

console.log("football".replace(new CustomReplacer("foo")));
// resultado esperado: "#!@?tball"
```

Veja também

- `Symbol.match`
- `Symbol.search`
- `Symbol.split`
- `RegExp.prototype[@@replace]()`

Symbol.search

O símbolo **`Symbol.search`** é um método conhecido por retornar o índice com uma string que corresponde a expressão regular. Essa função é chamada pelo método `String.prototype.search()`.

Para mais informação, veja `RegExp.prototype[@@search]()` e `String.prototype.search()`.

Experimente

```
class Search1 {
  constructor(value) {
    this.value = value;
  }
  [Symbol.search](string) {
    return string.indexOf(this.value);
  }
}

console.log("foobar".search(new Search1("bar")));
// Expected output: 3
```

Property attributes of `Symbol.search`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Pesquisa de string personalizada

```
class caseInsensitiveSearch {
  constructor(value) {
    this.value = value.toLowerCase();
  }
  [Symbol.search](string) {
    return string.toLowerCase().indexOf(this.value);
  }
}

console.log("foobar".search(new caseInsensitiveSearch("BaR")));
// resultado esperado: 3
```

Veja também

- `Symbol.match`
- `Symbol.replace`
- `Symbol.split`
- `RegExp.prototype[@@search]()`

Symbol.species

O símbolo **`Symbol.species`** especifica uma propriedade valor-função que a função construtora usa para criar objetos derivados.

Experimente

```
class Array1 extends Array {
  static get [Symbol.species]() {
    return Array;
  }
}

const a = new Array1(1, 2, 3);
```

```
const mapped = a.map((x) => x * x);

console.log(mapped instanceof Array1);
// Expected output: false

console.log(mapped instanceof Array);
// Expected output: true
```

Descrição

A propriedade acessora de espécies permite que subclasses substituam o construtor padrão para objetos.

Property attributes of `Symbol.species`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Usando espécies

Você pode querer que retorne objetos `Array` em sua class derivada de array `MyArray`. Por exemplo, quando usar métodos tais como `map()` que retorna o construtor padrão, você quer que esses métodos retornem um objeto `Array` pai, em vez do objeto `MyArray`. Esses símbolos espécies permitem que você fala isso:

```
class MyArray extends Array {
  // Substitui espécies para a Array pai construtora
  static get [Symbol.species]() {
    return Array;
  }
}
let a = new MyArray(1, 2, 3);
let mapped = a.map((x) => x * x);

console.log(mapped instanceof MyArray); // false
console.log(mapped instanceof Array); // true
```

Veja também

- `Map[@@species]`
- `Set[@@species]`

Symbol.split

O **`Symbol.split`** é um símbolo conhecido que especifica o método que divide uma string nos índices correspondentes a uma expressão regular. Essa função é chamada pelo método `String.prototype.split()`.

Para mais informações, veja `RegExp.prototype[@@split]()` e `String.prototype.split()`.

Experimente

```
class Split1 {
  constructor(value) {
    this.value = value;
  }
  [Symbol.split](string) {
    const index = string.indexOf(this.value);
    return `${this.value}${string.substring(0, index)}/${string.substring(
      index + this.value.length,
    )}`;
  }
}

console.log("foobar".split(new Split1("foo")));
// Expected output: "foo/bar"
```

Property attributes of `Symbol.split`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Divisão reversa personalizada

```
class ReverseSplit {
  [Symbol.split](string) {
    const array = string.split(" ");
    return array.reverse();
  }
}
```

```
console.log("Another one bites the dust".split(new ReverseSplit()));
// resultado esperado: [ "dust", "the", "bites", "one", "Another" ]
```

Veja também

- `Symbol.match`
- `Symbol.replace`
- `Symbol.search`
- `RegExp.prototype[@@split]()`

Symbol.toPrimitive

O **`Symbol.toPrimitive`** é um símbolo que específica uma propriedade com valor função que é chamada para converter um ojbeto para um valor primitivo correspondente.

Experimente

```
const object1 = {
  [Symbol.toPrimitive](hint) {
    if (hint === "number") {
      return 42;
    }
    return null;
  },
};

console.log(+object1);
// Expected output: 42
```

Descrição

Com a ajuda da propriedade **`Symbol.toPrimitive`** (usada como uma função valor), um objeto pode ser convertido para um valor primitivo. a função é chamada com um argumento string **`hint`**, que espcífica o tipo de preferência do resultado do valor primitivo. O argumento **`hint`** pode ser um `"number"`, `"string"`, e `"default"`.

Property attributes of `Symbol.toPrimitive`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Modificando valores primitivos convertendo para um objeto

O exemplo a seguir descreve que a propriedade **`Symbol.toPrimitive`** pode ser modificar o valor primitivo convertendo para um objeto.

```
// Um objeto sem propriedade Symbol.toPrimitive.
var obj1 = {};
console.log(+obj1); // NaN
console.log(`${obj1}`); // "[object Object]"
console.log(obj1 + ""); // "[object Object]"

// Um objeto com propriedade Symbol.toPrimitive
var obj2 = {
  [Symbol.toPrimitive](hint) {
    if (hint == "number") {
      return 10;
    }
    if (hint == "string") {
      return "hello";
    }
    return true;
  },
};
console.log(+obj2); // 10        -- dica é "number"
console.log(`${obj2}`); // "hello"   -- dica é "string"
console.log(obj2 + ""); // "true"    -- dica é "default"
```

Veja também

- `Date.prototype[@@toPrimitive]`
- `Symbol.prototype[@@toPrimitive]`
- `Object.prototype.toString()`
- `Object.prototype.valueOf()`

Symbol.toStringTag

O símbolo **`Symbol.toStringTag`** é uma propriedade com valor string que é usada para a criação de uma descrição de string de um objeto padrão. É acessado internalmente pelo método `Object.prototype.toString()`.

Experimente

```
class ValidatorClass {
  get [Symbol.toStringTag]() {
    return "Validator";
  }
}

console.log(Object.prototype.toString.call(new ValidatorClass()));
// Expected output: "[object Validator]"
```

Property attributes of `Symbol.toStringTag`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Tags padrões

```
Object.prototype.toString.call("foo"); // "[object String]"
Object.prototype.toString.call([1, 2]); // "[object Array]"
Object.prototype.toString.call(3); // "[object Number]"
Object.prototype.toString.call(true); // "[object Boolean]"
Object.prototype.toString.call(undefined); // "[object Undefined]"
Object.prototype.toString.call(null); // "[object Null]"
// ... e mais
```

Símbolos built-in toStringTag

```
Object.prototype.toString.call(new Map()); // "[object Map]"
Object.prototype.toString.call(function* () {}); // "[object GeneratorFunction]"
Object.prototype.toString.call(Promise.resolve()); // "[object Promise]"
// ... e mais
```

Classes personalizadas para objeto tag

Quando cria sua própria classe, o JavaScript padroniza para "Object" tag:

```
class ValidatorClass {}

Object.prototype.toString.call(new ValidatorClass()); // "[object Object]"
```

Tag costumizada com toStringTag

Agora, com a ajuda do `toStringTag`, você é capaz de costumizar sua própria tag:

```
class ValidatorClass {
  get [Symbol.toStringTag]() {
    return "Validator";
  }
}

Object.prototype.toString.call(new ValidatorClass()); // "[object Validator]"
```

toStringTag disponível em todos os objetos protótipos da DOM

Devido a uma mudança nas especificações WebIDL spec change em meados de 2020, navegadores estão adicionando uma propriedade `Symbol.toStringTag` para todos os objetos protótipos da DOM . Por exemplo, para acessar a propriedade `Symbol.toStringTag` no `HTMLButtonElement`:

```
let test = document.createElement("button");
test.toString(); // Retorna [object HTMLButtonElement]
test[Symbol.toStringTag]; // Retona HTMLButtonElement
```

Veja também

- `Object.prototype.toString()`

Symbol.unscopables

O símbolo **`Symbol.unscopables`** é usado para específicar um valor objeto cujo os nomes de propriedade próprio e herdados são excluídos das associações do ambiente `with` o objeto associado.

Experimente

```
const object1 = {
  property1: 42,
};

object1[Symbol.unscopables] = {
  property1: true,
};
```

```
with (object1) {
  console.log(property1);
  // Expected output: Error: property1 is not defined
}
```

Descrição

O `@@unscopables` símbolo (`Symbol.unscopables`) pode ser definido em qualquer objeto para impedir que os nomes da propriedade sejam expostos como variavéis lexicais `with` ligações de ambiente. Note que se usar o Strict mode, `with` as declarações não estão disponíveis e provavelmente não haverá necessidade desse símbolo.

Configurando a propriedade para `true` em um objeto `unscopables` tornará ele não *unscopable* e portanto não irá apareer nas variavéis de escopo lexicais. Configurando uma propriedade para `false` o tornará `scopable` e portanto irá aparecer no escopo de variavéis lexicais.

Property attributes of `Symbol.unscopables`	
Writable	no
Enumerable	no
Configurable	no

Exemplos

Escopo com declarações

O seguinte código funciona normalmente no ES5 e anteriores. Entretanto, no ECMAScript 2015 e superiores, o método `Array.prototype.keys()` foi introduzido. Isso significa que dentro do ambiente `with` "chaves" seria agora o método e não as variavéis. Isso qunado os símbolos `unscopables` foram introduzidos. Uma confiração built-in `unscopables` é implementada como `Array.prototype[@@unscopables]` para prevenir que alguns métodos de Array tenham escopo definido na desclaração `with`.

```
var keys = [];

with (Array.prototype) {
  keys.push("something");
}

Object.keys(Array.prototype[Symbol.unscopables]);
// ["copyWithin", "entries", "fill", "find", "findIndex",
//  "includes", "keys", "values"]
```

Objetos não escopáveis

Você também pode configurar `unscopables` para seu próprio objeot.

```
var obj = {
  foo: 1,
  bar: 2,
};

obj[Symbol.unscopables] = {
  foo: false,
  bar: true,
};

with (obj) {
  console.log(foo); // 1
  console.log(bar); // ReferenceError: bar is not defined
}
```

Veja também

- `Array.prototype[@@unscopables]`
- `with` statement (not available in Strict mode)

TypedArray.prototype.sort()

O método **`sort()`** ordena os elementos de uma matriz tipada *no local* e retorna a matriz ordenada. Esse método tem o mesmo algoritmo que `Array.prototype.sort()`. *TypedArray* é uma das maneiras de escrever matrizes.

Syntax

```
typedarray.sort([compareFunction])
```

Parâmetros

`compareFunction` Optional

Especifica uma função que define a ordem de classificação.

Valor de retorno

A matriz ordenada.

Exemplos

Para mais exemplos, acesse o método `Array.prototype.sort()`.

```
var numbers = new Uint8Array([40, 1, 5, 200]);
numbers.sort();
// Uint8Array [ 1, 5, 40, 200 ]
// A compare function is not required as in the case of Array
// to sort the numbers numerically.

var numbers = [40, 1, 5, 200];
numbers.sort();
// The elements are sorted as strings.
// [1, 200, 40, 5]

function compareNumbers(a, b) {
  return a - b;
}

numbers.sort(compareNumbers);
// [ 1, 5, 40, 200 ]
```

Ver também

- `Array.prototype.sort()`

TypeError

O objeto **`TypeError`** representa um erro de quando um valor não é do tipo esperado.

Sintaxe

```
new TypeError([message[, fileName[, lineNumber]]])
```

Parâmetros

`message`

Opcional. Mensagem, descrição do erro

`fileName`

Opcional. O nome do arquivo contendo o código que causou a exceção

`lineNumber`

Opcional. O número da linha do código que causou a exeção

Descrição

O `TypeError` é ativado quando um operador ou argumento passado para uma função é incompativel com o tipo esperado por esse operador ou função.

Propriedades

`TypeError`

Permite a adição de propriedades para o objeto `TypeError`.

Métodos

O TypeError global não contém métodos por si só, no entanto, ele herda alguns métodos através da cadeia prototype.

<code>Instâncias TypeError</code>

Propriedades

Métodos

Exemplos

Capturando um <code>TypeError</code>

```
try {
  null.f();
} catch (e) {
  console.log(e instanceof TypeError); // true
  console.log(e.message); // "null has no properties"
  console.log(e.name); // "TypeError"
  console.log(e.fileName); // "Scratchpad/1"
  console.log(e.lineNumber); // 2
  console.log(e.columnNumber); // 2
  console.log(e.stack); // "@Scratchpad/2:2:3\n"
}
```

Criando um <code>TypeError</code>

```
try {
  throw new TypeError("Hello", "someFile.js", 10);
} catch (e) {
  console.log(e instanceof TypeError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "TypeError"
  console.log(e.fileName); // "someFile.js"
  console.log(e.lineNumber); // 10
  console.log(e.columnNumber); // 0
  console.log(e.stack); // "@Scratchpad/2:2:9\n"
}
```

Veja também

- `Error`
- `TypeError`

undefined

Resumo

O valor global `undefined` representa um valor indefinido. Trata-se de um dos tipos primitivos do JavaScript.

Property attributes of `undefined`	
Writable	no
Enumerable	no
Configurable	no

Sintaxe

```
undefined
```

Descrição

O `undefined` é uma propriedade do objeto `global`, ou seja, é uma variável no escopo global. O valor inicial de `undefined` é o valor primitivo `undefined`.

Nos browsers modernos (JavaScript 1.8.5 / Firefox 4+), a especificação ECMAScript 5 define que o `undefined` é uma propriedade não configurável e somente leitura. Mesmo quando esse não for o caso, evite sobrescrevê-lo.

Uma variável que não teve um valor atribuído é do tipo `undefined`. Um método ou sentença também retorna `undefined` se a variável que está sendo avaliada não tem um valor atribuído. Uma função retorna `undefined` se um valor não for `retornado`.

Uma vez que undefined não é uma `palavra reservada`, ele pode ser usado como um identificador (nome de variável) em qualquer escopo que não seja o escopo global.

```
// escreve no console "foo string"
(function () {
  var undefined = "foo";
  console.log(undefined, typeof undefined);
})();

// escreve no console "foo string"
(function (undefined) {
  console.log(undefined, typeof undefined);
})("foo");
```

Exemplos

Igualdade estrita (===) e <code>undefined</code>

Voce pode utilizar `undefined` com os operadores de igualdade e desigualdade estritas para determinar se a variável possui um valor. No código a seguir, a variável `x` não foi definida, e o resultado do `if` é verdadeiro.

```
var x;
if (x === undefined) {
  // sentenças aqui são executadas
} else {
  // o código aqui não é executado
}
```

> **Nota:** O operador de igualdade estrita (`===`) tem que ser utilizado aqui uma vez que `x == undefined` também checa se `x` é nulo, enquanto o operador não o faz. `null` não é equivalente à `undefined`. Veja `operadores de comparação` (em inglês) para maiores detalhes.

<code>Operador Typeof</code> e <code>undefined</code>

Alternativamente, `typeof` pode ser usado:

```
var x;
if (typeof x === "undefined") {
  // esse código é executado
}
```

Uma das principais razões para usar o `typeof` é que ele não lança erros caso a variável não tenha sido inicializada.

```
// x não foi atribuída anteriormente
if (typeof x === "undefined") {
  // retorna verdadeiro sem lançar erros
  // esse código executa
}

if (x === undefined) {
  // lança um ReferenceError para x
}
```

No entanto, esse tipo de técnica deveria ser evitada. A linguagem JavaScript é uma linguagem com escopo estático, portanto o conhecimento sobre se uma variável está definida pode ser adquirido pela verificação de sua definição dentro do contexto à qual ela pertence. A única exceção é para o escopo global. No entanto, o escopo global é anexado ao objeto global, portanto a verificação da existência de uma variável no contexto global pode ser feita através de uma checagem na propriedade do objeto global usando o operador `in`, por exemplo.

<code>Operador Void</code> e <code>undefined</code>

O operador `void` é a terceira alternativa.

```
var x;
if (x === void 0) {
  // esse código executa
}

// y não foi definido antes
if (y === void 0) {
  // lança uma ReferenceError (ao contrário de `typeof`)
}
```

unescape()

A função **unescape()** obsolta computa uma nova string na qual as sequencias hexadecimal são esquecidas com o caractere que representa. As sequências de escape podem ser introduzidas como funções `escape`. Porque a função 'unescape' está obsoleta, ao invez disso, use `decodeURI` ou `decodeURIComponent`.

> **Nota:** Não use `unescape` para decodificar URIs, use `decodeURI` ao invez disso.

Syntax

```
unescape(str)
```

Parâmetros

`str`

Uma string a ser decodificada.

Valor retornado

Uma nova string na qual alguns caracteres tem que ser 'unescaped'.

Descrição

A função `unescape` é uma propriedade do objeto global.

Exemplos

```
unescape("abc123"); // "abc123"
unescape("%E4%F6%FC"); // "äöü"
unescape("%u0107"); // "ć"
```

Veja também

- `decodeURI`
- `decodeURIComponent`
- `escape`

WeakMap

Sumário

O objeto WeakMap é uma coleção de pares key/value na qual as chaves são fracamente referenciadas. As chaves **devem** ser objetos, e os valores podem ser de tipos arbitrários.

Você pode descobrir mais sobre `WeakMaps` na seção Objeto WeakMap.

Sintaxe

```
new WeakMap([iterable])
```

Parâmetros

`iterable`

> Iterable é um Array ou outro objeto iterável cujos elementos são pares key-value (Arrays de dois elementos). Cada par key-value será adicionados ao novo `WeakMap`. `null` é tratado como `undefined`.

Descrição

Por quê WeakMap?

Um programador JavaScript experiente vai notar que esta API pode ser implementada em JavaScript com dois arrays (um para chaves, um para valores) compartilhado pelos 4 métodos da API. Tal implementação teria duas principais incoveniências. A primeira é uma busca O(n) (n sendo o número de chaves no map). A segunda é o problema de memory leak. Com maps escritos manualmente, o array de chaves guardaria referências para objetos chave, prevenindo-os de serem coletados pelo Garbage Collector. Nos WeakMaps nativos, referências aos objetos chave são segurados de modo "fraco", o que significa que eles não previnem a coleção pelo GC no caso de não haver nenhuma outra referência ao objeto.

Por conta das referências serem fracas, chaves de `WeakMap` não são enumeráveis (i.e. não existe um método que dá a você uma lista de chaves). Se existisse tal método, a lista dependeria no estado da coleção do GC, introduzindo não-determinismo. Se você quiser ter uma lista de chaves, deve usar um `Map`.

Construtor

`WeakMap()`

> Cria um novo objeto `WeakMap`.

Métodos de instância

`WeakMap.prototype.delete(key)`

> Remove qualquer valor associado à chave(`key`). `WeakMap.prototype.has(key)` retornará `false` depois.

`WeakMap.prototype.get(key)`

> Retorna o valor associado à chave(`key`). Ou `undefined` se não houver.

`WeakMap.prototype.has(key)`

> Retorna um boolean afirmando se um valor foi associado à chave(`key`) no objeto `WeakMap` ou não.

`WeakMap.prototype.set(key, value)`

> Define o valor(`value`) da chave(`key`) no objeto `WeakMap`. Retorna o objeto `WeakMap`.

Exemplos

Exemplo: Usando <code>WeakMap</code>

```
var wm1 = new WeakMap(),
  wm2 = new WeakMap(),
  wm3 = new WeakMap();
var o1 = {},
  o2 = function () {},
  o3 = window;

wm1.set(o1, 37);
wm1.set(o2, "azerty");
wm2.set(o1, o2); // um valor pode ser qualquer coisa, incluindo um objeto or uma função
wm2.set(o3, undefined);
wm2.set(wm1, wm2); // chaves e valores pode ser quaisquer objetos. Até mesmo WeakMaps!

wm1.get(o2); // "azerty"
wm2.get(o2); // undefined, pois não existe valor para o2 em wm2
wm2.get(o3); // undefined, pois este é o valor definido

wm1.has(o2); // true
wm2.has(o2); // false
wm2.has(o3); // true (mesmo se o valor armazenado for 'undefined')

wm3.set(o1, 37);
wm3.get(o1); // 37
wm3.clear();
wm3.get(o1); // undefined, pois wm3 foi 'limpado' e não há mais valor para o1.

wm1.has(o1); // true
wm1.delete(o1);
wm1.has(o1); // false
```

Veja também

- WeakMap bug at Mozilla
- Hiding Implementation Details with ECMAScript 2015 WeakMaps
- `Map`
- `Set`
- `WeakSet`

WeakMap.prototype.delete()

O método **`delete()`** remover o elemento especificado ou setado a partir de um objeto instanciado do `WeakMap`.

Experimente

```
const weakmap1 = new WeakMap();
const object1 = {};

weakmap1.set(object1, 42);

console.log(weakmap1.delete(object1));
// Expected output: true

console.log(weakmap1.has(object1));
// Expected output: false
```

Sintaxe

```
wm.delete(key);
```

Parâmetros

chave(key)

Requerido. A chave(`key`) do elemento a ser removido do objeto instanciado do `WeakMap`.

Valores retornados

`true` se o emento do objeto do `WeakMap` tenha sido removido com sucesso. `false` se a chave(`key`) não for encontrada no `WeakMap` ou se a chave(`key`) não for um objeto.

Exemplos

Usando o método <code>delete</code>

```
var wm = new WeakMap();
wm.set(window, "foo");

console.log(wm.delete(window)); // Retorna true. Removido com sucesso.

wm.has(window); // Retorna false. O objeto window não é mais pertecente ao WeakMap.
```

Ver também

- `WeakMap`

WeakMap.prototype.get()

O método **`get()`** retorna um elemento especificado de um objeto `WeakMap`.

Experimente

```
const weakmap1 = new WeakMap();
const object1 = {};
const object2 = {};

weakmap1.set(object1, 42);

console.log(weakmap1.get(object1));
// Expected output: 42

console.log(weakmap1.get(object2));
// Expected output: undefined
```

Sintaxe

```
wm.get(chave);
```

Parâmetros

chave

Obrigatório. A chave do elemento a ser retornado pelo objeto `WeakMap`.

Valor de retorno

O elemento associado à chave especificada no objeto `WeakMap`. Se a chave não for encontrada, retorna-se `undefined`.

Exemplos

Utilizando o método <code>get</code>

```
var wm = new WeakMap();
wm.set(window, "foo");

wm.get(window); // Retorna "foo".
wm.get("baz"); // Retorna undefined.
```

Ver também

- `WeakMap`
- `WeakMap.set()`
- `WeakMap.has()`

WeakMap.prototype.has()

O método **`has()`** retorna um booleano indicando se um elemento com a chave especificada existe no objeto `WeakMap` ou não.

Experimente

```
const weakmap1 = new WeakMap();
const object1 = {};
const object2 = {};

weakmap1.set(object1, "foo");

console.log(weakmap1.has(object1));
// Expected output: true

console.log(weakmap1.has(object2));
// Expected output: false
```

Sintaxe

```
wm.has(chave);
```

Parameters

chave

Obrigatório. A chave do elemento cuja presença deve ser verificada no objeto `WeakMap`.

Valor de retorno

Boolean

Retorna `true` se um elemento com a chave especificada existir no objeto `WeakMap`. Caso contrário, retorna `false`.

Exemplos

Utilizando o método <code>has</code>

```
var wm = new WeakMap();
wm.set(window, "foo");

wm.has(window); // retorna true
wm.has("baz"); // retorna false
```

Ver também

- `WeakMap`
- `WeakMap.prototype.set()`
- `WeakMap.prototype.get()`

WeakMap.prototype.set()

O método **`set()`** adiciona um novo elemento com a `chave` e o `valor` especificados em um objeto `WeakMap`.

Experimente

```
const weakmap1 = new WeakMap();
const object1 = {};
const object2 = {};

weakmap1.set(object1, "foo");
weakmap1.set(object2, "bar");

console.log(weakmap1.get(object1));
// Expected output: "foo"
```

```
console.log(weakmap1.get(object2));
// Expected output: "bar"
```

Sintaxe

```
wm.set(chave, valor);
```

Parâmetros

chave

Obrigatório. A chave do elemento a ser adicionado no objeto `WeakMap`.

valor

Obrigatório. O valor do elemento a ser adicionado no objeto `WeakMap`.

Valor de retorno

O objeto `WeakMap`.

Exemplos

Utilizando o método <code>set</code>

```
var wm = new WeakMap();
var obj = {};

// Adicionar novos elementos ao WeakMap
wm.set(obj, "foo").set(window, "bar"); // encadeável

// Atualizar um elemento no WeakMap
wm.set(obj, "baz");
```

Notas específicas ao Firefox

- Antes do Firefox 33, `WeakMap.prototype.set` retornava `undefined` e, portanto, não era encadeável. Isso foi resolvido (Erro do Firefox 1031632). O comportamento também podia ser encontrado no Chrome/v8 (issue).

Ver também

- `WeakMap`
- `WeakMap.prototype.get()`
- `WeakMap.prototype.has()`

WeakSet

O objeto **WeakSet** pertmite que você armazene objetos mantidos "fracamente" na coleção.

Syntaxe

```
new WeakSet([iterable]);
```

Parâmetros

iterable

Se umobjeto interagível é passado, todos os seus elementos serão adicionados ao novo `WeakSet`.

Descrição

Objetos `WeakSet` são coleções de objetos. Um objeto no `WeakSet` pode ocorrer uma vez; é único na coleção do `WeakSet`.

As principais diferenças para o objeto `Set` são:

- Em contraste ao `Sets`, `WeakSets` são **coleções apenas de objetos** e não de valores arbitrários de qualquer tipo.
- O `WeakSet` é *fraco*. Referências aos objetos nas coleções são mantidas "fracamente". Se não há outra referência para um objeto mantido na `WeakSet`, eles pode ser coletados pelo gerenciamento de memória (*garbage collection*).

Propriedades

`WeakSet.length`

O valor da propriedade de comprimento é 0.

`WeakSet`

Representa o protóripo para o construtos `Set`. Permite a adição de propriedades para todos os objetos do `WeakSet`.

<code>Instâncias WeakSet</code>

Todas as instâncias do `WeakSet` herdam do `WeakSet`.

Propriedades

Métodos

Exemplos

Usando o objeto do <code>WeakSet</code>

```
var ws = new WeakSet();
var obj = {};
var foo = {};

ws.add(window);
ws.add(obj);

ws.has(window); // true
ws.has(foo); // false, foo não foi adicionado ao set

ws.delete(window); // remove window do set
ws.has(window); // false, window foi removido
```

Veja também

- `Map`
- `Set`
- `WeakMap`

Iteration protocols

Algumas adições do ECMAScript 2015 não são novos *built-ins* ou uma nova sintaxe, mas protocolos. Estes protocolos podem ser implementados por qualquer objeto desde que respeitando algumas convenções.Existem dois protocolos: O *protocolo iterável* (iterable protocol) e o *protocolo iterador* (iterator protocol).

O protocolo iterável (iterable protocol)

O protocolo **iterável** permite que objetos JavaScript definam ou personalizem seu comportamento de iteração, como valores em um loop do construtor `for..of`. Alguns tipos *built-in* são *built-in iterables* com um comportamento de iteração padrão, tal como `Array` ou `Map`, enquanto outros tipos (como `Object`) não são assim.

Para ser **iterável**, um objeto deve implementar o método **@@iterator**, o que significa que o objeto (ou um dos objetos acima de sua cadeia de protótipos) deve ter uma propriedade com uma chave **@@iterator** que está disponível via constante `Symbol.iterator`:

Property	Value
`[Symbol.iterator]`	Uma função de zero argumentos que retorna um objeto, em conformidade com o protocolo iterador.

Sempre que um objeto precisa ser iterado (como no início de um loop `for..of`), o método `@@iterator` é chamado sem argumentos e o retorno do **iterador** é usado para obter os valores a serem iterados.

O protocolo iterador (iterator protocol)

O protocolo **iterador** define uma maneira padrão de produzir uma sequência de valores (finito ou infinito).

Um objeto é um iterador quando implementa um método **`next()`** com a semântica adiante:

Propriedade	Valor
`next`	Uma função sem argumentos que retorna um objeto com duas propriedades: • `done` (boolean) ◦ Tem o valor `true` se o iterador ultrapassar o final da sequência iterada. Nesse caso `value` opcionalmente especifica o valor de retorno do iterador. ◦ Tem o valor `false` se o iterador foi capaz de produzir o próximo valor na sequência. Isso é equivalente a não especificar a propriedade `done`. • `value` - qualquer valor JavaScript retornado pelo iterador. Pode ser omitido quando `done` é `true`. O método `next` sempre retorna um objeto com propriedades apropriadas, incluindo `done` e `value`. Se um valor não-objeto é retornado (tal como `false` ou `undefined`), será lançado um `TypeError` ("iterator.next() retorna o valor de um não-objeto").

Nota: Não é possível saber de forma reflexiva se um determinado objeto implementa o protocolo do iterador, no entanto, é fácil criar um objeto que satisfaça tanto o iterador quanto os protocolos iteráveis (como mostrado no exemplo abaixo). Fazer isso permite que um iterador seja consumido pelas várias sintaxes que iteráveis esperam. Assim, raramente é desejável implementar o protocolo do iterador sem também implementar iteráveis.

```
var myIterator = {
  next: function () {
    // ...
  },
  [Symbol.iterator]: function () {
    return this;
  },
};
```

Exemplos de uso do protocolo de iteração

Uma `String` é um exemplo de um objeto iterable built-in:

```
var someString = "hi";
typeof someString[Symbol.iterator]; // "function"
```

O iterador padrão de uma string retorna as posições dos caracteres de uma string um por um:

```
var iterator = someString[Symbol.iterator]();
iterator + ""; // "[object String Iterator]"

iterator.next(); // { value: "h", done: false }
iterator.next(); // { value: "i", done: false }
iterator.next(); // { value: undefined, done: true }
```

Alguns construtores built-in, como *spread syntax*, usam o mesmo protocolo de iteração interiormente:

```
[...someString]; // ["h", "i"]
```

Podemos redefinir o comportamento de iteração fornecendo nosso próprio `@@iterator`:

```
var someString = new String('hi');           // need to construct a String object explicitly to avoid auto-boxing

someString[Symbol.iterator] = function() {
  return { // this is the iterator object, returning a single element, the string "bye"
    next: function() {
      if (this._first) {
        this._first = false;
        return { value: 'bye', done: false };
      } else {
        return { done: true };
      }
    },
    _first: true
  };
};
```

Observe como redefinir `@@iterator` afeta o comportamento built-in que faz uso do protocolo de iteração:

```
[...someString]; // ["bye"]
someString + ""; // "hi"
```

Exemplos de Iteráveis

Iteráveis Built-in

`String`, `Array`, `TypedArray`, `Map` and `Set` são todos iteráveis internos, porque cada um dos seus objetos protótipos implementa um método `@@iterator`.

Iteráveis definidos pelo usuário

Nós podemos fazer nossos próprios iterables assim:

```
var myIterable = {};
myIterable[Symbol.iterator] = function* () {
    yield 1;
    yield 2;
    yield 3;
};
[...myIterable]; // [1, 2, 3]
```

Built-in APIs que aceitam iteráveis

Existem muitas APIs que aceitam iteráveis, por exemplo: `Map([iterable])`, `WeakMap([iterable])`, `Set([iterable])` e `WeakSet([iterable])`:

```
var myObj = {};
new Map([[1, 'a'], [2, 'b'], [3, 'c']]).get(2);              // "b"
new WeakMap([[{}, 'a'], [myObj, 'b'], [{}, 'c']]).get(myObj); // "b"
new Set([1, 2, 3]).has(3);                                   // true
new Set('123').has('2');                                     // true
new WeakSet(function* () {
    yield {};
    yield myObj;
    yield {};
}()).has(myObj);                                             // true
```

Veja também `Promise.all(iterable)`, `Promise.race(iterable)` e `Array.from()`.

Sintaxe que espera iteráveis

Some statements and expressions expect iterables, for example the `for-of` loops, spread operator, `yield*`, and destructuring assignment:

```
for(let value of ['a', 'b', 'c']){
    console.log(value);
}
// "a"
// "b"
// "c"

[...'abc']; // ["a", "b", "c"]

function* gen() {
  yield* ['a', 'b', 'c'];
}

gen().next(); // { value:"a", done:false }

[a, b, c] = new Set(['a', 'b', 'c']);
a // "a"
```

Iteráveis não bem formados

If an iterable's `@@iterator` method doesn't return an iterator object, then it's a non-well-formed iterable. Using it as such is likely to result in runtime exceptions or buggy behavior:

```
var nonWellFormedIterable = {}
nonWellFormedIterable[Symbol.iterator] = () => 1
[...nonWellFormedIterable] // TypeError: [] is not a function
```

Iterator examples

Simple iterator

```
function makeIterator(array) {
    var nextIndex = 0;

    return {
       next: function() {
           return nextIndex < array.length ?
               {value: array[nextIndex++], done: false} :
               {done: true};
       }
    };
}

var it = makeIterator(['yo', 'ya']);

console.log(it.next().value); // 'yo'
console.log(it.next().value); // 'ya'
console.log(it.next().done);  // true
```

Infinite iterator

```
function idMaker() {
    var index = 0;

    return {
       next: function(){
           return {value: index++, done: false};
       }
    };
}

var it = idMaker();

console.log(it.next().value); // '0'
console.log(it.next().value); // '1'
console.log(it.next().value); // '2'
// ...
```

Com um gerador

```
function* makeSimpleGenerator(array) {
    var nextIndex = 0;

    while (nextIndex < array.length) {
        yield array[nextIndex++];
    }
}

var gen = makeSimpleGenerator(['yo', 'ya']);

console.log(gen.next().value); // 'yo'
console.log(gen.next().value); // 'ya'
console.log(gen.next().done);  // true
```

```
function* idMaker() {
    var index = 0;
    while (true)
        yield index++;
}

var gen = idMaker();

console.log(gen.next().value); // '0'
console.log(gen.next().value); // '1'
console.log(gen.next().value); // '2'
// ...
```

Com uma classe ES2015

```
class SimpleClass {
  constructor(data) {
    this.index = 0;
    this.data = data;
  }

  [Symbol.iterator]() {
    return {
      next: () => {
        if (this.index < this.data.length) {
          return {value: this.data[this.index++], done: false};
        } else {
          this.index = 0; //If we would like to iterate over this again without forcing manual update of the index
          return {done: true};
        }
      }
    };
  }
}

const simple = new SimpleClass([1,2,3,4,5]);

for (const val of simple) {
  console.log(val);  //'0' '1' '2' '3' '4' '5'
}
```

Um objeto gerador um iterador ou iterável?

Um objeto gerador é tanto iterador quanto iterável:

```
var aGeneratorObject = function* () {
    yield 1;
    yield 2;
    yield 3;
}();
typeof aGeneratorObject.next;
// "function", because it has a next method, so it's an iterator
typeof aGeneratorObject[Symbol.iterator];
// "function", because it has an @@iterator method, so it's an iterable
aGeneratorObject[Symbol.iterator]() === aGeneratorObject;
// true, because its @@iterator method returns itself (an iterator), so it's an well-formed iterable
[...aGeneratorObject];
// [1, 2, 3]
```

Veja também

- Para mais informações sobre *generators* ES2015, consulte a documentação de function*.

Visão geral das tecnologias JavaScript

Enquanto o HTML é usado para armazenar o conteúdo e a formatação de uma página web e o CSS define a formatação e a aparência, o JavaScript é usado para adicionar interatividade a uma página web e criar aplicações web ricas.

No entanto, o termo genérico "JavaScript" tal como é entendido no contexto do navegador contém vários elementos bem diferentes. Um deles é a linguagem principal (ECMAScript), outra é a coleção das Web APIs, incluindo o DOM (Document Object Model).

JavaScript, a linguagem principal (ECMAScript)

O núcleo da linguagem JavaScript é padronizado pelo comitê ECMA TC39 como uma linguagem chamada ECMAScript. A última versão da especificação é ECMAScript 5.1.

Este núcleo da linguagem é também usado em ambientes fora do navegador, por exemplo em node.js.

O que se enquadra no escopo ECMAScript?

Entre outras coisas, o ECMAScript define:

- A sintaxe da linguagem (regras de análise, palavras-chave, controle de fluxo, inicialização de objetos literais...)
- Mecanismos para manuseio de erros (`throw`, `try...catch`, habilidade para criar tipos de `Error` definido pelo usuário)
- Tipos (*boolean*, *number*, *string*, *function*, *object*...)
- O objeto global. Num navegador esse objeto global é o objeto *window*, mas o ECMAScript apenas define as APIs, não especificamente para navegadores, por exemplo: *parseInt*, *parseFloat*, *decodeURI*, *encodeURI*...
- Um mecanismo de herança baseado em protótipo
- Objetos e funções embutidas (`JSON`, `Math`, Array, *Object introspection*...)
- Modo estrito (rigoroso)

Suporte do navegador

Desde Agosto de 2014, as atuais versões dos principais navegadores implementaram o ECMAScript 5.1 e o ECMAScript 2015, mas versões mais antigas continuam usando a implementação do ECMAScript 3 e apenas partes do ECMAScript 5. Navegadores mais modernos já implementaram grande parte do ECMAScript 6.

Futuro

A 6ª edição principal do ECMAScript foi oficialmente aprovada e publicada como norma em 17 de junho de 2015 pela Assembléia Geral da ECMA. Desde então, as Edições ECMAScript são publicadas anualmente.

API de Internacionalização

A API de Especificação de Internacionalização do ECMASCRIPT é uma adição para a Especificação de Linguagem ECMAScript, também padronizada pela Ecma TC39. A API de internacionalização fornece intercalação (comparação de string), formatação numérica, formatação de data e tempo para aplicações JavaScript, permitindo que aplicativos escolham o idioma e adaptem a funcionalidade às suas necessidades. O padrão foi aprovado em dezembro de 2012; O status das implementações nos navegadores é rastreado na documentação do objeto Intl. A especificação de Internacionalização hoje em dia também é ratificada anualmente e os navegadores aprimoram as suas implementações constantemente.

As APIs Web e o DOM

WebIDL

A Especificação da WebIDL provê a junção entre as tecnologias DOM e o ECMAScript.

O núcleo do DOM

O Document Object Model (DOM) é uma **convenção multiplataforma** e **Independente de linguagem** para representação e interação com objetos em documentos HTML, XHTML, e XML. Objetos no **DOM tree** (árvore do DOM) podem ser adereçados e manipulados utilizando métodos em objetos. A W3C padroniza o Core Document Object Model (Núcleo do Documento do Modelo de Objeto), o qual define interfaces de agnóstico de linguagem que abstraem documentos HTML e XML como objetos, e também define mecanismos para manipular essa abstração. Entre as definições do DOM, podemos encontrar:

- A estrutura do documento, um tree model (modelo de árvore), e a arquitetura do DOM Event (Evento do DOM) no DOM core (núceo do DOM): `Node`, `Element`, `DocumentFragment`, `Document`, `DOMImplementation`, `Event`, `EventTarget`, ...
- Uma definição menos rigorosa do DOM Event Architecture, assim como eventos específicos no DOM events.
- Outras coisas como o DOM Traversal e o DOM Range.

Pela perspectiva do EXMScript, objetos definidos na especificação do DOM são chamados de "host objects".

HTML DOM

HTML, a linguagem de marcação da Web, é especificada nos termos do DOM. Situado além dos conceitos de abstração definidos no DOM Core, o HTML também define o *significado* dos elementos. O HTML DOM inclui coisas como a propriedade `className` em elementos HTML, ou APIs como `document.body`.

A especificação HTML também define restrições nos objetos; por exemplo, ela requer que todos os filhos de um elemento `ul`, que representam uma lista desordenada, sejam elementos `li`, que representem itens de lista. Em linhas gerais, ele também proíbe o uso de elementos e atributos que não estão definidos em um padrão.

Procurando pelo `Document` object, `Window` object, e outros elementos DOM? Leia a Documentação do DOM.

Outras APIs de destaque

- As funções `setTimeout` e `setInterval` foram inicialmente especificadas na interface `Window` no Padrão HTML.
- XMLHttpRequest. Possibilita enviar requisições HTTP assíncronas.
- A Fetch API fornece uma abstração mais ergonômica para requisições de rede.
- CSS Object Model. O CSSOM é utilizado para abstrair regras CSS como objetos
- WebWorkers. API que permite computação paralela.
- WebSockets. API que permite comunicação bi-direcional de baixo-nível.
- Canvas 2D Context. API de ilustração (desenho) para o elemento canvas.
- A interface WebAssembly fornece utilitários para comunicação entre código JavaScript e módulos WebAssembly.

Ambientes sem navegador (como Node.js) geralmente não têm APIs DOM — porque não interagem com um documento — mas ainda assim implementam muitas APIs da Web, como `fetch()` e `setTimeout()`.

Suporte de Navegadores

Todo desenvolvedor já percebeu que o DOM é uma bagunça. A uniformidade no suporte dos Navegadores varia muito de feature para feature. A principal razão para essa situação é o fato de que muitas features importantes do DOM possuem (quando possuem) especificações não muito claras. Além disso, diferentes Navegadores adicionaram features incompatíveis para a sobreposição de casos de uso (como o event model do Internet Explorer). A atual (Junho de 2011) tendência é que a W3C e particularmente a WHATWG estão definindo features mais antigas em detalhes, a fim de melhorar a interoperabilidade. Seguindo esta tendência, Navegadores estão melhorando suas implementações baseados nessas especificações.

Uma abordagem comum, porém não a mais confiável, para compatibilidade cross-browser é utilizar a biblioteca JavaScript. Essas bibliotecas abstrem as features do DOM e garantem que suas API's funcionem de forma similar nos navegadores. Alguns dos frameworks mais utilizados são o jQuery, o prototype, e o YUI.

Gramática léxica

Esta página descreve a gramática léxica da linguagem JavaScript. O código fonte de um script ECMAScript é lido da esquerda para direita e traduzido em uma sequência de elementos de entrada, como símbolos, caracteres de controle, finalizadores de linha, comentários ou espaços em branco. ECMAScript também define certas palavras-chave e literais e possui regras para inserção automática de ponto e vírgula ao final de declarações.

Caracteres de controle

Caracteres de controle não possuem representação visual, mas são usados para controlar a interpretação do código.

Código	Nome	Abreviação	Descrição
`U+200C`	Zero width non-joiner	<ZWNJ>	Colocado entre caracteres para evitar que sejam visualmente ligados (fundidos) em algumas linguagens (Wikipedia).
`U+200D`	Zero width joiner	<ZWJ>	Colocado entre caracteres que podem não estar normalmente conectados, para que estes caracteres sejam renderizados com suas devidas ligações em certas linguagens (Wikipedia).
`U+FEFF`	Byte order mark	<BOM>	Usado no início do script para marcá-lo como Unicode e especificar a ordem de bytes do código (Wikipedia).

Espaço em branco

Espaços em branco ajudam na legibilidade do código e separam os símbolos um do outro. Estes caracteres normalmente não são funcionais no código. Ferramentas de minificação podem ser usadas para remover espaços em branco do código reduzindo, assim, a quantidade de dados que precisam ser transferidos.

Código	Nome	Abreviação	Descrição	Sequência de escape
U+0009	Character tabulation	<HT>	Tabulação horizontal	\t
U+000B	Line tabulation	<VT>	Tabulação vertical	\v
U+000C	Form feed	<FF>	Pular para próxima página (Wikipedia).	\f
U+0020	Space	<SP>	Espaço normal	
U+00A0	No-break space	<NBSP>	Espaço normal, sem marcar quebra de linha	
Others	Other Unicode space characters	<USP>	Outros tipos de espaço. Veja na Wikipedia	

Finalizadores de linha

Além dos caracteres de espaço em branco, caracteres finalizadores de linha são usados para melhorar a legibilidade do código. Contudo, em alguns casos, finalizadores de linha podem influenciar a execução do código JavaScript, por isso você não poderá usá-los em alguns casos. Eles também afetam o processo automático de inserção de ponto e vírgula. Em expressões regulares os finalizadores de linha podem ser indicados com a classe **\s**.

Apenas os seguintes códigos Unicode são tratados como finalizadores de linha no ECMAScript, outros caracteres de quebra de linha são tratados apenas como espaço em branco (por exemplo, Next Line, NEL, U+0085 são considerados espaço em branco).

Código	Nome	Abreviação	Descrião	Sequência de escape
U+000A	Line Feed	<LF>	Caractere de nova linha em sistemas UNIX.	\n
U+000D	Carriage Return	<CR>	Caractere de nova linha nos sistemas Commodore e nos primeiros Mac's.	\r
U+2028	Line Separator	<LS>	Wikipedia	
U+2029	Paragraph Separator	<PS>	Wikipedia	

Comentários

Comentários são usados para inserir dicas, notas, sugestões ou alertas no código JavaScript. Isto pode tornar mais fácil ler ou entender o que o código faz. Também pode ser usado para desabilitar código, evitando sua execução; isto pode ser uma valiosa ferramenta na hora de debugar seu código.

JavaScript possui duas formas de inserir comentário no código.

O primeiro é com o uso de duas barras `//`; isso faz com que qualquer coisa escrita a seguir, e na mesma linha, se torne comentário. Por exemplo:

```
function comment() {
  // Isto é um comentário de uma linha em JavaScript
  console.log("Hello world!");
}
comment();
```

A segunda forma é com o uso de `/* */`, que é muito mais flexível.

Por exemplo, você pode usá-lo em uma única linha:

```
function comment() {
  /* Isto é um comentário de uma linha em JavaScript */
  console.log("Hello world!");
}
comment();
```

Ou pode fazer um comentário de várias linhas, desta forma:

```
function comment() {
  /* Este é um comentário de várias linhas. Observe que
 não precisamos encerrar o comentário até terminar. */
  console.log("Hello world!");
}
comment();
```

Você também pode usá-lo no meio de uma linha, se quiser, porém isto pode tornar seu código mais difícil de ler, portanto use com cautela:

```
function comment(x) {
  console.log("Hello " + x /* insira o valor de x */ + " !");
}
comment("world");
```

Além disso, você pode usá-lo para desativar algum código para evitar que seja executado, envolvendo o código em um comentário, assim:

```
function comment() {
  /* console.log('Hello world!'); */
}
comment();
```

Neste caso, a função `console.log()` nunca será invocada, pois está comentada. Qualquer quantidade de linhas de código podem ser desabilitadas desta forma.

Palavras-chave

Palavras-chave reservadas do ECMAScript 2015

- `break`
- `case`
- `catch`
- `class`
- `const`
- `continue`
- `debugger`
- `default`
- `delete`
- `do`
- `else`
- `export`
- `extends`
- `finally`
- `for`
- `function`
- `if`
- `import`
- `in`
- `instanceof`
- `new`
- `return`
- `super`
- `switch`
- `this`
- `throw`
- `try`
- `typeof`
- `var`
- `void`
- `while`
- `with`
- `yield`

Palavras-chave reservadas para uso futuro

As seguintes palavras-chave estão reservadas para uso futuro na especificação ECMAScript. Atualmente elas não tem funcionalidade, mas poderão ter no futuro, portanto elas também não podem ser usadas como identificadores.

Estes sempre são reservados:

- `enum`

Os seguintes são reservados apenas quando o código está em modo estrito (*strict mode*):

- `implements`
- `interface`
- `let`
- `package`

- `private`
- `protected`
- `public`
- `static`

Os seguintes são reservados apenas quando são encontrados no código do módulo:

- `await`

Palavras-chave reservadas para uso futuro em versões antigas

As seguintes palavras-chave foram reservadas para uso futuro em especificações antigas do ECMAScript (ECMAScript 1 até 3).

- `abstract`
- `boolean`
- `byte`
- `char`
- `double`
- `final`
- `float`
- `goto`
- `int`
- `long`
- `native`
- `short`
- `synchronized`
- `throws`
- `transient`
- `volatile`

Além disso, os literais `null`, `true`, e `false` não podem ser usados como identificadores em ECMAScript.

Uso de palavras-chave reservadas

Palavras reservadas atualmente só não podem ser usadas como identificadores . Como descrito em es5.github.com/#A.1, não há problema em usar palavras reservadas desta forma.

```
a.import
a['import']
a = { import: 'test' }.
```

Por outro lado, o seguinte é ilegal por quê a palavra reservada "*import*" está sendo usada como identificador. Identificadores podem ser usados em declaração de expressões e funções.

```
function import() {} // Proibido.
```

Literais

Null

Veja também `null` para mais informações.

```
null;
```

Booleanos

Veja também `Boolean` para mais informações.

```
true;
false;
```

Números

Decimal

```
1234567890;
42;

// Cuidado com o uso precedido de zero:
0888; // 888 convertido como decimal
0777; // Convertido como octal, 511 em decimal
```

Veja que números decimais podem começar com zero (`0`) seguido de outro dígito decimal, porém se todos dígitos após o zero inicial forem menores que 8, o número é interpretado como um número octal. Isso não vai disparar um erro em JavaScript, veja Erro do Firefox 957513. Veja também esta página sobre conversão de tipo `parseInt()`.

Binário

Números binários em JavaScript são iniciados em zero seguido da letra "B" em maiúsculo ou minúsculo (`0b` ou `0B`). Esta sintaxe é nova no ECMAScript 2015, por isso verifique a compatibilidade do navegador. Se o dígito seguinte ao `0b` não for 0 ou 1, o seguinte erro de sintaxe é disparado: "Missing binary digits after 0b".

```
var FLT_SIGNBIT = 0b10000000000000000000000000000000; // 2147483648
var FLT_EXPONENT = 0b01111111100000000000000000000000; // 2139095040
var FLT_MANTISSA = 0b00000000011111111111111111111111; // 8388607
```

Octal

Números octais em JavaScript são escritos com zero inicial seguido da letra "O" em maiúsculo ou minúsculo (`0o` ou `0O`). Esta sintaxe é nova no ECMAScript 2015, por isso verifique a compatibilidade do navegador. Se os dígitos após o `0o` não forem octais válidos (01234567), o seguinte erro de sintaxe será disparado: "Missing octal digits after 0o".

```
var n = 0o755; // 493
var m = 0o644; // 420

// Também é possível com apenas um zero no início (veja sobre decimais acima)
0755;
0644;
```

Hexadecimal

Números hexadecimais em JavaScript são iniciados com zero seguido da letra "X" em maiúsculo ou minúsculo (`0x` ou `0X`). Se os dígitos após o 0x não forem hexadecimais válidos (0123456789ABCDEF), o seguinte erro de sintaxe será disparado: "Identifier starts immediately after numeric literal".

```
0xffffffffffffffff; // 295147905179352830000
0x123456789abcdef; // 81985529216486900
0xa; // 10
```

Objetos

Veja também `Object` e Object initializer para mais informações.

```
var o = { a: "foo", b: "bar", c: 42 };

// Notação curta. Novidade no ES2015
var a = "foo",
  b = "bar",
  c = 42;
var o = { a, b, c };

// instead of
var o = { a: a, b: b, c: c };
```

Arrays

Veja também `Array` para mais informações.

```
[1954, 1974, 1990, 2014];
```

Strings

```
'foo';
"bar";
```

Caracteres de escape em hexadecimal

```
"\xA9"; // "©"
```

Caracteres de escape em Unicode

A representação de caracteres de escape em Unicode requer ao menos quatro caracteres após o `\u`.

```
"\u00A9"; // "©"
```

Codificação de caracteres Unicode

Novidade no ECMAScript 2015. Com a codificação de caracteres Unicode qualquer caractere pode ser representado usando sequência hexadecimal, desta forma podemos representar caracteres Unicode com sequência hexadecimal até `0x10FFFF`. Usando apenas Unicode, muitas vezes pode ser necessário utilizar mais de uma sequência para conseguir o mesmo resultado.

Veja também `String.fromCodePoint()` ou `String.prototype.codePointAt()`.

```
"\u{2F804}";

// O mesmo usando apenas Unicode
"\uD87E\uDC04";
```

Expressões regulares

Veja também `RegExp` para mais informações.

```
/ab+c/g

// Um "espaço vazio" em expressão regular
// Para evitar confusão com comentário de linha única
// faça o seguinte:
/(?:)/
```

Templates

Veja também template strings para mais informações .

```
`texto`;

`linha de texto 1
 linha de texto 2`;

`texto ${expressão} texto`;

tag`texto ${expressão} texto`;
```

Inserção automática de ponto e vírgula

Algumas declarações JavaScript precisam ser finalizadas com ponto e vírgula e, por isso, são afetadas pelo mecanismo automático de inserção de ponto evírgula (na silga ASI, em inglês):

- Declaração vazia
- `let`, `const`, declaração de variável
- `import`, `export`, declaração module
- Expressões
- `debugger`
- `continue`, `break`, `throw`
- `return`

A especificação ECMAScript menciona três regras para inserção de ponto e vírgula.

1. Um ponto e vírgula é inserido antes, quando um finalizador de linha ou um "}" é encontrado em local não permitido pela sintaxe.

```
{ 1 2 } 3

// É transformado pelo ASI em

{ 1 2 ;} 3;
```

2. Um ponto e vírgula é inserido ao final, quando o sinal de final de linha é detectado e o conversor não é capaz de unir a linha seguinte em um único comando.

Aqui o `++ não é interpretado como operador de incremento aplicado à variável b por que há um finalizador de linha entre o b e ++ .`

```
a = b
++c

// É transformado pelo ASI em

a = b;
++c;
```

3. Um ponto e vírgula é inserido ao final quando uma declaração com regra léxica restrita é encontrada seguida de um finalizador de linha. As declarações sem regra específica de finalizador de linha são:

- Operadores de incremento ou decremento (`++` and `--`)
- `continue`
- `break`
- `return`
- `yield`, `yield*`
- `module`

```
return
a + b

// É transformado pelo ASI em

return;
a + b;
```

Firefox-notas específicas

- Em versões anteriores ao Firefox 5 (JavaScript 1.8.6) palavras reservadas para uso futuro não pdem ser usadas em strict mode. Essa falha de ECMAScript foi corrigida no Firefox 5.

Veja também

- Jeff Walden: Binary and octal numbers
- Mathias Bynens: JavaScript character escape sequences
- `Boolean`
- `Number`
- `RegExp`
- `String`

Operadores

Este capítulo documenta todos os operadores, expressões e keywords da linguagem JavaScript .

Expressões e operadores por categoria

Para uma lista alfabética ver a barra lateral à esquerda.

Expressões primárias

Palavras-chave e expressões básicas em JavaScript.

`this`

: A palavra-chave `this` refere-se ao contexto de execução da função.

`function`

: A palavra-chave `function` define uma função.

`class`

: A palavra-chave `class` define uma classe.

`function*`

: A palavra-chave `function*` define um gerador de função.

`yield`

: Pausa e retorma uma função de gerador

`yield*`

: Delegar a outra função gerador ou objeto iterável.

`[]`

: Array initializer/literal syntax.

`{}`

: Object initializer/literal syntax.

`/ab+c/i`

: Regular expression literal syntax.

`[for (x of y) x]`

: Array comprehensions.

`(for (x of y) y)`

: Generator comprehensions.

`( )`

: Grouping operator.

Left-hand-side expressions

Left values are the destination of an assignment.

Property accessors

: Member operators provide access to a property or method of an object (`object.property` and `object["property"]`).

`new`

: The `new` operator creates an instance of a constructor.

`super`

: The `super` keyword calls the parent constructor.

`...obj`

The spread operator allows an expression to be expanded in places where multiple arguments (for function calls) or multiple elements (for array literals) are expected.

Incremento e decremento

Operadores sufixo/prefixo para incremento e sufixo/prefixo para decremento.

`A++`

Operador sufixo para incremento.

`A--`

Operador sufixo para decremento.

`++A`

Operador prefixo para incremento.

`--A`

Operador prefixo para decremento.

Unary operators

A unary operation is operation with only one operand.

`delete`

The `delete` operator deletes a property from an object.

`void`

The `void` operator discards an expression's return value.

`typeof`

The `typeof` operator determines the type of a given object.

`+`

The unary plus operator converts its operand to Number type.

`-`

The unary negation operator converts its operand to Number type and then negates it.

`~`

Bitwise NOT operator.

`!`

Logical NOT operator.

Operadores aritméticos

Arithmetic operators take numerical values (either literals or variables) as their operands and return a single numerical value.

`+`

Operador de soma.

`-`

Operador de subtração.

`/`

Operador de divisão.

`*`

Operador de multiplicação.

`%`

Remainder operator.

Operadores relacionais

A comparison operator compares its operands and returns a `Boolean` value based on whether the comparison is true.

`in`

The `in` operator determines whether an object has a given property.

`instanceof`

: The `instanceof` operator determines whether an object is an instance of another object.

`<`

: Operador "menor que".

`>`

: Operador "maior que".

`<=`

: Operador "menor ou igual que".

`>=`

: Operador "maior ou igual que".

Operadores de igualdade

O resultador de um operador de igualdade é do tipo `Booleano` baseado que a comparação seja verdadeira.

`==`

: Operadores de igualdade.

`!=`

: Inequality operator.

`===`

: Identity operator.

`!==`

: Nonidentity operator.

Bitwise shift operators

Operations to shift all bits of the operand.

`<<`

: Bitwise left shift operator.

`>>`

: Bitwise right shift operator.

`>>>`

: Bitwise unsigned right shift operator.

Binary bitwise operators

Bitwise operators treat their operands as a set of 32 bits (zeros and ones) and return standard JavaScript numerical values.

`&`

: Bitwise AND.

`|`

: Bitwise OR.

`^`

: Bitwise XOR.

Operadores Lógicos Binários

Operadores lógicos são normalmente usados com boolean (logical) valores, e quando eles são, eles retornam um valor Boolean.

`&&`

: AND.

`||`

: OR.

Operador de Condicional (ternário)

```
(condition ? ifTrue : ifFalse)
```

O operador condicional retorna um dos dois valores baseado no valor lógico da condição.

Assignment operators

An assignment operator assigns a value to its left operand based on the value of its right operand.

`=`

Assignment operator.

`*=`

Multiplication assignment.

`/=`

Division assignment.

`%=`

Remainder assignment.

`+=`

Addition assignment.

`-=`

Subtraction assignment

`<<=`

Left shift assignment.

`>>=`

Right shift assignment.

`>>>=`

Unsigned right shift assignment.

`&=`

Bitwise AND assignment.

`^=`

Bitwise XOR assignment.

`|=`

Bitwise OR assignment.

`[a, b] = [1, 2]` `{a, b} = {a:1, b:2}`

Destructuring assignment allows you to assign the properties of an array or object to variables using syntax that looks similar to array or object literals.

Comma operator

`,`

The comma operator allows multiple expressions to be evaluated in a single statement and returns the result of the last expression.

Non-standard features

Legacy generator function

The `function` keyword can be used to define a legacy generator function inside an expression. To make the function a legacy generator, the function body should contains at least one `yield` expression.

Expression closures

The expression closure syntax is a shorthand for writing simple function.

Veja também

- Operator precedence

Adição (+)

O operador de adição (+) produz a soma de operandos numéricos ou concatenação de strings.

Experimente

```
console.log(2 + 2);
// Expected output: 4

console.log(2 + true);
// Expected output: 3

console.log("hello " + "everyone");
// Expected output: "hello everyone"

console.log(2001 + ": A Space Odyssey");
// Expected output: "2001: A Space Odyssey"
```

Sintaxe

```
x + y
```

Descrição

O operador de adição está encarregado para duas operações distintas, adição numérica e concatenação de Strings. Ao avaliar, ele primeiro coage ambos os operandos para primitivos chamando a função do objeto `[@@toPrimitive]()`()(com "default" como dica), e os métodos valueOf(), e toString(), nessa ordem. Em seguida, são testados os tipos dos dois operandos:

- Se um lado é uma string, o outro operando também é convertido em uma string e eles são concatenados.
- Se ambos forem BigInts, a adição de BigInt será executada. Se um lado for um BigInt, mas o outro não, um `TypeError` será lançado.
- Caso contrário, ambos os lados são convertidos em números e a adição numérica é executada.

A concatenação de strings geralmente é considerada equivalente a literais de modelo ou String.prototype.concat(), mas não são. A adição força a expressão a uma chamada *primitive* , que chama `valueOf()` em prioridade; por outro lado, literais de modelo e `concat()` converte a expressão para uma string, que chama `toString()` em prioridade. Se a expressão tiver o método `@@toPrimitive`, a concatenação de strings o chama com "default" como dica, enquanto literais de modelo usam "string". Isso é importante para objetos que têm diferentes representações de string e primitivas — como Temporal, cujo método `valueOf()` lança.

```
const t = Temporal.Now.instant();
"" + t; // Throws TypeError
`${t}`; // '2022-07-31T04:48:56.113918308Z'
"".concat(t); // '2022-07-31T04:48:56.113918308Z'
```

Você é aconselhado a não usar "" + x para executar a coerção de string.

Exemplos

Adição numérica

```
// Number + Number -> adição
1 + 2; // 3
// Boolean + Number -> adição
true + 1; // 2
// Boolean + Boolean -> adição
false + false; // 0
```

Concatenação de String

```
// String + String -> concatenação
"foo" + "bar"; // "foobar"
// Number + String -> concatenação
5 + "foo"; // "5foo"
// String + Boolean -> concatenação
"foo" + false; // "foofalse"
```

Veja também

- Operador de subtração
- Operador de divisão
- Operador de multiplicação
- Operador restante
- Operador de exponenciação
- Operador de incremento
- Operador de decremento
- Operador de negação unário
- Operador unário plus

Expressão da função async

A palavra chave async pode ser usado para definir funções async dentro das expressões.Você tambem pode definir funções async usando a declaração de função async.

Sintaxe

```
async function [nome]([parametro1[, parametro2[, ..., parametroN]]]) {
    declarações
}
```

Como no ES2015, você tambem pode usar arrow functions.

Parâmetros

`nome`

O nome da função.Pode ser omitida, na qual a função se torna anonima . O nome é somente local para o corpo da função.

`parametroN`

O nome do argumento passado para a função.

`declarações`

As declarações que compoem o corpo da função .

Descrição

Uma expressão `async function` é muito similar, e tem quase a mesma sintaxe de uma `declaração async function`. A principal diferença entre uma expressão `async function` e uma declaração `async function` é o *nome da função*, que pode ser omitido em expressões `async function` para criar *funções anônimas*. Uma expressão `async function` pode ser utilizada como um IIFE (Immediately Invoked Function Expression, em tradução livre: Expressão de Função Invocada Imediatamente) que roda assim que definido. Veja também o capítulo sobre funções para mais informações.

Exemplos

Exemplo simples

```
function resolveAfter2Seconds(x) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

(async function (x) {
  // async function expression usada como uma IIFE
  var a = resolveAfter2Seconds(20);
  var b = resolveAfter2Seconds(30);
  return x + (await a) + (await b);
})(10).then((v) => {
  console.log(v); // imprime 60 após 2 segundo.
});

var add = async function (x) {
  // async function expression atribuída a uma variável
  var a = await resolveAfter2Seconds(20);
  var b = await resolveAfter2Seconds(30);
  return x + a + b;
};

add(10).then((v) => {
  console.log(v); // imprime 60 após 4 segundos.
});
```

Veja também

- `async function`
- `AsyncFunction` object
- `await`

await

O operador `await` é utilizado para esperar por uma `Promise`. Ele pode ser usado apenas dentro de uma `async function`.

Sintaxe

```
[rv] = await expressão;
```

`expressão`

Uma `Promise` ou qualquer valor para esperar uma resolução.

`rv`

Retorna um valor final da promise, ou o próprio valor se ele não for uma `Promise`.

Descrição

A expressão `await` faz a execução de uma função `async` pausar, para esperar pelo retorno da `Promise`, e resume a execução da função `async` quando o valor da Promise é resolvido. Ele então retorna o valor final da Promise. Se esse valor não for uma `Promise`, ele é convertido para uma `Promise` resolvida.

Se a `Promise` for rejeitada, a expressão `await` invoca uma Exception com o valor rejeitado.

Exemplos

Se uma `Promise` é passada para uma expressão `await`, ele espera pela sefinalização da `Promise` e retorna seu valor final.

```
function resolveAfter2Seconds(x) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

async function f1() {
  var x = await resolveAfter2Seconds(10);
  console.log(x); // 10
}
f1();
```

Se o valor não for uma `Promise`, ele converte o valor para uma `Promise` resolvida, e espera por ele.

```
async function f2() {
  var y = await 20;
  console.log(y); // 20
}
f2();
```

Se a `Promise` for rejeitada, o valor rejeitado é invocado em uma Exception.

```
async function f3() {
  try {
    var z = await Promise.reject(30);
  } catch (e) {
    console.log(e); // 30
  }
}
f3();
```

Veja também

- `async function`
- `async function expression`
- `AsyncFunction` object

Bitwise AND (&)

O operador bitwise AND (`&`) retorna `1` em cada posição do bit em que os bits corrrespondentes de ambos os operandos são `1`.

Experimente

```
const a = 5; // 00000000000000000000000000000101
const b = 3; // 00000000000000000000000000000011

console.log(a & b); // 00000000000000000000000000000001
// Expected output: 1
```

Sintaxe

```
a & b
```

Descrição

Os operandos são convertidos em inteiros de 32 bits e expressos por uma série de bits (zeros e uns). Números com mais de 32 bits têm seus bits mais significativos descartados. Por exemplo, o seguinte inteiro com mais de 32 bits será convertido em um de 32 bits inteiro:

```
Antes: 11100110111110100000000000000110000000000001
Depois:            10100000000000000110000000000001
```

Cada bit no primeiro operando é emparelhado com o bit correspondente no segundo operando: *primeiro bit* para o *primeiro bit* , *segundo bit* para o *segundo bit*, e assim por diante.

O operador é aplicado a cada par de bits e o resultado é construído em bitwase.

A tabela verdade para a operação AND é:

a	b	a AND b
0	0	0
0	1	0
1	0	0
1	1	1

```
     9 (base 10) = 00000000000000000000000000001001 (base 2)
    14 (base 10) = 00000000000000000000000000001110 (base 2)
                   --------------------------------
14 & 9 (base 10) = 00000000000000000000000000001000 (base 2) = 8 (base 10)
```

A operação AND bit a bit (Bitwase) de qualquer número `x` com `0` produz `0` .

Exemplos

Usando bitwase AND

```
//  5: 00000000000000000000000000000101
//  2: 00000000000000000000000000000010
5  e  2 ; // 0
```

Veja também

- Operadores Bitwise no JS guide
- Operador de atribuição Bitwise AND

Bitwise XOR (^)

O operador bitwise XOR (^) retorna o numero 1 em cada posição de bit para a qual os bits correspondentes de ambos, mas não de ambos os operandos, são `1`s.

Experimente

```
const a = 5; // 00000000000000000000000000000101
const b = 3; // 00000000000000000000000000000011

console.log(a ^ b); // 00000000000000000000000000000110
// Expected output: 6
```

Sintaxe

```
a ^ b
```

Descrição

Os operandos são convertidos em números inteiros de 32 bits e expressados por uma série de bits (zeros e uns). Numeros com mais de 32 bits descartam seus bits mais significativos. Por exemplo, o número inteiro a seguir com mais de 32 bits será convertido em um número inteiro de 32 bits:

```
Antes: 11100110111110100000000000000110000000000001
Depois:            10100000000000000110000000000001
```

Cada bit no primeiro operando é emparelhado com o bit correspondente no segundo operando: primeiro bit para o primeiro bit, segundo bit para o segundo bit e assim por diante.

O operador é aplicado para cada par de bits e o resultado é construído em bitwise.

A tabela verdade para a operação XOR é:

a	b	a XOR b
0	0	0
0	1	1
1	0	1
1	1	0

```
.    9 (base 10) = 00000000000000000000000000001001 (base 2)
    14 (base 10) = 00000000000000000000000000001110 (base 2)
                   --------------------------------
14 ^ 9 (base 10) = 00000000000000000000000000000111 (base 2) = 7 (base 10)
```

Bitwise XORing any number `x` with `0` yields `x`.

Exemplos

Usando bitwise XOR

```
// 9  (00000000000000000000000000001001)
// 14 (00000000000000000000000000001110)

14 ^ 9;
// 7  (00000000000000000000000000000111)
```

Leia também

- Bitwise operators in the JS guide
- Bitwise XOR assignment operator

class expression

A **expressão class** é uma das formas de definir uma classe no ECMAScript 2015. Parecido com as expressões function, expressões class poder ser nomeadas ou não. Se nomeada, o nome da classe é local para apenas o corpo da classe. Classes no JavaScript usam herança com base no prototype

Sintaxe

```
var MinhaClasse = class [nomeDaClass] [extends] {
  // corpo da classe
};
```

Descrição

Uma expressão de classe tem uma sintaxe similar a uma declaração de classe. No entanto, com expressões de classe, você poderá omitir o nome da classe ("ligação de identificadores"), que você não pode se usar declarações. Adicionalmente, expressões de classe permitem que você redefina/redeclare classes e **não invocar** nenhum tipo de erros como declaração de classe. A propriedade construtora é opcional. E o *typeof* de classes geradas usando essa palavra-chave sempre será "function".

Assim como declarações de classes, o corpo das expressões de classe são executados em strict mode.

```
"use strict";
var Foo = class {}; // propriedade de construtor é opcional
var Foo = class {}; // Re-declaração é permitida

typeof Foo; //retorna "function"
typeof class {}; //retorna "function"

Foo instanceof Object; // true
Foo instanceof Function; // true
class Foo {} // Throws TypeError, não permite re-declaração
```

Exemplos

Uma simples expressão de classe

Isso é apenas uma simples classe anônima que você pode referir usando a variável "Foo".

```
var Foo = class {
  constructor() {}
  bar() {
    return "Olá mundo!";
  }
};

var instance = new Foo();
instance.bar(); // "Olá mundo!"
Foo.name; // "Foo"
```

Expressões de classes nomeadas

Se você quiser referir a classe atual dentro do corpo da classe, você pode criar uma expressão de classe nomeada. Esse nome é visível apenas no escopo da expressão classe sozinha.

```
var Foo = class NamedFoo {
  constructor() {}
  whoIsThere() {
    return NamedFoo.name;
  }
};
var bar = new Foo();
bar.whoIsThere(); // "NamedFoo"
NamedFoo.name; // ReferenceError: NamedFoo is not defined
Foo.name; // "NamedFoo"
```

Veja também

- Expressão `function`
- Declaração `class`
- Classes

Operador Vírgula

Sumário

o **operador vírgula** avalia o valor de seus operandos (da esquerda para a direita) e retorna o valor do último operando.

Sintaxe

```
expr1, expr2, expr3...
```

Parameters

`expr1`, `expr2, expr3...`

Quaisquer expressões.

Descrição

Você pode usar o operador vírgula quando desejar incluir múltiplas expressões em um lugar que requer uma única expressão. O uso mais comum desse operador é suprir múltiplos parâmetros em um loop `for`.

Exemplo

Se `a` é um array de 2 dimensões com 10 elementos de um lado, o seguinte código usa o operador vírgula para incrementar duas variáveis mutuamente. Note que a vírgula na declaração `var` ***não*** é o operador vírgula, porque ele não existe dentro de uma expressão. Além disso, ela é uma caractere especial nas declarações `var` para combinar múltiplas delas em uma única. Embora praticamente a vírgula comporte-se quase que igualmente ao operador vírgula. O código imprime os valores dos elementos diagonais da matriz:

```
for (var i = 0, j = 9; i <= 9; i++, j--)
  document.writeln("a[" + i + "][" + j + "] = " + a[i][j]);
```

Veja também

- for loop

Operador Condicional Ternário

Sumário

O **operador condicional (ternário)** é o único operador JavaScript que possui três operandos. Este operador é frequentemente usado como um atalho para a instrução `if`.

Sintaxe

```
condition ? expr1 : expr2
```

Parâmetros

`condition`

Uma expressão que é avaliada como `true` ou `false`.

`expr1`, `expr2`

Expressões com valores de qualquer tipo.

Descrição

Se `condition` é `true`, o operador retornará o valor de `expr1`; se não, ele retorna o valor de `exp2`. Por exemplo, para exibir uma mensagem diferente baseada no valor da variável `isMember`, você poderá utilizar o código (statement) seguinte:

```
"The fee is " + (isMember ? "$2.00" : "$10.00");
```

Conforme o resultado da operação, você também poderá atribuir a variáveis:

```
var elvisLives = Math.PI > 4 ? "Yep" : "Nope";
```

Também são possíveis múltiplas avaliaçãoes ternárias (nota: o operador condicional é associativo a direita):

```
var firstCheck = false,
  secondCheck = false,
  access = firstCheck
    ? "Access denied"
    : secondCheck
      ? "Access denied"
      : "Access granted";

console.log(access); // logs "Access granted"
```

Você também pode usar avaliações ternárias no espaço livre de modo a fazer diferentes operações:

```
var stop = false,
  age = 16;

age > 18 ? location.assign("continue.html") : (stop = true);
```

Você também pode fazer mais do que uma única operação em cada caso, separando-os por vírgula:

```
var stop = false,
  age = 23;

age > 18
  ? (alert("OK, you can go."), location.assign("continue.html"))
  : ((stop = true), alert("Sorry, you are much too young!"));
```

Você também pode fazer mais de uma operação durante a atribuição de um valor. Neste caso, **o último valor separado por vírgula dentro** dos parênteses **será o valor a ser atribuído**.

```
var age = 16;

var url =
  age > 18
    ? (alert("OK, you can go."),
      // alert returns "undefined", but it will be ignored because
      // isn't the last comma-separated value of the parenthesis
      "continue.html") // the value to be assigned if age > 18
    : (alert("You are much too young!"),
      alert("Sorry :-("),
      // etc. etc.
      "stop.html"); // the value to be assigned if !(age > 18)

location.assign(url); // "stop.html"
```

Veja também

- Operador condicional if

Atribuição via desestruturação (destructuring assignment)

A sintaxe de **atribuição via desestruturação (destructuring assignment)** é uma expressão JavaScript que possibilita extrair dados de arrays ou objetos em variáveis distintas.

Sintaxe

```
var a, b, rest;
[a, b] = [1, 2];
console.log(a); // 1
console.log(b); // 2

[a, b, ...rest] = [1, 2, 3, 4, 5];
console.log(a); // 1
console.log(b); // 2
console.log(rest); // [3, 4, 5]

({ a, b } = { a: 1, b: 2 });
console.log(a); // 1
console.log(b); // 2

// ES2016 - não implementado em Firefox 47a01
({ a, b, ...rest } = { a: 1, b: 2, c: 3, d: 4 });
```

Descrição

As expressões de objeto e matriz literais fornecem uma maneira fácil de criar pacotes *ad hoc* de dados .

```
var x = [1, 2, 3, 4, 5];
```

A atribuição via desestruturação usa sintaxe similar, mas no lado esquerdo da atribuição são definidos quais elementos devem ser extraídos da variável de origem.

```
var x = [1, 2, 3, 4, 5];
var [y, z] = x;
console.log(y); // 1
console.log(z); // 2
```

Esse recurso é semelhante aos recursos presentes em linguagens como Perl e Python.

Desestruturação de array

Atribuição básica de variável

```
var foo = ["one", "two", "three"];

var [one, two, three] = foo;
console.log(one); // "one"
console.log(two); // "two"
console.log(three); // "three"
```

Atribuição separada da declaração

Uma variável pode ter seu valor atribuído via desestruturação separadamente da declaração dela.

```
var a, b;

[a, b] = [1, 2];
console.log(a); // 1
console.log(b); // 2
```

Valores padrão

Uma variável pode ser atribuída de um padrão, no caso em que o valor retirado do array é undefined.

```
var a, b;

[a = 5, b = 7] = [1];
console.log(a); // 1
console.log(b); // 7
```

Trocando variáveis

Os valores de duas variáveis podem ser trocados em uma expressão de desestruturação.

Sem atribuição via desestruturação, trocar dois valores requer uma variável temporária (ou, em algumas linguagens de baixo nível, o Algoritmo XOR Swap).

```
var a = 1;
var b = 3;

[a, b] = [b, a];
console.log(a); // 3
console.log(b); // 1
```

Analisando um array retornado de uma função

Sempre foi possível retornar uma matriz de uma função. A desestruturação pode tornar mais conciso o trabalho com um valor de retorno do tipo array.

Neste exemplo, `f()` retorna os valores `[1, 2]` como saída, que podem ser analisados em uma única linha com desestruturação.

```
function f() {
  return [1, 2];
}

var a, b;
[a, b] = f();
console.log(a); // 1
console.log(b); // 2
```

Ignorando alguns valores retornados

Você pode ignorar valores retornados que você não tem interesse:

```
function f() {
  return [1, 2, 3];
}

var [a, , b] = f();
console.log(a); // 1
console.log(b); // 3
```

Você também pode ignorar todos os valores retornados:

```
[, ,] = f();
```

Atribuindo o resto de um array para uma variável

Ao desestruturar um array, você pode atribuir a parte restante deste em uma viáriável usando o padrão rest:

```
var [a, ...b] = [1, 2, 3];
console.log(a); // 1
console.log(b); // [2, 3]
```

Extraindo valores do resultado de uma expressão regular

Quando o método de expressão regular `exec()` encontra um resultado, ele retorna um array que contém primeiro toda a porção resultante da string e depois cada uma das porções da string resultante envolvidas por parênteses na expressão regular. A atribuição via desestruturação lhe permite extrair as partes desses array facilmente, ignorando a porção resultante completa se não precisar.

```
var url = "https://developer.mozilla.org/pt-BR/Web/JavaScript";

var parsedURL = /^(\w+)\:\/\/([^\/]+)\/(.*)$/.exec(url);
console.log(parsedURL); // ["https://developer.mozilla.org/pt-BR/Web/JavaScript", "https", "developer.mozilla.org", "en-US/Web/JavaScript"]

var [, protocol, fullhost, fullpath] = parsedURL;

console.log(protocol); // "https"
```

Desestruturação de objeto

Atribuição básica

```
var o = { p: 42, q: true };
var { p, q } = o;

console.log(p); // 42
console.log(q); // true
```

Atribuição sem declaração

Uma variável pode ter seu valor atribuído via desestruturação separadamente da sua declaração.

```
var a, b;

({ a, b } = { a: 1, b: 2 });
```

> **Nota:** Os parênteses `( ... )` ao redor da declaração de atribuição é uma sintaxe necessária quando se utiliza a atribuição via desestruturação de objeto literal sem uma declaração.
>
> `{a, b} = {a:1, b:2}` não é uma sintaxe stand-alone válida, pois `{a, b}` no lado esquerdo é considarada um bloco, não um objeto literal.
>
> No entanto, `({a, b} = {a:1, b:2})` é valida, assim como `var {a, b} = {a:1, b:2}`

Atribuição para variáveis com novos nomes

Uma variável pode ser extraída de um objeto e atribuída a uma variável com um nome diferente da propriedade do objeto.

```
var o = { p: 42, q: true };
var { p: foo, q: bar } = o;

console.log(foo); // 42
console.log(bar); // true
```

Valores padrão

Uma variável pode ser atribuída de um padrão, no caso em que o valor retirado do objeto é undefined.

```
var { a = 10, b = 5 } = { a: 3 };

console.log(a); // 3
console.log(b); // 5
```

Definindo um valor padrão de parâmetro de função

Versão ES5

```
function drawES5Chart(options) {
  options = options === undefined ? {} : options;
  var size = options.size === undefined ? "big" : options.size;
  var cords = options.cords === undefined ? { x: 0, y: 0 } : options.cords;
  var radius = options.radius === undefined ? 25 : options.radius;
  console.log(size, cords, radius);
  // now finally do some chart drawing
}

drawES5Chart({
  cords: { x: 18, y: 30 },
  radius: 30,
});
```

Versão ES2015

```
function drawES2015Chart({
  size = "big",
  cords = { x: 0, y: 0 },
  radius = 25,
} = {}) {
  console.log(size, cords, radius);
  // do some chart drawing
}

drawES2015Chart({
  cords: { x: 18, y: 30 },
  radius: 30,
});
```

Objeto aninhado e desestruturação de array

```
var metadata = {
  title: "Scratchpad",
  translations: [
    {
```

```
      locale: "de",
      localization_tags: [],
      last_edit: "2014-04-14T08:43:37",
      url: "/de/docs/Tools/Scratchpad",
      title: "JavaScript-Umgebung",
    },
  ],
  url: "/pt-BR/docs/Tools/Scratchpad",
};

var {
  title: englishTitle,
  translations: [{ title: localeTitle }],
} = metadata;

console.log(englishTitle); // "Scratchpad"
console.log(localeTitle); // "JavaScript-Umgebung"
```

For de iteração e desestruturação

```
var people = [
  {
    name: "Mike Smith",
    family: {
      mother: "Jane Smith",
      father: "Harry Smith",
      sister: "Samantha Smith",
    },
    age: 35,
  },
  {
    name: "Tom Jones",
    family: {
      mother: "Norah Jones",
      father: "Richard Jones",
      brother: "Howard Jones",
    },
    age: 25,
  },
];

for (var {
  name: n,
  family: { father: f },
} of people) {
  console.log("Name: " + n + ", Father: " + f);
}

// "Name: Mike Smith, Father: Harry Smith"
// "Name: Tom Jones, Father: Richard Jones"
```

Extraindo campos de objetos passados como parâmetro de função

```
function userId({ id }) {
  return id;
}

function whois({ displayName: displayName, fullName: { firstName: name } }) {
  console.log(displayName + " is " + name);
}

var user = {
  id: 42,
  displayName: "jdoe",
  fullName: {
    firstName: "John",
    lastName: "Doe",
  },
};

console.log("userId: " + userId(user)); // "userId: 42"
whois(user); // "jdoe is John"
```

Isso extrai o `id`, `displayName` e `firstName` do objeto `user` e os imprime na tela.

Nomes computados de propriedade de objeto e desestruturação

Nomes computados de propriedades, como em objetos literais, podem ser usados com desestruturação.

```
let key = "z";
let { [key]: foo } = { z: "bar" };

console.log(foo); // "bar"
```

Notas específicas do Firefox

- O Firefox forneceu uma extensão não-padronizada de linguagem em JS1.7 para desestruturação. Esta extensão foi removida no Gecko 40. Consulte Erro do Firefox 1083498.
- A partir do Gecko 41 e para cumprir com a especificação ES2015, padrões de desestruturação com parênteses, como `([a, b]) = [1, 2]` or `({a, b}) = { a: 1, b: 2 }`, agora são considerados inválidos e lançarão um `SyntaxError`. Veja a postagem no blog de Jeff Walden e Erro do Firefox 1146136 para mais detalhes.

Veja também

- Operadores de Atribuição
- "ES6 in Depth: Destructuring" on hacks.mozilla.org (em inglês)

Divisão (/)

O operador de divisão (/) produz o quociente de seus operandos onde o operando esquerdo é o dividendo e o operando direito é o divisor.

Experimente

```
console.log(12 / 2);
// Expected output: 6

console.log(3 / 2);
// Expected output: 1.5

console.log(6 / "3");
// Expected output: 2

console.log(2 / 0);
// Expected output: Infinity
```

Sintaxe

```
x / y
```

Exemplos

Divisão básica

```
1 / 2; // 0.5

Math.floor(3 / 2); // 1

1.0 / 2.0; // 0.5
```

Divisão por zero

```
2.0 / 0; // Infinito

2.0 / 0.0; // Infinito, porque 0.0 === 0

2.0 / -0.0; // -Infinito
```

Veja também

- Operador de adição
- Operador de subtração
- Operador de multiplicação
- Operador restante
- Operador de exponenciação
- Operador de incremento
- Operador de decremento
- Operador de negação unário
- Operador unário plus

Operador Exponencial

Sumário

O **operador de exponenciação (**)** retorna o resultado de elevar o primeiro operando à potência do segundo operando. É equivalente ao método `Math.pow`, exceto que também aceita números do tipo BigInt como operandos.

Experimente

```
console.log(3 ** 4);
// Expected output: 81

console.log(10 ** -2);
// Expected output: 0.01

console.log(2 ** (3 ** 2));
// Expected output: 512
```

```
console.log((2 ** 3) ** 2);
// Expected output: 64
```

Sintaxe

```
x ** y;
```

Descrição

O operador de exponenciação é associativo à direita: `a ** b ** c` é igual a `a ** (b ** c)`.

Na maioria das linguagens, como PHP, Python e outras que têm um operador de exponenciação (`**`), o operador de exponenciação é definido para ter uma precedência maior do que os operadores unários, como o unário `+` e unário `-`, mas há algumas exceções. Por exemplo, no Bash, o operador `**` é definido para ter uma precedência menor do que os operadores unários.

Em JavaScript, é impossível escrever uma expressão de exponenciação ambígua. Ou seja, você não pode colocar um operador unário (`+ / - / ~ /! / delete / void / typeof`) imediatamente antes do número base; fazer isso causará um SyntaxError.

```
    -2 ** 2;
    // // 4 no Bash, -4 em outros idiomas.
    // Isso é inválido em JavaScript, pois a operação é ambígua.

    -(2 ** 2);
    // -4 em JavaScript e a intenção do autor não é ambígua.
```

Observe que algumas linguagens de programação usam o símbolo circunflexo `^` para exponenciação, mas o JavaScript usa esse símbolo para o operador `XOR lógico bit a bit`.

Exemplos

Exponenciação básica

```
2 ** 3; // 8
3 ** 2; // 9
3 ** 2.5; // 15.588457268119896
10 ** -1; // 0.1
NaN ** 2; // NaN
```

Associatividade

```
2 ** 3 ** 2; // 512
2 ** (3 ** 2); // 512
(2 ** 3) ** 2; // 64
```

Uso com operadores unários

Para inverter o sinal do resultado de uma expressão de exponenciação:

```
-(2 ** 2); // -4
```

Para forçar a base de uma expressão de exponenciação a ser um número negativo:

```
(-2) ** 2; // 4
```

Veja também

- Operador condicional if
- Operador de adição
- Operador de subtração
- Operador de multiplicação
- Operador de divisão
- Operador de resto
- Operador de incremento
- Operador de decremento
- Operador de negação unária
- Operador de adição unária

Expressão de função (function expression)

A palavra-chave **`function`** pode ser usada para definir uma função dentro de uma expressão.

Sintaxe

```
function [name]([param1[, param2[, ..., paramN]]]) {
   statements
}
```

Parâmetros

`name`

O nome da função. Pode ser omitido, neste caso a função será anônima. O nome só é local em relação ao corpo da função.

`paramN`

O nome de um argumento a ser passado para a função.

`statements`

As declarações que compreendem o corpo da função.

Descrição

Uma expressão de função (*function expression*) é muito similar e tem quase a mesma sintaxe de uma declaração de função (veja function statement para mais detalhes). A principal diferença entre uma expressão de função e a declaração de uma função é o nome da função (*function name*), o qual pode ser omitido em expressões de funções para criar funções anônimas. Veja ainda o capítulo sobre funções para maiores informações.

Exemplos

O exemplo a seguir define uma função sem nome e a atribui a x. A função retorna o quadrado de seu argumento:

```
var x = function (y) {
  return y * y;
};
```

Expressão de função nomeada

Se você quer se referir à função atual dentro do corpo da função, você terá que criar uma expressão de função nomeada. Este nome será local apenas para o corpo (escopo) da função. Isto ainda evita o uso da propriedade não-padronizada `arguments.callee`.

```
var math = {
  factorial: function factorial(n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
  },
};
```

Veja também

- `Funções e Escopos de Funções`
- `Função`
- `function statement`
- `function* statement`
- `function* expression`
- `GeneratorFunction`VersãoCompatívelDesconhecidaVersãoCompatívelDesconhecida

Operador de Agrupamento

Sumário

O operador de agrupamento `()` controla a precedência e a precedência de avaliação em expressões.

Sintaxe

```
( )
```

Descrição

O operador de agrupamento consiste em um par de parênteses em volta de uma expressão, ou sub-expressão, para ignorar a precedência normal de operadores de modo que expressões com menor precedência possam ser avaliadas antes de uma expressão com maior prioridade.

Exemplos

Ignorando multiplicação e divisão primeiro, então adição e substração para avarliar adição primeiramente.

```
var a = 1;
var b = 2;
var c = 3;

// default precedence
a + b * c; // 7
// evaluated by default like this
a + (b * c); // 7

// now overriding precedence
// addition before multiplication
(a + b) * c; // 9

// which is equivalent to
a * c + b * c; // 9
```

Veja também

- Operator precedence
- `delete`
- `typeof`

import.meta

O objeto **`import.meta`** mostra os metadados específicos do contexto de um módulo JavaScript. Ele contém informações sobre o módulo, como a sua URL.

Sintaxe

```
import.meta
```

Descrição

A sintaxe consiste na palavra chave `import`, um ponto, e o identificador `meta`. Normalmente no lado esquerdo do ponto é o objeto na qual a propriedade de acesso é realizada is, mas aqui `import` não é exatamente um objeto.

O objeto`import.meta` foi criado no implementação ECMAScript,com um protótipo `null`. O objeto é extensível, e suas propriedades são grávaveis, configuráveis, e enumeráveis.

Exemplos

Usando import.meta

Dado o modulo `my-module.js`

```
<script type="module" src="https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Operators/import.meta/my-module.js"></script>
```

Você pode acessar essa meta informação sobre o modulo usando o objeto `import.meta.`

```
console.log(import.meta); // { url: "file:///home/user/my-module.js" }
```

Irá retornar um objeto com propriedade URL indicando a base URL do módulo. Isso vai ser o URL da qual o script obteve, por scripts external, ou a base do documento URL contendo documento, para scripts inline.

Note que isso irá incluir parâmetros query e/ou cerquilha (i.e., seguindo o `?` ou `#`).

Por exemplo, seguindo esse HTML:

```
<script type="module">
  import "./index.mjs?someURLInfo=5";
</script>
```

O arquivo JavaScript a seguir vai registrar o parâmetro `someURLInfo`:

```
// index.mjs
new URL(import.meta.url).searchParams.get("someURLInfo"); // 5
```

O mesmo se aplica quando um arquivo importa outro:

```
// index.mjs
import "./index2.mjs?someURLInfo=5";

// index2.mjs
new URL(import.meta.url).searchParams.get("someURLInfo"); // 5
```

Note que enquanto o Node.js vai passar nos parâmetros query (ou na cerquilha) como no exemplo passsado, a partir do Node 14.1.0, a URL com parâmetro query vai dar erro quando carregar no formulário `node --experimental-modules index.mjs?someURLInfo=5` (é tratado como um arquivo ao invés de uma URL nesse contexto).

Nesse arquivo específico, o argumento passado pode ser complementar para ser usado na ampla aplicação `location.href` (com strings query ou cerquilha adicionada depois do caminho de arquivo HTML) (ou por Node.js, através do `process.argv`).

Veja também

- `import`
- `export`

in operator

O **operador `in`** retorna `true` se a propriedade especificada estiver no objeto especificado ou na sua cadeia de protótipo (prototype chain) desde objeto.

Sintaxe

```
prop in object
```

Parâmetros

`prop`

Um string ou símbolo representando o nome de uma propriedade ou o índice de um array (não-símbolos serão trazidos para strings).

`object`

Objeto para verificar se este (ou sua cadeia de protótipo) contém a propriedade com o nome especificado.

Descrição

Os exemplos a seguir mostram alguns usos do operador `in`.

```
// Arrays
var trees = ["redwood", "bay", "cedar", "oak", "maple"];
0 in trees; // retorna true
3 in trees; // retorna true
6 in trees; // retorna false
"bay" in trees; // retorna false (você precisa especificar o
// número do índice, não o valor naquele índice)
"length" in trees; // retorna true (length é uma propridade do Array)
Symbol.iterator in trees; // retorna true (arrays são iteráveis, funciona apenas na ES2015+)

// Objetos predefinidos
"PI" in Math; // retorna true

// Objetos personalizados
var mycar = { make: "Honda", model: "Accord", year: 1998 };
"make" in mycar; // retorna true
"model" in mycar; // retorna true
```

Você precisa especificar um objeto no lado direito do operador `in`. Por exemplo, você pode especifica um string criado com o construtor `String`, mas você não pode especificar um string literal.

```
var color1 = new String("green");
"length" in color1; // retorna true

var color2 = "coral";
// gera um erro (color2 não é um objeto String)
"length" in color2;
```

Usando <code>in</code> com propriedade removida ou undefined

Se você remover uma propriedade com o operador `delete` , o operador `in` retorna `false` para essa propriedade.

```
var mycar = { make: "Honda", model: "Accord", year: 1998 };
delete mycar.make;
"make" in mycar; // retorna false

var trees = new Array("redwood", "bay", "cedar", "oak", "maple");
delete trees[3];
3 in trees; // retorna false
```

Se você definir uma propriedade como `undefined`, mas não a remover, o operador `in` retorna `true` para essa propriedade.

```
var mycar = { make: "Honda", model: "Accord", year: 1998 };
mycar.make = undefined;
"make" in mycar; // retorna true
```

```
var trees = new Array("redwood", "bay", "cedar", "oak", "maple");
trees[3] = undefined;
3 in trees; // retorna true
```

Propriedades herdadas

O operador `in` retorna `true` para propriedades que estão na cadeida de protótipo (prototype chain).

```
"toString" in {}; // retorna true
```

Ver também

- `for...in`
- `delete`
- `Object.prototype.hasOwnProperty()`
- `Reflect.has()`
- Enumerability and ownership of properties

Desigualdade (!=)

O operador de desigualdade (`!=`) verifica se seus dois operandos não são igual, retornando um resultado booleano. Ao contrário do operador de desigualdade estrito, ele tenta converter os tipos e comparar operandos de tipos diferentes.

Experimente

```
console.log(1 != 1);
// Expected output: false

console.log("hello" != "hello");
// Expected output: false

console.log("1" != 1);
// Expected output: false

console.log(0 != false);
// Expected output: false
```

Sintaxe

```
x != y
```

Descrição

O operador de desigualdade verifica se seus operandos não são iguais. É a negação do operador de igualdade, de modo que as duas linhas a seguir sempre darão o mesmo resultado:

```
x != y;

!(x == y);
```

Para obter detalhes do algoritmo de comparação, consulte a página do operador de igualdade.

```
3 != "3"; // false
```

Para evitar isso e exigir que diferentes tipos sejam considerados diferentes, use o operador de desigualdade estrita em vez disso:

```
3 !== "3"; // true
```

Exemplos

Comparação sem conversão de tipo

```
1 != 2; // true
"hello" != "hola"; // true

1 != 1; // false
"hello" != "hello"; // false
```

Comparação com conversão de tipo

```
"1" != 1; // false
1 != "1"; // false
0 != false; // false
0 != null; // true
0 != undefined; // true
0 != !!null; // false, veja o operador lógico NOT
0 != !!undefined; // false, veja o operador lógico NOT
null != undefined; // false

const number1 = new Number(3);
const number2 = new Number(3);
number1 != 3; // false
number1 != number2; // true
```

Comparação de objetos

```
const object1 = {
  key: "value",
};

const object2 = {
  key: "value",
};

console.log(object1 != object2); // true
console.log(object1 != object1); // false
```

Veja também

- Operador de igualdade
- Operador de igualdade estrito
- Operador de desigualdade estrito

instanceof

O operador **`instanceof`** testa se um objeto tem, em seu prototype, a função construtora.

Experimente

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
const auto = new Car("Honda", "Accord", 1998);

console.log(auto instanceof Car);
// Expected output: true

console.log(auto instanceof Object);
// Expected output: true
```

Sintaxe

```
objeto instanceof construtor
```

Parâmetros

`objeto`

O objeto a ser testado

`construtor`

Função construtora a ser verificada

Descrição

O operador instanceof testa a presença da função construtora no prototype do objeto.

```
// definindo construtores
function C() {}
function D() {}

var o = new C();

// true, porque: Object.getPrototypeOf(o) === C.prototype
o instanceof C;

// false, porque D.prototype não está no prototype desse objeto
o instanceof D;

o instanceof Object; // true, porque:
C.prototype instanceof Object; // true

C.prototype = {};
var o2 = new C();

o2 instanceof C; // true

// false, porque C.prototype não está mais no prototype desse objeto
o instanceof C;

D.prototype = new C(); // use inheritance
var o3 = new D();
o3 instanceof D; // true
o3 instanceof C; // true
```

Note que o resultado do instanceof pode alterar quando a gente altera o prototype da função construtora. No entanto, a gente não pode alterar (por padrão) o prototype do objeto. Só é possível fazer essa alteração usando a pseudopropriedade __proto__.

<code>instanceof</code> and multiple context (e.g. frames or windows)

Different scope have different execution environments. This means that they have different built-ins (different global object, different constructors, etc.). This may result in unexpected results. For instance, `[] instanceof window.frames[0].Array` will return `false`, because `Array.prototype !== window.frames[0].Array` and arrays inherit from the former. This may not make sense at first but when you start dealing with multiple frames or windows in your script and pass objects from one context to another via functions, this will be a valid and strong issue. For instance, you can securely check if a given object is in fact an Array using `Array.isArray(myObj)`

> **Nota: Note for Mozilla developers:** In code using XPCOM `instanceof` has special effect: `obj instanceof xpcomInterface` (e.g. `Components.interfaces.nsIFile`) calls `obj.QueryInterface(xpcomInterface)` and returns `true` if QueryInterface succeeded. A side effect of such call is that you can use *`xpcomInterface`*'s properties on `obj` after a successful `instanceof` test. Unlike standard JavaScript globals, the test `obj instanceof xpcomInterface`works as expected even if `obj` is from a different scope.

Examples

Demonstrating that <code>String</code> and <code>Date</code> are of type <code>Object</code> and exceptional cases

The following code uses `instanceof` to demonstrate that `String` and `Date` objects are also of type `Object` (they are derived from `Object`).

However, objects created with the object literal notation are an exception here: Although the prototype is undefined, `instanceof Object` returns `true`.

```
var simpleStr = "This is a simple string";
var myString = new String();
var newStr = new String("String created with constructor");
var myDate = new Date();
var myObj = {};

simpleStr instanceof String; // returns false, checks the prototype chain, finds undefined
myString instanceof String; // returns true
newStr instanceof String; // returns true
myString instanceof Object; // returns true

myObj instanceof Object; // returns true, despite an undefined prototype
({}) instanceof Object; // returns true, same case as above

myString instanceof Date; // returns false

myDate instanceof Date; // returns true
myDate instanceof Object; // returns true
myDate instanceof String; // returns false
```

Demonstrating that <code>mycar</code> is of type <code>Car</code> and type <code>Object</code>

The following code creates an object type `Car` and an instance of that object type, `mycar`. The `instanceof` operator demonstrates that the `mycar` object is of type `Car` and of type `Object`.

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
var mycar = new Car("Honda", "Accord", 1998);
var a = mycar instanceof Car; // retorna true
var b = mycar instanceof Object; // retorna true
```

Veja também

- `typeof`

AND lógico (&&)

O operador lógico **AND (`&&`)** (conjunção lógica) para um conjunto de operandos booleanos será `true` se e somente se todos os operandos forem `true`. Caso contrário, será `false`.

Geralmente, o operador retorna o valor do primeiro operando falsy encontrado ao avaliar da esquerda para a direita ou retorna o valor do último operando se todos forem truthy.

Exemplo

Experimente

```
const a = 3;
const b = -2;

console.log(a > 0 && b > 0);
// Expected output: false
```

Sintaxe

```
expr1 && expr2
```

Descrição

O AND lógico (`&&`) avalia operandos da esquerda para a direita, retornando imediatamente com o valor do primeiro operando falsy que encontrar; se todos os valores forem truthy, o valor do último operando é retornado.

Se um valor puder ser convertido em `true`, o valor é denominado como truthy. Se um valor puder ser convertido em `false`, então valor será denominado falsy.

Exemplos de expressões que podem ser convertidas em `false` são:

- `false`;
- `null`;
- `NaN`;
- `0`;
- string vazia (`""` ou `''` ou ``` ``);
- `undefined`.

O operador AND lógico (`&&`) preserva valores não booleanos e os retorna como estão:

```
result = "" && "foo"; // result retorna "" (string vazia)
result = 2 && 0; // result retorna 0
```

```
result = "foo" && 4; // result retorna 4
```

Mesmo que o operador `&&` possa ser usado com operandos não booleanos, ele ainda é considerado um operador booleano, pois seu valor de retorno sempre pode ser convertido em um boolean primitive. Para converter explicitamente seu valor de retorno (ou qualquer expressão em geral) para o valor booleano correspondente, use um double `operador NOT` ou o construtor `Boolean`.

Avaliação de curto-circuito

A expressão AND lógica é um operador de curto-circuito. Ou seja, à medida que cada operando é convertido em um booleano, se o resultado de uma conversão for `false`, o operador AND para e retorna o valor original desse operando falso; ele **não** avalia nenhum dos operandos restantes.

Considere o pseudocódigo abaixo.

```
(alguma expressao falsa aqui) && expr
```

A `expr` **nunca é avaliada** porque o primeiro operando `(alguma expressao falsa aqui)` é avaliado como falsy. Se `expr` for uma função, a função nunca será chamada. Veja o exemplo abaixo:

```
function A() {
  console.log("chamou A");
  return false;
}
function B() {
  console.log("chamou B");
  return true;
}

console.log(A() && B());
// registra "chamou A" no console devido à chamada da função A,
// && é avaliado como false (porque a função A retorna false), então false é registrado no console;
// o operador AND entra em curto-circuito aqui e ignora a função B
```

Operador precedente

O operador AND tem uma precedência maior que o operador OR, o que significa que o operador `&&` é executado antes do operador `||` (consulte precedência de operadores).

```
true || (false && false); // true
true && (false || false); // false
2 === 3 || (4 < 0 && 1 === 1); // false
```

Exemplos

Usando AND

O código a seguir mostra exemplos do operador `&&` (AND lógico).

```
a1 = true && true; // t && t retorna true
a2 = true && false; // t && f retorna false
a3 = false && true; // f && t retorna false
a4 = false && 3 === 4; // f && f retorna false
a5 = "Cat" && "Dog"; // t && t retorna "Dog"
a6 = false && "Cat"; // f && t retorna false
a7 = "Cat" && false; // t && f retorna false
a8 = "" && false; // f && f retorna ""
a9 = false && ""; // f && f retorna false
```

Regras de conversão para booleanos

Convertendo AND para OR

A seguinte operação envolvendo **booleanos**:

```
bCondition1 && bCondition2
```

é sempre igual a:

```
!(!bCondition1 || !bCondition2)
```

Convertendo OR para AND

A seguinte operação envolvendo **booleanos**:

```
bCondition1 || bCondition2
```

é sempre igual a:

```
!(!bCondition1 && !bCondition2)
```

Removendo parênteses aninhados

Como as expressões lógicas são avaliadas da esquerda para a direita, sempre é possível remover os parênteses de uma expressão complexa desde que certas regras sejam seguidas.

A seguinte operação composta envolvendo **booleanos**:

```
bCondition1 || (bCondition2 && bCondition3)
```

é sempre igual a:

```
bCondition1 || bCondition2 && bCondition3
```

Veja também

- `Boolean`
- Truthy
- Falsy

NOT lógico (!)

O operador lógico **NOT (!)** (complemento lógico, negação) leva a verdade à falsidade e a falsidade à verdade, por isso, é conhecido como um operador de "negação". Normalmente é usado com valores booleanos (lógicos). Quando usado com valores não booleanos, retorna `false` se seu único operando puder ser convertido em `true`; caso contrário, retorna `true`.

Experimente

```
const a = 3;
const b = -2;

console.log(!(a > 0 || b > 0));
// Expected output: false
```

Sintaxe

```
!x
```

Descrição

Retorna `false` se seu único operando puder ser convertido em `true`; caso contrário, retorna `true`.

Se um valor puder ser convertido em `true`, o valor será chamado truthy. Se um valor puder ser convertido em `false`, o valor será denominado falsy.

Exemplos de expressões que podem ser convertidas em falso são:

- `null`;
- `NaN`;
- `0`;
- empty string (`""` or `''` or `` ``);
- `undefined`.

Mesmo que o operador `!` possa ser usado com operandos que não sejam valores booleanos, ele ainda pode ser considerado um operador booleano, pois seu valor de retorno sempre pode ser convertido em um primitivo booleano. Para converter explicitamente seu valor de retorno (ou qualquer expressão em geral) para o valor booleano correspondente, use um operador NOT duplo (`!!`) ou o construtor `Boolean` do JavaScript.

Exemplos

Usando NOT

O código a seguir mostra exemplos do operador lógico NOT `!`:

```
!true; // !t returns false
!false; // !f returns true
!""; // !f returns true
!"Cat"; // !t returns false
```

Duplo NOT (<code>!!</code>)

É possível usar alguns operadores NOT em série para forçar explicitamente a conversão de qualquer valor para um primitivo booleano correspondente. A conversão é baseada na "veracidade" ou "falsidade" do valor (veja truthy e falsy).

A mesma conversão pode ser feita através da função `Boolean()` do JavaScript.

```
!!true; // !!truthy returns true
!!{}; // !!truthy returns true: any object is truthy...
!!new Boolean(false); // ...even Boolean objects with a false .valueOf()!
!!false; // !!falsy returns false
!!""; // !!falsy returns false
!!Boolean(false); // !!falsy returns false
```

Conversão entre NOTs

A seguinte operação envolvendo **booleanos**:

```
!!bCondition
```

é sempre igual a:

```
bCondition
```

Veja também

- `Boolean`
- Truthy

new operator

O **operador** `new` cria uma instancia de um tipo de objeto definido pelo usuário ou de um dos tipos nativos (*built-in*) que possuem uma função construtora.

Experimente

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}

const car1 = new Car("Eagle", "Talon TSi", 1993);

console.log(car1.make);
// Expected output: "Eagle"
```

Sintaxe

```
new construtor[([argumentos])]
```

Parametros

`construtor`

Uma função que especifica o tipo da instância do objeto.

`argumentos`

Uma lista de valores com os quais o construtor será chamado.

Descrição

Criar um objeto definido pelo usuário requer dois passos:

1. Definir o tipo de objeto através da escrita de uma função.
2. Criar uma instancia do objeto utilizando `new`.

Para definir um tipo de objeto, crie uma função para o tipo de objetivo que especifica seu nome e propriedades. Um objeto pode ter uma propriedade que é um outro objeto. Veja os exemplos abaixo:

Quando o código `new Foo(...)` é executado, acontece o seguinte::

1. Um novo objeto é criado, herdando de `Foo.prototype`.
2. A função construtora *Foo* é chamado com os argumentos especificados, e com `this` vinculado ao novo objeto criado. `new Foo` é equivalente a `new Foo()`, ou seja, se nenhuma lista de argumentos for especificada, *Foo* é chamado sem argumentos.
3. O objeto retornado pela função construtora é o resultado de toda expressão `new`. Se a função construtora não retornar um objeto explicitamente, o objeto criado no passo 1 é então usado. (Normalmente construtores não retornam um valor, mas eles podem escolher fazê-lo se eles quiserem sobrescrever o processo normal de criação de objeto.)

Você sempre pode adicionar uma propriedade à um objeto definido previamente. Por exemplo, `carro1.cor = "preta"` adiciona a propriedade `cor` em `carro1`, e atribui a ela o valor de `"preta"`. Entretanto, isso não afeta os outros objetos. Para adicionar a nova propriedade a todos objetos do mesmo tipo, você deve adicionar a propriedade à definição do tipo de objeto `Carro`.

Você pode adicionar uma propriedade compartilhada à um tipo de objeto definido anteriormente através do uso da propriedade `Function.prototype`. Isso define uma propriedade que é compartilhada por todos os objetos criados com essa função, ao invés de apenas uma instancia do tipo de objeto. O código a seguir adiciona uma propriedade cor com valor `null` à todos objetos do tipo `carro`, e então sobrescreve aquele valor com a string `"preta"` somente no objeto de instancia `carro1`. Para mais informações veja prototype.

```
function Carro() {}
carro1 = new Carro();

console.log(carro1.cor); // undefined

Carro.prototype.cor = null;
console.log(carro1.cor); // null
```

```
carro1.cor = "preta";
console.log(carro1.cor); // preta
```

Exemplos

Tipo de objeto e instância de objeto

Suponha que você quer criar um tipo de objeto para carros. Você quer que esse tipo de objeto se chame `carro`, e quer que ele tenha propriedade para fabricante, modelo e ano. Para fazer isso, você escreveria a função a seguir:

```
function Carro(fabricante, modelo, ano) {
  this.fabricante = fabricante;
  this.modelo = modelo;
  this.ano = ano;
}
```

Agora você pode criar um objeto chamado `meucarro` como a seguir:

```
var meucarro = new Carro("Eagle", "Talon TSi", 1993);
```

Essa declaração cria `meucarro` e atribui a ele os valores especificados as suas propriedades. Então o valor de `meucarro.fabricante` é a string "Eagle", `meucarro.ano` é o inteiro 1993, e assim sucessivamente.

Você pode criar qualquer numero de objetos `carro` através de chamadas a `new`. Por exemplo:

```
var carrodoken = new Carro("Nissan", "300ZX", 1992);
```

Propriedade do objeto que é outro objeto

Suponha que você defina um objeto chamado `pessoa` como a seguir:

```
function Pessoa(nome, idade, sexo) {
  this.nome = nome;
  this.idade = idade;
  this.sexo = sexo;
}
```

E então instancia dois novos objetos `pessoa` como a seguir:

```
var rand = new Pessoa("Rand McNally", 33, "M");
var ken = new Pessoa("Ken Jones", 39, "M");
```

Então você pode reescrever a definição de `carro` para incluir uma propriedade dono que aceita um objeto pessoa, como a seguir:

```
function Carro(fabricante, modelo, ano, dono) {
  this.fabricante = fabricante;
  this.modelo = modelo;
  this.ano = ano;
  this.dono = dono;
}
```

Para instanciar os novos objetos, você então usa o seguinte:

```
var carro1 = new Carro("Eagle", "Talon TSi", 1993, rand);
var carro2 = new Carro("Nissan", "300ZX", 1992, ken);
```

Ao invés de passar uma string ou valor inteiro quando criar os novos objetos, as definições acima passam objetos `rand` e `ken` como os parâmetros para os donos. Para descobrir o nome do dono do `carro2`, você pode acessar a seguinte propriedade:

```
carro2.dono.nome;
```

Veja também

- `Function`
- `Reflect.construct()`

new.target

A propriedade `new.target` permite que você detecte quando uma função ou construtor foi chamado utilizando o operador new. Em construtores e funções instaciadas com o operador new , `new.target` retorna a referência ao construtor ou função. Em chamadas normais de funções , `new.target` é `undefined`.

Sintaxe

```
new.target
```

Descrição

A sintaxe `new.target` consiste na palavra- chave `"new"`, o ponto, e o nome da propriedade `"target"`. Normalmente `"new."` serve como um contexto para a propriedade de acesso, mas aqui `"new."` não é exatamente um objeto. Em chamadas de construtores, entretanto, `new.target` se refere ao construtor invocado pelo `new` e então

"`new.`" se torna um contexto virtual.

`new.target` é uma propriedade meta que é disponibilizada para todas as funções. Em funções do tipo flecha `=>`, `new.target` se refere ao `new.target` em torno da função.

Exemplos

new.target em chamadas de Função

Em chamadas normais de função (diferente de chamadas a funções do tipo construtor), `new.target` é `undefined`. Isso permite que você detecte se a função foi chamada como um novo construtor.

```
function Foo() {
  if (!new.target) throw "Foo() must be called with new";
  console.log("Foo instanciado com new");
}

Foo(); // throws "Foo() must be called with new"
new Foo(); // logs "Foo instanciado com new"
```

new.target em Construtores

Em classes construtoras, `new.target` se refere ao construtor que foi diretamente invocado pelo `new`. Isto também é o caso se o construtor é uma classe pai e foi delegado pelo construtor de um filho.

```
class A {
  constructor() {
    console.log(new.target.name);
  }
}

class B extends A {
  constructor() {
    super();
  }
}

var a = new A(); // logs "A"
var b = new B(); // logs "B"

class C {
  constructor() {
    console.log(new.target);
  }
}
class D extends C {
  constructor() {
    super();
  }
}

var c = new C(); // logs class C{constructor(){console.log(new.target);}}
var d = new D(); // logs class D extends C{constructor(){super();}}
```

A partir do exemplo acima das classes C e D, mostra que new.target aponta para a definição da classe na qual foi inicializada. Exemplo, quando D foi inicializado utilizando new, a definição da classe D foi impressa e similiarmente ao caso de C, a classe C foi impressa.

Veja também

- funções
- Classes
- `new`
- `this`

null

Resumo

O valor `null` é um literal em JavaScript que representa um valor nulo ou "vazio" (p/ex: que aponta para um objeto inexistente). É um dos valores primitivos do JavaScript.

Sintaxe

```
null
```

Descrição

O valor `null` é um literal, e não uma propriedade do objeto global (como o `undefined` pode ser). O desenhos das APIs, o null as vezes é devolvido no lugar de um objeto que era esperado. Quando fizer a checagem de um valor para `null` ou `undefined`, esteja ciente das diferenças entre o operador de igualdade (==) e o de igualdade estrita (===) (em inglês). Uma conversão de tipos é realizada na operação de igualdade.

```
// foo não existe, não foi definido e jamais foi inicializado:
> foo
"ReferenceError: foo is not defined"

// foo é conhecido e existe, mas não aponta para nenhum tipo ou valor:
> var foo = null; foo
"null"
```

Diferenças entre <code>null</code> e <code>undefined</code>

```
typeof null; // object (bug no ECMAScript, deveria ser null - http://2ality.com/2013/10/typeof-null.html)
typeof undefined; // undefined
null === undefined; // falso
null == undefined; // verdadeiro
```

Veja também

- `undefined`
- `NaN`

Operador de coalescência nula

O **operador de coalescência nula (`??`)** é um operador lógico que retorna o seu operando do lado direito quando o seu operador do lado esquerdo é `null` ou `undefined`. Caso contrário, ele retorna o seu operando do lado esquerdo.

Ao contrário do operador lógico OR (`||`), o operando esquerdo é retornado se houver um valor *falsy* (falso) que não seja `null` ou `undefined`. Em outras palavras, se você usar `||` para obter algum valor padrão para outra variável `foo`, você pode enfrentar comportamentos inesperados se você considerar algum valor falseável como utilizável (eg. `''` ou `0`). Veja abaixo alguns exemplos:

Experimente

```
const foo = null ?? "default string";
console.log(foo);
// Expected output: "default string"

const baz = 0 ?? 42;
console.log(baz);
// Expected output: 0
```

Sintaxe

```
exprEsq ?? exprDir
```

Descrição

O operador de coalescência nula retorna os resultados da expressão de seu lado direito se a expressão de seu lado esquerdo for `null` ou `undefined`.

Endereçando um valor padrão à variável

Inicialmente, quando se deseja endereçar um valor padrão à variável, um padrão comum é utilizar o operador lógico OR (`||`):

```
let foo;

//  foo nunca é endereçado a nenhum valor, portanto, ainda está indefinido
let someDummyText = foo || "Hello!";
```

Entretanto, devido ao `||` ser um operador lógico booleano, o operando do lado esquerdo é coagido para um valor booleano para sua avaliação e qualquer valor *falseável* (`0`, `''`, `NaN`, `null`, `undefined`) não é retornado. Este comportamento pode causar consequencias inesperadas se você considerar `0`, `''`, ou `NaN` como valores válidos.

```
let count = 0;
let text = "";

let qty = count || 42;
let message = text || "Olá!";
console.log(qty); // 42 e não 0
console.log(message); // "Olá!" e não ""
```

O operador de coalescência nula evita esta cilada pois retorna o segundo operando apenas quando o primeiro é avaliado entre os valores `null` ou `undefined` (mas nenhum outro valor falseável):

```
let myText = ""; // Uma string vazia (que também é um valor falseável)

let notFalsyText = myText || "Olá mundo";
console.log(notFalsyText); // Olá mundo

let preservingFalsy = myText ?? "Olá vizinhança";
console.log(preservingFalsy); // '' (Pois myText não é undefined e nem null)
```

Curto-circuito

Assim como os operadores lógicos OR e AND, a expressão do lado direito não é avaliada se o lado esquerdo não for avaliado entre `null` e nem `undefined`.

```
function A() {
  console.log("A foi chamado");
  return undefined;
}
function B() {
  console.log("B foi chamado");
  return false;
}
function C() {
  console.log("C foi chamado");
  return "foo";
}

console.log(A() ?? C());
// Imprime "A foi chamado" então "C foi chamado" e por fim "foo"
// Como A() retornou undefined então ambas expressões foram avaliadas

console.log(B() ?? C());
// Imprime "B foi chamado" então "false"
// Como B() retornou false (e não null ou undefined), a expressão
// do lado direito não foi avaliada.
```

Sem encadeamento com os operadores AND e OR

Não é possível encadear ambos operadores AND (`&&`) e OR (`||`) diretamente com o `??`. Um `SyntaxError` será disparado nesse tipo de caso.

```
null || undefined ?? "foo"; // Dispara um SyntaxError
true || undefined ?? "foo"; // Dispara um SyntaxError
```

Entretanto, explicitar diretamente a precedência por meio de parênteses resulta no comportamento correto:

```
(null || undefined) ?? "foo"; // retorna "foo"
```

Relacionamento com o operador de encadeamento opcional (<code>?.</code>)

O operador de coalescência nula trata `undefined` e `null` como valores específicos e então executa o operador de encadeamento opcional (`?.`) o qual é útil para acessar uma propriedade de um objeto, o qual pode ser `null` ou `undefined`.

```
let foo = { someFooProp: "oi" };

console.log(foo.someFooProp?.toUpperCase()); // "OI"
console.log(foo.someBarProp?.toUpperCase()); // undefined
```

Exemplo

Neste exemplo, nós iremos prover valores padrão, mas manter valores que não sejam (advinha???) `null` ou `undefined`.

```
const nullValue = null;
const emptyText = ""; // falseável (falsy)
const someNumber = 42;

const valA = nullValue ?? "padrão para A";
const valB = emptyText ?? "padrão para B";
const valC = someNumber ?? 0;

console.log(valA); // "padrão para A"
console.log(valB); // "" (pois a string vazia não é null ou undefined)
console.log(valC); // 42
```

Especificações

Especificação	Status	comentário
Proposal for the "nullish coalescing" operator	Stage 4	

Veja também

- The optional chaining operator
- The logical OR (||) operator
- Default parameters in functions

Inicializador de Objeto

Objetos podem ser inicializados utilizando `new Object()`, `Object.create()`, ou a notação *literal*. Um inicializador de objetos é uma lista de zero ou mais pares de *propriedade: valor*, separados por vírgula e fechado por um par de chaves (`{}`).

Sintaxe

```
var o = {};
var o = { a: "foo", b: 42, c: {} };
```

```
var a = "foo",
  b = 42,
  c = {};
var o = { a: a, b: b, c: c };

var o = {
  propriedade: function ([parâmetros]) {},
  get propriedade() {},
  set propriedade(valor) {},
};
```

Novas notações em ECMAScript 2015

Por favor, verifique o suporte das anotações na tabela de compatibilidade. Em ambientes que não dão suporte às anotações, ocorrerá erros de sintaxe.

```
// // Abreviação em nomes de propriedades (ES2015)
var a = "foo",
  b = 42,
  c = {};
var o = { a, b, c };

// // Abreviação em nomes de métodos (ES2015)
var o = {
  property([parameters]) {},
  get property() {},
  set property(value) {},
};

// Nomes de propriedades computados (ES2015)
var prop = "foo";
var o = {
  [prop]: "hey",
  ["b" + "ar"]: "there",
};
```

Descrição

Um inicializador de objetos é uma expressão que descreve a inicialização de um `Object`. Objects consiste de *propriedades*, as quais descrevem um objeto. Os valores das propriedades de um objeto podem ser tipos de dados primitivos ou outros objetos .

Criando objetos

Um objeto vazio, sem propriedades, pode ser criado como:

```
var object = {};
```

Contudo, a vantagem em utilizar a notação *literal* ou o *inicializador* é a possibilidade de rapidamente criar objetos com propriedades dentro de chaves (`{}`). Você simplesmente cria uma lista de pares *chave: valor*, separados por vírgula. O código abaixo cria um objeto com três propriedades, sendo as chaves "foo", "age" e "baz", com seus respectivos valores, tipo string de valor "bar", tipo number de valor 42 e, por último, um outro objeto com seus respectivos pares de *chave: valor*.

```
var object = {
  foo: "bar",
  age: 42,
  baz: { myProp: 12 },
};
```

Acessando propriedades

Uma vez que você criou um objeto, é interessante que possa ler ou alterá-lo. As propriedades de um objeto podem ser acessadas utilizando a notação de ponto ou colchetes. Veja assessores de propriedade para mais informações.

```
object.foo; // "bar"
object["age"]; // 42

object.foo = "baz";
```

Definições de propriedade

Nós temos aprendido como descrever propriedades utilizando a sintaxe *inicializador*. No entanto, às vezes, há variáveis que queremos inserir em nosso objeto. Então teremos um código parecido como abaixo:

```
var a = "foo",
  b = 42,
  c = {};

var o = {
  a: a,
  b: b,
  c: c,
};
```

Com ECMAScript 2015, há uma notação mais curta que possibilita atingir o mesmo resultado:

```
var a = "foo",
  b = 42,
  c = {};

// Abreviação em nomes de propriedades (ES2015)
var o = { a, b, c };

// Em outras palavras,
console.log(o.a === { a }.a); // true
```

Duplicação em nomes de propriedades

Quando se está utilizando o mesmo nome para suas propriedades, a última sobrescreverá as anteriores.

```
var a = { x: 1, x: 2 };
console.log(a); // { x: 2}
```

Em códigos ECMAScript 5 no modo estrito, duplicação em nomes de propriedades serão consideradas `SyntaxError`. Porém, com a introdução de "nomes de propriedades computadas", tornou-se possível a duplicação das propriedades em tempo de execução. Assim, ECMAScript 2015 removeu a restrição.

```
function haveES2015DuplicatePropertySemantics() {
  "use strict";
  try {
    ({ prop: 1, prop: 2 });

    // No error thrown, duplicate property names allowed in strict mode
    return true;
  } catch (e) {
    // Error thrown, duplicates prohibited in strict mode
    return false;
  }
}
```

Definição de métodos

Uma propriedade de um objeto pode se referir à function, ou aos métodos getter ou setter.

```
var o = {
  propriedade: function ([parâmetros]) {},
  get propriedade() {},
  set propriedade(valor) {},
};
```

No ECMAScript 2015, uma notação abreviada está disponível, dispensando o uso da palavra reservada "function".

```
// Abreviações em nomes de métodos (ES2015)
var o = {
  propriedade([parâmetros]) {},
  get propriedade() {},
  set propriedade(valor) {},
  *gerador() {},
};
```

Com ECMAScript 2015, há uma forma concisa em criar propriedades cujo valor é uma função gerador.

```
var o = {
  * gerador() {
    ...........
  }
};
```

Mas em ECMAScript 5, você escreveria (lembrar que em ES5 não há geradores):

```
var o = {
  generator: function *() {
    ...........
  }
};
```

Para mais informações e exemplos, veja definições de método.

Nomes de propriedades computados

Começando com ECMAScript 2015, a sintaxe *inicializador* de objeto também suporta "nomes de propriedades computados". Isso permite que você possa inserir uma expressão dentro de colchetes `[]`, que será computada como o nome de uma propriedade. Isto é semelhante à notação de chaves utilizado em acessor de propriedade, utilizado para ler a alterar as propriedades existentes em um objeto. Segue um exemplo utilizando a mesma sintaxe em objetos literais:

```
// Nomes de propriedades computados (ES2015)
var i = 0;
```

```
var a = {
  ["foo" + ++i]: i,
  ["foo" + ++i]: i,
  ["foo" + ++i]: i,
};

console.log(a.foo1); // 1
console.log(a.foo2); // 2
console.log(a.foo3); // 3

var param = "size";
var config = {
  [param]: 12,
  ["mobile" + param.charAt(0).toUpperCase() + param.slice(1)]: 4,
};

console.log(config); // { size: 12, mobileSize: 4 }
```

Mutação Prototype

Uma definição de propriedade na forma de `__proto__: valor` or `"__proto__": valor` não cria uma propriedade com o nome `__proto__`. Inclusive, se o valor fornecido for um objeto ou `null`, muda o `[[Prototype]]` do objeto criado para o valor informado. (Se o valor fornecido não é um objeto ou null, o valor não será alterado.)

```
var obj1 = {};
assert(Object.getPrototypeOf(obj1) === Object.prototype);

var obj2 = { __proto__: null };
assert(Object.getPrototypeOf(obj2) === null);

var protoObj = {};
var obj3 = { __proto__: protoObj };
assert(Object.getPrototypeOf(obj3) === protoObj);

var obj4 = { __proto__: "not an object or null" };
assert(Object.getPrototypeOf(obj4) === Object.prototype);
assert(!obj4.hasOwnProperty("__proto__"));
```

Apenas uma única mudança em prototype é permitida em um objeto: múltiplas mudanças gera erro de sintaxe.

Definições de propriedade que não utilizam da notação de ":", não são consideradas mudanças de prototype: são definições de propriedades que se comportam de forma semelhante às definições utilizando qualquer outro nome.

```
var __proto__ = "variable";

var obj1 = { __proto__ };
assert(Object.getPrototypeOf(obj1) === Object.prototype);
assert(obj1.hasOwnProperty("__proto__"));
assert(obj1.__proto__ === "variable");

var obj2 = {
  __proto__() {
    return "hello";
  },
};
assert(obj2.__proto__() === "hello");

var obj3 = { ["__prot" + "o__"]: 17 };
assert(obj3.__proto__ === 17);
```

Notação de objeto literal vs JSON

A notação de objeto literal não é a mesma de **J**ava**S**cript **O**bject **N**otation (JSON). Mesmo que possuam semelhanças, há as seguintes diferenças:

- JSON permite definições de propriedades utilizando apenas aspas duplas, como `"propriedade": valor`. E a definição não pode ser abreviada.
- Os valores JSON podem ser apenas strings, numbers, arrays, `true`, `false`, `null`, ou outro objeto JSON.
- Uma função como valor (veja "Métodos" acima) não pode ser atribuido em JSON.
- Objetos como `Date` serão do tipo string após `JSON.parse()`.
- `JSON.parse()` rejeitará "nomes de propriedades computados" e um erro será lançado.

Veja também

- Assesores de propriedade
- `get` / `set`
- Definições de métodos
- Lexical grammar

Precedência de Operadores

Resumo

A precedência de operadores determina a ordem em que os operadores são processados. Operadores com maior precedência são processados primeiro.

Um exemplo simples:

```
3 + 4 * 5 // returns 23
```

O operador de multiplicação ("*") tem maior precedência que o operador de adição ("+") e por isso será executado primeiro.

Associatividade

Associatividade determina a ordem em que operadores da mesma precedência são processados. Por exemplo, considere a expressão:

```
a OP b OP c
```

Associatividade à esquerda (esquerda para direita) significa que a expressão será processada como `(a OP b) OP c`, enquanto associatividade à direita (direita para esquerda) significa que será interpretada como `a OP (b OP c)`. Operadores de atribuição são associativos à direita, então você pode escrever:

```
a = b = 5; 5 = b = a
```

com o resultado esperado que `a` e `b` tenham o valor 5. Isto é porque o operador de atribuição retorna o valor que ele atribuiu. Primeiro, `b` é definido com o valor 5. Então `a` é definido com o valor de `b`.

Tabela

A tabela seguinte está ordenada da mais alta (20) para a mais baixa (0) precedência.

Precedência	Tipo do Operador	Associatividade	Operadores individuais
20	Agrupamento	n/a	`( … )`
19	Acesso a Membro	esquerda para direita	`… . …`
	Acesso a Membro Computado	esquerda para direita	`… [ … ]`
	new (com lista de argumentos)	n/a	`new … ( … )`
18	Chamada a Função	esquerda para direita	`… ( … )`
	new (sem lista de argumentos)	direita para esquerda	`new …`
17	Incremento Pós-fixado	n/a	`… ++`
	Decremento Pós-fixado	n/a	`… --`
16	NÃO lógico	direita para esquerda	`! …`
	NÃO bit-a-bit	direita para esquerda	`~ …`
	Positivo Unário	direita para esquerda	`+ …`
	Negativo Unário	direita para esquerda	`- …`
	Incremento Pré-fixado	direita para esquerda	`++ …`
	Decremento Pré-fixado	direita para esquerda	`-- …`
	typeof	direita para esquerda	`typeof …`
	void	direita para esquerda	`void …`
	delete	direita para esquerda	`delete …`
15	Exponenciação	direita para esquerda	`… ** …`
14	Multiplicação	esquerda para direita	`… * …`
	Divisão	esquerda para direita	`… / …`
	Resto	esquerda para direita	`… % …`
13	Adição	esquerda para direita	`… + …`
	Subtração	esquerda para direita	`… - …`
12	Deslocamento de bits para esquerda	esquerda para direita	`… << …`
	Deslocamento de bits para direita	esquerda para direita	`… >> …`
	Deslocamento de bits para direita, sem sinal	esquerda para direita	`… >>> …`
11	Menor Que	esquerda para direita	`… < …`
	Menor ou Igual a	esquerda para direita	`… <= …`
	Maior Que	esquerda para direita	`… > …`
	Maior ou Igual a	esquerda para direita	`… >= …`
	in	esquerda para direita	`… in …`
	instanceof	esquerda para direita	`… instanceof …`
10	Igualdade	esquerda para direita	`… == …`
	Desigualdade	esquerda para direita	`… != …`
	Igualdade Estrita	esquerda para direita	`… === …`
	Desigualdade Estrita	esquerda para direita	`… !== …`
9	E bit-a-bit	esquerda para direita	`… & …`

8	OU exclusivo bit-a-bit	esquerda para direita	`… ^ …`
7	OU bit-a-bit	esquerda para direita	`… | …`
6	E lógico	esquerda para direita	`… && …`
5	OU lógico	esquerda para direita	`… || …`
4	Condicional	direita para esquerda	`… ? … : …`
3	Atribuição	direita para esquerda	`… = …`
			`… += …`
			`… -= …`
			`… *= …`
			`… /= …`
			`… %= …`
			`… <<= …`
			`… >>= …`
			`… >>>= …`
			`… &= …`
			`… ^= …`
			`… |= …`
2	yield	direita para esquerda	`yield …`
	yield*		`yield* …`
1	Propagação	n/a	`... …`
0	Vírgula / Sequência	esquerda para direita	`… , …`

Encadeamento opcional

O operador de **encadeamento opcional `?.`** permite a leitura do valor de uma propriedade localizada internamente em uma cadeia de objetos conectados, sem que a validação de cada referência da cadeia seja expressivamente realizada.

O operador **`?.`** funciona de maneira similar ao operador **`.`** de encadeamento, exceto que, ao invés de causar um erro se a referência é nullish (`null` ou `undefined`), a expressão sofre um "curto-circuito" e retorna com um valor de `undefined`. Quando utilizado com uma chamada de função, retorna `undefined` se a função executada não existir.

Isso resulta em expressões mais curtas e simples ao acessar propriedades encadeadas quando a possibilidade de uma referência ser inexistente. Isso também pode auxiliar ao explorar o conteúdo de um objeto quando não existe garantia da existência de determinadas propriedades obrigatórias.

Experimente

```
const adventurer = {
  name: "Alice",
  cat: {
    name: "Dinah",
  },
};

const dogName = adventurer.dog?.name;
console.log(dogName);
// Expected output: undefined

console.log(adventurer.someNonExistentMethod?.());
// Expected output: undefined
```

Sintaxe

```
obj.val?.prop
obj.val?.[expr]
obj.arr?.[index]
obj.func?.(args)
```

Descrição

O operador de encadeamento opcional provê uma forma de simplificar o acesso a valores através de objetos conectados, quando é possível que uma referência ou função possa ser `undefined` ou `null`.

Por exemplo, considere um objeto `obj` que possui uma estrutura aninhada. Sem o encadeamento opcional, verificar proriedades profundamente aninhadas requer uma validação de referências intermediárias, algo como:

```
let nestedProp = obj.first && obj.first.second;
```

O valor de `obj.first` é confirmadamente não-`null` (e não-`undefined`) antes de acessar o valor de `obj.first.second`. Isso previne o erro que ocorreria se você simplesmente acessasse `obj.first.second` diretamente sem testar `obj.first`.

Com o operador de encadeamento opcional (`?.`), entretanto, você não precisa explicitamente testar e aplicar curto-circuito baseado no estado de `obj.first` antes de tentar acessar `obj.first.second`:

```
let nestedProp = obj.first?.second;
```

Ao utilizar o operador `?.` ao invés de apenas `.`, o JavaScript sabe que deve implicitamente checar e ter certeza que `obj.first` não é `null` ou `undefined` antes de tentar acessar `obj.first.second`. Se `obj.first` é `null` ou `undefined`, a expressão automaticamente sofrerá curto-circuito, retornando `undefined`.

Isso é equivalente ao seguinte, exceto que a variável temporária, de fato, não é criada:

```
let temp = obj.first;
let nestedProp = temp === null || temp === undefined ? undefined : temp.second;
```

Encadeamento opcional com chamadas de funções

Você pode usar o encadeamento opcional ao tentar chamar um método que pode não existir. Isso pode auxiliar, por exemplo, ao utilizar uma API em que o método está indisponível, seja pela época da implementação ou por causa de uma funcionalidade que ainda não está disponível no dispositivo do usuário.

Usar encadeamento opcional com chamadas de função faz com que a expressão automaticamente retorne `undefined` ao invés de lançar uma exceção se o método não é encontrado:

```
let result = someInterface.customMethod?.();
```

> **Nota:** Se existe uma propriedade com tal nome e que não é uma função, usando `?.` ainda lançará a exceção `TypeError` (`x.y is not a function`).

Lidando com callbacks opcionais ou manipuladores de eventos

Se você usa callbacks ou consulta métodos de objetos com atribuição via desestruturação, você pode ter valores não-existentes que você não conseguirá chamar como funções, a menos que você tenha testado sua existência. Usando `?.`, você pode evitar esse teste extra:

```
// Escrito como ES2019
function doSomething(onContent, onError) {
  try {
    // ... faz algo com os dados
  } catch (err) {
    if (onError) {
      // teste se onError realmente existe
      onError(err.message);
    }
  }
}
```

```
// Usando encadeamento opcional com chamadas de função
function doSomething(onContent, onError) {
  try {
    // ... faz algo com os dados
  } catch (err) {
    onError?.(err.message); // Nenhuma exceção se onError for undefined
  }
}
```

Encadeamento opcional com expressões

Você também pode usar o operador de encadeamento opcional ao acessar propriedades com uma expressão usando assessores de propriedade:

```
let nestedProp = obj?.["prop" + "Name"];
```

Encadeamento opcional não válido no lado esquerdo de uma atribuição

```
let object = {};
object?.property = 1; // Uncaught SyntaxError: Invalid left-hand side in assignment
```

Acesso a item de Array com encadeamento opcional

```
let arrayItem = arr?.[42];
```

Exemplos

Exemplo básico

Esse exemplo busca plo valor da propriedade `name` para o membro `bar` em um map quando não existe tal membro. Portanto, o resultado é `undefined`.

```
let myMap = new Map();
myMap.set("foo", { name: "baz", desc: "inga" });

let nameBar = myMap.get("bar")?.name;
```

Avaliação com curto-circuito

Ao usar o encadeamento opcional com expressões, se o operador do lado esquerdo é `null` ou `undefined`, a expressão não será avaliada. Por exemplo:

```
let potentiallyNullObj = null;
let x = 0;
let prop = potentiallyNullObj?.[x++];

console.log(x); // 0 já que x não foi incrementado
```

Empilhando o operator de encadeamento opcional

Com estruturadas aninhadas, é possível usar encadeamento opcional múltiplas vezes:

```
let customer = {
  name: "Carl",
  details: {
    age: 82,
    location: "Paradise Falls", // endereço detalhado é desconhecido
  },
};
let customerCity = customer.details?.address?.city;

// … isso também funcional com encadeamento opcional em chamada de função
let duration = vacations.trip?.getTime?.();
```

Combinando com o operador de coalescência nula (nullish coalescing)

O operador de coalescência nula pode ser usado após o encadeamento opcional, para construir um valor padrão quando nada é encontrado:

```
let customer = {
  name: "Carl",
  details: { age: 82 },
};
const customerCity = customer?.city ?? "Cidade desconhecida";
console.log(customerCity); // Cidade desconhecida
```

Especificações

Specification	Status	Comment
Proposal for the "optional chaining" operator	Stage 4	

Veja também

- O Nullish Coalescing Operator
- TC39 proposals

Assessores de propriedade

Os assessores de propriedade fornecem acesso as propriedades de um objeto usando a notação de ponto ou a notação de colchetes.

Experimente

```
const person1 = {};
person1["firstname"] = "Mario";
person1["lastname"] = "Rossi";

console.log(person1.firstname);
// Expected output: "Mario"

const person2 = {
  firstname: "John",
  lastname: "Doe",
};

console.log(person2["lastname"]);
// Expected output: "Doe"
```

Sintaxe

```
object.property
object["property"]
```

Descrição

Pode-se pensar em um objeto como uma matriz associativa (a.k.a. *map*, *dictionary*, *hash*, *lookup table*). As chaves nesta matriz são os nomes das propriedades dos objetos. É típico quando se fala de propriedades de um objeto para fazer uma distinção entre propriedades e métodos. No entanto, a distinção entre propriedade/método é um pouco mais do que convenção. Um método é simplesmente uma propriedade que pode ser chamada, por exemplo, se tiver uma referência a uma instância de Function como seu valor.

Há duas maneira de acessar propriedades: notação de ponto ou a notação de colchetes.

Notação de ponto

```
get = object.property;
object.property = set;
```

Neste código, a propriedade deve ser um identificador válido, i.e. uma sequência de carácteres alfanuméricos, incluíndo também o underline ("_") e o cifrão ("$"), não pode começar com um número. Por exemplo, `object.$1` é valido, enquanto `object.1` não é.

```
document.createElement("pre");
```

Aqui, o método chamado "createElement" é recuperado do documento e é chamado.

Se você usar um método para um literal numérico e o literal numérico não tem expoente e nenhum ponto decimal, deixar de espaço em branco antes do ponto que precede a chamada de método para evitar que o ponto seja interpretado como um ponto decimal.

```
(77).toExponential();
// ou
(77).toExponential();
// ou
(77).toExponential();
// ou
(77).toExponential();
// ou
(77.0).toExponential();
// porque 77. === 77.0, sem ambiguidade :p
```

Notação de colchete

```
get = object[property_name];
object[property_name] = set;
```

`property_name` é uma string. A string não precisa ser um identificador válido; pode ser qualquer valor, e.g. "1foo", "!bar!", ou até " " (um espaço).

```
document["createElement"]("pre");
```

Isto faz exatamente a mesma coisa que o exemplo anterior.

Um espaço antes da notação de colchete é permitido.

```
document["createElement"]("pre");
```

Nomes de propriedades

Nomes de propriedades devem ser strings. Isto significa que objetos não-string não podem ser usados como chave em um objeto. Qualquer objeto não-string, incluindo um número, é estereotipado como uma string pelo método toString.

```
var object = {};
object["1"] = "value";
console.log(object[1]);
```

A saída é "value", desde 1 é estereotipado como '1'.

```
var foo = { unique_prop: 1 },
  bar = { unique_prop: 2 },
  object = {};
object[foo] = "value";
console.log(object[bar]);
```

A saída também é "value", já que ambos foo e bar são convertidos para a mesma string. No motor de Javascript SpiderMonkey, esta string poderia ser "['object Object']".

Ligação de método

Um método não é limitado ao objeto de quem é metodo. Especificamente, `this` não é fixo em um método, i.e., `this` não se referece necessariamente ao objeto contendo o método. Ao invés disso, `this` é "passado" pela função call. Veja method binding.

Nota sobre <code>eval</code>

Novatos em JavaScript comentem muitas vezes o erro de usar eval onde a notação de colchete pode ser usada no lugar. Por exemplo, a sintaxe a seguir é muitas vezes vista em muitos scripts.

```
x = eval("document.forms.form_name.elements." + strFormControl + ".value");
```

`eval` é lento e deve ser evitado sempre que possível. Também, `strFormControl` would have to hold an identifier, which is not required for names and IDs of form controls. It is better to use bracket notation instead:

```
x = document.forms["form_name"].elements[strFormControl].value;
```

Veja também

- `Object`

- `Object/defineProperty`

Sintaxe de Espalhamento

Sintaxe de Espalhamento (Spread syntax) permite que um objeto iterável, como uma expressão de array ou uma string seja expandido para ser usado onde zero ou mais argumentos (para chamadas de funções) ou elementos (para arrays *literais*) são esperados, ou que um objeto seja expandido onde zero ou mais pares *propriedade:valor* (para objetos *literais*) são esperados.

Experimente

```
function sum(x, y, z) {
  return x + y + z;
}

const numbers = [1, 2, 3];

console.log(sum(...numbers));
// Expected output: 6

console.log(sum.apply(null, numbers));
// Expected output: 6
```

Sintaxe

Para chamada de funções:

```
myFunction(...iterableObj);
```

Para arrays literais ou strings:

```
[...iterableObj, '4', 'five', 6];
```

Para objetos literais (novo em ECMAScript 2018; stage 3 draft):

```
let objClone = { ...obj };
```

Exemplos

Espalhamento e chamadas de funções

Substituindo apply

É comum usar `Function.prototype.apply` em casos onde você quer usar os elementos de um array como argumentos para uma função.

```
function myFunction(x, y, z) {}
var args = [0, 1, 2];
myFunction.apply(null, args);
```

Com a sintaxe de espalhamento, o código acima pode ser escrito assim:

```
function myFunction(x, y, z) {}
var args = [0, 1, 2];
myFunction(...args);
```

Qualquer argumento numa lista de argumentos pode usar a sintaxe de espalhamento e pode ser usado mais de uma vez.

```
function myFunction(v, w, x, y, z) {}
var args = [0, 1];
myFunction(-1, ...args, 2, ...[3]);
```

Apply para new

Quando um construtor é chamado com `new`, não é possivel usar diretamente um array e `apply` (`apply` executa o `[[Call]]` e não o `[[Construct]]`). No entanto, um array pode facilmente ser usado com `new` graças ao operador de espalhamento:

```
var dateFields = [1970, 0, 1]; // 1 Jan 1970
var d = new Date(...dateFields);
```

Para usar o `new` com array de parâmetros sem a sintaxa de espalhamento, você teria que fazer isso **indiretamente** por meio da aplicação parcial:

```
function applyAndNew(constructor, args) {
  function partial() {
    return constructor.apply(this, args);
  }
  if (typeof constructor.prototype === "object") {
    partial.prototype = Object.create(constructor.prototype);
  }
  return partial;
}

function myConstructor() {
```

```
  console.log("arguments.length: " + arguments.length);
  console.log(arguments);
  this.prop1 = "val1";
  this.prop2 = "val2";
}

var myArguments = ["hi", "how", "are", "you", "mr", null];
var myConstructorWithArguments = applyAndNew(myConstructor, myArguments);

console.log(new myConstructorWithArguments());
// (internal log of myConstructor):           arguments.length: 6
// (internal log of myConstructor):           ["hi", "how", "are", "you", "mr", null]
// (log of "new myConstructorWithArguments"): {prop1: "val1", prop2: "val2"}
```

Espalhamento em arrays literais

Um array literal mais poderoso

Criar um novo array usando um array existente como parte dele, não é possível utilizando apenas a sintaxe de array literal. O código imperativo deve ser usado ao invés da combinação de `push`, `splice`, `concat`, etc. Com a sintaxe de espalhamento isso se torna muito mais sucinto:

```
var parts = ["shoulders", "knees"];
var lyrics = ["head", ...parts, "and", "toes"];
// ["head", "shoulders", "knees", "and", "toes"]
```

Assim como espalhar a lista de argumentos, `...` pode ser usado em qualquer lugar em um array literal e pode ser usado multiplas vezes.

Copiando um array

```
var arr = [1, 2, 3];
var arr2 = [...arr]; // like arr.slice()
arr2.push(4);

// arr2 becomes [1, 2, 3, 4]
// arr remains unaffected
```

Nota: A sintaxe de espalhamento efetivamente vai um nível mais profundo quando se copia um array. Assim sendo, pode ser inadequado para copiar arrays multidimensionais como o exemplo a seguir mostra (é o mesmo com `Object.assign()` e a sintaxe de espalhamento).

```
var a = [[1], [2], [3]];
var b = [...a];
b.shift().shift(); // 1
// Now array a is affected as well: [[], [2], [3]]
```

Uma maneira melhor de concatenar arrays

`Array.concat` é frequentemente usado para concatenar um array no final de um array existente. Sem a sintaxe de espalhamento é feito assim:

```
var arr1 = [0, 1, 2];
var arr2 = [3, 4, 5];
// Append all items from arr2 onto arr1
arr1 = arr1.concat(arr2);
```

Com a sintaxe de espalhamento se torna isso:

```
var arr1 = [0, 1, 2];
var arr2 = [3, 4, 5];
arr1 = [...arr1, ...arr2];
```

`Array.unshift` é frequentemente usado para inserir um array de valores no inicio de um array existente. Sem a sintaxe de espalhamento é feito assim:

```
var arr1 = [0, 1, 2];
var arr2 = [3, 4, 5];
// Prepend all items from arr2 onto arr1
Array.prototype.unshift.apply(arr1, arr2); // arr1 is now [3, 4, 5, 0, 1, 2]
```

Com a *sintaxe de espalhamento* isso se torna *[Note, no entanto, que isso cria um novo `arr1` array. Ao contrário de `Array.unshift`, isso não modifica o array original `arr1` array]*:

```
var arr1 = [0, 1, 2];
var arr2 = [3, 4, 5];
arr1 = [...arr2, ...arr1]; // arr1 is now [3, 4, 5, 0, 1, 2]
```

Espalhamento em objetos literais

A proposta Rest/Spread Properties for ECMAScript (stage 3) adiciona espalhamento de propriedades para objetos literais. Este copia as propriedades enumeráveis de um objeto informado em um novo objeto.

Cópia-rasa (Shallow-cloning) (excluindo o protótipo) ou fusão (**merge**) de objetos agora é possivel usando uma sintaxe mais curta que `Object.assign()`.

```
var obj1 = { foo: "bar", x: 42 };
var obj2 = { foo: "baz", y: 13 };

var clonedObj = { ...obj1 };
// Object { foo: "bar", x: 42 }

var mergedObj = { ...obj1, ...obj2 };
// Object { foo: "baz", x: 42, y: 13 }
```

Note que `Object.assign()` chamada os setters e a *sintaxe de espalhamento* não.

Apenas para iteráveis

A sintaxe de espalhamento (diferente de propriedades espalhadas) só pode ser utilizada com objetos iteráveis.

```
var obj = { key1: "value1" };
var array = [...obj]; // TypeError: obj is not iterable
```

Espalhamento com muitos valores

Quando usar a sintaxe de espalhamento para chamada de funções, esteja ciente da possibilidade de exceder tamanho máximo de argumentos do motor do Javascript. Veja `apply()` para mais detalhes.

Sintaxe Rest (parâmetros)

A *sintaxe rest* se parece exatamente como a *sintaxe de espalhamento*, mas esta é usada para desestruturar arrays e objetos. De certa forma, a *sintaxe rest* é o oposto da *sintaxe de espalhamento*: A *sintaxe de espalhamento (spread)* 'expande' um array em vários elementos, enquanto a *sintaxe rest* coleta multiplos elementos e 'condensa' eles em um único elemento. Veja parâmetros rest.

Veja também

- Parâmetros Rest (also '`...`')

Subtração (-)

O operador de subtração (-) é o operador aritmético de subtração usado para encontrar a diferença entre dois números

Experimente

```
console.log(5 - 3);
// Expected output: 2

console.log(3.5 - 5);
// Expected output: -1.5

console.log(5 - "hello");
// Expected output: NaN

console.log(5 - true);
// Expected output: 4
```

Sintaxe

```
x - y
```

Exemplos

Subtração com números

```
5 - 3; // 2
3 - 5; // -2
```

Subtração com strings

> **Aviso:** Note que o JavaScript sempre tentará converter uma String em numero número

```
"foo" - 3; // NaN
"5" - 1; // 4
"10" - 5; // 5
```

Veja também

- Operador de adição
- Operador de divisão
- Operador de multiplicação
- Operador de restante
- Operador de exponenciação
- Operador de incremento
- Operador de decremento

- Operador unário negation
- Operador unário plus

super

A palavra-chave **super** é usada para acessar o objeto pai de um objeto, em outros casos, é usada para acessar a classe pai de uma classe.

Síntaxe

```
// chama o objeto (ou construtor) pai
super(...[arguments]);
// chama um método da classe/objeto pai
super.metodo([arguments]);
```

Descrição

Quando usada no construtor de uma classe, a palavra-chave `super` deve ser usada apenas uma vez, e precisa ser usada antes que a palavra-chave `this` possa ser usada. Essa palavra-chave também pode ser usada para chamar uma função ou objeto pai.

Exemplo

Usando <code>super</code> em classes

Esse trecho de código foi obtido através de classes sample (demonstração). Aqui `super()` é chamado para evitar duplicar a parte do construtor que é comum entre `Polygon` e `Square`.

```
class Polygon {
  constructor(height, width) {
    this.name = "Polygon";
    this.height = height;
    this.width = width;
  }
  sayName() {
    console.log("Hi, I am a ", this.name + ".");
  }
}

class Square extends Polygon {
  constructor(length) {
    this.height; // ReferenceError, precisa chamar o super primeiro!

    // Aqui, ele chama a classe construtora pai com o tamanho
    // provido pelo Polygon -> width e height
    super(length, length);

    // Nota: Em classes derivadas, super() deve ser chamado antes de
    // usar this. Deixar isso de fora vai causar um ReferenceError.
    this.name = "Square";
  }

  get area() {
    return this.height * this.width;
  }

  set area(value) {
    this.area = value;
  }
}
```

Super chamando métodos estáticos

Você também pode chamar o super em métodos estáticos.

```
class Human {
  constructor() {}
  static ping() {
    return "ping";
  }
}

class Computer extends Human {
  constructor() {}
  static pingpong() {
    return super.ping() + " pong";
  }
}
Computer.pingpong(); // 'ping pong'
```

Ao deletar propriedades do super, será emitido um erro

Você não pode utilizar o delete operator e `super.prop` ou `super[expr]` para deletar uma propriedade da classe pai. Isto emitirá um `ReferenceError`.

```
class Base {
  constructor() {}
  foo() {}
}
class Derived extends Base {
  constructor() {}
  delete() {
    delete super.foo;
  }
}

new Derived().delete(); // ReferenceError: invalid delete involving 'super'.
```

<code>Super.prop</code> não pode sobrescrever propriedades não editáveis

`super`não pode sobrescrever o valor de uma propriedade quando esta houver sido definida como não editável ('writable: false') com, e.g., `Object.defineProperty`.

```
class X {
  constructor() {
    Object.defineProperty(this, "prop", {
      configurable: true,
      writable: false,
      value: 1,
    });
  }
  f() {
    super.prop = 2;
  }
}

var x = new X();
x.f();
console.log(x.prop); // 1
```

Usando <code>super.prop</code> em objetos literais

`super` também pode ser usado na inicialização da notação literal de objetos. No exemplo abaixo, cada objeto define um método. No segundo objeto, `super` chama o método do primeiro objeto. Isso funciona graças ao `Object.setPrototypeOf()`, com o qual é possível configurar o prototype do `obj2` para o `obj1`, tornando o `super` capaz de encontrar o `method1()` por meio do `obj2`.

```
var obj1 = {
  method1() {
    console.log("method 1");
  },
};

var obj2 = {
  method2() {
    super.method1();
  },
};

Object.setPrototypeOf(obj2, obj1);
obj2.method2(); // retorna "method 1"
```

Notas Gecko específicas

- `super()` ainda não funciona como esperado para prototypes nativos.

Veja mais

- Classes

this

A palavra-chave **`this`** comporta-se um pouco diferente em Javascript se comparado com outras linguagens. Também possui algumas diferenças entre o modo estrito e o modo não estrito.

Em muitos casos, o valor `this` é determinado pela forma como a função é chamada. Ele não pode ser assinado durante a execução, e isso pode ser diferente a cada vez que a função é chamada. ES5 introduziu o método `bind` para estabelecer o valor `this` da função, independentemente de como ela seja chamada, e ECMAScript 2015 introduziu o arrow functions, cujo `this` é lexicalmente delimitado (o valor `this` é estabelecido segundo o escopo de execução no qual está inserido).

Sintaxe

```
this
```

Contexto global

No contexto de execução global (fora de qualquer função), `this` refere-se ao objeto global, seja em modo estrito ou não.

```
console.log(this.document === document); // true

// Em navegadores web, o objeto window é também o objeto global:
```

```
console.log(this === window); // true

this.a = 37;
console.log(window.a); // 37
```

Contexto de função

Dentro de uma função, o valor de `this` depende de como a função é chamada.

Chamada simples

Como o código a seguir não está no modo estrito, o valor de `this` não é definido pela chamada. Por padrão, `this` será o objeto global que no navegador é o `window`.

```
function f1() {
  return this;
}

// No navegador
f1() === window; // true
```

Em modo estrito, o valor de `this` permanece seja qual for o definido ao entrar no contexto de execução, assim, no caso a seguir, `this` por padrão será indefinido (`undefined`):

```
function f2() {
  "use strict"; // assume modo estrito
  return this;
}

f2() === undefined; // true
```

Portanto, em modo estrito, se `this` não for definido durante o contexto da execução, ele permanecerá indefinido (`undefined`).

> **Nota:** No segundo exemplo, `this` deveria ser `undefined`, porque `f2` foi chamada diretamente e não como um método ou popriedade de um objeto (ou seja, `window.f2()`). Esta característica não foi implementada em alguns navegadores quando começaram a dar suporte ao strict mode (modo estrito). Como resultado, eles incorretamente retornavam o objeto `window`.

Funções Arrow (seta)

Nas arrow functions (funções seta), o `this` é definido lexicalmente, isto é, seu valor é definido pelo contexto de execução onde está inserido. Em um código global, this assume o objeto global:

```
var globalObject = this;
var foo = () => this;
console.log(foo() === globalObject); // true
```

Não importa como `foo` é chamado, o `this` continuará como o objeto global. Isto continua verdadeiro mesmo se chamá-lo como método de um determinado objeto (o que normalmente definiria seu this ao objeto), com `call` ou `apply` ou `bind` é usado:

```
// Chama como um método de um objeto
var obj = { foo: foo };
console.log(obj.foo() === globalObject); // true

// Tentativa de definir this usando call
console.log(foo.call(obj) === globalObject); // true

// Tentantiva de definir this usando bind
foo = foo.bind(obj);
console.log(foo() === globalObject); // true
```

Não importa como for, o this do foo mantém o valor que recebeu quando foi criado (no exemplo acima, o objeto global). O mesmo se aplica para funções arrow criadas dentro de outras funções: seus this são definidos em seus respectivos contextos de execução.

```
// Cria obj com um método bar que retorna uma função que
// retorna seu this. A função retornada é criada como
// uma função arrow, para que seu this esteja permanentemente
// ligado ao this da função que a envolve. O valor de bar pode ser // definido na chamada, que por sua vez define o valor da função
// retornada.
var obj = {
  bar: function () {
    var x = () => this;
    return x;
  },
};

// Chama bar como método de obj, configurando seu this como obj
// Assina à variável fn uma referência para a função retornada
var fn = obj.bar();

// Chamar fn, sem definir this, por padrão, referenciaria
```

```
// ao objeto global ou undefined em modo estrito (strict mode)
console.log(fn() === obj); // true
```

No exemplo acima, a função (chamemos função anônima A) atribuída a obj.bar retorna outra função (chamemos função anônima B) que é criada como uma função arrow (seta). Como resultado, o this da função B é permanentemente definido como o this de obj.bar (função A) quando chamado. Quando a função retornada (função B) é chamada, seu this sempre será aquele que foi definido inicialmente. No exemplo de código acima, o this da função B é definido com o this da função A, que é obj, por isso permanece definido para obj, mesmo quando chamado de uma maneira que normalmente definiria seu this como undefined ou como objeto global (ou qualquer outro método, como naquele exemplo anterior de contexto de execução global).

Como método de um objeto

Quando uma função é chamada como um método de um objeto, seu this toma o valor do objeto pertencente ao método chamado.

No exemplo a seguir, quando `o.f()` é invocado, o this dentro da função é vinculado ao objeto o.

```
var o = {
  prop: 37,
  f: function () {
    return this.prop;
  },
};

console.log(o.f()); // logs 37
```

Observe que esse comportamento não é afetado mesmo pela forma como (ou onde) a função foi definida. No exemplo anterior, nós definimos a função in-line (em linha) como o membro f durante a definição de o. No entanto, poderíamos ter apenas facilmente definido a função primeiro e depois anexado a o.f. Fazendo isso resulta no mesmo comportamento:

```
var o = { prop: 37 };

function independent() {
  return this.prop;
}

o.f = independent;

console.log(o.f()); // registra 37
```

Isto demonstra que é importante apenas que a função foi chamada a partir do membro f de o.

Da mesma forma, a vinculação de this só é afetada pela referência do membro mais imediato. No exemplo a seguir, quando invocamos a função, podemos chamá-la como um método g do objeto o.b. Desta vez, durante a execução, o this dentro da função irá se referir a o.b. O fato do objeto ser um membro de o não tem qualquer consequência; a referência mais imediata é tudo que importa.

```
o.b = { g: independent, prop: 42 };
console.log(o.b.g()); // registra 42
```

`this` na cadeia de protótipos (prototype chain) do objeto

A mesma noção vale para métodos definidos em algum lugar da cadeia de protótipos do objeto. Se o método está na cadeia de protótipo de um objeto, this refere-se ao objeto que é proprietário do método chamado, como se o método estivesse no objeto.

```
var o = {
  f: function () {
    return this.a + this.b;
  },
};
var p = Object.create(o);
p.a = 1;
p.b = 4;

console.log(p.f()); // 5
```

Neste exemplo, o objeto atribuído à variável p não tem sua própria propriedade f, ele o herda de seu protótipo. Mas não importa que a procura por f finalmente encontre um membro com esse nome em o; a procura começou como uma referência para a p.f, portanto o this dentro da função recebe o valor do objeto referido como p. Isto é, já que f é chamado como um método de p, seu this refere-se a p. Este é um recurso interessante de herança prototípica do JavaScript.

`this` com seletores (getter) ou modificadores (setter)

Mais uma vez, a mesma noção se aplica quando uma função é chamada a partir de um getter ou setter. A função usada como getter ou setter tem seu this ligado ao objeto do qual a propriedade está sendo modificada ou selecionada.

```
function modulus() {
  return Math.sqrt(this.re * this.re + this.im * this.im);
}

var o = {
  re: 1,
  im: -1,
  get phase() {
    return Math.atan2(this.im, this.re);
```

```
  },
};

Object.defineProperty(o, "modulus", {
  get: modulus,
  enumerable: true,
  configurable: true,
});

console.log(o.phase, o.modulus); // logs -0.78 1.4142
```

Como um construtor (constructor)

Quando a função é usada com um construtor (com a palavra chave `new`), seu this é vinculado ao novo objeto sendo contruído.

Nota: enquanto o padrão para um construtor é retornar o objeto referenciado por this, ele pode retornar, ao invés, algum outro objeto (se o valor de retorno não é um objeto, então o objeto this é retornado).

```
/*
 * Contrutores funcionam da seguinte forma:
 *
 * function MyConstructor(){
 *   // O código do corpo da função vai aqui.
 *   // Criam-se propriedades sobre |this| como
 *   // desejado, assinando-os. Ex.,
 *   this.fum = "nom";
 *   // etc...
 *
 *   // Se a função tem uma instrução que
 *   // retorna um objeto, esse objeto será o
 *   // resultado da expressão |new|. Caso contrário,
 *   // o resultado da expressão é o objeto
 *   // atualmente vinculado a |this|
 *   // (i.e., o caso mais comumente visto).
 * }
 */

function C() {
  this.a = 37;
}

var o = new C();
console.log(o.a); // logs 37

function C2() {
  this.a = 37;
  return { a: 38 };
}

o = new C2();
console.log(o.a); // registra 38
```

No último exemplo (C2), porque um objeto foi retornado durante a construção, o novo objeto que this foi vinculado simplesmente é descartado. (Isso essencialmente faz da expressão "this.a = 37;" código morto. Não é exatamente morto, pois ele é executado, mas ele pode ser eliminado sem efeitos colaterais.)

<code>call</code> e <code>apply</code>

Quando uma função usa a palavra-chave this em seu corpo, o seu valor pode ser vinculado a um determinado objeto na chamada utilizando os métodos `call` or `apply` que todas as funções herdam de Function.prototype.

```
function add(c, d) {
  return this.a + this.b + c + d;
}

var o = { a: 1, b: 3 };

// O primeiro parâmetro é o objeto a usar como
// 'this'; subsequentes parâmetros são passados como
// argumentos na função chamada
add.call(o, 5, 7); // 1 + 3 + 5 + 7 = 16

// O primeiro parâmetro é o objeto a usar como
// 'this', o segundo é um arranjo (array) cujos
// membros são usados como argumentos na função chamada
add.apply(o, [10, 20]); // 1 + 3 + 10 + 20 = 34
```

Observe que, com call e apply, se o valor passado como this não for um objeto, será feita uma tentativa de convertê-lo em um objeto usando a operação interna ToObject. Portanto, se o valor passado é um primitivo como 7 ou 'foo', ele será convertido para um objeto usando o construtor relacionado, de modo que o número primitivo 7 é convertido em um objeto, como realizado por new Number(7), e a cadeia de caracteres 'foo' em um objeto, como realizado por new String(' foo '), por exemplo.

```
function bar() {
  console.log(Object.prototype.toString.call(this));
}
```

```
bar.call(7); // [object Number]
```

O método <code>bind</code>

ECMAScript 5 introduziu `Function.prototype.bind`. Chamando f.bind(algumObjeto) cria-se uma nova função com o mesmo corpo e escopo que f, mas onde o this ocorrer na função original, na nova função ele será permanentemente ligado ao primeiro argumento de bind, independentemente de como a função esteja sendo usada.

```
function f(){
  return this.a;
}

var g = f.bind({a:"azerty"});
console.log(g()); // azerty

var h = g.bind({a: 'yoo'}); // bind só funciona uma vez!
console.log.(h()); // azerty

var o = {a:37, f:f, g:g, h: h};
console.log(o.a, o.f(), o.g(), o.h()); // 37, 37, azerty, azerty
```

Como um manipulador de eventos DOM

Quando uma função é usada como um manipulador de eventos, seu this está definido para o elemento do evento a partir do qual foi disparado (alguns navegadores não seguem essa convenção para os listeners adicionados dinamicamente com métodos que não sejam addEventListener).

```
// Quando chamado como listener, transforma o elemento blue
// relacionado
function bluify(e) {
  // sempre true
  console.log(this === e.currentTarget);
  // true quando currentTarget e target são o mesmo objeto
  console.log(this === e.target);
  this.style.backgroundColor = "#A5D9F3";
}

// Obtém uma lista de todo elemento no documento
var elements = document.getElementsByTagName("*");

// Adiciona bluify com um click listener (escutador de click)
// para que quando o elemento seja clicado se torne azul
for (var i = 0; i < elements.length; i++) {
  elements[i].addEventListener("click", bluify, false);
}
```

Em um manipulador de evento in-line (em linha)

Quando o código é chamado de um manipulador de evento in-line, seu this está definido para o elemento DOM em que o listener é colocado:

```
<button onclick="alert(this.tagName.toLowerCase());">Show this</button>
```

O alerta acima mostra button. Note, porém, que apenas o código exterior tem um this definido desta maneira:

```
<button onclick="alert((function(){return this}()));">Show inner this</button>
```

Neste caso, o this da função interior não está definido, portanto ele retorna o objeto global/objeto window (ou seja, o objeto padrão no modo não-estrito onde this não está definido pela chamada).

Veja também

- Strict mode
- All this, um artigo sobre this em diferentes contextos

typeof

Sumário

O operador `typeof` retorna uma string indicando o tipo de um operando.

Sintaxe

O operador `typeof` pode ser utilizado das seguintes maneiras:

```
typeof operando
```

Parâmetros

`operando` é a string, variável, keyword, ou objeto para que o tipo do mesmo seja retornado. O uso de parênteses é opcional.

Descrição

Esta tabela resume os possíveis valores que são retornados pelo `typeof`:

Tipo	Resultado
Undefined	`"undefined"`
Null	`"object"`
Boolean	`"boolean"`
Number	`"number"`
String	`"string"`
Host object (provided by the JS environment)	*Implementation-dependent*
Function object (implements [[Call]] in ECMA-262 terms)	`"function"`
E4X XML object	"xml"
E4X XMLList object	"xml"
Qualquer outro objeto	`"object"`

Exemplos

Casos comuns

```
// Números - Numéricos
typeof 37 === "number";
typeof 3.14 === "number";
typeof Math.LN2 === "number";
typeof Infinity === "number";
typeof NaN === "number"; // Apesar de ser "Não-Numérico"
typeof Number(1) === "number"; // mas nunca utilize desta forma!

// Strings - Seqüências de caracteres
typeof "" === "string";
typeof "bla" === "string";
typeof typeof 1 === "string"; // typeof sempre retorna uma string
typeof String("abc") === "string"; // mas nunca utilize desta forma!

// Booleans - Lógicos booleanos
typeof true === "boolean";
typeof false === "boolean";
typeof Boolean(true) === "boolean"; // mas nunca utilize desta forma!

// Undefined - Indefinidos
typeof undefined === "undefined";
typeof blabla === "undefined"; // uma variável indefinida

// Objetos
typeof { a: 1 } === "object";
typeof [1, 2, 4] === "object"; // use Array.isArray ou Object.prototype.toString.call para diferenciar os objetos das arrays
typeof new Date() === "object";

typeof new Boolean(true) === "object"; // isto é confuso, portanto não use desta forma!
typeof new Number(1) === "object"; // isto é confuso, portanto não use desta forma!
typeof new String("abc") === "object"; // isso também é confuso, assim evite usar esta construção!

// Funções
typeof function () {} === "function";
typeof Math.sin === "function";
```

<code>null</code>

```
// Desde os primóridos do JavaScript
typeof null === "object";
```

Na primeira implementação do JavaScript, valores em JavaScript foram representados com uma tag (etiqueta) de tipo e um valor. A tag de tipo para objetos foi 0. `null` foi representada com o ponteiro NULL (0x00 na maioria das plataformas). Consequentemente, null teve 0 como sua tag de tipo, portanto o `typeof` retorna esse valor. (necessário referência)

Está previsto para ser corrigido na próxima versão do ECMAScript (que estará disponível através de um opt-in). Isso resultará em um `typeof null === 'null'`.

Expressões regulares

Expressões regulares que podem ser chamadas foram uma adição não-padrão em alguns browsers (necessidade de referência para dizer qual).

```
typeof /s/ === "function"; // Chrome 1-12 ... // Não-conforme no ECMAScript 5.1
typeof /s/ === "object"; // Firefox 5+ ...    // Conforme no ECMAScript 5.1
```

Outras peculiaridades

`alert` versões antigas do Internet Explorer

No IE 6, 7 e 8, `typeof alert === 'object'`

Especificação

ECMA-262 section 11.4.3

Veja também

- instanceof

void operator

O **`operador void`** avalia a expressão dada e, em seguida, retorna `undefined`.

Sintaxe

```
void expressão
```

Descrição

Este operador permite a inserção de expressões que produzem efeitos secundários em lugares onde uma expressão que avalia a ação `undefined` é desejada.

O operador `void` é muitas vezes utilizado apenas para obter o valor primitivo `undefined`, geralmente usando "`void(0)`" (o que equivale a "`void 0`"). Nestes casos, a variável global `undefined` pode ser utilizado em vez (supondo que ele não tenha sido atribuído a um valor não-padrão).

Chamada imediata das expressões da função

Quando usamos uma Chamada imediata das expressões da função, valores nulos podem ser usado para forçar a palavra-chave da função a ser tratada como uma expressão em vez de uma declaração.

```
void (function iife() {
  var bar = function () {};
  var baz = function () {};
  var foo = function () {
    bar();
    baz();
  };
  var biz = function () {};

  foo();
  biz();
})();
```

JavaScript URIs

Quando o navegador segue uma URI `javascript`, ele avalia o código na URI e então troca o conteúdo da página pelo valor retornado por este, a não ser que o valor retornado seja `undefined`. O operador `void` pode ser usado para retornar `undefined`. Por exemplo:

```
<a href="javascript:void(0);">Clique aqui para não fazer nada</a>

<a href="javascript:void(document.body.style.backgroundColor='green');">
  Clique aqui para o papel de parede ser verde
</a>
```

Note que no entanto o pseudo protocolo `javascript:` tem seu uso desencorajado perante outras alternativas como *unobtrusive event handlers*.

Veja também

- `undefined`

yield

A palavra-chave `yield` é usada para pausar e resumir uma generator function (`function*` or generator function legada).

Sintaxe

```
[rv] = yield [expressão];
```

`expressão`

Define o valor que retorna de uma generator function via o protocolo iterator. Se omitido, será retornado `undefined`.

`rv`

Retorna o valor opcional passado para o metódo `next()` do generator e resume sua execução.

Descrição

A palavra-chave `yield` pausa a execução de uma generator function e o valor da expressão em frente a palavra-chave `yield` é retornado para a chamada do generator. Ele pode ser considerado uma versão da palavra-chave `return` para o generator.

A palavra-chave `yield` atualmente retorna um objeto do tipo `IteratorResult` com duas propriedades, `value` e `done`. A propriedade `value` é o resultado da avaliação da expressão `yield`, e `done` é `false`, indicando que a generator function não foi totalmente completada.

Uma vez pausado em uma expressão `yield`, a execução do código do generator permanece pausado até a próxima chamada do métod `next()`. Cada vez que o método `next()` do generator é chamado, o generator resume a execução e roda até atingir um dos seguintes estados:

- Um `yield`, que causa a pausa do generator novamente e retorna seu novo valor. Na próxima vez que `next()` for chamado, a execução resumirá com a declaração imediamente após `yield`.
- `throw` é usado para invocar uma exception à partir do generator. Isso irá parar completamente a execução do generator, e a execução resume para onde foi chamado o generator normalmente no caso que uma exception é invocada.
- O final de uma generator function é atingido; nesse caso, a execução do generator termina e um `IteratorResult` é retornado para onde o generator foi chamado no qual o `value` é `undefined` e `done` é `true`.
- Uma declaração `return` é atingida. Nesse caso, a execução do generator termina e um `IteratorResult` é retornado para onde o generator foi chamado no qual o `value` é o valor especificado pela declaração `return` statement e `done` é `true`.

Se um valor opcional é passado para o método `next()` do generator, esse valor se torna o valor retornado pela operação `yield` atual do generator.

Entre o caminho de código do generator, os seus operadores `yield`, e a habilidade de especificar um novo valor inicial passando isso para o `Generator.prototype.next()`, generators oferecem um poder e controle enormes

Exemplos

O código seguinte é a declaração de um exemplo de uma generator function.

```
function* foo() {
  var index = 0;
  while (index <= 2) yield index++;
}
```

Uma vez que a generator function é definida, isso pode ser usada para construir um iterator como mostrado a seguir:

```
var iterator = foo();
console.log(iterator.next()); // { value: 0, done: false }
console.log(iterator.next()); // { value: 1, done: false }
console.log(iterator.next()); // { value: 2, done: false }
console.log(iterator.next()); // { value: undefined, done: true }
```

Notas específicas do Firefox

- A partir do Gecko 29, uma generator function completada não invoca mais um `TypeError` "generator has already finished". Ao invés, isso retorna um objeto `IteratorResult` como `{ value: undefined, done: true }` (Erro do Firefox 958951).
- A partir do Gecko 33, a análise (parsing) de uma expressão `yield` foi atualizada para se conformar com a especificação do ES2015 (Erro do Firefox 981599):
  - A expressão após a palavra-chave `yield` é opcional e omitir isso não invoca mais um `SyntaxError`: `function* foo() { yield; }`

Veja também

- O protocolo Iterator
- `function*`
- `function* expression`
- `yield*`

Escape de classes de caracteres unicode: \p{...}, \P{...}

Um **escape de classe de caracteres unicode** é um tipo de escape de classe de caracteres que corresponde a um conjunto de caracteres especificado por uma propriedade Unicode. Este escape é suportado apenas no modo compatível com Unicode. Quando o marcador `v` está habilitado, também pode ser usado para corresponder com textos de tamanho finito.

Experimente

```
const sentence = "A ticket to 大阪 costs ¥2000 👌.";

const regexpEmojiPresentation = /\p{Emoji_Presentation}/gu;
console.log(sentence.match(regexpEmojiPresentation));
// Expected output: Array ["👌"]

const regexpNonLatin = /\P{Script_Extensions=Latin}+/gu;
console.log(sentence.match(regexpNonLatin));
// Expected output: Array [" ", " ", " 大阪 ", " ¥2000 👌."]

const regexpCurrencyOrPunctuation = /\p{Sc}|\p{P}/gu;
console.log(sentence.match(regexpCurrencyOrPunctuation));
// Expected output: Array ["¥", "."]
```

Sintaxe

```
\p{propriedadeSolitaria}
\P{propriedadeSolitaria}

\p{propriedade=valor}
\P{propriedade=valor}
```

Parâmetros

`propriedadeSolitaria`

Nome ou valor de uma propriedade Unicode solitária, seguindo a mesma sintaxe como `valor`. Ela especifica o valor para a propriedade `General_Category` (Categoria Geral), ou um nome binário de propriedade (en-US). No modo `v`, também pode ser uma propriedade Unicode binária de textos (en-US).

> **Nota:** A sintaxe ICU (en-US) permite omitir o nome da propriedade `Script` também, mas o JavaScript não suporta isso, porque na maioria dos casos a propriedade `Script_Extensions` é mais útil que a `Script`.

`propriedade`

O nome de uma propriedade Unicode. Deve ser composto de letras ASCII (`A–Z`, `a–z`) e underscores (_), e deve ser um dos nome de propriedades não binárias (en-US).

`valor`

O valor de uma propriedade Unicode. Deve ser composto de letras (`A–Z`, `a–z`), underscores (_), e dígitos (`0–9`), e deve ser um dos valores suportados listados em `PropertyValueAliases.txt (en-US)`.

Descrição

`\p` e `\P` são suportados apenas no modo compatível com Unicode. No modo compatível com Unicode, eles são escapes de caracteres para o caractere `p` ou `P`.

Todo caracter Unicode possui um conjunto de propriedades que o descreve. Por exemplo, o caracter `a (en-US)` possui a propriedade `General_Category` com o valor `Lowercase_Letter`, e a propriedade `Script` com o valor `Latn`. As sequências de escape `\p` e `\P` permitem encontrar caracteres com base em suas propriedades. Por exemplo, `a` pode corresponder a `\p{Lowercase_Letter}` (o nome da propriedade `General_Category` é opcional) assim como `\p{Script=Latn}`. `\P` cria uma *classe complementar* que consiste de pontos de código sem a propriedade especificada.

Para compor múltiplas propriedades, use a sintaxe de intersecção de conjunto de caracteres habilitada com o marcador `v`, ou veja padrão de subtração e intersecção.

No modo `v`, `\p` pode corresponder a uma sequências de pontos de código, definida no Unicode como "propriedades de textos". Isto é mais útil para emojis, que são frequentemente compostos por múltiplos pontos de código. Contudo, `\P` pode apenas complementar as propriedades do caracter.

> **Nota:** Existem planos de portar a funcionalidade de propriedades de texto para o modo `u` também.

Exemplos

Categorias gerais

Categorias gerais são utilizada para classificar caracteres Unicode e subcategorias estão disponíveis para definir uma categorização mais precisa. É possível utilizar ambas as formas, curta ou longa, no escape de uma propriedade Unicode.

Elas podem ser utilizadas para corresponder a letras, números, símbolos, pontuações, espaços, etc. Para uma lista mais extensa de categorias gerais, por favor consulte a especificação Unicode (en-US).

```
// encontrando todas as letras de um texto
const estoria =
  "É o Gato Listrado: agora eu deve ter alguém com quem conversar.";

// Forma mais explícita
estoria.match(/\p{General_Category=Letter}/gu);

// Não é obrigatório usar o nome da propriedade para categorias gerais
estoria.match(/\p{Letter}/gu);

// Isso é equivalente (forma curta):
estoria.match(/\p{L}/gu);

// Isso também é equivalente (conjunção de todas as subcategorias usando a forma curta)
estoria.match(/\p{Lu}|\p{Ll}|\p{Lt}|\p{Lm}|\p{Lo}/gu);
```

Escritas e extensões de escritas

Algumas linguagens utilizam escritas diferentes para escrever seus sistemas. Por exemplo, inglês e espanhol são escritos utilizando latim enquanto árabe e russo são escritos utilizando outras escritas (respectivamente árabe e cirílico). As propriedades Unicode `Script` e `Script_Extensions` permitem que expressões regulares correspondam com caracteres de acordo com a escrita com a qual elas são amplamente utilizadas (`Script`) ou de acordo com o conjunto de escritas as quais elas pertencem (`Script_Extensions`).

Por exemplo, `A` pertence a escrita `Latin` (latim) e `ε` a escrita `Greek` (Grega).

```
const caracteresMisturados = "aεЛ";

// Utilizando o canonico nome "longo" da escrita
caracteresMisturados.match(/\p{Script=Latin}/u); // a

// Utilizando a forma curta (código ISO 15924) para a escrita
caracteresMisturados.match(/\p{Script=Grek}/u); // ε

// Utilizando o nome curto sc para a propriedade Script
caracteresMisturados.match(/\p{sc=Cyrillic}/u); // Л
```

Para mais detalhes, consulte a especificação Unicode (en-US), a especificação ECMAScript da tabela de escritas (en-US), e a lista de códigos de escritas ISO 15924 (en-US).

Se o caracter é utilizado em um conjunto limitado de escritas, a propriedade `Script` vai apenas combinar para a escrita "predominantemente" utilizada. Se nós queremos que os caracteres correspondam com base na escrita "não predominante", nós podemos utilizar a propriedade `Script_Extensions` (`Scx` para a forma curta).

```
// ٢ é o dígito 2 na notação Árabe-Índico
// enquanto ela é predominantemente escrita com a escrita Árabe
// ela ainda pode ser escrita na escrita Thaana

"٢".match(/\p{Script=Thaana}/u);
// null já que Thaana não é a escrita predominante

"٢".match(/\p{Script_Extensions=Thaana}/u);
// ["٢", index: 0, input: "٢", groups: undefined]
```

Escape de propriedades Unicode vs. Classes de caracteres

Com expressões regulares no JavaScript, também é possível usar classes de caracteres e especialmente `\w` ou `\d` que correspondem a letras ou dígitos. Contudo, essas formas apenas correspondem a caracteres da escrita *Latin* (em outras palavras, de `a` à `z` e de `A` à `Z` para `\w` e de `0` à `9` para `\d`). Conforme mostrado neste exemplo, o trabalho com textos que não seguem a escrita Latim pode ser um pouco desajeitado.

Categorias de escape de propriedades Unicode abrangem muito mais caracteres e `\p{Letter}` ou `\p{Number}` vai funcionar para qualquer escrita.

```
// Tentando usar faixas de valores para evitar as limitações do \w:

const textoQueNaoELatim = "Приключения Алисы в Стране чудес";
const regexpPalavraBMP = /([\u0000-\u0019\u0021-\uFFFF])+/gu;
// BMP vai de U+0000 à U+FFFF mas o espaço é U+0020

console.table(textoQueNaoELatim.match(regexpPalavraBMP));

// Usando escapes de propriedades Unicode ao invés disso
const regexpUPE = /\p{L}+/gu;
console.table(textoQueNaoELatim.match(regexpUPE));
```

Correspondendo à preços

O exemplo a seguir corresponde a preços em um texto:

```
function pegarPrecos(str) {
  // Sc significa "currency symbol" (Símbolo de moeda)
  return [...str.matchAll(/\p{Sc}\s*[\d.,]+/gu)].map((match) => match[0]);
}

const str = `Rolinhos California $6.99
Rolinhos crocantes $8.49
Tempura de camarão $10.99`;
console.log(pegarPrecos(str)); // ["$6.99", "$8.49", "$10.99"]

const str2 = `Loja nos EUA $19.99
Loja na Europa €18.99
Loja no Japão ¥2000`;
console.log(pegarPrecos(str2)); // ["$19.99", "€18.99", "¥2000"]
```

Correspondendo à textos

Com o marcador `v`, `\p{…}` pode corresponder a textos que são potencialmente mais longos do que um caracter utilizando as propriedades de textos:

```
const marcador = "🇺🇳";
console.log(marcador.length); // 2
console.log(/\p{RGI_Emoji_Flag_Sequence}/v.exec(marcador)); // [ '🇺🇳' ]
```

Contudo, você não pode usar `\P` para corresponder a "um texto que não tem uma propriedade", porque não é claro quantos caracteres devem ser usados.

```
/\P{RGI_Emoji_Flag_Sequence}/v; // SyntaxError: Invalid regular expression: Invalid property name
```

Veja também

- Guia Classes de caracteres
- Expressões regulares
- Classe de caracter: `[...]`, `[^...]`
- Espace de classe de caracter: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`
- Escape de caracter: `\n`, `\u{...}`
- Disjunção: `|`
- Propriedade de caracter Unicode (en-US) na Wikipédia
- ES2018: Escapes de propriedades de RegExp Unicode (en-US) por Dr. Axel Rauschmayer (2017)
- Propriedades § de expressões regulares Unicode (en-US)
- Utilidades do Unicode: UnicodeSet (en-US)
- RegExp marcador v com conjunto de notações e propriedades de texto (en-US) no v8.dev (2022)

Instruções e declarações

Aplicações em JavaScript são realizadas através de instruções com uma sintaxe apropriada. Uma única instrução pode abranger multiplas linhas. Multiplas instruções podem estar contidas em uma única linha se cada instrução estiver separada por ponto e vírgula (;).

Instruções e declarações por categoria

Para visualizar em ordem alfabética use a barra de rolagem à esquerda.

Control flow

`Block`

Uma instrução de bloqueio é usada para um grupo de zero ou mais instruções. O bloqueio é delimitado por um par de chaves.

`break`

Termina o atual loop, troca, ou nomeia instruções e transfere o controle do programa para a próxima instrução após a que foi terminada agora.

`continue`

Termina a execução das instruções na atual iteração do atual loop, e continua a execução do loop com a seguinte iteração.

`Empty`

Uma instrução vazia é usada para gerar uma "não-instrução", ainda que o JavaScrpit espere uma.

`if...else`

Executa uma instrução se a condição especificada é verdadeira. Se a condição é falsa, outra instrução pode ser executada.

`switch`

Retorna uma expressão, combinando o valor da expressão a condição do caso, e executa a condição associada a este caso.

`throw`

Aplica uma exceção definida pelo usuário.

`try...catch`

Marca um bloco de instruções para tentar, e especifica a resposta, uma exceção deve ser retornada.

Declarações

`var`

Declara uma variável, opcionalmente inicializando com um valor.

`let`

Declara acesso ao local da variável, opcionalmente inicializando com um valor.

`const`

Declara uma constante de apenas leitura.

Funções e classes

`function`

Declara uma função com os parâmetros especificos.

`function*`

Gera funções habilitando iteradores de escrita mais facilmente.

`return`

Especifica o valor a ser retornado por uma função.

`class`

Declara uma classe.

Iterações

`do...while`

Cria um loop que executa uma especifica instrução até que a condição de teste seja falsa. A condição é retornada depois da execução da instrução, resultando na instrução especificada executando ao menos uma vez.

`for`

Cria um loop que consiste em três opções de expressões, entre parenteses e separado por ponto e vírgula, seguido pela instrução executada no loop.

`for each...in`

Itera uma variável especificada sobre todos os valores das propriedades do objeto. Para cada propriedade distinta, uma instrução especificada é executada.

`for...in`

Itera através de enumeráveis propriedades de um objeto, em ordem arbitrária. Para cada propriedade distinta, instruções podem ser executadas.

`for...of`

Itera através de objetos iteráveis (incluindo arrays, array-like objects, iterators and generators), invocando uma iteração personalizada com instruções para serem executadas pelo valor de cada propriedade.

`while`

Cria um loop que executa uma especifica instrução enquanto o teste da condição for positivo. A condição é retornada antes de executar a instrução.

Outros

`debugger`

Invoca qualquer funcionalidade de depuração disponível. Se não há funcionabilidade disponível, a instrução não tem efeito.

`export`

Usado para exportar funções que os deixa disponível para importação em módulos externos, outros scripts.

`import`

Usado para importar funções exportadas de um módulo externo, outro script.

`label`

Fornece uma instrução com um identificador que pode ser referido usando `break` ou `continue` instruções.

`with`

Extende o alcance de uma instrução.

Veja também

- Operators

Funções assíncronas

A declaração **`async function`** define uma *função assíncrona*, que retorna um objeto `AsyncFunction`.

Você também pode definir funções assíncronas usando uma `expressão async function`.

Sintaxe

```
async function nome([param[, param[, ... param]]]) {
   instruções
}
```

`nome`

O nome da função.

`param`

O nome de um parâmetro a ser passado para a função.

`instruções`

As instruções que compõem o corpo da função.

Descrição

Quando uma função assíncrona é chamada, ela retorna uma `Promise`. Quando a função assíncrona retorna um valor, a `Promise` será resolvida com o valor retornado. Quando a função assíncrona lança uma exceção ou algum valor, a `Promise` será rejeitada com o valor lançado.

Uma função assíncrona pode conter uma expressão `await`, que pausa a execução da função assíncrona e espera pela resolução da `Promise` passada, e depois retoma a execução da função assíncrona e retorna o valor resolvido.

> **Nota:** A proposta das funções `async/await` é de simplificar o uso de forma síncrona das `Promises` e executar alguns procedimentos em um grupo de `Promises`. Assim como `Promises` são similares a `callbacks` estruturados, funções `async/await` são similares à junção de `generators` com `Promises`.

Exemplos

Exemplo simples

```
function resolverDepoisDe2Segundos(x) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

async function adicionar1(x) {
  var a = resolverDepoisDe2Segundos(20);
  var b = resolverDepoisDe2Segundos(30);
  return x + (await a) + (await b);
}

adicionar1(10).then((v) => {
  console.log(v); // exibe 60 depois de 2 segundos.
});

async function adicionar2(x) {
  var a = await resolverDepoisDe2Segundos(20);
  var b = await resolverDepoisDe2Segundos(30);
  return x + a + b;
}

adicionar2(10).then((v) => {
  console.log(v); // exibe 60 depois de 4 segundos.
});
```

Reescrevendo uma cadeia de <code>Promise</code> com uma função <code>async</code>

Uma API que retorna uma `Promise` vai resultar em uma cadeia de `Promises` e separa a função em várias partes. Considere o seguinte código:

```
function pegarDadosProcessados(url) {
  return baixarDados(url) // retorna uma Promise
    .catch((e) => {
      return baixarDadosReservas(url); // retorna uma Promise
    })
    .then((v) => {
      return processarDadosNoWorker(v); // retorna uma Promise
    });
}
```

pode ser escrita em uma única função `async` desta forma:

```
async function pegarDadosProcessados(url) {
  let v;
  try {
    v = await baixarDados(url);
  } catch (e) {
    v = await baixarDadosReservas(url);
  }
  return processarDadosNoWorker(v);
}
```

Note que no exemplo acima não tem a instrução `await` na instrução do `return`, porque o valor retornado de uma função `async` é implícitamente passado por um `Promise.resolve`.

Notas específicas do Firefox

- A função expression closure syntax não é permitida com funções assíncronas e irão lançar a exceção SyntaxError a partir do Firefox 55.

Veja também

- async_function
- AsyncFunction
- await

block

Uma **declaração em bloco** (ou **declaração composta (compound)** em outras linguagens) é usado para agrupar nenhum ou mais declarações. O bloco é delimitado por um par de chaves e pode opcionalmente ser `nomeado`:

Sintaxe

Declaração de Bloco

```
{
  ListaDeDeclarações
}
```

Declaração de Bloco Nomeada

```
NomeIdentificador: {
  ListaDeDeclarações
}
```

`ListaDeDeclarações`

Declarações agroupadas com a declaração em bloco.

`NomeIdentificador`

Um `nome (label)` opcional para identificação visual ou um alvo para `break`.

Descrição

A declaração em bloco é frequentemente chamada de **declaração composta (compound)** em outras linguagens. Ela permite que você use multiplas declarações onde o JavaScript espera apenas uma declaração. Combinar declarações em blocos são uma prática comum em JavaScript. O comportamento oposto é possível usando uma declaração vazia, onde você fornece nenhuma declaração, mesmo que uma seja requerida.

Regras de Bloqueio de Escopo

Com `var`

Variáveis declaradas com `var` **não** possuem bloqueio de escopo. Variáveis introduzidas com um bloco são guardadas no escopo da função que elas estão ou script, e os efeitos de configura-las persistem pelo próprio bloco. Em outras palavras, declarações em bloco não introduzem um escopo. Mesmo blocos "standalone" são sintaxes válidas, você não vai querer usar blocos standalone em JavaScript, porque não fazem o que você pensa que eles fazem, se você pensa que eles fazer coisas como blocos em C ou Java. Por exemplo:

```
var x = 1;
{
  var x = 2;
}
console.log(x); // imprime 2
```

Isso loga 2 porque a declaração `var x` com o bloco está no mesmo escopo que a declaração `var x` antes do bloco. No C ou Java, o código equivalente iria imprimir 1.

Com `let` e `const`

Por contraste, variáveis declaradas com `let` e `const` **tem** escopo bloqueado:

```
let x = 1;
{
  let x = 2;
}
console.log(x); // imprime 1 1
```

`O x = 2` é limitado ao escopo do bloco em que ele foi definido.

O mesmo true de `const`:

```
const c = 1;
{
  const c = 2;
}
console.log(c); // imprime 1 e não invoca uma throw SyntaxError...
```

Note que o escope bloqueado de `const c = 2` *não invoca uma* `SyntaxError: Identifier 'c' has already been declared` porque ele foi unicamente declarado no bloco.

Veja também

- `while`
- `if...else`
- `let`

break

Sumário

O comando **break** encerra o loop atual, `switch`, ou o loop que foi informado no `label` e transfere o controle da execução do programa para o comando seguinte.

Síntaxe

```
break [label];
```

`label`

Opcional. Identificador associado ao label de um comando. Se a estrutura não for um loop ou `switch`, ele será um pré-requisito.

Descrição

O comando `break` inclui um label opcional que permite ao programa encerrar a execução da estrutura que possui o nome informado na label. O comando `break` deve estar dentro dessa estrutura informada no label. A estrutura que possui o nome informada na label pode ser qualquer comando `block`; não é necessário que seja precedida por um loop.

Exemplos

A função a seguir possui um comando `break` que encerra o loop `while` quando a variável i vale 3, e então retorna o valor 3 * `x`.

```
function testaBreak(x) {
  var i = 0;

  while (i < 6) {
    if (i == 3) {
      break;
    }
    i += 1;
  }
  return i * x;
}
```

O código a seguir possui o comando break dentro de uma estrutura nomeada. O comando `break` deverá estar dentro da estrutura na qual o label se refere. Veja que `inner_block` está dentro de `outer_block`.

```
bloco_externo: {
  bloco_interno: {
    console.log("1");
    break bloco_externo; // encerra bloco_interno e bloco_externok
    console.log(":-("); // não é executado
  }

  console.log("2"); // não é executado
}
```

O código a seguir também utiliza o comando break com blocos nomeados mas gera um erro de sintaxe pois o comando break está dentro do bloco_1 mas faz uma referência ao bloco_2. Um comando break sempre deverá estar dentro da estrutura nomeada na qual fizer referência.

```
bloco_1:{
  console.log ('1');
  break bloco_2;            // SyntaxError: label not found
}

bloco_2:{
  console.log ('2');
}
```

Veja também

- `continue`
- `label`
- `switch`

class

A **declaração class** cria uma nova classe com dado **nome** usando a herança do protótipo base.

Você também pode definir uma classe usando class expression.

Sintaxe

```
class name [extends] {
  // class body
}
```

Descrição

Assim como as expressões de classe, o corpo de uma declaração de classe é executado em strict mode.

Declarações de classe não são hoisted (ao contrário de function declarations).

Exemplos

Declaração simples de classe

No exemplo a seguir, primeiro definimos uma classe chamada *Polygon*, então extendemos essa classe para criar uma classe chamada *Square*. Note que *super()*, usada no construtor, só pode ser usada em construtores e deve ser chamada antes da chave *this* ser usada.

```
class Poligono {
  constructor(altura, largura) {
    this.nome = "Polígono";
    this.altura = altura;
    this.largura = largura;
  }
```

```
}

class Quadrado extends Poligono {
  constructor(altura) {
    super(altura, altura);
    this.nome = "Quadrado";
  }
}
```

Veja também

- `function` declaration
- `class` expression
- Classes

const

Constantes possuem escopo de bloco, semelhantes às variáveis declaradas usando o palavra-chave `let`. O valor de uma constante não pode ser alterado por uma atribuição, e ela não pode ser redeclarada.

Experimente

```
const number = 42;

try {
  number = 99;
} catch (err) {
  console.log(err);
  // Expected output: TypeError: invalid assignment to const 'number'
  // (Note: the exact output may be browser-dependent)
}

console.log(number);
// Expected output: 42
```

Resumo

A **declaração** `const` cria uma variável cujo o valor é fixo, ou seja, uma constante somente leitura. Isso não significa que o valor é imutável, apenas que a variável constante não pode ser alterada ou retribuída.

Sintaxe

```
const name1 = value1 [, name2 = value2 [, ... [, nameN = valueN]]]];
```

`nameN`

Nome da constante. Pode ser qualquer identificador válido.

`valueN`

Valor atribuido a constante. Pode ser qualquer expressão válida, incluindo uma função.

Descrição

Esta declaração cria uma constante que pode pertencer tanto ao escopo global (na janela ou objeto) quanto ao local do bloco em que é declarada. Constantes globais não se tornam propriedades do objeto `window`, diferente da criação de variáveis com `var`.

Toda constante requer um inicializador, ou seja, é preciso especificar um valor para a constante no momento em que ela é declarada (o que faz sentido, uma vez que esse valor não pode ser alterado).

A declaração `const` cria uma referência somente leitura a um valor. Isso **não** significa que esse valor é imutável, apenas que o identificador da variável constante não pode ser alterado. Se o conteúdo do identificador for um objeto, isso significa que o conteúdo do objeto (ex. seus parâmetros) podem ser alterados.

Todas as considerações de "temporal dead zone" se aplicam tanto a `let` quanto a `const`.

Uma constante não pode ter o mesmo nome que uma função ou variável que esteja no mesmo escopo.

Exemplos

O exemplo abaixo demonstra o comportamento de uma constante. Experimente executá-lo no console do seu navegador.

```
// NOTA: constantes podem ser declaradas em caixa alta ou baixa,
// mas uma convenção comum é usar apenas caixa alta

// define MY_FAV como uma constante e lhe atribui o valor 7
const MY_FAV = 7;

// isto falha mas não emite erros no Firefox e Chrome (porém não falha no Safari)
MY_FAV = 20;

// a variável MY_FAV possui o valor 7
console.log("my favorite number is: " + MY_FAV);
```

```
// tentar redeclarar a constante emite um erro - Uncaught SyntaxError: Identifier 'MY_FAV' has already been declared
const MY_FAV = 20;

// o nome MY_FAV está reservado para a constante acima, logo também irá falhar
var MY_FAV = 20;

// isso também vai falhar
let MY_FAV = 20;

// É importante notar a natureza de escopo por bloco
if (MY_FAV === 7) {
  // não tem problema fazer isso, pois cria uma variável de bloco MY_FAV
  // com escopo local (o nome MY_FAV poderia ser usado com let também)
  let MY_FAV = 20;

  // MY_FAV agora é 20
  console.log("meu número favorito é " + MY_FAV);

  // isso retorna um erro, pois tenta registrar a variável no contexto global
  var MY_FAV = 20;
}

//MY_FAV ainda é 7
console.log("meu número favorito é " + MY_FAV);

// const deve ser inicializada
const FOO; // SyntaxError: missing = in const declaration

// const também funciona com objetos
const MY_OBJECT = { key: "value" };

// Sobrescrever o objeto também falha (no Firefox e Chrome mas não no Safari) - Uncaught TypeError: Assignment to constant variable.
MY_OBJECT = { OTHER_KEY: "value" };

// Entretanto, atributos de objetos não estão protegidos,
// logo a seguinte instrução é executada sem problemas
MY_OBJECT.key = "otherValue"; // Utilize Object.freeze() se quiser tornar um objeto imutável

// o mesmo se aplica para arrays
const MY_ARRAY = [];
// É possível utilizar push para incluir itens no array
MY_ARRAY.push("A"); //["A"]
// Todavia, atribuir um novo array para a variável gera um erro
MY_ARRAY = ["B"];
```

Observações

Em versões anteriores do Firefox & Chrome e a partir de Safari 5.1.7 e Opera 12.00, se você define uma variável com `const`, você ainda consegue alterar o valor depois. Este recurso não é suportado no Internet Explorer 6-10, mas está incluído no Internet Explorer 11.

Observações específicas para Firefox

A declaração `const` foi implementada no Firefox muito antes de `const` aparecer na especificação ECMAScript 6. For `const` ES6 compliance see Erro do Firefox 950547 and Erro do Firefox 611388.

- Iniciando com o Gecko 36:
  - `{const a=1};a` passa a retornar `ReferenceError` e não retorna `1` devido block-scoping.
  - `const a;` passa a retornar `SyntaxError` ("missing = in const declaration"): É necessário incializar a constante.
  - `const a = 1; a = 2;` passa a retornar `SyntaxError` ("invalid assignment to const a").

Veja também

- `var`
- `let`
- Guia de constantes em JavaScript

continue

A palavra chave **continue** termina a atual iteração do laço em que ele se encontra ou de um laço rotulado, e continua a execução deste laço com a próxima iteração.

Syntax

```
continue [rótulo];
```

`rótulo`

Identificador associado ao laço.

Descrição

Diferentemente do `break`, o `continue` não termina a execução do laço completamente, em vez disso ele:

- Volta à condição, em um laço do tipo `while`.
- Volta à expressão, atualizando-a, em um laço do tipo `for`.

O `continue` pode incluir, opcionalmente, um rótulo que premite ao programa pular para a próxima iteração de um laço rotulado em vez de pular o loop em que ele se encontra. Neste caso, o `continue` necessita estar dentro deste laço rotulado.

Exemplos

Usando <code>continue</code> com <code>while</code>

O exemplo abaixo mostra um laço `while` que tem um `continue` que será executado quando o valor de `i` for 3. Assim, `n` terá os valores 1, 3, 7, e 12.

```
var i = 0;
var n = 0;

while (i < 5) {
  i++;

  if (i === 3) {
    continue;
  }

  n += i;
}
```

Usando o <code>continue</code> com um rótulo

No exemplo abaixo, um laço rotulado como `checkiandj` contém o laço rotulado `checkj`. Se o `continue` for alcançado, o programa continua a execução voltando ao topo do rótulo `checkj`. Cada vez que o `continue` for alcançado, `checkj` reiterará até sua condição for falsa. Quando retornar `false`, o restante de `checkiandj` será executado.

Se o `continue` tivesse o rótulo `checkiandj`, o programa iria continuar retornando a execução ao label `checkiandj`.

Veja também `label`.

```
var i = 0;
var j = 8;

checkiandj: while (i < 4) {
  console.log("i: " + i);
  i += 1;

  checkj: while (j > 4) {
    console.log("j: " + j);
    j -= 1;

    if (j % 2 == 0) continue checkj;
    console.log(j + " is odd.");
  }
  console.log("i = " + i);
  console.log("j = " + j);
}
```

Output:

```
"i: 0";

// start checkj
"j: 8";
"7 is odd.";
"j: 7";
"j: 6";
"5 is odd.";
"j: 5";
// end checkj

"i = 1";
"j = 4";

"i: 1";
"i = 2";
"j = 4";

"i: 2";
"i = 3";
"j = 4";

"i: 3";
"i = 4";
"j = 4";
```

Veja também

- `break`
- `label`

debugger

A **declaração debug** invoca qualquer funcionalidade de depuração disponivel, como definir um breakpoint. Se nenhuma funcionalidade de depuração estiver disponivel, essa declaração não tem nenhum efeito.

Sintaxe

```
debugger;
```

Exemplos

O exemplo a seguir mostra o código onde uma declaração debugger foi inserida para invocar um depurador (se um existir) quando a função for chamada.

```
function codigoPotencialmenteBugado() {
  debugger;
  // faça atividades de depuração como examinar, pular um passo etc.
}
```

Quando o depurador é invocado, a execução é parada na declaração debugger. É como um breakpoint no código-fonte.



Veja também

- Depurando JavaScript
- O Depurador nas Ferramentas de Desenvolvedor do Firefox

do...while

A declaração **`do...while`** cria um laço que executa uma declaração até que o teste da condição for falsa (false). A condição é avaliada depois que o bloco de código é executado, resultando que uma declaração seja executada pelo menos uma vez.

Sintaxe

```
do
   statement
while (condition);
```

`declarações`

> A declaração é executada pelo menos uma vez e re-executada cada vez que a condição (`condition`) for avaliada como verdadeira (true). Para executar múltiplas declarações dentro do laço, use um `block` declaração (`{ ... }`) ao grupo dessas declarações.

`condição`

> Uma expressão é validade depois de cada passagem pelo laço. Se a condição `(condition)` é avaliada como verdadeira (true) o bloco de código é executado novamente. Quando a condição `(condition)` é avaliada como falsa (false), o controle passa para a instrução seguinte ao laço **do...while**.

Exemplos

Usando <code>do...while</code>

No exemplo seguinte, o laço **do...while** soma pelo menos uma vez e executa novamente até `i` não ser menor que 5.

Conteúdo HTML

```
<div id="exemplo"></div>
```

Conteúdo JavaScript

```
var resultado = "";
var i = 0;
do {
  i += 1;
  resultado += i + " ";
} while (i < 5);
document.getElementById("exemplo").innerHTML = resultado;
```

Resultado

Veja também

- `while`
- `for`

empty

Uma **declaração vazia** é usada para fornecer nenhuma declaração, embora a sintaxe do JavaScript esperasse uma.

Experimente

```
const array1 = [1, 2, 3];

// Assign all array values to 0
```

```
for (let i = 0; i < array1.length; array1[i++] = 0 /* empty statement */);

console.log(array1);
// Expected output: Array [0, 0, 0]
```

Sintaxe

```
;
```

Descrição

A instrução vazia é um ponto-e-vírgula (;) indicando que nenhuma instrução será executada, mesmo se a sintaxe do JavaScript exigir uma.

O comportamento oposto, em que você deseja várias instruções, mas o JavaScript permite apenas uma única, é possível usando uma instrução de bloco ; combina várias declarações em uma única.

Exemplos

A instrução vazia às vezes é usada com instruções de loop. Veja o exemplo a seguir com um corpo de loop vazio:

```
var arr = [1, 2, 3];

// Assign all array values to 0
for (i = 0; i < arr.length; arr[i++] = 0 /* empty statement */);

console.log(arr);
// [0, 0, 0]
```

Nota: É uma boa ideia comentar o uso intencional da declaração vazia, pois não é realmente óbvio distinguir entre um ponto e vírgula normal. No exemplo a seguir, o uso provavelmente não é intencional:

```
if (condition) // Caution, this "if" does nothing!
  killTheUniverse(); // So this gets always executed!!!
```

Outro Exemplo: Uma declaração `if...else` declaração sem chaves ({ }). Se três for verdadeiro, nada acontecerá, quatro não importa e também a função launchRocket ( ) no caso contrário não será executada.

```
if (one) doOne();
else if (two) doTwo();
else if (three) // nothing here
else if (four) doFour();
else launchRocket();
```

Veja também

- `Block statement`

export

O export é utilizado quando criamos módulos JavaScript para exportar ligações em tempo real para suas funções, objetos ou valores primitivos de um módulo sejam utilizados por outros programas através de declarações `import`. Ligações que são exportadas ainda podem ser modificadas localmente; quando importadas, embora elas possam ser lidas somente pelo módulo que as importou, seu valor é atualizado sempre que ela modificada pelo módulo que a exportou.

Módulos exportados ficam em `strict mode`, independentemente se é declarado dessa forma, ou não. Export não pode ser utilizado em scripts embutidos.

Sintaxe

Há dois tipos de exportação:

1. Exportações Explícitas (Named Exports) (Zero ou mais exports por módulo)
2. Exportações Padrão (Default Exports) (Uma por módulo)

```
// Exportando recursos individuais
export let name1, name2, …, nameN; // também var, const
export let name1 = …, name2 = …, …, nameN; // também var, const
export function functionName(){...}
export class ClassName {...}

// Lista de exportações
export { name1, name2, …, nameN };

// Renomeando exports
export { variable1 as name1, variable2 as name2, …, nameN };

// Exportando atribuições desestruturadas renomeando
export const { name1, name2: bar } = o;

// Exportações Padrão (Default exports)
export default expression;
export default function (…) { … } // também class, function*
export default function name1(…) { … } // também class, function*
export { name1 as default, … };
```

```
// Agregando módulos
export * from …; // não define a exportação padrão
export * as name1 from …; // Draft ECMAScript® 2021
export { name1, name2, …, nameN } from …;
export { import1 as name1, import2 as name2, …, nameN } from …;
export { default } from …;
```

`nameN`

Identificador para ser exportado (assim ele pode ser importado via `import` em outro script).

Descrição

Há dois diferentes tipos de export, explícito(named) e padrão(default). Pode-se ter várias exportações explícitas por módulo, mas apenas uma padrão. Cada tipo corresponde à uma da síntaxe acima:

Exportações explícitas:

```
// exporta recursos declarados anteriomente
export { myFunction, myVariable };

// exporta recursos individuais (pode exportar var, let,
// const, function, class)
export let myVariable = Math.sqrt(2);
export function myFunction() { ... };
```

Exportação padrão (pode ser feita apenas uma por script):

```
// exporta um recurso declarado anteriormente como padrão
export { myFunction as default };

// exporta recursos individuais como padrão
export default function () { ... }
export default class { .. }
```

Exportações explícitas são úteis para exportar vários valores. Durante a importação, é obrigatório usar o mesmo nome do objeto correspondente.

Mas a exportação padrão pode ser importada com qualquer nome, por exemplo:

```
// arquivo test.js
let k;
export default (k = 12);
```

```
// algum outro arquivo
import m from "./test"; // note que temos a liberdade de usar import m ao invés de import k, porque k era uma exportação padrão
console.log(m); // vai retornar log 12
```

Você também pode renomear exportações explícitas para evitar conflitos e nome:

```
export { myFunction as function1, myVariable as variable };
```

Re-exportando / Agregando

É possível também "importar/exportar" de módulos diferentes em um módulo pai, de modo que eles estejam disponíveis para serem importados daquele módulo. Em outras palavras, pode-se criar um módulo único concentrando várias exportações de vários módulos.

Isto pode ser feito com a sintaxe "export from":

```
export { default as function1, function2 } from "bar.js";
```

O que é comparável com um combinação de import e export:

```
import { default as function1, function2 } from "bar.js";
export { function1, function2 };
```

Mas onde `function1` e `function2` não ficam disponíveis dentro do módulo atual.

> **Nota:** Os exemplos a seguir são sintaticamente inválidos apesar de sua equivalência com o import:

```
import DefaultExport from "bar.js"; // Válido
```

```
export DefaultExport from "bar.js"; // Inválido
```

O modo correto de fazer isso e renomeando o export:

```
export { default as DefaultExport } from "bar.js";
```

Exemplos

Usando exportações explícitas

Em um módulo `my-module.js` poderiamos usar o seguinte código:

```
// módulo "my-module.js"
function cube(x) {
  return x * x * x;
}

const foo = Math.PI + Math.SQRT2;

var graph = {
  options: {
    color: "white",
    thickness: "2px",
  },
  draw: function () {
    console.log("Da função draw de graph");
  },
};

export { cube, foo, graph };
```

Então, no módulo principal incluído sem sua página HTML, poderíamos ter:

```
import { cube, foo, graph } from "./my-module.js";

graph.options = {
  color: "blue",
  thickness: "3px",
};

graph.draw();
console.log(cube(3)); // 27
console.log(foo); // 4.555806215962888
```

É importante notar o seguinte:

- Você pode incluir esse script no seu código HTML através do elemento `<script>` do tipo="module", de modo que ele seja reconhecido e tratado apropriadamente.
- Você não pode executar módulos JS através de `file://` URL — você receberá errors CORS. Você deve rodá-los através de um servidor HTTP.

Usando a exportação padrão

Se queremos exportar um valor sozinho ou obter um valor de reserva para o nosso módulo, nós poderiamos usar `export default`:

```
// módulo "my-module.js"
export default function cube(x) {
  return x * x * x;
}
```

Daí em outro script podemos usar:

```
import cube from "my-module";
console.log(cube(3)); // 27
```

Veja também

- `import`
- ES6 in Depth: Modules, Hacks blog post by Jason Orendorff
- Axel Rauschmayer's book: "Exploring JS: Modules"

for

A instrução `for` cria um loop que consiste em três expressões opcionais, dentro de parênteses e separadas por ponto e vírgula, seguidas por uma declaração ou uma sequência de declarações executadas em sequência.

Sintaxe

```
for ([inicialização]; [condição]; [expressão final])
   declaração
```

`inicialização`

> Uma expressão (incluindo expressões de atribuição) ou declarações variáveis. Geralmente usada para iniciar o contador de variáveis. Esta expressão pode, opcionalmente, declarar novas variáveis com a palavra chave `var`. Essas variáveis não são locais no loop, isto é, elas estão no mesmo escopo que o loop `for` está. Variáveis declaradas com let são locais para a declaração.
>
> O resultado desta expressão é descartado.

`condição`

Uma expressão para ser avaliada antes de cada iteração do loop. Se esta expressão for avaliada para true, `declaração` será executado. Este teste da condição é opcional. Se omitido, a condição sempre será avaliada como verdadeira. Se a expressão for avaliada como falsa, a execução irá para a primeira expressão após a construção loop `for`.

`expressão final`

Uma expressão que será validada no final de cada iteração de loop. Isso ocorre antes da próxima avaliação da condição. Geralmente usado para atualizar ou incrementar a variável do contador.

`declaração`

Uma declaração que é executada enquanto a condição for verdadeira. Para executar múltiplas condições dentro do loop, use uma instrução de bloco (`{...}`) para agrupar essas condições. Para não executar declarações dentro do loop, use uma instrução vazia (`;`).

Exemplos de uso

Usando <code>for</code>

A declaração `for` começa declarando a variável `i` e inicializando-a como `0`. Ela verifica se `i` é menor que nove, executa as duas instruções subsequentes e incrementa 1 a variável `i` após cada passagem pelo loop.

```
for (var i = 0; i < 9; i++) {
  console.log(i);
  // more statements
}
```

Expressões <code>for</code> opcionais

Todas as três expressões na condição do loop `for` são opcionais.

Por exemplo, no bloco de *inicialização*, não é necessário inicializar variáveis:

```
var i = 0;
for (; i < 9; i++) {
  console.log(i);
  // more statements
}
```

Assim como ocorre no bloco de *inicialização*, a *condição* também é opcional. Se você está omitindo essa expressão, você deve certificar-se de quebrar o loop no corpo para não criar um loop infinito.

```
for (var i = 0; ; i++) {
  console.log(i);
  if (i > 3) break;
  // more statements
}
```

Você também pode omitir todos os três blocos. Novamente, certifique-se de usar uma instrução `break` no final do loop e também modificar (incrementar) uma variável, para que a condição do `break` seja verdadeira em algum momento.

```
var i = 0;

for (;;) {
  if (i > 3) break;
  console.log(i);
  i++;
}
```

Usando <code>for</code> sem uma declaração

O ciclo `for` a seguir calcula a posição de deslocamento de um nó na seção [expressão final] e, portanto, não requer o uso de uma declaração ou de um bloco de declaração, e no seu lugar é usado um ponto-vírgula - `;`.

```
function showOffsetPos(sId) {
  var nLeft = 0,
    nTop = 0;

  for (
    var oItNode = document.getElementById(sId); // inicialização
    oItNode; // condition
    nLeft += oItNode.offsetLeft,
      nTop += oItNode.offsetTop,
      oItNode = oItNode.offsetParent // expressão final
  ); /* empty statement */

  console.log(
    'Offset position of "' +
      sId +
      '" element:\n left: ' +
      nLeft +
```

```
      "px;\n top: " +
      nTop +
      "px;",
  );
}

// Exemplo de call:

showOffsetPos("content");

// Resultado:
// "Offset position of "content" element:
// left: 0px;
// top: 153px;"
```

> **Nota:** Nesse caso, quando você não usa a seção de declaração, **o ponto-e-vírgula é colocada imediatamente após a declaração do ciclo**.

Veja também

- `break`
- `continue`
- `empty`
- `while`
- `do...while`
- `for...in`
- `for...of`

for await...of

A **declaração** `for await...of` cria um loop que itera sobre objetos iteráveis assíncronos, bem como sobre iteráveis síncronos, incluindo: `String`, `Array`, `Array`-como objetos (e.g., `arguments` or `NodeList`), `TypedArray`, `Map`, `Set`, e iteráveis async/sync. Invoca um hook de iteração personalizado com instruções a serem executadas para o valor de cada propriedade do objeto.

Sintaxe

```
for await (variável of iterável) {
  // declaração
}
```

`variável`

Em cada iteração, o valor de uma propriedade diferente é atribuído à *variável*. A *variável* pode ser declarada como `const`, `let` ou `var`.

`iterável`

Objeto cujas propriedades iteráveis devem ser iteradas.

Iterando sobre iteráveis assíncronos

Você também pode iterar sobre um objeto que explicidamente implementa protocolo iterável assíncrono(*async iterable protocol*):

```
var asyncIterable = {
  [Symbol.asyncIterator]() {
    return {
      i: 0,
      next() {
        if (this.i < 3) {
          return Promise.resolve({ value: this.i++, done: false });
        }

        return Promise.resolve({ done: true });
      },
    };
  },
};

(async function () {
  for await (let num of asyncIterable) {
    console.log(num);
  }
})();

// 0
// 1
// 2
```

Iterando sobre generators assíncronos

Como os geradores assíncronos implementam o protocolo assíncrono Iterator, eles podem fazer um loop usando `for await... of`

```
async function* asyncGenerator() {
  var i = 0;
  while (i < 3) {
    yield i++;
  }
}

(async function () {
  for await (let num of asyncGenerator()) {
    console.log(num);
  }
})();
// 0
// 1
// 2
```

Para termos um exemplo mais concreto de iteração sobre um generator assíncrono usando `for await... of`, considere iterar sobre dados obtidos através de um fecth de uma API. Este exemplo cria primeiro um iterador assíncrono para um stream de dados e depois usa-o para encontrar o tamanho da resposta da API.

```
async function* streamAsyncIterator(stream) {
  const reader = stream.getReader();
  try {
    while (true) {
      const { done, value } = await reader.read();
      if (done) {
        return;
      }
      yield value;
    }
  } finally {
    reader.releaseLock();
  }
}
// Obtém dados do URL e calcula o tamanho da resposta usando o generator assíncrono
async function getResponseSize(url) {
  const response = await fetch(url);
  // Guardará o tamanho do response em bytes.
  let responseSize = 0;
  // O for-wait-loop irá iterar de forma assíncrona sobre cada parte do response.
  for await (const chunk of streamAsyncIterator(response.body)) {
    // Incrementa o valor do responseSize
    responseSize += chunk.length;
  }

  console.log(`Response Size: ${responseSize} bytes`);
  // output esperado:"Response Size: 1071472"
  return responseSize;
}
getResponseSize("https://jsonplaceholder.typicode.com/photos");
```

Veja também

- `Statements/for...of`

for...in

O laço **`for...in`** interage sobre propriedades enumeradas de um objeto, na ordem original de inserção. O laço pode ser executado para cada propriedade distinta do objeto.

Syntax

```
for (variavel in objeto) {...
}
```

`variavel`

Uma propriedade diferente do objeto é atribuida em cada iteração.

`objeto`

Objeto com as propriedades enumeradas.

Descrição

O laço for...in somente iterage sobre propriedades enumeradas. Objetos criados a partir de construtores built-in (arrays e object) herdam propriedades não enumeradas de object.prototype e String.prototype, assim como método `String`'s `indexOf()` ou `Object`'s `toString()`. O laço irá iterar sobre todas as propriedades enumeráveis do próprio objeto e somente aquelas enumeráveis herdadas de construtores de objetos prototype.

Propriedades deletadas, adicionadas ou modificadas

O laço `for...in` iterage sobre as propriedades de um objeto em uma ordem arbitrária (veja o `delete`). Se uma propriedade é deletada durante a execução do loop, ela se torna indisponível para ser acessada depois. É recomendável não adicionar, remover ou alterar propriedades novas ao objeto durante a execução do laço (durante o loop)

Iteração em Arrays e <code>for...in</code>

> **Nota:** `for...in` não deve ser usado para iteração em uma `Array` onde a ordem é importante, visto que ele iterage em uma ordem arbitrária.

Indices de arrays somente se tornam propriedades enumeradas com inteiros (integer). Não há garantia de que utilizando o laço for...in os indices de um array serão retornados em uma ordem particular ou irá retornar todas as propriedades enumeráveis. É recomendável utilizar o laço `for` com índices numéricos ou `Array.prototype.forEach()` ou ainda `for...of` quando iteragir sobre arrays onde a ordem é importante.

Iteração apenas sobre suas próprias propriedades

Se você quer considerar somente as propriedades do próprio objeto e não as herdadas via prototype, use `getOwnPropertyNames()` ou execute `hasOwnProperty()` ou (`propertyIsEnumerable`

Exemplos

A função seguinte toma como argumento um objeto. O laço for...in iterage sobre todos as propriedades enumeráveis do objeto e retorna uma string com o nome das propriedades e seus respectivos valores.

```
//Objeto
var obj = { a: 1, b: 2, c: 3 };

//Para prop (propriedade) in obj (objeto) faça
for (var prop in obj) {
  // ctrl+shift+k (para abrir o console no mozilla firefox)
  console.log("obj." + prop + " = " + obj[prop]);
}

//A saída (output) deverá ser:
// "obj.a = 1"
// "obj.b = 2"
// "obj.c = 3"
```

A função seguinte ilustra o uso de `hasOwnProperty()`: as propriedades herdadas não são exibidas.

```
var triangle = { a: 1, b: 2, c: 3 };

function ColoredTriangle() {
  this.color = "red";
}

ColoredTriangle.prototype = triangle;

var obj = new ColoredTriangle();

for (var prop in obj) {
  if (obj.hasOwnProperty(prop)) {
    console.log("obj." + prop + " = " + obj[prop]);
  }
}

// Output:
// "obj.color = red"
```

Compatibilidade: Initializer expressions

Anterior ao SpiderMonkey 40, era possível usar uma expressão inicial com o laço for...in, conforme exemplo que se segue:

```
var obj = { a: 1, b: 2, c: 3 };
for (var i = 0 in obj) {
  console.log(obj[i]);
}
// 1
// 2
// 3
```

Este não é o comportamento padrão e atualmente é ignorado a partir da versão 40 e posteriores e seu uso irá exibir o error `SyntaxError` ("for-in loop head declarations may not have initializers"). Outras engines como a v8 (Chrome), Chakra (IE/Edge) e JSC (Webkit/Safari) estão estudando o que fazer com esse comportamento.

Veja também

- `for...of` - laço similar que iterage sobre os valores das propriedades.
- `for each in` - (deprecated).
- `for`
- Generator expressions (uses the `for...in` syntax)
- Enumerability and ownership of properties
- `Object.getOwnPropertyNames()`
- `Object.prototype.hasOwnProperty()`
- `Array.prototype.forEach()`

for...of

O loop **`for...of`** percorre objetos iterativos (incluindo `Array`, `Map`, `Set`, o objeto arguments e assim por diante), chamando uma função personalizada com instruções a serem executadas para o valor de cada objeto distinto.

Sintaxe

```
for (variavel of iteravel) {
  declaração
}
```

`variável`

A cada iteração, um valor de uma propriedade diferente é atribuido à *variável*.

`iteravel`

Objeto cujos atributos serão iterados.

Exemplos

Iterando sobre um <code>Array</code>

```
let iterable = [10, 20, 30];

for (let value of iterable) {
  console.log(value);
}
// 10
// 20
// 30
```

Ao invés de `let`, você pode usar `const` se você não for modificar a variável dentro do bloco.

```
let iterable = [10, 20, 30];

for (const value of iterable) {
  console.log(value);
}
// 10
// 20
// 30
```

Iterando sobre uma <code>String</code>

```
let iterable = "boo";

for (let value of iterable) {
  console.log(value);
}
// "b"
// "o"
// "o"
```

Iterando sobre uma <code>TypedArray</code>

```
let iterable = new Uint8Array([0x00, 0xff]);

for (let value of iterable) {
  console.log(value);
}
// 0
// 255
```

Iterando sobre um <code>Map</code>

```
let iterable = new Map([["a", 1], ["b", 2], ["c", 3]]);

for (let entry of iterable) {
  console.log(entry);
}
// [a, 1]
// [b, 2]
// [c, 3]

for (let [key, value] of iterable) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterando sobre um <code>Set</code>

```
let iterable = new Set([1, 1, 2, 2, 3, 3]);

for (let value of iterable) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterando sobre um objeto arguments

```
(function() {
  for (let argument of arguments) {
    console.log(argument);
  }
})(1,2,3);

// 1
// 2
// 3
```

Iterando sobre uma coleção do DOM

Iterar sobre uma coleção do DOM como `NodeList`: o seguinte exemplo adiciona uma classe `read` aos parágrafos que são descendentes diretos de uma tag `article`:

```
// Nota: Isso irá funcionar somente em plataformas que tem
// suporte ao NodeList.prototype[Symbol.iterator]
let articleParagraphs = document.querySelectorAll("article > p");

for (let paragraph of articleParagraphs) {
  paragraph.classList.add("read");
}
```

Iterando sobre generators

Você pode também iterar sobre generators:

```
function* fibonacci() { // uma função geradora (generator)
  let [prev, curr] = [1, 1];
  while (true) {
    [prev, curr] = [curr, prev + curr];
    yield curr;
  }
}

for (let n of fibonacci()) {
  console.log(n);
  // Trunca a sequência em 1000
  if (n >= 1000) {
    break;
  }
}
```

Não reuse generators

Generators não devem ser re-usados, mesmo se o loop `for...of` for terminado precocemente, por exemplo através da palavra-chave `break`. Enquanto em um loop ativo, o generator é fechado e tentar iterar novamente sobre ele não produz (yield) nenhum resultado adicional. O Firefox ainda não implementou este comportamento (o generator pode ser reutilizado, violando o padrão do ES2015 (13.7.5.13, step 5m), mas isso irá mudar uma vez que o Erro do Firefox 1147371 for resolvido.

```
var gen = (function *(){
  yield 1;
  yield 2;
  yield 3;
})();
for (let o of gen) {
  console.log(o);
  break;  // Closes iterator
}

// O generator não deve ser reutilizado, o código a seguir não
// faz sentido!
for (let o of gen) {
  console.log(o); // Nunca será chamado.
}
```

Iterando sobre outros objetos iteráveis

Você pode também iterar sobre um objeto que implementa explicitamente um protocolo iterável protocol:

```
var iterable = {
  [Symbol.iterator]() {
    return {
      i: 0,
      next() {
        if (this.i < 3) {
          return { value: this.i++, done: false };
        }
        return { value: undefined, done: true };
      }
    };
  }
};

for (var value of iterable) {
  console.log(value);
}
// 0
// 1
// 2
```

Diferença entre <code>for...of</code> e <code>for...in</code>

O loop `for...in` irá iterar sobre todas as propriedades enumeráveis de um objeto.

A sintaxe do `for...of` é específica para **coleções**, ao invés de todos os objetos. Ela irá iterar desta maneira sobre os elementos de qualquer coleção que tiver uma propriedade `[Symbol.iterator]`.

O exemplo a seguir mostra a diferença entre um loop `for...of` e um loop `for...in`.

```
Object.prototype.objCustom = function () {};
Array.prototype.arrCustom = function () {};

let iterable = [3, 5, 7];
iterable.foo = "hello";

for (let i in iterable) {
  console.log(i); // escreve 0, 1, 2, "foo", "arrCustom", "objCustom"
}

for (let i of iterable) {
  console.log(i); // escreve 3, 5, 7
}
```

Consulte também

- for each...in - uma declaração similar, mas que itera entre os valores dos atributos de um objeto, ao invés dos seus nomes (obsoleto).
- `Array.prototype.forEach()`

Declaração de função

A **declaração function** define uma função com os especificados parâmetros.

Você pode também definir funções usando o construtor `Function` e uma `function expression`.

Sintaxe

```
function name([param,[, param,[..., param]]]) {
   [statements]
}
```

`name`

O nome da função.

`param`

O nome de um argumento a ser passado para a função. Uma função pode ter atè 255 argumentos.

`statements`

As instruções que compõem o corpo da função.

Descrição

Uma função criada com uma declaração function é um objeto `Function` e tem todas as propriedades, métodos e comportamentos dos objetos `Function`. Veja `Function` para informações detalhadas sobre as funções.

Uma função pode também ser criada usando uma expressão (veja `function expression`).

Por padrão, funções retornam `undefined`. Para retornar qualquer outro valor, a função precisa ter uma instrução `return` que especifica o valor para retorno.

Criando funções condicionalmente

Funções podem ser condicionalmente declaradas, isso é, uma instrução de função pode ser aninhada dentro de uma instrução `if`. A maioria dos navegadores que não sejam Mozilla ir tratar essas declarações condicionais como uma incondicional e criará a função se a condição for true ou não, veja esse artigo para uma visão geral. Portanto, ela não pode ser usada para criação de expressões de função condicional.

Declarações de funções hoisting

Declarações de funções em JavaScript são hoisted à definição da função. Você pode usar uma função antes de tê-la declarado:

```
hoisted(); // logs "foo"

function hoisted() {
  console.log("foo");
}
```

Note que `function expressions` não são hoisted:

```
notHoisted(); // TypeError: notHoisted is not a function

var notHoisted = function () {
  console.log("bar");
};
```

Exemplos

Usando funções

O seguinte código declara uma função que retorna a quantidade total das vendas, quando dados os números de unidades vendidas dos produtos a, b e c.

```
function calc_sales(units_a, units_b, units_c) {
  return units_a * 79 + units_b * 129 + units_c * 699;
}
```

Veja também

- `Functions and function scope`
- `Function`
- `function expression`
- `function* statement`
- `function* expression`
- `Arrow functions`
- `GeneratorFunction`

if...else

A condicional **if** é uma estrutura condicional que executa a afirmação, dentro do bloco, se determinada condição for verdadeira. Se for falsa, executa as afirmações dentro de **else**.

Sintaxe

```
if (condição) afirmação1 [else afirmação2]
```

`condição`

> Uma expressão (premissa) que pode ser avaliada como verdadeira (true) ou falsa (false), veja lógica de programação para entender melhor.

`condição1`

> Condicional que será executada caso a condição em *if* seja verdadeira (true). Pode ser qualquer instrução, incluindo mais condicionais *if* aninhadas à instrução. Para executar multiplas instruções, faça um agrupamento com uma instrução em bloco ({ ... }). Para não executar nenhuma instrução, utilize uma instrução vazia (empty).

`condição2`

> Condicional que será executada caso a condição em *if* seja avaliada como falsa (false) e a condição *else* exista. Pode ser qualquer instrução, incluindo instruões em bloco e mais condicionais *if* aninhadas.

Descrição

Múltiplas condicionais *if* ... *else* podem ser aninhados quando necessário. Observe que não existe elseif (em uma palavra). O correto é a instrução com espaços (else if), conforme abaixo:

```
if (condição1)
   instrução1
else if (condição2)
   instrução2
else if (condição3)
   instrução3
...
else
   instruçãoN
```

Para ver seu funcionamento, abaixo está a instrução como deveria ser caso identada corretamente.

```
if (condição1)
   instrução1
else
   if (condição2)
      instrução2
   else
      if (condição3)
...
```

Para executar multiplas instruções dentro de uma condição, utilize um bloco (`{ ... }`) . Em geral, é sempre uma boa prática utilizar instruções dentro de blocos, especialmente em códigos que envolvam condicionais *if* aninhadas:

```
if (condição) {
   instrução1
} else {
   instrução2
}
```

Não confunda os valores boolean primitivos `true` e `false` com os valores true e false do objeto Boolean. Qualquer valor que não for `undefined`, `null`, `0`, `NaN`, ou uma string vazia (`""`), e qualquer objeto, incluíndo um objeto Boolean cujo valor é false, é avaliado como true quando passado por uma condicional. Por exemplo:

```
var b = new Boolean(false);
if (b) // essa condição é avaliada como true
```

Exemplos

Usando <code>if...else</code>

```
if (cipher_char === from_char) {
  result = result + to_char;
  x++;
} else {
  result = result + clear_char;
}
```

Usando <code>else if</code>

Perceba que não existe sintaxe de `elseif` em JavaScript. Entretanto, você pode escrevê-la com um espaço entre o if e o else.

```
if (x > 5) {
} else if (x > 50) {
} else {
}
```

Atribuições junto de expressões condicionais

É recomendado não utilizar atribuições simples junto de uma expressão condicional, visto que atribuições podem ser confundidas com igualdade ao olhar o código. Por exemplo, não use o código abaixo:

```
if (x = y) {
  /* faça a coisa certa */
}
```

Caso você precise utilizar uma atribuição em uma expressão condicional, uma prática comum é inserir parênteses adicionais em volta da atribuição. Por exemplo:

```
if ((x = y)) {
  /* faça a coisa certa */
}
```

Veja também

- `block`
- `switch`

import

A declaração estática **`import`** é usada para importar vínculos que são exportados por um outro módulo. Os módulos importados estão em `strict mode`, declarado como tal ou não. A declaração `import` não pode ser usada em scripts embutidos, a menos que tal script tenha um `type="module"`.

Há também uma função dinâmica **`import()`**, que não requer scripts de `type="module"`.

A compatibilidade com versões anteriores pode ser garantida usando o atributo `nomodule` na tag de script.

A importação dinâmica é útil em situações em que você deseja carregar um módulo condicionalmente ou sob demanda. A forma estática é preferível para carregar dependências iniciais e pode se beneficiar mais prontamente de ferramentas de análise estática e tree shaking.

Sintaxe

```
import defaultExport from "module-name";
import * as name from "module-name";
import { export } from "module-name";
import { export as alias } from "module-name";
import { export1 , export2 } from "module-name";
import { foo , bar } from "module-name/path/to/specific/un-exported/file";
import { export1 , export2 as alias2 , [...] } from "module-name";
import defaultExport, { export [ , [...] ] } from "module-name";
import defaultExport, * as name from "module-name";
import "module-name"; var promise = import("module-name"); // This is a stage 3 proposal.
```

`defaultExport`

Nome que de referência para a exportação padrão do módulo.

`module-name`

O módulo para importar. Geralmente, esse é um nome de caminho relativo ou absoluto para o arquivo `.js` contendo esse módulo. Certos empacotadores podem permitir ou exigir o uso da extensão; verifique seu ambiente. Apenas aspas simples e aspas duplas são permitidas.

`name`

Nome do objeto do módulo que será usado como um tipo de espaço de nomes ao se referir às importações.

`export, exportN`

Nome das exportações a serem importadas.

`alias, aliasN`

Nomes que se referem às importações nomeadas.

Descrição

O parâmetro name é o nome do "objeto módulo", que será usado como um tipo de namespace para se referir às exportações. Os parâmetros de exportação especificam exportações nomeadas individuais, enquanto a importação * como sintaxe de nome importa todos eles. Abaixo estão alguns exemplos para esclarecer a sintaxe.

Importar o conteúdo de todo um módulo

Isso insere `myModule` no escopo atual, contendo todas as exportações do módulo no arquivo localizado em `/modules/my-module.js`.

```
import * as myModule from '/modules/my-module.js';
```

Aqui, acessar as exportações significa usar o nome do módulo ("myModule" neste caso) como um namespace. Por exemplo, se o módulo importado acima incluir um export `doAllTheAmazingThings()`, você o chamaria assim:

```
myModule.doAllTheAmazingThings();
```

Importar uma única exportação de um módulo

Dado um objeto ou valor chamado `myExport`que foi exportado do módulo `my-module` implicitamente (porque o módulo inteiro é exportado) ou explicitamente (usando a instrução `export` statement), isso insere `myExport` no escopo atual.

```
import {myExport} from '/modules/my-module.js';
```

Importar várias exportações do módulo

Isso inclui tanto `foo` como `bar` no escopo atual.

Importar uma exportação com um alias mais conveniente

Você pode renomear uma exportação ao importá-la. Por exemplo, isso insere `shortName` no escopo atual.

```
import {reallyReallyLongModuleExportName as shortName}
  from '/modules/my-module.js';
```

Renomear várias exportações durante a importação

Importe várias exportações de um módulo com aliases convenientes.

```
import {
  reallyReallyLongModuleExportName as shortName,
  anotherLongModuleName as short
} from '/modules/my-module.js';
```

Importar um módulo apenas para seus efeitos colaterais

Importe um módulo inteiro somente para efeitos colaterais, sem importar nada. Isso executa o código global do módulo, mas na verdade não importa nenhum valor.

```
import '/modules/my-module.js';
```

Importando Padrões

É possível ter um padrão `exportação` (seja um objeto, uma função, uma classe etc.). A declaração de importação pode então ser usada para importar esses padrões.

A versão mais simples importa diretamente o padrão:

```
import myDefault from '/modules/my-module.js';
```

Também é possível usar a sintaxe padrão com as vistas acima (importações de namespace ou importações nomeadas). Em tais casos, a importação padrão terá que ser declarada primeiro. Por exemplo:

```
import myDefault, * as myModule from '/modules/my-module.js';
// myModule used as a namespace
```

Ou

```
import myDefault, {foo, bar} from '/modules/my-module.js';
// specific, named imports
```

Importações Dinâmicas

A palavra-chave `import` pode ser chamada como uma função para importar dinamicamente um módulo. Quando usado dessa maneira, retorna uma promessa.

```
import('/modules/my-module.js')
  .then((module) => {
    // Do something with the module.
  });
```

Este formulário também suporta a palavra-chave await.

```
let module = await import('/modules/my-module.js');
```

Exemplos

Padrão Import

O código abaixo mostra como importar um módulo secundário para auxiliar no processamento de uma requisição AJAX que retorna um JSON.

O Módulo: file.js

```
function getJSON(url, callback) {
  let xhr = new XMLHttpRequest();
  xhr.onload = function () {
    callback(this.responseText)
  };
  xhr.open('GET', url, true);
  xhr.send();
}

export function getUsefulContents(url, callback) {
  getJSON(url, data => callback(JSON.parse(data)));
}
```

O programa principal: main.js

```
import { getUsefulContents } from "/modules/file.js";

getUsefulContents("http://www.example.com", (data) => {
  doSomethingUseful(data);
});
```

Dynamic Import

Este exemplo mostra como carregar a funcionalidade em uma página com base em uma ação do usuário, nesse caso, um clique no botão e, em seguida, chamar uma função dentro desse módulo. Esta não é a única maneira de implementar essa funcionalidade. A função `import()` também é compatível com `await`.

```
const main = document.querySelector("main");
for (const link of document.querySelectorAll("nav > a")) {
  link.addEventListener("click", (e) => {
    e.preventDefault();

    import("/modules/my-module.js")
      .then((module) => {
        module.loadPageInto(main);
      })
      .catch((err) => {
        main.textContent = err.message;
      });
  });
}
```

Veja também

- `export`
- `import.meta`
- Previewing ES6 Modules and more from ES2015, ES2016 and beyond
- ES6 in Depth: Modules, Hacks blog post by Jason Orendorff
- ES modules: A cartoon deep-dive, Hacks blog post by Lin Clark
- Axel Rauschmayer's book: "Exploring JS: Modules"

label (Etiqueta)

Provém uma declaração com um indentificador o qual pode ser referido posteriormente utilizando-se os comandos *break* ou *continue*.

Por exemplo, você pode usar um *label* para identificar um laço e então com um comando de *break* ou *continue* indicar se este deve ser interrompido ou então continuar sua execução.

Sintaxe

```
label :
   declaração
```

Parâmetros

`label`

Qualquer idenfiticador JavaScript válido que não é uma palavra reservada.

`declaração`

Uma declaração qualquer. *break* pode ser utilizado com qualquer declaração entiquetada (com *label*) , assim como o *continue* também pode.

Evite usar labels

Labels não são comunmente utilizados em JavaScript já que estes fazem com que programas fiquei mais difíceis de ler e entender. Sempre que possível evite utilizar labels e, dependendo dos casos, prefira chamar funções ou lançar um erro

Exemplos

<code>Exemplo com continue</code>

```
var i, j;

loop1: for (i = 0; i < 3; i++) {
  //O primeiro 'for' é etiquetado com "loop1"
  loop2: for (j = 0; j < 3; j++) {
    //O segundo é etiquetado com "loop2"
    if (i == 1 && j == 1) {
      continue loop1;
    } else {
      console.log("i = " + i + ", j = " + j);
    }
  }
}

// Sua saída é:
//   "i = 0, j = 0"
//   "i = 0, j = 1"
//   "i = 0, j = 2"
//   "i = 1, j = 0"
//   "i = 2, j = 0"
//   "i = 2, j = 1"
//   "i = 2, j = 2"
// Note como pula-se "i = 1, j = 1" e "i = 1, j = 2"
```

Exemplo 2 <code>(continue)</code>

Dado uma lista de itens e outra de testes, este exemplo conta o número de ítens que passam em todos os testes.

```
var itemsPassed = 0;
var i, j;

top: for (i = 0; i < items.length; i++) {
  for (j = 0; j < tests.length; j++) if (!tests[j].pass(items[i])) continue top;
  itemsPassed++;
}
```

Exemplo 3 <code>(break)</code>

Dado uma lista de ítens e uma lista de testes, neste exemplo determina-se se todos os ítens passam em todos os testes.

```
var allPass = true;
var i, j;
```

```
top: for (i = 0; items.length; i++)
  for (j = 0; j < tests.length; i++)
    if (!tests[j].pass(items[i])) {
      allPass = false;
      break top;
    }
```

Veja também

- break
- continue

let

Declara uma variável local no escopo do bloco atual, opcionalmente iniciando-a com um valor.

Sintaxe

Definição `let`:

```
let var1 [= value1] [, var2 [= value2]] [, ..., varN [= valueN]];
```

Expressão `let`:

```
let (var1 [= value1] [, var2 [= value2]] [, ..., varN [= valueN]]) expression;
```

Instrução `let`:

```
let (var1 [= value1] [, var2 [= value2]] [, ..., varN [= valueN]]) statement;
```

Parâmetros

Parameter	Description
`var1`, `var2`, ..., `varN`	Nome da variável. Pode ser qualquer identificador válido.
`value1`, `value2`, ..., `valueN`	Valor inicial da variável. Pode ser qualquer expressão válida.
`expression`	Qualquer expressão válida.
`statement`	Qualquer instrução válida.

Descrição

`let` permite que você declare variáveis limitando seu escopo no bloco, instrução, ou em uma expressão na qual ela é usada. Isso é inverso da keyword `var`, que define uma variável globalmente ou no escopo inteiro de uma função, independentemente do escopo de bloco.

> **Nota: Nota do tradutor:** o trecho acima: "independentemente do escopo de bloco", na verdade, significa dizer que variáveis declaradas dentro de blocos internos da função, por exemplo, são vinculadas no escopo da função, não no bloco no qual elas são declaradas. Se isso parece confuso - e realmente é -, apenas entenda que, ao contrário do que se poderia supor, em Javascript blocos não possuem escopo como em outras linguagens, somente funções têm! Isso quer dizer que mesmo uma váriavel definida com a keyword `var` dentro de um bloco de instrução `if`, será visível no resto inteiro da função.

Escopo de bloco

Variáveis declaradas com `let` são "içadas" para começo do bloco em que elas são definidas (isso é conhecido também pelo termo, hoisting).

Redeclaração de uma mesma variável num mesmo escopo de bloco causa um `TypeError`.

```
if (x) {
  let foo;
  let foo; // Emite um TypeError.
}
```

No entanto, corpos de funções não possuem essa limitação!

```
function do_something() {
  let foo;
  let foo; // Isso funciona.
}
```

> **Aviso: Atenção:** Os rascunhos do ECMAScript 6 (desde abril de 2012) tornam este comportamento ilegal. Isso significa, provavelmente, que as futuras versões do JavaScript levantarão um `TypeError`. Então, se você faz isso, deveria evitar essa prática!

Você pode encontrar erros em instruções `switch` porque existe apenas um bloco subjacente.

```
switch (x) {
  case 0:
    let foo;
    break;
```

```
  case 1:
    let foo; // TypeError para a redeclaração.
    break;
}
```

Exemplos

Uma *expressão let* limita o escopo de uma variável declarada somente para aquela expressão.

```
var a = 5;
let(a = 6) alert(a); // 6
alert(a); // 5
```

Usada dentro de um bloco, *let* limita os escopo das variáveis para aquele bloco. Observe a diferença entre *var* em que o escopo é definido dentro da função onde é declarada

```
var a = 5;
var b = 10;

if (a === 5) {
  let a = 4; // O escopo é dentro do bloco if
  var b = 1; // O escopo é dentro da função

  console.log(a); // 4
  console.log(b); // 1
}

console.log(a); // 5
console.log(b); // 1
```

É possível usar a keyword *let* para vincular variáveis localmente no escopo de loops em vez de usar uma variável global (definida usando *var*) para isso.

```
for (let i = 0; i < 10; i++) {
  alert(i); // 1, 2, 3, 4 ... 9
}

alert(i); // i não está definida
```

Veja também

- `var`
- `const`
- Novidades do JavaScript 1.7

return

A **declaração** `return` finaliza a execução de uma função e especifica os valores que devem ser retonados para onde a função foi chamada.

Sintaxe

```
return [[expression]];
```

`expression`

A expressão cujo valor será retornado. Se omitido, `undefined` é retornado.

Descrição

Quando uma declaração `return` é usada em um corpo de função, a execução dessa função é parada. Se especificado, um dado valor é retornado à quem chamou a função. Se a expressão for omitida, `undefined` será retornado. Todas as declarações return a seguir param a execução da função:

```
return;
return true;
return false;
return x;
return x + y / 3;
```

Inserção Automática de Ponto-e-Vírgula

A declaração `return` é afetada pela inserção automática de ponto e vírgula (ASI). Nenhum terminador de linha é permitido entre a palavra-chave `return` e a expressão.

```
return
a + b;
```

é transformado pelo ASI em:

```
return;
a + b;
```

O console irá alertar "unreachable code after return statement" (código inacessível após a declaração return).

> **Nota:** A partir do Gecko 40, um alerta é mostrado no console se um código inacessível é encontrado após uma declaração `return`.

Exemplos

return

A função a seguir retorna o quadro do seu argumento, `x`, quando `x` é um número.

```
function square(x) {
  return x * x;
}
```

Interrompendo uma função

Uma função para imediatamente no ponto em que `return` é chamado.

```
function counter() {
  for (var count = 1; ; count++) {
    // loop infinito
    console.log(count + "A"); // até 5
    if (count === 5) {
      return;
    }
    console.log(count + "B"); // até 4
  }
  console.log(count + "C"); // nunca aparece
}

counter();

// Saída:
// 1A
// 1B
// 2A
// 2B
// 3A
// 3B
// 4A
// 4B
// 5A
```

Retornando uma função

Veja também o artigo sobre Closures.

```
function magic(x) {
  return function calc(x) {
    return x * 42;
  };
}

var answer = magic();
answer(1337); // 56154
```

Veja também

- Funções
- Closures

switch

A condicional **switch** avalia uma expressão, combinando o valor da expressão para um cláusula **case**, e executa as instruções associadas ao **case**.

Experimente

```
const expr = "Papayas";
switch (expr) {
  case "Oranges":
    console.log("Oranges are $0.59 a pound.");
    break;
  case "Mangoes":
  case "Papayas":
    console.log("Mangoes and papayas are $2.79 a pound.");
    // Expected output: "Mangoes and papayas are $2.79 a pound."
    break;
  default:
    console.log(`Sorry, we are out of ${expr}.`);
}
```

Sintaxe

```
switch (expressão) {
  case valor1:
    //Instruções executadas quando o resultado da expressão for igual á valor1
    [break;]
  case valor2:
    //Instruções executadas quando o resultado da expressão for igual á valor2
    [break;]
  ...
  case valueN:
    //Instruções executadas quando o resultado da expressão for igual á valorN
    [break;]
  default:
    //Instruções executadas quando o valor da expressão é diferente de todos os cases
    [break;]
}
```

`expressão`

Uma expressão que será comparada á cada cláusula **case**.

`case expressão`

Uma cláusula **case** que será comparada á expressão.

Descrição

Se a condição for correspondida, o programa executa as instruções asssociadas. Se múltiplos casos corresponderem o valor, o primeiro caso que corresponder é selecionado, mesmo se os casos não forem iguais entre si.

O programa primeiro procura por um caso o qual a expressão avalie como tendo o mesmo valor que o input da expressão (usando a comparação de igualdade estrita, `===`) transferindo assim o controle para a cláusula encontrada e em seguida executando as instruções associadas. Caso nenhum caso seja correspondido, então o programa procura pela cláusula opcional `default`, que, se encontrado, tem o controle transferido a ele, executando suas instruções associadas. Se não houver uma cláusula `default`, o programa continua a execução da instrução seguindo para o final do `switch`. Por convenção, a cláusula default é a última, mas não é algo obrigatório.

A instrução opcional `break` associada com cada `case` garante que o programa saia da condicional `switch` assim que a instrução correspondente for executada e executa a instrução que segue logo após o switch. Caso `break` seja omitido, o programa continua a execução para a próxima instrução dentro de `switch`.

Exemplos

Exemplo: Usando <code>switch</code>

No exemplo a seguir, if `expr` é avaliado como "Bananas", o programa corresponde o valor com o `case` "Bananas" e executa a instrução associada. Quando `break` for encontrado, o programa para (break), saindo de `switch` e executa a instrução localizada após o `switch`. Se `break` fosse omitido, a instrução para "Cherries" também seria executada.

```
switch (expr) {
  case "Laranjas":
    console.log("As laranjas custam $0.59 o quilo.");
    break;
  case "Maçãs":
    console.log("Maçãs custam $0.32 o quilo.");
    break;
  case "Bananas":
    console.log("Bananas custam $0.48 o quilo.");
    break;
  case "Cerejas":
    console.log("Cerejas custam $3.00 o quilo.");
    break;
  case "Mangas":
  case "Mamões":
    console.log("Mangas e mamões custam $2.79 o quilo.");
    break;
  default:
    console.log("Desculpe, estamos sem nenhuma " + expr + ".");
}

console.log("Tem algo mais que você gostaria de levar?");
```

Exemplo: O que acontece se eu esquecer um break?

Se você esquecer um break então o script irá rodar a partir do caso onde o critério foi correspondido e irá rodar também o caso seguinte independentemente do critério ter sido correspondido ou não:

```
var foo = 0;
switch (foo) {
  case -1:
    console.log("1 negativo");
    break;
  case 0: // foo é 0 então aqui o critério foi correspondido, então esse bloco vai rodar
    console.log(0);
  // NOTA: o break esquecido deveria estar aqui
  case 1: // nenhuma instrução break em 'case 0:' então essa instrução vai rodar também
```

```
    console.log(1);
    break; // o programa encontra esse break então não vai continuar para o 'case 2:'
  case 2:
    console.log(2);
    break;
  default:
    console.log("default");
}
```

Exemplo: Métodos para múltiplos casos

Referência para esta técnica abaixo:

- Switch statement multiple cases in JavaScript (Stack Overflow)

Multi-Caso - Operação Simples

Esse método toma vantagem do fato de não existir um break após um case e irá continuara executar o próximo case independentemente se o case corresponde ao critério. Veja o título desta seção "O que acontece se eu esquecer um break?"

Esse é um exemplo de uma operação sequencial simples com a instrução switch, onde quatro valores diferentes fazem a mesma coisa..

```
var Animal = "Girafa";
switch (Animal) {
  case "Vaca":
  case "Girafa":
  case "Cachorro":
  case "Porco":
    alert("Esse animal irá para Arca de Noé");
    break;
  case "Dinossauro":
  default:
    alert("Esse animal não vai.");
}
```

Multi-Caso - Operações encadeadas

Esse é um exemplo de múltiplas operações squenciais usando a instrução `switch`, onde, dependendo do número inteiro, você poderá receber outputs diferentes. Isso mostra que você pode alterar a ordem que você insere as instruções de `case`, e isso não precisa ser uma sequência numérica. Em JavaScript, você pode até mesmo misturar definições de strings dentro desses `cases`.

This is an example of a multiple-operation sequential switch statement, where, depending on the provided integer, you can receive different output. This shows you that it wil traverse in the order that you put the case statements, and it does not have to be numerically sequential. In JavaScript, you can even mix in definitions of strings into these case statements as well.

```
var foo = 1;
var output = "Output: ";
switch (foo) {
  case 10:
    output += "Então ";
  case 1:
    output += "Qual ";
    output += "É ";
  case 2:
    output += "O Seu ";
  case 3:
    output += "Nome";
  case 4:
    output += "?";
    alert(output);
    break;
  case 5:
    output += "!";
    alert(output);
    break;
  default:
    alert("Favor escolher um número de 0 à 6!");
}
```

Output para esse exemplo:

Value	Alert Text
foo é NaN ou não é 1, 2, 3, 4, 5 ou 10	Favor escolher um número de 0 à 6!
10	Output: Então Qual É O Seu Nome?
1	Output: Qual É O Seu Nome?
2	Output: Seu Nome?
3	Output: Nome?
4	Output: ?
5	Output: !

See also

- `if...else`

throw

A **declaração `throw`** lança uma exceção definida pelo usuário. A execução da função atual vai parar (as instruções após o `throw` não serão executadas), e o controle será passado para o primeiro bloco `catch` na pilha de chamadas. Se nenhum bloco `catch` existe entre as funções "chamadoras", o programa vai terminar.

Sintaxe

```
throw expressão;
```

`expressão`

A expressão a ser lançada.

Descrição

Use a instrução `throw` para lançar uma exceção. Quando você lança uma exceção, `expressão` especifica o valor da exceção. Cada uma das intruções a seguir lança uma exceção:

```
throw "Erro2"; // gera uma exceção com um valor string
throw 42; // gera uma exceção com o valor 42
throw true; // gera uma exceção com o valor true
```

Note também que a instrução `throw` é afetada pela inserção automática de ponto-e-vírgula (ASI) como nenhum terminador de linha entre a palavra `throw` e a expressão é permitido.

Exemplos

Lançando um objeto

Você pode especificar um objeto quando você lança uma exceção. Você pode então referenciar as propriedades do objeto no bloco `catch`. O exemplo a seguir cria um objeto do tipo `UserException` e o usa na intrução `throw`.

```
function UserException(message) {
  this.message = message;
  this.name = "UserException";
}
function getMonthName(mo) {
  mo = mo - 1; // Ajusta o número do mês para index de array (1=Jan, 12=Dec)
  var months = [
    "Jan",
    "Feb",
    "Mar",
    "Apr",
    "May",
    "Jun",
    "Jul",
    "Aug",
    "Sep",
    "Oct",
    "Nov",
    "Dec",
  ];
  if (months[mo] !== undefined) {
    return months[mo];
  } else {
    throw new UserException("InvalidMonthNo");
  }
}

try {
  // statements to try
  var myMonth = 15; // 15 is out of bound to raise the exception
  monthName = getMonthName(myMonth);
} catch (e) {
  monthName = "unknown";
  logMyErrors(e.message, e.name); // pass exception object to err handler
}
```

Outro exemplo lançando um objeto

O exemplo a seguir testa uma string de entrada para um cep dos Estados Unidos. Se o CEP utiliza um formato inválido, a intrução throw lança uma exceção através da criação de um objeto do tipo `ZipCodeFormatException`.

```
/*
 * Cria um objeto ZipCode.
 *
 * Formatos aceitos para o CEP são:
 *    12345
 *    12345-6789
```

```
 *    123456789
 *    12345 6789
 *
 * Se o argumento passado para o construtor do ZipCode não atende
 * a um desses padrões uma exceção é lançada.
 */

function ZipCode(zip) {
  zip = new String(zip);
  pattern = /[0-9]{5}([- ]?[0-9]{4})?/;
  if (pattern.test(zip)) {
    // o valor do CEP será a primeira combinação na string
    this.value = zip.match(pattern)[0];
    this.valueOf = function () {
      return this.value;
    };
    this.toString = function () {
      return String(this.value);
    };
  } else {
    throw new ZipCodeFormatException(zip);
  }
}

function ZipCodeFormatException(value) {
  this.value = value;
  this.message = "does not conform to the expected format for a zip code";
  this.toString = function () {
    return this.value + this.message;
  };
}

/*
 * Isso poderia estar em um script que valida dados de endereços
 * para os endereços dos Estados Unidos.
 */

const ZIPCODE_INVALID = -1;
const ZIPCODE_UNKNOWN_ERROR = -2;

function verifyZipCode(z) {
  try {
    z = new ZipCode(z);
  } catch (e) {
    if (e instanceof ZipCodeFormatException) {
      return ZIPCODE_INVALID;
    } else {
      return ZIPCODE_UNKNOWN_ERROR;
    }
  }
  return z;
}

a = verifyZipCode(95060); // retorna 95060
b = verifyZipCode(9560); // retorna -1
c = verifyZipCode("a"); // retorna -1
d = verifyZipCode("95060"); // retorna 95060
e = verifyZipCode("95060 1234"); // retorna 95060 1234
```

Relançando uma exceção

Você pode usar `throw` para relançar uma exceção após você pegá-la. O exemplo a seguir pega uma exceção com um valor numérico e a relança se o valor for maior que 50. A exceção relançada propaga para a função encapsuladora ou para o nível superior para que o usuário a veja.

```
try {
  throw n; // lança uma exceção com um valor numérico
} catch (e) {
  if (e <= 50) {
    // instruções para tratar exceções 1-50
  } else {
    // não pode tratar esta exceção então relança
    throw e;
  }
}
```

Veja também

- `try...catch`

try...catch

As declarações **try...catch** marcam um bloco de declarações para testar (**try**), e especifica uma resposta, caso uma exceção seja lançada.

Sintaxe

```
try {
   try_statements
}
[catch (exception_var_1 if condition_1) {
   catch_statements_1
}]
...
[catch (exception_var_2) {
   catch_statements_2
}]
[finally {
   finally_statements
}]
```

`try_statements`

Declarações (statements) a serem executadas.

`catch_statements_1`, `catch_statements_2`

Declarações que são executadas se uma exceção é lançada no bloco `try`.

`exception_var_1`, `exception_var_2`

Um indentificador que leva um objeto exceção para uma cláusula `catch` associada.

`condition_1`

Uma expressão condicional.

`finally_statements`

Declarações que são executadas depois que a declaração `try` é completada. Essas declarações são executadas independetemente se uma exceção foi ou não lançada ou capturada.

Descrição

A declaração `try` consiste em um bloco `try`, que contém uma ou mais declarações, e ao menos uma cláusula `catch` ou uma cláusula `finally`, ou ambas. Ou seja, há 3 formas de declarações `try` :

1. `try...catch`
2. `try...finally`
3. `try...catch...finally`

Uma cláusula `catch` contém declarações que especificam o que fazer caso uma exceção seja lançada no bloco `try`. Ou seja, se você quer que o bloco `try` tenha êxito, e caso não tenha, você quer que o controle passe para o bloco `catch`. Caso qualquer declaração dentro do bloco `try` (ou em uma função chamada no interior do bloco `try`) lançar uma exceção o controle imediatamente muda para a cláusula `catch`. Se nenhuma exceção for lançada no bloco `try` a cláusula `catch` é ignorada.

A cláusula `finally` é executada após a excecução do bloco `try` e da(s) cláusula(s) `catch` porém antes das declarações seguintes ao `try`. Ela sempre é executada, independente se uma exceção for lançada ou capturada.

Você pode aninhar uma ou mais declarações `try`. Caso uma declaração `try` interior não tenha uma cláusula `catch`, a cláusula catch pertencente a declaração `try` que a envolve é introduzida.

Você pode usar a declaração try para manipular exceções em JavaScript. Veja o Guia Javascript para informações sobre exceções em JavaScript.

Cláusula catch incondicional

Quando uma única, incondicional cláusula `catch` é utilizada, o bloco `catch` é inserido quando qualquer exceção for lançada. Por exemplo, quando a exceção ocorre no código a seguir, o controle é transferido para a cláusula `catch`.

```
try {
  throw "myException"; // gera uma exceção
} catch (e) {
  // declarações para manipular quaisquer exceções
  logMyErrors(e); // passa o objeto de exceção para o manipulador de erro
}
```

Cláusulas <code>catch</code> condicionais

> **Não padrão:** Este recurso não é padronizado. Não recomendamos o uso de recursos não padronizados em produção, pois eles têm suporte limitado nos navegadores e podem mudar ou ser removidos. No entanto, podem ser uma alternativa viável em casos específicos onde não exista uma opção padrão.

Você pode utilizar uma ou mais cláusulas `catch` condicionals para manipular exceções específicas. Nesse caso a cláusula `catch` apropriada será inserida quando a exceção espeficica for lançada. No exemplo a seguir, o código no bloco `try` pode potencialmente jogar três exceções: `TypeError`, `RangeError`, e `EvalError`. Quando a exceção ocorre, o controle transfere para a cláusula `catch` apropriada. Caso a exceção não seja uma das especificadas e uma cláusula `catch` incondicional for encontrada, o controle é transferido para essa cláusula `catch`.

Se utilizar uma cláusula `catch` incondicional com uma ou mais cláusulas `catch` condicionais, o catch incondicional deve ser especificado por último. Caso contrário, o catch incondicional interceptará todos os tipos de exceção antes que eles alcancem os condicionais.

Nota: Essa funcionalidade não é parte da especificação ECMAScript.

```
try {
    myroutine(); // pode lançar três tipos de exceções
} catch (e if e instanceof TypeError) {
    // declarações para manipular exceções TypeError
} catch (e if e instanceof RangeError) {
    // declarações para manipular exceções RangeError
} catch (e if e instanceof EvalError) {
    // declarações para manipular exceções EvalError
} catch (e) {
    // declarações para manipular quaisquer exceções não especificadas
    logMyErrors(e); // passa o objeto de exceção para o manipulador de erro
}
```

E aqui temos como implementar as mesmas "cláusulas `catch` condicionais" utilizando apenas JavaScript puro conforme a especificação ECMAScript (obviamente é mais verboso, porém, funciona em qualquer lugar):

```
try {
    myroutine(); // pode lançar três tipos de exceções
} catch (e) {
    if (e instanceof TypeError) {
        // declarações para manipular exceções TypeError
    } else if (e instanceof RangeError) {
        // declarações para manipular exceções RangeError
    } else if (e instanceof EvalError) {
        // declarações para manipular exceções EvalError
    } else {
       // declarações para manipular quaisquer exceções não especificadas
       logMyErrors(e); // passa o objeto de exceção para o manipulador de erro
    }
}
```

O identificador de exceção

Quando uma exceção é lançada no bloco `try`, `exception_var` (ex. o `e` dentro de `catch (e)`) armazena o valor especificado pela declaração `throw`. Você pode usar esse identificador para conseguir informação sobre a exceção que foi lançanda.

Esse identificador é local para a cláusula `catch`. Ou seja, é criado quando a cláusula `catch` é introduzida e após terminar sua excecução o identificador não se torna mais disponível.

A cláusula <code>finally</code>

A cláusula `finally` é executada após a excecução do bloco `try` e da(s) cláusula(s) `catch` porém antes das declarações seguintes a declaração `try`. Ela sempre é executada, independente se uma exceção for lançada ou capturada.

A cláusula `finally` contém declarações para executarem após a execução do bloco `try` e da(s) cláusula(s) `catch` porém antes das declarações seguintes a declaração `try`. A cláusula `finally` é excutada independente se uma exceção for lançada ou não. Caso uma exceção seja lançada, as declarações no interior da cláusula `finally` são executadas mesmo que nenhum `catch` manipule a exceção.

Você pode utilizar a cláusula `finally` para fazer seu script falhar graciosamente quando uma exceção ocorrer; por exemplo, você pode precisar liberar um recurso que seu script possui vinculado. O exemplo seguinte abre um aquivo e então executa declarações que utilizam o arquivo (server-side JavaScript permite que você acesse arquivos). Se uma exceção for lançada enquanto o arquivo estiver aberto, a cláusula `finally` fecha o arquivo antes que o script falhe.

```
openMyFile();
try {
  // vincula o recurso
  writeMyFile(theData);
} finally {
  closeMyFile(); // sempre fecha o recurso
}
```

Exemplos

Blocos try aninhados

Primeiro vamos ver o que acontece com isso:

```
try {
  try {
    throw new Error("oops");
  }
  finally {
    console.log("finally");
  }
}
catch (ex) {
  console.error("outer", ex.message);
}

// Resultado
```

```
// "finally"
// "outer" "oops"
```

Agora, caso nós já capturamos a exceção no bloco `try` interno adicionando um bloco `catch`

```
try {
  try {
    throw new Error("oops");
  }
  catch (ex) {
    console.error("inner", ex.message);
  }
  finally {
    console.log("finally");
  }
}
catch (ex) {
  console.error("outer", ex.message);
}

// Resultado:
// "inner" "oops"
// "finally"
```

E agora, vamos relançar o erro.

```
try {
  try {
    throw new Error("oops");
  }
  catch (ex) {
    console.error("inner", ex.message);
    throw ex;
  }
  finally {
    console.log("finally");
  }
}
catch (ex) {
  console.error("outer", ex.message);
}

// Resultado:
// "inner" "oops"
// "finally"
// "outer" "oops"
```

Qualquer exceção lançada será capturada apenas uma vez pelo bloco `catch` envolvente mais próximo, a não ser que seja relançada. Obviamente qualquer nova exceção elevada no bloco "interno" (pois código em um bloco `catch` pode fazer algo que lance), será capturado pelo bloco "externo".

Retornando de um bloco <code>finally</code>

Caso o bloco `finally` retorne um valor, esse valor torna-se o valor de retorno do produto de `try-catch-finally` inteiro, independente de qualquer declaração return nos blocos `try` e `catch`. Isso inclui exceções lançadas dentro do bloco `catch`:

```
try {
  try {
    throw new Error("oops");
  }
  catch (ex) {
    console.error("inner", ex.message);
    throw ex;
  }
  finally {
    console.log("finally");
    return;
  }
}
catch (ex) {
  console.error("outer", ex.message);
}

// Resultado:
// "inner" "oops"
// "finally"
```

O "oops" externo não é lançado devido ao retorno do bloco `finally`. O mesmo se aplicaria a qualquer valor retornado do bloco `catch`.

Vejam também

- throw
- Error

var

Sumário

O **`variable` statement** declara uma variável, opcionalmente é possível atribuir à ela um valor em sua inicialização.

Sintaxe

```
var varname1 [= value1 [, varname2 [, varname3 ... [, varnameN]]]];
```

`varnameN`

Nome da variável. Pode ser utilizado qualquer identificador legal.

`valueN`

Valor inicial da variável. Pode ser qualquer expressão legal.

Descrição

Declarações de variáveis, onde quer que elas ocorram, são processadas antes que qualquer outro código seja executado. O escopo de uma variável declarada com `var` é seu contexto atual em execução, o qual é a função a qual pertence ou, para variáveis declaradas fora de qualquer função, o escopo é o global.

Atribuir um valor a uma variável não declarada anteriormente implica em criar uma variável global (ela se torna uma propriedade do objeto global) quando a atribuição é executada. As diferenças entre uma variável declarada e uma não delcarada são:

1. Variáveis declaradas estão relacionadas com o contexto de execução quando elas são criadas (por exemplo, uma função, objeto). Por outro lado, as variaveis não declaradas sempre são globais.

```
function x() {
  y = 1; // Lança a exceção ReferenceError em modo restrito (strict mode)
  var z = 2;
}

x();

console.log(y); // logs "1"
console.log(z); // Lança a exceção ReferenceError: z não foi definida fora da função x()
```

2. Variáveis declaradas são criadas antes de qualquer código ser executado. As variáveis não declaradas não existem até quando o código atribuir um valor a ela.

```
console.log(a); // Lança a exceção ReferenceError.
console.log("still going..."); // Nunca será executado.
```

```
var a;
console.log(a); // mostra "undefined" ou "" dependendo do naveador.
console.log("still going..."); // mostra "still going...".
```

3. Variáveis declaradas são propriedades não configuráveis no contexto de execução (função ou global). Variaveis não declaradas são configuráveis (por exemplo, podem ser excluídas).

```
var a = 1;
b = 2;

delete this.a; // Lança a exceção TypeError em modo restrito(strict mode). Caso contrário, Falha silenciosamente.
delete this.b;

console.log(a, b); // Throws a ReferenceError.
// A propriedade 'b' foi deletada e não existe mais.
```

Por conta dessas três diferenças, falha para declarar variáveis, muito provavelmente, levar a resultados inesperados. **Então, é recomendado sempre declarar as variáveis, independentemente se as variáveis estão em escopo de função ou escopo global**. E o modo restrito (strict mode) do ECMAScript 5 sempre lançará uma exceçao quando o código atribuir um valor a uma variável não declarada.

var hoisting

Como as declarações de variáveis (e declarações em geral) são processadas antes de qualquer código seja executado, declarar uma variável em qualquer lugar no código é equivalente a declarar no inicio. Isso também significa que uma variável pode aparecer para ser usada antes dela ser declarada. Esse comportamento é chamado de "hoisting", a variável é movida para o inicio da função ou do código global.

```
bla = 2;
var bla;
// ...

// é implicitamente entendido como:

var bla;
bla = 2;
```

Por essa razão, recomenda-se sempre declarar variáveis na parte superior do seu escopo de aplicação (o topo do código global e a parte superior do código da função). Por isso, é claro que as variáveis são função de escopo (local) e que são resolvidos na cadeia de escopo.

Exemplos

Declarando e inicializando duas variáveis

```
var a = 0,
  b = 0;
```

Atribuindo duas variáveis com uma única expressão

```
var a = "A";
var b = a;

// Equivalente a:

var a,
  b = (a = "A");
```

É sempre importante lembrar da ordem da declaração das variáveis:

```
var x = y,
  y = "A";
console.log(x + y); // undefinedA
```

Então, x e y são declarados antes de qualquer código seja executado, a atribuição ocorre posteriormente. Quando "x = y" for executado, y existe e nenhum exceção ReferenceError é lançada, e o valor de y será considerado como '`undefined`'. Por este motivo, este valor é atribuido a x. Depois disso, o valor 'A' é atribuido a variável y. Consequentemente, depois da primeira linha, x === undefined && y === 'A', então o resultado.

Iniciando diversas variáveis

```
var x = 0;

function f() {
  var x = (y = 1); // x é declarado localmente,y não é!
}
f();

console.log(x, y); // 0, 1
// x é uma variável global como experado
// y vazou para fora da função!
```

Variável global implícita e fora do escopo da função

Variáveis que aparecem como variáveis globais implicitas podem ser referenciadas como variáveis fora do escopo da função:

```
var x = 0; // x é declarada como global e é igual a 0

console.log(typeof z); // undefined, desde que z não tenha sido criada anteriormente

function a() {
  // quando a for chamada,
  var y = 2; // y é declarada como local desta função, e o valor 2 é atribuido

  console.log(x, y); // 0 2

  function b() {
    // quando b for chamado,
    x = 3; // atribui o valor 3 a variável global existente, ele não cria uma nova variável global
    y = 4; // atribui o valor 4 a uma variável fora, ele não cria uma nova variável
    z = 5; // cria uma nova variável global e atribui o valor 5.
  } // (Lança a exceção ReferenceError em modo restrito.)

  b(); // chamando b, o código irá criar z como variável global
  console.log(x, y, z); // 3 4 5
}

a(); // chamando a, também irá chamar b
console.log(x, z); // 3 5
console.log(typeof y); // indefinido já que y é uma variável local da função a
```

Veja também

- `let`
- `const`
- How One Missing `var` Ruined our Launch

while

A **declaração while** cria um laço que executa uma rotina especifica enquanto a condição de teste for avaliada como verdadeira. A condição é avaliada antes da execução da rotina.

Syntax

```
while (condição) {
  rotina
```

```
}
```

`condição`

> Uma expressão avaliada antes de cada passagem através do laço. Se essa condição for avaliada como verdadeira, a rotina é executada. Quando a condição for avaliada como falsa, a execução continua na declaração depois do laço `while`.

rotina

> Uma declaração que é executada enquanto a condição é avaliada como verdadeira. Para executar multiplas declarações dentro de um laço, use uma declaração em bloco (`{ ... }`) para agrupar essas declarações.

Exemplos

O seguinte laço `while` itera enquanto n é menor que três.

```
var n = 0;
var x = 0;

while (n < 3) {
  n++;
  x += n;
}
```

Cada iteração, o laço incrementa n e soma à x. Portanto, x e n assumem os seguintes valores:

- Depois da primeira passagem: n = 1 e x = 1
- Depois da segunda passagem: n = 2 e x = 3
- Depois da terceira passagem: n = 3 e x = 6

Depois de completar a terceira passagem, a condição n < 3 não é mais verdadeira, então o laço termina.

Veja Também

- `do...while`
- `for`

with

> **Aviso:** O uso da declaração `with` não é recomendado, isso porque ele pode ser a fonte de bugs confusos e problemas de compatibilidade. Veja o parágrafo "Contra de ambiguidade" na seção "Descrição" para mais detalhes.

A **declaração with** extende a cadeia de escopo para uma declaração.

Sintaxe

```
with (expressão)
  declaração
```

`expressão`

> Adiciona a dada expressão à cadeia de escopo quando estiver avaliando a declaração. O parênteses em volta da expressão é obrigatório.

`declaração`

> Qualquer declaração. Para a execução de multiplas declarações, utilize a declaração em bloco ({ ... }) para agrupar estas declarações.

Descrição

JavaScript procura por um nome não qualificado procurando uma cadeia de escopo associada à execução do contexto do script ou função contendo um nome não qualificado. A declaração 'with' adiciona o dado objeto à frente dessa cadeia de escopo durante a validação desse corpo de declarações. Se um nome não qualificado usado no corpo for igual ao de uma propriedade na cadeia de escopo, então o nome ficará ligado à propriedade e ao objeto contendo a propriedade. Senão, um `ReferenceError` será invocado.

> **Nota:** Usar `with` não é recomendado, e está proibido no strict mode do ECMAScript 5. A alternativa recomendada é atribuir o objeto cujas propriedades você quer acessar a uma variável temporária.

Pros & contras de perfomance

Pro: A declaração `with` pode ajudar o tamanho do arquivo por reduzir a necessidade de repetir a referência a um objeto longo sem penalidade na perfomance. A cadeia de escopo mudada por um 'with' não é computacionalmente cara. O uso de 'with' irá aliviar o interpretador de tratar repetidamente as referências. Note que, no entando, isso em muitos casos pode ser substituído usando uma variável temporária para armazenar a referência do objeto desejado.

Contra: A declaração `with` força que o objeto especificado a ser procurado primeiro por pesquisas de nome. Assim sendo, todos os indentificadores que não são membros do objeto espeficícado vão ser encontrados mais lentamente em um bloco 'with'. Onde a perfomance é importande, 'with' deve ser usado apenas para englobar blocos de código que acessam membros de um objeto especifíco.

Contra de ambiguidade

Contra: A declaração `with` faz ser difícil para um leitor humano ou compilador JavaScript decidir se um nome não qualificado vai ser encontrado em uma cadeia de escopo, e também, em qual objeto. Dado o exemplo seguinte:

```
function f(x, o) {
  with (o) {
    console.log(x);
  }
}
```

Apenas quando `f` é chamado `x` é encontrado ou não, e se for encontrado, ou em `o` ou (se nenhuma propriedade existir) no objeto de ativação de `f`, onde o nome de `x` é o primeiro argumento formal. Se você esquecer de definir `x` no objeto que você passou como segundo argumento, ou se há algum bug similar ou confusão, você não vai receber um erro — apenas resultados inesperados.

Contra: Código utilizando `with` talvez não seja compatível posteriormente, especialmente quando usado com algo que não seja um objeto simples. Considere esse exemplo:

```
function f(foo, values) {
  with (foo) {
    console.log(values);
  }
}
```

Se você chamar `f([1,2,3], obj)` em um ambiente ECMAScript 5, então a referência de `values` dentro da declaração `with` irá ser `obj`. No entando, ECMAScript 6 introduz uma propriedade `values` no `Array.prototype` (então isso estará disponível em todas as arrays). Então, em um ambiente JavaScript que suporta ECMAScript 6, a referência de `values` dentro da declaração `with` irá ser `[1,2,3].values`.

Exemplos

Usando <code>with</code>

A seguinte declaração `with` irá especificar que o objeto `Math` é o objeto padrão. As seguintes declarações seguindo a declaração `with` irão referir a propriedade `PI` e aos métodos `cos` e `sin`, sem especificar um objeto. JavaScript assume o objeto `Math` para essas referências.

```
var a, x, y;
var r = 10;

with (Math) {
  a = PI * r * r;
  x = r * cos(PI);
  y = r * sin(PI / 2);
}
```

Veja também

- `block`
- Strict mode
- `Symbol.unscopables`
- `Array.prototype[@@unscopables]`

Strict mode

O **strict mode** do ECMAScript 5 é uma forma de *optar* por uma variante restrita do JavaScript. O strict mode não é apenas um subconjunto: ele *intencionalmente* tem semânticas diferentes do código normal. Navegadores que não suportam strict mode executarão código strict mode com um comportamento diferente dos navegadores que suportam, então não confie no strict mode sem testar o suporte a funcionalidades dos aspectos relevantes do strict mode. Código strict mode e código não-strict mode podem coexistir, então scripts podem optar pelo strict mode incrementalmente.

O strict mode faz várias mudanças nas semânticas normais do JavaScript. Primeiro, o strict mode elimina alguns erros silenciosos do JavaScript fazendo-os lançar exceções. Segundo, o strict mode evita equívocos que dificultam que motores JavaScript realizem otimizações: código strict mode pode às vezes ser feito para executar mais rápido que código idêntico não-strict mode. Terceiro, strict mode proíbe algumas sintaxes que provavelmente serão definidas em versões futuras do ECMAScript.

Veja transitioning to strict mode se você quer mudar seu código para funcionar na variante restrita do JavaScript.

Invocando strict mode

Strict mode se aplica a *scripts* inteiros ou *funções individuais*. Ele não se aplica a declarações de bloco fechadas em chaves `{}`; tentar aplicá-lo a tais contextos não faz nada. Código `eval`, código `Function`, atributos *event handlers*, strings passadas para `setTimeout` e coisas do tipo são scripts inteiros, e invocar strict mode neles funciona como esperado.

Strict mode para scripts

Para invocar strict mode para um script inteiro, coloque *exatamente* a declaração `"use strict";` (ou `'use strict';`) antes de qualquer outra declaração.

```
// Sintaxe strict mode para todo o script
"use strict";
var v = "Oi! Eu sou um script strict mode!";
```

Essa sintaxe tem uma armadilha que já pegou um site grande: não é possível concatenar cegamente scripts não-conflitantes. Considere concatenar um script strict mode com um script não-strict mode: a concatenação inteira parece strict! O inverso também é verdade: não-strict mais strict parece não-strict. Concatenar scripts

strict mode com outros é bom, e concatenar scripts não-strict mode é bom. Simplesmente concatenar scripts strict e não-strict é problemático. Portanto é recomendado que você ative strict mode função-por-função (pelo menos durante o período de transição).

Você também pode usar a abordagem de embrulhar o conteúdo inteiro de um script em uma função e fazer a função externa usar strict mode. Isto elimina o problema da concatenação, mas significa que você tem que exportar explicitamente quaisquer variáveis globais fora do escopo da função.

Strict mode para funções

Da mesma forma, para invocar strict mode para uma função, coloque *exatamente* a declaração `"use strict";` (ou `'use strict';`) no corpo da função antes de qualquer outra declaração.

```
function strict() {
  // Sintaxe strict mode a nível de função
  "use strict";
  function nested() {
    return "E eu também!";
  }
  return "Oi! Eu sou uma função strict mode!  " + nested();
}
function notStrict() {
  return "Eu não sou strict.";
}
```

Mudanças no strict mode

Strict mode muda tanto a sintaxe quando o comportamento em tempo de execução. Mudanças geralmente se encaixam em uma destas categorias: mudanças que convertem equívocos em erros (como erros de sintaxe ou em tempo de execução), mudanças que simplificam como uma variável particular para um dado uso de um nome é computada, mudanças que simplificam `eval` e `arguments`, mudanças que facilitam JavaScript "seguro", e mudanças que antecipam a evolução futura do ECMAScript.

Convertendo equívocos em erros

Strict mode transforma em erros alguns equívocos anteriormente aceitos. O JavaScript foi projetado para ser fácil para novos desenvolvedores, e algumas vezes ele dá semânticas de não-erros a operações que deveriam ser erros. Algumas vezes isso resolve o problema pontual, mas outras vezes cria problemas piores no futuro. Strict mode trata esses equívocos como erros para que sejam descobertos e consertados prontamente.

Primeiro, o strict mode impossibilita criar variáveis globais acidentalmente. Em JavaScript normal, cometer um erro de digitação ao digitar uma variável em uma atribuição cria uma nova propriedade no objeto global e continua a "funcionar" (embora falhas futuras sejam possíveis: provavelmente, em JavaScript moderno). Atribuições que acidentalmente criariam variáveis globais lançam exceções em vez disso no strict mode:

```
"use strict";
// Assumindo que uma variável global variavelErrada existe
variavellErrada = 17; // esta linha lança ReferenceError por conta do
// erro de digitação da variável
```

Segundo, strict mode faz com que atribuições que outrora falhariam silenciosamente lançar uma exceção. Por exemplo, `NaN` é uma variável global não-atribuível. Em código normal, atribuir a `NaN` não faz nada; o desenvolvedor não recebe qualquer resposta de falha. No strict mode, atribuir a `NaN` lança uma exceção. Qualquer atribuição que falha silenciosamente em código normal (atribuir a uma propriedade não-atribuível, atribuir a uma propriedade getter-only, atribuir a uma propriedade de um objeto não-extensível) lançarão exceções em strict mode:

```
"use strict";

// Atribuir a uma propriedade não-atribuível
var obj1 = {};
Object.defineProperty(obj1, "x", { value: 42, writable: false });
obj1.x = 9; // lança TypeError

// Atribuir a uma propriedade getter-only
var obj2 = {
  get x() {
    return 17;
  },
};
obj2.x = 5; // lança TypeError

// Atribuir a uma nova propriedade de um objeto não-extensível
var fixed = {};
Object.preventExtensions(fixed);
fixed.newProp = "ohai"; // lança TypeError
```

Terceiro, strict mode faz com que tentativas de deletar propriedades não-deletáveis lancem exceções (onde anteriormente a tentativa simplesmente não faria efeito):

```
"use strict";
delete Object.prototype; // lança TypeError
```

Quarto, strict mode anterior ao Gecko 34 requer que todas as propriedades nomeadas em um objeto literal sejam únicas. Código normal pode duplicar nomes de propriedades, sendo que a última ocorrência determina o valor da propriedade. Mas como apenas o último faz algo, a duplicação é meramente um vetor de bugs, se o código for modificado para mudar o valor da propriedade por outro meio que não modificando a última instância. Nomes de propriedades duplicados são erro de sintaxe em strict mode:

> **Nota:** Não é mais o caso no ECMAScript 6 (Erro do Firefox 1041128).

```
"use strict";
var o = { p: 1, p: 2 }; // !!! erro de sintaxe
```

Quinto, strict mode exige que nomes de parâmetros de função sejam únicos. Em código normal, o último argumento duplicado esconde argumentos anteriores com nome semelhante. Estes argumentos anteriores continuam disponíveis através de `arguments[i]`, portanto não são completamente inacessíveis. Entretanto, esta ocultação faz pouco sentido e provavelmente é indesejável (pode esconder um erro de digitação, por exemplo), então em strict mode nomes de argumentos duplicados são erro de sintaxe:

```
function sum(a, a, c) {
  // !!! erro de sintaxe
  "use strict";
  return a + b + c; // errado se este código executasse
}
```

Sexto, strict mode em ECMAScript 5 proíbe sintaxe octal. Sintaxe ocatal não é parte do ECMAScript 5, mas é suportado em todos os navegadores prefixando o número octal com um zero: `0644 === 420` e `"\045" === "%"`. No ECMAScript 6, número octal é suportado prefixando um número com `"0o"`:

```
var a = 0o10; // ES6: Octal
```

Novos desenvolvedores algumas vezes acham que um prefixo zero não tem significado semântico, então eles o utilizam como mecanismo de alinhamento — mas isso muda o significado do número! Sintaxe octal é raramente útil e pode ser usada equivocadamente, então o strict mode torna octal erro de sintaxe:

```
"use strict";
var sum =
  015 + // !!! erro de sintaxe
  197 +
  142;
```

Sétimo, strict mode em ECMAScript 6 proíbe definir propriedades em valores primitivos. Sem strict mode, definir uma propriedade é simplesmente ignorado; com strict mode, no entanto, um `TypeError` é lançado.

```
(function () {
  "use strict";

  false.true = ""; // TypeError
  (14).navegandoPara = "casa"; // TypeError
  "com".voce = "distante"; // TypeError
})();
```

Simplificando usos de variáveis

Strict mode simplifica como nomes de variáveis se mapeiam a definições de variáveis particulares no código. Muitas otimizações de compiladores dependem da habilidade de dizer que variável *X* está guardada *naquele* local; isto é crítico para otimizar completamente o código JavaScript. JavaScript às vezes torna este mapeamento básico de nome para definição de variável no código impossível de executar até o tempo de execução. Strict mode remove a maioria dos casos onde isso acontece, então o compilador pode otimizar melhor código strict mode.

Primeiro, strict mode proíbe `with`. O problema com `with` é que qualquer nome dentro do block pode mapear tanto para uma propriedade do objeto passado quanto para uma variável no escopo envolvente (ou até global), em tempo de execução: é impossível saber qual de antemão. Strict mode torna `with` um erro de sintaxe, então não há chance de um nome em um `with` referir-re a uma localização deconhecida em tempo de execução:

```
"use strict";
var x = 17;
with (obj) {
  // !!! erro de sintaxe
  // Se isto não fosse strict mode, isso seria var x ou obj.x?
  // No geral, é impossível saber sem executar o código, então
  // o nome não pode ser otimizado.
  x;
}
```

A alternativa simples de atribuir o objeto a uma variável de nome curto, e então acessar a propriedade correspondente naquela variável, está pronta para substituir `with`.

Segundo, eval de código strict mode não introduz novas variáves no escopo envolvente. Em código normal, `eval("var x;")` introduz uma variável `x` na função envolvente ou no escopo global. Isto significa que, no geral, em uma função que contém uma chamada para `eval`, todo nome que não se refere a um argumento ou variável local deve ser mapeado para uma definição particular em tempo de execução (porque aquele `eval` pode ter introduzido uma nova variável que esconderia a variável externa). No strict mode, `eval` cria variáveis apenas para o código sendo avaliado, portanto `eval` não pode afetar se um nome se refere a uma variável externa ou alguma variável local:

```
var x = 17;
var evalX = eval("'use strict'; var x = 42; x");
console.assert(x === 17);
console.assert(evalX === 42);
```

Relacionadamente, se a função `eval` for invocada por uma expressão da forma `eval(...)` em código strict mode, o código será avaliado como código strict mode. O código pode invocar strict code explicitamente, mas não é necessário.

```
function strict1(str) {
  "use strict";
  return eval(str); // str será tratado como código strict mode
}
function strict2(f, str) {
  "use strict";
  return f(str); // não é eval(...): str é strict se e
  // somente se invocar strict mode
}
function nonstrict(str) {
  return eval(str); // str é strict se e somente se
  // invocar strict mode
}

strict1("'Código strict mode!'");
strict1("'use strict'; 'Código strict mode!'");
strict2(eval, "'Código não-strict.'");
strict2(eval, "'use strict'; 'Código strict mode!'");
nonstrict("'Código não-strict.'");
nonstrict("'use strict'; 'Código strict mode!'");
```

Portanto nomes em código `eval` de strict mode se comportam da mesma forma que nomes em código strict mode que não estão sendo avaliados como resultado de `eval`.

Terceiro, strict mode proíbe deletar nomes simples. `delete name` em strict mode é erro de sintaxe:

```
"use strict";

var x;
delete x; // !!! erro de sintaxe

eval("var y; delete y;"); // !!! erro de sintaxe
```

Tornando <code>eval</code> e <code>arguments</code> mais simples

Strict mode torna `arguments` e `eval` menos estranhamente mágicos. Ambos envolvem uma quantidade considerável de comportamento mágico em código normal: `eval` para adicionar ou remover *bindings* e para mudar valores de *bindings*, e `arguments` com suas propriedades indexadas como *alias* de argumentos nomeados. Strict mode dá grandes passos na direção de tratar `eval` e `arguments` como palavras-chave, embora reparos completos não venham até uma futura edição do ECMAScript.

Primeiro, os nomes `eval` e `arguments` não podem ser vinculados ou atribuídos na sintaxe da linguagem. Todas as tentativas são erros de sintaxe:

```
"use strict";
eval = 17;
arguments++;
++eval;
var obj = { set p(arguments) {} };
var eval;
try {
} catch (arguments) {}
function x(eval) {}
function arguments() {}
var y = function eval() {};
var f = new Function("arguments", "'use strict'; return 17;");
```

Segundo, código strict mode não faz *alias* para propriedades de objetos `arguments` criados dentro de si. Em código normal dentro de uma função cujo primeiro argumento é `arg`, definir `arg` também define `arguments[0]`, e vice-versa (a menos que nenhum argumento seja fornecido ou `arguments[0]` seja deletado). Objetos `arguments` para funções strict mode armazenam os argumentos originais quando a função é invocada. `arguments[i]` não rastreia o valor do argumento nomeado correspondente, nem um argumento nomeado rastreia o valor do `arguments[i]` correspondente.

```
function f(a) {
  "use strict";
  a = 42;
  return [a, arguments[0]];
}
var pair = f(17);
console.assert(pair[0] === 42);
console.assert(pair[1] === 17);
```

Terceiro, `arguments.callee` não é mais suportado. Em código normal, `arguments.callee` se refere à função envolvente. Esse caso de uso é frágil: apenas nomeie a função envolvente! Além disso, `arguments.callee` prejudica substancialmente otimizações como tornar funções *inline* porque deve ser possível fornecer uma referência à função não-*inline* se `arguments.callee` for acessado. `arguments.callee` para funções strict mode é uma propriedade não-deletável que lança exceção quando atribuída ou recuperada:

```
"use strict";
var f = function () {
  return arguments.callee;
};
f(); // lança TypeError
```

Tornando JavaScript "seguro"

Strict mode torna mais fácil escrever JavaScript "seguro". Alguns websites agora oferecem formas de escrever JavaScript que será executado pelo website *em nome de outros usuários*. JavaScript nos navegadores pode acessar informações privadas do usuário, então tal JavaScript deve ser parcialmente transformado antes de executado, para barrar acesso a funcionalidades proibidas. A flexibilidade do JavaScript torna isso efetivamente impossível sem várias verificações em tempo de execução. Algumas funções da linguagem são tão pervasivas que fazer verificações em tempo de execução tem um custo de desempenho considerável. Alguns ajustes do strict mode, mais a exigência de que o JavaScript submetido por usuários seja código strict mode e invocá-lo de uma certa maneira, reduz substancialmente a necessidade de realizar verificações em tempo de execução.

Primeiro, o valor passado como `this` para uma função em strict mode não é forçado a ser um objeto (entenda-se "encaixotamento"). Para uma função normal, `this` é sempre um objeto: seja o objeto fornecido se chamado com um `this` cujo valor é um objeto; o valor, encaixotado, se chamado com um `this` Boolean, string ou número; ou o objeto global se chamado com um `this` `undefined` ou `null`. Use `call`, `apply`, ou `bind` para especificar um determinado `this`. O encaixotamento automático não apenas é um custo de desempenho, como também expor o objeto global em navegadores é um risco de segurança, porque o objeto global oferece acesso a funcionalidades que ambientes JavaScript "seguros" devem restringir. Assim, para uma função strict mode, o `this` especificado não é encaixotado em um objeto, e se não especificado, `this` será `undefined`:

```
"use strict";
function fun() {
  return this;
}
console.assert(fun() === undefined);
console.assert(fun.call(2) === 2);
console.assert(fun.apply(null) === null);
console.assert(fun.call(undefined) === undefined);
console.assert(fun.bind(true)() === true);
```

Isso significa, além de outras coisas, que em navegadores não é mais possíveis referenciar o objeto `window` através de `this` dentro de uma função strict mode.

Segundo, em strict mode não é mais possível "caminhar" a pilha JavaScript via extensões do ECMAScript comumente implementadas. Em código normal com essas extensões, quando uma funçao `fun` está no meio de ser chamada, `fun.caller` é a função que chamou `fun` mais recentemente, e `fun.arguments` é o `arguments` para aquela invocação de `fun`. Ambas as extensões são problemáticas para JavaScript "seguro" porque permitem que código "seguro" acesse funções "privilegiadas" e seus argumentos (potencialmente inseguros). Se `fun` estiver em strict mode, tanto `fun.caller` quando `fun.arguments` são propriedades não-deletáveis que lançam exceção quando definidas ou recuperadas:

```
function restrita() {
  "use strict";
  restrita.caller; // lança TypeError
  restrita.arguments; // lança TypeError
}
function invocadorPrivilegiado() {
  return restrita();
}
invocadorPrivilegiado();
```

Terceiro, `arguments` para funções strict mode não oferecem mais acesso às variáveis da chamada de função correspondente. Em algumas implementações antigas do ECMAScript, `arguments.caller` era um objeto cujas propriedades faziam *alias* de variáveis naquela função. Isso é um risco de segurança porque quebra a habilidade de esconder valores privilegiados via abstração de função; isso também exclui a maioria das otimizações. Por essas razões, nenhum navegador moderno implementa isso. Ainda assim, por causa de sua funcionalidade histórica, `arguments.caller` para uma função strict mode também é uma propriedade não-deletável que lança exceção quando definida ou recuperada:

```
"use strict";
function fun(a, b) {
  "use strict";
  var v = 12;
  return arguments.caller; // lança TypeError
}
fun(1, 2); // não expõe v (or a or b)
```

Pavimentando o caminho para versões futuras do ECMAScript

Versões futuras do ECMAScript provavelmente introduzirão nova sintaxe, e o strict mode no ECMAScript 5 aplica algumas restrições para facilitar essa transição. Será mais fácil fazer algumas mudanças se as fundações dessas mudanças forem proibidas no strict mode.

Primeiro, no strict mode, uma pequena lista de identificadores tornam-se palavras-chave reservadas. Estas palavras são `implements`, `interface`, `let`, `package`, `private`, `protected`, `public`, `static` e `yield`. Em strict mode, portanto, você não pode nomear ou usar variáveis ou argumentos com esses nomes.

```
function package(protected) {
  // !!!
  "use strict";
  var implements; // !!!

  // !!!
  interface: while (true) {
    break interface; // !!!
  }

  function private() {} // !!!
}
function fun(static) {
  "use strict";
} // !!!
```

Duas ressalvas específicas à Mozilla: primeiro, se seu código for JavaScript 1.7 ou superior (por exemplo em código chrome ou usando o `<script type="">` correto) e for strict mode, `let` e `yield` têm a funcionaldade que eles tiveram desde que essas palavras-chave foram introduzidas pela primeira vez. Mas código strict mode na web, carregado com `<script src="">` ou `<script>...</script>`, não serão capazes de usar `let/yield` como identificadores. Segundo, enquanto o ES5 reserva incondicionalmente as palavras `class`, `enum`, `export`, `extends`, `import` e `super`, a Mozilla reservava-as apenas em strict mode antes do Firefox 5.

Segundo, strict mode proíbe declaração de funções fora do nível superior de um script ou função. Em código normal em navegadores, declarações de funções são permitidas "em todo lugar". *Isso não é parte do ES5 (nem do ES3)!* É uma extensão com semânticas incompatíveis em diferentes navegadores. Edições futuras do ECMAScript promissoramente especificarão novas semânticas para declarações de funções fora do nível superior de um script ou função. Proibir tais declarações de funções em strict mode "limpa a mesa" para especificação em uma *release* futura do ECMAScript:

```
"use strict";
if (true) {
  function f() {} // !!! erro de sintaxe
  f();
}

for (var i = 0; i < 5; i++) {
  function f2() {} // !!! erro de sintaxe
  f2();
}

function baz() {
  // correto
  function eit() {} // erro de sintaxe
}
```

Essa proibição não é adequada ao strict mode porque tais declarações de função são uma extensão de ES5 básico. Mas é a recomendação do comitê do ECMAScript e os navegadores a implementarão.

Strict mode em navegadores

Os principais navegadores agora implementam strict mode. Entretanto, não dependa deles cegamente já que ainda há várias versões de navegadores utilizadas por aí que têm apenas suporte parcial ao strict mode ou que não suportam de forma alguma (ex.: Internet Explorer abaixo da versão 10!). *Strict mode muda semântica.* Depender dessas mudanças causará equívocos e erros em navegadores que não implementam strict mode. Tenha cautela ao usar strict mode e gere confiança no strict mode com testes de funcionalidades que verificam se partes relevantes do strict mode estão implementadas. Finalmente, certifique-se de *testar seu código em navegadores que suportam e não suportam strict mode.* Se você testar apenas em navegadores que não suportam strict mode, você provavelmente terá problemas em navegadores que suportam e vice-versa.

Veja também

- Where's Walden? » New ES5 strict mode support: now with poison pills!
- Where's Walden? » New ES5 strict mode requirement: function statements not at top level of a program or function are prohibited
- Where's Walden? » New ES5 strict mode support: new vars created by strict mode eval code are local to that code only
- JavaScript "use strict" tutorial for beginners.
- John Resig - ECMAScript 5 Strict Mode, JSON, and More
- ECMA-262-5 in detail. Chapter 2. Strict Mode.
- Strict mode compatibility table
- Transitioning to strict mode

Template strings

Template Strings são *string*s que permitem expressões embutidas. Você pode *utilizar string* multi-linhas e interpolação de *string* com elas.

Basicamente é uma nova forma de criar strings e tornar o seu código um pouco mais legível.

Sintaxe

```
`corpo de texto`

`texto linha 1
 texto linha 2`

`texto string ${expression} texto string`

tag `texto string ${expression} texto string`
```

Descrição

Template strings são envolvidas por (acentos graves) (` `) em vez de aspas simples ou duplas. *Template strings* podem possuir *placeholders*. Estes são indicados por um cifrão seguido de chaves (`${expression}`). As expressões nos *placeholders*, bem como o texto em volta delas são passados a uma função. A função padrão apenas concatena as partes em uma *string* única. Se existir uma expressão precedendo a *template string* (função `tag` exemplo), a *template string* é definida como "*tagged template string*". No caso, a expressão *tag* (geralmente uma função) é chamada pela *template string* processada, que você pode manipular antes de produzir o resultado.

```
`\`` === "`"; // --> true
```

Strings multi-linhas

Qualquer caracter de nova linha inserido no código é parte da *template string*. Utilizando *strings* normais, você teria de usar a síntaxe a seguir para obter strings multi-linhas:

```
console.log("texto string linha 1\n" + "texto string linha 2");
// "texto string linha 1
// texto string linha 2"
```

Para obter o mesmo efeito com strings multi-linhas, você agora pode escrever:

```
console.log(`texto string linha 1
texto string linha 2`);
// "texto string linha 1
//  texto string linha 2"
```

Interpolação de Expressões

Para encapsular expressões dentro de strings, você precisava utilizar a seguinte sintaxe:

```
var a = 5;
var b = 10;
console.log("Quinze é " + (a + b) + " e\nnão " + (2 * a + b) + ".");
// "Quinze é 15 e
// não 20."
```

Agora, com template strings, você pode utilizar as substituições sintáticas tornando o código mais legível:

```
var a = 5;
var b = 10;
console.log(`Quinze é ${a + b} e
não ${2 * a + b}.`);
// "Quinze é 15 e
// não 20."
```

Tagged template strings

Uma forma mais avançada dos *template string* são os *template strings* com marcações ou tags, ou *tagged template strings*. Com eles, você tem a possibilidade de modificar a saída dos *template strings* usando uma função. O primeiro argumento contém um array de literais ("Hello" e "World" neste exemplo). Do segundo em diante e cada argumento subsequente contém valores previamente processados (algumas vezes chamados *cooked*) pelas expressões de substituição ("15" e "50" no caso do exemplo). No final, a função retorna a string ja manipulada:

```
var a = 5;
var b = 10;

function tag(strings, ...values) {
  console.log(strings[0]); // "Hello "
  console.log(strings[1]); // " world "
  console.log(values[0]); // 15
  console.log(values[1]); // 50

  return "Bazinga!";
}

tag`Hello ${a + b} world ${a * b}`;
// "Bazinga!"
```

Strings Raw

A propriedade especial `raw`, disponível no primeiro argumento da função da *tagged template string* acima, permite o acesso as strings de maneira pura (*raw*) exatamente como elas foram especificadas:

```
function tag(strings, ...values) {
  return strings.raw[0];
}

tag`string text line 1 \n string text line 2`;
// "string text line 1 \\n string text line 2"
```

Adicionalmente, o método `String.raw()` permite a criação de strings cruas, exatamente como as *template functions* e as concatenações deveram criar.

```
String.raw`Hi\n${2 + 3}!`; // "Hi\\n5!"
```

Veja também

- `String`
- `String.raw()`
- Gramática léxica
- Template-like strings in ES3 compatible syntax
- "ES6 in Depth: Template strings" on hacks.mozilla.org
- https://styled-components.com/

Trailing commas

Trailing commas (por vezes chamadas "final commas") podem ser úteis quando adicionar novos elementos, parâmetros, ou propriedades ao código JavaScript. Se desejar adicionar um nova propriedade, pode adicionar uma nova linha sem modificar a última linha anterior, se isso A linha já usa uma vírgula de trailing. Isto torna as diffs de controle de versão mais limpas e editadas, assim o controle de versão do código pode ser menos problemático.

O JavaScript tem permitido o uso de trailing commas em arrays literais desde o início, e posteriormente adicionou-as aos objetos literais, e mais recentemente, a parâmetros da função e a importações nomeadas e exportações nomeadas.

JSON, no entanto, não permitem o uso de trailing comma.

Syntax

```
,
```

Exemplos

Trailing commas em literais

Arrays

JavaScript ignora trailing commas em arrays:

```
const arr = [1, 2, 3];
arr; // [1, 2, 3]
arr.length; // 3
```

Se for utilizada mais do que uma trailing comma, é produzida uma elisão (ou buraco). Uma elisão com buracos é chamado de *sparse* (uma matriz *densa* não tem sparse). Quando arrays iteráveis por exemplo com `Array.prototype.forEach()` ou `Array.prototype.map()`, os buracos da matriz são saltados.

```
const arr = [1, 2, 3, , ,];
arr.length; // 5
```

Objetos

Trailing commas em objetos literais também são úteis:

```
const objeto = {
  foo: "bar",
  teclado: "qwerty",
  idade: 42,
};
```

Trailing commas em funções

Trailing commas também são permitidas nas listas de parâmetros de funções

Definição de parâmetros

Os seguintes pares de definição de funções são válidos e equivalentes um ao outro. As trailing commas não afetam a propriedade `length` das declarações de função ou o objeto `arguments`

```
function f(p) {}
function f(p) {}
(p) => {};
(p) => {};
```

As trailing commas também funcionam como definições de métodos para classes ou objetos:

```
class C {
  um(a) {}
  dois(a, b) {}
}
const objeto = {
  um(a) {},
  dois(a, b) {},
};
```

Chamadas de função

Os seguintes pares de invocação de funções são válidos e equivalentes um ao outro.

```
f(p);
f(p);
Math.max(10, 20);
Math.max(10, 20);
```

Trailing commas inválidas

Definições de parâmetros de funções ou invocações de funções que contenham apenas uma vírgula lançam um `SyntaxError`. Além disso, ao utilizar um parâmetros rest, não são permitidas trailling commas:

```
function f(,) {} // SyntaxError: parâmetro formal em falta
(,) => {};       // SyntaxError: expressão esperada, contém ','
f(,)             // SyntaxError: expressão esperada, contém ','
function f(...p,) {} // SyntaxError: parâmetro após parâmetro de repouso
(...p,) => {}        // SyntaxError: parênteses de fecho esperados, contém ','
```

Trailing commas em desestruturação

Também é permitida uma vírgula no lado esquerdo quando se usa atribuição de desestruturação:

```
// desestruturação de array com trailing comma
[a, b] = [1, 2];
// desestruturação de objeto com trailing comma
const o = {
  p: 42,
  q: true,
};
const { p, q } = o;
```

Mais uma vez, ao utilizar um elemento de rest, um `SyntaxError` será lançado:

```
const [a, ...b] = [1, 2, 3];
// SyntaxError: elemento rest pode nao ter um trailing comma
```

Trailing commas em JSON

Como o JSON se baseia num subconjunto muito restrito de sintaxe JavaScript, **não são permitidas trailing commas no JSON***.

Ambas as linhas irão lançar um `SyntaxError`:

```
JSON.parse("[1, 2, 3, 4, ]");
JSON.parse('{"foo" : 1, }');
// SyntaxError JSON.parse: carácter inesperado
// na linha 1 coluna 14 dos dados do JSON
```

Omitir trailing commas para analisar correctamente o JSON:

```
JSON.parse("[1, 2, 3, 4 ]");
JSON.parse('{"foo" : 1 }');
```

Trailling commas em importações nomeadas e exportações nomeadas

Trailing commas são válidas em importações nomeadas e exportações nomeadas.

importações nomeadas

```
import { A, B, C } from "D";
import { X, Y, Z } from "W";
import { A as B, C as D, E as F } from "Z"; //Renomeando as importações
```

exportações nomeadas

```
export { A, B, C };
export { A, B, C };
export { A as B, C as D, E as F }; // Renomeando as importações
```

Prefixo do quantificador

```
  //{ DecimalDigits[~Sep], DecimalDigits[~Sep] }
  x{n,}
  x{n,m}
  x{n,m}?
```

Veja também

- Proposta inicial do ECMAScript: trailing function commas por Jeff Morrison